cadence®

Jasper Apps Command Reference Manual

Product Version 2026.06
June 2026

© 2026 Cadence Design Systems, Inc.
Printed in the United States of America.

Cadence Design Systems, Inc. (Cadence), 2655 Seely Ave., San Jose, CA 95134, USA.

Open SystemC, Open SystemC Initiative, OSCI, SystemC, and SystemC Initiative are trademarks or registered trademarks of Open SystemC Initiative, Inc. in the United States and other countries and are used with permission.

Trademarks: Trademarks and service marks of Cadence Design Systems, Inc. (Cadence) contained in this document are attributed to Cadence with the appropriate symbol. For queries regarding Cadence's trademarks, contact the corporate legal department at the address shown above or call 800.862.4522.

All other trademarks are the property of their respective holders.

Restricted Permission: This publication is protected by copyright law and international treaties and contains trade secrets and proprietary information owned by Cadence. Unauthorized reproduction or distribution of this publication, or any portion of it, may result in civil and criminal penalties. Except as expressly permitted below, this publication may not be copied, reproduced, modified, published, uploaded, posted, transmitted, or distributed in any way, including automated processes, training or services involving a large language model, foundation model, deep machine learning, generative artificial intelligence, or any other similar process or technology, without prior written permission from Cadence. Unless otherwise agreed to by Cadence in writing, customers are granted permission to print one (1) hard copy of this publication subject to the following conditions:

1. The publication may be used only in accordance with a written agreement between Cadence and its customer.
2. The publication may not be modified in any way.
3. Any authorized copy of the publication or portion thereof must include all original copyright, trademark, and other proprietary notices and this permission statement.
4. The information contained in this document cannot be used in the development of like products or software, whether for internal or external use, and shall not be used for the benefit of any other party, whether or not for consideration.

Disclaimer: Information in this publication is subject to change without notice and does not represent a commitment on the part of Cadence. Except as may be explicitly set forth in such agreement, Cadence does not make, and expressly disclaims, any representations or warranties as to the completeness, accuracy or usefulness of the information contained in this document. Cadence does not warrant that use of such information will not infringe any third party rights, nor does Cadence assume any liability for damages or costs of any kind that may result from use of such information. Cadence is committed to using respectful language in our code and communications. We are also active in the removal and replacement of inappropriate language from existing content. This product documentation may however contain material that is no longer considered appropriate but still reflects long-standing industry terminology. Such content will be addressed at a time when the related software can be updated without end-user impact.

Restricted Rights: Use, duplication, or disclosure by the Government is subject to restrictions as set forth in FAR52.227-14 and DFAR252.227-7013 et seq. or its successor.

Contents

1 27
Introduction 27
What's New 27
Reference Manual Organization 28
Related Documentation 29
Jasper Apps Support for Tcl 30
UNIX Commands 30
Conventions Used in This Manual 31
2 34
Installation 34
Supported Platforms 34
Supported Platforms for Intel Architecture (x86_64) 34
Supported Platforms for ARM Architecture (AArch64) 35
Supported Languages and Standards 36
Jasper Apps Installation 37
License Installation 38
TIMEOUT and TIMEOUTALL 39
Redundant License Servers 39
Installing Your License 39
Appending Lines to a License File 40
Manually Starting a New Daemon 42
Using the Correct License Server 42
Using FlexNet Publisher Utilities 43
License-Related Environment Variables 46
3 48
Commands and Variables Overview 48
Command-Line Options 48
Environment Variables 58
Passing Information to the Jasper Tcl Environment 59
Environment Variables Used by Jasper 61
Command Summary 69
General Command Summary 70

General Configuration Command Summary 83
App-Specific Command Summary 118
App-Specific Configuration Command Summary 119
General Command Usage 151
Tab Completion 152
Stopping a Command 152
Prefix Matching 152
Specifying the Current Task 153
4 154
Regular Commands 154
abstract 155
abvip 165
agent 168
analyze 175
assert 199
assume 212
auto_setup 225
autoprove 227
blackbox_assistant 234
capture_testcase 237
cfe_debug 240
check_afv 245
check_assumptions 250
check_c2rtl 262
check_caf 290
check_cdc 310
check_code 349
check_conn 351
check_cov 368
check_csr 401
check_dlh 411
check_fsv 421
check_hls 453
check_loop 504
check_lpv 512
check_one_of 533

check_reset 535
check_rvv 538
check_sec 545
check_spv 622
check_superlint 637
check_timing_exception 669
check_unr 675
check_xprop 683
clear 712
clock 715
command_interface 726
compilation 728
compile 738
complexity_manager 748
config_rtlds 751
configure 771
connect 774
cover 780
custom_engine 798
custom_gui_action 801
custom_rule_rtlds 804
database 809
debug_explorer 813
debug_handoff 821
dglob 829
dump_function_stack 830
edit 831
elaborate 832
exit 867
expert_system 870
export 880
formal_profiler 882
fsm_checks 887
get_analyze_info 890
get_clock_info 901
get_decomposed_expression 904
get_design_info 908

get_fanin 968
get_fanout 974
get_filename 977
get_flop_info 978
get_inst_top 985
get_install_dir 986
get_latch_info 987
get_needed_assumptions 990
get_proj_dir 1000
get_property_info 1001
get_property_list 1014
get_reset_info 1019
get_root_name 1027
get_signal_info 1028
get_signal_list 1037
get_status 1040
get_tool 1043
get_top_module 1045
get_trace_info 1046
get_uxdb_status 1048
get_version 1049
graph 1050
gui 1054
help 1061
hunt 1064
include 1116
incremental_elaborate_wrapper_mode 1117
info_query 1119
ipk 1123
ipxact 1126
jasper_history 1134
jasper_model_divider 1137
jasper_model_mpram 1141
jasper_model_multiplier 1146
jasper_model_ram 1151
jasper_scoreboard_3 1156
jforeach 1158

liberty 1161
load_radix_file 1166
pretty_print 1173
proof_structure 1175
proofmaster 1201
property_triage 1209
prove 1212
put_message 1230
read_sdc 1233
redirect 1245
replace_driver 1247
report 1251
reset 1257
restore 1279
sanity_check 1283
save 1289
schematic_viewer 1296
session 1302
set_current_gui 1310
show_list 1312
show_source_browser 1313
source_lock 1314
sst 1317
stopat 1320
task 1324
view 1333
virtual_net 1334
visualize 1337
5 1429
Set Commands 1429
set_afv_generate_auto_deadlock 1430
set_afv_input_conditions_nested 1431
set_afv_prll_property_consolidation 1432
set_afv_strict_declarations 1434
set_allow_latch_enable_loops 1435
set_analyze_bbox_parse_mode 1438

set_analyze_libnonamehide 1439
set_analyze_librescan 1441
set_analyze_libunboundsearch 1442
set_analyze_y_and_v_command_line_order 1443
set_annotation 1445
set_auto_disable_related_covers 1447
set_automatic_library_search 1448
set_blackbox_assistant_require_clock_paths_in_connections 1451
set_blackbox_assistant_require_control_paths_in_connections 1452
set_c2rtl_disable_imp_assumption 1453
set_c2rtl_max_signals_to_plot 1454
set_cache_proof_simplification 1455
set_caf_engine_mode 1457
set_caf_prove_keep_traces 1458
set_caf_prove_parallel_tasks 1459
set_caf_prove_time_limit 1461
set_caf_signal_type 1462
set_caf_signals_customize 1463
set_caf_signals_extraction_heuristic 1464
set_caf_signals_filter_connected 1465
set_caf_signals_filter_constant 1466
set_caf_signals_filter_wide_signal 1468
set_caf_signals_max_bits 1469
set_caf_stuck_at_bitblasted 1470
set_capture_elaborated_design 1471
set_capture_property_expression 1472
set_cdc_calculate_boundary_data_during_extract 1473
set_cdc_clock_period_max_factor 1474
set_cdc_conditional_waiver_property_limit 1475
set_cdc_gate_level_analysis_mode 1476
set_cdc_machine_learning_flow 1477
set_cdc_parallel_extraction 1478
set_cell_define_debug_visibility 1479
set_cfe_compile_pedantic_errors 1481
set_cfe_profiler_default_function_name 1482
set_clock_loop_model 1483
set_cmd_time_limit 1485

set_command_license_wait_interval 1488
set_command_license_wait_retries 1489
set_compilation_cache 1490
set_compilation_cache_path 1492
set_compilation_command 1493
set_compilation_skip_errors_in_filelist 1494
set_compilation_unit_naming_mode 1495
set_compilation_unit_naming_prefix 1497
set_conn_prove_loop_handling 1498
set_counter_speculative_detection 1501
set_covergroup_bin_size_limit 1502
set_covergroup_cross_product_limit 1503
set_cte_enable_session_tcl_env_restore 1504
set_cumulative_reset 1505
set_database_auto_save_restore 1506
set_database_session_save_restore 1507
set_debug_handoff_source_lock 1508
set_design_exploration_include_clock_logic 1510
set_dollar_display_synthesis 1512
set_elab_time_limit 1513
set_elaborate_bbox_inout_simplification 1515
set_elaborate_bbox_output_driver 1516
set_elaborate_generate_sage_database 1517
set_elaborate_interrupt_on_error 1518
set_elaborate_limited_unrolled_loop_precondition 1519
set_elaborate_limited_unrolled_loop_property 1520
set_elaborate_procedural_loop_limit 1521
set_elaborate_quiet_trace 1523
set_elaborate_real_mode 1524
set_elaborate_single_run_mode 1526
set_elaborate_use_disk 1528
set_enable_preserve_property_status 1530
set_engine_algorithm_selection 1531
set_engine_algorithm_selection_dir 1533
set_engine_distributed_mode 1534
set_engine_mode 1535
set_engine_processes 1549

set_engine_solver 1551
set_engine_threads 1553
set_engineB_before_first_mode 1555
set_engineB_before_increment_mode 1557
set_engineB_first_trace_attempt 1559
set_engineB_processes 1562
set_engineB_trace_attempt_increment 1564
set_engineB_trace_attempt_time_limit 1566
set_engineB_trace_attempt_time_limit_incr 1568
set_engineBm_first_trace_attempt 1570
set_engineBm_processes 1573
set_engineBm_trace_attempt_increment 1575
set_engineBm_trace_attempt_time_limit 1577
set_engineBm_trace_attempt_time_limit_incr 1579
set_engineC_optimization 1581
set_engineCG_max_mem 1583
set_engineD_optimization 1584
set_engineG_optimization 1585
set_engineH_before_first_mode 1587
set_engineH_before_increment_mode 1589
set_engineH_first_proof_attempt 1591
set_engineH_first_trace_attempt 1592
set_engineH_last_proof_attempt 1595
set_engineH_restart 1596
set_engineH_trace_attempt_increment 1597
set_engineH_trace_attempt_time_limit 1599
set_engineH_trace_attempt_time_limit_incr 1601
set_engineJ_attempts 1603
set_engineJ_max_trace_length 1605
set_engineJ_migrate 1608
set_engineJ_restarts 1610
set_engineJ_seed 1612
set_engineJ_single_property 1614
set_engineL_asserts_are_intermediate 1615
set_engineL_cache 1616
set_engineL_generate_trails 1618
set_engineL_max_segment_length 1619

set_engineL_random_diversification 1620
set_engineL_search_mode 1621
set_engineL_seed 1623
set_engineL_segment_time_limit 1625
set_engineL_state_removal 1627
set_engineL_tail_length 1628
set_engineM_optimization 1630
set_engineQ3_max_trace_length 1631
set_engineQ3_random_diversification 1634
set_engineQ3_random_diversification_factor 1636
set_engineQ3_restarts 1638
set_engineQ3_seed 1640
set_engineQT_attempt_time_limit 1642
set_engineQT_optimization 1644
set_engineU2_max_trace_length 1645
set_engineU2_restarts 1647
set_engineU2_seed 1649
set_engineU_max_trace_length 1651
set_engineU_seed 1653
set_engineU_trace_attempt_time_limit 1655
set_engineWL_config 1657
set_evaluate_properties_on_formal_reset 1658
set_executable_path 1660
set_exit_handler 1661
set_expert_system_property_send_limit 1664
set_expert_system_regression_model_version 1665
set_expert_system_regression_tag 1666
set_expert_system_status 1667
set_expert_system_superlint_tasks_filter 1668
set_expert_system_tcl_design_name 1669
set_expert_system_unprocessed_property_status 1670
set_export_sva_bind_enable_directive 1671
set_export_sva_embed_lib_modules 1672
set_export_sva_handshake_delay_threshold 1673
set_export_sva_hierarchy_separator 1674
set_export_sva_hierarchy_separator_escape_string 1675
set_export_sva_target_environment 1676

set_export_sva_trace_suffix_cycles 1677
set_expr_32bit_constants 1678
set_extended_inout_port_alias_handling 1679
set_file_access_time_limit 1681
set_first_trace_attempt 1683
set_force_property_removal 1686
set_fsv_abstract_result_mode 1687
set_fsv_allow_faults_in_bbox 1688
set_fsv_bbox_abstraction_level 1689
set_fsv_break_loop_mode 1690
set_fsv_bus_fault_partition 1691
set_fsv_bus_fault_width 1692
set_fsv_classification_scheme 1693
set_fsv_clock_cycle_time 1694
set_fsv_constant_propagation_analysis_optimization 1695
set_fsv_engine_mode 1697
set_fsv_export_fault_relations 1698
set_fsv_export_propagated_constants 1699
set_fsv_fault_tw 1700
set_fsv_generate_activatability 1702
set_fsv_generate_always_detected 1703
set_fsv_generate_always_propagated 1704
set_fsv_generate_co_max_delay_time 1705
set_fsv_generate_copy_env 1707
set_fsv_generate_detectability 1708
set_fsv_generate_fo_co_max_delay_time 1709
set_fsv_generate_fo_max_delay_time 1711
set_fsv_generate_propagatability 1713
set_fsv_generate_propagated_always_detected 1714
set_fsv_generate_use_bus_faults 1715
set_fsv_generate_use_memory_faults 1716
set_fsv_import_include_ut 1717
set_fsv_import_prime_only 1718
set_fsv_max_checks_per_task 1719
set_fsv_max_faults_per_task 1720
set_fsv_memory_fault_width 1721
set_fsv_proof_min_cluster_size 1722

set_fsv_proof_sanity_check 1723
set_fsv_prove_keep_traces 1725
set_fsv_prove_strategy 1726
set_fsv_set_hold_time 1727
set_fsv_source_browser_max_line_count 1728
set_fsv_source_browser_size_threshold 1729
set_fsv_source_browser_updating 1730
set_fsv_stopat_abstraction_level 1731
set_fsv_strobe_checker_mode 1732
set_fsv_strobe_checker_mode_assert_value 1733
set_fsv_strobe_optimization 1734
set_fsv_structural_coi 1735
set_fsv_structural_constant 1736
set_fsv_structural_constant_propagation_analysis 1737
set_fsv_structural_distance 1738
set_fsv_structural_fault_relations 1739
set_fsv_structural_propagation_analysis 1740
set_fsv_structural_seq_constants_engine_mode 1741
set_fsv_structural_seq_constants_propagation 1742
set_fsv_structural_seq_constants_prove_time_limit 1743
set_fsv_unsafe_inference 1744
set_function_debug_visibility 1745
set_functional_loop_time 1746
set_gate_level_compilation_mode 1748
set_gdb_path 1749
set_hunt_single_proof_thread 1750
set_incremental_compilation_mode 1752
set_internal_from_mode 1755
set_jasper_history_per_command_limit 1756
set_jaspersim_max_jobs 1757
set_jaspersim_mode 1758
set_jaspersim_partition_depth 1760
set_jaspersim_partition_file 1762
set_jaspersim_partition_method 1764
set_jaspersim_partition_min_size 1766
set_legacy_expression_parser 1767
set_liberty_cells_auto_blackbox 1770

set_liberty_cells_override_rtl_definition 1771
set_liberty_enable_cell_define 1772
set_liberty_enable_function_synthesis 1773
set_liberty_enable_multithread_load 1774
set_liberty_enable_timing_model_synthesis 1775
set_library_cell_detection 1776
set_library_cell_optimization 1778
set_log_timestamp_format 1780
set_log_timestamp_mode 1782
set_loop_filter_latch_patterns 1783
set_loop_filter_startpoints 1785
set_loop_handling 1787
set_loop_handling_fallback 1791
set_lpv_enable_corrupt_annotation 1793
set_lpv_enable_pin_corruption_in_liberty_bbox 1794
set_lpv_enable_upf_path_format 1795
set_lpv_enable_upf_signals 1796
set_lpv_xprop_mode 1797
set_max_num_sub_expressions 1799
set_max_trace_length 1800
set_memory_aware_export_memory_report 1803
set_memory_aware_initial_mem 1805
set_memory_aware_max_mem 1807
set_memory_aware_mem_factor 1809
set_memory_aware_memory_report_path 1811
set_memory_aware_min_mem 1812
set_memory_aware_proofs 1814
set_memory_aware_wastage_factor 1817
set_message 1819
set_message_history_limit 1827
set_missing_entries_as_x_in_comb_udp 1828
set_module_elaboration_memory_limit 1829
set_no_fallback_in_use_clause 1831
set_optimize_cell_netlists 1832
set_parallel_synthesis_directory 1834
set_parallel_synthesis_dispatch_mode 1835
set_parallel_synthesis_dispatch_timeout 1837

set_parallel_synthesis_manual_partition_file 1839
set_parallel_synthesis_memory_limit 1841
set_parallel_synthesis_mode 1842
set_parallel_synthesis_num_process 1844
set_parallel_synthesis_partition_method 1846
set_parallel_synthesis_shell 1849
set_parallel_synthesis_shell_stop 1852
set_per_property_simplification 1853
set_persistent_proofgrid_timeout 1855
set_preference 1856
set_prompt1 1870
set_prompt2 1872
set_proof_simplification 1873
set_proof_simplification_max_iterations 1875
set_proof_structure_auto_propagation 1876
set_proof_structure_support_embedded_on_case_split 1877
set_proofgrid_bridge 1879
set_proofgrid_cloud_file_upload 1881
set_proofgrid_cloud_job_start_template 1882
set_proofgrid_cloud_job_status_template 1883
set_proofgrid_cloud_job_stop_template 1884
set_proofgrid_cloud_jobarray 1885
set_proofgrid_cloud_upload_storage 1887
set_proofgrid_cluster 1888
set_proofgrid_engine_file_path 1890
set_proofgrid_engineJ_max_jobs 1891
set_proofgrid_engineL_max_jobs 1893
set_proofgrid_engineQ3_max_jobs 1895
set_proofgrid_engineU2_max_jobs 1897
set_proofgrid_engineU_max_jobs 1899
set_proofgrid_extra_args 1901
set_proofgrid_manager 1904
set_proofgrid_max_jobs 1906
set_proofgrid_max_local_jobs 1907
set_proofgrid_max_partitions 1908
set_proofgrid_mem_scan 1909
set_proofgrid_mode 1912

set_proofgrid_pending_time_limit 1915
set_proofgrid_per_engine_max_jobs 1917
set_proofgrid_per_engine_max_jobs_per_session 1919
set_proofgrid_per_engine_max_local_jobs 1921
set_proofgrid_queue 1923
set_proofgrid_restarts 1925
set_proofgrid_resubmit_time_limit 1926
set_proofgrid_shell 1927
set_proofgrid_shell_stop 1934
set_proofgrid_socket_communication 1935
set_proofmaster 1937
set_proofmaster_dir 1940
set_proofmaster_initial_dir 1942
set_proofmaster_max_data_age 1944
set_property_compile_time_limit 1945
set_property_delay_cycles 1947
set_property_info_proof_timestamp 1948
set_property_table_dynamic_sort 1950
set_property_triage_max_threads 1952
set_prove_advanced_simplification 1953
set_prove_advanced_simplification_time_limit 1954
set_prove_cache 1955
set_prove_cache_in_parallel 1958
set_prove_cache_job_mode 1960
set_prove_cache_max_jobs 1962
set_prove_cache_mode 1964
set_prove_cache_path 1967
set_prove_cache_traces 1968
set_prove_check_loop_time 1969
set_prove_clock_optimization 1971
set_prove_dump_trace_type 1973
set_prove_evaluate_in_trace_mode 1975
set_prove_inferred_target_bound 1977
set_prove_invariants_import 1980
set_prove_no_cover_traces 1982
set_prove_no_traces 1984
set_prove_on_determined 1985

set_prove_orchestration 1987
set_prove_orchestration_hunt_trace_swarm_jobs_ratio 1990
set_prove_orchestration_hunt_trace_swarm_start_time_ratio 1991
set_prove_orchestration_hunt_trace_swarm_strategy 1992
set_prove_orchestration_stagnation 1994
set_prove_orchestration_with_trace_swarm 1995
set_prove_per_property_max_time_limit 1997
set_prove_per_property_time_limit 1999
set_prove_per_property_time_limit_factor 2002
set_prove_prefer_shortest 2004
set_prove_report_mem 2006
set_prove_report_period 2007
set_prove_report_task 2008
set_prove_start_timer_on_engine_start 2010
set_prove_stop_on_cex_limit 2011
set_prove_stop_on_target_bound 2012
set_prove_stop_on_unreachable_limit 2013
set_prove_summary 2014
set_prove_target_bound 2015
set_prove_time_limit 2017
set_prove_verbosity 2019
set_prove_with_soft 2021
set_prove_with_soft_mode 2023
set_proven_directive 2026
set_read_sdc_debug 2028
set_regexp_mode 2029
set_reset_effort 2033
set_reset_extended_mode 2034
set_reset_max_iterations 2035
set_resilient_compilation 2036
set_resilient_compilation_mode 2038
set_restore_resolve_file_path 2042
set_rtlds_auto_translate_rtl_to_gate_level_signals 2044
set_save_compress_database 2045
set_save_restore_capture_source_lock_rtl 2047
set_schematic_viewer_character_limit 2049
set_schematic_viewer_collapse_combo_logic 2050

set_schematic_viewer_collapse_constants 2052
set_schematic_viewer_info_pane_area 2054
set_schematic_viewer_show_collapsed_logic_as_cloud 2056
set_schematic_viewer_show_function_nets 2058
set_schematic_viewer_show_gate_cell_names 2060
set_schematic_viewer_show_info_pane 2062
set_schematic_viewer_show_instance_unconnected_pins 2064
set_schematic_viewer_show_register_name 2065
set_schematic_viewer_show_strength_information 2066
set_schematic_viewer_simplify_case_statements 2068
set_schematic_viewer_simplify_concatenations 2069
set_sdc_vhdl_expr_mode 2070
set_sec_auto_disable_violating_mappings 2072
set_sec_automap_aggregate_mappings 2073
set_sec_autoprove 2075
set_sec_autoprove_expose_internal_cex 2076
set_sec_autoprove_expose_status_progress 2078
set_sec_autoprove_include_related_properties 2080
set_sec_autoprove_max_number_of_internal_cex 2082
set_sec_autoprove_minimal_max_length_of_internal_cex 2083
set_sec_autoprove_post_proof_activities_after_stop 2085
set_sec_autoprove_skip_prequalification 2087
set_sec_autoprove_strategy 2088
set_sec_break_loop_mode 2090
set_sec_disable_imp_assumption 2092
set_sec_elaborate_mode 2093
set_sec_expose_non_app_related_properties 2094
set_sec_fractional_verification_mode 2095
set_sec_hierarchical_blackbox_failing_instance 2096
set_sec_hierarchical_check_assumption 2097
set_sec_hierarchical_force_job 2098
set_sec_hierarchical_generate_script 2099
set_sec_hierarchical_match_port_number 2100
set_sec_hierarchical_module_average_size 2101
set_sec_hierarchical_module_excludelist 2102
set_sec_hierarchical_module_includelist 2103
set_sec_hierarchical_module_min_size 2104

set_sec_hierarchical_num_session 2105
set_sec_hierarchical_regular_expression 2106
set_sec_hierarchical_report_incremental_status 2107
set_sec_hierarchical_run_time_limit 2108
set_sec_hierarchical_save_partition_database 2109
set_sec_hierarchical_signoff_waiver_list 2110
set_sec_hierarchical_use_analyze_snapshot 2111
set_sec_mapping_max_word_size 2112
set_sec_mpram_compound_port_mapping 2113
set_sec_proofgrid_utilization_warning_percentage_threshold 2114
set_sec_return_mapping_dictionary_from_gui 2115
set_sec_run_source_browser_diff 2117
set_sec_shallow_elaboration 2120
set_sec_show_bound_progress_in_log 2121
set_sec_show_proof_log 2122
set_sec_task_based_auto_mapping_on_setup 2125
set_sec_time_step_for_status_progress_log_entries 2126
set_sec_twin_debug_mode 2127
set_sec_use_uif_modeling 2128
set_sequence_wait_timeout 2130
set_show_checker_on_hierarchy_tree 2131
set_show_disabled_property_count 2132
set_show_interface_on_hierarchy_tree 2133
set_show_proven_directive 2134
set_simulation_max_loop_limit 2135
set_single_property_liveness 2136
set_spv_autoprove_basic_engine_mode 2138
set_spv_autoprove_expose_internal_cex 2139
set_spv_autoprove_expose_status_progress 2141
set_spv_autoprove_max_number_of_internal_cex 2143
set_spv_autoprove_minimal_max_length_of_internal_cex 2144
set_spv_autoprove_skip_prequalification 2146
set_spv_break_loop_mode 2147
set_spv_create_combined_precondition 2149
set_spv_graph_show_alternative_paths 2150
set_spv_include_control_logic 2152
set_spv_keep_driving_logic 2155

set_spv_proofgrid_utilization_warning_percentage_threshold 2157
set_spv_prove_single_thread 2158
set_spv_show_bound_progress_in_log 2160
set_spv_show_proof_log 2161
set_spv_taint_feedback_loop 2164
set_spv_time_step_for_status_progress_log_entries 2166
set_src_inline_macro_expansion 2167
set_src_loop_iteration_debug 2170
set_sst_default_trace_length 2171
set_stable_name_in_case_assert 2172
set_superlint_add_automatic_task_assumption_bitblasted 2174
set_superlint_arithmetic_overflow_counter_filter 2175
set_superlint_arithmetic_overflow_enable_exponentiation 2177
set_superlint_arithmetic_overflow_enable_multiplication 2178
set_superlint_conflicting_assertions_remove_design_constraints 2179
set_superlint_deadcode_full_case_filter 2180
set_superlint_deadcode_unified_loop 2181
set_superlint_debug_unreachable 2182
set_superlint_debug_unreachable_per_property_time_limit 2183
set_superlint_debug_unreachable_time_limit 2184
set_superlint_enable_compile_messages 2185
set_superlint_enable_hier_lint 2186
set_superlint_enable_observability_tags 2187
set_superlint_enable_rtl_inline_waiver 2189
set_superlint_engine_mode 2190
set_superlint_engine_mode_fsm_deadlock_livelock 2191
set_superlint_extract_default_case_x_assignment_checks 2192
set_superlint_extract_exclude_fsm 2193
set_superlint_extract_low_level_fsm 2194
set_superlint_filter_covered_reset 2195
set_superlint_fsm_automatic_assumptions_type 2196
set_superlint_fsm_ignore_self_transitions 2197
set_superlint_fsm_max_state 2198
set_superlint_fsm_no_mtrn_max_state 2199
set_superlint_fsm_strategy_per_level_time_limit 2200
set_superlint_fsm_strategy_prove_time_limit 2202
set_superlint_leverage_formal 2204

set_superlint_limit_observable_boundary_signal 2205
set_superlint_loop_count_limit 2206
set_superlint_loop_filter_startpoints 2207
set_superlint_machine_learning_flow 2209
set_superlint_observability_local_expression_levels 2210
set_superlint_parallel_extraction 2211
set_superlint_parallel_synthesis 2212
set_superlint_prove_keep_traces 2214
set_superlint_prove_memory_limit 2215
set_superlint_prove_time_limit 2216
set_superlint_report_negative_result_overflow 2217
set_superlint_report_same_drivers_multiple_assignment 2219
set_superlint_report_unsigned_casting 2220
set_superlint_rule_profiler 2222
set_superlint_signal_type 2223
set_superlint_signals_automatic_assumptions_type 2224
set_superlint_signals_customize 2225
set_superlint_signals_deadlock_bitblasted 2226
set_superlint_signals_deadlock_max_width 2227
set_superlint_signals_extraction_heuristic 2228
set_superlint_signals_filter_connected 2229
set_superlint_signals_filter_constant 2230
set_superlint_signals_filter_wide_signal 2231
set_superlint_signals_max_bits 2232
set_superlint_stuck_at_bitblasted 2233
set_superlint_sva_max_depth 2234
set_superlint_sva_max_length 2235
set_superlint_unified_deadcode_checks 2236
set_superlint_verbosity_level 2238
set_superlint_waiver_export_version 2239
set_tag_irrelevant_values 2240
set_task_compile_time_limit 2242
set_tcl_auto_show_error_info 2244
set_tcl_no_precondition 2245
set_tcl_related_covers 2246
set_terminal_binary_path 2247
set_time_format 2248

set_trace_extension 2249
set_trace_optimization 2251
set_trace_show_reset 2254
set_unr_break_clock_loop 2256
set_unr_functional_coverage 2257
set_unr_functional_coverage_mode 2258
set_unr_partition_max_jobs 2259
set_unr_prove_max_chunks 2260
set_unr_prove_max_properties_per_chunk 2261
set_use_low_power_port_definition 2262
set_visualize_annotate_clock_events 2264
set_visualize_auto_check_props 2265
set_visualize_auto_load_debugging_tables 2266
set_visualize_default_layout 2267
set_visualize_highlight_limit 2268
set_visualize_log_default_auto_sync 2269
set_visualize_multiple_traces 2270
set_visualize_preload_signal_list 2271
set_visualize_show_default_signals 2273
set_visualize_show_full_struct 2274
set_visualize_show_strength_information 2275
set_visualize_waveedit_transparency 2276
set_why_include_non_ticking_gated_clocks 2277
set_word_level_reduction 2278
set_word_level_time_limit 2280
set_xprop_autoprove_expose_internal_cex 2281
set_xprop_autoprove_max_number_of_internal_cex 2283
set_xprop_autoprove_minimal_max_length_of_internal_cex 2284
set_xprop_break_loop_mode 2286
set_xprop_ignore_non_resetting_flops_as_source 2287
set_xprop_proofgrid_utilization_warning_percentage_threshold 2288
set_xprop_show_proof_log 2289
set_xprop_use_all_undriven 2292
set_xprop_use_bbox_outputs 2295
set_xprop_use_bus_contention 2298
set_xprop_use_bus_floating 2300
set_xprop_use_init_abstraction 2302

set_xprop_use_inputs 2305
set_xprop_use_internal_undriven 2308
set_xprop_use_reset_abstraction 2311
set_xprop_use_reset_state 2314
set_xprop_use_stopats 2316
A 2320
Appendix A: PSL Support 2320
B 2327
Appendix B: SVA Support 2327
SVA Support Status 2327
Jasper Tcl Operators 2335
Additional Support for SystemVerilog 2009 and 2012 2336
C 2339
Appendix C: Liveness vs. Safety 2339
D 2343
Appendix D: Formal Property Verification Proof Setup 2343
E 2371
Appendix E: Deep Bug Hunting 2371
Overview 2372
Formal Strategy with Overconstraining 2373
Bound Swarm Strategy 2374
Loop Swarm Strategy 2378
Cycle Swarm Strategy 2380
State Swarm Strategy 2383
Trace Swarm Strategy 2388
Trace Search Strategy 2390
Guidepointing Strategy 2391
Simulation Strategy 2393
"hunt" Command Flow 2394
Hunt Limitations 2396
Deadlock Hunting 2399
Additional Resources 2401
F 2402
Appendix F: Custom Radix Files 2402

Radix File Format 2402
Definition Clause Syntax 2402
Use Clause Syntax 2405
Predefined Radix Names 2406
Example Radix Files 2408
Radix File with Tag-Value Definition for Values 0, 1, 2, and 3 2408
Radix File with Color Definitions 2408
Custom Radix JSON File 2410
Header Object 2410
Radices Object 2411
Floating-Point Radix 2412
Fixed-Point Radix 2413
Usages Object 2414
Custom Radix Related Documentation 2414
G 2416
Appendix G: Specifying Enum Values with Tcl 2416
Inside an Instance 2416
Within a SystemVerilog Package 2417
Within a Compilation Scope 2417
Jasper UNR App Case 2419
H 2420
Appendix H: Automatically Extracted Type Properties 2420
Introduction to VHDL Type Properties 2420
Introduction to SystemVerilog Type Properties 2421
Procedural Notes 2421
Type Property Limitations 2422
I 2424
Appendix I: Vacuity Checking with Automatically Extracted Covers 2424
Precondition Reachability Checking 2424
Property Examples 2425
SVA Immediate Properties 2425
Nested Implications 2426
Refining Vacuity Checking with Additional Covers 2427
Managing Vacuity Checking 2429
Enabling and Disabling Related Covers 2430

Infinite and Finite Precondition Proof by Disabling 2430
Limitations 2431
Excluded Cases 2431
Handling for Nested Implications for Local versus Non-Local Variables 2432
J 2433
Appendix J: Understanding Visualize Semantics with SVA Equivalents 2433
SystemVerilog BT/RC Keywords and NOCEX Boolean 2433
SystemVerilog BT Keyword 2433
SystemVerilog RC Keyword 2434
SystemVerilog NOCEX(p) Boolean 2434
SystemVerilog Equivalent of Visualize At Least Once Options 2434
SystemVerilog Equivalent of Visualize Force Options 2436
SystemVerilog Equivalent of Visualize Not Options 2437
SystemVerilog Equivalent of Visualize for a Signal Sequence 2437
K 2439
Appendix K: Jasper Apps-Xcelium Integration 2439
xrun Process Flow 2439
Supported Apps 2441
Jasper Apps-Xcelium Version Compatibility 2441
Launching Jasper with xrun: P2P and CPI2Jetlist 2442
Jasper-Specific xrun Command-Line Options 2444
Tips for Compiling VHDL Designs 2447
Differences in Synthesis for Mixed-Language Designs 2447
Mappings of Integer Parameters to Basic VHDL Enums 2448
Mappings of VHDL "time" to the Verilog "integer" Parameter 2450
L 2451
Appendix L: Covergroup Support 2451
M 2468
Appendix M: Disabled Jasper and Front-End Messages 2468
Disabled Warnings 2468
Disabled Warning or Info Messages 2471
Disabled Error Messages 2472
N 2473
Appendix N: get_design_info Transitivity 2473

Transitivity for Top Instance, "-instance", and "-module" 2473
Transitivity for SystemVerilog Interfaces 2482
O 2484
Appendix O: HDL Sort Analyze Flow 2484
21 2486
Jasper TclHelp Reference 2486
Jasper Command Help 2486

1

Introduction

This chapter is an introduction to the *Jasper Apps Command Reference Manual*. It includes the following sections:

- What's New
- Reference Manual Organization
- Related Documentation
- Jasper Apps Support for Tcl
- UNIX Commands
- Conventions Used in This Manual

Cadence® Design Systems, Inc. prohibits the use of our software in a way that does not comply with our written guidelines and documentation.

What's New

The release of Jasper® Apps Product Version 2026.03 includes many enhancements, new features, and resolved issues. Access details in the following places:

- Jasper Apps Technology Updates on Jasper Apps Technology Updates.
  This page displays a table that contains links to the following:
  - Incremental training slides – See the "General Jasper Technology Updates" column
  - Other collateral such as videos – See the "Feature-Specific Technology Updates" column
  - Release notes – See the "Release Notes" column
- Release notes in your Jasper build: `<path_to_jasper_dir>/doc`

Reference Manual Organization

This command reference manual is organized as follows:

- Chapter 1, "Introduction" Lists new and enhanced features in the current release, introduces knowledgebase resources, provides a basic overview of Jasper Apps Tcl, and defines the conventions used in Jasper Apps documentation.
- Chapter 2, "Installation" Lists platform, language, and standards support and describes product and license installation procedures.
- Chapter 3, "Commands and Variables Overview" Lists Jasper Apps command-line arguments, environment variables, and general usage information. It also provides an overview of Jasper Apps commands with graphical user interface (GUI) equivalents.
- Chapter 4, "Regular Commands" Documents Jasper Apps regular commands, including command options, definitions, and examples.
- Chapter 5, "Configuration Commands" Documents Jasper Apps configuration commands, including command options, definitions, and examples.
- Appendix A, "PSL Support" Lists Jasper Apps support for PSL.
- Appendix B, "SVA Support" Lists Jasper Apps support for SVA.
- Appendix C, "Liveness vs. Safety" Details the difference between liveness properties and safety properties and provides an example.
- Appendix D, "Formal Property Verification Proof Setup" Lists proof and Visualize™ setup commands and indicates whether they apply to the global or task environment.
- Appendix E, "Deep Bug Hunting" Introduces the deep bug hunting feature, supported strategies, and command flow.
- Appendix F, "Custom Radix Files" Provides guidance for formatting custom radix files.
- Appendix G, "Specifying Enum Values with Tcl" Details ways to define an enum.
- Appendix H, "Automatically Extracted Type Properties" Describes Jasper Apps support for VHDL and SystemVerilog type properties and includes procedural notes and limitations.
- Appendix I, "Vacuity Checking with Automatically Extracted Covers" Introduces proof vacuity checking features and includes examples and guidance for managing vacuity checking.

- Appendix J, "Understanding Visualize Semantics with SVA Equivalents" Provides SVA examples intended to explain the meaning of various Visualize commands.
- Appendix K, "Jasper Apps-Xcelium Integration" Presents the xrun process flow, Xcelium™Jasper Apps version compatibility, methods of launching Jasper Apps with the xrun flow, and so forth.
- Appendix L, "Covergroup Support" Lists Jasper Apps covergroup support with limitations and examples.
- Appendix M, "Disabled Jasper and Front-End Messages" Lists Jasper and front-end messages that are disabled by default. You can use the command `set_message` to enable the messages.
- Appendix N, "get_design_info Transitivity" Lists the transitivity of `get_design_info` scopes: top instance (no scope switch), `-instance`, and `-module`. This appendix also identifies limitations and exceptions.
- Appendix O, "HDL Sort Analyze Flow" Provides examples and limitations of the HDL Sort Analyze flow introduced in the 2026.03 FCS release.

Related Documentation

This book has a companion book, the *Jasper Platform and Formal Property Verification App User Guide*, which introduces the apps and related windows. You can access this book and the companion user guide from the tool's *Help* menu. Other resources available from the Jasper Apps *Help* menu include the following:

- Stand-alone app-specific user guides
- Stand-alone PDF mini guides that describe the Visualize GUI, guide engine selection, and so forth
- Proof Accelerator data sheets
- Searchable HTML Tcl command help

> You can also type `help` on the command line for a list of all commands or use `help <command_name> -gui` to display documentation on a specific command in the Jasper Help Browser.

In addition, Cadence offers Cadence Application Support and Knowledge (ASK), a searchable knowledgebase that includes release version information, quick tip videos, answers to frequently asked questions, application notes, manuals, tutorials, technology updates (incremental training), and other resources.

New and updated resources are continually added to the site. This site requires a one-time registration. To register, go to the Cadence ASK website (support.cadence.com) and follow the new-user registration instructions.

Jasper Apps Support for Tcl

Jasper Apps supports native Tcl. Tcl is a simple programming language. Regardless of whether you have experience programming in Tcl, you can learn enough to write interesting Tcl scripts within a few hours. Refer to the application note "Scripting with Tcl in Jasper Apps" for new-user tips and techniques to handle challenges, such as referencing objects in an HDL. The app note is available from the menu option *Help – Application Guides – Scripting with Tcl in Jasper Apps*.

> A single `#` in any Tcl expression is treated as a "starting comment" tag.

UNIX Commands

During a Jasper Apps session, you can run some UNIX® commands in one of the following ways:

```
exec {<command>}
```

```
exec "<command>"
```

You can also run a process in the background with `&`; however, this command returns a `pid` with a dummy value, so you cannot use it to identify the process.

Example:

```
exec "pwd"
```

```
exec xterm &
```

For commands with multiple arguments, specify each argument separately with any combination of pairs of curly braces or quotes, for example,

```
exec {ls} "-l"
```

```
exec {ls} {-l}
```

Specify sub-commands inside square brackets, for example,

```
exec {ls} "-l" [exec pwd]
```

- Commands that access the tty do not work. And in some cases, they can cause the tool to hang. For example, `top` must access the tty and, since `top` cannot open the tty, the tool generates an error message. Jasper Apps recognizes the following system commands (without `exec`): `as`, `ex`, `more`, `vi`, and `vim`. If you run any of them, the tool issues an error. Example:

  ```
  vi test.txt
  ```

  ```
  ERROR (ECD036): "vi" requires a tty and is not supported in Jasper.
  ```

  This handling prevents the console from hanging. If you use the `exec` command to run one of the system commands listed above, (for example, `exec more test.txt`), Jasper Apps does not issue an error.
- You can start a process in the background from the Tcl prompt.
  Example: `exec xterm &`

Conventions Used in This Manual

The following tables list conventions used in syntax and text.

Jasper Apps Syntax Convention	
Convention	**Definition**
`Courier font`	Indicates text you will type on the command line.
`-underscore_separation`	
	Indicates a command switch. Switches are not case-sensitive.

`[ ]`	Indicates optional arguments. Do not type the square brackets.
`|`	Indicates a choice (a logical OR) among alternatives. Do not type the vertical bar.
`\`	The backslash character (\) at the end of a line indicates that the command you are entering continues on the next line.
`*`	Indicates the preceding argument appears zero or more times per command.
`+`	Indicates the preceding argument appears one or more times per command.
`‘ ’`	Indicates the enclosed character(s) should be explicitly included on the command line. Do not type the single quotation marks.
`<`*`Italics`*`>`	A command option that you will replace with a valid value. Do not type the angle brackets.
`( )`	Used as a grouping convention. Do not type the parentheses. For example, parentheses in the following syntax indicate that if you use the `-bbox` option, you will follow it with either the `0` or `1`. `[-bbox (0 | 1)]`

Jasper Apps Text Conventions	
Convention	**Definition**

`Courier font`	This style indicates: • Text you will type in GUI fields • Commands and options • File names and paths • Code samples
Italics	User interface items such as button and dialog field names.
Menu – Option	GUI command sequence. Example: *Help – Command Reference Manual* Meaning: Click on the *Help* menu and choose the option *Command Reference Manual*.
Blue text	Hyperlinked cross-reference. When you view the PDF version of Jasper Apps manuals from a computer screen, click on the blue text to view related information.

2

Installation

This chapter provides instructions for installing Jasper Apps and licenses. It includes the following sections:

- Supported Platforms
- Supported Languages and Standards
- Jasper Apps Installation
- License Installation
- License-Related Environment Variables

Supported Platforms

There are two versions of Jasper Apps, one for the Intel® Architecture (x86_64) and another for the ARM Architecture (AArch64). Check below for the supported platforms for each.

Supported Platforms for Intel Architecture (x86_64)

Jasper Apps supports the following operating systems in Intel Architecture (x86_64):

- Red Hat Enterprise Linux 8.x (64-bit)
- Red Hat Enterprise Linux 9.x (64-bit)
- Rocky Linux 8.x (64-bit)
- Rocky Linux 9.x (64-bit)
- SUSE Linux Enterprise Server 15 SP3+ (64-bit)

The `lmutil` tool does not work with Red Hat Enterprise Linux 9.x (64-bit) and Rocky Linux 9.x (64-bit). While that is not fixed, you can use `lmutil` with any other supported OS or check the following workaround: Running lmgrd and cdslmd on RHEL 9.

For Jasper Apps built with Intel Architecture, Jasper engines can also be dispatched to ARM Architecture (AArch64) machines for the following operating systems:

- Red Hat Enterprise Linux 8.x (64-bit)
- Red Hat Enterprise Linux 9.x (64-bit)
- Rocky Linux 8.x (64-bit)

Supported Platforms for ARM Architecture (AArch64)

Jasper Apps supports the following operating systems in ARM Architecture (AArch64):

- Red Hat Enterprise Linux 8.x (64-bit)
- Red Hat Enterprise Linux 9.x (64-bit)
- Rocky Linux 8.x (64-bit)

The ARM version does not support:

- Decryption of encrypted files using algorithms RC2, RC4, and RC5
- CPI flow
- Dispatching engines to x86_64 processor

The `lmutil` tool does not work with Red Hat Enterprise Linux 9.x (64-bit). While that is not fixed, you can use `lmutil` with any other supported OS or check the following workarounds:

- Running lmgrd and cdslmd on RHEL 9
- ```
  sudo bash -c 'if [ ! -e /lib/ld-lsb-aarch64.so.3 ];
  then ln -s /lib/ld-linux-aarch64.so.1 /lib/ld-lsb-aarch64.so.3; fi'
  ```
  ```

CHANGE IN OPERATING SYSTEMS: Jasper is now compiled on Red Hat Enterprise Linux 8.4 (x86_64) and Red Hat Enterprise Linux 8.6 (AArch64). From now on, Red Hat Enterprise Linux 7, CentOS 7, Ubuntu 14.04 LTS, SUSE Linux Enterprise Server 12, and SUSE Linux Enterprise Server 15 SP0/SP1/SP2 systems are no longer able to startup Jasper.

> The SUSE Linux Enterprise Server 15 SP3+ (x86_64) systems are still supported by Jasper Apps.

IMPORTANT INFORMATION ON KERNEL SUPPORT: Currently, a bug in kernel versions 4.18.0-358.el8_6 through 4.18.0-372.19.el8_6, which are present in Red Hat Enterprise Linux 8.x operating systems, prevents you from running Jasper Apps. You must upgrade the kernel version to run Jasper. For additional information about this issue, consult the QTBUG-100174 bug report.

Supported Languages and Standards

Jasper Apps supports the following languages and standards:

- Verilog® 95 (IEEE® Std 1364™-1995) and 2K (IEEE Std 1364-2001)
- VHDL (IEEE Std 1076™-1993), 2K (IEEE Std 1076-2002), and VHDL ‘08 (IEEE Std 1076-2008)
- Mixed-language (Verilog and VHDL)
- IEEE Std 1850™-2005 *Standard for Property Specification Language (PSL)*:
  - PSL-Verilog
  - PSL-VHDL
- SystemVerilog (IEEE Std 1800™-2005, IEEE Std 1800-2009, IEEE Std 1800-2012, and IEEE Std 1800-2023) and embedded SVA in SystemVerilog designs
- Tcl 8.6.10 (https://www.tcl.tk/man/tcl/TclCmd/contents.htm)
- IEEE 1685™ IP-XACT
- UPF 1.0 and UPF 2.0 (IEEE Std 1801™-2009) and UPF 2.1 (IEEE Std 1801-2013)
- Synopsys® Liberty™ library modeling standard (version 2014.09)

Jasper Apps Installation

This installation guide assumes you are installing Jasper Apps under the `/usr/local` directory. If you choose to install the tool in any other directory, adjust the instructions by replacing `/usr/local` with your actual installation directory.

Jasper Apps requires you to set one or more environment variables. How you set variables depends on the shell you use. In the following installation example, we assume the shell is `csh`. Adjust the installation steps based on your unique shell. Also, we recommend you set environment variables in the shell startup file (for example, `.cshrc` for `csh`, `.profile` for `sh`).

1. Download the Jasper Apps tar kit from the Cadence software download site (downloads.cadence.com).

   - To download the tar kit for the Intel Architecture (x86_64), select the "Linux" option from the *Default Operating System* list. The ARM Architecture (AArch64) kit is downloaded if you choose the "Other" option.
   - Use your Cadence Application Support and Knowledge (ASK) login. If you are not a registered user, sign up at Cadence ASK website (support.cadence.com). If you cannot log in, contact Customer Support.

2. Uncompress and untar the kit using the `tar` command. This creates a directory named `jasper_<version>`.
   Example:

   ```
   tar -zxvf <jasper_kit_name>.tgz
   ```

3. Start the license manager (`lmgrd`) or reread the license file as appropriate. See "License Installation"
4. Set the `LM_LICENSE_FILE` variable. The appropriate setting depends on the existing licensing environment. Replace `host_name` with the license server host name and replace `port` with the TCP/IP port number that the license manager daemon was started on.

   ```
   setenv LM_LICENSE_FILE port@host_name
   ```

> Refer to "License-Related Environment Variables" to learn how you can resolve launch delays when running the tool from remote locations.

5. Modify the `PATH` variable to add `<path_to_jasper_dir>/bin` to the executable path.

   ```
   setenv PATH ${PATH}:/usr/local/<path_to_jasper_dir>/bin
   ```

6. Verify that the installation is complete by typing `jaspergold` on your command line.

> The `<path_to_jasper_dir>/doc/example_jasper_apps` directory includes scripts that illustrate sessions for VHDL PSL, VHDL SVA, Verilog SVA, and mixed-language VHDL/Verilog SVA. All the files you need to run these scripts are also located in the example directory.

License Installation

Licensing is usually site specific. The license file includes instructions. The instructions in this chapter and in the Jasper installation build at `/doc/INSTALL.pdf` is an overview. For complete details, consult your FlexNet Publisher documentation.

> Cadence has a longstanding advisory to avoid using the `lmremove` feature in FlexNet with `cdslmd` (see License Alert – LMREREAD and LMREMOVE on Cadence Application Support and Knowledge for more details).

This section includes the following topics:

- TIMEOUT and TIMEOUTALL
- Redundant License Servers
- Installing Your License
- Appending Lines to a License File
- Manually Starting a New Daemon
- Using the Correct License Server

- Using FlexNet Publisher Utilities

TIMEOUT and TIMEOUTALL

The FlexNet variables `TIMEOUT` and `TIMEOUTALL` specify the idle time-out for one or all features, returning them to the free pool for use by another user.

If the license server administrator specifies the `TIMEOUT` or `TIMEOUTALL` settings in the options file for the vendor daemon, the licenses checked out by an idle Jasper Apps session are automatically checked in after the specified time. Consult the FlexNet documentation for additional information.

- The minimum allowed value of `TIMEOUT` and `TIMEOUTALL` is 900 seconds (15 minutes).
- Jasper Apps informs the license server that it is idle if no activity has occurred for eight minutes. Idle state includes no running Tcl command in an analysis session, no running background proof, and no keystrokes or mouse clicks.

When the tool goes from being idle to non-idle (for example, when a Tcl command runs), the tool attempts to check out all licenses that were previously checked in. In batch mode, if the check-out fails, the tool exits. In an interactive GUI session, the tool displays a License Lost dialog with advice to save your work in case it is unable to recover licenses.

Redundant License Servers

FlexNet Publisher supports three-server redundancy for license servers. With this arrangement, two of the three license servers must be up and running for proper functioning. Three-server redundancy is designed to provide hardware failover protection. One of the three servers is the "master," the only server capable of issuing licenses, and all clients must contact the master.

Installing Your License

The following is an illustrative example only. It assumes that the license daemon will be run on the same machine that Jasper Apps will run on. If this assumption is not valid, contact your system administrator.

> Use `lmutil lmdown` instead of `lmutil lmreread` when you get a new Jasper license.

1. Install the license file in the customer-specific location. For example:

   ```
   cp license.dat /usr/local/<installation>/etc/license
   ```

2. Edit the license file as follows:
   - Edit the path to the vendor daemon (that is, the path following `VENDOR cdslmd` in the license file) so that it points to the path `/usr/local/<installation>/Linux64/bin/cdslmd`.
   - Replace `<installation>` with your installation name.

     > The path must be a fully expanded path. The FlexNet Publisher license manager will not expand an environment variable.

3. Start the license manager.

   ```
   /usr/local/<installation>/bin/lmgrd -c <PATH_TO_LICENSE_FILE> -l LM_LOG_FILE
   ```

   > The `-l LM_LOG_FILE` is optional but highly recommended. You can use it for debugging.

Licensing should work correctly at this stage. If you encounter license issues with a Jasper Apps software installation, check the environment variables `LM_LICENSE_FILE` and `CDS_LIC_FILE` to ensure that they are set to the correct paths. If you continue to have problems, look for a hidden `.flexlmrc` file in your home directory and remove the line that includes `CDS_LIC_FILE`.

Appending Lines to a License File

License files begin with a `SERVER` line. In the case of three-server redundancy, there are three `SERVER` lines. `SERVER` lines are followed by one or more `VENDOR` lines, which are followed by `FEATURE` lines. You can modify the following elements in the license file and use the line-continuation character (`\`) to break long lines.

- **SERVER lines –** Host names and optional TCP/IP port numbers

> Cadence ships license files with `Cadence_SERVER` as hostname in the `SERVER` line. Edit this field to enter the actual hostname prior to starting the license server. If you do not change the port number, end users must update `LM_LICENSE_FILE` or `CDS_LIC_FILE`. (See "License-Related Environment Variables".)

Example (bash):
`export LM_LICENSE_FILE=5280@server`

- **VENDOR lines –** paths, options file path, optional TCP/IP port numbers (for firewall support)
- **FEATURE lines –** values in `keyword=value` pairs

> In general, a `FEATURE` line describes the license required to use a product. An `INCREMENT` line incrementally adds licenses to a prior `FEATURE` or `INCREMENT` line.

The following is a sample of the license keys section of a license file.

```
SERVER qa2 0026b92a4b75 5280

DAEMON cdslmd ./cdslmd

USE_SERVER

# DO NOT REMOVE THE USE_SERVER LINE

FEATURE jasper_fpv cdslmd 2018.12 31-dec-2018 5 4EC2EF327ED1B81G75E5 \

         VENDOR_STRING=J:DEMO DUP_GROUP=NONE vendor_info=31-dec-2018 \

         ISSUED=31-dec-2018 SN=2016-09-24Q11:06:22:637 SIGN2="1961 CE67 \

         2A5C 5CBF 65FE 0C37 A52B 3FCB 0A3F 2967 746D BC94 1B16 6ABD \

         6A14 1031 B158 B973 641F 57B1 2380 C699 651A 260D 5D26 351D \

         F192 2674 54F5 5B4C" V7.1_LK=6E66EB3997CDAD2A8CCC

FEATURE jasper_interactive cdslmd 2018.12 31-dec-2018 5 \

         9E06ACA968L410130B9E VENDOR_STRING=J:DEMO DUP_GROUP=NONE \

         vendor_info=31-dec-2018 ISSUED=31-dec-2018 \

         SN=2016-06-25T21:06:21:637 SIGN2="1445 62G7 9686 BAB3 FC24 \

         1154 B74C CDD3 64EA FC8A 9FFA 725D A6C4 9477 80EB 2B77 2A10 \

         DD52 C7D6 C0A7 F299 FBC2 9045 3909 218C F6B1 0AAB 1310 A500 \
```

```
         4A30" V7.1_LK=BEA62BA93DCE41750BAD

# ... (snip)

#

# These portions of a sample of the product license keys section are provided as

# an example of what a license file looks like. Your actual license will include

# multiple FEATURE lines specific to your case.
```

Manually Starting a New Daemon

The license server system has two components, `lmgrd` and the vendor daemon. The job of `lmgrd` is to start and maintain the vendor daemons listed in the `VENDOR` lines of your license file and refer the Jasper Apps features checkouts to the correct daemon.

> If you are running a three-server redundant license server system, keep a separate copy of the license file and the `lmgrd` and the vendor daemon binaries on each of the three servers.

To manually start `lmgrd` from the command line, use the following syntax:

```
lmutil lmgrd -c <license_file_list> -l [+]<debug_log_path>
```

- **license_file_list –** This variable points to one or more files. You can include one or more paths to single license files or directories that contain license files. In the case of the latter, `lmgrd` uses all the license files in the directory.
- **debug_log_path –** This variable is the full path to the debugging log file. Use the `+` character to append logging entries.

> Use `lmutil lmgrd -help` for additional information.

Using the Correct License Server

Jasper Apps uses FlexNet Publisher licensing from Flexera Software, version `11.17.2.0`, which is included in the product package. If you use a FlexNet Publisher license server other than the one that ships with the tool, it must be version `11.16.4.0` or newer. When you upgrade the FlexNet

Publisher license server, if you have specified a path to the vendor daemon on a `VENDOR` line in the license file, you must change it so it points to a supported version of the daemon.

When you upgrade your license server, use the following procedure.

1. Stop the license server.
2. Copy the new `cdslmd` over the old `cdslmd`.
3. Start the license server.
4. Verify that the update was successful by running `lmstat`.
   When you run `lmstat`, it will tell you the version of the vendor daemon that is being used. For example, the following return is for a license server named `cyklop`:
   `Vendor daemon status (on cyklop): cdslmd: UP v11.16.4.0`
   If you do not see `v11.16.4.0` in the return, the update was not successful. In that case, carefully redo the steps listed above.

   > If Jasper tries to check out a license from an older version of the license server, the attempt will fail. However, Jasper does not "know" that the failure is fatal, which means that at start-up, it will make more check-out attempts according to the values specified by the `-wait_interval` and `-wait_retries` command-line arguments. (See "Command-Line Options".)

   > The default value of `-wait_interval` is 60 seconds, and the default value of `-wait_retries` is unlimited, which means that Jasper will attempt to check out licenses until it is interrupted (for example, by pressing `Ctrl+C`). When a license checkout attempt fails due to the license server being too old, an INFO message will be included in the log file for the console process (`jg_console.log`) containing the error number `-83` and the message text `Version of vendor daemon is too old.` Example:

```
2021-09-21 14:27:25.613477 (121704) lic_core INFO    (1) checking out
'jasper_interactive' ...

2021-09-21 14:27:25.935431 (121704) lic_core INFO    ... (1) checkout failed

(error: -83) ("FLEXnet ERROR(-83, 0, 0): Version of vendor daemon is too old.")
```

   If you see this text, you must update the license server.

Using FlexNet Publisher Utilities

The FlexNet Publisher utilities are packaged as a single executable called `lmutil`. You can install `lmutil` as individual commands by creating links to the individual command names or making copies of `lmutil` as the individual command names. Or you can install it as a wrapper that runs the individual command as the `lmutil` command (for example, `lmstat` or `lmdown`).

The following helpful utilities are described in this section:

- lmdown
- lmhostid
- lmstat

lmdown

`lmdown` shuts down selected license daemons (both `lmgrd` and selected vendor daemons).

> Use `lmutil lmdown` instead of `lmutil lmreread` when you get a new Jasper license.

`lmutil lmdown` `-c <license_file_list> [-vendor <vendor_daemon>] [-q] [-all] [-force]`
`-c license_file_list`
Specifies the license files you want to shut down.
`-vendor vendor_daemon`
Shuts down the specified vendor daemon, but `lmgrd` continues to run.
`-q`
Keeps the utility from prompting or printing a header. If you don't use this option, `lmdown` prompts with, "`Are you sure? [y/n]:`."
`-all`
Automatically shuts down all the specified servers.
`-force`
Use this option when you have borrowed licenses. It shuts down the license daemons for the machine where the license server system is running.

lmhostid

`lmhostid` returns the FlexNet Publisher licensing hostid of the current platform. If you run this command without any arguments, it displays the default hostid type for the current platform.

`lmutil lmhostid` `[-n] [-utf8]`	
`-n`	Returns the hostid as a string.
`-type`	Returns the default hostid for the current platform.
`-utf8`	Displays the hostid as a UTF-8 encoded string rather than an ASCII string. Use this option if your hostid contains other than alpha-numeric characters.

> Consult the FlexNet documentation for additional information.

lmstat

`lmstat` returns the status of all network licensing activities (for example, running daemons and individual feature users).

`lmutil lmstat`
`[-a] [-c <`*`license_file_list`*`>] [-f [<`*`feature`*`>]]`
`[-i [<`*`feature`*`>] [-s [<`*`server`*`>] [-S [<`*`vendor`*`>]] [-t <`*`timeout_value`*`>]`	

`-a`	Displays all information.
`-c` *`license_file_list`*	Reports on the specified license file(s).
`-f` *`feature`*	Displays a specified feature's users. If you do not specify a feature, it displays information for all features.
`-i` *`feature`*	Displays information from the `FEATURE` line for the specified feature. If you do not specify a feature, it displays information for all features.

`-s server`	Displays the status of all the license files that are listed in `$VENDOR_LICENSE_FILE` or `$LM_LICENSE_FILE` on the specified server. If you do not specify a server, it displays information for all servers.
`-S vendor`	Lists all users of the specified vendor's features.
`-t timeout_value`	Sets the connection time-out (time spent attempting to connect) to the value of *timeout_value*.

License-Related Environment Variables

This section includes information on the `CDS_LIC_FILE` and `CDS_LIC_ONLY` optional environment variables. Refer to "Environment Variables" for additional information about environment variables you might find helpful.

The location of the Jasper license file is defined by searching the following in order:

1. The setting of the `CDS_LIC_FILE` environmental variable.
2. The setting of the `LM_LICENSE_FILE` environmental variable.

There are two kinds of delays you might encounter when checking out licenses:

- The license exists, but Jasper Apps must search in many license file locations before it finds the license.
- The license does not exist or is not available, and Jasper Apps must search many license locations before finding out.

With the environment variables `CDS_LIC_FILE` and `LM_LICENSE_FILE`, you can expect the following optimizations:

- Defining `CDS_LIC_FILE` makes Jasper Apps look first into it for licenses and then at `LM_LICENSE_FILE`, if it is defined, which minimizes the first type of delay.
- Defining `CDS_LIC_ONLY` makes Jasper Apps ignore `LM_LICENSE_FILE` and only query the servers from the `CDS_LIC_FILE`, which minimizes the second type of delay.

Use the following example to define the environment variables `CDS_LIC_FILE` and `CDS_LIC_ONLY`:

```
export CDS_LIC_FILE=5280@my_license_server_machine
export CDS_LIC_ONLY=1
```

3

Commands and Variables Overview

This chapter provides a quick look at Jasper Apps commands. It includes tables for the following:

- Command-Line Options
- Environment Variables
- Command Summary
- General Command Usage

Command-Line Options

By default, Jasper Apps runs in GUI mode and opens the main Jasper Apps window when you launch the tool. To launch Jasper Apps with the default Formal Property Verification App, type either of the following at the command prompt:

```
jaspergold
jg
```

To launch a different app, use one of the app switches listed in the table below.

Syntax

```
jaspergold
    [options]
jaspergold
    -batch [options]
```

The following table lists and briefly describes the Jasper Apps command-line options.

Command-line options are case insensitive. For example, the tool recognizes all of the following:

- `jaspergold -COMMAND`
- `jaspergold -command`
- `jaspergold -CommAND`

Command-Line Options	Description

`<`*file_name*`>.tcl`	Run one or more Tcl scripts with the required `.tcl` extension in the order they appear on the command line as if they were entered interactively.
	Example:
	`// Run script1.tcl and then run script2.tcl.`
	`jaspergold script1.tcl script2.tcl`

`-help`	Display command-line help information (the contents of this table).
`-version`	Display the version string.
`-afv`	Launch the Architectural Formal Verification App.
`-c2rtl`	Launch the Jasper C2RTL App.
`-caf`	Launch the JasperC C Autoformal App.
`-cdc`	Launch the Clock Domain Crossing Verification App.
`-cfpv (cpp | systemc)`	Launch the C Formal Property Verification App.
`-conn`	Launch the Connectivity Verification App.
`-cov [-offline]`	Launch the Coverage App. Use `-offline` to launch the Coverage App in offline mode. With this mode, you can load the coverage database without loading the elaborated design. Also see `-cov_init "`*arguments*`"`.

`-csr`	Launch the CSR Verification App.
`-fpv`	Launch the Formal Property Verification App.
`-fsv`	Launch the Functional Safety Verification App.
`-hls`	Launch the HLS Equivalence Checking App.
`-lpv`	Launch the Low Power Verification App.
`-rvv`	Launch the RISC-V Verification App.
`-sec`	Launch the Sequential Equivalence Checking App.
`-spv`	Launch the Security Path Verification App.
`-superlint` `[-lint_dft]`	Launch the Superlint App. Use `-lint_dft` to launch the Superlint App with access to Lint checks only.
`-unr`	Launch the Coverage Unreachability App.
`-xprop`	Launch the X-Propagation Verification App.
`-no_jges`	Disable Jasper Expert System on invocation.
`-batch`	Launch the tool in batch mode.
`-no_gui`	Start the tool without launching the GUI, allowing Tcl command entry from the terminal.
`-proj <`*`dir_name`*`>`	Use the specified directory as the project directory. If you do not use `-proj` to specify a directory, the tool uses `./jgproject`. The project directory stores various files, including log files, debugging files, and settings related to the current project. `-proj` does not support a lone forward slash (`\`) or empty string.

`-acquire_proj`	Forcibly acquire ownership of the project directory if it is already locked.
`-tcl <tcl>`	Execute the specified Tcl script in `session_0`.
`-command <cmd>`	Run the specified Tcl command after running all specified Tcl scripts and before receiving interactive commands. Add multiple commands by enclosing multiple commands in single or double quotes and separating the commands with a semi-colon (`;`). Example: `jg -command 'include setup.tcl; prove -all; report -all -file foo -force'`
`-db <file_name>`	Load the specified database. Use this command to continue work on a saved database.

`-define`
`<tcl_variable>`
`<tcl_value>`
	Define a Tcl variable and its value for `session_0`.
	Implementation guidance:
	• This switch is especially useful for launching two analysis sessions, each with a different version of the RTL, so you can bring up the corresponding waveforms for comparison.
	• Define multiple variables by using one `-define` switch for each definition.

`-remote_mode <mode>`
 	
`-remote_console`
	Use the specified mode to define the configuration for launching a session on a remote machine.
	• `local` – Launch a remote analysis session on your local machine.
	• `lsf` – Launch a remote analysis session on the IBM® Platform LSF® cluster that is expected to be available using the `LSF_BINDIR` environment variable. See "LSF_BINDIR".
	• `oge` – Launch a remote analysis session on the Oracle® Grid Engine that is expected to be available using the `SGE_ROOT` environment variable. See "SGE_ROOT".
	Any other value is regarded as a custom shell script for the mode `shell`.
	Use `-remote_console` to launch the default `session_0` on the remote machine.
	Also see `session` to learn how to launch remote sessions from the console.

`-no_wait`	Do not wait for a Jasper Apps license to become available. By default, if a Jasper Apps license is unavailable, the tool waits for one to become available. With this switch, the tool errors out instead.
`-wait_interval N`	In the event a license is not available, wait the specified number of seconds and then retry license checkout. By default, Jasper Apps tries a license checkout every 60 seconds.
`-wait_retries N`	In the event a license is not available, try to check out a license the specified number of times. By default, the number of retries is unlimited.

`-block_licenses`
 	
`<`*`licenses`*`>`
	Tell Jasper to ignore the license specified in the argument. The argument must consist of valid Jasper license feature names separated by either a comma or spaces.
	
	• If both the environment variable `JASPER_BLOCK_LICENSES` (see Table 3-2) and this command line switch are present, the command-line switch takes precedence and the environment variable is completely ignored.
	• ABVIP- and IPK- licenses cannot be added to the block list.
	• You can specify an empty list, which means that Jasper will not ignore any license.

`-title <`*`title`*`>`	Prepend all main window titles with the specified string. If your string includes spaces, you must enclose it in quotation marks.
`-bg`	Change the background color of GUIs to the specified color. `-bg` supports x11 colors. For color choices, use a web browser and the search string `x11 color`. Also see the "GUI Settings" appendix in the Jasper user guide.
`-fg`	Change the foreground color of GUIs to the specified color. `-fg` supports x11 colors. For color choices, use a web browser and the search string `x11 color`. Also see the "GUI Settings" appendix in the Jasper user guide.
`-style (legacy | crisp)`	Configure the look and feel of the Jasper GUI. • `legacy`: Follows 2025.12 FCS and earlier releases • `crisp`: Available in 2026.03 FCS and later releases
`-display <`*`display`*`>`	Configure the display into which the remote session should be launched.

`---`	Specify the end of options and arguments to be handled by Jasper. The tool ignores all arguments after the first `---` with the exception that they will be available in Tcl's `argv` variable and present in log files. See "Passing Information to the Jasper Tcl Environment".
`-debug_handoff [<database_name> [-trace_id_list <id_list>]]`	Launch the tool with the Debug Handoff GUI. It provides a simplified user interface to browse and debug the properties and includes Visualize windows captured in the Debug Handoff database. Display only the listed traces when launching Debug Handoff with a specified database. Example: `debug_handoff db.dho -trace_id_list 1 3 5`
`-allow_unsupported_OS`	
	Suppress the non-supported OS warning and launch the tool if possible.
`-tool_inactivity_delay <n>`	
	Set a minimum inactivity delay for the tool in seconds. The minimum delay allowed by Jasper is 480 seconds (480 s). If a lower value is specified with the `-tool_inactivity_delay <n>` command line option, a warning is issued to `jg_console.log` and the value 480 seconds is used.

`-cov_init`
 	
"*arguments*"
	Enable the Coverage App with the specified options (see list of options in the next table). Enclose the arguments in quotes, for example:
	
	`jg -cov_init "-model all -type all -precision high"`
	
	`jg -cov_init "-coverage b -covfile ccf.txt"`
	
	This command also enables export of the active task as a UNICOV database to be read by IMC/Verisium Manager. To export multiple tasks, use the `-task` option with the following syntax:
	
	`-task {scope1::A:B C scope2::D}`
	
	• `scope1` and `scope2` are verification scopes.
	• `A`, `B`, `C`, and `D` are tasks.
	• `*` can be used to represent all tasks.
	
	• All options supported by `check_cov -init` command are supported. Options used to enable coverage in Xcelium simulator (`-coverage`, `-covfile`) are also supported. The two sets can be mixed. To combine Jasper and Xcelium coverage results, use the same options as Xcelium. Refer to the full `check_cov` help for supported `check_cov -init` options and to the `Integrated Coverage User Guide` in Xcelium for help with supported Xcelium options.
	• You must specify at least one option among `-model`, `-coverage`, and `-covfile`.

The table below lists the `-cov_init` arguments.

-cov_init Argument	**Description**
`-app`	Use this switch in the argument to specify a list of Jasper Apps for Unicov export. Supported options include `fpv`, `conn`, `cov`, `csr`, `unr`, `xprop`, `spv`, and `all` (the default). `all` selects all Jasper Apps **except** UNR.

`-coverage`
 	
`<coverage_option>`
	Enable coverage instrumentation.
	
	Supported options:
	• `block (b)`
	• `statement (s)`
	• `expression (e)`
	• `functional (u)`
	• `toggle (t)`
	• `all`
	
	Either specify the switch multiple times to specify multiple coverage types or specify types in colon-separated format.
	
	Use example: `-coverage b:s:e`
	`-coverage u`

`-covdut` `<DUT string>`	Select DUT for Coverage. Specify the switch multiple times to specify multiple DUTs. Example: `-covdut dut1 -covdut dut2`
`-covworkdir` `<dir_name>`	Select coverage workdir.
`-covscope` `<dir_name>`	Select coverage scope name.
`-covtest` `<dir_name>`	Select coverage test name.

`-covfile <file>`	Specify coverage instrumentation control file. Specify the switch multiple times to read multiple files.
`-covoverwrite`	Enable overwrite of coverage output files.
`-covcleanworkdir`	Remove coverage working directory.
`-write_metrics`	Dump the vsof file.
`-interval <time>`	Interval of dumping intermittent UNR results. Specify the time interval as an integer followed by `s`, `m`, or `h`. Example: `-interval 4h` (for 4 hours)
`-waveform`	Save waveform to Visualize property results.

`-witness_resolution`
`any_pass`
`| any_unr`
`| all`	

	Preference among witnesses to get saved to vsof. Default: `any_pass`
`-task`	Specify the tasks to be exported to Unicov database. Default : current set task at time of export Example: `-task {scope1::A:B C scope2::D}` . `scope1`, `scope2` are verification scopes. . `A`, `B`, `C`, `D` are tasks.

`-exclude`
`const`
`| assume`
`| reset`
`| waived`
`| disabled`
	Specify options to exclude cover items from the Unicov database.
	Specify the switch multiple times to specify multiple types.
	Default: `none`
	Example : `-exclude const -exclude waived`

`-disable_user_proven_status`	
	Do not export marked/bounded proven validity status on properties to Unicov database.
`-use_native_ucis`	Use the UCIS library shipped with Jasper.

`-launch_imc`	View the last generated Unicov database with IMC. The tool ignores this switch if you have defined multiple tasks with the `-task` switch.
`-tester_mode`	Launch Jasper in tester mode. In tester mode, the tool writes a vsof (which can be read in vManager) upon exiting. It does not generate a Unicov database.
`-no_vhdl_shortcircuit`	
	Disable short-circuited expression for VHDL operators `and`, `or`, `nand`, and `nor` of bit and Boolean type operands.
`-include_assign_scoring`	
	If you have already specified the "branch" model to be generated, use `-include_assign_scoring` to also generate branch cover items for Verilog continuous assignments.
`-save_setup_script`	
	If you specify the `-save_setup_script` switch, the elaborated database will not be saved and the flow will be the `no_setup` flow.

Environment Variables

This section includes the following:

- Passing Information to the Jasper Tcl Environment
- Environment Variables Used by Jasper

The log file for the Jasper console process (`<jgproject>/jg_console.log`) contains information about important environment variables that were in effect during the current run (such as those related to the license environment). You can find this information in the log in the section "Important environment variables."

Passing Information to the Jasper Tcl Environment

Jasper supports two features that work together to make it possible to easily pass information from your shell (for example, bash) to the Jasper Tcl environment without using environment variables. The two features are:

- The Tcl variables `argc`, `argv`, and `argv0`
- The command line option `---` (three hyphens)

Similar to the standard Tcl shell tclsh, Jasper sets the following Tcl variables when an analysis session starts:

- `argc`: An integer that is the number of elements in `argv` (that is, the same value as `llength $argv` returns)
- `argv`: A list that contains all arguments on the Jasper command line
- `argv0`: A string that is the full path to the Jasper start-up script
- `tcl_interactive`: Always has the value `1`, which means the Tcl shell is invoked interactively

- For an understanding of the settings for these variables, review the tclsh documentation available at https://www.tcl.tk/man/tcl8.6/UserCmd/tclsh.htm.
- The variables are available when Jasper sources the `JGRC` file, so it is possible to use them for the setup you might specify in this file.

Jasper assigns values to these variables in a way similar to the way tclsh assigns their values when running interactively (even in batch mode) because Jasper always treats the Tcl files specified on the command line as if the Tcl scripts in them were entered from the command prompt. That is, from the perspective of the Tcl interpreter, Jasper is always running in "interactive mode."

To make it possible for you to send any kind of arguments (even arguments and options that Jasper

normally considers invalid) to `argv` use the command-line option `---` (three hyphens). This command line option tells Jasper to stop parsing the command line arguments that follow it. The only thing Jasper does with the arguments that follow `---` is add them to `argv` (and potentially print them to log files). This feature is best illustrated with examples.

Example 1:

```
$ jg -batch test.tcl --- does-not-exist.tcl -illegal_argument
.
.
.
% puts $argv
-batch test.tcl --- does-not-exist.tcl -illegal_argument
% puts [llength $argv]
5
% puts [ls test.tcl]
test.tcl
% exit -force
.
.
.
```

(The file `test.tcl` is the same file generated in Example 2 below.)

Example 2:

This example illustrates how the Tcl interpreter in Jasper always run in interactive mode. Lines that start with `$` contain commands run in the shell (normally bash) and lines starting with % contain commands that are run by Jasper. In this example, you can see that abbreviated commands are expanded (for example, `llengt` expands to `llength`) and unrecognized commands are "auto executed" (`ls`). However, these things do not happen if you run `tclsh test.tcl`.

```
$ echo 'puts $argv' > test.tcl
$ echo 'puts [llengt $argv]' >> test.tcl
$ echo 'puts [ls test.tcl]' >> test.tcl
$ jg -batch test.tcl
.
.
.
% puts $argv
-batch test.tcl
% puts [llengt $argv]
```

```
2
% puts [ls test.tcl]
test.tcl
% exit -force
.
.
.
```

Environment Variables Used by Jasper

The following table lists and briefly describes the environment variables used by Jasper Apps.

Environment Variable	Description
CDS_LIC_FILE	This variable applies to all Jasper Apps products. It points to Jasper-specific license files. Also see "License-Related Environment Variables".
CDS_LIC_ONLY	If you have defined CDS_LIC_ONLY, Jasper Apps uses it along with CDS_LIC_FILE to find the license file and ignores LM_LICENSE_FILE (see "LM_LICENSE_FILE"
DISPLAY	Points to a valid display device for launching an X window application. If the DISPLAY environment variable does not point to a valid display, Jasper Apps prints an error message and exits immediately.
EDITOR	Points to the application launched by the edit command. The default is vi.

`JASPER_BLOCK_LICENSES`	Specifies the list of licenses to ignore. The licenses in the list must be separated by either commas or spaces. Two equivalent examples follow: • `env JASPER_BLOCK_LICENSES= "jasper_fpv jasper_interactive" jg` • `env JASPER_BLOCK_LICENSES= jasper_fpv,jasper_interactive jg` • If the environment variable contains strings that Jasper does not recognize as names of valid Jasper licenses, the tool issues a warning on `stderr` upon launch. One warning for each unrecognizable string is issued. • If there are warnings, the strings that Jasper does recognize as valid license feature names are added to the block list. • You can specify an empty list, which means that Jasper will not ignore any license.
`JASPER_CMD_LINE_ARGS`	Prepends its value to the list of command-line arguments at startup. This variable provides a way to always run the tool with a set of command-line options, even if they are not explicitly specified every time you start Jasper Apps.

`JASPER_COMM_DEBUG`	If you encounter a problem with a session connecting to the console, define this variable. We recommend setting it to `1`, and when the tool finishes, inspect the log files. • You can define any value, including an empty string, but if you set it to a non-empty string, the log files for the analysis sessions contain messages for the analysis session connecting to the console (IPL001) and disconnecting from the console (IPL002). • See also JASPER_COMM_LOCAL_SERVERNAME and JASPER_COMM_PORTS.
`JASPER_COMM_LOCAL_SERVERNAME`	

	If you encounter a problem starting the server in the main process, use the environment variable `JASPER_COMM_LOCAL_SERVERNAME` to specify the name of the host the analysis session will connect to. The tool only considers this value if you start the analysis session in local mode. The suggested values for this variable are `localhost` and `127.0.0.1`.
	• This environment variable might be warranted if you see an error message such as the following:
	`ERROR: Session labeled "session_0" has terminated due to socket error: 2 "Host not found" (server1:0) (server1:34260)`
	or
	`ERROR (EPL003): Cannot connect to the console...`
	• See also JASPER_COMM_DEBUG and JASPER_COMM_PORTS.

`JASPER_COMM_PORTS`	By default, the operating system assigns a port for the tool. If you encounter problems, use the environment variable `JASPER_COMM_PORTS` with a list of port ranges. When starting the server, the tool will attempt to use one of the ports in the specified ranges.
	
	Example:
	
	`1234,12345-23456,34567`
	
	• This environment variable might be warranted if you see an error message such as the following:
	
	`ERROR: Session labeled "session_0" has terminated due to socket error: 2 "Host not found" (server1:0) (server1:34260)`
	
	• Use a comma-separated list. Each element of the list must be either an integer or a pair of integers separated by a dash. The tool attempts to use the ports in the specified order. In the example above, it first attempts port 1234, then 12345 up to 23456, and then finally 34567.
	
	• See also JASPER_COMM_DEBUG and JASPER_COMM_LOCAL_SERVERNAME.

JASPER_FATAL_LICENSE_ERRORS	
	Jasper remembers when a license check out attempt fatally failed and does not attempt to check out more licenses of the same kind. This memory is per process, which means that the console (`jg_console`) process and the analysis session processes (`jg_session`) do not share this memory. The only license check out failure considered fatal is when the license server does not serve the license. The FlexNet error code for this kind of failure is "-5" and is described as "No such feature exists" in the FlexNet documentation. If this feature is causing issues, you can disable it by setting the environment variable `JASPER_FATAL_LICENSE_ERRORS` to an empty string. Following is an example of how to start Jasper with this feature disabled: `env JASPER_FATAL_LICENSE_ERRORS="" jg` This feature does not apply to ABVIP-licenses.
JASPER_FORCED_EXIT_TIMEOUT	

	Specifies a time-out in seconds. When the tool has been inactive for the specified amount of time, it autosaves the database and exits. There is no default time limit at which the tool automatically exits if inactive. The minimum allowed value is 3600 seconds. If you use a lower value, the tool defaults to 3600 seconds. After three fourths of the specified time passes, a dialog appears. It offers two options: • Click the *Exit Application* button to autosave the database and exit the tool. • Click the *Continue* button to close the dialog and continue working with the tool. The tool is considered inactive when all analysis sessions are inactive. An inactive analysis session neither runs a Tcl command nor runs a job in the background (for example, prove or Visualize). The timer starts as soon as the tool becomes inactive and stops as soon as it becomes active.
`JASPER_UXDB_ARGS`	Defines Jasper Expert System project, regression cycle, and user tags to customize Information System reports. For additional information, see the *Jasper Expert System User Guide* (*Help – Application Guides*).
`JASPER_UXDB_OPTS`	Defines specified server/client parameters, such as, client's connection timeout. For additional information, see the *Jasper Expert System User Guide* (*Help – Application Guides*).
`JASPER_UXDB_PATH`	Enables the Jasper Information System Server and provides access to the Information System Web interface. Set this variable as follows: `JASPER_UXDB_PATH {port}@{server_name}` The default `http_port` is `8080`.

JASPER_UXDB_TOKEN	Specifies a DAAPI access token. This token enables the `info_query` command to retrieve information from the Jasper Information System Server. Set this variable as follows: `JASPER_UXDB_TOKEN <token>` For additional information, see the *Jasper Expert System User Guide* (*Help – Application Guides*).
JGRC	Points to an optional default Tcl script that loads at startup. You can put initialization commands in this script. If you do not set this variable, the tool looks for files named `.jgrc` in the following order of directories, sourcing every one it finds: • Your home directory (`~/.jgrc`) • The current working directory (`./jgrc`) The system administrator responsible for the Jasper installation can edit the file located at `<jasper_install>/etc/res/settings/jgrc.tcl` to control the initialization script of the tool.
LM_LICENSE_FILE	This variable applies to all Jasper Apps products. It points to a valid license file. Also see "License-Related Environment Variables".
LSF_BINDIR	This variable is necessary when you configure your server farm using `proofgrid_mode lsf` or the `-remote_mode lsf` command-line switch. When you use `proofgrid_mode lsf`, Jasper Apps executes `$LSF_BINDIR/bsub`. Consult the IBM Platform LSF documentation to learn how to configure `$LSF_BINDIR`.

PDFVIEWER	Points to the PDF viewer application for viewing documents. • Use this variable only if there are no settings for PDF files in the settings file ~/.config/cadence/jasper.conf. You can specify the PDF viewer directly in this file or with the Preferences dialog (*Edit – Preferences*). • This variable is helpful if you encounter problems launching a PDF document from a GUI. • If acroread is your PDF viewer, and you get a "corrupted file" message when you attempt to open a manual from the *Help* menu, set the variable to a supported PDF viewer such as xpdf or evince. Cadence supports PDF version 1.6 and above, which is not supported by Adobe® Acrobat® Reader® software on Linux operating systems.
SGE_ROOT	This variable is necessary when you configure your server farm using proofgrid_mode oge or the -remote_mode oge command-line switch. You will not set this variable if you use proofgrid_mode shell. When you use proofgrid_mode oge, Jasper Apps executes '$SGE_ROOT/util/arch'/qrsh or '$SGE_ROOT/util/arch'/qsub. Consult the Oracle Grid Engine documentation to learn how to configure $SGE_ROOT.
TMPDIR	Points to the directory where temporary files are created. If you do not define this variable or this value is empty, the default is /tmp. When you start Jasper or create a new analysis session, the tool now checks that the temporary directory exists and is writable and prints an error if the check fails.

`VOVDIR`	This variable is necessary when you configure your server farm using `proofgrid_mode nc`. You will not set this variable if you use `proofgrid_mode shell`. When you use `proofgrid_mode nc`, Jasper Apps executes `$VOVDIR/scripts/nc`. Consult the Runtime Design Automation NetworkComputer documentation to learn how to configure `$VOVDIR`.
`XDG_CONFIG_HOME`	Defines the directory in which Jasper writes user-specific configuration files. If not specified, Jasper uses `$USER/.config/` for the user-specific configuration files. This environment variable is not specific to Jasper. That is, setting it permanently in your environment will likely affect other tools. Therefore, if you want this to only affect Jasper, create and use a script that sets this environment variable and then launches Jasper. Refer to the XDG Base Directory Specification available online and the Jasper user guide "GUI Settings" appendix section, "Changing Default Configuration Settings."

Command Summary

The following sections include tables that list and briefly describe Jasper Apps commands:

- General Command Summary
- General Configuration Command Summary
- App-Specific Command Summary
- App-Specific Configuration Command Summary

The summaries in this section include corresponding Jasper Apps GUI actions where applicable.

General Command Summary

The table in this section lists and briefly describes general commands. Look for GUI equivalents in the third column.

> To see the names of buttons, rest your cursor over the button in the active window.

Command	Description	Equivalent GUI Action
abstract	Create a counter, initial-value, register-value, or reset-value abstraction.	Menu: • *Application – Task Add Counter Abstraction* or *Add Initial Value Abstraction* *Task Tree –* • Right-click and choose *Task Context* Design Information window • Right-click on a counter name and choose *Add Counter Abstraction* or *Remove Counter Abstraction* Visualize window • *View* menu – *Task Context* • Click the *Add to Task Context* button and then the *Counter Abstractions* tab.
abvip	Print the requested ABVIP information or set the ABVIP installation directory.	None
analyze	Analyze the contents of an HDL or property file.	• Menu: *Design – Analyze RTL* • Click the *Analyze RTL* button.

assert	Add an assertion about the signals in the design to the current task. Also, remove, modify, convert, list, and so forth.	• Menu: *Application – Task – Add Property* • Click the *Capture Behavior from Waveform Selection* button. • Click the *Add Property* button.
agent	Enable and control the Jasper AI Agent.	The `agent -init` command enables the Jasper AI Agent GUI in the lower right corner of the window.
assume	Add an assumption about the signals in the design to the current task or all tasks. Also, remove, modify, convert, list, and so forth.	• Menu: *Application – Task – Add Property* • Click the *Capture Behavior from Waveform Selection button*. • Click the *Add Property* button.
auto_setup	Attempt to automatically generate a setup script for loading the specified RTL.	None
autoprove	Orchestrates a dynamic approach to verification based on the `prove` command.	None
blackbox_assistant	Black box modules and instances not required for the verification.	None
cfe_debug	Provide the ability to debug C/C++ code using GDB.	None
capture_testcase	Save a tarred file that contains all sourced scripts and design files.	None
check_assumptions	Check whether assumptions overconstrain the design and might cause vacuity.	*Task Tree* – Right-click and choose *Check Assumptions*

check_code	Check whether a Tcl command returns a specific code.	None
check_dlh	Use formal verification hunting to increase the chance of finding deadlocks.	None
check_loop	Check for signals that contribute to a combinational loop.	Refer to the "Combo Loop Viewer" chapter in the *Jasper Platform and Formal Property Verification App User Guide.*
check_one_of	Check whether an actual value matches the expectations.	None
check_reset	Set up your reset verification environment and run reset analysis.	None
clear	Clear proof and GUI information.	*Task Tree* – Right-click and choose *Clear Task*
clock	Specify the global clock-related configuration, list all clock configurations, and analyze the clock tree.	Refer to the "Clock Viewer" chapter in the *Jasper Platform and Formal Property Verification App User Guide.*
command_interface	Manage connections to Jasper command interface clients.	None
compilation	Handle RTL compilation configuration, profiling, and debugging.	None
compile	Compile C/C++ or SystemC/HLS sources for the current session.	None

complexity_manager	Launch a GUI display of design statistics to show complexity.	• Menu: *Window – Complexity Manager* • *Property Table*, right-click and choose *Complexity Manager* • Session tab, right-click and choose *Complexity Manager*
configure	List or unlimit the current configuration or change it to default values.	• Menu: *Application – Proof – Proof Settings* • Click the *Proof Settings* button.
connect	Instantiate and connect a module or entity to the top module of the elaborated design.	• Menu: *Design – Connect* • Click the *Connect* button.
cover	Add a cover directive for signals in the design to the current task. Also, remove, modify, convert, list, and so forth.	• Menu: *Application – Task – Add Property* • Click the *Capture Behavior from Waveform Selection* button. • Click the *Add Property* button.
custom_engine	Manage custom engines in the current session.	None
custom_gui_action	Add, remove, or list custom buttons to or from GUI windows.	None
database	Access and manipulate the database for Unicov and BPS.	None
debug_explorer	Run the Debug Assistant.	None
debug_handoff	Create, use, and modify the Debug Handoff database.	None
dglob	Sort and list files.	None

dump_function_stack	Dump the stack call of a function instantiation, given its name.	None
edit	Edit a specified file.	Right-click in a source pane and choose *Edit Source*.
elaborate	Synthesize and read the netlist.	• Menu: *Design – Elaborate RTL* • Click the *Elaborate RTL* button.
exit	Exit the current session.	• Menu: *File – Exit* • Click the *Close* (x) button in the top right corner of the console title bar.
expert_system	Define custom Expert System rules and manage built-in and custom rules.	Click the Expert System buttons in the top right corner of the main window or the Visualize window.
export	Export Tcl properties as SVA or PSL requirements files.	Menu: *File – Export*
formal_profiler	Profile and provide detailed information about an engine's work on a property.	None
fsm_checks	Generate properties to check the reachability of state machine transitions/crosses.	None
get_analyze_info	Returns information about analyzed HDL or property files.	None
get_clock_info	Provide detailed information about the clock configuration.	None
get_decomposed_expression		
	Print a list of expressions for the sub-expressions of the provided argument.	None

get_design_info	Report design information.	You can access the Design Information window from the context menu (right-click) from any one of multiple points: • Jasper Apps console, for example, the *Property Table* • Visualize window, for example, from the *Design* toolbar • Source Browser, for example, with the *Source* pane context menu (right-click) • ProofGrid™ Manager window, for example, the *Properties* pane context menu.
get_fanin	Report the signals in the fanin of a signal or property.	None
get_fanout	Report the signals in the immediate fanout of the specified signal.	None
get_filename	Report the file name for a specified module or instance name.	None
get_flop_info	Report information about signals related to one or more input flops.	None
get_inst_top	Report information about signals related to the input flop.	None
get_install_dir	Report the location of the installation directory.	None
get_latch_info	Get the top instance path to be analyzed formally.	None

get_needed_assumptions	Find the irreducible set of assumptions to achieve a target status.	None
get_proj_dir	Report the location of the project directory.	None
get_property_info	Report a list of key/value pairs with information about the property.	None
get_property_list	Report a list of properties whose attributes satisfy specified value criteria.	None
get_reset_info	Get a reset VCD or customized report.	None
get_root_name	Report the name of the root of the instance tree.	None
get_signal_info	Report information about a specified signal.	None
get_signal_list	Report a list of unique signals.	None
get_status	Report the aggregated proof status of a set of properties.	• Click the *RTL Modules* or *Task Tree* tab. • Visualize window *Visualize Configuration* pane options
get_tool	Report the name of the tool running your script.	None
get_top_module	Report the name of the top module that will be formally analyzed.	None
get_trace_info	Get information about a trace.	None
get_uxdb_status	Report whether Jasper is connected to the Expert System Information Server	None

get_version	Report the version number or build date of the product installation you are using.	Menu: *Help – About Jasper*
graph	Create visual graphs not based on the netlist or calculate the shortest path between two design signals.	None
gui	Provide interactivity to the GUI (query widgets and get input using dialogs).	None
help	Return information about all commands or a specific command. Return information about all Error, Warning, and Info messages or a specific message.	Menu: *Help –* • *Command Reference Manual* • *Tcl Command Help*
hunt	Use formal verification to increase the chances of finding CEXs (no proof).	None
include	Read and execute a script file of Tcl/Jasper Apps commands.	• Menu: *File – Tcl Scripts* • Click the *Source* button. • Click the *Source Recent/Database Script* button to run the last script. • Click the arrow adjacent to the *Source Recent/Database Script* button and choose a script from the list.
incremental_elaborate_wrapper_mode		
	Set the `elaborate` command incremental wrapper flow mode.	None

info_query	Return information from a connected Jasper Information System server.	None
ipk	Analyze an IPK and extract or load proof kit files.	None
ipxact	Print the requested IP-XACT information or execute the requested action.	None
jasper_history	Queries information about Jasper commands' run history.	None
jasper_model_divider	Manage the jasper_model_divider PA connection and configuration.	None
jasper_model_mpram	Manage the jasper_model_mpram PA connection and configuration.	None
jasper_model_multiplier	Manage the jasper_model_multiplier PA connection and configuration.	None
jasper_model_ram	Manage the jasper_model_ram PA connection and configuration.	None
jasper_scoreboard_3	Load definitions to configure jasper_scoreboard_3 PA.	None
jforeach	Optimize memory for large Tcl results by iterating through results.	None
liberty	Load specified Synopsys Liberty™ files and report cell information.	None
load_radix_file	Load a radix file.	Visualize window *View* menu – *File – Load Radix File*

pretty_print	Display any Tcl list in a more easy-to-read format.	None
proofmaster	Control the ProofMaster settings.	None
proof_structure	Set up formal methodologies using Jasper tasks.	None
property_triage	Group similar properties with CEX traces into buckets.	None
prove	Prove properties or control a `prove` command running in the background.	• Menu: *Application – Proof – Prove All* • Click the *Prove All* button. • *Task Tree*: Right-click and choose *Prove Task*. • *Property Table:* Right-click and choose *Prove Task* or *Prove Property.*
put_message	Add a user-specified message to the Jasper logging system.	None
redirect	Redirect `stdout` to a file or Tcl variable.	None
replace_driver	Disconnect the original driver of a signal and replaces it with a new one.	None
report	Report analysis results.	None
reset	Specify and report the global, design reset condition.	• Menu: *Design – Reset* • Click the *Reset* button.

restore	Restore a session saved by the `save` command.	• Menu: *File – Open Database* or *File – Elaborated Design – Restore Elaborated Design* • Click the *Open Database* button.
sanity_check	Facilitate the successful setup of the global environment.	None
save	Save the current session.	• Menu: *File – Save Database* or *File – Elaborated Design – Save Elaborated Design* • Click the *Save Database* button.
schematic_viewer	Draw a schematic of a portion of or an entire loaded design.	Menu: *Window – Schematic Viewer* *Window – New Window* Clock Viewer window – • Click the Schematic Viewer button • Right-click a clock and choose *View COI in Schematic* Combo Loop Viewer window • *Tools – Show Loop Graph and Schematic* • Click the Graph and Schematic Viewer button • Right-click a loop and choose *Show Loop Graph and Schematic* • Right-click a signal in the Source Browser and choose *Show in Schematic Viewer* Source Browser window – *Design Hierarchy* pane and source pane,

		right-click, and choose *Show in Schematic Viewer*. Visualize window – • Waveform signals list, right-click, and choose *Show in Schematic Viewer*. • *Source Pane*, right-click, and choose *Show in Schematic Viewer*. • *Signal Browser* pane, right-click, and choose *Show in Schematic Viewer*. Consult the individual app user guides for additional information on app-specific Schematic Viewer options (*Help – Application Guides*).
session	Manipulate analysis sessions.	Various
set_current_gui	Set the current GUI view to the specified app view.	• Menu: *View – Applications...* • Click the *Other Applications* button.
show_list	Format list values returned by a Tcl command.	None
show_source_browser	Show the Source Browser window.	• Menu: *Window – Source Browser* • *Property Table* – right-click and choose *View Source*
source_lock	Enables Source Lock on analyzed files.	None

sst	Check whether SST traces are still valid after adding new helper assertions.	None
stopat	Stop traversing a netlist at the specified signals or instances.	Menu – *Application – Task – Add Stopat* Task Context dialog – • *Stopats* tab – Click the *New* button. Visualize window – • Click the *Add to Task Context* button and then the *Stopats* tab. • Menu: *Visualize – Add to Task Context* • Drag and drop a signal into the *Task Context* pane. • Right-click on one or more signal names and choose *Add Stopat*. In the Visualize window, stopats remain in the background until you use the *Set Local Task Context as AR* feature.

task	Create, set, remove, edit, link, and report on a task.	• *Task Tree* – right-click and choose any of the following: ◦ *Create Task* ◦ *Remove Task* ◦ *Clear Task* ◦ *Task Context* ◦ *Set as Current Task* • *Property Table* – right-click and choose *Create Task*
view	View the contents of a file.	None
virtual_net	Create a new virtual net, optionally driven by a specified expression.	None
visualize	Generate a trace to Visualize a requirement with or without the design or to Visualize a design scenario.	Refer to the "Visualize" chapter in the *Jasper Platform and Formal Property Verification App User Guide*.

General Configuration Command Summary

The table in this section lists and briefly describes general Jasper configuration commands. In some cases, you can use the GUI in place of the command. See the "Equivalent GUI Actions" column.

Configuration commands begin with `set_`, and each `set_*` command has a complementary `get_*` command. Use the `get_*` commands to report the current configuration setting and refer.

Configuration Command Summary

Command	Description	Equivalent GUI Action
set_allow_latch_enable_loops		

	Assume all combinational loops through latch-enable pins are stable.	None
set_analyze_bbox_parse_mode		
	Control whether the tool will parse header (ANSI ports and parameters) of a blackboxed module discovered by the `analyze` command's `-y` directive.	None
set_analyze_libnonamehide	Enable or disable storing read modules in a temporary directory during compilation.	None
set_analyze_librescan	Enable or disable directory re-scanning to find Verilog modules.	None
set_analyze_libunboundsearch		
	Enable cross-file compilation unit `-L` option.	None
set_analyze_y_and_v_command_line_order		
set_annotation	Create an annotation for a property or signal.	None
	Modify the command line order of `-y` and `-v` arguments for the `analyze` command.	None
set_auto_disable_related_covers		
	Specify whether enabled related covers will be disabled along with their main properties.	None
set_automatic_library_search		
	Enable or disable searches for modules in user-declared libraries for `analyze` and `elaborate`.	None

set_blackbox_assistant_require_clock_paths_in_connections		
	Control whether blackbox assistant requires clock paths in connections.	None
set_blackbox_assistant_require_control_paths_in_connections		
	Control whether blackbox assistant requires control paths in connections.	None
set_cache_proof_simplification		
	Enable or disable proof simplification caching.	Proof Settings dialog – *General* tab – *Cache proof simplification results*
set_capture_elaborated_design		
	Enable capturing the elaborated netlist to speed up subsequent design reuse.	• Menu: *File* – *Elaborated Design* – *Capture Elaborated Design* • In the session control panel toolbar, click the down arrow adjacent to the *Launch New Session* button and choose *Capture Elaborated Design*
set_capture_property_expression		
	Disable or re-enable extracting expressions from embedded properties.	None
set_cell_define_debug_visibility		

	Consider cell define blocks in design traversals.	None
set_clock_loop_model	Control how Jasper handles loops through clock pins during proofs.	None
set_cmd_time_limit	Sets the time limit for most Tcl commands.	None
set_command_license_wait_interval		
	Set the time interval between license checkout attempts.	None
set_command_license_wait_retries		
	Set the number of checkout retries the tool should attempt before failing.	None
set_compilation_cache	Enable or disable compilation caching for "analyze" and "elaborate".	None
set_compilation_cache_path	Define a location for the storage of compilation cache files.	None
set_compilation_command	Provide a compilation command to compile C/C++.	None
set_compilation_skip_errors_in_filelist		
	Define if invalid "-f" file entries raise errors or are ignored with warnings.	None
set_compilation_unit_naming_mode		
	Set the automatic naming mode for Compilation Units.	None
set_compilation_unit_naming_prefix		
	Set the auto-naming prefix for Compilation Units.	None
set_counter_speculative_detection		

	Enable (or disable) the speculative counter detection algorithm.	None
set_covergroup_bin_size_limit		
	Set a limit for the size of covergroup vector bins.	None
set_covergroup_cross_product_limit		
	Set a limit for the product of cross coverage inside covergroups.	None
set_cte_enable_session_tcl_env_restore		
	Determine if SDC processing considers Jasper's Tcl environment.	None
set_cumulative_reset		
	Enable reset cumulative mode.	None
set_database_auto_save_restore		
	Enable (or disable) Jasper Apps to save the database automatically.	None
set_database_session_save_restore		
	Enable session-specific Jasper database save and restore.	None
set_design_exploration_include_clock_logic		
	Specify whether Design Information includes clock logic in its computations.	Design Information window menu – *View – Include Clock Logic*
set_dollar_display_synthesis	Enable `$display` synthesis in elaboration.	None
set_elab_time_limit	Specify the maximum time to spend on a single `elaborate` command.	None
set_elaborate_bbox_inout_simplification		

	Configure inout port simplification for blackboxed modules.	None
set_elaborate_bbox_output_driver		
	Configure output strength of ports in blackboxed modules.	None
set_elaborate_generate_sage_database		
	Control whether Verisium Debug Database (Sage) is generated during elaboration.	None
set_elaborate_interrupt_on_error		
	Enable or disables stopping `elaborate` earlier on errors.	None
set_elaborate_limited_unrolled_loop_precondition		
	Enable the creation of precondition property for limited unrolled loops.	None
set_elaborate_limited_unrolled_loop_property		
	Enable the creation of an automatic property for limited unrolled loops.	None
set_elaborate_procedural_loop_limit		
	Set the procedural loop limit for `elaborate`.	None
set_elaborate_quiet_trace	Specify whether `elaborate` will automatically generate global QuietTrace™ soft constraints.	None
set_elaborate_real_mode	Set how real and wreal signals are handled during elaboration.	None
set_elaborate_single_run_mode		
	Enable or disable `elaborate` single run mode.	None

set_elaborate_use_disk	Specify how the elaborated design is transferred to the current session.	None
set_enable_preserve_property_status		
	Set whether property status is preserved if valid after environment changes.	None
set_engine_algorithm_selection		
	Enable engine algorithm selection.	None
set_engine_algorithm_selection_dir		
	Set the directory to save models created by engine algorithm selection.	None
set_engine_distributed_mode	Enable running portfolios of distributed engines.	None
set_engine_mode	Optimize the proof process for different types of designs and requirements.	• Menu: *Application – Proof – Proof Settings* • Click the *Proof Settings* button.
set_engine_processes	Specify the number of distributed engines for engine B and Bm.	None
set_engine_solver	Specify the solver you want the engines to use in the decision procedure for the Boolean satisfiability problem.	None
set_engine_threads	Specify the upper limit on the number of threads used by each engine.	Advanced Engine Settings dialog – *Parallelism* section
set_engineB_before_first_mode		

	Specify how engine B will treat any cycles before its first trace attempt.	Advanced Engine Settings dialog – *Engine B* section
set_engineB_before_increment_mode		
	Specify how engine B treats any cycles between consecutive trace attempts.	Advanced Engine Settings dialog – *Engine B* section
set_engineB_first_trace_attempt		
	Specify the cycle at which engine B will make its first trace attempt.	None
set_engineB_processes	Specify the number of distributed engines for engine B.	None
set_engineBm_processes	Specify the number of distributed engines for engine Bm.	None
set_engineB_trace_attempt_increment		
	Specify the increment engine B uses between one trace attempt and the next.	None
set_engineB_trace_attempt_time_limit		
	Specify the limit on the amount of time engine B spends on each trace attempt.	None
set_engineB_trace_attempt_time_limit_incr		
	Specify the time limit increment for engine B when the specified limit for trace attempts expires	None
set_engineBm_first_trace_attempt		
	Specify the cycle at which engine Bm will make its first trace attempt.	None
set_engineBm_trace_attempt_increment		

	Specify the increment engine Bm uses between one trace attempt and the next.	None
set_engineBm_trace_attempt_time_limit		
	Specify the limit on the amount of time engine Bm spends on each trace attempt.	None
set_engineBm_trace_attempt_time_limit_incr		
	Specify the time limit increment for engine Bm when the specified limit for trace attempts expires	None
set_engineC_optimization	Specify the optimization mode for engines C and C2.	Advanced Engine Settings dialog – *Engines C, C2, G and G2* section
set_engineCG_max_mem	Specify the maximum memory allocation for engine C and G data structures.	Advanced Engine Settings dialog – *Engines C, C2, G and G2* section
set_engineD_optimization	Specify the optimization mode for engines D and I.	Advanced Engine Settings dialog – *Engine D* section
set_engineG_optimization	Specify the optimization mode for engines G and G2.	Advanced Engine Settings dialog – *Engines C, C2, G and G2* section
set_engineH_before_first_mode		
	Set how engine H treats undetermined cycles before its first trace attempt.	None
set_engineH_before_increment_mode		

	Set how engine H treats undetermined cycles between consecutive trace attempts.	None
set_engineH_first_proof_attempt		
	Specify the cycle at which engines H, Hp, and Hps make their first proof attempt.	None
set_engineH_first_trace_attempt		
	Specify the cycle at which engine H will make its first trace attempt.	None
set_engineH_last_proof_attempt		
	Specify the cycle at which engines H, Hp, and Hps make their last proof attempt.	None
set_engineH_restart	Restarts engine H's max length search when another engine reports a proof.	None
set_engineH_trace_attempt_increment		
	Specify the increment engine H uses between one trace attempt and the next.	None
set_engineH_trace_attempt_time_limit		
	Set a trace attempt time limit for engine H.	None
set_engineH_trace_attempt_time_limit_incr		
	Set a trace attempt time limit increment for engine H.	None
set_engineJ_attempts	Specify the maximum number of engine J attempts to solve all constraints at every clock cycle before restarting.	Advanced Engine Settings dialog – *Engine J* section

set_engineJ_max_trace_length		
	Specify an upper limit on the length of traces searched for by engine J.	Advanced Engine Settings dialog – *Engine J* section
set_engineJ_migrate	Enable (or disable) engine job handover to Q3 if the engineJ_attempts limit is exceeded.	None
set_engineJ_restarts	Specify the number of restarts engine J makes before abandoning the proof.	Advanced Engine Settings dialog – *Engine J* section
set_engineJ_seed	Specify a hexadecimal setting for the seed used by engine J.	Advanced Engine Settings dialog – *Engine J* section
set_engineJ_single_property		
	Specify multi- or single-property mode for engine J.	Advanced Engine Settings dialog – *Engine J* section
set_engineL_asserts_are_intermediate		
	Specify whether asserts should be treated as sources of intermediate states.	None
set_engineL_cache	Specify whether to cache cover points for engine L.	Advanced Engine Settings dialog – *Engine L* section
set_engineL_generate_trails	Enable (or disable) trail generation by engine L.	None
set_engineL_max_segment_length		
	Specify the segment length limit.	Advanced Engine Settings dialog – *Engine L* section
set_engineL_random_diversification		

	Specify whether engine L will find different traces for the same property to diversify start states for other properties	Advanced Engine Settings dialog – *Engine L* section
set_engineL_search_mode	Specify whether engine L will target properties in order or in parallel.	Advanced Engine Settings dialog – *Engine L* section
set_engineL_seed	Specify a hexadecimal setting as the seed for engine L.	None
set_engineL_segment_time_limit		
	Specify the engine L segment time limit.	Advanced Engine Settings dialog – *Engine L* section
set_engineL_state_removal	Specify whether engine L should pick start states based on the last-found start states (`fifo`) or the start states that have been most fruitful (`lru`).	Advanced Engine Settings dialog – *Engine L* section
set_engineL_tail_length	Specify a starting point for a new search that is *N* cycles before the end of found traces.	Advanced Engine Settings dialog – *Engine L* section
set_engineM_optimization	Specify the optimization mode for engine M.	None
set_engineQ3_max_trace_length		
	Specify the trace length upper limit for searches by Q3 when diversification is on.	Advanced Engine Settings dialog – *Engine Q3* section
set_engineQ3_random_diversification		
	Enable random diversification for engine Q3.	Advanced Engine Settings dialog – *Engine Q3* section
set_engineQ3_random_diversification_factor		

	Specify the random diversification factor used by engine Q3.	Advanced Engine Settings dialog – *Engine Q3* section
set_engineQ3_restarts	Limit engine Q3 restarts to the specified non-negative integer attempts.	Advanced Engine Settings dialog – *Engine Q3* section
set_engineQ3_seed	Specify a hexadecimal setting as the seed for engine Q3.	Advanced Engine Settings dialog – *Engine Q3* section
set_engineQT_attempt_time_limit		
	Specify the time limit for each engine QT quiet attempt.	None
set_engineQT_optimization	Enable optimization for engine QT.	None
set_engineU_max_trace_length		
	Specify the trace length upper limit for searches by U.	None
set_engineU_seed	Specify the hexadecimal setting for the seed used by engine U.	None
set_engineU_trace_attempt_time_limit		
	Specify a trace attempt time limit for engine U.	None
set_engineU2_max_trace_length		
	Specify the trace length upper limit for searches by U2.	None
set_engineU2_restarts	Limit engine U2 restarts to the specified non-negative integer attempts.	None
set_engineU2_seed	Specify the hexadecimal setting for the seed used by engine U2.	None
set_evaluate_properties_on_formal_reset		

	Controls property evaluation during the reset phase in the formal analysis.	None
set_executable_path	Provide path to the executable that will be used to debug C/C++ code with GDB.	None
set_exit_handler	Set a Tcl proc to run when an analysis session exits for predefined reasons.	None
set_expert_system_property_send_limit		
	Specify the number of property status updates to send to the information server.	None
set_expert_system_regression_model_version		
	Specify the regression model version used for jobs sent to the Jasper Information System server	None
set_expert_system_regression_tag		
	Specify the regression tag used with jobs sent to the Jasper Information System server	None
set_expert_system_status	Disable or enable Jasper Expert System.	Click the power button on the Expert System status bar.
set_expert_system_superlint_tasks_filter		
	Specify whether Superlint tasks are filtered from the content sent to the Jasper Information System server	None
set_expert_system_tcl_design_name		
	Specify a user-defined design name for the current job to be sent to the Jasper Information System server.	None

set_expert_system_unprocessed_property_status		
	Specify whether the tool includes properties with `unprocessed` status in updates sent to the Jasper Information System server.	None
set_expr_32bit_constants	Extend expressions with unsized numbers to 32-bit constants.	None
set_extended_inout_port_alias_handling		
	Enable or disable extended elaborate handling of alias through inout ports.	None
set_file_access_time_limit	Set the time limit used by "analyze" command while waiting for file access.	None
set_first_trace_attempt	Specify the cycle at which engines will make their first trace attempt.	Proof Settings dialog – *General* tab
set_force_property_removal	Control removal of converted embedded properties when using `-remove_original`.	None
set_function_debug_visibility	Enable (or disable) debugging for functions.	None
set_functional_loop_time	Specify the time limit for the functional loop analysis step in the `break_functional` flow.	None
set_gate_level_compilation_mode		
	Enable or disable gate-level compilation mode.	None
set_gdb_path	Provide path to GDB that will be used to debug C/C++ code.	None
set_hunt_single_proof_thread		
	Specify whether hunt properties will be proven in a single proof thread.	None

set_incremental_compilation_mode		
	Enable or disable advanced incremental compilation mode.	None
set_internal_from_mode	Specify the X-injection mode for the modeling of internal `-from` signals.	None
set_jaspersim_max_jobs	Define the maximum number of parallel jobs for the parallel simulator.	None
set_jaspersim_mode	Enable parallel simulation.	None
set_jaspersim_partition_depth		
	Define the depth of the instances that will become partitions.	None
set_jaspersim_partition_file	Define a partition file for parallel simulator partitioning.	None
set_jaspersim_partition_method		
	Enable (or disable) configuring partitions for the Jasper parallel simulator.	None
set_jasper_history_per_command_limit		
	Specify the limit of Jasper command entries to keep in history per command name.	None
set_legacy_expression_parser		
	Set the Tcl expression parser for design compilation, legacy or improved.	None
set_liberty_cells_auto_blackbox		
	Configures the tool to blackbox all instances of Liberty cells.	None
set_liberty_cells_override_rtl_definition		

	Configure the tool to prioritize Synopsys Liberty cell definitions over RTL.	None
set_liberty_enable_cell_define		
	Encapsulates generated Verilog model for Liberty cells by "`celldefine".	None
set_liberty_enable_function_synthesis		
	Specify whether the tool synthesizes Liberty cell functions.	None
set_liberty_enable_timing_model_synthesis		
	Configures Liberty enable timing model synthesis.	None
set_library_cell_detection	Enable automatic detection of library cells.	None
set_library_cell_optimization	Enable optimizations for synthesis of library cells.	None
set_log_timestamp_format	Specify the format for the timestamp of logging messages.	None
set_log_timestamp_mode	Specify whether logging messages will be prepended by a timestamp.	None
set_loop_filter_latch_patterns		
	Automatically filters loops that go through latch patterns and cause false loops	None
set_loop_filter_startpoints		
	Automatically filters startpoints in the design not connected to loops	None

set_loop_handling	Specify how you want the tool to handle loops in the COI of properties.	Loop Viewer window • *Tools* – Various options • Click the applicable button in the *Loop Handling* button group
set_loop_handling_fallback	Configure the default loop handling fallback method.	None
set_max_num_sub_expressions		
	Limit the number of sub-expressions for `get_decomposed_expression` signals.	None
set_max_trace_length	Specify the limit for the length of traces to search for.	Proof Settings dialog – *General* tab – *Limits* section
set_memory_aware_export_memory_report		
	Enable or disable exporting the Memory Aware Proofs (MAP) memory report.	None
set_memory_aware_memory_report_path		
	Set the path for the Memory Aware Proofs (MAP) memory report.	None
set_memory_aware_initial_mem	Define the initial memory limit for Memory Aware Proofs (MAP).	None
set_memory_aware_max_mem	Define the maximum memory limit for Memory Aware Proofs (MAP).	None
set_memory_aware_mem_factor	Define the memory scaling factor for Memory Aware Proofs (MAP).	None

set_memory_aware_min_mem	Define the minimum memory limit for Memory Aware Proofs (MAP).	None
set_memory_aware_proofs	Enable or disable Memory Aware Proofs (MAP).	None
set_memory_aware_wastage_factor		
	Set the memory wastage factor for ProofGrid workers.	None
set_message	Manipulates warning and info messages or changes message severity.	None
set_message_history_limit	Set the limit of stored user messages issued in the current session.	None
set_missing_entries_as_x_in_comb_udp		
	Set how missing entries in combinational UDPs are handled during elaboration.	None
set_module_elaboration_memory_limit		
	Set the maximum memory footprint that a module can reach during elaboration.	None
set_no_fallback_in_use_clause		
	Specify whether fallbacks are active when handling use clauses in SystemVerilog.	None
set_optimize_cell_netlists	Enable optimization for cell netlists that behave as a single gate.	None
set_parallel_synthesis_directory		
	Set the directory where parallel synthesis can write temporary files.	None
set_parallel_synthesis_dispatch_mode		

	Specify how to spawn multiple synthesis processes, locally or on a shell grid.	None
set_parallel_synthesis_dispatch_timeout		
	Set the time that the flow waits for jobs to be launched in "shell" mode.	None
set_parallel_synthesis_memory_limit		
	Set the memory limit for parallel synthesis jobs.	None
set_parallel_synthesis_manual_partition_file		
	Set the path to the file that specifies how to partition the design.	None
set_parallel_synthesis_mode		
	Specify the parallel synthesis mode for the `elaborate` command.	None
set_parallel_synthesis_num_process		
	Specify the number of processes spawned in parallel synthesis mode.	None
set_parallel_synthesis_partition_method		
	Specify the method for parallel synthesis partitioning for the design.	None
set_parallel_synthesis_shell		
	Specify the command to launch parallel jobs when the dispatch mode is `shell`.	None
set_parallel_synthesis_shell_stop		
	Specify the `jg_frontend` jobs clean-up command when the dispatch mode is `shell`.	None

set_per_property_simplification		
	Specify whether a property-specific simplification step should run during prove.	None
set_preference	Sets the preference to the specified value.	Menu: *Edit – Preferences*
set_prompt1	Set the Tcl script used to define the Tcl prompt.	None
set_prompt2	Set the Tcl script used to define the Tcl prompt for incomplete commands.	None
set_proof_simplification	Control simplification during proof.	Proof Settings dialog – *General* tab
set_proof_simplification_max_iterations		
	Set the maximum number of proof simplification iterations.	None
set_proof_structure_auto_propagation		
	Set whether Proof Structure auto propagation is enabled.	None
set_proof_structure_support_embedded_on_case_split		
	Set if Proof Structure supports embedded properties for case split nodes.	None
set_proofgrid_bridge	Enable or disable the use of Advanced Job Dispatch and Management feature.	None
set_proofgrid_cloud_file_upload		
	Enable file uploading mechanism with Bridge.	None
set_proofgrid_cloud_job_start_template		

	Specify the executable binary path used in job start command.	None
set_proofgrid_cloud_job_status_template		
	Specify the executable binary path used in job status command.	None
set_proofgrid_cloud_job_stop_template		
	Specify the executable binary path used in job stop command.	None
set_proofgrid_cloud_upload_storage		
	Specify the storage type used in file uploading.	None
set_proofgrid_cloud_jobarray	Control the use of job arrays when dispatching ProofGrid jobs.	None
set_proofgrid_cluster	Specify the sequence of hosts on which to spawn proof engine jobs.	Proof Settings dialog – *ProofGrid* tab
set_proofgrid_engine_file_path		
	Specify the custom engine file path.	None
set_proofgrid_engineJ_max_jobs		
	Control the number of engine J jobs.	None
set_proofgrid_engineL_max_jobs		
	Control the number of engine L jobs.	None
set_proofgrid_engineQ3_max_jobs		
	Control the number of engine Q3 jobs.	None
set_proofgrid_engineU_max_jobs		
	Control the number of engine U jobs.	None
set_proofgrid_engineU2_max_jobs		

	Control the number of engine U2 jobs.	None
set_proofgrid_extra_args	Set extra arguments that will be supplied to the active `proofgrid_mode`.	Proof Settings dialog – *ProofGrid* tab
set_proofgrid_manager	Turn on (or off) the ProofGrid Manager feature.	ProofGrid Manager – Click the *Enable* (or *Disable*) button
set_proofgrid_max_jobs	Specify the limit for jobs used by ProofGrid.	Proof Settings dialog – *ProofGrid* tab
set_proofgrid_max_local_jobs		
	Specify the limit for jobs used by ProofGrid when `proofgrid_mode` is `local`.	Proof Settings dialog – *ProofGrid* tab
set_proofgrid_max_partitions	Set the maximum number of partitions for partitionable engines.	None
set_proofgrid_mem_scan	Turn on memory scan.	None
set_proofgrid_mode	Set the grid on which to run parallel proofs.	Proof Settings dialog – *ProofGrid* tab
set_proofgrid_pending_time_limit		
	Set the time limit for a `job pending` warning.	None
set_proofgrid_per_engine_max_jobs		
	Specify the limit for engine jobs started by ProofGrid per engine.	Proof Settings dialog – *ProofGrid* tab
set_proofgrid_per_engine_max_jobs_per_session		
	Specify the upper limit for the number of per-engine jobs in analysis sessions.	None
set_proofgrid_per_engine_max_local_jobs		

	Specify an additional limit, beyond per engine max jobs, for engine jobs per engine started by ProofGrid when `proofgrid_mode` is `local`.	Proof Settings dialog – *ProofGrid* tab
set_proofgrid_queue	Set the queue name on which to run parallel proofs.	Proof Settings dialog – *ProofGrid* tab
set_proofgrid_restarts	Specify the number of restarts ProofGrid allows per engine job.	Proof Settings dialog – *ProofGrid* tab
set_proofgrid_resubmit_time_limit		
	Specify the time limit for job resubmission.	None
set_proofgrid_shell	Set the command used to launch ProofGrid jobs.	Proof Settings dialog – *ProofGrid* tab
set_proofgrid_shell_stop	Set the command used to clean up ProofGrid jobs when `proofgrid_mode` is `shell`.	Proof Settings dialog – *ProofGrid* tab
set_proofgrid_socket_communication		
	Specify whether communication happens over a TCP/IP socket or standard in/out.	Proof Settings dialog – *ProofGrid* tab
set_proofmaster	Enable or disable ProofMaster.	None
set_proofmaster_dir	Specify a location for ProofMaster data.	None
set_proofmaster_initial_dir	Import ProofMaster data from an existing directory and uses it immediately.	None
set_proofmaster_max_data_age		
	Delete unmatched proof cache data from the ProofMaster dir after N days.	None
set_property_compile_time_limit		

	Set the per-property time limit for compiling properties during elaboration.	None
set_property_delay_cycles		
	Specify the number of cycles you would like to delay properties.	None
set_property_info_proof_timestamp		
	Control the visibility of the "proof_timestamp" property attribute.	None
set_property_table_dynamic_sort		
	Specify your dynamic sorting preference for property tables.	None
set_property_triage_max_threads		
	Specify the maximum number of threads for "property_triage" analysis.	None
set_prove_advanced_simplification		
	Perform property-independent model simplification before running the proof command.	None
set_prove_advanced_simplification_time_limit		
	Specify the maximum time spent on the advanced simplification run.	None
set_prove_cache	Set the property proof results cache directory.	None
set_prove_cache_in_parallel	Control running proof caching in parallel.	None
set_prove_cache_job_mode	Set the grid on which to run the parallel proof caching.	None

set_prove_cache_max_jobs	Set the limit for proof cache parallel jobs.	None
set_prove_cache_mode	Control the mode of property proof results caching.	None
set_prove_cache_path	Control trace caching and retrieving from the cache during proofs.	None
set_prove_cache_traces	Specify the time limit for the combo loop enumeration step in the `break_functional` flow.	None
set_prove_check_loop_time	Specify the time limit for the combo loop enumeration step in the `break_functional` flow.	None
set_prove_clock_optimization		
	Specify whether clock optimization is enabled during the proof.	None
set_prove_dump_trace_type	Specify which trace types the tool will dump with `prove -dump_trace`.	None
set_prove_evaluate_in_trace_mode		
	Specify whether the tool evaluates part of the trace or the full trace when you use `prove -evaluate_in_trace`.	None
set_prove_inferred_target_bound		
	Configure the tool to automatically infer target bounds for FPV assertions.	None
set_prove_invariants_import	Enable import of invariants as implicit assumptions.	None
set_prove_no_cover_traces	Specify whether to generate cover traces during proofs.	None
set_prove_no_traces	Specify whether to generate traces during proofs.	None

set_prove_on_determined	Set a Tcl proc to call when a `prove` command determines a property.	None
set_prove_orchestration	Control orchestration of proof engines.	None
set_prove_orchestration_stagnation		
	Control stagnation monitoring in orchestration proofs.	None
set_prove_orchestration_with_trace_swarm		
	Control whether or not "trace_swarm" runs along with proof orchestration.	None
set_prove_orchestration_hunt_trace_swarm_start_time_ratio		
	Control the ratio of proof time after which "trace_swarm" runs.	None
set_prove_orchestration_hunt_trace_swarm_strategy		
	Set the "trace_swarm" strategy that runs along with proof orchestration.	None
set_prove_orchestration_hunt_trace_swarm_jobs_ratio		
	Control the percentage of proof jobs to be shifted to "trace_swarm".	None
set_prove_per_property_max_time_limit		
	Specify the single-property engine wall clock time limit for the proof of each property.	Proof Settings dialog – *General* tab – *Limits* section
set_prove_per_property_time_limit		
	Specify the wall clock time limit for proving each property.	Proof Settings dialog – *General* tab – *Limits* section
set_prove_per_property_time_limit_factor		

	Control iterative geometric progression of the time limit imposed per property.	Proof Settings dialog – *General* tab – *Limits* section
set_prove_prefer_shortest	Control whether `prove` will continue searching for a shorter trace after identifying a non-minimal trace.	None
set_prove_report_mem	Control the generation of frequent engine job memory consumption messages.	None
set_prove_report_period	Set the sampling period for requesting memory reports from proof processes.	None
set_prove_report_task	Specify whether Jasper Apps prefixes property names with the task name in proof messages.	None
set_prove_start_timer_on_engine_start		
	Start `prove` command timer only after the first proof job starts.	None
set_prove_stop_on_cex_limit	Set the value of the variable "prove_stop_on_cex_limit".	None
set_prove_stop_on_target_bound		
	Set the value of the variable "prove_stop_on_target_bound".	None
set_prove_stop_on_unreachable_limit		
	Set the value of the variable "prove_stop_on_unreachable_limit".	None
set_prove_summary	Specify whether the tool will suppress the `SUMMARY` table after the proof.	None
set_prove_target_bound	Specify the global target bound for all property proofs.	None

set_prove_time_limit	Specify the maximum time allowed for a `prove` or `visualize` command.	Proof Settings dialog – *General* tab – *Limits* section
set_prove_verbosity	Specify the verbosity level for the `prove` command.	None
set_prove_with_soft	Instruct proofs to apply any available user-created soft constraints.	None
set_prove_with_soft_mode	Control which property types have soft constraints applied during proofs.	None
set_proven_directive	Specify whether to use proven directives as assumptions.	Proof Settings dialog – *General* tab
set_read_sdc_debug	Enable SDC Debug Analysis.	None
set_regexp_mode	Change Jasper regular expression semantics.	None
set_reset_effort	Define algorithm effort level used during reset analysis.	None
set_reset_extended_mode	Use configuration and verification settings specified with `check_reset` for reset analysis.	None
set_reset_max_iterations	Set the maximum number of iterations used for reset analysis.	None
set_resilient_compilation		
	Enable or disable resilient compilation.	None
set_resilient_compilation_mode		
	Set the resilient compilation mode.	None
set_restore_resolve_file_path		
	Specify the method for resolving paths for `restore -elaborated_design`.	None

set_save_compress_database		
	Enable Jasper to compress generated databases.	None
set_save_restore_capture_source_lock_rtl		
	With Source Lock active, include source-locked RTL in `save -elaborated_design`.	None
set_schematic_viewer_character_limit		
	Specifies the maximum number of characters in a label before it is truncated.	None
set_schematic_viewer_collapse_combo_logic		
	Specify whether combinational logic should be collapsed into clouds.	None
set_schematic_viewer_info_pane_area		
	Specifies where in a schematic viewer window to show the information pane.	None
set_schematic_viewer_collapse_constants		
	Specify whether collapsed constants are shown and their format.	None
set_schematic_viewer_show_collapsed_logic_as_cloud		
	Specify whether collapsed combinational logic is displayed as a cloud or a box.	None
set_schematic_viewer_show_function_nets		
	Specify the type schematic the tool displays when you run the `schematic_viewer` command	None
set_schematic_viewer_show_gate_cell_names		

	Specify whether the gate cell names will be displayed.	None
set_schematic_viewer_show_info_pane		
	Specifies whether the schematic viewer information pane is displayed by default.	None
set_schematic_viewer_show_instance_unconnected_pins		
	Specify whether schematics will show unconnected instance pins.	None
set_schematic_viewer_show_register_name		
	Specify whether the schematic will show the visible net name below the flop or latch symbol.	None
set_schematic_viewer_show_strength_information		
	Specify whether strength information is displayed on the schematic.	None
set_schematic_viewer_simplify_case_statements		
	Specify whether case statements are simplified into a single gate.	None
set_schematic_viewer_simplify_concatenations		
	Specify whether concatenations of multidimensional arrays should be collapsed.	None
set_sequence_wait_timeout		
	Define the lower boundary time out for `$wait_until_convergence`.	None
set_show_checker_on_hierarchy_tree		
	Specify whether the tool displays SystemVerilog checkers on the *Design Hierarchy* tree.	None

set_show_disabled_property_count		
	Specify whether the disabled property count displays in the GUI.	None
set_show_interface_on_hierarchy_tree		
	Specify whether the tool displays SystemVerilog interfaces on the *Design Hierarchy* tree.	None
set_show_proven_directive	Display the proven properties assumed at the start of the current proof.	None
set_simulation_max_loop_limit		
	Limit the number of times the simulation can loop over the same net without converging.	None
set_single_property_liveness	Select multi-/single- property mode for liveness properties.	None
set_src_inline_macro_expansion		
	Use this command to enable or disable the inline macro expansion feature.	None
set_src_loop_iteration_debug		
	Enable advanced for loop debugging features in the Source Pane.	None
set_sst_default_trace_length		
	Specify the SST default trace length.	None
set_stable_name_in_case_assert		
	Specify whether the names for automatic case assertions are stable.	None

set_tag_irrelevant_values	Specify whether irrelevant values are tagged as X in the Visualize window.	Visualize window – • Menu – *View* – *Tag Irrelevant Values*
set_task_compile_time_limit	Set the per-task time limit for compiling properties during elaboration.	None
set_tcl_auto_show_error_info		
	Specify whether Jasper outputs the contents of the `errorInfo` variable.	None
set_tcl_no_precondition	Disable the automatic generation of precondition covers for liveness assertions created with the `assert` command.	None
set_tcl_related_covers	Enable the precondition check for Tcl properties.	None
set_terminal_binary_path	Provide the path to the terminal that will be opened with GDB.	None
set_time_format	Specify the time format that appears in messages.	None
set_trace_extension	Extend a trace *N* extra cycles beyond the cycle that triggers a property.	None
set_trace_optimization	Specify the default trace optimizations or reduce the number of optimizations.	None

set_trace_show_reset	Disable or re-enable the feature to prefix traces with the reset sequence.	• Visualize window – *View* menu – *Show Reset Cycles* • Preferences dialog – *Visualize* tab – *Show reset cycles in waveform viewer*
set_use_low_power_port_definition		
	Specify whether output is the new UPF_ISO port.	None
set_visualize_annotate_clock_events		
	Enable the annotation of slow or gated clock events on Visualize waveforms.	None
set_visualize_auto_check_props		
	Compute exercised covers and violated assertions.	Visualize window – *Indexed Behaviors* tab – Click the *Analyze automatically* button
set_visualize_auto_load_debugging_tables		
	Automatically load debugging tables.	Visualize window – *Debugging Table* – Click the *Load automatically* button
set_visualize_default_layout		

	Specify the classic or simplified Visualize window layout.	Visualize window – *View* menu – *Layouts –Classic Visualize Simplified Visualize*
set_visualize_highlight_limit		
	Specify a cover trace length threshold to suppress highlight calculation.	None
set_visualize_log_default_auto_sync		
	Specify whether the *Visualize Log* pane updates the data automatically.	Visualize window – *Visualize Log* pane – *Update all messages* button
set_visualize_multiple_traces		
	Enable or disable the capture of intermediary traces when running Visualize QuietTrace.	None
set_visualize_preload_signal_list		
	Specify the signal list file the tool loads in each new Visualize window.	None
set_visualize_show_default_signals		
	Specify whether the Visualize window loads the default signals when it opens a trace.	None
set_visualize_show_full_struct		
	Enable value generation for all struct nodes.	None
set_visualize_show_strength_information		
	Specify whether strength information is displayed in Visualize window tooltips.	None

set_visualize_waveedit_transparency		
	Set the WaveEdit background transparency.	None
set_why_include_non_ticking_gated_clocks		
	Specify whether non-ticking gated clocks are included in "Why" results.	None
set_word_level_reduction	Enable (or disable) word-level transformations or reductions during the proof flow.	None
set_word_level_time_limit	Specify the wall clock time limit for the word-level flow model generation step.	None

App-Specific Command Summary

The table in this section lists and briefly describes the main commands to initialize, configure, and manipulate each of the apps.

App-Specific Command Summary

Command	Description
check_afv	Print requested Architectural Modeling information or execute the requested action.
check_c2rtl	Compare a C++ model to RTL or a C++ model to another C++ model, spec and imp.
check_caf	Configure, manipulate, and waive checks or display/hide views in CAF.
check_cdc	Initiate structural and functional verification on clock domain crossings.
check_conn	Print requested Connectivity information or execute requested action.
check_cov	Compute, manage, and report proof core and stimuli coverage.
check_csr	Print out requested CSR information or execute the requested action.

check_fsv	Check for the observability of faults on user-defined strobing points.
check_hls	Compare a SystemC model to RTL or a SystemC model to another SystemC model.
check_lpv	Load, elaborate, and check power-aware designs.
check_rvv	Check compliance of the RTL design with the RISC-V specification.
check_sec	Compare two RTL designs, the specification and the implementation.
check_spv	Check for absence of undesired functional paths between sets of signals.
check_superlint	Configure, manipulate, and waive checks in the Superlint App.
check_timing_exception	Set up, manages, and reports Timing Exception flow.
check_unr	Analyze unreachable branch, expression, FSM, and toggle signals in a design.
check_xprop	Run X-propagation analysis on one or more signals.
config_rtlds	Configure different aspects of the RTL Designer flow.
custom_rule_rtlds	Specify the functionality of a new rule.
read_sdc	Load specified Synopsys® Design Constraints (SDC) files.

App-Specific Configuration Command Summary

The table in this section lists and briefly describes app-specific configuration commands. In some cases, you can use the GUI in place of the command. See the "Equivalent GUI Actions" column.

Configuration commands begin with `set_`, and each `set_*` command has a complementary `get_*` command. Use the `get_*` commands to report the current configuration setting.

App-Specific Configuration Command Summary

Command	Description	Equivalent GUI Action
set_afv_generate_auto_deadlock		

	Specify if AFV should automatically generate a deadlock assertion.	None
set_afv_input_conditions_nested		
	Specify input as single-line conditional or multiple nested if-statements.	None
set_afv_prll_property_consolidation		
	Specify the number of rows at which the tool generates only one parallel property.	None
set_afv_strict_declarations	Specify strict declarations for AFV signals.	None
set_c2rtl_disable_imp_assumption		
	Specify whether to automatically disable imp embedded assumptions.	None
set_c2rtl_max_signals_to_plot		
	Set the maximum number of signals plotted in a C/C++ plot without user prompt.	None
set_caf_engine_mode	Control the engine mode used by CAF.	None
set_caf_prove_keep_traces	Specify whether `check_caf -prove` keeps traces.	None
set_caf_prove_parallel_tasks	Control task parallelism during CAF proof stage.	None
set_caf_prove_time_limit	Specify the time limit for the `check_caf -prove` command.	None

set_caf_signals_customize	Specify signals to be used for CAF HLS_BT_STCK checks with custom types.	None
set_caf_signals_extraction_heuristic	Configure heuristic for extracting signal checks in CAF.	
set_caf_signals_filter_connected	Control filtering of connected signals for CAF signal checks.	None
set_caf_signals_filter_constant	Control filtering of constant signals for CAF signal checks.	None
set_caf_signals_filter_wide_signal	Configure whether CAF extracts checks on wide signals.	None
set_caf_signals_max_bits	Limit the number of bits for stuckat property generation in CAF.	None
set_caf_signal_type	Enable signal extraction for the specified signal types only.	None
set_caf_stuck_at_bitblasted	Control the bit blasting of the stuckat properties in CAF.	None
set_cdc_clock_period_max_factor		
	Specify the maximum normalized period of a CDC clock.	None
set_cdc_calculate_boundary_data_during_extract		
	Enable boundary data calculation during the CDC extraction phase.	None

set_cdc_conditional_waiver_property_limit	Set the maximum number of properties CDC generates for conditional waivers.	None
set_cdc_gate_level_analysis_mode		
	Enable or disable CDC gate-level analysis mode.	None
set_cdc_machine_learning_flow	Enable the machine-learning (ML) flow for suggested waivers in the CDC domain.	None
set_cdc_parallel_extraction	Set the number of threads for CDC structural analysis.	None
set_cfe_compile_pedantic_errors	Control whether the CFE compiler enforces strict C/C++ standard compliance.	None
set_cfe_profiler_default_function_name		
	Set the default function name display format in the CFE profiler.	None
set_conn_prove_loop_handling		
	Configure the default loop handling method for Connectivity proof strategies.	None
set_debug_handoff_source_lock		
	Specify whether Debug Handoff uses Source Lock as reference for the bundle.	None
set_export_sva_bind_enable_directive		
	Specify a conditional compilation directive for the SVA bind statement.	None
set_export_sva_embed_lib_modules		

	Specify whether the SVA export operation will embed SVA library modules.	None
set_export_sva_handshake_delay_threshold		
	Specify the threshold for the delay in the handshake properties.	None
set_export_sva_hierarchy_separator		
	Specify the hierarchy separator when exporting SVA.	None
set_export_sva_hierarchy_separator_escape_string		
	Specify the string used to replace the RTL hierarchy separator character.	None
set_export_sva_target_environment		
	Specify the target validation environment for SVA export.	None
set_export_sva_trace_suffix_cycles		
	Specify the number of cycles that will be added to the cover property.	None
set_fsv_abstract_result_mode		
	Specify the FSV global fault abstract result mode.	None
set_fsv_allow_faults_in_bbox		
	Specify whether FSV imports faults are injected in blackbox instances.	None
set_fsv_bbox_abstraction_level		

	Specify the level of black box abstraction during task generation.	None
set_fsv_break_loop_mode	Specify how FSV handles breaking combo loops.	None
set_fsv_bus_fault_partition	Specify the chunks a multi-bit signal is divided into when adding a bus fault.	None
set_fsv_bus_fault_width	Specify the threshold at which a multi-bit signal is considered a bus.	None
set_fsv_classification_scheme		
	Specify the scheme used to determine faults classification (ISO 26262 or diagnostic).	None
set_fsv_clock_cycle_time	Specify the clock cycle time.	None
set_fsv_constant_propagation_analysis_optimization		
	Specify the level of optimization during the computation of constant propagation analysis in the structural phase.	None
set_fsv_engine_mode	Set the AutoProve engine mode for the FSV App.	None
set_fsv_export_fault_relations		
	Specify whether to export fault relations to UCM.	None
set_fsv_export_propagated_constants		
	Specify whether FSV exports propagated constants to the `.csv` file.	None

set_fsv_fault_tw	Specify the fault injection time window.	None
set_fsv_generate_activatability		
	Enable or disable activatability analysis.	None
set_fsv_generate_always_detected		
	Enable or disable `always_detected` analysis.	None
set_fsv_generate_always_propagated		
	Enable or disable `always_propagated` analysis.	None
set_fsv_generate_co_max_delay_time		
	Set the max number of cycles to check for detection after fault injection.	None
set_fsv_generate_fo_co_max_delay_time		
	Set the max number of cycles to check for detection after fault propagation.	None
set_fsv_generate_fo_max_delay_time		
	Set the max number of cycles to check for propagation after fault injection.	None
set_fsv_generate_copy_env	Copy the environment from the current task to new tasks.	None
set_fsv_generate_detectability		
	Enable or disable `detectability` analysis.	None
set_fsv_generate_propagatability		

	Enable or disable `propagatability` analysis.	None
set_fsv_generate_propagated_always_detected		
	Enable or disable `propagated_always_detected` analysis.	None
set_fsv_generate_use_bus_faults		
	Automatically generate bus faults instead of bit faults that are part of that bus.	None
set_fsv_generate_use_memory_faults		
	Automatically generate memory faults instead of bit faults that are part of that memory.	None
set_fsv_import_include_ut	Specify whether FSV imports untestable faults.	None
set_fsv_import_prime_only	Specify whether FSV imports all faults or primary faults only.	None
set_fsv_max_checks_per_task		
	Specify the maximum number of checks per task.	None
set_fsv_max_faults_per_task		
	Specify the maximum number of faults per task.	None
set_fsv_memory_fault_width	Specify the threshold at which a multi-bit signal is considered a memory.	None
set_fsv_proof_min_cluster_size		

	Control the cluster size for grouping by instance.	None
set_fsv_proof_sanity_check		
	Specify whether sanity properties should be skipped, handled automatically by FSV, or included when using a regular `prove` command.	None
set_fsv_prove_keep_traces		
	Specify whether `check_fsv -prove` suppresses traces.	None
set_fsv_prove_strategy	Set the FSV prove strategy.	None
set_fsv_set_hold_time	Specify the SET fault hold time.	None
set_fsv_source_browser_max_line_count		
	Specify the maximum number of lines the FSV source browser processes.	None
set_fsv_source_browser_size_threshold		
	Specify the file size threshold (in GB) for the FSV source browser.	None
set_fsv_source_browser_updating		
	Specify whether the source browser is updated during the proof.	None
set_fsv_stopat_abstraction_level		
	Control the abstraction level of stopats in FSV.	None

set_fsv_strobe_checker_mode		
	Specify the checker mode that determines when the tool considers the fault detected at the checker strobe.	None
set_fsv_strobe_checker_mode_assert_value		
	Specify the value at which the bad machine strobe signal is asserted when the `fsv_strobe_checker_mode` value is `assert`.	None
set_fsv_strobe_optimization	Specify whether to generate strobes observer nets based on the strobes in the fanout of the fault only	None
set_fsv_structural_coi	Specify whether to run COI analysis on CO strobes, FO strobes, or all strobes when performing structural analysis.	None
set_fsv_structural_constant	Enable or disable activatability analysis when performing structural analysis.	None
set_fsv_structural_constant_propagation_analysis		
	Enable or disable constant propagation analysis when performing structural analysis.	None
set_fsv_structural_distance	Enable or disable distance calculation when performing structural analysis.	None
set_fsv_structural_fault_relations		

	Enable or disable fault relations analysis when performing structural analysis.	None
set_fsv_structural_propagation_analysis		
	Enable or disable structural sequential constant propagation analysis.	None
set_fsv_structural_seq_constants_engine_mode		
	Set the engine mode for the FSV sequential constants propagation proof.	None
set_fsv_structural_seq_constants_propagation		
	Enable or disable propagation analysis when performing structural analysis.	None
set_fsv_structural_seq_constants_prove_time_limit		
	Specify the time limit for FSV sequential constants propagation analysis proof.	None
set_fsv_unsafe_inference	Enable unsafe inference of check results from bus faults.	None
set_lpv_enable_corrupt_annotation		
	Enable or disable corrupt annotations in Visualize.	None
set_lpv_enable_pin_corruption_in_liberty_bbox		
	Enable pin corruption on blackboxed instances of Liberty cells.	None
set_lpv_enable_upf_path_format		

	Configure the path format used by the UPF query commands.	None
set_lpv_enable_upf_signals		
	Enable or disable UPF signals.	None
set_lpv_xprop_mode	Add the power net of targets to the preconditions of X-Prop targets in the Low Power Verification with X-Propagation Verification flow.	None
set_rtlds_auto_translate_rtl_to_gate_level_signals		
	Enable or disable automatic translation of signals from RTL to gate-level.	None
set_sdc_vhdl_expr_mode	Enable VHDL format processing of SDC files.	None
set_sec_auto_disable_violating_mappings		
	Specify whether SEC sanity check automatically disables violating mappings.	None
set_sec_automap_aggregate_mappings		
	Specify whether SEC auto mapping aggregates compound mappings.	None
set_sec_autoprove	Specify the proof flow the SEC App uses, `autoprove` or `prove`.	Proof Settings dialog – *SEC* tab – *Auto*
set_sec_autoprove_expose_internal_cex		
	Show identified counterexamples in the internal CEX task	None

set_sec_autoprove_expose_status_progress		
	Specify whether SEC displays a task for internal statuses.	None
set_sec_autoprove_include_related_properties		
	Include related verification tasks of the processed targets.	None
set_sec_autoprove_max_number_of_internal_cex		
	Set the max number of counterexamples shown in the internal CEX task.	None
set_sec_autoprove_minimal_max_length_of_internal_cex		
	Set the minimal max length of counterexamples in the internal CEX task.	None
set_sec_autoprove_post_proof_activities_after_stop		
	Specify whether to continue app-specific activity after stopping a proof.	None
set_sec_autoprove_skip_prequalification		
	Skip the SEC AutoProve prequalification step.	None
set_sec_autoprove_strategy		
	Configure SEC AutoProve to use the specified proof strategy.	Proof Settings dialog – *SEC* tab – *SEC AutoProve settings*
set_sec_break_loop_mode	Specify how SEC handles breaking combo loops.	None
set_sec_disable_imp_assumption		

	Disable the imp side assumptions automatically.	None
set_sec_elaborate_mode	Configure elaboration to run with SEC configurations and modeling.	None
set_sec_expose_non_app_related_properties		
	Specify whether to display non-SEC-related properties in the Property Table and report them in the Task Tree.	None
set_sec_fractional_verification_mode	Specify whether SEC fractional verification mode is enabled.	None
set_sec_hierarchical_blackbox_failing_instance		
	Control blackboxing during hierarchical verification.	None
set_sec_hierarchical_check_assumption		
	Include `assumptions_conflict` properties in proofs.	None
set_sec_hierarchical_force_job		
	Set the job results database update option value.	None
set_sec_hierarchical_generate_script		
	Control hierarchical script generation.	None
set_sec_hierarchical_match_port_number		
	Control instance top-level mapping.	None
set_sec_hierarchical_module_average_size		

	Set the average size of the partition.	None
set_sec_hierarchical_module_excludelist		
	Set a list of patterns for excluding modules for hierarchical verification.	None
set_sec_hierarchical_module_includelist		
	Set a list of patterns to match modules for hierarchy decomposition points.	None
set_sec_hierarchical_module_min_size		
	Set the minimum size of the partition.	None
set_sec_hierarchical_num_session		
	Set the number of sessions used for hierarchical verification to *N*.	None
set_sec_hierarchical_regular_expression		
	Set a list of regular expressions for instance matching.	None
set_sec_hierarchical_report_incremental_status		
	Control the writing of status updates in-between verification steps.	None
set_sec_hierarchical_run_time_limit		
	Get the job results database update option value.	None
set_sec_hierarchical_save_partition_database		

	Drive database file generation for the partition process.	None
set_sec_hierarchical_signoff_waiver_list		
	Specify the list of signoff waived categories.	None
set_sec_hierarchical_use_analyze_snapshot		
	Control the reanalysis of designs.	None
set_sec_mapping_max_word_size		
	Specify a maximum size for mapping entries created by the `check_sec -map` command.	None
set_sec_mpram_compound_port_mapping		
	Set whether the tool maps MPRam ports as compounds.	None
set_persistence_proofgrid_timeout_factor		
	Define a factor for persistent ProofGrid timeout configurations.	None
set_sec_proofgrid_utilization_warning_percentage_threshold		
	Specify ProofGrid utilization threshold in percentage for getting warnings.	None
set_sec_return_mapping_dictionary_from_gui		
	Specify whether SEC GUI commands use mapping dictionaries or mapping IDs.	None

set_sec_run_source_browser_diff		
	Specify whether SEC highlights differences between Spec and Imp designs.	None
set_sec_show_bound_progress_in_log		
	Specify whether SEC shows bound progress in the proof log.	None
set_sec_show_proof_log		
	Specify whether you want the Sequential Equivalence Checking (SEC) App to show the proof log.	None
set_sec_task_based_auto_mapping_on_setup		
	Enable generating task-based mappings for verification targets in SEC setup.	
set_sec_twin_debug_mode		
	Specify whether debugging tools open with the SEC twin signal mode enabled.	Visualize window – Click the *Twin* button
set_sec_use_uif_modeling		
	Set whether UIF is used (`on`) or not (`off`) to model blackboxed operators.	None
set_spv_autoprove_basic_engine_mode		

	Set the AutoProve engine mode for the SPV App when using `basic` strategy.	None
set_spv_autoprove_expose_internal_cex		
	Specify whether SPV displays a task for internal counterexamples.	None
set_spv_autoprove_expose_status_progress		
	Specify whether SPV displays a task for internal statuses.	None
set_spv_autoprove_max_number_of_internal_cex		
	Set the max number of counterexamples shown in the internal CEX task.	None
set_spv_autoprove_minimal_max_length_of_internal_cex		
	Set the minimal max length of counterexamples in the internal CEX task.	None
set_spv_autoprove_skip_prequalification		
	Report whether SPV skips the `autoprove` prequalification step.	None
set_spv_break_loop_mode	Specify how SPV handles breaking combo loops.	None
set_spv_create_combined_precondition		
	Specify whether a combined precondition is created with `check_spv -create`.	None
set_spv_graph_show_alternative_paths		

	Specify whether the SPV graph colors alternative paths in light red.	None
set_spv_include_control_logic		
	Specify whether SPV properties analyze control logic during the proof.	None
set_spv_keep_driving_logic	Specify whether SPV keeps driving logic of source signals from original design.	None
set_spv_proofgrid_utilization_warning_percentage_threshold		
	Specify ProofGrid utilization threshold for getting warnings in percentage.	None
set_spv_prove_single_thread		
	Specify whether SPV properties will be proven in a single proof thread.	None
set_spv_show_bound_progress_in_log		
	Specify whether SPV shows bound progress in the proof log.	None
set_spv_show_proof_log		
	Specify whether SPV shows the proof log.	None
set_spv_taint_feedback_loop		
	Specify whether SPV applies a taint barrier on the `-from` signal.	None
set_superlint_add_automatic_task_assumption_bitblasted		

	Enable (or disable) bit blasting of automatic task assumptions.	None
set_superlint_arithmetic_overflow_counter_filter		
	Enable filtering arithmetic overflow of non-critical-valued counters checks.	None
set_superlint_arithmetic_overflow_enable_exponentiation		
	Enable checking overflow on power operation.	None
set_superlint_arithmetic_overflow_enable_multiplication		
	Enable checking for overflow on the expression of each multiplication operation.	None
set_superlint_conflicting_assertions_remove_design_constraints		
	Specify whether `ASR_IS_CNFL` considers design constraints.	None
set_superlint_deadcode_full_case_filter		
	Specify whether Superlint generates dead code checks for defaults of case if the case is already full.	None
set_superlint_deadcode_unified_loop		
	Specify whether Superlint generates multiple properties or a single property for all possible values of loop variables on each line.	None
set_superlint_debug_unreachable		

	Enable computation of root causes for unreachable properties.	None
set_superlint_debug_unreachable_per_property_time_limit		
	Specify the time limit for per property root causes computation.	None
set_superlint_debug_unreachable_time_limit		
	Specify the overall time limit for root causes computation.	None
set_superlint_enable_compile_messages		
	Specify whether Superlint packages analysis and elaboration messages as Superlint checks.	None
set_superlint_enable_hier_lint		
	Specify whether Superlint enables the hierarchical flow in the LINT domain.	None
set_superlint_enable_observability_tags		
	Enable the filtering of given checks by observability expression.	None
set_superlint_enable_rtl_inline_waiver		
	Configure the tool to process lint pragmas.	None
set_superlint_engine_mode	Set the Superlint AutoProve engine mode.	None
set_superlint_engine_mode_fsm_deadlock_livelock		

	Control the engine mode used by Superlint for FSM Deadlock and Livelock checks.	None
set_superlint_extract_exclude_fsm		
	Disable extraction of the specified FSM.	None
set_superlint_extract_low_level_fsm		
	Enable extraction of the specified low-level FSM.	None
set_superlint_filter_covered_reset		
	Specify whether `check_superlint -extract` will report dead code properties covered during reset.	None
set_superlint_fsm_automatic_assumptions_type		
	Specify types of signals on which Superlint generates automatic assumptions for the FSM tasks.	None
set_superlint_fsm_ignore_self_transitions		
	Specify whether the tool generates self-transitions checks.	None
set_superlint_fsm_max_state		
	Specify the maximum limit on states allowed for FSM candidates in the FSM detection process.	None
set_superlint_fsm_no_mtrn_max_state		

	Specify the maximum limit on states allowed for FSM candidates in the extraction of FSM_NO_MTRN checks.	None
set_superlint_fsm_strategy_per_level_time_limit		
	Specify the time limit for each hierarchical level in the FSM divide-and-conquer strategy.	None
set_superlint_fsm_strategy_prove_time_limit		
	Specify the total time limit for the FSM divide-and-conquer strategy.	None
set_superlint_leverage_formal		
	Specify whether Superlint leverages formal engines to refine results and reduce noise.	None
set_superlint_limit_observable_boundary_signal		
	Specify the maximum number of boundaries you want added to the trace when debugging observable violations.	None
set_superlint_loop_count_limit		
	Specify the maximum number of loops in the design for which Superlint extracts properties.	None
set_superlint_loop_filter_startpoints		
	Automatically filter startpoints not connected to loops in Superlint extract.	None

set_superlint_machine_learning_flow	Enable the machine-learning (ML) flow for suggested waivers in the LINT domain.	None
set_superlint_observability_local_expression_levels		
	Specify the maximum number of levels the tool searches for observability gates.	None
set_superlint_parallel_extraction		
	Control the number of threads used during extraction.	None
set_superlint_parallel_synthesis		
	Specify whether Superlint enables the parallel synthesis in the LINT domain.	None
set_superlint_prove_keep_traces		
	Specify whether `check_superlint -prove` will generate traces.	None
set_superlint_prove_memory_limit		
	Specify the memory limit used for engine data structures during `check_superlint -prove`.	None
set_superlint_prove_time_limit		
	Specify the time limit for the `check_superlint -prove` command.	None
set_superlint_report_negative_result_overflow		

	Control reporting of overflow due to assigning negative results to variables.	None
set_superlint_report_same_drivers_multiple_assignment		
	Specify whether to report a SIG_IS_MLAS property when drivers are the same.	None
set_superlint_report_unsigned_casting		
	Control reporting of overflow due to unsigned casting.	None
set_superlint_rule_profiler		
	Enable profiling for Superlint rules.	None
set_superlint_signal_type	Limit signal coverage checks to signals of a particular type: primary inputs, primary outputs, internal signals, or a combination of types.	None
set_superlint_signals_automatic_assumptions_type		
	Specifies signal types for which to generate automatic fairness assumptions.	None
set_superlint_signals_customize		
	Enables signal extraction for the specified signal list.	None
set_superlint_signals_deadlock_bitblasted		
	Disable or re-enable bit blasting of deadlock signals properties.	None
set_superlint_signals_deadlock_max_width		

	Specify the maximum width of signals for which the tool generates signal deadlock checks when bit blasting is off.	None
set_superlint_signals_extraction_heuristic		
	Specify the heuristic Superlint uses during the extraction of signal checks.	None
set_superlint_signals_filter_connected		
	Specify whether Superlint filters connected signals when signal checks are enabled.	None
set_superlint_signals_filter_constant		
	Specify whether Superlint extracts checks on design constants.	None
set_superlint_signals_filter_wide_signal		
	Specify whether Superlint extracts wide signals	None
set_superlint_signals_max_bits		
	Limit the max bits on which stuck-at and toggle properties will be generated.	None
set_superlint_stuck_at_bitblasted		
	Control the bit blasting of stuck-at properties.	None
set_superlint_sva_max_depth		

	Specify the current maximum depth (in terms of gate complexity) for SVA expressions.	None
set_superlint_sva_max_length		
	Specify the current maximum length (in number of characters) for SVA expressions.	None
set_superlint_verbosity_level		
	Specify the verbosity level of Superlint extraction.	None
set_superlint_waiver_export_version		
	Include tool version in the header of the exported Superlint waivers file.	None
set_unr_break_clock_loop		
	Apply stopats on clock loops for the UNR flow.	None
set_unr_functional_coverage		
	Enable functional coverage for the UNR flow.	None
set_unr_functional_coverage_mode		
	Select mode of functional coverage for the UNR flow.	None
set_unr_partition_max_jobs		
	Set a limit on the number of partitions that run in parallel.	None
set_unr_prove_max_chunks	Set the maximum number of chunks for dividing proof properties in the UNR flow.	None

set_unr_prove_max_properties_per_chunk	Set the maximum number of properties per proof chunk in the UNR flow.	None
set_xprop_autoprove_expose_internal_cex	Specify whether XPROP displays a task for internal counterexamples.	None
set_xprop_autoprove_max_number_of_internal_cex		
	Set the max number of counterexamples shown in the internal CEX task.	None
set_xprop_autoprove_minimal_max_length_of_internal_cex		
	Set the minimal max length of counterexamples in the internal CEX task.	None
set_xprop_break_loop_mode	Specify how X-Prop handles breaking combo loops.	None
set_xprop_ignore_non_resetting_flops_as_source		
	Consider only uninitialized flops that are part of a reset block as X-sources.	None
set_xprop_proofgrid_utilization_warning_percentage_threshold		
	Specify ProofGrid utilization threshold for getting warnings in percentage.	None
set_xprop_show_proof_log	Specify whether XPROP shows the proof log.	None

set_xprop_use_all_undriven	Enable or disable treatment of all undriven nets as sources of X during X-Prop checks.	• Menu: *Application –Configuration – X-Propagation Setup Wizard* • Wizard buttons: *X-Propagation Setup Wizard* button
set_xprop_use_bbox_outputs		
	Disable or re-enable treatment of black-box outputs as sources of X during X-Prop checks.	• Menu: *Application –Configuration – X-Propagation Setup Wizard* • Wizard buttons: *X-Propagation Setup Wizard* button
set_xprop_use_bus_contention		

	Disable or re-enable treatment of multiple driven buses as sources of X during X-Prop checks.	• Menu: *Application –Configuration – X-Propagation Setup Wizard* • Wizard buttons: *X-Propagation Setup Wizard* button
set_xprop_use_bus_floating		
	Disable or re-enable treatment of undriven buses as sources of X during X-Prop checks.	• Menu: *Application –Configuration – X-Propagation Setup Wizard* • Wizard buttons: *X-Propagation Setup Wizard* button
set_xprop_use_init_abstraction		

	Enable or disable treatment of init abstractions as sources of X during X-Prop checks.	• Menu: *Application –Configuration – X-Propagation Setup Wizard* • Wizard buttons: *X-Propagation Setup Wizard* button
set_xprop_use_inputs	Enable or disable treatment of primary inputs as sources of X during X-Prop checks.	• Menu: *Application –Configuration – X-Propagation Setup Wizard* • Wizard buttons: *X-Propagation Setup Wizard* button
set_xprop_use_internal_undriven		

	Enable or disable treatment of internal undriven nets as sources of X during X-Prop checks.	• Menu: *Application –Configuration – X-Propagation Setup Wizard* • Wizard buttons: *X-Propagation Setup Wizard* button
set_xprop_use_reset_abstraction		
	Enable flop reset abstractions as sources of X or reinstate the default.	• Menu: *Application –Configuration – X-Propagation Setup Wizard* • Wizard buttons: *X-Propagation Setup Wizard* button

set_xprop_use_reset_state	Disable or re-enable treatment of uninitialized registers as sources of X during X-Prop checks	• Menu: *Application –Configuration – X-Propagation Setup Wizard* • Wizard buttons: *X-Propagation Setup Wizard* button
set_xprop_use_stopats	Enable or disable treatment of stopat nets as sources of X during X-Prop checks.	• Menu: *Application –Configuration – X-Propagation Setup Wizard* • Wizard buttons: *X-Propagation Setup Wizard* button

General Command Usage

This section provides general information on the efficient entry of commands and switches. It includes the following topics:

- Tab Completion
- Stopping a Command
- Prefix Matching

- Specifying the Current Task

Tab Completion

Jasper Apps supports tab completion for commands and for command Help. Tab completion behaves similar to a standard bash shell. That is, the first time you press the `Tab` key, nothing happens unless there is a unique completion for the command.

Examples:

- If we have the commands `cmd1` and `cmd2` and you type `c` and press `Tab`, the text will be expanded to `cmd`. The second time you press `Tab`, the tool lists (in the *Log* pane) all available commands that start with the characters to the left of the cursor. To distinguish this text from other output in this pane, the text is blue. In this example, the tool lists `cmd1` and `cmd2`. (You might need to scroll to the bottom of the *Log* pane to see the return.)
- In the case of the `help` command, if we have the commands `cmd1` and `cmd2` and you type `help c` and press `Tab`, the text will be expanded to `help cmd`. The second time you press `Tab`, the tool lists (in the log pane) all available commands that start with the characters to the left of the cursor.

> Tab completion for Tcl commands is only supported when the cursor is at the start of a line, after a semi-colon (`;`),
> and after a left-square-bracket (`[`), but not if there is a right-square-bracket (`]`) between the `[` and the cursor.

Stopping a Command

Jasper Apps supports the `Ctrl+C` stop command. This command is useful for stopping the currently running command (for example, halting a long-running `prove` command to identify its complexity). When you press `Ctrl+C`, the tool returns to the state that existed prior to running the current command.

Prefix Matching

Jasper Apps supports prefix matching for command switches. You can abbreviate a switch by typing the first one or more letters to form an unambiguous abbreviation. For example, with the `assert` command, you can abbreviate the `-rename` switch with `-rena` or `-ren`. However, you cannot abbreviate `-rename` with `-re` because that causes ambiguity with the `-remove` switch.

- If you attempt to use a prefix that is not unique, the tool does not accept the command. Instead, it prints a message advising that you have used an ambiguous switch.
- Prefix matching is not available for switches inside `-f` files. You must use full switch names.

Specifying the Current Task

In commands that include the `-task` option, replace *`task_name`* with a period (.) to specify the current task.

4

Regular Commands

This chapter provides descriptions of all Jasper Apps regular commands. Use this chapter as a guide to command definitions, syntax, and related commands.

abstract

Creates a counter, initial-value, reset-value, or register-value abstraction.

Syntax

```
Counter Abstraction
-------------------
abstract -counter [-task task_name |-env]
         (signal_name [-values critical_value+])+
         [-disable_condition disable_condition+]
         [-abstract_init_value]

abstract -counter [-task task_name |-env]
        ( -find [-silent]
         |-list [-include_values] [-silent]
         |-remove signal_name
         |-clear
        )

Initial-Value Abstraction
-------------------------
abstract -init_value [-task task_name] register_name_tcl_list

abstract -init_value [-task task_name]
        ( -list  [-silent]
         |-remove signal_name
         |-clear
        )

Reset Value Abstraction
-----------------------
abstract -reset_value
        flop_name_tcl_list [-expression expression |-constant]
        [-condition_expression condition_expression]
        [-env |-task task_name]
        [-bound N]
        [-sync_reset_hint sync_reset_signal]
        [-silent]
```

```
abstract -reset_value
        [-env |-task task_name]
        ( -list [-silent]
         |-remove flop_name_tcl_list [-silent]
         |-clear [-silent]
        )

abstract -reset_value -show flop_name [-typed_list]

Register Value Abstraction
--------------------------
abstract -register_value
        flop_name_tcl_list -expression expression
        [-env |-task task_name]
        [-bound N]

abstract -register_value
        [-env |-task task_name]
        ( -list [-silent]
         |-remove flop_name_tcl_list [-silent]
         |-clear [-silent]
        )
```

Returns

No return values.

Detailed Description

Use this command to create an abstraction for a counter, the initial value of a signal, a reset value, or a register value.

If the analysis region includes a counter that has been abstracted, the tool incorporates the abstraction into its analysis. Abstracted counters in the analysis region appear orange in the Visualize window.

If the tool abstracts the initial value of a signal, its reset value is not included in the analysis.

This command also provides utility switches to get information about abstractions and request actions related to specified abstractions.

```
Counter Abstraction
-------------------
-counter [-task task_name |-env]
         (signal_name [-values critical_value+])+
         [-disable_condition disable_condition+]
         [-abstract_init_value]

    Create a counter abstraction for the specified signal.

    . Use "-task" to specify a task other than the current task.

    . Use "-env" to specify the global environment instead of
      the current task.

    . Use "-values" to specify one or more values that are critical to the
      counter, that is, values that must not be abstracted out.

    . Use "-disable_condition" to specify one or more conditions under which the
      counter's original value should be used instead of the
      abstraction-computed one. For instance, synchronous reset conditions.

    . Use "-abstract_init_value" to convert a known initial value on the output
      of the counter abstraction into an undefined X value, which permits all
      possible initialization values for the specified signals to be explored.
      The undefined value will be present for the "prove" and "visualize"
      commands.

      NOTE:
      . The "-values" feature is expression-mode sensitive. If you use
        "elaborate -mode verilog", the tool only supports Verilog expressions,
        and if you use "elaborate -mode vhdl", the tool only supports VHDL
        expressions. The expressions can either be constant or change during
        the proof.
      . In addition to any critical values you specify, the tool includes
        minimum and maximum counter limits (0 and 2^counterWidth - 1).
      . If the specified signal does not appear to be a counter, the tool does
        not create an abstraction.
      . If any other abstraction is already applied to the counter, the tool
        issues an error message.
      . Specifying expression as a critical value is not supported
        in SPV or X-Prop.
```

```
-counter [-task task_name |-env]
    ( -find [-silent]
     |-list [-include_values] [-silent]
     |-remove signal_name
     |-clear)

    Perform the specified action or get the specified information.

    . Use "-task" to specify a task other than the current task.

    . Use "-env" to specify the global environment instead of
      the current task.

    . Use "-find" to return a list of counters (candidates for abstraction) in
      the cone of influence of the properties in the current or specified task.
      And use "-silent" to turn off the output to the screen and return
      information with a Tcl return value, which is helpful in Tcl scripts.
      Tcl return value is a list containing one element for each counter found.

      NOTE: Each of these elements is a sub-list containing three subelements:
      . First subelement  - Name of the counter
      . Second subelement - List of critical values
      . Third subelement  - List of disable conditions

    . Use "-list" to show counter abstractions in the current or specified
      task.

      . Use "-include_values" to specify the following Tcl return format:
        [<counter_name> <critical_values>]*

        NOTE: The format is a list of Tcl values. If the return includes more
        than one critical value, they are printed inside braces.

      . Use "-silent" to turn off the output to the screen and return
        information with a Tcl return value, which is helpful in Tcl scripts.

    . Use "-remove" to remove the counter abstraction on the specified signal
      in the current or specified task.

    . Use "-clear" to remove all counter abstractions in the current or
      specified task.

      NOTE: This command does not affect SEC properties or abstractions.
```

```
Initial-Value Abstraction
-------------------------
-init_value [-task task_name] register_name_tcl_list

    Create initial-value abstractions in the current or specified task for one
    or more specified signals.

    This command converts a known initial value on the output of a register
    into an undefined X value, which permits all possible initialization values
    for the specified signals to be explored. The undefined value will be
    present for the "prove" and "visualize" commands.

    . Specify a Tcl list of one or more register names or a wildcard string
      representing one or more signals to create initial-value abstractions in
      the current or specified task.

    . Use "-task" to specify a task other than the current task.

    NOTE:
    . This command supports wildcards (* and ?) in the register_name_tcl_list
      argument.
    . As this command ignores concrete values coming from reset analysis,
      if a flop is also reset during proof, this command has no effect.
      Therefore, if you want to also ignore the flop reset value during proof,
      see "abstract -reset_value".

-init_value [-task task_name]
    ( -list [-silent]
     |-remove signal_name
     |-clear)

    Take the specified action on one or more specified signals or get the
    specified information.

    . Use "-task" to specify a task other than the current task.

    . Use "-list" to show initial-value abstractions in the current or
      specified task. And use "-silent" to turn off the output to the screen
      and return signal names with a Tcl return value, which is helpful in Tcl
      scripts.

    . Use "-remove" to remove the abstraction on the specified signal from the
```

```
      current or specified task.

    . Use "-clear" to remove all initial value abstractions in the current or
      specified task.


Reset Value Abstraction
-----------------------
-reset_value
    flop_name_tcl_list [-expression expression |-constant]
    [-condition_expression condition_expression]
    [-env |-task task_name]
    [-bound N] [-silent]
    [-sync_reset_hint sync_reset_signal]

    Create reset-value abstractions for one or more specified flops.

    By default, this command applies to the current task. Use "-env" or "-task"
    to apply the abstraction to the global environment or a specified task.

    This command essentially disconnects the reset value pin of a flop from its
    driving logic, thus preventing the specified flops from receiving a
    concrete value when being reset (that is, during reset analysis or proof).

    . Use "-expression" to define an expression that determines a new reset
      value for the abstracted flops.

    . Use "-condition_expression" to specify an expression for the reset
      condition, which defaults to the reset condition pin of the specified
      flops.

    . Use "-env" to enter the abstraction into the global environment instead
      of the current task.

      NOTE: Using "-env" also affects reset analysis; thus, all loads will be
      affected during reset analysis as well.

    . Use "-task" to apply the command to the specified task instead of the
      current task.

    . Use "-constant" to specify that the flop has an unknown but constant
      reset value.
```

```
      NOTE: Using "-constant" only affects proof cycles, which means the
      abstracted flop might still be left uninitialized by reset analysis.

    . Use "-bound" to apply the specified abstraction for just N number of
      cycles after reset analysis. By default, this abstraction has bound $,
      that is, it applies to all proof cycles.

    . Use "-silent" to turn off the output to the screen.

    . Use "-sync_reset_hint" to specify a signal involved in synchronous reset
      condition logic. This hint is meant to help the tool find the appropriate
      synchronous reset condition for the specified target flop.

    NOTE:
    . This command supports Tcl lists.
    . If you apply the abstraction to a wide signal containing flops and
      elements other than flops, the tool only applies the abstraction to the
      signal flops.
    . This abstraction will only affect reset analysis when using "-env".
    . If you use "-bound" with a value other than the default $ (infinity), the
      abstraction affects the flop reset pin during proof until the specified
      cycle. If you want to apply this abstraction for the reset analysis
      alone, you must use the bound value 0 combined with the "-env" option.

-reset_value [-env |-task task_name]
    ( -list [-silent]
     |-remove flop_name_tcl_list [-silent]
     |-clear [-silent]
    )

    Take the specified action on one or more specified flops or reset-value
    abstractions or list reset-value abstractions.
    . Use "-env" to apply this command to the global environment instead of the
      current task.

    . Use "-task" to apply the command to the specified task instead of the
      current task.

    . Use "-list" to show reset-value abstractions. And use "-silent" to turn
      off the output to the screen and return flop names with a Tcl return
      value, which is helpful in Tcl scripts.

    . Use "-remove" to remove the abstraction on the specified flop. And use
```

```
      "-silent" to turn off the output to the screen.

      NOTE: If the abstraction you intend to remove was applied to a wide
      signal containing flops and elements other than flops, you can use a list
      of signals as the argument.

    . Use "-clear" to remove all reset-value abstractions. And use "-silent" to
      turn off the output to the screen.

      NOTE: This command does not affect SEC properties or abstractions.

-reset_value -show flop_name [-typed_list]
     Show all configurations for the specified abstraction.

     Use "-typed_list" to include the type information before each list in the
     result.

Register Value Abstraction
--------------------------
-register_value flop_name_tcl_list -expression expression
    [-env |-task task_name]
    [-bound N]

    Create register-value abstractions for one or more specified flops.

    WARNING: Unlike other "abstract" options, a register value abstraction is
    not a true abstraction since the "-expression" option might create an
    overconstraint. Thus, proven or unreachable results might not be valid
    if you use the "-register_value" option.

    By default, this command applies to the current task. Use "-env" or "-task"
    to apply the abstraction to the global environment or a specified task.

    This command essentially replaces a register value with an expression
    during proof for an infinite number of cycles or up to a specified number
    of cycles after reset analysis for one or more specified flops.

    . Use "-expression" to define an expression that determines a new value for
      the abstracted flops.

    . Use "-env" to enter the abstraction into the global environment instead
      of the current task.
```

```
    . Use "-task" to apply the command to the specified task instead of the
      current task.

    . Use "-bound" to apply the specified abstraction for the specified number
      of cycles after reset. By default, this abstraction applies to all proof
      cycles. The tool uses the flop's clock for the "specified number of
      cycles."

    NOTE:
    . This command supports Tcl lists.
    . If you apply the abstraction to a wide signal containing flops and
      elements other than flops, the tool only applies the abstraction to the
      flop signals.
    . Although this command replaces a register value with an expression,
      this case is different from an assumption because it does not constrain
      the underlying logic. Therefore, if you want to constrain the register
      and its underlying logic to an expression, see "assume".
    . Register value abstraction is not supported by SPV or X-Prop.

-register_value [-env |-task task_name]
    ( -list [-silent]
     |-remove flop_name_tcl_list [-silent]
     |-clear [-silent]
    )

    Take the specified action on one or more specified flops or
    register-value abstractions or list register-value abstractions.
    . Use "-env" to apply this command to the global environment instead of the
      current task.

    . Use "-task" to apply the command to the specified task instead of the
      current task.

    . Use "-list" to show register-value abstractions. And use "-silent" to
      turn off the output to the screen and return flop names with a Tcl return
      value, which is helpful in Tcl scripts.

    . Use "-remove" to remove the abstraction on the specified flop. And use
      "-silent" to turn off the output to the screen.

      NOTE: If the abstraction you intend to remove was applied to a wide
      signal containing flops and elements other than flops, you can use a list
      of signals as the argument.
```

. Use "-clear" to remove all register-value abstractions. And use "-silent" to turn off the output to the screen.

Example

```
abstract -reset_value [list a b c]
abstract -reset_value {a b c}
abstract -reset_value -reset [list a b c]
abstract -reset_value -reset {a b c}

abstract -counter my_counter

// List counters and show the counter width in parentheses
abstract -counter -find
cnt1 (3)
cnt2 (5)
big_cnt (64)

abstract -init_value a
abstract -reset_value q
abstract -reset_value q -expression {~a}
abstract -reset_value -reset q
abstract -reset_value q -expression {~b} -bound 4
abstract -reset_value q -sync_reset_hint rst
abstract -register_value q -expression 1'b0
abstract -register_value q -expression {b | c} -bound 2

// Show all the configurations for the given abstraction:

abstract -reset_value q -bound 10 -expression {a & b}
abstract -reset_value q -show
q {a & b} {10}

abstract -reset_value q -bound 10 -expression {a & b}
abstract -reset_value q -show -typed_list
target q expression {a & b} bound {10}
```

See Also

clock

abvip

Prints requested ABVIP information or sets the ABVIP installation directory.

Syntax

```
abvip -set_location abvip_directory

abvip -list

abvip -clear

abvip -version
```

Returns

The "-list" subcommand returns a list of ABVIP checkers.

Detailed Description

Use this command to set the ABVIP installation directory, access the documentation files, and so forth. Refer to the examples section, which includes a script and instructions to copy all the files you need to run the script.

The "Getting Started with Cadence Verification IP" manual is available in the tool after you run the "abvip" command. In the main Jasper window, go to Help - ABVIP User Guides or access the manual online at https://support.cadence.com/apex/techpubDocViewerPage?path=vip_getting_started_for_uvm_sv%2Fvip_getting_started_for_uvm_sv11.3%2Fvip_getting_started_for_uvm_svTOC.html.
The manual provides instructions for accessing the Cadence Verification IP online documentation and running examples. Use the following link to access a collection of protocol-specific documentation, videos, Rapid Adoption Kits, and so forth:
https://support.cadence.com/apex/platformPage?searchTerm=Verification%20IP&pageName=VerificationIP.

```
-set_location abvip_directory
    Set the ABVIP installation to the specified directory.

    NOTE:
    . All subsequent "analyze" commands automatically include "-y" and
      "+incdir+" switches as well as ".sv" and ".svp" library extensions to take
      advantage of this information to find ABVIP checker modules. To restore
      "analyze" default behavior, use "abvip -clear".
    . With the exception of "analyze" messages related to ports of top ABVIP
      modules:
      . Jasper does not issue warning messages inside ABVIP modules as the
        "analyze" and "elaborate" commands run.
      . The tool ignores user-specified message severity level changes during
        "analyze" and "elaborate".

-list
    List the ABVIP checkers available in the installation.

-clear
    Clear ABVIP settings. ABVIP checkers will no longer be listed and their
    documentation will no longer be accessible.

-version
    Print the current version of the installed ABVIP.

    NOTE: Prior to using this switch, set the installation using the "abvip
    -set_location abvip_directory" command.
```

Example

```
# The "abvip -set_location" command is not protocol specific. This command
# enables Jasper to analyze ABVIP files for all protocols.
# Refer to the "Detailed description" above for information about user messages.

abvip -set_location $VIPCAT_INST_DIR/tools/abvip/
analyze...
elaborate...

abvip -list
ace5 ace_sys ahb ahb5 amba_lpi apb apb3 apb4 atb axi3 axi4 axi4_ace axi4_stream axi5
chi ddr ddr2 ddr3 ddr5 dfi dfi3 dfi4 i2c lpddr lpddr2 lpddr3 lpddr5 ocp pipe sdram
```

```
# Running an Example
# ==================
# The following example checks ace-sys compilation/elaboration with dummy top.v
# Set up the following environment variables:

# Set the ABVIP_INST_DIR environment variable to the abvip folder in the vipcat
# installation directory:
setenv ABVIP_INST_DIR <VIPCAT_INSTALL_DIR>/tools/abvip

# Change to a working directory.

# Copy the ace_sys example directory:
cp -r $ABVIP_INST_DIR/ace_sys/examples .
cd examples/sys_check/

# Run Jasper:
jg CCI400r1.tcl

# Contents of Tcl script:
# ----------------------
set example_dir $env(ABVIP_INST_DIR)/ace_sys/examples
# set example_dir $env(example_dir)
abvip -set_location $env(ABVIP_INST_DIR)

# Example to check ace-sys compilation/elaboration with dummy top.v
analyze -sv09 $example_dir/sys_check/bind.sv $example_dir/sys_check/top.v
elaborate -top top -bbox_a 1024 -sv09_strong_embedding


clock ACLK
reset ~ARESETn

#Do not run prove. This example checks ace-sys compilation/elaboration.
```

agent

Enables and controls the Jasper Agent.

Syntax

```
Configuration and Initialization
====================================
agent -config -jedai_server jedai_server -model model_name
          [-model_setting model_settings] [-access_token access_token]
          | -show
          | -list list_type

agent -init

AgentRouter
================
agent -query query_text

Ask Jasper Agent
==================
agent -ask_jasper -query query_text

Context Selection
================
agent -context -show -available [-reload]

agent -context -show -available -files category_name [-reload]

agent -context -selected

agent -context -add dictionary

agent -context -add -all category1 [category2 ...]

agent -context -clear

Tcl SVA Generation Agent
================================
agent -tcl_sva_gen -query query_text
```

```
agent -tcl_sva_gen -create -id id
          [-task task_name] [-force]

Design Insight Agent
============================
agent -design_insight -query query_text

Clear History
=============
agent -clear_history
```

Returns

Returns various data structures depending on the "agent" mode used.

Detailed Description

Use this command to configure, initialize, and operate the AI Agent.

NOTE: "agent" is only supported in FPV.

NOTE: Jasper Agent requires the "ChipStack_Base" license.

NOTE: "agent" uses the JedAI LLM infrastructure, which needs to be configured. Contact your local Cadence support team for guidance on how to set up a JedAI LLM server.

NOTE: Required JedAI version: 25.14

```
Configuration and Initialization
================================
-config -jedai_server jedai_server -model model_name
    [-model_setting model_settings] [-access_token access_token]
    Configures the Jasper Agent to point to and authenticate the
    proper JedAI server.

    . Use "-jedai_server" to provide the JedAI server specification. It
      usually follows the format http://server:port.
    . Use "-model" to provide the large language model (LLM)
      specification.
```

```
    . Use "-model_setting" to optionally provide extra settings to the
      LLM.
    . Use "-access_token" to provide the JedAI server authentication
      credential. The access token can also be provided by the
      environment variable JEDAI_ACCESS_TOKEN. See the "Jasper AI
      Agent" appendix in the "Jasper Apps Platform and Formal
      Property Verification App User Guide" for details.
    . Use "-extra_headers" to provide additional HTTP headers for the
      JedAI server requests, as a key-value list.

-config -show
    Returns a dictionary with the current AI Agent configuration.

    . "jedai_server": the configured JedAI server URL.
    . "model": the configured LLM model.
    . "model_setting": extra LLM model settings, as a key-value list.
    . "access_token": the configured authentication credential.
    . "extra_headers": extra HTTP headers, if set.

-config -list list_type
    Lists available items of the specified type. Provide the type as
    an argument. Currently supported types:

    . "model": lists all available language models.

-init
    Initializes the "agent" GUI component. After this command is used,
    the agents' GUI is enabled in the bottom right corner of the
    Jasper window.

Agent Router
============
The "agent" command includes a generic query router that accepts general
AI queries and dispatches them to the most appropriate agent module.
Modules include Ask Jasper, Design Insight, and Tcl SVA Generation. The
router selects the appropriate module based on configuration and selected
context. The Agent chat UI uses this same routing mechanism.

-query query_text
    Sends a general AI query through the Agent Router. The router
    dispatches the query to the most appropriate agent module based
    on configuration and selected context. Use this switch to interact
    with the AI Agent without targeting a specific module directly.
```

```
Ask Jasper Agent
==================
-ask_jasper -query query_text
    Sends the query to the Ask Jasper Agent. The Ask Jasper
    Agent uses the LLM to answer questions about Jasper Apps and
    their usage. The answers are based on Jasper documentation, which is
    required to be present in the JedAI server. The context can be augmented by
    files selected using the context selection commands below.

Context Selection
=================
-context -show -available [-reload]
    Lists all available document categories from the JedAI server. These
    categories organize reference files that can be used to provide
    context to the AI Agent.

    . Use "-reload" to force refresh from the server instead of using
      cached data.
    . Without "-reload", displays an info message showing when the
      cached data was loaded. For example, "Using cached data from
      2025-12-01 14:30:15".

-context -show -available -files category_name [-reload]
    Lists all available files within the specified category from the
    JedAI server.

    . Provide "category_name" as an argument to the "-files" switch.
    . Use "-reload" to force refresh from the server instead of using
      cached data.
    . Without "-reload", displays an info message showing when the
      cached data was loaded.

-context -selected
    Returns all currently selected files that will be used as context
    for AI Agent queries.

    . Returns dictionary format:
      {category1 {file1 file2} category2 {file3}}
    . Returns empty string if no files are currently selected.

-context -add dictionary
    Adds specific files to the context selection. This command clears
```

```
    all previously selected files before adding the new ones specified
    in the dictionary.

    . Use dictionary format:
      {category1 {file1 file2} category2 {file3 file4}}
    . Each category must exist. Verify with "-show" "-available".
    . Each file must exist in its category. Verify with "-show"
      "-available" "-files".
    . Returns a dictionary with selected files organized by category:
      {category1 {file1 file2} category2 {file3 file4}}
    . Validates dictionary format and file existence. See error
      messages EAI047-EAI049 for validation failures.

-context -add -all category1 [category2 ...]
    Adds all files from one or more categories to the context selection.
    This command clears all previously selected files before adding the
    categories.

    . Provide one or more category names as arguments.
    . Each category must exist. Verify with "-show" "-available".
    . Returns a dictionary with selected files organized by category:
      {category1 {file1 file2 ...} category2 {file3 ...}}

-context -clear
    Removes all selected files from the context.

    . Returns "1" if files were cleared.
    . Returns "0" if context was already empty.

Tcl SVA Generation Agent
================================
-tcl_sva_gen -query query_text
    Sends the query to the Tcl SVA Generation. The Tcl SVA Generation
    Agent uses the LLM to convert a natural language sentence
    describing a design behavior to a proper SVA temporal expression.
    It returns the expression along with an associated ID and suggested
    name.

-tcl_sva_gen -create -id id [-task task_name] [-force]
    Uses the expression created in a previous query to create an
    assertion.

    . Use "-id" to provide the ID returned from a previous query
```

```
      command.
    . Use "-task" to provide the task in which to create the assertion.
      If you do not provide this switch, the assertion is created in
      the active task.
    . Use "-force" to replace an assertion with the same name, if it
      exists.

Design Insight Agent
============================
-design_insight -query query_text
    Sends the query to the Design Insight Agent. The Design
    Exploration uses the LLM to answer questions about the design and
    formal environment. It provides the answer as a generated Tcl script
    and its evaluation return.

Clear History
=============
-clear_history
    Clears the conversation history of the agent.
```

Example

```
agent -config -jedai_server http://server.com:9000 -model JEDAI_MODEL \
  -access_token my_access_token
agent -config -show
agent -init
agent -ask_jasper -query "How do I use the prove command?"

# Context Selection Examples
agent -context -show -available
category1 category2 category3

agent -context -show -available -files category1 -reload
jasper_fsv_userguide.pdf jasper_conn_userguide.pdf \
  file1.pdf ...

set files [agent -context -show -available -files category1]
agent -context -add [list category1 $files]
{category1 {file1.pdf file2.pdf ...}}

agent -context -add {category1 {file1.pdf file2.pdf}}
{category1 {file1.pdf file2.pdf}}
```

```
agent -context -add \
  {category1 {file1.pdf} category2 {file2_1.pdf}}
{category1 file1.pdf category2 file2_1.pdf}

agent -context -add -all category1
{category1 {file1.pdf file2.pdf ...}}

agent -context -add -all category1 category2
{category1 {file1.pdf ...} category2 {file2_1.pdf ...}}

agent -context -selected
{category1 {file1.pdf file2.pdf ...} category2 {file2_1.pdf ...}}

agent -context -clear
1

agent -context -selected


agent -context -clear
0

# Lists models
agent -config -list model
model1 model2 model3


# Property Generation Examples
agent -tcl_sva_gen -query "Signal data_valid should be high \
  when ready is asserted"
agent -tcl_sva_gen -create -id 12345 -task my_task

# Design Exploration Examples
agent -design_insight -query \
  "Show me all the registers in my design"

# Clear History Example
agent -clear_history
```

analyze

Analyzes the contents of an HDL or property file.

Syntax

```
analyze ( -vhdl |-vhdl93 |-vhdl2k |-vhdl08
          [-alias alias_name target_library]
         |-verilog |-v2k |-v95 |-vams
         |-sv |-sv05 |-sv09 |-sv12 |-sv17 )
        [-req |-psl |-repository repository_id]
        [-y dir_name]*
        [-v file_name]*
        [-lib default_lib_name]
        [(-L package_name)+ package_declaration.sv |-L library_name]
        [lib_name::]file_name*
        [+libext+ext[+ext]*]
        [+liborder] [+librescan] [+libnonamehide]
        [+libunboundsearch]
        [-ignore_embedded_psl]
        [-ignore_translate_off]
        [+define(+name[=value])+]
        [-define name[=value]]
        [+undef+macro_name[+macro_name]*]
        [-undef macro_name]
        [-f file_name]*
        [-f_relative_to_file_location file_name]*
        [+incdir+dir_name[+dir_name]*]
        [-incdir dir_name]
        [-hdl_sort]
        [-discover]
        [-file_ext_mode file_extension analyze_mode_mnemonic]
        [-bbox_m string]
        [-no_bbox_m string]
        [-sfcu |(-mfcu [-cu_name compilation_unit_name])]
        [-relaxed_case_matching]
        [-disable_ansi_include_file_search]
        [-disable_overwrite]
        [-disable_native_psl_in_vhdl]
        [-change_hdl_mode_inside_vunit]
```

```
        [-no_multi_concat_as_multi_assign_pattern]
        [-enable_early_macro_argument_expansion]
        [-disable_gzip_handling]
        [-treat_work_as_lib_name]
        [-treat_input_port_as_variable]

analyze -register
        ( -vhdl |-vhdl93 |-vhdl2k |-vhdl08
         [-alias alias_name target_library]
         |-verilog |-v2k |-v95 |-vams
         |-sv |-sv05 |-sv09 |-sv12 |-sv17 )
        [-repository repository_id]
        [+define(+name[=value])+]
        [-define name[=value]]
        [+undef+macro_name[+macro_name]*]
        [-undef macro_name]
        [-f file_name]*
        [-f_relative_to_file_location file_name]*
        [-lib default_lib_name]
        [-L library_name]
        [lib_name::]file_name*
        [+libext+ext[+ext]*]
        [+incdir+dir_name[+dir_name]*]
        [-incdir dir_name]
        [-sfcu |-mfcu]
        [-keep_logfile]
        [-treat_work_as_lib_name]
        [-disable_native_psl_in_vhdl]

analyze -register -clear

analyze -save [-dir dir_name] [-req]

analyze -restore [-dir dir_name] [-req]
        [-bbox_m string] [-no_bbox_m string] [-L library_name]

analyze -clear [-req |-repository repository_id]

analyze -gate_level file_name*
        [+define(+name[=value])+]
        [-define name[=value]]
        [-bbox_m string]
        [-no_bbox_m string]
```

```
[-f file_name]*
[-f_relative_to_file_location file_name]*
[-repository repository_id]
[-lib default_lib_name]
[-L library_name]
```

Returns

No return values

Detailed Description

Use this command to analyze the HDL or property files intended for verification. The tool does not synthesize and read in the netlist until you use the "elaborate" command.

NOTE: Jasper Apps ignores a limited set of front-end messages; that is, it does not print them by default. Refer to the command reference manual appendix "Disabled Front-End Messages" for a list of messages you can enable with the command "set_message".

Configurations

Jasper Apps supports SystemVerilog configurations (for example, as stated in Section 33 of the SystemVerilog '09 LRM). This feature helps the designer configure the implementation that will be used for each instance during the synthesis process.

A configuration is simply an explicit set of rules that specify the exact source description used to represent each instance in a design.

The configuration process can be divided into two steps: library mapping and configuring the instances.
. Library mapping places the module implementations in different libraries.
. Configuring is the operation that selects a source representation for an instance.

Refer to the "Libraries" section below and the "-lib" definition to learn more about how Jasper Apps handles library mapping.

To configure the design, you must do the following:

```
. Use the notation described in the LRM (for example, Section 33.4 of the
  SystemVerilog '09 LRM).
. Analyze the file.
. Elaborate the configuration cell as the top module. For example, if the
  configuration is called "cfg1", use the following "elaborate" command:

  elaborate -top cfg1

Libraries
---------
Jasper Apps supports the following library management options:

. -v: for library files
. -y: for library directories
. -lib: an alternative way to create library mappings
. -L: a way to specify search order for library files in the absence of a
  configuration

NOTE: "-lib" and "-L" are constructs defined by SystemVerilog. Refer to Chapter
33 of the SV09 LRM.

When the tool finds an instance of a module or user-defined primitive (UDP)
that cannot be resolved in the source description files, it scans for a
definition in the library files or directories that you specify on the command
line. The tool supports options to control library scanning precedence rules,
including:

. +libext+
. +liborder
. +librescan
. +libnonamehide

Because files analyzed with "-lib" are considered source description files, the
"-L" option has precedence over "-v" and "-y".

Example:

# In the following example, the tool first searches file "m.v" for the
# definition for module m instantiated by top in spite of the "-L" option
# appearing last in the "analyze" command.
#
analyze -sv -lib my_lib m.v
analyze -sv -y lib -v lib.v -L my_lib top.v
```

```
elaborate

NOTE: SystemVerilog configurations provide the most complete and robust
solution for library management; therefore, you should use them unless you
have a very simple case that is easily addressed with command-line library
switches. (Refer to the "Configurations" section above.)

HDL Files
---------
-vhdl |-vhdl93 |-vhdl2k |-vhdl08 [-alias alias_name target_Library]
    Analyze the specified VHDL files.

    . Use "-vhdl" for files that follow IEEE(R) Std 1076(TM)-1993.
    . Use "-vhdl93" for files that follow IEEE Std 1076-1993.
    . Use "-vhdl2k" for files that follow IEEE Std 1076-2002.
    . Use "-vhdl08" for files that follow IEEE Std 1076-2008.
    . Use "-alias" to specify an alias for a library.
      Important things to know about the "-alias" switch:
      . When creating "alias_name", there must be no previously created
        library alias with the same name.
      . Each "analyze" command can have more than one "-alias" switch.
      . Aliases remain in effect in subsequent "analyze" commands.
      . You can create an alias for an alias; for example:
        -alias A B -alias C A
      . Only use "-alias" with a "-vhdl*" switch; otherwise, the tool generates
        an error message.
    NOTE: If your design is written in a mixture of 1993 and 2008 VHDL
    standards, you must use the option "-vhdl08" for all "analyze" commands to
    ensure compatible design analysis.

-verilog |-v2k |-v95 |-vams
    Analyze the specified Verilog files.

    . Use "-verilog" for files that follow IEEE Std 1364(TM)-1995.
    . Use "-v2k" for files that follow IEEE Std 1364-2001 or IEEE Std 1364-2005.
    . Use "-v95" for files that follow IEEE Std 1364-1995.
      This switch is the same as "-verilog".
    . Use "-vams" for files that follow Accellera Verilog-AMS LRM Version 2.4.0.
    NOTE:
    . First note: If your design is written in a mixture of Verilog and VAMS files, you
    must use the option "-mcfu" or "-sfcu" to ensure compatible design analysis.
    . Second note: For more information run 'help -message VERI-2452'.
```

```
-sv |-sv05 |-sv09 |-sv12 |-sv17
    Analyze the specified SystemVerilog files.
    . Use "-sv" for files that follow IEEE Std 1800(TM)-2005.
    . Use "-sv05" for files that follow IEEE Std 1800-2005.
      This switch is the same as "-sv".
    . Use "-sv09" for files that follow IEEE Std 1800-2009.
    . Use "-sv12" for files that follow IEEE Std 1800-2012.
    . Use "-sv17" for files that follow IEEE Std 1800-2017.

Property Files
--------------
-req
    The specified files contain high-level requirements.

    NOTE: The tool does not evaluate "analyze -req" until you run "elaborate"
    or "elaborate -req". Thus, any messages for design errors are printed at
    that time.

-psl
    The specified files contain PSL.

    Use this switch if you are analyzing PSL that is not embedded in the RTL,
    for example, PSL in vunit files.

-repository repository_id
    The specified files contain RTL to be instantiated inside a specified
    repository, which is outside of the main design scope and outside the
    requirements scope.

    The argument "repository_id" is an integer specifying the repository
    inside which the analyzed RTL will be instantiated.

    Currently, two repositories are available: 0 and 1.

Library Management
------------------
-y dir_name*
    When the source code has module instances without a module
    definition, go to the specified directory of Verilog files to find
    the definitions.

    The default extension for the files in this directory is .v, but
    you can specify a different extension with the +libext argument.
```

```
    The tool treats the modules from this directory as regular
    Verilog modules rather than library modules. Also see the "+libext"
    definition below.

    NOTE:
    . Also refer to the "-f" definition for cases when you specify more than
      one dir_name.
    . Use multiple "-y dir_name" instances to specify multiple directories.
    . The tool stops parsing the files in the specified directory upon the
      first match for an instance declaration.
    . If the tool discovers black-boxed modules during this search, it does not
      read their files, so their interfaces are not read, and the tool creates
      an empty module during elaboration based on the module instantiations.
    . IMPORTANT: Jasper reads files in the library directory specified with
      "-y dir_name" only if they have the same name as a module or UDP that has
      been instantiated in the RTL but has not been resolved.

-v file_name*
    When the source code has module instances without a module
    definition, go to the specified Verilog library file to find the
    definition.

    NOTE:
    . Jasper Apps treats instances that are from the Verilog library module
      or analyzed using the "-v" switch as library cells, which are treated in
      the same way as cell define modules. All design traversal features (for
      example, "get_fanin") skip cell defines by default. See also
      "set_cell_define_debug_visibility".
    . The tool does not treat SystemVerilog interfaces from library files as
      library cells.
    . The tool does not recognize PSL properties in library files
      specified with "-v". As a workaround, you can analyze these files
      directly, that is, as part of the design.
    . The "analyze" command automatically loads files with the ".v"
      extension if you do not specify "+libext". However, if you do use
      "+libext", it only loads what you have specified with that command.
    . In accordance with the SystemVerilog LRM, the tool does not automatically
      load package definitions in files analyzed with "-v".
    . Use multiple "-v file_name" instances to specify multiple files.

-lib default_lib_name
    Use default_lib_name as lib_name for file names that do not have
```

```
    an explicit lib_name:: part.

(-L package_name)+ package_declaration.sv |-L library_name
    Use "-L" to specify Verilog packages or a VHDL or Verilog library for the
    tool to search. Also see "set_automatic_library_search".

lib_name::file_name...
    Read "file_name" into library "lib_name". The "lib_name::"
    part is only valid for VHDL, and if none is specified, the
    default library is "work".

+libext+ext...
    Specifies file extensions for one or more library directories.

    The "analyze" command automatically loads files with the ".v" extension if
    you do not specify "+libext". However, if you do use this switch, it only
    loads what you have specified.

+liborder
    Search for module definitions in the library where the instance is located
    and then search remaining libraries in the order they were specified.

+librescan
    Always scan all Verilog(R) directories in the directory list when looking
    for a new module.

+libnonamehide
    Keep all read modules in a temporary library during compilation.

    Important things to know:
    . For the default global file compilation unit (GFCU) mode of analyze, this
      setting is the default.
    . Use "set_analyze_libnonamehide off" to disable the feature.
    . For multi- and single-file compilation unit (MFCU and SFCU) mode, this
      switch is disabled by default.

+libunboundsearch
    Do not clear "-L" options before and after each compilation unit.

Utility Switches
----------------
-ignore_embedded_psl
    Ignore embedded PSL.
```

```
    NOTE:
    . By default, Jasper Apps analyzes embedded PSL.
    . In some cases, the tool is not able to parse certain types of PSL
      comments.
    . Use this command for cases when you decide the tool does not need to
      analyze the embedded PSL.

-ignore_translate_off
    Disable the translate_off pragma during analysis.

+define+name=value...
    Specifies one or more Verilog `define directives or PSL #define directives
    (available in VHDL flavor) with name "name" and value "value".

    NOTE:
    . Macro definitions using "analyze +define" override macro definitions
      using "`define" directive in the RTL, and the tool issues a warning
      message for each overridden "`define".
    . With the exception of SFCU mode, macro definitions with
      "analyze +define" *do not* override "`undef" directives in the RTL.
      Therefore, the tool undefines macros passed to "`undef" directives in
      the RTL even if they were defined in the command line. In SFCU mode,
      the tool ignores those undefinitions and issues a warning message.
    . According to the SystemVerilog 2009 LRM, macro text can be empty.
      Therefore, specifying "analyze -sv09 +define+A=+B=16", when you have the
      following module, the tool issues an error because "+define+A=+B=16" is
      equivalent to A="" (empty string) and B=16. Hence when evaluating the
      parameter value (2*`A), it becomes (2*).

    module EXAMPLE
       #(parameter DEPTH = (2*`A),
                   P_WIDTH = (`B+1),
                   D_WIDTH = ((`B/8) +
                              (`B) +
                              (`B/8) +
                              (`B))
       ) () ;
    endmodule

-define name=value
    Specifies one Verilog `define directives or PSL #define directives
    (available in VHDL flavor) with name "name" and value "value".
```

```
    NOTE:
    . Macro definitions using "analyze -define" override macro definitions
      using "`define" directive in the RTL, and the tool issues a warning
      message for each overridden "`define".
    . With the exception of SFCU mode, macro definitions with
      "analyze -define" *do not* override "`undef" directives in the RTL.
      Therefore, the tool undefines macros passed to "`undef" directives in
      the RTL even if they were defined in the command line. In SFCU mode,
      the tool ignores those undefinitions and issues a warning message.
    . According to the SystemVerilog 2009 LRM, macro text can be empty.
      Therefore, specifying "analyze -sv09 -define+A" is equivalent to
      A="" (empty string).
    . Switches "+define" and "-define" can be used together in the same
      "analyze" command.

+undef+macro_name...
    Specifies one or mode Verilog `undef directives with name "macro_name",
    preventing the macro from being defined in the RTL. It also undefines any
    macro named "macro_name" previously defined by earlier "analyze" commands
    when using Global Compilation Unit (GFCU) mode.

-undef macro_name
    Specifies one Verilog `undef directives with name "macro_name", preventing
    the macro from being defined in the RTL.  It also undefines any macro named
    "macro_name" previously defined by earlier "analyze" commands when using
    Global Compilation Unit (GFCU) mode.

    NOTE: Switches "+undef" and "-undef" can be used together in the same
    "analyze" command.

-f file_name*
-f_relative_to_file_location file_name*
    Specifies a file listing HDL sources, options, and other "-f" and
    "-f_relative_to_file_location" calls.

    NOTE:
    . "-f" searches for files and paths relative to the current directory.
    . "-f_relative_to_file_location" searches for files and paths relative to
      where the file file_name is located.
    . Do not specify a language switch (for example, -sv09) in an "-f*" file
      different from the one specified on the command line.
    . Subcommands are not supported inside "-f*" files.
```

```
    . Invalid or unknown entries in "-f*" files are ignored triggering a
      message. To prevent these errors from being skipped, enable the
      "set_compilation_skip_errors_in_filelist on" command.
    . The tool expands Tcl variables inside "-f*" files with Tcl's usual "$"
      syntax. If a Tcl variable referenced with "$" does not exist, the tool
      will search for an environment variable with that name.
    . Use multiple "-f* file_name" instances to specify multiple files.
    . Jasper Apps supports absolute paths declared inside .f* files, for
      example, /home/user/test.v.
    . Comments are allowed inside "-f*" files. Parsing of comments has higher
      precedence than any other syntax. The supported comment delimiters are:
      . "//" with the comment ending at the end of the line (valid in paths)
      . "#" with the comment ending at the end of the line
      . "--" with the comment ending at the end of the line
      . "/*" with the comment ending on the next matching "*/", without nesting
    . The character ";" is not recognized in "-f*" files.

+incdir+dir...
    Specifies an include directory. The tool uses this switch to locate files
    that are specified using the Verilog `include directive or the PSL #include
    directive that is available in the VHDL flavor.

    NOTE: Although you can use both "+incdir+dir..." and "-incdir" together,
    there is no guarantee that the tool will preserve the order of the
    directories when using them together.

-incdir dir_name
    Specifies an include directory. The tool uses this switch to locate files
    that are specified using the Verilog `include directive or the PSL #include
    directive that is available in the VHDL flavor.

    NOTE: Although you can use both "+incdir+dir..." and "-incdir" together,
    there is no guarantee that the tool will preserve the order of the
    directories when using them together.

-hdl_sort
    Analyze the registered library files in sorted order.

    This switch takes the list of previously registered file names to analyze
    them in a sorted order. It supports Verilog, VHDL, and mixed designs. Use
    this switch with any other of the main "analyze" arguments except "-req".

    NOTE:
```

```
    . Verilog sort is not able to sort macro dependencies.
    . Verilog sort is not able to support Libmaps/library declarations.
    . SystemVerilog Configurations are partially supported. See Appendix O
      in the Jasper Apps Cmd Ref for details.
    . VHDL sort is not able to sort files if any of the files has a
      configuration declaration that has some declaration before the binding
      indication.

-discover
    Analyze the files according to their file extension.

    The default file extensions and their associated analyze mode are
        | File Extension | Analyze Mode       |
        | .v             | Verilog 1995       |
        | .sv            | SystemVerilog 2005 |
        | .vhd           | VHDL 1993          |
        | .vhdl          | VHDL 1993          |
        | .vhdl08        | VHDL 2008          |
        | .vams          | Verilog AMS        |

    NOTE:
    . Any default file extension can be overridden using "-file_ext_mode".
    . Any non-default file extension must be explicitly associated with an
      analyze mode using "-file_ext_mode".

-file_ext_mode
    Define the analyze mode to be used when analyzing a file with a given file
    extension. It can be used with "analyze -hdl_sort".

    The following analyze modes are supported:
        | Analyze Mode       | Mnemonic                                        |
        | Verilog 1995       | verilog                                         |
        | Verilog 2000       | v2k, verilog_2000                               |
        | SystemVerilog 2005 | sv, sv05, system_verilog, system_verilog_2005 |
        | SystemVerilog 2009 | sv09, system_verilog_2009                       |
        | SystemVerilog 2012 | sv12, system_verilog_2012                       |
        | SystemVerilog 2017 | sv17, system_verilog_2017                       |
        | Verilog AMS        | vams, verilog_ams                               |
        | VHDL 1987          | vhdl87, vhdl_1987                               |
        | VHDL 1993          | vhdl93, vhdl_1993                               |
        | VHDL 2008          | vhdl08, vhdl_2008                               |

-bbox_m string
```

```
    Black box the specified set of modules.

    A black box is a module with no observable or controllable content (that
    is, no logical function). By default, the outputs of a black boxed module
    always produce a logic X. If you intend to bind in another module with an
    output port connected and output port wire, the output port wire will be
    polluted by the X value driven by the black box output port. Take note of
    this behavior. If you intend for your bind module to drive this wire, you
    must connect it to the black box module output port signal stub inside the
    module.

    NOTE:
    . This switch accepts Tcl lists. You can also specify multiple modules with
      multiple instances of "-bbox_m" or with a wildcard string.
      For example, the following argument black boxes all modules whose names
      match the wildcard string:
      -bbox_m FIFO_*_TWO_PORT
    . If you specify a name with an escape character, you must transform it to
      a list item.
      For example, change:
      analyze -bbox_m {\escaped }
      To one of the following:
      analyze -bbox_m {{\escaped }}
      analyze -bbox_m {\\escaped }
      analyze -bbox_m [list \\escaped ]
    . If you attempt to black box the "-top" module, the tool issues an error
      during elaboration; therefore, use care when creating the wildcard
      string.

-no_bbox_m string
    Do not black box the specified set of modules.

    NOTE:
    . This switch accepts Tcl lists. You can also specify multiple modules with
      multiple instances of "-no_bbox_m" or with a wildcard string.
      For example, the following argument turns off black boxing for all
      modules whose names match the wildcard string:
      -no_bbox_m FIFO_*_TWO_PORT
    . If you specify a name with an escape character, you must transform it to
      a list item.
      For example, change:
      analyze -no_bbox_m {\escaped }
      To one of the following:
```

```
      analyze -no_bbox_m {{\escaped }}
      analyze -no_bbox_m {\\escaped }
      analyze -no_bbox_m [list \\escaped ]
    . "-no_bbox_m" takes priority over "-bbox_m"; therefore, if you specify a
      module for both "-bbox_m" and "-no_bbox_m", the tool does not black box
      it.
    . Using "-no_bbox_m" without "-bbox_m" has no effect.

-sfcu
    Use the single-file compilation unit analyze mode.

    In this mode, the tool does not pass any declarations or macro definitions
    from one analyzed *file* to another.

    NOTE:
    . As defined by the Verilog 2001 LRM, SFCU mode is only applicable to
      SystemVerilog designs. If you attempt to use SFCU mode with Verilog 1995,
      Verilog 2001, or Verilog AMS, the tool issues a warning saying that this
      Compilation Unit mode is not supported for Verilog designs, and it will
      use MFCU model instead.
    . The Tcl parser recognizes accumulated macros regardless of compilation
      unit mode.
    . The default "analyze" mode is global; that is, "analyze" accumulates
      declarations and macro definitions. Using "-sfcu" can resolve potential
      conflicts that this handling introduces.
    . Avoid using  "analyze" commands with a mix of the global model with SFCU
      model as it might lead to some unpredictable (and unacceptable) behaviors.
    . SystemVerilog checkers are intended as verification library units.
      Therefore, they are visible across compilation units.

-mfcu [-cu_name compilation_unit_name]
    Use the multi-file compilation unit analyze mode.

    Use "-cu_name" to specify a compilation unit name.

    In MFCU mode, the tool does not pass any declarations or macro definitions
    from one *command* to another.

    NOTE:
    . The Tcl parser recognizes accumulated macros regardless of compilation
      unit mode.
    . The default "analyze" mode is global (GFCU); that is, "analyze"
      accumulates declarations and macro definitions.
```

```
    . Avoid using "analyze" commands with a mix of global model with MFCU model
      as it might lead to some unpredictable (and unacceptable) behaviors.
    . Use "-cu_name" to specify that you want the "analyze" command to create
      a new compilation unit with a unique name.
    . SystemVerilog checkers are intended as verification library units.
      Therefore, they are visible across compilation units.

-relaxed_case_matching
    Provide case-insensitive name searching inside a converted Verilog package
    for mixed-language designs.

    NOTE: When you use this switch, we recommend you also use it with the
    "elaborate" command.

-disable_ansi_include_file_search
    By default, when resolving a relative `include filename, Jasper Apps
    searches relative to the directory that contains the file that contains the
    include command. Use this switch to enable a search for the file relative
    to the present working directory of the command line.

    NOTE: If the tool does not find the file, it iterates through any "incdir"
    directories you specified.

-disable_overwrite
    By default, the tool overwrites any previous definitions of a
    SystemVerilog module or VHDL unit if you analyze it again. With this
    switch, the tool retains the first definition and discards any
    additional analyses of the same module or unit.

    NOTE: In addition to SystemVeriog modules and VHDL units, the behavior
    of the "-disable_overwrite" switch also applies to primitives, PSL units,
    interfaces, programs, packages, and checkers.

-disable_native_psl_in_vhdl
    As defined in the IEEE Standard for VHDL Language Reference Manual 2008,
    the IEEE Property Specification Language (PSL) has been incorporated into
    VHDL. This integration can cause "analyze" to produce errors when using
    PSL reserved words such as 'X', 'AG', 'in', and 'inf' in VHDL assertions,
    as these tokens may be interpreted as PSL keywords rather than standard
    VHDL identifiers. Using this switch treats these tokens as identifiers,
    thereby preventing conflicts with PSL reserved keywords.

-change_hdl_mode_inside_vunit
```

```
    Enable the compiler to change the parser mode to the HDL language specified
    by the analyze command when compiling a PSL verification unit. With this
    feature you can, for example, use SystemVerilog assertions inside PSL
    verification units.

-no_multi_concat_as_multi_assign_pattern
    By default, multi-concatenation to unpacked arrays or structs is treated
    as multi-assignment pattern. Use this switch to disable the behavior.

    NOTE: You must use this switch with both "analyze" and "elaborate".

-enable_early_macro_argument_expansion
    Enable the pre-2020.03, non-LRM-compliant behavior of early expansion of
    macro arguments.

-disable_gzip_handling
    Disable read of compressed files in the "analyze" command.

    NOTE: The data format supported is described by RFCs 1950 to 1952 in the
    files rfc1950 (zlib format), rfc1951 (deflate format), and rfc1952 (gzip format).

-treat_work_as_lib_name
    Analyze using work named library instead of the current library.

    NOTE: This switch also applies to library handling in SystemVerilog
    configurations. It complies with Chapter 33 of the SystemVerilog LRM.
    The default behavior in SystemVerilog configurations is to treat the
    library work as the current working library, and not as the library named
    work.

-treat_input_port_as_variable
    Let module input ports accept signals without an explicit nettype when
    using "`default_nettype none".

Registration
------------
-register
    -vhdl |-vhdl93 |-vhdl2k |-vhdl08
    [-alias alias_name target_library]
    |-verilog |-v2k |-v95 |-vams
    |-sv |-sv05 |-sv09 |-sv12 |-sv17 )
    [-repository repository_id]
    [+define(+name[=value])+]
```

```
[-define name[=value]]
[+undef+macro_name[+macro_name]*]
[-undef macro_name]
[-f file_name]*
[-f_relative_to_file_location file_name]*
[-lib default_lib_name]
[-L library_name]
[lib_name::]file_name*
[+libext+ext[+ext]*]
[+incdir+dir_name[+dir_name]*]
[-incdir dir_name]
[-sfcu |-mfcu]
[-keep_logfile]
[-treat_work_as_lib_name]
[-disable_native_psl_in_vhdl]
```

Register the specified library files for later sorting based on inter-file dependencies.

```
. Use "-vhdl" for files that follow IEEE(R) Std 1076(TM)-1993.
. Use "-vhdl93" for files that follow IEEE Std 1076-1993.
. Use "-vhdl2k" for files that follow IEEE Std 1076-2002.
. Use "-vhdl08" for files that follow IEEE Std 1076-2008.
. Use "-alias" to specify an alias for a library.
  Important things to know about the "-alias" switch:
  . When creating "alias_name", there must be no previously created
    library alias with the same name.
  . Each "analyze" command can have more than one "-alias" switch.
  . Aliases remain in effect in subsequent "analyze" commands.
  . You can create an alias for an alias; for example:
    -alias A B -alias C A
  . Only use "-alias" with a "-vhdl*" switch; otherwise, the tool generates
    an error message.
. Use "-verilog" for files that follow IEEE Std 1364(TM)-1995.
. Use "-v2k" for files that follow IEEE Std 1364-2001 or IEEE Std 1364-2005.
. Use "-v95" for files that follow IEEE Std 1364-1995.
  This switch is the same as "-verilog".
. Use "-vams" for files that follow Accellera Verilog-AMS LRM Version 2.4.0.
. Use "-sv" for files that follow IEEE Std 1800(TM)-2005.
. Use "-sv05" for files that follow IEEE Std 1800-2005.
  This switch is the same as "-sv".
. Use "-sv09" for files that follow IEEE Std 1800-2009.
. Use "-sv12" for files that follow IEEE Std 1800-2012.
```

- Use "-sv17" for files that follow IEEE Std 1800-2017.
- Use "-repository" to specify in which repository the "analyze" command must be performed.
- Use "+define" to specify one or more Verilog `define directives
- Use "-define" to specify one Verilog `define directives
- Use "+undef" to specify one or more Verilog `undef directives
- Use "-undef" to specify one Verilog `undef directives
- Use "-f" and "-f_relative_to_file_location" to specify a file listing HDL sources, options, and other "-f" and "-f_relative_to_file_location" calls. NOTE:
  - "-f" searches for files and paths relative to the current directory.
  - "-f_relative_to_file_location" searches for files and paths relative to where the file "file_name" is located.
  - Do not specify a language switch (for example, "-vhdl") in an "-f*" file different from the one specified on the command line.
  - Subcommands are not supported inside "-f*" files.
  - Invalid or unknown entries in "-f*" files are ignored, triggering a message. To prevent these errors from being skipped, enable the "set_compilation_skip_errors_in_filelist on" command.
  - The tool expands Tcl variables inside "-f*" files with Tcl's usual "$" syntax. If a Tcl variable referenced with "$" does not exist, the tool searches for an environment variable with that name.
  - Use multiple "-f* file_name" instances to specify multiple files.
  - Jasper Apps supports absolute paths declared inside .f* files, for example, /home/user/test.v.
  - Comments are permitted inside "-f*" files. Parsing of comments has higher precedence than any other syntax. The supported comment delimiters are:
    - "//" with the comment ending at the end of the line (valid in paths)
    - "#" with the comment ending at the end of the line
    - "--" with the comment ending at the end of the line
    - "/*" with the comment ending on the next matching "*/", without nesting
  - The character ";" is not recognized in "-f*" files.
- Use "-lib" to specify that you want to use default_lib_name as the lib_name for file names that do not have an explicit lib_name:: part.
- Use "-L" to specify a VHDL library for the tool to search. Also see "set_automatic_library_search".
- Specify "lib_name::file_name..." if you want the tool to read "file_name" into library "lib_name". The "lib_name::" part is only valid for VHDL, and if you do not specify one, the default library is "work".
- Use "-sfcu/-mfcu" to specify the Compilation Unit model to be used when analyzing the listed files to define the sort order.
- Use "+libext" to specify file extensions for one or more library directories.

```
    . Use "+incdir" to specify one or more include directories.
    . Use "-incdir" to specify one include directory.
    . Use "-keep_logfile" to save the "analyze" log file.
      Important things to know about this command:
      . Only use this switch when requested by your Cadence representative. It
        is designed for tool debugging only.
      . The tool stores the log in the following file:
        <project>/sessionLogs/<session identifier>/analyze*.log
    . Use "-treat_work_as_lib_name" to indicate that "-L work" refers to the
      library named "work" instead of the current working library.
    . Use "-disable_native_psl_in_vhdl" to treat PSL reserved keywords as
      identifiers in VHDL 2008.

    NOTE:
    . The commands "elaborate" and "clear -all" clear the registered file list.
    . If your design is written in a mixture of 1993 and 2008 VHDL
      standards, you must use the option "-vhdl08" for all "analyze" commands
      to ensure compatible design analysis.

    LIMITATIONS:
    . "-register" does not support mixed VHDL-SystemVerilog designs.
    . You can only register library files. Directories are not supported.
    . The tool cannot sort the files in the correct order if any of the files
      has a configuration declaration that has some declaration before the
      binding indication.

-register -clear
    Remove the list of previously registered files.

Save/Restore
------------
-save [-dir dir_name] [-req]
    Save the analyzed results of all Verilog and VHDL modules.

    . Use "-dir" to specify a directory name. If you do not specify a name, the
      tool uses the project directory:
      <project_dir>/sessionLogs/<session identifier>/).
    . Use "-req" to save only requirement analyzed results.

    NOTE:
    . Use this command just after the last "analyze" command and before the
      "elaborate" command.
    . Refer to the known limitations listed with the "-restore" documentation
```

```
      below.

-restore [-dir dir_name] [-req] [-bbox_m string] [-no_bbox_m string]
         [-L library_name]
    Restore the analyzed results available inside the specified directory.

    NOTE: If you do not specify a directory, the tool uses the project
    directory.

    . Use "-req" to restore only requirement analyzed results.
    . Use "-bbox_m" to black box the specified set of modules in the restored
      analysis.

      NOTE:
      . This switch accepts Tcl lists. You can also specify multiple modules
        with multiple instances of "-bbox_m" or with a wildcard string.
        For example, the following argument black boxes all modules whose names
        match the wildcard string:
        -bbox_m FIFO_*_TWO_PORT
      . If you specify a name with an escape character, you must transform it
        to a list item.
        For example, change:
        analyze -bbox_m {\escaped }
        To one of the following:
        analyze -bbox_m {{\escaped }}
        analyze -bbox_m {\\escaped }
        analyze -bbox_m [list \\escaped ]
      . If you attempt to black box the "-top" module, the tool issues an error
        during elaboration; therefore, use care when creating the wildcard
        string.

    . Use "-no_bbox_m" if you do not want the tool to black box the specified
      set of modules in the restored analysis.

      NOTE:
      . This switch accepts Tcl lists. You can also specify multiple modules
        with multiple instances of "-no_bbox_m" or with a wildcard string.
        For example, the following argument turns off black boxing for all
        modules whose names match the wildcard string:
        -no_bbox_m FIFO_*_TWO_PORT
      . If you specify a name with an escape character, you must transform it
        to a list item.
        For example, change:
```

```
        analyze -no_bbox_m {\escaped }
        To one of the following:
        analyze -no_bbox_m {{\escaped }}
        analyze -no_bbox_m {\\escaped }
        analyze -no_bbox_m [list \\escaped ]
      . "-no_bbox_m" takes priority over "-bbox_m"; therefore, if you specify a
        module for both "-bbox_m" and "-no_bbox_m", the tool does not black box
        it.
      . Using "-no_bbox_m" without "-bbox_m" has no effect.

    . Use "-L" to specify Verilog packages or a VHDL or Verilog library for the
      tool to search when restoring the analysis. Also see
      "set_automatic_library_search".

    Known limitation for the save and restore features:
    . When you restore the analysis, some tool capabilities (for example, the
      Source Browser) will not work if the HDL files are not accessible.
    . To restore the database for Superlint, use "restore".
      This command is necessary because "check_superlint -extract" does not
      extract information related to messages issued by "analyze" if you run
      it after you run "analyze -restore".
    . To restore the database for Coverage, use "restore".
    . Designs that redefine Proof Accelerators or standard library packages are
      not supported by "analyze -restore".

Clear Options
-------------

 -clear
    Clear all previously analyzed modules from memory but preserve
    the existing data structure to continue work in the current
    session.

    NOTE: Use this option for the following situations:
    . Before analyzing a design with a different top module and running the
      "elaborate" command.
    . After correcting errors that caused an unsuccessful elaboration and
      before re-analyzing and re-elaborating the design.

    Use "clear -all" to clear the entire data structure and proof
    information.

-clear -req
```

```
    Clear previously analyzed requirement files but preserve the design
    to continue work in the current session.

-clear -repository repository_id
    Clear RTL files previously analyzed inside the specified repository.

Gate-Level Netlist
------------
-gate_level file_name*
            [+define(+name[=value])+]
            [-define name[=value]]
            [-bbox_m string]
            [-no_bbox_m string]
            [-f file_name]*
            [-f_relative_to_file_location file_name]*
            [-repository repository_id]
            [-lib default_lib_name]
            [-L library_name]
    Reads a gate-level netlist. See a description of the options in the
    "Utility Switches" and "Property Files" sections of the help.

    NOTE:
    . A gate-level netlist is an output of synthesis tools expressed in Verilog
      and composed of bit-blasted cells with no higher-level RTL constructs, for
      example, always blocks, generates, and so forth.
    . This command enables the gate-level compilation mode and messages with
      the same ID are limited to 20 repetitions during elaboration.
    . This command can be used with Jasper FPV, CDC, and SEC Apps.
    . This command cannot be used with VHDL cells definition.
    . This command disables Blackbox Assistant, parallel elaboration, and
      resilient compilation.
    . The supported "elaborate" switches are "-bbox", "-bbox_m", "-no_bbox_m",
      "-top", "-f", "-clear", "-triple_equal", "-bbox_i", "-no_bbox_i",
      "-bbox_a", "-bbox_mul", "-bbox_div", "-bbox_mod", "-bbox_pow", "-L",
      "-model_tran_gates", "-disable_auto_bbox", "-loop_limit", and "-parameter".
    . "elaborate -parameter" is only supported if the top module is not from
      "analyze -gate_level" command.
```

Example

```
analyze -v2k my_hdl_file.v
analyze -v2k -req my_hlr_file.v
```

```
analyze -vhdl -psl my_property_file.vhdl.psl
analyze -verilog -ignore_translate_off my_hdl_file.v

// Importing packages not specified in the current or standard library:
analyze -sv -lib pkg p.w
analyze -sv -lib pkg2 q.w
analyze -sv -L pkg -L pkg2 pkg_decl.sv
elaborate

// analyzing libraries but has some definitions defined in both - use -mfcu to solve
the conflicts:
analyze -sv -mfcu -lib lib1 lib1.v
analyze -sv -mfcu -lib lib2 lib2.v
elaborate -top lib1.top

// Register and sort design files:
analyze -register -vhdl {lib3::dirA/dirB/libfile.vhd}
analyze -register -vhdl -lib lib2 libx_a.vhd
analyze -register -vhdl {lib1::dirA/dirB/libfile.vhd}
analyze -register -vhdl entity.vhdl
analyze -hdl_sort

// Register and sort mixed design files:
analyze -register -vhdl counter.vhd
analyze -register -sv09 top.v
analyze -register -vhdl08 middle.vhd
analyze -register -sv pkg.v
analyze -hdl_sort

// Save and restore "analyze" results:
analyze -save -dir my_save_dir
analyze -restore -dir my_save_dir

// Reading a gate-level file with liberty cells:
set_liberty_enable_function_synthesis on
liberty -load liberty.lib
analyze -gate_level top.v
elaborate -top top

// Reading a gate-level file with behavioral modules and UDPs cells:
analyze -verilog udps.v
analyze -verilog behavioral_modules.v
analyze -gate_level top.v
```

```
elaborate -top top

// Analyzing files based on their file extension:
analyze -discover test.vhd test.v -file_ext_mode .v sv05
analyze -discover lib1/test.vhd lib1/mod1.v -file_ext_mode .v sv09 -file_ext_mode .vhd
vhdl08 -lib lib1
analyze -discover lib2/test.vhd lib2/mod2.v -file_ext_mode .v sv12 -file_ext_mode .vhd
vhdl19 -lib lib2
elaborate -L lib1 -L lib2
```

See Also

elaborate,
set_analyze_libnonamehide,
set_analyze_librescan

assert

Adds an assertion. Also removes, modifies, converts, lists, etc.

Associated App(s)

Depending on the property to be created.

Syntax

```
assert [-task task_name |-name name
       |-name task_name::name]
       expression
       [-no_precondition]
       [-label label] [-annotation '{'annotation'}']

assert -replace
        (-name name)
       |(-name task_name::name)
       expression
       [-label label] [-annotation '{'annotation'}']

assert -helper
        (-name name)
       |(-name task_name::name)
       expression [-annotation '{'annotation'}']

assert -set_helper
        name
       |task_name::name

assert -clear_helper
        name
       |task_name::name

assert -set_sst
        name
       |task_name::name
```

```
assert -clear_sst
        name
       |task_name::name

assert -mark_proven
        name
       |task_name::name

assert -xprop
       [-from input_signal_tcl_list]
       (-to barrier_signal_tcl_list)
       [-precond expression [-no_precondition_cover]]
       [-sampling_clock clock] [-delay N]
       [(-task task_name) |(-name property_name)
       |(-name task_name::name)] [-annotation '{'annotation'}']

assert -k_liveness -from target_liveness_property -k N
       [-fairness list_fairness_assumptions]
       [-max_k N]
       [-step_k N]
       [-name property_name]

assert -from_assume assume_name+ [-regexp] [-remove_original]

assert -list [-silent] [-type (xprop | spv)] [-task task_name]

assert -clear [-task task_name]

assert -show name

assert -remove name+ [-regexp]

assert -disable name+ [-regexp [-case_insensitive]]

assert -enable name+ [-regexp [-case_insensitive]]

assert -clear_store_trace name+ [-regexp]

assert -set_store_trace [value] name+ [-regexp]

assert -set_trace_extension N [-posedge signal_tcl_list] name+ [-regexp]

assert -clear_trace_extension name+ [-regexp]
```

```
assert -clear_intermediate name+

assert -set_intermediate name+

assert -rename old_name new_name

assert -set_target_bound N {assert_name_list} [-regexp]

assert -clear_target_bound {assert_name_list} [-regexp]
```

Returns

Returns a string (identifier name) for the assertion.

Detailed Description

```
Use this command to add an assertion about the signals in the design.
By default, this command applies to the current task and does not automatically
update the database. Use "-task" to specify a different task.

NOTE:
. You can define a clocking and reset statement in your property.

  Example:

  assert { @(posedge clk) disable iff (!rstn) (a) }

. If you declare a Tcl property without an explicit clock (for example,
  assert {something}), the clocking of this property will be the fastest
  sampler possible, which depends on the clock setup.
  . If clock setup is clock clk, the property is clocked at @(posedge clk)
  . If clock setup is clock clk -both_edges, the property is clocked at @(clk)

  If you do not specify clocking, the property behaves as an immediate property.
  In Visualize, it will be sampled at all times and it can evaluate to true or
  false at the beginning of a cycle, reflecting input value changes immediately
  as they happen in the same cycle.

[-task task_name |-name name |-name task_name::name]
       expression
```

```
       [-no_precondition]
       [-label label] [-annotation {annotation}]
    Create an assertion for the specified Boolean expression that evaluates
    to either true or false.

    Use the "prove" command to verify this assertion.

    NOTE: Jasper Apps supports SVA expressions. Refer to your command
    reference manual for limitations and clarification.

    . Use "-task" to apply the command to the specified task instead of the
      current task.
    . Use "-name" to assign a name to the property.
    . Use "-no_precondition" if you are creating a liveness assertion and you
      do not want the tool to automatically generate precondition covers.
    . Use "-label" to add an element heading. The GUIs will display this
      heading instead of the property name.
    . Use "-annotation" to annotate the specified property with the description
      enclosed by the curly braces. You can view annotations in the Visualize
      window waveforms pane by positioning the mouse to hover over the property
      and reveal a tooltip. And you can print annotations to the screen with
      the "get_annotation" command.

    NOTE:
    . Do not use * or ? in property names.
    . Do not use the "-task" and "-name" switches in the same command.
      If you need to specify both a "name" and "task_name", use the "-name"
      switch and prepend the property name with "<task>::". (Replace <task>
      with an actual task name.) Example:

      assert -name taskA::A {~rdy => ~trx}

-replace (-name name) |(-name task_name::name)
         expression
         [-label label] [-annotation {annotation}]
    Replace the expression for the specified assertion.

    . Use this command to edit an assertion used as a Visualize target without
      triggering a clear-all. Replot the trace after you use this command.
    . Use "-name" to specify the assertion you are editing.
    . Use "-label" to add or change the property's element heading.
    . Use "-annotation" to add or change the property's annotation.
```

```
    NOTE: This command does not apply to embedded assertions.

-helper (-name name) |(-name task_name::name)
         expression [-annotation {annotation}]
    Create a helper assertion with the specified expression.

    NOTE: Use "prove -with_helpers" to run the proof with these helper
    assertions.

    . Use "-name" to specify the assertion's name.
    . Use "-annotation" to annotate the specified property with the description
      enclosed by the curly braces. You can view annotations in the Visualize
      window waveforms pane by positioning the mouse to hover over the property
      and reveal a tooltip. And you can print annotations to the screen with
      the "get_annotation" command.

    Use this command to create a helper assertion that is used by the engines
    to prove divergent properties. See "prove -sst" for more details.

-set_helper (name | task_name::name)
    Convert the specified regular assertion to a helper assertion.

    NOTE: Use "prove -with_helpers" to run the proof with these helper
    assertions.

-clear_helper (name | task_name::name)
    Convert the specified helper assertion to a regular assertion.

-set_sst (name | task_name::name)
    Convert the specified regular assertion to an SST assertion.
    See "prove -sst" for more details.

-clear_sst (name | task_name::name)
    Clear the specified SST assertion.

-mark_proven (name | task_name::name)
    Mark the specified undetermined, unprocessed, or ar_cex assertion as
    "proven" regardless of its actual status.

    This command is typically used with state-space tunneling (SST) to speed up
    proofs of complex properties.

    CAUTION: If you mark an assertion "proven", the tool treats the specified
```

```
    assertion the same way it treats the assertions it proves. So if you
    subsequently run commands such as "prove -all", "prove -task", or
    "prove -property -with_proven", the "marked_proven" status affects the
    results of other properties; that is, the tool does not consider situations
    where these "marked_proven" properties might not hold. To clear the
    "marked_proven" status, you must clear all results for the assertion's
    task.

    NOTE: The SEC, C2RTL, SPV, and XPROP Apps do not support this feature.

-xprop [-from input_signal_tcl_list]
    (-to barrier_signal_tcl_list)
    [-precond expression [-no_precondition_cover]]
    [-sampling_clock clock] [-delay N]
    [(-task task_name) |(-name property_name) |(-name task_name::name)]
    [-annotation {annotation}]
    Generate an assertion to check that X values cannot propagate between two
    points in the design.

    The tool extracts the internal nodes that have explicit X values and starts
    from there.

    NOTE: To include X-propagation checks for SVA assertions that use the
    $isunknown keyword, do the following:
    . Use the "assert" command with the following format instead of using
      the "-xprop" switch:
      assert -name p1 {$isunknown(<signal>)}
    . Use "-from" to add originating signals for the path. This switch adds
      X sources to the default list.
    . Use "-to" to select the end points for the path. This switch is required.
    . Use "-precond" to limit the X-propagation checking to situations where
      the precondition (triggering) expression is true; that is, instead of
      checking hasXInOutput, check preconditionExpression & hasXInOutput.
      The tool creates a related cover check on the precondition expression. An
      unreachable status of this cover helps in identifying the
      over-constrained scenario.
      NOTE: Only Boolean or sequence expressions are allowed in preconditions,
      that is, you cannot use property or implication operators in the
      precondition expression.
      Use "-no_precondition_cover" to specify that you do not want the tool to
      create the related cover check for the precondition expression.
    . Use "-sampling_clock" to enable handling for sampled properties. The
      signal name must be a single-bit signal (that is, a simple net or a bit
```

```
      of an array).
    . Use "-delay" to delay verification for the specified number of cycles.
      With this switch, there can be no CEX until after the specified cycle.
    . Use "-task" to assign the property to a task other than the current task.
    . Use "-name" to assign a name to the property.
    . Do not use both "-task" and "-name". If you need to specify both, use the
      "-name" switch and prepend the property name with "<task>::".
      (Replace <task> with an actual task name.)
    . Use "-annotation" to annotate the specified property with the description
      enclosed by the curly braces. You can view annotations in the Visualize
      window waveforms pane by positioning the mouse to hover over the property
      and reveal a tooltip. And you can print annotations to the screen with
      the "get_annotation" command.

    Important things to know about this command:
    - When you use "-from", the tool does not consider explicit assignments
      (for example, foo == 1'bx) as X for verification purposes. Uninitialized
      register handling is controlled by a different command,
      "set_xprop_use_reset_state".
    - The "-from" and "-to" switches support Tcl lists and lists of signals
      that can be formed using wildcards, for example:
      assert -xprop -from io_dram?_data_valid -to dram_io_cas0_l
      assert -xprop -from io_dram*_data_valid -to dram_io_cas0_l
    - Signals used with the "-from" switch must be inputs or stopats. They
      cannot be internal signals driven by some logic.
    - You can use ProofGrid to prove multiple "-xprop" properties in
      parallel. To enforce serial processing, put properties in different
      tasks.
    - X-Prop properties are not supported with tunneling. The tool ignores these
      properties when verifying tasks that contain them.

-k_liveness -from target_liveness_property -k N
    [-fairness list_fairness_assumptions]
    [-max_k N]
    [-step_k N]
    [-name property_name]
    k-Liveness is useful if:
    . You have liveness properties that time out or give you little insight.
    . You suspect missing or too-weak fairness assumptions.
    . You want a finite, debuggable trace that explains how progress is being
      deferred.

    k-Liveness proves that, once the fairness conditions have been satisfied k
```

```
    times while the liveness obligation is pending, the obligation occurs. Here,
    k is the count of satisfied fairness states observed during the pending
    period. Proving this safety assertion for some k establishes that the
    original liveness property is provable.

    Important: A k-Liveness counterexample does not necessarily refute the
    liveness property; it only shows you can defer it k times. You might still
    pass at a larger k, or after fixing fairness/assumptions.

    If you find a CEX for a k-Liveness property, you might consider increasing k
    or debugging the k-Liveness trace to see if there is any missing fairness
    constraint or if the trace has a behavior that is unexpected.

    The tool generates an assertion for the requested k. If "-max_k" and
    "-step" options are used, it generates assertions for a range of 1 to
    "-max_k" % "-step"

    . Use "-from" to specify the target liveness property.
    . Use "-fairness" to list the fairness assumptions that are considered in
      the safety k-Liveness assertion. If not specified, the tool considers all
      fairness assumptions.
    . Use "-max_k" to specify the max_k value.
    . Use "-step_k" to specify the step_k value.
    . Use "-name" to specify the prefix of the property name.

-from_assume assume_name+ [-regexp] [-remove_original]
    Create assertions from one or more specified assumptions and disable
    the original assumptions.
    . Use "-remove_original" to remove the original assumption instead of
      disabling it. This action is not propagated to all sessions, even if
      you used "-update_all_sessions".

    Important things to know about this command:
    . Wildcards and regular expressions are supported; for example:
      assert -from_assume {^.*$} -regexp
    . Jasper does not support converting environment assumptions. It
      ignores any environment assumptions matched by wildcard strings.
    . You cannot automatically reverse this command. You
      must manually remove or disable the assert and enable the
      assumption.
    . Related covers linked to the original property are linked to the new
      assertion. The command does not enable related covers by default when
      linking to the new assertion.
```

```
-list [-silent] [-type (xprop | spv)] [-task task_name]
    Show the list of assertions in the specified task.

    . Use "-silent" to return the list as Tcl return values without printing
      the list on the screen.
    . Use "-type" with "xprop" or "spv" to return only the
      list of X-Prop or SPV assertions.
    . Use "-task" to apply the command to the specified task instead of the
      current task.

-clear [-task task_name]
    Remove all assertions specified with the "assert" command.

    Use "-task" to apply the command to the specified task instead of the
    current task.

-show name
    Show the actual expression for the assertion with the specified name.

-remove name+ [-regexp]
    Remove assertions with the specified name.

    Important things to know:
    . Use this switch to specify one or more asserts.
    . Wildcards and regular expressions are supported; for example:
      assert -remove {^.*$} -regexp
    . You can only remove properties created with the GUI or on the command
      line. That is, you cannot remove properties added automatically (for
      example, SVA and PSL embedded properties). However, you can disable
      and enable these automatically added properties.

-disable name+ [-regexp [-case_insensitive]]
    Keep the assertion in the current task but do not
    include it in any analysis.

    Use "-regexp" to specify that you are using a regular expression to specify
    properties. And use "-case_insensitive" to specify that the expression is
    not case sensitive.

    Important things to know:
    . Specify one or more assertions.
    . Wildcards and regular expressions are supported; for example:
```

```
      assert -disable {^.*$} -regexp
    . When you disable a property, the tool discards associated traces and
      issues an info message.

-enable name+ [-regexp [-case_insensitive]]
    Re-enable the specified (previously disabled) assertion.

    Use "-regexp" to specify that you are using a regular expression to specify
    properties. And use "-case_insensitive" to specify that the expression is
    not case sensitive.

    Important things to know:
    . Specify one or more assertions.
    . Wildcards and regular expressions are supported, for example,
      assert -enable {^.*$} -regexp

-clear_store_trace name+ [-regexp]
    Do not generate and store traces for the specified properties during
    proofs.

    Important things to know:
    . The tool establishes property status and bounds as usual.
    . Use this switch to specify one or more assertions.
    . Wildcards and regular expressions are supported, for example,
      assert -enable {^.*$} -regexp

-set_store_trace [value] name+ [-regexp]
    Generate and store traces for the specified properties during proofs,
    according to the specified value.

    Possible values are "0", "1", or "unlimited":
    . 0:         The tool does not generate and store traces (unless shared with
                 and accounted for by another property). Specifying the value
                 0 is equivalent to specifying the switch "-clear_store_trace".
    . 1:         The tool generates and stores a single trace for each
                 property.
    . unlimited: The tool generates and stores an unlimited number of traces.

    Important things to know about specifying a value:
    . If you do not specify the value argument, the default is "1".
    . If you need to specify a property named "0", "1", or "unlimited", you
      must explicitly specify the first argument "0", "1", or "unlimited" and
      then specify the property name.
```

```
    NOTE:
    . The commands "prove -no_traces" and "set_prove_no_traces" take
      precedence over "assert -set_store_trace" values "1" and "unlimited".
    . Use this switch to specify one or more assertions.
    . Wildcards and regular expressions are supported, for example:
      assert -set_store_trace unlimited {^.*$} -regexp

-set_trace_extension N [-posedge signal_tcl_list] name+ [-regexp]
    Extend a trace with N extra posedge events on the specified signals--or
    on :global_clock if no signals are specified--beyond the cycle that
    triggers the property. "$" is a special value that means infinite.
    This switch overrides the global setting (see "set_trace_extension").

    Important things to know:
    . "-set_trace_extension 0" makes the effective value for this property 0
      regardless of the global setting; therefore, it is not the same as
      "-clear_trace_extension".
    . "$" is not supported when using "-posedge".
    . Wildcards and regular expressions are supported.

-clear_trace_extension name+ [-regexp]
    Remove property-specific trace extensions, allowing the global setting to
    apply (see "set_trace_extension").

    Wildcards and regular expressions are supported.

-clear_intermediate name+
    Do not use the specified properties as candidates for intermediate targets
    for engine L.

    NOTE: Use this switch to specify one or more assertions.

-set_intermediate name+
    Allow engine L to use the specified properties as candidates for
    intermediate targets.

    NOTE: Use this switch to specify one or more assertions.

-rename old_name new_name
    Rename an assertion.

-set_target_bound N {assert_name_list} [-regexp]
```

```
    Set the target bound for one or more specified properties.

    IMPORTANT: The status of properties reaching the target bound will change
    to "Bounded Proven (User)" or "Bounded Unreachable (User)"; however, this
    command does not affect proofs in any way. That is, it only changes how the
    results are reported.

    NOTE:
    . When the min_length of an assertion is equal or greater than the
      target bound, its status changes to "Bounded Proven (User)".
      This indicates the assert can still have a CEX of the exact length
      of the target bound, but not shorter.
    . If you want proofs to stop when this target bound is reached, the
      "set_prove_stop_on_target_bound" configuration must be "true". Otherwise,
      Jasper continues running proofs after reaching the target bound.
    . Liveness properties are not supported and ignored by this command.
    . Wildcards and regular expressions are supported.
      Example: assert -set_target_bound N task::*
    . Value 0 for N means an infinite target bound, which is equivalent to
      using "assert -clear_target_bound".
    . The target bound set with this command takes precedence over any bound
      specified with the "set_prove_target_bound" command.
    . If you restore a jasper_database using "restore -jdb" command, the
      status of an "Undetermined" assertion does not automatically change to
      "Bounded Proven (User)" according to the current target bound. You must
      specify the target bound again after restoring the database.

-clear_target_bound {assert_name_list} [-regexp]
    Clear a previously set target bound for one or more specified properties.

    NOTE: Wildcards and regular expressions are supported.
    Example: assert -clear_target_bound N task::*
```

Example

```
assert a==b

assert -name illegal_size {wr_size != 2'b11}

assert {scan_enable == 1'b0}

assert -name illegal_size {WR_SIZE /= "11"}

assert {SCAN_ENABLE = '0'}

# Use an expander and "get_property_list" to run the "assert" command for
# a list of properties:
assert -enable {*}[get_property_list -include {type assert}]

NOTE: Jasper Apps supports the implication overlapping operator
"->" on the command line to declare properties. This
operator indicates that the expressions on both sides happen at
the same iteration.

assert {~rdy -> ~trx}

assert {(rdy = '1') -> (trx = '1')}

# Pattern-matching with regular expressions:
# ------------------------------------------
# Disable assertions that match zero or more characters followed by either
# "_ended" or "_started"
assert -disable {.*(_ended|_started)} -regexp

# Remove assertions that match "trans" followed by 1 or more numbers and
# followed by zero or more characters
assert -remove {trans([0-9]+).*} -regexp
```

See Also

assume,
cover,
prove,
set_prove_target_bound,
set_prove_inferred_target_bound

assume

Adds an assumption. Also removes, modifies, converts, lists, etc.

Associated App(s)

Depending on the property to be created.

Syntax

```
assume [ -env [-name property_name]
         |-task task_name
         |-name property_name
         |-name task_name::property_name
       ]
       [-type (approved | temporary)]
       ([-bound N] expression
       |-constant signal_name
       )
       [-label label] [-annotation '{'annotation'}']
       [-replace]

assume -soft
       [ -env [-name property_name]
         |-task task_name
         |-name property_name
         |-name task_name::property_name
       ]
       [-bound N]
       [-label label] [-annotation '{'annotation'}']
       (expression|-quiet |-noisy |-quiet_from |-max_from)
       [-replace]

assume -xprop
       [-bound N]
       -to signal_name_tcl_list
       [-precond expression]
       [-sampling_clock clock]
       [-task task_name |-name property_name
```

```
        |-name task_name::property_name]
       [-annotation '{'annotation'}']

assume -xprop -is_x
       -to signal_name_tcl_list
       [-precond expression]
       [-task task_name |-name property_name
        |-name task_name::property_name]
       [-annotation '{'annotation'}']

assume -from_assert assert_name+ [-env]
       [-regexp] [-remove_original]

assume -from_assert assert_name+ -bound N [-regexp] [-remove_original]

assume -list [-silent] [-type xprop] [-env |-task task_name]

assume -clear [-env |-task task_name]

assume -show property_name [-env]

assume -remove property_name+ [-regexp] [-env]

assume (-disable |-enable) property_name+ [-regexp [-case_insensitive]] [-env]

assume -rename old_name new_name [-env]

assume -set_type property_name (approved | temporary)

assume -reset expression [-name property_name]
        [-replace |-replace_if_needed]

assume -reset ( -show property_name
                      |-remove property_name
                      |-disable property_name+ [-regexp]
                      |-enable property_name+ [-regexp]
                      |-list [-silent]
                      |-clear)
```

Returns

Returns a string (identifier name) for the assumption.

Detailed Description

Use this command to add an assumption (constraint) to signals in the design under verification or requirement specification. By default, this command applies to the current task and does not automatically update the database. Use "-env" or "-task" to apply the assumption to the global environment or a specified task. The command "assume -env" makes assumptions visible in all tasks and during reset. However, reset analysis only respects assumptions with constants or simple combinational logic. The tool does not respect temporal logic for "-env" assumptions, and in this case, it issues a warning.

NOTE:
. You can define a clocking and reset statement in your property.

  Example:

  assume { @(posedge clk) disable iff (!rstn) (a) }

. If you declare a Tcl property without an explicit clock (for example, assume {something}), the clocking of this property will be the fastest sampler possible, which depends on the clock setup.
  . If clock setup is clock clk, the property is clocked at @(posedge clk)
  . If clock setup is clock clk -both_edges, the property is clocked at @(clk)

  If you do not specify clocking, the property behaves as an immediate property. In Visualize, it will be sampled at all times and it can evaluate to true or false at the beginning of a cycle, reflecting input value changes immediately as they happen in the same cycle.

```
[ -env [-name property_name]
 |-task task_name
 |-name property_name
 |-name task_name::property_name
]
       [-type (approved | temporary)]
       ([-bound N] expression
       |-constant signal_name)
       [-label label] [-annotation {annotation}]
       [-replace]
```

Create an assumption for the specified SystemVerilog expression that evaluates to either true or false.

```
All proofs are carried out assuming that the expression is true.
Scenarios in which the expression is false are ignored in the proof.

NOTE: Jasper Apps supports SVA expressions. Refer to your command
reference manual for limitations and clarification.

. Use "-env" to enter the assumption into the global environment instead
  of the current task and use "-name" to assign a name to the property.
  NOTE:  The command "assume -env" makes assumptions visible in all tasks
  and during reset.
. Use "-name" to assign a name to the property in the current task.
. Use "-task" to apply the command to the specified task instead of the
  current task.
. Use "-type" to add a tag to the assumption:
  . approved - The user has approved the assumption as valid.
  . temporary - The user will discard the assumption later. This is the
    default tag.
. Use "-bound" to apply the specified assumption for N number of
  cycles (after reset). By default an assumption is applicable for
  all cycles.
. Use "-constant" to assume that the signal is pseudo constant, that is,
  the signal has a fixed but unknown value throughout the analysis.
  The tool analyzes the design for all possible values for the specified
  signal; however, the value of the signal remains constant for a trace.
  For example, a design register that defines an input mode can have
  an arbitrary value ranging from 0 to 3. The analysis must consider
  all possible mode values, but it can exploit the fact that the
  mode does not change.
  NOTE: The command "assume -env -constant some_inputs" does not apply
  during the reset phase. As a workaround, you can do one of the following:
  . Specify a constant value.
  . Use "reset -none" and manually specify the reset scheme using
    assumptions.
. Use "-label" to add an element heading. The GUIs will display this
  heading instead of the property name.
. Use "-annotation" to annotate the specified property with the
  description enclosed by the curly braces. You can view annotations in
  the Visualize window waveforms pane by positioning the mouse to hover
  over the property and reveal a tooltip. And you can print annotations to
  the screen with the "get_annotation" command.
. Use "-replace" to modify the specified assumption.
  Important things to know:
  . The "-name" switch is required with "-replace".
```

```
      . Replot any existing trace after you use this command.
      . This command does not apply to embedded assumptions.
      . This command does not apply to X-Prop assumptions.

    NOTE:
    . The following command is the equivalent of using "-env":
      assume -name ::<property_name>
    . Do not use * or ? in property names.
    . Do not use the "-task" and "-name" switches in the same command.
      If you need to specify both a "property_name" and "task_name", use the
      "-name" switch and prepend the property name with "<task>::".
      (Replace <task> with an actual task name.) Example:
      assume -name taskA::A {~rdy => ~trx}

-soft [ -env [-name property_name]
       |-task task_name
       |-name property_name
       |-name task_name::property_name
      ]
       [-bound N]
       [-label label] [-annotation '{'annotation'}']
       (expression |-quiet |-noisy |-quiet_from |-max_from)
       [-replace]
    Create a soft assumption, which is a mere preference on delivered traces
    rather than a hard constraint that must unconditionally be satisfied.
    Jasper only attempts to apply this soft assumption in a proof if
    "set_prove_prefer_quiet on" was executed, "-prefer_quiet" was added to
    the "prove" command, or engine QT was added to the engine list.

    . Use "-env" to enter the soft assumption into the global environment
      instead of the current task and use "-name" to assign a name to the
      property.
      NOTE:  The command "assume -env" makes assumptions visible in all tasks
      and during reset.
    . Use "-name" to assign a name to the property in the current task.
    . Use "-task" to apply the command to the specified task instead of the
      current task.
    . Use "-bound" to apply the specified assumption for N number of cycles
      (after reset). By default, an assumption applies to all cycles.
      NOTE: The switches "-quiet_from" and "-max_from" do not yet support this
      switch.
    . Use "-label" to add an element heading. The GUIs will display this
      heading instead of the property name.
```

```
    . Use "-annotation" to annotate the specified property with the
      description enclosed by the curly braces. You can view annotations in
      the Visualize window waveforms pane by positioning the mouse to hover
      over the property and reveal a tooltip. And you can print annotations to
      the screen with the "get_annotation" command.
      NOTE: This command supports wide signals.
    . Use "-quiet" to generate traces that prefer low inputs.
    . Use "-noisy" to generate traces that prefer edges on all outputs.
    . Use "-quiet_from" to generate traces that rarely propagate originating
      signals ("from" signals).
    . Use "-max_from" to generate traces that prefer propagated originating
      signals ("from" signals).
    . Use "-replace" to modify the specified assumption.
      Important things to know:
      . The "-name" switch is required with "-replace".
      . Replot any existing trace after you use this command.
      . This command does not apply to embedded assumptions.
      . This command does not apply to X-Prop assumptions.

    NOTE:
    . The following command is the equivalent of using "-env":
      assume -name ::<property_name>
    . Do not use * or ? in property names.
    . Do not use the "-task" and "-name" switches in the same command.
      If you need to specify both a "property_name" and "task_name", use the
      "-name" switch and prepend the property name with "<task>::".
      (Replace <task> with an actual task name.) Example:
      assume -name taskA::A {~rdy => ~trx}
    . Do not use $isunknown keyword with this command.

-xprop [-bound N] -to signal_name_tcl_list [-precond expression]
    [-sampling_clock clock]
    [-task task_name |-name property_name |-name task_name::name]
    [-annotation {annotation}]

    Create an assumption that instructs the verification to ignore traces where
    X propagates to the specified signals.

    Using this command not only disables propagation of X from the specified
    signals, it also forbids traces where X propagates to the signals.

    NOTE: As an alternative to the "-xprop" switch, you can specify SVA
    assumptions that use the $isunknown keyword. Do the following:
```

. Use the "assume" command with the following format instead of using
  the "-xprop" switch:
  assume -name p1 {!$isunknown(<signal>)}
. Use "-bound" to apply the specified assumption for N number of
  cycles (after reset). By default an assumption is applicable for
  all cycles.
. Use "-to" to select the end points for the path. This switch is required.
. Use "-precond" to limit the X-propagation assumption to situations where
  the precondition (triggering) expression is true; that is, instead of
  assuming hasXInOutput, assume preconditionExpression & hasXInOutput.
  NOTE: Only Boolean or Sequence expressions are allowed in preconditions,
  that is, no property or implication operators can be used in the precondition
  expression.
. Use "-sampling_clock" to enable handling for sampled properties. The
  signal name must be a single-bit signal (that is, a simple net or a bit
  of an array).
  NOTE: If you specify "-sampling_clock", the tool will produce no
  traces where the "-to" net is X at the rising edge of the sampling clock.
  However, X can still appear and be propagated on the "-to" net in time
  points where the sampling clock does not rise.
. Use "-task" to assign the property to a task other than the current task.
. Use "-name" to assign a name to the property.
. Do not use both "-task" and "-name". If you need to specify both, use the
  "-name" switch and prepend the property name with "<task>::".
  (Replace <task> with an actual task name.)
. Use "-annotation" to annotate the specified property with the
  description enclosed by the curly braces. You can view annotations in
  the Visualize window waveforms pane by positioning the mouse to hover
  over the property and reveal a tooltip. And you can print annotations to
  the screen with the "get_annotation" command.

Important information about this command:
. Global X-Prop assumptions (-env) are not supported.
. Assumptions created with this command are relevant for verification of
  X-propagation (XPROP) and "cover -path", but the tool ignores them
  for other kinds of properties (such as those from the Formal Property
  Verification App and Sequential Equivalence Checking App).
. X-Prop properties are not supported with tunneling. The tool ignores
  these properties when verifying tasks that contain them.
. The "-to" switch supports Tcl lists and lists of signals that can be
  formed using wildcards, for example:
  assume -xprop -to io_dram?_data_valid
  assume -xprop -to io_dram*_data_valid

```
-xprop -is_x -to signal_name_tcl_list [-precond expression]
    [-task task_name |-name property_name |-name task_name::name]
    [-annotation {annotation}]

    Create an assumption that instructs the verification to assume
    X values to one or more specified signal names.

    When you use this command, any specified signal behaves like 1'bX,
    which propagates to its fanout.

    . Use "-to" to select the end points for the path. This switch is required.
    . Use "-precond" to limit the X-propagation assumption to situations where
      the precondition (triggering) expression is true; that is, instead of
      assuming hasXInOutput, assume preconditionExpression & hasXInOutput.
      NOTE: Only Boolean or Sequence expressions are allowed in preconditions,
      that is, no property or implication operators can be used in the precondition
      expression.
    . Use "-task" to assign the property to a task other than the current task.
    . Use "-name" to assign a name to the property.
    . Do not use both "-task" and "-name". If you need to specify both, use the
      "-name" switch and prepend the property name with "<task>::".
      (Replace <task> with an actual task name.)
    . Use "-annotation" to annotate the specified property with the
      description enclosed by the curly braces. You can view annotations in
      the Visualize window waveforms pane by positioning the mouse to hover
      over the property and reveal a tooltip. And you can print annotations to
      the screen with the "get_annotation" command.

    Important information about this command:
    . Global X-Prop assumptions (-env) are not supported.
    . Assumptions created with this command are relevant only for the
      verification of x-propagation.
    . X-Prop properties are not supported with tunneling. The tool ignores these
      properties when verifying tasks that contain them.
    . The "-to" switch supports Tcl lists and lists of signals that can be
      formed using wildcards, for example:
      assume -xprop -to io_dram?_data_valid
      assume -xprop -to io_dram*_data_valid
    . The "-to" switch supports inputs and stopats. You cannot specify an
      internal signal driven by some logic.

-from_assert assert_name+ [-env] [-regexp] [-remove_original]
```

```
    Create an assumption from one or more specified assertions and disable
    the original assertions.
    . Use "-env" to apply the assumption to the global environment instead
      of the current task.
    . Use "-remove_original" to remove the original assertion instead of
      disabling it. This action is not propagated to all sessions, even if
      you used "-update_all_sessions".

    Important information about this command:
    . Wildcards and regular expressions are supported; for example:
      assume -from_assert {^.*$} -regexp
    . You cannot automatically reverse this command. You must
      manually remove or disable the assumption and enable the
      assertion.
    . Related covers linked to the original property are linked to the new
      assumption. The command does not enable related covers by default when
      linking to the new assumption.
    . Trace extension is ignored. See "assert -set_trace_extension" and
      "set_trace_extension".

-from_assert assert_name+ -bound N [-regexp] [-remove_original]
    Create an assumption from one or more specified assertions to be applied
    for the specified number of cycles N (after reset). The original assertion
    cannot produce a CEX within the specified bound, but it might fail after
    this point. For this reason, the original assertion is not disabled and we
    recommend that you formally verify the assertion for failures.
    . Use "-remove_original" to remove the original assertion. This action is
      not propagated to all sessions, even if you used "-update_all_sessions".

    Important information about this command:
    . Wildcards and regular expressions are supported; for example:
      assume -from_assert {^.*$} -regexp -bound N
    . You cannot automatically reverse this command. You
      must manually remove or disable the assumption.
    . Related covers linked to the original property are linked to the new
      assumption. The command does not enable related covers by default when
      linking to the new assumption.
    . Trace extension is ignored. See "assert -set_trace_extension" and
      "set_trace_extension".

-list [-silent] [-type xprop] [-env |-task task_name]
    Show the list of assumptions in the current or specified task.
    . Use "-silent" to return the list as Tcl return values without printing
```

```
      the report on the screen.
    . Use "-type" with "xprop" to return only the list of X-Prop assumptions.
    . Use "-env" to list the assumption set for the global environment.
    . Use "-task" to apply the command to the specified task instead of the
      current task.

-clear [-env |-task task_name]
    Remove all assumptions created with the "assume" command from the current
    or specified task.
    . Use "-env" to remove the assumption set for the global environment.
    . Use "-task" to apply the command to the specified task instead of the
      current task.

-show property_name [-env]
    Show the actual expression for the assumption with the specified
    name.

    Use "-env" to apply this command to the global environment.

-remove property_name+ [-regexp] [-env]
    Remove the assumption with the specified name.

    Use "-env" to apply this command to the global environment.

    Important things to know:
    . Use this switch to specify one or more assumptions.
    . Wildcards and regular expressions are supported; for example:
      assume -remove {^.*$} -regexp
    . You can only remove properties created with the GUI or on the command
      line. That is, you cannot remove properties added automatically (for
      example, SVA and PSL embedded properties). However, you can disable
      and enable these automatically added properties.

-disable |-enable property_name+ [-regexp [-case_insensitive]] [-env]
    Disable or re-enable the assumption with the specified name.

    When you disable an assumption, it remains in the database, but the tool
    does not include it in any analysis.

    . Use "-env" to apply this command to the global environment.

    . Use "-regexp" to specify that you are using a regular expression to
      specify properties. And use "-case_insensitive" to specify that the
```

```
      expression is not case sensitive.

    Important things to know:
    . Specify one or more assumptions.
    . Wildcards and regular expressions are supported; for example:
      assume -disable {^.*$} -regexp

-rename old_name new_name [-env]
    Rename an assumption.
    . Use "-env" to apply this command to the global environment.

-set_type property_name (approved | temporary)
    Add a tag to the specified existing assumption:
    . Use "approved" to specify that you have approved the assumption as valid.
    . Use "temporary" to specify that you  will discard the assumption later.
      This is the default tag.

    Important things to know:
    . This switch supports wildcards (* and ?) in the property name.
    . This switch does not apply to "-soft" assumptions.

-reset expression [-name property_name]
    [-replace |-replace_if_needed]
    Create an assumption that is only valid during reset analysis.
    . Use "-reset expression" to create a reset-specific assumption.
    . Use "-name" to assign a name to the assumption. If you do not specify a
      name, the tool assigns one automatically in the form
      assume:<N>.
    . Use "-replace" to replace an existing assumption with the same name.
      If there is no assumption with the same name, this command fails.
    . Use "-replace_if_needed" to replace an existing assumption or create a
      new assumption if it does not exist.

-reset ( -show property_name
        |-remove property_name
        |-disable property_name+ [-regexp]
        |-enable property_name+ [-regexp]
        |-list [-silent]
        |-clear)
    Manage reset-specific assumptions as specified.
    . Use "-show" to see the actual expression for the assumption with the
      specified name.
    . Use "-remove" to remove the specified reset-specific assumption.
```

```
    . Use "-disable" or "-enable" to disable or re-enable one or more specified
      reset-specific assumptions and use "-regexp" to support regular
      expressions.
      NOTE: This command supports wildcards.
    . Use "-list" to list all declared reset-specific assumptions and use
      "-silent" to return the list as Tcl return values without printing the
      report on the screen.
    . Use "-clear" to remove all declared reset-specific assumptions.
```

Example

```
assume -name illegal_size {wr_size != 2'b11}

assume -env {scan_enable == 1'b0}

assume -name illegal_size {WR_SIZE /= "11"}

assume -env {SCAN_ENABLE = '0'}

# Use an expander and "get_property_list" to run the "assume" command for
# a list of properties:
assume -enable {*}[get_property_list -include {type assume}]

NOTE: Jasper Apps supports the implication overlapping operator
"->" on the command line to declare properties. This
operator indicates that the expressions on both sides happen at
the same iteration.

assume {~rdy -> ~trx}

assume {(rdy = '1') -> (trx = '1')}

# X-Prop examples:
# ----------------
# Assume that the X won't occur for 10 cycles.
assume -xprop -bound 10 -to target_counter

# The following command tells the tool to assume that "signal" WILL NOT have
# an X value during the verification.
assume -xprop -to signal

# The following command tells the tool to assume that "signal" will not have
```

```
# an X value during the verification when "p" holds.
 assume -xprop -to signal -precond p

# For the assumption "assume -env {!rstN |-> din==0}", use the following
# command to randomize inputs:
assume -xprop -is_x -to din -precond ~rstN

# Pattern-matching with regular expressions:
# ------------------------------------------
# Disable assumptions that match zero or more characters followed by either
# "_ended" or "_started"
assume -disable {.*(_ended|_started)} -regexp

# Remove assumptions that match "trans" followed by 1 or more numbers and
# followed by zero or more characters
assume -remove {trans([0-9]+).*} -regexp
```

See Also

assert,
cover

auto_setup

Attempts to automatically generate a script for loading the specified RTL.

Syntax

```
auto_setup ( -vhdl |-verilog |-v2k |-v95 |-sv )
           [-path path]
           [-suffix suffix]
           [-top_mod module_name]
           [-top_file file_name]
           [-save -file file_name [-force]]
```

Returns

No return values

Detailed Description

Use this command to generate a setup script that will analyze and elaborate the RTL files and set up the clocks and resets. The generated setup is intended to be a starting point for further development, and it might need adjustments in terms of `defines, parameters, clocks, and resets.

NOTE: Although this command can jump-start the setup process, you must have a sign-off process because this feature can, in some cases, create unintended side-effects, including the following:
. Incorrect files analyzed
. Erroneous elaboration switches

-vhdl |-verilog |-v2k |-v95 |-sv
    Tailor the setup script for the specified language.

    NOTE: This feature does not support mixed-language designs.

-path path
    Use the specified path when locating files for automatic setup.

```
    If you do not specify a path, the tool uses the current path.

-suffix suffix
    Filter the file set according to the specified suffix.

    If you do not specify a suffix, the tool does not filter by suffix.

-top_mod module_name
    Use the specified module as the top module.

    If you do not specify a module, the tool attempts to determine it
    automatically.

-top_file file_name
    Locate the top module in the specified file.

    If you do not specify a file, the tool attempts to determine it
    automatically.

-save -file file_name [-force]
    In addition to generating the setup script and printing it to the screen,
    save it to the specified file.

    Use "-force" if the specified setup file already exists and you want to
    overwrite it.
```

Example

```
auto_setup -sv -suffix v -path ./design/rtl -top_file my_design.v \
             -top_mod my_design -save -file setup.tcl -force
```

autoprove

Orchestrates a dynamic approach to verification based on the "prove" command.

Syntax

```
autoprove
     ( -all
      |-task [task_name_list]
      |-property {property_name_list} [-regexp]
     )
      [-bg]
      [-with_helpers] [-with_proven]
      [-time_limit time_budget]
      [-mem_limit memory_budget]
      [-chunks N]
      [-mode (default | prove | cover | hard | superlint | coverage | xprop)]
      [-effort (low | medium | high | (user [-engines engine_list]))]
      [-asserts]
      [-covers]
      [-suppress_traces]
      [-keep_traces]
      [-assumption_lifting]
      [-dump_vcd]
      [-dump_trace
        [-dump_trace_type (shm | vcd | fsdb)]
        [-dump_trace_dir dir_name]]
      [-verbosity N]
      [-silent]
```

Returns

For autoprove return values, see the "prove" command.

Detailed Description

Autoprove is an automatic engine orchestration command that attempts to increase the verification results using a given set of resources, both time and

licenses. It is a hybrid command in the sense that it calls the basic "prove" commands several times during the execution while dynamically deciding on the execution parameters, engines, and order of execution.

-all
    Prove all assertions and cover properties in all tasks.

-task [task_name_list]
    Prove all assertions and cover properties in the specified tasks.

    NOTE:
    . If you do not supply a task name, the tool proves all assertions and
      covers in the current task.
    . When specifying a list of task names, enclose the list in curly braces.
    . This switch supports wildcards (* and ?) in task names.

-property property_name_list [-regexp]
    Prove the specified properties.

    NOTE:
    . When specifying a list of properties, enclose the list in curly braces.
    . To prove properties not in the current task prepend the property
      names with "<task>::".
    . All listed properties must belong to the same task.
    . This switch supports wildcards (* and ?) and regular expressions in
      property names, for example:
      autoprove -property top.ing?.i_ingress.c_read
      autoprove -property top.ing\[1-3\].i_ingress.c_valid -regexp

-bg
    Run the "autoprove" command in the background and continue to accept
    and run new commands.

-with_helpers
    Prove with helper assertions.

    You can prioritize helper assertions over regular assertions by submitting
    them first to the engines with "assert -helper...".

-with_proven
    Use all proven assertions as assumptions regardless of the value of
    proven_directive.

```
-time_limit time_budget
    Override the default two hours "prove_time_limit" for "autoprove".

    Specify "time_budget" values as an integer followed by s, m, or h (no
    spaces).
    . s = seconds
    . m = minutes
    . h = hours

-mem_limit memory_budget
    Override the default 2048MB "engineCG_max_mem" for "autoprove".

    Specify "memory_budget" values as an integer followed by m or g (no
    spaces).
    . m = megabytes
    . g = gigabytes

-chunks N
    Split the given job into N subjobs.

    This option is useful for big designs. By default, the tool issues one
    chunk.

    NOTE:
    . When using chunks, an unlimited "prove_time_limit" is not supported.
    . "-chunks" is not supported in combination with "-mode superlint" or
      "-mode hard".

-mode (default | prove | cover | hard | superlint | coverage | xprop)
    Focus "autoprove" on the specified mode:
    . prove    - Assertions
    . cover    - Covers
    . hard     - Hard-to-prove/cover properties. For this argument, the tool
                 must provide an adequate "time_budget".
    . superlint      - Properties generated by the Superlint App
    . coverage - Properties generated by the Coverage App
    . xprop    - Properties generated by the X-Prop App

    NOTE: "-mode superlint" and "-mode coverage" suppress trace generation by
    default. "superlint" completely suppresses trace generation. "coverage" partly
    suppresses trace generation. Use "-keep_traces" if you want the tool to
    generate traces for these modes.
```

```
-effort (low | medium | high | (user [-engines engine_list]))
    Override the number of engines used by "autoprove", which is translated
    to the amount of load "autoprove" puts on the system.

    Levels: The actual number of engines the following levels use is
    app-specific.
    . low    - This level is useful for a local run with limited resources.
    . medium - Use the default "autoprove" engine set.
    . high   - Use all suitable engines. This level is very stressful. It
               consumes considerable resources.
    . user   - Override the engines used by "autoprove".
               Follow "user" with "-engines" and engine_list, for example,

               autoprove -effort user -engines {B D}

               NOTE:
               . If you do not use "-engines" and engine_list, "autoprove" uses
                 the current engine_mode set.
               . "autoprove" uses the new engine_list (or engine_mode)
                 through all phases.

-asserts
    Prove only assertions, not covers. This switch is an alias for operation
    mode "prove".

-covers
    Prove only covers, not assertions. This switch is an alias for operation
    mode "cover".

-suppress_traces
    Suppress trace generation.

-keep_traces
    Disable trace suppression in all modes.

    NOTE: "-mode superlint" and "-mode coverage" suppress trace generation by
    default. "superlint" completely suppresses trace generation. "coverage" partly
    suppresses trace generation. Use "-keep_traces" if you want the tool to
    generate traces for these modes.

-assumption_lifting
    Only include a relevant subset of assumptions that is sufficient for the
    prove.
```

```
    NOTE: This option might be useful in cases with too many assumptions.

-dump_vcd
    Dump a VCD file for each Covered/Cex property found.

    The VCD files are saved in the "traces" directory, in the current
    Jasper project directory.

    NOTE:
    . Any previously existing "traces" directory is renamed "traces.bak".
    . Any previously existing "traces.bak" directory is deleted.

-dump_trace
    [-dump_trace_type (shm | fsdb | vcd)]
    [-dump_trace_dir dir]
    Dump traces to the disk as soon as the engines find them. The tool uses
    the SHM format as the default and writes files in the project directory.
    The trace has values for all signals in the analysis region of the proof
    target.
    . Use "-dump_trace_type" with "shm", "fsdb", or "vcd" to
      specify the file dump type. VCD files are dumped as compressed files to
      save disk space.

      IMPORTANT:
      . Jasper Apps writes 6.3 FSDB traces and reads version 6.6 FSDB
        traces. FSDB Reader 6.6 was introduced by Verdi version X-2025.06.
      . FSDBs you generate with the latest Jasper are backwards compatible
        with all Jasper versions from 2025.03 FCS and newer. In addition,
        they are backwards compatible with older versions of Verdi released
        during that same period.
      . Jasper Apps supports backwards compatibility, that is, the tool can
        read FSDBs generated from older versions of Jasper Apps and
        Synopsys Verdi.

   . Use "-dump_trace_dir" to specify the dump directory where the file will
     be written.

-verbosity N
    Override the default "prove_verbosity" level, which is 6.

    This switch controls the amount of information printed to the log, the
    console, and the proof messages tab. 0 means "silent". The higher the
```

```
    value, the more output.

-silent
    Alias for verbosity level 0.
```

Example

```
autoprove -task taskA
autoprove -time_limit 1h
autoprove -mem_limit 4g
autoprove -task { taskA taskB taskC }
autoprove -property { propertyA propertyB propertyC }

# Setting up a session and proving with autoprove
# ------------------------------------------------
analyze...
elaborate...
clock...
reset...
task -create task1
assert {##5 ff0.dout} -task task1
assert {##5 ff1.dout} -task task1

task -create task2
assert {##5 ff2.dout} -task task2
assert {##10 ff2.dout} -task task2

#set engine_list {Ht Hp K L M N J D I}
set engine_list {Hp J}
autoprove -all -effort user -engines $engine_list
```

See Also

get_status,
prove,
set_engine_mode,
set_max_trace_length,
set_proof_simplification,
set_proofgrid_per_engine_max_jobs,
set_proofgrid_per_engine_max_local_jobs,
set_proofgrid_mode,
set_prove_per_property_time_limit,
set_prove_time_limit,
set_proven_directive,
visualize

blackbox_assistant

Generates a list of modules and instances not required for the verification.

Syntax

```
blackbox_assistant -config
                   (-connectivity_map csv_file_tcl_list
                    | -max_depth N)
                   [-export export_file_name]

blackbox_assistant -clear
```

Returns

No return values.

Detailed Description

Use this command to black box modules and instances not required for the verification. The goal is to reduce the design complexity for elaboration or proof.

NOTE: "blackbox_assistant", which helps find blackbox candidates to improve performance in Connectivity proofs, is not compatible with LPV.

NOTE: "blackbox_assistant" detects and preserves the signal paths between source, destination, and the conditions described in the connection. When a connection latency requires detecting and preserving clock paths, Blackbox Assistant should not be used.

NOTE: "blackbox_assistant" is not supported by CDC, COV, FSV, SUPERLINT, UNR, SEC, SPV, X-Prop, or C2RTL.

```
-config
        (-connectivity_map csv_file_tcl_list
         | -max_depth N)
        [-export export_file_name]
```

```
Configure "blackbox_assistant" with the provided options.
When configured, subsequent "elaborate" commands automatically
trigger "blackbox_assistant", which black boxes the modules
and instances not required for the verification without additional
user intervention.
NOTE: Use this before the "elaborate" command.

Configuration options
---------------
-connectivity_map csv_file_tcl_list
  Use the provided list of connectivity maps to compute the black boxes.
  With this option, all instances not in the paths required to
  verify the connections are black boxed.
  For additional information about the syntax of the connectivity map,
  see the Connectivity App User Guide.

  NOTE: "blackbox_assistant" supports undefined modules only when the
  following assumptions are valid for the module instantiation:
  . There is no hierarchical reference or implicit declaration.
  . The undefined module is a Verilog module instantiated by named port
    connection, and there is no complex expression other than full nets
    or bit slices.

  Example:
    module top;
        wire w1, w2;
        undefMod unk(.inA(w1), .inB(w2));
    endmodule

-max_depth N
    Use this switch to black box all instances bellow the specified depth.
    If the specified depth is "0", for example, all instances in the "1"
    depth level will be black boxed.
    NOTE: "0" is the depth of the top instance, "1" the depth of its children,
    and so forth.

Other switches
---------------
. Use "-export" to specify a file that will contain the results of the
  analysis, in a format that is compatible with "elaborate -f".
  By exporting the results to a file, it is possible to reuse them
  in future runs without having to rerun the "blackbox_assistant" analysis.
```

```
      NOTE: Reuse the file only if you know that the design or inputs have
      not changed since it was generated.
      NOTE: The results are still automatically black boxed during elaboration.


-clear
    Clear the results and configurations stored by "blackbox_assistant" so that
    they do not affect the next calls to "elaborate".
```

Example

```
blackbox_assistant -config -connectivity_map conn.csv -export [get_proj_dir]/results.f

# Triggers "blackbox_assistant" using "conn.csv", and automatically
# blackboxes the modules/instances not required:

elaborate
```

capture_testcase

Saves a tarred file of all sourced scripts and design and reset files.

Syntax

```
capture_testcase testcase
    [-include_logs] [-include_files file_name_list]
    [-exclude_env][-force]
```

Returns

The name of the generated archive.

Detailed Description

Run this command at any point after you analyze and elaborate a design to save a testcase as a compressed file that contains all Tcl scripts run with the "source" and "include" commands and all referenced design and property files. If you used a reset file during setup, this command includes it in the compressed file. You will generally use this command to capture a testcase to share with the Cadence representative supporting you.

NOTE:
. The "captured.env" file, which is at the top of the directory, records the PWD at the time "capture_testcase" is run. This information might be helpful for finding the directory where scripts were launched.
. This command does not add symbolic link targets to the testcase. Instead, it saves the links as files with the contents of their respective targets.
. The tool attempts to use "gzip" to create a compressed file, and if it does not find it, "bzip2". If it cannot find either utility, the files remain uncompressed and the tool prints a warning.

Implementation guidance: "testcase" can be a full or relative path to the file you are saving.

-include_logs
    Add all the files in the current project directory to the set of files

```
    captured by this command.

    This switch is helpful when you have used the GUI to run the tool rather
    than scripts alone.

-include_files {file_name_list}
    Add the specified files to the set of files captured by this command.

    Important things to know:
    . Use this switch to include signal files, screen shots, README files, and
      so forth.
    . Surround file lists with curly braces.
    . This switch accepts a list of files and directories. For any directories
      you specify, the tool includes all files within that directory and all of
      its sub-directories.
    . Symbolic links are recursively replaced with the contents of the file
      they point to until the tool finds a regular file.
    . Broken or circular links are not included.
    . Use the Tcl "glob" command to specify a list of files to include using
      wildcards (see "Example" section).

-exclude_env
    Do not save a "captured.env" file; that is, do not capture environment
    variables.

    NOTE: Using this switch can, in some cases, hinder our ability to run
    the captured testcase.

-force
    If the specified testcase name already exists, overwrite it.
```

Example

```
capture_testcase -include_files {fileA ../fileB ../dirC/fileC}

// Launch the tool and use scripts to run your session.
capture_testcase testcase1
Testcase tar file: testcase1.tgz
Files captured: 7

// Launch the tool and use the GUI or a combination of GUI and scripts to run
// your session.
capture_testcase -include_logs testcase2
Testcase tar file: testcase2.tgz
Files captured: 19

// Capture all files within "../my_scripts/" and any of its sub-directories,
// and include any files matching the pattern "../../*.txt".
capture_testcase my_capture -include_files "../my_scripts/ [glob ../..*.txt]"
```

cfe_debug

Provides the ability to debug C/C++ code using GDB.

Associated App(s)

C2RTL

Syntax

```
Setup
-----
cfe_debug -config
    [-compilation_command compilation_command]
    [-gdb_path path_to_gdb]
    [-terminal_binary_path path_to_terminal]
    [-executable_path path_to_executable]

cfe_debug -config -show

Compilation
-----
cfe_debug -compile
    [-compilation_command compilation_command]
    [-compile_systemc]
    [-include_undriven]

Open
-----
cfe_debug -open
    (-visualize_window visualize_window |-property property_name)
    -cycle cycle_number
    [(-compile |-use_executable)]
    [-compile_systemc]
    [-include_undriven]

Export Trace For Gdb
--------------------
cfe_debug -export_trace_for_gdb
    -cycle N
    [(-property property_name)
     |(-visualize_window visualize_window_name)]
```

Returns

Various, context sensitive return values.

Detailed Description

```
This command integrates GDB debugging into C2RTL.

---------
Configurations
---------

This command uses the following Jasper configurations:

. gdb_path: Provides the path to GDB. Defaults to /usr/bin/gdb.
    - Can be set using "set_gdb_path {}"
    - Can be retrieved with "get_gdb_path"
. terminal_binary_path: Provides the path to the terminal where GDB opens.
Defaults to /usr/bin/xterm.
    - Can be set using "set_terminal_binary_path {}"
    - Can be retrieved using "get_terminal_binary_path"
. compilation_command: Provides the command to compile the C/C++ code.
No default value.
    - Can be set using "set_compilation_command {}"
    - Can be retrieved using "get_compilation_command"
. executable_path: Provides the path to a pre-compiled executable.
No default value.
    - Can be set using "set_executable_path {}"
    - Can be retrieved using "get_executable_path"


-config
-----
cfe_debug -config
    [-compilation_command compilation_command]
    [-gdb_path path_to_gdb]
    [-terminal_binary_path path_to_terminal]
    [-executable_path path_to_executable]

      Use this sub-command to set the values of the four configurations
      used by this command.

-config -show
-----
cfe_debug -config -show
    Use this sub-command to show the current values of all four configurations.


-compile
```

```
-----
cfe_debug -compile [-compilation_command compilation_command]
    [-compile_systemc]
    [-include_undriven]

    . Use "-compilation_command" to use the value specified instead of the value
      in the configuration. This command sets the value of the "executable_path".
    . Use "-compile_systemc" to compile SystemC datatype classes with the
      source code.
    . Use "include_undriven" to compile an executable that can inject
      undriven signals from Jasper buffer macros.

-open
-----
cfe_debug -open
        [(-visualize_window window_name |-property property_name)]
        -cycle cycle_number
        [(-compile |-use_executable)]
        [-compile_systemc]
        [-include_undriven]

    Use this sub-command to open a terminal with GDB opened.

    . Use only one of "-visualize_window" or "-property".
      If a "-visualize_window" is used, GDB will be opened with
      a trace for the property in the Visualize window.
    . Use "-cycle" to specify the C/C++ cycle being debugged in the code.
    . Use only one of "-compile" or "-use_executable" to select the mode of
      operation. The default is "-use_executable". If "-compile" is present, the
      source code will be compiled with the command present in
      the "set_compilation_command" configuration.
    . Use "-compile_systemc" to compile SystemC datatype classes with the
      source code.
    . Use "-include_undriven" to add undriven signals from Jasper buffer
      macros in the injected trace."

-export_trace_for_gdb
---------------------
cfe_debug -export_trace_for_gdb (-cycle N)
       [(-property property_name)
        | (-visualize_window visualize_window_name)]

    Retrieve trace information for the specified property and cycle.
```

```
Output of this command is stored as a CSV file, and the tool
message specifies the location of the trace file.
. Use "-property" to specify the property for exporting input values.
. Use "-visualize_window" to specify the Visualize window for exporting
  input values.
  NOTE: You can only use "-property" or "-visualize_window."
. Use "-cycle" to specify the trace cycle for exporting input values.
```

Example

```
cfe_debug -config -compilation_command "gcc main.c -o main" -exe "main" -terminal
"xterm" -gdb "gdb"
cfe_debug -config -show
cfe_debug -compile
cfe_debug -compile -compilation_command "gcc main2.c -o main"
cfe_debug -open -property main.cpp_UserAssert_1 -cycle 1
cfe_debug -open -visualize_window visualize:0 -cycle 1
cfe_debug -open -property main.cpp_UserAssert_1 -cycle 1 -compile
cfe_debug -open -property main.cpp_UserAssert_1 -cycle 1 -use_executable
```

check_afv

Prints requested Architectural Modeling info or executes requested action.

Associated App(s)

Architectural Formal Verification

Syntax

```
check_afv -table table_name
          -list (for | state | in | wire | tag)

check_afv [-table table_name]
          -list (property | assume | assert | cover)

check_afv -list (table | param | lparam | input | step)

check_afv -load (file_name.xlsx | file_name.xml)
          [-parameter param_name param_value]
          [-define macro_names]*
          [-analyze_opts analyze_options_list]
          [-elaborate_opts elaborate_options_list]

check_afv -bind (file_name.xlsx | file_name.xml)
          [instance_to_bind]
          [-define macro_name]*
          [-analyze_opts analyze_options_list]
          [-elaborate_opts elaborate_options_list]

check_afv -export -to_sva sva_file.sv
          [-to_tcl tcl_file.tcl]

check_afv -prove [-table table_name]

check_afv -clear
```

Returns

A list of requested objects or output of requested action.

Detailed Description

Use this command to get information from an XLSX/XML worksheet file that defines a module in Architectural Modeling format or to request actions related to this module.

This command has two scopes, the loaded worksheet and a specified table. The default is the loaded worksheet. To specify a table, use the "-table" switch.

List Switches

-table table_name
    -list (for | state | in | wire | tag)
    List the information that matches the specified criteria.

    NOTE: The following switches are valid for the "-table" scope.

    . Use "for" to list the FOR cell ranges defined in the specified table
      with their index variables.
    . Use "state" to list STATE signals defined in the specified table.
    . Use "in" to list IN signals defined in the specified table.
    . Use "wire" to list WIRE signals defined in the specified table.
    . Use "tag" to list all tags specified for the specified table. Valid tags
      are COVER, FULL, NEVER, NEVER_INIT, PRLL, RLX_PRLL.

[-table table_name]
    -list (property | assume | assert | cover)
    List the information that matches the specified criteria.

    NOTE: The following switches are valid for both the default and "-table"
    scopes.

    . Use "property" to list all the properties generated by the loaded
      Architectural Model worksheets.
    . Use "assume" to list all the assumptions generated by the loaded
      Architectural Model worksheets.
    . Use "assert" to list all the assertions generated by the loaded
      Architectural Model worksheets.

```
    . Use "cover" to list all the covers generated by the loaded
      Architectural Model worksheets.

-list (table | param | lparam | input | step)
    List the information that matches the specified criteria.

    NOTE: The following switches are valid for the worksheet scope. They do not
    support the "-table" switch.

    . Use "table" to list all tables defined in the loaded Architectural Model
      worksheets.
    . Use "param" to list all the loaded Architectural Model worksheet's
      parameters and their values.
    . Use "lparam" to list all the loaded Architectural Model worksheet's local
      parameters and their values.
    . Use "input" to list all the loaded Architectural Model worksheet's
      inputs.
    . Use "step" to list all the loaded Architectural Model worksheet's step
      tags.

Action Switches
---------------
-load (file_name.xlsx | file_name.xml)
           [-parameter param_name param_value]
           [-define macro_name]*
           [-analyze_opts analyze_options_list]
           [-elaborate_opts elaborate_options_list]
    Load the module generated by the specified XLSX/XML worksheet file as the top
module.

    . Use "-parameter" to specify the parameter values for the top-level
      module.
    . Use "-define" to specify any macro definitions for the IFNDEF keyword.
      Specify a list of macros or reuse switch to specify multiple macros.
    . Use "-analyze_opts" to pass the specified arguments to the "analyze"
      command.
    . Use "elaborate_opts" to pass the specified arguments to the "elaborate"
      command.

    NOTE: This command also loads the GUI, and it is not valid for the table
    scope.

-bind (file_name.xlsx | file_name.xml)
```

```
            [instance_to_bind]
            [-define macro_name]*
            [-analyze_opts analyze_options_list]
            [-elaborate_opts elaborate_options_list]
    Binds the module generated by the specified XLSX/XML worksheet file.

    . To specify a name for the instance you are binding, use instance_to_bind;
      otherwise, the instance name will be the same as the module name.
    . Use "-define" to specify any macro definitions for the IFNDEF keyword.
      Specify a list of macros or reuse switch to specify multiple macros.
    . Use "-analyze_opts" to pass the specified arguments to the "analyze"
      command.
    . Use "elaborate_opts" to pass the specified arguments to the "elaborate"
      command.

    NOTE:
    . This switch also loads the GUI.
    . This switch is not valid for the table scope.
    . The module connection is handled as a requirement module.

-export -to_sva sva_file.sv [-to_tcl file_name.tcl]
    Export the properties generated by AFV from the loaded model.

    Use the "-to_tcl" switch to generate a Tcl file, which contains commands
    for loading the generated SVA properties.

    NOTE: Properties dependent on the NEXT keyword are not supported.

-prove [-table table_name]
    Prove the properties generated by the loaded Architectural Model worksheet.

    Use the "-table" scope to prove only properties related to the specified
    table.

    NOTE: The "set_cmd_time_limit" command has no effect on "-prove"; instead,
    it honors prove time-limit commands.

-clear
    Clear the Architectural Model worksheet.

    NOTE: You must manually clear the properties that were created.
```

Example

```
check_afv -table table1 -list for
check_afv -list table
check_afv -table table1 -list assert
check_afv -list assert
check_afv -load ace.xml
check_afv -load ace.xml -define DEBUG -define disable_deadlock
check_afv -load ace.xml -define {DEBUG disable_deadlock}
check_afv -bind ace.xml
check_afv -bind ace.xml ace_2
check_afv -table table1 -prove
check_afv -prove

# Example session
# ---------------
# Load the Archtectural Model
check_afv -load L1L2.xml

# Limit the proof time
set_prove_per_property_time_limit 10s
set_prove_time_limit 30s

# Prove the Arch Model
check_afv -prove

# Report results
report

# Visualize parts of the Arch Model
visualize -violation -property {<embedded>::L1L2.AST_table_L1_CMP_full} -new_window
```

check_assumptions

Checks whether assumptions overconstrain the design and might cause vacuity.

Syntax

```
check_assumptions [-all |-task task_name_tcl_list]
                  [-conflict]
                  [-dead_end]
                  [-live]
                  [-deadlock]
                  [-sanity]
                  [-bg]
                  [-force]
                  [-max_length N]
                  [-time_limit time_limit]
                  [-verbosity N |-silent]
                  [-from property_name [-trace_id N] [-cycle N]]
                  [-simplification_only]]

check_assumptions -minimize
                  [-assumes assumption_tcl_list]
                  [-include_env_assumes]
                  [-include_clock]
                  [-include_reset]
                  [-required_assumes assumption_tcl_list]
                  [-attempts number_of_prove_attempts]
                  [-attempts_time_limit time_limit]
                  [-from_reset]
                  [-all |-task task_name_tcl_list]
                  [-conflict]
                  [-dead_end]
                  [-live]
                  [-deadlock]
                  [-sanity]
                  [-force]
                  [-max_length N]
                  [-time_limit time_limit]
                  [-verbosity N |-silent]
                  [-from property_name [-trace_id N] [-cycle N]]
                  [-simplification_only]

check_assumptions -trace_id N

check_assumptions -show [-all |-task task_name_tcl_list] [-dead_end] [-live] [-
deadlock]
```

Returns

Return Value	Definition
conflict	There is an unavoidable finite conflict from reset at a given cycle. All "proven" and "unreachable" results are vacuous, and the property table displays a red exclamation point for them.
no_dead_end	There are no dead end states. This implies that there also are no conflicts.
dead_end	There is a reachable finite conflict - a dead end state.
no_conflict	There are no unavoidable finite conflicts from reset, and it is unknown whether there are dead end states.
unfair	There is an unavoidable infinite conflict involving fairness constraints from reset. All "proven" and "unreachable" results for liveness or infinite properties are vacuous.
fair	There are no unavoidable infinite or finite conflicts from reset, and it is unknown whether there are dead end states.
sanity	There are no unavoidable finite conflicts from reset up to a given min_length bound.
no_deadlock	There are no deadlock states. All fairness constraints can be satisfied from all reachable states.
deadlock	There is a reachable infinite conflict - a deadlock state. A fairness constraint cannot be satisfied from a reachable state.
undetermined	Not even a sanity trace is available.
background	The command is running in the background.
background thread_id	The command is running in the background and is associated with the specified "thread_id".
failed	The command stopped due to an error.

Detailed Description

Use this command to prove the "check_assumptions" properties, which are special properties built into every task to check the effects of assumptions. The properties ":noConflict", ":noDeadEnd", ":live", and ":noDeadlock" will be shown in your task when you run this command. For example, you can see them displayed in the Property Table. The property types are "Assert(noConflict)", "Assert(noDeadEnd)", and "Cover(infinite)". The following list defines the properties and explains what you can learn from the proof results they produce on your design.

. :noConflict This property asserts that the assumptions and proof setup should allow infinite prefix traces. The property will have one of the following statuses:

    . cex A violation trace will illustrate that at a certain cycle (the last in the trace) it is impossible to satisfy all assumptions. We call this an unavoidable "conflict". All "proven" and "unreachable" results are vacuous and the property table displays a red exclamation point for them.

    . undetermined A sanity trace is likely available. See "-sanity". It will illustrate that there are no conflicts this short. A longer and longer sanity trace will become available during proof. The bounds show how many cycles are conflict free.

    . proven A looping sanity trace is available. It will illustrate that there are infinite traces; that is, there is "no_conflict".

      NOTE: Since :noConflict ignores liveness assumptions, an infinite trace might not actually exist due to an overconstraint by the liveness assumptions. The "no_conflict" property only checks for conflicts derived by safety or finite properties. See :live below.

. :noDeadEnd This property asserts that every reachable state has a next cycle satisfying all assumptions. The property will have one the following statuses:

    . cex A violation trace will illustrate that at a certain cycle (the last in the trace) it is impossible to satisfy all assumptions, given the preceding state. We call this a "dead_end".

```
    . undetermined The bounds show how many cycles are dead end free.

    . proven       There is no dead end. The bound is infinite.

. :live        This property covers a situation where all fairness constraints
               are satisfied infinitely often.

    . unreachable  It is impossible to satisfy all fairness constraints
                   infinitely often. All "proven" and "unreachable" results for
                   liveness or infinite properties are vacuous. Use "-minimize"
                   to narrow down the conflict.

    . covered      A looping cover trace illustrates that all constraints,
                   including fairness, are satisfied.

. :noDeadlock  This property states that it is possible to satisfy all
               fairness constraints from every reachable state. The property
               will have one the following statuses:

    . cex          A violation trace illustrates that from a certain cycle
                   (the last in the trace), it is impossible to satisfy all
                   liveness assumptions. This is known as a "deadlock".

    . undetermined No information available.

    . proven       There is no deadlock.

This feature helps you understand the effects assumptions have on the design.
When a cex is found, it warns you about conditions that might indicate
that the assumptions are stronger than intended and provides a trace for
inspection. In effect, "check_assumptions" gives you confidence that you have
verified what you intended.

NOTE:
. Sequential assumptions might in themselves introduce benign dead ends that
  are hard to avoid, so a dead end does not necessarily indicate a problem in
  itself.
. The command uses a custom engine_mode, but beyond that, it respects all
  ProofGrid and "prove" settings. See "help prove -gui". In particular, if you
  want minimal dead_end or conflict traces, see "help set_prove_prefer_shortest
  -gui".
. Potentially unstable loops, for instance through latches, can contribute to
```

```
  conflicts, and the tool checks them exactly when they are in the cone of
  influence (COI) of the properties in the task. This means that when you add
  or remove properties such that the set of latches in the COI might have
  changed, we recommend that you rerun "check_assumptions".
. The command might, as a side effect, prove other properties as well.
. By default, if you do not specify "-conflict", "-dead_end", "-live" or
  "deadlock" the tool checks only conflict.

Implementation Guidance
-----------------------
It is always recommended that you run the "check_assumptions" command at least
once when the proof environment is finalized with all the constraints to detect
any unexpected overconstraint problem.

NOTE: Assumptions might be stronger than you intended even if they do not
cause any conflicts. Assumptions still need to be validated by other means,
that is, assume-guarantee.

[-all |-task task_name_tcl_list]
    Prove the "check_assumptions" assertions in "-all" tasks or in the tasks
    specified with "task task_name_tcl_list".

    NOTE:
    . If you do not use "-all" or "-task", the command applies to the current
      task.
    . When specifying a list of task names, enclose the list in curly braces.
    . The "-task" switch supports wildcards (* and ?).

[-conflict] [-dead_end] [-live] [-deadlock] [-sanity]
    Check the specified property types.

    NOTE: "check_assumptions" with no optional switches checks the current task
    for conflicts only (:noConflict).

    . Use no switch or only "-conflict" to run the engines focused on checking
      for unavoidable finite conflicts from reset (:noConflict).
    . Use "-dead_end" to run the engines focused on checking for reachable
      finite conflicts (:noDeadEnd). A dead end is a reachable scenario that
      cannot be extended due to unavoidable conflicts of one or more
      assumptions with another or conflicts between some assumptions and the
      design.
    . Use "-live" to run the engines focused on checking for unavoidable
      infinite conflicts from reset (:live). A live conflict suggests that all
```

of your pure liveness or infinite properties will be vacuously proven (mixed safety or finite and liveness or infinite might still fail).

. use "-deadlock" to check if it is possible to reach a state from which all fairness constraints cannot be satisfied. If you show the property first, you can then use "assert -set_store_trace unlimited :noDeadlock" to instruct the tool to generate and store multiple deadlock traces. This is useful to get a trace fore each deadlocking fairness constraint.

. With the exception of "-dead_end" and "-deadlock", which are mutually exclusive, you can combine the switches to check multiple properties in the same proof.

. Use "-sanity" if you want a longer and longer sanity trace to become available during the proof.

When you use the switches "-conflict", "-dead_end", "-live" and "deadlock", the proof is committed to determining the status of the corresponding properties (:noConflict, :noDeadEnd, :live and :noDeadlock), and it runs until it reaches the goal unless the proof prematurely stops, for example, due to a time out. Meanwhile, the proof might also find incidental results for any property in the task as a side effect (including, but not limited to, the other conflict properties).

Some engines focused on dead end are affected by engineCG_max_mem, see "set_engineCG_max_mem". Use verbosity > 7 to see progress during their initial transformation phase.

-bg

Run the proof in the background and continue to accept and run new commands. See "prove" for operations available on a proof running in the background.

NOTE: "-bg" and "-minimize" are mutually exclusive.

-force

Clear existing proof results for the selected properties before beginning. See "prove -force".

-max_length N

Override the current "max_trace_length" for this command. See "set_max_trace_length".

-time_limit time_limit

Override the current "prove_time_limit" for this command. See "set_prove_time_limit".

```
-verbosity N
    Override the current "prove_verbosity" for this command. See
    "set_prove_verbosity".

-silent
    Alias for -verbosity 0.

-from property_name [-trace_id N] [-cycle N]
    Use the state at cycle N from the existing trace for the specified property
    as the reset value for this proof.

    Use "-trace_id" to specify a trace other than the default. The default property
    trace is the last shortest trace. See "assert/cover -set_store_trace" for the
    commands to configure the number of traces generated and stored by the tool.

    If you do not specify a cycle, or if you specify 0, the tool uses the cycle in
    which the property was triggered. This is often, but not always, the last
    cycle of the trace. See "set_trace_extension".

    If you specify $, or if you specify -1, the tool uses the last cycle of the
    existing trace. A negative N selects the cycle relative to the end of the
    trace.

    NOTE:
    . This is a trace-hunting feature. The tool does not generate :noConflict
      cex results because all paths from reset are not examined when you use
      "-from". The command might, however, generate both bounds and proven
      results for :noConflict. A conflict from the selected start state is
      reported as a dead end.
    . Found traces will start from reset and include the first N cycles from
      the selected trace as a prefix.
    . "check_assumptions -from" is included as an initial release to gather
      feedback from early adopters and finalize implementation for an upcoming
      release. Contact support@cadence.com to provide feedback or comments.

-simplification_only
    Run only preprocessing and simplification without invoking the regular
    check assumptions (Q) engines.

    NOTE: This can be useful when proof_simplification is able to determine the
    property without providing a trace, and the Q engines do not converge.
```

```
-minimize [-from_reset]
    [-assumes assumption_tcl_list] [-include_env_assumes]
    [-include_clock] [-include_reset] [-required_assumes assumption_tcl_list]
    [-attempts number_of_prove_attempts] [-attempts_time_limit time_limit]

    [-all |-task task_name_tcl_list] [-conflict] [-dead_end] [-live] [-deadlock]
    [-sanity] [-force] [-max_length N] [-time_limit time_limit] [-verbosity N
    |-silent] [-from property_name [-trace_id N] [-cycle N]]
    [-simplification_only]

    If the tool has established, or can establish, a conflict or dead_end or
    unreachable :live or a deadlock, run "get_needed_assumptions" to provide a
    minimized debugging environment for the result.

    By default, for conflicts, dead_ends and deadlocks, only the last cycle (the
    cycle that violates some constraints) is minimized. The rest of the existing
    trace is retained as is. Use "-from_reset" to minimize all cycles from
    reset. Minimizing only the last cycle has some noteworthy consequences:

    . The cost of "-minimize" is significantly reduced, especially when the
      conflict is deep. Minimizing deep conflicts from reset has a high risk of
      not converging.

    . For :noConflict, the set of needed assumptions might be smaller than the
      set of needed assumptions from reset. For example, the result might
      exclude a bounded assumption (see "assume -bound") that constrains the
      prefix trace in a way that contributes to the conflict but does not
      constrain the last cycle. Because the result is not guaranteed to be a
      :noConflict cex from reset, the minimized conflict trace appears as a
      :noDeadEnd cex.

    . For :noDeadEnd, minimizing only the last cycle is often more useful
      because when minimizing from reset, the end result can be a different,
      less relevant dead_end caused by disabling some assumption that should
      not have been disabled.

    . For :noDeadlock, minimizing only the last cycle is often more useful
      because when minimizing from reset, the end result can be a different,
      less relevant deadlock caused by disabling some assumption that should
      not have been disabled.

    NOTE:
```

```
    . See "get_needed_assumptions" for a description of the first set of
      switches.
    . See "check_assumptions", the base command, for a description of the last
      set of switches.
    . "get_needed_assumptions" runs with "-include_liveness_assumes". Beginning
      with version 2020.06, it is no longer necessary to specify the
      "-include_liveness_assumes" switch with "check_assumptions -minimize".
    . "-include_env_assumes", "-include_clock", and "-include_reset" implies
      "-from_reset".

-trace_id N
    Display the specified trace.

    NOTE: See "assert/cover -set_store_trace" for the commands to configure the
    number of traces generated and stored by the tool.

-show [-all |-task task_name_tcl_list] [-dead_end] [-live] [-deadlock]
    Show the "check_assumptions" properties in the current or specified tasks.

    By default, the tool only shows conflict. Use "-dead_end" to show the
    dead_end property, "-live" to show the live property and "-deadlock" to show
    the deadlock property.
```

Example

```
check_assumptions -task <embedded>

# Example session
# ---------------
{module a (input clk, a, b);
    c1: assume property (@(posedge clk) a s_until b);
    c2: assume property (@(posedge clk) !a s_until b);
endmodule}

analyze -sv09; elaborate

reset -none; clock clk

assume -bound 1 !b
INFO (IPM005): The name "<embedded>::assume:0" is assigned to assumption "!b".
assume:0
```

```
check_assumptions -minimize
INFO (IPF036): Starting proof on task: "<embedded>", 1 properties to prove with 0
already proven/unreachable
INFO (IRS029): Starting reset analysis: phase 1 of 4.
...
INFO (IRS039): Reset analysis complete.
INFO (IPF031): Settings used for proof thread 0:
    orchestration                = off
    time_limit                   = 86400s
    per_property_time_limit      = 1s * 10 ^ scan
    engine_mode                  = Q1 Q2 Q3
    proofgrid_per_engine_max_jobs = 1
    max engine jobs              = Q1 Q2 Q3, total 3
    proofgrid_mode               = local
    proofgrid_restarts           = 10
INFO (IPF055): 0.0.PRE: A counterexample (cex) with 1 cycles was found for the property
":noConflict" in 0.00 s.
INFO (IPF051): 0.0.PRE: The cover property ":live" was proven unreachable in 0.00 s.
use check_assumptions -show -live to show this property in the property table
INFO (IPF057): 0.0.PRE: The property ":noDeadEnd" was proven in 0.00 s.
use check_assumptions -show -dead_end to show this property in the property table
WARNING (WAS006): The task is inconsistent at cycle 1.
INFO (IPF059): Completed proof on task: "<embedded>" for proof thread 0
minimize <embedded>:conflict:
Using target status cex
Settings used for this command:
    Original task                 = <embedded>
    Property                      = :noConflict
    Target status                 = cex
    Prove attempts time limit     = 120s
    Prove attempts max trace length = 1
    - Including liveness assumes

Determined: assume:0 , remaining 2 (66.6667%)
Determined: a.c2 assume:0 , remaining 1 (33.3333%)
Determined: a.c1 a.c2 assume:0 , remaining 0 (0%)
WARNING (WAS006): The task is inconsistent at cycle 1.
Completed successfully.
============================================================
SUMMARY
============================================================
        Resulting task            : <embedded>:conflict
             Prove attempts       : 6
```

```
Achieved status       : cex
Assumptions
   - needed           : a.c1 a.c2 assume:0
```

See Also

get_needed_assumptions,
prove,
set_engineCG_max_mem,
set_prove_time_limit,
set_prove_verbosity

check_c2rtl

Compares a C++ model to RTL or a C++ model to another C++ model, spec and imp.

Associated App(s)

C2RTL

Syntax

```
Setup
-----
check_c2rtl -compile (-spec |-imp)
    ( list_of_source_files |
     [list_of_source_files]
     (-f file_name)+)
    [-I include_dir]+
    [-D macro_definition]+
    [-U macro_name]+
    [-include file_name]+
    [-std version]
    [-pedantic_errors]
    [-top model_top_name]
    [-unroll_limit N]
    [-max_vla_size N]
    [-max_array_sroa N]
    [-force_cpp]
    [-resolve_getenv]
    [-create_valid_enum_value_input_constraints ]
    [-create_related_covers
                {(precondition | late_precondition
                  | infinite_precondition | witness | contrapositive
                  | infinite_contrapositive)+}]
    [-verbosity N]
    [-gcc_compatibility_version version]

check_c2rtl -analyze (-spec |-imp)
    analyze_switches_and_arguments
```

```
check_c2rtl -elaborate (-spec |-imp)
    [-rename_top new_imp_top_name]
      elaborate_switches_and_arguments

check_c2rtl -setup
    [-no_auto_mapping]
    [-include_undriven_mapping]


Clear
-----
check_c2rtl -clear (all)


Signal Mapping
--------------
check_c2rtl -map
       (  [-auto]
          [(-signal_type (input | output | primary_input | primary_output
                         | bbox_input | bbox_output | user_bbox_input
                         | user_bbox_output | auto_bbox_input | auto_bbox_output
                         (-auto |-regexp)))]
          [-spec_delay spec_delay_value]
          [-imp_delay imp_delay_value]
          [-map_condition {map_cond_expr}]
          [-clock {clocking_event}]
          [-disable {disable_cond_expr}]
          [-edge (posedge | negedge | both_edge)
           |-helper]
          [-tag {tag_name}]
          [-task {task_name} |-global]
          [-respect_connections_during_reset]
       )
      |( ( -spec {signal_list} [-imp {signal_list}])
            [-inverse]
            [-return (text | id | none)]
            [-spec_delay spec_delay_value]
            [-imp_delay imp_delay_value]
            [-map_condition {map_cond_expr}]
            [-clock {clocking_event}]
            [-disable {disable_cond_expr}]
            [-edge (posedge | negedge | both_edge)]
             |-helper
```

```
             |-constraint]
            [-tag {tag_name}]
            [-task {task_name} |-global]
            [-respect_connections_during_reset]
       )
```

Controlling Automatic Init Mapping

```
check_c2rtl -auto_map_reset_x_values (on | off)
          [-map_one_side_reset_x (on | off)]
          [-task task_name]
          [-exclude_module {regexp_or_module_list}
          [-module_regexp]]
          [-exclude_instance {regexp_or_instance_list}
           [-instance_regexp]]
```

Signal Unmapping

```
check_c2rtl -unmap
         ( -id mapping_id_list
          |-spec {signal_list}
          |-imp {signal_list}
         )
           [-mapping_type (  connection_all | connection_primary_input
                           | ver_target_all | ver_target_primary_output
                           | ver_target_misc | helper ) ]
           [-proof_status (proven | cex | unprocessed | undetermined)+]
           [-signal_type (  input | output | primary_input | primary_output)]
           [-tag {tag_name}]
           [(-task task_name [-include_global_mapping])
            |-global
           ]
```

Generating Verification

```
check_c2rtl -generate_verification
        [-task task_name]
```

Utility Switches

```
----------------
check_c2rtl (-get_spec_top_instance |-get_imp_top_instance)

check_c2rtl -import_property property_name_tcl_list [-regexp]


Result Validation
-----------------
check_c2rtl -validate_results
              [-task task_name]
              [-summary]
              [-file file_name [-force]]
              [-list_sanity_asserts
                [-status {status1 status2 ...}]


Setting Mapping Context
-----------------------
check_c2rtl -set_context_of_mapping
 (   -id {mapping_id_list}
         | ( -spec {signal_list}
            |-imp {signal_list})
               [-mapping_type (connection_all | connection_primary_input
                                | connection_bbox_output
                                | connection_undriven
                                | connection_misc
                                | ver_target_all
                                | ver_target_primary_output
                                | ver_target_mpram_port
                                | ver_target_bbox_input
                                | ver_target_misc | helper)
               ]
               [-proof_status (proven | cex | unprocessed | undetermined)+]
               [-signal_type (primary_input | primary_output)]
               [-tag {tag_name}]
               [([-task task_name] [-include_global_mapping])
                |-global
               ]
        |-set_default
        |-clear_default
        )
        [-spec_delay spec_delay_value]
        [-imp_delay imp_delay_value]
```

```
        [-map_condition {map_cond_expr}]
        [-clock {clocking_event}]
        [-disable {disable_cond_expr}]
        [-edge (posedge | negedge | both_edge)]



C/C++ Model Profiler for C2RTL
------------------
check_c2rtl -cfe_profile (-spec |-imp |-both)
            (-enable |-disable |-report)
            [-memory]
            [-output_file output_file_path]


CFE Configuration
-----------------
check_c2rtl -cfe_config (-spec |-imp |-both)
            -dynamic_pruning
      [-total_time_limit total_time]
      [-per_branch_time_limit per_branch_time]
      [-check_branches (none | functions | all)]
      [-recheck_freq_level (0 | 1 | 2)]
      [-ignore_branch_reachability (true | false)]
      [-use_valid_enum_value_input_constraints (true | false)]
check_c2rtl -cfe_config (-spec |-imp |-both)
            -unroll_limit
      ( -line line_number
       |-name jasper_name
       |-label c_label)
      -file file_name
      -bound bound_value
check_c2rtl -cfe_config (-spec |-imp |-both)
            -ignore_unimplemented_functions
      (-names {function_name_list} |-clear)
check_c2rtl -cfe_config -unroll_pragma_name
      {pragma_name_list}
check_c2rtl -cfe_config (-spec |-imp) -show
       (dynamic_pruning | unroll_limit | ignore_unimplemented_functions |
unroll_pragma_name)
```

Returns

Various, context sensitive return values.

Detailed Description

The Jasper C2RTL Equivalence Checking (C2RTL) App formally compares two models, one C++ and the other either C++ or RTL. The models are called the specification (spec) and implementation (imp). C2RTL synthesizes the C++ model and analyzes and elaborates the RTL in a fashion that is similar to other Jasper apps, but in addition, it links (connects) them and creates one or more tasks for comparing pairs of signals from both models.

```
Setup
-----
-compile (-spec |-imp)
    ( list_of_source_files |
     [list_of_source_files](-f file_name)+)
    [-I include_dir]+
    [-D macro_definition]+
    [-U macro_name]+
    [-include file_name]+
    [-std version]
    [-pedantic_errors]
    [-l library]+
    [-top model_top_name]
    [-unroll_limit N]
    [-max_vla_size N]
    [-max_array_sroa N]
    [-force_cpp]
    [-resolve_getenv]
    [-create_valid_enum_value_input_constraints]
    [-create_related_covers (precondition | late_precondition
            | infinite_precondition | witness | contrapositive
            | infinite_contrapositive)+]
    [-verbosity N]
    [-gcc_compatibility_version version]

    Parse C or C++ code and synthesize a netlist whose behavior mimics the
    behavior that the code would have if compiled by GCC for an Intel x86 64
    bit CPU.
```

```
. "list_of_source_files" accepts a list of C/C++ source files.
. Use "-I include_dir" to add "include_dir" to the list of directories
  searched to resolve "#include" directives to header files. C2RTL
  implements exactly the same algorithm as GCC to search for a header file
  when preprocessing an "#include" directive. For details, refer to GCC's
  documentation about option "-I".
. Use "-D macro_definition" to define a macro. You can pass either
  "name" or "name=def". If you pass "name", C2RTL treats it as
  "name=1".
. Use "-U macro_name" to cancel any previous definition of "macro_name",
  either built-in or provided with "-D".
. Use "-include file_name" to preinclude "file_name" before compilation,
  as if GCC had received option "-include".
. Use "-l library" to link the specified library. Available libraries are:
    - systemc: Links the Jasper-provided SystemC library.
    - softfloat: Links the Berkeley SoftFloat (SoftFloat-3e) library.
    - ac: Links the Jasper-provided AC datatypes library.
    - m: Enables support for additional <math.h> (or <cmath>) functions.

    Functions available WITHOUT "-l m" (always supported):
        . ceil*
        . copysign*
        . fabs*
        . fdim*
        . floor*
        . fmax*
        . fmin*
        . fmod*
        . fpclassify
        . isfinite+
        . isinf*
        . isnan*
        . isnormal+
        . llrint*
        . llround*
        . lrint*
        . lround*
        . modf*
        . nan*
        . nearbyint*
        . remainder*
        . rint*
```

```
    . round*
    . signbit+
    . trunc*

Additional functions enabled WITH "-l m":
    . acos* (c)
    . atan* (c)
    . exp* (c)
    . exp2* (c)
    . fma*
    . frexp*
    . ilogb*
    . ldexp*
    . log* (c)
    . log2* (c)
    . log10* (c)
    . pow* (c)
    . scalbln*
    . scalbn*
    . sincos* (c)
    . sqrt* (c)

NOTES:
    . (*) means that all the C99 "overloads" and gcc __builtin versions
      are available. For instance, for "fmax", the following functions
      are available: fmax(), fmaxf(), fmaxl(), __builtin_fmax(),
      __builtin_fmaxf(), and __builtin_fmaxl().
    . (+) means that the function is also available as a gcc __builtin.
      For instance, for "isfinite", the following functions are
      available: isfinite() and __builtin_isfinite().
    . (c) means that the implementation of this function is complex,
      and that repeatedly calling it with non-constant arguments
      increases the size and complexity of the extracted formal model.
    . Since the "long double" datatype is synthesized in exactly the
      same way as "double" (using 64 bits), all the "l" overloads
      (for instance, fmaxl or nanl) behave as their "double"
      counterparts (fmax or nan).
    . The implementation of the additional functions is generally as
      precise as what you might expect of any modern C library. In
      particular, they do not guarantee to return a correctly-rounded
      result (the nearest floating-point representation of the exact
      result).
NOTE:
```

```
      . You do not need to explicitly pass the C source files from these
        libraries in the list_of_cpp_files argument of the
        "check_c2rtl -compile" command.
      . If you want to use your own version of any of these libraries,
        do not use the "-l library" option.

. Use "-std version" to set the version of the C++ standard
  (ISO/IEC 14882) used to determine how to accept C++ input files. Valid
  values for "version" are:
      . c++11
      . c++14 (default value when no -std option is given)
      . c++17
      . c++20
  C programs are parsed using the C07 standard, and currently it is not
  possible to configure a different standard.

. Use "-pedantic_errors" to direct C2RTL to do the following:
      . disable all C/C++ language extensions, and
      . emit all diagnostics (compilation warning/error messages) that
        the standard requires the compiler to issue.

  The "standard" means either ISO C 2007 for C files or the standard
  indicated by switch "-std" for C++ files.

  By default, C2RTL (and GCC) lets the input C/C++ program use language
  extensions (language or library features that are not defined in the ISO
  C/C++ standard). Such extensions might reject standard-conforming
  programs, or even alter the behavior that the standard specifies for
  them.

  Switch "-pedantic_errors" switches off all such extensions, which
  precisely means that C2RTL:

      . accepts the input program if and only if it strictly conforms to
        the ISO C/C++ standard (it only uses language/library features
        specified in the standard), and
      . assigns behavior to the program as specified by the standard.

  C2RTL treats files with extension .cc, .cp, .cxx, .cpp, .CPP, .c++, and .C
  as C++ code. It treats files with extension .c as C code. Any other file
  extension causes an error.

  The "-std" and "-pedantic_errors" switches control the exact dialect of
```

the C and C++ languages that C2RTL accepts. Generally speaking, passing "-std c++NN" causes C2RTL to parse:

- C files the same way as "gcc -std=gnu99" would, and
- C++ files the same way as "g++ -std=gnu++NN" would.

Passing "-pedantic_errors" has the same effect as GCC option "-pedantic-errors". It disables all language extensions (see below) and makes C2RTL accept programs exactly as specified by the standard given by "-std".

More specifically, C and C++ files are parsed as described below.

C files are matched against a dialect of C07 (ISO/IEC 9899:2007). C07 is identical to C99 plus the corrections introduced in the three ISO Technical Corrigenda published in 2001, 2004, and 2007.

The precise C dialect accepted is C07 when you use "-pedantic_errors". When you do not use it, C2RTL accepts C07 extended by:

- a large number of minor extensions commonly available on other C compilers
- a large list of GCC language extensions (see chapter 6 of the GCC manual) which emulate the GCC version indicated by option "-gcc_compatibility_version".

Most of these extensions enlarge the accepted language (make C2RTL accept all C07 programs and more), but some of them shrink it (some C07 strictly-conformant programs get rejected or their behavior is altered). This is because some extensions contradict ISO C 2007.

You cannot change the language dialect used to accept C input files.

C++ files are matched against a dialect of C++ defined as follows. When you use "-pedantic_errors", passing "-std c++NN" makes C2RTL accept C++ files as defined by language standard C++NN (ISO/IEC 14882, version NN). When you do not use "-pedantic_errors", C2RTL accepts C++NN extended by:

- the same minor extensions used for C files, and
- a small number of C++ language extensions, and
- a large list of GCC language extensions (see chapters 6 and 8 of the GCC manual) that emulate the GCC version indicated by option "-gcc_compatibility_version".

As for C files, some of the language extensions might conflict with ISO C++NN, making C2RTL reject or alter the behavior of C++ programs that strictly conform with ISO C++NN.

. Use "-top" to give a name to the top-level of the synthesized netlist. If you do not specify a name, the tool uses "main" as the default name.
. Use "-unroll_limit N" to specify the maximum number of loop iterations and the maximum recursion depth in the C++ code. The default value is 256.
  . If the tool is able to determine that the upper bound on the number of iterations is smaller than the specified value "N", it uses the smaller value.
  . If the tool is not able to determine this upper bound or the upper bound is greater than "N", it unrolls the loops and the recursive calls "N" number of times.
. Use "-max_vla_size N" to specify the default hardware synthesized for all VLAs in the C++ model. The default value is 256 bytes.
  . Example: check_c2rtl -compile -spec "main.cpp" -max_vla_size 512.
  . If the array expands to use more memory than max_vla_size, then the corresponding VLASizeLimit assertion will fail.
. Use "-max_array_sroa N" to modify the maximum length of arrays to which the tool applies Scalar Replacement Of Aggregates (SROA).
  SROA replaces every C/C++ array of length L (with L <= N) by L scalar variables. This makes it possible to propagate constants through array read or write operations, and makes other optimizations to work better.
  NOTE:
  . By default, SROA uses N = 65536.
  . If your C/C++ model does not have large arrays, there is generally no benefit in modifying the default N.
  . If the model has large arrays, formal-model generation might run faster if you lower N, at the cost of possibly extracting a larger formal model.
. Use "-force_cpp" to compile all .c files as C++.
. Use "-resolve_getenv" to resolve the environment variable value during compilation in "getenv" function with a string constant only. Otherwise, calling getenv has no effect.
. Use "-create_valid_enum_value_input_constraints" to generate valid enum value assumptions, one for every enum variable declared using JASPER_INPUT. Each assumption ensures that the input is only one of the declared enum values.
. Use "-create_related_covers" to enable the precondition check for one or more specified types of cover statements.
  The name of the precondition property indicates the precondition type, for example, propertyA:witness1.
  The tool creates related covers as follows. In these examples, s1 and s2 are sequences.
  . The precondition cover of an SVA implication s1 |=> P, is a

```
    finite trace where sequence s1 matches somewhere on the trace. This is
    the default.
  . The late precondition cover of an SVA implication s1 |=> s2 is a
    cover of the sequence (s1 ##1 1'b1).
  . The infinite precondition cover of an SVA implication s1 |=> P, where P
    is a infinite property, is an infinite trace where s1 matches
    somewhere on the trace.
  . The witness cover of an SVA implication s1 |=> s2, where s2 is a sequence,
    is the cover of the sequence (s1 ##1 s2).
  . The witness cover of an SVA implication s1 |-> s2, where s2 is a sequence,
    is the cover of the sequence (s1 ##0 s2).
  . The contrapositive cover of an SVA implication s1 |-> P, is a finite
    trace where P is not true somewhere on the trace.
  . The infinite contrapositive cover of an SVA implication
    s1 |=> s_eventually B, where B is a Boolean expression, is an infinite
    trace where, starting at some point, B never matches.
  Jasper creates the contrapositive cover property only when the right
  hand side of the property is Boolean. The feature does not support
  sequence/property operators.
  The following example triggers a contrapositive property only when y is
  Boolean.
  x |=> y  creates cover (~y)
  NOTE:
  . The tool does not support witness precondition covers for assumptions with
    local variable assignments.
  . By default, the tool creates "precondition" covers.
  . This command accepts Tcl lists.
. Use "-verbosity N" to specify the level of message verbosity you want.
  If N = 5, the tool shows a brief description of warning messages.
  The default is N = 6, which leads the tool to print all warning and error
  messages as is.
. Use "-f" to specify a list of files for C or C++ model compilation.
  This switch lets you specify additional source files from a file. When
  you use this command, it is optional to specify a spec or imp file list.
  Both inputs are used for compilation.
  NOTE:
    . These commands do not support file names with whitespace or the
      character "$".
    . Use multiple "-f* file_name" instances to specify multiple files.
    . Unknown and prohibited switches in "-f*" file(s) will be ignored.
    . Jasper Apps supports absolute paths declared inside .f* file(s) for
      example, /home/user/test.cpp. All other paths are related to the
      the current working directory of Jasper.
```

```
        . You can specify C or C++ source files.
        . Use one source file per line in the "-f* file_name".
        . Coroutines and modules are not supported in C++20.

    . Use "-gcc_compatibility_version version" to specify the version of gcc
      to be compatible with the compilation. The default version is equivalent
      to GCC version 12.3.0.
      Available versions are: gcc12_3, and gcc9_3.

-analyze (-spec |-imp) analyze_switches_and_arguments
    Analyze the HDL or property files intended for verification of either spec
    side, imp side, or both sides.
    You can use all the "analyze" switches with this command except "-save",
    "-restore", "-req", and "-repository".
    For more details, run "help analyze -gui".

-elaborate (-spec |-imp [-rename_top new_imp_top_name])
elaborate_switches_and_arguments
    Synthesize and read the netlist for the Verilog(R) and VHDL files that have
    been analyzed for spec or imp using the "check_c2rtl -analyze" command.
    You can use all the "elaborate" switches with this command except
    "-load_cpi", "-req", and "-repository".
    For more details, run "help elaborate -gui".
    Use the optional "-rename_top" switch to specify a new imp top instance name.

    NOTE:
    . You must elaborate the imp side ONLY after the spec side has been
      elaborated.
    . To re-elaborate, you must first run "check_c2rtl -setup -clear".

-setup
    [-no_auto_mapping]
    [-include_undriven_mapping]

    Connect the previously analyzed and elaborated spec and imp designs and
    auto map primary_input, primary_output, bbox_input, bbox_output,
    reset_x, and stopat signals.
    Setup uses name-based mapping; thus, the tool maps only signals with the
    same name on both spec and imp sides.

    NOTE: The setup auto mapping creates global mapping entries that will be
    exposed in all C2RTL tasks. The created connection mappings on boundary
    input signals are respected during reset.
```

```
    Use "-no_auto_mapping" to specify that you do not want the tool to
    run auto mapping during setup. If you use this switch, you can run auto
    mapping after the setup using "check_c2rtl -map -auto".

    Use "-include_undriven_mapping" to instruct C2RTL to include undriven
    signals mapping during the setup auto mapping.
```

```
Clear
-----
-clear (all)

    Clear C2RTL settings.
    . Use "all" to clear all C2RTL settings in the current session.
```

```
Signal Mapping
--------------
-map   ([-auto]
          [-signal_type (  input | output | primary_input | primary_output
                         | bbox_input | bbox_output | user_bbox_input
                         | user_bbox_output | auto_bbox_input | auto_bbox_output
                         (-auto |-regexp))]
          [-spec_delay spec_delay_value]
          [-imp_delay imp_delay_value]
          [-map_condition {map_cond_expr}]
          [-clock {clocking_event}]
          [-disable {disable_cond_expr}]
          [-edge (posedge | negedge | both_edge)]
           |-helper]
          [-tag {tag_name}]
          [-task {task_name} |-global]
          [-exclude_signal {signal_list}]
          [-respect_connections_during_reset]
       )
      |( ( -spec {signal_list} [-imp {signal_list}])
            [-inverse]
            [-return (text | id | none)]
            [-spec_delay spec_delay_value]
```

```
        [-imp_delay imp_delay_value]
        [-map_condition {map_cond_expr}]
        [-clock {clocking_event}]
        [-disable {disable_cond_expr}]
        [-edge (posedge | negedge | both_edge)]
         |-helper
         |-constraint]
        [-tag {tag_name}]
        [-task {task_name} |-global]
        [-respect_connections_during_reset]
   )
  | -map_file map_file_name

Map spec signals to imp signals.

Auto Mapping
------------
Use "-auto" to automatically map primary inputs and outputs, inputs and
output signals of black boxes, and undriven signals that have the same
names in the spec and imp.
. Use "-spec" with "-auto" to specify the spec instance list whose signals
  are to be auto mapped.
. Use "-imp" with "-auto" to specify the imp instance list whose signals
  are to be auto mapped. If you use "-spec" but not "-imp", the tool
  attempts to map the spec instance signals to imp signals in an instance
  of the same name, taking into account the imp prefix. If you use both
  "-spec" and "-imp" switches, the tool considers pairs of instances by
  matching each spec instance from the given list to the imp instance in
  the same position on the imp list. Therefore, the two supplied spec and
  imp lists must be of the same length.
. Use "-signal_type" and a signal type to auto map only signals of the
  specified type.
. Use "-tag" to add a tag (property name prefix) to the generated
  mappings.
. Use "-task" to consider the specified task's mapping instead of the
  current task mapping.
. Use "-global" to consider global mapping, that is, mapping that is
  exposed in all tasks.
. Use "-respect_connections_during_reset" to create connections that are
  respected during reset. This switch requires the "-global" switch.
  This switch affects only connection mappings (such as primary inputs,
  bbox outputs, undrivens, and so forth); therefore, you cannot use it with
  the "-helper" switch.
```

```
Manual Mapping
--------------
If you do not use "-auto", manually map the spec to the imp with the
following switches:
. Use "-spec" to define the names of the spec signals to map.
  A signal can be a bit-vector signal. In such cases, the imp
  signal must be a bit-vector signal of the same length. Specify
  sub-vectors of such bit-vector signals with an expression of the
  form "[left_index:right_index]", for example, [9:13] or [3:0]. The
  bits are mapped according to their order from the right vector bound
  to the left one.
. Use "-imp" to define the names of the imp signals, bits, or
  bit-vector expressions.
  . If you use "-spec" but not "-imp", then for each signal
    from the given list, the tool attempts to map the spec signals to imp
    signals of the same name, taking into account the imp prefix.
  . If you use both "-spec" and "-imp" switches, the tool considers pairs
    of signal by matching each spec signal from the
    given list to the imp signal in the same position on the imp
    list. Therefore, the two supplied spec and imp lists must be of the
    same length.
  . You can use multi-bit vector expressions as in {signal}.
. Use "-inverse" to specify that the mapped signals are the inverse (not)
  of each other. The tool generates assume or assert properties
  accordingly.
. Use "-tag" to add a tag (property name prefix) to the generated
  mappings.
. Use "-task" to relate the generated mappings to a specified task instead
  of the current task.
. Use "-global" to create global mapping, that is, mapping that will be
  exposed in all tasks.
. Use "-constraint" to create connection mapping, that is, to connect the
  signal pair by adding equivalence assumptions on them.
. Use "-respect_connections_during_reset" to create connections that are
  respected during reset. This switch requires the "-global" switch.
  This switch affects only connection mappings (such as primary inputs,
  bbox outputs, undrivens, and so forth); therefore, you cannot use it with
  the "-helper" switch.
. Use "-respect_abstraction_during_reset" to create reset_value
  abstractions that are respected during reset. This switch requires the
  "-global" switch.
```

```
Task-Related Mapping
--------------------
With either auto or manual mapping, the tool relates the generated mappings
to the current task; that is, the generated mapping pairs will be exposed
as properties for the current task only.
. Use "-task" to relate the generated mappings to a specified task instead
  of the current task.
. Use "-global" to create global mapping, that is, mapping that will be
  exposed in all tasks.
Any auto-generated connection mapping is global regardless of the supplied
task switches because connection mapping is exposed as "-env" properties,
which affect all tasks.

Return Type
-----------
With either auto or manual mapping, use "-return" and one of
the following arguments to return information:
. Use "text" to return the result as mapping text. (This is the
  default behavior.)
. Or use "id" to return the result as mapping IDs.
. Or use "none" to return nothing (empty list of strings).

Mapping Pairs as Helpers
------------------------
With either auto or manual mapping, use "-helper" to create helper mapping
pairs. Helper mapping pairs direct the tool to create helper assertions
during the generation phase ("check_c2rtl -generate_verification"). Refer to
the C2RTL User Guide for a detailed explanation of helper assertions.

The purpose of generating helpers is to prevent any unintended change in
the environment. For example, if you add a new mapping pair on a primary
input or an undriven signal, the tool generates a new "-env" assumption
(even if that was not your intention); however, using "-helper" prevents
the tool from generating an assumption and only exposes helper assertions.


Temporal and Logical Context
----------------------------
Use the following switches to specify temporal and logical context for new
mappings. This set of switches generates various combinations of temporal
and logical conditions on spec and imp equivalence.
```

```
  . -spec_delay
  . -imp_delay
  . -map_condition
  . -clock
  . -disable

  Important things to know:
  . If you do not specify delay values, the default is 0.
  . Only one of "-spec_delay" or "-imp_delay" can be greater than 0.
  . All condition values default to 1 (true).
  . Delay values are non-negative integers.
  . The following condition expressions are valid SVA expressions, and each
    of them might contain signals on both sides.
    . {map_cond_expr}
    . {disable_cond_expr}
  . {clocking_event} is "negedge" or "posedge" plus an SVA expression for
    clock signals.
  . If you supply all of these values along with the spec_signal mapped to
    the imp_signal, the tool generates an expression equivalent to the
    following:
@ (clocking_event)
        disable iff (disable_cond_expr)
              (
                 (##spec_delay_value  1'b1).triggered and map_condition.triggered
                 |->
                 spec_signal == $past(imp_signal,  spec_delay_value)
              )

    NOTE:
    . This expression specifies a condition on spec_signal and imp_signal that
      is checked on every {clocking_event} and when {disable_cond_expr} is
      false.
    . This expression is built out of the antecedent on the left of the
      implication (|->) and the consequent on its right.
    . The equality between the mapped signals is the consequent, but it might
      be more than a simple equality; that is, the mapped signals might
      differ on the relative time when they are sampled.
    . All the delays in the context expression are relative to the time of
      the event of the implication ("the |-> time").
    . {map_cond_expr} can be any Boolean expression;
      that is, it is not necessary for it to contain signals only from one
      side.
    . The consequent is an equality between the mapped signals such that the
```

```
      spec signal is sampled spec_delay_value time cycles after the
      implication time and the imp signal is sampled imp_delay_value time
      cycles after the implication time.
    . If imp_delay_value >= 0, then the consequent is of the following form:
      ( $past(spec_signal, imp_delay_value) == imp_signal )
    . The whole implication condition is valid only if, at implication time,
      {disable_cond_expr} is false.
    . The time frames are counted according to the {clocking_event}.

  Logical Context for Clocks
  --------------------------
  Use the following switches to specify logical context for new
  mappings on clock signals.
  . -map_condition
  . -edge

  Important things to know:
  . These clock signals need a specific mapping switch due to the SVA
    semantics of clock signal values inside properties. To enable checking
    the clock signal under a specific condition, we must check the
    equivalence of the @({edge} clock_signal) event of the spec and imp
    clocks when the map_condition is met.
  . A clocking signal is a signal that is combinationally derived from
    user-defined clock signals.
  . The tool does not accept "-edge" without "-map_condition".
    However, you can achieve the same effect with "-map_condition 1".
  . If you do not specify "-edge", the tool provides a value.
  . The {map_cond_expr} condition expression is a non-temporal valid SVA
    expression, and it contains signals from both sides (spec and imp).




Controlling Automatic Init Mapping
----------------------------------
-auto_map_reset_x_values (on | off)
    [-map_one_side_reset_x (on | off)]
    [-task task_name]
    [-exclude_module {regexp_or_module_list} [-module_regexp]]
    [-exclude_instance {regexp_or_instance_list} [-instance_regexp]]

    Enable or disable the automatic mapping of pairs of uninitialized signals on
    the current task.
```

```
    Use "on" to instruct C2RTL to automatically map flop and latch pairs that
    are nonresettable or uninitialized. Auto init mapping is completely
    automatic, which means it adds or removes init mappings when the reset
    changes.
    The only control you have over auto init mapping is to turn it on or off.

    . Use "-map_one_side_reset_x" with "on" if you would also like to
      automatically map flops and latches that are nonresettable or
      uninitialized on just one side (spec or imp). If you do not use this
      switch, the default is "off".
    . Use "-task" to specify a task other than the current task.
    . Use "-exclude_module" to instruct C2RTL to exclude uninitialized registers
      inside module(s), and use "-module_regexp" to specify that the
      "-exclude_module" argument list is a regular expression list.
    . Use "-exclude_instance" to instruct C2RTL to exclude uninitialized
      registers inside instance(s), and use "-instance_regexp" to specify that
      the "-exclude_instance" argument list is a regular expression list.


Signal Unmapping
----------------
-unmap
        ( -id mapping_id_list
         |-spec {signal_list}
         |-imp {signal_list}
        )
           [-mapping_type (  connection_all | connection_primary_input
                           | connection_misc
                           | ver_target_all | ver_target_primary_output
                           | ver_target_misc | helper ) ]
           [-proof_status (proven | cex | unprocessed | undetermined)+]
           [-signal_type (primary_input | primary_output)]
           [-tag {tag_name}]
           [(-task task_name [-include_global_mapping])
            |-global
           ]
    Cancel a specified mapping pair or all mappings that involve signals that
    match the spec or imp signal_list.
    The tool also cancels all properties that might have been generated from
    these mappings.
    . Use "-id" with a list of IDs for the mapping pairs you want to cancel.
    . Use "-spec" to define the set of signal names on the spec side that you
```

```
      want to unmap.
    . Use "-imp" to define the set of signal names on the imp side that you
      want to unmap.
      . Use "-signal_type" to unmap only signals of the specified type.
        You can only use this switch with "-spec" or "-imp"
        That is, if you specify mapping IDs, all types are unmapped.
      . Use "-mapping_type" to unmap the mapping pairs of the specified mapping
        type.
      . Use "-instance" to specify that you want to unmap mapping pairs of
        signals that are in the specified instance.
      . Use "-proof_status" to unmap mapping entries according to their proof
        status. This switch accepts one or more of the following arguments:
        "proven", "cex", "unprocessed", "undetermined".
      . Use "-tag" to unmap only mappings with the specified tag.
    . Use "-task" to unmap the specified task's mapping instead of the current
      task's mapping.
    . Use "-global" to unmap exposed global mapping in all tasks.
    . Use "-include_global_mapping" to unmap both the global and task-specific
      mapping pair(s).


Generating Verification
-----------------------
-generate_verification
        [-task task_name]

    Create a task that contains all the conditions, assumptions, and assertions
    to verify equivalence.

    Use "-task" to generate the properties to the specified task. If you do not
    specify a task, this command applies to the current task.

    NOTE: When you run this command, the tool also runs an interface check and
    caches the results.


Utility Switches
----------------
-get_spec_top_instance |-get_imp_top_instance
    Return the name of the spec or imp top instance.

-import_property property_name_tcl_list [-regexp]
    Import non-C2RTL properties to the C2RTL App as C2RTL properties.
```

```
    Use "-regexp" to specify that the argument is a regular expression.


Result Validation
-----------------
-validate_results
            [-task task_name]
            [-summary]
            [-file file_name [-force]]
            [-list_sanity_asserts [-status {status_1 status_2...}]

    Report the possible risks to the validity of the C2RTL proof. The risks that
    it checks are associated with the assumptions made by the Jasper-CFE
    synthesis engine on the C/C++ model to be able to synthesize it into a
    hardware model. Any risks highlighted here point to an invalid synthesis
    model that can potentially lead to false results.

    . Use "-task" to specify a task other than the current task.
    . Use "-summary" to limit the command output to the final line of the
      summary without detailing errors.
    . Use "-file" to direct the command output to the specified file. And use
      "-force" to overwrite an existing file.
    . Use "-list_sanity_asserts" to return a dictionary with sanity assertion
      types (for example, insufficient unroll limit) as keys and lists of
      corresponding assertions as values. Use "-status" with a list of statuses
      as argument to return only assertions whose status corresponds to one of
      the options in the list. Without "-status" all sanity assertions will be
      returned.

      The following options are supported by "-status":

      . cex
      . undetermined
      . unknown
      . proven

Setting Mapping Context
-----------------------
-set_context_of_mapping
        (   -id mapping_id_list
         | ( -spec {signal_list}
             |-imp {signal_list})
```

```
            [-mapping_type (  connection_all | connection_primary_input
                            | connection_bbox_output | connection_undriven
                            | connection_misc
                            | ver_target_all | ver_target_primary_output
                            | ver_target_mpram_port | ver_target_bbox_input
                            | ver_target_misc | helper) ]
            ]
            [-proof_status (proven | cex | unprocessed | undetermined)+]
            [-signal_type (  primary_input | primary_output)]
            [-tag {tag_name}]
            [([-task task_name] [-include_global_mapping])
             |-global
            ]
        |-set_default
        |-clear_default
        )
        [-spec_delay spec_delay_value]
        [-imp_delay imp_delay_value]
        [-map_condition {map_cond_expr}]
        [-clock {clocking_event}]
        [-disable {disable_cond_expr}]
        [-edge (posedge | negedge | both_edge)]

Set or change the temporal and logical context of a set of mappings.
. Use "-id" with a list of IDs for the mapping pairs whose context you
  want to change.
. Use "-spec" to define the set of signal names on the spec side whose
  mapping context you want to change.
. Use "imp" to define the set of signal names on the imp side whose
  mapping context you want to change.
  . Use "-signal_type" to set the context only for mappings of signals of
    the specified type.
    You can only use this switch with "-spec" or "-imp".
    That is, if you specify mapping IDs, the context is set for all types.
  . Use "-mapping_type" to set the context only for mapping pairs of the
    specified mapping type.
  . Use "-proof_status" to set the context for mapping entries according to
    their proof status.
    This switch accepts one or more of the following arguments:
    "proven", "cex", "unprocessed", "undetermined".
  . Use "-tag" to set the context only for mappings with the specified tag.
. Use "-task" to set context for the specified task's mapping instead of
  the current task's mapping.
```

```
    . Use "-global" to set context for exposed global mapping in all tasks.
    . Use "-include_global_mapping" to set context for both the global and
      task-specific mapping pair(s).
    . Use "-set_default" to set the default context. Once you set the default
      context, every new mapping with no explicit context has the default
      context. To use the default context in the setup for automatic mapping,
      use "check_c2rtl -setup... -setup_file...".
      NOTE: The default context applies to all mapping, including pairs of
      inputs and clock and reset signals.
    . Use "-clear_default" to clear the default context.
    . Use "-spec_delay", "-imp_delay", "-map_condition",
      "-clock", and "-disable" to specify temporal and logical context to the
      specified mappings. This set of switches generates various combinations
      of temporal and logical conditions on spec and imp equivalence. For
      usage, refer to the subsection "Temporal and Logical Context" in the
      section "Signal Mapping".
    . Use "-map_condition" and "-edge" to specify the logical context for the
      specified mappings on clock signals. For usage, refer to the subsection
      "Logical Context for Clocks" in the section "Signal Mapping".



C/C++ Model Profiler for C2RTL
------------------
-cfe_profile (-spec |-imp |-both)
             (-enable |-disable |-report)
             [-memory]
             [-output_file output_file_path]

  Control and query the C/C++ model profiler used during C2RTL compilation.

  . Use "-spec" to apply the command only to the spec side design.
  . Use "-imp" to apply the command only to the imp side design.
  . Use "-both" to apply the command to both spec and imp designs.

  . Use "-enable" to enable the profiler for the selected side(s).
    . Use "-memory" to enable memory profiling, default is VM RSS (Virtual
      Memory Resident Set Size) as the metric.

  . Use "-disable" to disable the profiler for the selected side(s) and, for
    "-both", to reset the memory profiling metric to "none".

  . Use "-report" to dump the profiler data for the selected side(s) to
```

```
    text file(s).
    . Use "-output_file" to specify the output file path.
    . If no path is specified, the dump is saved to:
        jgproject/sessionLogs/session_{i}/c2rtl_profiler_report.txt.
    . When you specify a single side ("-spec" or "-imp"), the profiler data
      for the corresponding design is written to the given output file.
    . When you specify "-both", two reports are generated, one for each
      design and the tool writes separate profiler data files for the spec
      and imp designs.



Dynamic Pruning
------------------
-cfe_config (-spec |-imp |-both) -dynamic_pruning
            [-total_time_limit total_time]
            [-per_branch_time_limit per_branch_time]
            [-check_branches branches_to_check]
            [-recheck_freq_level recheck_frequency_level]
            [-use_valid_enum_value_input_constraints (true | false)]

    Configure dynamic path pruning for C2RTL compilation.
    You must run this command before the "check_c2rtl -compile" command
    to have an effect.
    By default, dynamic path pruning settings are applied to both side models.

    . Use "-spec" to apply dynamic path pruning settings to the spec side model
      only.
    . Use "-imp" to apply dynamic path pruning settings to the imp side model
      only.
    . Use "-both" to apply dynamic path pruning settings to both sides of the
      model.
      NOTE: If you do not specify "-spec", "-imp", or "-both", the tool issues
      a warning and proceeds as if you had specified "-both".
    . Use "-total_time_limit" to limit the total time spent on branch
      feasibility decisions during C2RTL compilation. The argument must be in
      one of the time formats supported by Jasper. The default
      value is 60s. If dynamic pruning exceeds the total time limit during C2RTL
      compilation, the tool makes no additional path feasibility decisions.
      The tool interprets a total time limit of 0 seconds as "unlimited".
    . Use "-per_branch_time_limit" to limit the time spent on a single branch
      feasibility decision during C2RTL compilation. The argument must be in
      one of the time formats supported by Jasper. The default value is 1s.
```

```
      The tool interprets a per-branch time limit of 0 seconds as "unlimited".
    . Use "-check_branches" to define the type of branches the compilation
      process performs feasibility checks on. Possible values are:
      . none: Disable dynamic pruning.
      . functions: Perform feasibility checks on calls to virtual functions
        and calls to functions using function pointers.
      . all: Perform feasibility checks on all program branches. This is the
        default value.
    . Use "-recheck_freq_level" to configure the checking frequency of a branch
      if the program runs it repeatedly. Possible values are:
      . 0: Checks a branch for feasibility until it is no longer considered
        infeasible, then does not recheck it again.
      . 1: Once a branch is considered feasible, recheck its feasibility every
        2^N-th visit during program runs. It the branch proves to be
        infeasible, reset the counter. This is the default value.
      . 2: Check every branch at every visit during simulation.
    . Use "-use_valid_enum_value_input_constraints" to make dynamic path pruning
      consider only the valid enum values for inputs declared using
      JASPER_INPUT, while pruning the C++ model.
      All other values for the inputs that are not part of the enum encoding
      will be considered infeasible for doing reachability analysis of the C++
      code branches.


Unroll Limit
---------
-cfe_config (-spec |-imp |-both) -unroll_limit
                   (-line line_number
                    |-name jasper_name
                    |-label c_label)
                   -file file_name
                   -bound bound_value

    Add a bound to loops, recursive function calls, or implicit loops
    that initialize object VLA.
    The bound must be added before the "check_c2rtl -compile"
    command to have an effect.

    . Use "-spec" to apply bound settings to the spec side model.
    . Use "-imp" to apply bound settings to the imp side model.
    . Use "-both" to apply bound settings to both the spec and imp side models.
    . Use "-line line_number" to specify the loop line or the line in which the
      object VLA or recursive function is declared. Loop line is the line of
```

```
      the "for", the "do", or the "while" keyword of the loop. For do-while
      loops, the loop line is the line of the "do" keyword.
    . Use "-name jasper_name" to specify the C/C++ construct (a loop, a
      recursive function declaration, or object VLA declaration) by its name.
      Declare the name on a new line just above the construct as follows:
      #pragma JASPER_NAME <label_name>
    . Use "label c_label" to specify the loop by its C label (the label
      appearing just before the loop).
    . Use "-file file_name" to specify the name of the file containing
      the loop, recursive function declaration, or object VLA declaration.
      The file provided must be the same file provided for the "check_c2rtl -compile"
      command.
    . Use "-bound bound_value" to specify your required bound.
    . The tool produces an error if a loop bound is set inside an unused inline
      function, use "set_message -warning ECTR004; set_message -warning ECTR068;
      set_message -warning ECTR069; set_message -warning ECC032; set_message
      -warning ECC037; set_message -warning ECC038" to downgrade errors from this
command
      into warnings.


Set Ignored Unimplemented Functions
---------------------------------------
-cfe_config (-spec |-imp |-both) -ignore_unimplemented_functions
       ( -names list_of_function_names
        |-clear )

    Define which functions should be ignored by the formal model if they are
    unimplemented. By default, the formal model will contain assertions to
    verify that unimplemented functions are not reachable. This command lets you
    prevent these assertions from being generated.
    The elements in the list of function names that are defined by this command
    are called "ignored names" and might contain the wildcard character "*".
    If a function or method in the model is not implemented and has a base name
    that matches an ignored name, then the formal model will not contain
    reachability assertions for this function.
    . Use "-spec" to define the ignored names for the spec side model only.
    . Use "-imp" to define the ignored names for the imp side model only.
    . Use "-both" to define the ignored names for both sides of the model.
      NOTE: If you do not specify "-spec", "-imp", or "-both", the tool issues
      a warning and proceeds as if you had specified "-both".
    . Use "-names" to add to the set of ignored names.
    . Use "-clear" to clear the set of ignored names.
```

```
Set Unroll Pragma Name
----------------------
-cfe_config -unroll_pragma_name {pragma_name_list}

    Add a list of pragma names to specify unroll limits within the C++ code. You
    must add the pragma name list before the "check_c2rtl -compile" command to
    have any effect. Pragma names are case insensitive.


Show Configuration
------------------
-cfe_config (-spec |-imp) -show
       (dynamic_pruning | unroll_limit | ignore_unimplemented_functions |
unroll_pragma_name)

    Return the current configuration for the specified config name.
    . Use "-spec" to return the configuration for the spec side model.
    . Use "-imp" to return the configuration for the imp side model.
    . Use "dynamic_pruning" to show the dynamic pruning configuration.
    . Use "unroll_limit" to show the unroll limit configuration.
    . Use "ignore_unimplemented_functions" to show the sorted list
      of ignored function names.
    . Use "unroll_pragma_name" to show the list of unroll pragma names.
```

check_caf

Configures, manipulates, and waives checks or display/hides views in CAF.

Associated App(s)

CAF-C Auto Formal

Syntax

```
Initializing and Configuring Checks
-----------------------------------
check_caf -init
    [-systemc |-cpp]

check_caf -configure
    (-load_rule_file file_name+)

check_caf -configure
    (-save_rule_file file_name) [-force]

Extracting and Manipulating Checks
----------------------------------------------
check_caf -extract
    [ -instances instance_list |-hierarchies instance_list
     |-exclude_hierarchies instance_list |-top]
    [-extended_property_name]
    [-task task_name]
    [-list]

check_caf -list
    [-type (Assert | Assume | Cover)+]
    [-waived (none | only)]
    [-regexp property_expression_regexp]
    [-status (  all | cex | proven | covered | unprocessed
         | unreachable | undetermined )+]
    [-severity (error | warning | info)]
    [-domain domain_list]
    [-category category_list]
```

```
    [-tag tag_list]
    [-message message]
    [-label label]
    [-source_location source_location]
    [-instance instance]
    [-module module]
    [-grouping]
    [-silent]

check_caf -prove
    [-time_limit time_limit]
    [-mem_limit memory_limit]
    [-task task_name_list]
    [-keep_traces [assert | all | none]]
    [-property property_name_list]
    [-bg]

check_caf -clear

Debugging Checks
--------------
check_caf -waiver -add
    -id (message_id_tcl_list | property_id_tcl_list)
    -comment comment
    [-export file_name [-do_not_apply] [-append]]
    [-silent]
    [-force]

check_caf -waiver -add
    [-instance instance_name]
    [-expression expr]
    [-category message_category]
    [-tag message_tag]
    [-argument argument [-relaxed] [-regexp_argument |-wildcard_argument]]
    [-module module_name]
    [-hierarchy module_or_instance]
    [-source_file file_name
        [-start_line line_num] [-end_line line_num]]
        [-start_column column_num] [-end_column column_num]]
    [-top]
    -comment comment
    [-export file_name [-do_not_apply] [-append]]
    [-silent]
```

```
    [-force]

check_caf -waiver -remove waiver_id_tcl_list
    [-silent]

check_caf -waiver -show waiver_id_tcl_list
    [-silent]

check_caf -waiver -list
    [-instance instance_name]
    [-expression expr]
    [-category message_category]
    [-tag message_tag]
    [-module module_name]
    [-hierarchy module_or_instance]
    [-source_file file_name]
    [-state waiver_state]
    [-top]
    [-silent]

check_caf -waiver -list_matching_checks waiver_id_tcl_list
    [-silent]

check_caf -waiver -export -file_name file_name
    [waiver_id_tcl_list]
    [-append]
    [-version]
    [-unmatched]
    [-updated]

check_caf -waiver -import -file_name file_name
    [-silent]
    [-force]

check_caf -waiver -accept_update waiver_id_tcl_list
    [-silent]

Reporting CAF App Setup
----------------------------------------
check_caf -report -setup
    [-file file_name [-force]
      [[ -html [-launch_html_browser]
          [-source_path_mapping paths_mapping]
```

```
          [-relative_file_path]]
        |-xml |-csv]]

Reporting CAF App Checks
----------------------------------------
check_caf -report
    [-property
        [-status (  cex | unreachable | proven
                          | covered | undetermined | unprocessed )+]
        [-type (assert | cover)+]
        [-task task_list]]
    [-violation
        [-severity (error | warning | info)+]
        [-order (  category | tag | filename | instance(module)
                           | label | severity )]
        [-include_design_build]]
    [-detailed [-verbose (tag_list | all)]]
    [-domain domain_list]
    [-category category_list]
    [-tag tag_list]
    [-instance instance_list]
    [-regexp]
    [-waived (none | only [ -user_tcl |-user_rtl
                                    |-auto]+)]
    [-grouping (none | basic | advanced)]
    [-file file_name [-force]
      [ [-html [-launch_html_browser]
          [-source_path_mapping paths_mapping]
          [-relative_file_path]]
       |-xml |-csv]]

Reporting CAF App Waivers
----------------------------------------
check_caf -report -waiver
    [only ( -user_tcl |-user_rtl |-auto )+]
    [-waiver_status (matched | unmatched)+]
    [-detailed]
    [-file file_name [-force]
      [ [-html [-launch_html_browser]
          [-source_path_mapping paths_mapping]
          [-relative_file_path]]
       |-xml |-csv]]
```

```
Reporting CAF App Signoff
-----------------------------------------
check_caf -report -signoff
    [-file file_name [-force]
      [ [-html [-launch_html_browser]
          [-source_path_mapping paths_mapping]
          [-relative_file_path]]
       |-xml |-csv]]

GUI Operations
--------------
check_caf -gui
   (  [-reinvoke_mode (reinvoke | rerun)]
    | [-order_by (domain | tag | message | filename | severity | label |
instance(module))]
    | [-grouping_mode (none | basic | advanced)] )+
    | [-view -display (design_hierarchy)]
    | [-view -hide (design_hierarchy)]
    | [-show_file_mode (file_name | full_path)]
```

Detailed Description

```
Use this command to configure, manipulate, and waive checks in the
CAF App extraction process or display/hides views in the CAF App.

Initializing and Configuring Checks
-----------------------------------
-init
    [-systemc |-cpp]
    Initialize the CAF app in either systemC mode or C/C++ mode.

-configure
    (-load_rule_file file_name_list)
    Load the configuration from one or more .def files. If you do not provide
    any files, the tool loads the default configuration.

-configure
    (-save_rule_file file_name) [-force]
    Save the configuration to a <file_name>.def file.

    If a file with the same name already exists, use "-force" to overwrite it.
```

```
NOTE: To configure CAF checks, use "config_rtlds" command.


Extracting and Manipulating Checks
-------------------------------------------
-extract
    [ -instances instance_list |-hierarchies instance_list
     |-exclude_hierarchies instance_list |-top]
    [-extended_property_name]
    [-task task_name]
    [-list]

    Extract checks from the design.

    . Use "-instances" to limit extraction to the specified instances.
    . Use "-hierarchies" to limit extraction to the specified hierarchies.
    . Use "-exclude_hierarchies" to exclude the specified hierarchies.

      NOTE: If you want to exclude all instances of a module recursively on
      down, leverage the following command:

      get_design_info -module <module> -list instance

      This provides a list of paths to each instance. Pass this list to the
      "-exclude_hierarchies" switch as follows:

      set mod_list [get_design_info -module <module> -list instance -silent]
      check_caf -extract -exclude_hierarchies $mod_list

    . Use "-top" to limit extraction to the top instance.
    . Use "-extended_property_name" to specify the following naming convention
      for checks:
      \<instance>_<file><start line><end line><check type>prop<prop id>
    . Use "-task" to extract to a specified task.
      NOTE: If the task you provide does not exist, the tool creates it first.
    . Use "-list" to list the IDs of the extracted properties or lint messages.
      NOTE: If you do not use the optional "-list" switch, "check_caf
      -extract" returns the total number of extracted properties or lint
      messages.


-list
    [-type (Assert | Assume | Cover)]+
```

```
[-waived (none | only)]
[-regexp property_expression_regexp]
[-status (  all | cex | proven | covered | unprocessed
          | unreachable | undetermined )]+
[-severity (error | warning | info)]
[-domain domain_list]
[-category category_list]
[-tag tag_list]
[-message message]
[-label label]
[-source_location source_location]
[-instance instance]
[-module module]
[-grouping]
[-silent]
```

List the IDs for the property candidates or messages that match the specified criteria.

. Use "-type" to list properties of a specific type.
. Use "-waived" to list messages with the specifed waived status.
. Use "-regexp" to filter properties or messages by expression using regular expression.
. Use "-status" to list properties of a specific status.
. Use "-severity" to list only properties and messages of the specified severity.
. Use "-domain" to list only the properties and messages in the specified domains.
. Use "-category" to list only the properties and messages in the specified categories.
. Use "-tag" to list only properties and messages with the specified tags.
. Use "-message" to list only properties and messages with the specified message.
. Use "-label" to list properties and messages with the specified label.
. Use "-source_location" to list properties and messages with the specified source location. The source location can be a file, starting line, or a range of starting and ending lines.
. Use "-instance" to list properties and messages with the specified instance.
. Use "-module" to list properties and messages with the specified module.
. Use "-grouping" to list properties and messages of a related group.
  NOTE: The grouping mode preference (basic or advanced) is the same as that selected for the run.

```
-prove
    [-time_limit time_limit]
    [-mem_limit memory_limit]
    [-task task_name_list]
    [-property (property_name_list | task::property_name_list)]
    [-keep_traces [assert | all | none]]
    [-bg]

    Prove CAF properties using one of the following strategies:
    . "orchestration" - If the "caf_prove_strategy" variable is set
      to "hybrid" and there is a single task to run or multiple tasks to run
      in parallel mode, CAF uses this strategy. This is the default.
    . "autoprove" - If the "caf_prove_strategy" variable is set
       to "autoprove" or there are multiple tasks to run in serial mode,
       CAF uses this strategy.
    . Use "-time_limit" to override the default "caf_prove_time_limit" of
      "2h".  Specify "time_limit" values as an integer followed by s, m, or h
      (no spaces).
      . s = seconds
      . m = minutes
      . h = hours
    . Use "-mem_limit" to override the default 2048MB "engineCG_max_mem" for
      "autoprove". Specify "memory_limit" values as an integer followed by m or
      g (no spaces).
      . m = megabytes
      . g = gigabytes
    . Use "-task" to specify the task you want proven.
      NOTE:
      . If you provide no task, the tool proves all CAF tasks.
      . If you do not supply a task name, the tool proves all assertions and
        covers in the current task.
    . Use "-keep_traces" to keep traces generated during proof.
      . Use "assert" to keep traces for assertions only.
      . Use "all" to keep all traces during proof.
      . Use "none" to discard all traces during proof.
    . Use "-property" to prove the specified properties only.
      NOTE:
      . When specifying a list of properties, enclose the list in curly
        braces.
      . To prove properties not in the current task, prepend the property
        names with "<task>::".
```

```
      . All listed properties must belong to the same task.
    . Use "-bg" to run the proof in the background.

-clear
    Clear all CAF App information without clearing the results of the
    analysis and elaboration.

Debugging Checks
--------------
-waiver -add -id (message_id_tcl_list | property_id_tcl_list)
    -comment comment
    [-export file_name [-do_not_apply] [-append]]
    [-silent]
    [-force]

    Add waivers on the specified messages or properties.

    . Use "-comment" to add a comment to the waiver.
    . Use "-export" to save waivers to the specified file.
      . Use "-do_not_apply" to prevent applying the waiver.
      . Use "-append" to append to the file if it already exists

    . Use "-silent" to suppress output and generate only Tcl return values.
    . Use "-force" to add waivers even if there are no matching messages or
      properties.

-waiver -add
    ( [-instance instance_name]
      [-expression expr]
      [-category message_category]
      [-tag message_tag]
      [-argument argument_list [-relaxed] [-regexp_argument |-wildcard_argument]]
      [-module module_name]
      [-hierarchy module_or_instance]
      [-source_file file_name
        [-start_line line_num] [-end_line line_num]]
        [-start_column column_num] [-end_column column_num]]
      [-top]
    )+
    -comment comment
    [-export file_name [-do_not_apply] [-append]]
    [-silent]
    [-force]
```

```
Add waivers as specified and return an ID list of the added waivers.

NOTE: You must use one or more of the following switches to add waivers to
properties: "-tag", "-instance", "-expression", and "-source_file".

. Use "-instance" to add waivers on properties of the specified
  instance. This switch supports wildcard and regular expression.
  Add "_REG_EXPR:" prefix to specify regular expression and "_WILD_CARD:"
  to specify wildcard.
. Use "-top" to add waivers on properties of the top instance.
. Use "-expression" to add waivers on properties with the specified
  expression.
. Use "-category" to add waivers on messages in the specified category.
  This switch supports wildcard and regular expression.
  Add "_REG_EXPR:" prefix to specify regular expression and "_WILD_CARD:"
  to specify wildcard.
. Use "-tag" to add waivers on messages with the specified tag.
  This switch supports wildcard and regular expression.
  Add "_REG_EXPR:" prefix to specify regular expression and "_WILD_CARD:"
  to specify wildcard.
. Use "message" to add waivers on the specified message.
  This switch supports wildcard and regular expression.
  Add "_REG_EXPR:" prefix to specify regular expression and "_WILD_CARD:"
  to specify wildcard.
. Use "-argument" to add waivers on the specified message arguments.
  NOTE:
  . Specified message arguments in the waiver need to exactly match message
    arguments.
  . Use "-regexp_argument" or "-wildcard_argument" to consider the
    arguments regular expressions or wildcards respectively.
  . Use "-relaxed" to match message arguments partially and in an unordered
    manner.
  . "-relaxed" ignores any duplication in the arguments specified.
. Use "-module" to add waivers on messages from the specified module.
  This switch supports wildcard and regular expression.
  Add "_REG_EXPR:" prefix to specify regular expression and "_WILD_CARD:"
  to specify wildcard.
. Use "-hierarchy" to add waivers on messages from the specified module or
  instance.
  NOTE:
  . If you specify a module, the tool adds waivers on messages from the
    module specified, all instances of the module as well as all
```

```
        subinstances and exclusive submodules. For example, module B is an
        exclusive submodule of module A if all the instances of module B are
        subinstances of module A instances.
      . If you specify an instance, the tool adds waivers on messages from the
        instance specified, all subinstances and exclusive submodules.
      . If you specify a name that matches both a module and an instance, the
        tool adds waivers as if you had specified a module.
      . This switch does not support Wildcard or regular expression.
    . Use "-source_file" to add waivers on checks from the specified file.
      This switch supports wildcard and regular expression.
      Add "_REG_EXPR:" prefix to specify regular expression and "_WILD_CARD:"
      to specify wildcard.
    . Use "-start_line" to add waivers on checks after the specified
      line.
    . Use "-end_line" to add waivers on checks before the specified line.
    . Use "-start_column" to add waivers on checks after the specified
      column.
    . Use "-end_column" to add waivers on checks before the specified column.
    . Use "-comment" to add a required comment to the waiver.
    . Use "-export" to save waivers to the specified file.
      . Use "-do_not_apply" to prevent applying the waiver.
      . Use "-append" to append to the file if it already exists
    . Use "-silent" to suppress output and generate only Tcl return values.
    . Use "-force" to add waivers even if there are no matching properties or
      messages.

-waiver -remove waiver_id_tcl_list [-silent]
    Remove the specified waivers and return an ID list of the matching messages
    or properties.

    Use "-silent" to suppress output and generate only Tcl return values.

-waiver -show waiver_id_tcl_list [-silent]
    Show details for the specified waivers.

    Use "-silent" to suppress output and generate only Tcl return values.

-waiver -list
    [-instance instance_name]
    [-expression expr]
    [-category message_category]
    [-tag message_tag]
    [-module module_name]
```

```
    [-hierarchy module_or_instance]
    [-source_file file_name]
    [-state (matched | unmatched | updated)]
    [-top]
    [-silent]

    List the IDs of the waivers that match the specified criteria.

    . Use "-instance" to list waivers added on properties of the specified
      instances. This switch supports wildcards as follows:
    . You can use a string such as "a.*.b" but not a string such as "a*".
    . Use "*" to add waivers on properties of all instances including "top"
      and ".*" to exclude "top".
    . Use "-top" to list waivers added on properties of the "top" instance.
    . Use "-expression" to list waivers added on properties with the specified
      expression.
    . Use "-category" to add waivers on messages in the specified category.
    . Use "-tag" to add waivers on messages with the specified tag.
    . Use "message" to add waivers on the specified message.
    . Use "-module" to add waivers on messages from the specified module.
    . Use "-hierarchy" to add waivers on messages from the specified module or
instance.
    . Use "-source_file" to list waivers added on checks from the specified
      file.
    . Use "-state" to list waivers added on checks from the specified
      waiver state.
    . Use "-silent" to suppress output and generate only Tcl return values.

-waiver -list_matching_checks waiver_id_tcl_list [-silent]
    List the IDs of properties or messages that match the specified waivers.

    Use "-silent" to suppress output and generate only Tcl return values.

-waiver -export -file_name file_name
    [waiver_id_tcl_list]
    [-append]
    [-version]
    [-unmatched]
    [-updated]
    Export waivers to the specified file.

    This command generates a Tcl file with the specified file name.
```

```
    . Use "waiver_id_tcl_list" to export only the listed waivers.
    . Use "-append" to append the extracted waivers to the specified file.
    . Use "-version" to include tool version information to the header of the
      specified file.
    . Use "-unmatched" to generate waiver file with unmatched waivers only.
    . Use "-updated" to generate waiver file with updated waivers only.

-waiver -import -file_name file_name
    [-module_mapping {old_module_name=new_module_name}]
    [-silent] [-force]
    Import waivers from the specified file.

    . Use "-force" to import waivers even if there are no matching properties.
    . Use "-module_mapping" to be help the tool to update waivers from a design
    with a different top module.

    NOTE: When you import waivers after the design has
    changed, the tool tries to update waivers automatically. Waivers that are
    automatically imported and updated are added to the tool and reported in
    the console. However, if the tool could not update the waivers, they are
    are reported as unmatched.

-waiver -accept_update waiver_id_tcl_list [-silent]
    Accept the update done by the tool to the specified waivers and return an
    ID list of the matching properties or messages.

    Use "-silent" to suppress output and generate only Tcl return values.

Reporting CAF App Setup
---------------------------------------
-report -setup
    [-file file_name [-force]
      [ [-html [-launch_html_browser]
          [-source_path_mapping paths_mapping]
          [-relative_file_path]]
       |-xml |-csv]]

    Generate a setup report.

    Setup reports consist of one section, which is Tool Configuration.

    . Use "-file" to specify a file name. And use "-force" to overwrite an
      existing file.
```

```
      . Use "-html" to report results as HTML.
        . Use "-launch_html_browser" to launch an HTML browser with results.
        . Use "-source_path_mapping" to specify the mapping between UNIX(R) and
          Microsoft(R) Windows(R) when using Microsoft(R) Internet Explorer(R).
          To link directly to the source files, the mapped paths should
          be separated with the character '='; for example, /home/user_name=z:
        . Use "-relative_file_path" to open the source files of the relative
          file path in the HTML report. The same relative file path is used in
          the elaboration.
      . Use "-xml" to report results as XML.
      . Use "-csv" to report results as CSV.
      . If you do not specify a format, the default is text.
      NOTE: If you do not specify a file, the report is printed to the console.

Reporting CAF App Checks
----------------------------------------
-report
    [-property
        [-status (  cex | unreachable | proven
                  | covered | undetermined | unprocessed)+]
        [-type (assert | cover)+]
        [-task task_list]]
    [-violation
        [-severity (error | warning | info)+]
        [-order (  category | tag | filename | instance(module)
                 | label | severity)]
        [-include_design_build]]
    [-detailed [-verbose (tag_list | all)]]
    [-domain domain_list]
    [-category category_list]
    [-tag tag_list]
    [-instance instance_list]
    [-regexp]
    [-waived (none | only [ -user_tcl |-user_rtl
                           |-auto]+)]
    [-grouping (none | basic | advanced)]
    [-file file_name [-force]
      [[ -html [-launch_html_browser]
          [-source_path_mapping paths_mapping]
          [-relative_file_path]]
        |-xml |-csv]]

    Generate a checks report.
```

When you use this command without additional arguments, the tool prints a violations summary to the console.

. Use "-property" to generate a properties report.
  . Use "-status" to generate a report for the specified statuses only. The default is all.
  . Use "-type" to generate a report for the specified types only. The default is all.
  . Use "-task" to generate a report for the specified tasks only. The default is all. This switch supports regular expressions.
. Use "-violation" to generate a violations report.
  . Use "-severity" to generate a report for the specified violation severities only. The default is all.
  . Use "-order" to generate a report ordered by the specified criteria. The default is "category".
    NOTE: Use "mode" to generate a report sorted by modes of violations. This is applicable only in the Multimode Summary View.
  . Use "-include_design_build" to include violations of design build.
  NOTE: Running the violations report command in the Multimode Summary View generates a report that includes all violations and their modes.
. Use "-domain" to generate a report for the specified domains.
  This option works with rule violations and formal domains only.
. Use "-category" to generate a report for the specified categories.
  This option works with rule violations and formal categories only.
. Use "-tag" to generate a report for the specified tags.
  This option works with rule violations and formal tags only.
. Use "-instance" to generate a report for the specified instances.
. Use "-regexp" to consider the category, tag, or instance name a regular expression.
. Use "-waived only" to generate a report for waived checks that were waived by waivers of the specified types only. If no waiver type is specified, a report of all waived checks will be generated.
. Use "-waived none" to exclude waived checks from the report. By default, waived checks are included in the report.
. Use "-grouping none" to generate a report with no grouping information.
. Use "-grouping basic" to generate a report where only same tags can be grouped together, reporting only the primary.
. Use "-grouping advanced" to generate a report where only specific tags can be grouped together, reporting only the primary.

NOTE: Grouping is a GUI concept that groups related checks and shows only one check, which is the primary. By default, the report is generated with the app's current grouping option.

```
    . Use "-detailed" to generate a detailed report. By default, the tool
      generates a summary report only. And use "-vebose" to include verbose
      information on the specified tags.
    NOTE: "-verbose" switch works only with Violations reports.
    . Use "-file" to specify a file name. And use "-force" to overwrite an
      existing file.
      . Use "-html" to report results as HTML.
        . Use "-launch_html_browser" to launch an HTML browser with results.
        . Use "-source_path_mapping" to specify the mapping between UNIX(R) and
          Microsoft(R) Windows(R) when using Microsoft(R) Internet Explorer(R).
          To link directly to the source files, the mapped paths should
          be separated with the character '='; for example, /home/user_name=z:
        . Use "-relative_file_path" to open the source files of the relative
          file path in the HTML report. The same relative file path is used in
          the elaboration.
      . Use "-xml" to report results as XML.
      . Use "-csv" to report results as CSV.
      . If you do not specify a format, the default is text.
      NOTE: If you do not specify a file, the report is printed to the console.

Reporting CAF App Waivers
-----------------------------------------
-report -waiver
    [only (-user_tcl |-user_rtl |-auto)+]
    [-waiver_status (matched | unmatched)+]
    [-detailed]
    [-file file_name [-force]
      [[-html [-launch_html_browser]
          [-source_path_mapping paths_mapping]
          [-relative_file_path]]
      |-xml |-csv]]

    Generate a waivers report.

    When you use this command without additional arguments, the tool prints
    to the console a waivers report containing a summary of all waivers in
    addition to one section for every waiver type that contains information of
    each waiver of this type.

    . Use "only" to include waivers of the specified types only. By default,
      waivers of all types are included.
    . Use "-waiver_status" to include waivers with the specified waiver
      statuses only. By default, waivers with all statuses are included.
```

```
    . Use "-detailed" to add the checks that are waived by this waiver beneath
      every waiver's information. By default, waived checks are not added.
    . Use "-file" to specify a file name. And use "-force" to overwrite an
      existing file.
      . Use "-html" to report results as HTML.
        . Use "-launch_html_browser" to launch an HTML browser with results.
        . Use "-source_path_mapping" to specify the mapping between UNIX(R) and
          Microsoft(R) Windows(R) when using Microsoft(R) Internet Explorer(R).
          To link directly to the source files, the mapped paths should
          be separated with the character '='; for example, /home/user_name=z:
        . Use "-relative_file_path" to open the source files of the relative
          file path in the HTML report. The same relative file path is used in
          the elaboration.
      . Use "-xml" to report results as XML.
      . Use "-csv" to report results as CSV.
      . If you do not specify a format, the default is text.
      NOTE: If you do not specify a file, the report is printed to the console.

Reporting CAF App Signoff
----------------------------------------
-report -signoff
    [-file file_name [-force]
      [[-html [-launch_html_browser]
          [-source_path_mapping paths_mapping]
          [-relative_file_path]]
      |-xml |-csv]]

    Generate a signoff report.

    Signoff reports include the signoff information
    (delta of used settings and rule file with the default settings and rule
    file) of the tool configuration, and summaries of the violations,
    waived violations and waivers.
    When you use this command without additional arguments, the tool prints
    the signoff report to the console.

    . Use "-file" to specify a file name. And use "-force" to overwrite an
      existing file.
      . Use "-html" to report results as HTML.
        . Use "-launch_html_browser" to launch an HTML browser with results.
        . Use "-source_path_mapping" to specify the mapping between UNIX(R) and
          Microsoft(R) Windows(R) when using Microsoft(R) Internet Explorer(R).
          To link directly to the source files, the mapped paths should
```

```
          be separated with the character '='; for example, /home/user_name=z:
        . Use "-relative_file_path" to open the source files of the relative
          file path in the HTML report. The same relative file path is used in
          the elaboration.
      . Use "-xml" to report results as XML.
      . Use "-csv" to report results as CSV.
      . If you do not specify a format, the default is text.
      NOTE: If you do not specify a file, the report is printed to the console.

GUI Operations
--------------
-gui
   ( [-reinvoke_mode (reinvoke | rerun)]
    |[-order_by (domain | tag | message | filename | severity | label |
instance(module))]
    |[-grouping_mode (none | basic | advanced)])+
    |[-view -display (design_hierarchy)]
    |[-view -hide (design_hierarchy)]
    |[-show_file_mode (file_name | full_path)]

    Perform operations specific to the GUI.
    . Use "-reinvoke_mode" to specify the mode used by the "Reinvoke" button.
      . Use "reinvoke" to clear all and then reapply "analyze", "elaborate",
        "clock", "reset", and signals configuration commands as well as waivers.
      . Use "rerun" to clear all and then reapply all commands and settings
        used in the previous run.
    . Use "-order_by" to specify the order preference for the violation
      messages view. The default is "domain".
    . Use "-grouping_mode" to specify the grouping mode preference for the
      violation messages view.
      . Use "none" to view violation messages with no grouping information.
      . Use "basic" to view violation messages where only same tags can be
        grouped together, viewing only the primary.
      . Use "advanced" to view violation messages where only specific tags can be
        grouped together, viewing only the primary.
      NOTE: The default is "basic".
    . Use -view -display to display the specified view.
    . Use -view -hide to hide the specified view.
    . Use "-show_file_mode" to modify the file information displayed in the "File"
      column of the "Violation Messages View".
      This command also affects the display when you order by "Filename".
      . Use "file_name" to view the file name only.
      . Use "full_path" to view the full path.
```

Example

```
check_caf -init -systemc
config_rtlds -rule -enable -tag { HLS_IS_OOBI HLS_AS_OVFL }
config_rtlds -rule -enable -category CAF_AUTO_FORMAL_SHIFT
config_rtlds -rule -enable -domain all
compile -cpp v2.cpp -I include/ -D CAF -l ac_datatypes -unroll_limit 3
compile -hls -top DUT dut1.cc
check_caf -extract
check_caf -list -source_location /a/b/c.cpp
check_caf -waiver -add -property {1 2 3} -comment "Expected to fail in this
configuration"
check_caf -waiver -add -instance a.b.* -domain CAF_AUTO_FORMAL_SHIFT -comment "Waive
all properties for type CAF_AUTO_FORMAL_SHIFT under a.b subinstances"
check_caf -waiver -add -tag HLS_IS_OOBI -source_file ..designsreference_dut.cc -
start_line 65 -end_line 68 -comment "waive all violations in dut from lines 65 through
68 having HLS_IS_OOBI tag"
check_caf -waiver -remove {1 2 3}
check_caf -waiver -remove [check_caf -waiver -list]
check_caf -waiver -export -file /a/b/mywaivers.tcl
check_caf -waiver -import -file /a/b/c/mywaivers.tcl
check_caf -waiver -list_matching_checks {1 2}
check_caf -waiver -list -source_file /a/b/c.v -expression ~a.rstN
check_caf -prove -task .
check_caf -report -setup
check_caf -report -violation -property -csv -file checks.csv -detailed
check_caf -report -waiver only -auto -user_tcl -waiver_status matched -detailed
check_caf -report -signoff -file signoff_report.html -html -launch_html_browser
check_caf -gui -order_by tag
check_caf -gui -grouping_mode advanced
check_caf -gui -view -display design_hierarchy
check_caf -gui -show_file_mode file_name
```

See Also

config_rtlds,
check_c2rtl,
check_hls,
set_caf_signal_type,
set_caf_stuck_at_bitblasted,
set_caf_signals_max_bits,
set_caf_signals_filter_wide_signal,
set_caf_signals_extraction_heuristic,
set_caf_signals_filter_connected,
set_caf_signals_filter_constant,
set_caf_signals_customize

check_cdc

Initiates structural and functional verification on clock domain crossings.

Associated App(s)

CDC Verification

Syntax

```
Environment Setup
-----------------
check_cdc -init

check_cdc -name_map -file file_name

CDC Extraction
--------------
check_cdc -extract

Clearing Data
-------------
check_cdc -clear (-analysis |-all)

Launching Multimode
-------------------
check_cdc -multimode

Rule Configuration
------------------
check_cdc -check
          ( -rule -set {{n_min | reset_min_dff value}+}
            ([-source_clock clock_signal]
            [-destination_clock clock_signal]
            [-module module_name]
            |-hier_group_level level ))

check_cdc -glitch_free
          ( -add -module  {module_name_list}
```

```
           |-clear )

Structural Analysis
-------------------
check_cdc -scheme
          ( -create scheme_type -formal_list {formal_signals}
           |-add ( ndff | mux_loop | mux_ndff | mux_pulse
                  | handshake | fifo | pulse | edge | sync_enabler
                  | glitch_protector | reset )
            -map {{formal_signal signal}+}
            [-module module_name])

check_cdc -reset
          ( -set_order ({reset_signal+ {reset_signal+}+}
                               |-clear
                               |-preview) )

check_cdc -violation
          (-group (none | basic | advanced))

Functional Analysis
-------------------
check_cdc -protocol_check
          ( -add expression_name -expression expression_template -scheme scheme_type
           |-remove expression_name
           |-generate [-include_failed_schemes]
           |-prove [-scheme {scheme_list}]
                  [-property {protocol_check_name_list}]
           |-export -file file_name [-force]
                  [ -property {protocol_check_name_list} [-regexp]
                   |-default_clock default_clock_name
                   |-filter filter_id]
                  [-default_disable_cond expression]
           |-clear )

Metastability Injection
-----------------------
check_cdc -metastability
        ( -inject [-include_reset] [-include_protocol_check]
            [-include_inactive_pairs]
         | -disable ( -pair key_list
                     |-source key_list
                     |-destination key_list
```

```
                |-property key_list )
        | -enable ( -pair key_list
                  |-source key_list
                  |-destination key_list
                  |-property key_list )
        | -injection_status ( -pair [key_list]
                            |-source [key_list]
                            |-destination [key_list]
                            |-property [key_list] )
        |-prove [-property property_list]
        |-export
          ( -time_window (N | time_unit | N time_unit)
             [-tolerance (N | time_unit | N time_unit)]
             [-dir dir_name] [-report] [-include_comboglitch]
             [-include_reset] [-include_inactive_pairs]
             [-max_operands N]
           | -to_formal -file file_name [-force])
        |-import -file file_name -from_formal
        |-clear )
```

Filter

```
check_cdc -filter
          ( -add ( -clock_domain domain_name
                           |-source_clock_domain domain_name
                           |-destination_clock_domain domain_name
                           |-local_source_clock clock_signal
                           |-local_destination_clock clock_signal
                           |-source_unit signal
                           |-destination_unit signal
                           |-occurrence design_element
                           |-pair_classification (data | control | datacontrol)
                           |-scheme scheme_name
                           |-scheme_type scheme_type
                           |-cdc_group cdc_group
                           |-scheme_property scheme_property_name
                           |-user_property user_property_type
                           |-tag tag_name
                           |-category category_name
                           |-domain domain_name
                           |-severity severity_value
                           |-module module_name
                           |-instance instance_name
```

```
                            |-local_source_reset reset_signal
                            |-local_destination_reset reset_signal
                            |-source_reset_domain source_reset_domain
                            |-destination_reset_domain destination_reset_domain
                            |-unit_type unit_type
                            |-liberty_type liberty_type
                            |-reset_domain reset_domain
                            |-declared_clock (declared | undeclared)
                            |-violation_key violation_key
                            |-number_of_pairs number_of_pairs
                            |-port_type (clock | reset | bbox_input
                                        | bbox_output | primary_input
                                        | primary_output | primary_inout)
                            |-signal_config_type (ConstantValue | StaticValue
                                                  | ExclusiveValue |
GrayCodedSig)
                            |-source_type (Console | TCL | SDC)
                            |-detection_type detection_type
                            |-undeclared_resets undeclared_reset_signals
                            |-undeclared_clocks undeclared_clock_signals)+
                           [-regexp]
                           [-literal]
           |-clear )

Waiver
------
check_cdc -waiver -add -reason reason [-comment comment]
        (  (-filter filter_id [-export export_to_file [-do_not_apply]]
           [-expression sva_expression |-expression_list sva_expression_list])
         | (-hierarchy module_or_instance [-filter filter_id]) )

check_cdc -waiver
      ( -remove (-all | id_tcl_list |-confidence_score confidence_score_percentage)
       |-generate [-filter filter_id] [-reason reason_tcl_list]
       |-prove [-id id_tcl_list] [-task task_list]
       |-accept (id_tcl_list |-confidence_score confidence_score_percentage)
       |-import -file file_name
       |-export -file file_name [-force]
         [(-user_tcl |-imported |-hierarchy)+]
         [(-safe |-unsafe |-unmatched |-unvalidatable |-mixed |-undetermined)+] )

CDC Export
----------
```

```
check_cdc -export
           ( ( -type (signal_config_property | waiver_cond_property))
            |-database
            |-to_formal
            |-default_clock default_clock_name )
           (-file file_name [-force])
           [-default_disable_cond expression]

CDC Import
----------
check_cdc -import
( (-waiver -database file_name [-module module_name])
 |(-from_formal -file file_name) )

Bottom-Up Hierarchical CDC
--------------------------
check_cdc -hierarchical
          ( (-load [-directory directory_name] [-instance instance_name]
             -file_name file_name_or_pattern_list )
           |(-report report_file_name -info database_file_name )
           |(-get_version -file file_name )
           |(-generate [-module module_name] [-instance instance_name] )
           |-prove )

Reporting CDC App Setup
---------------------------------------
check_cdc -report -setup
    [only ( -design_statistics |-tool_configuration
           |-design_configuration)+]
    [-file file_name [-force]
      [ [-html [-launch_html_browser]
          [-source_path_mapping paths_mapping]
          [-relative_file_path]]
      |-xml |-csv]]

Reporting CDC App Checks
----------------------------------------
check_cdc -report
    [-property
        [-status (  cex | unreachable | proven
                          | covered | undetermined | unprocessed)+]
        [-type (assert | cover)+]
        [-task task_list]]
```

```
    [-violation
        [-severity (error | warning | info)+]
        [-order (  category | tag | filename | instance(module)
                           | label | severity | mode)]
    [-detailed]
    [-domain domain_list]
    [-category category_list]
    [-tag tag_list]
    [-instance instance_list]
    [-regexp]
    [-waived (none | only [ -hierarchy |-imported
                                    |-user_tcl |-auto]+)]
    [-grouping (none | basic | advanced)]
    [-file file_name [-force]
      [ [-html [-launch_html_browser]
          [-source_path_mapping paths_mapping]
          [-relative_file_path]]
      |-xml |-csv]]

Reporting CDC App Waivers
-----------------------------------------
check_cdc -report -waiver
    [only ( -hierarchy |-imported |-user_tcl
           | -auto)+]
    [-waiver_status (  safe | unsafe | unvalidatable | undetermined
                             | unmatched | mixed)+]
    [-detailed]
    [-file file_name [-force]
      [ [-html [-launch_html_browser]
          [-source_path_mapping paths_mapping]
          [-relative_file_path]]
      |-xml |-csv]]

Reporting CDC App Signoff
-----------------------------------------
check_cdc -report -signoff
    [-file file_name [-force]
      [ [-html [-launch_html_browser]
          [-source_path_mapping paths_mapping]
          [-relative_file_path]]
      |-xml |-csv]]

Objects List
```

```
------------
check_cdc -list ( domains | filters | rules | user_defined_schemes
                        | pairs | scheme_properties| violations
                        | violation_groups | signoff | waivers
                        | signal_config | reset | groups
                        | schemes | domain_crossings | units | clock_signals
                        | inactive_pairs | inactive_reset_pairs
                        | all_reset_pairs | clock_groups | clock_matrix
                        | clock_synchronicity | setup | ports
                        | reset_paths | reset_definitions | declared_resets
                        | reset_schemes
                        | reset_schemes_and_rejected_user_defined | ipqc
                        | ipqc_no_clock_matrix | reset_order | rdc_pairs
                        | boundary_info | virtual_clocks | icg_cells
                        | virtual_resets | glitch_free_cells | false_paths
                        | convergence_schemes | clock_sense
                        | inactive_destinations | user_properties
                        | hier_db_info | timing_arcs | design_resets
                        | waiver_reasons )
                      [-filter filter_id] [-include_indexes]
                      [-file file_name [-force]]

check_cdc -list ( ipqc [-no_clock_matrix] | ipqc_no_clock_matrix )
               -file file_name [-force]

Debug
-----
check_cdc -debug -clock_domain_all_nets (true | false)
check_cdc -debug -reset_domain_all_nets (true | false)
check_cdc -debug -color_clock_logic (true | false)
check_cdc -debug -metastability -property property_name
check_cdc -debug -show_clock_domain signal_name
check_cdc -debug -show_reset_domain signal_name

GUI
---
check_cdc -gui -order_by (tag | domain | message | severity | instance(module))

Show Schematic
--------------
check_cdc -show_schematic
          [ -signal signal_list [-all |-driver |-port]
           |-instance instance_name
```

```
          |-path -from signal_name -to signal_name]
         [-context (cdc | rdc)]
         [-new_window new_window_name]


CDC Machine Learning
--------------------
check_cdc -ml
         ( -dump_data -dir directory_path
          |-train -dir directory_path -projects {project_directories}
          |-infer -dir directory_path
 )
```

Returns

Multiple return values.

Detailed Description

Use this command to set up your CDC verification environment, initiate the structural analysis, complete the functional checks, and provide the metastability modeling and injection.

IMPORTANT: At various points, this help references CDC units. Units are always a design element, that is, flops, top inputs and outputs, blackboxed inputs and outputs, and signals with stopats.

```
Environment Setup
-----------------
-init

    Initialize settings required for the CDC App.
    For example, this switch enables CDC to use information in SDC files that
    you previously loaded with the "read_sdc" command.

-name_map -file file_name

    Use this command to load a Conformal LEC .map.do file that contains
    keypoint mapping information in the form of Conformal LEC add mapped
    point commands.
    The mapped signals are used to translate "config_rtlds" commands and
```

```
    waivers from an RTL analysis to the mapped signals in the gate-level
    analysis.
    NOTE: This command is applied only when "set_cdc_gate_level_analysis_mode"
    is enabled.

CDC Extraction
--------------
-extract

    Run the entire structural CDC analysis.

    After loading a Liberty(TM) file with the "liberty -load" command,
    the timing information contained in the loaded file will be considered
    during clock domain detection. When you include this information,
    the tool creates clock domain associations between ports of cells in the
    library and then uses these associations during clock domain detection
    if any blackboxed instances of a cell are found.

    KNOWN LIMITATIONS:
    . Currently, the tool considers only the following timing arcs when
      trying to extract clock domain associations between cell ports:

      . Combinational timing arcs ("combinational", "combinational_rise",
        "combinational_fall")
      . Sequential timing arcs:
        . Edge-sensitive ("rising_edge", "falling_edge")
      . Setup or hold ("setup_rising", "setup_falling", "hold_rising",
          "hold_falling")

    . The tool only creates clock domain associations for whole ports of a
      cell, that is, clock domain inference for individual pins of a port is
      not currently supported. Every pin of a port "B" must have a supported
      timing relationship with every pin of another port "A" for the clock
      domain of "B" to be inferred from that of "A".

    . Topological ordering of black boxes is currently unavailable. Black
      boxes in a cascade or loop configuration are processed in an
      undeterministic order, and clock inference of ports may produce
      overall incomplete results as only the information already computed
      at the time of processing is used for each black box.

Clearing Data
-------------
```

```
-clear (-analysis |-all)

    . Use "-clear -analysis" to clear the results from the extraction phase.
    . Use "-clear -all" to clear CDC and RTLDS configurations, SDC, and the
      results from the extraction phase.

Launching Multimode
-------------------
-multimode
    Enables the multimode feature, which launches mode sessions for multiple
    configurations of a design.
    NOTE:
    . For this command to work, modes information must be provided.

Rule Configuration
------------------
-check (( -rule -set {{n_min | reset_min_dff value}+}
        [-source_clock clock_signal]
        [-destination_clock clock_signal]
        [-module module_name]
       |-hier_group_level level ))

    Changes the "n_min" or "reset_min_dff" conditional value, or
    changes the hierarchical level for the grouping of violations.

    . Use "-rule -set" to change value for one of the n_min or reset_min_dff
      parameters when certain conditions are met.
      To configure condition:
      . Use "-source_clock" to apply the rule to crossings that match the
        specified source signal.
      . Use "-destination_clock" to apply the rule to crossings that match the
        specified destination signal.
      . Use "-module" to apply the rule to synchronizers that have all the
        flops in the specified module.

      The following parameters are supported:
        . n_min int
          Specifies the minimum number of DFFs in the NDFF synchronizer.
          The default is 2.
        . reset_min_dff int
          Specifies the minimum number of flops in the reset synchronizer.
          The default is 2.
    . Use "-hier_group_level" to specify the maximum hierarchical level in which
```

```
      violations should be grouped. If a violation belongs to an instance that
      has a greater level than the one specified, this violation is related to the
      parent instance corresponding to the maximum level. The default is 3.

-glitch_free ( -add -module {module_name_list}
                 | -clear)
    Configures glitch-free modules. Glitch violations inside these modules will
    be automatically waived by the tool.
    . Use "-add" to add modules to the list of glitch-free cells.
    . Use "-clear" to remove all modules from the list of glitch-free cells.

Structural Analysis
-------------------
-scheme ( -create scheme_type -formal_list {formal_signals}
         |-add (  ndff | mux_loop | mux_ndff | mux_pulse
                | handshake | fifo | pulse | edge | sync_enabler
                | glitch_protector | reset | icg  | enable_based )
          -map {{formal_signal signal}+}
         [-module module_name])

    Use the specified design configuration to identify structural problems.

    . Use "-create" to create a new custom scheme type and insert it into the
    library. Use this switch to extend the tool's off-the-shelf library of
      synchronization schemes with your own custom synchronizers.
      NOTE:
      . The scheme_type provided must be a new type. If you provide a name that
        conflicts with a name that already exists in the library, the tool
        errors out.
      . Formal signals are names for the relevant signals inside each
        synchronizer. The tool maps these to the actual RTL signals in your
        design for each instance of a synchronizer. The tool already has a set
        of these signals for the supported types, but it can be extended with
        new ones. A valid formal signal name can only have letters, numbers,
        and underscores. If you provide  an invalid formal signal, the tool
        errors out.
      . You must provide "dout" as a formal signal for all custom
        synchronizers. This signal is the output of the synchronizer.
    . Use "-add" to add a user-defined CDC scheme that maps to a pre-defined or
      custom synchronizer using the specified signal mappings. To add a custom
      synchronizer, you first need to create it using "check_cdc -scheme -create".
    . Use "-map" with a list of mapping signals pairs.
      . NDFF required formal signals               : data, dout
```

```
  This scheme is a DFF chain used to synchronize single bit and wide paths.
. MUX_LOOP required formal signals           : data, dout, dready
  This scheme is used to synchronize a data path using a MUX controlled by an
  enable signal.
. MUX_NDFF required formal signals           : data, dout, sready, dready
  This scheme is used to synchronize a data path using a NDFF
  to synchronize the control signal.
. MUX_PULSE required formal signals          : data, dout, sready, dready,
                                               enable
  This scheme is used to synchronize a data path using a Pulse
  to synchronize the control signal.
. FIFO required formal signals               : wfull, rempty, winc, rinc,
                                               wdata, rdata,
                                               (wptr | sclk),
                                               (rptr | dclk)
  NOTE:
  . At least one of the parameters "wptr" and "sclk" must be present
    in the parameter map.
  . At least one of the parameters "rptr" and "dclk" must be present
    in the parameter map.
. Handshake required formal signals          : data, sreq, dreq, dack, sack
. Pulse required formal signals              : data, dout
. Edge required formal signals               : data, dout,
                                               activity (high | low),
                                               sensitivity (rising | falling)
. Sync_enabler required formal signals:      : enable,
                                               (srcdomain | srcdomainlist),
                                               dstdomain
  This scheme uses an enabling signal to synchronize a data path between one
  or more source domains and a specified destination domain.
  NOTE:
  . Parameters "srcdomain" and "srcdomainlist" are mutually exclusive,
    and at least one of them must be present in the parameter map.
  . Sync_enabler scheme can only be added as an instance-based
    scheme, that is, it cannot be added in combination with "-module".
. Glitch_protector required formal signals   : dout
  Use this scheme to add a control-based synchronizer. "glitch_protector"
  can be added as a user-defined module-based or instance-based scheme
  only.
  NOTE: Formal signals "gpdata" and "control" are optional when adding a
  module-based glitch protector.
. Reset required formal signals              : srst, dclk,
                                               (dout | fsrst | snrst)
```

```
        Use this scheme to add a custom module that acts as a classic reset
        synchronizer (the synchronized output signal is asynchronously asserted
        and synchronously de-asserted) or as a direct reset synchronizer (the
        synchronized output signal is fully synchronized).
        NOTE:
        . Reset synchronizers are identified as a result of the
          "check_cdc -extract" command.
        . An optional parameter "fsrst" can be used to specify an output
          of the synchronizer as fully synchronized.
        . An optional parameter "snrst" can be used to specify an output
          of the synchronizer as asynchronously asserted and synchronously
          de-asserted.
        . An optional parameter "dout" can be used to specify an output
          of the synchronizer as fully synchronized or asynchronously asserted
          and synchronously de-asserted. The behavior of that output is
          determined based on the structure and inputs to the synchronizer (that
          combination of structure and inputs determines whether that synchronizer
          behaves like a classical or a direct synchronizer).
        . At least one of parameters "dout", "fsrst", and "snrst" must be
          specified.
        . Parameter "dout" cannot be used along with parameters "fsrst"
          and/or "snrst".
        . Parameters "fsrst", "snrst", and "srst" can be reused multiple times
          in a single declaration.
      . NOTE: You can add the polarity of signals in the pairs mapping.
        For example, you can define the following:
        check_cdc -scheme -add sync_enabler -map {{enable ~en} {srcdomain clk1}
{dstdomain clk2}}
      . ICG to add a user-defined Integrated Clock Gating (ICG) cell
        that maps to a pre-defined template using the specified signal mapping.
        . Use "-module" to add an instantiated module as an ICG cell.
        . Use "-map" with a list of mapping signal pairs.
          ICG requires the following formal signals: clkin, enable, and clkout.
          You can use an optional parameter "testenable" to specify the test
          enable signal.
          An example of a valid ICG declaration follows: check_cdc -scheme
          -add icg -module modulename -map {{clkin inputclockpinname}
          {enable enablepinname} {testenable testenable_pin}
          {clkout outputclockpinname}}
      . Enable_based to add a convergence scheme with the specified signals
        mapping to the convergence analysis. The required "enable_based"
        formal parameters follow: enable
    . Use "-module" to add an instantiated module as a CDC scheme.
```

```
-reset -set_order ({reset_signal+ {reset_signal+}+}
                   |-clear
                   |-preview )
```

. Use "-set_order" to specify an order among reset signals during CDC setup phase. CDC uses this order during "check_cdc -extract" to automatically waive different_reset violations accordingly. With this command you inform the tool that resets which appear later in the list are asserted before the resets that appear earlier in the list. For example, "check_cdc -reset -set_order {ARST1 ARST2}" automatically waives different_reset violations for ARST1->ARST2 reset domain crossings (that is, for back-to-back flops where the first is reset by ARST1 and the second is reset by ARST2).
Reset signals that are asserted simultaneously can be grouped inside second-level lists. For example, with "check_cdc -reset -set_order {ARST1 ARST2 {ARST3 ARST4}}", you guarantee that ARST3 and ARST4 are always asserted at the same time and that they are always asserted before ARST2, which is always asserted before ARST1.
. Use "-set_order -clear" to remove all reset order definitions previously entered during CDC setup phase.
. Use "-set_order -preview" to list all the reset-domain crossings that will be automatically waived by the tool, given the reset order definitions provided. This command returns a Tcl dictionary where the keys correspond to source resets and the values correspond to destination resets. That is, if a given key-value pair is present in the Tcl dictionary, a reset-domain crossing between a flop reset by the source reset (key) and a flop reset by the destination reset (value) will be automatically waived by the tool.
NOTE:
. Reset order information can be specified in successive "check_cdc -reset -set_order" commands. The tool keeps an internal representation of the cumulative effect of all the reset order definitions provided.

```
-violation (-group (none | basic | advanced))
```

. Use "-group" to specify the grouping preference for the violation messages view.
. Use "none" to view violation messages with no grouping information.
. Use "basic" to view violation messages with same tags only grouped together, viewing only the primary.
. Use "advanced" to view violation messages with different tags grouped together, viewing only the primary.

Functional Analysis

```
-------------------
-protocol_check ( -add expression_name -expression expression_template -scheme
scheme_type
                |-remove expression_name
                |-generate [-include_failed_schemes]
                |-prove [-scheme {scheme_list}]
                        [-property {protocol_check_name_list} [-regexp]]
                |-export -file file_name [-force]
                         [-property {protocol_check_name_list} [-regexp]
                          |-default_clock default_clock_name
                          |-filter filter_id]
                         [-default_disable_cond expression]
                |-clear )
```

Generate and verify scheme properties for all CDC schemes already detected.

. Use "-add" to add new protocol checks for a scheme type.
  NOTE:
  . The expression name must be a new name. If you provide a name that conflicts with one that already exists in the tool library, the tool errors out. To check the expression names existent in the tool library, refer to "CDC App Functional Checks" section of the CDC User Guide.
  . The expression template must be a valid SVA or PSL property. To build this template, you must replace the signal names with the corresponding formal signal. All formal signals used in the template must be related to the synchronizer in question and should be preceded by a "%" character. The tool then replaces all %formal_signal by the actual formal signal when creating the protocol checks for the synchronizer instance. An example of a valid property template follows:
  {@(posedge %dclk) $changed(%sready) |=> $stable(%sready) until_with $rose(%dclk)}
  . The scheme_type provided must be a valid one and must already by in the tool library.
. Use "-remove" to remove a user created protocol check.
. Use "-generate" to automatically create the scheme properties. And use "-include_failed_schemes" to include schemes that have structural violations or waivers.
. Use "-prove" to validate the scheme properties created.
  . Use "-scheme" to specify the list of CDC schemes you want to verify.
  . Use "-property" to specify the list of scheme properties you want to verify.
  NOTE: CDC considers embedded assumptions and signal configuration assumptions when proving protocol checks.

```
      KNOWN LIMITATION: Changes in the <embedded> task do not change the
      protocol checks results. You need to clear the results manually and rerun.
    . Use "-export" to export the scheme properties to a file.
      . Use "-file" to export to the specified file. And use "-force" to
        overwrite an existing file.
      . Use "-property" to specify the list of scheme properties you want to
        export. And use "-regexp" to consider the property name a regular
        expression.
      . Use "-default_clock" to specify the default assertion clock.
      . Use "-filter" to export properties that match the filter criteria.
      . Use "-default_disable_cond" to set which disable condition is
        prepended to exported properties.
    . Use "-clear" to clear all scheme properties generated and return CDC to
      the scheme phase.

Metastability Injection
-----------------------
-metastability ( -inject [-include_reset] [-include_protocol_check]
                   [-include_inactive_pairs]
                 | -disable ( -pair key_list
                              |-source key_list
                              |-destination key_list
                              |-property key_list )
                 | -enable ( -pair key_list
                             |-source key_list
                             |-destination key_list
                             |-property key_list )
                 | -injection_status ( -pair [key_list]
                                       |-source [key_list]
                                       |-destination [key_list]
                                       |-property [key_list] )
                 |-prove [-property property_list]
                 |-export
                 ( -time_window (N | time_unit | N time_unit)
                       [-tolerance (N | time_unit | N time_unit)]
                       [-dir dir_name] [-report] [-include_comboglitch]
                       [-include_reset] [-include_inactive_pairs]
                       [-max_operands N]
                  | -to_formal -file file_name [-force])
                 |-import -file file_name -from_formal
                 |-clear )

    Inject metastability effect in all active CDC pairs in the COI of all user
```

```
properties and verify the properties in the presence of metastability.

. Use "-inject" to automatically inject metastability in all user properties.
  . Use "-include_reset" to inject metastability considering reset pairs.
  . Use "-include_protocol_check" to include protocol checks that were
    automatically generated by the tool as targets for metastabilty
    injection.
    NOTE: "check_cdc -metastability -inject -include_reset" and
    "check_cdc -metastability -inject -include_protocol_check" are
    included as an initial release to gather feedback from early adopters
    and finalize implementation for an upcoming release. Contact
    support@cadence.com to provide feedback or comments targeting the
    upcoming production version.
  . Use "-include_inactive_pairs" to also model the metastability effect
    in inactive CDC pairs.
. Use "-disable" to disable metastability injection in one or more
  clock-domain crossings.
. Use "-enable" to re-enable metastability injection in one or more
  clock-domain crossings.
. Use "-injection_status" to get the status of metastability injection
  (enabled or disabled) in one or more clock-domain crossings. The tool will
  return all injection statuses if you do not provide a key.
  NOTE: All three switches, "-disable", "-enable", and "-injection_status",
  provide the option to select the target crossings by pair, destination,
  source, or property:
  . Use "-pair" to select one or more crossings directly by providing
    their CDCPair keys as a list. Use "check_cdc -list pairs" to find
    the available CDCPair keys.
  . Use "-source" or "-destination" to select all the crossings to one or more
    sources or destinations, providing their CDCUnit keys as a list, or a wildcard
    expression. Use "check_cdc -list units" to find the available CDCUnit keys.
    NOTE:
  . The wildcard expression tries to match any CDCUnit.
  . Use "-property" to select all the crossings in the COI of one or more
    user properties by providing their UserProperty keys as a list. Use
    "check_cdc -list user_properties" to find the available UserProperty
    keys.
. Use "-prove" to check the result of the metastability injection.
  By default, all user properties in all tasks are selected to be proven.
  . Use "-property" to specify a list of properties to be proven
    (instead of all user properties). The switch requires fully qualified
    property names (<task>::<property>), but the task <embedded> is
    assumed if none is specified for a property.
```

```
    . Use "-export" to export the metastabilty candidates to a file to
      be used in simulation or in a different formal environment.
      . Use "-file" to export to the specified file.
      . Use "-force" to overwrite an existing file.
      . Use "-dir" to export the simulation files to the specified directory.
        If you do not specify a directory, the simulation files are written to
        the session directory.
      . Use "-time_window" to specify the time window:
        . "N" is mandatory if you do not specify the time unit.
          . If you use "N" and do not specify a "time_unit", the default is
            nanoseconds (ns).
          . "N" is a natural non-zero number.
        . "time_unit" is mandatory if you do not specify "N".
          . If you use "time_unit" and do not specify "N", the default is 1.
          . "time_unit" is s, ms, us, ns, ps, or fs.
        . If you override the setup and hold times when binding the
          metastability injection model to your design, you need to specify
          the value in seconds, using scientific notation. For example:
          "bind top : dut meta_injection #(.TSETUP_clk(5e-9)) i_meta_injection();"
          overrides the value of the setup window associated with clock "clk"
          to be 5ns. Refer to the CDC User Guide for additional information.
      . Use "-tolerance" to specify the tolerance for near-border changes in hold
violations:
        . "N" is mandatory if you do not specify the time unit.
          . If you use "N" and do not specify a "time_unit", the default is
              nanoseconds (ns).
          . "N" is a natural non-zero number.
        . "time_unit" is mandatory if you do not specify "N".
          . If you use "time_unit" and do not specify "N", the default is 0.
          . "time_unit" is s, ms, us, ns, ps, or fs.
      . Use "-report" to generate a report with the results of the "-export"
        command, including the list of exported and discarded candidates.
        NOTE: CDC groups injection points exported to simulation by destination
        and displays these by destination in the report generated with the
        "-report" option. For example, even when two different CDC pairs,
        data1[1:0]->q[1:0] and data2[1:0]->q[3:2], are shown in the Pairs
        tab, a single grouped injection point "q[3:0]" is exported to
        simulation and shown in the metastability injection report. Consider
        this fact when noting divergences between the number of CDC pairs
        reported and the number of candidates for metastability injection
        exported to simulation.
      . Use "include_comboglitch" to explicitly generate the extra RTL for
        combo-glitch mode.
```

```
        NOTE: +define+CDC_COMBOGLITCH has no effect unless you generate the
        model with this switch.
      . Use "-include_reset" to explicitly generate the extra RTL for a
        simplified model of injection due to asynchronous reset de-assertion.
      . Use "-include_inactive_pairs" to also include inactive CDC pairs in
        the model for metastability injection in simulation. When you do not
        use this switch, the tool exports active CDC pairs only.
      . Use "-to_formal -file" to export metastability candidates to a file
        for import to a different Jasper formal environment.
      . Use "-max_operands" to change the maximum number of operands in an
        exportation candidate. Default: 64.
    . Use "-import" to import metastability candidates in a different formal
      environment. The design setup (RTL, clocks, resets, configuration) must
      be the same as when the imported file was generated.
    . Use "-clear" to clear the metastability effect and return CDC to
      the previous phase.


Filter
------
-filter ( -add ( -clock_domain domain_name
                |-source_clock_domain domain_name
                |-destination_clock_domain domain_name
                |-local_source_clock clock_name
                |-local_destination_clock clock_name
                |-source_unit signal
                |-destination_unit signal
                |-occurrence design_element
                |-pair_classification (data | control | datacontrol)
                |-scheme scheme_name
                |-scheme_type scheme_type
                |-cdc_group cdc_group
                |-scheme_property scheme_property_name
                |-user_property user_property_name
                |-tag tag_name
                |-severity severity_value
                |-module module_name
                |-instance instace_name
                |-local_source_reset reset_signal
                |-local_destination_reset reset_signal
                |-source_reset_domain source_reset_domain
                |-destination_reset_domain destination_reset_domain
                |-unit_type unit_type
```

```
            |-liberty_type liberty_type
            |-reset_domain reset_domain
            |-declared_clock (declared | undeclared)
            |-violation_key violation_key
            |-number_of_pairs number_of_pairs
            |-port_type (clock | reset | bbox_input | bbox_output
                        | primary_input | primary_output
                        | primary_inout)
            |-signal_config_type (ConstantValue | StaticValue
                                  | ExclusiveValue | GrayCodedSig)
            |-source_type (Console | TCL | SDC)
            |-detection_type detection_type
            |-undeclared_resets undeclared_reset_signals
            |-undeclared_clocks undeclared_clock_signals)+
           [-regexp]
           [-literal]
     |-clear )

Manage CDC filters.

. Use "-add" to add a new filter. The filtering criteria is dependent on the
  category you want to filter as defined below.
  NOTE: All the criteria support a wildcard expression by default, or you
  can specify one of the following switches.
  . Use "-regexp" to filter the criteria using regular expressions.
  . Use "-literal" to filter the criteria using fixed strings.

  . CDC Pairs:
    . Use "-clock_domain" to filter all CDC pairs in the specified clock
      domain.
    . Use "-source_clock_domain" to filter all CDC pairs that have the source
      unit in the specified clock domain.
    . Use "-destination_clock_domain" to filter all CDC pairs that have the
      destination unit in the specified clock domain.
    . Use "-local_source_clock" to filter all CDC pairs that have the source unit
      being clocked by the specified clock signal.
    . Use "-local_destination_clock" to filter all CDC pairs that have the
      being clocked by the specified clock signal.
    . Use "-source_unit" to filter all CDC pairs that have the specified
      unit as the source unit.
    . Use "-destination_unit" to filter all CDC pairs that have the
      specified flop as the destination unit.
    . Use "-pair_classification" to filter all CDC pairs that have the
```

```
    specified classification. Options include "data", "control" and
    "dataControl".
    NOTE:
    . "-pair_classification" criteria can only be used to set rules
    of type pair.
. CDC Schemes:
  . Use "-scheme" to filter the specified scheme.
  . Use "-scheme_type" to filter the schemes that have the specified
    type.
. CDC Groups (convergence and glitch):
  Use "-cdc_group" to filter the specified group.
. CDC Units:
  . Use "-unit_type" to filter the units with the specified unit type.
  . Use "-liberty_type" to filter the units that have the specified
    Liberty type.
  . Use "-declared_clock" to filter the units associated with a
    declared/undeclared clock.
  . Use "-port_type" to filter the ports with the specified port type.
  . Use "-source_type" to filter units with the specified source type.
. Scheme Properties:
  Use "-scheme_property" to filter the specified scheme property.
. User Properties:
  Use "-user_property" to filter the specified user property.
. Tags:
  . Use "-tag" to filter the CDC violations with the specified
    tag name.
  . Use "-category" to filter the CDC violations with tags belonging to
    the specified category name.
  . Use "-domain" to filter the CDC violations with tags belonging to
    domain name.
  . Use "-severity" to filter CDC violation by the specified
    severity.
. Violations:
  . Use "-occurrence" to filter all violations that have the
    specified occurrence point.
  . Use "-module" to filter the CDC violations that occur inside a
    particular module.
  . Use "-instance" to filter the CDC violations that occur inside a
    particular instance.
  . Use "-local_source_reset" to filter the CDC violations with the
    specified local source reset signal.
  . Use "-local_destination_reset" to filter the CDC violations with the
    specified local destination reset signal.
```

```
        . Use "-source_reset_domain" to filter the CDC violations whose source
          reset is derived from the specified reset domain.
        . Use "-destination_reset_domain" to filter the CDC violations whose
          destination reset is derived from the specified reset domain.
        . Use "-reset_domain" to filter the CDC violations with the
          specified reset domain.
        . Use "-violation_key" to filter violations with the specified key.
        . Use "-number_of_pairs" to filter violations by the number of associated
          pairs.
        . Use "-detection_type" to filter violations by the scheme detection type.
        . Use "-undeclared_resets" to filter FLP_RS_UDCL violations by the
          undeclared reset signals.
        . Use "-undeclared_clocks" to filter FLP_CK_UDCL violations by the
          undeclared clock signals.
      . Signals:
        . Use "-signal_config_type" to filter signals with the specified
          signal configuration type.

        NOTE:
        . The use of "-scheme", "-scheme_type", "-scheme_property", "-tag"
          "-severity", "-cdc_group", and "-user_property"
          switches is restricted to violation filtering options and can only
          be applied on waivers.
    . Use "-clear" to remove all filters.

Waiver
------
-waiver -add -reason reason [-comment comment]
        (  (-filter filter_id [-export export_to_file [-do_not_apply]]
           [-expression sva_expression |-expression_list sva_expression_list])
         | (-hierarchy module_or_instance [-filter filter_id]) )

    . Use "-add" to create a new waiver for violations.
      . Use "-reason" to specify why the violation is considered safe. The tool
        converts reasons to formal expressions. You can validate them with
        "check_cdc -waiver -prove". The safety of the waiver can then be
        reviewed during the sign-off process.
        NOTE:
         . To list all available reasons, run "check_cdc -list waiver_reasons".
         . "SEE_COMMENTS" is always available for all tags.
           . When you use "SEE_COMMENTS", the waiver is "Unvalidatable" and
             cannot be verified by formal engines.
           . The "-comment" switch is mandatory for this reason.
```

```
      . Use "-comment" to add a note about the waiver.
        NOTE:
          . If you specify a reason other than "SEE_COMMENTS", this switch is
            optional. Reasons include built-in comments.
          . The tool prepends your comment to the built-in comment.
      . Use "-filter" to base the waiver on a specific filter.
      . Use "-expression" to add a custom condition to the waiver.
      . Use "-expression_list" to add a list of custom conditions to the waiver.
      . Use "-export" to save waivers to the specified file.
        . Use "-do_not_apply" to prevent the waiver from being applied.
      . Use "-hierarchy" to specify a module or instance for which the tool
        should report no internal violations. You can also use "-filter" to add
        criteria to the new waiver.

-waiver
      ( -remove (-all | id_tcl_list |-confidence_score confidence_score_percentage)
       |-generate [-filter filter_id] [-reason reason_tcl_list]
       |-prove [-id id_tcl_list] [-task task_list]
       |-accept (id_tcl_list |-confidence_score confidence_score_percentage)
       |-import -file file_name
       |-export -file file_name [-force]
         [(-user_tcl |-imported |-hierarchy)+]
         [(-safe |-unsafe |-unmatched |-unvalidatable |-mixed |-undetermined)+])

    . Use "-remove" to delete all waivers or only the specified ones.
      . Use "-confidence_score" to remove ML-generated waivers with
        confidence scores at or below the specified value.
        NOTE:
        . This switch accepts values in the range from 0 to 100.
        . This switch is only available when the Machine Learning flow is enabled.
    . Use "-generate" to create waivers with formal reasons for all supported
      tags. The tool converts reasons to formal expressions. You can validate
      them with "check_cdc -waiver -prove".
      . Use "-filter" to base the waiver generation on a specific filter.
      . Use "-reason" to provide a custom list of reasons.
        NOTE: To list all reasons, run "check_cdc -list waiver_reasons".
    . Use "-prove" to validate waivers related to the specified reasons.
      Use "-id" to validate only the specified waivers.
      Use "-task" to validate waivers related to properties in the specified
      tasks.
      NOTE:
      . If the waiver is proven, it is "safe". Otherwise, it is "unsafe".
      . Available tasks are "<CDC_conditional_waiver_properties>" and
```

```
        "<RDC_conditional_waiver_properties>".
    . Use "-accept" to mark waivers as user waivers. User waivers are always
      considered valid, regardless of their status.
      . Use "-confidence_score" to accept ML-generated waivers. Waivers with
        confidence scores at or above the specified value will be accepted.
        NOTE:
        . This switch accepts values in the range from 0 to 100.
        . This switch and "id_tcl_list" argument are mutually exclusive.
        . This switch is only available when the Machine Learning flow is enabled.
    . Use "-import" to load waivers from a file.
    . Use "-export" to save all waivers to a Tcl file. Each waiver is written
      as a "check_cdc -waiver -add" command.
      . Use "-user_tcl" to export only waivers added in the current session.
      . Use "-imported" to export only waivers imported from scripts in the
        current session.
      . Use "-hierarchy" to export hierarchical waivers.
      . Use "-safe", "-unsafe", "-unvalidatable", "-unmatched", "-mixed", or
        "-undetermined" to export waivers with the specified status only.
        NOTE: By default, the tool exports all waivers.
      . Use "-force" to overwrite an existing file. You can later source the
        file in this or another Jasper session.

CDC Export
----------
-export ( ( -type (signal_config_property | waiver_cond_property))
         |-database
         |-to_formal
         |-default_clock default_clock_name )
         (-file file_name [-force] )
         [-default_disable_cond expression]

    Export CDC information to simulation or in another Jasper session.
    . Use "-type" to specify the report type you want generated to the
      specified file.
    . Use "-database" to export the current CDC database to the specified file,
      which you can then load in another Jasper session using the "-import"
      switch.
    . Use "-to_formal" to export signal config, conditional waiver, and scheme
      properties that you want to import to Jasper in a different formal
      environment.
    . Use "-default_clock" to specify the default assertion clock.
    . Use "-file" to export to the specified file. And use "-force" to
      overwrite an existing file.
```

```
    . Use "-default_disable_cond" to set which disable condition is
      prepended to exported properties.

CDC Import
----------
-import ( (-waiver -database file_name [-module module_name])
         |(-from_formal -file file_name) )
    Imports CDC information from a previously exported CDC database file.
    . Use "-waiver" to import waivers from a CDC database file and automatically
      translate them to match the current design hierarchy.
      . Use "-database" to specify the database path.
      . Use "-module" to specify the module name that generated the source
        database. If omitted, the module name is read from the database file
        itself. This option is useful when the module has been renamed in the
        current design.
      NOTE: The tool does not support the criteria of design clock, source
      clock, or destination clock. If any of these criteria are present, the
      tool ignores the corresponding waiver and issues a warning.

    . Use "-from_formal" to import signal config, conditional waiver, and scheme
      properties from a file previously generated with "check_cdc -export
      -to_formal".

Bottom-Up Hierarchical CDC
--------------------------
-hierarchical ( (-load [-directory directory_name] [-instance instance_name]
                  -file_name file_name_or_pattern_list )
                |(-report report_file_name -info database_file_name )
                |(-get_version -file file_name )
                |(-generate [-module module_name] [-instance instance_name] )
                |-prove )

    Load information from previous analysis, report the version of Jasper
    used to produce the database, or generate consistency checks for the hierarchical
    bottom-up analysis.

    . Use "-load" to load and open database files.
      . Use "-directory" with "-file" to open all database files in the
        directory that match the file name or pattern list.
      . Use "-instance" to specify the name of the blackboxed instance
      . Use "-file_name" to specify file names or patterns that will be loaded from the
        given directory. The specified files must be valid CDC databases generated with
        the "check_cdc -export -database" command.
```

```
. Use "-report" to generate a report from the database specified with the
  "-info" switch.

. Use "-get_version" with "-file" to report the version of the database
  used to produce the specified file.

. Use "-generate" to automatically generate properties based on the block-level
  properties loaded with the hierarchical database.
  . Use "-module" to specify the module name for which the properties will
    be generated.
  . Use "-instance" to specify the instance name for which the properties
    will be generated.

NOTE:
. Environmental assumptions defined at block level will generate top-level
  assertions (assume-guarantee flow).
. Conditional waivers with the non-overlapping (|=>) or the overlapping
  (|->) implication operator, will generate top-level covers based on the
  expression's precondition.

. Use "-prove" to prove all properties previously generated with the
  "generate" command.

NOTE:
. If a hierarchical assertion fails, an "invalid_assumption" violation
  is generated.
. If a hierarchical cover fails, an "invalid_precondition" violation is
  generated.
. If a hierarchical property cannot be proven due to the presence of a
  signal in the expression that is unknown to the current design (internal
  to the block), an "internal_property" violation is generated.

Important things to know about bottom-up hierarchical CDC:

The tool uses the information you provide to create automatic abstractions
for the blackboxed instances of the specified module. It also checks the
consistency between the imported database and the current setup and
generates integration violations for any problems detected.

For parameterized designs, if the instance parameters used at the top level
do not match the parameters used during the IP level CDC analysis, the tool
issues an error when you attempt to load the hierarchical database at the
```

```
    top level.

    To fix this error, do one of the following:

    . Provide the matching parameter values while loading the database.
    . When different instances of the same module use different parameter
      values, generate multiple databases with appropriate parameter values
      for use at the top level.

Reporting CDC App Setup
---------------------------------------
-report -setup
    [only (-design_statistics |-tool_configuration |-design_configuration)+]
    [-file file_name [-force]
      [ [-html [-launch_html_browser]
          [-source_path_mapping paths_mapping]
          [-relative_file_path]]
      |-xml |-csv]]

    Generate a setup report.

    Setup reports consist of three sections, which are Design Statistics,
    Tool Configuration, and Design Configuration.
    When you use this command without additional arguments, the tool prints a
    setup report containing all sections to the console.

    The three sections contain the following information:
      . Design Statistics: statistics about the elaborated design, reported in
        the following tables:
        . Design Size: the number of gates and the number of flops.
        . Black Boxes: each blackboxed module and its blackbox type/reason.
        . Liberty Modules: the module/cell names loaded from Liberty(TM)
          libraries and their source file names.
        . Design Parameters: the parameter names and values of the top module.
      . Tool Configuration: the rules and settings in effect for the current
        CDC session, reported in the following sub-sections:
        . Rule Configuration:
          . Enabled Rules Summary: for each enabled category, the number of
            enabled error, warning, and info tags.
          . Enabled Rules Detailed: each enabled tag, its alias, and its
            severity.
        . Parameter Configuration:
          . Rule Parameters: every parameter value coming from the rule file
```

```
            used in the current CDC session. Each row reports the parameter
            name, the scope in which it applies (Global, a specific domain, or
            a specific tag), and its current value.
          . Tool Settings: the CDC application configuration keys and their
            current values.
        . Design Configuration: the user-provided design setup, reported in the
          following tables and sub-sections:
          . Declared Clocks: the declared clock signals, their clock names,
            source, and the command that declared them.
          . SDC Command Summary: the SDC commands processed and their status.
          . Declared Resets: the declared reset signals.
          . Applied Constraints: the signal configurations (constant, static,
            exclusive, and gray-coded signals), and, in a detailed report, false
            paths, externally synchronized signals, clock rating of ports,
            structural cells, user-defined synchronizers, reset order, and reset
            rating of ports.

      . Use "only" to include specified sections only.
        . Use "-design_statistics" to include the Design Statistics section.
        . Use "-tool_configuration" to include the Tool Configuration section.
        . Use "-design_configuration" to include the Design Configuration section.
      . Use "-file" to specify a file name. And use "-force" to overwrite an
        existing file.
        . Use "-html" to report results as HTML.
          . Use "-launch_html_browser" to launch an HTML browser with results.
          . Use "-source_path_mapping" to specify the mapping between UNIX(R) and
            Microsoft(R) Windows(R) when using Microsoft(R) Internet Explorer(R).
            To link directly to the source files, the mapped paths should
            be separated with the character '='; for example, /home/user_name=z:
          . Use "-relative_file_path" to open the source files of the relative
            file path in the HTML report. The same relative file path is used in
            the elaboration.
        . Use "-xml" to report results as XML.
        . Use "-csv" to report results as CSV.
        . If you do not specify a format, the default is text.
        NOTE: If you do not specify a file, the report is printed to the console.

Reporting CDC App Checks
------------------------------
-report
    [-property
        [-status (  cex | unreachable | proven
                  | covered | undetermined | unprocessed)+]
```

```
    [-type (assert | cover)+]
    [-task task_list]]
[-violation
    [-severity (error | warning | info)+]
    [-order (  category | tag | filename | instance(module) | label
             | severity | mode)]
[-detailed]
[-domain domain_list]
[-category category_list]
[-tag tag_list]
[-instance instance_list]
[-regexp]
[-waived (none | only [-hierarchy |-imported |-user_tcl |-auto]+)]
[-grouping (none | basic | advanced)]
[-file file_name [-force]
  [ [-html [-launch_html_browser]
      [-source_path_mapping paths_mapping]
      [-relative_file_path]]
  |-xml |-csv]]

Generate a checks report.

When you use this command without additional arguments, the tool prints a
violations summary to the console.

. Use "-property" to generate a properties report.
  . Use "-status" to generate a report for the specified statuses only. The
    default is all.
  . Use "-type" to generate a report for the specified types only. The
    default is all.
  . Use "-task" to generate a report for the specified tasks only. The
    default is all. This switch supports regular expressions.
. Use "-violation" to generate a violations report.
  . Use "-severity" to generate a report for the specified violation
    severities only. The default is all.
  . Use "-order" to generate a report ordered by the specified criteria.
    The default is "category".
    NOTE: Use "mode" to generate a report sorted by modes of
    violations. This is applicable only in the Multimode Summary View.
  . Use "-include_design_build" to include violations of design build.
  NOTE: Running the violations report command in the Multimode Summary View
  generates a report that includes all violations and their modes.
. Use "-domain" to generate a report for the specified domains.
```

```
    . Use "-category" to generate a report for the specified categories.
    . Use "-tag" to generate a report for the specified tags.
    . Use "-instance" to generate a report for the specified instances.
    . Use "-regexp" to consider the category, tag, or instance name a regular
      expression.
    . Use "-waived only" to generate a report for waived checks that were
      waived by waivers of the specified types only. If no waiver type is
      specified, a report of all waived checks will be generated.
    . Use "-waived none" to exclude waived checks from the report. By default,
      waived checks are included in the report.
    . Use "-grouping none" to generate a report with no grouping information.
    . Use "-grouping basic" to generate a report where only same tags can be
      grouped together, reporting only the primary.
    . Use "-grouping advanced" to generate a report where violations from
      different tags can be grouped together, reporting only the primary.
    NOTE: Grouping is a GUI concept that groups related checks and shows only
    one check, which is the primary. By default, the report is generated with
    the app's current grouping option.
    . Use "-detailed" to generate a detailed report. By default, the tool
      generates a summary report only.
    . Use "-file" to specify a file name. And use "-force" to overwrite an
      existing file.
      . Use "-html" to report results as HTML.
        . Use "-launch_html_browser" to launch an HTML browser with results.
        . Use "-source_path_mapping" to specify the mapping between UNIX(R) and
          Microsoft(R) Windows(R) when using Microsoft(R) Internet Explorer(R).
          To link directly to the source files, the mapped paths should
          be separated with the character '='; for example, /home/user_name=z:
        . Use "-relative_file_path" to open the source files of the relative
          file path in the HTML report. The same relative file path is used in
          the elaboration.
      . Use "-xml" to report results as XML.
      . Use "-csv" to report results as CSV.
      . If you do not specify a format, the default is text.
      NOTE: If you do not specify a file, the report is printed to the console.

Reporting CDC App Waivers
-----------------------------------------
-report -waiver
    [only (-hierarchy |-imported |-user_tcl |-auto)+]
    [-waiver_status (  safe | unsafe | unvalidatable | undetermined
                     | unmatched | mixed)+]
    [-detailed]
```

```
    [-file file_name [-force]
      [ [-html [-launch_html_browser]
          [-source_path_mapping paths_mapping]
          [-relative_file_path]]
      |-xml |-csv]]

    Generate a waivers report.

    When you use this command without additional arguments, the tool prints
    to the console a waivers report containing a summary of all waivers in
    addition to one section for every waiver type that contains information of
    each waiver of this type.

    . Use "only" to include waivers of the specified types only. By default,
      waivers of all types are included.
    . Use "-waiver_status" to include waivers with the specified waiver
      statuses only. By default, waivers with all statuses are included.
    . Use "-detailed" to add the checks that are waived by this waiver beneath
      every waiver's information. By default, waived checks are not added.
    . Use "-file" to specify a file name. And use "-force" to overwrite an
      existing file.
      . Use "-html" to report results as HTML.
        . Use "-launch_html_browser" to launch an HTML browser with results.
        . Use "-source_path_mapping" to specify the mapping between UNIX(R) and
          Microsoft(R) Windows(R) when using Microsoft(R) Internet Explorer(R).
          To link directly to the source files, the mapped paths should
          be separated with the character '='; for example, /home/user_name=z:
        . Use "-relative_file_path" to open the source files of the relative
          file path in the HTML report. The same relative file path is used in
          the elaboration.
      . Use "-xml" to report results as XML.
      . Use "-csv" to report results as CSV.
      . If you do not specify a format, the default is text.
      NOTE: If you do not specify a file, the report is printed to the console.

Reporting CDC App Signoff
-----------------------------------------
-report -signoff
    [-file file_name [-force]
      [ [-html [-launch_html_browser]
          [-source_path_mapping paths_mapping]
          [-relative_file_path]]
      |-xml |-csv]]
```

```
    Generate a signoff report.

    Signoff reports include the design statistics, the signoff information
    (delta of used settings and rule file with the default settings and rule
    file) of the tool configuration, the design configuration, and summaries of
    the violations, waived violations and waivers.
    When you use this command without additional arguments, the tool prints
    the signoff report to the console.

    . Use "-file" to specify a file name. And use "-force" to overwrite an
      existing file.
      . Use "-html" to report results as HTML.
        . Use "-launch_html_browser" to launch an HTML browser with results.
        . Use "-source_path_mapping" to specify the mapping between UNIX(R) and
          Microsoft(R) Windows(R) when using Microsoft(R) Internet Explorer(R).
          To link directly to the source files, the mapped paths should
          be separated with the character '='; for example, /home/user_name=z:
        . Use "-relative_file_path" to open the source files of the relative
          file path in the HTML report. The same relative file path is used in
          the elaboration.
      . Use "-xml" to report results as XML.
      . Use "-csv" to report results as CSV.
      . If you do not specify a format, the default is text.
      NOTE: If you do not specify a file, the report is printed to the console.

Objects List
------------
-list ( domains | filters | rules | user_defined_schemes | pairs
        | scheme_properties | violations | violation_groups | signoff
        | waivers | signal_config | reset | groups | schemes | domain_crossings
        | units | clock_signals | inactive_pairs | inactive_reset_pairs
        | all_reset_pairs | clock_groups | clock_matrix | clock_synchronicity
        | setup | ports | reset_paths | reset_definitions | declared_resets
        | reset_schemes | reset_schemes_and_rejected_user_defined | ipqc
        | ipqc_no_clock_matrix | reset_order | rdc_pairs | boundary_info
        | virtual_clocks | icg_cells | virtual_resets | glitch_free_cells
        | false_paths | convergence_schemes | clock_sense
        | inactive_destinations | user_properties | hier_db_info | timing_arcs
        | design_resets | waiver_reasons )
      [-filter filter_id] [-include_indexes]
      [-file file_name [-force]]
```

```
Returns a Tcl dictionary that lists the detected objects for the specified
type for the current CDC setup.

. Use "-file" to generate a report of the specified type in CSV format for
  the current setup. And use "-force" to overwrite an existing file.
  NOTE: This command appends a .csv extension to file_name if it does not
  already have one.
. Use "-include_indexes" to include signal indexes for source_unit and
  destination_unit of pairs.
. Use "-filter" to list only the IDs of the objects related to the
  specified filter. The filter criteria accepted by each type of object
  are the following:
NOTE: Most of the objects do not accept any filter criteria. The ones
that do follow:
  . Pairs
      . source_clock
      . destination_clock
      . source_unit
      . destination_unit
      . pair_classification
      . scheme
  . Violations
      . category
      . domain
      . tag
      . severity
      . source_unit
      . destination_unit
      . scheme_property
      . scheme_type
      . source_clock
      . destination_clock
  . Signoff
      . tag
      . category
      . domain
      . severity
      . source_unit
      . destination_unit
      . scheme_property
      . scheme_type
      . source_clock
      . destination_clock
```

```
. Scheme Properties
    . scheme_property
    . tag
    . scheme
    . scheme_type
. Domain Crossings
    . source_domain
    . destination_domain
. Groups
    . cdc_group
. Schemes
    . scheme
    . scheme_type
    . tag
. Clock Signals
    . clock_domain
. Clock Matrix
    . clock_domain
. Inactive Pairs
    . source_clock
    . destination_clock
    . source_unit
    . destination_unit
    . pair_classification
. Glitch-free Cells
    . module
. Declared Resets
    . Reset Signal
    . Signal Type
    . Virtual
    . Polarity
. Waiver Reasons
    . Tags
    . Comments
```

Since all information in the GUI is listed, you can create a number of scripts using the "-list" information. For example, suppose "check_cdc -list schemes" returns the following:

```
FSM_block.control_counter_pulse {Status Passed Scheme FSM_block.control_
counter_pulse {Scheme Type} Edge {Detection Type} Automatic Pairs control_
block.control_counter_r-FSM_block.counter_sync.data_meta Violation {}}
control_block.i_manager.msynchronizer.data_sync {Status Failed Scheme
```

```
control_block.i_manager.msynchronizer.data_sync {Scheme Type} MUX_NDFF
{Detection Type} Automatic Pairs {FSM_block.i_code-control_block.i_manager
.msynchronizer.data_sync, FSM_block.i_valid-control_block.i_manager.
msynchronizer.en1} Violation sync_chain_logic}

This result is a Tcl dictionary for which the keys are the scheme names:
. FSM_block.control_counter_pulse
. control_block.i_manager.msynchronizer.data_sync

For each scheme, there is another Tcl dictionary containing the scheme
information as follows: Status, Scheme, {Scheme Type}, {Detection Type},
Pairs and Violation.

To access all the information for a certain scheme, pass the key (scheme
name) as index: dict get $DICTIONARY scheme_name
For example,
  set dictionary [check_cdc -list schemes]
  dict get $dictionary FSM_block.control_counter_pulse
would return
  Status Passed Scheme FSM_block.control_counter_pulse {Scheme Type} Edge
  {Detection Type} Automatic Pairs control_block.control_counter_r-FSM_block
  .counter_sync.data_meta Violation {}

To list all keys, use "dict keys $dictionary", which in this example, would
return the following:
  FSM_block.control_counter_pulse, control_block.i_manager.msynchronizer.data_sync

With the scheme dictionary, you can access any information inside a scheme
providing a key.
For example,
  set scheme_info [dict get $dictionary FSM_block.control_counter_pulse]
  dict get $scheme_info Pairs
would return
  control_block.control_counter_r-FSM_block.counter_sync.data_meta

Also, if you want to count how many "sync_chain_logic" violations the tool
found, you can use the following script:
  set scheme_list [check_cdc -list schemes]
  set schemes {}
  foreach key [dict keys $scheme_list] {
     if {[lsearch [dict get [dict get $scheme_list $key] Violation]
sync_chain_logic] >= 0} {
          lappend schemes $key
```

```
        }
      }
      puts $schemes

    Another way to list specific objects is to use "-filter". For instance,
    if you want to remove all schemes of type "edge", you can create a filter
    as follows,
      set filter_id [check_cdc -filter -add -scheme_type edge]
    and then use the filter to list only "edge" schemes:
      set edge_schemes [check_cdc -list schemes -filter $filter_id]

    The result would be as follows:
      FSM_block.control_counter_pulse {Status Passed Scheme FSM_block.control_
      counter_pulse {Scheme Type} Edge {Detection Type} Automatic Pairs control_
      block.control_counter_r-FSM_block.counter_sync.data_meta Violation {}}

-list ( ipqc [-no_clock_matrix] | ipqc_no_clock_matrix )
      -file file_name [-force]

    Generate the following ipqc reports for the current CDC setup:
      file_name_clockmatrix.csv
      file_name_info.txt
      file_name_error.txt
    . Use "ipqc -no_clock_matrix" or "ipqc_no_clock_matrix" to avoid
      generating the clockmatrix CSV file.
    . Use "-file" to save the report to the specified file.
    . Use "-force" to overwrite an existing file.

Debug
-----
-debug -clock_domain_all_nets (true | false)

    Enables the annotation of the clock domain in the schematic viewer
    for all signals in all debugging windows. The default is true.

-debug -reset_domain_all_nets (true | false)

    Enables the annotation of the reset domain in the schematic viewer
    for all signals in all debugging windows. The default is true.

-debug -color_clock_logic (true | false)

    Enables the coloring of clock logic in the schematic viewer according
```

```
    to which clock domain includes the signal. This command enables the
    coloring of clock logic for all debug windows. The default is false.

-debug -metastability -property property_name

    Returns a Tcl dictionary that lists relevant signals for all clock domain
    crossings in the COI of the property. The keys of the returned dictionary
    are the destination units, and the value of each key is a Tcl list of
    destination clock, "msiw_injection" signal, and "msiw_q_no_msi" signal.

    NOTE: If there is an opened visualize window when the command is invoked, the
    visualize window will be cleared and the signals that suffered from metastability
are
    added to the window.

-debug -show_clock_domain signal_name

    Returns a Tcl dictionary that lists the inferred clock domains for the
    target signal. If different bits of the signal have different
    clock domains, each bit will be a different key in the Tcl dictionary.
    If all bits of the signal have the same clock domains, the full
    signal will be the key in the Tcl dictionary.

    NOTE: Run this command after "check_cdc -extract".

-debug -show_reset_domain signal_name

    Returns a Tcl dictionary that lists the inferred reset domains for the
    target signal. If different bits of the signal have different
    reset domains, each bit is a different key in the Tcl dictionary.
    If all bits of the signal have the same reset domains, the full
    signal is the key in the Tcl dictionary.

GUI
---
-gui -order_by (tag | domain | message | severity | instance(module))

    Use this command to specify the order preference for the violations view.
    The default is "domain".

Show Schematic
--------------
```

Use this command to open a schematic viewer window with CDC or RDC context and draw the top module schematic, its instances, or specific signals.

```
-show_schematic [ -signal {signal_list} [-all |-driver |-port]
                  |-instance instance_name
                  |-path -from signal_name -to signal_name]
                 [-context (cdc | rdc)]
                 [-new_window]
```

When you use "check_cdc -show_schematic" without additional arguments, the tool draws a schematic of the entire loaded design with CDC context.

- Use "-signal" to draw the schematic of the specified signal list on the schematic viewer window.
  - Use "-all" to draw all gates connected to the signals specified by the signal list.
  - Use "-driver" to draw the gates that drive the signals specified by the signal list.
  - Use "-port" to draw the signals that are ports specified by the signal list.
- Use "-instance" to draw a specified instance instead of the top module on the schematic viewer window.
- Use "-path" to show all paths between two sets of signals. If there is no path between the sets of signals,
  the viewer displays the signals themselves and their driving logic.
- Use "-context" to switch between CDC and RDC schematic window context.
- Use "-new_window" to open the schematic in a new window.
  NOTE:
  - If you do not specify a context, the default is CDC.
  - RDC context changes how constraints (constant, static, reset) are propagated through the design, changing net color in schematic and changing information displayed in signal info add-in window when selecting a net.

CDC Machine Learning

```
-ml ( -dump_data -dir directory_path
     |-train -dir directory_path -projects {project_directories}
     |-infer -dir directory_path)
```

Use machine learning to improve CDC analysis by training models on existing violation data and inferring waiver suggestions for new designs.

```
. Use "-dump_data" to export violation data in a format suitable for
  machine learning training. The data is saved to the specified directory.
  A directory is considered a project for further "-train" use.
. Use "-train" to train machine learning models using violation data from
  multiple project directories specified by "-project".
  The trained models are saved to the specified directory in "-dir" for
  later use with the "-infer" option.
. Use "-infer" to use previously trained models to suggest waivers for
  current CDC violations. The tool loads models from the specified
  directory and applies them to suggest which violations might be safely
  waived based on learned patterns.
```

Example

```
check_cdc -glitch_free -add -module libcell_and2
check_cdc -scheme -add ndff -map {{data input} {dout output}} -module sync
check_cdc -scheme -add enable_based -map {{Enable en}}
check_cdc -protocol_check -generate
check_cdc -protocol_check -export -file cdc_properties.svp
check_cdc -metastability -inject
check_cdc -waiver -add -reason SEE_COMMENTS -comment "All violations from filter 1 are
safe" -filter 1
check_cdc -report -setup only -design_statistics -design_configuration
check_cdc -report -violation -file filename.csv -csv -grouping none
check_cdc -report -waiver only -auto -user_tcl -hierarchy -waiver_status {safe
unvalidatable} -detailed
check_cdc -report -signoff -file signoff_report.html -html -launch_html_browser
check_cdc -gui -order_by severity
check_cdc -hierarchical -load -dir [get_proj_dir] -file {*.jdb|*.db}
check_cdc -hierarchical -load -file {{/a/b/file_name.db} {/a/b/c/file_name.db}}
check_cdc -show_schematic -context rdc
check_cdc -show_schematic -signal rst -context cdc
check_cdc -show_schematic -instance flop1 -context rdc
check_cdc -show_schematic -path -from rst -to out
```

check_code

Stops command interpretation if actual and expected values do not match.

Syntax

```
check_code actual expected [on_error]
```

Returns

No return values.

Detailed Description

Use this command to check whether an actual value matches the expectations. If it does not, "check_code" returns with an error code and stops command interpretation unless you also use "catch". The tool evaluates the optional "on_error" block before returning with an error code.

This command is useful when creating a regression script.

actual
: Actual value.

expected
: Expected value.

on_error
: A command to evaluate when the actual and the expected values do not match.

Example

```
check_code [ prove ~pci_code.error ] cex

# Adding an "on_error" to be run in case check_code fails.
check_code [ prove -status -silent ] 0.H {prove -stop}

check_code [ join [get_design_info -list fsm -silent] ] "fsm1 fsm2"
```

See Also

check_one_of

check_conn

Prints requested Connectivity information or executes the requested action.

Associated App(s)

Connectivity Verification

Syntax

```
check_conn -init

check_conn -connection connection_name
            -list ( clock | condition | condition_stable | cond_expr
                          | latency |  src_signal | dest_signal | reset | stage
                          | property | assert | cover | path [-limit N] )

check_conn -interface_connection connection_name
            -list (src_signal | dest_signal)

check_conn -list (  connection | interface_connection | interface
                         | interface_map | assume | functional | non_functional )

check_conn -load csv_file_name
               [-filter [-regexp]]
               [-log file_name]

check_conn -validate [connection_name_list] [-include ((condition | disable | clock)+ |
all)]

check_conn -enable connection_name_list

check_conn -disable connection_name_list

check_conn -save_as file_name [-force]

check_conn -prove [-connection connection_name_list |-property property_name_list]

check_conn -connection connection_name -debug (bbox_out | loop_stopat | register)
```

```
check_conn -connection connection_name -debug validate

check_conn -clear

check_conn -generate_template file [-force] [-verbose |-compact]

check_conn -report
           [-file file_name] [-force] [-csv] [-full_name] [-dict]
           [-show_rtl_info] [-sort_by (connection_name | src_block | src_signal
                              | dest_block | dest_signal | classification
                              | src_rtl_info | dest_rtl_info)+]

check_conn -signoff -port
           [-file file_name] [-force] [-csv]
           [-full_name] [-instance instance_name]
           [-sort_by (instance_name | port_name | port_direction
                         | #connections_src | #connections_dest | rtl_info)+]

check_conn -expand_interface_connection
           (interface_connection_tcl_list |-all)

check_conn -generate_toggle_checks connection_name_list

check_conn -reverse
            (  [-src name_list]
               [-dest name_list]
             | [-target name_list] )
            (-load [-append] |-save_as file_name [-force] [-append] |-dict)+
            [-wildcard]
            [-complexity (straightforward | simple | conditional | complex)]
            [-level (down | same | up)]
            [-relative_depth N]
            [-stop_on_target_boundary (on | off)]
            [-radix (2 | 8 | 10 | 16 | bin | oct | dec | hex)]
            [-semicolon_separated]
            [-prefix_name connection_prefix_name]
            [-sort_by (connection_name | src_block | src_signal
                      | dest_block | dest_signal | classification
                      | src_rtl_info | dest_rtl_info)+]
            [-limit N]
            [-max_path_length N]
            [-silent]
```

```
check_conn -load_ipxact
             [spirit:design_name [-instance instance_name]]
             [-interfaces]
             [-infer_interface_definitions]
             [-no_debug]

check_conn -connection connection_name -show_schematic [-limit N] [-annotated]
```

Returns

A list of requested objects or the output of a requested action.

Detailed Description

Use this command to get information from a CSV worksheet that defines properties in the Connectivity format or to request actions related to this worksheet.

This command has two scopes, the loaded worksheet and a specified connection. The default is the loaded worksheet. To specify a connection, use the "-connection" switch.

NOTE: CONN cannot be used in conjunction with SEC, SPV, and X-Prop.

List Switches

-init
  This command enables CONN App handling of Digital Mixed Signal (DMS) types and relevant elaboration settings.

  NOTE:
  . "check_conn -init" sets the values of the following commands:
    . set_extended_inout_port_alias_handling on
    . set_elaborate_bbox_output_driver highz
    . set_elaborate_bbox_inout_simplification off
  . DMS types are synthesized as 1-bit signals at the basic level. Supported types include "real", "wreal", and types present on Cadence Real Number Modeling Package (cds_rnm_pkg.sv) and Cadence Electrical Equivalent package (EE_pkg.sv). EEnet is a structure, each of the three members are synthesized as 1-bit signals. This command also enables the use of "wreal" types inside

```
    SystemVerilog files.

  NOTE:
  . You must use this command before the "analyze" command.
  . You can find additional information regarding supported connections using
    DMS types in the Jasper Connectivity Verification App User Guide
    (Help - Application Guides - Connectivity Verification App User Guide).

-connection connection_name
    -list (  clock | condition | condition_stable | cond_expr | latency
           | src_signal | dest_signal | reset | stage
           | property | assert | cover | path [-limit N] )

    List the information that matches the specified criteria.

    NOTE: The following switches are valid for the loaded worksheet (default
    scope) or a specified connection.

    . Use "-connection" to specify a connection other than the loaded
      worksheet.
    . Use "clock" to list clock information. The result is a pair, clock
      signal and edge, unless the connection is a PIPE connection, in
      which case the result is an ordered list of clock signals for each PIPE.
    . Use "condition" to list all CONDITION clauses defined in the worksheet or
      for the specified connection.
    . Use "condition_stable" to list all CONDITION(STABLE) clauses defined in
      the worksheet or for the specified connection.
    . Use "cond_expr" to list all COND_EXPR clauses defined in the worksheet or
      for the specified connection. If the connection is a PIPE connection, the
      tool returns an ordered list of pairs of input condition and output
      condition for each PIPE.
    . Use "latency" to list all LATENCIES clauses defined in the worksheet or
      for the specified connection.
    . Use "src_signal" to list source signals defined in the worksheet or for a
      specified connection.
    . Use "dest_signal" to list destination signals defined in the worksheet or
      for a specified connection.
    . Use "reset" to return an ordered list of reset signals defined in the
      worksheet for a specified PIPE connection.
    . Use "stage" to return an ordered list of the stages defined in the
      worksheet for a specified PIPE connection.
    . Use "property" to list all properties generated by Connectivity from the
      loaded worksheet or for a specified connection.
```

```
    . Use "assert" to list all assertions generated by Connectivity from the
      loaded worksheet or for a specified connection.
    . Use "cover" to list all precondition cover directives generated by
      Connectivity for assertions.
    . Use "path" to list paths between the source and destination of a
      connection. If multiple paths exist, the tool returns a list of up to
      ten paths by default. To modify the default limit, use the "-limit N"
      switch, where "N" is the maximum number of paths you want listed.

      SPECIAL CASE: Running "check_conn -reverse ... -load" on connections
      prior to running "check_conn -connection connection_name -list path"
      on the specified connections causes
      "check_conn -connection connection_name -list path -limit 1"
      to return the single path based on CONDITION/COND_EXPR.

-interface_connection connection_name
    -list (src_signal | dest_signal)
    List the information that matches the specified criteria.

    NOTE: The following switches are valid for the specified interface
    connection. The source and destination signals are listed in the same order.

    . Use "src_signal" to list the signals defined for the source interface.
    . Use "dest_signal" to list the signals defined for the destination
      interface.

-list (  connection | interface_connection | interface
       | interface_map | assume | functional | non_functional )
    List the information that matches the specified criteria.

    NOTE: The following switches are valid for the worksheet scope. They do not
    support the "-connection" scope.

    . Use "connection" to list all CONNECTION names defined in the worksheet.
    . Use "interface_connection" to list all INTERFACE_CONNECTION names defined
      in the worksheet.
    . Use "interface" to list all INTERFACE names defined in the worksheet.
    . Use "interface_map" to list all INTERFACE_MAP relationships defined in
      the worksheet.
    . Use "assume" to list all assumptions generated by Connectivity from the
      loaded worksheet.
    . Use "functional" to list functional connections. Functional connections
      have been validated (the source signal is in the cone of influence of
```

```
      the destination signal) and all of their related properties are "proven"
      or "covered".
    . Use "non_functional" to list connections that are not valid (have a
      failed validation status) or have one or more of their related properties
      with status "unreachable" or "CEX".

    NOTE: Run "check_conn -validate" and "check_conn -prove" before listing
    "functional" and "non_functional" connections; otherwise, the result
    might be empty or incomplete.

Action Switches
---------------
-load csv_file_name
      [-filter [-regexp]]
      [-log file_name]
    Load the Connectivity map present in the CSV workbook file and the
    properties generated from its specification.

    . Use "-filter" to specify that you want the tool to consider wildcards for
      source or destination signal inputs. And use "-regexp" to consider
      regular expression matching.
      . Wildcards are allowed in...
        . The source or destination signal in a CONNECTION row
        . The source or destination interface instance name in
          an INTERFACE_CONNECTION row
      . You can only specify wildcard and regular expression filtering for
        a source *or* destination, not for both sides.
      . You can specify any of the following reserved constants in the opposite
        side of a wildcard specification:

        Constant     Definition                Applies to
        --------     ----------                ----------
        JDA:LOW      All bits of each matched  CONNECTION, INTERFACE_CONNECTION
                     signal are tied to zero
        JDA:HIGH     All bits of each matched  CONNECTION, INTERFACE_CONNECTION
                     signal are tied to one
        JDA:SAME     Signal names are the      CONNECTION
                     same in both sides
        JDA:DEFAULT  Each matched signal is    INTERFACE_CONNECTION
                     tied to the default
                     value specified in the
                     interface map
    . Use "-log" to print output messages to a file.
```

```
      NOTE: Use this switch when the load operation generates more than 500
      warnings or errors and you want to see them all.


    Important things to know:
    . "-load" also loads the GUI.
    . These switches are not valid for the connection scope.
    . You can load multiple Connectivity maps in the same setup in an analysis
      session.
    . The Connectivity App generates expressions in Verilog syntax. To use
      "check_conn -load", you must use "elaborate -mode verilog" when
      elaborating a design written in VHDL.
    . When specifying values in CSV format, do not use spaces between a comma
      and the double quote used to enclose a field as this causes unintended
      parsing.

      Examples:
      # The following causes a parsing error:
      COND_EXPR, "bridge_addr.bridge_i.fsm_tx[1]"

      # The following produces the desired results:
      COND_EXPR,"bridge_addr.bridge_i.fsm_tx[1]"

-validate [connection_name_list] [-include ((condition | disable | clock)+ | all)]
    Check, for each connection, if the source is in the cone of influence of
    the destination, and update the Connections Table to indicate the result.
    This command runs for all connections, unless you provide a connection list.
    This is a sanity check and should be the first action performed after
    loading a connectivity map.

    Use "-include" to include the condition, disable, clock, or all of these
    signals in the cone of influence of the destination.

-enable connection_name_list
    Enable one or more connections in the Connections Table. By default, all
    connections are enabled when they are loaded. When a connection is
    enabled, all of its related properties are enabled in the Property Table.
    Only enabled connections are exported with the "-save_as" subcommand.

    NOTE: To enable connections based on wildcard or regular expression patterns
    for connection names, you can use the Tcl "lsearch" command to filter the
    connection list.
```

```
    . Wildcard search example:
      set wildcard_expression {wildcard_expression}
      set conn_list [check_conn -list connection]
      check_conn -enable [lsearch -inline -all $conn_list $wildcard_expression]

    . Regular expression search example:
      set regexp_expression {regexp_expression}
      set conn_list [check_conn -list connection]
      check_conn -enable [lsearch -inline -all -regexp $conn_list $regexp_expression]

-disable connection_name_list
    Disable one or more connections in the Connections Table. When a connection
    is disabled, all of its related properties are disabled in the Property
    Table. Disabled connections are not exported when you use the "-save_as"
    subcommand.

    NOTE: To disable  connections based on wildcard or regular expression
    patterns for connection names, you can use the Tcl "lsearch" command to
    filter the connection list.

    . Wildcard search example:
      set wildcard_expression {wildcard_expression}
      set conn_list [check_conn -list connection]
      check_conn -disable [lsearch -inline -all $conn_list $wildcard_expression]

    . Regular expression search example:
      set regexp_expression {regexp_expression}
      set conn_list [check_conn -list connection]
      check_conn disable [lsearch -inline -all -regexp $conn_list $regexp_expression]

-save_as file_name [-force]

    Save enabled connections loaded into the Connectivity App to the specified
    file.

    This command saves connections in expanded format, as displayed in the
    Connections Table.

    Use "-force" if the specified file already exists and you want to overwrite
    it.

-prove [-connection connection_name_list |-property property_name_list]
    Prove all properties generated by the Connectivity App.
```

```
    . Use "-connection" to limit the proof to properties generated for the
      specified connections.
    . Use "-property" to limit the proof to the specified properties in the
      Connectivity task.

    NOTE:
    . The session-specific commands, such as "visualize -violation", "visualize
      -cover", and "get_status" do not return the data from this analysis.
    . The "set_cmd_time_limit" command has no effect on "-prove"; instead, it
      honors prove time-limit commands.

-connection connection_name -debug (bbox_out | loop_stopat | register)
    Debug a connection after "check_conn -prove". The command returns a Tcl
    dictionary in which properties are keys and, for each one of them, the
    value is a list of signals.

    . Use "bbox_out" or "register" to list blackbox outputs or register ports
      in the trace of a failing assertion related to the specified connection.
    . Use "loop_stopat" to list the stopat signals applied when breaking
      combinational loops in the cone of influence of the connection properties.

-connection connection_name -debug validate
    Use "validate" to report the status of the "check_conn -validate" command
    for the specified connection and, if the status is "Outside COI", it will
    return a list containing all signals not found during COI validation. If all
    signals are inside COI, it will return the status "Inside COI" and an empty
    list of signals. If the connection is not processed, it will return the
    status "unprocessed" and an empty list of signals.

    NOTE:
    . check_conn -validate must be executed before debugging the validate status.

-clear
    Remove the Connectivity properties from the current connectivity map(s) and
    the "Connectivity" task created by the "check_conn" command.

    NOTE:
    . This switch stops periodic updates to existing databases.

-generate_template file [-force] [-verbose |-compact]
    Generate a CSV template in which to complete the connectivity map.
```

```
    . Use "-force" to overwrite the existing file.
    . Use "-verbose" to generate a verbose template, which includes column
      headers for the conditions. This is the default setting.
    . Use "-compact" to generate a compact template, which prints the
      conditions directly beneath the connection.

-report  [-file file_name] [-force] [-csv] [-full_name] [-dict]
    [-show_rtl_info] [-sort_by (connection_name | src_block | src_signal
                      | dest_block | dest_signal | classification
                      | src_rtl_info | dest_rtl_info)+]
    Use this command to report connectivity table information with combined
    proof status of all the properties related to a particular connection.
    This report contains a list of all connections, except disabled ones, names
    (with source and destination information) and their proof status. If you do
    not include any options with this command, the tool prints out a header and
    a table of results on the console.

    . Use "-file" to save the report to a specified file instead of displaying
      it on the screen.
    . Use "-force" if the specified report file already exists and you want to
      overwrite it.
    . Use "-csv" to generate the report as comma-separated values (CSV).
    . Use "-full_name" to generate the report with block name appended to
      signals.
    . Use "-dict" to return the generated reports as a Tcl dictionary, where
      the keys are the connection names and the values are nested Tcl
      dictionaries containing the connection's name, source block, source
      signal, destination block, destination signal, proof status, toggle
      status, COI status, classification, and RTL information for both source
      and destination.
    . Use "-sort_by" to sort the listed connections by one or more attributes.
    . Use "-show_rtl_info" to display the RTL file and line of a signal's
      declaration.
      NOTE: "-show_rtl_info" is an initial release to gather feedback from
      early adopters and finalize implementation for an upcoming release.
      Contact support@cadence.com to provide feedback or comments.

-signoff -port
    [-file file_name] [-force] [-csv]
    [-full_name] [-instance instance_name]
    [-sort_by] (instance_name | port_name | port_direction
                     | #connections_src | #connections_dest | rtl_info)+]
```

Use this command to report connections information for each instance port. This report contains a list of all connections, except disabled ones, (with their complexity) for each instance port, including its name, direction, instance name, and number of connections in which the given port acts as source and destination. If you do not include any optional switches with this command, the tool prints a header and a table of results on the console.

The signoff report helps you understand how ports are referenced in each connection, but more importantly, it tells you whether there are ports of an instance that are not referenced in any connection. This enables you to identify changes in RTL where ports might have been added, but no connections were specified in the connection spec.

. Use "-file" to save the report to a specified file instead of displaying it on the screen.
. Use "-force" if the specified report file already exists, and you want to overwrite it.
. Use "-csv" to generate the report as comma-separated values (CSV).
. Use "-full_name" to generate the report with block name appended to signal names.
. Use "-instance" to get a list connections for the specified instance. The signoff report helps you understand how ports are referenced in each connection, but more importantly, it tells you whether there are ports of an instance that are not referenced in any connection. This enables you to identify changes in RTL where ports might have been added, but no connections were specified in the connection spec.
. Use "-sort_by" to sort the listed connections by one or more report columns.

NOTE: See the "Jasper Connectivity Verification App User Guide" for additional information and a usage example (Help - Application Guides).

-expand_interface_connection (interface_connection_tcl_list |-all)
Expand interface connections, creating properties for each pair of signals.

Use "-all" to expand all interface connections. Otherwise, provide a list of interface connections. This command does not accept an empty list.

-generate_toggle_checks connection_name_list
IMPORTANT: This switch is not required to validate connections and is no longer recommended. However, you can still use it to understand whether signals are tied off in the environment.

```
    Generate toggle checks for the specified connections. To generate toggle
    for all checks, specify "{}" as the connection_name_list.
    Example: check_conn -generate_toggle_checks {}

    NOTE: Toggle checks are environment cover properties that ensure that
    each source signal bit will change value (from 0 to 1 and from 1 to 0).

Reverse Connectivity Switches
-----------------------------
-reverse
    (  [-src name_list]
       [-dest name_list]
     | [-target name_list] )
    (-load [-append] |-save_as file_name [-force] [-append] |-dict)+
    [-wildcard]
    [-complexity (straightforward | simple | conditional | complex)]
    [-level (down | same | up)]
    [-relative_depth N]
    [-stop_on_target_boundary (on | off)]
    [-radix (2 | 8 | 10 | 16 | bin | oct | dec | hex)]
    [-semicolon_separated]
    [-prefix_name connection_prefix_name]
    [-sort_by (connection_name | src_block | src_signal
              | dest_block | dest_signal | classification
              | src_rtl_info | dest_rtl_info)+]
    [-limit N]
    [-max_path_length N]
    [-silent]

    Extract connections from the design using structural analysis, optionally
    loading them in the Connections Table in GUI. You have the option to
    select a path to save the connectivity map.

    Connections are organized in four categories depending on
    the gates that the connection can go through:
    . straightforward - buffers
    . simple          - buffers, flops, and inverters
    . conditional     - buffers, flops, inverters, latches, muxes, AND gates,
                        and inouts
    . complex         - any gate

    To guide the extraction, you must provide either a list of sources, a
    list of destinations, or both. An alternative approach is to provide a
```

list of targets. Each of these arguments is a Tcl list that can contain signal names, instance names, and module names. For instances and modules, the extraction starts with their inputs, outputs, and inouts. The most common use models follow:

. Provide instances or modules as arguments to search for connections between their ports in the same hierarchical depth, without searching inside them.
. Provide signals as arguments to search for connections in the same hierarchical depth.

The following switches define the direction of the extraction:

. Use "-src" to look for connections leaving the specified source and arriving at any destination.
. Use "-dest" to look for connections arriving at the specified destination from any source.
. Use "-src" and "-dest" to look for connections between the specified source(s) and destination(s).
. Use "-target" to provide a list of targets. When you provide a target, the tool searches all connections leaving or arriving at this target.

The following switches define the action to be performed:

. Use "-load" to open the generated connectivity map directly in the Connections Table in GUI. And use "-append" to append connections to the map.
. Use "-save_as" to specify the name of the file where the connectivity map should be saved.
  . Use "-force" to overwrite any existing data in the file.
  . Use "-append" to append connections to the specified file.
. Use "-dict" to return the generated connections as a Tcl dictionary, where the keys are the connection names and the values are nested Tcl dictionaries containing the connection's source signal, destination signal, condition expression, disable expression, latencies, and clock.

The optional switches function as follows:

. Use "-wildcard" to enable wildcard matching for the source, destination, or target.
. Use "-complexity" to choose a category between "straightforward", "simple", "conditional" and "complex". The default value for this switch is "conditional".
. Use "-level" to limit the extraction by specifying the direction the

```
  algorithm should follow.
  . down - Guides the extraction to inspect only subinstances.
  . same - Both source and destination should be either signals
           of the same instance or signals of sibling instances.
  . up   - Guides the extraction to inspect only instances directly above.
  NOTE: You cannot use "-level" if you use both "-src" and "-dest".
. Use "-relative_depth" to limit the extraction by stopping the search for
  a connection once the search algorithm exceeds the relative hierarchical
  depth specified from the target provided. For example,
  use "-relative_depth 1" to find connections in direct subinstances or the
  parent instance. The default value for this switch is 0, meaning that the
  algorithm only searches connections in the same hierarchical depth.
. Use "-stop_on_target_boundary" to prevent reverse from searching inside
  modules and instances used as source and/or destination. The default
  value for this switch is "on", meaning that the algorithm does not search
  connections inside the provided targets.
. Use "-radix" to return number literals in the specified radix for the
  saved connectivity map file.
. Use "-semicolon_separated" to generate a semicolon-separated value file.
  The default format is comma-separated value (CSV) file.
. Use "-prefix_name" to specify a prefix for the connection names. If no
  prefix is specified, the tool uses the default prefix "c_".
. Use "-sort_by" to sort the listed connections by one or more attributes.
. Use "-limit" to specify the maximum number of connections the command will
  search for. The analysis stops when the tool finds N connections. The default
  value for this switch is $, which means that the algorithm searches for an
  unlimited number of connections.
. Use "-max_path_length" to limit the length of the path of the extracted
  connections. The value represents the number of signals traversed by the
  algorithm and must be greater than 1. By default, the length of extracted
  paths is unlimited.
. Use "-silent" to suppress the extraction summary.

Important things to know:
. The "-src", "-dest", and "-target" arguments accept Tcl lists of
  signal, module, or instance names. Because these arguments are parsed
  as Tcl lists, special characters in names (such as square brackets,
  braces, and backslashes) must be properly quoted or escaped according
  to Tcl list quoting rules.
. When using "-wildcard" with signal names that contain square brackets
  (e.g., "inst[0].sig"), you must apply two levels of escaping:
  1. The first level of escaping is needed for Tcl list parsing of the
     argument.
```

```
      2. The second level is needed to protect the literal brackets from
         being interpreted as wildcard character classes by the glob-style
         pattern matcher.
      For example, to match signals under "inst[0]" with a wildcard:
        check_conn -reverse -src {inst\\[0\\].sig*} -wildcard -load
      Alternatively, you can use a variable or "regsub" to prepare the
      escaped name:
        set escaped_name [regsub -all {([\[\]])} $name {\\\1}]
        check_conn -reverse -src $escaped_name -wildcard -load
      NOTE: Fully qualified names without wildcards do not require this
      double escaping. The extra escaping is only necessary when "-wildcard"
      is used and the names contain glob-special characters such as
      "[", "]", "*", or "?".
    . Reverse Connectivity supports generating expressions in Verilog only.
      Thus, if you are using VHDL, you must elaborate the design with the
      "-mode verilog" option.
    . Reverse Connectivity can only generate expressions that are valid in the
      Jasper Tcl interpreter. Thus, PSL properties and a few other complex
      structures are not supported as condition/clock/reset expressions.

IP-XACT-Related Switches
------------------------
-load_ipxact
             [spirit:design_name [-instance instance_name]]
             [-interfaces] [-infer_interface_definitions] [-no_debug]
    Generate a Connectivity map based on connections found in spirit:design_name
    and load it into the Connectivity App. You must load the corresponding
    IP-XACT specifications through "ipxact -load" before running
    "check_conn -load_ipxact". See also "ipxact -generate_connectivity_map".

    The resulting map has interface definitions and interface connections.

    . Use "-instance" when spirit:design_name is implemented by instance_name.
      Otherwise, the tool assumes that the top module implements
      spirit:design_name.
    . Use "-interfaces" to generate a map without connections. The map
      contains only interface definitions and port maps.
    . If you do not specify spirit:design_name, the tool generates connectivity
      information for all <spirit:component> elements loaded. Connections are
      only generated for <spirit:component> elements with corresponding
      <spirit:design> elements.
    . Use "-infer_interface_definitions" to generate INTERFACE_MAP definitions for
      missing <spirit:abstractionDefinition> elements.
```

```
    . Use "-no_debug" to load the map without any reference to the IP-XACT file
      names and line numbers.
    . The Connectivity App generates expressions in Verilog syntax. To use
      "check_conn -load_ipxact", you must use "elaborate -mode verilog" when
      elaborating a design written in VHDL.

Schematic Switches
------------------
-connection connection_name -show_schematic [-limit N] [-annotated]
    Run the Jasper Apps Schematic Viewer and show the path between source
    and destination signals.

    . Use "-limit" to specify the maximum number of paths you want shown when
      multiple paths exist. By default, the schematic shows up to ten paths.
    . Use "-annotated" to add signal annotations to the schematic. This switch
      applies only if the assertion related to the connection failed with a
      counterexample. Signal values come from the trace of the CEX.
```

Example

```
check_conn -reverse -src [list instA {\signal-1 }] -dest {instB} -relative_depth 2 -
load -sort_by {dest_block dest_signal src_block src_signal}
check_conn -clear
check_conn -reverse -src top_module -relative_depth $ -complexity simple -save_as
conn_map.csv -force -load -sort_by {src_rtl_info dest_rtl_info}
check_conn -reverse -target {instC} -load -append
check_conn -reverse -dest {inst_*.in1} -wildcard -max_path_length 50 -
stop_on_target_boundary off -dict
check_conn -reverse -src {block\\[0\\].*} -wildcard -load
check_conn -connection conn1 -list condition
check_conn -list connection
check_conn -connection conn1 -list assert
check_conn -list assert
check_conn -load conn.csv
check_conn -connection conn1 -prove
check_conn -prove
check_conn -validate
check_conn -validate {conn1 conn2}
check_conn -validate -include {disable clock}
```

See Also

blackbox_assistant,
ipxact

check_cov

Computes, manages, and reports proof core and stimuli coverage.

Syntax

```
check_cov -init
          [-model (all | (statement | branch | expression | functional | toggle |
fsm)+)]
          [-type (all | stimuli | coi | proof
              | proof_core_high_precision | bound | mutation)+]
          [-covdut {module_name}]+
          [-covfile CCF file]*
          [-expr_type (all | bitwise | conditional | relational | reduction )+]
          [-expr_coverable_statements (all | procassign | contassign )+]
          [-expr_successive_xor_limit N]
          [-max_expr_terms N]
          [-expr_struct_scoring]
          [-vlog_short_circuit]
          [-no_vhdl_shortcircuit]
          [-enable_prove_based_proof_core]
          [-create_cover_item_property]
          [-proof_core_include_failing]
          [-enable_cover_item_traces]
          [-skip_ternary_branches]
          [-skip_async_blocks]
          [-include_instance {instance_list} [-regexp]]+
          [-exclude_instance {instance_list} [-regexp]]+
          [-include_module {module_list}] [-regexp]]+
          [-exclude_module {module_list}] [-regexp]]+
          [-exclude_cover_expression_calculation]
          [-include_x_cover_items]
          [-exclude_empty_branch_cover_items]
          [-include_celldefine_cover_items]
          [-enable_vy]
          [-exclude_bind_hierarchies]
          [-include_assume_coi]
          [-include_sim_align_cover_items]
          [-toggle_no_ports]
          [-toggle_ports_only]
```

```
          [-toggle_sv_mda N]
          [-toggle_sv_mda_of_struct]
          [-toggle_sv_generate]
          [-covergroup_per_instance_default_one]
          [-statement_type (blocking | non_blocking)]
          [-toggle_type (flop | latch | port)]
          [-toggle_mutation_type (stopat | stuckat)]
          [-include_assign_scoring]
          [-toggle_sv_modport]
          [-toggle_auto_interface]

check_cov -export [file_name] [-force]

check_cov -import [file_name] [-db_file file_name]+ [-overwrite]

check_cov -measure
          [-type (all | stimuli | coi | proof
              | proof_core_high_precision | bound | mutation)+]
          [-type per_property_checker+]
          [-all |-task task_name+]
          [-property {property_name_list} [-regexp]]+
          [-include_deadcode_assume {assumption_list}]
          [-refresh]
          [-bg]
          [-verbose]
          [-stimuli_engine_mode engine_mode_tcl_list]
          [-checker_engine_mode engine_mode_tcl_list]
          [-iter N]
          [-max_jobs N]
          [-per_engine_max_jobs N]
          [-time_limit time_limit]
          [-per_property_time_limit time_limit]
          [-per_property_time_limit_factor N]
          [-enable_unprocessed]
          [-checker_cover {cover_item_name_list} [-regexp]]+
          [-proof_core_job_ratio percentage]
          [-simplification_only]
          [-skip_covered_or_checked_from_other_tasks]
          [-skip_deadcode]
```

When a coverage measurement is running in the background:

```
check_cov -measure
```

```
    (-stop |-wait)
        [-id index]

check_cov -measure -clear
         [-task {task_list}]

check_cov -extend cover_item_id_list
         (-precondition expression |-postcondition expression)+
         [-task {task_list}]

check_cov -extend cover_item_id_list -clear
         [-task {task_list}]

check_cov -create_cover_item_property
         [-include_hierarchies included_hierarchies]
         [-exclude_hierarchies excluded_hierarchies]
         [-task {task_list}]
         [(-cover_item_id {cover_items_list}) | (-cover_item_name {cover_items_list})]

check_cov -cover_item_tag -add tag_name
    ( ([-source_file file
         [-start_line start_line] [-end_line end_line]
         [-start_column start_column] [-end_column end_column]]
       [-instance {instance_list}]
       [-expression expression])+
       [-type type]
     |-cover_item_id id_list
    )
     [-force]

check_cov -cover_item_tag -remove (-all |-tag tag_name)

check_cov -cover_item_tag -untag (-all |-cover_item_id id_list)

check_cov -cover_item_tag -get -all

check_cov -cover_item_tag -list_matching_cover_items
    (-all |-tag {tag_list})

check_cov -report
                 [-exclude ([const] [assume] [reset] [waived] [unreachable] [deadcode]
                       [disabled] [synthesis_optimized] [constant_input_port])+]
                 [(-task task_list)]
```

```
                  [-model (statement | branch | expression | functional | toggle |
fsm)]
                  [-type (checker | stimuli | formal | bounded)]
                  [-checker_mode (coi | proof_core | proof_core_high_precision |
mutation)]
                  [-no_return]
                  [-silent]
                  [-report_file file_name [-force] [-html]]
                  [-exclude_module {module_list} [-regexp]]
                  [-include_module {module_list} [-regexp]]
                  [-exclude_instance {instance_list} [-regexp]]
                  [-include_instance {instance_list} [-regexp]]
check_cov -list [-status
            (covered | uncovered | unreachable | deadcode | unprocessed
                        | checked | unchecked | undetectable | in_bound | out_bound
                        | covered_and_checked | uncovered_or_unchecked
                        | unreachable_or_undetectable)]
                  [-exclude_module {module_list} [-regexp]]
                  [-include_module {module_list} [-regexp]]
                  [-exclude_instance {instance_list} [-regexp]]
                  [-include_instance {instance_list} [-regexp]]
                  [-bound override_bound]
                  [-exclude ([const] [assume] [reset] [waived] [unreachable] [deadcode]
                        [disabled] [output_coi] [synthesis_optimized]
[constant_input_port]
                        [signal_coi -signal {signal_list}])+]
                  [-include ([case_default]
                        [signal_coi -signal {signal_list}])]
                  [-assert assert_name [-regexp]]
                  [(-task task_list)]
                  [-checker_mode (coi | proof_core | proof_core_high_precision |
mutation)]
                  [-no_return]
                  [-silent]
                  [-model ([branch] [statement] [expression] [functional] [toggle]
[fsm])+]
                  [-cid]

check_cov -get_cover_item_info cover_item_id_list [-task tasks_list]
       ([-name]
        [-expression]
        [-instance]
        [-module]
```

```
        [-file]
        [-source_location]
        [-tag]
        [-prove_status]
        [-deadcode_analysis_status]
        [-exclusions]
        [-bound]
        [-engine]
        [-time]
        [-precondition]
        [-postcondition])

check_cov -get_target_info target_id_list
       ([-name]
        [-prove_status]
        [-bound]
        [-engine]
        [-task]
        [-time]
        [-coi]
        [-proof_core])

check_cov -get_cover_id [-name |-signal |-cover_name cover_property_name]
    [ -source_signal {source_signal_arg_list} [-direction ([fanin] [fanout])+]
     | {target_id_list}]

check_cov -get_assert_id [-name]

check_cov -waivers
                        -import -file_name file_name [-force] [-silent]
                              [-vrefine [-ucdb db_path [-target target -source
source]]]
                        |-export (-file_name file_name)
                              [-id waiver_id_list |-tag waiver_tag_list]
                              [-vrefine [-ucdb db_path [-target target -source
source]]]
                              [-force] [-silent]
                        |-add -comment comment
                              ([-source_file file_name]
                               [-start_line start_line]
                               [-end_line end_line]
                               [-start_column start_column]
                               [-end_column end_column]
```

```
                              [-instance instance]
                              [-expression {expression}]+
                              [-tag tag]
                              [-cover_item_tag tag_name]
                              [-type type]
                              [-persistency persistency]
                              ([-cover_item_id id_list (  [-enclosed_ci_marking]
                                            | [-waive_connected_net])]
                              [(-task task_name] | [-env]))
                             [-force] [-silent]
                        |-remove (-all
                                         |-id waiver_id [-enclosed_ci_marking] [-
waive_connected_net]
                                         |-tag tag_name)
                        |-edit -id waiver_id
                              [-source_file file_name]
                              [-start_line start_line]
                              [-end_line end_line]
                              [-start_column start_column]
                              [-end_column end_column]
                              [-instance instance]
                              [-comment comment]
                              [-expression expression]
                              [-tag tag]
                              [-type type]
                              ([-task task_name] | [-env])
                              [-force] [-silent]
                        |-get (-all |-id waiver_id_list |-tag tag_list) [-no_return]
                        |-list_matching_cover_items (-all |-id waiver_id |-tag
waiver_tag)
                        |-accept_update waiver_id_list

check_cov -list_related_group
                  [-status (unchecked | undetectable)]
                  [-signal {source_signal_arg_list}]

check_cov -assert_analysis -assert assertion_name [-set_bound bound_value | -
clear_bound]

check_cov -configure_gui
                  ([-task (task_list)]
                   [-exclude (  waived | disabled | unreachable | deadcode
                              | reset | const | assume | synthesis_optimized)])
```

```
check_cov -configure_gui -instance instance -model cover_item_model
                  ([-hierarchical])

check_cov -configure
                   [-checker_mode (coi | proof_core | proof_core_high_precision |
mutation)]
```

Detailed Description

Use this command to compute, manage, and report proof core and stimuli coverage.

```
Setup Switches
--------------
-init [-model (all | (statement | branch | expression | functional | toggle | fsm)+)]
      [-type (all | stimuli | coi | proof
          | proof_core_high_precision | bound | mutation)+]
      [-covdut {module_name}]+
      [-covfile ccf_file]*
      [-expr_type (all | bitwise | conditional | relational | reduction )+]
      [-expr_coverable_statements (all | procassign | contassign )+]
      [-expr_successive_xor_limit N]
      [-max_expr_terms N]
      [-expr_struct_scoring]
      [-vlog_short_circuit]
      [-no_vhdl_shortcircuit]
      [-enable_prove_based_proof_core]
      [-create_cover_item_property]
      [-proof_core_include_failing]
      [-enable_cover_item_traces]
      [-skip_ternary_branches]
      [-skip_async_blocks]
      [-include_instance {instance_list} [-regexp]]+
      [-exclude_instance {instance_list} [-regexp]]+
      [-include_module {instance_list} [-regexp]]+
      [-exclude_module {module_list} [-regexp]]+
      [-exclude_cover_expression_calculation]
      [-include_x_cover_items]
      [-exclude_empty_branch_cover_items]
      [-include_celldefine_cover_items]
      [-enable_vy]
      [-exclude_bind_hierarchies]
```

```
  [-include_assume_coi]
  [-include_sim_align_cover_items]
  [-toggle_no_ports]
  [-toggle_ports_only]
  [-toggle_sv_mda [N]]
  [-toggle_sv_mda_of_struct]
  [-toggle_sv_generate]
  [-covergroup_per_instance_default_one]
  [-statement_type (blocking | non_blocking)]
  [-toggle_type (flop | latch | port)]
  [-toggle_mutation_type (stopat | stuckat)]
  [-include_assign_scoring]
  [-toggle_sv_modport]
  [-toggle_auto_interface]

Initializes Coverage App with the specified settings.

Coverage Initialization is supported at any point in the verification flow.
In the current version of the tool, the following features require
initialization before elaborating a design:
  . Expression model.
  . Stimuli Coverage for branch/statement models inside loops, functions,
    and VHDL.

. Use the "-model" switch to select the coverage models you desire.
  Default value for this option is "all".
  . Use "all" to initialize all coverage models, except "fsm". This is
    equivalent to "{statement branch expression toggle functional}". The
    "fsm" model is only enabled if it is explicitly passed in the model
    list.
  . Use "statement" to extract statement cover items from the design.
  . Use "branch" to extract branch cover items from the design.
  . Use "expression" to extract expression cover items from the design.
  . Use "functional" to set up covergroups and user covers.
  . Use "toggle" to extract toggle cover items from the design.
  . Use "fsm" to extract finite-state machine cover items from the design.
. Use "-type" to select the measurement types for coverage setup.
  Default value for this option is "{stimuli coi bound proof}".
  . Use "all" to enable stimuli, coi, bound and proof.
  NOTE:
  . You cannot use the types "proof", "proof_core_high_precision"
    and "mutation" together.
  . When exporting Checker Coverage metrics to IMC, the tool gives
```

```
    precedence to "mutation" and "proof" over "coi". To export COI metrics,
    enable type "coi" and do not enable "proof" or "mutation" types.
. Use "-covdut" to select a module and its hierarchy for coverage setup.
  The default is the top module.
  NOTE:
  . The tool inserts additional observability points in branches and
    statements inside branches, which increases precision for the
    corresponding cover items. Other cover items might incidentally
    benefit from the improved precision.
. Use "-covfile" to specify the coverage configuration file.
  Specify the switch multiple times to specify multiple files.
. Use "-expr_type" to restrict the generation of expression cover items
  with the specified type of operator.
        -all           All Verilog operators and VHDL operators except
                       events and VHDL not
        -bitwise       ~ , & , | , ^ , ^~ , ~^ , xor , and xnor
        -conditional   ?:
        -relational    > , >= , < , === , !== , == , !=, <= , = , and /=
        -reduction     & , ~& , | , ~| , ^ , ~^ ,and ^~
  Verilog logical operators (|| and &&) and VHDL logical operators (and, or,
  nand, nor) in condition expressions is scored by default.
. Use "-expr_coverable_statements" to enable the coverage of operators in
  various Verilog and VHDL statements. By default, expression cover items
  are created for conditions only.
        -all           enables scoring of expressions in all supported
                       statements.
        -procassign    enables scoring of expressions in procedural
                       statements for Verilog and sequential statements for
                       VHDL.
        -contassign    enables scoring of expressions in continuous
                       assignment for Verilog and concurrent statements for
                       VHDL.
. Use "-expr_successive_xor_limit N" to restrict the generation of
  expression cover items for "N" successive XOR?XNOR terms. If you do not
  specify otherwise, the tool takes 4 by default.
. Use "-max_expr_terms N" to restrict the generation of expression cover
  items to "N" leaf terms. If you do not specify otherwise, the tool takes
  1024 by default. Possible values of "N" are 16, 32, 64, 128, 256, 512,
  1024, and 2048.
. Use "-expr_struct_scoring" to enable scoring of expressions containing
  members of union and struct.
. Use "-vlog_short_circuit" to enable short-circuited expression tables
  for Verilog operators &&, ||, |, &. Ternary operator ?: is
```

by default scored in short-circuited form.
. Use "-no_vhdl_shortcircuit" to disable short-circuited expressions for VHDL operators and, or, nand, and nor of bit and Boolean type operands.
. Use "-enable_prove_based_proof_core" to collect proof core data from the engines using the "prove" command.
. Use "-create_cover_item_property" to create cover properties for all cover items in the < embedded> task.
. Use "-proof_core_include_failing" to measure proof core coverage for failing assertions.
. Use "-enable_cover_item_traces" to enable cover item traces, which are disabled by default.
. Use "-skip_ternary_branches" to exclude ternary cover items.
. Use "-skip_async_blocks" to suppress cover item generation for asynchronous blocks in always blocks.
. Use "-include_instance" to generate cover items from the matching instances only.
. Use "-exclude_instance" to exclude generation of cover items from the matching list of instances.
. Use "-include_module" to generate cover items from the matching modules only.
. Use "-exclude_module" to exclude generation of cover items for the specified list of modules.
NOTE:
  . The "-exclude_module", "-include_module", "-exclude_instance", and "-include_instance" switches accept wildcards (* and ?) and filter using case-sensitive wildcard strings by default.
  . Use "-regexp" with any of these switches to change the expression to a case-sensitive regular expression.
  . Initially, the tool includes cover items from all instances. You can use these switches to exclude/include instances and modules.
  . Use "-exclude_instance" and "-exclude_module" first to exclude the generation of cover items from the specified list of instances and modules. You can then use "-include_instance" and "-include_module" to generate cover items from the specified instances and modules that are excluded using "-exclude_instance"/"exclude_module" switches.
  . You can specify these switches multiple times. The overall result is a combination of all switches with the last switch having highest priority.
  . If "-covdut" switch is specified, then these switches filter out modules and instances from instance hierarchy included by the "-covdut" switch.
  . "-exclude_instance" and "-include_instance" only match and apply the filter on an instance and not its hierarchy. Use wildcards or regexp pattern to filter the complete hierarchy.

. Use "-exclude_cover_expression_calculation" to prevent the tool from calculating the expression of the cover items. Expressions are calculated by default.
. Use "-include_x_cover_items" to include cover items generated from default branches with X assignments or null statements in VHDL.
. Use "-exclude_empty_branch_cover_items" to exclude cover items generated for implicit else branches and implicit default case statements.
. Use "-include_celldefine_cover_items" to include cover items generated from `celldefine blocks.
. Use "-enable_vy" to include cover items from modules analyzed with the "-v" or "-y" switches.
. Use "-exclude_bind_hierarchies" to exclude cover items inside hierarchies that come from SVA bind statements. This only applies to code coverage.
. Use "-include_assume_coi" to include logic referenced by constraints in COI and proof-core reporting.
. Use "-include_sim_align_cover_items" to generate new cover items or modify the existing ones to match the blocks generated by the simulation tool.
. Use "-toggle_no_ports" to skip generation of toggle cover items for module ports so that a net is scored only once within a hierarchy.
. Use "-toggle_ports_only" to generate toggle cover items for module ports only.
  NOTE: The "-toggle_no_ports" and "-toggle_ports_only" switches are mutually exclusive.
. Use "-toggle_sv_mda N" to generate cover items for packed and unpacked multidimensional vectors and static arrays for reg, logic, bit, and wire SystemVerilog data types inside modules and interfaces.
  In this switch, N is an optional argument and specifies the maximum number of bits for which toggle cover items will be generated.
  Valid Values for N are 10, 12 (default), 14, 16, 18, 20, 22, and 24.
. Use "-toggle_sv_mda_of_struct" to generate cover items for members of packed and unpacked multidimensional static arrays (MDA) of structs declared inside modules and interfaces.
  NOTE: The optional argument N specified with "-toggle_sv_mda" also applies to "-toggle_sv_mda_of_struct".
. Use "-toggle_sv_generate" to generate toggle cover items for registers and nets inside all for-generate, if-generate, and case-generate blocks.
. Use "-covergroup_per_instance_default_one" to report the instance coverage of all the covergroups where option.per_instance is not set explicitly. If the value is set explicitly, instance coverage is reported as per setting of the option per_instance.
. Use "statement_type" to restrict statement model cover items generation for blocking or non-blocking statements.

```
    . Use "toggle_type" to restrict toggle model cover items generation
      individually or as a combination of flop, latch, and port.
    . Use "toggle_mutation_type" to specify the mutation type for calculating
      toggle coverage.
    . If you have already specified the "branch" model be generated, use
      "include_assign_scoring" to also generate branch cover items for Verilog
      continuous assignments. These items can also be generated using the
      CCF "set_assign_scoring" command.
    NOTE: Use the options specified above to configure automatic setup, which
    automatically creates a coverage model and adds all assertions to the
    coverage database.
    . Use "toggle_sv_modport" to enable scoring of modport signals for all
      interfaces.
    . Use "toggle_auto_interface" to enable scoring of all interface
      and modport signals for the selected module.

-export [file_name] [-force]
    Exports the Coverage database. Use this command to export Coverage results for
    later import and analysis.

    . Use file_name to specify the file name. If you do not specify a file
      name, the "-export" command refers to covdb.db by default.
    . Use "-force" to overwrite the existing database file.

    NOTE: To save and restore the complete session analysis work, use "save"
    and "restore" instead as these commands include Coverage App data by
    default.

-import [file_name] [-db_file file_name]+ [-overwrite]
    Imports the coverage database(s) specified by "file_name" to the main database.
    By default, this automatically merges specified databases to the main
    coverage database.

    . Use "-db_file" to specify the file name. It takes one argument, however
      you can use this switch multiple times to import multiple database files.
      Also, it is optional when importing only single file.
      NOTE: You must run "analyze" and "elaborate" before importing the database
      unless you launched the tool with "jg -cov -offline".
    . Use "-overwrite" to clear main database and then import the specified
      databases to main database. This switch works for offline mode only.

-measure [-type ( all | stimuli | coi | proof
                | proof_core_high_precision | bound | mutation)+]
```

```
    [-type per_property_checker]
    [-all |-task {task_name_list}]
    [-property {property_name_list} [-regexp]]+
    [-include_deadcode_assume {assumption_list}]
    [-refresh]
    [-bg]
    [-verbose]
    [-stimuli_engine_mode {engine_mode_tcl_list}]
    [-checker_engine_mode {engine_mode_tcl_list}]
    [-iter {N}]
    [-no_auto]
    [-max_jobs {N}]
    [-per_engine_max_jobs {N}]
    [-time_limit {time_limit}]
    [-per_property_time_limit {time_limit}]
    [-per_property_time_limit_factor {N}]
    [-enable_unprocessed]
    [-checker_cover {cover_item_name_list} [-regexp]]+
    [-proof_core_job_ratio {percentage}]
    [-simplification_only]
    [-skip_covered_or_checked_from_other_tasks]
    [-skip_deadcode]
Generate all coverage metrics: stimuli, checker, and bounded coverage.

NOTE:
  . "check_cov -measure" with no additional arguments runs "-measure"
    on the current task with "-init -type" options (when available), and the
    checker type is defined by the current active Checker Mode, which can be
    selected in COV GUI or by using "check_cov -configure -checker_mode".
  . If Coverage is not initialized yet, the tool attempts to initialize
    it with the options required to run the issued "-measure" command, which
    might not be possible due to limitations of the automatic setup.
    See the "-init" section for more information on initialization options.

The command "check_cov -measure" calculates multiple coverage metrics
in parallel. To achieve this, it requires
"(n1+n2+n3) * per_engine_max_jobs" engine jobs where n1, n2, and n3
corresponds to checker, stimuli, and deadcode engines respectively.
The default value for "per_engine_max_jobs" is taken from
"proofgrid_per_engine_max_local_jobs" for local mode and
"proofgrid_per_engine_max_jobs" for lsf mode.
In the default mode, the amount of engines required for checker, stimuli,
and deadcode are 9, 7 and 7, respectively.
```

```
Hence, this command needs at least (9+7+7)*1, that is, 23 engine jobs
in local mode, and (9+7+7)*2, that is, 46 engine jobs in lsf mode.
If the specified "max_jobs" value is less than the required engine
jobs, then "per_engine_max_jobs" will be capped to comply with
max_jobs requirement.
Also, "proofgrid_per_engine_max_jobs_per_session" should be equal or
greater than the engine jobs required by the "check_cov -measure"
command.

The stimuli coverage deadcode analysis aims to re-prove the unreachable
cover items in an unconstrained environment (also reset unconstrained)
and report them as deadcode if they are still unreachable.

. Use "-type" to override the "-init -type" options for coverage
  measurement by this "-measure" command.
. Use "-all" to generate coverage metrics for all tasks.
. Use "-task" to specify the tasks for coverage measurement.
. Use "-property" to specify a list of assertions and covers for
  coverage measurement. The switch accepts wildcards (* and ?).
  Use "-regexp" to provide the input as a regular expression.
. Use "-include_deadcode_assume" to specify a list of assumptions
  for deadcode analysis.
. Use "-exclude_functional_stimuli" to skip stimuli measure for
  functional cover items.
. Use "-refresh" to refresh coverage metrics in the GUI.
  NOTE: "-refresh" cannot be used with "-type".
. Use "-bg" to run the "-measure" command in the background.
  NOTE: This switch returns a numeric identifier that represents this
  measurement and can be used along with "-wait" and "-stop" options.
. Use "-verbose" to print proof messages.
  The proof messages will use the prove_verbosity configuration.
  (See "set_prove_verbosity" for additional information.)
. Use "-stimuli_engine_mode" to override the default engine selection
  for stimuli measurement.
. Use "-checker_engine_mode" to override the default engine selection
  for proof core measurement.
  Valid options are any subset of engines {C K I N A*}.
  When available, the tool automatically adds custom variants for engines
  N, AD, and AM.
. Use "-iter" to override the current "max_trace_length" for this
  "-measure" command. The default value comes from "set_max_trace_length".
  (See "set_max_trace_length" for additional information.)
. Use "-no_auto" switch to disable auto prove strategy for stimuli measure.
```

. Use "-max_jobs" to override the maximum number of engine_jobs used
  by the "-measure" command.
  The default value comes from "proofgrid_max_jobs" and
  "proofgrid_max_local_jobs" (See "set_proofgrid_max_jobs" and
  "set_proofgrid_max_local_jobs" for additional information.)
. Use "-per_engine_max_jobs" to override the number of engine_jobs
  per engine per coverage-metrics computation by the "-measure" command.
  The default value comes from "proofgrid_per_engine_max_jobs"
  and "proofgrid_per_engine_max_local_jobs". (See
  "set_proofgrid_per_engine_max_jobs" and
  "set_proofgrid_per_engine_max_local_jobs" for additional information.)
. Use "-time_limit" to override the time limit for this "-measure" command.
  The default value comes from "prove_time_limit". (See
  "set_prove_time_limit".
. Use "-per_property_time_limit" to override the current
  "prove_per_property_time_limit" for this "-measure" command.
  The default value comes from "prove_per_property_time_limit". (See
  "set_prove_per_property_time_limit".
. Use "-per_property_time_limit_factor" to override the current
  "prove_per_property_time_limit_factor" for this "-measure" command.
  The default value comes from "prove_per_property_time_limit_factor". (See
  "set_prove_per_property_time_limit_factor" for additional information.)
. Use "-enable_unprocessed" to run mutation mode on unprocessed, proven,
  and undetermined assertions.
. Use "-checker_cover" to run mutation mode on a selective set of cover
  items. The switch is applicable to coverage type mutation only.
  The switch accepts wildcards (* and ?). Use "-regexp" to provide
  the input as a regular expression.
. Use "-proof_core_job_ratio" to change the percentage of jobs allocated
  to the auxiliary proof-core instance that runs together with mutation.
  By default, when a mutation measure starts, 45 percent of its jobs are
  allocated to an auxiliary proof-core instance, that runs in parallel to
  mutation to speed-up the classification of undetectable cover items.
  Once proof-core finishes, the jobs allocated to it are passed to mutation
  until the measure run is complete. Set this parameter to zero to prevent
  proof-core from running together with mutation.
. Use "-simplification_only" to run only preprocessing and simplification
  without invoking the regular formal engines.
. Use "-skip_covered_or_checked_from_other_tasks" to prevent spending
  resources on cover items that are already checked in other tasks,
  or covered in case of stimuli.
. Use "-skip_deadcode" to avoid running deadcode analysis for this
  "-measure" command. The tool allocates all resources that would be spent

```
      on deadcode analysis to the stimuli proof for this "-measure" command.

    . Use "-type per_property_checker" to get independent checker analysis
      results for each property.

      Results are returned directly from the command, rather than being shown
      in the GUI. The return format is a dictionary that maps property names
      to their checker results. For each analyzed property, it shows:
        . A status indicating how the property was classified after the
          analysis finished.
        . How many toggle cover items there are in its COI.
        . How many toogle cover items were checked.
        . A set of lengths where cover items got checked and their respective
          cover item IDs.
        . The "min_length" reached by the engine.

      The tool ensures a fair split of resources is given to each property.
      To achieve that, each property runs on a dedicated engine partition and
      the overall "time_limit" is evenly split between them.
      All proof jobs are executed on a custom version of the engine Bm.
      Only toggle items are considered by this analysis type.
      For background runs, results can be fetched by
      "check_cov -measure -status [-id index]", instead of being returned by
      the analysis command itself.

-measure -stop [-id index]
    Stop all "check_cov -measure" commands running in the background, or
    provide an optional numeric identifier (index) to stop a single command
    running in the background.

    NOTE: "-stop" is not blocking and might return before the measurement
    has actually stopped. If you want blocking behavior, use
    "check_cov -measure -stop" and then "check_cov -measure -wait".

-measure -wait [-id index]
    Wait for the currently running "check_cov -measure" commands to complete
    before running additional commands.

    Provide an "index" to tell the tool to wait for a specific instance of the
    "check_cov -measure" command running in background.

    This option is useful when "check_cov -measure" is running in the
    background (-bg). In effect, it moves "check_cov -measure" back
```

```
    to the foreground, and the tool accepts no additional commands until
    measurement completes.

-measure -status [-id index]
    Return the status of all "check_cov -measure" commands running in the
    background.

    Provide an "index" to tell the tool to return the status of a specific
    instance of the "check_cov -measure" command running in background.

    In the case of measure run with "-type per_property_checker", checker
    results for each property are also returned. Check the help for
    "-type per_property_checker" for more detailed information.

-measure -clear [-task {task_list}]
    Clear the coverage measurement results.

    Use "-task" to specify the tasks for which you want to clear coverage
    measurement results.

-extend cover_item_id_list
      (-precondition expression |-postcondition expression)+
      [-task tasks_list]
    Set the precondition and/or postcondition of the list of cover items
    specified by "cover_item_id_list" in the set of tasks provided.

    . If no task is specified, the current working task is used.
    . Stimuli proof results for the cover items are cleared in the
      tasks that are affected by this command.
    . At least one of "-precondition" or "-postcondition" must be used.
    . The precondition or postcondition, if not used, will remain unchanged.

-extend cover_item_id_list -clear
      [-task tasks_list]
    Clear both the precondition and postcondition of the list of cover items
    specified by "cover_item_id_list" in the set of tasks provided.

    . If no task is specified, the current working task is used.
    . Stimuli proof results for the cover items are cleared in the
      tasks that are affected by this command.

-create_cover_item_property
      [-include_hierarchies included_hierarchies]
```

```
      [-exclude_hierarchies excluded_hierarchies]
      [-task {task_list}]
      [(-cover_item_id {cover_items_list}) | (-cover_item_name {cover_items_list})]
    Convert cover items to cover properties.
    . Use "-include_hierarchies" if you want the tool to generate cover
      properties for the specified hierarchies only.
    . Use "-exclude_hierarchies" if you do not want the tool to generate cover
      properties for the specified hierarchies.
    . Use "-task" to specify the tasks for which you want to create cover
      properties.
    . Use "-cover_item_id" to specify by ID the cover items you want to create
      cover properties for in the current or specified task.
    . Use "-cover_item_name" to specify by name the cover items you want to
      create properties for in the current or specified task.

-cover_item_tag -add tag_name
    ( ([-source_file file [-start_line start_line] [-end_line end_line]
                          [-start_column start_column] [-end_column end_column]]
       [-instance {instance_list}]
       [-expression -expression])+
       [-type type]
     |-cover_item_id id_list
    )
    [-force]
    Add a tag to one or more cover items.

    . Use "-source_file" to tag all cover items in the specified source file.
      . Use "-start_line" to tag cover items from the specfied start line in
        the given file.
      . Use "-end_line" to tag cover items to the specified end line in the
        given file.
      . Use "-start_column" to tag cover items from the specified start column
        in the given file.
      . Use "-end_column" to tag cover items to the specified end column in
        the given file.
    . Use "-instance" to tag cover items in the specified instance list.
    . Use "-expression" to tag cover items with the specified expression.
    . Use "-type" to tag cover items of a particular type (that is, statement,
      branch, expression, functional, or toggle).
    . Use "-cover_item_id" to tag the list of specified cover items.
    . Use "-force" to re-tag an already tagged cover item.

-cover_item_tag -remove (-all |-tag tag_name)
```

```
    Remove tags from cover items.

    . Use "-all" to remove all tags from all cover items.
    . Use "-tag" to remove tags from cover items with the specified tag only.

-cover_item_tag -untag (-all |-cover_item_id id_list)
    Remove tag from a specified list of cover items.

    . Use "-all" to untag all cover items.
    . Use "-cover_item_id" to remove tag from the list of specified cover
      items.

-cover_item_tag -get -all
    Return a list of currently existing tags.

-cover_item_tag -list_matching_cover_items (-all |-tag {tag_list})
    Return a list of cover item IDs.

    . Use "-all" to return a list of cover item IDs for all tags.
    . Use "-tag" to return a list of cover item IDs associated with the
      specified tags.

Report Switches
---------------
-report
    [-exclude ([const] [assume] [reset] [waived] [unreachable]
        [deadcode] [synthesis_optimized] [disabled] [constant_input_port])+]
    [-task task_list]
    [-model (statement | branch | expression | functional | toggle | fsm)]
    [-type (checker | stimuli | formal | bounded)]
    [-checker_mode (coi | proof_core | proof_core_high_precision | mutation)]
    [-no_return]
    [-silent]
    [-report -file file_name [-force] [-html]]
    [-exclude_module {module_list} [-regexp]]
    [-include_module {module_list} [-regexp]]
    [-exclude_instance {instance_list} [-regexp]]
    [-include_instance {instance_list} [-regexp]]
    Report both coverage and assertion summary in text and Tcl format.

    Coverage Summary
    Print report with the following information for each instance:
    . Formal coverage shows the percentage of "covered_and_checked" items
```

```
      based on checker setting (proof/coi as provided in init type) with
      total cover items.
    . Stimuli coverage shows the percentage of "covered" items with total
      cover items.
    . Checker coverage shows the percentage of "checked" items based on
      checker setting (proof/coi as provided in init type) with total
      cover items.

    Assert Summary
    Print report with the following information for each undetermined assertion:
    . The bounded coverage percentage.
    . The assert bound.
    . The number of cover items in coi.
    . The number of cover items covered beyond bound.
    . Covered CIs max bound.
    . The number of cover items undetermined (and not dead).
    . Undetermined CIs max bound.

    The following switches can be used to further refine the summary result:
    . Use "-exclude" with one or more of the following arguments to omit cover
      items that are not reachable due to propagation or initial simplification.
      . const: constant propagation
      . assume: assumption propagation
      . reset: reset propagation
      . waived: user-specified waivers
      . unreachable: unreachable cover items
      . deadcode: deadcode cover items
      . synthesis_optimized: cover items related to RTL code that was optimized during
        synthesis and is not present in resulting netlist. These cover items will
        usually be undetectable (out-of-coi) and/or deadcode.
        Some examples of these are:
          . Default labels of full case blocks.
          . Branches that are not executed due to module parameterization.
          . Blocking assignments that were overriden.
          . Variables assigned to RTL constants in sequential blocks
            that can lead to synthesis optimizations in branch and loop conditions.
      . disabled: disabled cover items
      . constant_input_port: cover items with stimuli status defined by constants
assigned to ports
    . Use "-task" to specify the tasks for which you want to report cover
      properties. If task is not specified, the default is from the currently
      selected task in console.
    . Use the "-model" switch to select the coverage model for which you want
```

```
      to report cover properties. By default, the report includes all "init"
      models.
    . Use the "-type" switch to select the coverage metrics type for which you
      want to report cover properties. By default, the report includes all four
      metric types.
    . Use the "-checker_mode" switch to select the checker mode to be reported
      in the checker metrics. By default, the tool reports the current active
      checker mode that is displayed in GUI. The active checker mode can be
      selected in GUI or by using "check_cov -configure -checker_mode".
    . Use "-silent" to return Tcl return values without printing to the screen.
      This option is helpful in Tcl scripts.
    . Use "-report_file" to export the coverage report to the specified
      file format.
      . Use "-force" to overwrite an existing file.
      . Use "-html" to export the coverage report to an HTML file.
      NOTE: When you use the "-html" switch with "-report", the Coverage App
      generates HTML files for Summary and Bounded coverage report.
    . Use "-exclude_module" to exclude cover items from the matching
      module.
    . Use "-include_module" to include only cover items in the matching
      module.
    . Use "-exclude_instance" to exclude cover items from the matching
      instance.
    . Use "-include_instance" to include only cover items in the matching
      instance.
      NOTE:
      . The "-exclude_module", "-include_module", "-exclude_instance", and
        "-include_instance" switches accept wildcards (* and ?) and filter
        using case-sensitive wildcard strings by default.
      . Use "-regexp" with any of these switches to change the expression to
        a case-sensitive regular expression.

List Switches
---------------
-list [-status (covered | uncovered | unreachable | deadcode | unprocessed
        | checked | unchecked | undetectable | in_bound | out_bound
        | covered_and_checked | uncovered_or_unchecked
        | unreachable_or_undetectable)]
        [-exclude_module {module_list} [-regexp]]
        [-include_module {module_list} [-regexp]]
        [-exclude_instance {instance_list} [-regexp]]
        [-include_instance {instance_list} [-regexp]]
        [-bound override_bound]
```

```
        [-exclude ([const] [assume] [reset] [waived] [unreachable]
              [deadcode] [synthesis_optimized] [disabled] [output_coi]
[constant_input_port]
              [signal_coi -signal {signal_list}])+]
        [-include ([case_default]
              [signal_coi -signal {signal_list}])]
        [-assert assert_name [-regexp]]
        [-task task_list]
        [-checker_mode (coi | proof_core | proof_core_high_precision | mutation)]
        [-no_return]
        [-silent]
        [-model ([branch] [statement] [expression] [functional] [toggle] [fsm])+]
        [-cid]
    List formal status, stimuli status, checker status, description,
    source location and cover item ID per cover item level.

    Use "-status" and specify one of the following arguments:
    . "covered" - List reachable cover items in the current or specified task.
    . "uncovered" - List undetermined cover items in the current or specified
      task.
    . "unreachable" - List unreachable cover items in the current or specified
      task.
    . "deadcode" - List deadcode cover items in the current or specified task.
    . "checked" - List in_proof_core or in_coi cover items based on init type
      in the current or specified task.
    . "unchecked" - List only those in_coi cover items that are not present in
      the in_proof_core cover items of the current or specified task.
    . "undetectable" - List out_coi cover items in the current or specified
      task.
    . "covered_and_checked" - List reachable and checked cover items based on
      init type in the current or specified task.
    . "uncovered_or_unchecked" - List undetermined or out_proof_core cover items
      if init type checker setting is "proof" or "mutation".
      It lists undetermined and in_coi cover items if init type checker setting
      is "only_coi" in the current or specified task.
    . "unreachable_or_undetectable" - List unreachable or out_coi cover items in
      the current or specified task.
    . "unprocessed" - List unprocessed cover items in the current or specified
      task.
    . "in_bound" - List in_bound cover items for an undetermined assertion in
      the current or specified task.
    . "out_bound" - List out_bound cover items for an undetermined assertion in
      the current or specified task.
```

```
   By default, it list all cover items in the current or specified task.

   Use the following switches to further refine the coverage list:
   . Use "-exclude_module" to exclude cover items from the matching
     module.
   . Use "-include_module" to include only cover items in the matching
     module.
   . Use "-exclude_instance" to exclude cover items from the matching
     instance.
   . Use "-include_instance" to include only cover items in the matching
     instance.
     NOTE:
     . The "-exclude_module", "-include_module", "-exclude_instance", and
       "-include_instance" switches accept wildcards (* and ?) and filter
       using case-sensitive wildcard strings by default.
     . Use "-regexp" with any of these switches to change the expression to
       a case-sensitive regular expression.
   . Use "-bound override_bound" with "in_bound" or "out_bound" status to
     override the current bound with the specified bound.
   . Use "-exclude" with one or more of the following arguments to omit cover
     items that are not reachable due to propagation or initial simplification.
     . const: constant propagation
     . assume: assumption propagation
     . reset: reset propagation
     . waived: user-specified waivers
     . unreachable: unreachable cover items
     . deadcode: deadcode cover items
     . synthesis_optimized: cover items related to RTL code that was optimized during
       synthesis and is not present in resulting netlist. These cover items will
       usually be undetectable (out-of-coi) and/or deadcode.
       Some examples of these are:
         . Default labels of full case blocks.
         . Branches that are not executed due to module parameterization.
         . Blocking assignments that were overriden.
         . Variables assigned to RTL constants in sequential blocks
           that can lead to synthesis optimizations in branch and loop conditions.
     . disabled: disabled cover items
     . output_coi: cover items in the COI of primary outputs
     . constant_input_port: cover items with stimuli status defined by constants
assigned to ports
     . signal_coi: cover items in the COI of specified signal list
   . Use "-include" with one of the following arguments to list cover items
     that fall in the following category:
```

```
      . case_default: cover items enclosed in case default statements
      . signal_coi: cover items in the COI of specified signal list
    . Use "-assert" with "in_bound" or "out_bound" status to include only cover
      items in the bounded coverage of the specified assertion.
    . Use "-task" to specify the tasks for which you want to list cover
      properties. If task is not specified, the default is from the currently
      selected task in console.
    . Use the "-checker_mode" switch to select the checker mode to be reported
      in the checker metrics. By default, the tool reports the current active
      checker mode that is displayed in GUI. The active checker mode can be
      selected in GUI or by using "check_cov -configure -checker_mode".
    . Use "-model" to list only cover items from the specified model or
      models. By default, list include all "init" models.
    . Use "-cid" to report cover items in cover item ID list format.

Utility Switches
----------------
-get_cover_item_info cover_item_id_list [-task tasks_list]
    [-name]
    [-expression]
    [-instance]
    [-module]
    [-file]
    [-source_location]
    [-tag]
    [-prove_status]
    [-deadcode_analysis_status]
    [-exclusions]
    [-bound]
    [-engine]
    [-time]
    [-precondition]
    [-postcondition]
    Return information about cover items in the specified Tcl list.

    . The information is returned in the format of a list of dictionaries.
      Each dictionary represents a cover item in "cover_item_id_list".
      The dictionary keys are the same as the options available for the
      command.
    . All keys after "prove_status" in the list above might have different
      values for each task. For this reason, each cover item result contains
      a key called "task", whose value is a list of dictionaries, one per task.
      Each dictionary for a task contains a key called "name", whose value is
```

```
      the task name, and then all the specified keys in the list above.
    . Use "-task" to return information about cover items in the specified
      tasks only. By default, the tool returns information for all tasks.
    . Use one or more additional switches to return information for the
      specified fields only. By default, the tool returns information for all
      fields except "expression" and "deadcode_analysis_status".
    . The expression returned does not include clock information if
      the cover item is inside a block sampled by a clock.
    . For toggle cover items, this command returns the name of the
      signal bit as being its expression.

-get_target_info target_id_list
    [-name]
    [-prove_status]
    [-bound]
    [-engine]
    [-task]
    [-time]
    [-coi]
    [-proof_core]
    Return information about targets in the specified Tcl list.

    . Use "-name", "-prove_status", "-bound", "-engine", "-task", "-time",
      "-coi" and/or "-proof_core" to return information on only the
      specified options. By default, "-get_target_info" returns
      information on all options.
    . The information is returned in the format of a list of dictionaries.
      Each dictionary represents one target inside "target_id_list".
      The dictionary keys are the same as the options available for the
      command.

-get_cover_id [-name |-signal |-cover_name cover_property_name]
    [ -source_signal {source_signal_arg_list} [-direction ([fanin] [fanout])+]
     | {target_id_list}]
    Return a list of the cover item IDs.
    . Use "-name" to return the list of all cover item names.
    . Use "-cover_name" to return the cover item ID of a given cover item name.

      The specifics of the naming convention for branch, statement, expression
      or toggle follow:


_automatic_coveritem_<type>_condition_<subtype>__<cover_item_index>__<type>__<line>_<su
```

```
btype_num>_<type_index>

      . type - branch, statement, or expression
      . subtype - Type of branch, statement, expression, or toggle cover item
      . cover item index - Index of all cover items generated
      . type - S for statement, B for branch, E for expression, or T for toggle
      . line number - The location of the cover item
      . subtype number - For conditions, 0 for true path and 1 for
        corresponding false path
        Example: if_expr_then is 0 while if_expr_else is 1
        NOTE: This information is irrelevant in the case of statements,
        expression, or toggle.
      . type index - The index of cover items with the same subtypes, line
        numbers, and subtype number
        Example: mux_out = enable ? enable2 ? din_1: din2: din_0; has two
        if_expr_then on the same line; therefore, their type indexes are 0 and 1.

      The specifics of the naming convention for covergroup follow:


cover_item_covergroup<covergroup_name>_inst_<covergroup_instance>_coverpoint_<coverpoin
t_name>_bin_<bin_name>

      . covergroup name - Name of covergroup
      . covergroup instance - Instance of covergroup
      . coverpoint name - Name of coverpoint
      . bin name - Name of bin

    . Use "-signal" to return the cover item names without the COVER_ITEM_ID
      suffix.
    . Use "-cover_name" to specify the cover item name and return the
      corresponding cover item ID.
    . Use "-source_signal" to return a list of cover item IDs in both the fanin
      and fanout of the specified signal list. And use "-direction" if you want
      to get a list of IDs for only the fanin or fanout.
    . Specify a "target_id_list" to return the cover items in the COI of the
      specified target ID list.

-get_assert_id [-name]
    Return the target IDs in the database.

    . Use "-name" to return the names of the targets.
```

```
-waivers
        -import -file_name file_name [-force] [-silent]
              [-vrefine [-ucdb db_path [-target target -source source]]]
        |-export -file_name file_name
              [-id waiver_id_list |-tag waiver_tag_list]
              [-vrefine [-ucdb db_path [-target target -source source]]]
              [-force] [-silent]
        |-add -comment comment
              ([-source_file file_name]
               [-start_line start_line]
               [-end_line end_line]
               [-start_column start_column]
               [-end_column end_column]
               [-instance instance]
               [-expression ipk_switches_and_expression_arg]+
               [-cover_item_tag tag_name]
               [-tag tag]
               [-type type]
               [-persistency persistency]
               ([-cover_item_id id_list (  [-enclosed_ci_marking]
                              | [-waive_connected_net])]
               [(-task task_name] | [-env]))
              [-force] [-silent]
        |-remove (-all
                 |-id waiver_id [-enclosed_ci_marking] [-waive_connected_net]
                 |-tag tag_name)
        |-edit -id waiver_id
              [-source_file file_name]
              [-start_line start_line]
              [-end_line end_line]
              [-start_column start_column]
              [-end_column end_column]
              [-instance instance]
              [-comment comment]
              [-expression expression]
              [-tag tag]
              [-type type]
              ([-task task_name] | [-env])
              [-force] [-silent]
        |-get (-all |-id waiver_id_list |-tag tag_list)  [-no_return]
        |-list_matching_cover_items (-all |-id waiver_id |-tag waiver_tag)
        |-accept_update waiver_id_list
```

```
Handle waived cover items as specified.

. Use "-import" to import waivers from a Tcl file to the Coverage App.
  By default, the tool imports waivers from a Tcl file generated by
  the "export" command from Coverage app. After running the command,
  a Tcl dictionary is returned with the following information:
  . Number of waivers successfully imported (with the updated count).
  . Number of waivers that failed to import.
  . Number of waivers not imported due to repetition.
  Additionally, two textual files can be generated in the project
  directory, in case of updates and/or failures, grouping waiver entries
  by reason, file, and instance:
  . Failed waivers file: contains waivers that failed to import,
    grouped by their corresponding reasons.
  . Updated waivers file: contains waivers that were successfully imported,
    but were updated. Here also lies waivers not added due to repetition.
    The failed waivers file is grouped in six different reasons:
    . Waiver expression mismatch: the expression field has changed since
      last export and no longer matches a cover item.
    . Waiver RTL mismatch: the RTL design has changed since last export
      and the persistency algorithm was not able to map it correctly.
    . Waiver RTL and expression empty intersection: the waiver entry
      points to code in the RTL and has an expression field,
      the intersection of this information can not relate to a cover item.
    . Waiver type mismatch: cover items were found by the RTL design and/or
      expression; however, not by the cover item model type provided in the
      waiver entry.
    . Waiver invalid instance: the provided instance in the waiver entry
      no longer exists in the design hierarchy.
    . Waiver invalid file: the design source file related to the waiver
      can not be found in the project hierarchy.
    The updated waivers file is grouped in six different reasons:
    . File name updated: this means that, since last export, the file
      related to the waiver has changed names.
    . RTL update: this means that the persistency algorithm was able to
      identify the changes in RTL and map the waiver accordingly.
    . Hierarchy update: this is related to a change in the design, in which
      the new RTL has a different top module; however, the persistency
      algorithm was able to find the old top module and include
      its hierarchy in the design.
    . RTL mismatch: this means that the old RTL could not be found in
      this version of the design; however, the waiver entry still had
      enough information to find the cover items related to it
```

```
      (that is, found by expression, source location, instance, type).
    . Persistency field error: this means that the "-persistency" field was
      not successfully read, this should not happen unless you have
      specifically changed this field (not a recommended flow).
      More details on the persistency algorithm can be found below.
    The tool uses the "-persistency" checksum field to increase the
    successfully imported waivers number even if the RTL source file
    has changed. If any RTL lines related to the waiver changes
    or the number of times these lines appear changes,
    the waiver will not be updated and, as a result, the waiver
    might not be imported. Comments, trailing white spaces, and blank lines
    are disconsidered for this purpose and thus do not affect the match.
. Use "-export" to export waivers to the Tcl file.
  The tool computes a "-persistency" checksum field by default,
  for more details refer to "-persistency" in the "-import" switch.
  Such field is later used by "-import" to match waivers in case of RTL
  changes, such as the addition of blank lines, comments, and spaces.
  . Use "-force" to ignore the waivers that have not been updated and
    waivers that have no data without stopping the export.
  . Use "-silent" to return Tcl return values without printing to the
    screen.
  . Use "-id" to export only waivers with ids in the specified list.
  . Use "-tag" to export only waivers with the associated waiver tag.
. Use "-vrefine" to export/import to/from the format of Cadence IMC tool.
    . In case this refinement file is applied to or generated from
      another database, including those generated with simulation,
      you can provide its path with the "-ucdb" switch.
      The tool will merge current formal results with this
      reference database prior to importing/exporting refinements.
      This is done to make the results more robust, as there is no
      guarantee that the identifiers present in the refinement file
      will match those in the current elaborated design.
    . When importing a refinement file without providing a reference
      database, block cover items are only imported in case there
      is a single element in the related scope/source code line.
    . When importing a refinement file without providing a reference
      database, expression cover items are ignored.
    . Using a refinement file generated without a reference database
      is only safe when applied to a UCDB created from the current
      working session, using the "database -export_unicov" command.
    . In case the reference database has a different design hierarchy
      than the one in the current elaborated design, you can use the
      "-target" and "-source" switches to adjust the hierarchical paths.
```

```
    . Jasper requires IMC to be set in the PATH to use this feature.
. Use "-add" to add a waiver. This command returns a new waiver ID.
  NOTE: Added waivers are not excluded from Coverage reports by default.
  Use "check_cov -report -exclude waived" to exclude user-specified waivers.
  . Use "-comment" to specify a comment for the new waiver.
    NOTE: This field is mandatory, so each waiver must have a comment.
  . Use "-source_file" to specify a file that contains the waiver.
  . Use "-start_line" to specify the start line of the waiver in the given
    file.
  . Use "-end_line" to specify the end line of the waiver in the given file.
  . Use "-start_column" to specify the start column of the waiver in the
    given file.
  . Use "-end_column" to specify the end column of the waiver in the given
    file.
  . Use "-instance" to specify the instance of the file.
  . Use "-expression" to specify an expression that describes the waivers.
    NOTE: For toggle cover items, the waiver is applied on the signal bit
    name instead of the cover item expression.
  . Use "-cover_item_tag" to waive all cover items with the specified tag.
  . Use "-tag" to specify a tag for the waiver.
  . Use "-type" to specify the type of the waiver.
  . The "-persistency" switch is computed during the "-export" flow to
    specify the encrypted checksum of the source file and checksum of its waivers.
    The tool then uses this information to consider file changes when importing
    waiver information.
  . Use "-cover_item_id" to add waivers by cover item ID.
    . Use "-enclosed_ci_marking" to waive all cover items enclosed within the
      source location of the selected cover item ID. The switch
      "-enclosed_ci_marking" is valid for the Branch model only.
    . Use "-waive_connected_net" to waive all cover items in the fanin and
      fanout of the signal bits connected through the buffer gate of the
      selected cover item ID. The switch "-waive_connected_net" is valid for
      the Toggle model only.
  . Use "-task" to add waivers for a specific task.
    NOTE: If omitted, the waiver is a global waiver.
  . Use "-env" to add waivers globally.
    NOTE: Global waivers are equivalent to applying waivers to all tasks.
  . Use "-force" to add the waiver when it does not match any cover item.
. Use "-remove" to delete waivers.
  . Use "-all" to delete all waivers.
  . Use "-id" to delete the waiver with the specified ID.
  . Use "-tag" to delete waivers with the specified tag.
  . Use "-enclosed_ci_marking" to delete waivers with the specified
```

```
      enclosed_ci_marking waiver ID.
    . Use "-waive_connected_net" to delete waivers with the specified
      waive_connected_net waiver ID.
  . Use "-edit -id" to edit the details of the waiver with the specified ID.
    Example: check_cov -waivers -edit -id 3 -end_line 27.
    NOTE: Use "-force" to add the waiver if the waiver matched cover items
    before the edit but failed to match any items after the edit.
  . Use "-get" to get information about one or more waivers.
    . Use "-all" to get information about all waivers.
    . Use "-id" to get information about one or more waivers with the
      specified ID(s).
    . Use "-tag" to get information about waivers with one or more specified
      tag(s).
    . Use "-no_return" to suppress the output of the command.
    NOTE: The return of "-get" is always a dictionary with waiver ID as keys,
    and it can be suppressed with the "-no_return" switch.
  . Use "-list_matching_cover_items" to get information about one or more
    waivers.
    . Use "-all" to get a list of the matching cover items for all waivers.
    . Use "-id" to get a list of the matching cover items for the specified
      waiver ID(s).
    . Use "-tag" to get information about waivers with one or more specified
      tag(s).
  . Use "-accept_update" to accept source location updates for persistent
    waivers, that is, waivers that have been imported after RTL changes.

  . Scenarios for waiver applicability when RTL is out of sync with earlier
    applied waivers follow:
    . Comments are added to RTL file causing lines out of sync - Waiver is
      added and source location is updated as per new source location of
      cover-items.
    . New RTL lines are added or some expression is modified causing file
      out of sync - Waiver not added by default. Use "-force" to add the
      waiver.
    . Waiver imported from different file location with relative or absolute
      file path in two different runs is updated as long as the file name is
      the same.
    . Waiver imported with invalid file name reports an error and the waiver
      is not imported.

-list_related_group
    [-status (unchecked | undetectable)]
    [-signal {source_signal_arg_list}]
```

```
    Lists group of related cover items for a given signal list.

    The following switches can be used to further refine the output result:
    . Use "-status" to list the cover items of the specified checker status
      category. If "-status" is not specified, the default comes from the
      checker settings based on the "init" type. If no checker type is
      specified in the "init" command, the tool reports an error.
      . Use "unchecked" to report unchecked (yellow) cover items in the output
        signal to cover item list.
      . Use "undetectable" to report undetectable (red) cover items in the
        output signal to cover item list.
    . Use the "-signal" switch to specify the coverage signals for which you
      want to list cover properties. By default, "list_related_group" includes
      all calculated signals based on the "status" switch.

-assert_analysis -assert assertion_name [-set_bound bound_value | -clear_bound]
    Set an artificial bound to an assertion in coverage assert analysis.
    NOTE:
    . This artificial bound will not alter proof results.
    . One command must be issued per assertion.
    . "-set_bound" will compute assert analysis independently of FPV result.
    . Use "-assert" to specify the assertion which bound will be altered.
      If "-assert" is not specified, the tool issues an error.
      NOTE: Assertions cannot be in Proven or CEX status.
    . Use "-set_bound" to specify the bound value for your assertion.
      NOTE: All bound values must be positive integers.
    . Use "-clear_bound" to reset a specific assertion's bound to the FPV value.

-configure_gui
    [-task (task_list)]
    [-exclude (  waived | disabled | unreachable | deadcode | reset | const
               | assume | synthesis_optimized)]
    Set up the coverage GUI environment through the command line.

    . Use "-task" to select the tasks you want to specify
      for "Coverage Analysis" and "Assert Analysis".
    . Use "-exclude" to exclude from "Coverage Analysis" cover items that were
      waived, disabled, or unreachable, or caused by deadcode, reset,
      constant propagation, assumptions, or synthesis optimized.
      NOTE: Use an empty list to clean this field.

-configure_gui -instance instance -model cover_item_model
    [-hierarchical]
```

```
    Open Coverage "Detailed" view through the command line.

    . Use "-instance" to go to the "Detailed" view of "Coverage Analysis"
      of the specified instance.
    . Use "-model" to specify which cover item model the report will have.
    . Use "-hierarchical" to open the report in hierarchical mode.
      NOTE: The report default mode is local.

-configure
    [-checker_mode (coi | proof_core | proof_core_high_precision | mutation)]

    Configure Coverage settings for the current session. This command applies
    to the Coverage GUI and the commands "check_cov -list", "check_cov -report",
    "check_cov -measure", and "database -export_unicov".

    Use "-checker_mode" to select the active Checker Mode for the current
    session. This applies to the Coverage GUI and the commands "check_cov
    -list", "check_cov -report", "check_cov -measure", and "database
    -export_unicov".
```

check_csr

Prints requested CSR information or executes the requested action.

Associated App(s)

CSR Verification

Syntax

```
check_csr -list instance

check_csr -list (register | addr_offset | addr_width | data_width)
                 (-instance instance_name)

check_csr -list (data_width | addr_offset | addr | access_type | reset_value | field)
                 (-instance instance_name)
                 (-register register_name)

check_csr -list (data_width | msb | lsb | access_type | reset_value)
                 (-instance instance_name)
                 (-register register_name)
                 (-field field)

check_csr -merge -ipxact ipxact_file_name
          -csv csv_file_name -ipxact_out ipxact_out_file_name

check_csr -generate_extension output_csv_file
    (-ipxact |-csv) input_file
    [-component component_name]
    [-maps map_name]
    [-force]

check_csr -load csv_file_name
           [-use_long_field_names]
           -instance instance_name
           [-load_extension ext_file]
           [-unified_check]
           [-partitioned_checks]
```

```
          [-addr_interaction_covers]
          [-unassigned_field_checks (constant | zero)]
          [-create_vplan]

check_csr -clear [-instance instance_name]

check_csr -prove
        [-instance instance_name
            [-register register_name
                [-field field_name]
            ]
        | -task task_name
        | -property property_list]
        [-bg]

check_csr -generate_template file [-force]

check_csr -load_ipxact [ipxact_file_name] -generate_csv csv_file_name
          [-maps map_list]
          [-force]
          [-component spirit:component_name]
          [-no_debug]

check_csr -load_ipxact [ipxact_file_name] -instance instance_name
          [-load_extension ext_file]
          [-maps map_list]
          [-unified_check]
          [-partitioned_checks]
          [-addr_interaction_covers]
          [-unassigned_field_checks (constant | zero)]
          [-component spirit:component_name]
          [-no_debug]
          [-volatile (read_undefined | write_undefined | ignore)]
```

Returns

A list of requested objects or the output of a requested action.

Detailed Description

Use this command to get information from the CSV workbook that defines

```
the CSR registers format or to request actions related to this workbook. This
command supports the Jasper Apps CSR format and IEEE 1685(TM) IP-XACT.

You can instantiate multiple CSR maps in each CSR verification execution flow
in an analysis session. Each workbook is associated with one instance. For each
instance, the tool generates properties for each field defined in the workbook,
according to the field access type.

This command has three scopes, the instance/workbook scope, the register
scope, and the field scope.

NOTE:
. You must elaborate the design before using this command.
. CSR cannot be used in conjunction with SEC, SPV, and X-Prop.

List Switches
-------------
Important things to know:
. Use "-instance" to get a list related to the specified instance.
. Use "-register" to get a list related to the specified register for the
  specified instance.
. Use "-field" to list only information defined for the specified field
  for the specified register for the specified instance.

-list instance
    List the defined instance name.

-list (register | addr_offset | addr_width | data_width)
      -instance instance_name
    List the information for the given instance.
    . Use "register" to list all CSR names defined in the workbook.
    . Use "addr_offset" to list the address offset defined for all CSR
      definitions.
    . Use "addr_width" to list the address width defined for all CSR
      definitions.
    . Use "data_width" to list the default data width defined for all CSR
      definitions.

-list (data_width | addr_offset | addr | access_type | reset_value | field)
      -instance instance_name -register register_name
    List the information for the given register of the given instance.

    . Use "data_width" to list the data_width for the specified register.
```

```
    . Use "addr_offset" to list the address offset defined for the specified
      register.
    . Use "addr" to list the address to check for each register.
    . Use "access_type" to list the access type for the specified register.
    . Use "reset_value" to list the reset value for the specified register.
    . Use "field" to list all fields defined for the specified register

-list (data_width | msb | lsb | access_type | reset_value)
      -instance instance_name -register register_name -field field_name
    List the information for the field defined under the given register and instance
name.

    . Use "data_width" to list the data_width for the specified field.
    . Use "msb" to list the most significant bit value of each field.
    . Use "lsb" to list the least significant bit value of each field.
    . Use "access_type" to list the access type for the specified field.
    . Use "reset_value" to list the reset value of each field.

Action Switches
---------------

-merge -ipxact ipxact_file_name
       -csv csv_file_name -ipxact_out ipxact_out_file_name
    Merge the ipxact input file with the csv file similarly to the REG App
    corresponding merge command.

-generate_extension output_csv_file
     (-ipxact |-csv) input_file
     [-component component_name]
     [-maps map_name]
     [-force]
     Generate a template extension file for the specified IP-XACT or CSV file.
     You can modify this file to add register- or field-specific vendor
     extensions. You can load the modified file can using the command
     "check_csr -load -load_extension".
     . Use "-ipxact" to specify IP-XACT input.
     . Use "-csv" to specify CSV input.
     . Use "-map" to generate a template for the specified map name.
     . Use "-component" to generate a template for the specified IP-XACT
       component.
     . Use "-force" if the file already exists and you want to overwrite it.

-load csv_file_name [-use_long_field_names]
```

```
  -instance instance_name
  [-load_extension ext_file]
  [-unified_check]
  [-partitioned_checks]
  [-addr_interaction_covers]
  [-unassigned_field_checks (constant | zero)]
  [-create_vplan]
Load the properties generated by the specified workbook file in CSV
format.
. Use "-use_long_field_names" to postfix property names with "_msb_lsb".
  NOTE: This switch is particularly useful if you provide the same name for
  multiple fields, such as "reserved".
. Use "-instance" to bind atomic-level properties to the glue logic of the
  CSR bus interface.
. Use "-load_extension" to load a file containing vendor extensions. You
  can generate the template for vendor extension files using the command
  "check_csr -generate_extension". This feature helps you modify the
  IP-XACT file, and it helps in merging vendor extensions across IP-XACT
  versions.
. Use "-unified_check" to generate the single assertion AST_DATA_INTEGRITY,
  the unified data integrity check, which comprises the reset value check
  and read after write checks.
. Use "-partitioned_checks" to generate the partitioned checks:
  . AST_RESET_VALUE
  . AST_BUS_READ_AFTER_BUS_WRITE
  . AST_BUS_READ_AFTER_DIRECT_WRITE
  . AST_BUS_WRITE_BUS_READ_IMMEDIATE
  . AST_BUS_WRITE_DIRECT_READ_IMMEDIATE
  . AST_DIRECT_READ_AFTER_BUS_WRITE
  . AST_DIRECT_READ_AFTER_DIRECT_WRITE
  . AST_DIRECT_WRITE_BUS_READ_IMMEDIATE
  . AST_DIRECT_WRITE_DIRECT_READ_IMMEDIATE
  NOTE:
  . If you do not specify "-unified_check" or "-partitioned_checks", the
    default behavior is that of "-unified_check".
  . AST_DATA_INTEGRITY comprises the following:
    . AST_RESET_VALUE
    . AST_BUS_READ_AFTER_BUS_WRITE (it comprises
      AST_BUS_WRITE_BUS_READ_IMMEDIATE)
    . AST_BUS_READ_AFTER_DIRECT_WRITE (it comprises
      AST_DIRECT_WRITE_BUS_READ_IMMEDIATE)
    . AST_DIRECT_READ_AFTER_BUS_WRITE (it comprises
      AST_BUS_WRITE_DIRECT_READ_IMMEDIATE)
```

```
    . AST_DIRECT_READ_AFTER_DIRECT_WRITE (it comprises
      AST_DIRECT_WRITE_DIRECT_READ_IMMEDIATE)

. Use "-addr_interaction_covers" to generate the following covers:
  . For fields:
    . COV_BUS_READ_OTHER_ADDR_THEN_BUS_READ
    . COV_BUS_WRITE_OTHER_ADDR_THEN_BUS_READ
  . For instances:
    . COV_BUS_READ_OUTSIDE_ADDR_BLOCKS
    . COV_BUS_WRITE_OUTSIDE_ADDR_BLOCKS

. Use "-unassigned_field_checks" to generate the checks for the undefined
  fields according to the two possible semantics: read constant undefined
  value or read zero.

  NOTE: "-unassigned_field_checks" is an initial release to gather feedback
  from early adopters and finalize implementation for an upcoming release.
  Contact support@cadence.com to provide feedback or comments.

. Use "-create_vplan" to export a .csv vPlan file that can be used with the
  Cadence vManager tool. The tool saves the exported file to the session
  directory as "csr_vplan.csv".
  When you import CSR results to vManager, the vPlan presents the results
  in a logical structure that follows the register map definition.
  Register names can be expanded to show fields, which in turn expand to
  show low-level checks and coverage for individual access policies.

You can instantiate multiple CSR maps, in each CSR verification execution
flow in an analysis session. Each workbook is associated with one
instance.

For each instance, the tool generates one property module subinstance for
each field defined in the workbook. Each subinstance presents properties
according to the access type defined for the field.

NOTE:
. The "-load" switch also loads the GUI.
. "-load" is not valid for the "-register" scope.
. Register and field names specified in the CSV file must use legal Verilog
  identifier syntax. That is, if you use spaces or any special characters
  (for example: /, [, or ]) in register or field names, the tool issues an
  error during compilation of the CSV file.
```

```
-clear [-instance instance_name]
    Clear the data loaded for the CSR Verification App.

    Use "-instance" to clear only the data related to the specified instance.

-prove
        [-instance instance_name
            [-register register_name
                [-field field_name]
            ]
        | -task task_name
        | -property property_list]
        [-bg]
    Prove the properties generated by CSR Verification App from the loaded
    workbook.
    . Use "-instance" to prove only properties related to the specified
      instance.
    . Use "-register" to prove only properties related to the specified
      register.
    . Use "-field" to prove only properties related to the specified field for
      the specified register.
    . Use "-task" to prove the CSR properties in the specified task.
    . Use "-property" to prove the given list of CSR properties.
    . Use "-bg" to run the command in the background and continue to accept and
      run new commands.

-generate_template file [-force]
    Generate a CSV template in which to complete the CSR map.
    . Use "-force" to overwrite an existing file.
    NOTE: Register and field names specified in the CSV file must use legal
    Verilog identifier syntax. That is, if you use spaces or any special
    characters (for example: /, [, or ]) in register or field names, the tool
    issues an error during compilation of the CSV file.

IP-XACT-Related Switches
------------------------
-load_ipxact [ipxact_file_name] -generate_csv csv_file_name [-maps maps] [-force]
    [-component spirit:component_name] [-no_debug]
    Generate a CSV file for "check_csr -load".

    . Use "-maps" to limit the CSV file content to specific maps.
    . Use "-force" to overwrite an existing CSV file.
    . Use "-component" when the IP-XACT files describe multiple
```

```
      instances of the specified component.
    . Use "-no_debug" to generate the map without any reference to the IP-XACT
      file names and line numbers.

    NOTE:
    . Use "ipxact -load" prior to using this command.
    . The IP-XACT file must contain a single IP-XACT component description
      (Section 6 of the IEEE 1685).

-load_ipxact [ipxact_file_name] -instance instance_name
    [-load_extension ext_file] [-maps maps]
    [-unified_check] [-partitioned_checks]
    [-addr_interaction_covers]
    [-unassigned_field_checks (constant | zero)]
    [-component spirit:component_name] [-no_debug]
    [-volatile (read_undefined | write_undefined | ignore)]
    Load the maps in the IP-XACT file as if they are from a CSV file.

    . Use "-load_extension" to load a file containing vendor extensions. You
      can generate the template for vendor extension files using the command
      "check_csr -generate_extension". This feature helps you modify the
      IP-XACT file, and it helps in merging vendor extensions across IP-XACT
      versions.
    . Use "-maps" to limit the CSV file content to specific maps.
    . Use "-component" when the IP-XACT files describe multiple
      instances of the specified component.
    . Use "-unified_check" to generate the single assertion AST_DATA_INTEGRITY,
      the unified data integrity check, which comprises the reset value check
      and read after write checks.
    . Use "-partitioned_checks" to generate the partitioned checks.
    . Use "-addr_interaction_covers" to generate the following covers:
      . For fields:
        . COV_BUS_READ_OTHER_ADDR_THEN_BUS_READ
        . COV_BUS_WRITE_OTHER_ADDR_THEN_BUS_READ
      . For instances:
        . COV_BUS_READ_OUTSIDE_ADDR_BLOCKS
        . COV_BUS_WRITE_OUTSIDE_ADDR_BLOCKS
    . Use "-unassigned_field_checks" to generate the checks for the undefined
      fields according to the two possible semantics: read constant undefined
      value or read zero.
    . Use "-no_debug" to load the map without any reference to the IP-XACT
      file names and line numbers.
    . Use "-volatile" to configure how the tool handles the IP-XACT specific
```

```
    volatile element.
    . Use "write_undefined" to generate the attribute BUS_WRITE_CHECK =
      write_undefined.
    . Use "read_undefined" to generate the attribute BUS_READ_CHECK =
      read_undefined.
    . Use "ignore" to treat the container as if the volatile attribute was not
      set.
    NOTE:
    . You can find more information about the attributes in the "Jasper
      Control and Status Register Verification App User Guide" (Help -
      Application Guides - CSR Verification App User Guide).
    . According to Section 6 of the IEEE 1685(TM), "volatile (optional) when
      true indicates in the case of a write followed by read, or in the case
      of two consecutive reads, there is no guarantee as to what is returned
      by the read on the second transaction or that this return value is
      consistent with the write or read of the first transaction."

  NOTE:
  . Use "ipxact -load" prior to using this command.
  . The IP-XACT file must contain a single IP-XACT component description
    (Section 6 of the IEEE 1685).
```

Example

```
check_csr -list instance
check_csr -list field -instance inst0 -register reg1
check_csr -list msb -instance inst0 -register reg1 -field fA
check_csr -list addr -instance inst0 -register reg1
check_csr -load csr1.csv -instance csr1_inst
check_csr -prove -instance inst0
check_csr -prove -instance inst0 -register r1
check_csr -prove -instance inst0 -register r2 -field fA

// Loading multiple maps
analyze -sv design.v
check_csr -load file1.csv -instance instance1
check_csr -load file2.csv -instance instance2
check_csr -load file3.csv -instance instance3
elaborate -top design
check_csr -prove
```

See Also

ipxact

check_dlh

Uses formal verification hunting to increase the chance of finding deadlocks.

Syntax

```
check_dlh -generate -cover [-depth N] [-stride N] [-seed N] [-num N]
   [-instance instance_tcl_list]

check_dlh -generate -trap [-disable_condition expression] [-changed N]
   [-stride N] [-depth N] [-signal signal_list] [-instance instance_tcl_list]
   [-type (fsm | counter | output | pattern)]
   [-pattern reg_expr | reg_expr_list]

check_dlh -verify [-task task_name] [-property property_name [-trace_id N]]
   [-strategy] [-engine_mode engine_list] [-time_limit time] [-max_jobs N]
   [-tail_length N] [-bg]

check_dlh -report [-engine_config] [-engine_stats] [-job_score]
   [-sort_by (cex | covered | first_trace_attempt | job | local_overconstraint
              | max_segment_length | max_trace_length | mode
              | random_diversification_factor | rarity_job_score | restarts
              | results_job_score | searches_from_reset
              | segment_length_exceeded | segment_time_limit
              | segment_time_out | start_state_count | start_time
              | tail_length | total_searches | unprocessed)]
```

Returns

When verifying deadlock without "-bg", these are the return values:

. "check_dlh -generate" does not have any return value. Properties are created in the Property Table.

. "check_dlh -verify" returns the number of deadlocks found.

. "check_dlh -report" returns the list of hunt tags run to verify deadlock along with their respective specific data such as mode, strategy, and the number of properties with each status. In addition, it returns a list of deadlock properties.

Detailed Description

Use this command to hunt for deadlocks using formal verification.

NOTE: This command is an initial release to gather feedback from early adopters and finalize implementation for an upcoming release.

Contact support@cadence.com to provide feedback or comments targeting the upcoming production version.

```
-generate -cover [-depth N] [-stride N] [-seed N] [-num N]
                 [-instance instance_tcl_list]
    Configure the environment with covers to help find deadlocks.

-depth N
    . Use "-depth" to set the depth of enable properties to be generated.
      Default value is 32.

      This switch determines how deep in a design the covers will be generated
      (related just to flop/clock enable signals).

-stride N
    . Use "-stride" to set the stride between enable cover properties to be
      generated. Default value is 4.

      This switch specifies the size of the "jump" between auto-generated
      properties (related just to flop/clock enable signals).
```

```
-seed N
    . Use "-seed" to make possible re-generating identical covers in multiple
      runs. Default is 0, which means random covers are generated (not possible
      to re-generate).

-num N
    . Use "-num" to set the number of cover properties the tool will try to
      generate. Default value is 300.

-instance instance_tcl_list
    . Use "-instance" to set the list of instances from which signals will be
      used to generate covers. Default is all instances.

-generate -trap [-disable_condition expression] [-changed N] [-stride N]
                [-depth N] [-signal signal_list] [-instance instance_tcl_list]
                [-type (fsm | counter | output | pattern)]
                [-pattern reg_expr | reg_expr_list]
    Configure the environment with properties to help find deadlocks.

-disable_condition expression
    . Use "-disable_condition" to set the expression that will be added to
      "disable iff" conditions of the covers and assertions to be generated.
      Default is no condition to disable.

-changed N
    . Use "-changed" to set the number of times you want to see important
      covers and assertions changed. Default value is 1.

-signal signal_list
    . Use "-signal" to set the list of signals to generate properties for.
      If you do not specify "-signal", the default value is all meaningful
      signals based on the "-instance" and "-pattern" switches.

-type (fsm | counter | output | pattern)
    . Use "-type" to specify the type of signals to generate traps for. If
      you do not specify a type, the tool generates properties for counters,
      FSMs, and flop-related signals that match the pattern list. Options
      and default values are {fsm, counter, output, pattern}.

-pattern reg_expr | reg_expr_list
    . Use "-pattern" to generate properties based on a pattern list. The tool
      filters flop|counter|fsm related signals based on the specified pattern.
      The default pattern list is {abort alloc blocked blocking bypassing drop
```

```
      full locked locking pend pending pop push stall wait}

-verify [-task task_name] [-property property_name [-trace_id N]] [-strategy]
        [-engine_mode engine_list] [-time_limit time] [-max_jobs N]
        [-tail_length N] [-bg]
    Hunt for possible deadlocks.

-task task_name
    . Use "-task" to specify the target task that contains DLH trap assertions
      to search for deadlocks.
      Default is current task.

-property property_name [-trace_id N]
    . Use "-property" to specify the target DLH trap assertions to search
      for deadlocks.

-strategy
    . Use "-strategy" to specify a hunt strategy to be used in parallel when searching
      for DLH trap assertions. For now, only state_swarm mode strategies are allowed.
      In case any DLH trap assertion is still undetermined or unprocessed, besides
      validating determined DLH trap assertions, state_swarm is also dispatched
      to search for more DLH trap assertions.

-engine_mode engine_list
    . Use "-engine_mode" to specify the engines to be used when verifying
      possible deadlocks. Default value is {B AM N I K U}.

-time_limit time
    . Use "-time_limit" to specify the deadlock verification time limit.
      Default value is 30m.

-max_jobs N
    . Use "-max_jobs" to specify the number of threads to be dispatched when
      hunting for deadlocks. Default value is 10.

-tail_length N
    . Use "-tail_length" to specify the number of cycles to go backward from
      the cycle where the possible deadlock is found when verifying possible
      deadlocks. Default value is 10.

-bg
    . Use "-bg" to run deadlock verification in the background.
```

```
-report [-engine_config] [-engine_stats] [-job_score]
        [-sort_by (cex | covered | first_trace_attempt | job
                   | local_overconstraint | max_segment_length
                   | max_trace_length | mode | random_diversification_factor
                   | rarity_job_score | restarts | results_job_score
                   | searches_from_reset | segment_length_exceeded
                   | segment_time_limit | segment_time_out | start_state_count
                   | start_time | tail_length | total_searches | unprocessed)]
    Report results of deadlock verifications.

    The report lists the number of properties for each tag along with their
    respective status. It also prints the list of jobs that were fired during
    its execution.

    . Use "-engine_config" to see the specific configurations of each job.

      NOTE: Configurations are only shown if you use one of the following
      engines: L, B, or Bm.

    . Use "-engine_stats" to see the specific statistics of each job.

      NOTE: Statistics are only shown if you use engine L.

    . Use "-job_score" to get two kinds of scores for each job: "Results Job
      Score" and "Rarity Job Score." The former is based on the number of
      determined properties by each job. The latter is based on the rarity
      of properties found by each job. Properties that are found by many jobs
      are considered "easy" targets, while properties that are found by just
      one or a few jobs have bigger weights and thus elevate the rarity job
      score. The higher the scores, the better.

      NOTE: You can use scores to compare jobs within the same tag. Do not
      use scores to compare jobs with different tags.

    . Use "-sort_by" to specify the sorting column of the output table.

      By default, the tool lists tags in the order they were executed
      and sorts jobs of the same tag in ascending order. When sorting by
      "start_time" or "mode", you are changing the way tags are listed.
      Otherwise, you are choosing columns with job-related information.
      Some columns only appear when you use the switches "-engine_stats" and
      "-engine_config" along with some modes. Others will only be available
      when using the "-job_score" switch. When you try to sort by a column that
```

```
  is not available in the output table, the tool issues a warning message
  and uses the default sorting.

. Meaning of each column in the report:

 Default Columns
 ---------------
 . Mode: strategy mode. Example: "state_swarm", "cycle_swarm".
 . Strategy: Strategy name. If you do not configure a name, the tool
   generates it. Example: "<default>".
 . Tag: Run name. If you do not configure a name, the tool generates it.
   Example: "<default>:0".
 . Properties: Number of properties considered by the run/job.
 . Processing: Number of properties that are being processed by the
   run/job. This column is only available if you use the switch "-status".
 . Unproc: Number of properties unprocessed by the run/job.
 . Total CEX: Number of assertion properties for which the job/tag found a
   counterexample. See also "-detailed" switch.
 . Total Covered: Number of cover properties for which the job/tag found a
   cover trace. See also "-detailed" switch.
 . Seed: Seed value used in the run. See also "-seed".
 . Result: Result string returned by the Tcl command at the end of the
   run. This column is not available if you use the switch "-status".

 Engine Configuration Columns
 ----------------------------
 These columns are available when using the switch "-engine_config".
 . Start State Count: Engine L job start state used to search for new
   traces. See also "-start_state_count".
 . Tail Length: Engine L job starting point for a new search that is N
   cycles before the end of found traces. See also "-tail_length".
 . Max Segment Length: Maximum number of trace attempts for an engine L job
   from its start state. The trace attempts number corresponds to the job
   segment length. See also "-max_segment_length".
 . Segment Time Limit: Amount of time the engine L job spends searching
   from each start state. See also "-segment_time_limit".
 . Random Diver. Factor: Diversification value used by the engine L job.
   See also "-random_diversification_factor".
 . First Trace Attempt: The cycle at which jobs from engines L, B, and Bm
   will make their first trace attempt. See also "-first_trace_attempt".
 . Per Prop. Time Limit: Amount of time allowed for the engine B job to
   spend on each property. See also "-per_property_time_limit" and
   "set_prove_per_property_time_limit".
```

```
    . Per Prop. Time Limit Fact.: Engine B job factor used to grow the time
      shown in "Per Prop. Time Limit" column. See also
      "-per_property_time_limit_factor" and
      "set_prove_per_property_time_limit_factor".

    NOTE: Engine configurations are used to configure the jobs in the
    beginning of the run.

    Engine Statistics Columns
    -------------------------
    These columns are available when using the switch "-engine_stats".
    . Segment Time Out: Number of times the engine L job timed out searching
      from the current start state before switching to a new unused start
      state.
    . Segment Length Exceeded: Number of times the engine L job reached a
      segment length limit.
    . Searches From Reset: Number of times the engine L job took "reset" as
      the start state.
    . Total Searches: Total number of start states used by the engine L job.
      NOTE: "Total CEX" and "Total Covered" might be greater than the total
      number of searches due to incidental traces.
    . Local Overcon.: Number of times the engine L job found a dead-end
      situation because of overcontraints.
    . Restarts: Number of times the engine L job was restarted by ProofGrid
      or by the user.
    . Max Trace Length: Max trace length reached by the engine L job.

    NOTE: Engine statistics are collected during the run and reflect each
    job behavior, which is influenced directly by the job configurations.

    Job Score Columns
    -----------------
    These columns are available when using the switch "-job_score".
    . Results Job Score: Score value based on the number of determined
      properties by each job.
    . Rarity Job Score: Score value based on the rarity of properties found by
      each job. Properties that are found by many jobs are considered "easy"
      targets, while properties that are found by just one or a few jobs have
      bigger weights and thus elevate this score.

-engine_config
    . Use "-engine_config" to see the specific configurations of each job.
```

```
      NOTE: Configurations are only shown if you use one of the following
      engines: L, B, or Bm.

-engine_stats
    . Use "-engine_stats" to see the specific statistics of each job.

      NOTE: Statistics are only shown if you use engine L.

-job_score
    . Use "-job_score" to get two kinds of scores for each job: "Results Job
      Score" and "Rarity Job Score." The former is based on the number of
      determined properties by each job. The latter is based on the rarity
      of properties found by each job. Properties that are found by many jobs
      are considered "easy" targets, while properties that are found by just
      one or a few jobs have bigger weights and thus elevate the rarity job
      score. The higher the scores, the better.

      NOTE: You can use scores to compare jobs within the same tag. Do not
      use scores to compare jobs with different tags.

-sort_by (cex
          | covered
          | first_trace_attempt
          | job
          | local_overconstraint
          | max_segment_length
          | max_trace_length
          | mode
          | random_diversification_factor
          | rarity_job_score
          | restarts
          | results_job_score
          | searches_from_reset
          | segment_length_exceeded
          | segment_time_limit
          | segment_time_out
          | start_state_count
          | start_time
          | tail_length
          | total_searches
          | unprocessed)
    . Use "-sort_by" to specify the sorting column of the output table.
```

By default, the tool lists tags in the order they were executed and sorts jobs of the same tag in ascending order. When sorting by "start_time" or "mode", you are changing the way tags are listed. Otherwise, you are choosing columns with job-related information. Some columns only appear when you use the switches "-engine_stats" and "-engine_config" along with some modes. Others will only be available when using the "-job_score" switch. When you try to sort by a column that is not available in the output table, the tool issues a warning message and uses the default sorting.

Example

```
# Verifying Deadlocks with the Default Settings
# ---------------------------------------------
# Checking deadlocks supports a default setting that does not require any
# configuration. This flow configures the deadlock environment and uses bug
# hunting to search for deadlocks. It first uses the default "hunt" strategy
# to determine covers and traps previously generated and then verifies the
# traces looking for deadlocks.
analyze requirement.v
elaborate
reset rst
clock clk
task -create deadlock -set
check_dlh -generate -cover
check_dlh -generate -trap
assert -set_store_trace unlimited *
hunt -run -task deadlock
check_dlh -verify

# Customizing a Sequential Deadlock Hunting Strategy Flow
# -------------------------------------------------------
# You can customize the strategy used to hunt for deadlocks by creating a new
# strategy with the desired mode and then customizing it. This flow hunts using
# newly created state swarm strategy ST1, which has been customized to change
# the max number of jobs and the time limit.
analyze requirement.v
elaborate
reset rst
clock clk
task -create deadlock -set
check_dlh -generate -cover
```

```
check_dlh -generate -trap
hunt -config -strategy ST1 -max_jobs 30 -time_limit 2h
hunt -run -strategy ST1 -task deadlock
check_dlh -verify

# Customizing a Parallel Deadlock Hunting Strategy Flow
# ----------------------------------------------------
# You can customize the state_swarm strategy used to hunt for deadlocks by
# creating a new strategy and customizing it. In this flow, instead of running
# "hunt" in a preliminary step, you include the recently created strategy with
# the "check_dlh -verify" command using the "-strategy" switch. This flow
# dispatches a parallel process for deadlock verification.
analyze requirement.v
elaborate
reset rst
clock clk
task -create deadlock -set
check_dlh -generate -cover
check_dlh -generate -trap
hunt -config -strategy ST1 -max_jobs 30
check_dlh -verify -task deadlock -strategy ST1 -time_limit 2h

# Customizing the Instances to Find Deadlocks
# -------------------------------------------
# You can customize the instances to search for deadlocks. This flow hunts
# for deadlocks in "list_ctrl" instance and configures helper covers in 3
# instances ("list_ctrl", "drop_ctrl", and "intr_ctrl").
analyze requirement.v
elaborate
reset rst
clock clk
task -create deadlock -set
check_dlh -generate -cover -instance {list_ctrl drop_ctrl intr_ctrl}
check_dlh -generate -trap {list_ctrl}
hunt -run -max_jobs 30 -task deadlock -time_limit 1h
check_dlh -verify
```

check_fsv

Checks for the observability of faults on user-defined strobing points.

Associated App(s)

FSV

Syntax

```
check_fsv -init
      [-no_miter]

check_fsv -fault
      ( -add signal_tcl_list
            [-type (sa0 | sa1 | seu | set | open | all)]
            [-set_hold_time start_time[:end_time]]
            [-time_window start_time[:end_time]]
            [-condition sva_condition]
            [-show_skipped]
            [-bus_fault_width N]
            [-bus_fault_partition N]
            [-memory_fault_width N]
            [-return (id | dict)]
            [-regexp]
            [-silent]
       |-add [-from_file file_name]
            [-hier_path path_to_design_top]
            [-ifss_class ifss_classification]
            [-set_hold_time time]
            [-time_window start_time[:end_time]]
            [-condition sva_condition]
            [-show_skipped]
            [-return id]
            [-silent]
       |-add [-multiple id_tcl_list]
            [-number max_fault_number]
            [-include_sub]
            [-return id]
```

```
     [-silent])
|-remove id_tcl_list
|-remove ( -node signal_tcl_list [-regexp]
   |-type (sa0 | sa1 | seu | set | multi | open)+)+
     [-return dict]
     [-silent]
|-show id_tcl_list
|-list
     [-node signal_tcl_list [-regexp]]
     [-type (sa0 | sa1 | seu | set | multi | open)+]
     [-class (unprocessed | safe | dangerous | unknown)+]
     [-coi_fo (unprocessed | in | out | unknown)+]
     [-coi_co (unprocessed | in | out | unknown)+]
     [-constant (unprocessed | activated | unactivatable | unknown)+]
     [-time_window start_time[:end_time] [-exact_match]]
     [-primary]
     [-dominant]
     [-dominated]
     [-equivalent]
     [-propagatability (  unprocessed | propagated | unpropagatable
                        | unknown)+ [-formal |-structural]]
     [-detectability (unprocessed | detected | undetectable | unknown)+
          [-formal |-structural]]
     [-always_propagated (  unprocessed | always_propagated
                          | not_always_propagated | unknown)+]
     [-always_detected (  unprocessed | always_detected
                        | not_always_detected | unknown)+]
     [-propagated_always_detected (  unprocessed
                                   | propagated_always_detected
                                   | propagated_not_always_detected
                                   | unknown)+]
     [-activatability (  unprocessed | activated | unactivatable
                       | unknown)+ [-formal |-structural]]
     [-abstract_result (constrained | default)
          [ -fo_result (  unprocessed | unobserved | unobservable
                        | dangerous | alwaysobserved | unknown)+
           |-co_result (  unprocessed | undetected | undetectable
                        | detected| alwaysdetected | unknown)+] ]
     [-task task_tcl_list]
     [-silent]
|-get (  node | type | class | injection | settime | coi_fo | coi_co
       | constant | property | task | result | observed_strobes
       | primary | equivalent | dominated | dominant | weight
```

```
              | fanin_bbox | fanout_bbox | co_result | fo_result
              | distance_co | distance_fo)
        id_tcl_list
            [-dominance_type (observed | unobservable)]
            [-check_type (  propagatability | detectability | always_propagated
                          | always_detected | propagated_always_detected
                          | activatability)]
            [-silent]

check_fsv -strobe
        -add (signal_tcl_list | property_list)
            [-functional]
            [-checker [-checker_mode (diff | assert [-assert_value (0 | 1)])]]
            [-condition sva_condition]
       |-remove ( id_tcl_list
                 |(-node (signal_tcl_list | property_tcl_list)
                   [-regexp] [-functional |-checker]+)
                 |(-functional |-checker)+]
       |-show id_tcl_list
       |-list
            [-node (signal_tcl_list | property_tcl_list) [-regexp]]
            [-type (fo | co)]*
            [-silent]
       |-get (node | type | checker_mode | condition | fanin_bbox | fanout_bbox |
fanin_fault)
        id_tcl_list
            [-silent]

check_fsv -structural
        [-coi (on | fo | co | off)]
        [-constant (on | off)]
        [-fault_relations (on | fo | co | off)]
        [-propagation_analysis (on | fo | co | off)]
        [-constant_propagation (on | fo | co | off)]
        [-distance (on | fo | co | off)]
        [-silent]
        [-list_constants (seq | all)]
        [-clear_seq_constants]

check_fsv -generate
        [-max_faults_per_task N]
        [-clock_cycle_time cycle_time]
        [-fo_max_delay cycle_time]
```

```
        [-co_max_delay cycle_time]
        [-fo_co_max_delay cycle_time]
        [-propagatability (on | off)]
        [-detectability (on | off)]
        [-always_propagated (on | off)]
        [-always_detected (on | off)]
        [-propagated_always_detected (on | off)]
        [-activatability (on | off)]
        [-strobe_optimization (on | off)]
        [-task task_name]
        [-force]
        [-copy_env]
        [-use_bus_faults]
        [-use_memory_faults]
        [-id id_tcl_list]
        [-sanity]
        [-silent]

check_fsv -report
       ( [-text text_file]
        |[-html html_file [-launch_html_browser]]
        |[-xml xml_file])
       ( [-type (sa0 | sa1 | seu | set | multi | open | all)+]
         [-class (unprocessed | safe | dangerous | unknown)+]
         [-coi_fo (unprocessed | in | out | unknown)+]
         [-coi_co (unprocessed | in | out | unknown)+]
         [-constant (unprocessed | activated | unactivatable | unknown)+]
         [-propagatability (unprocessed | propagated | unropagatable | unknown)+]
         [-detectability (unprocessed | detected | undetectable | unknown)+]
         [-always_propagated (  unprocessed | always_propagated
                              | not_always_propagated | unknown)+]
         [-always_detected (  unprocessed | always_detected
                            | not_always_detected | unknown)+]
         [-propagated_always_detected (  unprocessed
                                       | propagated_always_detected
                                       | propagated_not_always_detected
                                       | unknown)+]
         [-activatability (unprocessed | activated | unactivatable | unknown)+]
        |[-id id_tcl_list])
       ([-fault_relations
            [-show_dominance [-dominance_type (observed | unobservable)]]
            [-primary_only]
            [-hier_path path_to_design_top]])
```

```
        [-force]
        [-summary]
        [-aggregate]
        [-show_source_location]
        [-show_fault_ids]
        [-abstract_result (constrained | default)
             [ -fo_result (  unprocessed | unobserved | unobservable
                           | dangerous | alwaysobserved | unknown)+
              |-co_result (  unprocessed | undetected | undetectable
                           | detected| alwaysdetected | unknown)+] ]

check_fsv -abstract
       (-instance instance_tcl_list)
           [-mode (comb | mixed | seq)]
           [-connections connection_spec_tcl_list]
      |(-operator (multiplier | shift | adder | subtract | comparator | pow))
           [-include_instance {instance_list} [-regexp]]
           [-exclude_instance {instance_list} [-regexp]]
           [-include_module {module_list} [-regexp] [-recursive]]
           [-exclude_module {module_list} [-regexp] [-recursive]]
           [-minimum_width N]
           [-skip_const_operand]
      |(-bbox_mul |-bbox_div |-bbox_mod)
      |(-counter signal_list)
        (-task task_tcl_list |-env)
        [-silent]
      |-remove
        (-id id_tcl_list)
       |(-instance instance_tcl_list)
           [-task task_tcl_list]
        [-silent]
     |-get id_tcl_list
      [-silent]
     |-list
        [-instance instance_tcl_list]
        [-task task_tcl_list]
        [-env]
        [-type (instance | adder | subtract | multiplier | shift | comparator)]
        [-silent]
     |-clear

check_fsv -barrier
     -add (signal_tcl_list)
```

```
        [-condition sva_condition]
        [-value (0 | 1)]
    |-remove (signal_tcl_list)
    |-list
    |-clear

check_fsv -prove
      [-id id_tcl_list]
      [-check_type (propagatability | detectability | always_propagated
          | always_detected | propagated_always_detected | activatability)]*
      [-property property_name_tcl_list]
      [-task task_name_tcl_list]
      [-strategy (basic | clustering)]
      [-group_by (instance | signal | bit)]
      [-time_limit N]
      [-engine_mode engine_tcl_list]
      [-verbosity N]
      [-bg]
      [-keep_traces]
      [-sanity_check]
      [-silent]
      [-force]

check_fsv -visualize
      (-id id
       -check_type (activatability | propagatability | detectability
                    | always_propagated | always_detected
                    | propagated_always_detected))
     |(-sanity -task task_name)

check_fsv -fault_work
       ( -import directory_name
            [-use_test_const [-show_skipped]]
            [-fault_top fault_top]
            [ -hier_path hier_path
             |-hier_map hier_map (-from from -to to)]
            [-ucd_file ucd_file]
                [-fo_result (  unprocessed | unobserved | unobservable
                             | dangerous | alwaysobserved | unknown)+]
                [-co_result (  unprocessed | undetected | undetectable
                             | detected| alwaysdetected | unknown)+]
                [-type (sa0 | sa1 | seu | set)+]
            [-fault_list fault_list]
```

```
            [-fault_node fault_node [-regexp]]
            [-include_ut]
            [-include_equivalent]
            [-show_skipped]
        |-export [-id id_tcl_list]
            [-abstract_result (constrained | default)])
            [-fault_test test_name]

check_fsv -graph
 (-from signal_tcl_list -to signal_tcl_list)
 [-type (all | register | hierarchy | logic)]
 [-exclude_cell_contents]
 [-cells]

check_fsv -show_schematic ( -signal signal_list [-all |-driver | -port]
                           [-full_hierarchy])
                           |-instance instance_list

check_fsv -clear [all | result [-task task_tcl_list]]
```

Returns

Multiple return values.

Detailed Description

Use the "check_fsv" command for Functional Safety Verification (FSV). With this command, you can check for the observability of faults on user-defined strobing points.

NOTE: The FSV App supports only the loop handling modes "none", "break", and "break_functional".

```
-init
    [-no_miter]

    Initialize the settings required for the FSV App.

    Use "-no_miter" to disable the creation of a miter after elaborating
    the design.
    Note:
```

```
    . This option prevents you from using FSV formal analysis.
    . FSV does not accept the following "check_fsv" commands in this mode:
      "-abstract", "-generate" and "-prove".
    . FSV does not accept the following platform commands in this mode:
      "save", "restore".

-fault
    -add signal_tcl_list
        [-type (sa0 | sa1 | seu | set | open | all)]
        [-set_hold_time start_time[:end_time]]
        [-time_window start_time[:end_time]]
        [-condition sva_condition]
        [-show_skipped]
        [-bus_fault_width N]
        [-bus_fault_partition N]
        [-memory_fault_width N]
        [-return (id | dict)]
        [-regexp]
        [-silent]
   |-add [-from_file file_name]
        [-hier_path path_to_design_top]
        [-ifss_class ifss_classification])]
        [-set_hold_time start_time[:end_time]]
        [-time_window start_time[:end_time]]
        [-condition sva_condition]
        [-show_skipped]
        [-return id]
        [-silent]
   |-add [-multiple id_tcl_list]
        [-number max_fault_number]
        [-include_sub]
        [-set_hold_time time]
        [-time_window start_time[:end_time]]
        [-condition sva_condition]
        [-return id]
        [-silent]
   |-remove id_tcl_list [-return (id)] [-silent]
   |-remove ( -node signal_tcl_list [-regexp]
             |-type (sa0 | sa1 | seu | set | multi | open)+)+
        [-return dict]
        [-silent]
   |-show id_tcl_list
   |-list
```

```
        [-node signal_tcl_list [-regexp]]
        [-type (sa0 | sa1 | seu | set | multi | open)+]
        [-class (unprocessed | safe | dangerous | unknown)+]
        [-coi_fo (unprocessed | in | out | unknown)+]
        [-coi_co (unprocessed | in | out | unknown)+]
        [-constant (unprocessed | activated | unactivatable | unknown)+]
        [-time_window start_time[:end_time] [-exact_match]]
        [-primary]
        [-dominant]
        [-dominated]
        [-equivalent]
        [-propagatability (  unprocessed | propagated | unpropagatable
                           | unknown)+ [-formal |-structural]]
        [-detectability (unprocessed | detected | undetectable | unknown)+
             [-formal |-structural]]
        [-always_propagated (  unprocessed | always_propagated
                             | not_always_propagated | unknown)+]
        [-always_detected (  unprocessed | always_detected
                           | not_always_detected | unknown)+]
        [-propagated_always_detected (  unprocessed | propagated_always_detected
                                      | propagated_not_always_detected
                                      | unknown)+]
        [-activatability (unprocessed | activated | unactivatable | unknown)+
             [-formal |-structural]]
        [-abstract_result (constrained | default)
            [ -fo_result (  unprocessed | unobserved | unobservable | dangerous
                          | alwaysobserved | unknown)+
             |-co_result (  unprocessed | undetected | undetectable | detected
                          | alwaysdetected | unknown)+] ]
        [-task task_tcl_list]
        [-silent]
   |-get ( node | type | class | injection | settime | coi_fo | coi_co
          | constant | property | task | result | observed_strobes | primary
          | equivalent | dominated | dominant | weight | fanin_bbox
          | fanout_bbox | co_result | fo_result | distance_co | distance_fo)
id_tcl_list
        [-dominance_type (observed | unobservable)]
        [-check_type (  propagatability | detectability | always_propagated
                      | always_detected | propagated_always_detected
                      | activatability)] id_tcl_list
        [-silent]

    FSV Faults:
```

```
. Use "-add" to add fault nodes based on the specified signal list.
  NOTE:
  . The target should be a signal name.
  . Use "-regexp" to specify regular expression as target in addition to
    signal names.
  . Wide signals will be bit-blasted.

  . Use "-type" to specify the fault type you want to inject. You can
    specify more than one type with "+" as a delimiter or use "all" to
    inject all types. The default is "all".

    Supported types include the following:
    . sa0 - Stuck-at 0 forces a signal to a value (0).
    . sa1 - Stuck-at 1 forces a signal to a value (1).
    . seu - Single event upset inverts the value of a signal and holds the
            modified value until it is assigned a new value.
    . set - Single event transient inverts the value of a signal and holds
            the value for a certain period of time.
    . open - Forces the signal to be treated as a free input. In other
             words, it can have any possible value.

  . Use "-set_hold_time" to specify the hold time for SET type faults.
    Specify a time, [scale][unit], or a range, [scale][unit]:[scale][unit].
    NOTE: The default hold time for SET type faults is 1cc.

  . Use "-time_window" to specify the fault injection time window. The
    FSV App injects the fault within the specified time window.
    NOTE:
    . Use integers for (start:end) to specify the time interval followed by
      units (no spaces):
      . cc = clock cycle (default)
      . fs = femtosecond (10^-15) second
      . ps = picosecond  (10^-12) second
      . ns = nanosecond  (10^-9)  second
      . us = microsecond (10^-6)  second
      . ms = milisecond  (10^-3)  second
      . s  = second
    . Use "$" to specify infinite end time.
    . Use "0:$" to lift all restrictions and inject faults at any time.
    NOTE:
    . The time window "0" is the default for SA0 and SA1 faults.
    . The time window "0:$" is the default for SEU and SET faults.
```

```
  . Use "-condition" to specify fault injection condition. This injects
    the fault when the SVA condition specified evaluates to true.
      NOTE: Only sequence expressions are allowed in the fault condition,
      that is, no property or implication operators can be used in the
      fault condition expression.

  . Use "-bus_fault_width N" to automatically add a bus fault for multi-bit
    signals whose size is greater than "N". A bus fault is a single fault
    that represents an "or" grouping of faults that are injected on the bits
    of a multi-bit signal with the same fault type.
    NOTE:
    . "N" must be greater than or equal to 0.
    . The default value is 0.

  . Use "bus_fault_partition N" to split the automatically added bus faults
    into "N" chunks.
    NOTE:
    . "N" must be greater than or equal to 1.
    . The default value is 1.

  . Use "-memory_fault_width N" to automatically add a memory fault for
    multi-bit signals whose size is greater than "N". A memory fault is a
    single fault that represents an "or" grouping of faults that are
    injected on the bits of a multi-bit signal with the same fault type.
    NOTE:
    . "N" must be greater than or equal to 0.
    . The default value is 0.

  . Use "-silent" to avoid printing a summary of added faults.

. Use "-add" with "-from_file" to add fault nodes based on a specified
  file.

  The FSV App reads the three simulation fault file formats listed below:
  . IFSS elaboration faults file format (flt_target_dbg.list)
  . IFSS good simulation faults file format (fault.set)
  . IFR merge file format (ifr_merge.out)

  You can manually create a file (similar to IFSS elaboration format), for
  example:
            tb.module_inst.load_dato sa0+sa1+set
            tb.module_inst.dsr sa0
```

```
            tb.module_inst.reset sa0+sa1+set
            tb.module_inst.eoc sa0+sa1
            tb.module_inst.mux_en all
            tb.module_inst.error sa0+sa1+set

    . Use "-hier_path" to specify the path to the design top.

    . Use "-show_skipped" to view the faults that were not added.

  . Use "-add" with "-multiple" to add a multi fault with the specified
    list of fault IDs.
    NOTE:
    . Multi faults can be added only on IDs of non-multi faults.
    . If both the multi-fault and one of its fault elements have an
      injection specification ("-time_window" or "-condition"), then the
      fault is injected when both specifications are met.
    . You cannot add a multi fault with two or more fault IDs on the same
      signal.

    . Use "-number" to specify the maximum fault number to be injected out of
      the multiple faults specified.

    . Use "-include_sub" to include sub-fault groups injection (injected
      faults less than or equal number).

    . Use "-return" to specify the return type.
      . Use "-return id" to return the IDs of the inserted faults.
      . Use "-return dict" to return the result as a list of
        fault dictionaries.

    . Use "-silent" to avoid printing a summary of added faults.

    . Examples:
      . N = 1
        check_fsv -fault -add -multi {1 2 3 4 5} -number = 1
        Adds a multi fault that is injected separately on all five faults.
        Activation/Propagation/Detection of any of the five faults leads to
        the same result for the multi fault.

      . N = 3
        check_fsv -fault -add -multi {1 2 3 4 5} -number = 3
        Adds a multi fault that is injected separately on all fault subsets
        that contain exactly three faults.
```

```
      The injection of each fault subset is applied on all faults in the
      subset simultaneously.
      Activation/Propagation/Detection of any of the subsets leads to the
      same result for the multi fault
      . With "-include_sub" provided:
        All fault subsets have up to three faults (including).
        Activation/Propagation/Detection of any of the five faults leads
        to the same result for the multi fault.

. Use "-remove" to remove fault nodes with the specified IDs.
. Use "-remove" with "-node" to remove all the faults on the specified
  nodes.
  . Use "-remove" with "-type" to remove faults with the specified
    types from all nodes.
  . Use "-silent" to avoid printing a summary of removed faults.
  NOTE: The switches "-node" and "-type" can be combined. For example,
  the following command removes only the SA0 and SA1 faults from "sig":
  "check_fsv -fault -remove -node {sig} -type {sa0 sa1}"

. Use "-show" to show fault list data for fault nodes with the specified
  IDs. This command prints the following information to the console:
  [ID] [Name] [Type] [Class]

. Use "-list" to list fault IDs. And use "-node", "-type", "-class",
  "-coi_fo", "-coi_co", "-constant", "-time_window", "-primary",
  "-dominant", "-dominated", "-equivalent", "-propagatability",
  "-detectability", "-always_propagated", "-always_detected",
  "-propagated_always_detected", "-activatability", "-abstract_result",
  "-fo_result", "-co_result", and "-task" to filter IDs based on one or
  more specified attributes. These switches are not mutually exclusive,
  and you can specify each multiple times with multiple values.
  NOTE:
  . "-exact_match" is applicable with "-time_window" only.
  . All the previous switches accept a list.
  . Use "-regexp" with "-node" to evaluate the signal_tcl_list as
    regular expression(s).
  . "-fo_result" and "-co_result" options are available only when you
    use "-abstract_result" and specify the required mode, "constrained" or
    "default", as an argument. "default" mode considers only sound results
    while "constrained" mode considers some unsound results.
  . The abstract result mode can also be set with the following
    configuration variable:
    fsv_abstract_result_mode (default | constrained | none)
```

```
        The default is "none".
        NOTE: The "-abstract_result" switch value overrides the value of the
        "fsv_abstract_result_mode" variable.
      . This command returns the fault IDs for the specified filter's
        matching faults. If you do not specify a filter, the tool returns all
        fault IDs.
      . This command also prints the returned fault information to the console
        unless you use "-silent".
      . Use "-formal" or "-structural" with "-propagatability",
        "-detectability", and "-activatability" to list results from formal or
        structural analysis only.
        NOTE: "-structural" is not applicable with "-propagatability"
        propagated or unknown, "-detectability" detected or unknown , or
        " -activatability" unknown.

    . Use "-get" to get information about the faults argument for the specified
      ID Tcl list.
      Possible arguments include "node", "type", "class", "injection",
      "settime", "coi_fo", "coi_co", "constant", "property", "task", "result",
      "observed_strobes", "primary", "equivalent", "dominated", "dominant",
      "weight", "fanin_bbox", "fanout_bbox", "co_result", "fo_result",
      "distance_co", and "distance_fo" with the specified ID(s).
      . Use "-dominance_type" to specify the desired dominance with "weight",
        "dominated", or "dominant" only.
      . Use "-check_type" with "property", "task", "result", and
        "observed_strobes" to specify the desired properties.
        NOTE: The option "observed_strobes" requires a valid trace and returns
        the strobes whose value differs from their bad_machine counterpart
        due to the fault.
      . "fanin_bbox" or "fanout_bbox" gets a list of all the black-boxed
        instances that are in the fanin or fanout of the fault.
      . "-distance_fo" and "distance_fo" options are available only when you
        set "fsv_structural_distance" to "on", "fo", or "co" or add
        "-distance" to the structural analysis command.

-strobe
    -add (signal_tcl_list | property_list)
        [-functional]
        [-checker [-checker_mode (diff | assert [-assert_value (0 | 1)])]]
        [-condition sva_condition]
   -remove ( id_tcl_list
          |(-node (signal_tcl_list | property_tcl_list) [-regexp]
            [-functional |-checker]+)
```

```
        |(-functional |-checker)+]
  |-show id_tcl_list
  |-list
       [-node (signal_tcl_list | property_tcl_list) [-regexp]]
       [-type (fo | co)]*
       [-silent]
  |-get (node | type | checker_mode | condition | fanin_bbox | fanout_bbox |
fanin_fault) id_tcl_list
       [-silent]

   FSV Strobes:

   . Use "-add" to add strobe points to the specified signals or SVA
     properties. SVA properties should be added as CO strobes with checker mode
     "assert" as follows:

     check_fsv -strobe -add P1 -checker_mode assert

     These will be "detected" when the assertion is "violated".

     . Use "-functional" or "-checker" to add functional or checker type strobe
       points. The default strobe type is functional.
     . Use "-condition" to specify a condition that must to be met when
       a fault is observed. For example, use the following command to add a
       strobe on "data" and consider a fault observed only when "valid" is
       true:
       check_fsv -strobe -add {data} -condition {valid}
     . Use "checker_mode" to specify when the fault is considered "detected" at
       the checker strobe.
       . With "diff", the fault is detected when the strobe signals of the good
         machine and bad machine differ.
       . With "assert", the fault is detected when the bad machine strobe
         signal is asserted. Use "-assert_value" to control whether the strobe
         is asserted with the value 0 or 1.
       The default checker mode is "diff".

   . Use "-remove" to remove strobe points with the specified IDs.
     . Use "-remove" with "-node" to remove all the strobes on the specified
       nodes. And use "-remove" with "-node" and "-checker" or "-functional" to
       remove strobes with the specified type from all specified nodes.
     . Use "-remove" with "-functional" or "-checker" to remove all strobes
       with the specified type.
```

```
    . Use "-list" to list strobe IDs.
      . Use "-node" to list the IDs for the specified signal_tcl_list or
        property_tcl_list only. And use "-regexp" with "-node" to consider the
        signal_tcl_list or property_tcl_list as regular expressions.
      . Use "-type" to list the IDs for the specified strobe type only.

      NOTE: This command returns the strobe IDs for the specified filter's
      matching strobes. If you do not specify a filter, the tool returns
      all strobe IDs.

    . Use "-get" to get information about the strobe's argument for the
      specified ID Tcl list. "fanin_bbox" or "fanout_bbox" gets a list of all
      the blackboxed instances that are in the fan-in or fanout of the strobe
      respectively. "fanin_fault" gets a list of all faults present in the
      fan-in of the strobe.


-structural
    [-coi (on | fo | co | off)]
    [-constant (on | off)]
    [-fault_relations (all | on | fo | co | off)]
    [-propagation_analysis (on | fo | co | off)]
    [-constant_propagation (on | fo | co | off)]
    [-distance (on | fo | co | off)]
    [-silent]
    [-list_constants (seq | all)]
    [-clear_seq_constants]
    Run structural analysis with all the faults.

    . Use "-coi" to run COI analysis.
      . Use "on" to run COI analysis on all functional and checker output
        strobes, skipping the faults out of the strobes' COI.
      . Use "fo" to run COI analysis on functional strobes only.
      . Use "co" to run COI analysis on checker strobes only.
      . Use "off" to disable COI analysis.

    . Use "-constant" to run activatability analysis, skipping unactivatable
      faults.

    . Use "-fault_relations" to run dominance and equivalence analysis to
      generate a fault relations graph.
      NOTE:
      . The dominance direction is from gates' output towards inputs,
```

```
        that is, observed dominance.
      . Use "fo" or "co" to run fault relations analysis from the FO or CO
        strobes only. Use "on" to run fault relations analysis from both
        the FO and CO strobes.
      . Use "all" to analyze fault relations for all faults, regardless of the
        strobes' COI.

    . Use "-propagation_analysis" to run structural propagation analysis.
      The tool checks the local observability of the faults based on the
      structure of the design.

    . Use "-constant_propagation" to run propagation analysis based on
      constants. The tool checks the observability of the faults based on the
      structure of the design and the design constants.
      Note: You can optimize constant propagation analysis with
      "fsv_constant_propagation_analysis_optimization".
    . Use "-distance" to compute the distance of fault nodes to the strobes.
      NOTE:
      . Distance is the number of cells with two or more inputs on the shortest
        path from the node to any of the strobes.
      . Use "fo" or "co" to compute the distances of nodes to the "FO" or "CO"
        strobes respectively.
      . Use "on" to compute the distances of nodes to both "FO" and "CO"
        strobes.
      . Use "off" do disable computation of distances from nodes to strobes.
        This is the default.

    . Use "-silent" to avoid printing messages to the console.

    . Use "-list_constants" with "all" to list all propagated constants.
      Use "-list_constants" with "seq" to list sequential constants that were
      calculated by structural analysis.

    . Use "-clear_seq_constants" to clear cached sequential constants.

-generate
    [-max_faults_per_task N]
    [-clock_cycle_time time]
    [-fo_max_delay time]
    [-co_max_delay time]
    [-fo_co_max_delay time]
    [-propagatability (on | off)]
    [-detectability (on | off)]
```

```
[-always_propagated (on | off)]
[-always_detected (on | off)]
[-propagated_always_detected (on | off)]
[-activatability (on | off)]
[-strobe_optimization (on | off)]
[-task task_name]
[-force]
[-copy_env]
[-use_bus_faults]
[-use_memory_faults]
[-id id_tcl_list]
[-sanity]
[-silent]
Generate one or more FSV tasks with the injected faults.

. Use "-max_faults_per_task" to control the max number of faults per task.
  NOTE:
  . FSV generates tasks and distributes the fault nodes based on the
    specified threshold.
  . If the threshold equals (0), then all faults are generated in one task.
  . If you do not specify "-max_faults_per_task", the tool uses the value
    of "get_fsv_max_faults_per_task".

. Use "-clock_cycle_time" to specify the time of a single clock cycle.
  NOTE:
  . The tool uses this information to calculate the SET faults clock cycles
    and/or the fault injection time window when time is specified
    with a non-clock-cycle unit.
  . The format of time is [scale][unit], and unit can be any of the
    following: fs, ps, ns (default), us, ms, s.
  . If you do not specify "-clock_cycle_time", the tool uses the value of
    "get_fsv_clock_cycle_time".

. Use "-fo_max_delay" to provide the maximum number of clock cycles
  to check for propagation after a fault was injected during the proof
  of either the "propagatability" or "always_propagated" checks.
. Use "-co_max_delay" to provide the maximum number of clock cycles
  to check for detection after a fault was injected during the proof
  of either the "detectability" or "always_detected" checks.
. Use "-fo_co_max_delay" to provide the maximum number of clock cycles
  to check for detection after a fault propagation was found during the
  proof of the "propagated_always_detected" check.
```

. Use "-propagatability" to generate cover property checks for FO strobes.

. Use "-detectability" to generate cover property checks for CO strobes.

. Use "-always_propagated" to generate assertion checks for FO strobes.

. Use "-always_detected" to generate assertion checks for CO strobes.

. Use "-propagated_always_detected" to generate correlated FO/CO assertions.

. Use "-activatability" to generate the activatability cover property.

. Use "-strobe_optimization" (on | off) to create strobes observer nets
based on the strobes in the fanout of the fault only.

. Use "-task task_name" to name the generated task.

. Use "-force" to ignore structural analysis results and create properties
based on the specified checks only.

. Use "-copy_env" to copy all the task assumptions and task abstractions
(except stopats) from the current task to the new task(s).
NOTE: By default, newly generated FSV tasks contain only the environment
assumptions and abstractions.

. Use "-use_bus_faults" to generate bus faults instead of the faults that
are injected on their corresponding single-bit signals.
NOTE: By default, this option is disabled.

. Use "-use_memory_faults" to generate memory faults instead of the faults
that are injected on their corresponding single-bit signals.
NOTE:
. By default, this option is disabled.
. "-use_memory_faults" is an initial release to gather feedback from
early adopters and finalize implementation for an upcoming release.
Contact support@cadence.com to provide feedback or comments.

. Use "-id" to generate the specified fault IDs only.

. Use "-sanity" to generate sanity assertions based on existing functional
output strobes (FO) and checker output strobes (CO).

. Use "-silent" to avoid printing the info messages.

```
    NOTE:
    . Use "set_fsv_bbox_abstraction_level" (0 | 1 | 2) to set the level of
      abstraction of all black boxes when executing "-generate".

      Level 0: Disable black box abstraction.
      Level 1: Abstract black boxes so that faults propagate through them.
      Level 2: Treat black-boxed inputs in the COI of strobes as strobe points.

      NOTE: Multipliers, dividers, and modulo black boxes are abstracted with
      a combinatorial abstraction and the rest are abstracted with a mixed
      abstraction.

    . Use "set_fsv_max_checks_per_task" to specify the maximum number of
      checks that the tool generates in a single task. The default is 150,000

-report
       ( [-text text_file]
        |[-html html_file [-launch_html_browser]]
        |[-xml xml_file])
       ( [-type (sa0 | sa1 | seu | set | multi | open | all)+]
         [-class (unprocessed | safe | dangerous | unknown)+]
         [-coi_fo (unprocessed | in | out | unknown)+]
         [-coi_co (unprocessed | in | out | unknown)+]
         [-constant (unprocessed | activated | unactivatable | unknown)+]
         [-propagatability (  unprocessed | propagated | unpropagatable
                            | unknown)+]
         [-detectability (unprocessed | detected | undetectable | unknown)+]
         [-always_propagated (  unprocessed | always_propagated
                              | not_always_propagated | unknown)+]
         [-always_detected (  unprocessed | always_detected
                            | not_always_detected | unknown)+]
         [-propagated_always_detected (  unprocessed
                                       | propagated_always_detected
                                       | propagated_not_always_detected
                                       | unknown)+]
         [-activatability (unprocessed | activated | unactivatable | unknown)+] )
        |[-id id_tcl_list])
       ([-fault_relations [-show_dominance
                            [-dominance_type (observed | unobservable)]]
                          [-primary_only]
                          [-hier_path path_to_design_top]])
         [-force]
```

```
      [-summary]
      [-aggregate]
      [-show_source_location]
      [-show_fault_ids]
      [-abstract_result (constrained | default)
        [ -fo_result (  unprocessed | unobserved | unobservable | dangerous
                      | alwaysobserved | unknown)+
         |-co_result (  unprocessed | undetected | undetectable | detected
                      | alwaysdetected | unknown)+] ]
```

Generate FSV reports.

. Use "-text" to generate an FSV text report.

. Use "-html" to generate an FSV HTML report and use
  "-launch_html_browser" to automatically open the report in the browser.

. Use "-xml" to generate an FSV XML report.

. Use "-type", "-class", "-coi_fo", "-coi_co", "-constant",
  "-propagatability", "-detectability", "-always_propagated",
  "-always_detected", "-propagated_always_detected", "-activatability",
  "-fo_result", and "-co_result" to filter reported faults based on one
  or more specified attributes.
  NOTE:
  . "-type", "-class", "-coi_fo", "-coi_co", "-constant",
    "-propagatability", "-detectability", "-always_propagated",
    "-always_detected", "-propagated_always_detected", and "-activatability"
    are not mutually exclusive, and you can specify each one multiple times
    for multiple values.
  . "-fo_result" and "-co_result" options are available only when
    "-abstract_result" is used.

. Use "id" to filter the reported faults based on the specified IDs.

. Use "-fault_relations" to add relations relevant columns to the report.
  . Use "-show_dominance" to generate an extended report, which includes
    dominance information.
  . Use "-dominance_type" to specify dominance type relevant to "weight",
    "dominated", and "dominant" columns.
  . Use "-primary_only" to filter the reported faults to primary faults
    only.

```
      . Use "-hier_path" to prefix each fault name with the specified
        hierarchical path. Generally, this is the inst top or fault top path.
    . Use "-force" to overwrite an existing report file.
    . Use "-summary" to produce a report with the faults summary data only.
    . Use "-aggregate" to group faults on multi-bit signals based on their
      results.
      NOTE: The report format will be changed.
    . Use "-show_source_location" to add the surce location to the report.
    . Use "-show_fault_ids" to add the fault IDs to the report.
    . Use "-abstract_result" to generate abstract fault results. Specify the
      required mode, "constrained" or "default", as an argument. "default"
      mode considers only sound results while "constrained" mode considers
      some unsound results.
      NOTE: If you do not use the "-abstract_result" switch, the
      tool reads the abstract result mode from the "fsv_abstract_result_mode"
      property.

    NOTE:
    . If you do not specify "-text", "-html", or "-xml" the report prints
      to the console.
    . If you do not specify "-coi_fo", "-coi_co", "-propagatability",
      "-detectability", "-always_propagated", "-always_detected",
      "-propagated_always_detected", or "-activatability" the report will not
      include information regarding those checks.
    . If not specified, "-dominance_type" will be "observed" dominance by
      default.


-abstract
       (-instance instance_tcl_list)
           [-mode (comb | mixed | seq)]
           [-connections connection_spec_tcl_list]
      |(-operator (adder | subtract | multiplier | shift | comparator))
           [-include_instance {instance_list} [-regexp]]
           [-exclude_instance {instance_list} [-regexp]]
           [-include_module {module_list} [-regexp] [-recursive]]
           [-exclude_module {module_list} [-regexp] [-recursive]]
           [-minimum_width N]
           [-skip_const_operand]
      |(-bbox_mul | -bbox_div | -bbox_mod)
      |(-counter signal_list)
      (-task task_tcl_list | -env)
      [-silent]
```

```
|-remove
    (-id id_tcl_list)
   |(-instance instance_tcl_list)
       [-task task_tcl_list]
    [-silent]
|-get id_tcl_list
  [-silent]
|-list
    [-instance instance_tcl_list]
    [-task task_tcl_list]
    [-env]
    [-type (  instance | adder | subtract | multiplier | shift
            | comparator)]
    [-silent]

|-clear
```

Create FSV abstractions.

. Create an abstraction that does not eliminate functional paths for the specified instances, for multiplier, divider, or modulo instances or for adder, subtract, multiplier, shift or comparator gates.

  . Use "-instance" to abstract instances in the design (including black boxes).

    . Use "-connections" to specify a connection_spec_list. The specification must have the following format:
      { { { (input_port)+ } { (output_port)+ } (comb | seq | mixed) }+ }
      See the "Example" section.

      NOTE: The "-connections" switch is allowed only with one instance.

    . Use "-mode" to specify the default mode for the abstraction. The default mode is used for abstracting the outputs that are not specified in the connections specification Tcl list.
     . Use "-mode comb" to mark the abstraction as combinational, that is, the fault propagates immediately.
     . Use "-mode mixed" to mark the abstraction as mixed, that is, the fault is sticky and propagates now and in all future cycles.
     . Use "-mode seq" to mark the abstraction as sequential, that is, the fault is sticky and propagates in all future cycles.

```
    . Use "-operator" with "adder", "subtract", "multiplier", "shift", or
      "comparator" argument to abstract the specified operators.
     . Use "-exclude_module" to exclude matching modules from abstraction.
     . Use "-include_module" to include matching modules in abstraction.
       NOTE: Use "-recursive" with "-include_module" or "-exclude_module" to
       also include or exclude the instance hierarchies inside the module.
     . Use "-exclude_instance" to exclude matching instances from abstraction.
     . Use "-include_instance" to include matching instances in abstraction.
     NOTE: The "-exclude_module", "-include_module", "-exclude_instance", and
     "-include_instance" switches accept wildcards (* and ?) and filter
     using case-sensitive wildcard strings by default.

    . Use "-bbox_mul", "-bbox_div" or "-bbox_mod" to abstract all the
      multipliers, dividers or modulo black-box instances in the design
      respectively.
      NOTE: Use "set_fsv_bbox_abstraction_level (0 | 1 |2)" to control
      abstraction of all the black boxes in the design automatically when
      calling generate. Afterwards, manual abstraction of instances is
      possible using the "-abstract" command.
      Example:
      check_fsv -abstract -instance m -mode mixed -connections { {{in1 in2} {out1}
comb} {{in3} {out2} seq} }

    . Use "-counter" to add a fault-aware counter abstraction on the
      specified signals.

    . Use "-task" to add the abstractions to specified tasks. By default,
      abstractions are added to the global environment

    . Use "-remove" to remove abstractions. Specify abstraction IDs, instance
      names and/or task names.
      Note:
      . You can specify instance names and task names. In this case, the
        abstraction from any specified instance is removed from any specfied
        task.
      . If no task name is specified, abstractions are removed from all tasks,
        including the global environment.
      . If only a task name is specified, all the abstractions from that task
        are removed.
      . Instance names accept regular expressions.

    . Use "-get" to return a list of strings representing the abstraction IDs
      in the specified ID list.
```

```
      . Use "-list" to list fault IDs. And use "-instance", "-task", "-env" and
        "-type" to filter the listed strobes.
        NOTE:
        . Use "-type" with the value "instance" to see information about the
          abstraction of the specified instances.
        . Use the switch "-type" with the switch "-instance" to see
          information about the abstractions with the specified type inside
          the specified instances.
        . This command returns information about the specified filter's
          matching abstraction. If you do not specify a filter, the tool returns
          information about all the abstractions.

-barrier
    -add (signal_tcl_list)
      [-condition sva_condition]
      [-value (0 | 1)]
   |-remove (signal_tcl_list)
   |-list
   |-clear

    Create FSV barriers.
    . Use "-add" to create a barrier that blocks the propagation of a fault.
      NOTE: Barriers are added to the global environment and invalidate all
      current results.
      . Use "-condition" to specify a condition for the barrier. The fault
        will not propagate after this barrier if the condition is true.
      . Use "-value" to specify a special condition in which the fault is
        blocked only if it corrupts the signal to be the specified value.
      NOTE: The switches "-condition" and "-value" are mutually exclusive.

-prove
    [-id id_tcl_list]
    [-check_type (propagatability | detectability | always_propagated
        | always_detected | propagated_always_detected | activatability)]*
    [-property property_name_tcl_list]
    [-task task_name_tcl_list]
    [-strategy (basic | clustering))]
    [-group_by (instance | signal | bit)]
    [-time_limit N]
    [-engine_mode engine_tcl_list]
    [-verbosity N]
    [-bg]
```

```
[-keep_traces]
[-sanity_check]
[-silent]
[-force]

Prove the FSV properties.
. Use "check_fsv -prove" without additional switches to prove all the
  faults' properties.
. Use "-id" to specify the fault IDs for the properties you want to
  prove.
. Use "-check_type" to specify the property type you want to prove. This
  command supports multiple check types.
. Use "-task" to prove properties in the specified task(s).
. Use "-property" to specify the names of the FSV properties you want to
  prove.
. Use "-strategy" to specify an FSV proof strategy. The default strategy is
  "basic".
  . Use "basic" to run the proof on specified faults in one group.
  . Use "clustering" to leverage clustering of faults for better
    convergence.
  The "clustering" strategy runs in the following phases using stagnation
  limits:
  . All Activatability properties are run first together in a single
    group.
  . FSV proves all the properties together in a single group (focusing
    on finding counterexamples).
  . FSV groups the remaining properties into chunks and proves them
    serially (focusing on finding proofs).
  . FSV iterates over non-empty groups and proves them serially up to
    the total proof time limit.

. Use "-group_by" to specity how faults are grouped in the "clustering"
  strategy.
  . Use "instance" to specify that each hierarchical instance becomes a
    group.
  . Use "signal" to specify that all faults on one signal become one group.
  . Use "bit" to specify that all faults on one bit become one group.
. Use "-time_limit" to configure the proof time limit.
. Use "-engine_mode" to configure which engines to use during the proof.
. Use "-verbosity N" to override the current "prove_verbosity" for this
  "-prove" command. See "set_prove_verbosity" for a summary of the
  verbosity levels.
. Use "-bg" to run the command in the background and continue to accept and
```

```
      run new commands.
    . Use "-keep_traces" to disable trace suppression.
      . By default, FSV runs the proof internally with "-no_traces" to suppress
        traces.
      . FSV reruns the proof for a dedicated fault when you run "check_fsv
        -visualize".
      NOTE: Keeping traces for all faults might produce large amounts of data.
    . Use "-sanity_check" to prove only the FO and CO sanity assertions
      generated in the task.
    . NOTE:
      . When checker strobes with checker mode "assert" exist, FSV
        automatically generates and proves the required sanity properties.
      . Use "set_fsv_proof_sanity_check off" to disable the proof of the
        sanity properties.
      . Use "set_fsv_proof_sanity_check auto" enable automatic handling of
        sanity properties. This is the default option.
      . Use "set_fsv_proof_sanity_check on" to include the sanity properties
        when calling "check_fsv -prove" without any other filter.
    . Use "-silent" for "-verbosity 0" (see "-verbosity" description above).
    . Use "-force" to clear existing proof results for the properties before
      starting the prove.
      NOTE: You can use this switch in combination with other "-prove" switches.

-visualize
    (-id id
     -check_type (activatability | propagatability | detectability
                 | always_propagated | always_detected
                 | propagated_always_detected))
    |(-sanity -task task_name)

    Open a Visualize trace for the specified property.

    . Specify a fault ID and check type to open a Visualize trace of the
      property for the specified fault.
      NOTE: FSV reruns the proof for the dedicated fault unless "-keep_traces"
      was used during the proof.
    . Specify "-sanity" and provide a task name to open a Visualize trace for
      the CO sanity property in the specified task.


-fault_work
        -import directory_name
            [-use_test_const [-show_skipped]]
```

```
            [-fault_top fault_top]
            [-hier_path hier_path
                |-hier_map (-from from -to to)]
            [-ucd_file ucd_file]
                [-fo_result (  unprocessed | unobserved | unobservable
                             | dangerous | alwaysobserved | unknown)+]
                [-co_result (  unprocessed | undetected | undetectable
                             | detected| alwaysdetected | unknown)+]
                [-type (sa0 | sa1 | seu | set)+]
            [-fault_list fault_list]
            [-fault_node fault_node [-regexp]]
            [-include_ut]
            [-include_equivalent]
            [-show_skipped]
        |-export [-id id_tcl_list]
            [-abstract_result (constrained | default)]
            [-fault_test test_name]

   Integrate FSV with "Incisive Functional Safety Simulator" (IFSS).

   . Use "-import" to import the specified IFSS fault work directory.
     . Use "-use_test_const" to import all FO Test-Based Unpropagatable and
Test-Based CO Undetectable faults with test-based annotations.
And use "-show_skipped" to view signals that were not added as constants
       during import. With this option, the tool prints up to 100 signals from
       each of the following three categories: 1) test-based constants out of
       inst_top, 2) test-based constants in black boxes, and 3) invalid
       signals.
       NOTE:
       . The expected input file follows: ./fault_db/test_const.csv
         Starting with the 2020.06 release, FSV also accepts a gzipped file
         format, "test_const.csv.gz", which is unzipped while loading. To
         maintain backward compatibility, this file is processed only if
         "test_const.csv" does not exist.
       . You can also optionally include the SHM input file:
         ./fault_db/init.shm
       . The output UCD will contain TB-FU+ and/or TB-CU+ annotations only.
         No normal annotations will be exported if you use this option on
         import.
     . Use "-fault_top" to specify the top module of the faults required for
       import.
     . Fault databases are usually generated during simulation where the
       testbench  module is the top module for the verification run. When
```

```
      elaborating a  different sub hierarchy with a standalone Jasper
      invocation in the FSV  App, use "-hier_path" to specify the hierarchical
      path difference from the testbench module to the design top module.
    . Use "-hier_map" for fault databases generated during simulation if you
      have a wrapper module.
      . Use "-from" to specify the source (simulation environment) hierarchy.
      . Use "-to" to specify the destination (formal environment) hierarchy.
      NOTE: The "-hier_path" and "-hier_map" switches are mutually exclusive.
    . Use "-ucd_file" to import faults from the specified UCD file only,
      including untestable faults.
    . Use "-fo_result" to import faults with the specified results for
      FO strobes only from the specified UCD file.
    . Use "-co_result" to import faults with the specified results for
      CO strobes only from the specified UCD file.
    . Use "-type" to import faults with the specified type from the specified
      UCD file.
    . Use "-fault_list" to import faults from the specified fault list
      file only, including untestable faults. This file is generated in CSV
      format and represents a subset of the faults in the UCM file. You
      cannot use "-fault_list" with "-ucd_file".
      NOTE: This file must include a header line <ftime>,<ftype>,<fnode>.
    . Use "-fault_node" to filter imported faults by node name. And use
      "-regexp" to filter by regular expression.
    . Use "-include_ut" to include untestable faults. You cannot use
      "include_ut" with "-fault_list" or "-ucd_file".
    . Use "-include_equivalent" to import equivalent faults.
    . Use "-show_skipped" to view the faults that were not added during
import of the fault work.

  . Use "-export" to export FSV results to the imported fault works.
    . Use "-abstract_result" to export abstract fault results. Specify the
      required mode, constrained or default, as an argument. "default"
      mode considers only sound results while "constrained" mode considers
      some unsound results.
      NOTE: If you do not use the "-abstract_result" switch, the
      tool reads the abstract result mode from the "fsv_abstract_result_mode"
      property.
    . Use "-id" to specify fault work IDs to be exported. If you do not
      specify IDs, the tool exports all fault works.
    . Use "set_fsv_export_fault_relations true" before the "check_fsv -fault_work
      -export" command to export all fault relations into a UCM file.
    . Use "-fault_test" to specify the name of the test that was run as part
      of the FSV TC flow.
```

```
-graph
    (-from signal_tcl_list
     -to signal_tcl_list)
    [-type (all | register | hierarchy | logic)]
    [-exclude_cell_contents]
    [-cells]

    Calculate the shortest path between two signal lists.

    . Use "-from" to specify the source signal list.
    . Use "-to" to specify the destination signal list.
    . Use "-type" to specify the length of the path between the "-from" signal
      list and the "-to" signal list.
      . Use "all" to consider all signals in the path.
      . Use "register" to consider the number of registers in the path.
      . Use "hierarchy" to consider the number of hierarchy changes in the path.
      . Use "logic" to consider the number of gates in the path.
    . Use "-cells" to get the list of cell defined instances that are on the
      shortest path between the "-from" signal list and the "-to" signal list.
    . Use "-exclude_cell_contents" to exclude signals that are inside a cell
      from the returned path.

-show_schematic ( -signal signal_list [-all |-driver | -port]
                  [-full_hierarchy])
                  |-instance instance_list

    Draw the schematic of the specified signal or instance list on the
    schematic viewer window.
    . Use "signal" to specify the signal list for the schematic.
      . Use "-all" to draw all gates connected to the signals specified by the
        signal list.
      . Use "-driver" to draw the gates that drive the signals specified by the
        signal list.
      . Use "-port" to draw the ports of the signals specified by the signal
        list.
      . Use "-full_hierarchy" to draw all instance levels related to the signals
        specified by the signal list.
    . Use "-instance" to specify an instance list for the schematic.

-clear [all | result [-task task_tcl_list]]

    Clear FSV data.
```

```
. Use "all" to clear all FSV data structures (default).
. Use "result" to clear only FSV results.
. Use "-task" with "result" to clear the FSV results from the
  specified tasks.
```

Example

```
check_fsv -init
check_fsv -fault -add {sig1 sig2 sig3} -type sa0+set -set_hold_time 5cc
check_fsv -fault -from_file faults.list
check_fsv -strobe -add {out1 out2 out3}
check_fsv -structural
set t [check_fsv -generate]
prove -task t
check_fsv -report -text fsv.rpt
check_fsv -clear
```

See Also

set_fsv_abstract_result_mode,
set_fsv_bbox_abstraction_level,
set_fsv_stopat_abstraction_level,
set_fsv_bus_fault_partition,
set_fsv_bus_fault_width,
set_fsv_classification_scheme,
set_fsv_clock_cycle_time,
set_fsv_constant_propagation_analysis_optimization,
set_fsv_engine_mode,
set_fsv_export_fault_relations,
set_fsv_export_propagated_constants,
set_fsv_fault_tw,
set_fsv_generate_activatability,
set_fsv_generate_always_detected,
set_fsv_generate_always_propagated,
set_fsv_generate_copy_env,
set_fsv_generate_detectability,
set_fsv_generate_propagatability,
set_fsv_generate_propagated_always_detected,
set_fsv_generate_use_bus_faults,
set_fsv_generate_use_memory_faults,

set_fsv_generate_fo_max_delay_time,
set_fsv_generate_co_max_delay_time,
set_fsv_generate_fo_co_max_delay_time,
set_fsv_import_include_ut,
set_fsv_import_prime_only,
set_fsv_max_checks_per_task,
set_fsv_max_faults_per_task,
set_fsv_memory_fault_width,
set_fsv_unsafe_inference,
set_fsv_proof_sanity_check,
set_fsv_prove_keep_traces,
set_fsv_set_hold_time,
set_fsv_source_browser_max_line_count,
set_fsv_source_browser_size_threshold,
set_fsv_source_browser_updating,
set_fsv_strobe_checker_mode,
set_fsv_strobe_checker_mode_assert_value,
set_fsv_strobe_optimization,
set_fsv_structural_coi,
set_fsv_structural_constant,
set_fsv_structural_constant_propagation_analysis,
set_fsv_structural_distance,
set_fsv_structural_fault_relations,
set_fsv_structural_propagation_analysis,
set_fsv_structural_seq_constants_engine_mode,
set_fsv_structural_seq_constants_propagation,
set_fsv_structural_seq_constants_prove_time_limit,
set_fsv_allow_faults_in_bbox

check_hls

Compares a SystemC model to RTL or a SystemC model to another SystemC model.

Associated App(s)

HLS

Syntax

```
Extract Stratus BDW
-------------------
check_hls -extract -stratus_bdw
          -path path_to_stratus_workdir
          -module module_name
          -config config_name
          (-auto_setup |-proof_hints)

Pipeline
--------
check_hls -pipeline (-spec |-imp)
    -latency N
    -loop_label {label}
    -expand_label {label}
    -initiation_interval N
    -remove_states_before_first_io

Pipeline List
-------------
check_hls -pipeline -list

Pipeline Remove
---------------
check_hls -pipeline -remove id

Pipeline Clear
--------------
check_hls -pipeline -clear
```

```
Setup
-----
check_hls -compile (-spec |-imp)
    [-no_top_wrapper]
    [-top model_top_name]
    [-flags string]
    [-top_template_params string]
    [-bbox_m string]
    [-use_bundled_stratus_headers]

check_hls -analyze -imp
    analyze_switches_and_arguments

check_hls -elaborate -imp
      elaborate_switches_and_arguments

check_hls -setup
    [-no_auto_mapping]
    [-auto_map_reset_x_values]

Clear
-----
check_hls -clear
         (all | mapping | (result [-task task_name]))

Utility Switches
----------------
check_hls (-get_spec_top_instance |-get_imp_top_instance)

Interface Check
---------------
check_hls -interface
        [-mapped |-unmapped]
        [-spec |-imp]
        [-signal_type (  primary_input | primary_output
                                | dut_input | dut_output
                                | bbox_input | bbox_output | user_bbox_input
                                | user_bbox_output | auto_bbox_input | auto_bbox_output
                                | undriven | internal_undriven
                                | setup_stopat | reset_x | x_assignment |
x_default_assignment
                                | x_divide_by_zero | x_index_out_of_range |
x_misc_undriven
```

```
                                  | x_bus_contention | x_bus_floating
                                  | user_env_stopat | user_task_stopat)]
        [-task task_name]
        [-ignore_user_stopats]

Signals List
------------
check_hls -list
       [ [-signal]
           [-signal_type (  input | output | primary_input | primary_output
                                  | dut_input | dut_output
                                  | bbox_input | bbox_output | user_bbox_input
                                  | user_bbox_output | auto_bbox_input |
auto_bbox_output
                                  | mpram_port | undriven | internal_undriven
                                  | flop | latch | internal | stopat | reset_x
                                  | x_assignment | x_default_assignment |
x_divide_by_zero
                                  | x_index_out_of_range | x_misc_undriven
                                  | x_bus_contention | x_bus_floating)]
           [-spec |-imp]
           [(-coi_of |-fanout_of) signal_list]
           [ -instance instance_full_path
            |-name {regexp} [-verbose]
       ]

Signal Mapping List
-------------------
check_hls -list -mapping
           [-mapping_type (  connection_all | connection_primary_input
                                  | connection_bbox_output | connection_undriven
                                  | connection_stopat | connection_misc
                                  | ver_target_all | ver_target_primary_output
                                  | ver_target_mpram_port | ver_target_bbox_input
                                  | ver_target_misc | cutpoint_all | cutpoint_both
                                  | cutpoint_spec | cutpoint_imp | cutpoint_both_init
                                  | stopat_all | abstract_reset_value |
abstract_counter | abstract_init_value
                                  | stopat_both | stopat_spec  | stopat_imp |
stopat_both_env
                                  | helper | init_user | init_auto | init_all) ]
           [-signal_type (  input | output | primary_input | primary_output
                                  | dut_input | dut_output
```

```
                                    | bbox_input | bbox_output | user_bbox_input
                                    | user_bbox_output | auto_bbox_input |
auto_bbox_output
                                    | mpram_port | undriven | internal_undriven
                                    | flop | latch | internal | stopat | reset_x
                                    | x_assignment | x_default_assignment |
x_divide_by_zero
                                    | x_index_out_of_range | x_misc_undriven
                                    | x_bus_contention | x_bus_floating)]
           [  (-instance instance_full_path)
             |(-exclude_signal signal_list)
             |(-of_signal {regexp_or_signal_list}
                [ -spec_regexp [-verbose]
                 |-imp_regexp [-verbose] ] )
           ]
           [-tag {tag_name}]
           [([-task task_name] [-include_global_mapping])
            |-global
           ]
           [-return (text | id | dict)]
           [-proof_status (proven | cex | unprocessed | undetermined)+]
           [-proof_task proof_task_name]
           [ (-coi_of signal_list [-get_first_level_coi])
            |(-fanout_of signal_list [-get_first_level_fanout])]
           [-status (enabled | disabled)]

Mapping Info
------------
check_hls -mapping_info
        (    -id mapping_id_list
          |  -dict mapping_dict_list [-force]
          | (  -spec {spec_regexp_or_signal_list}
              |-imp {imp_regexp_or_signal_list})
        )
        [-regexp [-verbose]]
        [-signal_type (  input | output | primary_input | primary_output
                               | dut_input | dut_output
                               | bbox_input | bbox_output | user_bbox_input
                               | user_bbox_output | auto_bbox_input | auto_bbox_output
                               | mpram_port | undriven | internal_undriven
                               | flop | latch | internal | stopat | reset_x
                               | x_assignment | x_default_assignment |
x_divide_by_zero
```

```
                                | x_index_out_of_range | x_misc_undriven
                                | x_bus_contention | x_bus_floating)])
        [-mapping_type (  connection_all | connection_primary_input
                                | connection_bbox_output | connection_undriven
                                | connection_stopat | connection_misc
                                | ver_target_all | ver_target_primary_output
                                | ver_target_mpram_port | ver_target_bbox_input
                                | ver_target_misc | cutpoint_all | cutpoint_both
                                | cutpoint_spec | cutpoint_imp | cutpoint_both_init
                                | stopat_all | abstract_reset_value | abstract_counter
| abstract_init_value
                                | stopat_both | stopat_spec  | stopat_imp
|stopat_both_env
                                | helper | init_user | init_auto | init_all) ]
        [  (-instance instance_full_path)
          |(-of_signal {signal_list}
              [ -spec_regexp [-verbose]
               |-imp_regexp [-verbose] ] )
        ]
        [-proof_status (  proven | cex | unprocessed | undetermined)+]
        [-tag {tag_name}]
        [([-task task_name] [-include_global_mapping])
         |-global
        ]
        [-list (  spec_signal | imp_signal | context | source | id
                        | sva_expression | property | is_enabled
                        | is_inverse | type | task_name | tag | status)]

Signal Mapping
--------------
check_hls -map
        ([-auto]
           [-spec {spec_instance_list}
           [-imp {imp_instance_list}]]
           [-signal_type (  input | output | primary_input | primary_output
                                | dut_input | dut_output
                                | bbox_input | bbox_output | user_bbox_input
                                | user_bbox_output | auto_bbox_input |
auto_bbox_output
                                | mpram_port | undriven | internal_undriven
                                | flop | latch | internal | stopat | reset_x
                                | x_assignment | x_default_assignment |
x_divide_by_zero
```

```
                              | x_index_out_of_range | x_misc_undriven
                              | x_bus_contention | x_bus_floating)]
      [-coi_of signal_list]
      [-set_value (  0 | 1)]
      [-max_word_size max_word_size_value]
      [-return (  text | id | dict | none)]
      [-spec_delay spec_delay_value]
      [-imp_delay imp_delay_value]
      [-map_condition {map_cond_expr}]
      [-clock {clocking_event}]
      [-disable {disable_cond_expr}]
      [-edge (  posedge | negedge | both_edge)]
      [ -cut (  both | both_init | both_no_init | spec | imp)
       |-stopat (  both | spec | imp | both_env)
       |-helper
       |-constraint]
      [-tag {tag_name}]
      [-task task_name |-global]
      [-respect_connections_during_reset]
      [-exclude_module {regexp_or_module_list}
                  [-module_regexp]]
      [-exclude_instance {regexp_or_instance_list}
                  [-instance_regexp]]
  )
| ( (-spec {spec_regexp_or_signal_list}
        [-imp {imp_regexp_or_signal_list}])
      [-regexp
        [-verbose]
      [-max_word_size max_word_size_value]
      [-inverse]
      [-return (  text | id | dict | none)]
      [-spec_delay spec_delay_value]
      [-imp_delay imp_delay_value]
      [-map_condition {map_cond_expr}]
      [-clock {clocking_event}]
      [-disable {disable_cond_expr}]
      [-edge (  posedge | negedge | both_edge)]
      [ -cut (  both | both_init | spec | imp)
       |-stopat (  both | spec | imp | both_env)
       |-helper
       |-constraint]
      [-tag {tag_name}]
      [-task task_name |-global]
```

```
            [-init]
            [-respect_connections_during_reset]
          )
        |-map_file map_file_name

Signal Unmapping
----------------
check_hls -unmap
         ( -id mapping_id_list
          |-dict mapping_dict_list
          |-spec {spec_regexp_or_signal_list}
          |-imp {imp_regexp_or_signal_list}
         )
           [-regexp [-verbose]]
           [-mapping_type (  connection_all | connection_primary_input
                                  | connection_bbox_output | connection_undriven
                                  | connection_stopat | connection_misc
                                  | ver_target_all | ver_target_primary_output
                                  | ver_target_mpram_port | ver_target_bbox_input
                                  | ver_target_misc | cutpoint_all | cutpoint_both
                                  | cutpoint_spec | cutpoint_imp | cutpoint_both_init
                                  | stopat_all | abstract_reset_value |
abstract_counter | abstract_init_value
                                  | stopat_both | stopat_spec  | stopat_imp |
stopat_both_env
                                  | helper | init_user | init_auto | init_all) ]
           [  (-instance instance_full_path)
             |(-of_signal {signal_list}
                 [ -spec_regexp [-verbose]
                  |-imp_regexp [-verbose] ] )
           ]
           [-proof_status (  proven | cex | unprocessed | undetermined)+]
           [-signal_type (  input | output | primary_input | primary_output
                                 | dut_input | dut_output
                                 | bbox_input | bbox_output | user_bbox_input
                                 | user_bbox_output | auto_bbox_input |
auto_bbox_output
                                 | mpram_port | undriven | internal_undriven
                                 | flop | latch | internal | stopat | reset_x
                                 | x_assignment | x_default_assignment |
x_divide_by_zero
                                 | x_index_out_of_range | x_misc_undriven
                                 | x_bus_contention | x_bus_floating)]
```

```
          [-tag {tag_name}]
          [(-task task_name [-include_global_mapping])
           |-include_global_mapping
          ]

AutoProve Unmapping
-------------------
check_hls -unmap -autoprove_mappings [-task task_name]

Setting Mapping Context
-----------------------
check_hls -set_context_of_mapping
        (   -id mapping_id_list
         |  -dict mapping_dict_list [-force]
         | ( -spec {spec_regexp_or_signal_list}
            |-imp {imp_regexp_or_signal_list})
               [-regexp [-verbose]]
               [-mapping_type (  connection_all | connection_primary_input
                                 | connection_bbox_output | connection_undriven
                                 | connection_stopat | connection_misc
                                 | ver_target_all | ver_target_primary_output
                                 | ver_target_mpram_port | ver_target_bbox_input
                                 | ver_target_misc | cutpoint_all |
cutpoint_both
                                 | cutpoint_spec | cutpoint_imp |
cutpoint_both_init
                                 | stopat_all | abstract_reset_value |
abstract_counter | abstract_init_value
                                 | stopat_both | stopat_spec  | stopat_imp |
stopat_both_env
                                 | helper | init_user | init_auto | init_all) ]
              [  (-instance instance_full_path)
                |(-of_signal {signal_list}
                   [ -spec_regexp [-verbose]
                    |-imp_regexp [-verbose] ] )
              ]
              [-proof_status (  proven | cex
                                | unprocessed | undetermined)+]
              [-signal_type (  input | output | primary_input | primary_output
                              | dut_input | dut_output
                              | bbox_input | bbox_output | user_bbox_input
                              | user_bbox_output | auto_bbox_input
                              | auto_bbox_output | mpram_port | undriven |
```

```
internal_undriven
                                        | flop | latch | internal | stopat | reset_x
                                        | x_assignment | x_default_assignment |
x_divide_by_zero
                                        | x_index_out_of_range | x_misc_undriven
                                        | x_bus_contention | x_bus_floating)]
                [-tag {tag_name}]
                [([-task task_name] [-include_global_mapping])
                 |-global
                ]
        |-set_default
        |-clear_default
        )
        [-spec_delay spec_delay_value]
        [-imp_delay imp_delay_value]
        [-map_condition {map_cond_expr}]
        [-clock {clocking_event}]
        [-disable {disable_cond_expr}]
        [-edge (  posedge | negedge | both_edge)]

Clearing Mapping Context
------------------------
check_hls -clear_context_of_mapping
        (   -id mapping_id_list
         |  -dict mapping_dict_list [-force]
         | ( -spec {spec_regexp_or_signal_list}
            |-imp {imp_regexp_or_signal_list})
               [-regexp [-verbose]]
               [-mapping_type (  connection_all | connection_primary_input
                                        | connection_bbox_output | connection_undriven
                                        | connection_stopat | connection_misc
                                        | ver_target_all | ver_target_primary_output
                                        | ver_target_mpram_port | ver_target_bbox_input
                                        | ver_target_misc | cutpoint_all |
cutpoint_both
                                        | cutpoint_spec | cutpoint_imp |
cutpoint_both_init
                                        | stopat_all | abstract_reset_value |
abstract_counter | abstract_init_value
                                        | stopat_both | stopat_spec  | stopat_imp |
stopat_both_env
                                        | helper | init_user | init_auto | init_all) ]
                [  (-instance instance_full_path)
```

```
                |(-of_signal {signal_list}
                   [ -spec_regexp [-verbose]
                    |-imp_regexp [-verbose] ] )
              ]
              [-proof_status (  proven | cex
                                | unprocessed | undetermined)+]
              [-signal_type (  input | output | primary_input | primary_output
                               | dut_input | dut_output
                               | bbox_input | bbox_output | user_bbox_input
                               | user_bbox_output | auto_bbox_input
                               | auto_bbox_output | mpram_port | undriven |
internal_undriven
                               | flop | latch | internal | stopat | reset_x
                               | x_assignment | x_default_assignment |
x_divide_by_zero
                               | x_index_out_of_range | x_misc_undriven
                               | x_bus_contention | x_bus_floating)]
              [-tag {tag_name}]
              [[-task {task_name} [-include_global_mapping]]
               |-global
              ]
       )

Enabling and Disabling Mapping
---------------------------------------
check_hls (-enable_mapping |-disable_mapping)
       (   -id mapping_id_list
        |  -dict mapping_dict_list [-force]
        |(-spec {spec_regexp_or_signal_list} |-imp {imp_regexp_or_signal_list})
             [-regexp [-verbose]]
             [-mapping_type (  connection_all | connection_primary_input
                               | connection_bbox_output | connection_undriven
                               | connection_stopat | connection_misc
                               | ver_target_all | ver_target_primary_output
                               | ver_target_mpram_port | ver_target_bbox_input
                               | ver_target_misc | cutpoint_all |
cutpoint_both
                               | cutpoint_spec | cutpoint_imp |
cutpoint_both_init
                               | stopat_all | abstract_reset_value |
abstract_counter | abstract_init_value
                               | stopat_both | stopat_spec  | stopat_imp |
stopat_both_env
```

```
                                        | helper | init_user | init_auto | init_all) ]
               [ (-instance instance_full_path)
                 |(-of_signal {signal_list} [ -spec_regexp [-verbose]
                                                          |-imp_regexp [-verbose]
] )]
               [-proof_status (  proven | cex
                                        | unprocessed | undetermined)+]
               [-signal_type (  input | output | primary_input | primary_output
                                      | dut_input | dut_output
                                      | bbox_input | bbox_output | user_bbox_input
                                      | user_bbox_output | auto_bbox_input
                                      | auto_bbox_output | mpram_port | undriven |
internal_undriven
                                      | flop | latch | internal | stopat | reset_x
                                      | x_assignment | x_default_assignment |
x_divide_by_zero
                                      | x_index_out_of_range | x_misc_undriven
                                      | x_bus_contention | x_bus_floating)]
               [-tag {tag_name}]
               [[-task {task_name} [-include_global_mapping]]
                |-global
               ]
        )

Generating Verification
-----------------------
check_hls -generate_verification
        [-task task_name]

Infer Relations
---------------
check_hls -infer_relations
        [-valid_condition {valid_cond_expr}]
        [-input_signals signal_list]
        [-output_signals signal_list]

Prove
-----
check_hls -prove
        [-task task_name
         |-property property_name_tcl_list [-regexp]
         |(-mapping_id {mapping_id_list}
         |-dict {mapping_dict_list} [-force]
```

```
         | ( -spec {spec_regexp_or_signal_list}
             |-imp {imp_regexp_or_signal_list})
           [-regexp [-verbose]])
           [-signal_type (  input | output | primary_input | primary_output
                                  | dut_input | dut_output
                                  | bbox_input | bbox_output | user_bbox_input
                                  | user_bbox_output | auto_bbox_input |
auto_bbox_output
                                  | mpram_port | undriven | internal_undriven
                                  | flop | latch | internal | stopat | reset_x
                                  | x_assignment | x_default_assignment |
x_divide_by_zero
                                  | x_index_out_of_range | x_misc_undriven
                                  | x_bus_contention | x_bus_floating)]
           [-mapping_type (  connection_all | connection_primary_input
                                   | connection_bbox_output | connection_undriven
                                   | connection_stopat | connection_misc
                                   | ver_target_all | ver_target_primary_output
                                   | ver_target_mpram_port | ver_target_bbox_input
                                   | ver_target_misc | cutpoint_all | cutpoint_both
                                   | cutpoint_spec | cutpoint_imp | cutpoint_both_init
                                   | stopat_all | abstract_reset_value |
abstract_counter | abstract_init_value
                                   | stopat_both | stopat_spec  | stopat_imp |
stopat_both_env
                                   | helper | init_user | init_auto | init_all) ]
           [  (-instance instance_full_path)
             |(-of_signal {signal_list}
                 [ -spec_regexp [-verbose]
                  |-imp_regexp [-verbose] ] )
           ]
           [-proof_status (  proven | cex | unprocessed | undetermined)+]
           [-tag tag_name_tcl_list]
        ]
        [-time_limit time_budget]
        [-failure_extension]
        [-bg]
        [-silent |-verbosity N]
        [-with_helpers]
        [-with_proven]

Sign-Off Report
---------------
```

```
check_hls -signoff
        [-task task_name]
        [-file file_name [-json] [-force]]
        [-silent]
        [-summary]
        [-detailed]
        [-verbose]
        [-waive_category (  boundary_outputs_mapping | boundary_outputs_proof
                          | user_cutpoints | auto_cutpoints
                          | reset_value_abstraction | imported_assertions
                          | imported_covers | x_signals_and_undrivens
                          | mapped_stopats | assumptions_conflict
                          | assumptions_on_driven_signals
                          | assumptions_related_preconditions
                          | imp_assumptions)+ ]
        [[-waive_boundary_outputs_mapping {port_hierarchical_name_list}]
         [-waive_boundary_outputs_proof {port_name_list}]
         [-waive_user_cutpoints_proof {assertion_name_list}]
         [-waive_auto_cutpoints_proof {assertion_name_list}]
         [-waive_imported_assertion_proof {assertion_name_list}]
         [-waive_imported_cover_proof {cover_name_list}]
         [-waive_reset_value_abstraction {assert_name_list}]
         [-waive_x_signals_and_undrivens {signal_name_list}]
         [-waive_mapped_stopats {mapping_pair_list}]
         [-waive_assumption_on_driven_signals {assumption_name_list}]
         [-waive_assumption_related_preconditions {precondition_name_list}]
         [-waive_imp_assumptions {assumption_name_list}] [-regexp]]
        [-consider_bounded_proven_incomplete]
        [-get_valid_status
          [-summary] [-propagate_bbox_input_status_to_target] [-consider_noConflict] [-
no_cutpoint_status_propagation_to_target]]

check_hls -signoff_info
        [-task task_name]
        [-file file_name [-json] [-force]]
        [-silent]
        [-detailed]
        ( -get_category_status (  boundary_outputs_mapping
                                | boundary_outputs_proof
                                | imported_assertions | imported_covers
                                | reset_value_abstraction
                                | x_signals_and_undrivens
                                | mapped_stopats | assumptions_conflict
```

```
                                        | assumptions_on_driven_signals
                                        | assumptions_related_preconditions
                                        | imp_assumptions | user_cutpoints
                                        | auto_cutpoints)
         |-get_formal_environment (  clock | reset) )
```

Returns

Various, context sensitive return values.

Detailed Description

```
Extract Stratus BDW
-------------------
-extract -stratus_bdw
          -path path_to_stratus_workdir
          -module module_name
          -config config_name
          (-auto_setup |-proof_hints)

    Extract info from Stratus BDW

    . Use "-auto_setup" to automatically align the proof environment with the
      module synthesis setup in Stratus.
    . Use "-proof_hints" to apply hints extracted for the proof.

    NOTE: To revert changes done by this command, you must run
    "check_hls -clear all" first.

Pipeline
--------
-pipeline (-spec |-imp)
    -latency N
    -loop_label {label}
    -expand_label {label}
    -initiation_interval N
    -remove_states_before_first_io

    Returns the id of the pipeline being created
    No error checking is performed at this point
```

```
    . Use "-latency" to specify the latency interval of the pipeline.
    . Use "-loop_label" to identify the loop label in the design.
    . Use "-expand_label" to mark the expansion point of the pipeline.
    . Use "-initiation_interval" to specify the initiation interval of the pipeline.
      If you do not specify initiation interval, the default is 1.
    . Use "-remove_states_before_first_io" to allow the tool to remove wait
      calls in the beginning of the pipeline before any input/output operation.
      This can be useful to synchronize the timing to that of your
      implementation model.

Pipeline List
-------------
-pipeline -list

    Returns a list of pipelines configured by the previous command

Pipeline Remove
---------------
-pipeline -remove id

    Remove the pipeline by the id

Pipeline Clear
--------------
-pipeline -clear

    Remove all pipelines

Setup
-----
-compile (-spec |-imp)
    [-no_top_wrapper]
    [-top model_top_name]
    [-flags string]
    [-top_template_params string]
    [-bbox_m string]
    [-use_bundled_stratus_headers]

    Compile SystemC code, analyze and elaborate it.

    . Use "-no_top_wrapper" to disable generation of an sc_main
      with top instantiation.
    . Use "-top" to give a name to the top-level of the synthesized netlist.
```

```
    . Use "-flags" to give hlsfe compilation switches design attribute.
    . Use "-top_template_params" to use string as top wrapper module
      template params.
    . Use "-bbox_m" to declare a list of modules to be blackboxed.
    . Use "-use_bundled_stratus_headers" to add Stratus header files to the
      include path. These header files contain custom modifications to
      support pipelining of "cynw_p2p" interfaces.

-analyze -imp analyze_switches_and_arguments

    Analyze the HDL or property files intended for verification of the imp side.
    You can use all the "analyze" switches with this command
    except "-save", "-restore", "-req", and "-repository".
    For more details, run "help analyze -gui".

-elaborate -imp elaborate_switches_and_arguments

    Synthesize and read the netlist for the Verilog(R) and VHDL files that have
    been analyzed for imp using the "check_hls -analyze" command.
    You can use all the "elaborate" switches with this command except
    "-load_cpi", "-req", and "-repository".
    For more details, run "help elaborate -gui".

    NOTE:
    . The commands "check_hls -elaborate -log" and "check_hls -elaborate
      -list" do not support "-imp".
    . You cannot run "check_hls -elaborate" on a non-analyzed design.
      Use "check_hls -analyze" first.
    . To re-elaborate, you must first run "check_hls -clear all".
    . When specifying elaborate_switches_and_arguments for the imp side, do not
      include the imp top prefix.

-setup
    [-no_auto_mapping]
    [-auto_map_reset_x_values]

    Connect the previously analyzed and elaborated spec and imp designs, and
    auto map primary_input, primary_output, bbox_input, bbox_output, undriven,
    reset_x, and stopat signals.

    NOTE:
    . The setup auto mapping creates global mapping entries that will be
      exposed in all HLS tasks. The created connection mappings on boundary
```

```
      input signals are respected during reset.
    . Use "-no_auto_mapping" to specify that you do not want the tool to
      run auto mapping during setup. If you use this switch, you can run the
      auto mapping after the setup using
      "check_hls -setup -auto_map_reset_x_values on; check_hls -map -auto".
    . To avoid adding init mappings, run "check_hls -setup
      -auto_map_reset_x_values off".

Clear
-----
-clear (all | mapping | (result [-task task_name]))

    Clear HLS settings in the current session.

    . Use "all" to clear all HLS settings in the current session.
    . Use "mapping" to clear all mappings and their proof properties.
    . Use "result" to clear all proof information in the current or specified
      task.

Utility Switches
----------------
-get_spec_top_instance |-get_imp_top_instance
    Return the name of the spec or imp top instance.

Interface Check
---------------
-interface [-mapped |-unmapped]
           [-spec |-imp]
           [-signal_type (  primary_input | primary_output
                          | dut_input | dut_output
                          | bbox_input | bbox_output | user_bbox_input
                          | user_bbox_output | auto_bbox_input | auto_bbox_output
                          | undriven | internal_undriven | setup_stopat | reset_x
                          | x_assignment | x_default_assignment | x_divide_by_zero
                          | x_index_out_of_range | x_misc_undriven
                          | x_bus_contention | x_bus_floating
                          | user_env_stopat | user_task_stopat)]
           [-imp_assumptions [-assumption_type (  none | assumptions_embedded)]]
         [-instance instance_full_path]
         [-task task_name]
         [-ignore_user_stopats]
         [-ignore_imp_assumptions]
```

```
    Run the interface check to determine the outcome of mapping.

    Use this command without additional switches to report, for each signal
    type, whether the check passed or failed. And for failures, report how many
    signals of the specified type in the spec and imp are unmapped.

    Or specify additional switches defined below to refine the report.
    . Use "-mapped" to list mapped signals.
    . Use "-unmapped" to list unmapped signals. This is the default if you do
      not specify "-mapped".
    . Use "-spec" to list only signals from the spec design. This is the
      default if you do not specify "-imp".
    . Use "-imp" to list only signals from the imp design.
    . Use "-signal_type" to list only signals of the specified type.
    . Use "-instance" to list only signals that are in the specified instance.
    . Use "-task" to specify a task other than the current task.
    . Use "-ignore_user_stopats" to list only signals that are not formed by
      the "stopat" command.
    . Use "-ignore_imp_assumptions" to list only signals.
    . Use "-imp_assumptions" to list only the enabled imp assumptions.

    NOTE: When you run "check_hls -generate_verification", the tool also runs
    a full interface check on the current task and caches the results.

Signals List
------------
-list [ [-signal]
          [-signal_type (  input | output | primary_input | primary_output
                         | dut_input | dut_output
                         | bbox_input | bbox_output | user_bbox_input
                         | user_bbox_output | auto_bbox_input | auto_bbox_output
                         | mpram_port | undriven | internal_undriven
                         | flop | latch | internal | stopat | reset_x
                         | x_assignment | x_default_assignment | x_divide_by_zero
                         | x_index_out_of_range | x_misc_undriven
                         | x_bus_contention | x_bus_floating
                         | user_env_stopat | user_task_stopat)]
          [-spec |-imp]
          [(-coi_of |-fanout_of) {signal_list}]
          [ -instance instance_full_path
           |-name {regexp} [-verbose]
      ]
    "-list -signal" or just "-list" with no additional switches lists all
```

```
    signals.

    NOTE: The "-name" and "-instance" switches are mutually exclusive.

    . Use "-spec" or "-imp" to indicate whether the "-instance" or "-name"
      argument specifies an instance or signal(s) on the spec or imp side.
    . Use "-coi_of" to specify that you want to list signals in the COI of the
      specified signal list.
    . Use "-fanout_of" to specify that you want to list signals in the fanout
      of the specified signal list.
    . Use "-name" to define the set of signal names on the specified
      side that you want to list.
      The "-name" argument is a regular expression.
      Include the "-verbose" switch to print info messages detailing the
      regexp matching operation.
    . Use "-instance" to list all signals that are in the specified instance.
    . Use "-signal_type" to list only signals of the specified type.

Signal Mapping List
-------------------
-list -mapping
           [-mapping_type (  connection_all | connection_primary_input
                           | connection_bbox_output | connection_undriven
                           | connection_stopat | connection_misc
                           | ver_target_all | ver_target_primary_output
                           | ver_target_mpram_port | ver_target_bbox_input
                           | ver_target_misc | cutpoint_all | cutpoint_both
                           | cutpoint_spec | cutpoint_imp | cutpoint_both_init
                           | stopat_all | abstract_reset_value | abstract_counter |
abstract_init_value
                           | stopat_both | stopat_spec  | stopat_imp | stopat_both_env
                           | helper | init_user | init_auto | init_all) ]
           [-signal_type (  input | output | primary_input | primary_output
                          | dut_input | dut_output
                          | bbox_input | bbox_output | user_bbox_input
                          | user_bbox_output | auto_bbox_input
                          | auto_bbox_output | mpram_port | undriven |
internal_undriven
                          | flop | latch | internal | stopat | reset_x
                          | x_assignment | x_default_assignment | x_divide_by_zero
                          | x_index_out_of_range | x_misc_undriven
                          | x_bus_contention | x_bus_floating
                          | user_env_stopat | user_task_stopat)]
```

```
        [  ( -instance {instance_full_path} )
          |(-exclude_signal {signal_list} )
          |( -of_signal {regexp_or_signal_list}
             [ -spec_regexp [-verbose]
              |-imp_regexp [-verbose] ] )
        ]
        [-tag {tag_name}]
        [([-task {task_name}] [-include_global_mapping])
         |-global
        ]
        [-return (text | id | dict)]
        [-proof_status (proven | cex | unprocessed | undetermined)+]
        [-proof_task {proof_task_name}]
        [ (-coi_of {signal_list} [-get_first_level_coi])
         |(-fanout_of {signal_list} [-get_first_level_fanout])]
        [-status (enabled | disabled)]
"-list -mapping" with no additional switches lists the current
  task's mapping pairs. Use "-global" to list global mappings.

NOTE: The "-of_signal" and "-instance" switches are mutually exclusive.

. Use "-mapping_type" to list only mapping pairs of the specified mapping
  type.
. Use "-signal_type" to list only mapping pairs with signals of the
  specified signal type.
. Use "-of_signal" to define the set of signal names whose mappings you
  want to list.
    . Use "-spec_regexp" or "-imp_regexp" to specify that the "-of_signal"
      argument is a regular expression list on the spec or imp side.
      The default is spec side; otherwise, it is a list of specific names
      of signals, bits, or bit-vector expressions.
    . Use "-verbose" to print info messages detailing the regexp matching
      operations.
. Use "-instance" to specify that you want to list only mapping pairs of
  signals that are in the specified instance.
. Use "-exclude_signal" to instruct HLSEC to exclude mappings mapped on the
  passed signal(s).
. Use "-tag" to list only mapping pairs with the specified tag.
. Use "-task" to list the specified task's mapping instead of the current
  task's mapping.
. Use "-include_global_mapping" to list both the global and task-specific
  mapping pair(s).
. Use "-global" to list global mapping.
```

```
    . Use "-return" to specify the return type.
      . Use "-return text" to return the result as mapping text. (This is the
        default behavior.)
      . Use "-return id" to return the result as mapping IDs.
      . Use "-return dict" to return the result as a list of mapping
        dictionaries. See the section "Mapping Dictionary" for more info.
    . Use "-proof_status" to filter mapping entries according to their proof
      status. This switch accepts one or more of the following arguments:
      "proven", "cex", "unprocessed", "undetermined".
    . Use "-proof_task" to filter mapping entries according to their proof
      status in the proof task specified. Use this switch in combination with
      the switches "-global" and "-proof_status".
    . Use "-coi_of" to list mapping pairs in the COI of the specified signal
      list, and use "-get_first_level_coi" to retrieve mapping pairs for the
      first level of the COI.
    . Use "-fanout_of" to list mapping pairs in the fanout of the specified
      signal list, and use "-get_first_level_fanout" to get mapping pairs
      for the first level of the fanout.
    . Use "-status" to filter mapping entries according to their status. This
      switch accepts one of the following arguments: "enabled", "disabled".

Mapping Info
------------
-mapping_info (  -id mapping_id_list
               | -dict {mapping_dict_list} [-force]
               | ( -spec {spec_regexp_or_signal_list}
                  |-imp {imp_regexp_or_signal_list}
                   [-regexp [-verbose]])
             )
             [-signal_type (  input | output | primary_input | primary_output
                            | dut_input | dut_output
                            | bbox_input | bbox_output | user_bbox_input
                            | user_bbox_output | auto_bbox_input | auto_bbox_output
                            | mpram_port | undriven | internal_undriven
                            | flop | latch | internal | stopat | reset_x
                            | x_assignment | x_default_assignment | x_divide_by_zero
                            | x_index_out_of_range | x_misc_undriven
                            | x_bus_contention | x_bus_floating)]
             [-mapping_type (  connection_all | connection_primary_input
                             | connection_bbox_output | connection_undriven
                             | connection_stopat | connection_misc
                             | ver_target_all | ver_target_primary_output
                             | ver_target_mpram_port | ver_target_bbox_input
```

```
                              | ver_target_misc | cutpoint_all | cutpoint_both
                              | cutpoint_spec | cutpoint_imp | cutpoint_both_init
                              | stopat_all | abstract_reset_value | abstract_counter
                              | stopat_both | stopat_spec  | stopat_imp |
stopat_both_env
                              | helper | init_user | init_auto | init_all) ]
              [  ( -instance {instance_full_path} )
                |( -of_signal {regexp_or_signal_list}
                   [ -spec_regexp [-verbose]
                    |-imp_regexp [-verbose] ] )
              ]
              [-tag {tag_name}]
              [([-task {task_name}] [-include_global_mapping])
               |-global
              ]
              [-list (  spec_signal | imp_signal | context | source | id
                      | sva_expression | property | is_enabled | is_inverse
                      | type | task_name | tag | status)]
              [-silent]
```

List info for specific mapping pairs.
Unless a specific field is requested, this command returns, for each
specified mapping pair, the following tuple:
{ mapping pair ID, spec_signal, imp_signal, context,
  mapping source, assert/assume expression,
  property name, standard textual representation,
  is enabled, is verification pair, is inverse,
  mapping type, task name, tag }.

. Use "-task" to list the specified task's mapping instead of the current
  task's mapping.
  . When using "-list property", the tool only shows the properties for the
    specified task instead of the current task.
  . When using "-list status", the tool only shows the statuses for the
    specified task instead of the current task.
. Use "-global" to list global mapping.
. Use "-include_global_mapping" to list both the global and the
  task-specific mapping pair(s).
. Use "-id" to specify the list of IDs of the specific mapping pairs.
. Use "-dict" to specify the list of mapping dictionaries of specific
  mapping pairs. See the section on "Mapping Dictionary" for more info.
. Use "-spec" to define the set of signal name(s) on the spec side whose
  mapping info you want to list.

```
    . Use "-imp" to define the set of signal name(s) on the implementation side
      whose mapping info you want to list.
    . Use "-regexp" to specify that the "-spec" or "-imp" argument list
      is a regular expression list; otherwise, it is a list of specific names
      of signals, bits, or bit-vector expressions. Include the "-verbose"
      switch to print info messages detailing the regexp matching operation.
    . Use "-signal_type" to list only mappings of signals of the specified
      type.
    . Use "-mapping_type" to list only mapping pairs of the specified mapping
      type.
    . Use "-of_signal" to define the set of signal names whose mappings you
      want to list.
        . Use "-spec_regexp" or "-imp_regexp" to specify that the "-of_signal"
          argument is a regular expression list on the spec or imp side.
          The default is spec side; otherwise, it is a list of specific names
          of signals, bits, or bit-vector expressions.
        . Use "-verbose" to print info messages detailing the regexp matching
          operations.
    . Use "-instance" to specify that you want to list only mapping  pairs of
      signals that are in the specified instance.
    . Use "-proof_status" to list mapping entries according to their proof
      status.
      This switch accepts one or more of the following arguments:
      "proven", "cex", "unprocessed", "undetermined".
    . Use "-tag" to list only mappings with the specified tag.
    . Use "-list" to get specific information on the specified mapping entries.
      This switch accepts one of the following arguments:
      "spec_signal", "imp_signal", "context", "source", "sva_expression", "id",
      "property", "is_enabled", "is_inverse", "type",
      "task_name", "tag", or "status".
    . Use "-silent" to turn off output to the screen and return mapping
      information with a Tcl return value.

Signal Mapping
--------------
-map   ([-auto]
          [-spec {spec_instance_list}
          [-imp {imp_instance_list}]]
          [-signal_type (  input | output | primary_input | primary_output
                         | dut_input | dut_output
                         | bbox_input | bbox_output | user_bbox_input
                         | user_bbox_output | auto_bbox_input | auto_bbox_output
                         | mpram_port | undriven | internal_undriven
```

```
                    | flop | latch | internal | stopat | reset_x
                    | x_assignment | x_default_assignment | x_divide_by_zero
                    | x_index_out_of_range | x_misc_undriven
                    | x_bus_contention | x_bus_floating)]
    [-coi_of {signal_list}]
    [-set_value (  0 | 1)]
    [-max_word_size max_word_size_value]
    [-return (  text | id | dict | none)]
    [-spec_delay spec_delay_value]
    [-imp_delay imp_delay_value]
    [-map_condition {map_cond_expr}]
    [-clock {clocking_event}]
    [-disable {disable_cond_expr}]
    [-edge (  posedge | negedge | both_edge)]
    [ -cut (  both | both_init | both_no_init | spec | imp)
     |-stopat (  both | spec | imp | both_env)
     |-constraint
     |-helper]
    [-tag {tag_name}]
    [-task {task_name} |-global]
    [-exclude_signal {regexp_or_signal_list} [-signal_regexp]]
    [-respect_connections_during_reset]
    [-exclude_module {regexp_or_module_list} [-module_regexp]]
    [-exclude_instance {regexp_or_instance_list} [-instance_regexp]]
 )
|( ( -spec {spec_regexp_or_signal_list} [-imp {imp_regexp_or_signal_list}])
      [-regexp [-verbose]]
      [-max_word_size max_word_size_value]
      [-inverse]
      [-return (  text | id | dict | none)]
      [-spec_delay spec_delay_value]
      [-imp_delay imp_delay_value]
      [-map_condition {map_cond_expr}]
      [-clock {clocking_event}]
      [-disable {disable_cond_expr}]
      [-edge (  posedge | negedge | both_edge)]
      [ -cut (  both | both_init | spec | imp)
       |-stopat (  both | spec | imp | both_env)
       |-helper
       |-constraint]
      [-tag {tag_name}]
      [-task {task_name} |-global]
      [-init]
```

```
        [-respect_connections_during_reset]
   )
  | -map_file map_file_name

Map specification signals to implementation signals.

Auto Mapping
------------
Use "-auto" to automatically map primary inputs and outputs, inputs and
output signals of black boxes, and undriven signals that have the same
names in the specification and implementation.
. Use "-spec" with "-auto" to specify the spec instance list whose signals
  are to be auto mapped.
. Use "-imp" with "-auto" to specify the imp instance list whose signals
  are to be auto mapped. If you use "-spec" but not "-imp", the tool
  attempts to map the spec instance signals to imp signals in an instance
  of the same name, taking into account the imp prefix. If you use both
  "-spec" and "-imp" switches, the tool considers pairs of instances by
  matching each spec instance from the given list to the imp instance in
  the same position on the imp list. Therefore, the two supplied spec and
  imp lists must be of the same length.
. Use "-signal_type" and a signal type to auto map only signals of the
  specified type.
. Use "-coi_of" to specify that you want to auto map signals in the COI of
  the specified signal list.
. Use "-set_value" to set the mapped undriven signals to a constant value,
  0 or 1.
. Use "-max_word_size" to specify a maximum size for mapping entries.
  Mapping of vector signals or bit ranges that are wider than the specified
  value splits the entries into small "chunks" with a width up to
  "max_word_size" each. Specify the value 0 to use the full range of bits.
. Use "-tag" to add a tag (property name prefix) to the generated
  mapping(s).
. Use "-task" to consider the specified task's mapping instead of the
  current task mapping.
. Use "-global" to consider global mapping, that is, mapping that is
  exposed in all tasks.
. Use "-respect_connections_during_reset" to create connections that are
  respected during reset. This switch requires the "-global" switch.
  This switch affects only connection mappings (such as primary inputs,
  bbox outputs, undrivens, and so forth); therefore, you cannot use it with
  the following switches: -cut, -helper, -init.
  NOTE:
```

```
    . Using "-respect_connections_during_reset" with "-stopat" is
      not supported. Combining the switches triggers an error.
    . Using "-respect_connections_during_reset" with "-cut" is forbidden
      because the assertion added by the cutpoint is not respected during
      reset. Combining the switches triggers an error.
  . Use "-exclude_signal" to instruct HLSEC to exclude signals(s) from auto
    mapping and use "-signal_regexp" to specify that the "-exclude_signal"
    argument list is a regular expression list.
  . Use "-exclude_module" to instruct HLSEC to exclude module(s) from auto
    mapping and use "-module_regexp" to specify that the "-exclude_module"
    argument list is a regular expression list.
  . Use "-exclude_instance" to instruct HLSEC to exclude instance(s) from
    auto mapping and use "-instance_regexp" to specify that the
    "-exclude_instance" argument list is a regular expression list.
NOTE: The "-exclude_module", "-exclude_instance", and "-exclude_signal"
switches are not allowed with "check_hls -map -auto -signal_type reset_x".

Manual Mapping
--------------
If you do not use "-auto", manually map the spec to the imp with the
following switches:
  . Use "-spec" to define the name(s) of the spec signals to map.
    . The signal can be a bit-vector signal. In such cases, the imp
      signal must be a bit-vector signal of the same length. Specify
      sub-vectors of such bit-vector signals with an expression of the
      form "[left_index:right_index]", for example, [9:13] or [3:0]. The
      bits are mapped according to their order from the right vector bound
      to the left one.
  . Use "-imp" to define the name(s) of the imp signals, bits, or
    bit-vector expressions.
    . If you use "-spec" but not "-imp", then for each spec_regexp_or_signal
      from the given list, the tool attempts to map the spec signals to imp
      signals of the same name, taking into account the imp prefix.
    . If you use both "-spec" and "-imp" switches, the tool considers pairs
      of regexp_or_signal by matching each spec regexp_or_signal from the
      given list to the imp regexp_or_signal in the same position on the imp
      list. Therefore, the two supplied spec and imp lists must be of the
      same length.
    . You can use multi-bit vector expressions as in {spec_regexp_or_signal}.
  . Use "-max_word_size" to specify a maximum size for mapping entries.
    Mapping of vector signals or bit ranges that are wider than the specified
    value splits the entries into small "chunks" with a width up to
    "max_word_size" each. Specify the value 0 to use the full range of bits.
```

```
. Use "-inverse" to specify that the mapped signals are the inverse (not)
  of each other. The tool generates assume or assert properties
  accordingly.
. Use "-tag" to add a tag (property name prefix) to the generated
  mapping(s).
. Use "-task" to relate the generated mappings to a specified task instead
  of the current task.
. Use "-global" to create global mapping, that is, mapping that will be
  exposed in all tasks.
. Use "-init" to create init mapping (that is, bound 1 equivalence
  assumptions).
. Use "-constraint" to create connection mapping, that is, to connect the
  signal pair by adding equivalence assumptions on them.
. Use "-respect_connections_during_reset" to create connections that are
  respected during reset. This switch requires the "-global" switch.
  This switch affects only connection mappings (such as primary inputs,
  bbox outputs, undrivens, and so forth); therefore, you cannot use it with
  the following switches: -cut, -helper, -init.
  NOTE:
  . Using "-respect_connections_during_reset" with "-stopat" is
    not supported. Combining the switches triggers an error.
  . Using "-respect_connections_during_reset" with "-cut" is forbidden
    because the assertion added by the cutpoint is not respected during
    reset. Combining the switches triggers an error.
. Use "-respect_abstraction_during_reset" to create reset_value
  abstractions that are respected during reset. This switch requires the
  "-global" switch.

Mapping Using Regular Expressions
---------------------------------
Use "-regexp" to inform the tool that the argument of the "-spec" switch,
{spec_regexp_or_signal_list}, and the argument of the "-imp" switch,
{imp_regexp_or_signal_list}, are regular expressions.
. If you use "-regexp" with "-spec" but without "-imp", the tool evaluates
  the regular expression {spec_regexp_or_signal_list} and then maps each
  spec signal that matches the expression to its matching imp signal.
. If you use "-regexp" with both "-spec" and "-imp", consider using (?:RE)
  patterns in {spec_regexp_or_signal_list} to group sub-patterns.
  Accordingly, in {imp_regexp_or_signal_list}, use references to group
  values matched by {spec_regexp_or_signal}: $0, $1, $2, $3, ..., $9.
  . If you do not use (?:RE) patterns, at least one of the arguments
    {spec_regexp_or_signal_list} or {imp_regexp_or_signal_list} must be an
    expression that is evaluated to a single signal; otherwise, if both
```

```
    expressions are evaluated to a list of signals,
    the HLSEC App will not be able to perform the mapping due to ambiguity
    in signal pairs selection.
. Include the "-verbose" switch to print info messages detailing the
  regexp matching operation.
. Use "-exclude_signal" to instruct HLSEC to exclude signals(s) from
  mapping and use "-signal_regexp" to specify that the "-exclude_signal"
  argument list is a regular expression list.

Task-Related Mapping
--------------------
With either auto or manual mapping, the tool relates the generated mappings
to the current task; that is, the generated mapping pairs will be exposed
as properties for the current task only.
. Use "-task" to relate the generated mappings to a specified task instead
  of the current task.
. Use "-global" to create global mapping, that is, mapping that will be
  exposed in all tasks.
Any auto-generated connection mapping is global regardless of the supplied
task switches because connection mapping is exposed as "-env" properties,
which affect all tasks.

Return Type
-----------
With either auto or manual mapping, use "-return" and one of
the following arguments to return information:
  . Use "text" to return the result as mapping text. (This is the
    default behavior.)
  . Or use "id" to return the result as mapping IDs.
  . Or use "dict" to return the result as list of mapping dictionaries.
    See the section on "Mapping Dictionary" for more info.
  . Or use "none" to return nothing (empty list of strings).

Mapping Pairs as Helpers
------------------------
With either auto or manual mapping, use "-helper" to create helper mapping
pairs. Helper mapping pairs direct the tool to create helper assertions
during the generation phase ("check_hls -generate_verification"). Refer to
the HLSEC User Guide for a detailed explanation of helper assertions.

The purpose of generating helpers is to prevent any unintended change in
the environment. For example, if you add a new mapping pair on a primary
input or an undriven signal, the tool generates a new "-env" assumption
```

```
(even if that was not your intention); however, using "-helper" prevents
the tool from generating an assumption and only exposes helper assertions.

Mapping Pairs as Cutpoints
--------------------------
With either auto or manual mapping, use "-cut" to create cutpoint mapping
pairs. Refer to the HLSEC User Guide for a detailed explanation of
cutpoints.
. Use "-cut both" to add cutpoints on the signals on both sides
  without the concrete value init assumption.
. Use "-cut both_init" to add cutpoints on the signals on both sides.
. Use "-cut spec" to add a cutpoint only on the signal on the spec side.
. Use "-cut imp" to add a cutpoint only on the signal on the imp side.

Mapping Pairs as Stopats
------------------------
With either auto or manual mapping, use "-stopat" to create stopat mapping
pairs. During the generation phase ("check_hls -generate_verification"),
the tool adds stopats to mapped signals (see the "stopat" command) and
generates an equivalence assumption on them.
. Use "-stopat both" to add stopats on the signals on both sides.
. Use "-stopat spec" to add a stopat only on the signal on the spec side.
. Use "-stopat imp" to add a stopat only on the signal on the imp side.
. Use "-stopat both_env" to add stopats that are respected during reset on
  the signals on both sides.

Temporal and Logical Context
----------------------------
Use the following switches to specify temporal and logical context for new
mappings. This set of switches generates various combinations of temporal
and logical conditions on spec and imp equivalence.
. -spec_delay
. -imp_delay
. -map_condition
. -clock
. -disable

Important things to know:
. If you do not specify delay values, the default is 0.
. Only one of "-spec_delay" or "-imp_delay" can be greater than 0.
. All condition values default to 1 (true).
. Delay values are non-negative integers.
. The following condition expressions are valid SVA expressions, and each
```

```
    of them might contain signals on both sides.
    . {map_cond_expr}
    . {disable_cond_expr}
  . {clocking_event} is "negedge" or "posedge" plus an SVA expression for
    clock signals.
  . If you supply all of these values along with the spec_signal mapped to
    the imp_signal, the tool generates an expression equivalent to the
    following:
@ (clocking_event)
          disable iff (disable_cond_expr)
                (
                   (##spec_delay_value  1'b1).triggered and map_condition.triggered
                   |->
                   spec_signal == $past(imp_signal,  spec_delay_value)
                )

    NOTE:
    . This expression specifies a condition on spec_signal and imp_signal that
      is checked on every {clocking_event} and when {disable_cond_expr} is
      false.
    . This expression is built out of the antecedent on the left of the
      implication (|->) and the consequent on its right.
    . The equality between the mapped signals is the consequent, but it might
      be more than a simple equality; that is, the mapped signals might
      differ on the relative time when they are sampled.
    . All the delays in the context expression are relative to the time of
      the event of the implication ("the |-> time").
    . {map_cond_expr} can be any Boolean expression;
      that is, it is not necessary for it to contain signals only from one
      side.
    . The consequent is an equality between the mapped signals such that the
      spec signal is sampled spec_delay_value time cycles after the
      implication time and the imp signal is sampled imp_delay_value time
      cycles after the implication time.
    . If imp_delay_value >= 0, then the consequent is of the
      following form:
      ( $past(spec_signal, imp_delay_value) == imp_signal )
    . The whole implication condition is valid only if, at implication time,
      {disable_cond_expr} is false.
    . The time frames are counted according to the {clocking_event}.

  Logical Context for Clocks
  --------------------------
```

```
    Use the following switches to specify logical context for new
    mappings on clock signals.
    . -map_condition
    . -edge

    Important things to know:
    . These clock signals need a specific mapping switch due to the SVA
      semantics of clock signal values inside properties. To enable checking
      the clock signal under a specific condition, we must check the
      equivalence of the @({edge} clock_signal) event of the spec and imp
      clocks when the map_condition is met.
    . A clocking signal is a signal that is combinationally derived from
      user-defined clock signals.
    . The tool does not accept "-edge" without "-map_condition".
      However, you can achieve the same effect with "-map_condition 1".
    . If you do not specify "-edge", the tool provides a value.
    . The {map_cond_expr} condition expression is a non-temporal valid SVA
      expression, and it contains signals from both sides (spec and imp).

    Manual Mapping with a Map File
    ------------------------------
    Use "-map_file" to run a user-defined Tcl file that specifies mapping.

Signal Unmapping
----------------
-unmap
        ( -id mapping_id_list
         |-dict mapping_dict_list
         |-spec {spec_regexp_or_signal_list}
         |-imp {imp_regexp_or_signal_list}
        )
           [-regexp [-verbose]]
           [-mapping_type (  connection_all | connection_primary_input
                           | connection_bbox_output | connection_undriven
                           | connection_stopat | connection_misc
                           | ver_target_all | ver_target_primary_output
                           | ver_target_mpram_port | ver_target_bbox_input
                           | ver_target_misc | cutpoint_all | cutpoint_both
                           | cutpoint_spec | cutpoint_imp | cutpoint_both_init
                           | stopat_all | abstract_reset_value | abstract_counter |
abstract_init_value
                           | stopat_both | stopat_spec  | stopat_imp | stopat_both_env
                           | helper | init_user | init_auto | init_all) ]
```

```
        [  (-instance instance_full_path)
          |(-of_signal {signal_list}
              [ -spec_regexp [-verbose]
               |-imp_regexp [-verbose] ] )
        ]
        [-proof_status (  proven | cex | unprocessed | undetermined)+]
        [-signal_type (  input | output | primary_input | primary_output
                       | dut_input | dut_output
                       | bbox_input | bbox_output | user_bbox_input
                       | user_bbox_output | auto_bbox_input | auto_bbox_output
                       | mpram_port | undriven | internal_undriven
                       | flop | latch | internal | stopat | reset_x
                       | x_assignment | x_default_assignment | x_divide_by_zero
                       | x_index_out_of_range | x_misc_undriven
                       | x_bus_contention | x_bus_floating)]
        [-tag {tag_name}]
        [-include_global_mapping]
        [(-task task_name [-include_global_mapping])
         |-include_global_mapping
        ]
Cancel a specified mapping pair or all mappings that involve signals that
match spec_regexp_or_signal_list or imp_regexp_or_signal_list.
The tool also cancels all properties that might have been generated from
these mappings.
. Use "-id" with a list of IDs for the mapping pairs you want to cancel.
. Use "-dict" with a list of mapping dictionaries for the mapping pairs
  you want to cancel. See the section on "Mapping Dictionary" for more info.
. Use "-spec" to define the set of signal name(s) on the spec side that you
  want to unmap.
. Use "-imp" to define the set of signal name(s) on the imp side that you
  want to unmap.
  . Use "-regexp" to specify that the "-spec" or "-imp" argument list
    is a regular expression list; otherwise, it is a list of specific names
    of signals, bits, or bit-vector expressions. Include the "-verbose"
    switch to print info messages detailing the regexp matching operation.
  . Use "-signal_type" to unmap only signals of the specified type.
    You can only use this switch with "-spec" or "-imp" and "-regexp".
    That is, if you specify mapping IDs, all types are unmapped.
  . Use "-mapping_type" to unmap the mapping pairs of the specified mapping
    type.
  . Use "-of_signal" to define the set of signal names whose mappings you
    want to unmap.
    . Use "-spec_regexp" or "-imp_regexp" to specify that the "-of_signal"
```

```
          argument is a regular expression list on the spec or imp side. The
          default is spec side; otherwise, it is a list of specific names of
          signals, bits, or bit-vector expressions.
        . Use "-verbose" to print info messages detailing the regexp matching
          operations.
      . Use "-instance" to specify that you want to unmap mapping pairs of
        signals that are in the specified instance.
      . Use "-proof_status" to unmap mapping entries according to their proof
        status. This switch accepts one or more of the following arguments:
        "proven", "cex", "unprocessed", "undetermined".
      . Use "-tag" to unmap only mappings with the specified tag.
    . Use "-task" to unmap the specified task's mapping instead of the current
      task's mapping.
    . Use "-global" to unmap exposed global mapping in all tasks.
    . Use "-include_global_mapping" to unmap both the global and task-specific
      mapping pair(s).

AutoProve Unmapping
-------------------
-unmap -autoprove_mappings [-task task_name]
    Unmap all mappings created by HLSEC AutoProve.

    Use "-task" to unmap the specified task's mapping instead of the current
    task's mapping.

Setting Mapping Context
-----------------------
-set_context_of_mapping
         (   -id mapping_id_list
          |  -dict mapping_dict_list [-force]
          | ( -spec {spec_regexp_or_signal_list}
              |-imp {imp_regexp_or_signal_list})
                [-regexp [-verbose]]
                [-signal_type (  input | output | primary_input | primary_output
                               | dut_input | dut_output
                               | bbox_input | bbox_output | user_bbox_input
                               | user_bbox_output | auto_bbox_input | auto_bbox_output
                               | mpram_port | undriven | internal_undriven
                               | flop | latch | internal | stopat | reset_x
                               | x_assignment | x_default_assignment | x_divide_by_zero
                               | x_index_out_of_range | x_misc_undriven
                               | x_bus_contention | x_bus_floating)]
                [-mapping_type (  connection_all | connection_primary_input
```

```
                                | connection_bbox_output | connection_undriven
                                | connection_stopat | connection_misc
                                | ver_target_all | ver_target_primary_output
                                | ver_target_mpram_port | ver_target_bbox_input
                                | ver_target_misc | cutpoint_all | cutpoint_both
                                | cutpoint_spec | cutpoint_imp | cutpoint_both_init
                                | stopat_all | abstract_reset_value | abstract_counter
| abstract_init_value
                                | stopat_both | stopat_spec  | stopat_imp |
stopat_both_env
                                | helper | init_user | init_auto | init_all) ]
              [  ( -instance {instance_full_path} )
                |( -of_signal {regexp_or_signal_list}
                   [ -spec_regexp [-verbose]
                    |-imp_regexp [-verbose] ] )
              ]
              [-proof_status (  proven | cex | unprocessed | undetermined)+]
              [-tag {tag_name}]
              [([-task {task_name}] [-include_global_mapping]) |-global]
         |-set_default
         |-clear_default
        )
        [-spec_delay spec_delay_value]
        [-imp_delay imp_delay_value]
        [-map_condition {map_cond_expr}]
        [-clock {clocking_event}]
        [-disable {disable_cond_expr}]
        [-edge (  posedge | negedge | both_edge)]
```

Set or change the temporal and logical context of a set of mappings.

- Use "-id" with a list of IDs for the mapping pairs whose context you want to change.
- Use "-dict" with a list of mapping dictionaries for the mapping pairs whose context you want to change. See the section on "Mapping Dictionary" for more info.
- Use "-spec" to define the set of signal names on the spec side whose mapping context you want to change.
- Use "imp" to define the set of signal names on the imp side whose mapping context you want to change.
  - Use "-regexp" to specify that the "-spec" or "-imp" argument list is a regular expression list; otherwise, it is a list of specific names of signals, bits, or bit-vector expressions. Include the "-verbose" switch to print info messages detailing the regexp matching operation.

    . Use "-signal_type" to set the context only for mappings of signals of
      the specified type.
      You can only use this switch with "-spec" or "-imp" and "-regexp".
      That is, if you specify mapping IDs, the context is set for all types.
    . Use "-mapping_type" to set the context only for mapping pairs of the
      specified mapping type.
    . Use "-of_signal" to define the set of signal names whose mappings you
      want to list.
      . Use "-spec_regexp" or "-imp_regexp" to specify that the "-of_signal"
        argument is a regular expression list on the spec or imp side.
        The default is spec side; otherwise, it is a list of specific names
        of signals, bits, or bit-vector expressions.
      . Use "-verbose" to print info messages detailing the regexp matching
        operations.
    . Use "-instance" to specify that you want to set the context only for
      mapping pairs of signals that are in the specified instance.
    . Use "-proof_status" to set the context for mapping entries according to
      their proof status.
      This switch accepts one or more of the following arguments:
      "proven", "cex", "unprocessed", "undetermined".
    . Use "-tag" to set the context only for mappings with the specified tag.
  . Use "-task" to set context for the specified task's mapping instead of
    the current task's mapping.
  . Use "-global" to set context for exposed global mapping in all tasks.
  . Use "-include_global_mapping" to set context for both the global and
    task-specific mapping pair(s).
  . Use "-set_default" to set the default context. Once you set the default
    context, every new mapping with no explicit context has the default
    context. To use the default context in the setup for automatic mapping,
    use "check_hls -setup... -setup_file...".
    NOTE: The default context applies to all mapping, including pairs of
    inputs and clock and reset signals.
  . Use "-clear_default" to clear the default context.
  . Use "-spec_delay", "-imp_delay", "-map_condition",
    "-clock", and "-disable" to specify temporal and logical context to the
    specified mappings. This set of switches generates various combinations
    of temporal and logical conditions on spec and imp equivalence. For
    usage, refer to the subsection "Temporal and Logical Context" in the
    section "Signal Mapping".
  . Use "-map_condition" and "-edge" to specify the logical context for the
    specified mappings on clock signals. For usage, refer to the subsection
    "Logical Context for Clocks" in the section "Signal Mapping".

```
Clearing Mapping Context
------------------------
-clear_context_of_mapping
         (   -id mapping_id_list
          |  -dict mapping_dict_list [-force]
          | ( -spec {spec_regexp_or_signal_list}
              |-imp {imp_regexp_or_signal_list})
                [-regexp [-verbose]]
                [-signal_type (  input | output | primary_input | primary_output
                               | dut_input | dut_output
                               | bbox_input | bbox_output | user_bbox_input
                               | user_bbox_output | auto_bbox_input | auto_bbox_output
                               | mpram_port | undriven | internal_undriven
                               | flop | latch | internal | stopat | reset_x
                               | x_assignment | x_default_assignment | x_divide_by_zero
                               | x_index_out_of_range | x_misc_undriven
                               | x_bus_contention | x_bus_floating)]
                [-mapping_type (  connection_all | connection_primary_input
                                | connection_bbox_output | connection_undriven
                                | connection_stopat | connection_misc
                                | ver_target_all | ver_target_primary_output
                                | ver_target_mpram_port | ver_target_bbox_input
                                | ver_target_misc | cutpoint_all | cutpoint_both
                                | cutpoint_spec | cutpoint_imp | cutpoint_both_init
                                | stopat_all | abstract_reset_value | abstract_counter
| abstract_init_value
                                | stopat_both | stopat_spec  | stopat_imp |
stopat_both_env
                                | helper | init_user | init_auto | init_all) ]
                [  ( -instance {instance_full_path} )
                  |( -of_signal {regexp_or_signal_list}
                     [ -spec_regexp [-verbose]
                      |-imp_regexp [-verbose] ] )
                ]
                [-proof_status (  proven | cex | unprocessed | undetermined)+]
                [-tag {tag_name}]
                [[-task {task_name} [-include_global_mapping]] |-global]
         )

    Clear the temporal and logical context of a set of mappings.
    . Use "-id" with a list of IDs for the mapping pairs whose context you
      want to clear.
    . Use "-dict" with a list of mapping dictionaries for the mapping
```

```
      pairs whose context you want to clear. See the section on
      "Mapping Dictionary" for more info.
    . Use "-spec" to define the set of signal names on the spec side whose
      mapping context you want to clear.
    . Use "imp" to define the set of signal names on the imp side whose
      mapping context you want to clear.
      . Use "-regexp" to specify that the "-spec" or "-imp" argument list
        is a regular expression list; otherwise, it is a list of specific names
        of signals, bits, or bit-vector expressions. Include the "-verbose"
        switch to print info messages detailing the regexp matching operation.
      . Use "-signal_type" to clear the context only for mappings of signals of
        the specified type.
        You can only use this switch with "-spec" or "-imp" and "-regexp".
        That is, if you specify mapping IDs, the context is cleared for all types.
      . Use "-mapping_type" to clear the context only for mapping pairs of the
        specified mapping type.
      . Use "-of_signal" to define the set of signal names whose mappings you
        want to list.
        . Use "-spec_regexp" or "-imp_regexp" to specify that the "-of_signal"
          argument is a regular expression list on the spec or imp side.
          The default is spec side; otherwise, it is a list of specific names
          of signals, bits, or bit-vector expressions.
        . Use "-verbose" to print info messages detailing the regexp matching
          operations.
      . Use "-instance" to specify that you want to clear the context only for
        mapping pairs of signals that are in the specified instance.
      . Use "-proof_status" to clear the context for mapping entries according to
        their proof status.
        This switch accepts one or more of the following arguments:
        "proven", "cex", "unprocessed", "undetermined".
      . Use "-tag" to clear the context only for mappings with the specified tag.
    . Use "-task" to clear context for the specified task's mapping instead of
      the current task's mapping.
      NOTE: "-task" cannot be used with "-id".
    . Use "-global" to clear context for exposed global mapping in all tasks.
    . Use "-include_global_mapping" to clear context for both the global and
      task-specific mapping pair(s).

Enabling and Disabling Mapping
---------------------------------------
-enable_mapping |-disable_mapping
    (  -id mapping_id_list
     | -dict mapping_dict_list [-force]
```

```
    |(-spec {spec_regexp_or_signal_list} |-imp {imp_regexp_or_signal_list})
        [-regexp [-verbose]]
        [-signal_type (  input | output | primary_input | primary_output
                       | dut_input | dut_output
                       | bbox_input | bbox_output | user_bbox_input
                       | user_bbox_output | auto_bbox_input | auto_bbox_output
                       | mpram_port | undriven | internal_undriven
                       | flop | latch | internal | stopat | reset_x
                       | x_assignment | x_default_assignment | x_divide_by_zero
                       | x_index_out_of_range | x_misc_undriven
                       | x_bus_contention | x_bus_floating)]
        [-mapping_type (  connection_all | connection_primary_input
                        | connection_bbox_output | connection_undriven
                        | connection_stopat | connection_misc
                        | ver_target_all | ver_target_primary_output
                        | ver_target_mpram_port | ver_target_bbox_input
                        | ver_target_misc | cutpoint_all | cutpoint_both
                        | cutpoint_spec | cutpoint_imp | cutpoint_both_init
                        | stopat_all | abstract_reset_value | abstract_counter |
abstract_init_value
                        | stopat_both | stopat_spec  | stopat_imp | stopat_both_env
                        | helper | init_user | init_auto | init_all) ]
        [ (-instance {instance_full_path} )
         |(-of_signal {regexp_or_signal_list} [ -spec_regexp [-verbose]
                                              |-imp_regexp [-verbose] ] )]
        [-proof_status (proven | cex | unprocessed | undetermined)+]
        [-tag {tag_name}]
     [[-task {task_name} [-include_global_mapping]] |-global]
    )

    Enable or disable specific mappings.
    If you enable or disable mappings, their properties are enabled or
    disabled accordingly.
    . Use "-dict" to specify the list of mapping dictionaries of the specific
      mapping pairs you want to enable or disable. See the section on
      "Mapping Dictionary" for more info.
    . Use "-id" to specify the list of IDs of the specific mapping pairs you
      want to enable or disable.
    . Use "-spec" instead of IDs to define the set of signal name(s) on the
      spec side whose mappings you want to enable or disable.
    . Use "-imp" instead of IDs to define the set of signal name(s) on the imp
      side whose mappings you want to enable or disable.
      . Use "-regexp" to specify that the "-spec" or "-imp" argument list
```

```
        is a regular expression list; otherwise, it is a list of specific names
        of signals, bits, or bit-vector expressions. Include the "-verbose"
        switch to print info messages detailing the regexp matching operation.
      . Use "-signal_type" to enable or disable only mappings of signals of the
        specified type.
        You can only use this switch with "-spec" or "-imp" and "-regexp".
        That is, if you specify mapping IDs, all types are enabled or disabled.
      . Use "-mapping_type" to enable or disable only mapping pairs of the
        specified mapping type.
      . Use "-of_signal" to define the set of signal names whose mappings you
        want to list.
        . Use "-spec_regexp" or "-imp_regexp" to specify that the "-of_signal"
          argument is a regular expression list on the spec or imp side.
          The default is spec side; otherwise, it is a list of specific names
          of signals, bits, or bit-vector expressions.
        . Use "-verbose" to print info messages detailing the regexp matching
          operations.
      . Use "-instance" to specify that you want to enable or disable only
        mapping pairs of signals that are in the specified instance.
      . Use "-proof_status" to enable or disable mapping entries according to
        their proof status.
        This switch accepts one or more of the following arguments:
        "proven", "cex", "unprocessed", "undetermined".
      . Use "-tag" to enable or disable only mappings with the specified tag.
    . Use "-task" to enable or disable the specified task's mappings instead of
      the current task's mappings. And use "-include_global_mapping" to enable
      or disable both the global and the task-specific mapping pair(s).
    . Use "-global" to enable or disable the global mappings.

Generating Verification
-----------------------
-generate_verification [-task task_name]

    Create a task that contains all the conditions, assumptions, and assertions
    to verify equivalence.

    Use "-task" to generate the properties to the specified task. If you do not
    specify a task, this command applies to the current task.

    NOTE: When you run this command, the tool also runs an interface check and
    caches the results. Refer to the definition for "check_hls
    -interface".
```

```
Infer Relations
---------------
-infer_relations
    [-valid_condition {valid_cond_expr}]
    [-input_signals {signal_list}]
    [-output_signals {signal_list}]

    Infer relations between signals on the spec and imp sides. The
    inferred relations are exposed as proven helper assertions that
    can be used later with "check_hls -prove -with_proven".
    . NOTE: Use this command only after "check_hls -prove" has failed to prove
      the design as relation inference can be computationally expensive.
    . Use "-valid_condition" to specify the logical context in which
      outputs are written. The valid condition must be a valid SVA
      expression, and it contains signals from both sides (spec and
      imp). Providing a valid condition helps the tool infer more
      precise relations by restricting analysis to relevant execution
      cycles.
    . Use "-input_signals" to restrict relation inference to input
      signals relevant to the intended core behavior. These inputs
      guide the exploration performed to infer relations. By default,
      the tool considers all unconstrained input signals.
    . Use "-output_signals" to restrict relation inference to output
      signals relevant to the intended core behavior. These outputs
      guide the exploration performed to infer relations. By default,
      the tool considers all output signals.

Prove
-----
-prove
    [ -task task_name
     |-property property_name_tcl_list [-regexp]
     | (-mapping_id {mapping_id_list}
     | -dict {mapping_dict_list} [-force]
     | ( -spec {spec_regexp_or_signal_list}
        |-imp {imp_regexp_or_signal_list})
       [-regexp [-verbose]])
           [-signal_type (  input | output | primary_input | primary_output
                         | dut_input | dut_output
                         | bbox_input | bbox_output | user_bbox_input
                         | user_bbox_output | auto_bbox_input | auto_bbox_output
                         | mpram_port | undriven | internal_undriven
                         | flop | latch | internal | stopat | reset_x
```

```
                        | x_assignment | x_default_assignment | x_divide_by_zero
                        | x_index_out_of_range | x_misc_undriven
                        | x_bus_contention | x_bus_floating)]
          [-mapping_type (  connection_all | connection_primary_input
                          | connection_bbox_output | connection_undriven
                          | connection_stopat | connection_misc
                          | ver_target_all | ver_target_primary_output
                          | ver_target_mpram_port | ver_target_bbox_input
                          | ver_target_misc | cutpoint_all | cutpoint_both
                          | cutpoint_spec | cutpoint_imp | cutpoint_both_init
                          | stopat_all | abstract_reset_value | abstract_counter |
abstract_init_value
                          | stopat_both | stopat_spec  | stopat_imp | stopat_both_env
                          | helper | init_user | init_auto | init_all) ]
          [  ( -instance {instance_full_path} )
              |( -of_signal {signal_list}
                [ -spec_regexp [-verbose]
                 |-imp_regexp [-verbose] ] )
          ]
          [-proof_status (  proven | cex | unprocessed | undetermined)+]
          [-tag tag_name_tcl_list]
    ]
    [-time_limit time_budget]
    [-failure_extension]
    [-bg]
    [-silent |-verbosity N]
    [-with_helpers]
    [-with_proven]

    Try to prove all HLSEC properties that were generated by HLSEC.
    Return values are the same as the "prove" command, while each proof
    strategy defines which properties are considered when calculating the
    return value.
    For more info, see return value section for each proof strategy.

    . HLSEC provable properties are:
        . HLSEC properties from the task-specific and global mapping pairs.
        . Imported properties and their related covers.
        . Assumptions' related covers.
    . Use "-task" to prove HLSEC properties in the specified task.
      If you do not specify a task, this command applies to the current task.
      In both cases, it runs on HLSEC provable properties.
    . Use "-property" to specify the names of the properties you want to prove.
```

These properties must be HLSEC provable properties. By default, the "check_hls -prove" subcommand attempts to prove all the HLSEC provable properties in the task.

. Use "-mapping_id" to generate properties only from mapping pairs of the specified IDs.
. Use "-dict" to prove properties only from mapping pairs of the specified mapping dictionaries. See the section on "Mapping Dictionary" for more info.
. Use "-spec" instead of IDs to define the set of signal name(s) on the spec side whose properties you want to generate.
. Use "-imp" instead of IDs to define the set of signal name(s) on the imp side whose properties you want to generate.
  . Use "-regexp" to specify that the "-spec" or "-imp" argument list is a regular expression list; otherwise, it is a list of specific names of signals, bits, or bit-vector expressions. Use the "-verbose" switch to print info messages detailing the regexp matching operation.
  . Use "-signal_type" to generate properties only for mappings of signals of the specified type. You can only use this switch with "-spec" or "-imp" and "-regexp". That is, if you specify mapping IDs, the tool generates properties for all types.
  . Use "-mapping_type" to prove only mapping pairs of the specified mapping type.
  . Use "-instance" to specify that you want to prove only mapping pairs of signals that are in the specified instance.
  . Use "-of_signal" to define the set of signal names whose mappings you want to prove.
    . Use "-spec_regexp" or "-imp_regexp" to specify that the "-of_signal" argument is a regular expression list on the spec or imp side. The default is spec side; otherwise, it is a list of specific names of signals, bits, or bit-vector expressions.
    . Use "-verbose" to print info messages detailing the regexp matching operations.
  . Use "-proof_status" to prove mapping entries according to their proof status.
    This switch accepts one or more of the following arguments: "proven", "cex", "unprocessed", "undetermined".
  . Use "-tag" to generate properties only from mappings with the specified tags.
  . A "clock_tree_node" is a signal automatically identified by the tool to be in the clock tree with counterpart of the same name in the clock tree on the other side.
. Use "-time_limit" to override the default two-hour "prove_time_limit" for "autoprove".

```
      Specify "time_budget" values as an integer followed by s, m, or h (no
      spaces).
      . s = seconds
      . m = minutes
      . h = hours
    . Use "-failure_extension" to run a dedicated bug-hunting strategy in
      parallel to the proof-oriented strategy. This bug-hunting strategy
      uses current failures to find deep bugs. It cannot prove that the
      designs are equivalent. You must use at least two licenses to run the two
      strategies in parallel.
    . Use "-bg" to run the command in the background and continue to accept and
      run new commands.
    . Use "-silent" for "-verbosity 0" (see below).
    . Use "-verbosity N" to override the current "prove_verbosity" for this
      "-prove" command. See "set_prove_verbosity" for a summary of the
      verbosity levels.
    . Use "-with_helpers" to prove with helper assertions.
    . Use "-with_proven" to keep already proven properties in the same task and
      use them as assumptions.

  NOTE:
    . You can run multiple HLSEC proofs in the
      background only for different tasks.
    . HLSEC can infer the status of a precondition from the status of its main
      property and vice-versa.
      . A counterexample on the main property means that the precondition is
        covered.
      . If a precondition is unreachable, the main property is vacuously proven.
      . If a new bound was found for a precondition, the main property's bound
        may be increased to match it.
      In case of inference, the indication "(Inferred)" appears
      immediately after the engine name, for example:
      "INFO (IMTR106): (0.basic) MITER basic strategy (B)(Inferred): A min_length
      bound of 252 was found for the property "prop" in 4.14 s."

Sign-Off Report
---------------
-signoff
    [-task task_name]
    [-file file_name [-json] [-force]]
    [-silent]
    [-summary]
    [-detailed]
```

```
    [-verbose]
    [-waive_category (  boundary_outputs_mapping | boundary_outputs_proof
                      | user_cutpoints | auto_cutpoints
                      | reset_value_abstraction | imported_assertions
                      | imported_covers | x_signals_and_undrivens
                      | mapped_stopats | assumptions_conflict
                      | assumptions_on_driven_signals
                      | assumptions_related_preconditions
                      | imp_assumptions)+]
      [[-waive_boundary_outputs_mapping {port_hierarchical_name_list}]
       [-waive_boundary_outputs_proof {port_name_list}]
       [-waive_user_cutpoints_proof {assertion_name_list}]
       [-waive_auto_cutpoints_proof {assertion_name_list}]
       [-waive_imported_assertion_proof {assertion_name_list}]
       [-waive_imported_cover_proof {cover_name_list}]
       [-waive_reset_value_abstraction {assert_name_list}]
       [-waive_x_signals_and_undrivens {signal_name_list}]
       [-waive_mapped_stopats {mapping_pair_list}]
       [-waive_assumption_on_driven_signals {assumption_name_list}]
       [-waive_assumption_related_preconditions {precondition_name_list}]
       [-waive_imp_assumptions {assumption_name_list}] [-regexp]]
      [-consider_bounded_proven_incomplete]
      [-get_valid_status [-summary] [-propagate_bbox_input_status_to_target] [-
consider_noConflict] [-no_cutpoint_status_propagation_to_target]]

    Write a report that summarizes the HLSEC proof status for a specific task.

    The sign-off report highlights the possible risks to the validity of the
    HLSEC proof. The risks are associated with a list of categories, where for
    every category, the report sets a grade.
    The grades are:
    . Complete: This category poses no risk to the HLSEC proof.
    . Complete (with waiver): The HLSEC proof is valid due to waiving data.
    . Waived: This category has been waived. (Though it might contain data
      results in a different status, these will be ignored.)
    . Incomplete: The HLSEC proof can be valid, but that depends on further
      action or user knowledge. For example, an HLSEC proof can be valid when you
      disable X-handling. However, the validity of the HLSEC proof depends on the
      assumption that Xs do not propagate to valid outputs in both spec and
      imp. You should check this assumption, possibly with the X-Prop App.
    . Invalid: The HLSEC proof is invalid, and you must take further actions.
    NOTE: The default output of the report includes the output provided when
    calling "check_hls -signoff -get_valid_status -summary", and results for
```

```
the targets are reported and taken into consideration in the "output
primary bounds" report.
. Use "-task" to generate a sign-off report for a specified task. By default,
  the tool generates a report for the current task.
. Use "-file" and a file name to specify the file you want to write the
  report to. By default, the tool generates a text file.
  . Use "-json" to write the report in JSON format.
  . Use "-force" to overwrite any existing file that has the specified name.
. Use "-silent" to get a dictionary containing reported data.
. Use "-summary" to print only the "Signoff Summary" section of the report,
  while returning a dictionary representing the "Signoff Summary" report.
  If you use "-get_valid_status", summary dictionary will contain
  valid status summary data, including the number of proven targets,
  number of CEX targets, number of undetermined, number of unprocessed,
  number of unreachable preconditions, number of undetermined
  preconditions, and number of unprocessed preconditions. In addition, if
  imported covers exist, the dictionary will include the following data:
  number of covered covers, unprocessed covers, undetermined covers,
  and unreachable covers.
. Use "-detailed" to get a detailed dictionary report that includes
  all categories with details. Properties and signals sub-categories are
  included.
. Use "-verbose" to get a dictionary report that includes
  HLSEC proof environment.
. Use "-waive_category boundary_outputs_mapping" to turn off the report for
  unmapped outputs and blackboxed inputs.
. Use "-waive_category boundary_outputs_proof" to turn off the report for
  outputs and blackboxed inputs.
. Use "-waive_category user_cutpoints" to turn off the report for user
  cutpoints.
. Use "-waive_category auto_cutpoints" to turn off the report for auto
  cutpoints.
. Use "-waive_category reset_value_abstraction" to turn off the report for
  reset value abstraction assert list.
. Use "-waive_category x_signals_and_undrivens" to turn off the report for
  undriven or X mapping sign-off failures.
  NOTE: Only use this switch for cases when the report makes the HLSEC proof
  incomplete or invalid.
. Use "-waive_category mapped_stopats" to turn off the report for mapped
  stopats.
. Use "-waive_category assumptions_on_driven_signals" to turn off the
  report showing assumptions on driven signals.
  NOTE: Assumptions on driven signals are assumptions that refer to signals
```

```
  that are not boundary inputs. These assumptions are considered risky.
  Assumptions on driven signals might impose overconstraint to the HLSEC
  model and provide false positive results.
  You must review all of these assumptions and decide whether including
  them is safe, and only then, use this waive check command.
  To get a list of the assumptions on driven signals, use
  "dict get [dict get [dict get [check_hls
  -signoff_info -get_category_status assumptions_on_driven_signals
  -detailed] "Assumption management"] "Assumptions on driven signals"]
  "assumptions"".
. Use "-waive_category assumptions_conflict" to turn off the report
  from "check_assumptions -conflict".
. Use "-waive_category assumptions_related_preconditions" to turn off the report
  for assumptions' related preconditions.
. Use "-waive_category imported_assertions" to turn off the report for
  imported assertions.
. Use "-waive_category imported_covers" to turn off the report for imported
  covers.
. Use "-waive_category imp_assumptions" to turn off the report for
  imp assumptions.
. Use "-waive_boundary_outputs_mapping" to turn off the report for all
  outputs specified in the port name list.
  NOTE: Output port name and blackboxed input should be specified as full
  hierarchy name.
. Use "-waive_boundary_outputs_proof" to turn off the report for all
  output properties specified in the list.
. Use "-waive_user_cutpoints_proof" to turn off the report for all
  user cutpoint assertions specified in the list.
. Use "-waive_auto_cutpoints_proof" to turn off the report for all
  automatically generated assertions produced due to state_matching
  specified in the list.
. Use "-waive_imported_assertion_proof" to turn off the report for all
  assertions specified in the list.
. Use "-waive_imported_cover_proof" to turn off the report for all
  imported covers specified in the list.
. Use "-waive_reset_value_abstraction" to turn off the report for all
  reset value abstraction properties specified in the list.
. Use "-waive_x_signals_and_undrivens" to turn off the report for all
  x signals specified in the list.
. Use "-waive_mapped_stopats" to turn off the report for all mapping pairs
  specified in the list.
  NOTE: Mapping pair format should follow the same format used in GUI for
  mapped stopat. You can drag and drop to console to get exact format.
```

. Use "-waive_assumption_on_driven_signals" to turn off the report for all assumptions specified in the list.
NOTE: You can waive properies only if the property is either an assumption with no preconditions or an assumption whose preconditions are covered by related covers.
. Use "-waive_assumption_related_preconditions" to turn off the report for all covers specified in the list.
. Use "-waive_imp_assumption" to turn off the report for all imp assumptions specified in the list.
. Use "-regexp" to specify that the arguments of "-waive_boundary_outputs_mapping", "-waive_boundary_outputs_proof", "-waive_user_cutpoints_proof", "-waive_auto_cutpoints_proof", "-waive_imported_assertion_proof", "-waive_imported_cover_proof", "-waive_reset_value_abstraction", "-waive_assumption_on_driven_signals", and "-waive_assumption_related_preconditions", represent regular expressions.
. Use "-consider_bounded_proven_incomplete" to treat any bounded proven target as incomplete instead of complete.
. Use "-get_valid_status" to get a dictionary containing verification targets (targets on boundary output signals and imported properties) and their valid status - in addition to the regular signoff result.
If you use this switch, the returned dictionary will include, for each user-defined verification target, a dictionary with the following keys:
{property name, engine, Time, mapping pair ID, valid status, bound }
where valid status is one of the following: {unprocessed, proven, cex, covered, unreachable-precondition, undetermined-precondition, unprocessed-precondition}. When the valid status is not proven or unreachable, the bound figure is added to the dictionary.
The bound item contains the min length of a possible trace, and in some cases, the max length figure too.
  . Use "-propagate_bbox_input_status_to_target" to propagate blackboxed input status to the boundary targets during sign-off reporting (valid-result calculation).
  . Use "-consider_noConflict" to include an evaluation of the bound of the :noConflict property when calculating the reported min length of the verification targets. That is, if the :noConflict min length is N, there will be no proven target and no target assertion with a bound greater than N.
  . Use "-no_cutpoint_status_propagation_to_target" to disable propagating user cutpoints status to the boundary targets during sign-off reporting (valid-result calculation).
NOTE: If an assertion property has a precondition property, the signoff report includes data about one of the properties only. See the following table:

```
      +--------------------------+------------------------------+--------------------
--+
      | Assertion Property Status | Precondition Property Status | Reported Property
|
      +--------------------------+------------------------------+--------------------
--+
      | Proven                   | Covered                      | Assertion property
|
      | Proven                   | Uncovered                    | Precondition
property |
      | Cex                      | Any                          | Assertion property
|
      | Undetermined             | Unreachable                  | Precondition
property |
      | Undetermined             | NOT Unreachable              | Assertion property
|
      | Unprocessed              | Unreachable                  | Precondition
property |
      | Unprocessed              | NOT Unreachable              | Assertion property
|
      +--------------------------+------------------------------+--------------------
--+

-signoff_info
    [-task task_name]
    [-file file_name [-json] [-force]]
    [-silent]
    [-detailed]
    ( -get_category_status (  boundary_outputs_mapping | boundary_outputs_proof
                            | imported_assertions | imported_covers
                            | reset_value_abstraction | x_signals_and_undrivens
                            | mapped_stopats | assumptions_conflict
                            | assumptions_on_driven_signals
                            | assumptions_related_preconditions
                            | imp_assumptions | user_cutpoints | auto_cutpoints)
     |-get_formal_environment (  clock | reset) )

    Get the specified sign-off info.

    . Use "-get_category_status" to get a Tcl dictionary representing the
      specified category data including category status and description.
    . Use "-get_formal_environment" to get the specified formal environment.
    . Use "-task" to generate a sign-off report for a specified task.
```

```
      By default, the tool generates a report for the current task.
    . Use "-file" and a file name to specify the file you want to write the
      report to. By default, the tool generates a text file.
      . Use "-force" to overwrite any existing file that has the specified name.
      . Use "-json" to write the report in JSON format.
    . Use "-silent" to get only the returned dictionary.
    . Use "-detailed" to get a detailed dictionary including all category data
      (including property and signal names).
```

Example

```
# Example for using HLSEC:
#-------------------------
check_hls -clear all
check_hls -compile -spec -top dut dut.cpp
check_hls -analyze -imp -sv dut.v
check_hls -elaborate -imp -top dut
check_hls -setup
clock clk
reset rst
check_hls -generate_verification
check_hls -prove
check_hls -signoff

# Example for pipeline command:
#-------------------------
# my_loop:
# while(true) {
#   // read inputs
#   my_expansion_point: {}
#   // write outputs
#   wait();
#   }
check_hls -pipeline -spec -loop_label my_loop -expand_label my_expansion_point

# Examples of mapping using regular expressions:
# ----------------------------------------------
check_hls -map -spec {inst1..*_reg} -regexp
check_hls -map -spec {inst1.sig1\[([0-9]+)\]} -imp {modelB_imp.inst1.sig1\[$1\]} -
regexp
check_hls -map -spec {specsig(_i|_o|_reg)\[0:2\]} -imp {modelB_imp.impsig$1\[2:4\]} -
regexp
```

```
# You can map one spec signal to multiple imp signals (or vice versa) by using
# the entire name of the spec signal as the regular expression and terminating
# it with the metacharacter "$". Example:

check_hls -map -spec {specsig_reg_0$} -imp {impsig_reg_[0-9]+} -regexp -helper

# NOTE: This operation will fail if attempting to create cutpoint mappings,
# reset value abstractions, or counter abstractions.

# Since a regular expression can match any substring within a signal name and
# does not necessarily have to match the entire signal name, make sure that you
# use start-of-string and end-of-string anchors when needed. For example,
# consider the following case:

# Spec Design              Imp Design
# -----------              ----------
# specSig                  top_imp.impSig
# specSig_2                top_imp.impSig_2

# The command:

check_hls -map -spec {specSig(_.*)?} -imp {top_imp.impSig$1} -regexp

# creates the following mappings:

# {specSig -> top_imp.impSig}
# {specSig -> top_imp.impSig_2}
# {specSig_2 -> top_imp.impSig_2}

# This is because after replacing $1 with the empty string, the regular
# expression top_imp.impSig matches both signals top_imp.impSig and
# top_imp.impSig_2.
# If you do not intend to create the mapping {specSig -> top_imp.impSig_2},
# use the following command (notice the anchor at the end of the imp side
# regular expression):

check_hls -map -spec {specSig(_.*)?} -imp {top_imp.impSig$1$} -regexp

# In the example above, consider 2 instances of instA_*: instA_4 and instA_5.
# The command maps signals found in, for example, instances {instA_4.instB} and
# {instA_5.instB} and maps them accordingly to the imp side using the matching
# instance numbers:
```

```
# {modelB_imp.instX_4.instY} and {modelB_imp.instX_5.instY}.
```

check_loop

Checks for signals that cause a combinational loop.

Syntax

```
check_loop (signal_name [-strict]) |(-property property_name)
              [-env |-task task_name]
              [-classify]
              [-include_clock]
              [-latch_data]
              [-silent]
              [-limit N]
              [ ([-merge] [-potentially_unstable])
               | [-compress_inner_loops]
               | [-functional]]
              [-without_cp]
              [-show_schematic]
              [-suggest_disables [-minimal_group]]


check_loop -global
           [-env |-task task_name]
           [-classify]
           [-include_clock]
           [-latch_data]
           [-silent]
           [-limit N]
           [ ([-merge] [-potentially_unstable])
            | [-compress_inner_loops]
            | [-functional]]
           [-without_cp]
           [-suggest_disables [-minimal_group]]


check_loop previously_used_args -next_loops

check_loop -formal ([glitch] [spurious_posedge])+
           [-task task_name]
           [-bg]
```

```
           [-force]
           [-suggest_break]

check_loop -formal (-show_schematic |-show_path)
           [-task task_name]

check_loop -breakpoints property_name

check_loop -viewer
```

Returns

Returns a list of loops, and each loop is a list of signals.

Detailed Description

Use this command to search for a combinational loop starting from the specified signal or all loops in the cone of influence (COI) of a property. If the tool does not find a loop, it returns an empty string; otherwise, it returns a list of signals that contribute to the loop. If you specify a property, the command returns a list of all loops within the property's COI.

This command is helpful when you see an error message that identifies the presence or possible presence of a combinational loop in the COI of a signal. Specify the signal name reported in the error message as the starting point for a search to identify signals that contribute to the loop. Remove the loop in the design or add a stopat.

To learn more about the combo loop flow, go to https://support.cadence.com and use the following search string:

Jasper Combo Loop Handling Flow

```
signal_name [-strict] |-property property_name
    [-env |-task task_name]
    [-classify]
    [-include_clock]
    [-latch_data]
    [-silent]
    [-limit N]
    [ ([-merge] [-potentially_unstable])
```

```
 | [-compress_inner_loops]
 | [-functional]]
[-without_cp]
[-show_schematic]
[-suggest_disables [-minimal_group]]
Search for combinational loops starting from the specified signal or
property.

. Use "-strict" with a signal name to only report a loop over a signal if
  it is part of the loop chain itself.

. Use "-property" to search for all combinational loops in the COI of the
  specified property.

. Use "-env" to specify the global environment instead of the current task.

. Use "-task" to specify a task other than the current task.

. Use "-classify" to get more information about loops. Each one of the
  classification categories has its own type in the returned typed list.

. Use "-include_clock" to include the clock pin of flops in the search for
  combinational loops.

. Use "-latch_data" to include the data pin of latches in the search for
  combinational loops.

  NOTE: Matching against latch patterns that cause loops to be functionally
  false requires the "loop_filter_latch_patterns" configuration to be on,
  which is the default setting. This matching does not happen when using
  "-compress_inner_loops".

. Use "-silent" to turn off the output to the screen and return information
  with a Tcl return value, which is helpful in Tcl scripts.

. Use "-limit N" to set the maximum number of combinational loops the
  command will look for. Loop detection stops when the tool finds N loops.

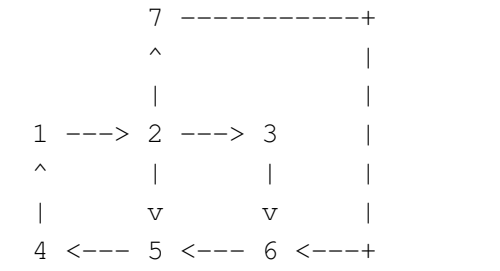
  NOTE: By default, "check_loop" finds up to 10 loops. To find all
  combinational loops, use "-limit $".

. Use "-merge" to collapse loops that comprise the same list of bit-blasted
  signals and use "-potentially_unstable" to filter out trivially stable
```

loops. This algorithm is pessimistic and might report stable loops as unstable.

. Use "-compress_inner_loops" to return a simplified representation of concentric loops.

Example:



In the case above, instead of reporting:

```
. 1 2 3 6 5 4
. 1 2 7 6 5 4
. 1 2 5 4
```

It will inline all loops and only report:

```
. 1 2 3 7 6 5 4
```

. Use "-functional" to return only the functionally true combinational loops, filtering out the loops that are only structural (that is, filtering out the false loops).

NOTE: The switch "-functional" requires previous declaration of a clock and a reset.

. Use "-without_cp" if you want to check for loops without the constant propagation step. The tool issues an error if the constant propagation step of loop checking detects an overconstraint.

. Use "-show_schematic" to run the Jasper Apps Schematic Viewer for the loops that were found in the COI of the specified signal.

. Use "-suggest_disables" to get suggestions for constraints that can functionally disable a loop, and use "-minimal_group" to return the minimal_group of constraints that disable the max number of loops.

Example:

```
assign y = ~b;
assign b = y & c;
assign c = scan_en;
```

```
      In the case above, the tool reports ~scan_en as a disable suggestion as
      it functionally disables the loop path between signals y and b.

-global [-env |-task task_name]
        [-classify]
        [-include_clock]
        [-latch_data]
        [-silent]
        [-limit N]
        [ ([-merge] [-potentially_unstable])
         | [-compress_inner_loops]
         | [-functional]]
        [-without_cp]
        [-suggest_disables [-minimal_group]]

    Search for combinational loops in the full design.

    . Use "-env" to specify the global environment instead of the current task.

    . Use "-task task_name" to specify a task other than the current task.

    . Use "-classify" to get more information about loops. Each one of the
      classification categories has its own type in the returned typed list.

    . Use "-include_clock" to include the clock pin of flops in the search for
      combinational loops.

    . Use "-latch_data" to include the data pin of latches in the search for
      combinational loops.

      NOTE: Matching against latch patterns that cause loops to be functionally
      false requires the "loop_filter_latch_patterns" configuration to be on,
      which is the default setting. This matching does not happen when using
      "-compress_inner_loops".

    . Use "-silent" to turn off the output to the screen and return information
      with a Tcl return value, which is helpful in Tcl scripts.

    . Use "-limit N" to set the maximum number of combinational loops the
      command will look for. Loop detection stops when the tool finds N loops.
      By default, "check_loop" finds up to 10 loops. To find all
      combinational loops, use "-limit $".
```

```
    . Use "-merge" to collapse loops that comprise the same list of bit-blasted
      signals and use "-potentially_unstable" to filter out trivially stable
      loops. This algorithm is pessimistic and might report stable loops as
      unstable.

    . Use "-compress_inner_loops" to return a simplified representation of
      concentric loops. For more information about this switch, see the example
      in the "-compress_inner_loops" definition above.

    . Use "-functional" to return only the functionally true combinational
      loops, filtering out the loops that are only structural (that is,
      filtering out the false loops).

      NOTE: The switch "-functional" requires previous declaration of a clock and
      a reset.

    . If the constant propagation step of loop checking detects an
      overconstraint, the tool issues an error. Use "-without_cp" if you want
      to check for loops without the constant propagation step.

    . Use "-suggest_disables" to get suggestions for constraints that can
      functionally disable a loop, and use "-minimal_group" to return the
      minimal_group of constraints that disable the max number of loops.

      Example:
          assign y = ~b;
          assign b = y & c;
          assign c = scan_en;
      In the case above, the tool reports ~scan_en as a disable suggestion as
      it functionally disables the loop path between signals y and b.

previously_used_args -next_loops
    Continue loop enumeration from the point where the last enumeration ended
    due to reaching the limit (default or specified with "-limit").

    NOTE: It is only possible to continue an execution if the command
    configuration remains the same. That is, you must explicitly specify the
    same arguments you used in the command you previously ran.

-formal ([glitch] [spurious_posedge])+
        [-task task_name]
        [-bg]
        [-force]
```

```
        [-suggest_break]

    Use this command to prove the presence or absence of functional loops
    through clock pins. When such loops are impossible or absent,
    "assume_no_loop" is safe. See "set_clock_loop_model assume_no_loop".
    Functional loops through clock pins can cause glitches or spurious
    posedges. Each effect is captured by a special property built into the task
    - :noGlitch and :noSpuriousPosedge respectively.
    When no loop property is specified, the command proves all of them.

    NOTE: "check_loop -formal" uses a specific engine mode and engine
    configuration that might be different from what you have specified.

    . Use "-task" to specify the task name where the proof should run.
      NOTE: If you do not use "-task", the command applies to the current task.

    . Use "-bg" to run the proof in the background and continue to accept and
      run new commands. See "prove" for operations available on a proof running
      in the background.

    . Use "-force" to clear existing proof results for the selected properties
      before beginning. See "prove -force".

    . Use "-suggest_break" to return a net where you can break a loop
      through the clock pin, removing a functional glitch.

-formal (-show_schematic |-show_path)
        [-task task_name]

    Use this command to display the schematic or the list of signals related to
    a loop property counterexample found by "check_loop -formal".

    . Use "-show_schematic" to draw a schematic of the loop contained within
      the loop property counterexample. The schematic has special annotations and
      colors to differentiate between nets with glitches and late edges.
    . Use "-show_path" to return a list of signals that are contained within
      the loop property counterexample.
    . Use "-task" to specify the task where the loop properties were evaluated.
      NOTE: If you do not use "-task", the command applies to the current task.

    NOTE: "-show_schematic" and "-show_path" support is limited to the :noGlitch
    property.
```

```
-breakpoints property_name
    Use this command to display all breakpoints associated with a specific
    property, including both visible signals and internal signal expressions
    where combo loop breakpoints were added during proof setup.

    Breakpoints are added per proof when using "set_loop_handling break".

    NOTE: This command returns all breakpoints added during the proof that
    processed this property. If the property was proved together with other
    properties that have combinational loops in their COI, all breakpoints
    from the proof are reported, not only those in the COI of the specified
    property.

    NOTE: This command requires that the property has been processed, as
    breakpoints are calculated during proof setup. If the property has not
    been processed, the tool issues error ECB006.

-viewer
    Launch the combo loop viewer.
```

Example

```
check_loop sig_A
check_loop -global
check_loop -global -classify
```

See Also

set_loop_filter_latch_patterns,
set_loop_filter_startpoints,
stopat

check_lpv

Loads, elaborates, and checks power-aware designs.

Syntax

```
check_lpv -init

check_lpv -load_upf pf_file
          [-allow_unescaped_bus_index]
          [-upf_version (1.0 | 2.0 | 2.1 | 3.0 | 3.1)]
          [-scope instance_path]

check_lpv -load_query_upf_file query_file
          [-scope instance_path]

check_lpv -generate_power_design
          [-invert_retention_event_semantics]
          [-corrupt_sequential_only]

check_lpv -create assert_all
          [-task task_name]
          [-power_domain power_domain_list]
          [-after N]

check_lpv -create (  assert_iso_up_all
                           | assert_iso_up_before
                           | assert_iso_up_during
                           | assert_iso_up_after)
          [-task task_name]
          [ -port port_name |-power_domain power_domain_list]
          [-after N]

check_lpv -create assert_iso_up_clk_stable
          [-task task_name]
          [-power_domain power_domain_list [-transitive]]
          [-after N]

check_lpv -create (assert_iso_supply | assert_iso_change_signal_stable)
          [-task task_name]
```

```
         [ -port port_name |-power_domain power_domain_list]
         [-after N]

check_lpv -create (  assert_supply_onehot
                           | assert_clock_stable_after_restore
                           | assert_ret_supply_on_save
                           | assert_ret_supply_on_restore
                           | assert_ret_supply_on_from_save_to_restore
                           | assert_ret_no_save_on_restore
                           | assert_pd_supply_change_clk_stable)
          [-task task_name]
          [-power_domain power_domain_list]
          [-after N]

check_lpv -create assert_rst_non_ret_flops
          [-precond combinational_expression]
          [-power_domain power_domain_list]
          [-after N]

check_lpv -create (  cover_power_switch_ctrl
                           | cover_iso_ctrl
                           | cover_ret_save_ctrl
                           | cover_ret_restore_ctrl
                           | cover_all)
          [-power_domain power_domain_list]
          [-after N]

check_lpv -create cover_pst_states

check_lpv -verify (  check_iso_clk_all [-use_declared_clocks] [-detailed]
                           | check_iso_clk_ctrl_data
                           | check_iso_clk_ctrl
                           | check_iso_outputs
                         )
          [-power_domain power_domain
           |-instance instance_name]
          [-report file_name [-force]]

check_lpv -verify check_iso_inputs
          [-power_domain power_domain
           |-instance instance_name]
          [-report file_name [-force]]
```

```
check_lpv -verify check_iso_default_value
          [-exceptions file_name]
          [-power_domain power_domain]
          [-report file_name [-csv] [-force]]

check_lpv -verify (  check_iso_connections
                           | check_ret_connections
                           | check_power_switch_connections
                          )
          [-power_domain [power_domain_list]
           |-instance instance_name]
          [-report file_name [-force]]

check_lpv -verify (  check_iso_ctrl_rst
                           | check_ret_ctrl_rst
                           | check_power_switch_ctrl_rst
                          )
          [-power_domain [power_domain_list]
           |-instance instance_name]
          [-report file_name [-force]]

check_lpv -verify check_supply_parallel
          [-power_domain [power_domain_list]
          [-report file_name [-force]]

check_lpv -verify check_iso_invalid_power_pin_connection
          [-power_domain [power_domain_list]
           |-instance instance_name]
          [-report file_name [-force]]

check_lpv -verify check_iso_source_and_destination_domains
          [-power_domain [power_domain_list]
           | -instance instance_name ]
          [-report file_name [-force]]

check_lpv -verify check_iso_output_dangling
          [-power_domain [power_domain_list]
           |-instance instance_name]
          [-report file_name [-force]]

check_lpv -verify check_iso_back_to_back
          [-power_domain [power_domain_list]
           |-instance instance_name]
```

```
          [-report file_name [-force]]

check_lpv -verify check_iso_ctrl_constant
          [-power_domain [power_domain_list]
           |-instance instance_name]
          [-report file_name [-force]]

check_lpv -verify check_iso_inputs_domains
          [-power_domain [power_domain_list]
           |-instance instance_name]
          [-report file_name [-force]]

check_lpv -verify check_ret_output_floating
          [-power_domain [power_domain_list]
           |-instance instance_name]
          [-report file_name [-force]]

check_lpv -verify check_ret_signal_tied_to_constant
          [-power_domain [power_domain_list]
           |-instance instance_name]
          [-report file_name [-force]]

check_lpv -verify check_ret_ctrl_signal_undriven
          [-power_domain [power_domain_list]
           |-instance instance_name]
          [-report file_name [-force]]

check_lpv -verify check_iso_cell_type
          -power_domain [power_domain_list]
           |-instance instance_name]
          [-report file_name [-force]]

check_lpv -verify check_iso_ctrl_signal_undriven
          [-power_domain [power_domain_list]
           |-instance instance_name]
          [-report file_name [-force]]

check_lpv -verify check_iso_input_tied_to_constant
          [-power_domain [power_domain_list]
           |-instance instance_name]
          [-report file_name [-force]]

check_lpv -verify check_iso_input_output_voltages
```

```
          [-power_domain [power_domain_list]
           |-instance instance_name]
          [-report file_name [-force]]

check_lpv -get_status

check_lpv -signal signal_name -list (  source_supply | sink_supply | source_domain
                                                                    | sink_domain)+

check_lpv -sec (-spec |-imp)
```

Returns

```
"-get_status" returns the following:
  . An error when no design is elaborated.
  . An empty string when a design is elaborated but has no power intent file
    loaded.
  . POWER_INTENT_LOADED when you have used "check_lpv -load_upf" to load a
    power format file.
  . POWER_AWARE when you have made the design power-aware with the command
    "check_lpv -generate_power_design".
```

Detailed Description

```
Use the Low Power Verification (LPV) App command "check_lpv" to load a Unified
Power Format (UPF) file, elaborate a power-aware design, and create and verify
properties, that is, structural checks, power reset checks and auto assertions.

-init
    Initialize settings for the LPV App.

    Use this command before running "analyze" if the design source uses types
    and functions defined in the UPF package.

-load_upf pf_file
        [-allow_unescaped_bus_index]
        [-upf_version (1.0 | 2.0 | 2.1 | 3.0 | 3.1)]
        [-scope instance_path]
    Load and parse the specified power format file.

    . Use "-allow_unescaped_bus_index" if your UPF does not use curly braces or
```

```
      escape signal names that contain square brackets.

      NOTE: "-allow_unescaped_bus_index" is an all-or-nothing switch. That is,
      the entire file must be unescaped or use no curly braces around bus
      indexes.

    . Use "-upf_version" to specify the UPF file version. The default is 2.1.

    . Use "-scope" to specify the instance where the UPF commands contained in
      the power format file are executed.

-load_query_upf_file query_file
        [-scope instance_path]
    Load and execute the specified file containing query commands.

    Use "-scope" to specify the instance where the commands contained in the
    query file are executed. If you do not specify this switch, the top
    instance is the scope.

-generate_power_design
        [-invert_retention_event_semantics]
        [-corrupt_sequential_only]
    Build a power model of the design.

    . Use "-invert_retention_event_semantics" to specify that you want the tool
      to consider high or low sensitivities declared in "set_retention_control"
      UPF commands as posedge or negedge, respectively.

      NOTE: The following UPF commands can create supply nets and ports inside
      excluded modules:
      . connect_supply_net
      . create_supply_net
      . create_supply_port

    . Use "-corrupt_sequential_only" to corrupt the output of sequential gates
      only. By default, "check_lpv -generate_power_design" corrupts the output of
      the gates connected to the boundary interface of the power domains and all
      sequential gates.

      . To enable corruption modeling for top ports, you must use the UPF
        command "set_port_attributes port_name -driver_supply supply_set".
        Only the ports you specify with this command will have corruption
        modeling enabled.
```

```
    NOTE:
    . You must use "check_lpv -load_*" before using this command.
    . This command invalidates any property not directly extracted from the
      design. Either prove or remove the unprocessed properties before
      running the command.
      For additional information, see the help for "prove", "assert", or
      "cover".

-create assert_all
        [-task task_name]
        [-power_domain power_domain_list]
        [-after N]
    Create all of the auto assertions, which are based on the design and the
    power intent.
    Refer to the assert_* definitions below.

    . Use "-task" to apply the command to the specified task instead of the
      <LowPower> task.
    . Use "-power_domain" to generate assertions for the specified power domains
      instead of all domains.
    . Use "-after" to delay the property check by the specified number of
      cycles.

    NOTE: This command generates assertions. You must manually run the proof of
    the assertions.

-create (  assert_iso_up_all
         | assert_iso_up_before
         | assert_iso_up_during
         | assert_iso_up_after)
        [-task task_name]
        [-port port_name |-power_domain power_domain_list]
        [-after N]
    Create the specified isolation cell auto assertions, which are based on the
    design and the power intent:
    . assert_iso_up_all - Creates auto assertions for the following:
            . assert_iso_up_before
            . assert_iso_up_during
            . assert_iso_up_after.
            (See the definitions below.)
```

```
    . assert_iso_up_before - Verifies that whenever a power domain of the
            signal driving the isolation cell is off and the power domain of
            the load of the isolation cell is on, the isolation conditions
            related to this isolation cell will be true strictly before the
            power is off.
    . assert_iso_up_during - Verifies that whenever a power domain of the
            signal driving the isolation cell is off and the power domain of
            the load of the isolation cell is on, the isolation conditions
            related to this isolation cell are true.
    . assert_iso_up_after - Verifies that whenever a power domain of the signal
            driving the isolation cell is off and the power domain of the load
            of the isolation cell is on, the isolation conditions related to
            this isolation cell are still true strictly after the power is
            turned on again.

    . Use "-task" to apply the command to the specified task instead of the
      <LowPower> task.
    . Use "-port" to specify  a port in the RTL that is isolated. This switch
      is useful if you want the assertions to be generated only for specific
      isolation ports.
    . Use "-power_domain" to generate assertions for the specified power domains
      instead of all domains.
    . Use "-after" to delay the property check by the specified number of
      cycles.

    NOTE:
    . "-port" and "-power_domain" are mutually exclusive switches.
    . This command generates assertions. You must manually run the proof of the
      assertions.

-create assert_iso_up_clk_stable
        [-task task_name]
        [-power_domain power_domain_list [-transitive]]
        [-after N]
    Create the auto assertion "assert_iso_up_clk_stable", which verifies that
    the clocks in the power domain are stable when the power domain is up and
    its isolations are enabled.

    With this command, the tool generates assertions for each clock of each
    power domain in the design. This command generates only one precondition
    property for each power domain.

    . Use "-task" to apply the command to the specified task instead of the
```

```
      <LowPower> task.
    . Use "-power_domain" to generate assertions for the specified power domains
      instead of all domains.
    . Use "-transitive" to recursively consider flops inside nested power
      domains without isolation rules in order to generate the clocks of the
      power domain specified by the "-power_domain" switch.
    . Use "-after" to delay the property check by the specified number of
      cycles.

    NOTE: This command generates assertions. You must manually run the proof of
    the assertions.

-create (assert_iso_supply | assert_iso_change_signal_stable)
        [-task task_name]
        [-port port_name |-power_domain power_domain_list]
        [-after N]
    Create the specified isolation cell auto assertions, which are based on the
    design and the power intent:
    . assert_iso_supply - Verifies that whenever a power domain of the signal
            driving the isolation cell is on and the power domain of the
            load of the isolation cell is on, the power supply of the
            isolation cell is on.
    . assert_iso_change_signal_stable - Verifies that an isolation control
            toggle does not generate a toggle between an isolation target and
            isolation output.

    . Use "-task" to apply the command to the specified task instead of the
      <LowPower> task.
    . Use "-port" to specify  a port in the RTL that is isolated. This switch
      is useful if you want the assertions to be generated only for specific
      isolation ports.
    . Use "-power_domain" to generate assertions for the specified power domains
      instead of all domains.
    . Use "-after" to delay the property check by the specified number of
      cycles.

    NOTE:
    . "-port" and "-power_domain" are mutually exclusive switches.
    . This command generates assertions. You must manually run the proof of the
      assertions.

-create (  assert_supply_onehot
         | assert_clock_stable_after_restore
```

```
        | assert_ret_supply_on_save
        | assert_ret_supply_on_restore
        | assert_ret_supply_on_from_save_to_restore
        | assert_ret_no_save_on_restore
        | assert_pd_supply_change_clk_stable)
      [-task task_name]
      [-power_domain power_domain_list]
      [-after N]
  Create the specified auto assertions, which are based on the design and the
  power intent:
  . assert_supply_onehot - Verifies that if a power supply net has the
          resolution semantic onehot, then there is never more than one
          active driver.
  . assert_clock_stable_after_restore - Verifies that there is no clock
          change in the last cycle of a restore.
  . assert_ret_supply_on_save - Verifies that whenever the value of an
          element is saved to the retention logic, the power supply of the
          corresponding retention logic is on.
  . assert_ret_supply_on_restore - Verifies that whenever the value of an
          element is restored from the retention logic, the power supply of
          the corresponding retention logic is on.
  . assert_ret_supply_on_from_save_to_restore - Verifies that the power
          supply of the retention logic is always on between save and
          restore.
  .  assert_ret_no_save_on_restore - Verifies that for each retention rule,
          the save and restore signals are not enabled at same time.
  .  assert_pd_supply_change_clk_stable - Verifies that if the power supply
          of a power domain is changing, then the clock is disabled.

  . Use "-task" to apply the command to the specified task instead of the
    <LowPower> task.
  . Use "-power_domain" to generate assertions for the specified power domains
    instead of all domains.
  . Use "-after" to delay the property check by the specified number of
    cycles.

  NOTE: This command generates assertions. You must manually run the proof of
  the assertions.

-create assert_rst_non_ret_flops
      [-precond combinational_expression]
      [-power_domain power_domain_list]
      [-after N]
```

```
    Create an auto assertion that checks whether the values of the
    non-retention flops are equal to the reset values if the precondition is
    true.
    . Use "-precond" to specify the precondition expression that must be true.
      If you do not use the "-precond" switch, the tool uses the restore signal
      of the retention rule as the precondition of the assertion.
    . Use "-power_domain" to generate the assertion for the specified power
      domains instead of all domains.
    . Use "-after" to delay the property check by the specified number of
      cycles.

-create (  cover_power_switch_ctrl
         | cover_iso_ctrl
         | cover_ret_save_ctrl
         | cover_ret_restore_ctrl
         | cover_all)
        [-power_domain power_domain_list] [-after N]
    Create the specified cover properties to check control signals for power
    element toggles:
    . cover_power_switch_ctrl - Checks whether control signals for power
            switches can toggle.
    . cover_iso_ctrl - Checks whether control signals for isolations can
            toggle.
    . cover_ret_save_ctrl - Checks whether save control signals for retentions
            can toggle.
    . cover_ret_restore_ctrl - Checks whether restore control signals for
            retentions can toggle.
    . cover_all - Creates all of the cover properties listed above.

    . Use "-power_domain" to generate cover properties for the specified power
      domains instead of all domains.
    . Use "-after" to delay the property check by the specified number of
      cycles.

-create cover_pst_states
    Create cover properties for each state specified by "add_pst_state".

-verify (  check_iso_clk_all [-use_declared_clocks] [-detailed]
         | check_iso_clk_ctrl_data
         | check_iso_clk_ctrl
         | check_iso_outputs )
        [-power_domain [power_domain_list] | -instance instance_name]
        [-report file_name [-force]]
```

```
    Run the specified structural checks.
    These checks target rules that should be followed when defining the power
    configurations. The tool provides a textual report showing violations to
    the rules.
    . Use "check_iso_clk_all" to verify that all input clocks of a power domain
      are isolated with an isolation cell.
      . Use "-use_declared_clocks" to show only instance inputs that are
        directly connected to declared clocks. By default, the violation report
        shows all instance inputs that lead to the clock/enable logic inside
        the instance.
      . Use "-detailed" to get a verbose textual report that includes, beyond
        all violations, a list of all instance ports that passed the check.
    . Use "check_iso_clk_ctrl_data" to verify that for each isolated signal,
      the clocks of the isolation control signal and the isolated signal are
      derived from the same primary clock.
      NOTE: The tool warns "The clocks of the isolation element and the
      isolation control overlap but are not the same" if the two lists have
      some declared clocks in common but also have some different declared
      clocks.
    . Use "check_iso_clk_ctrl" to verify that for each isolated signal that
      isolates a clock signal, the isolation control is derived from the same
      primary clock as the isolated clock.
    . Use "check_iso_outputs" to verify whether the outputs of the boundary
      instances of each power domain are isolated (OK), not isolated (ERROR),
      or partially isolated (PARTIALLY OK).
      NOTE: The tool ignores supply ports in this check.
    . Use "-instance" to run the specified checks on the specified
      instance.
    . Use "-power_domain" to run the check for the specified power domains. If
      no power domains are provided, the tool runs it for all domains.
    . Use "-report" with any of the structural checks to save the report to
      the specified file and use "-force" if the specified file already exists,
      and you want to overwrite it.

    NOTE: "-instance" and "-power_domain" are mutually exclusive switches.

-verify check_iso_inputs
        [-power_domain [power_domain_list] | -instance instance_name]
        [-report file_name [-force]]
    Run the "check_iso_inputs" structural check.

    This check verifies whether the inputs of the boundary instances of each
    power domain are isolated (OK), not isolated (ERROR), or partially isolated
```

```
    (PARTIALLY OK).
    . Use "-power_domain" to run the check for the specified power domains. If
      no power domains are provided, the tool runs it for all domains.
    . Use "-report" to save the report to the specified file and use "-force"
      if the specified file already exists, and you want to overwrite it.

    NOTE: "-instance" and "-power_domain" are mutually exclusive switches.

-verify check_iso_default_value
        [-exceptions file_name]
        [-power_domain [power_domain_list]]
        [-report file_name [-csv] [-force]]
    Run the "check_iso_default_value" power reset check.

    This check verifies that the clamp value of an isolation cell coincides
    with the reset value of the isolated signal and returns a Tcl list with the
    exact signals bits that failed.

    Implementation guidance:
    . Use this command after "check_lpv -load_upf".
    . You can use the "check_iso_default_value" check as a power reset
      check before the design is power aware. This usage does not model
      corruption of isolation cells; however, using the check in this way
      avoids requirements such as power sequence or constraints on the power
      signals. If you run "check_iso_default_value" after power transformation,
      you must provide constraints and/or a valid power sequence to generate
      realistic results.

    . Use "-exceptions" to specify isolation values that are different from the
      reset value. The format of the specified file is as follows:
      (<signal_name> value [1 | 0] | <signal_name> cycle <natural_number>)
      . <signal_name> value [1 | 0]: Use this format to verify that the
        specified signal has the specified clamp value instead of the value
        from the last reset cycle.
      . <signal_name> cycle <natural_number>: Use this format to
        verify that the specified signal has the clamp value from a specified
        reset cycle instead of the last reset cycle.
      Refer to the examples below.
    . Use "-power_domain" to run the check for the specified power domains. If
      no power domains are provided, the tool runs it for all domains.
    . Use "-report" to save the report to the specified file.
      . Use "-csv" to export the report in CSV format for use with spreadsheet
        applications.
```

```
      . Use "-force" if the specified file already exists, and you want to
        overwrite it.

-verify (  check_iso_connections
         | check_ret_connections
         | check_power_switch_connections)
        [-power_domain [power_domain_list] | -instance instance_name]
        [-report file_name [-force]]
    Run the specified structural checks.
    These checks target inputs and outputs of power elements to ensure that
    they are not empty.

    . Use "check_iso_connections" to verify that the power net, the isolation
      signal, and the isolation elements of an isolation rule are not empty.
    . Use "check_ret_connections" to verify that the power net, the save and
      restore signals, and the retention elements of a retention rule are not
      empty.
    . Use "check_power_switch_connections" to verify that the output supply
      net, the input supply nets, and the control signal of a power switch are
      not empty.
    . Use "-power_domain" to run the check for the specified power domains. If
      no power domains are provided, the tool runs it for all domains.
    . Use "-report" to save the report to the specified file and use "-force"
      if the specified file already exists, and you want to overwrite it.

    NOTE: "-instance" and "-power_domain" are mutually exclusive switches.

-verify (  check_iso_ctrl_rst
         | check_ret_ctrl_rst
         | check_power_switch_ctrl_rst)
        [-power_domain [power_domain_list] | -instance instance_name]
        [-report file_name [-force]]
    Run the specified power reset checks.
    These checks verify that control of power elements are not X in the last
    cycle of reset.

    . Use "check_iso_ctrl_rst" to verify that the power net and the isolation
      signal of an isolation rule are not X in the last cycle of reset.
    . Use "check_ret_ctrl_rst" to verify that the power net and save and
      restore signals of a retention rule are not X in the last cycle of
      reset.
    . Use "check_power_switch_ctrl_rst" to verify that the output supply net,
      input supply nets, and control signal of a power switch are not X in the
```

```
      last cycle of reset.
    . Use "-power_domain" to run the check for the specified power domains. If
      no power domains are provided, the tool runs it for all domains.
    . Use "-report" to save the report to the specified file and use "-force"
      if the specified file already exists, and you want to overwrite it.

    NOTE: "-instance" and "-power_domain" are mutually exclusive switches.

-verify check_supply_parallel
        [-power_domain [power_domain_list]]
        [-report file_name [-force]]
    Verify that all sources of a parallel supply net share the same supply
    driver.

    To run this check, the tool traverses backward through the design from
    the parallel supply net to confirm that the same root supply driver exists
    for all possible drivers of the parallel supply net.
    . Use "-power_domain" to run the check for the specified power domains. If
      no power domains are provided, the tool runs it for all domains.
    . Use "-report" to save the report to the specified file and use "-force"
      if the specified file already exists, and you want to overwrite it.

-verify check_iso_invalid_power_pin_connection
        [-power_domain [power_domain_list] | -instance instance_name]
        [-report file_name [-force]]
    Verify whether the isolation supply is connected to a supply port or a
    power switch.

    . Use "-power_domain" to run the check for the specified power domains. If
      no power domains are provided, the tool runs it for all domains.
    . Use "-report" to save the report to the specified file and use "-force"
      if the specified file already exists, and you want to overwrite it.

    NOTE: "-instance" and "-power_domain" are mutually exclusive switches.

-verify check_iso_source_and_destination_domains
        [-power_domain [power_domain_list] | -instance instance_name]
        [-report file_name [-force]]
    Verify whether the source and destination power domains are the same in a
    path through the isolation cell.

    . Use "-instance" to run the check on the specified instance.
    . Use "-power_domain" to run the check for the specified power domains. If
```

```
      no power domains are provided, the tool runs it for all domains.
    . Use "-report" to save the report to the specified file and use "-force"
      if the specified file already exists, and you want to overwrite it.

    NOTE: "-instance" and "-power_domain" are mutually exclusive switches.

-verify check_iso_output_dangling
        [-power_domain [power_domain_list] | -instance instance_name]
        [-report file_name [-force]]
    Verify whether an isolation cell output is dangling.

    . Use "-instance" to run the check on the specified instance.
    . Use "-power_domain" to run the check for the specified power domains. If
      no power domains are provided, the tool runs it for all domains.
    . Use "-report" to save the report to the specified file and use "-force"
      if the specified file already exists, and you want to overwrite it.

    NOTE: "-instance" and "-power_domain" are mutually exclusive switches.

-verify check_iso_back_to_back
        [-power_domain [power_domain_list] | -instance instance_name]
        [-report file_name [-force]]
    Verify whether back-to-back isolation cells exist.

    An output pin of an isolation cell drives only one input, which is an
    input pin of another isolation cell.

    . Use "-power_domain" to run the check for the specified power domain
      instead of running it for all domains.
    . Use "-report" to save the report to the specified file and use "-force"
      if the specified file already exists, and you want to overwrite it.

    NOTE: "-instance" and "-power_domain" are mutually exclusive switches.

-verify check_iso_ctrl_constant
        [-instance instance_name |-power_domain [power_domain_list]]
        [-report file_name [-force]]
    Verify whether the isolation control pin of the isolation cell instance is
    connected to a constant value.

    . Use "-instance" to run the check on the specified instance.
    . Use "-power_domain" to run the check for the specified power domains. If
      no power domains are provided, the tool runs it for all domains.
```

```
    . Use "-report" to save the report to the specified file and use "-force"
      if the specified file already exists, and you want to overwrite it.

    NOTE: "-instance" and "-power_domain" are mutually exclusive switches.

-verify check_iso_inputs_domains
        [-power_domain [power_domain_list] | -instance instance_name]
        [-report file_name [-force]]

    Verify if the isolation cell inputs have different power domains.
    Some isolation cells have a dedicated power pin (switchable_power)
    that is powered on during the isolation. If those isolation cells are
    placed in the same domain of their driving domain (such as, type FROM)
    and the switchable_power pin is different than the power pin that powers
    the isolation data pin, it can cause a power leakage in the isolation
    cell when the power of data input is off and switchable_power is on.
    To prevent this issue, connect the switchable_power pin of the isolation
    cell to the same power net that drives the data input pin of the
    isolation cell.

    . Use "-instance" to run the check on the specified instance.
    . Use "-power_domain" to run the check for the specified power domains. If
      no power domains are provided, the tool runs it for all domains.
    . Use "-report" to save the report to the specified file and use "-force"
      if the specified file already exists, and you want to overwrite it.

    NOTE: "-instance" and "-power_domain" are mutually exclusive switches.

-verify check_ret_output_floating
        [-power_domain [power_domain_list] | -instance instance_name]
        [-report file_name [-force]]

    Verify if the retention cell output is floating.

    . Use "-instance" to run the check on the specified instance.
    . Use "-power_domain" to run the check for the specified power domains. If
      no power domains are provided, the tool runs it for all domains.
    . Use "-report" to save the report to the specified file and use "-force"
      if the specified file already exists, and you want to overwrite it.

    NOTE: "-instance" and "-power_domain" are mutually exclusive switches.

-verify check_ret_signal_tied_to_constant
```

```
        [-power_domain [power_domain_list] | -instance instance_name]
        [-report file_name [-force]]

    Verify if the retention control pin of a retention instance is
    tied to a constant value, either directly or through a chain of
    buffers or inverters. In such cases, the retention functionality
    is either always enabled or always disabled.
    To correct the issue, connect the retention control signal to a power
    controller module, which should control when retention is activated.

    . Use "-instance" to run the check on the specified instance.
    . Use "-power_domain" to run the check for the specified power domains. If
      no power domains are provided, the tool runs it for all domains.
    . Use "-report" to save the report to the specified file and use "-force"
      if the specified file already exists, and you want to overwrite it.

    NOTE: "-instance" and "-power_domain" are mutually exclusive switches.

-verify check_ret_ctrl_signal_undriven
        [-power_domain [power_domain_list] | -instance instance_name]
        [-report file_name [-force]]

    Verify if control signal specified by retention strategy is undriven.

    . Use "-instance" to run the check on the specified instance.
    . Use "-power_domain" to run the check for the specified power domains. If
      no power domains are provided, the tool runs it for all domains.
    . Use "-report" to save the report to the specified file and use "-force"
      if the specified file already exists, and you want to overwrite it.

    NOTE: "-instance" and "-power_domain" are mutually exclusive switches.

-verify check_iso_cell_type
        [-power_domain [power_domain_list] | -instance instance_name]
        [-report file_name [-force]]

    Indicates that a constant data input is isolated to an opposite value.
    If the constant net is isolated to an incorrect value, it might result
    in incorrect operation of the destination domain when the domain driving
    the constant is OFF, and isolation is enabled.

    . Use "-instance" to run the check on the specified instance.
    . Use "-power_domain" to run the check for the specified power domains. If
```

```
      no power domains are provided, the tool runs it for all domains.
    . Use "-report" to save the report to the specified file and use "-force"
      if the specified file already exists, and you want to overwrite it.

    NOTE: "-instance" and "-power_domain" are mutually exclusive switches.

-verify check_iso_ctrl_signal_undriven
        [-power_domain [power_domain_list] | -instance instance_name]
        [-report file_name [-force]]

    Verify whether the control signal specified by isolation strategy is
    undriven.

    . Use "-instance" to run the check on the specified instance.
    . Use "-power_domain" to run the check for the specified power domains. If
      no power domains are provided, the tool runs it for all domains.
    . Use "-report" to save the report to the specified file and use "-force"
      if the specified file already exists, and you want to overwrite it.

    NOTE: "-instance" and "-power_domain" are mutually exclusive switches.

-verify check_iso_input_tied_to_constant
        [-power_domain [power_domain_list] | -instance instance_name]
        [-report file_name [-force]]

    Indicates that the input data pin of an isolation instance is connected
    to a constant signal (includes local tied and propagated constant), a tied
    cell, or a supply net.

    . Use "-instance" to run the check on the specified instance.
    . Use "-power_domain" to run the check for the specified power domains. If
      no power domains are provided, the tool runs it for all domains.
    . Use "-report" to save the report to the specified file and use "-force"
      if the specified file already exists, and you want to overwrite it.

    NOTE: "-instance" and "-power_domain" are mutually exclusive switches.

-verify check_iso_input_output_voltages
        [-power_domain [power_domain_list] | -instance instance_name]
        [-report file_name [-force]]

    Verify whether the isolation instance input domain and output domain have
    different voltages.
```

```
    You cannot use a cell defined as an isolation cell with two power pins when
    the power for both is ON and at different voltages. This check indicates if
    a power mode has both an input domain and output domain ON and at different
    nominal conditions.

    . Use "-power_domain" to run the check for the specified power domains. If
      no power domains are provided, the tool runs it for all domains.

    NOTE: "-instance" and "-power_domain" are mutually exclusive switches.

-get_status
    Check whether the loaded design is power-aware.
    This command returns the following:
    . An error when no design is elaborated.
    . An empty string when a design is elaborated but has no power intent file
      loaded.
    . POWER_INTENT_LOADED when you have used "check_lpv -load_pf" to load a UPF
      file.
    . POWER_AWARE when you have made the design power-aware with the command
      "check_lpv -generate_power_design".

-signal signal_name -list (  source_supply | sink_supply | source_domain
                           | sink_domain)+
    List the specified power information about a specified signal.

    NOTE: Specify one or more enums with this "-list" command.

-sec (-spec |-imp)
    Populate the LPV App GUI (Power Domain Info Browser and Design Hierarchy
    pane) with content for the specified SEC side (spec or imp).

    Refer to command documentation for "check_sec -spec_upf_file" and
    "check_sec -imp_upf_file".
```

Example

```
check_lpv -create assert_all
check_lpv -generate_power_design

# Exceptions File Example
# -----------------------
check_lpv -verify check_iso_default_value -exceptions my_exceptions

# my_exceptions contains the following lines:
sig1 cycle 10
sig2 value 1

# In this example, the tool compares the clamp value for sig1 against the reset
# value from 10 cycles before the end of the reset. In addition, it compares
# the clamp value for sig2 against 1.
```

check_one_of

Stops command interpretation if actual is not an element in ok_list.

Syntax

```
check_one_of actual ok_tcl_list [on_error]
```

Returns

No return values.

Detailed Description

Use this command to check whether an actual value matches the expectations. If it does not, "check_one_of" returns with an error code and stops command interpretation unless you also use "catch". The tool evaluates the optional on_error block before returning with an error code.

This command is useful when creating a regression script.

actual
    Actual value.

ok_tcl_list
    List of values that is expected to include actual.

    This argument can be a Tcl list.

on_error
    A command to evaluate when actual is not an element in ok_list.

Example

```
check_one_of [visualize -get_value x 1] {2'b01 2'b0x}
```

See Also

check_code

check_reset

Assists structural and functional verification of reset.

Syntax

```
Running Reset Analysis
----------------------
check_reset -run

Exclude Registers from Reset Convergence Analysis
-------------------------------------------------
check_reset -no_convergence
              (-add signal_list
               |-remove signal_pattern_list [-regexp |-exact]
               |-clear
               |-list)
```

Returns

"check_reset -run" return the following:

. completed : Indicates that reset analysis reached the end without stopping for reset verification.

. error : Indicates that an error occurred during reset analysis.

Detailed Description

```
Use this command to set up an advanced reset verification and configuration
environment and explicitly run reset analysis.

NOTE:
. Running "check_reset -run" invalidates Visualize traces and proof results.
. The tool uses reset verification or configuration changes specified with
  "check_reset" during any reset analysis called by other commands (for example,
  "prove" or "get_reset_info"). Use "set_reset_extended_mode" to turn it off.

Running Reset Analysis
----------------------
-run
    Run reset analysis with the "check_reset" verification and configuration setup.

Ignore Registers from Reset Convergence Analysis
------------------------------------------------
-no_convergence (-add signal_list
                 |-remove tag_list [-regexp |-exact]
                 |-clear
                 |-list)

    Specify flops and latches that you want the tool to ignore during
    reset analysis that requires convergence evaluation, which means
    that convergence checks will not evaluate stability for them.

    . Use "-add" to specify a list of registers that will be ignored during
      convergence checks.

      NOTE: The tool supports "-no_convergence" when you specify reset with
      "-expression". It does not support it when you specify reset with
      "-sequence" unless you also specify "trace_show_reset false".

    . Use "-remove" to delete a list of previously added non-convergent
      registers, and use "-regexp" to support regular expressions or "-exact"
      for literal matching.

    . Use "-clear" to remove all registers from the list of those that were
      previously added with "-no_convergence -add".

    . Use "-list" to list registers omitted from convergence checks.
```

Example

```
check_reset -no_convergence -add {flop1 flop2}
```

See Also

get_reset_info,
reset,
set_reset_extended_mode

check_rvv

Checks compliance of the RTL design with the RISC-V specification.

Associated App(s)

Jasper RISC-V Verification

Syntax

```
Initializing RVV
-------------------
check_rvv -init

RVV Configure ISA
-------------------
check_rvv -configure_isa [isa_string] [-mode (Machine | User)]

RVV Configure Spec
-------------------
check_rvv -configure_spec (-import json_file_name
                  |-detect_csrs json_file_name)

RVV Generate
-------------------
check_rvv -generate

RVV Setup
-------------------
check_rvv -setup [-enable_csr_equivalence] [-disable_csr_address_constraints]

RVV Clear
-------------------
check_rvv -clear

RVV Prove
-------------------
check_rvv -prove ( -task [task_name_tcl_list]
                   |-property property_name_tcl_list [-regexp])
                  [-bg]

RVV Debug
-------------------
check_rvv -debug (-property property_name)

RVV Report
-------------------
check_rvv -report [-property property_name |-window window_name]
```

Detailed Description

```
Use this command to configure, generate, setup, prove and debug in the RISC-V
Verification App.

Initializing RVV
------------------
-init
    Initialize the settings required for RVV App.

RVV Configure ISA
------------------
-configure_isa [isa_string] [-mode (Machine | User)]
    Configure the module supported RISC-V extensions. "isa_string" is written
    based on the RISC-V naming convention; the default is "RV32I". Extensions
    must be separated by "_".

    Use "-mode" to specify the privilege level. The default is "Machine"
    mode.

    Supported Extensions:
    . I      - Integer "I"
    . E      - Embedded "E"
    . M      - Integer Multiplication and Division "M"
    . A      - Atomic "A" (includes Zaamo, Zalrsc)
    . C      - Compressed "C" (includes Zca)
    . B      - Bit Manipulation (includes Zba, Zbb, Zbc, Zbs)
    . ZCA    - Code Size Reduction "Zca"
    . ZCB    - Code Size Reduction "Zcb"
    . ZICSR  - Control and Status Register (CSR) Instructions "Zicsr"
    . ZMMUL  - Integer Multiplication "Zmmul"
    . ZAAMO  - Atomic Memory Operations "Zaamo"
    . ZALRSC - Load-Reserved/Store-Conditional Instructions "Zalrsc"
    . ZBA    - Address generation instructions "Zba"
    . ZBB    - Basic bit-manipulation "Zbb"
    . ZBC    - Carry-less multiplication "Zbc"
    . ZBS    - Single-bit instructions "Zbs"
    . ZKN    - NIST Algorithm Suite "Zkn" (includes Zbkb, Zbkc, Zbkx, Zknd, Zkne, Zknh)
    . ZBKB   - Bitmanip instructions for Cryptography "Zbkb"
    . ZBKC   - Carry-less multiply instructions "Zbkc"
    . ZBKX   - Crossbar permutation instructions "Zbkx"
    . ZKND   - NIST Suite: AES Decryption "Zknd"
    . ZKNE   - NIST Suite: AES Encryption "Zkne"
    . ZKNH   - NIST Suite: Hash Function Instructions "Zknh"
```

```
    . ZICOND - Integer Conditional Operations "Zicond"
    . ZIFENCEI - Instruction-Fetch Fence "Zifencei"
    . ZIMOP  - May-Be-Operations "Zimop"

RVV Configure Spec
------------------
-configure_spec (-import json_file_name |-detect_csrs json_file_name)
    Import the module configuration file. The file must have initial state and
    input data signals.

    -import json_file_name
        Import an existing module configuration file.

    -detect_csrs json_file_name
        Automatically detect Control and Status Registers (CSRs) from the
        design. The tool analyzes the RTL to identify CSR registers. The output
        configuration file with detected CSRs is written to the session logs
        directory using the original file name with a "_detected" suffix.

RVV Generate
------------------
-generate
    Generate RVV module, create I/O data lines, and prepare the module to be
    set up.

RVV Setup
------------------
-setup [-enable_csr_equivalence] [-disable_csr_address_constraints]
    Set up the RVV formal environment and extract RVV properties.

    -enable_csr_equivalence
        Enable the creation of equivalence assertions that verify if CSR values
        in the RTL match the expected values from the spec model. These
        assertions are generated for both read-write (WR) and read-only (RO)
        CSRs.

    -disable_csr_address_constraints
        Disable the automatic generation of CSR address constraints.
        By default, the tool constrains CSR addresses to valid CSRs defined in
        the configuration. Use this switch to permit unconstrained CSR
        addresses.

RVV Clear
```

```
-------------------
-clear
    Clear all RVV settings in the current session.

RVV Prove
-------------------
-prove ( -task [task_name_tcl_list]
        |-property property_name_tcl_list [-regexp])
       [-bg]
    Prove all RVV assertions and cover properties in the current task.

    -task [task_name_tcl_list]
        Prove all RVV assertions and cover properties in the specified tasks.

        NOTE:
        . If you do not supply a task name, the tool proves all assertions and
          covers in the current task.
        . When specifying a list of task names, enclose the list in curly
          braces.

    -property property_name_tcl_list [-regexp]
        Prove the specified RVV properties.

        NOTE:
        . To prove properties not in the current task, prepend the property
          names with "<task>::".
        . All listed properties must belong to the same task.
        . Any non-RVV property is ignored.
        . This switch supports wildcards and regular expressions, for example,
          "check_rvv -prove -property {expression}".

    -bg
        Run the "check_rvv -prove" command in the background and continue to
        accept and run new commands.

RVV Debug
-------------------
-debug (-property property_name)
    Open the RVV debugging utility in Visualize. Use "-property" to specify a
    property name.

    NOTE: The property must be an RVV property.
```

```
RVV Report
-------------------
-report [-property property_name |-window window_name]
    Generate the RVV assembly report associated with a debug trace. The
    report is returned as the Tcl result of the command.

    The source of the trace is selected by one of the following mutually
    exclusive modes:

    -property property_name
        Generate the report for the specified RVV property.

    -window window_name
        Generate the report from the trace currently displayed in the
        specified Visualize window.

    No switches
        Generate the report from the trace in the currently active Visualize
        window.

    NOTE: The property associated with the trace must be an RVV property.
```

Example

```
# Example for using RVV:
#-------------------------
check_rvv -clear
check_rvv -init
clock clk
reset rst
check_rvv -configure_isa RV32I_M_C_ZiCSR -mode USER
check_rvv -configure_spec -import configure.json
check_rvv -generate
check_rvv -setup
check_rvv -prove -bg

# Example for using RVV with CSR equivalence checking:
#-------------------------
check_rvv -clear
check_rvv -init
clock clk
reset rst
check_rvv -configure_isa RV32I_M_C_ZiCSR -mode USER
check_rvv -configure_spec -import configure.json
check_rvv -generate
check_rvv -setup -enable_csr_equivalence
check_rvv -prove -bg

# Example for using RVV with CSR auto-detection:
#-------------------------
check_rvv -clear
check_rvv -init
clock clk
reset rst
check_rvv -configure_isa RV32I_M_C_ZiCSR -mode USER
check_rvv -configure_spec -detect_csrs configure.json
check_rvv -configure_spec -import $session_#/rvv/configure_detected.json
check_rvv -generate
check_rvv -setup
check_rvv -prove -bg

# Example for using RVV Debug Utility:
#-------------------------
check_rvv -debug -property rvv_property_name
```

check_sec

Compares two RTL designs, the specification and the implementation.

Syntax

```
Setup
-----
Split Setup
-----------

check_sec -compile_context (spec | imp | both | fpv)

check_sec -analyze (-spec |-imp |-both)
    analyze_switches_and_arguments

check_sec -ipk -both
    ipk_switches_and_arguments

check_sec -elaborate (-spec |-imp [-spec_dut spec_instance] |-both)
    elaborate_switches_and_arguments

check_sec -setup
          [-spec_dut spec_dut_instance_name_list]
          [-imp_dut imp_dut_instance_name_list]
          [-no_auto_mapping]
          [-exclude_module_auto_mapping {regexp_or_module_list}
              [-module_regexp]]
          [-exclude_instance_auto_mapping {regexp_or_instance_list}
              [-instance_regexp]]
          [-exclude_undriven_mapping]
          [-hierarchical]

check_sec -setup -clear

HSEC Reports
------------
check_sec -hierarchical -report (  failed_run
                                                  | verification_result
                                                  | verification_result_tree) [-
```

```
silent]

check_sec -set_dut_instance
         -spec spec_dut_instance_name_list -imp imp_dut_instance_name_list

Clear
-----
check_sec -clear
         (all | mapping | (result [-task task_name]))

Interface Check
---------------
check_sec -interface
        [-mapped |-unmapped]
        [-spec |-imp]
        [-signal_type (  primary_input | primary_output
                                | dut_input | dut_output
                                | bbox_input | bbox_output | user_bbox_input
                                | user_bbox_output | auto_bbox_input | auto_bbox_output
                                | undriven | internal_undriven
                                | setup_stopat | reset_x | x_assignment |
x_default_assignment
                                | x_divide_by_zero | x_index_out_of_range |
x_misc_undriven
                                | x_bus_contention | x_bus_floating
                                | user_env_stopat | user_task_stopat)]
        [-imp_assumptions [-assumption_type (none | assumptions_embedded)]]
        [-instance instance_full_path]
        [-task task_name]
        [-ignore_user_stopats]
        [-ignore_imp_assumptions]

Signals List
------------
check_sec -list
       [ [-signal]
           [-signal_type (  input | output | primary_input | primary_output
                                  | dut_input | dut_output
                                  | bbox_input | bbox_output | user_bbox_input
                                  | user_bbox_output | auto_bbox_input |
auto_bbox_output
                                  | mpram_port | undriven | internal_undriven
                                  | flop | latch | internal | stopat | reset_x
```

```
                                    | x_assignment | x_default_assignment |
x_divide_by_zero
                                    | x_index_out_of_range | x_misc_undriven
                                    | x_bus_contention | x_bus_floating | gated_clock)]
           [-spec |-imp]
           [(-coi_of |-fanout_of) signal_list]
           [ -instance instance_full_path
            |-name {regexp} [-verbose]
       ]

Hierarchy Mapping Rule List
---------------------------
check_sec -list -mapping -hierarchy
          [-return (text | id)]

Signal Mapping List
-------------------
check_sec -list -mapping
           [-mapping_type (  connection_all | connection_primary_input
                                | connection_bbox_output | connection_undriven
                                | connection_stopat | connection_misc
                                | ver_target_all | ver_target_primary_output
                                | ver_target_mpram_port | ver_target_bbox_input
                                | ver_target_misc | cutpoint_all | cutpoint_both
                                | cutpoint_spec | cutpoint_imp | cutpoint_both_init
                                | stopat_all | abstract_reset_value | abstract_counter
| abstract_init_value
                                | stopat_both | stopat_spec  | stopat_imp |
stopat_both_env
                                | helper | init_user | init_auto | init_all |
clock_gating_toggle_coverage) ]
           [-signal_type (  input | output | primary_input | primary_output
                                    | dut_input | dut_output
                                    | bbox_input | bbox_output | user_bbox_input
                                    | user_bbox_output | auto_bbox_input |
auto_bbox_output
                                    | mpram_port | undriven | internal_undriven
                                    | flop | latch | internal | stopat | reset_x
                                    | x_assignment | x_default_assignment |
x_divide_by_zero
                                    | x_index_out_of_range | x_misc_undriven
                                    | x_bus_contention | x_bus_floating | gated_clock)]
           [  (-instance instance_full_path)
```

```
            |(-exclude_signal signal_list
            |(-of_signal {regexp_or_signal_list}
               [ -spec_regexp [-verbose]
                |-imp_regexp [-verbose] ] )
          ]
          [-tag {tag_name}]
          [([-task task_name] [-include_global_mapping])
           |-global
          ]
          [-return (text | id | dict)]
          [-proof_status (proven | cex | unprocessed | undetermined)+]
          [-proof_task proof_task_name]
          [ (-coi_of signal_list [-get_first_level_coi])
           |(-fanout_of signal_list [-get_first_level_fanout])]
          [-status (enabled | disabled)]

Violating Mapping List
----------------------
check_sec -list -mapping
         -violating_mapping(  all | clock_signal | init_diff
                                  | init_mapping_ranges_diff
                                  | init_mapping_with_context
                                  | init_mapping_with_stopat
                                  | assumption_comb_coi
                                  | comb_loop_comb_coi
                                  | assumption_and_comb_loop_comb_coi
                                  | context_both_edge
                                  | dscp_on_constant
                                  | fast_rated_signal)
        [-mapping_type (  connection_all | connection_primary_input
                        | connection_bbox_output | connection_undriven
                        | connection_stopat | connection_misc
                        | ver_target_all | ver_target_primary_output
                        | ver_target_mpram_port | ver_target_bbox_input
                        | ver_target_misc | cutpoint_all | cutpoint_both
                        | cutpoint_spec | cutpoint_imp | cutpoint_both_init
                        | stopat_all | abstract_reset_value | abstract_counter |
abstract_init_value
                        | stopat_both | stopat_spec  | stopat_imp |
stopat_both_env
                        | helper | init_user | init_auto | init_all |
clock_gating_toggle_coverage) ]
         [-return (text | id | dict)]
```

```
         [-detailed]

Disabled Mapping List - Fractional SEC Mode
-------------------------------------------
check_sec -list -mapping -disabled_out_of_target_coi
           [-return (text | id)]

Related Verification
--------------------
check_sec -related_verification
        (  -property property_list
          |-id mapping_id_list
          |-dict mapping_dict_list
        )
        [-task task_name]

Mapping Info
------------
check_sec -mapping_info
        (    -id mapping_id_list
          |  -dict mapping_dict_list [-force]
          | (  -spec {spec_regexp_or_signal_list}
               |-imp {imp_regexp_or_signal_list})
        )
        [-regexp [-verbose]]
        [-signal_type (  input | output | primary_input | primary_output
                               | dut_input | dut_output
                               | bbox_input | bbox_output | user_bbox_input
                               | user_bbox_output | auto_bbox_input | auto_bbox_output
        | mpram_port | undriven | internal_undriven
                               | flop | latch | internal | stopat | reset_x
                               | x_assignment | x_default_assignment |
x_divide_by_zero
                               | x_index_out_of_range | x_misc_undriven
                               | x_bus_contention | x_bus_floating | gated_clock)])
        [-mapping_type (  connection_all | connection_primary_input
                              | connection_bbox_output | connection_undriven
                              | connection_stopat | connection_misc
                              | ver_target_all | ver_target_primary_output
                              | ver_target_mpram_port | ver_target_bbox_input
                              | ver_target_misc | cutpoint_all | cutpoint_both
                              | cutpoint_spec | cutpoint_imp | cutpoint_both_init
                              | stopat_all | abstract_reset_value | abstract_counter |
```

```
abstract_init_value
                              | stopat_both | stopat_spec  | stopat_imp
|stopat_both_env
                              | helper | init_user | init_auto | init_all |
clock_gating_toggle_coverage) ]
        [  (-instance instance_full_path)
          |(-of_signal {signal_list}
              [ -spec_regexp [-verbose]
               |-imp_regexp [-verbose] ] )
        ]
        [-proof_status (proven | cex | unprocessed | undetermined)+]
        [-tag {tag_name}]
        [([-task task_name] [-include_global_mapping])
         |-global
        ]
        [-list (spec_signal | imp_signal | context | source | id
                       | sva_expression | property | is_enabled
                       | is_inverse | type | task_name | tag | status
                       | target_bound)]

Hierarchy Mapping Rule Info
---------------------------
check_sec -mapping_info -hierarchy
          -id hierarchy_aliasing_rule_id_list

Signal Mapping
--------------
check_sec -map
        ([-auto]
           [-spec {spec_instance_list}
           [-imp {imp_instance_list}]]
           [-signal_type (  input | output | primary_input | primary_output
                                 | dut_input | dut_output
                                 | bbox_input | bbox_output | user_bbox_input
                                 | user_bbox_output | auto_bbox_input |
auto_bbox_output
                                 | mpram_port | undriven | internal_undriven
                                 | flop | latch | internal | stopat | reset_x
                                 | x_assignment | x_default_assignment |
x_divide_by_zero
                                 | x_index_out_of_range | x_misc_undriven
                                 | x_bus_contention | x_bus_floating)]
           [-coi_of signal_list]
```

```
    [-set_value (0 | 1)]
    [-max_word_size max_word_size_value]
    [-return (text | id | dict | none)]
    [-spec_delay spec_delay_value]
    [-imp_delay imp_delay_value]
    [-map_condition {map_cond_expr}]
    [-clock {clocking_event}]
    [-disable {disable_cond_expr}]
    [-edge (posedge | negedge | both_edge)]
    [ -cut (both | both_init | spec | imp)
     |-abstract (reset_value | counter | init_value)
     |-stopat (both | spec | imp | both_env)
     |-helper]
    [-tag {tag_name}]
    [-task task_name |-global]
    [-exclude_signal {regexp_or_signal_list}
         [-signal_regexp]]
    [-respect_connections_during_reset]
    [-respect_abstraction_during_reset]
    [-exclude_module {regexp_or_module_list}
                 [-module_regexp]]
    [-exclude_instance {regexp_or_instance_list}
                 [-instance_regexp]]
    [-mapping_type ( connection_primary_input
                           | connection_bbox_output
                           | connection_undriven | connection_stopat
                           | connection_misc
                           | ver_target_primary_output
                           | ver_target_mpram_port
                           | ver_target_bbox_input | ver_target_misc
                           | cutpoint_both | cutpoint_spec
                           | cutpoint_imp | cutpoint_both_init
                           | abstract_reset_value | abstract_counter
                           | abstract_init_value | stopat_both
                           | stopat_spec  | stopat_imp
                           | stopat_both_env | helper | init_user
                           | init_auto) ]
  )
| ( (-spec {spec_regexp_or_signal_list}
       [-imp {imp_regexp_or_signal_list}])
     [-regexp
       [-verbose]
       [-exclude_signal {regexp_or_signal_list}
```

```
            [-signal_regexp]]]
        [-max_word_size max_word_size_value]
        [-inverse]
        [-return (text | id | dict | none)]
        [-spec_delay spec_delay_value]
        [-imp_delay imp_delay_value]
        [-map_condition {map_cond_expr}]
        [-clock {clocking_event}]
        [-disable {disable_cond_expr}]
        [-edge (posedge | negedge | both_edge)]
        [ -cut (both | both_init | spec | imp)
         |-abstract (reset_value | counter | init_value)
         |-stopat (both | spec | imp | both_env)
         |-helper
         |-constraint]
        [-tag {tag_name}]
        [-task task_name |-global]
        [-init]
        [-respect_connections_during_reset]
        [-respect_abstraction_during_reset]
        [-mapping_type ( connection_primary_input
                               | connection_bbox_output
                               | connection_undriven | connection_stopat
                               | connection_misc
                               | ver_target_primary_output
                               | ver_target_mpram_port
                               | ver_target_bbox_input | ver_target_misc
                               | cutpoint_both | cutpoint_spec
                               | cutpoint_imp | cutpoint_both_init
                               | abstract_reset_value | abstract_counter
                               | abstract_init_value | stopat_both
                               | stopat_spec  | stopat_imp
                               | stopat_both_env | helper | init_user
                               | init_auto
                               | clock_gating_toggle_coverage) ]
    )
  | (-hierarchy
       -spec {spec_regexp_or_hierarchy_list}
       -imp  {imp_regexp_or_hierarchy_list}
       [-regexp]
    )
  |-map_file map_file_name
```

```
Signal Unmapping
----------------
check_sec -unmap
         ( -id mapping_id_list
          |-dict mapping_dict_list
          |-spec {spec_regexp_or_signal_list}
          |-imp {imp_regexp_or_signal_list}
         )
          [-regexp [-verbose]]
          [-mapping_type ( connection_all | connection_primary_input
                         | connection_bbox_output | connection_undriven
                         | connection_stopat | connection_misc
                         | ver_target_all | ver_target_primary_output
                         | ver_target_mpram_port | ver_target_bbox_input
                         | ver_target_misc | cutpoint_all | cutpoint_both
                         | cutpoint_spec | cutpoint_imp | cutpoint_both_init
                         | stopat_all | abstract_reset_value | abstract_counter
| abstract_init_value
                         | stopat_both | stopat_spec  | stopat_imp |
stopat_both_env
                         | helper | init_user | init_auto | init_all |
clock_gating_toggle_coverage) ]
          [  (-instance instance_full_path)
            |(-of_signal {signal_list}
                [ -spec_regexp [-verbose]
                 |-imp_regexp [-verbose] ] )
          ]
          [-proof_status (proven | cex | unprocessed | undetermined)+]
          [-signal_type (  input | output | primary_input | primary_output
                         | dut_input | dut_output
                         | bbox_input | bbox_output | user_bbox_input
                         | user_bbox_output | auto_bbox_input |
auto_bbox_output
                         | mpram_port | undriven | internal_undriven
                         | flop | latch | internal | stopat | reset_x
                         | x_assignment | x_default_assignment |
x_divide_by_zero
                         | x_index_out_of_range | x_misc_undriven
                         | x_bus_contention | x_bus_floating | gated_clock)]
          [-tag {tag_name}]
          [(-task task_name [-include_global_mapping])
           |-global
          ]
```

```
Hierarchy Unmapping
-------------------
check_sec -unmap -hierarchy -id hierarchy_aliasing_rule_id_list

AutoProve Unmapping
-------------------
check_sec -unmap -autoprove_mappings [-task task_name]

Controlling Automatic Init Mapping
----------------------------------
check_sec -auto_map_reset_x_values (on | off)
          [-map_one_side_reset_x (on | off)]
          [-task task_name]
          [-exclude_module {regexp_or_module_list}
         [-module_regexp]]
          [-exclude_instance {regexp_or_instance_list}
           [-instance_regexp]]

Controlling Automatic Clock Gating Toggle Coverage Mapping
----------------------------------
check_sec -auto_map_clock_gating_toggle_coverage (on | off)
          [-task task_name]

Ungating Clocks
---------------
check_sec -ungate [-task task_name |-env]
       (-auto [-spec |-imp |-both]
           | gated_clock_tcl_list
           |-remove gated_clock_tcl_list
           |-list [-status] [-silent]
           |-clear )

Splitting Mapping
-----------------
check_sec -split_mapping
              -id mapping_id_list
              -max_word_size max_word_size_value

Copying Mapping
---------------
check_sec -copy_mapping
              (-source_task source_task |-id mapping_id_list)
```

```
              [-target_task target_task]

Copying Imported Properties
---------------------------
check_sec -copy_imported_properties
              (-source_task source_task |-property property_name_tcl_list [-regexp])
              [-target_task target_task]

Setting Mapping Context
-----------------------
check_sec -set_context_of_mapping
        (   -id mapping_id_list
         |  -dict mapping_dict_list [-force]
         | ( -spec {spec_regexp_or_signal_list}
            |-imp {imp_regexp_or_signal_list})
               [-regexp [-verbose]]
               [-mapping_type (  connection_all | connection_primary_input
                                 | connection_bbox_output | connection_undriven
                                 | connection_stopat | connection_misc
                                 | ver_target_all | ver_target_primary_output
                                 | ver_target_mpram_port | ver_target_bbox_input
                                 | ver_target_misc | cutpoint_all | cutpoint_both
                                 | cutpoint_spec | cutpoint_imp | cutpoint_both_init
                                 | stopat_all | abstract_reset_value |
abstract_counter | abstract_init_value
                                 | stopat_both | stopat_spec  | stopat_imp |
stopat_both_env
                                 | helper | init_user | init_auto | init_all |
clock_gating_toggle_coverage) ]
               [  (-instance instance_full_path)
                 |(-of_signal {signal_list}
                    [ -spec_regexp [-verbose]
                     |-imp_regexp [-verbose] ] )
               ]
               [-proof_status (proven | cex | unprocessed | undetermined)+]
               [-signal_type (  input | output | primary_input | primary_output
                                | dut_input | dut_output
                                | bbox_input | bbox_output | user_bbox_input
                                | user_bbox_output | auto_bbox_input
                                | auto_bbox_output | mpram_port | undriven |
internal_undriven
                                | flop | latch | internal | stopat | reset_x
                                | x_assignment | x_default_assignment |
```

```
x_divide_by_zero
                                        | x_index_out_of_range | x_misc_undriven
                                        | x_bus_contention | x_bus_floating |
gated_clock)]
               [-tag {tag_name}]
               [([-task task_name] [-include_global_mapping])
                |-global
               ]
        |-set_default
        |-clear_default
        )
        [-spec_delay spec_delay_value]
        [-imp_delay imp_delay_value]
        [-map_condition {map_cond_expr}]
        [-clock {clocking_event}]
        [-disable {disable_cond_expr}]
        [-edge (posedge | negedge | both_edge)]

Setting Mapping Type
--------------------
check_sec -set_mapping_type -id mapping_id_list
                                | -dict mapping_dict_list [-force]
        [ -cut (both | both_init | spec | imp)
         |-stopat (both | spec | imp | both_env)
         |-abstract (reset_value | counter | init_value)
         |-helper
         |-cover_gated_clock_toggle]
        [-set_tag tag_name]

Enabling and Disabling Mapping
------------------------------------
check_sec (-enable_mapping |-disable_mapping)
        (  -id {mapping_id_list}
         | -dict {mapping_dict_list}
         | ( -spec {spec_regexp_or_signal_list}
            |-imp {imp_regexp_or_signal_list})
            [-regexp [-verbose]]
            [-mapping_type (  connection_all | connection_primary_input
                            | connection_bbox_output | connection_undriven
                            | connection_stopat | connection_misc
                            | ver_target_all | ver_target_primary_output
                            | ver_target_mpram_port | ver_target_bbox_input
                            | ver_target_misc | cutpoint_all | cutpoint_both
```

```
                                  | cutpoint_spec | cutpoint_imp | cutpoint_both_init
                                  | stopat_all | abstract_reset_value | abstract_counter
| abstract_init_value
                                  | stopat_both | stopat_spec  | stopat_imp |
stopat_both_env
                                  | helper | init_user | init_auto | init_all |
clock_gating_toggle_coverage) ]
            [  (-instance instance_full_path)
              |(-of_signal {signal_list}
                [ -spec_regexp [-verbose]
                 |-imp_regexp [-verbose] ] )
            ]
            [-proof_status (proven | cex | unprocessed | undetermined)+]
            [-signal_type (  input | output | primary_input | primary_output
                                  | dut_input | dut_output
                                  | bbox_input | bbox_output | user_bbox_input
                                  | user_bbox_output | auto_bbox_input |
auto_bbox_output
                                  | mpram_port | undriven | internal_undriven
                                  | flop | latch | internal | stopat | reset_x
                                  | x_assignment | x_default_assignment |
x_divide_by_zero
                                  | x_index_out_of_range | x_misc_undriven
                                  | x_bus_contention | x_bus_floating | gated_clock)]
            [-tag {tag_name}]
            [([-task task_name] [-include_global_mapping])
             |-global
            ]
        )

Waiving Signals
---------------
check_sec -waive
        ( -waive_signals {signals list} [-exclude_init_mapping]
                                                          [-only_init_mapping]
                                                          [-description description]
         |-unwaive_signals {signals list}
         |-list_waived [-of_description description] [-verbose] )

Get Twin Signal
---------------
check_sec -get_twin_of_signal
  signal_list [-regexp]
```

```
Undriven
--------
check_sec -set_auto_mapped_undriven_value (0 | 1)

Prove
-----
check_sec -prove
        [-task task_name
         |-property property_name_tcl_list [-regexp]
         |(-mapping_id {mapping_id_list}
         |-dict {mapping_dict_list} [-force]
         | ( -spec {spec_regexp_or_signal_list}
            |-imp {imp_regexp_or_signal_list})
           [-regexp [-verbose]])
           ([-signal_type (  input | output | primary_input | primary_output
                                  | dut_input | dut_output
                                  | bbox_input | bbox_output | user_bbox_input
                                  | user_bbox_output | auto_bbox_input |
auto_bbox_output
                                  | mpram_port | undriven | internal_undriven
                                  | flop | latch | internal | stopat | reset_x
                                  | x_assignment | x_default_assignment |
x_divide_by_zero
                                  | x_index_out_of_range | x_misc_undriven
                                  | x_bus_contention | x_bus_floating | gated_clock)]
          [-mapping_type (  connection_all | connection_primary_input
                                  | connection_bbox_output | connection_undriven
                                  | connection_stopat | connection_misc
                                  | ver_target_all | ver_target_primary_output
                                  | ver_target_mpram_port | ver_target_bbox_input
                                  | ver_target_misc | cutpoint_all | cutpoint_both
                                  | cutpoint_spec | cutpoint_imp | cutpoint_both_init
                                  | stopat_all | abstract_reset_value |
abstract_counter | abstract_init_value
                                  | stopat_both | stopat_spec  | stopat_imp |
stopat_both_env
                                  | helper | init_user | init_auto | init_all |
clock_gating_toggle_coverage) ]
            [-include_global_mapping])
           [  (-instance instance_full_path)
             |(-of_signal {signal_list}
                 [ -spec_regexp [-verbose]
```

```
                |-imp_regexp [-verbose] ] )
          ]
          [-proof_status (proven | cex | unprocessed | undetermined)+]
          [-tag tag_name_tcl_list]
        ]
        [-time_limit time_budget]
        [-cex_limit number_of_failures]
        [-failure_extension]
        [-bg]
        [-silent |-verbosity N]
        [-with_helpers]
        [-with_proven]

check_sec -prove -status [-thread thread_id]


No Auto Prove
-------------
check_sec -prove -no_auto
        [-suppress_traces]

Advanced Prove Strategy - Basic
-------------------------------
check_sec -prove -strategy basic
        [-failure_extension]

Advanced Prove Strategy - Proof
-------------------------------
check_sec -prove -strategy proof
        [-property property_name_tcl_list [-regexp]]

Advanced Prove Strategy - State Matching
----------------------------------------
check_sec -prove -strategy state_matching
        [-property property_name_tcl_list [-regexp]]

Advanced Prove Strategy - Clock Gating
--------------------------------------
check_sec -prove -strategy clock_gating
        [-property property_name_tcl_list [-regexp]]

Sign-Off Report
---------------
```

```
check_sec -signoff
        [-task task_name]
        [-file file_name [-json] [-force]]
        [-silent]
        [-summary]
        [-detailed]
        [-waive_category (  boundary_outputs_mapping | boundary_outputs_proof
                          | user_cutpoints | auto_cutpoints
                          | reset_value_abstraction | imported_assertions
                          | imported_covers | x_signals_and_undrivens
                          | mapped_stopats | assumptions_conflict
                          | assumptions_on_driven_signals
                          | assumptions_related_preconditions
                          | clock_gating | imp_assumptions
                          | loop_handling)+ ]
        [[-waive_boundary_outputs_mapping {port_hierarchical_name_list}]
         [-waive_boundary_outputs_proof {port_name_list}]
         [-waive_user_cutpoints_proof {assertion_name_list}]
         [-waive_auto_cutpoints_proof {assertion_name_list}]
         [-waive_imported_assertion_proof {assertion_name_list}]
         [-waive_imported_cover_proof {cover_name_list}]
         [-waive_reset_value_abstraction {assert_name_list}]
         [-waive_x_signals_and_undrivens {signal_name_list}]
         [-waive_mapped_stopats {mapping_pair_list}]
         [-waive_assumption_on_driven_signals {assumption_name_list}]
         [-waive_assumption_related_preconditions {precondition_name_list}]
         [-waive_imp_assumptions {assumption_name_list}]
         [-waive_unmapped_gated_clocks {signal_name_list}]
         [-waive_mapped_gated_clocks {cover_name_list}] [-regexp]]
        [-consider_bounded_proven_incomplete]
        [-get_valid_status
          [-summary] [-propagate_bbox_input_status_to_target] [-consider_noConflict] [-
no_cutpoint_status_propagation_to_target]]

check_sec -signoff_info
        [-task task_name]
        [-file file_name [-json] [-force]]
        [-silent]
        [-detailed]
        -get_category_status (  boundary_outputs_mapping | boundary_outputs_proof
                              | imported_assertions | imported_covers
                              | reset_value_abstraction |
x_signals_and_undrivens
```

```
                                        | mapped_stopats | assumptions_conflict
                                        | assumptions_on_driven_signals
                                        | assumptions_related_preconditions
                                        | clock_gating | imp_assumptions |
user_cutpoints
                                        | auto_cutpoints | loop_handling)


Proof Utility
-------------
check_sec -get_needed_constraints -property property_name
        [-task task_name]
        [-attempts number_of_prove_attempts]
        [-attempts_time_limit time_limit]
        [-include_cutpoint_mappings]
        [-include_init_mappings]
        [-include_mappings_of_undriven_signals]
        [-include_primary_input_mappings]
        [-include_stopat_mappings]
        [-include_user_env_assumptions]
        [-include_user_liveness_assumptions]
        [-required_assumptions assumptions_tcl_list]
        [-silent]
        [-status target_status_tcl_list]
        [-stop_status stop_status_tcl_list]
        [-user_assumptions assumptions_tcl_list]

Utility Switches
----------------
check_sec (-get_spec_top_instance |-get_imp_top_instance)

check_sec -import_property property_name_tcl_list [-regexp]
```

Returns

Various, context sensitive return values.

Detailed Description

The Jasper Sequential Equivalence Checking (SEC) App formally compares two RTL designs, the specification (spec) and implementation (imp). SEC analyzes

```
and elaborates the two designs in a fashion that is similar to other Jasper
apps, but in addition, it links (connects) them and creates one or more tasks
for comparing pairs of signals from both models.

. In the following cases, SEC shows proof results on the targets that are
  valid only if all the SEC abstractions in the task are valid, that is, if all
  the abstraction's assertions are proven.
  . -prove -no_auto

. For "-prove -strategy state_matching",
  every cutpoint is an SEC target, and all results are considered valid as we
  intend to prove the combinational cones' equivalence for the two designs.

. In the following cases, SEC shows valid proof results on the verification
  targets (boundary output targets and imported properties). These valid
  results are calculated by taking into consideration the abstraction's
  assertion results.
  . -prove -strategy basic
  . -prove -strategy proof
  . -prove -strategy clock_gating

Setup
-----
NOTE:
    . The spec side is analyzed in the main design compilation slot.
    . The imp side is analyzed in the internal repository slot 2.
      This slot cannot be used in "analyze" or "elaborate" commands, but you
      can refer to it to configure compilation (see "compilation -config").
    . If you use "-both", the tool analyzes both sides in the main design
      compilation slot.
    . For more information about repositories see the "analyze -repository" command.

Split Setup
-----------
-compile_context (spec | imp | both | fpv)
    Use the specified compilation context.

    . Specify "spec", "imp", or "both", if you want all subsequent FPV
      "analyze" and "elaborate" commands to automatically translate to SEC
      "analyze" and "elaborate" commands.
    . Low Power commands for UPF load and transformation are SEC context aware.
      Run "check_lpv -load_upf" to load an UPF file on current context and then
      run "check_lpv -generate_power_design" to transform it.
```

```
    . Running "restore -elaborated_design dir_or_file" after specifying "spec",
      "imp", or "both" restores the design for the SEC spec side, imp side, or
      both sides respectively.
      NOTE:
      . A directory specified with the "restore" command must contain one
        elaborated design only.
      . The provided elaborated design must be elaborated with SEC commands.
    . Specify "fpv" if you want to delete the previous translation and run
      all FPV "analyze" and "elaborate" commands as expected in FPV.

-analyze (-spec |-imp |-both) analyze_switches_and_arguments
    Analyze the HDL or property files intended for verification of either spec
    side, imp side, or both sides.
    You can use all the "analyze" switches with this command
    except "-save", "-restore", "-req", and "-repository".
    For more details, run "help analyze -gui".

-ipk -both ipk_switches_and_arguments
    Analyze, for spec and imp, an Intelligent Proof Kit (IPK) for use with
    formal verification, to extract proof kit files for use with simulation, to
    load wrapper files, or to load a customized debugging table.
    You can use all the "ipk" switches with this command except "-req" and
    "-repository". For more details, run "help ipk -gui".

    NOTE: Use "-ipk" before "-elaborate".

-elaborate (-spec |-imp [-spec_dut spec_instance] |-both)
elaborate_switches_and_arguments

    Synthesize and read the netlist for the Verilog(R) and VHDL files that have
    been analyzed for spec or imp using the "check_sec -analyze" command.
    You can use all the "elaborate" switches with this command except
    "-load_cpi", "-req", and "-repository".
    For more details, run "help elaborate -gui".
    Use the "-spec_dut" switch with a valid instance name to copy
    all parameters from "spec_instance" on the spec side and pass
    them to the top instance on the imp side.

    NOTE:
    . The commands "check_sec -elaborate -log" and "check_sec -elaborate
      -list" do not support "-spec", "-imp", or "-both".
    . You must run "check_sec -elaborate -imp" after "check_sec -elaborate
      -spec".
```

```
    . You cannot run "check_sec -elaborate" on a non-analyzed design. Instead,
      do one of the following:
      . Use "check_sec -analyze -both" with "check_sec -elaborate -both".
      . Use "check_sec -analyze -spec" and "check_sec -elaborate -spec" with
        "check_sec -analyze -imp" and "check_sec -elaborate -imp".
    . To re-elaborate, you must first run "check_sec -setup -clear".
    . When specifying elaborate_switches_and_arguments for the imp side, do not
      include the imp top prefix.
    . You must elaborate the spec side before running
      "check_sec -elaborate -imp -spec_dut spec_instance".

-setup
    [-spec_dut spec_dut_instance_name_list]
    [-imp_dut imp_dut_instance_name_list]
    [-no_auto_mapping]
    [-exclude_module_auto_mapping {regexp_or_module_list}
        [-module_regexp]]
    [-exclude_instance_auto_mapping {regexp_or_instance_list}
        [-instance_regexp]]
    [-exclude_undriven_mapping]
    [-hierarchical]

    Connect the previously analyzed and elaborated spec and imp designs, and
    auto map primary_input, primary_output, bbox_input, bbox_output, undriven,
    reset_x, and stopat signals.

    NOTE: The setup auto mapping creates global mapping entries that will be
    exposed in all SEC tasks. The created connection mappings on boundary input
    signals are respected during reset.
    . Use "-spec_dut" to specify a list of one or more spec instances that will
      be used in SEC analysis. If you do not supply this switch, the tool
      considers the spec top instance as the DUT instance.
    . Use "-imp_dut" to specify a list of one or more imp instances that will
      be used in SEC analysis. If you do not supply this switch, the tool
      considers the imp top instance as the DUT instance.
    . Use "-no_auto_mapping" to specify that you do not want the tool to
      run auto mapping during setup. If you use this switch, you can run the
      auto mapping after the setup using
      "check_sec -auto_map_reset_x_values on; check_sec -map -auto".
    . Use "-exclude_module_auto_mapping" to instruct SEC to exclude one or more
      modules during the setup auto mapping.
      And use "-module_regexp" to specify that the
      "-exclude_module_auto_mapping" argument list is a regular expression
```

```
      list.
    . Use "-exclude_instance_auto_mapping" to instruct SEC to exclude instances
      during the setup auto mapping.
      And use "-instance_regexp" to specify that the
      "-exclude_instance_auto_mapping" argument list is a regular expression
      list.
    . Use "-exclude_undriven_mapping" to instruct SEC to exclude undriven
      signals mapping during the setup auto mapping.
    . You cannot use the switches -spec_dut or -imp_dut after elaborating the
      imp side with the switch "-spec_dut spec_instance".
      The specified "spec_instance" in "elaborate" command is used as the spec_dut,
      and the top instance on the imp side as the imp_dut.

    NOTE: To avoid adding init mappings, run "check_sec
    -auto_map_reset_x_values off".

-setup -clear
    Clear the SEC database and the connection between spec and imp.

    Hierarchical SEC (HSEC)
    =======================
    Use "-hierarchical" to run hierarchical SEC (HSEC). The HSEC flow supports
    running Sequential Equivalence Checking on SOC-level designs.
    This flow targets state-matching designs.
    . Using "-hierarchical" is only possible if the "sec_shallow_elaboration"
      configuration is set before design elaboration.
    . HSEC creates the "hsec" directory in the Jasper project directory and
      dumps the "hsec_script.tcl" file there during design setup. This Tcl file
      contains commands that run for each partition (job).
    . By default, the tool creates the "hsec_script.tcl" file in the
      "<jg_project_dir>/hsec" directory.
    . Use "source hsec_script.tcl" to run all partitions and generate the
      final results.

    HSEC Commands
    -------------
    For the complete command documentation for the following commands, run the
    "help" command:

    help <command_name> -gui

    . set_sec_hierarchical_num_session: Use the
      "set_sec_hierarchical_num_session" command to set the number of
```

sessions you want to launch when partitioning to improve the turnaround time. The tool consolidates results and shows them in the main session.
NOTE: You can also use the reporting command to get results.

. set_sec_hierarchical_blackbox_failing_instance: Use the "set_sec_hierarchical_blackbox_failing_instance" command to change verification behavior.
When verifying an instance with child instances that have failed, the tool can include this instance as part of the parent instance. And if the parent instance passes, we can ensure that a failure from the child instance is not propagated to the parent. By default, this setting black boxes the child instances.
. You can use the following commands to control partitioning:
  . set_sec_hierarchical_module_excludelist: Use the "set_sec_hierarchical_module_excludelist" command to specify a list of modules you want to exclude from partitioning. During partitioning, the tool ignores the specified modules.
  . set_sec_hierarchical_module_includelist: Use the "set_sec_hierarchical_module_includelist" command to specify a list of modules you want to include in partitioning.
  . set_sec_hierarchical_module_average_size: Use the "set_sec_hierarchical_module_average_size" command to specify the average size of the partition and to ensure that the size of each partition complies with the given value. By default, this setting value is "50000".
  NOTE: The size of the partition is a function of the number of storage elements and the number of nets.
  . set_sec_hierarchical_module_min_size: Use the "set_sec_hierarchical_module_min_size" command to specify the minimum size of the partition. If the partition size is smaller than the specified minimum size value, the tool merges the partition into its parent partition. By default, this setting value is "5000".
  NOTE: The size of the partition is a function of the number of storage elements and the number of nets.
. set_sec_hierarchical_save_partition_database: Use the "set_sec_hierarchical_save_partition_database" command to drive database file generation for the partition process. By default, this setting value is "none".
. set_sec_hierarchical_report_incremental_status: Use the "set_sec_hierarchical_report_incremental_status" command to write status updates between verification steps and re-read when running scriptFile again. By default, this setting value is "true".
. set_sec_hierarchical_force_job: Use the "set_sec_hierarchical_force_job" command to update the job results database even if the variable

```
      "set_sec_hierarchical_report_incremental_status" is selected. By default,
      this setting value is "true".
    . set_sec_hierarchical_use_analyze_snapshot: When running a new job, if a
      previously loaded "analyze" snapshot is available, use the
      "set_sec_hierarchical_use_analyze_snapshot" command to avoid rerunning
      the "analyze" command. By default, this setting value is "false".
    . set_sec_hierarchical_regular_expression: Use the
      "set_sec_hierarchical_regular_expression" command to provide a regular
      expression for instance matching in the HSEC flow when the spec and imp
      names are different. "sec_hierarchical_regular_expression" takes a list of
      spec and imp mapping pair regular expressions.
    . set_sec_hierarchical_match_port_number: Use the
      "set_sec_hierarchical_match_port_number" command to force top-level
      mapping regardless of same number of instance ports. By default, this
      setting value is "true".
    . set_sec_hierarchical_run_time_limit: Use the
      "set_sec_hierarchical_run_time_limit" command to specify the restart
      timeout period for unresponsive HSEC worker sessions that have not
      updated their logfile after a user-defined timeout period. The tool
      handles this scenario as if the timeout had occurred inside the session
      itself, that is, the error module is included at the parent level again.
      By default, this setting value is "false".
    . set_sec_hierarchical_generate_script: Use the
      "set_sec_hierarchical_generate_script" command to generate the HSEC script
      when called with "check_sec -setup -hierarchical". By default, this
      setting value is "true".
    . set_sec_hierarchical_signoff_waiver_list: Use the
      "set_sec_hierarchical_signoff_waiver_list" command to set the list of signoff
      waived categories, which turn off the report for specified categories.
      By default, the following categories are waived ("x_signals_and_undrivens",
      "assumptions_conflict", "mapped_stopats", "assumptions_on_driven_signals").
    . set_sec_hierarchical_check_assumption: Use the
      "set_sec_hierarchical_check_assumption" command to include the
assumptions_conflict
      properties in proofs.
      These properties are special properties built into every task to check the
      effects of assumptions.
      By default, this setting value is "false".

    NOTE: You must run the HSEC configuration commands before running the
    "check_sec -setup" command.

HSEC Reports
```

```
------------
-hierarchical -report (  failed_run
                       | verification_result
                       | verification_result_tree) [-silent]

    Running Partitions by sourcing "hsec_script.tcl" runs all determined design
    partitions with the "-setup" command and generates the final results.
    When "running partitions" completes, do the following:
    . Run the "check_sec -hierarchical -report verification_result [-silent]"
      command to verify the results summary for all partitions.
    . Run the "check_sec hierarchical -report verification_result_tree
      [-silent]" command to verify the results summary tree for all partitions.
    . Run the "check_sec -hierarchical -report failed_run [-silent]" command to
      combine and debug partitions in a single session.
    . Run the "hsec::report_hierarchy_tree" command to view design hierarchy.

    NOTE: Running the "check_sec -hierarchical_report" command with any
    of the switches listed above creates a report file under the
    Jasper project directory, in the hsec directory, with the name
    "hsec_failed_run.rpt", "hsec_verification_result.rpt", or
    "hsec_verification_result_tree.rpt", accordingly.

-set_dut_instance
    -spec spec_dut_instance_name_list -imp imp_dut_instance_name_list

    Change the previously specified DUT instances to the instances specified
    with this command.

    NOTE: "check_sec -set_dut_instance" is not respected after running
    "check_sec -elaborate -imp -spec_dut spec_instance".

Clear
-----
-clear (all | mapping | (result [-task task_name]))

    Clear SEC settings.
    . Use "all" to clear all SEC settings in the current session.
    . Use "mapping" to clear all mappings and their proof properties.
    . Use "result" to clear all proof information in the current or specified
      task.
```

```
Interface Check
---------------
-interface [-mapped |-unmapped]
           [-spec |-imp]
           [-signal_type (  primary_input | primary_output
                          | dut_input | dut_output
                          | bbox_input | bbox_output | user_bbox_input
                          | user_bbox_output | auto_bbox_input | auto_bbox_output
                          | undriven | internal_undriven | setup_stopat | reset_x
                          | x_assignment | x_default_assignment | x_divide_by_zero
                          | x_index_out_of_range | x_misc_undriven
                          | x_bus_contention | x_bus_floating
                          | user_env_stopat | user_task_stopat)]
           [-imp_assumptions [-assumption_type (none | assumptions_embedded)]]
         [-instance instance_full_path]
         [-task task_name]
         [-ignore_user_stopats]
         [-ignore_imp_assumptions]

    Run the interface check to generate properties based on existing mappings,
    and report which signals are mapped/unmapped.

    Use this command without additional switches to report, for each signal
    type, whether the check passed or failed. And for failures, report how many
    signals of the specified type in the spec and imp are unmapped.

    Or specify additional switches defined below to refine the report.
    . Use "-mapped" to list mapped signals.
    . Use "-unmapped" to list unmapped signals. This is the default if you do
      not specify "-mapped".
    . Use "-spec" to list only signals from the spec design. This is the
      default if you do not specify "-imp".
    . Use "-imp" to list only signals from the imp design.
    . Use "-signal_type" to list only signals of the specified type.
    . Use "-instance" to list only signals that are in the specified instance.
    . Use "-task" to specify a task other than the current task.
    . Use "-ignore_user_stopats" to list only signals that are not formed by
      the "stopat" command.
    . Use "-ignore_imp_assumptions" to list only signals.
    . Use "-imp_assumptions" to list only the enabled imp assumptions.

Mapping Dictionary
==================
```

This section describes the mapping dictionary, which you can use to identify mapping pairs for "check_sec" commands. You can provide a dictionary as an argument for the "-dict" switch or specify it as a return type with "-return dict".

Use the "set_sec_return_mapping_dictionary_from_gui" command to control whether SEC commands issued from the GUI include mapping IDs or a dictionary.

Structure

The "-dict" switch accepts a list of mapping dictionaries wherein each dictionary includes a list of key-value pairs. Keys are mapping properties that help identify mapping pairs. Build your dictionary as follows:

. Specify each key-value pair with the key followed by its value enclosed in curly braces.
  Example: is_global {true}
. Enclose a list of key-value pairs, in a set of curly braces to denote a mapping dictionary.
  Example: {key1 {value1} key2 {value2} .... }
. Enclose a list of mapping dictionaries in a set of braces.
  Example: {{key1 {value1} key2 {value2}} {key1 {value1} key2 {value2}}}

NOTE: To specify just one mapping dictionary, you must also enclose it in braces to denote a list of dictionaries.
Example: {{key1 {value1} key2 {value2} .... }}

Dictionary Keys

```
-dict { {
          [spec_signal {spec_signal_name}]
          [imp_signal {imp_signal_name}]
          [type { connection_primary_input | ver_target_primary_output
                 | connection_dut_input | ver_target_dut_output
                 | ver_target_bbox_input | connection_bbox_output
                 | connection_misc | ver_target_misc
                 | connection_all | ver_target_all | ver_target_mpram_port
                 | connection_stopat | stopat_spec | stopat_imp | stopat_all
                 | stopat_both | stopat_both_env
                 | cutpoint_spec | cutpoint_imp | cutpoint_both
                 | cutpoint_both_init | cutpoint_all
                 | abstract_reset_value | abstract_counter | abstract_init_value
                 | connection_undriven | helper | init_user | init_auto | init_all}]
```

```
      [context {context_format_string}]
      [map_to_constant {-1 | 0 | 1}]
      [is_inverse {true | false}]
      [ ( task {task_name} )
        | ( is_global {true | false} )]
      [tag {tag_name}]
      [is_respected_during_reset {true | false}] }}
```

NOTE: The "task" and "is_global" keys are mutually exclusive.

. spec_signal: Defines the spec signal name that should be present in mapping pairs to be returned.
. imp_signal: Defines the imp signal name that should be present in mapping pairs to be returned.

NOTE: If you include both "spec_signal" and "imp_signal" keys, the dictionary returns an intersection of the mappings in which these are the spec and imp signals respectively, and not a union.

. type: Gets only mapping pairs of the specified mapping type.
. context: Gets only mapping pairs with the specified context attributes. Example: for a mapping with spec delay 2, and imp delay 1, you can specify: context {{2,1} {,} {0,0} {} {}}
. map_to_constant: Gets either mapping pairs that are mapped to a constant value (0 or 1) or are not constant mapping (-1).
. is_inverse: Gets only mapping pairs with the specified value for inverse attribute.
. task or is_global: Gets task-specific mapping pairs or global mappings. The "task" and "is_global" keys are mutually exclusive, so you can specify only one of them. If you specify neither of them, or if you specify "is_global" with value "false", the tool returns mappings from only the current task.
. tag: Gets only mapping pairs with the specified tag.

. is_respected_during_reset: Gets only mapping pairs that are in effect during reset.

  NOTE:
  . You must specify the "type" key to specify "is_respected_during_reset". Also, you can only specify the key for boundary input mapping types, namely: connection_primary_input, connection_dut_input, connection_bbox_output, connection_undriven, connection_stopat, connection_misc, abstract_reset_value.

```
    . Only global mappings can be respected during reset. To specify the
      "is_respected_during_reset" key as "true", the "is_global" key must also
      be included as "true".

  Under-Specification of Keys
  ---------------------------
  Each key in the mapping dictionary helps specify an attribute of mapping
  pairs, and hence filters based on that attribute. Therefore, specifying one
  or only a few keys can also help in filtering and return a list of mapping
  pairs. However, this under-specification of keys might not provide enough
  filtering criteria, and the dictionary might return many mappings. This can
  result in unwanted outcomes, especially with proactive commands like
  "check_sec -unmap", where you might unknowingly cancel many mapping pairs
  instead of one. Therefore, we recommend that you specify as many keys as
  possible in the mapping dictionary.

Lists
=====
This section describes the many lists available with "check_sec". Use these
commands to list signals, hierarchy mapping, signal mapping, violating mapping,
or disabled mapping.

NOTE: The "-signal" and "-mapping" switches are mutually exclusive. If you
specify none of these switches (check_sec -list), the command lists signals.

Signals List
------------
-list [ [-signal]
          [-signal_type (  input | output | primary_input | primary_output
                         | dut_input | dut_output
                         | bbox_input | bbox_output | user_bbox_input
                         | user_bbox_output | auto_bbox_input | auto_bbox_output
                         | mpram_port | undriven | internal_undriven
                         | flop | latch | internal | stopat | reset_x
                         | x_assignment | x_default_assignment | x_divide_by_zero
                         | x_index_out_of_range | x_misc_undriven
                         | x_bus_contention | x_bus_floating
                         | user_env_stopat | user_task_stopat | gated_clock)]
          [-spec |-imp]
          [(-coi_of |-fanout_of) {signal_list}]
          [ -instance instance_full_path
           |-name {regexp} [-verbose]
      ]
```

```
    "-list -signal" or just "-list" with no additional switches lists all
    signals.

    NOTE: The "-name" and "-instance" switches are mutually exclusive.

    . Use "-spec" or "-imp" to indicate whether the "-instance" or "-name"
      argument specifies an instance or signal(s) on the spec or imp side.
    . Use "-coi_of" to specify that you want to list signals in the COI of the
      specified signal list.
    . Use "-fanout_of" to specify that you want to list signals in the fanout
      of the specified signal list.
    . Use "-name" to define the set of signal names on the specified
      side that you want to list.
      The "-name" argument is a regular expression.
      Include the "-verbose" switch to print info messages detailing the
      regexp matching operation.
    . Use "-instance" to list all signals that are in the specified instance.
    . Use "-signal_type" to list only signals of the specified type.

Hierarchy Mapping Rule List
---------------------------
-list -mapping -hierarchy [-return (text | id)]
    "-list -mapping -hierarchy" lists all hierarchy mapping rules.
    . Use "-return text" to return the result as rules text. (This is the
      default behavior.)
    . Use "-return id" to return the result as hierarchy mapping rules IDs.

Signal Mapping List
-------------------
-list -mapping
           [-mapping_type (  connection_all | connection_primary_input
                           | connection_bbox_output | connection_undriven
                           | connection_stopat | connection_misc
                           | ver_target_all | ver_target_primary_output
                           | ver_target_mpram_port | ver_target_bbox_input
                           | ver_target_misc | cutpoint_all | cutpoint_both
                           | cutpoint_spec | cutpoint_imp | cutpoint_both_init
                           | stopat_all | abstract_reset_value | abstract_counter |
abstract_init_value
                           | stopat_both | stopat_spec  | stopat_imp | stopat_both_env
                           | helper | init_user | init_auto | init_all |
clock_gating_toggle_coverage) ]
           [-signal_type (  input | output | primary_input | primary_output
```

```
                          | dut_input | dut_output
                          | bbox_input | bbox_output | user_bbox_input
                          | user_bbox_output | auto_bbox_input
                          | auto_bbox_output | mpram_port | undriven |
internal_undriven
                          | flop | latch | internal | stopat | reset_x
                          | x_assignment | x_default_assignment | x_divide_by_zero
                          | x_index_out_of_range | x_misc_undriven
                          | x_bus_contention | x_bus_floating
                          | user_env_stopat | user_task_stopat | gated_clock)]
          [  ( -instance {instance_full_path} )
            |(-exclude_signal {signal_list}
            |( -of_signal {regexp_or_signal_list}
               [ -spec_regexp [-verbose]
                |-imp_regexp [-verbose] ] )
          ]
          [-tag {tag_name}]
          [([-task {task_name}] [-include_global_mapping])
           |-global
          ]
          [-return (text | id | dict)]
          [-proof_status (proven | cex | unprocessed | undetermined)+]
          [-proof_task {proof_task_name}]
          [ (-coi_of {signal_list} [-get_first_level_coi])
           |(-fanout_of {signal_list} [-get_first_level_fanout])]
          [-status (enabled | disabled)]
    "-list -mapping" with no additional switches lists the current
      task's mapping pairs. Use "-global" to list global mappings.

    NOTE: The "-of_signal" and "-instance" switches are mutually exclusive.

    . Use "-mapping_type" to list only mapping pairs of the specified mapping
      type.
    . Use "-signal_type" to list only mapping pairs with signals of the
      specified signal type.
    . Use "-of_signal" to define the set of signal names whose mappings you
      want to list.
        . Use "-spec_regexp" or "-imp_regexp" to specify that the "-of_signal"
          argument is a regular expression list on the spec or imp side.
          The default is spec side; otherwise, it is a list of specific names
          of signals, bits, or bit-vector expressions.
        . Use "-verbose" to print info messages detailing the regexp matching
          operations.
```

```
    . Use "-instance" to specify that you want to list only mapping pairs of
      signals that are in the specified instance.
    . Use "-exclude_signal" to instruct SEC to exclude mappings mapped on the
      passed signal(s).
    . Use "-tag" to list only mapping pairs with the specified tag.
    . Use "-task" to list the specified task's mapping instead of the current
      task's mapping.
    . Use "-include_global_mapping" to list both the global and task-specific
      mapping pair(s).
    . Use "-global" to list global mapping.
    . Use "-return" to specify the return type.
      . Use "-return text" to return the result as mapping text. (This is the
        default behavior.)
      . Use "-return id" to return the result as mapping IDs.
      . Use "-return dict" to return the result as a list of mapping
        dictionaries. See the section "Mapping Dictionary" for more info.
    . Use "-proof_status" to filter mapping entries according to their proof
      status. This switch accepts one or more of the following arguments:
      "proven", "cex", "unprocessed", "undetermined".
    . Use "-proof_task" to filter mapping entries according to their proof
      status in the proof task specified. Use this switch in combination with
      the switches "-global" and "-proof_status".
    . Use "-coi_of" to specify that you want to list mapping pairs in the
      COI of the specified signal list. And use "-get_first_level_coi" to get
      mapping pairs for the first level of the COI.
    . Use "-fanout_of" to specify that you want to list mapping pairs in the
      fanout of the specified signal list. And use "-get_first_level_fanout" to
      get mapping pairs for the first level of the fanout.
    . Use "-status" to filter mapping entries according to their status.
      This switch accepts one of the following arguments: "enabled", "disabled".

Violating Mapping List
----------------------
-list -mapping
    -violating_mapping (  all | clock_signal | init_diff
                        | init_mapping_ranges_diff
                        | init_mapping_with_context
                        | init_mapping_with_stopat
                        | assumption_comb_coi
                        | comb_loop_comb_coi
                        | assumption_and_comb_loop_comb_coi
                        | context_both_edge
                        | dscp_on_constant
```

```
                          | fast_rated_signal)
    [-mapping_type (  connection_all | connection_primary_input
                          | connection_bbox_output | connection_undriven
                          | connection_stopat | connection_misc
                          | ver_target_all | ver_target_primary_output
                          | ver_target_mpram_port | ver_target_bbox_input
                          | ver_target_misc | cutpoint_all | cutpoint_both
                          | cutpoint_spec | cutpoint_imp | cutpoint_both_init
                          | stopat_all | abstract_reset_value | abstract_counter
                          | stopat_both | stopat_spec  | stopat_imp | stopat_both_env
                          | helper | init_user | init_auto | init_all |
clock_gating_toggle_coverage) ]
    [-return (text | id | dict)]
    [-detailed]

    "-list -mapping -violating_mapping" with one of the following arguments
    lists mappings that violated rules in the format provided by "-return" switch.
    . Use "all" to list all violating mappings from all violating reasons.
    . Use "-detailed" with "all" to return the violating mappings as a dictionary
      with the following format:
      {reason1 {list of violating by reason1} reason2 {list of violating by reason2}}.
    . Use "clock_signal" to list cutpoints whose signals are in the
      design clock tree.
    . Use "init_diff" to list mappings whose signals in spec and imp
      have different init values.
    . Use "init_mapping_ranges_diff" to list cutpoints for which the mapped
      range between spec and imp is different than the mapped range in the
      related init mapping.
    . Use "init_mapping_with_context" to list cutpoints on bits that have init
      mapping with context.
    . Use "init_mapping_with_stopat" to list init mappings with a stopat mapping
      on the same signal, which makes init mapping redundant.
    . Use "assumption_comb_coi" to list cutpoints where the cut signal is under
      a combinational influence of a user-specified assumption. That is, there
      is a combinational trace from the cut signal to a signal that is used in
      the user-specified assumption.
    . Use "comb_loop_comb_coi" to list cutpoints where the cut signal is under
      a combinational influence of a structural combinational loop. That is,
      there is a combinational trace from the cut signal to a signal that is
      part of a combinational loop.
    . Use "assumption_and_comb_loop_comb_coi" to list double-sided cutpoints
      where the cut signal on the spec/imp side is under the combinational
      influence of a user-specified assumption while the signal on the
```

```
      complementary side is under the combinational influence of a
      combinational loop.
    . Use "context_both_edge" to list cutpoints with context when any clock
      is declared with "-both_edges".
    . Use "dscp_on_constant" to list double-sided cutpoints that have been
      applied on signals that have constant bits.
    . Use "fast_rated_signal" to list all mappings whose signals might
      change on the negative edge of the fastest clock.
    . Use "-mapping_type" to list only mapping pairs of the specified mapping
      type.
    . Use "-return" to specify the return type.
      . Use "-return text" to return the result as mapping text. (This is the
        default behavior.)
      . Use "-return id" to return the result as mapping IDs.
      . Use "-return dict" to return the result as a list of mapping
        dictionaries. See the section "Mapping Dictionary" for more info.

Disabled Mapping List - Fractional SEC Mode
-------------------------------------------
-list -mapping -disabled_out_of_target_coi
    List mapping pairs that were disabled due to their signals being out of
    any target COI. In these cases, the signals are irrelevant to the prove
    process.
    NOTE: This command is relevant only when using "fractional SEC
    verification mode."

Related Verification
--------------------
-related_verification ( -property {property_list}
                        |-id {mapping_id_list}
                        |-dict {mapping_dict_list}
                       )
                       [-task {task_name}]

    List the verification properties in the COI of the specified targets. These
    include related covers of specified targets, related covers of assumptions,
    bbox_input assertions, clock gating covers, abstractions and their related
    covers.
    . Use "-id" to specify a list of IDs of mapping pairs.
    . Use "-dict" to specify a list of mapping dictionaries of mapping pairs.
      See the section on "Mapping Dictionary" for more info.
    . Use "-property" to specify a list of properties.
    . Use "-task" to use the specified task's properties/mappings instead of the
```

```
current
      task's properties/mappings.
      NOTE:
      . Providing task with properties in a different task than the
        one specified causes an error.
      . Providing properties from different tasks causes an error.


Mapping Info
------------
-mapping_info (  -id mapping_id_list
               | -dict {mapping_dict_list} [-force]
               | ( -spec {spec_regexp_or_signal_list}
                  |-imp {imp_regexp_or_signal_list}
                   [-regexp [-verbose]])
             )
             [-signal_type (  input | output | primary_input | primary_output
                            | dut_input | dut_output
                            | bbox_input | bbox_output | user_bbox_input
                            | user_bbox_output | auto_bbox_input | auto_bbox_output
                            | mpram_port | undriven | internal_undriven
                            | flop | latch | internal | stopat | reset_x
                            | x_assignment | x_default_assignment | x_divide_by_zero
                            | x_index_out_of_range | x_misc_undriven
                            | x_bus_contention | x_bus_floating | gated_clock)]
             [-mapping_type (  connection_all | connection_primary_input
                             | connection_bbox_output | connection_undriven
                             | connection_stopat | connection_misc
                             | ver_target_all | ver_target_primary_output
                             | ver_target_mpram_port | ver_target_bbox_input
                             | ver_target_misc | cutpoint_all | cutpoint_both
                             | cutpoint_spec | cutpoint_imp | cutpoint_both_init
                             | stopat_all | abstract_reset_value | abstract_counter
                             | stopat_both | stopat_spec  | stopat_imp |
stopat_both_env
                             | helper | init_user | init_auto | init_all |
clock_gating_toggle_coverage) ]
             [  ( -instance {instance_full_path} )
               |( -of_signal {regexp_or_signal_list}
                  [ -spec_regexp [-verbose]
                   |-imp_regexp [-verbose] ] )
             ]
             [-tag {tag_name}]
```

```
          [([-task {task_name}] [-include_global_mapping])
           |-global
          ]
          [-list (spec_signal | imp_signal | context | source | id
                | sva_expression | property | is_enabled | is_inverse
                | type | task_name | tag | status)]
          [-silent]

List info for specific mapping pairs.
Unless a specific field is requested, this command returns, for each
specified mapping pair, the following tuple:
{ mapping pair ID, spec_signal, imp_signal, context,
  mapping source, assert/assume expression,
  property name, standard textual representation,
  is enabled, is verification pair, is inverse,
  mapping type, task name, tag }.

. Use "-task" to list the specified task's mapping instead of the current
  task's mapping.
  . When using "-list property", the tool only shows the properties for the
    specified task instead of the current task.
  . When using "-list status", the tool only shows the statuses for the
    specified task instead of the current task.
. Use "-global" to list global mapping.
. Use "-include_global_mapping" to list both the global and the
  task-specific mapping pair(s).
. Use "-id" to specify the list of IDs of the specific mapping pairs.
. Use "-dict" to specify the list of mapping dictionaries of specific
  mapping pairs. See the section on "Mapping Dictionary" for more info.
. Use "-spec" to define the set of signal name(s) on the spec side whose
  mapping info you want to list.
. Use "-imp" to define the set of signal name(s) on the implementation side
  whose mapping info you want to list.
. Use "-regexp" to specify that the "-spec" or "-imp" argument list
  is a regular expression list; otherwise, it is a list of specific names
  of signals, bits, or bit-vector expressions. Include the "-verbose"
  switch to print info messages detailing the regexp matching operation.
. Use "-signal_type" to list only mappings of signals of the specified
  type.
. Use "-mapping_type" to list only mapping pairs of the specified mapping
  type.
. Use "-of_signal" to define the set of signal names whose mappings you
  want to list.
```

```
        . Use "-spec_regexp" or "-imp_regexp" to specify that the "-of_signal"
          argument is a regular expression list on the spec or imp side.
          The default is spec side; otherwise, it is a list of specific names
          of signals, bits, or bit-vector expressions.
        . Use "-verbose" to print info messages detailing the regexp matching
          operations.
    . Use "-instance" to specify that you want to list only mapping  pairs of
      signals that are in the specified instance.
    . Use "-proof_status" to list mapping entries according to their proof
      status.
      This switch accepts one or more of the following arguments:
      "proven", "cex", "unprocessed", "undetermined".
    . Use "-tag" to list only mappings with the specified tag.
    . Use "-list" to get specific information on the specified mapping entries.
      This switch accepts one of the following arguments:
      "spec_signal", "imp_signal", "context", "source", "sva_expression", "id",
      "property", "is_enabled", "is_inverse", "type",
      "task_name", "tag", or "status".
    . Use "-silent" to turn off output to the screen and return mapping
      information with a Tcl return value.

Hierarchy Mapping Rule Info
---------------------------
-mapping_info -hierarchy -id {hierarchy_aliasing_rule_id_list}
    List info for specific hierarchy mapping rules.
    This command returns, for each specified hierarchy mapping rule ID, the
    following tuple:
    { hierarchy mapping rule ID, spec hierarchy expression,
      imp hierarchy expression, isRegEx }.
    . Use "-id" to specify the list of IDs of the specific hierarchy mapping
      rules.



Signal Mapping
--------------
-map   ([-auto]
          [-spec {spec_instance_list}
          [-imp {imp_instance_list}]]
          [-signal_type (  input | output | primary_input | primary_output
                         | dut_input | dut_output
                         | bbox_input | bbox_output | user_bbox_input
                         | user_bbox_output | auto_bbox_input | auto_bbox_output
```

```
                    | mpram_port | undriven | internal_undriven
                    | flop | latch | internal | stopat | reset_x
                    | x_assignment | x_default_assignment | x_divide_by_zero
                    | x_index_out_of_range | x_misc_undriven
                    | x_bus_contention | x_bus_floating)]
    [-coi_of {signal_list}]
    [-set_value (0 | 1)]
    [-max_word_size max_word_size_value]
    [-return (text | id | dict | none)]
    [-spec_delay spec_delay_value]
    [-imp_delay imp_delay_value]
    [-map_condition {map_cond_expr}]
    [-clock {clocking_event}]
    [-disable {disable_cond_expr}]
    [-edge (posedge | negedge | both_edge)]
    [ -cut (both | both_init | spec | imp)
     |-stopat (both | spec | imp | both_env)
     |-helper]
    [-tag {tag_name}]
    [-task {task_name} |-global]
    [-exclude_signal {regexp_or_signal_list} [-signal_regexp]]
    [-respect_connections_during_reset]
    [-respect_abstraction_during_reset]
    [-exclude_module {regexp_or_module_list} [-module_regexp]]
    [-exclude_instance {regexp_or_instance_list} [-instance_regexp]]
    [-target_bound {target_bound_value}]
    [-mapping_type ( connection_primary_input
                    | connection_bbox_output
                    | connection_undriven | connection_stopat
                    | connection_misc
                    | ver_target_primary_output
                    | ver_target_mpram_port
                    | ver_target_bbox_input | ver_target_misc
                    | cutpoint_both | cutpoint_spec
                    | cutpoint_imp | cutpoint_both_init
                    | abstract_reset_value | abstract_counter
                    | abstract_init_value | stopat_both
                    | stopat_spec  | stopat_imp
                    | stopat_both_env | helper | init_user
                    | init_auto
                    | clock_gating_toggle_coverage) ]
 )
|( ( -spec {spec_regexp_or_signal_list} [-imp {imp_regexp_or_signal_list}])
```

```
            [-regexp [-verbose] [-exclude_signal {regexp_or_signal_list} [-
signal_regexp]]]
            [-max_word_size max_word_size_value]
            [-inverse]
            [-return (text | id | dict | none)]
            [-spec_delay spec_delay_value]
            [-imp_delay imp_delay_value]
            [-map_condition {map_cond_expr}]
            [-clock {clocking_event}]
            [-disable {disable_cond_expr}]
            [-edge (posedge | negedge | both_edge)]
            [ -cut (both | both_init | spec | imp)
             |-abstract (reset_value | counter | init_value)
             |-stopat (both | spec | imp | both_env)
             |-helper
             |-constraint]
            [-tag {tag_name}]
            [-task {task_name} |-global]
            [-init]
            [-respect_connections_during_reset]
            [-respect_abstraction_during_reset]
            [-target_bound {target_bound_value}]
       )
      |(-hierarchy
            -spec {spec_regexp_or_hierarchy_list}
            -imp {imp_regexp_or_hierarchy_list}
            [-regexp]
       )
      | -map_file map_file_name

   Map specification signals to implementation signals.

   Auto Mapping
   ------------
   Use "-auto" to automatically map primary inputs and outputs, inputs and
   output signals of black boxes, and undriven signals that have the same
   names in the specification and implementation.
   . Use "-spec" with "-auto" to specify the spec instance list whose signals
     are to be auto mapped.
   . Use "-imp" with "-auto" to specify the imp instance list whose signals
     are to be auto mapped. If you use "-spec" but not "-imp", the tool
     attempts to map the spec instance signals to imp signals in an instance
     of the same name, taking into account the imp prefix. If you use both
```

"-spec" and "-imp" switches, the tool considers pairs of instances by matching each spec instance from the given list to the imp instance in the same position on the imp list. Therefore, the two supplied spec and imp lists must be of the same length.

. Use "-signal_type" and a signal type to auto map only signals of the specified type.
. Use "-coi_of" to specify that you want to auto map signals in the COI of the specified signal list.
. Use "-set_value" to set the mapped undriven signals to a constant value, 0 or 1.
. Use "-max_word_size" to specify a maximum size for mapping entries. Mapping of vector signals or bit ranges that are wider than the specified value splits the entries into small "chunks" with a width up to "max_word_size" each. Specify the value 0 to use the full range of bits.
. Use "-tag" to add a tag (property name prefix) to the generated mapping(s).
. Use "-task" to consider the specified task's mapping instead of the current task mapping.
. Use "-global" to consider global mapping, that is, mapping that is exposed in all tasks.
. Use "-respect_connections_during_reset" to create connections that are respected during reset. This switch requires the "-global" switch. This switch affects only connection mappings (such as primary inputs, bbox outputs, undrivens, and so forth); therefore, you cannot use it with the following switches: -cut, -helper, -abstract, -init, -hierarchy, and -cutpoint_to_helper.
  NOTE:
  . Using "-respect_connections_during_reset" with "-stopat" is not supported. Combining the switches triggers an error.
  . Using "-respect_connections_during_reset" with "-cut" is forbidden because the assertion added by the cutpoint is not respected during reset. Combining the switches triggers an error.
. Use "-respect_abstraction_during_reset" to create reset_value abstractions that are respected during reset. This switch requires the "-global" switch.
. Use "-exclude_signal" to instruct SEC to exclude signals(s) from auto mapping and use "-signal_regexp" to specify that the "-exclude_signal" argument list is a regular expression list.
. Use "-exclude_module" to instruct SEC to exclude module(s) from auto mapping and use "-module_regexp" to specify that the "-exclude_module" argument list is a regular expression list.
. Use "-exclude_instance" to instruct SEC to exclude instance(s) from auto mapping and use "-instance_regexp" to specify that the

```
  "-exclude_instance" argument list is a regular expression list.
NOTE: The "-exclude_module", "-exclude_instance", and "-exclude_signal"
switches are not allowed with "check_sec -map -auto -signal_type reset_x".
. Use "-mapping_type" to create mapping pairs of the specified mapping type.
. Use the "-target_bound" switch to specify the target bound value
  for the generated mapping properties. By default, the target bound is
  set to 0.
  NOTE: This switch is only applicable when using provable mapping types.
  . The target bound set with this command takes precedence over any bound
    specified with the "set_prove_target_bound" command.
  . When the min_length of the corresponding assertion is >= the target bound,
    its status changes to "Bounded Proven (User)".

Hierarchy Mapping (Aliasing)
----------------------------
Use "-hierarchy" to add hierarchy mapping rules. These hierarchy mapping
rules help the tool automatically map primary inputs and outputs, input and
output signals of black boxes, and undriven signals that have the same
names but are under different hierarchy names in the spec and imp.
Each time the auto mapping tries to find a matching imp signal of some spec
signal, it applies these rules to it to find the matching imp signal. It
applies the rules one by one in the same order you added them.
. Use "-spec" to define the name(s) or sub-name(s) of the spec hierarchies
  to map.
. Use "-imp" to define the name(s) or sub-name(s) of the imp hierarchies
  to map.
  NOTE:
  . For both "-spec" and "-imp", use the full instance names including
    all parents.
. Use "-regexp" to specify that the "-spec" and "-imp" argument lists are
  regular expression lists; otherwise, they are lists of specific hierarchy
  names.
  NOTE:
  . Any imp regular expression can contain references to group values
    matched by the spec regular expression: $0, $1, $2, $3, ..., $9
  . Use (?:RE) pattern to group a sub-pattern but not capture the result in
    a reference ($0, ..., $9).

Manual Mapping
--------------
If you do not use "-auto", manually map the spec to the imp with the
following switches:
. Use "-spec" to define the name(s) of the spec signals to map.
```

```
      . The signal can be a bit-vector signal. In such cases, the imp
        signal must be a bit-vector signal of the same length. Specify
        sub-vectors of such bit-vector signals with an expression of the
        form "[left_index:right_index]", for example, [9:13] or [3:0]. The
        bits are mapped according to their order from the right vector bound
        to the left one.
    . Use "-imp" to define the name(s) of the imp signals, bits, or
      bit-vector expressions.
      . If you use "-spec" but not "-imp", then for each spec_regexp_or_signal
        from the given list, the tool attempts to map the spec signals to imp
        signals of the same name, taking into account the imp prefix.
      . If you use both "-spec" and "-imp" switches, the tool considers pairs
        of regexp_or_signal by matching each spec regexp_or_signal from the
        given list to the imp regexp_or_signal in the same position on the imp
        list. Therefore, the two supplied spec and imp lists must be of the
        same length.
      . You can use multi-bit vector expressions as in {spec_regexp_or_signal}.
    . Use "-max_word_size" to specify a maximum size for mapping entries.
      Mapping of vector signals or bit ranges that are wider than the specified
      value splits the entries into small "chunks" with a width up to
      "max_word_size" each. Specify the value 0 to use the full range of bits.
    . Use "-inverse" to specify that the mapped signals are the inverse (not)
      of each other. The tool generates assume or assert properties
      accordingly.
    . Use "-tag" to add a tag (property name prefix) to the generated
      mapping(s).
    . Use "-task" to relate the generated mappings to a specified task instead
      of the current task.
    . Use "-global" to create global mapping, that is, mapping that will be
      exposed in all tasks.
    . Use "-init" to create init mapping (that is, bound 1 equivalence
      assumptions).
      NOTE:
      . Init mappings cannot be created between signals with at least one non-storage
bits.
    . Use "-constraint" to create connection mapping, that is, to connect the
      signal pair by adding equivalence assumptions on them.
    . Use "-respect_connections_during_reset" to create connections that are
      respected during reset. This switch requires the "-global" switch.
      This switch affects only connection mappings (such as primary inputs,
      bbox outputs, undrivens, and so forth); therefore, you cannot use it with
      the following switches: -cut, -helper, -abstract, -init,
      -hierarchy, and -cutpoint_to_helper.
```

```
  NOTE:
  . Using "-respect_connections_during_reset" with "-stopat" is
    not supported. Combining the switches triggers an error.
  . Using "-respect_connections_during_reset" with "-cut" is forbidden
    because the assertion added by the cutpoint is not respected during
    reset. Combining the switches triggers an error.
. Use "-respect_abstraction_during_reset" to create reset_value
  abstractions that are respected during reset. This switch requires the
  "-global" switch.

Mapping Using Regular Expressions
---------------------------------
Use "-regexp" to inform the tool that the argument of the "-spec" switch,
{spec_regexp_or_signal_list}, and the argument of the "-imp" switch,
{imp_regexp_or_signal_list}, are regular expressions.
. If you use "-regexp" with "-spec" but without "-imp", the tool evaluates
  the regular expression {spec_regexp_or_signal_list} and then maps each
  spec signal that matches the expression to its matching imp signal.
. If you use "-regexp" with both "-spec" and "-imp", consider using (?:RE)
  patterns in {spec_regexp_or_signal_list} to group sub-patterns.
  Accordingly, in {imp_regexp_or_signal_list}, use references to group
  values matched by {spec_regexp_or_signal}: $0, $1, $2, $3, ..., $9.
  . If you do not use (?:RE) patterns, at least one of the arguments
    {spec_regexp_or_signal_list} or {imp_regexp_or_signal_list} must be an
    expression that is evaluated to a single signal; otherwise, if both
    expressions are evaluated to a list of signals, the SEC App will not be
    able to perform the mapping due to ambiguity in signal pairs selection.
. Include the "-verbose" switch to print info messages detailing the
  regexp matching operation.
. Use "-exclude_signal" to instruct SEC to exclude signals(s) from
  mapping and use "-signal_regexp" to specify that the "-exclude_signal"
  argument list is a regular expression list.
. When using the switch "-hierarchy", only the argument of the "-spec"
  switch is considered as regular expression, while the argument of the
  "-imp" switch is still considered as regular string. However, you
  can use references to group values matched by
  {spec_regexp_or_instance_list}: $0, $1, $2, $3, ..., $9 on the
  {imp_regexp_or_instance_list}.
. Using the switch "-hierarchy" requires squared parentheses to be escaped
  for spec expression as follows:
  1) Part of the instance name - escape with '\\'.
  2) Part of the regular expression - escape with '\'.
```

```
Task-Related Mapping
--------------------
With either auto or manual mapping, the tool relates the generated mappings
to the current task; that is, the generated mapping pairs will be exposed
as properties for the current task only.
. Use "-task" to relate the generated mappings to a specified task instead
  of the current task.
. Use "-global" to create global mapping, that is, mapping that will be
  exposed in all tasks.
Any auto-generated connection mapping is global regardless of the supplied
task switches because connection mapping is exposed as "-env" properties,
which affect all tasks.

Return Type
-----------
With either auto or manual mapping, use "-return" and one of
the following arguments to return information:
  . Use "text" to return the result as mapping text. (This is the
    default behavior.)
  . Or use "id" to return the result as mapping IDs.
  . Or use "dict" to return the result as list of mapping dictionaries.
    See the section on "Mapping Dictionary" for more info.
  . Or use "none" to return nothing (empty list of strings).

Mapping Pairs as Helpers
------------------------
With either auto or manual mapping, use "-helper" to create helper mapping
pairs. Helper mapping pairs direct the tool to create helper assertions
during the generation phase (under "check_sec -interface"). Refer to
the SEC User Guide for a detailed explanation of helper assertions.

The purpose of generating helpers is to prevent any unintended change in
the environment. For example, if you add a new mapping pair on a primary
input or an undriven signal, the tool generates a new "-env" assumption
(even if that was not your intention); however, using "-helper" prevents
the tool from generating an assumption and only exposes helper assertions.

Mapping Pairs as Cutpoints
--------------------------
With either auto or manual mapping, use "-cut" to create cutpoint mapping
pairs. Refer to the SEC User Guide for a detailed explanation of
cutpoints.
. Use "-cut both" to add cutpoints on the signals on both sides
```

without the concrete value init assumption.
. Use "-cut both_init" to add cutpoints on the signals on both sides.
. Use "-cut spec" to add a cutpoint only on the signal on the spec side.
. Use "-cut imp" to add a cutpoint only on the signal on the imp side.

Mapping Pairs as Abstractions

With either auto or manual mapping, use "-abstract" to create a
Jasper abstraction on both the spec and imp designs and connect them in
a way that keeps the complexity reduction and the safety of the abstraction
without introducing unnecessary false negatives.

. Use "-abstract reset_value" to create a reset value abstraction on both
the spec and imp and apply it on the current task.

. Use "-abstract counter" to create a counter abstraction on both the spec
and imp.

. Use "-abstract init_value" to create an init abstraction on both the spec
and imp.

Mapping Pairs as Stopats

With either auto or manual mapping, use "-stopat" to create stopat mapping
pairs. During the generation phase (under "check_sec -interface"),
the tool adds stopats to mapped signals (see the "stopat" command) and
generates an equivalence assumption on them.
. Use "-stopat both" to add stopats on the signals on both sides.
. Use "-stopat spec" to add a stopat only on the signal on the spec side.
. Use "-stopat imp" to add a stopat only on the signal on the imp side.
. Use "-stopat both_env" to add stopats that are respected during reset on
the signals on both sides.

Temporal and Logical Context

Use the following switches to specify temporal and logical context for new
mappings. This set of switches generates various combinations of temporal
and logical conditions on spec and imp equivalence.
. -spec_delay
. -imp_delay
. -map_condition
. -clock
. -disable

```
 Important things to know:
 . If you do not specify delay values, the default is 0.
 . Only one of "-spec_delay" or "-imp_delay" can be greater than 0.
 . All condition values default to 1 (true).
 . Delay values are non-negative integers.
 . The following condition expressions are valid SVA expressions, and each
   of them might contain signals on both sides.
   . {map_cond_expr}
   . {disable_cond_expr}
 . {clocking_event} is "negedge" or "posedge" plus an SVA expression for
   clock signals.
 . If you supply all of these values along with the spec_signal mapped to
   the imp_signal, the tool generates an expression equivalent to the
   following:
@ (clocking_event)
        disable iff (disable_cond_expr)
              (
                 (##spec_delay_value  1'b1).triggered and map_condition.triggered
                 |->
                 spec_signal == $past(imp_signal,  spec_delay_value)
              )

   NOTE:
   . This expression specifies a condition on spec_signal and imp_signal that
     is checked on every {clocking_event} and when {disable_cond_expr} is
     false.
   . This expression is built out of the antecedent on the left of the
     implication (|->) and the consequent on its right.
   . The equality between the mapped signals is the consequent, but it might
     be more than a simple equality; that is, the mapped signals might
     differ on the relative time when they are sampled.
   . All the delays in the context expression are relative to the time of
     the event of the implication ("the |-> time").
   . {map_cond_expr} can be any Boolean expression;
     that is, it is not necessary for it to contain signals only from one
     side.
   . The consequent is an equality between the mapped signals such that the
     spec signal is sampled spec_delay_value time cycles after the
     implication time and the imp signal is sampled imp_delay_value time
     cycles after the implication time.
   . If imp_delay_value >= 0, then the consequent is of the
     following form:
```

```
        ( $past(spec_signal, imp_delay_value) == imp_signal )
      . The whole implication condition is valid only if, at implication time,
        {disable_cond_expr} is false.
      . The time frames are counted according to the {clocking_event}.

    Logical Context for Clocks
    --------------------------
    Use the following switches to specify logical context for new
    mappings on clock signals.
    . -map_condition
    . -edge

    Important things to know:
    . These clock signals need a specific mapping switch due to the SVA
      semantics of clock signal values inside properties. To enable checking
      the clock signal under a specific condition, we must check the
      equivalence of the @({edge} clock_signal) event of the spec and imp
      clocks when the map_condition is met.
    . A clocking signal is a signal that is combinationally derived from
      user-defined clock signals.
    . The tool does not accept "-edge" without "-map_condition".
      However, you can achieve the same effect with "-map_condition 1".
    . If you do not specify "-edge", the tool provides a value.
    . The {map_cond_expr} condition expression is a non-temporal valid SVA
      expression, and it contains signals from both sides (spec and imp).

    Manual Mapping with a Map File
    ------------------------------
    Use "-map_file" to run a user-defined Tcl file that specifies mapping.

Signal Unmapping
----------------
-unmap
        ( -id mapping_id_list
         |-dict mapping_dict_list
         |-spec {spec_regexp_or_signal_list}
         |-imp {imp_regexp_or_signal_list}
        )
           [-regexp [-verbose]]
           [-mapping_type (  connection_all | connection_primary_input
                           | connection_bbox_output | connection_undriven
                           | connection_stopat | connection_misc
                           | ver_target_all | ver_target_primary_output
```

```
                                  | ver_target_mpram_port | ver_target_bbox_input
                                  | ver_target_misc | cutpoint_all | cutpoint_both
                                  | cutpoint_spec | cutpoint_imp | cutpoint_both_init
                                  | stopat_all | abstract_reset_value | abstract_counter
| abstract_init_value
                                  | stopat_both | stopat_spec  | stopat_imp |
stopat_both_env
                                  | helper | init_user | init_auto | init_all |
clock_gating_toggle_coverage) ]
           [  (-instance instance_full_path)
             |(-of_signal {signal_list}
                 [ -spec_regexp [-verbose]
                  |-imp_regexp [-verbose] ] )
           ]
           [-proof_status (proven | cex | unprocessed | undetermined)+]
           [-signal_type (  input | output | primary_input | primary_output
                                  | dut_input | dut_output
                                  | bbox_input | bbox_output | user_bbox_input
                                  | user_bbox_output | auto_bbox_input |
auto_bbox_output
                                  | mpram_port | undriven | internal_undriven
                                  | flop | latch | internal | stopat | reset_x
                                  | x_assignment | x_default_assignment |
x_divide_by_zero
                                  | x_index_out_of_range | x_misc_undriven
                                  | x_bus_contention | x_bus_floating | gated_clock)]
           [-tag {tag_name}]
           [(-task task_name [-include_global_mapping])
            |-global
           ]
```

Cancel a specified mapping pair or all mappings that involve signals that match spec_regexp_or_signal_list or imp_regexp_or_signal_list.
The tool also cancels all properties that might have been generated from these mappings.

- Use "-id" with a list of IDs for the mapping pairs you want to cancel.
- Use "-dict" with a list of mapping dictionaries for the mapping pairs you want to cancel. See the section on "Mapping Dictionary" for more info.
- Use "-spec" to define the set of signal name(s) on the spec side that you want to unmap.
- Use "-imp" to define the set of signal name(s) on the imp side that you want to unmap.
  - Use "-regexp" to specify that the "-spec" or "-imp" argument list is a regular expression list; otherwise, it is a list of specific names

```
        of signals, bits, or bit-vector expressions. Include the "-verbose"
        switch to print info messages detailing the regexp matching operation.
      . Use "-signal_type" to unmap only signals of the specified type.
        You can only use this switch with "-spec" or "-imp" and "-regexp".
        That is, if you specify mapping IDs, all types are unmapped.
      . Use "-mapping_type" to unmap the mapping pairs of the specified mapping
        type.
      . Use "-of_signal" to define the set of signal names whose mappings you
        want to unmap.
        . Use "-spec_regexp" or "-imp_regexp" to specify that the "-of_signal"
          argument is a regular expression list on the spec or imp side. The
          default is spec side; otherwise, it is a list of specific names of
          signals, bits, or bit-vector expressions.
        . Use "-verbose" to print info messages detailing the regexp matching
          operations.
      . Use "-instance" to specify that you want to unmap mapping pairs of
        signals that are in the specified instance.
      . Use "-proof_status" to unmap mapping entries according to their proof
        status. This switch accepts one or more of the following arguments:
        "proven", "cex", "unprocessed", "undetermined".
      . Use "-tag" to unmap only mappings with the specified tag.
    . Use "-task" to unmap the specified task's mapping instead of the current
      task's mapping.
    . Use "-global" to unmap exposed global mapping in all tasks.
    . Use "-include_global_mapping" to unmap both the global and task-specific
      mapping pair(s).

Hierarchy Unmapping
-------------------
-unmap -hierarchy -id hierarchy_aliasing_rule_id_list
    Use "-hierarchy" to remove hierarchy aliasing rules, and use "-id" with a
    list of IDs for the hierarchy aliasing rules you want to cancel.

AutoProve Unmapping
-------------------
-unmap -autoprove_mappings [-task task_name]
    Unmap all mappings created by SEC AutoProve.

    Use "-task" to unmap the specified task's mapping instead of the current
    task's mapping.
```

```
Controlling Automatic Init Mapping
----------------------------------
-auto_map_reset_x_values (on | off)
    [-map_one_side_reset_x (on | off)]
    [-task task_name]
    [-exclude_module {regexp_or_module_list} [-module_regexp]]
    [-exclude_instance {regexp_or_instance_list} [-instance_regexp]]

    Turn on or off the automatic mapping of pairs of uninitialized signals on
    the current task.

    Use "on" to instruct SEC to automatically map flop and latch pairs that are
    nonresettable or uninitialized. Auto init mapping is completely automatic,
    which means it adds or removes init mappings when the reset changes.
    The only control you have over auto init mapping is to turn it on or off.

    . Use "-map_one_side_reset_x" with "on" if you would also like to
      automatically map flops and latches that are nonresettable or
      uninitialized on just one side (spec or imp). If you do not use this
      switch, the default is "off".
    . Use "-task" to specify a task other than the current task.
    . Use "-exclude_module" to instruct SEC to exclude uninitialized registers
      inside module(s) and use "-module_regexp" to specify that the
      "-exclude_module" argument list is a regular expression list.
    . Use "-exclude_instance" to instruct SEC to exclude uninitialized
      registers inside instance(s) and use "-instance_regexp" to specify that
      the "-exclude_instance" argument list is a regular expression list.



Controlling Automatic Clock Gating Toggle Coverage Mapping
----------------------------------------------------------
-auto_map_clock_gating_toggle_coverage (on | off)
          [-task task_name]

    Enable or disable the automatic mapping of clock gating toggle coverage
    on the current task.

    The actual mappings are first generated when "check_sec -interface"
    or "check_sec -prove" is run, not during "check_sec -setup".  The
    option might therefore be set at any point before
```

"check_sec -interface" or "check_sec -prove" — it does not need to precede "check_sec -setup".

Changing the setting after mappings have already been generated has the following effects:

. Turning it off removes all automatically created clock gating toggle coverage mappings and marks the task as not ready for proof. User-created toggle coverage mappings are not affected.
. Turning it back on re-creates the automatic mappings on the next "check_sec -interface" or "check_sec -prove".

Once created, the individual mappings can be enabled or disabled at any point using "check_sec -enable_mapping" or "check_sec -disable_mapping", like any other mapping.

The command fails with an error if a proof is currently running in the background on the task.

. Use "-task" to specify a task other than the current task.

Ungating Clocks

```
-ungate [-task task_name |-env]
       (-auto [-spec |-imp |-both]
           | gated_clock_tcl_list
           |-remove gated_clock_tcl_list
           |-list [-status] [-silent]
           |-clear )
```

Enable the modeling of gated clocks as ungated clocks.
The ungate command creates a replace driver abstraction where the target is the gated clock signal and the driver is its non-gated clock source.

. Use "gated_clock_tcl_list" to specify gated clock signals to ungate.
. Use "-auto" to automatically detect and ungate gated clocks on the specified side (spec, imp, or both). Default is spec if no side specified.
. Use "-remove" to remove previously added ungated clock abstractions.
. Use "-list" to list all ungated clocks on the specified task. Use "-status" with "-list" to show the driving clock, source (user/auto), exposure status, and violation reason if exist.

```
    . Use "-clear" to remove all ungated clock abstractions from the task.
    . Use "-task" to specify the task to perform the ungate on.
      Without specifying explicit task, the ungate will be performed on the
      current task.
    . Use "-env" to perform the ungate at the env (global) level instead of
      on a specific task, and also during reset. Cannot be combined
      with "-task".
    . Use "-silent" with "-list" to get only the returned dictionary.

    Return value:
    . "-ungate" with "-task", "-env", or "-auto" returns an integer count
      of ungated clock signals that were created.
    . "-ungate -remove" and "-ungate -clear" return an integer count of ungated
      clock signals that were removed.
    . "-ungate -list" returns a list of signal names, or a dictionary
      (keyed by signal name) when "-status" is specified.

Splitting Mapping
-----------------
-split_mapping -id mapping_id_list
                | -dict mapping_dict_list [-force]
               -max_word_size max_word_size_value
    Break the specified mappings into small mapping entries with a bit size up
    to the specified vector size threshold. This command returns the mapping
    IDs of the newly generated mappings together with the IDs of unchanged
    mappings for those whose size does not exceed the specified threshold.
    . Use "-id" with a list of IDs for the mapping pairs you want to split.
    . Or use "-dict" with a list of mapping dictionaries for the mapping
      pairs you want to split. See the section on "Mapping Dictionary" for
      more info.
    . Use "-max_word_size" to specify a maximum size for mapping entries.

Copying Mapping
---------------
-copy_mapping (-source_task source_task |-id mapping_id_list)
              [-target_task target_task]
    Copy mapping entries to the target task.
    Use the switch "-source_task" to copy all mapping entries from
    "source_task" to "target_task".
    Use the switch "-id" to copy all provided task-based mapping IDs to
    "target_task".

    NOTES:
```

```
    . If you do not specify a target task, the tool copies mapping entries
      to the current task.
    . Provided global mapping IDs, invalid mapping IDs, and mapping IDs from
      target task will be ignored.
    . Providing a mapping ID in the list that exists in a task with
      "auto_map_reset_x_values" on also turns on the "auto_map_reset_x_values"
      for the target task. This applies to "map_one_side_reset_x" as well.
    . Auto init mappings will not be copied to the task. However, all auto init
      mappings will be generated in the "check_sec -interface" step
      even if the list provided includes only one init mapping ID.

Copying Imported Properties
---------------------------
-copy_imported_properties (-source_task source_task |-property property_name_tcl_list
[-regexp])
              [-target_task target_task]
    Copy imported properties to the target task:
    . Use the "-source_task" switch to copy all the imported properties in the
      source task to the target task.
    . Use the "-property" switch to copy specified imported properties to the
      target task.

    NOTES:
    . If you do not specify a target task, the tool copies the imported
      properties to the current task.
    . Provided non-imported properties will be filtered out.
    . Properties' related covers will be copied automatically.
    . You cannot copy a property's related cover without the property.
    . You cannot copy a property with a name similar to a property name in the
      target task.
    . Copying properties from different tasks with the same name copies the
      first property with that name only and only if no property with that name
      already exists in the target task.

Setting Mapping Context
-----------------------
-set_context_of_mapping
         (   -id mapping_id_list
          |  -dict mapping_dict_list [-force]
          | ( -spec {spec_regexp_or_signal_list}
              |-imp {imp_regexp_or_signal_list})
                [-regexp [-verbose]]
                [-signal_type (  input | output | primary_input | primary_output
```

```
                                | dut_input | dut_output
                                | bbox_input | bbox_output | user_bbox_input
                                | user_bbox_output | auto_bbox_input | auto_bbox_output
                                | mpram_port | undriven | internal_undriven
                                | flop | latch | internal | stopat | reset_x
                                | x_assignment | x_default_assignment | x_divide_by_zero
                                | x_index_out_of_range | x_misc_undriven
                                | x_bus_contention | x_bus_floating | gated_clock)]
                [-mapping_type (  connection_all | connection_primary_input
                                | connection_bbox_output | connection_undriven
                                | connection_stopat | connection_misc
                                | ver_target_all | ver_target_primary_output
                                | ver_target_mpram_port | ver_target_bbox_input
                                | ver_target_misc | cutpoint_all | cutpoint_both
                                | cutpoint_spec | cutpoint_imp | cutpoint_both_init
                                | stopat_all | abstract_reset_value | abstract_counter
| abstract_init_value
                                | stopat_both | stopat_spec  | stopat_imp |
stopat_both_env
                                | helper | init_user | init_auto | init_all) ]
                [  ( -instance {instance_full_path} )
                  |( -of_signal {regexp_or_signal_list}
                     [ -spec_regexp [-verbose]
                      |-imp_regexp [-verbose] ] )
                ]
                [-proof_status (proven | cex | unprocessed | undetermined)+]
                [-tag {tag_name}]
                [([-task {task_name}] [-include_global_mapping]) |-global]
          |-set_default
          |-clear_default
         )
         [-spec_delay spec_delay_value]
         [-imp_delay imp_delay_value]
         [-map_condition {map_cond_expr}]
         [-clock {clocking_event}]
         [-disable {disable_cond_expr}]
         [-edge (posedge | negedge | both_edge)]
         [-target_bound {target_bound_value}]
         [-clear_target_bound]
```

Set or change the temporal and logical context of a set of mappings.
. Use "-id" with a list of IDs for the mapping pairs whose context you
  want to change.

```
. Use "-dict" with a list of mapping dictionaries for the mapping
  pairs whose context you want to change. See the section on
  "Mapping Dictionary" for more info.
. Use "-spec" to define the set of signal names on the spec side whose
  mapping context you want to change.
. Use "imp" to define the set of signal names on the imp side whose
  mapping context you want to change.
  . Use "-regexp" to specify that the "-spec" or "-imp" argument list
    is a regular expression list; otherwise, it is a list of specific names
    of signals, bits, or bit-vector expressions. Include the "-verbose"
    switch to print info messages detailing the regexp matching operation.
  . Use "-signal_type" to set the context only for mappings of signals of
    the specified type.
    You can only use this switch with "-spec" or "-imp" and "-regexp".
    That is, if you specify mapping IDs, the context is set for all types.
  . Use "-mapping_type" to set the context only for mapping pairs of the
    specified mapping type.
  . Use "-of_signal" to define the set of signal names whose mappings you
    want to list.
    . Use "-spec_regexp" or "-imp_regexp" to specify that the "-of_signal"
      argument is a regular expression list on the spec or imp side.
      The default is spec side; otherwise, it is a list of specific names
      of signals, bits, or bit-vector expressions.
    . Use "-verbose" to print info messages detailing the regexp matching
      operations.
  . Use "-instance" to specify that you want to set the context only for
    mapping pairs of signals that are in the specified instance.
  . Use "-proof_status" to set the context for mapping entries according to
    their proof status.
    This switch accepts one or more of the following arguments:
    "proven", "cex", "unprocessed", "undetermined".
  . Use "-tag" to set the context only for mappings with the specified tag.
. Use "-task" to set context for the specified task's mapping instead of
  the current task's mapping.
. Use "-global" to set context for exposed global mapping in all tasks.
. Use "-include_global_mapping" to set context for both the global and
  task-specific mapping pair(s).
. Use "-set_default" to set the default context. Once you set the default
  context, every new mapping with no explicit context has the default
  context. To use the default context in the setup for automatic mapping,
  use "check_sec -setup... -setup_file...".
  NOTE: The default context applies to all mapping, including pairs of
  inputs and clock and reset signals.
```

```
    . Use "-clear_default" to clear the default context.
    . Use "-spec_delay", "-imp_delay", "-map_condition",
      "-clock", and "-disable" to specify temporal and logical context to the
      specified mappings. This set of switches generates various combinations
      of temporal and logical conditions on spec and imp equivalence. For
      usage, refer to the subsection "Temporal and Logical Context" in the
      section "Signal Mapping".
    . Use "-map_condition" and "-edge" to specify the logical context for the
      specified mappings on clock signals. For usage, refer to the subsection
      "Logical Context for Clocks" in the section "Signal Mapping".
    . Use the "-target_bound" switch to specify the target bound value
      for the generated mapping properties. By default, the target bound is
      set to 0.
      NOTE: This switch is only applicable when using provable mapping types.
      . Value 0 means an infinite target bound, which is equivalent to
        using "-clear_target_bound".
      . The target bound set with this command takes precedence over any bound
        specified with the "set_prove_target_bound" command.
      . When the min_length of the corresponding assertion is >= the target bound,
        its status changes to "Bounded Proven (User)".
    . Use the "-clear_target_bound" switch to clear the target bound value.

Setting Mapping Type
--------------------
-set_mapping_type -id mapping_id_list
                  |-dict mapping_dict_list [-force]
        [ -cut (both | both_init | spec | imp)
         |-abstract (reset_value | counter | init_value)
         |-stopat (both | spec | imp | both_env)
         |-helper
         |-cover_gated_clock_toggle]
        [-set_tag tag_name]

    Set or change the type and tag of a set of mappings.
    The tool will remove the existing properties of the changed mapping pairs,
    and replace them with new properties according to the new mapping type.
    . Use "-id" with a list of IDs for the mapping pairs whose type you want to
      change.
    . Or use "-dict" with a list of mapping dictionaries for the mapping pairs
      you want to change. See the section on "Mapping Dictionary" for more info.
    . Use "-cut" to set the mapping type to cutpoint.
      . Use "-cut both" to add cutpoints on the signals on both sides
        without the concrete value init assumption.
```

```
      . Use "-cut both_init" to add cutpoints on the signals on both sides.
      . Use "-cut spec" to add a cutpoint only on the signal on the spec side.
      . Use "-cut imp" to add a cutpoint only on the signal on the imp side.
    . Use "-abstract" to create a Jasper abstraction on both the spec and
      imp designs.
      . Use "-abstract reset_value" to create a reset value abstraction on both
        the spec and imp and apply it on the current task.
      . Use "-abstract counter" to create a counter abstraction on both the
        spec and imp.
      . Use "-abstract init_value" to create an init abstraction on both the
        spec and imp.
    . Use "-stopat" to set the mapping type to stopat.
      . Use "-stopat both" to add stopats on the signals on both sides.
      . Use "-stopat spec" to add a stopat only on the signal on the spec side.
      . Use "-stopat imp" to add a stopat only on the signal on the imp side.
      . Use "-stopat both_env" to add stopats that are respected during reset on
        the signals on both sides.
    . Use "-helper" to set the mapping type to helper.
    . Use "-cover_gated_clock_toggle" to set the mapping type to clock gating
      toggle coverage.
    . Use "-set_tag" to set a new tag for the mapping pairs.

    NOTE: The command "check_sec -set_mapping_type -id mapping_id_list" without
    any other switches sets the mapping_type based on the signal attributes.

Enabling and Disabling Mapping
------------------------------------------
-enable_mapping |-disable_mapping
    ( -id {mapping_id_list}
     |-dict {mapping_dict_list}
     | ( -spec {spec_regexp_or_signal_list}
        |-imp {imp_regexp_or_signal_list})
        [-regexp [-verbose]]
        [-mapping_type (  connection_all | connection_primary_input
                       | connection_bbox_output | connection_undriven
                       | connection_stopat | connection_misc
                       | ver_target_all | ver_target_primary_output
                       | ver_target_mpram_port | ver_target_bbox_input
                       | ver_target_misc | cutpoint_all | cutpoint_both
                       | cutpoint_spec | cutpoint_imp | cutpoint_both_init
                       | stopat_all | abstract_reset_value | abstract_counter |
abstract_init_value
                       | stopat_both | stopat_spec  | stopat_imp | stopat_both_env
```

```
                    | helper | init_user | init_auto | init_all |
clock_gating_toggle_coverage) ]
        [  ( -instance {instance_full_path} )
          |( -of_signal {signal_list}
             [ -spec_regexp [-verbose]
              |-imp_regexp [-verbose] ] )
        ]
        [-proof_status (proven | cex | unprocessed | undetermined)+]
        [-signal_type (  input | output | primary_input | primary_output
                       | dut_input | dut_output
                       | bbox_input | bbox_output | user_bbox_input
                       | user_bbox_output | auto_bbox_input | auto_bbox_output
                       | mpram_port | undriven | internal_undriven
                       | flop | latch | internal | stopat | reset_x
                       | x_assignment | x_default_assignment | x_divide_by_zero
                       | x_index_out_of_range | x_misc_undriven
                       | x_bus_contention | x_bus_floating | gated_clock)]
        [-tag {tag_name}]
        [([-task {task_name}] [-include_global_mapping]) | -global]
    )

    Enable or disable specific mappings.
    If you enable or disable mappings, their properties are enabled or
    disabled accordingly.
    . Use "-dict" to specify the list of mapping dictionaries of the specific
      mapping pairs you want to enable or disable. See the section on
      "Mapping Dictionary" for more info.
    . Use "-id" to specify the list of IDs of the specific mapping pairs you
      want to enable or disable.
    . Use "-spec" instead of IDs to define the set of signal name(s) on the
      spec side whose mappings you want to enable or disable.
    . Use "-imp" instead of IDs to define the set of signal name(s) on the imp
      side whose mappings you want to enable or disable.
      . Use "-regexp" to specify that the "-spec" or "-imp" argument list
        is a regular expression list; otherwise, it is a list of specific names
        of signals, bits, or bit-vector expressions. Include the "-verbose"
        switch to print info messages detailing the regexp matching operation.
      . Use "-signal_type" to enable or disable only mappings of signals of the
        specified type.
        You can only use this switch with "-spec" or "-imp" and "-regexp".
        That is, if you specify mapping IDs, all types are enabled or disabled.
      . Use "-mapping_type" to enable or disable only mapping pairs of the
        specified mapping type.
```

```
      . Use "-of_signal" to define the set of signal names whose mappings you
        want to list.
        . Use "-spec_regexp" or "-imp_regexp" to specify that the "-of_signal"
          argument is a regular expression list on the spec or imp side.
          The default is spec side; otherwise, it is a list of specific names
          of signals, bits, or bit-vector expressions.
        . Use "-verbose" to print info messages detailing the regexp matching
          operations.
      . Use "-instance" to specify that you want to enable or disable only
        mapping pairs of signals that are in the specified instance.
      . Use "-proof_status" to enable or disable mapping entries according to
        their proof status.
        This switch accepts one or more of the following arguments:
        "proven", "cex", "unprocessed", "undetermined".
      . Use "-tag" to enable or disable only mappings with the specified tag.
    . Use "-task" to enable or disable the specified task's mappings instead of
      the current task's mappings.
    . Use "-global" to enable or disable the global mappings.
    . Use "-include_global_mapping" to enable or disable both the global and
      the task-specific mapping pair(s).



Waiving Signals
---------------
-waive ( -waive_signals {signals list} [-exclude_init_mapping]
           [-only_init_mapping] [-description description]
         |-unwaive_signals {signals list}
         |-list_waived [-of_description description] [-verbose] )

    Manage waivers on signals.

    NOTE: The tool does not map waived signals and the signoff command does not
    report them.

    . Use "-waive_signals" to waive a list of signals. This command removes any
      existing mapping from these signals.

    . Use "-exclude_init_mapping" to apply waivers to all mapping types except
      init mappings. The tool will not create any mappings on the waived
      signals (and will remove existing mappings), except for init mappings.

      NOTE: The tool might still create init mappings on specified signals and
```

```
      it will not remove existing ones.

    . Use "-only_init_mapping" to apply waivers only when dealing with init
      mappings.

      NOTE: The tool might still map the waived signals, but it will not create
      init mapping on the specified signals (and it will remove any existing
      init mappings on the signals).

      Use "-description" to add a description of the waiver.

    . Use "-unwaive_signals" to unwaive a list of signals. The tool does not
      restore any previous mapping.

    . Use "-list_waived" to list all waived signals.
      . Use "-of_description" to list all waived signals with a specified
        description.
      . Use "-verbose" to see the descriptions for the waivers.

Get Twin Signal
---------------
-get_twin_of_signal {signals_list} [-regexp]

    Get the twin of a specific signal or list of signals.
    The twin of a signal can be from the existing mapping or name matching.
    Existing mapping takes precedence.

    Use "-regexp" to specify that the signal or signals_list is a regular
    expression; otherwise, it is the specific name of a signal.

Undriven
--------
-set_auto_mapped_undriven_value (0 | 1)

 Sets the value of the undriven mapping to a constant, 0 or 1, for
 either an existing undriven mapping or for future undriven mapping.



Prove
-----
-prove
    [ -task task_name
```

```
     |-property property_name_tcl_list [-regexp]
     | (-mapping_id {mapping_id_list}
     | -dict {mapping_dict_list} [-force]
     | ( -spec {spec_regexp_or_signal_list}
        |-imp {imp_regexp_or_signal_list})
       [-regexp [-verbose]])
           ([-signal_type (  input | output | primary_input | primary_output
                           | dut_input | dut_output
                           | bbox_input | bbox_output | user_bbox_input
                           | user_bbox_output | auto_bbox_input | auto_bbox_output
                           | mpram_port | undriven | internal_undriven
                           | flop | latch | internal | stopat | reset_x
                           | x_assignment | x_default_assignment | x_divide_by_zero
                           | x_index_out_of_range | x_misc_undriven
                           | x_bus_contention | x_bus_floating | gated_clock)]
           [-mapping_type (  connection_all | connection_primary_input
                            | connection_bbox_output | connection_undriven
                            | connection_stopat | connection_misc
                            | ver_target_all | ver_target_primary_output
                            | ver_target_mpram_port | ver_target_bbox_input
                            | ver_target_misc | cutpoint_all | cutpoint_both
                            | cutpoint_spec | cutpoint_imp | cutpoint_both_init
                            | stopat_all | abstract_reset_value | abstract_counter |
abstract_init_value
                            | stopat_both | stopat_spec | stopat_imp | stopat_both_env
                            | helper | init_user | init_auto | init_all |
clock_gating_toggle_coverage) ]
           [-include_global_mapping])
           [  ( -instance {instance_full_path} )
               |( -of_signal {signal_list}
                  [ -spec_regexp [-verbose]
                   |-imp_regexp [-verbose] ] )
           ]
           [-proof_status (proven | cex | unprocessed | undetermined)+]
           [-tag tag_name_tcl_list]
    ]
    [-time_limit time_budget]
    [-cex_limit number_of_failures]
    [-failure_extension]
    [-bg]
    [-silent |-verbosity N]
    [-with_helpers]
    [-with_proven]
```

```
Try to prove all SEC properties that were generated by SEC.
Return values are the same as the "prove" command except if the
proof stops on "cex_limit", it returns "cex_threshold_reached".
Each proof strategy defines which properties are considered when
calculating the return value.
For more info, see return value section for each proof strategy.

. SEC provable properties are:
    . SEC properties from the task-specific and global mapping pairs.
    . Imported properties and their related covers.
    . Assumptions' related covers.
. Use "-task" to prove SEC properties in the specified task.
  If you do not specify a task, this command applies to the current task.
  In both cases, it runs on SEC provable properties.
. Use "-property" to specify the names of the properties you want to prove.
  These properties must be SEC provable properties. By default, the
  "check_sec -prove" subcommand attempts to prove all the SEC provable
  properties in the task.
. Use "-mapping_id" to generate properties only from mapping pairs of
  the specified IDs.
. Use "-dict" to prove properties only from mapping pairs of the
  specified mapping dictionaries. See the section on "Mapping Dictionary"
  for more info.
. Use "-spec" instead of IDs to define the set of signal name(s) on the
  spec side whose properties you want to generate.
. Use "-imp" instead of IDs to define the set of signal name(s) on the imp
  side whose properties you want to generate.
  . Use "-regexp" to specify that the "-spec" or "-imp" argument list
    is a regular expression list; otherwise, it is a list of specific names
    of signals, bits, or bit-vector expressions. Use the "-verbose"
    switch to print info messages detailing the regexp matching operation.
  . Use "-signal_type" to generate properties only for mappings of signals
    of the specified type. You can only use this switch with "-spec" or
    "-imp" and "-regexp". That is, if you specify mapping IDs, the tool
    generates properties for all types.
  . Use "-mapping_type" to prove only mapping pairs of the specified
    mapping type.
  . Use "-instance" to specify that you want to prove only mapping  pairs
    of signals that are in the specified instance.
  . Use "-of_signal" to define the set of signal names whose mappings you
    want to prove.
```

```
      . Use "-spec_regexp" or "-imp_regexp" to specify that the "-of_signal"
        argument is a regular expression list on the spec or imp side.
        The default is spec side; otherwise, it is a list of specific names
        of signals, bits, or bit-vector expressions.
      . Use "-verbose" to print info messages detailing the regexp matching
        operations.
    . Use "-proof_status" to prove mapping entries according to their proof
      status.
      This switch accepts one or more of the following arguments:
      "proven", "cex", "unprocessed", "undetermined".
    . Use "-tag" to generate properties only from mappings with the specified
      tags.
    . A "clock_tree_node" is a signal automatically identified by
      the tool to be in the clock tree with counterpart of the same
      name in the clock tree on the other side.
  . Use "-time_limit" to override the default two-hour "prove_time_limit" for
    "autoprove".
    Specify "time_budget" values as an integer followed by s, m, or h (no
    spaces).
    . s = seconds
    . m = minutes
    . h = hours
  . Use "-cex_limit" to specify that you want the prove flow to stop
    the proof job after it reaches a certain threshold of failures on SEC
    properties. If the prove flow stops on cex_limit, it returns
    "cex_threshold_reached".
  . Use "-failure_extension" to look for bugs with a dedicated search
    strategy running in parallel with the proof-oriented strategy. This
    dedicated strategy uses current failures to find deep bugs. It cannot
    prove that the designs are equivalent.
  . Use "-bg" to run the command in the background and continue to accept and
    run new commands.
  . Use "-silent" for "-verbosity 0" (see below).
  . Use "-verbosity N" to override the current "prove_verbosity" for this
    "-prove" command. See "set_prove_verbosity" for a summary of the
    verbosity levels. See "set_sec_show_proof_log" to specify
    whether you want the SEC App to show the proof log.
  . Use "-with_helpers" to prove with helper assertions.
  . Use "-with_proven" to keep already proven properties in the same task and
    use them as assumptions.


NOTE:
```

```
    . Use "-include_global_mapping" to include both the global and the
      task-specific mapping pair(s) in the prove.
    . You can run multiple SEC proofs in the background only for different tasks.
    . SEC can infer the status of a precondition from the status of its main
      property and vice-versa.
      . A counterexample on the main property means that the precondition is
        covered.
      . If a precondition is unreachable, the main property is vacuously proven.
      . If a new bound was found for a precondition, the main property's bound
        may be increased to match it.
      In case of inference, the indication "(Inferred)" appears immediately after
      the engine name, for example:
      "INFO (ISEC106): (0.basic) SEC basic strategy (B)(Inferred): A min_length
      bound of 252 was found for the property "prop" in 4.14 s."


-prove -status [-thread thread_id]
    Use this command to get the SEC proof status.

    Use "-thread" to list the proof status for the specified SEC thread.

    NOTE:
    . The command is non-blocking, that is, proof activity does not affect it.
      It returns immediate results.
    . "check_sec -prove -status" lists the status for all running proof
      threads. The tool prefixes statuses with the associated thread ID.

    Below is the list of possible return values:
    . "NeverStarted": Nothing has run in this session.
    . "Running": SEC main proof thread is running.
    . "Completed": Proof thread is not currently running. It was previously
      run.
    . "Error": SEC main proof thread ended with an error.
    . "NotSecThread": Proof thread is unrelated to an SEC proof.
    . "InternalSecThread": Proof thread is running. It was launched
                           automatically by an SEC main proof thread.

No Auto Prove
-------------
-prove -no_auto
       [-suppress_traces]
    Run the main proof flow with user-specified settings instead of the
    "autoprove" settings on all proof targets.
```

```
    Return values are the same as the "prove" command.
    For details on all other possible return values, use "help prove -gui".

    . Use "-suppress_traces" to suppress trace generation.

Advanced Prove Strategies
-------------------------
Basic prove:
-prove -strategy basic
       [-failure_extension]
    Run a proof with SEC-oriented proof settings on all proof targets.

    Return values are the same as the "prove" command.
    For details on all other possible return values, use "help prove -gui".

    With this proof strategy, the tool does not add helpers or cutpoints.

    Use "-failure_extension" to help the basic strategy find deep
    counterexamples by extending counterexamples it already identified.

    NOTE: Use "set_engine_mode" to override the engine mode for this proof.
    Jasper respects the "set_engine_mode" command only if both
    "set_prove_orchestration" and "set_proofmaster" are off.

Prove with Cutpoint Refinement:
-prove -strategy proof
       [-property property_name_tcl_list [-regexp]]
    Run a proof with automatically applied cutpoints on flops and internal
    signals of the design, using cutpoint refinement loop handling.

    Return values are the same as the "prove" command. But notice that
    AutoProve auto-generated helpers are not considered in the return value of
    the proof, while AutoProve auto-generated cutpoints and the specified
    verification targets are considered there. For details on all possible
    return values, use "help prove -gui".

    . Use "-property" to specify the names of the properties you want to prove.
      These properties have to be exposed SEC properties, but they cannot be
      AutoProve auto-generated properties since AutoProve cannot ensure that it
      will not remove or change them while running.
      By default, the "check_sec -prove" command attempts to prove all the
      provable properties in the task.
```

```
      This switch supports wildcards (* and ?). Use "-regexp" to specify that
      the argument is a regular expression.

State-matching prove:
-prove -strategy state_matching
    [-property property_name_tcl_list [-regexp]]

    Return values are the same as the "prove" command. But notice that
    AutoProve auto-generated helpers are not considered in the return value of
    the proof, while AutoProve auto-generated cutpoints and the specified
    verification targets are considered there. For details on all possible
    return values, use "help prove -gui".

    . Use "-property" to specify the names of the properties you want to prove.
      These properties must be exposed SEC properties, but they cannot be
      AutoProve auto-generated properties since AutoProve cannot ensure that it
      will refrain from removing or changing them while running.
      By default, the "check_sec -prove" command attempts to prove all the
      provable properties in the task.
      This switch supports wildcards (* and ?). Use "-regexp" to specify that
      the argument is a regular expression.

    NOTE:
    . Using "state_matching" might cause false negatives on verification targets.
    . "state_matching" can not be used with memory-aware proofs.

Clock-gating prove:
-prove -strategy clock_gating
    [-property property_name_tcl_list [-regexp]]

    Return values are the same as the "prove" command. But notice that
    AutoProve auto-generated helpers are not considered in the return value of
    the proof, while AutoProve auto-generated cutpoints and the specified
    verification targets are considered there. For details on all possible
    return values, use "help prove -gui".

    . Use "-property" to specify the names of the properties you want to prove.
      These properties must be exposed SEC properties, but they cannot be
      AutoProve auto-generated properties since AutoProve cannot ensure that it
      will refrain from removing or changing them while running.
      By default, the "check_sec -prove" command attempts to prove all the
      provable properties in the task.
      This switch supports wildcards (* and ?). Use "-regexp" to specify that
```

```
      the argument is a regular expression.


   After each advanced SEC autoprove strategy, run "check_sec -unmap
   -autoprove" to allow the next "autoprove" run to start from scratch.

Sign-Off Report
---------------
-signoff
    [-task task_name]
    [-file file_name [-json] [-force]]
    [-silent]
    [-summary]
    [-detailed]
    [-waive_category (  boundary_outputs_mapping | boundary_outputs_proof
                      | user_cutpoints | auto_cutpoints
                      | reset_value_abstraction | imported_assertions
                      | imported_covers | x_signals_and_undrivens
                      | mapped_stopats | assumptions_conflict
                      | assumptions_on_driven_signals
                      | assumptions_related_preconditions
                      | clock_gating | imp_assumptions
                      | loop_handling)+]
      [[-waive_boundary_outputs_mapping {port_hierarchical_name_list}]
       [-waive_boundary_outputs_proof {port_name_list}]
       [-waive_user_cutpoints_proof {assertion_name_list}]
       [-waive_auto_cutpoints_proof {assertion_name_list}]
       [-waive_imported_assertion_proof {assertion_name_list}]
       [-waive_imported_cover_proof {cover_name_list}]
       [-waive_reset_value_abstraction {assert_name_list}]
       [-waive_x_signals_and_undrivens {signal_name_list}]
       [-waive_mapped_stopats {mapping_pair_list}]
       [-waive_assumption_on_driven_signals {assumption_name_list}]
       [-waive_assumption_related_preconditions {precondition_name_list}]
       [-waive_imp_assumptions {assumption_name_list}]
       [-waive_unmapped_gated_clocks {signal_name_list}]
       [-waive_mapped_gated_clocks {cover_name_list}] [-regexp]]
      [-consider_bounded_proven_incomplete]
      [-get_valid_status [-summary] [-propagate_bbox_input_status_to_target] [-
consider_noConflict] [-no_cutpoint_status_propagation_to_target]]

    Write a report that summarizes the SEC proof status for a specific task.
```

```
The sign-off report highlights the possible risks to the validity of the
SEC proof. The risks are associated with a list of categories, where for
every category, the report sets a grade.
The grades are:
. Complete: This category poses no risk to the SEC proof.
. Complete (with waiver): The SEC proof is valid due to waiving data.
. Waived: This category has been waived. (Though it might contain data
  results in a different status, these will be ignored.)
. Incomplete: The SEC proof can be valid, but that depends on further
  action or user knowledge. For example, an SEC proof can be valid when you
  disable X-handling. However, the validity of the SEC proof depends on the
  assumption that Xs do not propagate to valid outputs in both spec and
  imp. You should check this assumption, possibly with the X-Prop App.
. Invalid: The SEC proof is invalid, and you must take further actions.
NOTE: The default output of the report includes the output provided when
calling "check_sec -signoff -get_valid_status -summary", and results for
the targets are reported and taken into consideration in the "output
primary bounds" report.
. Use "-task" to generate a sign-off report for a specified task. By default,
  the tool generates a report for the current task.
. Use "-file" and a file name to specify the file you want to write the
  report to. By default, the tool generates a text file.
  . Use "-json" to write the report in JSON format.
  . Use "-force" to overwrite any existing file that has the specified name.
. Use "-silent" to get a dictionary containing reported data.
. Use "-summary" to print only the "Signoff Summary" section of the report,
  while returning a dictionary representing the "Signoff Summary" report.
  . If you use "-get_valid_status", summary dictionary will contain
    valid status summary data, including the number of proven targets,
    number of CEX targets, number of undetermined, number of unprocessed,
    number of unreachable preconditions, number of undetermined
    preconditions, and number of unprocessed preconditions. In addition, if
    imported covers exist, the dictionary will include the following data:
    number of covered covers, unprocessed covers, undetermined covers,
    and unreachable covers.
. Use "-detailed" to get a detailed dictionary report that includes
  all categories with details. Proprties and signals sub-categories are
  included.
. Use "-waive_category boundary_outputs_mapping" to turn off the report for
  unmapped outputs and blackbox inputs.
. Use "-waive_category boundary_outputs_proof" to turn off the report for
  outputs and blackbox inputs.
. Use "-waive_category user_cutpoints" to turn off the report for user
```

```
  cutpoints.
. Use "-waive_category auto_cutpoints" to turn off the report for auto
  cutpoints.
. Use "-waive_category reset_value_abstraction" to turn off the report for
  reset value abstraction assert list.
. Use "-waive_category x_signals_and_undrivens" to turn off the report for
  undriven or X mapping sign-off failures.
  NOTE: Only use this switch for cases when the report makes the SEC proof
  incomplete or invalid.
. Use "-waive_category mapped_stopats" to turn off the report for mapped
  stopats.
. Use "-waive_category assumptions_on_driven_signals" to turn off the
  report showing assumptions on driven signals.
  NOTE: Assumptions on driven signals are assumptions that refer to signals
  that are not boundary inputs. These assumptions are considered risky.
  Assumptions on driven signals might impose overconstraint to the SEC
  model and provide false positive results.
  You must review all of these assumptions and decide whether including
  them is safe, and only then, use this waive check command.
  To get a list of the assumptions on driven signals, use
  "dict get [dict get [dict get [check_sec
  -signoff_info -get_category_status assumptions_on_driven_signals
  -detailed] "Assumption management"] "Assumptions on driven signals"]
  "assumptions"".
. Use "-waive_category assumptions_conflict" to turn off the report
  from "check_assumptions -conflict".
. Use "-waive_category assumptions_related_preconditions" to turn off the report
  for assumptions' related preconditions.
. Use "-waive_category imported_assertions" to turn off the report for
  imported assertions.
. Use "-waive_category imported_covers" to turn off the report for imported
  covers.
. Use "-waive_category clock_gating" to turn off the report for mapped
  gated-clock toggle covers.
. Use "-waive_category imp_assumptions" to turn off the report for
  imp assumptions.
. Use "-waive_boundary_outputs_mapping" to turn off the report for all
  outputs specified in the port name list.
  NOTE: Output port name and blackbox input should be specified as full
  hierarchy name.
. Use "-waive_boundary_outputs_proof" to turn off the report for all
  output properties specified in the list.
. Use "-waive_user_cutpoints_proof" to turn off the report for all
```

user cutpoint assertions specified in the list.
. Use "-waive_auto_cutpoints_proof" to turn off the report for all automatically generated assertions produced due to state_matching specified in the list.
. Use "-waive_imported_assertion_proof" to turn off the report for all assertions specified in the list.
. Use "-waive_imported_cover_proof" to turn off the report for all imported covers specified in the list.
. Use "-waive_reset_value_abstraction" to turn off the report for all reset value abstraction properties specified in the list.
. Use "-waive_x_signals_and_undrivens" to turn off the report for all x signals specified in the list.
. Use "-waive_mapped_stopats" to turn off the report for all mapping pairs specified in the list.
NOTE: Mapping pair format should follow the same format used in GUI for mapped stopat. You can drag and drop to console to get exact format.
. Use "-waive_assumption_on_driven_signals" to turn off the report for all assumptions specified in the list.
NOTE: You can waive properies only if the property is either an assumption with no preconditions or an assumption whose preconditions are covered by related covers.
. Use "-waive_assumption_related_preconditions" to turn off the report for all covers specified in the list.
. Use "-waive_imp_assumption" to turn off the report for all imp assumptions specified in the list.
. Use "-waive_unmapped_gated_clocks" to turn off the report for all signals specified in the list.
. Use "-waive_mapped_gated_clocks" to turn off the report for all covers specified in the list.
. Use "-regexp" to specify that the arguments of "-waive_boundary_outputs_mapping", "-waive_boundary_outputs_proof", "-waive_user_cutpoints_proof", "-waive_auto_cutpoints_proof", "-waive_imported_assertion_proof", "-waive_imported_cover_proof", "-waive_reset_value_abstraction", "-waive_assumption_on_driven_signals", "waive_unmapped_gated_clocks", "-waive_mapped_gated_clocks", and "-waive_assumption_related_preconditions", represent regular expressions.
. Use "-consider_bounded_proven_incomplete" to treat any bounded proven target as incomplete instead of complete.
. Use "-get_valid_status" to get a dictionary containing verification targets (targets on boundary output signals and imported properties) and their valid status - in addition to the regular signoff result.
If you use this switch, the returned dictionary will include, for each user-defined verification target, a dictionary with the following keys:

```
      {property name, engine, Time, mapping pair ID, valid status, bound } where valid
status
      is one of the following: {unprocessed, proven, cex, covered,
      unreachable-precondition, undetermined-precondition,
      unprocessed-precondition}. When the valid status is not proven
      or unreachable, the bound figure is added to the dictionary.
      The bound item contains the min length of a possible trace, and in some
      cases, the max length figure too.
      . Use "-propagate_bbox_input_status_to_target" to propagate black box
        input status to the boundary targets during sign-off reporting
        (valid-result calculation).
      . Use "-consider_noConflict" to include an evaluation of the bound of the
        :noConflict property when calculating the reported min length of the
        verification targets. That is, if the :noConflict min length is N, there
        will be no proven target and no target assertion with a bound greater
        than N.
      . Use "-no_cutpoint_status_propagation_to_target" to disable propagating
        user cutpoints status to the boundary targets during sign-off reporting
        (valid-result calculation).
      NOTE: If an assertion property has a precondition property, the
      signoff report includes data about one of the properties only. See the
      following table:
      +--------------------------+------------------------------+--------------------
--+
      | Assertion Property Status | Precondition Property Status | Reported Property
|
      +--------------------------+------------------------------+--------------------
--+
      | Proven                   | Covered                      | Assertion property
|
      | Proven                   | Uncovered                    | Precondition
property |
      | Cex                      | Any                          | Assertion property
|
      | Undetermined             | Unreachable                  | Precondition
property |
      | Undetermined             | NOT Unreachable              | Assertion property
|
      | Unprocessed              | Unreachable                  | Precondition
property |
      | Unprocessed              | NOT Unreachable              | Assertion property
|
      +--------------------------+------------------------------+--------------------
```

```
--+

-signoff_info
    [-task task_name]
    [-file file_name [-json] [-force]]
    [-silent]
    [-detailed]
    -get_category_status (  boundary_outputs_mapping | boundary_outputs_proof
                          | imported_assertions | imported_covers
                          | reset_value_abstraction | x_signals_and_undrivens
                          | mapped_stopats | assumptions_conflict
                          | assumptions_on_driven_signals |
assumptions_related_preconditions
                          | clock_gating | imp_assumptions | user_cutpoints
                          | auto_cutpoints | loop_handling)

    Get the specified sign-off info.
    NOTE: Use this command only after "check_sec -signoff".

    . Use "-get_category_status" to get a Tcl dictionary representing the
      specified category data including category status and description.
    . Use "-task" to generate a sign-off report for a specified task. By default,
      the tool generates a report for the current task.
    . Use "-file" and a file name to specify the file you want to write the
      report to. By default, the tool generates a text file.
      . Use "-force" to overwrite any existing file that has the specified name.
      . Use "-json" to write the report in JSON format.
    . Use "-silent" to get only the returned dictionary.
    . Use "-detailed" to get a detailed dictionary including all category data
      (including property and signal names).

Proof Utility
-------------
-get_needed_constraints -property property_name
    [-task task_name]
    [-attempts number_of_prove_attempts]
    [-attempts_time_limit time_limit]
    [-include_cutpoint_mappings]
    [-include_init_mappings]
    [-include_mappings_of_undriven_signals]
    [-include_primary_input_mappings]
    [-include_stopat_mappings]
    [-include_user_env_assumptions]
```

```
[-include_user_liveness_assumptions]
[-required_assumptions assumptions_tcl_list]
[-silent]
[-status target_status_tcl_list]
[-stop_status stop_status_tcl_list]
[-user_assumptions assumptions_tcl_list]
```

Systematically disable and enable assumptions and mappings to eventually find a minimal set of constraints needed to achieve the target proof status on the target property.

The command creates a task called NDD_CONSTRAINTS and copies the current task's user assumptions, mappings, and target property to the new task. Then, it iteratively uses bisection of the set of assumptions and mappings to find the needed constraints.

The command returns a tuple consisting of four elements: status, the number of prove attempts/iteration, a set of needed assumptions, and a set of needed mappings.

Interpret the status as follows:

Status	Definition
completed	A minimal set of constraints has been found.
attempts_limit	The command was stopped because the attempts limit was reached. The set of constraints returned is an overapproximation of the minimal needed set.
attempts_time_limit	The command was stopped because the attempts time limit was reached. In this case, none of the other elements is returned.
stopped_by_user	The command was stopped by the user. The set of constraints returned is an overapproximation of the minimal needed set.
stopped_by_status	The command was stopped because the status specified by "-stop_status" was reached.
aborted	The command stopped due to an error. In this case, none of the other elements is returned.

. Use "-task" to get needed constraints for the specified task instead of the current task.
. Use "-attempts" to stop after the specified number of proof attempts. The default is 0 (unlimited).
. Use "-attempts_time_limit" to limit time per iteration, measured in wall clock time. The time limit is a positive integer that takes an optional trailing unit: "s" for seconds, "m" for minutes, "h" for hours, or "d" for days. If you do not specify a unit, the unit is seconds. The default is 120s.
. Use "-property" to find the needed constraints for the specified property.
. Use "-required_assumptions" to include the specified assumptions in the set of needed constraints. The tool assumes the specified assumptions are needed and includes them in the result.
  . NOTE: The summary report includes the specified assumptions in a "required by user" field, which is separated from all the other needed assumptions.
. Use "-silent" to get the final result only and not print messages to the log and not report progress and results in the console.
. Use "-status" to try to achieve one of the specified statuses. If you do not specify a status and the property has been processed and has a proof result, the tool uses that status. If it does not have a proof result, the default is "proven" for assertions and "unreachable" for covers.
. Use "-stop_status" to stop immediately when the tool reaches one of the specified statuses. The resulting list of constraints are the assumptions and mappings used to reach that status, and they are not necessarily a minimal set.
. Use "-user_assumptions" to use the specified user assumptions only. The default is to use all enabled user assumptions.
. By default, the tool adds all connection_misc mappings to the starting set that will undergo bisection to find the minimal set of needed constraints.
  To include additional mappings and assumptions to this starting set, use the following switches:
  . Use "-include_cutpoint_mappings" to include cutpoint mappings.
  . Use "-include_init_mappings" to include init mappings.
  . Use "-include_mappings_of_undriven_signals" to include mappings of undriven signals.
  . Use "-include_primary_input_mappings" to include primary input mappings.
  . Use "-include_stopat_mappings" to include stopat mappings.

```
      . Use "-include_user_env_assumptions" to include assumptions in the
        global environment. This switch invalidates all proof results in all
        tasks.
      . Use "-include_user_liveness_assumptions" to consider liveness
        assumptions for safety properties and finite covers. By default, the
        tool does not consider liveness assumptions when the specified property
        is a safety or finite property.

Utility Switches
----------------
-get_spec_top_instance |-get_imp_top_instance
    Return the name of the spec or imp top instance.

-import_property property_name_tcl_list [-regexp]
    Import non-SEC properties to the SEC App for proof with "check_sec -prove".

    Use "-regexp" to specify that the argument is a regular expression.
```

Example

```
# Example for using the Split Setup flow for both spec and imp:
#-------------------------------------------------------------
check_sec -clear all
check_sec -analyze -both -sv -f model.vfile
check_sec -elaborate -both -top top
check_sec -setup

# Example for using the Split Setup flow for different spec and imp:
#-------------------------------------------------------------------
check_sec -clear all
check_sec -analyze -spec -sv -f modelA.vfile
check_sec -elaborate -spec modelA
check_sec -analyze -imp -sv -f modelB.vfile
check_sec -elaborate -imp -top modelB
check_sec -setup
# change only the imp side
check_sec -setup -clear
check_sec -elaborate -imp -clear
check_sec -analyze -imp -sv -f modelA.vfile
check_sec -elaborate -imp -top modelA
check_sec -setup
```

```
# Example for reuse of existing FPV script:
-----------------------------------------
check_sec -compile_context spec
analyze -sv -f modelA.vfile
# The tool runs "check_sec -analyze -spec -sv -f modelA.vfile"
elaborate -top modelA
# The tool runs "check_sec -elaborate -spec modelA"
check_sec -compile_context imp
analyze -sv -f modelB.vfile
# The tool runs "check_sec -analyze -imp -sv -f modelB.vfile"
elaborate -top modelB
# The tool runs "check_sec -elaborate -imp modelB"
check_sec -setup

# Examples of mapping using regular expressions:
# ----------------------------------------------
check_sec -map -spec {inst1..*_reg} -regexp
check_sec -map -spec {inst1.sig1\[([0-9]+)\]} -imp {modelB_imp.inst1.sig1\[$1\]} -
regexp
check_sec -map -spec {specsig(_i|_o|_reg)\[0:2\]} -imp {modelB_imp.impsig$1\[2:4\]} -
regexp

# You can map one spec signal to multiple imp signals (or vice versa) by using
# the entire name of the spec signal as the regular expression and terminating
# it with the metacharacter "$". Example:

check_sec -map -spec {specsig_reg_0$} -imp {impsig_reg_[0-9]+} -regexp -helper

# NOTE: This operation will fail if attempting to create cutpoint mappings,
# reset value abstractions, or counter abstractions.

# Since a regular expression can match any substring within a signal name and
# does not necessarily have to match the entire signal name, make sure that you
# use start-of-string and end-of-string anchors when needed. For example,
# consider the following case:

# Spec Design              Imp Design
# -----------              ----------
# specSig                  top_imp.impSig
# specSig_2                top_imp.impSig_2

# The command:
```

```
check_sec -map -spec {specSig(_.*)?} -imp {top_imp.impSig$1} -regexp

# creates the following mappings:

# {specSig -> top_imp.impSig}
# {specSig -> top_imp.impSig_2}
# {specSig_2 -> top_imp.impSig_2}

# This is because after replacing $1 with the empty string, the regular
# expression top_imp.impSig matches both signals top_imp.impSig and
# top_imp.impSig_2.
# If you do not intend to create the mapping {specSig -> top_imp.impSig_2},
# use the following command (notice the anchor at the end of the imp side
# regular expression):

check_sec -map -spec {specSig(_.*)?} -imp {top_imp.impSig$1$} -regexp

# Hierarchy aliasing examples:
# ----------------------------
check_sec -map -hierarchy -spec {instA_1.instB} -imp {modelB_imp.instX_1.instY}
check_sec -map -hierarchy -spec {instA_1\\[(\[0-9\]?)\\]} -imp
{modelB_imp.instX_1.instY[$1]}
check_sec -list -hierarchy -return id
check_sec -mapping_info -hierarchy -id 5
check_sec -unmap -hierarchy -id {7 3}

# Use regexp to automatically map spec signals to imp signals when their names
# are the same but their hierarchies are different. Example:

check_sec -map -hierarchy -spec {instA_(\[0-9\]?).instB} -imp
{modelB_imp.instX_$1.instY]} -regexp

# In the example above, consider 2 instances of instA_*: instA_4 and instA_5.
# The command maps signals found in, for example, instances {instA_4.instB} and
# {instA_5.instB} and maps them accordingly to the imp side using the matching
# instance numbers:
# {modelB_imp.instX_4.instY} and {modelB_imp.instX_5.instY}.

# Examples for data waiving in "check_sec -signoff":
#-------------------------------------------------
check_sec -signoff -waive_boundary_outputs_mapping {sig1 sig2 top_imp.sig3}
check_sec -signoff -waive_boundary_outputs_proof {prop1 prop2}
check_sec -signoff -waive_user_cutpoints_proof {prop3 prop4}
```

```
check_sec -signoff -waive_auto_cutpoints_proof {prop5 prop6}
check_sec -signoff -waive_imported_assertion_proof {prop7 prop8}
check_sec -signoff -waive_imported_cover_proof {prop9}
check_sec -signoff -waive_reset_value_abstraction {prop10}
check_sec -signoff -waive_x_signals_and_undrivens {sig4}
check_sec -signoff -waive_mapped_stopats {{s1 -> top_imp.s1}}
check_sec -signoff -waive_mapped_stopats {{s1 -> top_imp.s1} {s2 -> top_imp.s3}}
check_sec -signoff -waive_assumption_on_driven_signals {prop11}
check_sec -signoff -waive_assumption_related_preconditions {prop12}
check_sec -signoff -waive_imp_assumptions {prop13}
check_sec -signoff -waive_unmapped_gated_clocks {sig5}
check_sec -signoff -waive_mapped_gated_clocks {prop14}
```

See Also

set_sec_autoprove,
set_sec_autoprove_strategy,
set_sec_show_proof_log

check_spv

Checks for absence of undesired functional paths between sets of signals.

Associated App(s)

Security Path Verification

Syntax

```
check_spv -init

check_spv -create
              -from signal_tcl_list
              (-to) signal_tcl_list
              [-cover]
              [-from_precond expression]
              [-to_precond expression]
              [-no_precondition_covers]
              [-not_through signal_or_instance_tcl_list]
              [-include_control_logic |-exclude_control_logic]
              [-keep_driving_logic]
              [-taint_feedback_loop]
              [-task task_name |-name name |-name task_name::name]

check_spv -load csv_file


check_spv -abstract
           ( -from signal_tcl_list
                -to signal_tcl_list
            |-bbox_instance
            |-instance instance_list
               [-from_precondition expression]
               [-to_precondition expression]
               [-mode (seq | comb | mixed)]
               [-task task_name |-env]
               [-name name])
```

```
check_spv -abstract
          ( -list
             [-task task_name |-env]
             [-silent]
            |-remove name
            |-clear
               [-task task_name |-env])

check_spv -abstract
          ( -remove abstraction_name_tcl_list
             [-task task_name |-env])

check_spv -abstract
          ( -clear
             [-task task_name |-env]

check_spv -prove
               [-task task_name
                |-property property_name_tcl_list
                [-regexp] [-strategy (basic | proof)]
                [-cex_limit number_of_failures]
               ]
               [-force]
               [-time_limit]
               [-bg]
               [-engine_mode engine_tcl_list]
               [-skip_related_covers]
               [-cex_limit number_of_failures]
               [-from property_name]

check_spv -list (connection_specs | properties)
              [-silent]
              [-task task_name |-env |-all]

check_spv -consistency
          [ -task regexp_task_list
           |-property property_list
           |-from signal_list -to signal_list
           [-context task_name]]
           [-list (instance | signal | both)]
           [-include_stopats]
           [-prepare_consistency_graph]
```

```
check_spv -highlight property_list

check_spv -clear
         (all | result [-task task_name])

check_spv -signoff
               [ -task task_name
                |-properties property_name_tcl_list
               ]
               [-waive (  consistency | use_taint_feedback_loop
                                | use_keep_driving_logic
                                | undetermined_preconditions
                                | unreachable_preconditions
                                | loop_handling )+ ]
               [-silent]
```

Returns

Multiple return values.

Detailed Description

Use this command to confirm the absence of undesired functional paths between specified source and destination signals.

This command creates security properties (properties that check the absence or existence of undesired functional paths between two signals) and provides some tools, such as abstractions, to enable these verifications.

NOTE: The SPV App supports all loop handling modes except "assume_stable".

Initialize Security Environment

-init
    Initialize the settings required for the SPV App.
    NOTE:
    . This command must be issued before "analyze" and "elaborate" commands.
    . All other SPV commands must follow this command.
    . This command automatically disables the Expert System. This limitation
      will be removed in a future release.

```
Creating Security Properties
----------------------------
-create -from signal_tcl_list (-to) signal_tcl_list
               [-cover]
               [-from_precond expression]
               [-to_precond expression]
               [-no_precondition_covers]
               [-not_through signal_or_instance_tcl_list]
               [-include_control_logic |-exclude_control_logic]
               [-keep_driving_logic]
               [-taint_feedback_loop]
               [-task task_name |-name name |-name task_name::name]

    Create security properties to check for functional paths between source
    and destination signals.
    NOTE: This command ignores user-defined clocks and reset as source signals.
    . Use "-cover" to create a cover instead of an assertion to prove the
      existence of a cover path.
    . Use "-from" to define the specified signals as source signals.
      NOTE: This switch supports wildcards.
    . Use "-to" to define the specified signals as disjunctive destination
      signals, that is, if you want to check whether data can propagate to any
      of the specified signals.
      NOTE: This switch supports wildcards.
    . Use "-from_precond" to define an expression that determines when the
      value on the "-from" point will be allowed to propagate.
    . Use "-to_precond" to define an expression that must be true when the
      value that was on the "-from" has propagated to the "-to".
      NOTE: Only sequence expressions are allowed in preconditions, that is,
      no property or implication operators can be used in the precondition
      expression.
      NOTE: The "-from_precond" expression is not allowed to have a sampling
      clock. Examples:
        This expression is not allowed: {@(posedge clk) expr}
        This expression is allowed: {$past(a, 2,, @(posedge clk))}

      NOTE: When you define "-from_precond" and "-to_precond" expressions, the
      tool creates related covers by default.
      A combined related cover is created when both from and to preconditions
      are specified. Use the "-no_precondition_covers" switch
      to disable the creation of related covers when defining "-from_precond"
      and "-to_precond" expressions.
```

```
    . Use "-no_precondition_covers" to disable the creation of related covers
      when using the "-from_precond" and "-to_precond" switches.
      NOTE: Related covers are not created when "-cover" is specified.

    . Use "-not_through" to define the specified signals or instances that
      cannot be on the security path that might be found.
      NOTE: This switch supports wildcards.
    . Use "-include_control_logic" to analyze control paths during the
      property proof even if the "spv_include_control_logic" variable is set
      to "off". See "set_spv_include_control_logic" for additional information.
    . Use "-keep_driving_logic" to keep the driving logic of the source signals
      driven from the original design.
      NOTE: This switch:
      . Causes justified values for the "from" signal instead of random values.
      . Disables an optimization, so it might cause performance degradation.
      . Is mandatory if the property or the design depends on the "from" signal
        value.
    . Use "-taint_feedback_loop" to enable taint to be reinjected if it arrives
      at the "-from" signal again.
      NOTE:
      . This does not impact the concrete values or edges on the "-from" signal.
      . Using this switch might cause performance degradation.
      . Might cause CEX as well.
    . Use "-exclude_control_logic" to specify that control paths not be
      analyzed during the property proof even if the "spv_include_control_logic"
      variable is set to "on". The list of signals considered part of the
      control logic follows:
      . Mux selectors
      . Latch enables
      . Flop clock or reset pins
      . Output pin of comparison gates (equal, not-equal, greater-than,
        less-than)
      . Shift or rotate control pins
      . Reduce gates outputs (reduce OR, NOR, AND, XOR, XNOR)
      See "set_spv_include_control_logic" for additional information.
    . Use "-task" to apply the command to the specified task instead of the
      current task.
    . Use "-name" to assign a name to the property.

-load csv_file
    Create a set of SPV properties from the specified CSV file.

    The CSV file must have the following comma-separated elements in
```

```
    each line:

    . name
    . from
    . to
    . from_precond
    . to_precond
    . not_through
    . exclude_control_logic
    . include_control_logic
    . no_precondition_covers

    IMPORTANT:
    . These elements must be in the order specified above.
    . Elements can have an empty value as long as they are properly
      comma-separated.
    . The argument semantics for "name", "from", "to", "from_precond",
      "to_precond", and "not_through" are the same as their counterparts in the
      "check_spv -create" command.
    . For the rest of the elements, an empty value indicates that the switch
      is not present while a non-empty element indicates the switch is present.
    . For commas to be considered valid characters and not separators, they
      must be properly escaped.
    . Any line that has a number of columns that is different from the number
      of elements mentioned above will be ignored.

    NOTE: "check_spv -load" is an initial release to gather feedback from early
    adopters and finalize implementation for an upcoming release. Contact
    support@cadence.com to provide feedback or comments targeting the upcoming
    production version.

Adding Security Abstractions
----------------------------
-abstract  ( -from signal_tcl_list -to signal_tcl_list
            |-bbox_instance
            |-instance instance_list)
               [-from_precondition expression]
               [-to_precondition expression]
               [-mode (seq | comb | mixed)]
               [-task task_name | -env]
               [-name name])

-abstract ( -list
```

```
          [-task task_name | -env]
          [-silent]
       |-remove abstraction_name
       |-clear
          [-task task_name | -env])
Create a security abstraction to define when to allow a taint to be
propagated from a given signal list into another signal list.

. Use "-list" to list all existing SPV abstractions.
  . Use "-task" to list all existing SPV abstractions in the specified
    tasks.
  . Use "-env" to list all the global SPV abstractions.
  NOTE: If you do not specify additional switches, "-list" lists all  SPV
  abstractions in the current task.
. Use "-remove" with list of SPV abstractions to remove those abstractions.
. Use "-clear" to clear all SPV abstractions.
  . Use "-task" to clear all existing SPV abstractions from the specified
    tasks.
  . Use "-env" to clear all the global SPV abstractions.
  NOTE: If you do not specify additional switches, "-clear" clears all
  SPV abstractions in the current task.
. Use "-from" to define the specified signals as sources of the SPV
  propagation abstraction.
. Use "-to" to define the specified signals as targets of the SPV taint
  propagation
  abstraction.
. Use "-bbox_instance" to create SPV abstractions for all blackbox
  instances.
. Use "-instance instance_list" to create SPV abstractions on the list of
  provided instance names.
. Use "-from_precondition" to define an expression that determines when to
  allow a taint to be injected into the SPV abstraction.
. Use "-to_precondition" to define an expression that must be true for a
  taint inside the SPV abstraction to propagate out of it.
. SPV abstractions do not allow specifying driven signals as destination
  signals since the original driving logic is replaced by the logic of
  the abstraction. This blocks taint propagation through the original
  design and might cause false positives. If you specificy driven signals
  as destination signals, the tool issues ESPV326. To allow driven signals
  as destination signals, downgrade the error to a warning as follows:
  set_message -warning ESPV326.
. Use "-mode comb" to specify that the taint propagates out of the taint
  propagation abstraction in the same phase as the taint enters.
```

```
    . Use "-mode seq" to specify that the taint propagates out of the taint
      propagation abstraction in the phases following the phase that the taint
      enters the abstraction.
    . Use "-mode mixed" to specify that the taint propagates out of the taint
      propagation abstraction in the same phase or phases following the phase
      that the taint enters the abstraction.
    . Use "-task" to apply the abstraction to the specified task instead of the
      current task.
    . Use "-env" to apply the abstraction globally into all tasks.
    . Use "-name" to specify a name for the abstraction.
    . Note:
      . You cannot use the "-name" option with the "-bbox_instance" switch.
      . When using the "-instance" switch, you can abstract only a single
        instance when specifying a name for the abstraction.
      . You cannot use the "-mode", "-to_precondition", "-from_precondition"
        options with Mpram instances.

    NOTE:
    . "-to" signals can be constrained.
    . You cannot create SPV abstractions with intersecting "-to" signals.
    . You cannot create SPV abstractions with "-to" signals that intersect with
      the "-from" signal of an SPV property.
      with "-from" signal of an SPV property.
    . If you do not use "-mode", the default mode is "mixed".
    . For Mpram instances, a special "mpram" mode is used.
      If you use "-from", "-to" on signals of Mpram instances,
      it will not use the special mode. This might lead to false results.
    . Adding SPV abstractions might cause performance degradation.

-abstract -remove abstraction_name_tcl_list
    [-task task_name | -env]
    Remove the specified abstractions in the specified task.
    NOTE:
    . If you do not use the "-env" or "-task" option, the command applies to the
      current task.

-abstract -clear
    [-task task_name | -env]
    Remove all abstractions in the specified task.
    NOTE:
    . If you do not use the "-env" or "-task" option, the command applies to the
      current task.
```

```
Proving Security Properties
---------------------------
-prove
    [-task task_name [-strategy (basic | proof)]
     |-property property_name_tcl_list [-regexp] [-strategy (basic | proof)]
     [-cex_limit number_of_failures]
    ]
    [-force]
    [-time_limit]
    [-bg]
    [-engine_mode engine_tcl_list]
    [-skip_related_covers]
    [-cex_limit number_of_failures]
    [-from property_name]
    Start the proof of the specified security properties.

    . By default, SPV will prove the current task.
    . Use "-task" to prove all the SPV properties in the specified task.
    . Use "-property" to prove the specified SPV properties.
    . The "-property" switch supports wildcards and regular expressions.
      for example:
      check_spv -prove -property {expression} -regexp
    . Use "-property" with "-strategy" to choose a strategy that trades in
      resources versus proof convergence on the specified properties.
      . Use "basic" to run the proof in a single-threaded proof environment.
      . Use "proof" to run the proof in a sophisticated, multi-threaded, and
        collaborating proof environment.
      . By default, the tool applies your "prove" command using the "basic"
        strategy.
    . Use "-property" with "-cex_limit" to specify that you want the prove
      flow to stop the proof job after it reaches a certain threshold of
      failures on SPV properties. If the prove flow stops on cex_limit,
      it returns "cex_threshold_reached".

    . Use "-bg" to run the proof in the background and continue to accept and
      run new commands.
    . Use "-force" to clear existing proof results for the selected property
      before beginning. See "clear -result".
    . Use "-time_limit" to override the current "prove_time_limit" for this
      "prove" command. See "set_prove_time_limit".
    . Use "-engine_mode" to override the current engine mode for this proof.
      NOTE: Only the "basic" proof strategy provides the option to specify
      an engine mode and considers the "engine_mode" setting only if
```

```
  "set_prove_orchestration" and "set_proofmaster" are both off. See
  "set_engine_mode".
. By default, SPV properties and their related covers are proven together.
. Use "-skip_related_covers" to prevent the engines from automatically
  proving related covers. With this setting, the tool might still apply
  results to related covers if inferred from the main property's status or
  if the load cached previous results.
. Use "-cex_limit" to specify that you want the prove flow to stop
  the proof job after it reaches a certain threshold of failures on SPV
  properties. If the prove flow stops on cex_limit, it returns
  "cex_threshold_reached".
. Use "-from" to use an existing trace as the initial value
  for another formal proof.
  Use this feature when a formal proof finds an interesting scenario
  counterexample and you want to perform another search from that point.
  By doing so, you might find scenarios that are deeper than what you could
  reach with other formal proofs.
  NOTE:
  . This feature is only supported by the "basic" proof strategy.
  . This can only be used when the "-from" property and all target
    properties are in the same task and have the same signature
    ("-from", "-not_through", "-from_precondition",
    "-taint_feedback_loop", "-keep_driving_logic",
    "-include_control_logic"/"-exclude_control_logic").
  . This is a bug-hunting feature. The tool is not able to prove assertions
    when you use "check_spv -prove -from".
  . Found traces start from reset and include the trace to the specified
    property as a prefix.
  . If undetermined properties are triggered in the existing trace, the trace
    is used as the result for those properties. (The first time a property
    is triggered along a trace is the most significant event.)

NOTE:
. The strategies work as follows:
  . Basic strategy is the default strategy and runs a proof with
    SPV-oriented proof settings on the target property. It can run with 1
    job, but the recommendation is at least 8 jobs. Most easy results will
    be found by that strategy, often in less than one hour.  Run this
    strategy first.
  . Proof strategy runs a parallel basic proof, a proof with automatically
    applied cutpoints on flops and internal signals of the design using
    cutpoint refinement loop handling and failure extension to help the
    other strategies find deep counterexamples by extending counterexamples
```

```
        it already identified. It can find all easy and additional hard results
        with significantly higher resources and run time. Allow at least 36
        jobs and 1h of run time for this strategy.
        Run this strategy on remaining targets after "basic" strategy no longer
        produces results.
    . propagation environment is defined by the following:
      . from signals
      . not-through signals
      . keep driving logic
      . taint feedback loop
      . control logic
      . task
    . Return values are the same as the "prove" command. But internally
      auto-generated helpers are not considered in the return value of the
      proof. For details on all possible return values, use "help prove -gui".
    . Max jobs and per engine max jobs limitations are applied per property
      not per prove command.
    . Use the "set_spv_prove_single_thread off" command to prove multiple
      properties in individual proof threads.
      NOTE: Only the basic proof strategy supports this configuration.
    . Related covers are automatically proven in the same proof thread when
      proving the associated SPV properties. To disable the automatic proof
      of related covers, use "-skip_related_covers".
    . SPV can infer the status of the "to" precondition from the status of its
      main property and vice-versa.
      . A counterexample on the main property means that the "to" precondition
        is covered.
      . If the "to" precondition is unreachable, the main property is vacuously
        proven.
      . If a new bound was found for the "to" precondition, the main property's
        bound may be increased to match it.
      In case of inference, the indication "(Inferred)" appears immediately after
      the engine name, for example:
      "INFO (ISPV006): (0.basic) SEC basic strategy (B)(Inferred): A min_length
      bound of 252 was found for the property "prop" in 4.14 s."

Getting Security Information
----------------------------
-list (connection_specs | properties)
      [-silent]
      [-task task_name |-env |-all]
    List useful information regarding security abstractions and properties.
```

```
    . Use exactly one of the following arguments with each command:
      . connection_specs: Lists and returns the name and connection specs for
        all abstractions with the same format required when
        specifying a manual abstraction.
      . properties: Lists and returns all security properties with the
        corresponding from and to signals and from and to
        preconditions.
    . Use "-silent" to turn off the output to the screen and return information
      with a Tcl return value, which is helpful in Tcl scripts.
    . Use "-task" to list connection specs or properties for a specified task.
    . Use "-env" to list connection specs for environment abstractions.
      This switch works with "connection_specs" only.
    . Use "-all" to list connection specs or properties for all tasks.
    NOTE: If you do not use the "-task", "-env", or "-all" option, this command
    applies to the current task.

-consistency
    [ -task regexp_task_list
     |-property property_list
     |-from signal_list -to signal_list [-context task_name]]
     [-list (instance | signal | both)]
     [-include_stopats]
     [-prepare_consistency_graph]

    Check whether any blackbox or stopat on the path of any security property
    was not handled by an SPV abstraction for a black box or removing a stopat.

    NOTE: Use this command to identify blackboxes and stopats that have not been
    handled. This command can help you avoid a false positive proof result.

    . Use one of the following switches to run the command only for a given
      target:
      . Use "-task" with a list of tasks. This command supports regular
        expressions.
      . Use "-property" with a list of properties.
      . Use "-from" and "-to" to specify a list of from signals and a list of
        to signals.
        NOTE: In this case, the tool assumes only the environment
        abstractions. If you want the tool to consider a task's abstractions
        also, use "-context" with the desired task name as its argument.
    . Use "-list" to report either the instances' names, the instances'
      ports, or both. The instances' ports are reported in this format:
      {inputs} {outputs} {inouts}.
```

```
      If you do not use "-list", the command lists the properties that do not
      have abstracted blackboxes along with their names.
    . This command does not include stopats by default. Use "-include_stopats"
      to get a list of relevant stopats.
    . Use "-prepare_consistency_graph" if you want to open the consistency
      graph afterwards.
      NOTE: Using this switch can make this command significantly slower, but
      opening the consistency graph afterwards should take a lot less time.


-highlight property_list

    Generates highlight information for each property in the property_list.
    This information will be shown in the SPV Analysis Browser as follows:

    . "-from" fanout - Light purple
    . "-to" fanin - Light blue
    . "-from" fanout intersection with "-to" fanin - Light green

Manipulating the Security Environment
-------------------------------------
-clear (all | result [-task task_name])

    Clear the SPV environment.
    . Use "all" to clear all the SPV environment from the current session.
    . Use "result" to clear all proof information from the specified
      task.

SPV Signoff
-----------
-signoff
    [ -task task_name
     |-properties property_name_tcl_list
    ]
    [ -waive (  consistency | use_taint_feedback_loop
              | use_keep_driving_logic
              | undetermined_preconditions
              | unreachable_preconditions
              | loop_handling )+ ]
    [-silent]

    Write a report that summarizes the SPV proof status for a specific task or
    for specific SPV properties. The sign-off report highlights the possible
```

```
risks to the validity of the SPV proof.

. If "-task" or "-properties" are not used, the signoff command runs on the
  current task.
. The report might contain the following categories for each property:
  . Consistency: this category includes two types of possible taint
    blockers:
    . Blackboxes in the path of the property.
    . Stopats in the path of the property.
  . Using taint feedback loop: this category indicates that you might need
    to use "-taint_feedback_loop" for the property since there is a path in
    the design that starts and ends with a "from" signal.
  . Using keep driving logic: this category indicates that you might need
    to use "-keep_driving_logic" for the property. There are two possible
    reasons to do so:
    . An undriven "from" signal is in the COI of one of the property's
      precondition.
    . A "from" signal is in the COI of an assumption.
    NOTE: This switch is contraindicated in the context of proven properties.
    That is, if a property is proven, there is no need to use
    "-keep_driving_logic" even if the property meets the criteria listed
    above.
  . Undetermined preconditions: this category lists any undetermined
    preconditions.
  . Unreachable preconditions: this category lists any unreachable
    preconditions.
  . Loop handling: this category indicates that a property might have a
    spurious result due to the current loop handling mode. For more
    details, use "help set_loop_handling" or
    "help set_spv_break_loop_mode".
. Use "-waive" to waive specific categories for all properties.
```

Example

```
check_spv -create -from {source} -to {destination}
check_spv -abstract -from {source} -to {destination} -mode comb
check_spv -abstract -from {source} -to {destination} -mode seq -from_precondition
{condition expression} -to_precondition {condition expression}
check_spv -abstract -list
check_spv -abstract -bbox_instance
check_spv -prove
```

See Also

elaborate -bbox,
elaborate -bbox_a,
elaborate -bbox_div,
elaborate -bbox_m,
elaborate -bbox_mod,
elaborate -bbox_mul,
elaborate -bbox_pow,
set_spv_include_control_logic,
set_spv_prove_single_thread

check_superlint

Configures, manipulates, and waives checks or display/hide views in Superlint.

Syntax

```
Initializing and Configuring Checks
-----------------------------------
check_superlint -init

check_superlint -configure
    (-load_rule_file file_name+)

check_superlint -configure
    (-save_rule_file file_name) [-force]

check_superlint -configure
    (-load_design_file file_name+)

Extracting and Manipulating Checks
----------------------------------------------
check_superlint -extract
    [ -instances instance_list |-hierarchies instance_list
     |-exclude_hierarchies instance_list |-top]
    [-extended_property_name]
    [-task task_name]
    [-list]
    [-verbosity]

check_superlint property_id_tcl_list
   -get (  type | waived | expression | argument | status
                | waiver | task | fpv_property | label | severity
                | tag | category | source_location
                | module | instance | state | signal )
                [-silent]

check_superlint message_id_tcl_list
   -get (  waived | expression | label | waiver | severity
                | tag | category | source_location
                | module | instance | message | argument | session )
```

```
   [-silent]

check_superlint -show property_id_tcl_list
    [-silent]

check_superlint -show message_id_tcl_list
    [-silent]

check_superlint -set_label label
    (message_id_tcl_list | property_id_tcl_list)

check_superlint -list
    [-type (Assert | Assume | Cover)]+
    [-waived (none | only)]
    [-regexp property_expression_regexp]
    [-status (  all | cex | proven | covered | unprocessed
                     | unreachable | undetermined )]+
    [-severity (error | warning | info)]
    [-domain domain_list]
    [-category category_list]
    [-tag tag_list]
    [-message message]
    [-label label]
    [-source_location source_location]
    [-instance instance]
    [-module module]
    [-file file_name]
    [-grouping]
    [-silent]

check_superlint -update_fsm state_signal_name
    -idle_state_value idle_state_value
    -disabling_state_value disabling_state_value

check_superlint -layout (violation | formal)

check_superlint -prove
    [-time_limit time_limit]
    [-mem_limit memory_limit]
    [-task task_name_list]
    [-keep_traces [assert | all | none]]
    [-property property_name_list]
    [-bg]
```

```
    [-fsm]

check_superlint -clear

check_superlint -leverage

check_superlint -multimode

check_superlint -ml
    [-dump_data -dir directory_name]
    [-train -dir directory_name -projects directory_name]
    [-infer -dir directory_name]

Debugging Checks
--------------
check_superlint -waiver -add
    -id (message_id_tcl_list | property_id_tcl_list)
    -comment comment
    [-export file_name [-do_not_apply] [-append]]
    [-silent]
    [-force]

check_superlint -waiver -add
     [-instance instance_name]
     [-expression expr]
     [-category message_category]
     [-tag message_tag]
     [-exclude_tag exclude_tag_list]
     [-argument argument [-relaxed] [-regexp_argument |-wildcard_argument]]
     [-module module_name]
     [-hierarchy module_or_instance]
     [-source_file file_name
       [-start_line line_num] [-end_line line_num]]
       [-start_column column_num] [-end_column column_num]]
     [-top]
    )+
    -comment comment
    [-export file_name [-do_not_apply] [-append]]
    [-silent]
    [-force]

check_superlint -waiver -remove waiver_id_tcl_list
    [-silent]
```

```
check_superlint -waiver -show waiver_id_tcl_list
    [-silent]

check_superlint -waiver -list
    [-instance instance_name]
    [-expression expr]
    [-category message_category]
    [-tag message_tag]
    [-exclude_tag exclude_tag_list]
    [-module module_name]
    [-hierarchy module_or_instance]
    [-source_file file_name]
    [-state waiver_state]
    [-top]
    [-silent]

check_superlint -waiver -list_matching_checks waiver_id_tcl_list
    [-silent]

check_superlint -waiver -export -file_name file_name
    [waiver_id_tcl_list]
    [-append]
    [-version]
    [-unmatched]
    [-updated]

check_superlint -waiver -import -file_name file_name
    [-module_mapping {old_module_name=new_module_name}]
    [-silent]
    [-force]

check_superlint -waiver -accept_update
    [waiver_id_tcl_list]
    [-confidence_score confidence_score_percentage]
    [-silent]

check_superlint -waiver -remove_update
    [ml_generated_waiver_id_tcl_list]
    [-confidence_score confidence_score_percentage]
    [-silent]

check_superlint -show_schematic
```

```
  ( [-id check_id [-constants_values]]
   |[-instance instance_name]
   |[-signal signal_list])+
    [-mode (functional | shift | capture | none)]
    [-hide_full_hierarchy]
    [-new_window [new_window_name]]

check_superlint -observability -show check_id
    [-paths] [-boundaries]
    [-silent]

check_superlint -gated_clock

check_superlint -constant (zero | one | any)

check_superlint -debug -deadcode -compute_root_cause
    -id (message_id_tcl_list | property_id_tcl_list)

check_superlint -debug -view -deadcode_analyzer
    -id (message_id | property_id)

Reporting Superlint App Setup
-------------------------------------
check_superlint -report -setup
    [only ( -design_statistics |-tool_configuration
           |-design_configuration)+]
    [-file file_name [-force]
      [ [-html [-launch_html_browser]
          [-source_path_mapping paths_mapping]
          [-relative_file_path]]
      |-xml |-csv]]

Reporting Superlint App Checks
-------------------------------------
check_superlint -report
    [-property
        [-status (  cex | unreachable | proven
                          | covered | undetermined | unprocessed)+]
        [-type (assert | cover)+]
        [-task task_list]]
    [-violation
        [-severity (error | warning | info)+]
        [-order (  category | tag | filename | instance(module)
```

```
                          | label | severity | mode)]
        [-include_design_build]]
    [-detailed [-verbose (tag_list | all)]]
    [-domain domain_list]
    [-category category_list]
    [-tag tag_list]
    [-instance instance_list]
    [-regexp]
    [-waived (none | only [ -user_tcl |-user_rtl
                                    |-auto]+)]
    [-grouping (none | basic | advanced)]
    [-file file_name [-force]
      [ [-html [-launch_html_browser]
          [-source_path_mapping paths_mapping]
          [-relative_file_path]]
      |-xml |-csv]]
```

Reporting Superlint App Waivers

```
check_superlint -report -waiver
    [only (-user_tcl |-user_rtl |-auto)+]
    [-waiver_status (matched | unmatched)+]
    [-detailed]
    [-file file_name [-force]
      [ [-html [-launch_html_browser]
          [-source_path_mapping paths_mapping]
          [-relative_file_path]]
      |-xml |-csv]]
```

Reporting Superlint App Signoff

```
check_superlint -report -signoff
    [-file file_name [-force]
      [ [-html [-launch_html_browser]
          [-source_path_mapping paths_mapping]
          [-relative_file_path]]
      |-xml |-csv]]
```

Exporting Superlint App Properties

```
check_superlint -export
    -to_sva sva_file_name
    [-to_tcl tcl_file_name]
```

```
    [-extended_property_name]
    [-remove_duplicate_properties]
    [-module sva_module_name]
    [-initial_block sva_initial_block]
    [-action_block sva_action_block]
    [-type (all | Assert | Assume | Cover)+]
    [-waived (none | only)]
    [-status (all | cex | proven | unprocessed | unreachable | undetermined)+]
    [-category category_tcl_list]
    [-instance instance_tcl_list]
    [-top]
    [-property id_tcl_list]


Bottom-Up Hierarchical Analysis
-------------------------------
check_superlint -import -file file_name

Superlint FSM Checks
----------------------------------------
check_superlint -fsm
    [ -list |-show fsm_signal_name [-export_xml file_name]
     |-get_property fsm_signal_name [-state state_name]]

GUI Operations
--------------
check_superlint -gui
   ( [-reinvoke_mode (reinvoke | rerun)]
    |[-order_by (domain | tag | message | filename | severity | label |
instance(module))]
    |[-grouping_mode (none | basic | advanced)])+
    |[-view -display (design_hierarchy)]
    |[-view -hide (design_hierarchy)]
    |[-show_file_mode (file_name | full_path)]
```

Returns

Various

Detailed Description

```
Use this command to configure, manipulate, and waive checks in the
Superlint App extraction process, or display/hide views in the Superlint App.

Initializing and Configuring Checks
-----------------------------------
-init
    Initialize the settings required for Superlint.

-configure
    (-load_rule_file file_name_list)
    Load the configuration from one or more .def files. If you do not provide
    any files, the tool loads the default configuration.

    NOTE: See HAL manuals available from Cadence Online Support for additional
    information.

-configure
    (-save_rule_file file_name) [-force]
    Save the configuration to a 'file_name'.def file.

    If a file with the same name already exists, use "-force" to overwrite it.

    NOTE: See HAL manuals available from Cadence Online Support for additional
    information.

-configure
    (-load_design_file file_name_list)
    Load the design info configuration from one or more files. If you do not provide
    any files, the tools assumes that no configuration has been provided.

    NOTE: See HAL manuals available from Cadence Online Support for additional
    information.

NOTE: To configure Superlint checks please use config_rtlds command.


Extracting and Manipulating Checks
----------------------------------------------
-extract
    [ -instances instance_list |-hierarchies instance_list
     |-exclude_hierarchies instance_list |-top]
    [-extended_property_name]
    [-task task_name]
```

```
    [-list]
    [-verbosity]

    Extract checks (properties or lint messages) from the design.

    . Use "-instances" to limit extraction to the specified instances.
    . Use "-hierarchies" to limit extraction to the specified hierarchies.
    . Use "-exclude_hierarchies" to exclude the specified hierarchies.

      NOTE: If you want to exclude all instances of a module recursively on
      down, leverage the following command:

      get_design_info -module <module> -list instance

      This provides a list of paths to each instance. Pass this list to the
      "-exclude_hierarchies" switch as follows:

      set mod_list [get_design_info -module <module> -list instance -silent]
      check_superlint -extract -exclude_hierarchies $mod_list

    . Use "-top" to limit extraction to the top instance.
    . Use "-extended_property_name" to specify the following naming convention
      for checks:
      \<instance>_<file><start line><end line><check type>prop<prop id>[more indexes]
      NOTE: The naming convention uses Verilog escaped identifiers; that is,
      they begin with a backslash (\) and end with a space.
    . Use "-task" to extract to a specified task.
      NOTE:
      . If the task you provide does not exist, the tool creates it first.
    . Use "-list" to list the IDs of the extracted properties or lint messages.
      NOTE: If you do not use the optional "-list" switch, "check_superlint
      -extract" returns the total number of extracted properties or lint
      messages.
    . Use "-verbosity" to override the current "superlint_verbosity_level" for
      this extraction command. See "set_superlint_verbosity_level"

property_id_tcl_list -get (  type | expression | expression | argument
                           | status | waiver | task | fpv_property | label
                           | severity | tag | category | source_location
                           | module | instance | state | signal )
    [-silent]

    Report the specified attribute of the specified property candidates.
```

```
    . Use "type" to report the property candidate type: assertion, assumption
      or coverage hole.
    . Use "waived" to report the waived status for property
    . Use "expression" to report the expressions for property candidates.
    . Use "argument" to report the arguments for property candidates.
    . Use "status" to report the proof status for the property candidates.
    . Use "waiver" to report the waivers for the property candidates.
    . Use "task" to report the tasks for which the property candidates were
      exported.
    . Use "fpv_property" to report the FPV property names in the active
      exported task for the property candidates.
    . Use "label" to report labels for the specified property candidates.
    . Use "severity" to report the severity of any violation messages for the
      specified property candidates.
    . Use "tag" to report the message tag for the specified property candidates.
    . Use "category" to report the message category for the specified property
      candidates.
    . Use "source_location" to report the property source locations.
    . Use "module" to report the module names for the specified formal
      properties.
    . Use "instance" to report the instances for the specified formal
      properties.
    . Use "state" to report the states for the specified FSM formal
      properties.
    . Use "signal" to report the signals for the specified formal
      properties.

message_id_tcl_list -get (  waived | expression | label | waiver
                          | severity | tag | category  | source_location
                          | module | instance | message | argument | session )
    [-silent]

    Report the specified attribute of the specified messages.

    . Use "waived" to get if the message is waived or not
    . Use "expression" to report the expressions for the specified messages.
    . Use "label" to report the labels for the specified messages.
    . Use "waiver" to report the waivers for the specified messages.
    . Use "severity" to report the severity of the specified messages.
    . Use "tag" to report the tag for the specified messages.
    . Use "category" to report the category for the specified messages.
    . Use "source_location" to report the source location for the specified
```

```
      messages.
    . Use "module" to report the module name for the specified messages.
    . Use "instance" to report the instance for the specified messages.
    . Use "message" to report the short message for the specified IDs.
    . Use "argument" to report the arguments for the specified messages.
    . Use "session" to report the session IDs for the specified messages.

-show property_id_tcl_list [-silent]
    Show the details of the property candidates.

    . Use "-silent" to turn off the output to the screen and return
      information with a Tcl return value, which is helpful in Tcl scripts.

-show message_id_tcl_list [-silent]
    Show the details of the specified messages.

    Use "-silent" to turn off the output to the screen and return
    information with a Tcl return value, which is helpful in Tcl scripts.

-set_label label (message_id_tcl_list | property_id_tcl_list)
    Add a label for properties or messages that persists across different runs.

-list
    [-type (Assert | Assume | Cover)]+
    [-waived (none | only)]
    [-regexp (property_expression_regexp | message_expression_regexp)]
    [-status ( all | cex | proven | covered | unprocessed
               | unreachable | undetermined )]+
    [-severity (error | warning | info)]
    [-domain domain_list]
    [-category category_list]
    [-tag tag_list]
    [-message message]
    [-label label]
    [-source_location source_location]
    [-instance instance]
    [-module module]
    [-file file_name]
    [-grouping]
    [-silent]

    List the IDs for the property candidates or messages that match the
    specified criteria.
```

```
    . Use "-type" to list properties of a specific type.
    . Use "-waived" to list messages with the specifed waived status
    . Use "-regexp" to filter properties or messages by expression
      using regular expression.
    . Use "-status" to list properties of a specific status.
    . Use "-severity" to list only properties and messages of the specified
      severity.
    . Use "-domain" to list only the properties and messages in the specified
      domains.
    . Use "-category" to list only the properties and messages in the specified
      categories.
    . Use "-tag" to list only properties and messages with the specified tags.
    . Use "-message" to list only properties and messages with the specified message.
    . Use "-label" to list properties and messages with the specified label.
    . Use "-source_location" to list properties and messages with the
      specified source location. The source location can be a file, starting
      line, or a range of starting and ending lines.
    . Use "-instance" to list properties and messages with the specified
      instance.
    . Use "-module" to list properties and messages with the specified module.
    . Use "-file" to dump properties and messages to the specified file.
    . Use "-grouping" to list properties and messages of a related group.
      NOTE: The grouping mode preference (basic or advanced) is the same
      as that selected for the run.

-update_fsm state_signal_name -idle_state_value idle_state_value
                              -disabling_state_value disabling_state_value

    . Use "-idle_state_value" to specify the idle state value of the specified FSM.
    . Use "-disabling_state_value" to specify the disabling state value of the
specified FSM.

    NOTE: You must use this command before extracting checks of the specified FSM;
    otherwise, you will need to run "check_superlint -clear" and re-extract.

-layout (violation | formal)
    Set the main window layout to the violation view or formal property view.

    NOTE: Once set, the layout remains persistent across runs.

-prove
    [-time_limit time_limit]
```

```
    [-mem_limit memory_limit]
    [-task task_name_list]
    [-property (property_name_list | task::property_name_list)]
    [-keep_traces [assert | all | none]]
    [-bg]
    [-fsm]

    . Use "-time_limit" to override the default "superlint_prove_time_limit" of
      "2h".  Specify "time_limit" values as an integer followed by s, m, or h
      (no spaces).
      . s = seconds
      . m = minutes
      . h = hours
    . Use "-mem_limit" to override the default 2048MB "engineCG_max_mem" for
      "autoprove". Specify "memory_limit" values as an integer followed by m or
      g (no spaces).
      . m = megabytes
      . g = gigabytes
    . Use "-task" to specify the task you want proven.
      NOTE:
      . If you provide no task, the tool proves all Superlint tasks.
      . If you do not supply a task name, the tool proves all assertions and
        covers in the current task.
    . Use "-keep_traces" to keep traces generated during proof.
      . Use "assert" to keep traces for assertions only.
      . Use "all" to keep all traces during proof.
      . Use "none" to discard all traces during proof.
    . Use "-property" to prove the specified properties only.
      NOTE:
      . When specifying a list of properties, enclose the list in curly
        braces.
      . To prove properties not in the current task, prepend the property
        names with "<task>::".
      . All listed properties must belong to the same task.
    . Use "-bg" to run the proof in the background.
    . Use "-fsm" to run the proof with the FSM divide-and-conquer strategy,
      which proves FSM deadlock and livelock properties hierarchically by
      dividing the design into levels and adding stop-ats at instance
      boundaries. The strategy runs in parallel with a normal proof.

-clear
    Clear all Superlint App information without clearing the results of the
    analysis and elaboration.
```

```
-leverage
    Leverage formal results to refine Lint and DFT results. This feature
    leverages a mapping between formal properties and Lint/DFT violations
    in the following way:
    . If the proof finds a counter example for a formal property, the
      matching Lint or DFT violation is grouped with it.
    . If a formal property is proven, the matching Lint or DFT violation
      is waived.
    NOTE:
    . For this command to work, you must extract properties and run the proof.
    . To use this command, the "set_superlint_leverage_formal" variable must
      be enabled before extracting checks and running the proof. The default
      is "true".

-multimode
    Enables the multimode feature, which launches mode sessions for multiple
    configurations of a design.
    NOTE:
    . For this command to work, modes information must be provided.

Machine Learning
--------------------
-ml
    [-dump_data -dir dump_data_directory]
    [-train -dir train_model_directory -projects {dump_data_1 dump_data_2}]
    [-infer -dir train_model_directory]

    Machine Learning waiver suggestion is done using the options below:
    . "-dump_data" switch to dump the data for machine learning
      model "dump_data_directory" argument.
    . "-train" switch to train the model using the  data from the projects and
      dump the model in "train_model_directory".
      Multiple dumps can be used to train the model.
    . "-infer" switch to get waiver suggestions using the ML model
      from "train_model_directory".

Debugging Checks
--------------
-waiver -add -id (message_id_tcl_list | property_id_tcl_list)
    -comment comment
    [-export file_name [-do_not_apply] [-append]]
    [-silent]
```

```
    [-force]

    Add waivers on the specified messages or properties.

    . Use "-comment" to add a comment to the waiver.
    . Use "-export" to save waivers to the specified file.
      .Use "-do_not_apply" to prevent applying the waiver.
      .Use "-append" to append to the file if it already exists

    . Use "-silent" to suppress output and generate only Tcl return values.
    . Use "-force" to add waivers even if there are no matching messages or
      properties.

-waiver -add
      [-instance instance_name]
      [-expression expr]
      [-category message_category]
      [-tag message_tag]
      [-exclude_tag exclude_tag_list]
      [-argument argument_list [-relaxed] [-regexp_argument |-wildcard_argument]]
      [-module module_name]
      [-hierarchy module_or_instance]
      [-source_file file_name
        [-start_line line_num] [-end_line line_num]]
        [-start_column column_num] [-end_column column_num]]
      [-top]
    )+
    -comment comment
    [-export file_name [-do_not_apply] [-append]]
    [-silent]
    [-force]

    Add waivers as specified and return an ID list of the added waivers.

    NOTE: You must use one or more of the following switches to add waivers to
    properties: "-tag", "-instance", "-expression", and "-source_file".

    . Use "-instance" to add waivers on properties of the specified
      instance. This switch supports wildcard and regular expression.
      Add "_REG_EXPR:" prefix to specify regular expression and "_WILD_CARD:"
      to specify wildcard.
    . Use "-top" to add waivers on properties of the top instance.
    . Use "-expression" to add waivers on properties with the specified
```

```
  expression.
. Use "-category" to add waivers on messages in the specified category.
  This switch supports wildcard and regular expression.
  Add "_REG_EXPR:" prefix to specify regular expression and "_WILD_CARD:"
  to specify wildcard.
. Use "-tag" to add waivers on messages with the specified tag.
  This switch supports wildcard and regular expression.
  Add "_REG_EXPR:" prefix to specify regular expression and "_WILD_CARD:"
  to specify wildcard.
. Use the "-exclude_tag" switch to add waivers to messages, excluding
  those that have specific tags.
  NOTE: "-exclude_tag" switch and "-tag" switch are mutually exclusive.
. Use "-argument" to add waivers on the specified message arguments.
  NOTE:
  . Specified message arguments in the waiver need to exactly match message
    arguments.
  . Use "-regexp_argument" or "-wildcard_argument" to consider the
    arguments regular expressions or wildcards respectively.
  . Use "-relaxed" to match message arguments partially and in an unordered
    manner.
  . "-relaxed" ignores any duplication in the arguments specified.
. Use "-module" to add waivers on messages from the specified module.
  This switch supports wildcard and regular expression.
  Add "_REG_EXPR:" prefix to specify regular expression and "_WILD_CARD:"
  to specify wildcard.
. Use "-hierarchy" to add waivers on messages from the specified module or
  instance.
  NOTE:
  . If you specify a module, the tool adds waivers on messages from the
    module specified, all instances of the module as well as all
    subinstances and exclusive submodules. For example, module B is an
    exclusive submodule of module A if all the instances of module B are
    subinstances of module A instances.
    IMPORTANT: If you are using the Xcelium front end, Superlint waives both
    exclusive and non-exclusive submodules.
  . If you specify an instance, the tool adds waivers on messages from the
    instance specified, all subinstances and exclusive submodules.
  . If you specify a name that matches both a module and an instance, the
    tool adds waivers as if you had specified a module.
  . This switch does not support Wildcard or regular expression.
. Use "-source_file" to add waivers on checks from the specified file.
  This switch supports wildcard and regular expression.
  Add "_REG_EXPR:" prefix to specify regular expression and "_WILD_CARD:"
```

```
      to specify wildcard.
    . Use "-start_line" to add waivers on checks after the specified
      line.
    . Use "-end_line" to add waivers on checks before the specified line.
    . Use "-start_column" to add waivers on checks after the specified
      column.
    . Use "-end_column" to add waivers on checks before the specified column.
    . Use "-comment" to add a required comment to the waiver.
    . Use "-export" to save waivers to the specified file.
      .Use "-do_not_apply" to prevent applying the waiver.
      .Use "-append" to append to the file if it already exists
    . Use "-silent" to suppress output and generate only Tcl return values.
    . Use "-force" to add waivers even if there are no matching properties or
      messages.

-waiver -remove waiver_id_tcl_list [-silent]
    Remove the specified waivers and return an ID list of the matching messages
    or properties.

    Use "-silent" to suppress output and generate only Tcl return values.

-waiver -show waiver_id_tcl_list [-silent]
    Show details for the specified waivers.

    Use "-silent" to suppress output and generate only Tcl return values.

-waiver -list
    [-instance instance_name]
    [-expression expr]
    [-category message_category]
    [-tag message_tag]
    [-exclude_tag exclude_tag_list]
    [-module module_name]
    [-hierarchy module_or_instance]
    [-source_file file_name]
    [-state (matched | unmatched | updated)]
    [-top]
    [-silent]

    List the IDs of the waivers that match the specified criteria.

    . Use "-instance" to list waivers added on properties of the specified
      instances. This switch supports wildcards as follows:
```

```
    . You can use a string such as "a.*.b" but not a string such as "a*".
    . Use "*" to add waivers on properties of all instances including "top"
      and ".*" to exclude "top".
    . Use "-top" to list waivers added on properties of the "top" instance.
    . Use "-expression" to list waivers added on properties with the specified
      expression.
    . Use "-category" to add waivers on messages in the specified category.
    . Use "-tag" to add waivers on messages with the specified tag.
    . Use the "-exclude_tag" switch to add waivers to messages, excluding
      those that have specific tags.
      NOTE: "-exclude_tag" switch and "-tag" switch are mutually exclusive.
    . Use "message" to add waivers on the specified message.
    . Use "-module" to add waivers on messages from the specified module.
    . Use "-hierarchy" to add waivers on messages from the specified module or
instance.
    . Use "-source_file" to list waivers added on checks from the specified
      file.
    . Use "-state" to list waivers added on checks from the specified
      waiver state.
    . Use "-silent" to suppress output and generate only Tcl return values.

-waiver -list_matching_checks waiver_id_tcl_list [-silent]
    List the IDs of properties or messages that match the specified waivers.

    Use "-silent" to suppress output and generate only Tcl return values.

-waiver -export -file_name file_name
    [waiver_id_tcl_list]
    [-append]
    [-version]
    [-unmatched]
    [-updated]
    Export waivers to the specified file.

    This command generates a Tcl file with the specified file name.

    . Use "waiver_id_tcl_list" to export only the listed waivers.
    . Use "-append" to append the extracted waivers to the specified file.
    . Use "-version" to include tool version information to the header of the specified
file.
    . Use "-unmatched" to generate waiver file with unmatched waivers only.
    . Use "-updated" to generate waiver file with updated waivers only.
```

```
-waiver -import -file_name file_name
    [-module_mapping {old_module_name=new_module_name}]
    [-silent] [-force]
    Import waivers from the specified file.

    Use "-force" to import waivers even if there are no matching properties.
    Use "-module_mapping" to be help the tool to update waivers from a design
    with a different top module.

    NOTE: When you import waivers after the design has
    changed, the tool tries to update waivers automatically. Waivers that are
    automatically imported and updated are added to the tool and reported in
    the console. However, if the tool could not update the waivers, they are
    are reported as unmatched.

-waiver -accept_update
    [waiver_id_tcl_list]
    [-confidence_score confidence_score_percentage]
    [-silent]

    Accept the update done by the tool to the specified waivers and return an
    ID list of the matching properties or messages.

    . Use "waiver_id_tcl_list" to accept updates for the specified waiver IDs.
    . Use "-confidence_score" to accept ML-generated waivers. Waivers with
      confidence scores at or above the specified value will be accepted.
      NOTE:
      . This switch accepts values in the range from 0 to 100.
      . This switch and "waiver_id_tcl_list" argument are mutually exclusive.
      . This switch is only available when the Machine Learning flow is enabled.
    . Use "-silent" to suppress output and generate only Tcl return values.

-waiver -remove_update
    [ml_generated_waiver_id_tcl_list]
    [-confidence_score confidence_score_percentage]
    [-silent]

    Remove specified ML-generated waivers. Also return a list of matching message IDs.

    . Use "ml_generated_waiver_id_tcl_list" to remove ML-generated waivers.
      This command removes the specified ML-generated waiver IDs.
    . Use "-confidence_score" to remove ML-generated waivers. This option
      removes waivers that have confidence scores equal to or below the
```

```
      specified threshold.
      NOTE:
      . This switch accepts values from 0 to 100.
      . This switch is mutually exclusive with "ml_generated_waiver_id_tcl_list".
      . This switch is only available when the Machine Learning flow is enabled.
    . Use "-silent" to suppress output and generate only Tcl return values.

-show_schematic
  ( [-id check_id [-constants_values]]
   |[-instance instance_name]
   |[-signal signal_list])+
    [-mode (functional | shift | capture | none)]
    [-hide_full_hierarchy]
    [-new_window [new_window_name]]

    Draw the schematic of the top module in the schematic viewer window,
    with the default mode "functional".
    . Use "-id" to draw schematic for the specified check.
      The violation path is highlighted in red.
      Add "-constants_values" to show constants values of nets in the design.
    . Use "-instance" to draw a specified instance instead of the top module
      in the schematic viewer window.
    . Use "-signal" to draw the schematic of the specified signal list in the
      schematic viewer window.
    . Use "-mode" to specify DFT constraints mode.
      By default, the schematic viewer opens in the "functional" mode.
    . Use "-hide_full_hierarchy" to hide upper instance hierarchies and
      keep the local hierarchy only.
    . Use "-new_window" to open the schematic viewer in a new window. And use
      the optional "new_window_name" argument to specify the new window name.
      NOTE: If you do not provide the "new_window_name" argument, the tool
      automatically generates a name.

    NOTE:
    . With the Incisive front end, schematic views are currently available for
      the following DFT checks only:
      ADRSND, CLKNTP, MULTCC, CGCENC, CMBPAU, TCDFDT, MEMNCB, ASRTSC, ACNCPI,
      CLKDAT, CLKLDT, CDFDAT, CDLDAT, CAAFSR, CACSRF, CACSRL, CAALSR, TCDLDT,
      TCKDAT, TCKLDT, ASRTCK, MULTCK, NOTCLK, NOTSCN, ASNCFL, MTOMPH,
      SEDFRS, DEEPSG, SMCKRS, UVRFCN, CONVRF, RSTENA, MOTINP, OPDNMT, FENCNT,
      TPIUNR, IPRTNR, SYNPRT, CONCLK, DATUNB, RSTINF, CKSMFC, CDRNCF, SAMESR,
      CGECCP, RSTNFR, DTUNCP, INSTCL, INSTCN, RSTCCK, CGCPCK, RSTCLK.
    . With the Jasper front end, schematic views are available for all
```

```
      structural checks.

-observability -show check_id
    [-paths] [-boundaries]
    [-silent]
    Use this command for debugging observable checks.

    . Use "-paths" to get the structural path for observable checks.
    . Use "-boundaries" to get the structural path boundaries for observable
      checks.
    . Use "-silent" to reduce the amount of path information printed to the
      screen.

    NOTE:
    This command provides additional information for debugging observability
    in Visualize. Expect path or boundary duplications as the tool returns
    only user-visible signals.

In all subcommands use "-silent" to turn off the output to the screen and
return information with a Tcl return value, which is helpful in Tcl scripts.

-gated_clock
    Returns a list of the gated clocks in the design.
    This command considers loaded Liberty pins declared with "attribute clock :
    true" as design clocks.

-constant (zero | one | any)
    Get a list of design constants.

    . Use "zero" to get constants with value equal to zero.
    . Use "one" to get constants with value equal to one.
    . Use "any" to get all design constants.
Note: "zero" and "one" return bit-level design constants.

-debug -deadcode -compute_root_cause
       -id (message_id_tcl_list | property_id_tcl_list)
    Compute dead code root causes on the specified messages or properties with
    unreachable proof status.
    Note: This command accepts only dead code properties with unreachable
    proof status; otherwise, the tool issues an error.

-debug -view -deadcode_analyzer -id (message_id | property_id)
    Open the dead code anaylzer window with the debug data of the specified
```

```
    dead code property or message.
    If there is not at least one root cause available, the command computes
    a root cause before opening the window.
    Note: This command accepts only dead code properties with unreachable
    proof status; otherwise, the tool issues an error.

Reporting Superlint App Setup
-------------------------------------
-report -setup
    [only (-design_statistics |-tool_configuration |-design_configuration)+]
    [-file file_name [-force]
      [ [-html [-launch_html_browser]
          [-source_path_mapping paths_mapping]
          [-relative_file_path]]
      |-xml |-csv]]

    Generate a setup report.

    Setup reports consist of three sections, which are Design Statistics,
    Tool Configuration, and Design Configuration.
    When you use this command without additional arguments, the tool prints a
    setup report containing all sections to the console.

    . Use "only" to include specified sections only.
    . Use "-file" to specify a file name. And use "-force" to overwrite an
      existing file.
      . Use "-html" to report results as HTML.
        . Use "-launch_html_browser" to launch an HTML browser with results.
        . Use "-source_path_mapping" to specify the mapping between UNIX(R) and
          Microsoft(R) Windows(R) when using Microsoft(R) Internet Explorer(R).
          To link directly to the source files, the mapped paths should
          be separated with the character '='; for example, /home/user_name=z:
        . Use "-relative_file_path" to open the source files of the relative
          file path in the HTML report. The same relative file path is used in
          the elaboration.
      . Use "-xml" to report results as XML.
      . Use "-csv" to report results as CSV.
      . If you do not specify a format, the default is text.
      NOTE: If you do not specify a file, the report is printed to the console.

Reporting Superlint App Checks
------------------------------
-report
```

```
[-property
    [-status (  cex | unreachable | proven
              | covered | undetermined | unprocessed)+]
    [-type (assert | cover)+]
    [-task task_list]]
[-violation
    [-severity (error | warning | info)+]
    [-order (  category | tag | filename | instance(module) | label
             | severity | mode)]
    [-include_design_build]]
[-detailed [-verbose (tag_list | all)]]
[-domain domain_list]
[-category category_list]
[-tag tag_list]
[-instance instance_list]
[-regexp]
[-waived (none | only [-user_tcl |-user_rtl |-auto]+)]
[-grouping (none | basic | advanced)]
[-file file_name [-force]
  [ [-html [-launch_html_browser]
      [-source_path_mapping paths_mapping]
      [-relative_file_path]]
  |-xml |-csv]]

Generate a checks report.

When you use this command without additional arguments, the tool prints a
violations summary to the console.

. Use "-property" to generate a properties report.
  . Use "-status" to generate a report for the specified statuses only. The
    default is all.
  . Use "-type" to generate a report for the specified types only. The
    default is all.
  . Use "-task" to generate a report for the specified tasks only. The
    default is all. This switch supports regular expressions.
. Use "-violation" to generate a violations report.
  . Use "-severity" to generate a report for the specified violation
    severities only. The default is all.
  . Use "-order" to generate a report ordered by the specified criteria.
    The default is "category".
    NOTE: Use "mode" to generate a report sorted by modes of
    violations. This is applicable only in the Multimode Summary View.
```

```
  . Use "-include_design_build" to include violations of design build.
  NOTE: Running the violations report command in the Multimode Summary View
  generates a report that includes all violations and their modes.
. Use "-domain" to generate a report for the specified domains.
  This option works with rule violations and formal domains only.
. Use "-category" to generate a report for the specified categories.
  This option works with rule violations and formal categories only.
. Use "-tag" to generate a report for the specified tags.
  This option works with rule violations and formal tags only.
. Use "-instance" to generate a report for the specified instances.
. Use "-regexp" to consider the category, tag, or instance name a regular
  expression.
. Use "-waived only" to generate a report for waived checks that were
  waived by waivers of the specified types only. If no waiver type is
  specified, a report of all waived checks will be generated.
. Use "-waived none" to exclude waived checks from the report. By default,
  waived checks are included in the report.
. Use "-grouping none" to generate a report with no grouping information.
. Use "-grouping basic" to generate a report where only same tags can be
  grouped together, reporting only the primary.
. Use "-grouping advanced" to generate a report where only specific tags
  can be grouped together, reporting only the primary.
NOTE: Grouping is a GUI concept that groups related checks and shows only
one check, which is the primary. By default, the report is generated with
the app's current grouping option.
. Use "-detailed" to generate a detailed report. By default, the tool
  generates a summary report only. And use "-vebose" to include verbose
  information on the specified tags.
NOTE: "-verbose" switch works only with Violations reports.
. Use "-file" to specify a file name. And use "-force" to overwrite an
  existing file.
  . Use "-html" to report results as HTML.
    . Use "-launch_html_browser" to launch an HTML browser with results.
    . Use "-source_path_mapping" to specify the mapping between UNIX(R) and
      Microsoft(R) Windows(R) when using Microsoft(R) Internet Explorer(R).
      To link directly to the source files, the mapped paths should
      be separated with the character '='; for example, /home/user_name=z:
    . Use "-relative_file_path" to open the source files of the relative
      file path in the HTML report. The same relative file path is used in
      the elaboration.
  . Use "-xml" to report results as XML.
  . Use "-csv" to report results as CSV.
  . If you do not specify a format, the default is text.
```

```
      NOTE: If you do not specify a file, the report is printed to the console.

Reporting Superlint App Waivers
---------------------------------------
-report -waiver
    [only (-user_tcl |-user_rtl |-auto)+]
    [-waiver_status (matched | unmatched)+]
    [-detailed]
    [-file file_name [-force]
      [ [-html [-launch_html_browser]
          [-source_path_mapping paths_mapping]
          [-relative_file_path]]
      |-xml |-csv]]

    Generate a waivers report.

    When you use this command without additional arguments, the tool prints
    to the console a waivers report containing a summary of all waivers in
    addition to one section for every waiver type that contains information of
    each waiver of this type.

    . Use "only" to include waivers of the specified types only. By default,
      waivers of all types are included.
    . Use "-waiver_status" to include waivers with the specified waiver
      statuses only. By default, waivers with all statuses are included.
    . Use "-detailed" to add the checks that are waived by this waiver beneath
      every waiver's information. By default, waived checks are not added.
    . Use "-file" to specify a file name. And use "-force" to overwrite an
      existing file.
      . Use "-html" to report results as HTML.
        . Use "-launch_html_browser" to launch an HTML browser with results.
        . Use "-source_path_mapping" to specify the mapping between UNIX(R) and
          Microsoft(R) Windows(R) when using Microsoft(R) Internet Explorer(R).
          To link directly to the source files, the mapped paths should
          be separated with the character '='; for example, /home/user_name=z:
        . Use "-relative_file_path" to open the source files of the relative
          file path in the HTML report. The same relative file path is used in
          the elaboration.
      . Use "-xml" to report results as XML.
      . Use "-csv" to report results as CSV.
      . If you do not specify a format, the default is text.
      NOTE: If you do not specify a file, the report is printed to the console.
```

```
Reporting Superlint App Signoff
---------------------------------------
-report -signoff
    [-file file_name [-force]
      [ [-html [-launch_html_browser]
          [-source_path_mapping paths_mapping]
          [-relative_file_path]]
      |-xml |-csv]]

    Generate a signoff report.

    Signoff reports include the design statistics, the signoff information
    (delta of used settings and rule file with the default settings and rule
    file) of the tool configuration, the design configuration, and summaries of
    the violations, waived violations and waivers.
    When you use this command without additional arguments, the tool prints
    the signoff report to the console.

    . Use "-file" to specify a file name. And use "-force" to overwrite an
      existing file.
      . Use "-html" to report results as HTML.
        . Use "-launch_html_browser" to launch an HTML browser with results.
        . Use "-source_path_mapping" to specify the mapping between UNIX(R) and
          Microsoft(R) Windows(R) when using Microsoft(R) Internet Explorer(R).
          To link directly to the source files, the mapped paths should
          be separated with the character '='; for example, /home/user_name=z:
        . Use "-relative_file_path" to open the source files of the relative
          file path in the HTML report. The same relative file path is used in
          the elaboration.
      . Use "-xml" to report results as XML.
      . Use "-csv" to report results as CSV.
      . If you do not specify a format, the default is text.
      NOTE: If you do not specify a file, the report is printed to the console.

Exporting Superlint App Properties
---------------------------------------
-export
    -to_sva sva_file_name
    [-to_tcl tcl_file_name]
    [-remove_duplicate_properties]
    [-module sva_module_name]
    [-initial_block sva_initial_block]
    [-action_block sva_action_block]
```

```
[-type (all | Assert | Assume | Cover)+]
[-waived (none | only)]
[-status (all | cex | proven | unprocessed
          | unreachable | undetermined)+ ]
[-category category_tcl_list]
[-instance instance_tcl_list]
[-top]
[-property id_tcl_list]

Export structural properties as SVA modules.

. Use "-to_sva" to specify the name of the exported SVA file.
. Use "-to_tcl" to create a Tcl script for loading the exported SVA file
  in Jasper Apps.
. Use "-remove_duplicate_properties" to export properties with the same
  expression and clock only once.
. Use "-module" to specify a module name for SVA export, that is, to
  overwrite the default module name.
. Use "-initial_block" to add an initial block to the exported SVA file.
  For example,

  check_superlint -export -to_sva ... -initial_block {
    begin
      $display("Loaded Jasper assertions.")
    end
   }

  NOTE: You must use the "set_export_sva_target_environment" command to
  change the target environment to simulation or emulation when you use
  this switch.
. Use "-action_block" to add an action block to assertions and
  assumptions in the exported SVA file. For example,

  check_superlint -export -to_sva ... -action_block {
    begin
      `UVM_ERROR("Assertion %name% failed.");
      $assertoff(%name%);
    end
   }

  NOTE:
  . In this example, the tool replaces %name% in the action block with
    the property name.
```

```
      . You must use the "set_export_sva_target_environment" command to
        change the target environment to simulation or emulation when you
        use this switch.
    . Use "-type" to specify the types of properties to export.
      The default is "assert" and "cover".
    . Use "-waived only" to export waived properties only, and use
      "-waived none" to omit waived properties from the export. By default,
      all properties are exported regardless of waived status.
    . Use "-status" to specify the statuses of properties to export.
      The default is all.
    . Use "-category" to specify the categories of properties for export.
      The default is all.
    . Use "-instance" to specify the instances of properties to export.
    . Use "-top" to include the top instance with the instances of properties
      to export.
      Note: This switch is equivalent to including the top instance name in
      the list given to "-instance" switch.
    . Use "-property" to specify a set of structural property IDs to export.
      NOTE: The set of property IDs is filtered based on any other criteria
      provided.

Bottom-Up Hierarchical Analysis
-------------------------------
-import -file file_name

    Import a previously saved Superlint database to the SoC-level to verify
    constraint information for black-boxed instances.


Superlint FSM Checks
----------------------------------------
-fsm
    [ -list |-show fsm_signal_name [-export_xml file_name]
     |-get_property fsm_signal_name [-state state_name]]

    Query the FSMs extracted by the Superlint App.

    . Use "-list" to get a list of signals Superlint detected as FSMs.

    . Use "-show" with the FSM name to get information about the specified FSM,
      that is, its states, transitions, idle state, and mapping to state names.

      . Use "-export_xml" with a file name to export the specified FSM information,
```

```
that is, its states, transitions and their statuses to an XML file.

    . Use "-get_property" to get the task name, property name, status, and tag
      for the properties related to the specified FSM.

      For example:
      check_superlint -fsm -show brdg.state
      states {`EG_IDLE `EG_ADDR `EG_DATA} transitions {(`EG_IDLE |=> `EG_IDLE)
(`EG_IDLE |=> `EG_ADDR) (`EG_ADDR |=> `EG_ADDR) (`EG_ADDR |=> `EG_DATA) (`EG_DATA |=>
`EG_IDLE) (`EG_DATA |=> `EG_ADDR) (`EG_DATA |=> `EG_DATA)} state_map {`EG_IDLE = 0
,`EG_ADDR = 1 ,`EG_DATA = 2} idle_state `EG_IDLE

      array set info [check_superlint -fsm -show brdg.state];
      foreach attr [array names info] { puts "$attr: $info($attr)" }
      states: `EG_IDLE `EG_ADDR `EG_DATA
      idle_state: `EG_IDLE
      state_map: `EG_IDLE = 0 ,`EG_ADDR = 1 ,`EG_DATA = 2
      transitions: (`EG_IDLE |=> `EG_IDLE) (`EG_IDLE |=> `EG_ADDR) (`EG_ADDR |=>
`EG_ADDR) (`EG_ADDR |=> `EG_DATA) (`EG_DATA |=> `EG_IDLE) (`EG_DATA |=> `EG_ADDR)
(`EG_DATA |=> `EG_DATA)

      check_superlint -fsm -get_property brdg.state
      {task SL_AUTO_FORMAL_FSM property fsm_state_cover_EG_ADDR_prop_2 status covered
tag FSM_NO_RCHB} {task SL_AUTO_FORMAL_FSM property
fsm_trans_cover_EG_IDLE_EG_ADDR_prop_5 status covered tag FSM_NO_TRRN}

      check_superlint -fsm -get_property brdg.state -state `EG_IDLE
      {task SL_AUTO_FORMAL_FSM property fsm_trans_cover_EG_IDLE_EG_ADDR_prop_5 status
covered tag FSM_NO_TRRN}

GUI Operations
---------------------------------------
-gui
   ( [-reinvoke_mode (reinvoke | rerun)]
    |[-order_by (domain | tag | message | filename | severity | label |
instance(module))]
    |[-grouping_mode (none | basic | advanced)])+
    |[-view -display (design_hierarchy)]
    |[-view -hide (design_hierarchy)]
    |[-show_file_mode (file_name | full_path)]

    Perform operations specific to the GUI.
    . Use "-reinvoke_mode" to specify the mode used by the "Reinvoke" button.
```

```
    . Use "reinvoke" to clear all and then reapply "analyze", "elaborate",
      "clock", "reset", and signals configuration commands as well as waivers.
    . Use "rerun" to clear all and then reapply all commands and settings
      used in the previous run.
. Use "-order_by" to specify the order preference for the violation
  messages view. The default is "domain".
. Use "-grouping_mode" to specify the grouping mode preference for the
  violation messages view.
    . Use "none" to view violation messages with no grouping information.
    . Use "basic" to view violation messages where only same tags can be
      grouped together, viewing only the primary.
    . Use "advanced" to view violation messages where only specific tags can be
      grouped together, viewing only the primary.
  The default is "basic".
. Use -view -display to display the specified view.
. Use -view -hide to hide the specified view.
. Use "-show_file_mode" to modify the file information displayed in the "File"
  column of the "Violation Messages View".
  This command also affects the display when you order by "Filename".
    . Use "file_name" to view the file name only.
    . Use "full_path" to view the full path.
```

Example

```
check_superlint -extract
check_superlint -extract -task .
check_superlint -list -regexp {.*my_instance_name.*}
check_superlint -list -source_location /a/b/c.v:2
check_superlint -list -source_location /a/b/c.v:2-100
check_superlint -list -file propsAndMsgs
check_superlint -waiver -add -property {1 2 3} -comment "Expected to fail in this
configuration"
check_superlint -waiver -add -instance a.b.* -category AUTO_FORMAL_FSM fsm -comment
"Waive all properties for type FSM under a.b subinstances"
check_superlint -waiver -add -tag BLK_NO_RCHB -source_file "/a/b/c.v" -start_line 100 -
start_column 2 -comment "Waive unreachable code in the /a/b/c.v file from line 100"
check_superlint -waiver -add -exclude_tag {BLK_NO_RCHB} -comment "Waive all messages
except for those with the BLK_NO_RCHB tag"
check_superlint -waiver -remove {1 2 3}
check_superlint -waiver -remove [check_superlint -waiver -list]
check_superlint -waiver -export -file /a/b/mywaivers.tcl
check_superlint -waiver -import -file /a/b/c/mywaivers.tcl
check_superlint -waiver -accept_update {1 2 3}
check_superlint -waiver -accept_update -confidence_score 85.5
check_superlint -waiver -remove_update {1 2 3}
check_superlint -waiver -remove_update -confidence_score 85.5
check_superlint -waiver -list_matching_checks {1 2}
check_superlint -waiver -list -source_file /a/b/c.v -expression ~a.rstN
check_superlint -prove -task .
check_superlint -report -setup only -tool_configuration -design_configuration
check_superlint -report -violation -property -csv -file checks.csv -detailed
check_superlint -report -waiver only -auto -user_tcl -waiver_status matched -detailed
check_superlint -report -signoff -file signoff_report.html -html -launch_html_browser
check_superlint -export -to_sva my_props.sva -category {AUTO_FORMAL_FSM_REACHABILITY
AUTO_FORMAL_OVERFLOW} -waived none -top
check_superlint -export -to_sva my_sva.sva -to_tcl sva_loader.tcl -property {2 3 5 7
11} -type all -status {cex proven}
check_superlint -gui -reinvoke_mode rerun
check_superlint -gui -order_by tag
check_superlint -gui -grouping_mode advanced
check_superlint -gui -view -display design_hierarchy
check_superlint -gui -show_file_mode file_name
```

See Also

config_rtlds,
set_superlint_fsm_automatic_assumptions_type,
set_superlint_signals_automatic_assumptions_type,
set_superlint_signal_type,
set_superlint_stuck_at_bitblasted,
set_superlint_verbosity_level
set_superlint_loop_filter_startpoints

check_timing_exception

Sets up, manages, and reports Timing Exception flow.

Syntax

```
check_timing_exception -init

check_timing_exception -export -to file_name
                       [-force]

check_timing_exception -generate
                       [-task task_name]
                       [-exception_type (all | mcp | fp)+]

check_timing_exception -prove
                       [-check_id check_id |-command_id command_id]
                       [-threads num_parallel_threads]
                       [-bg]

check_timing_exception -list (timing_exception_command | timing_exception_check)

check_timing_exception -show -command_id command_id
                       [-list (file_name | line_number
                               | exception_type | analysis_type
                               | end_point_transition_type
                               | clock_edge | number_of_cycles
                               | from_point | from_transition_type
                               | through_points
                               | through_transition_types
                               | to_points | to_transition_type
                               | comment | status)+]

check_timing_exception -show -check_id check_id
                       [-list (timing_exception_command_id
                               | analysis_type | number_of_cycles
                               | launch_clock_name | capture_clock_name
                               | start_points | through_points
                               | end_points | status)+]
```

```
check_timing_exception -report
                       [-check_id check_id |-command_id command_id]
                       [-file file_name [-force]]
                       [-detailed]

check_timing_exception -debug
                       (-check_id check_id |-command_id command_id)
                       [-violation (visualize | schematic)]

check_timing_exception -audit
                       [-file file_name [-force]]
                       [-detailed]

check_timing_exception -waiver
                       ( -add ( -command (-source_loc source_loc |-hash hash)+
                              |{-check (-source_loc source_loc|-hash hash) -launch_clk
launch_clk}+
                             ) [-comment comment]
                        |-list
                        |-remove waiver_ids
                        |-export (-file file_name [-force])
                        |-edit -id id -comment comment
                        |-clear
                       )

check_timing_exception -clear
```

Returns

Multiple return values.

Detailed Description

Use the "check_timing_exception" command to set up Timing Exception flow with Timing Exception constraints from SDC, export Timing Exception database to be read in Xcelium, generate Timing Exception formal checkers (MCP only), or generate a Timing Exception verification report for sign-off.

-init

    Initialize settings required for the Timing Exception flow.

```
-export -to file_name
    [-force]

    Export Timing Exception database to be read in Xcelium.

    . Use "-to" to specify the JSON file name.
    . Use "-force" to overwrite any existing file that has the specified file
      name.

-generate [-task task_name]

    Generate Timing Exception formal checkers. The Timing Exception
    constraints supported are MCP and FP.

    . Use "-task" to specify a task name (default: "MCP").
    . Use "-exception_type" to specify the verification type. "fp" for false
      path, "mcp" for multi cycle path, or "all" for both (default: "mcp").

-prove
    [-check_id check_id | -command_id command_id]
    [-threads num_parallel_threads]
    [-bg]

    Try to prove MCP properties that were generated using
    "check_timing_exception -generate".
    Return values are the same as the "prove" command.

    . Use "-check_id" to prove MCP properties based on a specific check ID, or
      "-command_id" to prove MCP properties based on a specific command ID. If
      neither switch is provided, the tool proves properties from all commands.
    . Use "-threads" to specify the maximum number of parallel proof threads.
    . Use "-bg" to run the command in the background and continue to accept and
      run new commands.

-list (timing_exception_command | timing_exception_check)
    Returns a list of existing command or check IDs.

-show -command_id command_id
    [-list { [file_name] [line_number] [exception_type] [analysis_type]
             [end_point_transition_type] [clock_edge] [number_of_cycles]
             [from_point] [from_transition_type] [through_points]
             [through_transition_types] [to_points] [to_transition_type]
```

```
            [comment] [status] }]

    Returns information in the form of a Tcl dict based on the provided ID.
    The first element in each pair is the name of an attribute and the second
    element in each pair is the value of the previous attribute.

    Use "-list" to return a Tcl list of specified values. Each element in the
    returned list is the value of the corresponding attribute in the attribute
    list.

-show -check_id check_id
    [-list { [timing_exception_command_id] [analysis_type] [number_of_cycles]
             [launch_clock_name] [start_points]
             [through_points] [end_points] [status] }]

    Returns information in the form of a Tcl dict based on the provided ID.
    The first element in each pair is the name of an attribute and the second
    element in each pair is the value of the previous attribute.

    Use "-list" to return a Tcl list of specified values. Each element in the
    returned list is the value of the corresponding attribute in the attribute
    list.

-report
    [-check_id check_id |-command_id command_id]
    [-file file_name [-force]]
    [-detailed]

    Generate a Timing Exception verification report for sign-off.

    . Use "-check_id" to generate a report based on a specific check ID, or
      "-command_id" to generate a report based on a specific command ID. If
      neither switch is provided, the tool generates a full report.
    . Use "-file" to export the report to a file. If this switch is not
      provided, the report is only printed on the console.
    . Use "-force" to overwrite any existing file that has the specified file
      name.
    . Use "-detailed" to get the detailed report. If the switch is not provided,
      a summary report is created.

-debug (-command_id command_id | -check_id check_id)
    [-violation (visualize | schematic)]
```

```
    Generates a violation trace and returns the information through a Tcl
    dictionary, Visualize window or a Schematic Viewer window with MCP context.
    The Visualize window displays the values of "FROM", "TO" and "THROUGH"
    signals specified in the MCP/FP command and all the signals in the violation
    path.
    The Schematic Viewer window displays all signals from the start to the end
    point in the violation path.

    . Use "-check_id" to specify the check ID or "-command_id" to randomly
      select a check from the specified command ID that contains a violation.
    . Use "-violation visualize" to open a Visualize window for the violation.
    . Use "-violation schematic" to open a Schematic Viewer window for the
      violation.

-audit
    [-file file_name [-force]]
    [-detailed]

    Generates a full MCP App flow report, highlighting incomplete steps that may
    compromise the integrity of the overall MCP proof result.
    The report is divided into five major sections:
    . SDC Commands Parsing: Analysis of syntax and basic checks for SDC
      commands.
    . SDC Constraints Extraction: In-depth path analysis required to generate
      formal checks.
    . Formal Checks Generation: Formal verification checks required to run MCP
      proof.
    . Formal Verification: Formal verification results.
    . Audit Summary: Summarization of all former results.

    . Use "-file" to export the report to a file. If this switch is not
      provided, the report is only printed on the console.
    . Use "-force" to overwrite any existing file that has the specified file
      name.
    . Use "-detailed" to get the detailed report. If the switch is not provided,
      a summary report is created.

-waiver
    ( -add ( -command (-source_loc source_loc |-hash hash)+
            |{-check (-source_loc source_loc |-hash hash) -launch_clk launch_clk}+
          ) [-comment comment]
    |-list
    |-remove waiver_ids
```

```
    |-export (-file file_name [-force])
    |-edit -id id -comment comment
    |-clear
    )

    . Use "-add" to add a new waiver for MCP pairs based on command or checks.
      . Use "-command" to specify a waiver in the command level.
      . Use "-check" to specify a waiver in the check level.
      . Use "-source_loc" to specify the source location (file and line).
      . Use "-launch_clk" to specify the launch clock of the given check.
      . Use "-hash" to specify the hash of the given command.
      . Use "-comment" to specify a comment to describe the waiver.
    NOTE: The "-hash" is solely used in import/export flow.
    . Use "-list" to list all waivers.
    . Use "-remove" to remove waivers.
    . Use "-export" to export all waivers into a file.
      . Use "-file" to specify the file name.
      . Use "-force" to overwrite an existing file.
    . Use "-edit" to edit a given waiver id.
      . Use "-id" to specify the waiver.
      . Use "-comment" to specify the new waiver comment.
    . Use "-clear" to clean all waivers.

-clear
    Clear all the Timing Exception environment from the current session.
```

Example

```
check_timing_exception -generate
check_timing_exception -export -to database.json
check_timing_exception -show -command_id 1 -list {file_name line_number status}
check_timing_exception -show -check_id 1 -list {launch_clock_name status}
check_timing_exception -report -detailed
check_timing_exception -debug -check_id 1 -violation visualize
check_timing_exception -generate -exception_type {fp}
```

See Also

read_sdc

check_unr

Analyzes unreachable branch, expression, fsm, and toggle signals in a design.

Associated App(s)

UNR Analysis

Syntax

```
check_unr -init
            [-coverage {block | expr | toggle | fsm | functional}+]
            [-covdb covdb_path]
            [-dutinst dutInst]
            [-load_refinement refine_files]
            [-enable_assumption]
            [-covtest covtest_name]
            [-covscope covscope_name]
            [-covworkdir covworkdir_name]
            [-inst_top instance_name]
            [-write_metrics]
            [-covoverwrite]
            [-jg_periodicexport period_number]

check_unr -setup
            [-task task_name]

check_unr -enable_property
            (property_name_tcl_list [-regexp]
            |-task task_name
            |-all
            )

check_unr -disable_property
            (property_name_tcl_list [-regexp]
            |-task task_name
            |-all
            )
```

```
check_unr -list
            [properties]
            [-type (  reachable | unreachable | bounded | unprocessed
                            | enabled | disabled)]
            [-regexp regular_expression]
            [-task task_name]

check_unr -clear
            [-task task_name]

check_unr -prove
            [-all |-property property_name_list |-task task_name]
            [-max_properties_per_chunk N]
            [-max_chunks N]
            [-shallow_analysis]
            [-time_limit time]
            [-engine_mode engine_list]
            [-verbosity N]
            [-with_ppd path_to_ppd_file]
            [-save_ppd path_to_ppd_file]
           [-keep_trace]
            [-silent]
            [-bg]

check_unr -generate_partition
            [-threshold N]
            [-save path_of_xml_file]
     [-silent]
     [-force]

check_unr -load_partition
            [path_of_xml_file_to_load]
     [-silent]

check_unr -run_partition
            [-input input_tcl]
     [-max_jobs N]
     [-keep_run]

check_unr -reason
            [-file file_name]
```

Returns

Multiple return values.

Detailed Description

Use this command to find unreachable block, toggle, fsm or expression signals in a design.

```
-init
       [-coverage {block | expr | toggle | fsm | functional}+]
       [-covdb covdb_path]
       [-dutinst dutInst]
       [-load_refinement refine_files]
       [-enable_assumption]
       [-covtest covtest_name]
       [-covscope covscope_name]
       [-covworkdir covworkdir_name]
       [-inst_top instance_name]
       [-write_metrics]
       [-covoverwrite]
       [-jg_periodicexport period_number]

       Initialize UNR app to generate coverage targets.
       NOTE: This command must be used before the "elaborate" command.

       . Use "-coverage" to specify the types of UNR coverage properties you
         want to generate.

         NOTE:
         . Use "set_unr_functional_coverage true" to enable functional coverage for the
           UNR analysis.
         . Use "set_unr_functional_coverage_mode assertion |covergroup |all" to select
           functional coverage mode for the UNR analysis.
         . Use "assertion" to select only assertion witnesses and covers items.
         . Use "covergroup" to select only covergroup items.
         . Use "all" to select all functional coverage items. It is also the default
mode.

       . Use "-covdb" to specify the simulation coverage database used for the
         UNR target generation.
```

```
      . Use "-dutinst" to specify top level instance name enabled for
        coverage.
      . Use "-load_refinement" to specify waiver file to filter the coverage
        targets. Only ".vRefine" files are supported.
      . Use "-enable_assumption" to enable embedded (SVA) assumptions for the
        UNR analysis.
      . Use "-covtest" to specify covtest directory name for UNR coverage
        database.
      . Use "-covscope" to specify covscope directory name for UNR coverage
        database.
      . Use "-covworkdir" to specify covworkdir directory name for UNR coverage
        database.
      . Use "-inst_top" to specify top hierarchical instance path enabled for UNR
        coverage.
      . Use "-write_metrics" to generate the .vsof file
      . Use "-covoverwrite" to overwrite the existing database.
      . Use "-jg_periodicexport" to specify the interval to export database.

-setup
      [-task task_name]

      Extract and enable all UNR coverage properties in the current task.

      NOTE:
      . By default, the UNR App creates its own task named "unr".
      . In a UNR task, the user-defined assumptions are enabled by default
        while other user properties are not enabled.
      . No task is created if you have already created a task and launched
        this command on that task.
      . No environment except user assumptions will be copied from the
        <embedded> task.
      . If a UNR task already exists, then the integer number is suffixed in
        the UNR task name.
      . Once the setup is successful, you cannot provide another "-setup"
        command without using the "-clear" command.
      . The command creates the file "discarded_items_report.txt" under the
        current session directory, containing a human-readable text report of
        uncovered items from the simulation coverage database that cannot be
        evaluated, along with the respective reason for each.

      . Use "-task" to enable UNR coverage properties in a specified task.

-enable_property
```

```
       property_name_tcl_list -regexp |-task task_name |-all

       Enable the specified UNR coverage properties.

       . Use "-regexp" to enable the UNR coverage properties that match
         the specified regular expression.
       . Use "-task" to enable UNR coverage properties in a specified task.
       . Use "-all" to enable all UNR coverage properties.

-disable_property
       property_name_tcl_list -regexp |-task task_name |-all

       Disable the specified UNR coverage properties.

       . Use "-regexp" to disable the UNR coverage properties that match the
         specified regular expression.
       . Use "-task" to disable UNR coverage properties in a specified task.
       . Use "-all" to disable all UNR coverage properties.

-list
       [{properties}]
       [-type (  reachable | unreachable | bounded | unprocessed | enabled
              | disabled)]
       [-task task_name]
       [-regexp]

       Display all UNR properties along with their status as described below:

         BND - Bounded    : No conclusive result found for property during UNR
                            analysis.
         DSB - Disabled   : Property is not active for UNR analysis.
         RCH - Reachable  : Property is reachable.
         UNP - Unprocessed: Property is not processed for UNR analysis.
         UNR - Unreachable: Property is not reachable.

       . Use "-type" to filter out UNR coverage properties based on the
         specified status.
       . Use "-task" to list the UNR coverage properties of the specified task.
       . Use "-regexp" to list the UNR coverage properties that match the
         specified regular expression.

-clear
       [-task task_name]
```

```
       Clear all UNR coverage properties from the current task.

       Use "-task" to clear UNR coverage properties in a specified task.

-prove
       [-all |-property {property_name_list} |-task task_name]
       [-max_properties_per_chunk N]
       [-max_chunks N]
       [-shallow_analysis]
       [-time_limit time]
       [-engine_mode {engine_list}]
       [-verbosity N]
       [-with_ppd path_to_ppd_file]
       [-save_ppd path_to_ppd_file]
       [-keep_trace]
       [-silent]
       [-bg]

       Find the reachable UNR coverage properties using prove orchestration.

       . Use "-all" to prove all UNR coverage properties in all tasks.
       . Use "-property" to prove the specified UNR coverage properties.
       . Use "-task" to prove UNR coverage properties of a given task.
       . Use "-max_properties_per_chunk" to set the maximum number of
         properties per proof chunk, overriding the global configuration.
         See "set_unr_prove_max_properties_per_chunk".
       . Use "-max_chunks" to set the maximum number of chunks for dividing
         proof properties, overriding the global configuration. See
         "set_unr_prove_max_chunks".
       . Use "-shallow_analysis" to prove properties with minimum effort.
       . Use "-time_limit" to override the current "prove_time_limit" for
         this "prove" command. See "set_prove_time_limit".
       . Use "-engine_mode" to override the current "engine_mode" for this
         "prove" command. See "set_engine_mode".
       . Use "-verbosity" to override the current "prove_verbosity"
         for this "prove" command. See "set_prove_verbosity".
       . Use "-with_ppd" to read proof profiling data (PPD) previously saved
         with "-save_ppd". The tool uses this information to make future
         runs faster and consume less resources.
       . Use "-save_ppd" to save PPD for use with future runs to potentially
         make them run faster and consume less resources.
       . Use "-keep_trace" to disable trace suppression.
```

```
        NOTE: In UNR prove, default trace suppression is enabled.
      . Use "-silent" to ignore proof messages. This option is an alias to
        "-verbosity 0".
      . Use "-bg" to run the command in the background and continue to accept
        and run new commands.

-generate_partition
       [-threshold N]
       [-save path_of_xml_file]
       [-silent]
       [-force]

       Generate partitions of the given design for UNR Analysis.

       . Use "-threshold" to specify the maximum individual partition size.
         The default value is around 2 million.
         NOTE: Threshold is a recommendation and may not be strictly followed.
         It can be increased or decreased to generate balanced partitions.
       . Use "-save" to save the generated partitions in a xml file,
         which can be loaded and run later.
       . Use "-silent" to ignore partition generation messages.
       . Use "-force" to rewrite in the existing xml file.


-load_partition
       [path_of_xml_file_to_load]
       [-silent]

       Load the specified file to run the partitions for UNR Analysis.

       . Use "-silent" to ignore messages.

-run_partition
       [-input input_tcl]
       [-max_jobs N]
       [-keep_run]

       Run each partition in a different session for UNR Analysis.

       . Use "-input" to specify the input file with customized
         UNR Analysis steps.
       . Use "-max_jobs" to specify the number of partitions
         that run parallel. The default value is 3.
```

```
      . Use "-keep_run" to keep the session alive of each partition.
        By default, each session closes automatically after
        finishing that partition's UNR Analysis.

-reason
      [-file file_name]

      Creates the files "unr_reason.txt" and "unr_reason.cmd" under the current
      session directory; the first being a human-readable report containing the
      reason for UNRs detected via constant propagation, and the second being a
      IMC script to generate a refinement file excluding the same UNRs.

      . Use "-file" to change the root name and path of the created files (for
        example, using "-file ./path/file_name" creates both
        "./path/file_name.txt" and "./path/file_name.cmd").
```

Example

```
check_unr -init -coverage all
check_unr -setup -coverage {block expr toggle fsm} -covdb test -load_refinement
refine1.vRefine
check_unr -disable_property .* -regexp
check_unr -enable_property .*block.* -regexp
check_unr -list
check_unr -prove
database -export_unicov
check_unr -clear
check_unr -generate_partition -threshold 50000 -save ./partition.xml
check_unr -load_partition ./partition.xml
check_unr -run_partition -input unr.tcl -max_jobs 2
check_unr -reason -file ./path/file_name
```

See Also

set_unr_break_clock_loop,
set_unr_functional_coverage,
set_unr_functional_coverage_mode,
set_unr_prove_max_properties_per_chunk,
set_unr_prove_max_chunks

check_xprop

Runs X-propagation analysis on one or more signals.

Associated App(s)

X-Propagation Verification

Syntax

```
Initializing X-Prop Analysis
----------------------------
check_xprop -init

check_xprop ((-init_control (true | false) [-sequential |-expression])
             ((-init_data | -init_all_data) (true | false))
             (-init_index (true | false))
             (-exclude_bind_hierarchies))+

check_xprop ((-init_all_control (true | false) [-sequential |-expression])
             ((-init_data | -init_all_data) (true | false))
             (-init_index (true | false))
             (-exclude_bind_hierarchies))+

Creating X-Prop Assertions
--------------------------
check_xprop -create
            [-task task [-no_prefix]]
            [-name name]
            [-from signal_name]
            [-to signal_list]
            [-clocks_and_resets]
            [-control [-property_naming_style (default | long_filename |
short_filename)]
                [-no_flops_with_reset_pin [-include_sync_resets]]]
            [-data
                [-property_naming_style (default | long_filename | short_filename)]]
            [-index
                [-property_naming_style (default | long_filename | short_filename)]]
```

```
            [-outputs]
            [-bbox_inputs]
            [-flops_with_reset_pin [-include_sync_resets] [-no_temporary_buffers]
                [-property_naming_style (default | long_filename | short_filename)]]
            [-flops_with_reset_value [-include_sync_resets]
                [-property_naming_style (default | long_filename | short_filename)]]
            [-counters]
            [-fsms]
            [-precond expression]
            [-no_internal_precond]
            [-no_precondition_cover]
            [-create_precondition_cover]
            [-sampling_clock clock]
            [-from_file file_name]
            [-bit_blast |-no_bit_blast]
            [-instance {instance_list} [-regexp]]
            [-exclude_hierarchies {instance_list}]
            [-filter filter_expression [-regexp] [-case_sensitive]]
            [-filter_out filter_expression [-regexp] [-case_sensitive]]
            [-delay N]

X-Prop Abstractions
-------------------
check_xprop -abstract (multiplier | shift | adder | subtract | comparator)
            [-exclude_module {module_list} [-regexp] [-recursive]]
            [-include_module {module_list} [-regexp] [-recursive]]
            [-exclude_instance {instance_list} [-regexp]]
            [-include_instance {instance_list} [-regexp]]
            [-minimum_width integer]
            [-skip_const_operand]
            [ -env
             |-task task_name]

check_xprop -abstract (-instance instance_list)
            [-mode (seq | comb | mixed)]
            [-task task_name |-env]

check_xprop -abstract -remove abstraction_name_tcl_list
            [ -env
             |-task task_name]

check_xprop -abstract -clear
            [ -env
```

```
             |-task task_name]

check_xprop -abstract [multiplier | shift | adder | subtract | comparator] -list
            [ -env
             |-task task_name]
            [-silent]

Reporting Property Expressions
------------------------------
check_xprop -get_expression
            -property property_name
            [-precond |-target]

Proving X-Prop Assertions
-------------------------
check_xprop -prove
               [ -task task_name
                |-property property_name_tcl_list [-regexp]
                  [-strategy (basic | proof)]
               ]
               [-force]
               [-time_limit]
               [-bg]
               [-engine_mode engine_tcl_list]
               [-skip_related_covers]
               [-cex_limit number_of_failures]
               [-from property_name]
               [-with_proven]
               [-with_helpers]

Debugging X-Prop Assertions
---------------------------
check_xprop -waiver -add
            -property property_name_list
            -comment comment
            [-regexp]

check_xprop -waiver -add
            ([-instance instance_name | -top]
             [-category (clocks_and_resets | bbox_inputs | control_points
                | data_points | array_index | outputs | flops_with_reset_pin
                | counters | fsms | user_defined)]
             [-module module_name]
```

```
             [-source_file file_name
               [-start_line line_num] [-end_line line_num]
               [-start_column column_num] [-end_column column_num]])+
             -comment comment

check_xprop -waiver -remove waiver_id_tcl_list

check_xprop -waiver -list

check_xprop -waiver -export -file_name file_name

check_xprop -waiver -import -file_name file_name
            [-silent]
            [-force]

check_xprop -waiver -accept_update waiver_id_tcl_list
            [-silent]

check_xprop -waiver -list_matching_checks waiver_id_tcl_list
            [-silent]
X-Prop Reporting
----------------
check_xprop -report
            (  source
                 [-property property_name]
             | reason
                 [-include_failing_properties |-include_location])
            [-task task_name]
            [-txt file_name |-csv file_name]
            [-detailed] [-silent]

check_xprop -report
           [-type (precondition | reason | source)]
           [-file file_name [-force]
               [(-html [-launch_html_browser]
                |-xml |-csv)] ]
           [-category (clocks_and_resets | bbox_inputs | control_points
               | data_points | array_index | outputs | flops_with_reset_pin
               | flops_with_reset_value | counters | fsms | user_defined)+]
           [-instance {instance_list}]
           [-waived (none | only)]
           [-status (cex | proven | undetermined | unprocessed)]
           [-task task]
```

```
           [-detailed [-xsrc_info [N]]]

X-Prop Highlighting
-------------------
check_xprop -highlight_failure_path [-clear]
            [-task task_name]
            [-property property_name]

Clearing X-Prop Data Structures
-------------------------------
check_xprop -clear

Exporting X-Prop Properties
---------------------------
check_xprop -export
           (-task task_name
            |-property {property_name_tcl_list}
           )
            [-cex]
            [-proven]
            [-unprocessed]
            [-undetermined]
            [-file file_name]
            [-include_disable_iff]
            [-env]
            [-silent]
```

Returns

No return value

Detailed Description

Use this command to run X-propagation (X-Prop) analysis on one or more signals.

This command provides a set of flows that:

* Check the clocks and resets to ensure that they are free of Xs
* Check the control structures to see which areas of the code Xs can propagate to

```
 * Check data assignment structures to ensure that they are free of Xs
 * Check index expressions structures to ensure that they are free of Xs
 * Check the outputs to see if Xs can be propagated out of the design
 * Check the black-boxed inputs to see if Xs can propagate to black-boxed
   instances.
 * Use assume-guarantee reasoning to achieve convergence on difficult
   X-Prop proofs

In addition to assume-guarantee reasoning, the flow also provides property
grouping to take advantage of parallel processing provided by engine H and
distributed processing provided by ProofGrid. This capability is helpful when
the "prove" command alone is not converging on X-Prop assertions.

Important information about this command:
. This command does not guarantee convergence, but it helps in some cases. Be
  aware that it can require some time to complete.
. It is possible to open and interact with traces while the flow is still
  running.
. The X-Propagation Verification App supports all loop handling modes
  except "assume_stable".

The X-Propagation Verification App supports custom modeling for $isunknown,
that is, non-LRM compliant in the "realistic X" modeling context.

The X-Propagation Verification App checks for SVA properties with
$isunknown, provided the property follows one of these patterns:
  . assert property (@(posedge clk) ! $isunknown(<user-defined signal>));
  . assert property (@(posedge clk) not $isunknown(<user-defined signal>));
  . assert property (@(posedge clk) <pre-condition> |-> ! $isunknown(<user-defined
signal>));
  . assert property (@(posedge clk) <pre-condition> |-> not $isunknown(<user-defined
signal>));
  . assume property ($isunknown(<signal>));
  . assume property (<pre-condition> |-> $isunknown(<signal>));
  . assume property (@(posedge clk) disable iff (<disable_condition>) |-> !
$isunknown(<user_defined_signal>) );
  . assert property (@(posedge clk) disable iff (<disable_condition>) |-> !
$isunknown(<user_defined_signal>) );

  NOTE:
  . If there is a default clocking statement in the module, the following
    property is not supported because it inherits the clock from the
    default clocking statement:
```

```
    default clocking cb1 @(posedge clk); endclocking
    assume property ($isunknown(<signal>));
  . The tool does not insert X on the first cycle when you have an
    assumption of the following form:
    p1: assume property (@(posedge clk) $isunknown(a));
    In addition, this X-Prop pattern is not supported in Tcl expressions
    because you cannot have an expression using part of another expression.
  . The tool does not support properties with the following format:
    assert/assume property (@(posedge clk) disable iff (<disable_condition>)
    <pre-condition> |-> ! $isunknown(<user_defined_signal>));
    where <pre-condition> is a sequence (s1 ##1 s2).

Initializing X-Prop Analysis
----------------------------
-init
    Initialize the settings required for the X-Prop App.
    NOTE:
    . This command must be issued before the "analyze" and "elaborate" commands.
    . All other X-Prop commands must follow this command.
    . This command automatically disables the Expert System. This limitation will
      be removed in a future release.

(-init_control (true | false) [-sequential |-expression])
    Enable or disable analysis of the control structures:
    . true - Enable the analysis.
    . false - Disable the analysis.

    NOTE: You must run "check_xprop" with "-init_control" before loading the
    design.

    . Use "-sequential" to check for X-Prop only on sequential always blocks.
    . Use "-expression" to check for X-Prop by comparing all case items with
      case variables (still generating one property per case statement),
      instead of the default one property per variable in case variables or
      case items.

(-init_all_control (true | false) [-sequential |-expression])
    Enable or disable analysis of the control structures, including structures
    of the type "a?b:c", which are not done by -init_control:
    . true - Enable the analysis.
    . false - Disable the analysis.

    NOTE: You must run "check_xprop" with "-init_all_control" before loading
```

```
    the design.

    . Use "-sequential" to check for X-Prop only on sequential always blocks.
    . Use "-expression" to check for X-Prop by comparing all case items with
      case variables (still generating one property per case statement),
      instead of the default one property per variable in case variables or
      case items.

-init_data (true | false)
    Enable or disable analysis of the data assignment structures:
    . true - Enable the analysis.
    . false - Disable the analysis.

    NOTE: You must run "check_xprop" with "-init_data" before loading the
    design.

-init_all_data (true | false)
    Enable or disable analysis of the data assignment structures without
    filtering data assignments of constants where X cannot propagate in any
    way:
    . true - Enable the analysis.
    . false - Disable the analysis.

    NOTE: You must run "check_xprop" with "-init_all_data" before loading the
    design.

-init_index (true | false)
    Enable or disable analysis of the array index expressions structures:
    . true - Enable the analysis.
    . false - Disable the analysis.

    NOTE: You must run "check_xprop" with "-init_index" before loading the
    design.

-exclude_bind_hierarchies
    Use "-exclude_bind_hierarchies" to exclude creation of XProp targets on
    items generated from bind statements.

Creating X-Prop Assertions
--------------------------
-create
    Create the specified X-Prop assertions in the specified task.
```

```
    You can specify which signals to target with the following switches:
        -clocks_and_resets
        -control
        -data
        -index
        -outputs
        -bbox_inputs
        -flops_with_reset_pin
        -flops_with_reset_value
        -counters
        -fsms
        -to <signal_list>

    If you do not specify target signals, this command creates assertions for
    the outputs of the design.

-task task_name [-no_prefix]
    Create the specified X-Prop assertions in the specified task.

    . Replace "task_name" with a period (.) to specify the current task.
    . If you use "-create" without "-task", the tool generates default task
      names, according to the type of properties it creates. For example, for
      "-outputs", the tool creates the task XP_outputs. This naming convention
      makes it easy to identify the property type. However, if you specify a
      task, the type of the properties can be difficult to determine;
      therefore, in these cases, the tool adds a prefix to the name of each
      property to identify its type. If you do not want the tool to add this
      prefix, use the switch "-no_prefix".

-name property_name
    Use "-name" to assign a name to the property. If multiple properties are
    created by this command, the tool uses the name assigned with this command
    as the base name for the properties.

-from signal_name
    Allow the specified signal to be a source of Xs.

    Important information about this command:
    . Signals used with the "-from" switch must be inputs or stopats. They
      cannot be internal signals driven by some logic.
    . Xs in the design are allowed to propagate in accordance with how the
      design was elaborated (-disable_x_handling) and the current setting for
      "set_xprop_use_bus_contention" and "set_xprop_use_bus_floating".
```

```
    . Besides signals specified in "-from", sources of X are also captured from
      configurations defined by the following commands:
      . set_xprop_use_all_undriven
      . set_xprop_use_bbox_outputs
      . set_xprop_use_inputs
      . set_xprop_use_internal_undriven
      . set_xprop_use_init_abstraction
      . set_xprop_use_reset_abstraction
      . set_xprop_use_reset_state
      . set_xprop_use_stopats
      . set_xprop_use_bus_contention
      . set_xprop_use_bus_floating
    . Multiple drivers are always a source of X.
    . Reset abstracted flops are always a source of X.

-to signal_list
    Run X-Prop analysis on the specified signals. The tool will create a single
    property with all specified signals as the analysis points.

-clocks_and_resets
    Run X-Prop analysis on the clocks and resets in the design.

-control [-property_naming_style (default | long_filename | short_filename)
         [-no_flops_with_reset_pin [-include_sync_resets]]
    Run X-Prop analysis on the control structures in the design.

    . Use "-property_naming_style" with one of the following arguments to
      change the format of property names.
      . default: Property name has the following format:
        instance__signalName1_signalName2..signalNameN__line_x__col_y
      . long_filename: Property name has the following format:

instance__signalName1_signalName2..signalNameN__dir1/dir2/fileName__line_x__col_y
      . short_filename: Property name has the following format:
        instance__signalName1_signalName2..signalNameN__fileName__line_x__col_y

    . Use "-no_flops_with_reset_pin" to omit control checks for asynchronous
      flops with reset pins. And use "-include_sync_resets" to also omit them
      for synchronous flops.

-data [-property_naming_style (default | long_filename | short_filename)]
    Run X-Prop analysis on the data assignment structures in the design.
```

```
    Use "-property_naming_style" with one of the following arguments to change
    the format of property names.
    . default: Property name has the following format:
      instance__signalName1_signalName2..signalNameN__line_x__col_y
    . long_filename: Property name has the following format:
      instance__signalName1_signalName2..signalNameN__dir1/dir2/fileName__line_x__col_y
    . short_filename: Property name has the following format:
      instance__signalName1_signalName2..signalNameN__fileName__line_x__col_y

-index [-property_naming_style (default | long_filename | short_filename)]
    Run X-Prop analysis on the array index expression structures in the design.

    Use "-property_naming_style" with one of the following arguments to change
    the format of property names.
    . default: Property name has the following format:
      instance__signalName1_signalName2..signalNameN__line_x__col_y
    . long_filename: Property name has the following format:
      instance__signalName1_signalName2..signalNameN__dir1/dir2/fileName__line_x__col_y
    . short_filename: Property name has the following format:
      instance__signalName1_signalName2..signalNameN__fileName__line_x__col_y

-outputs
    Run X-Prop analysis on the outputs of the design.

    NOTE: This is the default behavior.

-bbox_inputs
    Run X-Prop analysis on the inputs of any black-boxed modules.

-flops_with_reset_pin [-include_sync_resets] [-no_temporary_buffers]
    [-property_naming_style (default | long_filename | short_filename)]
    Run X-Prop analysis on flops with a reset pin.

    . Use "-include_sync_resets" to include flops with a synchronous reset in
      the X-Prop analysis. By default, the tool excludes them.
    . Use "-no_temporary_buffers" to exclude properties on flops used as
      temporary buffers. Temporary buffers are uninitialized flops having
      blocking assignment. These are only used inside always block, that is, no
      fanout outside always block.

    . Use "-property_naming_style" with one of the following arguments to
      change the format of property names.
      . default: Property name has the following format:
```

```
        instance__signalName1_signalName2..signalNameN
      . long_filename: Property name has the following format:

instance__signalName1_signalName2..signalNameN__dir1/dir2/fileName__line_x__col_y
      . short_filename: Property name has the following format:
        instance__signalName1_signalName2..signalNameN__fileName__line_x__col_y

-flops_with_reset_value [-include_sync_resets]
    [-property_naming_style (default | long_filename | short_filename)]
    Run X-Prop analysis on flops with a reset value.

    . Use "-include_sync_resets" to include flops with a synchronous reset in
      the X-Prop analysis. By default, the tool excludes them.

    . Use "-property_naming_style" with one of the following arguments to
      change the format of property names.
      . default: Property name has the following format:
        instance__signalName1_signalName2..signalNameN__line_x__col_y
      . long_filename: Property name has the following format:

instance__signalName1_signalName2..signalNameN__dir1/dir2/fileName__line_x__col_y
      . short_filename: Property name has the following format:
        instance__signalName1_signalName2..signalNameN__fileName__line_x__col_y

-counters
    Run X-Prop analysis on counter flops of the design.



-fsms
    Run X-Prop analysis on finite state machine (FSM) flops of the design.

-precond expression
    Limit X-Prop checking to situations where the precondition (triggering)
    expression is true; that is, instead of checking hasXInOutput, check
    preconditionExpression & hasXInOutput.
    A related cover check is created on precondition expression. An unreachable
    status of this cover helps in identifying the over-constrained scenario.

    NOTE: Only Boolean or Sequence expressions are allowed in preconditions,
    that is, no property or implication operators can be used in the
    precondition expression.
    NOTE: The precondition expression is not allowed to have a sampling
```

```
    clock when using the "-sampling_clock" switch.

-no_internal_precond
    Do not use tool-generated preconditions for control, data and index targets.
    The tool ignores this option for other targets.

-no_precondition_cover
    Do not create the automatically extracted cover for the precondition
    checks.

    By default, if target has precondition then the tool automatically extracts
    covers for precondition expressions except for control, data, index and
    enables with flop sampling targets.

-create_precondition_cover
    Create cover of tool generated preconditions also. Control, data and index
    targets have tool generated preconditions. This option is useful with
    -control, -data, -index and -flop_sampling option only. This option is
    ignored for other targets.

    By default, if target has precondition then the tool automatically extracts
    covers for precondition expressions except for control, data, index and
    enables with flop sampling targets.

-sampling_clock clock
    Enable handling of sampled properties. The signal name must be a single-bit
    signal, that is, a simple net or a bit of an array.

    NOTE: Cannot be used with "-delay".

-from_file file_name
    Treat the signals in the specified file as allowable sources of X.

    NOTE: Construct the file with one signal per line.

-bit_blast
    Analyze the target signals and bit blast each signal before running
    X-Prop analysis.

    By default, "check_xprop" does not bit blast each signal. Using this switch
    might greatly increase the number of properties created and can lead to
    slower performance.
```

```
-no_bit_blast
    Analyze the target signals at the word level.

    This is the default behavior.

-instance {instance_list} [-regexp]
    Only analyze targets of the specified type from within the specified
    instance or instances.

    By default, the tool analyzes the entire design.

    . Use "-regexp" to specify the instance list to a case-insensitive
      wildcard strings or regular expressions.

-exclude_hierarchies {instance_list}
    Do not create targets in the specified instance hierarchies.

-filter filter_expression [-regexp] [-case_sensitive]
    Generate properties that match the specified expression based on the
    property names.

    By default, this switch supports case-insensitive strings and wildcard
    strings.

    . Use "-regexp" to specify the filter expression to a case-insensitive
      regular expression.
    . Use "-case_sensitive" to filter output using case-sensitive wildcard
      strings or regular expressions.

-filter_out filter_expression [-regexp] [-case_sensitive]
    Generate properties that do not match the specified expression based on the
    property names.

    By default, this switch supports case-insensitive strings and wildcard
    strings.

    . Use "-regexp" to specify that the filter expression to a case-insensitive
      regular expression.
    . Use "-case_sensitive" to filter output using case-sensitive wildcard
      strings or regular expressions.

-delay N
    When the "-delay" option is specified, the evaluation of generated
```

```
    properties starts N cycles after the reset.

    NOTE: Cannot be used with "-sampling_clock".

X-Prop Abstractions
-------------------
-abstract (multiplier | shift | adder | subtract | comparator)
    [-exclude_module {module_list} [-regexp] [-recursive]]
    [-include_module {module_list} [-regexp] [-recursive]]
    [-exclude_instance {instance_list} [-regexp]]
    [-include_instance {instance_list} [-regexp]]
    [-minimum_width integer]
    [-skip_const_operand]
    [-env |-task task_name]

-abstract (-instance instance_list)
    [-mode (seq | comb | mixed)]
    [-task task_name |-env]

    Create an abstraction that propagates X on the output net if any input net
    has X.

    . Use "multiplier" to abstract multipliers. If any input net of a
      multiplier operator has X, then the output of the multiplier becomes X.
    . Use "shift" to abstract left and right shift operators. If any input net
      of a shift operator has X, then the output of the shift becomes X.
    . Use "adder" to abstract adder(+) operator. If any input net of an adder operator
      has X, then the output of the adder becomes X.
    . Use "subtract" to abstract subtract(-) operator. If any input net of a subtract
      operator has X, then the output of the subtract becomes X.
    . Use "comparator" to abstract these operators (<, <=, >, >=). If any
      input net of these operator has X, then the output of operator becomes X.
    . Use "-exclude_module" to exclude matching modules from abstraction.
    . Use "-include_module" to include matching modules from abstraction.
      NOTE: Use "-recursive" with "-include_module/-exclude_module" to also
      include/exclude the instance hierarchies inside the module.
    . Use "-exclude_instance" to exclude matching instances from abstraction.
    . Use "-include_instance" to include matching instances from abstraction.
      NOTE:
      . The "-exclude_module", "-include_module", "-exclude_instance", and
        "-include_instance" switches accept wildcards (* and ?) and filter
        using case-sensitive wildcard strings by default.
      . Use "-regexp" with any of these switches to change the expression to
```

```
      a case-sensitive regular expression.
    . Use "-minimum_width" to abstract operators having a width greater than or
      equal to the specified width. Comparator operators have an output of size
      1; therefore, for "-comparator", the width of an input net is checked
      with minimum_width. For other abstraction options, the output width of an
      operator is compared with the minimum_width.
    . Use "-skip_const_operand" to skip abstraction for operators having a
      constant operand.
    . Use "-instance instance_list" to create X-Prop abstractions on the list
      of provided instance names.
    . X-Prop abstractions do not allow specifying driven signals as destination
      signals since the original driving logic is replaced by the logic of
      the abstraction. This blocks X propagation through the original
      design and might cause false positives. If you specificy driven signals
      as destination signals, the tool issues EDA020. To allow driven signals
      as destination signals, downgrade the error to a warning as follows:
      set_message -warning EDA020.
    . Use "-mode comb" to specify that the X propagates out of the
      X-propagation abstraction in the same phase as the X enters.
    . Use "-mode seq" to specify that the X propagates out of the
      X-propagation abstraction in the phases following the phase that the X
      enters the abstraction.
    . Use "-mode mixed" to specify that the X propagates out of the
      X-propagation abstraction in the same phase or phases following the phase
      that the X enters the abstraction.
    . Use "-env" to add an abstraction that affects all tasks.
    . Use "-task" to add an abstraction that affects only the specified task.
      NOTE:
      . An abstraction created with "-env" takes precedence over a "-task"
        abstraction for the same target (that is, "-env" overrides "-task").
      . The same target might be abstracted differently for each task.
      . If you do not use the "-env" or "-task" option, the command applies to
        the current task.

-abstract -remove abstraction_name_tcl_list
    [-env |-task task_name]
    Remove the specified abstractions in the specified task.
    NOTE:
    . If you do not use the "-env" or "-task" option, the command applies to the
      current task.

-abstract -clear
    [-env |-task task_name]
```

```
    Remove all abstractions in the specified task.
    NOTE:
    . If you do not use the "-env" or "-task" option, the command applies to the
      current task.

-abstract [multiplier | shift | adder | subtract | comparator] -list
    [-env |-task task_name]
    [-silent]
    Lists and returns abstractions with their abstraction IDs, instance, filename,
    line numbers, task and whether it is an environment abstraction.
    . Use "multiplier", "shift", "adder", "subtract", "comparator" to list only
      one type of abstraction. If no argument is given for "-abstract" then all
      type of abstraction are listed.
    . Use "-env" to list environment abstractions.
    . Use "-task" to list abstractions that are applied in the specified task.
    . Use "-silent" to return the abstraction IDs as Tcl return values without printing
      the information on the screen. This option is helpful in Tcl scripts.

Reporting Property Expressions
------------------------------
-get_expression
    -property prop_name
    [-precond |-target]
    Report the property expression in terms of HDL nets.

    . Use "-precond" to get only the precond expression in terms of HDL nets.
    . Use "-target" to get only the target expression in terms of HDL nets.

    NOTE:
    . "assert -show" for  control, data, and index targets shows internal nets
      like _automatic_*.
    . "cover -show" for related covers of control, data, and index targets also
      shows the internal net.
    . This command shows the actual HDL expression driving the internal nets.
    . This command is useful for control, data, and index targets only.
    . This command accepts X-Prop assertions and their related covers only.
    . By default, the command return is similar to "assert -show" but in terms
      of HDL nets.

Proving X-Prop Assertions
-------------------------
-prove
    [-task task_name
```

```
 |-property property_name_tcl_list [-regexp] [-strategy (basic | proof)]
]
[-force]
[-time_limit]
[-bg]
[-engine_mode engine_tcl_list]
[-skip_related_covers]
[-cex_limit number_of_failures]
[-from property_name]
[-with_proven]
[-with_helpers]
Start the proof of the specified security properties.

. By default, X-Prop proves the current task.
. Use "-task" to prove all the X-Prop properties in the specified task.
. Use "-property" to prove the specified X-Prop properties.
. The "-property" switch supports wildcards and regular expressions.
  Example:
  check_xprop -prove -property {expression} -regexp
. Use "-property" with "-strategy" to choose a strategy that trades in
  resources versus proof convergence on the specified properties.
  . Use "basic" to run the proof in a single-threaded proof environment.
  . Use "proof" to run the proof in a sophisticated, multi-threaded, and
    collaborating proof environment.
    NOTE: This option is available only with a set of properties that
    share the same X sources.
  . By default, the tool applies your "prove" command using the "basic"
    strategy.

. Use "-bg" to run the proof in the background and continue to accept and
  run new commands. To run several "prove" commands in parallel, use
  multiple "basic" strategy runs.
. Use "-force" to clear existing proof results for the selected property
  before beginning. See "clear -result".
. Use "-time_limit" to override the current "prove_time_limit" for this
  "prove" command. See "set_prove_time_limit".
. Use "-engine_mode" to override the current engine mode for this prove.
  NOTE: Only the "basic" proof strategy provides the option to specify
  an engine mode and considers the "engine_mode" setting only if
  "set_prove_orchestration" and "set_proofmaster" are both off. See
  "set_engine_mode".
. By default, X-Prop properties and their related covers are proven together.
. Use "-skip_related_covers" to prevent the  engines from automatically
```

```
  proving related covers. With this setting, the tool might still apply
  results to related covers if inferred from the main property's status or
  if the load cached previous results.
  NOTE: If you explicitly mention the related cover using the "-property"
  switch, the related cover will be processed.
. Use "-cex_limit" to specify that you want the prove flow to stop
  the proof job after it reaches a certain threshold of failures on X-Prop
  properties. If the prove flow stops on cex_limit, it returns
  "cex_threshold_reached".
. Use "-from" to use an existing trace as the initial value for another formal
  proof. Use this feature when a formal proof finds an interesting scenario
  counterexample and you want to perform another search from that point.
  By doing so, you might find scenarios that are deeper than what you could
  reach with other formal proofs.
  NOTE:
  . This feature is only supported by the "basic" proof strategy.
  . This can only be used when the "-from" property and all target
    properties are in the same task and have the same signature ("-from").
  . This is a bug-hunting feature. The tool is not able to prove assertions
    when you use "check_xprop -prove -from".
  . Found traces start from reset and include the trace to the specified
    property as a prefix.
  . If undetermined properties are triggered in the existing trace,
    the trace is used as the result for those properties. (The first time a
    property is triggered along a trace is the most significant event.)
. Use "-with_proven" to send proven X-Prop and FPV properties in the
  same task as the target properties to the engines as invariants. Proven
  properties are defined as proven assertions and unreachable covers.
  This feature is skipped if target properties have more than one
  signature. Only X-Prop properties with the same signature as the target
  properties are included as invariants. If no signature is detected,
  one signature is chosen from the proven X-Prop properties in the task.
  As long as the target properties share at most one signature, all proven
  FPV properties are included as invariants.
. Use "-with_helpers" to include helper assertions from the same task as
  the target properties as auxiliary proof targets. If a helper is
  already proven, it is also used as an invariant. This feature is skipped
  if more than one X-Prop signature is detected among the target
  properties.

NOTE:
. The strategies work as follows:
  . Basic strategy is the default strategy and runs a proof with
```

```
      X-Prop-oriented proof settings on the target property. It can run with 1
      job, but the recommendation is at least 8 jobs. Most easy results will
      be found by that strategy, often in less than one hour. Multiple basic
      strategies can run in parallel. Run this strategy first.
    . Proof strategy runs a parallel basic proof, a proof with automatically
      applied cutpoints on flops and internal signals of the design using
      cutpoint refinement loop handling and failure extension to help the
      other strategies find deep counterexamples by extending counterexamples
      it already identified. It can find all easy and additional hard results
      with significantly higher resources and run time. Allow at least 36
      jobs and 1h of run time for this strategy. Unlike "basic"
      strategy, you cannot run multiple proof strategies in parallel.
      Run this strategy on remaining targets after the "basic" strategy no
      longer produces results.
  . Return values are the same as the "prove" command. But internally
    auto-generated helpers are not considered in the return value of the
    proof. For details on all possible return values, use "help prove -gui".
  . Max jobs and per engine max jobs limitations are applied per property
    not per "prove" command.
    NOTE: Only the "basic" proof strategy supports this configuration.
  . Related covers are automatically proven in the same proof thread when
    proving the associated X-Prop properties. To disable the automatic proof
    of related covers, use "-skip_related_covers".
  . XPROP can infer the status of a precondition from the status of its main
    property and vice-versa.
    . A counterexample on the main property means that the precondition is
      covered.
    . If a precondition is unreachable, the main property is vacuously proven.
    . If a new bound was found for a precondition, the main property's bound
      may be increased to match it.
    In case of inference, the indication "(Inferred)" appears immediately after
    the engine name, for example:
    "INFO (IXP018): (0.basic) SEC basic strategy (B)(Inferred): A min_length
    bound of 252 was found for the property "prop" in 4.14 s."

Debugging X-Prop Assertions
---------------------------
-waiver -add -property property_name_list
    -comment comment
    [-regexp]

    Add waivers on the specified properties.
    The tool does not prove waived properties or related covers of waived properties.
```

```
    . Use "-property" to add a waiver on the list of properties.
      Use "-regexp" to consider the property name as a regular expression.
    . Use "-comment" to add a comment to the waiver.

-waiver -add
    [-instance instance_name | -top]
    [-category (clocks_and_resets | bbox_inputs | control_points | data_points
          | array_index | outputs | flops_with_reset_pin | counters | fsms |
user_defined)]
    [-module module_name]
    [-source_file file_name
      [-start_line line_num] [-end_line line_num]]
      [-start_column column_num] [-end_column column_num]]
   )+
    -comment comment

    Add waivers as specified and return an ID list of the added waivers.
    The tool does not prove waived properties or related covers of waived properties.

    . Use "-instance" to add waivers on properties of the specified
      instance.
    . The "-instance" switch accepts wildcards (* and ?). Wildcards are useful
      for ignoring the instance hierarchies.
    . Use "-top" to add waivers on properties of the top instance.
    . Use "-category" to add waivers on properties of specified category.
    . Use "-module" to add waivers on messages from the specified module.
    . Use "-source_file" to add waivers on checks from the specified file.
    . Use "-start_line" to add waivers on checks after the specified
      line.
    . Use "-end_line" to add waivers on checks before the specified line.
    . Use "-start_column" to add waivers on checks after the specified
      column.
    . Use "-end_column" to add waivers on checks before the specified column.
    . Use "-comment" to add a comment to the waiver.

-waiver -remove waiver_id_tcl_list
    Remove the specified waivers and return an ID list of the matching
    properties.

-waiver -list
    List the IDs of the waivers that are applied.
```

```
-waiver -export -file_name file_name
    Export waivers to the specified file.

    This command generates a Tcl file with the specified file name.

-waiver -import -file_name file_name [-silent] [-force]
    Import waivers from the specified file.

    . Use "-force" to import waivers even if there are no matching properties.
    . Use "-silent" to suppress output and generate only Tcl return values.

    NOTE: When you import waivers after the design has changed, the tool
    tries to update waivers automatically. Waivers that are automatically
    imported and updated are added to the tool and reported in the console.
    However, if the tool could not update the waivers, they are reported
    as unmatched.

-waiver -accept_update waiver_id_tcl_list [-silent]
    Accept the update done by the tool to the specified waivers and return an
    ID list of the matching properties or messages.

    Use "-silent" to suppress output and generate only Tcl return values.

-waiver -list_matching_checks waiver_id_tcl_list [-silent]
    List the names of properties that match the specified waivers.

    Use "-silent" to suppress output and generate only Tcl return values.

X-Prop Reporting
----------------
-report source
    [-task task_name |-property prop_name]
    [-detailed] [-txt file_name |-csv file_name] [-silent]
    Report the X sources configuration the tool will use in the X-Prop analysis
    for the current task.

    This report prints the status (on or off) of each kind of X source that can
    be enabled or disabled with "set_xprop_use_*" Tcl commands and, for each
    enabled kind, the names of signals that will be sources of X during X-Prop
    analysis. It also prints the sources of X that cannot be disabled
    (multiple drivers, reset-abstracted flops, X-Prop assumptions, and X-Prop
    properties with a specific "-from" signal).
    . Use "-task" to list the X sources for a specified task.
```

```
      If you do not specify a task, this command applies to the current task.
    . Use "-property" to list just the sources in the analysis of the specified
      property.
    . Use "-detailed" to print all possible sources of X, including those for
      disabled kinds of X source. With this switch, the report can contain
      sources that will not be used in the X-Prop analysis.
    . Use "-txt" to save the report output in a text format.
    . Use "-csv" to save the report output in CSV format.
    . Use "-silent" to return the report as Tcl return values without printing
      the report on the screen. This option is helpful in Tcl scripts.

    The output format is as follows:
    <x source type:>
      <x signal>
    Also see the "Example" section.

-report reason
    [-task task_name]
    [-include_failing_properties |-include_location |-detailed]
    [-txt file_name |-csv file_name] [-silent]
    Report the reasons for X-Prop failures.

    This report prints the x signal name, its type (as specified with the
    commands "set_xprop_use_*", from multiple drivers, or from reset abstracted
    flops), and the number of properties that failed due to this signal.
    . Use "-task" to list the X sources for a specified task.
      If you do not specify a task, this command applies to the current task.
    . Use "-include_failing_properties" to list the failing properties.
    . Use "-include_location" to list the failing properties and the source
      code file and line number of the X assignment for each property that
      failed due to "X Assignment".
    . Use "-detailed" to create a report that contains all of the information
      specified by the switches above.
    . Use "-txt" to save the report output in a text format.
    . Use "-csv" to save the report output in CSV format.
    . Use "-silent" to return the report as Tcl return values without printing
      the report on the screen. This option is helpful in Tcl scripts.

    The output format is as follows:
    <x signal> (<x reason>, #Failing Properties: <number of properties failing due to
this signal>)
    [<list of failing properties> (# cycles)]
    Also see the "Example" section.
```

```
-report
      [-type (precondition | reason | source)]
      [-file file_name [-force]
         [(-html [-launch_html_browser] | -xml | -csv)] ]
      [-category (clocks_and_resets | bbox_inputs | control_points | data_points
            | array_index | outputs | flops_with_reset_pin | flops_with_reset_value
            | counters | fsms | user_defined)+]
      [-instance instance_list]
      [-waived (none | only)]
      [-status (cex | proven | undetermined | unprocessed)]
      [-task task]
      [-detailed [-xsrc_info [N]]]

     Generate report of X-Prop properties.
     NOTE: By default, this command includes a property status summary and
     prints results to the console.

    . Use "-type" to specify type of the report.
      . Use "precondition" to report undetermined X-Prop assertions with
        trace length lesser than the trace length of their respective covers.
      . Use "reason" to generate a report of the reasons for X-Prop failures.
      . Use "source" to report the X sources configuration the tool uses
        in the X-Prop analysis for the current task.
      NOTE: If you use the "-type" switch, the property status summary is
      excluded.
    . Use "-file" to specify a file name. And use
      "-force" to overwrite an existing file.
      . Use "-html" to report results as HTML.
        . Use "-launch_html_browser" to launch an HTML browser with results.
      . Use "-xml" to report results as XML.
      . Use "-csv" to report results as CSV.
      . By default, the results are reported in .txt format.
      NOTE:
      . If you do not specify a file, the report is printed to the console.
      . If you use the "-csv", "-html", or "-xml" switch, the property status
        summary is excluded.
    . Use "-category" to generate a report for the specified X-Prop target types.
    . Use "-instance" to generate a report for the specified instances.
      NOTE: X-sources for bus contention checks, bus floating checks, and assumptions
      are not filtered from the report with the "instance" switch.
    . Use "-waived only" to generate a report for waived check only, and use
      "-waived none" to exclude waived checks from the report. By default,
```

```
      waived checks are reported.
    . Use "-status" to generate a report for the specified status. By
      default, the report is generated for all status.
    . Use "-task" to generate a report for the specified task. By default,
      the report is generated only for the current task.
    . Use "-detailed" to get a detailed report. By default, the tool generates
      only a summary report.
      . Use "-xsrc_info" to get a detailed report along with X-Sources
        information, that is, "X-Source Name" and "X-Source Type".
      . Use "-xsrc_info N" to view multiple sources information,
        where N specifies the maximum number of sources to be displayed.
        To view all sources use N = 0.
        NOTE: X-Sources of a property are dependent on traces found by engines,
        that may change between different runs.

X-Prop Highlighting
-------------------
-highlight_failure_path [-clear]
    [-task task_name |-property prop_name]
    Compute failure path highlights for display in the X-Propagation Analysis
    Browser.

    Important things to know:
    . Failure path highlights represent the recursive Why of a violation
      trace. They indicate the X path in the source code with red highlights.
      NOTE: These highlights depend on the generated trace.
    . The failure path highlight is not computed automatically. It occurs when
      you do one of the following:
      . Request failure path highlights with this command.
      . Click the button "Compute X Sources" in the "X-Prop Failure Analysis"
        dialog.
      . Choose the menu option Application > Highlight > Compute All Failure
        Path Highlights.

    . Use "-task" to compute the failure path highlight of all properties of
      the specified task.
      If you do not specify a task, this command applies to the current task.
    . Use "-property" to compute the failure path highlights for the specified
      property.
    . Use "-clear" to clear all computed failure path highlights, highlights of
      specific tasks (-task), or highlights of specific properties (-property).

Clearing X-Prop Data Structures
```

```
-------------------------------
-clear
    Clear the check_xprop data structures.

Exporting X-Prop Properties
---------------------------
-export
    [ -task task_name
     |-property property_name_tcl_list]
    [-cex]
    [-proven]
    [-unprocessed]
    [-undetermined]
    [-file file_name]
    [-include_disable_iff]
    [-env]
    [-silent]
    Export X-Prop properties to a partially encrypted file, to be used for
    simulation.

    . Use "-task" to export X-Prop properties for the specified task. The
      default is all.
    . Use "-property" to specify the list of properties you want to export. The
      default is all.
    . Use "-cex", "-proven", "-unprocessed", and "-undetermined" to export
      properties with the specified statuses. The default is all.
      NOTE: These status filters are also applicable to the "-task" and
      "-property" switches.
    . Use "-file" to export the properties to the specified file name. If you
      do not specify a file name, a default file is generated, which includes
      top module name and status filters.
    . Use "-include_disable_iff" to add a condition in property's target
      expression to disable checks during reset.
    . Use "-env" to run export analysis based on the entire design region
      ignoring the command's current task context, such as assumptions and
      stopats.
    . Use "-silent" to return the exported properties as Tcl return values
      without printing the information on the screen.
    NOTE:
    . An exported property name can differ from the corresponding property name
      to handle special characters/escape sequence present in the name.
    . X-Prop covers are not exported.
```

Example

```
check_xprop -init_control true -sequential
elaborate
check_xprop -create -clocks_and_resets -control
check_xprop -create -to data_out -precond data_out_valid -from data_in
check_xprop -prove
check_xprop -clear
check_xprop -export

# X Sources Report Example:
[<embedded>] check_xprop -report source -task XP_outputs
X Sources Report
----------------
Target: TASK: XP_outputs
Uninitialized Registers (on): 0
BBoxed Outputs (on): 0
Primary Inputs (off)
Stopats (off)
Internal Undriven (on): 1
  cout
X-Assignments (on): 0
--------
Multiple Driver: 0
Flop Reset Abstractions: 0
Assumes: 0
XProp Properties with specific -from: 0

# Detailed X Sources Report Example:
[<embedded>] check_xprop -report source -task XP_outputs
X Sources Report
----------------
Target: TASK: XP_outputs
Uninitialized Registers (on): 0
BBoxed Outputs (on): 0
Primary Inputs (off): 1
  ain
Stopats (off): 0
Internal Undriven (on): 1
  cout
X-Assignments (on): 0
--------
Multiple Driver: 0
```

```
Flop Reset Abstractions: 0
Assumes: 0
XProp Properties with specific -from: 0

# X Reasons Report Example:
[<embedded>] check_xprop -report reason
X Reasons Report
----------------
Target: TASK: <embedded>

brdg.int2ig_data[0] (BBoxed Outputs, #Failing Properties: 4)
brdg.int2ig_data[10] (BBoxed Outputs, #Failing Properties: 4)
brdg.int2ig_data[11] (BBoxed Outputs, #Failing Properties: 4)
.
.
.
eg.eg_ad_dataout[5] (Uninitialized Registers, #Failing Properties: 1)
eg.eg_ad_dataout[6] (Uninitialized Registers, #Failing Properties: 1)
size0[0] (Primary Inputs, #Failing Properties: 1)
size0[1] (Primary Inputs, #Failing Properties: 1)
valid0 (Primary Inputs, #Failing Properties: 4)

# Detailed X Reasons Report Example:
[<embedded>] check_xprop -report reason -detailed
X Reasons Report
----------------
Target: TASK: <embedded>

brdg.int2ig_data[0] (BBoxed Outputs, #Failing Properties: 4)
   XP_outputs::rdata3[0] (2 cycles)
   XP_outputs::rdata2[0] (2 cycles)
   XP_outputs::rdata1[0] (2 cycles)
   XP_outputs::rdata0[0] (1 cycles)
.
.
.
valid0 (Primary Inputs, #Failing Properties: 4)
   XP_outputs::ready3 (2 cycles)
   XP_outputs::eg_valid (2 cycles)
   XP_outputs::ready0 (6 cycles)
   XP_outputs::ready2 (2 cycles)
valid1 (Primary Inputs, #Failing Properties: 1)
   XP_outputs::ready1 (3 cycles)
```

.
.
.

See Also

elaborate,
set_xprop_ignore_non_resetting_flops_as_source,
set_xprop_use_all_undriven,
set_xprop_use_bbox_outputs,
set_xprop_use_inputs,
set_xprop_use_internal_undriven,
set_xprop_use_init_abstraction,
set_xprop_use_reset_abstraction,
set_xprop_use_reset_state,
set_xprop_use_stopats,
set_xprop_use_bus_contention,
set_xprop_use_bus_floating

clear

Clears proof/GUI information.

Syntax

```
clear  [-task task_name] [-result]
     | [-engineL_cache]
     | [-cache_proof_simplification]
     | [-all]
     | [-log]
```

Returns

No return values

Detailed Description

Use this command to clear the proof and/or GUI information. If you do not use any switches, the tool preserves the analyzed and elaborated design and some caches, for example, the proof simplification cache and the engine L cache. Be aware that if you previously specified clocks or reset, "clear" without switches also clears them, so you will need to specify them again before moving forward with your session.

Use switches to customize the clear operation, for example, to clear the analysis region and any prior proof information for a specified task or to clear a cache.

NOTE:
. This command does not clear user-defined Tcl variables or procedures.
. This command is not allowed while a background proof is in progress.
. This command stops periodic updates to existing databases.

-task task_name
    Clear the analysis region and any prior proof information in the specified task.

```
    NOTE:
    . This command clears task stopats and counter abstractions.
    . Replace "task_name" with a period (.) to specify the current task.
    . In the SEC App, this command unmaps all SEC task mappings.

-result
    Clear the proof result without clearing the proof
    setup such as the analysis region.

-engineL_cache
    Clear the engine L cache. See "set_engineL_cache".

-cache_proof_simplification
    Clear the proof simplification cache (see
    "set_cache_proof_simplification").

-all
    Clear GUI windows launched from the current session, such as the Visualize
    window and the Source Browser.

    Important things to know:
    . This command applies to the current session; that is, if you are running
      multiple sessions, the others are not affected.
    . This command has no effect on the database.
    . This command does not clear Proof Accelerator data structures. Use the
      Proof Accelerator command's "-clear" switch instead.
    . This command does not clear user-defined Tcl variables, procedures, or
      custom buttons.
    . This command does not clear run-time configurations.
    . The "clear -all" command runs the following commands:
      . abvip -clear
      . check_cdc -clear
      . check_spv -clear
      . check_superlint -clear
      . hunt -clear
      If Jasper has an elaborated design, "clear -all" also runs the
      following commands:
      . check_conn -clear
      . check_csr -clear
      . sanity_check -waiver -clear
    . The "clear -all" command releases allocated memory to the OS.

-log
```

Clear the log file for the current session.

Example

```
clear
clear -result -task my_task
```

See Also

set_cache_proof_simplification

clock

Specifies clock-related configuration.

Syntax

```
clock primary_clock
        [-both_edges]

clock clock_1 clock_tcl_list [factor [phase]]
        [-both_edges]

clock -rate
    ( input_tcl_list clock_signal [-both_edges]
     |-default clock_signal [-both_edges]
     |-clear )

clock -rate -task task_name
    ( task_stopat_tcl_list clock_signal [-both_edges]
     |-clear )

clock clock_name -factor N [-phase N]
clock clock_name -both_edges -pattern pattern
clock clock_name -from N -to M -both_edges
clock -clear
clock -remove clock_tcl_list
clock -analyze [-include_latches] [-typed_list] [-silent]
clock -analyze [-trace] [-shm shm_directory_name]
        [ [-hier_path path_difference]
         |[-hier_map -from source_hier -to destination_hier]]
clock -list [signal | configuration] [-typed_list] [-silent]
clock -list configuration -task task_name [-silent]

clock -infer
clock -none

clock -domain
    ( -signal_list clocking_signal_list [-target_domain domain_ID]
     |-list [-target_domain clocking_signal]
     |-clear)
```

Returns

"clock -analyze" returns a Tcl list containing inferred clocking signals.
If procedural assignments are sensitive to both edges of a clocking signal,
the return follows this example:
{<clocking_signal> ~<clocking_signal>}.

Detailed Description

Use this command to specify a global clock-related configuration, assign
signals to a new domain, get information about clocks and clock domains, or
analyze the clock tree. Clock tree analysis can provide the information you
need to specify the clock environment properly.

Before you can run a proof or Visualize a target, you must specify at least one
of the following commands:
. Use "clock <clk>" to declare a signal as a clock.
. Use "clock -none" for a free-running clock environment.
. Use "clock -infer" to specify that you want the tool to automatically infer
  primary clocks from every flop it encounters.

The "clock" analysis only considers global stopats and assumptions (specified
with "assume -env"). Task-specific information (including assumption and stopat
information) does not affect inferred clocks.

Clock arguments must be a signal or negation of a signal. An
argument cannot be an expression. For Verilog designs, use
"~<primary-clock>" for negedge clocks. For VHDL designs, use
"clock {not <primary-clock>}" unless you elaborate with
"-mode verilog".

Implicit Clocking Behavior with "-both_edges"

If you write a Tcl property with implicit clocking, that is,
@(posedge clk) and use "-both_edges" with the "clock" command, the implicit
@(posedge clk) becomes @(clk).

Example:

```
clock clk
assume { a |=> b }
```

```
NOTE: To use the standard (built-in) Tcl "clock" command, specify "tcl_clock".
For additional information about using Tcl with Jasper Apps, refer to the
application note "Scripting with Tcl in Jasper Apps" (Help - Application
Guides - Scripting with Tcl).

primary_clock
    The specified clock is a primary clock for the design.

    NOTE: For Verilog designs, use "~<primary_clock>" for
    negedge clocks. For VHDL designs, use "clock {not <primary_clock>}"
    unless you elaborate with "-mode verilog".

-both_edges
    The analysis region has one of the following conditions:
    . Some flops trigger on the posedge of the clock and others trigger
      on the negedge of the clock.
    . The high phase of clk_in enables a latch.

    NOTE:
    . Only use "-both_edges" if the tool prompts you to do so when running
      the "prove" or "visualize" command.
    . The tool will automatically declare the clock as both edges when there is
      no proof degradation.

clock_1 clock_tcl_list [factor [phase]]
        [-both_edges]
    Specifies the relationship of the specified clock to one or more specified
    clocks.

    . clock_1        : The base clock for the current analysis.
    . clock_tcl_list : One or more derived clocks specified for the current
                       analysis.
    . factor         : Integer ratio of the base clock frequency to the
                       frequency of one or more derived clocks (default 1).
    . phase          : Use an integer to indicate the relative phase between
                       clocks. The derived clock(s) will have their first
                       positive edge at the Nth positive edge of the base clock
                       (default 1).
                       When used with clock_1 clock_tcl_list, "-both_edges"
                       applies to the listed clock(s).

    NOTE: This command accepts lists of signals and Tcl lists.
```

```
-rate
    ( input_tcl_list clock_signal [-both_edges]
     |-default clock_signal [-both_edges]
     |-clear )
    The specified inputs represented by input_tcl_list, which are boundary input
    nets to the current analysis region, change value at the rate of the
    specified clock_signal. With the "-rate" switch, the clock_signal can be any
    signal in the design, even a driven signal that was not previously declared
    as clock.

    If you use "-default" and specify a clock_signal (instead of specifying
    inputs), the tool uses the specified clock signal to rate all undriven
    signals from the design. The initial value of "-default" is the posedge of
    the tool's fastest clock.

    Use "-both_edges" to specify that you want inputs rated on both edges. This
    feature only works if the specified clock_signal is a declared clock.

    Use "-clear" to remove all existing clock rate configurations.

    Important things to know about this command:
    . This command supports wildcards (* and ?) in the input_tcl_list variable.
    . Specified signals in the input_tcl_list variable must be free inputs or
      environmental stopats.
    . In the case of buffers, the tool uses their equivalent free inputs or
      environmental stopats.
    . Any signal that does not exist triggers a warning.
    . This command accepts lists of signals and Tcl lists.
    . By default, "clock -rate" applies to the rising edge of the specified
      clocks. To apply the command to the falling edge, invert the clock (~clk).

    CAUTION:
    . If you configure the rate of a stopat with a slower sampler than
      its original logic, the tool might be unable to find some traces that it
      otherwise can find with the original driver.
    . Rating an input that is in the COI of the specified clock signal
      can create a combinational loop passing through clock logic, which might
      lead to errors in the proof flow.

    Return: The signals that were added to the "-rate" list.

-rate -task task_name
    ( task_stopat_tcl_list clock_signal [-both_edges]
```

```
     |-clear )
    The specified signals represented by task_stopat_tcl_list, which are stopats
    from the specified task, change value at the rate of the specified
    clock_signal.

    Use "-both_edges" to specify that you want stopats rated on both edges. This
    feature only works if the specified clock_signal is a declared clock.

    Use "-clear" to remove all task clock rate configurations.

    Important things to know about this command:
    . This command supports wildcards (* and ?) in the task_stopat_tcl_list
      variable.
    . Any signal that does not exist triggers a warning.
    . This command accepts lists of signals and Tcl lists.
    . By default, "clock -rate -task" applies to the rising edge of the
      specified clocks. To apply the command to the falling edge, invert the
      clock (~clk).

    CAUTION: If you configure the rate of a stopat with a slower sampler than
    its original logic, the tool might be unable to find some traces that it
    otherwise can find with the original driver.

    Return: The signals that were added to the "-rate -task" list.

clock_name -factor N [-phase N]
    The specified "clock_name" has the specified factor.

    Use "-phase" with an integer to specify the relative phase between clocks.

    NOTE:
    . If you have declared a primary clock, the specified clock has its first
      positive edge at the Nth positive edge of the primary clock.
    . If you have not declared the primary clock, the specified clock has its
      first positive edge between cycles N and N+1.

clock_name -both_edges -pattern pattern
    The specified "clock_name" behaves like the wave described by "pattern".

    The argument "pattern" is a sequence of 0's and 1's, where
    each "0" corresponds to a cycle where "clock_name" will remain low and
    each "1" corresponds to a cycle where "clock_name" will be high.
```

```
    Currently, every clock declared with "clock -pattern" must be allowed
    to sample registers on both of its edges. Therefore, the switch
    "-both_edges" is mandatory.

    Important things to know about this command:
    . Clock inversion is not supported when using the "-pattern" switch.
      For example, the following command triggers an error:
      clock ~clk -both_edges -pattern 01

    . A patterned clock cannot be the base clock for another clock declaration.
      For example, the following command set is not supported:
      clock clk -both_edges -pattern 01
      clock clk clk1

    . Pattern clocks do not rate inputs by default. When you use "clock
      -pattern" and you want an input to change only on the rising edge of the
      clock, use "clock -rate <input> <pattern_clk_name>". Without "clock
      -rate", inputs are allowed to change at every cycle, that is, at every
      digit of the pattern.

clock_name -from N -to M -both_edges
    The specified "clock_name" works with multiple clock setups.
    The engines are able to select any clock factor between N and M with any
    valid phase. However, it is not possible to change the clock configuration
    between cycles.

    A range-based clock declaration is equivalent to running multiple "prove"
    commands using each valid clock declaration within the specified N-to-M
    range.

    Currently, every clock declared with "clock -from -to" must be allowed
    to sample registers on both of its edges. Therefore, the switch
    "-both_edges" is mandatory.

    Important things to know about this command:
    . Range-based clocks do not rate inputs by default. When you use "clock
      -from -to" and you want an input to change only on the rising edge of the
      clock, use "clock -rate <input> <clock_name>".
    . "clock -rate -default" is not supported by this feature.
    . The reset analysis in the proof will use a clock with a factor N that is
      the fastest clock possible.

-clear
```

```
    Remove all existing clock configurations from the global environment.

    Use this command to clear your clock declarations before attempting to
    specify a new factor and/or phase for a previously declared clock.

-remove clock_tcl_list
    Remove existing clock configurations for the specified signals from the
    global environment and return a Tcl list of successfully removed clocks.

    NOTE:
    . The tool also removes the following:
      . Clocks derived from signals in the "clock_tcl_list"
      . Input ratings driven by removed clocks
    . When removing both edges clocks:
      . Use the same polarity used in the declaration of the clock.
      . Get the original polarity with "clock -list configuration".
      . Example:
        clock ~clk -both_edges
        clock -remove ~clk
        clock clk -both_edges
        clock -remove clk

-analyze [-include_latches] [-typed_list] [-silent]
    Analyze the design's clock structure and return a report that includes
    inferred clocks for the global environment.
    . Use "-include_latches" to include clocking signals from latches in the
      analysis report and the Tcl return.
    . Use "-typed_list" to return a list of all information contained in the
      clock report summarized as Tcl return values.
    . Use "-silent" to return the list as Tcl return values without printing
      the list on the screen.

    NOTE:
    . The tool automatically extracts inferred clocks when you use the
      command "clock -infer". They are based on setup (that is, environment
      assumptions and stopats and "clock" command options).
    . The Tcl return includes the following:
      . All declared clocks that are in the combinational COI of the clocking
        signals.
      . All clocking signals, including constant clocks, gated clocks, and
        those driven by a complex logic clock.

    The clock report includes the following information:
```

```
    . Name
    . Status - defined, undefined, or propagated
    . Type - simple clock, AND-gated tree, and so forth
    . Number of PSL/SVA flops connected to the clock
    . Number of design flops connected to the clock
    . Names and types of signals driving the clock

-analyze [-trace] [-shm shm_directory_name]
         [ [-hier_path path_difference]
          |[-hier_map -from source_hier -to destination_hier]]
    Analyze the trace's signals and return potential clock signals.
    . Use "-hier_path" when the trace file contains a hierarchy that is
      different from the one in the design loaded in Jasper Apps.
    . Use "-hier_map" for trace files generated during simulation if you have a
      wrapper module.
      . Use "-from" to specify the source (simulation environment) hierarchy.
      . Use "-to" to specify the destination (formal environment) hierarchy.

-list [signal | configuration] [-typed_list] [-silent]
    List the user-defined clock configurations for the global environment.
    . Use "signal" to list all clock signal names, suppressing the sampling
      edge information.
    . Use "configuration" to list the full clock configuration, including
      clock phase, periods, clock patterns, and rated inputs. You can use each
      element in the list as an argument to the "clock" command to recreate the
      clock configuration.
    . Use "-typed_list" to add a type element before each of the requested
      lists.
      . The list format is as follows:

        $clockConfig(clocks) {<list of clock signal names>}
        $clockConfig(configuration) {<lists the complete clock configuration>}
        $clockConfig(none) {true/false <is clock none>}
        $clockConfig(clock_declared) {true/false <is clock declared>}

        Each of these fields is empty by default ( {} ) except for
        $clockConfig(none) and $clockConfig(clock_declared), which is false or
        true, depending on the clock configuration.
      . Refer to the "Example" section.
    . Use "-silent" to return the list as Tcl return values without printing the
      list on the screen. This option is helpful in Tcl scripts.

-list configuration -task task_name [-silent]
```

```
    List the clock configurations for a given task.

    You can use each element in the list as an argument to the "clock" command
    to recreate the clock configuration.

    Use "-silent" to return the list as Tcl return values without printing the
    list on the screen. This option is helpful in Tcl scripts.

-infer
    Execute clock inference explicitly and declare and return all inferred
    clocks for the global environment.

    The tool automatically extracts all inferred clocks by analyzing primary
    inputs that are connected to flip-flop clock pins.

-none
    Allow the tool to run with free clocks.

    NOTE: To run a proof (or Visualize) you must either have declared your
    clocks or used "clock -none".

-domain
    ( -signal_list clocking_signal_list [-target_domain domain_ID]
     |-list [-target_domain clocking_signal]
     |-clear)
    Assign specified signals to a new clock domain, list information about the
    current clock domain, or clear previous reassignments.

    . Use "-signal_list" to specify the list of clocking signals that will be
      removed from their current clock domain and added to another domain. If
      you do not specify a domain with "-target_domain", the tool creates a new
      clock domain.

      NOTE: Use this command if you want to redefine clock domains, that is,
      change the clock domain definitions that were built based on the clock
      tree analysis.

    . Use "-list" to list the current clock domain's information. And use the
      switch "-target_domain" to return the clock domain ID of the specified
      clocking signal.

    . Use "-clear" to revert your previous reassignments.
```

Example

```
clock clk {in1 in2}
clock clk {in1 in2} 2 1

clock -list -typed_list
User-defined clocks:
------------
0: clk clk 1 1
------------
Rated :
------------

clocks clk configuration {{clk clk 1 1}} none false clock_declared true

// Supported formats for "clock -rate":
clock -rate sig1 sig2 sig3 clk
clock -rate {sig1 sig2 sig3} clk
clock -rate {sig1} {sig2} {sig3} clk
clock -rate {sig1 sig2 sig3} clk -both_edges

// In the following command, the ratio of the periods of clk2 to clk1 is 4:1,
// and both the primary and derived clocks tick together right after reset.
clock clk1 clk2 4 1

// In the following command, "-both_edges" applies to clkB.
clock clkA clkB 2 1 -both_edges

// In the following command, the ratio of the periods of clk2 to clk3 is 2:3.
clock clk2 -factor 2
clock clk3 -factor 3

// The following clock declaration will force clk to behave like a square wave
// with a 25duty cycle.
clock clk -pattern 0100 -both_edges

// The following clock declaration specifies that the clock setup can have any
// possible clock configuration with factors 1, 2, or 3.
clock clk -from 1 -to 3 -both_edges
```

See Also

abstract,
assume,
prove,
reset,
sanity_check,
set_prove_clock_optimization

command_interface

Manages connections to Jasper command interface clients.

Syntax

```
command_interface -connect -local server_name
   [-protocol_version protocol_version] [-client_id client_id]

command_interface -connect -tcp -port port [-hostname hostname]
   [-protocol_version protocol_version] [-client_id client_id]

command_interface -disconnect
```

Returns

No return values

Detailed Description

Use this command to connect this Jasper analysis session to or
disconnect from Jasper command interface clients. Connected clients
can request the analysis session to run Jasper commands.

The only type of command interface client currently supported by Jasper is
Jasper's Python API. This API can start a socket server (local socket or
TCP-socket) to which one or more Jasper analysis sessions can connect. This API
is normally also responsible for launching Jasper and asking the first analysis
session to connect to its server. This means that users usually do not have to
use the "command_interface" command themselves.

-connect -local server_name [-protocol_version protocol_version]
                [-client_id client_id]
    Use this command to connect to a local socket server, typically one started
    by Jasper's Python API.

server_name
    Specifies the name of the server, usually a file

```
    under "/tmp".

-protocol_version protocol_version
    Specifies the version of the protocol used between
    Jasper and the command interface client. Use the
    value "latest" for using the most recent version
    of the protocol.

-client_id client_id
    Specifies an identifier which Jasper uses for to
    acknowledge the connection. Failing to provide a
    correct identifier when connecting to Jasper's
    Python API will make the API to drop the
    connection.

-connect -tcp -port port [-hostname hostname]
              [-protocol_version protocol_version] [-client_id client_id]
    Use this comment to connect to a TCP-socket server, typically one started
    by Jasper's Python API.

-port port
    Specifies the port number of the server.

-hostname hostname
    Specifies the name of the host on which the server
    runs. If not specified, Jasper assumes that the
    server runs on the local host.

-disconnect
    Use this command to disconnect from a server previously connected to with
    "command_interface -connect -local" or "command_interface -connect -tcp".

    NOTE: a Jasper analysis session can be connected to only one command interface
    client at a time. An attempt to connect while already connected fails with an
    error.
```

compilation

Handles RTL compilation configuration, profiling, and debugging.

Syntax

```
Compilation Configuration
-------------------------
compilation -config [-repository (0 | 1) |-req]
           (-module |-instance) pattern [-regexp] {blackbox (0 | 1)}
compilation -config [-repository (0 | 1) |-req]
            -module module_name -array array_list {blackbox (0 | 1)}
compilation -config [-repository (0 | 1) |-req]
            -module '*' -dir pattern_dir {blackbox (0 | 1)}
compilation -config [-repository (0 | 1) |-req]
            -module '*' -file pattern_file {blackbox (0 | 1)}
compilation -config [-repository (0 | 1) |-req] -list
compilation -config [-repository (0 | 1) |-req] -show config_name_list
compilation -config [-repository (0 | 1) |-req] -remove config_name_list
compilation -config [-repository (0 | 1) |-req] -clear

Compilation CFE Configuration
-----------------------------
compilation -cfe_config -interface_mode (none | valid | nondet)
compilation -cfe_config -compact_signal_naming (true | false)
compilation -cfe_config
            -create_enable_signal_for_cpp_asserts (true | false)
compilation -cfe_config -dynamic_pruning
           [-total_time_limit total_time]
           [-per_branch_time_limit per_branch_time]
           [-check_branches (none | functions | all)]
           [-recheck_freq_level (0 | 1 | 2)]
           [-ignore_branch_reachability (true | false)]
           [-use_valid_enum_value_input_constraints (true | false)]
compilation -cfe_config -unroll_limit
           ( -line line_number
            |-name jasper_name
            |-label c_label)
           -file file_name
           -bound bound_value
```

```
compilation -cfe_config -unroll_pragma_name
           {pragma_name_list}
compilation -cfe_config -ignore_unimplemented_functions
           (-names {function_name_list} |-clear)
compilation -cfe_config -show config_name_list

Compilation CFE Debug Configuration
-----------------------------------
compilation -cfe_debug -gdb_path path
compilation -cfe_debug -terminal_binary_path path
compilation -cfe_debug -executable_path path
compilation -cfe_debug -compilation_command command

Compilation Profiling
---------------------
compilation -profile (-enable |-disable)
compilation -profile -report [-sort_by (time | memory)] [-limit limit_value] [-silent]

Compilation CFE Profiling
-------------------------
compilation -cfe_profile (-enable |-disable |-report)
           [-memory] [-output_file output_file_path] [-repository]

Compilation Debugging
---------------------
compilation -debug
compilation -debug -find string

Compilation Report
---------------------
compilation -report -resilient
```

Returns

```
Various return values
```

Detailed Description

```
Use this command to handle multiple aspects of compilation.

Compilation Configuration
```

```
-------------------------
-config [-repository (0 | 1) |-req]
    (-module |-instance) pattern [-regexp] {blackbox (0 | 1)}
    Add a compilation configuration to the specified compilation slot.
    If you do not specify a slot with "-repository" or "-req", the change
    affects the main design.

    . The configuration applies to modules or instances matching the given
      pattern.
    . Use "-regexp" to use regular expression matching. By default, the tool
      does wildcard matching.
      NOTE: The tool ignores invalid regular expressions silently.

    Current configurations supported are:
    blackbox (0 | 1): Choose whether to black box the modules or instances
    matching the pattern.

    A black box is a module with no observable or controllable content (that
    is, no logical function). By default, the outputs of a black-boxed module
    always produce a logic X. If you intend to bind in another module with an
    output port connected and output port wire, the output port wire will be
    polluted by the X value driven by the black box output port. Take note of
    this behavior. If you intend for your bind module to drive this wire, you
    must connect it to the black-box module output port signal stub inside the
    module.

    NOTE:
    . "blackbox 0" takes priority over "blackbox 1".
    . "blackbox 1" for modules takes priority over "blackbox 0" for instances.
    . Instance black box configurations only apply to the "elaborate" command.
    . Module black-box configurations apply to both "analyze" and "elaborate"
      commands. In addition to creating a black-box module, "analyze" black
      boxes also have the following semantics:
      . The tool will try to ignore syntax errors in matching modules.
      . The tool might ignore port definitions for matching modules.
      . If you attempt to black box the "-top" module, the tool issues an
        error during elaboration.
      If any of these is undesirable, you must run "compilation -config" after
      all the "analyze" commands and before the "elaborate" command.

-config [-repository (0 | 1) |-req]
    -module [library_name.]module_name -array array_list {blackbox (0 | 1)}
```

```
    Add a compilation configuration to the specified compilation slot.
    If you do not specify a slot with "-repository" or "-req", the change
    affects the main design.

    The configuration applies to arrays in "array_list" defined under module
    "module_name". If a library "library_name" is specified, the configuration
    applies only to the module inside "library_name". If no library is specified,
    the tool searches for module "module_name" on all libraries and applies the
    configuration on all ocurrences.

    Current configurations supported are:
    blackbox (0 | 1): Choose whether to black box the arrays in "array_list".

    An array black box is a simplified model of an array. It contains limited
    observable or controllable content. By default, the outputs of a black-boxed
    array always produce a logic X.

    This model is often referred to as MPRAM (Multi-Port RAM).

    NOTE: "blackbox 0" takes priority over "blackbox 1".

    LIMITATIONS: Blackboxing arrays is not supported in the following cases:
    . Arrays that are module ports
    . Arrays connected to user-defined instances
    . Arrays that are not read-only
    . Arrays inside scopes (for example, generate blocks or always blocks)
    . Slices of arrays

-config [-repository (0 | 1) |-req]
    -module * -dir pattern_dir {blackbox (0 | 1)}

    Add a compilation configuration to the specified compilation slot.
    If you do not specify a slot with "-repository" or "-req", the change
    affects the main design.

    The configuration applies to all modules or entities defined in files under
    directories that match the "pattern_dir". It considers files inside
    subdirectories and supports wildcard matching.

    The tool does not apply the configuration to modules defined inside files
    that are included with an `include directive if they do not match the given
pattern.
```

```
    NOTE: This command applies to the "elaborate" command only. It has no
    effect on the "analyze" command.

     Supported configurations:
    . -module *: Apply the command to all modules or entities.
    . blackbox (0 | 1): Choose whether to black box all modules or entities
      in files under directories that match the "pattern_dir".

    NOTE: "blackbox 0" takes priority over "blackbox 1".

-config [-repository (0 | 1) |-req]
    -module * -file pattern_file {blackbox (0 | 1)}

    Add a compilation configuration to the specified compilation slot.
    If you do not specify a slot with "-repository" or "-req", the change
    affects the main design.

    The configuration applies to all modules or entities defined in files
    that match the "pattern_file" and supports wildcard matching.

    The tool does apply the configuration to modules defined inside files
    that are included with an `include directive if they do not match the given
pattern.

    NOTE: This command applies to the "elaborate" command only. It has no
    effect on the "analyze" command.

    Supported configurations:
    . -module *: Apply the command to all modules or entities.
    . blackbox (0 | 1): Choose whether to black box all modules or entities
      in files that match the "pattern_file".

-config [-repository (0 | 1) |-req] -list
    List all compilation configurations in the main or specified compilation
    slot.

-config [-repository (0 | 1) |-req] -show config_name_list
    Show the details for the given configurations applied to the main or
    specified compilation slot.

-config [-repository (0 | 1) |-req] -remove config_name_list
    Remove configurations with the specified names from the main or specified
    compilation slot.
```

```
-config [-repository (0 | 1) |-req] -clear
    Remove all configurations applied to the main or specified compilation slot.


Compilation CFE Configuration
-----------------------------
-cfe_config -interface_mode (none | valid | nondet)
    Provide the same controls as "set_cfe_interface_mode".
    Choose how the CFE models outputs, probes, and buffers whose defining
    code might never execute.
    . none: Leave the signal unconstrained when unreachable.
    . valid: Emit a companion NAME_jasper_valid signal.
    . nondet: Emit NAME_jasper_valid plus a NAME_jasper_nondet free variable
      that drives the signal whenever it is unreachable.

-cfe_config -compact_signal_naming (true | false)
    Mirrors "set_cfe_compact_signal_naming".
    Enable or disable compact (zero-indexed) signal naming in the model.

-cfe_config -create_enable_signal_for_cpp_asserts (true | false)
    Matches "set_cfe_create_enable_signal_for_cpp_asserts".
    Control whether the model contains the global SVA enable signal
    "jg_cpp_asserts_enable" to gate assertions generated from C++ asserts.

-cfe_config -dynamic_pruning
            [-total_time_limit total_time]
            [-per_branch_time_limit per_branch_time]
            [-check_branches (none | functions | all)]
            [-recheck_freq_level (0 | 1 | 2)]
            [-ignore_branch_reachability (true | false)]
            [-use_valid_enum_value_input_constraints (true | false)]
    Run this switch before compiling your C/C++ source so the generated
    model honors the limits.
    . Use "-total_time_limit" to cap the cumulative pruning time. A value of
      0 disables the cap.
    . Use "-per_branch_time_limit" to cap the time spent on a single branch
      feasibility check. A value of 0 disables the cap.
    . Use "-check_branches" with value "none", "functions" (virtual/function
      pointer calls), or "all" (default) to pick which branches are checked.
    . Use "-recheck_freq_level" to control how often the tool revalidates a
      branch that previously proved feasible (0, 1 [default], or 2).
    . Use "-ignore_branch_reachability" to treat currently infeasible
```

```
      branches as unreachable without recording additional constraints.
    . Use "-use_valid_enum_value_input_constraints" to restrict dynamic
      pruning to enumerated values declared through JASPER_INPUT enums.

-cfe_config -unroll_limit (-line line_number |-name jasper_name |-label c_label)
      -file file_name -bound bound_value
    Set loop, recursion, or VLA bounds without inserting pragmas. The
    file must match the path used during "check_c2rtl -compile" or
    "analyze". The switch accepts one locator ("-line", "-name", or "-label")
    together with a bound and file name. Bounds set inside unused inline
    functions raise an error unless you downgrade the related messages.

-cfe_config -unroll_pragma_name {pragma_name_list}
    Update the set of pragma identifiers (default: jasper_unroll_limit)
    that represent local unroll directives. Configure the list before
    running "check_c2rtl -compile".

-cfe_config -ignore_unimplemented_functions (-names {function_name_list}
      |-clear)
    Manage the list of functions that the CFE should treat as ignored if
    they are not implemented.

-cfe_config -show config_name_list
    Display the current value of one or more configuration names.
    You can query "interface_mode", "compact_signal_naming",
    "create_enable_signal_for_cpp_asserts", "dynamic_pruning","unroll_limit",
    "unroll_pragma_name", or "ignore_unimplemented_functions".

Compilation CFE Debug Configuration
-----------------------------------
-cfe_debug -gdb_path path
    Set or query the path to GDB used by cfe_debug. Equivalent to
    "set_gdb_path".

-cfe_debug -terminal_binary_path path
    Set or query the terminal binary path used by cfe_debug. Equivalent to
    "set_terminal_binary_path".

-cfe_debug -executable_path path
    Set or query the executable path used by cfe_debug. Equivalent to
    "set_executable_path".

-cfe_debug -compilation_command command
```

```
    Set or query the compilation command used by cfe_debug. Equivalent to
    "set_compilation_command".

Compilation Profiling
---------------------
-profile (-enable |-disable)
    Enable or disable compilation profiling.

-profile -report [-sort_by (time | memory)] [-limit limit_value] [-silent]
    List the profile information from all modules and return a Tcl dictionary
    with profiling information from all modules.
    Profiling information includes time in seconds and memory consumption in
    MB per module.
    . Use "-sort_by" to sort the list in descending order by memory or time values.
    . Use "-limit" to limit the size of the list returned.
    . Use "-silent" to suppress text output to the console and log files.

    NOTE: Currently, profiling information is only computed during hierarchical
    netlist compilation. Other compilation steps are not profiled.


Compilation CFE Profiling
-------------------------
-cfe_profile (-enable |-disable |-report)
    Control the C/C++ model profiler used during compilation of C/C++ designs.

    . Use "-enable" to enable the C/C++ model profiler.
      . Use "-memory" to additionally enable memory profiling of the C/C++
        model (using a VM RSS-based metric).

    . Use "-disable" to disable the C/C++ model profiler and reset
      the memory profiling metric.

    . Use "-report" to dump the C/C++ model profiler data to a text file.
      . Use "-output_file output_file_path" to specify the output file path.
      . If you do not specify "-output_file", the tool writes the report
        to a default file in the current project directory.


Compilation Debugging
---------------------
-debug
    Debug an error or warning message on a previous "analyze" or "elaborate"
```

```
    command. The following messages are currently supported:
    . VERI-1063
    . VERI-1128
    . VERI-1158
    . VERI-1207

    NOTE: Currently, messages coming from commands using "-req" or "-repository"
    switches are not supported.

-debug -find string
    Find the specified string using the same heuristics as "compilation -debug".

    Use this command to leverage previous "analyze" commands to find a
    specified string. This command is useful when you have a compilation issue
    due to a missing definition.


Compilation Report
----------------------
-report -resilient
    Prints the Resilient Compilation Report. This report lists all resilient-
    related messages, grouped by "Resilient Action", and gathers the complete
    text of the "Resilient Message", the "Reason" for the resilient action,
    and the summarized action. The tool issues the Resilient Compilation Report
    to the log or the Jasper GUI Console tab.

Example:
================================================================================
================================
Resilient Compilation Report
================================================================================
================================

 Resilient Action   | always block statement ignored
 Resilient Message  | (VERI-9084)design.sv(11): resilient compilation has ignored
statement in always block, due to
                    | (VERI-2289) at design.sv(11)
 Reason             | (VERI-2289)design.sv(11): signal clocked multiple times is not
synthesizable
--------------------------------------------------------------------------------
--------------------------------

================================================================================
```

```
==================================
```

Example

```
# Configuring instance "arb" under top module "top" to be black boxed. Notice the top
module name is implied
compilation -config -instance arb {blackbox 1}
# Configuring modules containing the word "FIFO" to be black boxed while not black
boxing module FIFO_32
compilation -config -module {FIFO} -regexp {blackbox 1}
compilation -config -module {FIFO_32} {blackbox 0}
# Configuring modules containing the word "FIFO" to be black boxed in compilation slot
for repository 0
compilation -config -repository 0 -module {FIFO} -regexp {blackbox 1}

# Enable profiling and get profiling information
compilation -profile -enable
elaborate -top mytop
compilation -profile -report

# Investigate causes for an error during elaboration
catch {elaborate -top mytop}
compilation -debug

# Configure the C/C++ frontend with the unified switch
compilation -cfe_config -interface_mode nondet
compilation -cfe_config -unroll_limit -line 120 -file c_model.cpp -bound 64
compilation -cfe_config -show {dynamic_pruning}
```

See Also

analyze,
elaborate

compile

Compiles C/C++ or SystemC/HLS sources for the current session.

Associated App(s)

CFPV

Syntax

```
C/C++ Compilation
-----------------
compile -cpp
    ( list_of_source_files|
     [list_of_source_files]
     (-f file_name)+)
    [-I include_dir]+
    [-D macro_definition]+
    [-U macro_name]+
    [-include file_name]+
    [-std version]
    [-pedantic_errors]
    [-l library]+
    [-top model_top_name]
    [-unroll_limit N]
    [-max_vla_size N]
    [-max_array_sroa N]
    [-force_cpp]
    [-resolve_getenv]
    [-create_valid_enum_value_input_constraints]
    [-create_related_covers
        {(precondition | late_precondition | infinite_precondition
            | witness | contrapositive | infinite_contrapositive)+}]
    [-verbosity N]
    [-gcc_compatibility_version version]

SystemC/HLS Compilation
-----------------------
compile -hls
    [-no_top_wrapper]
    [-top model_top_name]
    [-flags string]
    [-top_template_params string]
    [-bbox_m string]
    [-use_bundled_stratus_headers]
```

Returns

Various, context sensitive return values.

Detailed Description

```
Use this command to compile source code for C/C++ or HLS/SystemC flows from a
single top-level command. The "compile" command provides dedicated subcommands
for each language family:

. "compile -cpp" for C/C++ model compilation.
. "compile -hls" for HLS/SystemC model compilation.

C/C++ Compilation
-----------------
-cpp
    ( list_of_source_files |
     [list_of_source_files](-f file_name)+)
    [-I include_dir]+
    [-D macro_definition]+
    [-U macro_name]+
    [-include file_name]+
    [-std version]
    [-pedantic_errors]
    [-l library]+
    [-top model_top_name]
    [-unroll_limit N]
    [-max_vla_size N]
    [-max_array_sroa N]
    [-force_cpp]
    [-resolve_getenv]
    [-create_valid_enum_value_input_constraints]
    [-create_related_covers (precondition | late_precondition
            | infinite_precondition | witness | contrapositive
            | infinite_contrapositive)+]
    [-verbosity N]
    [-gcc_compatibility_version version]

    Parse C or C++ code and synthesize a netlist whose behavior mimics the
    behavior that the code would have if compiled by GCC for an Intel x86 64
    bit CPU.

    . "list_of_source_files" accepts a list of C/C++ source files.
    . Use "-I include_dir" to add "include_dir" to the list of directories
      searched to resolve "#include" directives to header files. C2RTL
      implements exactly the same algorithm as GCC to search for a header file
      when preprocessing an "#include" directive. For details, refer to GCC's
```

```
  documentation about option "-I".
. Use "-D macro_definition" to define a macro. You can pass either
  "name" or "name=def". If you pass "name", C2RTL treats it as
  "name=1".
. Use "-U macro_name" to cancel any previous definition of "macro_name",
  either built-in or provided with "-D".
. Use "-include file_name" to preinclude "file_name" before compilation,
  as if GCC had received option "-include".
. Use "-l library" to link the specified library. Available libraries are:
    - systemc: Links the Jasper-provided SystemC library.
    - softfloat: Links the Berkeley SoftFloat (SoftFloat-3e) library.
    - ac: Links the Jasper-provided AC datatypes library.
    - m: Enables support for additional <math.h> (or <cmath>) functions.

    Functions available WITHOUT "-l m" (always supported):
        . ceil*
        . copysign*
        . fabs*
        . fdim*
        . floor*
        . fmax*
        . fmin*
        . fmod*
        . fpclassify
        . isfinite+
        . isinf*
        . isnan*
        . isnormal+
        . llrint*
        . llround*
        . lrint*
        . lround*
        . modf*
        . nan*
        . nearbyint*
        . remainder*
        . rint*
        . round*
        . signbit+
        . trunc*

    Additional functions enabled WITH "-l m":
        . acos* (c)
```

```
    . atan* (c)
    . exp* (c)
    . exp2* (c)
    . fma*
    . frexp*
    . ilogb*
    . ldexp*
    . log* (c)
    . log2* (c)
    . log10* (c)
    . pow* (c)
    . scalbln*
    . scalbn*
    . sincos* (c)
    . sqrt* (c)

NOTES:
    . (*) means that all the C99 "overloads" and gcc __builtin versions
      are available. For instance, for "fmax", the following functions
      are available: fmax(), fmaxf(), fmaxl(), __builtin_fmax(),
      __builtin_fmaxf(), and __builtin_fmaxl().
    . (+) means that the function is also available as a gcc __builtin.
      For instance, for "isfinite", the following functions are
      available: isfinite() and __builtin_isfinite().
    . (c) means that the implementation of this function is complex,
      and that repeatedly calling it with non-constant arguments
      increases the size and complexity of the extracted formal model.
    . Since the "long double" datatype is synthesized in exactly the
      same way as "double" (using 64 bits), all the "l" overloads
      (for instance, fmaxl or nanl) behave as their "double"
      counterparts (fmax or nan).
    . The implementation of the additional functions is generally as
      precise as what you might expect of any modern C library. In
      particular, they do not guarantee to return a correctly-rounded
      result (the nearest floating-point representation of the exact
      result).
NOTE:
. You do not need to explicitly pass the C source files from these
  libraries in the list_of_cpp_files argument of the
  "compile -cpp" command.
. If you want to use your own version of any of these libraries,
  do not use the "-l library" option.
```

. Use "-std version" to set the version of the C++ standard (ISO/IEC 14882) used to determine how to accept C++ input files. Valid values for "version" are:
  . c++11
  . c++14 (default value when no -std option is given)
  . c++17
  . c++20

  C programs are parsed using the C07 standard, and currently it is not possible to configure a different standard.

. Use "-pedantic_errors" to direct C2RTL to do the following:
  . disable all C/C++ language extensions, and
  . emit all diagnostics (compilation warning/error messages) that the standard requires the compiler to issue.

  The "standard" means either ISO C 2007 for C files or the standard indicated by switch "-std" for C++ files.

  By default, C2RTL (and GCC) lets the input C/C++ program use language extensions (language or library features that are not defined in the ISO C/C++ standard). Such extensions might reject standard-conforming programs, or even alter the behavior that the standard specifies for them.

  Switch "-pedantic_errors" switches off all such extensions, which precisely means that C2RTL:

  . accepts the input program if and only if it strictly conforms to the ISO C/C++ standard (it only uses language/library features specified in the standard), and
  . assigns behavior to the program as specified by the standard.

  C2RTL treats files with extension .cc, .cp, .cxx, .cpp, .CPP, .c++, and .C as C++ code. It treats files with extension .c as C code. Any other file extension causes an error.

  The "-std" and "-pedantic_errors" switches control the exact dialect of the C and C++ languages that C2RTL accepts. Generally speaking, passing "-std c++NN" causes C2RTL to parse:
  . C files the same way as "gcc -std=gnu99" would, and
  . C++ files the same way as "g++ -std=gnu++NN" would.

  Passing "-pedantic_errors" has the same effect as GCC option "-pedantic-errors". It disables all language extensions (see below) and

makes C2RTL accept programs exactly as specified by the standard given by "-std".

More specifically, C and C++ files are parsed as described below.

C files are matched against a dialect of C07 (ISO/IEC 9899:2007). C07 is identical to C99 plus the corrections introduced in the three ISO Technical Corrigenda published in 2001, 2004, and 2007.

The precise C dialect accepted is C07 when you use "-pedantic_errors". When you do not use it, C2RTL accepts C07 extended by:

- a large number of minor extensions commonly available on other C compilers
- a large list of GCC language extensions (see chapter 6 of the GCC manual) which emulate the GCC version indicated by option "-gcc_compatibility_version".

Most of these extensions enlarge the accepted language (make C2RTL accept all C07 programs and more), but some of them shrink it (some C07 strictly-conformant programs get rejected or their behavior is altered). This is because some extensions contradict ISO C 2007.

You cannot change the language dialect used to accept C input files.

C++ files are matched against a dialect of C++ defined as follows. When you use "-pedantic_errors", passing "-std c++NN" makes C2RTL accept C++ files as defined by language standard C++NN (ISO/IEC 14882, version NN). When you do not use "-pedantic_errors", C2RTL accepts C++NN extended by:

- the same minor extensions used for C files, and
- a small number of C++ language extensions, and
- a large list of GCC language extensions (see chapters 6 and 8 of the GCC manual) that emulate the GCC version indicated by option "-gcc_compatibility_version".

As for C files, some of the language extensions might conflict with ISO C++NN, making C2RTL reject or alter the behavior of C++ programs that strictly conform with ISO C++NN.

. Use "-top" to give a name to the top-level of the synthesized netlist. If you do not specify a name, the tool uses "main" as the default name.
. Use "-unroll_limit N" to specify the maximum number of loop iterations and the maximum recursion depth in the C++ code. The default value is 256.
. If the tool is able to determine that the upper bound on the number of

iterations is smaller than the specified value "N", it uses the smaller value.
  . If the tool is not able to determine this upper bound or the upper bound is greater than "N", it unrolls the loops and the recursive calls "N" number of times.
. Use "-max_vla_size N" to specify the default hardware synthesized for all VLAs in the C++ model. The default value is 256 bytes.
  . Example: compile -cpp "main.cpp" -max_vla_size 512.
  . If the array expands to use more memory than max_vla_size, then the corresponding VLASizeLimit assertion will fail.
. Use "-max_array_sroa N" to modify the maximum length of arrays to which the tool applies Scalar Replacement Of Aggregates (SROA).
  SROA replaces every C/C++ array of length L (with L <= N) by L scalar variables. This makes it possible to propagate constants through array read or write operations, and makes other optimizations to work better.
  NOTE:
  . By default, SROA uses N = 65536.
  . If your C/C++ model does not have large arrays, there is generally no benefit in modifying the default N.
  . If the model has large arrays, formal-model generation might run faster if you lower N, at the cost of possibly extracting a larger formal model.
. Use "-force_cpp" to compile all .c files as C++.
. Use "-resolve_getenv" to resolve the environment variable value during compilation in "getenv" function with a string constant only. Otherwise, calling getenv has no effect.
. Use "-create_valid_enum_value_input_constraints" to generate valid enum value assumptions, one for every enum variable declared using JASPER_INPUT. Each assumption ensures that the input is only one of the declared enum values.
. Use "-create_related_covers" to enable the precondition check for one or more specified types of cover statements.
  The name of the precondition property indicates the precondition type, for example, propertyA:witness1.
  The tool creates related covers as follows. In these examples, s1 and s2 are sequences.
  . The precondition cover of an SVA implication s1 |=> P, is a finite trace where sequence s1 matches somewhere on the trace. This is the default.
  . The late precondition cover of an SVA implication s1 |=> s2 is a cover of the sequence (s1 ##1 1'b1).
  . The infinite precondition cover of an SVA implication s1 |=> P, where P is a infinite property, is an infinite trace where s1 matches

```
      somewhere on the trace.
    . The witness cover of an SVA implication s1 |=> s2, where s2 is a sequence,
      is the cover of the sequence (s1 ##1 s2).
    . The witness cover of an SVA implication s1 |-> s2, where s2 is a sequence,
      is the cover of the sequence (s1 ##0 s2).
    . The contrapositive cover of an SVA implication s1 |-> P, is a finite
      trace where P is not true somewhere on the trace.
    . The infinite contrapositive cover of an SVA implication
      s1 |=> s_eventually B, where B is a Boolean expression, is an infinite
      trace where, starting at some point, B never matches.
    Jasper creates the contrapositive cover property only when the right
    hand side of the property is Boolean. The feature does not support
    sequence/property operators.
    The following example triggers a contrapositive property only when y is
    Boolean.
    x |=> y  creates cover (~y)
    NOTE:
    . The tool does not support witness precondition covers for assumptions with
      local variable assignments.
    . By default, the tool creates "precondition" covers.
    . This command accepts Tcl lists.
  . Use "-verbosity N" to specify the level of message verbosity you want.
    If N = 5, the tool shows a brief description of warning messages.
    The default is N = 6, which leads the tool to print all warning and error
    messages as is.
  . Use "-f" to specify a list of files for C or C++ model compilation.
    This switch lets you specify additional source files from a file.
    NOTE:
      . These commands do not support file names with whitespace or the
        character "$".
      . Use multiple "-f* file_name" instances to specify multiple files.
      . Unknown and prohibited switches in "-f*" file(s) will be ignored.
      . Jasper Apps supports absolute paths declared inside .f* file(s) for
        example, /home/user/test.cpp. All other paths are related to the
        the current working directory of Jasper.
      . You can specify C or C++ source files.
      . Use one source file per line in the "-f* file_name".
      . Coroutines and modules are not supported in C++20.

  . Use "-gcc_compatibility_version version" to specify the version of gcc
    to be compatible with the compilation. The default version is equivalent
    to GCC version 9.3.0.
    Available versions are: gcc12_3, gcc9_3, and gcc6_3.
```

```
      GCC 12.3 is automatically used when C+20 is specified for compilation.

SystemC/HLS Compilation
-----------------------
-hls [-no_top_wrapper]
     [-top model_top_name]
     [-flags string]
     [-top_template_params string]
     [-bbox_m string]
     [-use_bundled_stratus_headers]

    Compile SystemC code, analyze and elaborate it.

    . Use "-no_top_wrapper" to disable generation of an sc_main
      with top instantiation.
    . Use "-top" to give a name to the top-level of the synthesized netlist.
    . Use "-flags" to give hlsfe compilation switches design attribute.
    . Use "-top_template_params" to use string as top wrapper module
      template params.
    . Use "-bbox_m" to declare a list of modules to be blackboxed.
    . Use "-use_bundled_stratus_headers" to add Stratus header files to the
      include path. These header files contain custom modifications to
      support pipelining of "cynw_p2p" interfaces.
```

complexity_manager

Launches a GUI display of design statistics to show complexity.

Syntax

```
complexity_manager (no arguments)

complexity_manager -task task_name
                   [-include_coi]
                   [-include_assumption]
                   [-exclude_constant]

complexity_manager -property {property_list}
                   [-include_coi]
                   [-include_assumption]
                   [-exclude_constant]
```

Returns

No return values

Detailed Description

Use this command to launch the Complexity Manager Viewer. Complexity Manager is a tool that displays design statistics to help you learn about the COI and find sources of complexity. This window shows data for the entire design or the COI of one or more properties or the combined COI of all properties in a task.

Complexity Manager includes columns for nets, gates, registers, and so forth to expose complexity hot spots.

To learn more about the Complexity Manager, consult the "Jasper Platform and Formal Property Verification App User Guide" (Help - User Guide).

no arguments
    Launch the Complexity Manager Viewer for the entire design logic.
    In this mode, Complexity Manager Viewer does not consider properties or

```
    abstractions included on any task.
    When analyzing the full design, the Complexity Manager considers environment
    abstractions but ignores stopats.

-task task_name
      [-include_coi] [-include_assumption]
      [-exclude_constant]
    Launch the Complexity Manager Viewer for the specified task.

    This analysis respects hard stopats and instance stopats in the specified
    task.

    . Use "-include_coi" to include logic in the COI of the target.

    . Use "-include_assumption" to include logic from the assumptions.

    . Use "-exclude_constant" to exclude logic due to constant propagation.

-property {property_list}
          [-include_coi] [-include_assumption]
          [-exclude_constant]
     Launch the Complexity Manager Viewer for one or more specified properties.

     This analysis respects hard stopats and instance stopats in the COI of
     corresponding properties.

    . Use "-include_coi" to include logic in the COI of the target.

    . Use "-include_assumption" to include logic from the assumptions.

    . Use "-exclude_constant" to exclude logic due to constant propagation.
```

Example

```
# Launch Complexity Manager for a property list
complexity_manager -property {prop1 prop2}

# Launch Complexity Manager for a task, including its assumptions in the
# analysis
complexity_manager -task task -include_assumption
```

See Also

get_design_info

config_rtlds

Configures different aspects of RTL Designer flow.

Syntax

```
config_rtlds -port signal_list -clock (clock_list | async_to_all | sync_to_all) [-add]

config_rtlds -port signal_list -module module_name
             ( -input input_list
              |-clock clock_list
              |-reset reset_list )+
            [-add]

config_rtlds -port signal_list -external_sync -destination_clock clock_list
            [-source_clock clock] [-add]

config_rtlds -port signal_list -reset reset_list [-add]

config_rtlds -port -reset -clear

config_rtlds -clock [-create {target_signal_list}]
            [-name clock_name] [-factor N] [-add]

config_rtlds -clock -group
            ((-async |-sync |-physically_exclusive |-logically_exclusive)
{{clock_list}+}
            [-name group_name])

config_rtlds -rule
            ((-enable |-disable)
                ( -tag tag_name_list
                 |-category category_name_list
                   [-severity (info | warning | error)]
                 |-domain (domain_name_list | all)
                   [-severity (info | warning | error)]))
            [-regexp]

config_rtlds -rule -load {file_name_list+}
```

```
config_rtlds -rule -save file

config_rtlds -rule
           ((-exclude |-include) (-tag |-category |-domain) {items_name_list} (-
instance {instance}+ |-hierarchy) {instance}+)

config_rtlds -rule
           ( -severity severity -tag tag_list
            |-parameter {key=value}+ -app app_list
            |-parameter {key=value}+ -domain domain_list
            |-parameter {key=value}+ -tag tag_list
            |-show (-tag {tag_list} |-category {category_list} |-domain {domain_list})

config_rtlds -rule
           -set_default rule_file_name

config_rtlds -test_clock signal
            ([-mode (shift | capture)])

config_rtlds -reset
            ( -sync signal_list -clock clock_list
             |-async signal_list
             |-synchronized signal_list -clock clock_list)
            -polarity (high | low)
            [-virtual]

config_rtlds -reset -set_order {reset_signal+ {reset_signal+}+}

config_rtlds -signal
       ( -constant {{signal_name value}+} [-condition expression] [-force]
        |-static signal_list [-condition expression] [-force]
        |-exclusive signal_list [-condition expression]
        |-gray_code signal_list [-condition expression]
        |-controllable signal_list)
        [-mode (global | functional | shift | capture)]
        [-override] [-condition expression]
        |-false_path ( -from list
                        |-to list
                        |-through list)+
                        [-regexp]
        |-reset reset_port (-on_assert |-on_deassert |-both)
              -inactive_clock clock_port_list
              |-constant_data {{data_port_list value}+}
```

```
        |-power_control power_control_port
              -inactive_clock clock_port_list
              |-inactive_reset reset_port_list
              |-constant_bit {{signal value}+}


config_rtlds -signal -check_conflict

config_rtlds -signal -prove

config_rtlds -signal -clear

config_rtlds -signal (-remove id_tcl_list)

config_rtlds -mode
            (((-parameter {{parameter_name value}+}
              |-tcl tcl_file
              |-sdc sdc_file
              |-db db_file)
                [-name name] [-description description])
           |-remove mode_name
           |-close)

config_rtlds -module module_name
       ( -controllable [-port {port_name}]
        |-constant {{port_name value}+}
        |-reset {{port_name (async | sync | synchronized)}+}
       [-mode (functional | shift | capture)+]
       [-override] )

config_rtlds -memory_cell module_name
            [-input signal_name]
            [-output signal_name]
            [-chip_select signal_name [-chip_select_value constant_value]]
            [-write_enable signal_name [-write_enable_value constant_value]]
            [-write_clock signal_name]

config_rtlds -list signal_config

config_rtlds -list (tag | domain)

config_rtlds -list test_clock
```

```
config_rtlds -list mode

config_rtlds -list connectivity

config_rtlds -connectivity
           ( (-from signal_list |-from_one_of signal_list)
             (-to signal_list |-to_one_of signal_list)
              -logic_type (wire | logic)
              -allow_modules_in_path module_list
              -type (mandatory | invalid)
              -destination_type (flop_data | flop_reset | latch_data | latch_en)
            [-comment comment] )

config_rtlds -no_scan
            [-module module_name [-regexp]]
            [-instance instance_name [-regexp]]
            [-flop flop_name_list]

config_rtlds -custom_rule tag_name
            (-type (structural | basic_lint)
             -tcl_proc proc_name
             -tcl_file tcl_file_name
            [-callbacks call_back_name_list])
```

Returns

Various

Detailed Description

Use this command to configure the RTL Designer flow.

NOTE:
. A tag is a type of rule extracted in apps that are part of RTL Designer signoff.
. A category is the name of a certain group of tags or other categories that you might specify. Categories can contain many other categories or tags, but each category must have no more than one parent. Use categories to more easily manage tags.
. A domain is a super category that contains tags relevant to its type. Currently supported domains include LINT, DFT, AUTO_FORMAL, and CDC. Some

```
  domains might be unavailable depending on the rule file provided.

-port signal_list -clock (clock_list | async_to_all | sync_to_all) [-add]

    Defines the clock signals associated with the specified list of top inputs
    and outputs, black-boxed inputs and outputs, and signals with stopats.

    . Use "-add" to define clock associations for the specified signals
      incrementally.
    . Use "async_to_all" to associate the signals to a clock asynchronous
      to all other declared clocks in the design.
    . Use "sync_to_all" to associate the signals to a clock synchronous
      to all other declared clocks in the design.

    NOTE: By default, the tool treats all ports as synchronous to other CDC
    units when not manually rated. This behavior is controlled by the parameter
     "treat_data_ports_as_async".

-port signal_list -module module_name (-input input_list |-clock clock_list
    |-reset reset_list)+ [-add]

    Creates clock or reset rating to the ports of all instances of a
    given module.

    . Use the combination of "-module" and "-clock" to create rating
      between the module data ports in the "-port" argument and the module
      clock ports in the "-clock" argument. These ratings are applied
      to all instantiations of the given module by rating the instance data
      ports to the primary clocks that are actually connected to the instance's
      clock ports.

      NOTE: In the absence of manual rating of ports, the parameter
      "treat_data_ports_as_async" determines the rating of those ports. By
      default, all ports are treated as synchronous to other CDC units when
      not manually rated.

    . Use the combination of "-module" and "-reset" to create reset rating
      between data ports and reset signals.

      NOTE: In the absence of manual reset rating of ports, the parameter
      "treat_data_ports_as_async" determines the reset rating of those ports. By
      default, all ports are treated as synchronous to all resets when not
      manually rated.
```

```
    . Use the combination of "-module" and "-input" to consider both the clock and
      reset driving the pin specified in the "-input" argument as the ratings of
      the ports specified in the "-port" argument.

      NOTE:
        . Only module-level ports can be passed in the "-port", "-clock", "-input",
          and "-reset" arguments.
        . You can use the "-clock", "-input", and "-reset" switches in the same
          command.

    . Use "-add" to define the associations for the specified signals
      incrementally.

-port signal_list -external_sync -destination_clock clock_list
    [-source_clock clock] [-add]

    Specify that the ports in "signal_list" are externally synchronized into
    the destination_clock.

    . Use "-source_clock" to specify the source clock from which these ports
      are synchronized.
    . Use "-add" to define clock associations for the specified signals
      incrementally.

-port signal_list -reset reset_list [-add]

    Specify that the ports in "signal_list" are rated by the resets in
    "reset_list".

    . Special resets available:
      . Use "reset_async_to_all" to associate the signals to a reset
        asynchronous to all other resets in the design.
      . Use "reset_sync_to_all" to associate the signals to a reset that
        behaves the same as all other resets in the design and will
        generate an RDC pair.
    . Use "-add" to incrementally define reset ratings for the specified
      signals.

    NOTE: By default, the tool treats all ports as synchronous to all resets
    when not manually rated. This behavior is controlled by the parameter
     "treat_data_ports_as_async".
```

```
-port -reset -clear

    Remove all previously declared reset port ratings.

-clock [-create {target_signal_list}] [-name clock_name] [-factor N] [-add] [-virtual]
    Use this command to create a clock definition driving the target signals
    or to change the factor of an existing clock.

    . Creating a clock on a target with a previous definition overwrites the
      previous definition on that target. Use "-add" to superpose different
      clock definitions on the same target.
    . If the "-name" switch is omitted, a procedurally generated name is used
      for the clock definition.
    . The default factor for clock definitions is 1. Use "-factor" to specify a
      different value.
    . If the "-create" switch is omitted, the factor of the clock specified
      with the "-name" switch is changed to the value specified by the
      switch "-factor".
    . Use "-virtual" to create signals that can be used as
      if they existed in the design for clock definition, manipulation, and
      association.
    NOTE: Clocks defined with this command are available for formal use only
    after running the CDC command "check_cdc -extract".

-clock -group
    ((-async |-sync |-physically_exclusive |-logically_exclusive) {{clock_list}+}
    [-name group_name])
    Use "-async", "-sync", "-physically_exclusive", or "-logically_exclusive"
    to declare synchronicity between groups of clocks.
    And use "-name" to specify a name for the group.
    NOTES:
      . The synchronicity among clocks of the same group is determined by
        parameter "all_clocks_sync_by_default".
      . A command containing a single group sets the synchronicity between
        every clock of that group and all other clocks not contained in that group.
      . A command containing multiple groups sets the synchronicity between
        every clock of every group and every clock of the other groups.


-rule
    ((-enable |-disable)
      ( -tag tag_name_list
       |-category category_name_list
```

```
        [-severity (info | warning | error)]
      |-domain (domain_name_list | all)
        [-severity (info | warning | error)]))
    [-regexp]
    Use "-enable" or "-disable" to enable or disable the specified tags,
    categories, or domains. Enabling or disabling a category or domain also
    enables or disables all corresponding tags or categories.
    NOTE: These switches are effective only if a rule file was previously loaded.
    You can use "config_rtlds -rule -load" to load a rule file.

    . Use "-domain all" to enable or disable all checks.
    . Use "-severity" to apply the configuration to a category or domain
      with the specified severity only.
    NOTE: This command supports wildcards and regular expressions. Use "-regexp"
    to configure all checks that match the specified regular expression.

-rule -load {file_name_list+}
    Load a predefined configuration in a rule file. The tool checks the
    loaded rule file for validity against the XML file that contains
    default static data and rules about dependencies between tags, domains,
    and categories.
    NOTE: If you try to load a rule file after elaboration, tool will issue an error.

-rule -save file
    Save current configuration items, that is, domains, categories, and tags,
    with their parameters to a rule file.
    NOTE: This switch is effective only if a rule file was previously loaded.
    You can use "config_rtlds -rule -load" to load a rule file.

-rule
    ((-exclude |-include) (-tag |-category |-domain) {items_name_list}+
      (-instance {instance}+ |-hierarchy {instance}+))
    Exclude or include the specified checks.

    . Use "-exclude" to exclude from extraction one or more tags, categories,
      or domains from the specified instances or hierarchies. By default, all
      enabled tags are extracted.
    . Use "-include" to include one or more tags, categories, or domains that
      you previously excluded with the "-exclude" command.

-rule
    ( -severity severity -tag tag_list
     |-parameter {key=value }+ -app app_list
```

```
     |-parameter {key=value }+ -domain domain_list
     |-parameter {key=value }+ -tag tag_list
     |-show (-tag {tag_list} |-category {category_list} |-domain {domain_list}))

    Manipulate default values or show check details.

    . Use "-severity" with "-tag" to change the default severity of tags in
      the "tag_list". Supported severity levels include "Note" , "Warning",
      " Error", and "Fatal".
    . Use "-parameter" with "-app" to change the default values of different
      parameters used as input for the apps in "app_list".
    . Use "-parameter" with "-domain" to change the default values of different
      parameters used as input for the domains in "domain_list".
    . Use "-parameter" with "-tag" to change the default values of different
      parameters used as input for the tags in "tag_list".
    . Use "-show" to return a Tcl list of tags along with their statuses
      (enabled=ON or disabled=OFF). For example,
      {{IDN_NF_NMCV ON} {IDN_NR_AMKY ON} {MOD_NS_GLGC OFF}}

-rule
    -set_default rule_file_name
        configures the default rule file that will be used in "include default"
        directive in rule files and in signoff reports for comparing configurations

-test_clock signal
    [-mode (shift | capture)]
    Declare the specified signal as test clock for use in DFT analysis. By
    default, the test clock is applied to both modes. Use the optional
    "-mode" switch to limit the mode in which test clock is active to either
    "shift" or "capture".

    NOTE: The most recently issued "-test_clock" command overrides any previous
    "-test_clock" command for the specified signal.

-reset
    ( -sync signal_list -clock clock_list
     |-async signal_list
     |-synchronized signal_list -clock clock_list)
    -polarity (high | low)
    [-virtual]

    Specify design resets as synchronous, asynchronous, or synchronized and set
    the order of priority.
```

```
    . Use "-sync" to specify design resets as synchronous (synchronous assertion
      and deassertion).
    . Use "-async"  to specify design resets as asynchronous (asynchronous
      assertion and deassertion) .
    . Use "-synchronized" to specify design resets as equivalent to the output
      of a reset synchronizer (asynchronous assertion and synchronous
      deassertion).
      NOTE:
      . The "-clock" switch is required when declaring a reset signal as
        synchronous or synchronized, and the clock signal should be declared as
        a clock using "config_rtlds -clock".
      . Use "-polarity" to specify if the reset is active high or active low.
        This switch is mandatory.
    . Use "-virtual" to indicate that the specified reset signals are virtual,
      that is, they do not exist in the current design. This switch creates
      virtual abstractions for each signal in the "-sync", "-async", and
      "-synchronized" arguments.

-reset -set_order {reset_signal+ {reset_signal+}+}
    Specify the priority of reset signals from highest to lowest.
    NOTE: This command does not apply to the CDC app. To specify reset
    ordering in CDC, use "check_cdc -reset -set_order".

-signal ( -constant {{signal_name value}+} [-condition expression] [-force]
         |-static signal_list [-condition expression] [-force]
         |-exclusive signal_list [-condition expression]
         |-gray_code signal_list [-condition expression]
         |-controllable signal_list)
         [-mode (global | functional | shift | capture)+]
         [-override] [-condition expression]
         |-false_path ( -from list
                       |-to list
                       |-through list)+
                       [-regexp]
          |-reset reset_port
            |-inactive_clock clock_port
              (-on_assert |-on_deassert |-both)
            |-constant_data {{data_port_name value}+}
          |-power_control {power_control_port}
          |-inactive_clock {clock_port_list}
           |-inactive_reset {reset_port_list}
          |-constant_bit {{signal (high|low)}+}
```

```
Configures the signal constraints used in the RTL Designer Signoff Flow.

. Use "-constant" to define one or more signals as constant.
  The syntax of the constant value should be consistent with the syntax of
  the elaborated expression mode.
  . Use "-force" to add stopats for driven or design signals.
  . Use "-condition" to define a precondition for the
    exclusive signals configuration check.
. Use "-static" to define one or more signals as static.
  . Use "-force" to add stopats for driven or design signals.
  . Use "-condition" to define a precondition for the
    exclusive signals configuration check.
. Use "-exclusive" to define a list of signals that are mutually toggle
  exclusive.
  . Use "-condition" to define a precondition for the
    exclusive signals configuration check.
. Use "-gray_code" to define a list of signals that are gray coded.
. Use "-controllable" to define a list of signals that are controllable.
. Use "-mode" to define the mode or list of modes for which that constraint
  is active.
  . Use "global" to apply constraints globally. This is the only mode
    accepted by CDC.
  . Use "functional" to apply constraints to the "RTLDS_functional" task.
  . Use "shift" to to apply constraints to the "RTLDS_shift" task.
  . Use "capture" to to apply constraints to the "RTLDS_capture" task.
  NOTE:
  . By default, the constraints are applied globally.
  . In Superlint, you can specify multiple modes, for example,
    {shift capture}.
  . "shift" and "capture" modes are applicable only in Superlint.
  . Using "functional" mode with CDC converts the signal to the "global"
    mode and generates a warning.
. Use  "-override" to overwrite previous configurations.
  . Use "-condition" to define a precondition for the
    exclusive signals configuration check.
. Use "-false_path" to specify paths that should make their
  corresponding CDC pairs considered inactive.
    . Use "-from" and "-to" to specify the path's startpoint and endpoint,
      respectively. The argument of both switches can be a signal, a module,
      an instance, a wildcard expression, or a list of elements of any of
      the previous types.
    . Use "-through" to specify the signals the false path should go through.
      The argument for this switch can be a signal, a wildcard expression,
```

```
          or a list of elements of any of the previous types.
        NOTE:
        . The wildcard expression tries to match any of the allowed elements.
        . To use a regular expression, use "regexp".
        . It is mandatory to specify the "-from", "-to", or "-through" switch.
        . Switches "-from" and "-to" cannot be applied to individual bits or
            ranges, but must be applied to full signals only.

    . Use "-reset" to specify that a group of reset signals are asserted,
      de-asserted, change only when one of the following switches is present:
      .inactive_clock: Use this switch to specify that a list of clock ports
        is inactive.
      .constant_data: Use this switch to specify that a list of data ports
        remain stable at a value.

    . Use "-power_control" to specify which power control port (isolation
      control, retention save, or retention restore) the constraint refers to.
      . Use "-inactive_clock" to specify the list of clock signals constrained
        as stable during the "power_control_port" transitions.
      . Use "-inactive_reset" to specify the list of reset signals constrained
        to de-asserted state during the "power_control_port" transitions.
      . Use "-constant_bit" to specify the signal bits constrained as constant
        during "power_control_port" transitions. The switch must be followed
        by the list of tuples {signal value} (where value can be high or low).

    NOTE:
    . Hierarchy in "signal_name" should be separated by dots.
    . "-gray_code" and "-false_path" are applicable only in CDC.

-signal -check_conflict
    Checks for configuration conflicts in Superlint functional mode and CDC
    functional analysis.

-signal -prove
    Runs a proof on functional signal configuration checks.
    NOTE: Applicable in CDC only.

-signal -clear
    Clears all signal configurations.

-signal (-remove id_tcl_list)
    Remove the specified signal configuration list.
```

```
-mode
  (((-parameter {{parameter_name value}+}
    |-tcl tcl_file
    |-sdc sdc_file
    |-db db_file)
   [-name name] [-description description])
  |-remove mode_name
  |-close)
    Configures and manipulates a multimode analysis.
    . Use "-parameter" to create a mode with a list of pairs (parameter x value)
overwriting parameter(s) with provided value(s) during elaboration.
      NOTE:
      . "elaborate" command launches multimode analysis.
      . Allowed before "elaborate".
    . Use "-tcl" to create a mode in which the tool runs the provided Tcl file before
extract.
      IMPORTANT: This Tcl file must have Jasper's constraints with setup information
specifically for this mode.
      NOTE:
      . "check_cdc -extract" or "check_superlint -extract" launches multimode analysis
      . Allowed before "check_cdc -extract" or "check_superlint -extract"
    . Use "-sdc" to create a mode that parses the provided "sdc_file" using the
"read_sdc" command in the mode before the analysis.
      NOTE:
      . "check_cdc -extract" launches multimode analysis.
      . Allowed before "check_cdc -extract".
    .Use "-db" to create a mode that loads the provided CDC/SUPERLINT
     app database file in the current mode session.
      NOTE:
      . After configuring all modes, use the following commands to launch
        multimode analysis for CDC and Superlint:
       . For CDC: "check_cdc -load"
       . For Superlint: "check_superlint -multimode"
    . Use "-name" to name the mode being created. If not provided, tool automatically
generates a name for the mode.
    . Use "-description" to add an optional description to the mode. It will be shown
in Modes Table and list command.
    . Use "-remove" to remove a previously created mode.
      NOTE: Allowed before launching multimode analysis.
    . Use "-close" to close all mode sessions and take Summary View back to a state
before launching multimode analysis.

-module module_name
```

```
    ( -controllable [-port {port+}]
     |-constant {{port value}+}
     |-reset {{port_name (async | sync | synchronized)}+})
    [-mode (functional | shift | capture)]
    [-override]
    Configures the module constraints used in the RTL Designer Signoff Flow.
    . Use "-controllable" to configure modules as controllable. A controllable
      module is a module in which all outputs are controllable. And use "-port"
      to define a module port as controllable.
    . Use "-constant"  to define module ports as constants.
    . Use "-reset" to specify the ports and their types that act as reset inputs
      to the module.
      This switch is applicable to CDC only.
      NOTE: This action is not equivalent to adding a reset to the tool. Resets
      should be added using 'config_rtlds -reset' command.
    . Use "-mode" to define the mode in which that constraint is active.
      By default, the constraints are applied to the "functional" mode.
    . Use "-override" to add cutpoints for driven or design constant or
      controllable modules and ports.

-memory_cell module_name
    [-input signal_name]
    [-output signal_name]
    [-chip_select signal_name [-chip_select_value constant_value]]
    [-write_enable signal_name [-write_enable_value constant_value]]
    [-write_clock signal_name]

    Configures a memory cell for DFT analysis. This command can be used
    without a design.

    Important things to know about this command:
    . The only mandatory argument is the module name.
    . It does not verify whether module or signals names are actually valid
      because it runs before elaboration.
    . "constant_value" means any constant valid in Verilog or VHDL.

-list signal_config
    Returns a Tcl dictionary that lists the signal configurations.
    This dictionary can be used for scripts and interact with other commands.

    For example, suppose "config_rtlds -list signal_config" returns the
    following:
```

```
    1 {Id 1 Signals {in in2} Type Exclusive Mode Shift Value {} Property
<RTLDS_shift>::1_rtlds_exclusive}
    2 {Id 2 Signals in3 Type Constant Mode Capture Value 0 Property
<RTLDS_capture>::in3_rtlds_const}
    3 {Id 3 Signals in4 Type Constant Mode Capture Value 1 Property
<RTLDS_capture>::in4_rtlds_const}
    4 {Id 4 Signals in5 Type Static Mode Functional Value {} Property
<RTLDS_functional>::in5_rtlds_static}

    This result is a Tcl dictionary for which the keys are the ID of the signal
    configuration names:
    . 1
    . 2
    . 3
    . 4

    For each signal configuration, there is another Tcl dictionary containing
    the information as follows: ID, Signals, Type, Mode, Value, Property.

    To access all the information for a certain signal configuration, pass the
    key (signal_config_id) as index: dict get $DICTIONARY signal_config_id
    For example,
      set dictionary [config_rtlds -list signal_config]
      dict get $dictionary 1
    would return
      Id 1 Signals {in in2} Type Exclusive Mode Shift Value {} Property
<RTLDS_shift>::1_rtlds_exclusive

    To list all keys, use "dict keys $dictionary", which in this example would
    return the following:
      1 2 3 4

    With the signal configuration dictionary, you can access any information
    inside one of them providing a key.
    For example,
      set sig_info [dict get $dictionary 1]
      dict get $sig_info Mode
    would return
      Shift

-list (tag | domain)
    Similar to signal_config above, this command returns a Tcl dictionary of
    tags/domains with their values.
```

```
    The supported keys are: name, enabled, severity, message, parameters.
    Examples:
        (1) dict get [config_rtlds -list tag] INP_NR_ASGN
            Returns a list of values for INP_NR_ASGN as follows:
            name inp_nr_asgn enabled false message {"Primary input port %s of module %s
may be driven inside the
            module"} severity warning parameters {}
        (2) dict get [config_rtlds -list tag] INP_NR_ASGN message
            Returns the value for "message":
            "Primary input port %s of module %s may be driven inside the module"
        (3) dict get [config_rtlds -list tag] BLK_NF_NMCV parameters
            Returns a list of parameters with current values:
            local_regex_style {} pattern _blk % dict get [dict get [config_rtlds -list
tag] blk_nf_nmcv parameters] pattern *_blk
        (4) dict get [dict get [config_rtlds -list tag] BLK_NF_NMCV parameters] pattern
            Returns the value of the parameter "pattern" in  BLK_NF_NMCV:
            *_blk

-list test_clock
    This command returns a Tcl list of the signals currently marked as
    test clocks and their modes (shift, capture, or both).

-list mode
    Returns a Tcl dictionary that lists the multimode's modes configuration.

-list connectivity
    Similar to connectivity below, this command returns a Tcl list of
    connectivity configurations.
    For each configuration, this command specifies the from signals, to signals,
    type of from signals ("-from" or "-from_one_of"), type of to signals ("-to"
    or "-to_one_of"), and the comment of the configuration.

-connectivity ( (-from |-from_one_of) signal_list
                (-to |-to_one_of) signal_list
                 -allow_modules_in_path module_list
                 -logic_type (wire | logic)
                 -type (mandatory | invalid)
                 -destination_type (flop_data | flop_reset | latch_data | latch_en)
               [-comment comment] )
    . Use "-from" to specify the signals of the start point and check that all
      of the signals exist in the path.
    . Use "-from_one_of" to specify the signals of the start point and check
      that at least one of the signals exists in the path.
```

```
    . Use "-to" to specify the signals of the end point and check that all of
      the signals exist in the path.
    . Use "-to_one_of" to specify the signals of the end point and check that
      at least one of the signals exists in the path.
    . Use "-logic_type" to specify the required logic type in the path.
      Supported logic types include "wire" for wires and buffers and "logic" for
      combinational paths.
    . Use "-type" to specify the path type.
      . Use "mandatory" to issue an error if the path does not exist.
      . Use "invalid" to issue an error if the path exists.
    . Use "-allow_modules_in_path" to allow logic in the path inside the
      specified modules.
    . Use "-destination_type" to specify which type of signal is the
      destination of the listed flops in the "-to" list.
    . Use "-comment" to insert a comment.

-no_scan
       [-module module_name [-regexp]]
       [-instance instance_name [-regexp]]
       [-flop flop_name_list]

      Waives all flops for checks related to non-scannability.

    . Use "-module" to waive flops for the entire hierarchy of the specified
      module. And use "-regexp" to specify that the module name is a regular
      expression.
    . Use "-instance" to waive flops for the entire heirarchy of the specified
      instance. And use "-regexp" to specify that the instance name is a
      regular expression.
   . Use "-flop" to waive the specified flop.

-custom_rule tag_name
    (-type (structural | basic_lint)
     -tcl_proc proc_name
     -tcl_file tcl_file_name
    [-callbacks call_back_name_list])

     Use this command to add a new custom rule with the specified name.

   . Use "-type" to define the type of the custom rule. This provides details
     about when the rule should be implemented, that is, during elaboration
     for basic design checking or after elaboration with structural checks.
     . Use "structural" to add a structural custom rule.
```

```
  . Use "basic_lint" to add a basic lint custom rule.
. Use "-tcl_proc" to define the name of the procedure where patterns or
  callbacks defining the custom rule need to be matched.
. Use "-tcl_file" to define the name of the Tcl file where the Tcl procedure
  is written.
. Use "-callbacks" to add callbacks when a new custom rule type is
  "basic_lint". Superlint traverses the RTL and provides some callback points
  on design elements like module, always blocks, statements, and so forth.
  With the "-callbacks" switch, you can provide a list of points where
  you want the Tcl procedure to be invoked.

NOTE: For a complete list of callbacks, see the "Superlint App
User Guide" (Help - Application Guides - Superlint App User Guide).
```

Example

```
config_rtlds -rule -enable -tag { IDN_NF_ALCA }
config_rtlds -rule -enable -category ALL_BASIC_LINT
config_rtlds -rule -severity error -tag IDN_NF_ALCA
config_rtlds -rule -enable -domain all
config_rtlds -rule -exclude -tag ESCNTA -instance a.b
config_rtlds -rule -parameter { lower_bound=0 } -tag IDN_NF_ALCA
config_rtlds -module test -constant {{data1 1'b0}} -mode functional -override
config_rtlds -module test -controllable -port {clk} -mode capture -override
config_rtlds -memory_cell super_mem -input in -output out -chip_select cselect -
write_enable wen -write_clock clk
config_rtlds -memory_cell super_mem -chip_select cselect -chip_select_value 1'b1 -
write_enable wen -write_enable_value 1'b1
config_rtlds -connectivity -from {a b} -to {c d} -type invalid -logic_type logic -
comment "example comment"
config_rtlds -no_scan -module to.* -regexp
config_rtlds -custom_rule CLK_GATE -type structural -tcl_proc proc_clkgate -tcl_file
clkgate.itcl
config_rtlds -custom_rule MOD_NAME_RULE -type basic_lint -tcl_proc proc_modname -
tcl_file custom.tcl -callbacks {rtldsModuleStart}
config_rtlds -port {out bbox_in} -clock clk
config_rtlds -port {out bbox_in} -clock clk2 -add
config_rtlds -port {in} -clock async_to_all
config_rtlds -port {en} -clock sync_to_all
config_rtlds -port  din -reset rstN -module top
config_rtlds -port din2 -reset rstN2
config_rtlds -clock -virtual {virtual_1 virtual_2}

config_rtlds -port {m_out m_out_inv} -input m_clk -module mod
config_rtlds -port {m_out m_out_inv} -input m_clk2 -module mod -add
Vhdl Examples:
config_rtlds -signal -constant {{arb.ig_req \"0100\"}} -mode shift
config_rtlds -signal -constant {{"ing0.ready" '1'}} -mode shift
config_rtlds -signal -constant {{"ing0.wr_rd_reg" 0}} -mode shift
Verilog Examples:
config_rtlds -signal -constant {{"ing0.wr_rd_reg" 1'b1}} -mode shift
config_rtlds -signal -constant {{brdg.int2eg_data[7:0] 8'hFF}} -mode shift
config_rtlds -signal -constant {{"ing0.wr_rd_reg" 'b0}} -mode shift
config_rtlds -signal -constant {{"ing0.wr_rd_reg" 0}} -mode shift
config_rtlds -signal -constant {{A 1'b0} {B 1'b1}}
```

See Also

check_superlint,
check_cdc,
custom_rule_rtlds

configure

Lists or unlimits the current configuration or changes it to default values.

Syntax

```
Setting Configuration
---------------------
configure -default
    | (-unlimit [ all | proof | trace | cmd | synthesize] )

Listing Configuration
---------------------
configure -list [-silent]

configure -list_changed [-silent]
```

Returns

Returns the configurations and their values

Detailed Description

```
Setting Configuration
---------------------
Use this command to manipulate the configuration of the tool.

-default
    Reset the configuration settings to their default values.

    NOTE: This command is not allowed while a background proof is in progress.

-unlimit ...
    Remove the limits using the following settings:
    . cmd_time_limit = 7200s                (cmd)
    . prove_per_property_time_limit = 0s    (proof)
    . prove_time_limit = 0s                 (proof)
    . max_trace_length = 0                  (trace)
```

```
    . task_compile_time_limit = 120s        (synthesize)
    . property_compile_time_limit = 5s      (synthesize)
    The default is "all", which removes limits for the configuration
    variables listed above.

    NOTE: In the case of the configuration variables
    "task_compile_time_limit" and "property_compile_time_limit", the
    "-unlimit" switch resets the limit to the highest supported value,
    which is 10000s.

Listing Configuration
---------------------
Use this command to list the configuration of the tool.

NOTE: Use "-silent" with the list switches to turn off output to the screen
and return information with a Tcl return value. The "-silent" option is helpful
in Tcl scripts.

-list [-silent]
    List the current settings.
    This command returns a list of space-separated pairs of configuration name
    and value.

    The list format is as follows:
    { { <configuration_name> <configuration_value> }+ }

-list_changed [-silent]
    List the settings that the user has changed.
    This command returns a list of space-separated pairs of configuration name
    and value.

    The list format is as follows:
    { { <configuration_name> <configuration_value> }+ }
```

Example

```
configure -list
----------------------------------------
analyze_libnonamehide                          : default
analyze_librescan                              : off
analyze_libunboundsearch                       : off
arch_prll_property_consolidation               : 50
...
----------------------------------------
{analyze_libnonamehide default} {analyze_librescan off} {analyze_libunboundsearch off}
{arch_prll_property_consolidation 50}...

configure -unlimit cmd
configure -list_changed
```

connect

Instantiates and connects a module/entity to the main elaborated design's top.

Syntax

```
connect unit_name instance_name [-auto]
        (-connect formal_name actual_name)*
        [-main_design |-repository repository_id]

"-bind" Switches
----------------
connect -bind unit_name instance_name [instance_to_bind_to]
        (-parameter param_name param_value)*
        [-auto] (-connect formal_name actual_name)*
        [-elaborate] [-vhdl]

Utility Switches
----------------
connect -elaborate [-vhdl]
connect -list [-silent] [instance_name]
connect -remove instance_name
connect -clear [-repository repository_id]
```

Returns

Returns the specified instance name.

Detailed Description

Use this command to do the following:

a) Instantiate and connect a module or entity containing a set of
   requirements from "analyze -req".
b) Instantiate and connect a module or entity elaborated in the
   main design.
c) Instantiate and connect a module or entity elaborated with
   "elaborate -repository repository_id".

```
d) Instantiate and bind a module or entity to the top module
   or an instance inside the top module.
e) Instantiate and connect a Proof Accelerator or Formal
   Scoreboard to the design.

NOTE: This command clears any previous proof and Visualize results.

unit_name instance_name [-auto] [-main_design |-repository repository_id]
        (-connect formal_name actual_name)*
    Instantiate and connect the specified module or entity to the top module.

    . "unit_name" is the name of an elaborated module or entity.

    . "instance_name" is the unique instance name you want to use to
      instantiate an elaborated module or entity.

    . By default, the command searches for "unit_name" in the design compiled
      with "analyze -req" and "elaborate -req".

    . Use "-main_design" to search for "unit_name" in the main design.

    . Use "-repository" with a repository ID to search for "unit_name" in the
      specified repository.

    . Use "-auto" to automatically connect the input port of the specification
      module to the design signal with the same name.

      NOTE: The tool does not autoconnect signals of different widths.

    . Use "-connect" to connect the specified port (formal_name) from the
      specification module to a specified signal or port in the design
      (actual_name).

      NOTE: If you are connecting to a signal or port of an instance, you
      cannot specify "actual_name" signals that are in the top module.

"bind" Switches
---------------
Implementation guidance for "-bind":
. This command supports connections to constants, multibits, and
  expressions.
. When connecting to an expression, use the full hierarchical path
  for signals. (The command does not support relative paths.)
```

```
. You must specify the "-auto" switch to automatically connect the
  input port of the specified module to the design signal with the
  same name.
. It is not possible to connect a Verilog and a VHDL module in the
  same "connect -elaborate" command. You must use two separate
  commands. Refer to example section below.
. Do not use "analyze -req" in combination with "connect -bind". Refer to
  example section below.

-bind unit_name instance_name [instance_to_bind_to]
        (-parameter param_name param_value)*
        [-auto] (-connect formal_name actual_name)*
        [-elaborate] [-vhdl]
    Instantiate and bind the specified module or entity to the top module or an
    instance inside the top module.

    . "unit_name" is the name of an elaborated module or entity.
    . "instance_name" is the unique instance name you want to use to
      instantiate an elaborated module or entity.
    . "instance_to_bind_to" is the unique instance name for the instance you
      want to bind the new instance to.

    . Use "-parameter" to specify the parameter values for the instance that is
      being created.

    . Use "-auto" to automatically connect the input port of the specification
      module to the design signal with the same name.

      NOTE: The tool does not autoconnect signals of different widths.

    . Use "-connect" to connect the specified port (formal_name) from the
      specification module to a specified signal or port in the design
      (actual_name).

      NOTE: If you are connecting to a signal or port of an instance, you
      cannot specify "actual_name" signals that are in the top module.

    . Use "-elaborate" to execute the "-bind" specification you entered
      previously and use "-vhdl" to bind a VHDL module.

      After a successful elaboration, you can end the separate frontend
      process (jg_frontend ) to free the associated memory with the
      "analyze -clear" command; however, if you end the "jg_frontend"
```

```
      process beforehand, the tool skips subsequent "connect -elaborate"
      commands and prints the WNL102 warning.

-elaborate [-vhdl]
    Execute the "-bind" specification you entered previously.

    Use "-vhdl" to bind a VHDL module.

    After a successful elaboration, you can end the separate frontend process
    (jg_frontend ) to free the associated memory with the "analyze -clear"
    command; however, if you end the "jg_frontend" process beforehand,
    the tool skips subsequent "connect -elaborate" commands and prints the
    WNL102 warning.

Utility Switches
----------------
-list [-silent] [instance_name]
    List information for the instances that are connected
    with the "connect" command.

    . Use "-silent" to show on the screen only the list of instances connected
      and return them as Tcl return values. This option is helpful in Tcl
      scripts.

    . Supply an instance name if you want to list connection information for
      a specified "instance_name".

    NOTE: This switch does not list instances connected with "-bind".

-remove instance_name
    Remove a previously created connect instance.

    NOTE: This switch does not remove the ones connected with "-bind".

-clear [-repository repository_id]
    Remove all connect instances.

    Use "-repository" with a repository ID to remove the connect instance
    derived from the specified repository.

    NOTE: The "-clear" switch also removes instances connected with "-bind",
    but you  must run "connect -elaborate" to complete the clear process.
    NOTE: Removing all connect instances is blocked by the apps: SEC, SPV,
```

XPROP, and FSV.

Example

```
// Connect instance derived from Requirements (analyze/elaborate -req)
connect pci_Requirement Requirement_inst \
  -connect clk   PCI_CLK_IN \
  -connect gntN  PCI_GNTn_IN \
  -connect idsel PCI_IDSEL_IN \
  -connect mAd   PCI_AD_OUT \
  -connect sAd   PCI_AD_IN \
  -connect mAd_enable 1'b1

// Connect instance deriving from repository (analyze/elaborate -repository)
connect -repository 0 \
  pci_Requirement Requirement_inst \
  -connect clk   PCI_CLK_IN \
  -connect gntN  PCI_GNTn_IN \
  -connect idsel PCI_IDSEL_IN \
  -connect mAd   PCI_AD_OUT \
  -connect sAd   PCI_AD_IN \
  -connect mAd_enable 1'b1


connect -remove Requirement_inst

// Proper command flow for "connect -bind" (Do not use "analyze -req".)
analyze -sv top.sv
analyze -sv req.sv
elaborate
clock clk
reset rst

connect -bind req req1 -parameter width 1 -connect clk clk -connect rst rst -connect in
in1 -connect out out1 -elaborate
connect -bind req req2 -parameter width 2 -connect clk clk -connect rst rst -connect in
in2 -connect out out2 -elaborate
connect -bind req req3 -parameter width 3 -connect clk clk -connect rst rst -connect in
in3 -connect out out3 -elaborate

// Binding a Formal Scoreboard to the DUV
connect -bind jasper_scoreboard_2 my_pa -connect {clk1} {clk} \
```

```
  -parameter {LOG_PKT_LATENCY} {5} -parameter {PKT_LATENCY} {7} -elaborate


// Connecting a Verilog and a VHDL module in two separate
// commands
analyze -verilog top.v
analyze -verilog vlogmod.v
analyze -vhdl vhdmod.vhdl
elaborate -top top
connect -bind vlogmod mod1 -auto -elaborate
connect -bind vhdlmod mod2 -auto -elaborate -vhdl
```

See Also

analyze -req

cover

Adds a cover directive. Also removes, modifies, lists, etc.

Associated App(s)

Depending on the property to be created.

Syntax

```
cover [-task task_name |-name name |-name task_name::name]
      expression
      [-label label] [-annotation '{'annotation'}']

cover -replace
       -name name
      |-name task_name::name
      expression
      [-label label] [-annotation '{'annotation'}']

cover -path
      -from input_signal_tcl_list
      [-to barrier_signal_tcl_list]
      [-task task_name |-name name
      |-name task_name::name] [-annotation '{'annotation'}']

cover -generate -signal signal
      [ -task task_name
       |-name name
       |-name task_name::name]
      (-range range |-values values)
      [-after after_expr]
      [-condition cond_expr]
      [-count (cover_count | cover_count_range)]

cover -generate -fsm fsm_signal
      [ -task task_name
       |-name name
       |-name task_name::name]
```

```
      ( -transitions
       |-states [-count (cover_count | cover_count_range])+
      [ -use_enum]

cover -generate -expression expression
      [  -task task_name
       | -name name
       | -name task_name::name]
      [-after after_expression]
      [-count (cover_count | cover_count_range) ]

cover -generate -auto
      [-num N]
      [-seed N]
      [-property property_name_tcl_list]
      [-instance instance_name_tcl_list]
      [-task task_name]
      [-type {cover_type+}]
      [-pattern {pattern+} [-regexp]]
      [-signal {signal_name+}]
      [-list {cover_info+}]
      [-exclude_bind_hierarchies]

cover -k_liveness -from target_liveness_property -k N
       [-fairness list_fairness_assumptions]
       [-max_k N]
       [-step_k N]
       [-name property_name]
       [-type (witness)]

cover -list [-silent] [-type (path | spv)]
      [-task task_name]

cover -clear
      [-task task_name]

cover -show name

cover -remove name+ [-regexp]

cover -disable name+ [-regexp [-case_insensitive]]

cover -enable name+ [-regexp [-case_insensitive]]
```

```
cover -clear_store_trace name+ [-regexp]

cover -set_store_trace [value] name+ [-regexp]

cover -set_trace_extension N [-posedge signal_tcl_list] name+ [-regexp]

cover -clear_trace_extension name+ [-regexp]

cover -clear_intermediate name+

cover -set_intermediate name+

cover -rename old_name new_name

cover -set_target_bound N {cover_name_list} [-regexp]

cover -clear_target_bound {cover_name_list} [-regexp]

cover -extend
      -all |(-property {property_name_list} [-regexp])
      [-task task_name]
      (-precondition precondition_expression
       |-postcondition postCondition_expression)+
      [-disable_original] [-silent]
```

Returns

Returns a string (identifier name) for the cover directive.

Detailed Description

Use this command to add a cover directive. By default, this command applies to the current task and does not automatically update the database. Use "-task" to specify a different task.

NOTE:
. You can define a clocking and reset statement in your property.

  Example:

```
  cover { @(posedge clk) disable iff (!rstn) (a) }

. If you declare a Tcl property without an explicit clock (for example,
  cover {something}), the clocking of this property will be the fastest
  sampler possible, which depends on the clock setup.
  . If clock setup is clock clk, the property is clocked at @(posedge clk)
  . If clock setup is clock clk -both_edges, the property is clocked at @(clk)

  If you do not specify clocking, the property behaves as an immediate property.
  In Visualize, it will be sampled at all times and it can evaluate to true or
  false at the beginning of a cycle, reflecting input value changes immediately
  as they happen in the same cycle.


[-task task_name |-name name |-name task_name::name]
       expression
       [-label label] [-annotation {annotation}]
    Create a cover directive for the specified Boolean expression that
    evaluates to either true or false.

    Use the "prove" command to verify this cover.

    NOTE: Jasper Apps supports SVA expressions. Refer to your command
    reference manual for limitations and clarification.

    . Use "-task" to apply the command to the specified task instead of the
      current task.

    . Use "-name" to assign a name to the property.

    . Use "-label" to add an element heading. The GUIs will display this
      heading instead of the property name.

    . Use "-annotation" to annotate the specified property with the description
      enclosed by the curly braces. You can view annotations in the Visualize
      window waveforms pane by positioning the mouse to hover over the property
      and reveal a tooltip. And you can print annotations to the screen with
      the "get_annotation" command.

    NOTE:
    . Do not use * or ? in property names.
    . Do not use the "-task" and "-name" switches in the same command.
      If you need to specify both a "name" and "task_name", use the "-name"
```

```
      switch and prepend the property name with "<task>::". (Replace <task>
      with an actual task name.) Example:
    cover -name taskA::A {~rdy => ~trx}

-replace (-name name) |(-name task_name::name)
         expression
         [-label label] [-annotation {annotation}]
    Replace the expression for the specified cover.

    Use this command to edit a cover used as a Visualize target without
    triggering a clear-all. Replot the trace after you use this command.

    . Use "-name" to specify the cover you are editing.

    . Use "-label" to add or change the property's element heading.

    . Use "-annotation" to add or change the property's annotation.

    NOTE: This command does not apply to embedded covers.

-path -from input_signal_tcl_list -to barrier_signal_tcl_list
     [-task task_name |-name name |-name task_name::name]
     [-annotation {annotation}]
    Generate a cover property that checks whether the value of a signal
    can be propagated to another signal, that is, whether one signal can
    control another signal.

    . Use "-from" to select the originating signals for the path.

    . Use "-to" to select the end points for the path.

    . If you do not use "-to", the tool uses the list of output signals.

    . Use "-task" to assign the property to a task other than the current task.

    . Use "-name" to assign a name to the property.

    . Do not use * or ? in property names.

    . Do not use both "-task" and "-name". If you need to specify both, use the
      "-name" switch and prepend the property name with "<task>::".
      (Replace <task> with an actual task name.)
```

```
    . Use "-annotation" to annotate the specified property with the description
      enclosed by the curly braces. You can view annotations in the Visualize
      window waveforms pane by positioning the mouse to hover over the property
      and reveal a tooltip. And you can print annotations to the screen with
      the "get_annotation" command.

    Important things to know about this command:
    . You can use ProofGrid to prove multiple "-path" properties in parallel or
      sequentially. To enforce serial processing, put properties in different
      tasks.
    . Path properties are not supported with tunneling. The tool ignores these
      properties when verifying tasks that contain them.
    . The "-from" and "-to" switches support Tcl lists and lists of signals
      that can be formed using wildcards, for example:
      cover -path -from io_dram?_data_valid -to dram_io_cas0_l
      cover -path -from io_dram*_data_valid -to dram_io_cas0_l
      If you use lists, the tool generates one "Cover(path)" property. You can
      interpret its proof results as follows:
      . Covered: There is a path from ONE of the "-from" signals to ONE of the
        "-to" signals.
      . Unreachable: There is no path from ANY of the "-from" signals to ANY of
        the "-to" signals.
    . In versions prior to 2015.09, signals specified with "-from" were
      restricted to stopats and primary inputs. This limitation no longer
      exists; however, the expanded support is an initial release to gather
      feedback from early adopters and finalize implementation for an upcoming
      release. Contact support@cadence.com to provide feedback or comments
      targeting the upcoming production version.

-generate -signal signal
          [-task task_name |-name name_prefix |-name -task_name::name_prefix]
          (-range range) |(-values values)
          [-after after_expr]
          [-condition cond_expr]
          [-count (cover_count | cover_count_range)]
    For each of the values specified by "-range" or "-values", create a cover
    directive where the expression tests whether "signal" reaches the value,
    given the auxiliary conditions provided by "-after" and "-condition".

    . Use "-task" to apply the command to the specified task instead of the
      current task.

    . Use "-name" to assign a name prefix to each of the generated properties.
```

. Use "-range" to specify values in a range using the format "M:N" or "M:s:N", where "M" is the lower bound of the range, "N" the upper bound, and "s" is the step between consecutive values.

NOTE: The command also accepts the "N:M" and "N:-s:M" formats.

. Use "-values" to manually specify the list of values.

. Use "-after" to specify an expression that has to be satisfied before "signal" reaches the value in each of the covers generated.

. Use "-condition" to specify an expression that must be satisfied by each of the covers generated.

. Use "-count" to specify how many times the condition must be met for the cover to be reached. If you specify a range, the tool creates one cover directive for each of the values in the range.

NOTE: The generated covers will be a combination of the values specified by the "-values" or "-range" switch with the values specified by "-count".

```
-generate -fsm fsm_signal
          [-task task_name |-name name |-name -task_name::name]
          ( -transitions
           |-states [-count (cover_count | cover_count_range)])+
          [-use_enum]
```

Creates cover directives for each of the states or transitions of the finite state machine (FSM) represented by the specified FSM signal.

. Use "-task" to apply the command to the specified task instead of the current task.

. Use "-name" to assign a name prefix to each of the generated properties.

. Use "-transitions" to create covers for each of the state transitions in the FSM.

. Use "-states" to create covers for each of the states in the FSM and use "-count" with "-states" to specify expressions where the state reachability expressions must be met a certain number of times for the cover to be reached. If you specify a range, the tool creates one cover

```
      directive for each of the values in the range.

    . Use "-use_enum" to generate cover expressions with enums, when
      available, instead of the default naming, which uses binary numbers.

-generate -expression expression
          [-task task_name |-name name |-name task_name::name]
          [-after after_expression]
          [-count (cover_count | cover_count_range)]
    Create cover directives using "expression" combined with the expressions
    generated by the "-after" and "-count" switches.

    NOTE: The number of covers generated will be the number of elements in the
    range passed to the "-count" switch

    . Use "-task" to apply the command to the specified task instead of the
      current task.

    . Use "-name" to assign a name prefix to each of the generated properties.

    . Use "-after" to specify an expression that must be satisfied before the
      specified signal reaches the value in each of the covers generated.

    . Use "-count" to specify how many times "expression" must be evaluated for
      the cover to be reached. If you specify a range, the tool creates one
      cover directive for each of the values in the range.

-generate -auto
          [-num N]
          [-seed N]
          [-property property_name_tcl_list]
          [-instance instance_name_tcl_list]
          [-task task_name]
          [-type {cover_type+}]
          [-pattern {pattern+} [-regexp]]
          [-signal {signal_name+}]
          [-list {cover_info+}]
          [-exclude_bind_hierarchies]
    Use this command to randomly create cover directives automatically based
    on design structure. The Property Table lists the covers, and the tool
    includes them in any proof you run with the "-task" or "-with_helper"
    switch.
```

The intent of automatically generated covers is to boost the performance of engine L.

. Use "-num" to specify the number of covers to be automatically generated. If you do not specify this switch, the tool automatically computes the number of covers based on the number of assertions in the target task.

. Use "-seed" to specify a natural number to be used as a seed when generating covers.

  NOTE:
  . If you do not specify this switch, or if you specify a seed equal to zero, the tool automatically generates a random seed and the seed is printed in an info message so you can use it to reproduce a cover generation run.
  . When using a seed greater than zero, the list of generated covers always contains the same set of covers for identical seeds. Therefore, it might be that the command generates fewer covers than requested by "-num", which can happen if any of the covers already exists in the target task. However, when using no seed or a seed equal to zero, this command generates the exact number of requested covers.

. Use "-property" to generate covers for signals in the COI of the specified properties.

. Use "-instance" to generate covers for signals in the instances specified or down in the hierarchy.

. Use "-task" to generate covers in the specified task instead of the current task. Covers are generated for signals in the COI of the specified task.

. Use "-type" to specify the type of covers you want the tool to automatically generate. Types include the following:
  . fsm: Generate covers for FSM signals.
  . pattern: Generate covers for register signals matching pre-defined string patterns. Examples of pre-defined patterns are: full, empty, valid, hit, and so forth.
  . enable: Generate covers for critical flop and clock enable signals.

  If you do not specify types, the tool generates covers of all fsm and pattern types.

```
    . Use "-pattern" to customize pattern list used when generating "pattern"
      type covers.

      . Use "-regexp" to specify that you are using a regular expression to
        specify patterns.

      Important things to know:
      . Use this switch to specify one or more patterns.
      . Wildcards and regular expressions are supported.

    . Use "-signal" to specify the list of signals to generate covers for, in
      case the signal is of one of the specified types.
      . When "-signal" is used, it does not mean covers will be generated for
        specified signals. Specified signals must belong to one of the specified
        types.
      . "-signal" switch supports Tcl lists and lists of signals that can be
        formed using wildcards, for example:
        cover -generate -auto -signal {full* ready*}

    . Use "-list" to list specific cover information without generating any
      covers. In the current version of the tool, cover information includes
      just one argument:
      . signal:  List signals detected by the generator. Signals will be
                 related to all types specified. If you do not specify "-type"
                 the tool lists FSM and pattern.

    . Use "-exclude_bind_hierarchies" to avoid generating covers for signals in
      bind instances.

-k_liveness -from target_liveness_property -k N
       [-fairness list_fairness_assumptions]
       [-max_k N]
       [-step_k N]
       [-name property_name]
       [-type (witness)]
    The k-Liveness cover shows scenarios where the fairness conditions have
    been satisfied k times while the liveness obligation is pending. If the
    k-Liveness cover is unreachable, then the original liveness property is
    provable.

    See more detail in "assert -k-liveness" TclHelp.
```

```
    The tool generates N properties, starting from k value and increasing by
    step_k value until max_k value is reached.

    . Use "-from" to specify the target liveness property.
    . Use "-fairness" to list the fairness assumptions that will be considered
      in the safety k-Liveness assertion. If not specified, the tool considers
      all fairness assumptions.
    . Use "-max_k" to specify the max_k value.
    . Use "-step_k" to specify the step_k value.
    . Use "-name" to specify a prefix of the property name.
    . Use "-type k_witness" to create a witness cover property, which generates
      a trace showing a scenario that the k-cover is reachable and
      the liveness obligation is reached after k times.
    . Use "-type k_fairness" to create a fairness cover property, which
      generates a trace showing a scenario where the k-cover precondition is
      reached after k times.

-list [-silent] [-type (path | spv)]
    [-task task_name]
    Show the list of cover directives in the current or specified task.

    . Use "-silent" to return the list as Tcl return values without
      printing the list on the screen.

    . Use "-type path" to return only the list of covers generated
      by path sensitizing analysis.
    . Use "-type spv" to return only the list of covers generated by the SPV
      App.

-clear
    [-task task_name]
    Remove all cover directives specified with the "cover" command in the
    current or specified task.

-show name
    Show the actual expression for the cover directive with the
    specified name.

-remove name+ [-regexp]
    Remove the cover directive with the specified name.

    Important things to know:
    . Use this switch to specify one or more covers.
```

```
    . Wildcards and regular expressions are supported; for example:
      cover -remove {^.*$} -regexp
    . You can only remove properties created with the GUI or on the command
      line. That is, you cannot remove properties added automatically (for
      example, SVA and PSL embedded properties). However, you can disable
      and enable these automatically added properties.

-disable name+ [-regexp [-case_insensitive]]
    Keep the cover directive in the current task but do not
    include it in any analysis.

    Use "-regexp" to specify that you are using a regular expression to specify
    properties. And use "-case_insensitive" to specify that the expression is
    not case sensitive.

    Important things to know:
    . Use this switch to specify one or more covers.
    . Wildcards and regular expressions are supported; for example:
      cover -disable {^.*$} -regexp
    . When you disable a property, the tool discards associated traces and
      issues an info message.

-enable name+ [-regexp [-case_insensitive]]
    Re-enable the specified (previously disabled) cover directive.

    Use "-regexp" to specify that you are using a regular expression to specify
    properties. And use "-case_insensitive" to specify that the expression is
    not case sensitive.

    Important things to know:
    . Use this switch to specify one or more covers.
    . Wildcards and regular expressions are supported; for example:
      cover -enable {^.*$} -regexp

-clear_store_trace name+ [-regexp]
    Do not generate and store traces for the specified cover directives during
    proofs.

    Important things to know:
    . The tool establishes property status and bounds as usual.
    . Use this switch to specify one or more covers.
    . Wildcards and regular expressions are supported, for example,
      cover -clear_store_trace {^.*$} -regexp
```

```
-set_store_trace [value] name+ [-regexp]
    Generate and store traces for the specified properties during proofs,
    according to the specified value.

    Possible values are "0", "1", or "unlimited":
    . 0:         The tool does not generate and store traces (unless shared with and
                 accounted for by another property). Specifying the value "0" is
                 equivalent to specifying the switch "-clear_store_trace".
    . 1:         The tool generates and stores a single trace for each
                 property.
    . unlimited: The tool generates and stores an unlimited number of traces.

    Important things to know about specifying a value:
    . If you do not specify the value argument, the default is "1".
    . If you need to specify a property named "0", "1", or "unlimited", you
      must explicitly specify the first argument "0", "1", or "unlimited" and
      then specify the property name.

    NOTE:
    . The commands "prove -no_traces" and "set_prove_no_traces" take
      precedence over "cover -set_store_trace" values "1" and "unlimited".
    . Use this switch to specify one or more covers.
    . When using "hunt -run" command with "state_swarm" mode, switch
      "-no_cover_traces" is enabled by default and takes precedence over
      "cover -set_store_trace" values "1" and "unlimited". See also help for
      "hunt" command.
    . Wildcards and regular expressions are supported, for example,
      cover -set_store_trace unlimited {^.*$} -regexp

-set_trace_extension N [-posedge signal_tcl_list] name+ [-regexp]
    Extend a trace with N extra posedge events on the specified signals--or
    on the tool's fastest clock if no signals are specified--beyond the cycle
    that triggers the property. "$" is a special value that means infinite.
    This switch overrides the global setting (see "set_trace_extension").

    Important things to know:
    . "-set_trace_extension 0" makes the effective value for this property 0
      regardless of the global setting; therefore, it is not the same as
      "-clear_trace_extension".
    . "$" is not supported when using "-posedge".
    . Wildcards and regular expressions are supported.
```

```
-clear_trace_extension name+ [-regexp]
    Remove property-specific trace extensions, allowing the global setting to
    apply (see "set_trace_extension").

    Wildcards and regular expressions are supported.

-clear_intermediate name+
    Do not use the specified cover directives as candidates for intermediate
    targets for engine L.

    Important things to know:
    . Use this switch to specify one or more covers.
    . This switch supports wildcards (* and ?).

-set_intermediate name+
    Allow engine L to use the specified cover directives as candidates for
    intermediate targets.

    Important things to know:
    . Use this switch to specify one or more covers.
    . This switch supports wildcards (* and ?).

-rename old_name new_name
    Rename a cover directive.

-set_target_bound N {cover_name_list} [-regexp]
    Set the target bound for one or more specified properties.

    IMPORTANT: The status of properties reaching the target bound will change
    to "Bounded Proven (User)" or "Bounded Unreachable (User)"; however, this
    command does not affect proofs in any way. That is, it only changes how the
    results are reported.

    NOTE:
    . When the min_length of a cover is equal or greater than the
      target bound, its status changes to "Bounded Unreachable (User)".
      This indicates the cover can still have a trace of the exact length
      of the target bound, but not shorter.
    . If you want proofs to stop when this target bound is reached, the
      "set_prove_stop_on_target_bound" configuration must be "true". Otherwise,
      Jasper continues running proofs after reaching the target bound.
    . Liveness properties are not supported and ignored by this command.
    . Wildcards and regular expressions are supported.
```

```
      Example: cover -set_target_bound N task::*
    . Value 0 for N means an infinite target bound, which is equivalent to
      using "cover -clear_target_bound".
    . The target bound set with this command takes precedence over any bound
      specified with the "set_prove_target_bound" command.
    . If you restore a jasper_database using the "restore -jdb" command, the
      status of an "Undetermined" cover does not automatically change to
      "Bounded Unreachable (User)" according to the current target bound. You
      must specify the target bound again after restoring the database.

-clear_target_bound {cover_name_list} [-regexp]
    Clear a previously set target bound for one or more specified properties.

    NOTE: Wildcards and regular expressions are supported.
    Example: cover -clear_target_bound N task::*

-extend
    -all |(-property {property_name_list} [-regexp])
    [-task task_name]
    ( -precondition precondition_expression
     |-postcondition postcondition_expression)+
    [-disable_original] [-silent]
    Generate additional cover properties using pre-existing covers in the
    current or specified task.

    . Use "-all" to extend all covers of the current or specified task.

    . Use "-property" to extend only covers specified by {property_name_list}.
      NOTE: This command supports wildcards and regular expressions. The
      default is wildcards. Use the "-regexp" switch to use regular
      expressions instead.

    . Use "-task" to use and generate covers in the specified task.

    . Use "-precondition" to specify an expression that is added as a
      precondition to one or more existing covers.

    . Use "-postcondition" to specify an expression that is added as a
      postcondition to one or more existing covers.

    . By default, the tool keeps original covers when it creates extended
      covers. Use "-disable_original" to disable the original covers.
```

```
    . Use "-silent" to return the new extended cover list as Tcl return values
      without printing the list on the screen.

    NOTE: If the cover has property operators, the tool automatically tries to
    convert to sequence operators. The conversion is supported for the
    following operators:
    . #-# is converted to ##0
    . #=# is converted to ##1
    . s_eventually is converted to ##[0:$]
```

Example

```
cover a==b

cover -name illegal_size {wr_size != 2'b11}

cover {scan_enable == 1'b0}

cover -name illegal_size {WR_SIZE /= "11"}

cover {SCAN_ENABLE = '0'}

# Use an expander and "get_property_list" to run the "cover" command for
# a list of properties:
cover -enable {*}[get_property_list -include {type cover}]

NOTE: Jasper Apps supports the implication overlapping operator
"->" on the command line to declare properties. This
operator indicates that the expressions on both sides happen at
the same iteration.

cover {~rdy -> ~trx}

cover {(rdy = '1') -> (trx = '1')}

# Pattern-matching with regular expressions:
# ----------------------------------------
# Disable cover properties that match zero or more characters followed by either
# "_ended" or "_started"
cover -disable {.*(_ended|_started)} -regexp

# Remove cover properties that match "trans" followed by 1 or more numbers and
```

```
# followed by zero or more characters
cover -remove {trans([0-9]+).*} -regexp

cover -generate -name cover_sig_value_ -signal sig -range 50:10:70 -after start -
condition !bypass
cover -generate -name cover_sig_value_ -signal sig -values {50 60 70} -after start -
condition !bypass

# Are both equivalent to the following set of covers:
# cover -name cover_sig_value_50 {start ##[1:$] (sig == 50 && !bypass)}
# cover -name cover_sig_value_60 {start ##[1:$] (sig == 60 && !bypass)}
# cover -name cover_sig_value_70 {start ##[1:$] (sig == 70 && !bypass)}

# If you use the "-count" switch:
# cover -generate -name cover_sig_value_ -signal sig -values {1 2} -count 2
#
# The tool generates the following set of covers:
# cover {sig == 1 [=2]}
# cover {sig == 2 [=2]}
#
# If you specify a range for "-count", you get:
# cover -generate -signal sig -values {1 2} count 1:2
#
# cover {sig == 1 [=1]}
# cover {sig == 1 [=2]}
# cover {sig == 2 [=1]}
# cover {sig == 2 [=2]}

# Consider an FSM signal with allowed states "IDLE" and "BUSY":
cover -generate -name cover_fsm_ -fsm state -transitions -states -count 3 -use_enum

# Is equivalent to the following set of covers:
# cover -name cover_fsm_IDLE {state == IDLE [=3]}
# cover -name cover_fsm_BUSY {state == BUSY [=3]}
# cover -name cover_fsm_IDLE_BUSY {state == IDLE ##1 state == BUSY [=3]}
# cover -name cover_fsm_BUSY_IDLE {state == BUSY ##1 state == IDLE [=3]}
```

See Also

assert,
assume,
set_prove_target_bound,
set_prove_stop_on_target_bound

custom_engine

Manages custom engines in the current session.

Syntax

```
custom_engine -add -code code_string

custom_engine -list

custom_engine -show engine_list
```

Returns

Multiple return values.

Detailed Description

Proof orchestration dynamically adjusts proof settings during proofs to improve performance. And it explores custom engines, which are engines created automatically with non-default engine settings. The "custom_engine" command provides a way to inspect and add custom engines and use them with "set_engine_mode" and "prove".

Custom engines incorporate complex engine settings such as changing the internal solver (see "set_engine_solver"). We usually do not recommend using the "custom_engine" command directly since custom engines are automatically created during ProofMaster Proof Orchestration and reused via ProofMaster PPD files. However, this can occasionally be useful when you want to exactly replicate a recent orchestration proof for debugging purposes. The codes for any custom engines are displayed in the log by orchestration.

-add -code code_string
    Add custom engine using the given code_string.

    You can get code_string with the "-show" switch or by searching the log file for "custom engine code" messages.

    NOTE:
    . If a custom engine with the given code already exists, the tool returns its name. Otherwise, it returns a unique name (per current session) for the added engine.
    . The code_string is portable, and you can use it to redefine custom engines across runs.

-list
    List all custom engines currently defined.

-show engine_list
    Return a Tcl dictionary with information about the given engines, code strings included.
    If the list contains undefined engines or regular engines, the tool issues an error.

Example

```
custom_engine -add -code hT3NYLhPPfiYWAdCgXmBG+rvbLaeNeNeSC/1ZzEBAA
custom_engine -list
custom_engine -show AMcustom1
custom_engine -show [ custom_engine -list ]
```

See Also

prove,
set_engine_mode,
set_engine_solver

custom_gui_action

Add, remove, or list custom button(s) to/from GUI windows.

Syntax

```
custom_gui_action -add button_name
            -window (  visualize | console | superlint | afv | cdc
                     | conn | cov | csr | fpv | fsv | lpv | sec
                     | spv | unr | xprop)
            -script tcl_script
            [-tooltip tooltip_string]
            [-iconfile path_of_icon_file]
            [-force]

custom_gui_action -remove button_name
            -window (  visualize | console | superlint | afv | cdc
                     | conn | cov | csr | fpv | fsv | lpv | sec
                     | spv | unr | xprop)

custom_gui_action -list
            -window (  visualize | console | superlint | afv | cdc
                     | conn | cov | csr | fpv | fsv | lpv | sec
                     | spv | unr | xprop)
```

Returns

No return values.

Detailed Description

Use this command to add, remove, or list custom button(s) from the main Jasper App views or Visualize windows.

```
-add button_name
    -window (  visualize | console | superlint | afv | cdc | conn | cov
             | csr | fpv | fsv | lpv | sec | spv | unr
             | xprop)
```

```
    -script tcl_script
    [-tooltip tooltip_string]
    [-iconfile path_of_icon_file]
    [-force]

    Add a custom button to the specified window or app view.
    . Use "-window" to specify the window or app view to add this button to.
      . visualize - Jasper Visualize window
      . console   - All Jasper App views
      . superlint - Superlint App
      . afv       - Architectural Formal Verification App
      . cdc       - CDC Verification App
      . conn      - Connectivity Verification App
      . cov       - Coverage App
      . csr       - CSR Verification App
      . fpv       - Formal Property Verification App
      . lpv       - Low Power Verification App
      . sec       - Sequential Equivalence Checking App
      . spv       - Security Path Verification App
      . unr       - UNR Analysis App
      . xprop     - X-Propagation Verification App
    . Use "-script" to specify the Tcl script you want the tool to run when you
      click the button. The argument to "-script" can contain a macro "%n", which
      will be translated to current app view or window where the button is added
      before the tool runs the script.
    . Use "-tooltip" to specify the tooltip you want the tool to display on
      mouse hover over the button.
    . Use "-iconfile" to specify the PNG image you want to use for the button
      icon.
      NOTE: For optimal rendering, use images with 16x16 pixels.
    . Use "-force" to force the addition of a button even if there exists a button
      with the same name. Using this option removes the existing button
      and adds the new button at the position of the existing button.

-remove button_name
    -window (  visualize | console | superlint | afv | cdc | conn | cov
             | csr | fpv | fsv | lpv | sec | spv | unr
             | xprop)
    Remove the specified custom button from the specified window or app view.

    Use "-window" to specify the window or app view you want to remove this
    button from.
```

```
-list
    -window (  visualize | console | superlint | afv | cdc | conn | cov
             | csr | fpv | fsv | lpv | sec | spv | unr
             | xprop)

    Return a list of button names for the specified window or app view.

    Use "-window" to specify the window or app view that you want a list for.
```

Example

```
custom_gui_action -add VisualizeCover -window fpv -script {visualize -cover -property
$propertyName -bg}
custom_gui_action -add CopyCurrentWindow -window visualize -script {visualize -
new_window -copy} -iconfile ~/copy.png
custom_gui_action -remove VisualizeCover -window fpv
custom_gui_action -list -window fpv
```

custom_rule_rtlds

Creates different structural and basic lint custom rules.

Syntax

```
custom_rule_rtlds -element (flop | comb | latch | and | or | clock_gate | multi_driven
| wire)
                  [-data signal_name]
                  [-clock signal_name]
                  [-reset signal_name]
                  [-enable signal_name]
                  [-output signal_name]

custom_rule_rtlds -complete_signal signal_name
                   -bit bit_name
                   -connection_type (only_one | more_than_one | all | none | any)

custom_rule_rtlds -attribute (  test_clock | const_0 | const_1 | clock
                              | reset | const | primary_input | primary_output
                              | input | output | multi_bit | bbox
                              | (naming [-pattern matching_pattern] [-none]))
                              | (module [-pattern matching_pattern]))

custom_rule_rtlds -mode (functional | shift | capture)

custom_rule_rtlds -source signal_name

custom_rule_rtlds -args {signal_list}

custom_rule_rtlds -get (info | handle | list)
                  (-handle handle
                   -property property_name)

custom_rule_rtlds -report_violation
                  (-args arg_list
                   -handle handle)
```

Detailed Description

```
Use this command inside the custom rule Tcl file to specify the
functionality of the new rule.

-element (flop | comb | latch | and | or | clock_gate | multi_driven | wire)
      [-data signal_name]
      [-clock signal_name]
      [-reset signal_name]
      [-enable signal_name]
      [-output signal_name]

    Use "-element" to declare elements such as flop, comb, latch, and gate,
    or gate, and clock gate in a structural custom rule.
    . Use "-data" to specify a data signal name.
    . Use "-clock" to specify a clock signal name.
    . Use "-reset" to specify a reset signal name.
    . Use "-enable" to specify an enable signal name.
    . Use "-output" to specify an output signal name.

-complete_signal signal_name
      -bit bit_name -connection_type (only_one | more_than_one | all | none | any)

    Use "-complete_signal" to declare complete signal.
    . Use "-bit" to specify the bit_name.
      This bit_name should start a pattern that should match.
    . Use "-connection_type" to specify how many bits should match.
    NOTE: "-source" and "-args" should always be the complete_signal in such rules.

-attribute (  test_clock | const_0 | const_1 | clock | reset | const
            | primary_input | primary_output | input | output | multi_bit | bbox
            | (naming [-pattern matching_pattern] [-none]))
            | (module [-pattern matching_pattern]))
    [-signal signal_name]
    [-not]

    Use "-attribute" to specify various attributes such as test_clock, const_0,
    const_1, clock, reset, const, primary_input, primary_output, naming,
    multi_bit and bbox associated with signal name in a structural custom rule.
    . Use "-pattern" along with the "naming" attribute to specify the
      associated naming pattern.
    . Use "-none" along with the "naming" attribute when no signal should
      match the pattern.
```

```
    . Use "-pattern" along with the "module" attribute to specify the
      associated naming pattern.
    . Use "-not" to negate the effect of an attribute.

-mode (functional | shift | capture)
    Use "-mode" to define the mode of a structural custom rule.

-source signal_name
    Use "-source" to define the signal that will be the source of a structural
    custom rule.

-args {signal_list}
    Use "-args" to define the arguments in a structural custom rule.

-get (info | handle | list)
     (-handle $handle
      -property property_name)

    Use "-get" to extract additional information from provided objects when
    writing a basic lint custom rule.
    . Use "info" to extract string information from the handle. For example,
      to get name of an object, use this switch.
    . Use "handle" to get another handle from "handle". For example, to get
      the selector from a case statement, use this command to return the
      selector handle.
    . Use "list" to get a list of handles from "handle". For example, to get
      ports of modules, use this command to return a list of all port handles.

    Use "-handle" to add an object from which you want to extract information.
    For example, if you have enabled a trigger on the callback
    "rtldsModuleStart" using the "config_rtlds -custom_rule" command, the tool
    runs the Tcl procedure with the module object. If you want to get the name
    of the module, the "-handle" for this command would be module_handle.

    Use "-property" to specify the kind of information you want to extract.
    For example, if you want to extract the name of a module, pass the
    property "rtldsName" with "-property".

    NOTE:
    . "-handle" and "-property" are mandatory for the "-get" switch.
    . For complete lists of allowable objects and properties, see the
      "Superlint App User Guide" (Help - Application Guides - Superlint App
      User Guide).
```

```
-report_violation
      (-args arglist
      -handle $handle)

    . Use "-args" to provide a list of variable arguments. This completes
      the basic lint custom rule argument "%s" written in the rule file.
    . Use "-handle" to define the object you want to highlight in the RTL
      when returning the basic lint custom rule violation. For example, if
      you are reporting a violation on identifiers that are not following
      some specified naming convention, provide the identifier handle with
      this switch.

    NOTE: "-args" and "-handle" are mandatory for the "-report_violation"
    switch.
```

Example

```
Structural Custom Rule Examples:
custom_rule_rtlds -element flop -output OUT -clock CLK
custom_rule_rtlds -attribute naming -signal CLK -not -pattern .*clk
custom_rule_rtlds -mode Functional
custom_rule_rtlds -source OUT
custom_rule_rtlds -args {CLK}

custom_rule_rtlds -element flop -data DATA -output OUT
custom_rule_rtlds -element flop -reset RST
custom_rule_rtlds -element and -data RST -output DATA
custom_rule_rtlds -mode Functional
custom_rule_rtlds -source OUT
custom_rule_rtlds -args {RST OUT}

custom_rule_rtlds -element clock_gate -output OUT -clock CLK
custom_rule_rtlds -mode Functional
custom_rule_rtlds -source OUT
custom_rule_rtlds -args {OUT}

custom_rule_rtlds -element flop -output bit
custom_rule_rtlds -complete_signal no_flop -bit bit -connection_type none
custom_rule_rtlds -attribute primary_output -signal bit
custom_rule_rtlds -mode functional
custom_rule_rtlds -source no_flop
```

```
custom_rule_rtlds -args {no_flop}

custom_rule_rtlds -element wire -output OUT -data in
custom_rule_rtlds -mode Functional
custom_rule_rtlds -source OUT
custom_rule_rtlds -args {OUT}

Basic Lint Custom Rule Examples:
set name [custom_rule_rtlds -get info -handle $handle -property rtldsName]
set pattern "^mod_.*"
if { ![regexp $pattern $name match] } {
  custom_rule_rtlds -report_violation -args $name -handle $handle
}

set portIter [custom_rule_rtlds -get list -handle $handle -property rtldsPorts]
foreach formalActual $portIter {
   set formal [custom_rule_rtlds -get handle -handle $formalActual -property
rtldsHighConn]
   set actual [custom_rule_rtlds -get handle -handle $formalActual -property
rtldsLowConn]
   set formalname [custom_rule_rtlds -get info -handle $formal -property rtldsName]
   set isDotStar [custom_rule_rtlds -get info -handle $formalActual -property
rtldsIsDotStar]
   if {$isDotStar} {
     set duName [custom_rule_rtlds -get info -handle $du -property rtldsName]
     set iName [custom_rule_rtlds -get info -handle $handle -property rtldsName]
     custom_rule_rtlds -report_violation -args $formalname $duName $iName -handle
$handle
   }
}
```

See Also

check_superlint,
check_cdc,
config_rtlds

database

Accesses and manipulates the database for Unicov.

Syntax

```
Initialize and export Unicov-based database:
database -init_unicov
    [-app {app_type_list} ]
    [-write_metrics]
    [-work_dir work] [-scope scope] [-test test]
    [-waveform] [-overwrite] [-clean_work_dir] [-interval time_interval]
    [-witness_resolution (any_pass | any_unr | all)]

database -export_unicov
    [-task task_list]
    [-exclude {[const] [assume] [reset] [waived] [disabled]
               [unreachable] [deadcode] [synthesis_optimized]}]
    [-checker_mode (coi | proof_core | proof_core_high_precision | mutation)]
```

Returns

Various

Detailed Description

```
Initialize and Export Unicov-Based Database
-------------------------------------------

-init_unicov
    [-app {app_type_list}]
    [-write_metrics]
    [-work_dir work] [-scope scope] [-test test]
    [-waveform] [-overwrite] [-clean_work_dir] [-interval time_interval]
    [-witness_resolution (any_pass | any_unr | all)]
    Initialize the Unicov database setup.

    This command handles initialization for Unicov export functionality
```

```
    only. It does not generate properties.

    Property generation depends on the respective app-specific functions. For
    example, "check_cov -init" handles property generation in the Coverage App
    and "check_xprop -create target" handles property generation in X-Prop.

    Use the "database -export_unicov" command (see below) to export all
    properties generated by supported Jasper Apps. Jasper Apps supported by
    vManager include FPV, COV, CONN, CSR, XPROP, UNR and SPV.

    . Use "-app" to specify a list of Jasper Apps for Unicov export.
      Supported options include "fpv", "conn", "cov", "csr", "unr",
      "xprop", "spv", and "all" (the default).
      NOTE: "all" selects all Jasper Apps except UNR.
    . Use "-work_dir" to specify the work directory.
      The default is "cov_work".
    . Use "-scope" to specify the scope directory.
      The default is "scope".
    . Use "-test" to specify the test directory.
      The default is "test".
      NOTE:
      . If more than one task is exported or the verification scope is not
        default, the tool names the test directories for each task as follows:
        [test option specified]_[verification scope]_[task name].
      . Refer to the "database -export_unicov" command (see below) to understand
        how to specify verification scope and multiple tasks for export.
    . Use "-write_metrics" to generate the .vsof file. In the vManager-based
      flow, this is the default option.
    . Use "-overwrite" to overwrite the Unicov database.
      NOTE: This switch overwrites the coverage model. Use "-clean_work_dir"
      if you want to also remove the data.
    . Use "-clean_work_dir" to remove coverage working directory before
      saving new Unicov database.
    . Use "-waveform" to export waveforms along with the .vsof file.
    . Use "-interval" to export the Unicov database at the specified time
      intervals.
    . Use "-witness_resolution" to specify the criteria for selecting a witness
      to export to the Unicov database. The default is "any_unr".

-export_unicov [-launch_imc] [-task task_list]
    [-exclude {[const] [assume] [reset] [waived] [disabled]
               [unreachable] [deadcode] [synthesis_optimized]}]
    Export the Unicov database for the specified tasks using the "-init_unicov"
```

```
options. If "-init_unicov" is not issued, the tool will use
the default configurations, which include exporting information about all
apps to the default work directory "cov_work/scope/test".

. Use "-task" to export the Unicov database for one or more specified tasks
  instead of the default task. If no task is specified, the default task
  is the current task.
  NOTE: The "-task" switch accepts a list of tasks in colon-joined, space-
  separated sets of tasks with verification scopes prepended by a double-
  colon delimiter. If you do not provide the scope for a set, the tool
  takes the default scope.
  In a non-vManager flow, the default scope is "" (empty string), but in
  vManager-based runs, the default scope is the verification scope as
  mentioned in the VSIF file.
  In the following example, "-task {scope1::A:B C:D scope2::E}", the tool
  groups tasks A and B into one set using verification scope "scope1" and
  tasks C and D into another set using the default verification scope.
  Task E has its own verification scope (scope2).
. Use "-exclude" to exclude the specified cover items from the Unicov
  database.
  NOTE:
  . The "-exclude" switch affects Unicov databases generated for the
    Coverage App only.
  . The excluded items will be marked as "n/a" for Formal Metrics and as 1
    for  "Formal Excluded" attribute in the vManager/IMC.
  . The tool applies this switch to report results before exporting to
    Unicov.
  . The proof-core report is calculated on all targets for given tasks when
    you run "database -export_unicov -task task_list". All tasks are
    included if no task is specified.
. Use the "-checker_mode" switch to select the checker mode to be exported
  in the checker metrics. By default, the tool exports the currently
  active checker mode displayed in the GUI. Select the active checker mode
  in the GUI or by using the "check_cov -configure -checker_mode" command.
```

Example

```
// Initialize and export Unicov-based database:
database -init_unicov -app {fpv conn} -work_dir cov_work -scope scope -test test -
overwrite
database -export_unicov
```

See Also

session

debug_explorer

Runs the Debug Explorer.

Syntax

```
debug_explorer -init
    [ -why_limit N |-decision_limit N
     |-until node]
    [-visualize_window window_name]

debug_explorer -change_decision from_node [to_node]
    [ -why_limit N |-decision_limit N
     |-until node]
    [-visualize_window window_name]

debug_explorer -continue [node]
    [ -why_limit N |-decision_limit N
     |-until node]
    [-visualize_window window_name]

debug_explorer -refresh [node]
    [-visualize_window window_name]

debug_explorer -constrain from_node to_node
    (-force |-forbid |-release)
    [-visualize_window window_name]

debug_explorer -predicate
    ( (-register predicate_name)
     |(-list {[user_defined] [native]}))

debug_explorer -rule
    ( (-add -predicate predicate_name
            -scope (shallow | deep)
            -behavior (force | forbid | prefer | avoid)
           [-priority priority])
     |-remove priority
     |(-move old_priority -new_priority new_priority)
     |-list)
    [-visualize_window window_name]

debug_explorer -root_cause_finder sec_clock_gating
    [-visualize_window window_name]
```

Returns

```
-init
-change_decision [node] [node]
-continue [node]
-refresh [node]
  Returns a sorted list of lists, where each internal list contains 2 elements:
  a signal (or property) and a cycle.

-constrain
-predicate -register predicate_name
-rule -create
-rule -remove
-rule -move
  Does not return a value.

-predicate -list
  Returns a list of strings.

-rule -list
  Returns a list of string lists.

-root_cause_finder sec_clock_gating
  Returns a dictionary.
```

Detailed Description

```
Use this command to create a why path, a subset of the why analysis graph
that assists in waveform debugging. It creates a "debug_explorer" annotation
with the following color-coded plots:
. Signal-cycle pairs in the path:
  . Use green for a signal that does not have any possible children available in
    the "why" path.
  . Use gray for a signal that has only one possible child available in the
    "why" path.
  . Use purple for a signal that has multiple children available, from which the
     tool selects one to use in the path.
. Arrows point from a signal cycle to its selected child, which follows the
  same color-coding as its parent.

Stop conditions
---------------
```

```
The stop conditions are used to define until which node the why path is
computed. The path might finish before reaching the stop condition if it reaches
a node without children.
. Use "-why_limit" to create a path with N why operations.
. Use "-decision_limit" to create a path that does up to N decisions.
  . A decision happens when a signal-cycle pair that has multiple available
    children and the tool selects one of them to continue the path.
  . After reaching N decisions, the path continues until it would need to make
    another decision.
. Use "-until" to direct the path torwards a node and stop when it arrives at
  the provided node.
  . A node is defined as a signal-cycle pair, for example: {sig1 10}.
  . If it is not possible to create a path, "-init" creates an initial path with
    only the first node. "-change_decision" and "-continue" error out and do not
    modify the path.
. The default behavior is to create a path with a maximum of three decisions.

-init
    [-why_limit N |-decision_limit N |-until node]
    [-visualize_window window_name]
  Generate a first why path. The path begins at the triggering cycle of the
  target property and finishes when it reaches a stop condition.
  . Rules can be automatically applied according to what is commonly used for
    the target property. These rules can be modified or removed using the
    "debug_explorer -rule" command. The following rules are automatically
    applied:
    . For any property:
      . "avoid choosing reset signals"
    . For SEC properties:
      . "prefer choosing signals with different value on twin"
      . "prefer choosing imp signals"
  . The automatically applied rules can be moved to a different priority or
    removed using the "debug_explorer -rule" command.
  . Use "-visualize_window" to apply your command to a window other than the
    current window.

-change_decision from_node [to_node]
    [-why_limit N |-decision_limit N |-until node]
    [-visualize_window window_name]
  Edit the current why path by modifying the choice for "from_node" to
  "to_node".
  . For the change to take effect, the "from_node" must be a signal-cycle pair
```

```
    that has multiple children in the current why path.
  . The "to_node" must be a signal-cycle pair that is a reason for "from_node"
    and is not already in the current why path. If omitted, the tool selects a
    new child from the other reasons for "from_node."
  . The path continues from the specified choice until reaching a stop
    condition.
  . Use "-visualize_window" to apply your command to a window other than the
    current window.

-continue [node]
    [-why_limit N |-decision_limit N |-until node]
    [-visualize_window window_name]
  Continue the path from where the last constructed path stopped until reaching
  a stop condition.
  . Provide a node as an argument to indicate from where to continue the path.
    This node must be a signal-cycle pair that is a reason for the last node.
    If omitted, the tool selects a child from the other reasons for the last
    node.
  . Use "-visualize_window" to apply your command to a window other than the
    current window.

-refresh [node]
    [-visualize_window window_name]
  Recompute the path following the current Debug Explorer environment. Tha path
  might change if a decision was manually modified or if there were changes in
  the rules or constraints.
  . Provide a node as an argument to indicate from where to reload the path. If
    omitted, the tool selects the first node of the path.
  . Use "-visualize_window" to apply your command to a window other than the
    current window.

-constrain from_node to_node
    (-force |-forbid |-release)
    [-visualize_window window_name]
  Constrain the decision specified by a "from_node", which is a signal-cycle
  pair, and a "to_node", which is one of its reasons.
  . Use "-force" to force the decision to be made.
  . Use "-forbid" to prevent the decision from being made.
  . Use "-release" to remove the constraint from the decision.
  . Adding or releasing a constraint updates the "from_node" color.
  . The path might change if the current decision is no longer valid.
  . Use "-visualize_window" to apply your command to a window other than the
    current window.
```

```
-predicate
    ( (-register predicate_name)
     |(-list {[user_defined] [native]}))
  Manipulate predicates, which define a logic that evaluates either "0" or "1"
  for a given candidate node given its parent. Predicates are one of the
  parameters needed to create a rule.
  . Use "-register" to register a new  user-defined predicate with the specified
    name.
    . If the predicate already exists, the command does nothing.
    . There must exist a Tcl procedure with the specified name. It must have the
      following characteristics:
      . Return a boolean (1 or 0). If, during the predicate evaluation, it
        throws an error or returns a non-boolean value, the predicate is ignored
        and the candidate is kept in the list of available children for the
        rule under evaluation.
      . Accept three arguments:
        . "parent" refers to the current node.
        . "candidate" refers to the candidate child of the parent node being
          evaluated.
        . "window" refers to the Visualize window being used to generate the
          path.
    . Example:
      proc ingress_interface {parent candidates window} {
          set candidate_signal [lindex $candidate 0]
          return [regexp "^ig_" $candidate_signal]
      }
      debug_explorer -predicate -register ingress_interface
  . Use "-list" to list all predicates of the specified type.
    . The "user_defined" type lists predicates that were created using the
      "debug_explorer -predicate -register" command.
    . The "native" type lists predicates that are provided by the tool.
    . If the predicate type is not specified, the tool lists all predicates.

-rule
    ( (-add -predicate predicate_name
            -scope (shallow | deep)
            -behavior (force | forbid | prefer | avoid)
           [-priority priority])
     |-remove priority
     |(-move old_priority -new_priority new_priority)
     |-list)
    [-visualize_window window_name]
```

```
  Manipulate rules, which are used to change the amount of available children of
  the nodes, reducing the amount of possible paths by using domain-specific
  knowledge provided by the user.
  . Use "-add" to add a new rule with the specified predicate, scope, behavior,
    and priority.
    . The "scope" must be either "shallow" or "deep". A "shallow" scope only
      evaluates the predicate for the candidate nodes. A "deep" scope evaluates
      the predicate for the candidate nodes and their descendants within an
      interval of 2 cycles.
    . The "behavior" indicates how to handle the predicate evaluation. According
      to the specified behavior, a candidate node is included in the list of
      available children of a node if:
      . "force": The predicate evaluates to "1".
      . "forbid": The predicate does not evaluate to "1".
      . "prefer": Either the predicate evaluates to "1" or no candidates
        evaluate to "1" for the parent node.
      . "avoid": Either the predicate does not evaluate to "1" or all candidates
        evaluate to "1" for the parent node.
    . The "priority" must be a natural number where the smaller the number, the
      higher the priority. If not specified, the tool assigns the lowest
      priority to the new rule.
  . Use "-remove" to remove the rule with the specified priority.
  . Use "-move" to move the rule with the specified old priority to the new
    priority.
  . When a path is computed, the rules are evaluated following the priority
    order.
  . When a rule is added, removed, or moved, the colors are updated to reflect
    the new available children of the nodes but the current path is not
    modified. An exclamation mark is displayed in the arrows that are no longer
    available. "debug_explorer -refresh" can be used to recompute the path
    following the new rules. It is also possible to manually modify the path by
    using the "debug_explorer -change_decision" command.
  . Use "-list" to list all rules.
  . Use "-visualize_window" to apply your command to a window other than the
    current window.

-root_cause_finder sec_clock_gating
    [-visualize_window window_name]
  Performs path postprocessing tailored to SEC clock-gating verification.
    This postprocessing extracts information about the culprit gated clock
    and the flop it drives from the specified path. This information is
    returned as a Tcl dictionary as well as displayed in a new Visualize
    window.
```

```
  Use "-visualize_window" to apply your command to a window other than the
    current window.
```

Example

```
debug_explorer -init -why_limit 5
debug_explorer -change_decision {sig1 10} {sig2 9}
debug_explorer -continue -visualize_window visualize:1
debug_explorer -refresh {sig1 10}
debug_explorer -constrain {sig1 10} {sig2 9} -force
debug_explorer -predicate -list user_defined
debug_explorer -rule -add \
  -predicate reset_signal -scope shallow -behavior prefer
```

See Also

visualize

debug_handoff

Creates, manipulates, and modifies the Debug Handoff database.

Syntax

```
Creating the Debug Handoff Database
-----------------------------------
debug_handoff -save
              -property {property_name_list}
              database_name

debug_handoff -save
              -visualize_window visualize_window_name
              [-no_history]
              database_name

Using the Debug Handoff Database
--------------------------------
debug_handoff -load database_name

debug_handoff -elaborate

debug_handoff -dump_trace_table [database_name]
              [-file file_name]
              [-silent]


Debug Handoff Confirm Flow
--------------------------
debug_handoff -config database database_name
debug_handoff -config rtl_export (on | off)

debug_handoff -export_rtl path [-force]

debug_handoff -reload_rtl (-from_db |-external path)

debug_handoff -confirm ([-property {property_name_list}
                                  [-two_valued_simulation]]
                                [-report filename [-force]])+

Information
-----------
debug_handoff -get_info (version [file_name]| database)
```

Returns

None

Detailed Description

Use this command to create, use, and modify the Debug Handoff database.

NOTE:
. Debug Handoff currently supports the FPV App. Check the documentation for other apps for information about the supported save and restore flow.
. Debug Handoff does not support "set_database_auto_save_restore on".
. Debug Handoff does not support the "restore -include elaborated_design" command.
Databases saved after this operation are not accessible in the Debug Handoff mode.

The use model is:

1. In Jasper, use the Debug Handoff save command to create a database.
   The database is a single file that contains all the information needed to debug traces after closing the tool.
2. From a shell command line, launch Jasper in Debug Handoff mode to load the database and debug a trace.
3. With the Debug Handoff window, use the menu options to elaborate, open traces in Visualize, and run the confirm flow.

Commands:
debug_handoff -config database database_name
debug_handoff -save -property property_name

$ jg -debug_handoff database_name
Also run "jg -help" and refer to "debug_handoff" for command-line options.

NOTE: Also see the standalone Jasper Debug Handoff guide (Help - Application Guides - Debug Handoff).

CAUTION: Using encrypted scripts with Debug Handoff can produce unexpected, and in some cases, undesirable results. The problem arises in scripts that include internal commands.

Creating the Debug Handoff Database

```
-----------------------------------
-save -property {property_name_list}
       database_name
    Save a list of properties to the Debug Handoff database.
    . Use "-property" to specify a list of properties you want to add to the
      Debug Handoff database. A trace (proof-engine-generated trace)
      associated with the specified property must exist. Otherwise, the
      command triggers an error.
    . The name of the database can be an existing database or a new database.

      NOTE:
      . If you specify an existing database, the tool appends the specified
        properties to the database.
      . If you do not specify a database, the tool reuses the previously
        specified database or issues an error if none exists.
      . If the database name you specify does not include the extension
        ".dho", the tool automatically adds it.
      . Debug Handoff does not append traces in an existing database after
        environment changes such as adding or removing stopats, assumptions,
        or abstrations using the "-env" option. Provide a new database name.

-save -visualize_window visualize_window_name [-no_history]
       database_name
    Save the specified Visualize window to the Debug Handoff database.
    . Use "-visualize_window" to specify the Visualize window you want to
      add to the Debug Handoff database. The Visualize window must have a
      Visualize target. Otherwise, the command triggers an error.

    . Use "-no_history" to save only the trace the Visualize window is
      currently displaying.
      NOTE: The restored trace does not preserve other traces in the window's
      history. This option is recommended for a faster load time when you have
      used commands that might take a long time to run (for example,
      "visualize -quiet").

    . The name of the database can be an existing database or a new database.

    . All procedures and commands from sourced and included files are saved in
      the database. You can automatically load these files by enabling the
      "debug_handoff_source_files" variable before elaboration, that is,
      "set_debug_handoff_source_files on".

      NOTE:
```

```
      . If you specify an existing database, the tool appends the specified
        properties to the database.
      . If you do not specify a database, the tool reuses the previously
        specified database or issues an error if none exists.
      . If the database name you specify does not include the extension
        ".dho", the tool automatically adds it.
      . Debug Handoff does not support saving and loading traces if the
        auto-save feature was enabled ("set_database_auto_save_restore on").
      . Debug Handoff does not append traces in an existing database after
        environment changes such as adding or removing stopats, assumptions,
        or abstrations using the "-env" option. Provide a new database name.

Using the Debug Handoff Database
--------------------------------
-load database_name
    Load the Debug Handoff database in the Debug Handoff window.

    NOTE:
    . If a Debug Handoff database is already open, the tool closes the
      current database and opens the specified database.
    . Use "debug_handoff -load" in a Tcl script to be executed in the Debug
      Handoff Window

-elaborate
    Elaborate the design captured in the current Debug Handoff database.

-dump_trace_table [database_name]
                  [-file file_name]
                  [-silent]
    Dump the property name and trace ID for all traces saved in the Debug
    Handoff database.

      . Use "-file" to save the dumped information to the specified file.
      . Use "-silent" to suppress output to the console.

      NOTE:
      . If you specify an existing database, the tool reads the properties
        from the database.
      . If you do not specify a database, the tool uses the previously
        specified database or issues an error if none exists.
      . If a Debug Handoff database is already open and you specify a different
        database, the tool closes the current database and opens the specified
        database.
```

```
      . Also run "jg -help" and refer to "-trace_id_list", which is used
        in conjunction with "-dump_trace_table".

Debug Handoff Confirm Flow
--------------------------
-config database database_name
    Specify the current database for Debug Handoff commands. The name of the
    database can be an existing database or a new database. If the database
    name you specify does not include the extension ".dho", the tool
    automatically adds it.

-config rtl_export (on | off)
    Set the configuration in Debug Handoff to enable or disable the RTL export.
    Use this command after specifying the database name.

-export_rtl path [-force]
    Export the RTL contents from the database to the specified directory. The
    tool stores RTL files in the database using the absolute path from the
    "analyze" files and creates the same directory hierarchy inside the export
    folder.

    To use this capability, you must first enable "rtl_export".

    Use "-force" to overwrite existing files.

-reload_rtl (-from_db |-external path)
    Reload the RTL used in the Debug Handoff context. This command replaces the
    current design and requires a new elaboration.

    . Use "-from_db" to use the RTL stored in the database file.
    . Use "-external" to provide a path to new RTL you want to use to confirm
      traces. The new RTL must have the same directory hierarchy, and the files
      must have the same names.

-confirm ([-property {property_name_list} [-two_valued_simulation]]
          [-report filename [-force]])+
    Confirm one or more properties in the Debug Handoff database.

    . Use "-property" to specify a list of properties you want to confirm.
    . Use "-report" to export results to the specified file.
    . Use "-force" to overwrite an existing report.
    . By default, the tool applies three-valued logic semantics (1, 0, X) when
      confirming the trace. To change to two-valued logic semantics (1, 0), use
```

```
      "-two_valued_simulation".
    . The tool also applies two-valued logic semantics (1, 0) when the trace
      optimization is set to standard. In this case, opening a Visualize window
      concretizes any missing values, making all signal values available in the
      trace. This ensures the command is consistent with "visualize -confirm"
      when a trace is saved for later use.

    NOTE:
    . Run this command after reloading an external RTL and elaborating the
      design.
    . You must specify at least one switch in combination with "-confirm".
    . The "debug_handoff -confirm" flow does not support liveness or infinite
      properties.
      The tool does not draw conclusions for these properties. That is, the
      tool does not report whether a liveness or infinite property passed or
      failed confirmation.

Information
-----------
-get_info (version [file_name]| database)
    . Use "version" to print the Debug Handoff database version supported by the
      Jasper version currently running. Jasper writes the Debug Handoff
      database with this version and can only read databases with the same
      version.
    . Use "file_name" to print the version number of a specific Debug Handoff
      database file.
    . Use "database" to print the full path and name of the current Debug
      Handoff database.
```

Example

```
debug_handoff -save -property {prop_a prop_b prop_c} test.dho
debug_handoff -save -visualize_window visualize:0 test.dho
debug_handoff -load test.dho
debug_handoff -dump_trace_table -file props.txt

jg -debug_handoff test.dho

debug_handoff -config database test.dho
debug_handoff -config rtl_export on
debug_handoff -reload_rtl -external ./analyzed_files
debug_handoff -elaborate
```

```
debug_handoff -confirm -property {prop_a prop_b prop_c}
debug_handoff -confirm -report report.out

# Example Flow
# ------------
analyze...
elaborate...
clock clk
reset ~reset

cover {out == 8'b000001000} -name cover1
...
assert {out == 8'b000001100 |-> reset == 1 } -name assert1
...

prove -all

visualize -cover -property <embedded>::cover1

debug_handoff -config database file.dho
debug_handoff -config rtl_export on

debug_handoff -save -visualize_window visualize:0

debug_handoff -save -property {<embedded>::cover2 <embedded>::cover3 <embedded>::cover4
<embedded>::assert1 <embedded>::assert2 <embedded>::assert3}

exit -f

# Open the Debug Handoff database in the Debug Handoff GUI
jg -debug_handoff file.dho
```

dglob

Sorts and lists files.

Syntax

```
dglob expression
```

Returns

Returns a list of files that match the regular expression.

Detailed Description

Use this Tcl command to produce a list of files that match your specified regular expression argument. "dglob" is similar to the standard Tcl command "glob". Jasper Apps alphabetically sorts the list of files this command produces. For example, use "dglob" to determine which files should be analyzed with the "analyze" command.

expression
    A shell-style regular expression used to match a directory or set of files. See a Tcl reference manual for details.

Example

```
dglob ../design/*.v
```

See Also

analyze

dump_function_stack

Dumps the stack call of a function instantiation, given its name.

Syntax

```
dump_function_stack function_instance_name
```

Example

```
dump_function_stack mod.:func_myFunc_0
```

edit

Edits a file.

Syntax

```
edit file_name
```

Returns

No return values

Detailed Description

Use this command to edit a file using vi in a new xterm session.
To use a different editor, set the EDITOR environment variable.

file_name
    Name of the file you want to edit.

Example

```
edit my_file.v
```

elaborate

Synthesizes and reads the netlist.

Syntax

```
elaborate [-vhdl]
          [-sv09_expression_mode]
          [-bbox (0 | 1)]
          [-bbox_m string]*
          [-no_bbox_m string]*
          [-bbox_i string]*
          [-no_bbox_i string]*
          [-bbox_a value]
          [-bbox_mul value]
          [-bbox_div value]
          [-bbox_mod value]
          [-bbox_pow value]
          [-disable_auto_bbox]
          [-ignore_translate_off]
          [-top [lib_name'.']string['('arch_name')']]
          [-inst_top instance_path [-inst_top_hierarchy_mode (preserve | discard)]]
          [-parameter param_name param_value]*
          [-req |-repository repository_id]
          [-no_psl_tasks]
          [-no_type_properties]
          [-no_preconditions]
          [-create_related_covers
                {(precondition | late_precondition
                  | infinite_precondition | witness | contrapositive
                  | infinite_contrapositive)+}]
          [-mode (verilog | vhdl [-disable_sva_vhdl])]
          [-weaken_embedding]
          [-sv09_strong_embedding]
          [-multiple_clock]
          [-enable_assert_action_block]
          [-enable_sv_type_properties]
          [-enable_extended_optimization]
          [-disable_x_handling]
          [-disable_x_in_assign_expression]
```

```
[-triple_equal]
[-x_value (0 | 1)]
[-extract_case_assertions [-parallel_equal_assignment]]
[-ignore_if_case_qualifiers]
[-ignore_embedded_sva]
[-abstract_ram [-mpram_init_value (0 | 1)]]
[-max_array_dim_addr_size N]
[-loop_limit loop_count]
[-procedural_loop_limit loop_count]
[-non_constant_loop_limit loop_count]
[-enable_limited_unrolled_loops]
[-global_embedded_assumes]
[-unshare_instances]
[-keep_array_for_module string]*
[-model_tran_gates]
[-unbuffered_concatenation]
[-use_expanded_module_name]
[-f file_name]*
[(-L package_name)+ package_declaration.sv |-L library_name]
[-treat_work_as_lib_name]
[-cov_ignore_m string]
[-cov_include_m string]
[-prefer_universal_automata]
[-no_universal_automata]
[-disable_skip_buffer_for_merge]
[-disable_specify_block_connection]
[-disable_adder_extension_warning]
[-disable_extended_udp_handling]
[-enable_shared_tree_nodes (none | all)]
[-automatic_global_clock]
[-enable_synchronous_abort]
[-free_unassigned_vhdl_signals]
[-extract_covergroups]
[-optimize_implication_assert]
[-static]
[-witness_cover_property]
[-lrm_cover_property]
[-show_internal_flops]
[-no_multi_concat_as_multi_assign_pattern]
[-relaxed_port_connection_for_mixed_language]
[-remove_dangling_logic]
[-vhdl_enum_to_verilog_int]
[-relaxed_case_matching]
```

```
          [-respect_vhdl_others_clause]
          [-ignore_reg_initial_value]
          [-treat_clock_in_datapath_as_constant]

elaborate -log

elaborate -list (top | configuration) [-silent]

elaborate -clear [-req |-repository repository_id]

elaborate -reload_tb [-keep_environment]
```

Returns

Returns the name of the top module selected for analysis, including any existing parameters and their values. If there are multiple top-level candidates (more than one module not instantiated inside another module), and you did not use the "-top" option, the tool selects an arbitrary one.

Detailed Description

Use this command to synthesize and read the netlist for the Verilog(R) and VHDL files that have been analyzed using the "analyze" command.

Jasper prints a synthesis summary at the end of each successful elaboration. The summary includes the number of analyzed/actually synthesized design units/packages and the important elaboration flows that were enabled.

Example:

```
INFO (INL208): Elaboration synthesis summary:
Number of synthesized netlists 8 (0 packages)
Single run mode On
Pipeline On (8 pipelined netlists)
Late hierarchical reference connection Off
Number of analyzed Verilog modules 14 (8 synthesized)
Number of analyzed VHDL entities 0 (0 synthesized)
```

The following list includes additional information about entries in the summary.

. Single run mode: On by default. Prevents additional "elaborate" commands for

```
  improved performance in terms of memory use.
. Pipeline: Deletes netlists after they are elaborated to reduce elaboration
  memory peak.
. Late hierarchical reference connection: Postpones hierarchical references
  between modules to allow pipelining and reduce elaboration memory peak.

NOTE: Jasper Apps ignores a limited set of front-end messages; that is, it
does not print them by default. Refer to the command reference manual appendix
"Disabled Front-End Messages" for a list of messages you can enable with the
command "set_message".

Usage tips:
-----------
. By default, "elaborate_single_run_mode" is enabled, and in this mode, the
  tool uses less memory, but it only supports a single "elaborate" command.
  Subsequent "elaborate" commands without "analyze -clear" or "clear -all"
  trigger an error.

. After an unsuccessful elaboration, correct the issues that caused the
  elaboration error, run "analyze -clear", re-analyze the design, and elaborate
  again.

. By default, the tool uses the disk to serialize the elaborated design to the
  session. Use the command "set_elaborate_use_disk" before you run "elaborate"
  to specify that you do not want to use the disk.

Handling for inout ports:
-------------------------
. If your design has inout ports in the top module or in the module of the
  instance specified with the "-inst_top" switch, the tool converts them to
  output ports.

. If a module in your design contains inout ports, the tool converts them
  into input ports. Additionally, if these ports have driving logic, the tool
  also creates an associated output port for them.

  In the following example, Jasper recognizes that port1 drives port2 and
  port2 drives port1. Consequently, mod_alias has two associated output ports.

  Example:
  module mod_alias (inout port1, port2);
      alias port1 = port2;
  endmodule
```

. In the case of inout ports contained in black-boxed modules, the tool converts them into either an input port or an output port, depending on whether the given port has a driver or not. If the port is driven, the tool converts it into an input port; otherwise, the tool converts it into an output port.

. Port connections are like assignments. So when an input port is connected, it is equivalent to the following,

```
assign in_port = actual_connection ;
```

and you need to extend the actual connection if it is smaller in size.

When the port is inout or output, it is equivalent to the following,

```
assign actual_connection = out_port ;
```

and if the actual connection is smaller, a truncated value is assigned.

Also, if you want to extend the inout or output port actuals, this will either drive a constant 0 (for unsigned) or create a multiple driver (for signed) actuals. Both are incorrect.

Important things to know:

. When using the extended built-in functions "next" or "driver" in expressions on the command line, you must prepend them as shown below to avoid name clashes with signal names.

  . Verilog expression mode - use "$next" and "$driver".

  . VHDL expression mode - use "::next" and "::driver".

. During elaboration, the tool recognizes several types of memories, including memories with asynchronous or synchronous reset pins, multidimensional arrays, multiple read and write ports, and so forth. As a consequence, you might encounter some tool-generated signals. The original net in the RTL that represented a RAM appears with the naming convention <ram_net>.ram_net. In addition, you might see the following signals, for example, in the Visualize window or the "get_design_info" return:

  . For each write port of a RAM:
    <ram_name>.clk_#

```
    <ram_name>.write_enable_#
    <ram_name>.write_address_#
    <ram_name>.write_data_#

  . For each asynchronous write port of a RAM:
    <ram_name>.async_write_enable_#
    <ram_name>.async_write_address_#
    <ram_name>.async_write_data_#

  . For each read port of a RAM:
    <ram_name>.read_enable_#
    <ram_name>.read_address_#
    <ram_name>.read_data_#

. During elaboration, the tool might black box some arithmetic operators. (See
  "-bbox_mul", "-bbox_div", "-bbox_mod", and "-bbox_pow".) As a consequence,
  you might encounter some tool-generated signals. For the  multiply, divide,
  modulus, and power operators, you might see the following signals, for
  example, in the "get_design_info" return:

  . For each multiply operator:
    mult_#.left
    mult_#.right
    mult_#.out

  . For each divide operator:
    div_#.left
    div_#.right
    div_#.out

  . For each modulus operator:
    mod_#.left
    mod_#.right
    mod_#.out

  . For each power operator:
    pow_#.a
    pow_#.b
    pow_#.o

. Jasper Apps supports SystemVerilog configurations, as stated in the LRM
  (for example, Section 33 of the SystemVerilog '09 LRM). Elaborate the
  configuration cell as the top module. For example, if the configuration is
```

called "cfg1", use the following "elaborate" command:

```
elaborate -top cfg1
```

See "analyze" for additional information about SystemVerilog configurations.

Jasper Apps also supports VHDL configurations, and as with SystemVerilog configurations, you must elaborate the configuration as the top module, for example:

```
elaborate -vhdl -top CONFIG_ADDSUB_STRUCT
```

. According to the SystemVerilog LRM, interfaces cannot be instantiated in ports; however, Jasper accepts this coding in the port of the top module, but without its internal logic. Thus, if you define an interface as a port in the top module, Jasper does not synthesize any inner construction of it, only the signals.

In the following example, only the properties associated with interface myif_i2 are created, including the one inside the modport.

```
interface myif (input clk,rst);
logic in,out;
ast1 : assert property (in |-> out);
modport sram (input in, output out);
ast2 : assert property (in |-> out);
endinterface

module mymod (
  myif.sram myif_i
);
  myif myif_i2();
endmodule
```

. When defining the value of a SystemVerilog package or SystemVerilog Compilation Unit parameter or localparam with the "elaborate" command, you must use the following syntax:

```
pkg::P1
```

no-arguments
    Elaborate all Verilog/SystemVerilog modules and...

```
    . If "-top" is not specified, choose one module as top. (The top
      module will be the last module elaborated that is not
      instantiated by any other module.)
    . Elaborate any other modules or VHDL entities that are needed by
      instantiations.

    The default assumption is that the top-level module is from Verilog
    or SystemVerilog.

-vhdl
    Elaborate all VHDL entities.

    Important things to know:
    . If you do not specify "-top", the tool chooses one entity-architecture
      pair as top. (The top module will be the last entity elaborated
      that is not instantiated by any other architecture.)
    . To avoid undesirable results, always use the "-vhdl" switch with
      "elaborate" when the top is a VHDL entity.
    . The tool elaborates any other modules or VHDL entities that are needed by
      instantiations.
    . When you specify the "-vhdl" switch, the default expression mode is VHDL
      (see the "-mode" switch below). Always use "-mode verilog" to change the
      default if you are running the Connectivity App or if you want to use
      Verilog expressions in successive commands.

-sv09_expression_mode
    Elaborate all Verilog/SystemVerilog modules and support Tcl expressions
    that follow the SystemVerilog 2009 (IEEE Std 1800-2009) semantics for
    implicit embedding.

    NOTE: This switch is silently ignored when included in "-f" files.

-bbox
    By default, all missing modules are errors.
    Use this option to override the default behavior.
      0 = missing modules are errors.
      1 = missing modules are automatically black boxed.

 -bbox_m string*
    Black box the specified set of modules.

    A black box is a module with no observable or controllable content (that
```

```
    is, no logical function). By default, the outputs of a black boxed module
    always produce a logic X. If you intend to bind in another module with an
    output port connected and output port wire, the output port wire will be
    polluted by the X value driven by the black box output port. Take note of
    this behavior. If you intend for your bind module to drive this wire, you
    must connect it to the black box module output port signal stub inside the
    module.

    NOTE:
    . This switch accepts Tcl lists. You can also specify multiple modules with
      multiple instances of "-bbox_m" or with a wildcard string.
      For example, the following argument black boxes all modules whose names
      match the wildcard string:
      -bbox_m FIFO_*_TWO_PORT
    . If you specify a name with an escape character, you must transform it to
      a list item.
      For example, change:
      elaborate -bbox_m {\escaped }
      To one of the following:
      elaborate -bbox_m {{\escaped }}
      elaborate -bbox_m {\\escaped }
      elaborate -bbox_m [list \\escaped ]
    . The tool does not black box the "-top" module when it is a match for
      the wildcard string.
    . To black box a nested module inside a module, use
      "-bbox_m parent.nested".
    . Also see "elaborate -no_bbox_m".
    . The "analyze" command also supports this switch, and in some cases,
      you will want to use it there so the modules are black boxed earlier in
      the flow.

-no_bbox_m string*
    Do not black box the specified set of modules.

    NOTE:
    . This switch accepts Tcl lists. You can also specify multiple modules with
      multiple instances of "-no_bbox_m" or with a wildcard string.
      For example, the following argument turns off black boxing for all
      modules whose names match the wildcard string:
      -no_bbox_m FIFO_*_TWO_PORT
    . If you specify a name with an escape character, you must transform it to
      a list item.
      For example, change:
```

```
      elaborate -no_bbox_m {\escaped }
      To one of the following:
      elaborate -no_bbox_m {{\escaped }}
      elaborate -no_bbox_m {\\escaped }
      elaborate -no_bbox_m [list \\escaped ]
    . Using "-no_bbox_m" without "-bbox_m" has no effect.
    . The "analyze" command also supports this switch, and in some cases, you
      will want to use it there to avoid black boxing earlier in the flow.

-bbox_i string*
    Black box the specified set of instances.

    NOTE:
    . This switch accepts Tcl lists. You can also specify multiple instances
      with multiple instances of "-bbox_i" or with a wildcard string.
      For example, the following argument black boxes all instances whose names
      match the wildcard string:
      -bbox_i inst?_*
    . The top module is implied; therefore, do not include it in the instance
      name. For example, instance "arb" is under top module "top"; therefore,
      the command argument is: -bbox_i arb
    . If you specify a name with an escape character, you must surround
      the name with curly braces, for example:
      elaborate... -bbox_i {\arb$.()z}
    . Also see "elaborate -no_bbox_i".

-no_bbox_i string*
    Do not black box the specified set of instances.

    NOTE:
    . This switch accepts Tcl lists. You can also specify multiple instances
      with multiple instances of "-no_bbox_i" or with a wildcard string. For
      example, the following argument turns off black boxing for all instances
      whose names match the wildcard string:
      -no_bbox_i inst?_*
    . The top module is implied; therefore, do not include it in the instance
      name. For example, instance "arb" is under top module "top"; therefore,
      the command argument is: -no_bbox_i arb
    . If you specify a name with an escape character, you must surround
      the name with curly braces, for example:
      elaborate... -no_bbox_i {\arb$.()z}
    . "-no_bbox_i" takes priority over "-bbox_i"; therefore, if you specify an
      instance for both "-bbox_i" and "-no_bbox_i", the tool does not black box
```

```
     it.
   . Using "-no_bbox_i" without "-bbox_i" has no effect.


-bbox_a value
   Black box arrays if their size exceeds the specified value.

   By default, the tool bit-blasts arrays until their size reaches 2048 bits.
   At that point, the tool black boxes them. Use this switch to set a
   different threshold for black boxing.

   NOTE: Jasper Apps does not black box arrays inside interfaces.

   Important things to know about black-boxed arrays:
   . Use the following list as a guide to the behavior you can expect when
     using "elaborate -bbox_a":

     VHDL
     ----
     . 1D arrays - Jasper Apps does not black box them.
     . 2D arrays - All types, except record types, are black boxed above
       the threshold value.
     . 3D arrays - Same behavior as 2D arrays.

     SystemVerilog
     -------------
     . Packed arrays:
       . 1D arrays - Jasper Apps does not black box them.
       . 2D arrays - Wire, wand, tri, triand, trior, tri0, tri1, and trireg
         are not black boxed. All other types are black boxed above the
         threshold value.
       . 3D arrays - Same behavior as 2D arrays.

     . Unpacked arrays:
       . 1D arrays - Only wire, wand, tri, triand, trior, tri0, tri1, trireg,
         real, shortreal, packed struct/union, and unpacked struct/union are
         not black boxed. All other types are black boxed above the threshold
         value.
       . 2D arrays - Same as 1D, but now real, shortreal, packed
         struct/array/union, and unpacked struct/array/union types are black
         boxed also.
       . 3D arrays - Same behavior as 2D arrays.
```

NOTE: Multi-dimensional arrays of a unique element ([1] or [0:0]) declared in the dimension that varies more slowly will not be black boxed. See the following examples:
. logic [7:0] array [1] [127:0];
. logic array [0:0] [7:0] [1023:0];
. logic [0:0] [2047:0] array;

. The tool collapses the array to a single bit that does not drive any RTL logic.

. The <<black box>> notation appears adjacent to the array name in hierarchy trees (for example, the Formal Property Verification App Design Hierarchy pane).

. Array references in the RTL code are now free inputs into fanout logic and assertions.

. For properties created with "assert", "assume", or "cover" and tool-generated properties (for example, those created by the Connectivity App), if you refer to array members, the tool reports that they do not exist. In addition, you cannot constrain array members. To constrain or reference logic driven by the array entry, reference the logic that is downstream from the array reference. For example:

```
module ...
logic array2 [5:0]; // Declare logic array that is black boxed
                    // (6 locations 1 bit wide each)

wire out = array2[0];
endmodule
```

To reference the undriven input for "array2[0]" (which no longer exists when array2 is black boxed) you must reference the wire "out" instead.

NOTE: To avoid black boxing arrays in a set of modules, regardless of the "-bbox_a" values, see the "-keep_array_for_module" switch.

-bbox_mul value
Black box multipliers if their size (sum of input bits) exceeds the specified value.

By default, Jasper Apps black boxes all multiplier "gates" whose size

```
   is greater than or equal to a threshold of 1.

   NOTE: In most cases, each multiplication has a multiplier "gate". However,
   power-of-two multiplication (for example, sig * 4) is the exception to this
   rule. In these cases, the multiplier is implemented as a shift
   (sig << 2) operation and there is no multiplier to black box.


-bbox_div value
   Black box divide operators if their output exceeds the specified
   value.

   Important things to know:
   . If you do not use this switch, the tool black boxes all dividers.
   . The output size is the size of the dividend. For example,
     for a/b, the output has the size of a.


-bbox_mod value
   Black box modulus operators if their output exceeds the specified
   value. The default value is 1.

   Important things to know:
   . If you do not use this switch, the tool black boxes all modulus
     operators whose size exceeds the default value.
   .  The output size is the size of the right operand. For example,
      in the operation a%b, the output has the size of b.

-bbox_pow value
   Black box power operators if their output exceeds the specified value.

   Important things to know:
   . If you do not use this switch, the tool black boxes all power operators.
   . The output size is the size of the left operand or the size of the left
     operand times two to the power of the size of the right operand,
     whichever is smaller. For example, in the operation a ^ b, the output
     has the size of a or the size of (a * (2 ^ b)), whichever is smaller.

-disable_auto_bbox
   Disable all automatic black boxing.

   NOTE:
   . You cannot use "-disable_auto_bbox" in combination with switches that
```

```
      change the default threshold for automatic blackboxing, that is,
       "-bbox_a", "-bbox_mul", "-bbox_div", "-bbox_mod", or "-bbox_pow".
      UNR APP ONLY EXCEPTION: While "-disable_auto_bbox" will disable the
      blackboxing of arrays, you can still use the "-bbox_a" switch with UNR to
      blackbox arrays based on the threshold provided.
    . Using this switch with very large designs increases the probability of
      erroring out due to insufficient memory.

-ignore_translate_off
    Ignore synthesis pragmas when reading files from libraries during
    elaboration.

-top lib_name.string(arch_name)
    Use the specified module/entity/configuration as "top level" for synthesis
    and analysis.

    . If no "-top" option is present, the tool elaborates, synthesizes,
      and reads in all modules.
    . Supplying the library name is optional (default: work).
    . "string" is an entity or configuration name for VHDL or a module name for
      Verilog.
    . The architecture name is optional. If multiple architectures
      are present for a VHDL entity, the tool picks an arbitrary one.

    NOTE:
    . "arch_name" is only supported when used with the "-vhdl" option and a
      string that is an entity.
    . Proof accelerators cannot be specified as top-level modules.

-inst_top instance_path [-inst_top_hierarchy_mode (preserve | discard)]
    Use the specified path as the top instance.
    The tool black boxes all instances in the design that are not in the
    hierarchy of the specified path. The flow assumes the first part of
    the hierarchical path is the top module. If it matches, it looks for
    a complete match of the hierarchical path down to the specified
    instance; otherwise an error is generated.

    If the first part of the hierarchical path does not match the top module,
    the tool checks whether it is an instance under the top level module.
    If it matches an instance under the top level module, then the tool
    looks for a complete match of the hierarchical path down to the specified
    instance; otherwise, an error is generated.
```

For example, consider a design where the hierarchy, including the top module, is "twrapper.inst1". Specification for "-inst_top twrapper.inst1" blackboxes all instances except "twrapper.inst1", as it finds a hierarchical match, including the top module. Alternatively, the specification "-inst_top inst1" would behave the same way because "inst1" does not match the top module name but does match an instance just under the top module. Any other specification would result in elaboration terminating with the following messages depending on the resilient compilation settings:

set_resilient_compilation on: ENL191 error indicating the hierarchy was not found
set_resilient_compilation off: HIER-8000 error indicating the hierarchy was not found

NOTE:

. You can refer to instance paths inside "for generate" blocks just as you do in any Tcl command. You can use brackets for Verilog (for example, "genblk [0].inst1") or parentheses for VHDL (for example, "genblk (0).inst1").
. You can use ":" as a hierarchy delimiter. For example, you can use the following:
  elaborate -inst_top inst1:inst2
. The "elaborate" command does issue an error if it fails to find the specified path.
. You must escape instance names in the path using the convention supported by the language used to instantiate them.
  . If "top" is Verilog, and it instantiates "inst$", use Verilog escaping, even if "inst$" points to a VHDL design unit. For example:
    elaborate -inst_top \inst$
  . In some cases you must mix escaping styles to avoid ambiguity. Using the example above, if "inst$" points to a VHDL design unit, and it further instantiates "inst*", and you want to set the corresponding path as the top instance, mix escaping styles as follows:
    elaborate -inst_top \inst$.\inst*\

Use "-inst_top_hierarchy_mode" as follows:

. Specify "preserve" to maintain the behavior of "-inst_top". In this case, the tool uses the specified path as the top instance and black boxes all instances in the design that are not in the hierarchy of the specified path.
. Specify "discard" to elaborate a specific partition of the design (instance path) as the actual "synthesis top". In this case, the tool does

```
      not black box any instance in the design.

-parameter param_name param_value*
    Specifies the parameter values for the top-level module or for its
    subinstances.

    NOTE:
    . Only use this argument when you also use "-top" to specify the top-level
      module for analysis.
    . Overriding localparams is not LRM compliant; however, Jasper
      supports using this switch to override localparams in the top
      module or for its subinstances.
    . Use multiple "-parameter param_name param_value" instances to specify
      multiple parameter values.
    . For subinstances, provide the instance path together with the parameter
      name.

-req
    Only elaborate the requirement files.

    NOTE:
    . Use "-top" with "-req" to precisely control what is elaborated into
      the requirements namespace.
    . You must run "elaborate -req" after the main design has already been
      elaborated.
    . You must use "analyze -req" before using "elaborate -req".

-repository repository_id
    Elaborate RTL files previously analyzed inside the specified repository.
    NOTE:
    . Use the "connect" command with the "elaborate -repository" option
    to ensure that modules are created.
    . If you run "elaborate" again, any connections or instances created by the
    "connect" command will be cleared and must be recreated.

-no_psl_tasks
    Do not generate a separate task for each vunit. Instead, create properties
    in the <embedded> task.

-no_type_properties
    Do not generate type properties.

    The tool supports VHDL type properties. During elaboration, it extracts
```

type properties on the signals of the design from the allowed values and places them in the <constraints> task.

Refer to the appendix "Automatically Extracted Type Properties" in the command reference manual.

NOTE: "-no_type_properties" and "-enable_sv_type_properties" are mutually exclusive.

-no_preconditions
Disable the precondition check.

By default, the precondition check is enabled. This check creates new cover points (trigger points) in the assertion or assumption, which create cover statements in the task.

CAUTION: If you choose not to run precondition checking, vacuous proofs can go undetected.

NOTE: It is an error to use this switch in combination with the switch "-create_related_covers precondition" (see below).

-create_related_covers
{(precondition | late_precondition | infinite_precondition | witness | contrapositive | infinite_contrapositive)+}
Enable the precondition check for one or more specified types of cover statements.

The name of the precondition property indicates the precondition type, for example, propertyA:witness1.

The tool creates related covers as follows. In these examples, s1 and s2 are sequences.

. The precondition cover of an SVA implication s1 |=> P, is a finite trace where sequence s1 matches somewhere on the trace. This is the default.
. The late precondition cover of an SVA implication s1 |=> s2 is a cover of the sequence (s1 ##1 1'b1).
. The infinite precondition cover of an SVA implication s1 |=> P, where P is a infinite property, is an infinite trace where s1 matches somewhere on the trace.
. The witness cover of an SVA implication s1 |=> s2, where s2 is a sequence, is the cover of the sequence (s1 ##1 s2).

```
    . The witness cover of an SVA implication s1 |-> s2, where s2 is a sequence,
      is the cover of the sequence (s1 ##0 s2).
    . The contrapositive cover of an SVA implication s1 |-> P, is a finite
      trace where P is not true somewhere on the trace.
    . The infinite contrapositive cover of an SVA implication
      s1 |=> s_eventually B, where B is a Boolean expression, is an infinite
      trace where, starting at some point, B never matches.


      Jasper creates the contrapositive cover property only when the right
      hand side of the property is Boolean. The feature does not support
      sequence/property operators.

      The following example triggers a contrapositive property only when y is
      Boolean.

      x |=> y  creates cover (~y)



    NOTE:
    . The tool does not support witness precondition covers for assumptions with
      local variable assignments.
    . By default, the tool creates "precondition" covers.
    . It is an error to use "-create_related_covers precondition" in
      combination with the switch "-no_preconditions".
    . This command accepts Tcl lists.
    . Using "-create_related_covers" in "-f" files is not supported. It
      triggers a warning.

-mode (verilog | vhdl [-disable_sva_vhdl])
    Interpret expressions used in commands (for example,
    "assert", "assume", "cover", and "visualize") as follows:

    . verilog = Interpret expressions using Verilog format.
    . vhdl = Interpret expressions using VHDL format.

    The VHDL parser accepts SVA operators. To avoid name clashes with VHDL
    operators, suppress SVA keywords in the VHDL expression parser with
    "-disable_sva_vhdl". Otherwise, define SVA keywords "and", "or", and "not",
    as "s_and", "s_or", and "s_not".

    NOTE: Use the "-mode" option when the expression mode is different from
```

```
    that of the top module entity.

-weaken_embedding
    During elaboration, convert strongly embedded SVA sequences
    into weakly embedded ones. This switch can be useful when
    you want to avoid creating a liveness assertion
    unnecessarily.


 -sv09_strong_embedding
    During elaboration, treat all embedded SVA sequences defined in
    SystemVerilog 2009 (and newer) files as strongly embedded SVA sequences.

    Important things to know:
    . In SystemVerilog 2009 and newer, all embedded SVA sequences are treated as
      weakly embedded SVA sequences by default, which is the opposite of
      the SystemVerilog 2005 behavior.
    . This option does not affect embedded SVA sequences specified with an
      explicit strong or weak keyword.

-multiple_clock
    Elaborate PSL properties sampled by multiple clocks.

    NOTE: SVA properties with multiple clocks are always elaborated.

-enable_assert_action_block
    Enable assert action blocks.

    By default, the tool ignores assert action blocks.

-enable_sv_type_properties
    Generate SystemVerilog type properties for enum variables to ensure that
    enum values are obeyed.

    When you use this switch, the tool extracts SystemVerilog type properties
    on the signals of the design from the allowed values and places them in the
    <constraints> task. By default, the <embedded> task inherits the
    properties in the <constraints> task. For user-defined tasks, you
    must copy the assertions and assumptions explicitly or link the
    <constraints> task if applicable.

    NOTE:
    . If an enum is complete (that is, all values are possible), the tool does
```

```
      not create any properties for it.
    . "-no_type_properties" and "-enable_sv_type_properties" are mutually
      exclusive.

-enable_extended_optimization
    Enable additional optimization options during synthesis.

-disable_x_handling
    Do not treat literal Xs as free variables (that is, assign 0 to Xs).

    NOTE:
    . By default, the tool ensures a "pessimistic" verification by allowing
      signals that are assigned X to take all potential values.
    . By default, if you use "-disable_x_handling", the tool maps X values into
      the concrete value 0 prior to exposure to engines. Use "-x_value" if you
      want to define this value. (See the "-x_value" definition below.)

-disable_x_in_assign_expression
    Do not treat Xs as free variables in unary, binary, and question-colon
    expressions (that is, assign 0 to Xs).

    NOTE: By default, the tool ensures a "pessimistic" verification by allowing
    Xs in assign expressions to take all potential values.

-triple_equal
    Enable "case equality" synthesis in which the operators === and !==
    explicitly check for four-state values.

    The reset analysis domain uses a four-value analysis in which the tool
    respects X and Z values while performing comparisons. The comparisons can
    result in either a match or mismatch, but they never result in X.

    The truth table, for the Reset Analysis domain, for the operators === and
    !== follows.

    NOTE: Signals do not evaluate to X values for the operators in this truth
    table.

     a      b     ===    !==
    ------------------------
     0      0      1      0
     0      1      0      1
     1      0      0      1
```

```
 1      1      1      0
 X      0      0      1
 X      1      0      1
 0      X      0      1
 1      X      0      1
 X      X      1      0
 0      Z      0      1
 Z      0      0      1
 1      Z      0      1
 Z      1      0      1
 Z      Z      1      0
```

NOTE:

. The "-triple_equal" switch turns on the "case equality" semantics, which change the semantics of the "case" statement.
. The "case equality semantics" provide a definitive result when you have X and Z values in a Verilog "case" statement, just as if the comparison was being performed by a === operator.
. Refer to the Verilog LRM for a better understanding of the semantics.

  SystemVerilog LRM - IEEE 1800-2012 section 12.5:

  The reason for providing a case_expression comparison that handles the x and z values is that it provides a mechanism for detecting such values and reducing the pessimism that can be generated by their presence.

. If you do not use "-triple_equal", the tool treats the operators === and !== as == and !=. That is, by default, the tool does not respect "case equality" semantics.
. The previous truth table and the "case equality" semantics are only true in the reset analysis context.
. For the Formal Property Verification App (FPV), the X value refers only to the domain of reset analysis; however, the Z value refers to both the reset analysis and formal verification engines domains.
. When using "-triple_equal", case equality comparisons with an explicit X are correctly honored by the FPV proof domain. In FPV, a signal can never have value X; therefore, a case equality comparison between any signal and X will always evaluate to false. See table below.

. The truth table for the operators === and !== follows (FPV proof domain). "exp X" refers to explicit X in the case equality comparison.

a	b	===	!==
0	0	1	0
0	1	0	1
1	0	0	1
1	1	1	0
0	exp X	0	1
exp X	0	0	1
1	exp X	0	1
exp X	1	0	1
exp X	exp X	1	0
0	Z	0	1
Z	0	0	1
1	Z	0	1
Z	1	0	1
Z	Z	1	0

. It is noteworthy to say that, in the FPV proof domain, there is no concept of X as a value. Any free inputs, including X-assignments or X-comparisons, receive arbitrary values assigned by the proof engines.
. A signal that is carrying X in X-Prop App's X modeling cannot honor case equality as shown in the Reset Analysis domain truth table, even when using "-triple_equal".

-x_value
In assignment statements, treat Xs as specified:
. 0 - Assign 0 to Xs.
. 1 - Assign 1 to Xs.

NOTE: By default, the tool treats Xs as free variables; therefore, this switch has no effect unless you have also used "-disable_x_handling" or "-disable_x_in_assign_expression".

-extract_case_assertions [-parallel_equal_assignment]
Automatically extract assertions for priority, unique, and unique0 case statements and add them to the <embedded> task.

The naming convention is:
<path>._automatic_unique_case_Sig1.
<path>._automatic_unique0_case_Sig2.
<path>._automatic_priority_case_Sig3.

Where SigN is a signature based on the case statement structure.

```
    The tool generates unique_case assertions if the design includes any of the
    following:
    . UNIQUE keyword
    . full_case and parallel_case pragmas
    . SystemVerilog full_case and parallel_case attributes

    The tool generates unique0_case assertions if the design includes any of the
    following:
    . UNIQUE0 keyword
    . parallel_case pragma without full_case pragma
    . SystemVerilog parallel_case attribute without full_case attribute

    The tool generates priority_case assertions if the design includes no
    unique_case assertion as described above and any of the following:
    . PRIORITY keyword and no *default case*
    . full_case pragma
    . SystemVerilog full_case attribute

    Use "-parallel_equal_assignment" to prove parallelism between the
    assertions extracted for unique and unique0 case statements that pass the
    following test:

    . All case items have only one basic assignment statement.
    . The left-hand assignment variable is the same for all basic assignments.
    . Although some case items have the same conditions, they also have the
      same assignment expressions for the same variable.

-ignore_if_case_qualifiers
    Do not use SystemVerilog if/case qualifiers and Synopsys pragmas (full_case
    and parallel_case) during synthesis.

-ignore_embedded_sva
    Ignore embedded SystemVerilog assertions during synthesis.

-abstract_ram [-mpram_init_value (0 | 1)]
    Replace automatically extracted dual-port and multiport RAMs in the DUV
    with jasper_model_mpram PA instances.

    Use "-mpram_init_value" to specify the initial value of RAMs this command
    abstracts.

-max_array_dim_addr_size N
```

```
    Increase the maximum dimension of a vector or array to the specified size
    (in terms of address bits).

    NOTE:
    . N must be less than or equal to 31.
    . The default value is 20.

-loop_limit loop_count
    Change the loop count limit to the specified size.

    By default, the tool elaborates a loop count up to 5000. If the RTL
    exceeds the 5000-loop limit (or a limit specified with this switch),
    elaboration stops, and the tool generates an error. If that happens, you can
    use this option to specify a larger loop limit to ensure design elaboration
    finishes.

    The loop limit encompasses both procedural and generate-for loops.
    Procedural loops are loops defined in a sequential area in Verilog or in a
    subprogram or process in VHDL. The generate-for counterparts are resolved
    in compilation time to write replicable RTL.

    NOTE: Use the value 0 to re-enable the default behavior.

-procedural_loop_limit loop_count
    Change the limit on the number of iterations of procedural loops.

    This switch is a subset of "-loop_limit" that only affects procedural loops.
    If both "-loop_limit" and "-procedural_loop_limit" are configured, the
    tool respects the smaller of the two for the procedural loop limit.

    NOTE: Use the value 0 to make the loop_count unlimited.

-non_constant_loop_limit loop_count
    Change the loop count limit to the specified size for scenarios where
    the stop condition is not constant.

    By default, the tool elaborates a loop count up to 200 for these scenarios.
    If the RTL exceeds the 200-loop limit (or a limit specified with this switch),
    elaboration stops, and the tool generates an error. If that happens, you can
    use this option to specify a larger loop limit to ensure design elaboration
    finishes.

    NOTE: Use the value 0 to re-enable the default behavior.
```

```
-enable_limited_unrolled_loops
    Enable a limited unroll of loops whose stop condition is not constant. If
    the stop condition is not satisfied after the loop is unrolled a certain
    threshold, the unrolled process stops and an automatic property called
    "_automatic_limited_unrolled_loop_XXX" is created to help verify whether
    the loop has been unrolled a sufficient number of times to avoid an early
    exit when been run.

    Use the "-non_constant_loop_limit loop_count" switch to control how many
    times the loop should be unrolled in the process. By default, the loop
    is unrolled 200 times.

    Use the "set_elaborate_limited_unrolled_loop_property" command to control
    the creation of the associated automatic property.

    Use the "set_elaborate_limited_unrolled_loop_precondition" command to
    control the creation of a precondition for the property. This
    precondition checks whether the unrolled loop can have any scenario for
    an early exit.

-global_embedded_assumes
    Use embedded assumptions as environment assumptions.

    Important things to know about this command:
    . Use this option to leverage embedded assumptions for reset analysis.
    . This command applies to assumptions in the <embedded> task. It
      does not apply to assumptions in vunits.
    . This switch does not apply to embedded assumptions created with
      $isunknown.

-unshare_instances
    Enable netlist duplication in earlier stages of "elaborate" commands to try
    to avoid later conflicts in the HR solver.

    You must use this switch when requested by the "elaborate" command.

-keep_array_for_module string*
    Do not black box arrays in the specified set of modules.

    NOTE:
    . This switch accepts Tcl lists. You can also specify multiple modules with
      multiple instances of "-keep_array_for_module" or with a wildcard string.
```

```
      For example, the following argument turns off black boxing for all arrays
      in all modules whose names match the wildcard string:
      -keep_array_for_module FIFO_*_TWO_PORT
    . If you specify a name with an escape character, you must transform it to
      a list item.
      For example, change:
      elaborate -keep_array_for_module {\escaped }
      To one of the following:
      elaborate -keep_array_for_module {{\escaped }}
      elaborate -keep_array_for_module {\\escaped }
      elaborate -keep_array_for_module [list \\escaped ]

-model_tran_gates
    Synthesize Verilog transistor primitives (tran, rtran, tranif0, rtranif0,
    tranif1, rtranif1).

    By default, the tool black boxes all transistor primitives.

    NOTE: This synthesis does not handle strength reduction in resistive
    transistors.

-unbuffered_concatenation
    Do not add "Why" buffers inside concatenate operations.

    Use this switch when "Why" buffers are breaking constant propagation and
    preventing the tool from elaborating the design.

    CAUTION: If you use this switch, Why results will not highlight the values
    inside concatenates.

-use_expanded_module_name
    Do not rename excessively long module names.

    NOTE: By default, Jasper Apps renames modules if their length is
    excessively long, and the new name includes the string
    "renamed_due_excessive_length".

-f file_name*
    Specifies a file listing HDL sources, options, and other "-f"
    calls.

    NOTE:
    . "-f" searches for files and paths relative to the current directory.
```

```
    . Subcommands are not supported inside "-f" files.
    . Invalid or unknown entries in "-f" files are ignored triggering a
      message. To prevent these errors from being skipped, enable the
      "set_compilation_skip_errors_in_filelist on" command.
    . The tool expands Tcl variables inside "-f" files with Tcl's usual "$"
      syntax. If a Tcl variable referenced with "$" does not exist, the tool
      searches for an environment variable with that name.
    . Use multiple "-f file_name" instances to specify multiple files.
    . Jasper Apps supports absolute paths declared inside "-f" files, for
      example, /home/user/test.v.
    . Comments are allowed inside "-f" files. Parsing of comments has higher
      precedence than any other syntax. The supported comment delimiters are:
      . "//" with the comment ending at the end of the line (valid in paths)
      . "#" with the comment ending at the end of the line
      . "--" with the comment ending at the end of the line
      . "/*" with the comment ending on the next matching "*/", without nesting
    . The character ";" is not recognized in "-f" files.


(-L package_name)+ package_declaration.sv |-L library_name
    Use "-L" to specify Verilog packages or a VHDL or Verilog library for the
    tool to search. Also see "set_automatic_library_search".

    NOTE:
    . If you use "-L work", the "elaborate" process first searches for a
      cell in its parent library. If it does not exist in its parent library,
      the tool attempts to resolve the  cell according to the SystemVerilog
      config directive, if it exists. And finally, the tool searches in the
      order you specified the libraries.
    . The tool does not support this switch in -f files.

-treat_work_as_lib_name
    Elaborate using work named library instead of the current library.

    NOTE:
    . This switch is helpful with the P2P flow when your design includes
      code that is analyzed without any named libraries.
    . This switch also applies to library handling in SystemVerilog
      configurations, which complies with Chapter 33 of the SystemVerilog LRM.
      The default behavior in SystemVerilog configurations is to treat the
      library work as the current working library, and not as the library named
      work.
```

```
-cov_ignore_m string
    Ignore coverage generation for the specified set of modules.

-cov_include_m string
    Include coverage generation for the specified set of modules only.

    NOTE:
    . -cov_include_m switch has priority over -cov_ignore_m switch
    . These switches accepts only one string; however, you can specify multiple
      modules with a wildcard string.
    . If you specify a name with an escape character, you must surround
      the name with curly braces, for example:
      elaborate... -cov_ignore_m {\arb$.()z}

-prefer_universal_automata
    Use the universal automata algorithm for additional cases not included in
    the default set.

    For some properties, compilation can take much longer than expected. This
    algorithm can deliver a considerable property compilation and proof time
    reduction for many of those cases. This algorithm is on by default for a
    subset of cases, and this switch enables it for additional cases.

-no_universal_automata
    Disable the universal automata algorithm.

-disable_skip_buffer_for_merge
    Use the pre-2015.12 handling for instance merging.

    NOTE: By default, the tool now skips buffers to preserve linefile
    information for Why operations.

-disable_specify_block_connection
    Disable the connections on $setuphold calls and $recrem timing checks.

    By default, the delayed version of reference and data ports are
    directly connected to their counterparts in $setuphold calls and $recrem
    timing checks. Use this switch if you want the tool to ignore $setuphold
    calls and $recrem timing checks.

-disable_adder_extension_warning
    Do not issue VERI-9002 for adder carry_out extensions.
```

```
-disable_extended_udp_handling
    Use the pre-2023.09 handling for UDPs.

    NOTE:
    . By default, the tool now uses the enhanced UDP handling to
      achieve the same UDP support that was available with IFV and IEV.
    . Primitives containing multiple columns that are edge-sensitive
      trigger the VERI-9087 elaboration error. If you downgrade the error,
      the tool treats all entries as level-sensitive.

-enable_shared_tree_nodes (none | all)
    When synthesizing the design, handle nodes in the hierarchical tree as
    specified:
    . none: Do not share nodes in the hierarchical tree.
    . all:  Fold identical nodes. This is the default.
      NOTE: If the tool succeeds, it reduces elaboration time and memory
            consumption.

-automatic_global_clock
    Use the SystemVerilog clock reference $global_clock as a reference to the
    internal Jasper global clock signal.

-enable_synchronous_abort
    Enable synchronous abort operations:
    . SVA operators
      . disable iff
      . accept_on
      . reject_on
    . PSL
      . abort
      . async_abort

    Example: With this setting, "async_abort", which is asynchronous, behaves
    like "sync_abort".

    CAUTION: If you use this switch, the disable signal must be high on a clock
    cycle of the property clock to have an effect. This stipulation applies to
    all embedded and Tcl properties, that is, both user-defined and
    automatically created properties.
```

```
-free_unassigned_vhdl_signals
    Treat undriven VHDL signals as free inputs.

-extract_covergroups
    Automatically extract SystemVerilog Covergroup from the synthesizable
    section of RTL.

    NOTE:
    . Refer to the command reference manual appendix "Covergroup Support."
    . This is the default setting if you previously ran "check_cov -init -model
      functional".

-optimize_implication_assert
    Optimize the translation for liveness and safety assertions of the types
    s1|->s2 and s1|=>s2 where both s1 and s2 are sequences, and s2 is strongly
    and weakly embedded respectively.

    NOTE:
    . This optimization is only used when the property is an assertion of
      the form described above with no local variable assignments.
    . The switch "-optimize_implication_assert" is an initial release to gather
      feedback from early adopters and finalize implementation for an upcoming
      release. Contact support@cadence.com to provide feedback or comments
      targeting the upcoming production version.

-static
    Perform a "soft elaboration," stopping the elaboration process after
    creating the hierarchical tree.
    NOTE:
    . Only hierarchical information like instance tree, instances, and modules
      will be available. No information about signals or ports is extracted.
    . Use "elaborate -clear" to clear static elaboration before running full
      elaboration.
    LIMITATIONS: The following are not supported by static elaboration:
    . BBA
    . FSV App
    . VHDL designs
    . elaborate -mode

-witness_cover_property
    Use the witness semantics to generate covers instead of the LRM semantics.
    This is the default when you do not specify semantics.
    According to the SystemVerilog Language Reference Manual (LRM), a property
```

```
    of the form "A |-> B" is vacuously true whenever A is false; hence, a
    cover of this property matches at a cycle where A is false. In most cases,
    when you write "cover property (A |-> B)", you really want to find a
    witness trace where A actually occurs, which would correctly be expressed
    as "cover property (A #-# B)". By default, the Jasper command
    "elaborate -witness_cover_property" automatically completes this conversion
    to match your intent. In contrast, the "-lrm_cover_property" switch
    follows the LRM strictly, and any trace with a cycle in which the
    antecedent A is false is considered a valid match for
    "cover property (A |-> B)".

    NOTE: The switches "-witness_cover_property" and "-lrm_cover_property" are
    mutually exclusive.

-lrm_cover_property
    Use the LRM semantics to generate covers.

    NOTE:
    . The switches "-witness_cover_property" and "-lrm_cover_property" are
      mutually exclusive.
    . See "-witness_cover_property" for more details about this behavior.

-show_internal_flops
    Expose all internal flops and latches used in tristate logic. The tool
    synthesizes these registers to represent flops and latches with Z output.

-no_multi_concat_as_multi_assign_pattern
    By default, multi-concatenation to unpacked arrays or structs is treated
    as multi-assignment pattern. Use this switch to disable the behavior.

    NOTE: You must use this switch with both "analyze" and "elaborate".

-relaxed_port_connection_for_mixed_language
    Allow open connections when connecting VHDL and SystemVerilog instances in
    mixed-language designs.

-remove_dangling_logic
    Remove any user-declared logic that does not drive outputs of modules.

    NOTE: In VHDL designs, this feature is restricted to "variable"
    declarations.

-vhdl_enum_to_verilog_int
```

```
    Match basic VHDL enums (std_logic, bit, Boolean, and character) as integers
    in Verilog.

    NOTE: This switch applies the mapping used by xrun. Refer to the command
    reference appendix "Jasper Apps-Xcelium
    Integration" for additional information.

-relaxed_case_matching
    Provide case-insensitive name searching inside a converted Verilog package
    for mixed-language designs.

    NOTE: When you use this switch, we recommend you also use it with the
    "analyze" command.

-respect_vhdl_others_clause
    Synthesize latches for missing default FSM case assignments in VHDL with an
    incomplete enum specification.

-ignore_reg_initial_value
    Ignore the initial value of SystemVerilog/Verilog variables when there is no
    assignment to them.

    NOTE: By default, Jasper treats the initial value of SystemVerilog/Verilog
    variables as a continuous assignment when there is no assignment to them.

-treat_clock_in_datapath_as_constant
    Treat clock signal inside an always block as a constant signal when the
    signal is used as a data signal.

    The value of a signal on the datapath depends on the polarity of the clock
    signal on the clock event associated with the always block and the signal
    use as follows:
        . When using posedge on the clock event, the constant value is 1'b1
          if the signal is used in a positive context; otherwise, the constant
          value is 1'b0.
        . The same reasoning is valid when using negedge in the clock event.
          The value is 1'b1 if the signal is used in a negative context;
          otherwise, the constant value is 1'b0.

    NOTE: Warning VERI-2334 is always issued whenever the clock signal is used
    as a data signal

-log
```

```
    Print the log file generated by the "elaborate" command.

    The log file from the "elaborate" command is written to the directory for
    temporary files. Use TMPDIR to specify a different location.

-list (top | configuration) [-silent]
    . top: List elaborated top module names, including the main design, requirements,
      and repositories.
      The returned list has the following format:
      {{main design_top} {req {req_0_top req_1_top ...}} {repo {{0 repo_0_top} {1
repo_1_top} ...}}}
      Use "-silent" to suppress printing the report to the screen.

    . configuration: Return "elaborate" configuration switches and arguments.
      The list returned has the following format:
      {vhdl 0} {sv09ExpressionMode 0} {bbox_m {mod mod2}} {no_bbox_m {}} ...
      This is not an exhaustive list.

      NOTE: The command default is "elaborate -list" or "elaborate -list top".

-clear
    Clear the effects of the last elaboration.

    Use this option in cases where you need to change the
    parameters of an elaboration. Refer to the examples section
    below.

-clear -req
    Clear the effects of the previous requirements elaboration.

    NOTE: In general, you will use this option only if you want
    to clear the effects of the last requirement file elaboration
    so you can elaborate again with different command options or
    modified requirements.
    NOTE: This option is blocked by the apps: SEC, SPV, XPROP, and FSV.

-clear -repository repository_id
    Clear the effects of the previous elaboration in the specified repository.

-reload_tb
    If Incremental Wrapper Mode is turned on, re-elaborate the design's
    testbench without modifying the target DUT.
```

```
    NOTE: This command runs all previous "analyze" commands in the background.

    For more information see "incremental_elaborate_wrapper_mode".

-reload_tb -keep_environment
    Reload the testbench and maintain the previous formal setup configuration.

    WARNING:
    . Proof results will be cleared.
    . The tool sanitizes the formal setup, ignoring and issuing warnings for
      configurations that are no longer valid.

    LIMITATION: Keep environment does not support the following kind of
    configurations:
    . Setups from apps
    . Converted properties (assume -from_assert and assert -from_assume)
```

Example

```
elaborate -vhdl -top ALU -mode verilog
elaborate -create_related_covers {precondition late_precondition witness}

// Using "-clear" to change the parameters of an elaboration
// Elaborate the design.
elaborate
[WARN] foo.v(11): multiply mult_40u_40u is BLACK BOXED

// This multiplier is 40 x 40 bits.
// Clear the effects of elaborate so you can tell the tool to
// elaborate again and only black box multipliers if they are
// larger than 80 bits.
elaborate -clear

// Elaborate again and only black box multipliers larger than 80 bits.
elaborate -bbox_mul 80
```

See Also

analyze,
dglob,
set_elab_time_limit,
set_elaborate_interrupt_on_error,
set_elaborate_quiet_trace,
set_elaborate_single_run_mode,
set_elaborate_use_disk,
set_module_elaboration_memory_limit,
incremental_elaborate_wrapper_mode

exit

Use this command to exit the tool.

Syntax

```
exit [return_value] [-force]
```

Returns

No return values

Detailed Description

The command supports specifying an exit status code and can bypass confirmation dialogs.

return_value

Important information about this command:

. If you provide a return value argument, the tool uses it as the exit status to return to the parent process. For example, you can use "exit 1" to indicate a session that included an error.

. If you do not supply an argument, the tool exits with a status of 0.

. Use 0, 1, or a code greater than 10.

. Refer to the table below for additional information on available codes.

Exit Code	Definition
0	Represents normal exit.
1	Represents generic failure (not license specific).
2	Indicates failure to check out a license at startup for any reason other than those related to exit codes 3 and 4 below.
3	Indicates failure to check out a license at startup because the FlexNet Publisher option MAX limit has been reached.*
4	Indicates failure to check out a license at startup because the current user/host/... is excluded (FlexNet Publisher option EXCLUDE/EXCLUDEALL) or is not included (option INCLUDE/INCLUDEALL).*
5	Indicates failure to reconnect to the license server after successful license checkout. This can happen if you have not specified the "-no_wait" switch at launch. Refer to the command reference manual to learn more about "-no_wait".
6-10	Used internally by the tool and not available to the user.
0-1 and >10	Exit codes available to the user.

* Refer to FlexNet Publisher documentation available online.

-force

Exit immediately without confirmation dialogs.

Example

```
exit

# exit with a status code of 1
exit 1

exit –force
```

expert_system

Defines Expert System custom rules and manages built-in and custom rules.

Syntax

```
expert_system -rule -add rule_id -type (general | welcome) -title rule_title
   -description rule_description -procedure rule_tcl_procedure
   [-trigger (post_analyze_cmd | post_elaborate_cmd | netlist_change
              | clock_change | reset_change | formal_env_change
              | property_change | proof_setting_change | post_prove_cmd
              | debug_property_start | post_visualize_replot_cmd)+]
   [-category (analyze | elaborate | clock | reset | environment
               | proof_convergence | debug | verification_signoff | misc)]
   [-link links_list]
   [-min_fv_experience_level (new | limited | experienced | expert)]
   [-max_fv_experience_level (new | limited | experienced | expert)]
   [-min_jg_experience_level (new | limited | experienced | expert)]
   [-max_jg_experience_level (new | limited | experienced | expert)] [-force]

expert_system -rule -edit rule_id [-type (welcome | general)]
   [-title rule_title] [-description rule_description]
   [-procedure rule_tcl_procedure]
   [-trigger (post_analyze_cmd | post_elaborate_cmd | netlist_change
              | clock_change | reset_change | formal_env_change
              | property_change | proof_setting_change | post_prove_cmd
              | debug_property_start | post_visualize_replot_cmd)+]
   [-category (analyze | elaborate | clock | reset | environment
               | proof_convergence | debug | verification_signoff | misc)]
   [-link links_list]
   [-min_fv_experience_level (new | limited | experienced | expert)]
   [-max_fv_experience_level (new | limited | experienced | expert)]
   [-min_jg_experience_level (new | limited | experienced | expert)]
   [-max_jg_experience_level (new | limited | experienced | expert)]

expert_system -rule -list [(custom | built_in | all)]

expert_system -rule -is_enabled rule_id

expert_system -rule -enable rule_id
```

```
expert_system -rule -disable rule_id

expert_system -rule -remove rule_id

expert_system -rule -show rule_id [-type] [-category] [-title] [-description]
   [-procedure] [-trigger] [-link] [-min_fv_experience_level]
   [-max_fv_experience_level] [-min_jg_experience_level]
   [-max_jg_experience_level]

expert_system -rule -history [(matched)]

expert_system -log message

expert_system -generate_return -no_match

expert_system -generate_return -match
   [-action [-action_button_txt action_button_txt]]
   [-ask_user user_query -answer answer_list [-multi_choice]] -priority N
   -match_reason_title rule_match_reason_title
   [-match_reason_body rule_match_reason_body]

expert_system -waiver -add [-signal signal_expression]
   [-property property_expression] [-task task_expression]
   [-file file_expression] -comment [-regexp] [-rule_id rule_id_expression]

expert_system -waiver -remove waiver_id

expert_system -waiver -list [-rule_id rule_id_expression]

expert_system -waiver -export -file file_name [-force]

expert_system -waiver -import -file file_name

expert_system -tag -add tag_tcl_list

expert_system -tag -list

expert_system -set_server port@host [-project project_name]
```

Returns

Various

Detailed Description

Use this command to define, execute, and manage custom rule development and to customize information about the current invocation of Jasper.

Rule definition arguments provide for adding and editing rule IDs, titles, descriptions, procedures, triggers, links, and experience levels, and you can modify these throughout the run without the need to exit the tool. For example, to change the precondition of the rule, simply re-source the Tcl code for the procedure.

Rule execution arguments, which you can use only from within the rule's Tcl procedure, facilitate rule implementation.

Rule management arguments list rules, enable or disable rules, remove rules, and return rule information.

NOTE: You must define all mandatory attributes of a rule before you can enable it. Until you have defined all mandatory attributes, which include rule ID, title, description, procedure, and trigger, the tool considers the rule to be in a "partial" state. The tool automatically disables any rule it considers to be partial, and you must specifically enable it using "expert_system -rule -enable rule_id" once you have defined all attributes.

Use the following environment variables to connect to and customize the Jasper Information System interface:

. JASPER_UXDB_PATH - Enables connection to the Information System.
  You must define this variable as follows before loading Jasper:
  $ setenv JASPER_UXDB_PATH port@server_name

. JASPER_UXDB_ARGS - Defines project, regression cycle, and user tags to
  customize reports for the Information System. See the following example:

  setenv JASPER_UXDB_ARGS "project=proj1:tags=tag1,tag2,tag3"

  NOTE: You can subsequently use the "expert_system -tag" command to add or list
  tags.

```
. JASPER_UXDB_OPTS - Defines a way to control server/client parameters,
  such as, client's connection timeout. Define this value as follows:
  $ setenv JASPER_UXDB_OPTS "connection_timeout=N"
  Providing a timeout of 100s, for example, instructs Jasper to wait
  100s to get a response from the server. The default timeout is 30s.

NOTE:
. Expert System does not support the SPV and XPROP apps.
. Running "check_spv -init" or "check_xprop -init" automatically disables the
  Expert System. This limitation will be removed in a future release.
. See the "Jasper Expert System User Guide" for additional information
  (Help - Application Guides - Expert System User Guide).

-rule -add rule_id -type (general | welcome) -title rule_title
           -description rule_description -procedure rule_tcl_procedure
           [-trigger (post_analyze_cmd | post_elaborate_cmd | netlist_change
                     | clock_change | reset_change | formal_env_change
                     | property_change | proof_setting_change
                     | post_prove_cmd | debug_property_start
                     | post_visualize_replot_cmd)+]
           [-category (analyze | elaborate | clock | reset | environment
                      | proof_convergence | debug | verification_signoff
                      | misc)]
           [-link links_list]
           [-min_fv_experience_level (new | limited | experienced | expert)]
           [-max_fv_experience_level (new | limited | experienced | expert)]
           [-min_jg_experience_level (new | limited | experienced | expert)]
           [-max_jg_experience_level (new | limited | experienced | expert)]
           [-force]
    Add a custom recommendation and assign an ID.

    NOTE: Custom rule IDs must start with a "U" and contain letters, digits,
    and underscores only.

-type (general | welcome)
    . Use "-type" to specify the rule type.
      . Use "welcome" to specify a rule that triggers when you start Jasper.
      . Use "general" to specify a general rule.
        NOTE: This type of rule requires that you specify a trigger.

-title rule_title
    . Use "-title" to add a rule title.
```

```
-description rule_description
    . Use "-description" to add a rule description.

-procedure rule_tcl_procedure
    . Use "-procedure" to specify the associated Tcl procedure.

-trigger (post_analyze_cmd
          | post_elaborate_cmd
          | netlist_change
          | clock_change
          | reset_change
          | formal_env_change
          | property_change
          | proof_setting_change
          | post_prove_cmd
          | debug_property_start
          | post_visualize_replot_cmd)+
    . Use "-trigger" to specify what triggers rule commputation. You can
      specify more than one trigger.
      . Use "post_analyze_cmd" to trigger computation each time an "analyze"
        command ends.
      . Use "post_elaborate_cmd" to trigger computation each time an
        "elaborate" command ends.
      . Use "netlist_change" to trigger computation each time the netlist
        changes.
      . Use "clock_change" to trigger computation each time the clock
        definition changes.
      . Use "reset_change" to trigger computation each time the reset
        definition changes.
      . Use "formal_env_change" to trigger computation each time the formal
        environment changes.
      . Use "property_change" to trigger computation each time the list of
        properties changes.
      . Use "post_prove_cmd" to trigger computation each time a "prove"
        command ends.
      . Use "proof_setting_change" to trigger computation each time the
        proof settings change, for example, each time you run a "set_engine*"
        or "set_prove*" command.
      . Use "debug_property_start" to trigger computation each time a
        Visualize window opens for a violation or a cover trace.
      . Use "post_visualize_replot_cmd" to trigger computation each time a
        "visualize -replot" commands ends.
```

```
-category (analyze
           | elaborate
           | clock
           | reset
           | environment
           | proof_convergence
           | debug
           | verification_signoff
           | misc)
    . Use "-category" to specify the category the rule is associated with.
      The default is "misc".

-link links_list
    . Use "-link" to add HTML links to related information.
      The link should be specified in HTML format (see example below).

-min_fv_experience_level (new | limited | experienced | expert)
    . Use "-min_fv_experience_level" to assign a minimum formal verification
      experience level. The default is "new".

-max_fv_experience_level (new | limited | experienced | expert)
    . Use "-max_fv_experience_level" to assign a maximum formal verification
      experience level. The default is "expert".

-min_jg_experience_level (new | limited | experienced | expert)
    . Use "-min_jg_experience_level" to assign a minimum Jasper Apps
      experience level. The default is "new".

-max_jg_experience_level (new | limited | experienced | expert)
    . Use "-max_jg_experience_level" to assign a maximum Jasper Apps
      experience level. The default is "expert".

-force
    . Use "-force" to overwrite an existing recommendation.

-rule -edit rule_id [-type (welcome | general)] [-title rule_title]
            [-description rule_description] [-procedure rule_tcl_procedure]
            [-trigger (post_analyze_cmd | post_elaborate_cmd | netlist_change
                       | clock_change | reset_change | formal_env_change
                       | property_change | proof_setting_change
                       | post_prove_cmd | debug_property_start
                       | post_visualize_replot_cmd)+]
            [-category (analyze | elaborate | clock | reset | environment
```

```
                        | proof_convergence | debug | verification_signoff
                        | misc)]
            [-link links_list]
            [-min_fv_experience_level (new | limited | experienced | expert)]
            [-max_fv_experience_level (new | limited | experienced | expert)]
            [-min_jg_experience_level (new | limited | experienced | expert)]
            [-max_jg_experience_level (new | limited | experienced | expert)]
    Edit a custom recommendation.

-rule -list [(custom | built_in | all)]
    List rule IDs.
    . Use "custom" to list only custom rules.
    . Use "built_in" to list only system-defined rules.
    . Use "all" to list all rule IDs. This is the default.
    NOTE: The tool lists IDs even if you have previously disabled the
    recommendation.

-rule -is_enabled rule_id
    Return "true" if the specified rule is currently enabled or "false" if
    the specified rule is currently disabled.

-rule -enable rule_id
    Enable the specified rule.

-rule -disable rule_id
    Disable the specified rule.

-rule -remove rule_id
    Remove the specified rule.

-rule -show rule_id [-type] [-category] [-title] [-description] [-procedure]
            [-trigger] [-link] [-min_fv_experience_level]
            [-max_fv_experience_level] [-min_jg_experience_level]
            [-max_jg_experience_level]
    Return information on the specified rule.
    NOTE:
    . For system-defined rules, this command returns the category, title,
      description, links, and associated minimum and maximum experience
      levels.
    . For custom rules, this command also returns the rule type, associated
      procedure, and triggers if the custom rule is of type "general".

-rule -history [(matched)]
```

```
    List all rules that matched at some point during the current run.

-log message
    Print the specified message to the Expert System log file.

-generate_return -no_match
    Format the return value for the rule's Tcl procedure when the
    rule does not match.

-generate_return -match [-action [-action_button_txt action_button_txt]]
                        [-ask_user user_query -answer answer_list [-multi_choice]]
                        -priority N -match_reason_title rule_match_reason_title
                        [-match_reason_body rule_match_reason_body]
    Format the return value for the rule's Tcl procedure when the
    rule matches.

-action [-action_button_txt action_button_txt]
    . Use "-action" to specify what action Jasper takes when you
      apply the rule, and use "-action_button_txt" to customize the text
      for the action button.

-ask_user user_query -answer answer_list [-multi_choice]
    . Use "-ask_user" to define a user query with answers, and use
      "-multi_choice" to specify a multiple-choice question.
      NOTE: The answer_list argument must be a Tcl list of the possible
      answers to the question argument.

-priority N
    . Use "-priority" to set the rule's priority (0-100).

-match_reason_title rule_match_reason_title
    . Use "-match_reason_title" to summarize the reason for the match in
      the rule's precondition.
      NOTE: You must pass the match reason data to this command in HTML.

-match_reason_body rule_match_reason_body
    . Use "-match_reason_body" to add examples of the match that are visible
      when you click the "Show me" button.
      NOTE: You must pass the match reason data to this command in HTML.

-waiver -add [-signal signal_expression] [-property property_expression]
             [-task task_expression] [-file file_expression] -comment [-regexp]
             [-rule_id rule_id_expression]
```

```
    Add waivers as specified and return an ID list of the added waivers.

    NOTE: You must use exactly one of the following switches to add waivers to
    rules: "-signal", "-property", "-task", or "-file".

-signal signal_expression
    . Use "-signal" to add waivers on rules based on the specified signals.

-property property_expression
    . Use "-property" to add waivers on rules based on the specified properties.

-task task_expression
    . Use "-task" to add waivers on rules based on the specified tasks.

-file file_expression
    . Use "-file" to add waivers on rules based on the specified files.
      NOTE: The "-signal", "-property", "-task", and "-file" switches support
      wildcards or regular expressions ("-regexp" switch required).

-comment
    . Use "-comment" to add a required comment to the waiver.

-regexp
    . Use "-regexp" to specify the expression passed as argument to the
      switches "-signal", "-property", "-task", or "-file" as a regular
      expression.

-rule_id rule_id_expression
    . Use "-rule_id" to apply the waiver on rules that match the specified
      "rule_id" only.

-waiver -remove waiver_id
    Remove the specified waiver.

-waiver -list [-rule_id rule_id_expression]
    Show details for the specified waivers.

-waiver -export -file file_name [-force]
    Export all waivers to a Tcl file. For each waiver, an
    "expert_system -waiver -add" command is written to the file.
    You can later source the file in this or another Jasper Apps session.

-force
```

```
    Use "-force" to overwrite an existing file.

-waiver -import -file file_name
    Import waivers from the specified file.

-tag -add tag_tcl_list
    Add the specified tags, send the updated list of tags to the Information
    System, and return the updated list.
    NOTE: Each tag can contain the following characters only and cannot be
    empty: [A-Za-z_0-9]

-tag -list
    Show the current list of tags.

-set_server port@host [-project project_name]
    Set the Information System server path from within Jasper.

    NOTE:
    . If Jaspergold is already connected to a server, you cannot change the path
      with this command.
    . You must modify the server path before you load the design.

-project project_name
    Use "-project" to specify a project name. If you do not specify a  project
    name, the tool uses the default empty project.
```

Example

```
expert_system -rule -add "UDR001" -type general -title "Change engine settings" -
procedure "UDR001_procedure" -trigger {post_prove_cmd proof_setting_change} -link <a
href=\"www.cadence.com\">cadence</a>
```

export

Exports Tcl properties as SVA or PSL requirements files.

Syntax

```
export -to_sva file [-keep_hierarchy] [-force] [-task task_name] [-env]

export -to_psl file [-force] [-task task_name] [-env]
```

Returns

No return values

Detailed Description

Use this command to export Tcl properties as SVA or PSL requirements files.

```
-to_sva file [-keep_hierarchy] [-force] [-task task_name] [-env]
    Export Tcl properties created with Tcl commands to an SVA or PSL
    requirements file.

    . Use "-keep_hierarchy" to export SVA that directly references signals
      through hierarchy rather than mapping SVA signals through ports binding,
      which is the default behavior.
    . Use "-force" to overwrite any existing file that has the specified name.
    . Use "-task" to specify a task other than the current task.
    . Use "-env" to include global assumptions created with "assume -env".

-keep_hierarchy
    . Use "-keep_hierarchy" to export SVA that directly references signals
      through hierarchy rather than mapping SVA signals through ports binding,
      which is the default behavior.

-force
    . Use "-force" to overwrite any existing file that has the specified name.

-task task_name
```

```
    . Use "-task" to specify a task other than the current task.

-env
    . Use "-env" to include global assumptions created with "assume -env".

-to_psl file [-force] [-task task_name] [-env]
    Export Tcl properties created with Tcl commands to an SVA or PSL
    requirements file.

    . Use "-keep_hierarchy" to export SVA that directly references signals
      through hierarchy rather than mapping SVA signals through ports binding,
      which is the default behavior.
    . Use "-force" to overwrite any existing file that has the specified name.
    . Use "-task" to specify a task other than the current task.
    . Use "-env" to include global assumptions created with "assume -env".

-force
    . Use "-force" to overwrite any existing file that has the specified name.

-task task_name
    . Use "-task" to specify a task other than the current task.

-env
    . Use "-env" to include global assumptions created with "assume -env".
```

Example

```
export -to_sva my_properties.sva -force -task my_task -env
```

formal_profiler

Profiles and provides detailed information about an engine's work on a property.

Syntax

```
formal_profiler -property property
    [-engine_mode (B | M)] [-silent]
    [-time_limit maximum_time]
    [-initial_bound bound_number]
    [-report_period period]
    [-viewer]
    [-file file_name [-force]]

formal_profiler -report [run_name] [-bound bound_number]
    (-instance instance_list
    |-property property_list
    |-signal signal_list)

formal_profiler -list

formal_profiler -show run_list [-list (bound)]

formal_profiler -viewer

formal_profiler -stop [run_name]
```

Returns

The "-property" switch returns the name of the new run.
The "-report" switch returns a list of pairs with the relevant types
and their effort for a specific run.
The "-list" switch returns a list of available runs.
The "-show" switch returns a list of pairs with each run and its parameters.

Detailed Description

Use this command to control the Formal Profiler run for a single property.

```
Each new run is stored individually, and its name is returned.

LIMITATIONS:
The following is a list of Formal Profiler limitations:
. Requires background proof
. Does not clear results In the GUI. It only replaces them when you run the
  command again.
. Supports only one window at a time
. Supports only one engine thread
. Does not support SPV and XPROP properties
. Does not support proof flows with word-level transformations or reductions
  (see "set_word_level_reduction")

-property property_name [-engine_mode (B | M)] [-silent] [-time_limit limit]
    [-initial_bound bound_number] [-report_period period] [-viewer]
    [-file file_name [-force]]

    Start the Formal Profiler and return the name of the run. This command can
    be called repeatedly to track multiple properties in parallel.

    The Formal Profiler tracks the engine's effort to prove the specified
    property and reports it with info messages in the console. Use it to find
    possible proof bottlenecks that can be solved for improved convergence.

    . Use "-engine_mode" to select either engine B or M. By default, the tool
      uses engine B.

    . Use "-file" to also redirect the output of Formal Profiler to a specified
      file. Use "-force" if the specified file already exists and you want to
      overwrite it. The proof output is not redirected.

    . Use "-silent" to suppress the info messages in the console.

    . Use "-time_limit" to set the proof maximum run time.
      Formal Profiler respects the following configuration commands if you do
      not use this switch:
      . set_prove_time_limit
      . set_prove_per_property_time_limit
      . set_prove_per_property_time_limit_factor

    . Use "-initial_bound" to set the bound at which the profiling starts.
      Skipped bounds are shown as trivial ones, that is, with no tracked effort.
```

```
    . Use "-report_period" to control intermediate reports during a bound.
      With this switch, Formal Profiler repeatedly reports information at every
      "specified value" period during a bound, and at the end of a bound.
      If you do not use this switch, Formal Profiler reports information every
      60s and at the end of a bound by default. Use "-report_period 0" to disable
      intermediate reports and receive reports at the end of a bound only.

    NOTE:
    . Only engine B supports "-initial_bound".
    . The default time unit for "-report_period" is seconds, that is, if no
      unit is explicitly defined, it assumes a time period measured in
      seconds. However, you can write "1m" for one minute or "1h30m" for
      ninety minutes.

    . Use "-viewer" to open the Formal Profiler GUI, which arranges all
      the collected information into easier-to-use-and-explore tables.

-report [run_name] [-bound bound_number]
    (-instance instance_list |-property property_list
    |-signal signal_list)

    Return the run's effort for one or more specified entities on the specified
    bound.
    If you do not specify a run, the command returns data from the last run.
    Effort might not add up to 100% in some cases where the struggle is bound
    to internal structures not mappable to user-visible elements.

    . Use "-instance" to get the effort for one or more instances.

    . Use "-property" to get the effort for one or more properties.

    . Use "-signal" to get the effort for one or more signals.

-list

    Return all available runs.

-show run_list [-list (bound)]

    Return all attributes of the specified runs.
    Use "-list" to return the specified argument for each run.

-viewer
```

```
    Open the Formal Profiler GUI for the last run.

-stop [run_name]

    Stop the Formal Profiler. If you do not specify a run, the command stops
    this last run.
```

Example

```
# Start Formal Profiler with engine B
formal_profiler -property prop:0 -engine_mode B

# Start Formal Profiler with engine B, redirecting the output to the file
  "profile.log", overwriting if it already exists
formal_profiler -property prop:0 -engine_mode B -file profile.log -force

# Get effort for a set of signals from run "formalprofiler:1"
formal_profiler -report formalprofiler:1 -bound 4 -signal {sig1 sig2 sig3}

# Get a list of saved runs
formal_profiler -list

# Get all parameters from formalprofiler:1 and formalprofiler:2
formal_profiler -show {formalprofiler:1 formalprofiler:2}

# Get a list of bounds from formalprofiler:1 and formalprofiler:2
formal_profiler -show {formalprofiler:1 formalprofiler:2} -list bound

# Stop the last Formal Profiler run
formal_profiler -stop

# Stop the Formal Profiler run "formalprofiler:1"
formal_profiler -stop formalprofiler:1

# Start Formal Profiler with engine M and show its GUI
formal_profiler -property prop:1 -engine_mode M -viewer
```

See Also

prove,
set_prove_time_limit

fsm_checks

Generates properties to check the reachability of state machine transitions.

Syntax

```
fsm_checks [-add_fsm fsm_spec]
           [-add_cross cross_spec]
           [-cross [-auto [-depth depth_value]]]
           [-trans [-no_auto]]
           [-states]
           [-deadend]
           [-remove_fsm fsm_signal]
```

Returns

None

Detailed Description

Use this command to generate properties for proving the reachability of:

- State machine transitions
- Combinations of states across pairs of state machines (crosses)

To run FSM checks, the command automatically detects state variables or accepts user-specified state variables and extracts cover properties you can prove or Visualize.

no-arguments
Print a list of FSMs, their states, and any crosses that will be acted upon if the "fsm_checks" command is run with "-trans" or "-cross".

-add_fsm fsm_spec
Define a new FSM to be used when creating the properties.

Important things to know about this command:
. Supply an FSM spec of the form:

```
      {<fsm_signal> <list of FSM states>}
    . "list of FSM states" is an alternating list of values and labels.

-add_cross cross_spec
    Define the pair of FSMs to "cross", that is, prove or Visualize the
    reachability of each combination of states across the two FSMs.

    Supply a "cross spec" of the form:
    {<FSM1> <FSM2>}

-cross [-auto [-depth depth_value]]
    Generate the specified cross-state properties.

    Important things to know about this command:
    . It creates a new task for each state machine pair.
    . The task naming format is:
      CROSS__<full path to FSM1>__<full path to FSM2>
    . The automatically extracted property name format is:
      <value>_&&_<value>
    . If you use "-auto", the tool attempts to automatically determine which
      FSMs should be crossed. In this mode, the tool evaluates the fanin of
      each FSM, and if there is another FSM within "depth_value" ranks of
      registers (as defined by the "-depth" switch and with a default value of
      1), then those two FSMs are crossed.
    . "depth_value" is the level of registers to look through. It must be a
      positive integer.

-trans [-no_auto]
    Generate properties for state transition reachability for both user-defined
    and automatically identified FSMs.

    Use "-no_auto", if you want the tool to generate properties for
    user-defined FSMs only.

    Important things to know about this command:
    . It creates a new task for each state machine pair.
    . The task naming format is:
      TRANS__<full path to FSM>
    . The automatically extracted property name format is:
      <value>_to_<value>

-states
    Generate properties for state reachability for both user-defined and
```

```
    automatically identified FSMs.

    Important things to know about this command:
    . It creates a new task for each state machine.
    . The task naming format is:
      STATE__<full path to FSM>
    . The automatically extracted property name format is:
      <value>

-deadend
    Generate dead end properties for both user-defined and automatically
    identified FSMs.

    Important things to know about this command:
    . It creates a new task for each state machine.
    . The task naming format is:
      DEADEND__<full path to FSM>
    . The automatically extracted property name format is:
      <value>_deadend

-remove_fsm fsm_signal
    Remove the specified user-defined FSM.

    Important things to know about this command:
    . Supply an FSM signal of the form:
      <fsm_signal>
    . It removes the tasks related to this FSM.
```

Example

```
// Define a new FSM to be used when creating the properties
fsm_checks -add_fsm {my_fsm {0 `IDLE 1 `WRITE 2 `READ 3 `WAIT}}

// Define the pair of FSMs to "cross"
fsm_checks -add_cross {my_fsm1 my_fsm2}

// Construct a conceptual FSM from the concatenation of sig1, sig2, and sig3
fsm_checks -add_fsm {{{sig1,sig2,sig3}} {val lab val lab val lab}}
```

get_analyze_info

Returns information about analyzed HDL or property files.

Syntax

```
get_analyze_info -list (file | library | macro | defined_macro | undefined_macro)
                 [-req |-repository repository_id]
                 [-silent]


"-name" Scope
-------------
get_analyze_info -name name_list
                 [-req |-repository repository_id]
                 [-silent]

"-library" Scope
----------------
get_analyze_info -library library_name_list
                 [-req |-repository repository_id]
                 [-silent]

"-module" Scope
---------------
get_analyze_info -module module_name_list
                 [-req |-repository repository_id]
                 [-silent]

"-primitive" Scope
------------------
get_analyze_info -primitive primitive_name_list
                [-req |-repository repository_id]
                [-silent]

"-package" Scope
----------------
get_analyze_info -package package_name_list
                 [-req |-repository repository_id]
                 [-silent]
```

```
"-file" Scope
-------------
get_analyze_info -file file_name_list
                 [-req |-repository repository_id]
                 [-silent]

"-macro" Scope
--------------
get_analyze_info -macro macro_name_list
                 [-evaluate]
                 [-req |-repository repository_id]
                 [-silent]
                 [-include_usage_info]

"-location" Scope
-----------------
get_analyze_info -location location_list
                 [-req |-repository repository_id]
                 ([-include {((name | file) {pattern})+}]
                  [-exclude {((name | file) {pattern})+}]
                  [-regexp])
                 [-silent]

"-compilation_unit" Scope
--------------
get_analyze_info -compilation_unit compilation_unit_name_list
                 [-req |-repository repository_id]
                 [-silent]
```

Returns

A list of dictionaries or values.

Detailed Description

Use this command to get information about the HDL or property source files intended for verification.

-list (file | library | macro | defined_macro | undefined_macro) [-req |-repository repository_id]

```
                          [-silent]
    List "analyze" information for the design.

    . Use "-req" to return only information from analyzed requirement files.

    . Use "-repository" with a repository ID to return only information from
      the specified repository.
      NOTE: SEC analyzes the imp side in repository slot 2.

    . Use "-silent" to suppress text output to the console and log files.

    Supported "-list" Enums
    -----------------------
    . file: List all analyzed files. The tool returns a list
        of absolute path file names.

      NOTE:
      . Files that are analyzed multiple times, such as `include files, are
        listed only once.
      . The order of the file names in the return of the command is the order
        that each analyzed file is first processed.

    . library: List all analyzed libraries. The tool returns
        a list of library names.

    . macro: List all analyzed macros. The tool returns a list with the macro
      name, the last definition location, and the definition status
      (Defined/Undefined).

    . defined_macro: List all analyzed macros that were defined. The tool
      returns a list with the macro name and the last definition location.

    . undefined_macro: List all analyzed macros that were undefined. The tool
      returns a list with the macro name and the last undefinition location.

"-name" Scope
-------------
-name name_list [-req |-repository repository_id]
                [-silent]
    Get analyze information for any design object identified by the specified
    names.
    The tool returns a list of dictionaries containing the name, the location
    of the definition and the type of the definition.
```

```
    The tool currently supports the following design objects:
    . Verilog/SystemVerilog modules
    . VHDL entities
    . Packages
    . Functions
    . Verilog/SystemVerilog macros

    . Use "-req" to return only information from analyzed requirement files.

    . Use "-repository" with a repository ID to return only information from
      the specified repository.
      NOTE: SEC analyzes the imp side in repository slot 2.

    . Use "-silent" to suppress text output to the console and log files.

    NOTE:
    . For VHDL entities, the tool returns the type "module" to make
      scripting easier.
    . For macros added on the command line, we do not have an associated file
      line. In this case, the tool returns the macro location
      "<analyze_command_definition>(1)".

"-library" Scope
----------------
-library library_name_list [-req |-repository repository_id]
                           [-silent]
    Get analyze information for the specified library. The tool returns a
    list of dictionaries containing the library name, its Compilation Units, a
    list of its analyzed modules or entities, and a list of its analyzed
    packages.

    . Use "-req" to return only information from analyzed requirement files.

    . Use "-repository" with a repository ID to return only information from
      the specified repository.
      NOTE: SEC analyzes the imp side in repository slot 2.

    . Use "-silent" to suppress text output to the console and log files.

"-module" Scope
---------------
-module module_name_list [-req |-repository repository_id]
```

```
                          [-silent]
    Get analyze information for the specified module or entity. The tool
    returns a list of dictionaries containing the module or entity name, the
    library in which it was analyzed, its location in the source HDL file,
    and a list of function definitions. Each function definition is a
    dictionary containing its name and its location in the source HDL file.

    . Use "-req" to return only information from analyzed requirement files.

    . Use "-repository" with a repository ID to return only information from
      the specified repository.
      NOTE: SEC analyzes the imp side in repository slot 2.

    . Use "-silent" to suppress text output to the console and log files.

"-primitive" Scope
------------------
-primitive primitive_name_list [-req |-repository repository_id]
                               [-silent]
    Get analyze information for the specified primitive. The tool
    returns a list of dictionaries containing the primitive name, the
    library in which it was analyzed, its location in the source HDL file,
    and a list of function definitions. Each function definition is a
    dictionary containing its name and its location in the source HDL file.

    . Use "-req" to return only information from analyzed requirement files.

    . Use "-repository" with a repository ID to return only information from
      the specified repository.
      NOTE: SEC analyzes the imp side in repository slot 2.

    . Use "-silent" to suppress text output to the console and log files.

"-package" Scope
----------------
-package package_name_list [-req |-repository repository_id]
                           [-silent]
    Get analyze information for the specified package. The tool returns a
    list of dictionaries containing the package name, the library in which it
    was analyzed, its location in the source HDL file, and a list of function
    definitions. Each function definition is a dictionary containing its name
    and its location in the source HDL file.
```

```
    . Use "-req" to return only information from analyzed requirement files.

    . Use "-repository" with a repository ID to return only information from
      the specified repository.
      NOTE: SEC analyzes the imp side in repository slot 2.

    . Use "-silent" to suppress text output to the console and log files.

"-file" Scope
-------------
-file file_name_list [-req |-repository repository_id]
                     [-silent]
    Get analyze information for the specified file. The tool returns a
    list of dictionaries containing the "analyze" command used, the file name,
    the Compilation Unit, and a list of macro definitions processed when
    analyzing the file. Each macro definition is a dictionary containing macro
    name and its location in the source HDL file.

    . Use "-req" to return only information from analyzed requirement files.

    . Use "-repository" with a repository ID to return only information from
      the specified repository.
      NOTE: SEC analyzes the imp side in repository slot 2.

    . Use "-silent" to suppress text output to the console and log files.

    NOTE:
    . This command supports Verilog/SystemVerilog files only.
    . This command also returns information for files specified in Verilog
      through `include directives.

"-macro" Scope
--------------
-macro macro_name_list [-evaluate]
                       [-req |-repository repository_id]
                       [-silent]
                       [-include_usage_info]
    Get analyze information for the specified macro. The tool returns a list
    of dictionaries containing the macro name, the "analyze" command, the
    compilation unit name, and a list of dictionaries containing the macro
    type (DEFINE, UNDEF and UNDEFALL), location and the definition, when applicable,
    for its macro definitions.
```

```
    . Use "-evaluate" to evaluate the macro text as an expression.
      The tool returns a value or a value range, for example, 0:1, represented
      by the macro expression. It also recursively evaluates and displays the
      value of each macro expanded inside the original macro text.
      The command evaluates the latest macro definition.
      If the macro text is not a valid SystemVerilog expression, the return
      value is empty.
      If the macro text is a string, the return value is a representative value
      of the string.

    . Use "-req" to return only information from analyzed requirement files.

    . Use "-repository" with a repository ID to return only information from
      the specified repository.
      NOTE: SEC analyzes the imp side in repository slot 2.

    . Use "-silent" to suppress text output to the console and log files.

    . Use "-include_usage_info" to list all macro usages processed by the analyzer.

    NOTE:
    . "get_analyze_info -macro" does not support PSL macros.
    . For macros added on the command line, we do not have an associated file
      line. In this case, the tool returns the macro location
      "<analyze_command_definition>(1)".

"-location" Scope
-----------------
-location location_list [-req |-repository repository_id]
                        ([-include {((name | file) {pattern})+}]
                         [-exclude {((name | file) {pattern})+}]
                         [-regexp])
                        [-silent]
    Get analyze information for the specified RTL location. The tool returns
    the "analyze" command, the Compilation Unit, the list of dictionaries
    containing the location, the file's inclusion backtrace (which is a list of
    locations), and the list of active macro definitions up to the specified
    file line (not including the specified line). Each active macro definition
    is a dictionary containing the macro name and its location in the source
    HDL file (except for the command-line macro definitions in which a virtual
    file location is used).

    If the given location is analyzed multiple times, the command reports
```

```
    information for each "analyze" command.

    . Use "-req" to return only information from analyzed requirement files.

    . Use "-repository" with a repository ID to return only information from
      the specified repository.
      NOTE: SEC analyzes the imp side in repository slot 2.

    . Use "-include" to include in the returned Tcl list only the specified
      macro (name) or macro definition file (file) pattern. Use the following
      format to specify the argument:
      {filter_type filter_value}

    . Use "-exclude" to exclude from the returned Tcl list the specified
      macro (name) or macro definition file (file) pattern. Use the following
      format to specify the argument:
      {filter_type filter_value}

    . Use "-regexp" to specify that you are using a regular expression to specify
      attributes for "-include" and "-exclude".

    . Use "-silent" to suppress text output to the console and log files.

    NOTE:
    . This command supports Verilog/SystemVerilog files only.
    . Use the following format to specify the location:
      {file_name line_number}
    . The inclusion backtrace is an ordered list of locations corresponding
      to include directives needed to analyze the RTL location specified in the
      command.
    . Exclusion filter takes precedence over inclusion, which means that if you
      use an attribute value with both the "-include" and "-exclude" switches,
      any active macro having this attribute value is excluded from the returned
      Tcl list.
    . "-include" and "-exclude" support wildcards (* and ?) in the pattern
      argument if you do not also use the "-regexp" switch.
    . For macros added on the command line, we do not have an associated file
      line. In this case, the tool returns the macro location
      "<analyze_command_definition>(1)".

"-compilation_unit" Scope
----------------
-compilation_unit compilation_unit_name_list [-req |-repository repository_id]
```

```
                                        [-silent]
Get analyze information for the specified Compilation Unit. The tool
returns a list of dictionaries containing the Compilation Unit name, the
library in which it was analyzed, its location in the source HDL file, and
a list of function definitions. Each function definition is a dictionary
containing its name and its location in the source HDL file.

. Use "-req" to return only information from analyzed requirement files.

. Use "-repository" with a repository ID to return only information from
  the specified repository.
  NOTE: SEC analyzes the imp side in repository slot 2.

. Use "-silent" to suppress text output to the console and log files.
```

Example

```
get_analyze_info -list file
get_analyze_info -list library
get_analyze_info -list macro
get_analyze_info -list defined_macro
get_analyze_info -name {name1 name2}
get_analyze_info -library {lib1 lib2}
get_analyze_info -module {mod1 mod2}
get_analyze_info -primitive {prim1 prim2}
get_analyze_info -package {pack1 pack2}
get_analyze_info -file {defs1.sv defs2.v}
get_analyze_info -macro {MACRO1 MACRO2}
get_analyze_info -location {{defs1.sv 3} {defs2.v 1}}
get_analyze_info -location {my_file.v 10} -include {name {.*PCIE.*}} -regexp
-----

# Consider the macro0.v RTL file:
`define INSTANTIATE
module m0 ();
    `ifdef INSTANTIATE
        `define CHILD_INST
        m2 m2();
    `endif
endmodule

# Consider the macro1.v RTL file:
`ifdef CHILD_INST
    `define INSTANTIATE
`endif

# If you run:
analyze -sv09 macro0.v macro2.v
get_analyze_info -name INSTANTIATE

# get_analyze_info command returns a list of 2 dictionaries:
{name INSTANTIATE location {macro0.v 1} type macro} {name INSTANTIATE
  location {macro1.v 2} type macro}
```

See Also

analyze,
get_design_info,
check_sec

get_clock_info

Provides detailed information about the clock configuration.

Syntax

```
get_clock_info -gui

get_clock_info declared_clock
    -list {[flop] [latch] [load] [connected_driver]}+
    [-transitive] [-silent]

get_clock_info -fastest_clocks

get_clock_info -load_all_related_info

get_clock_info -csv file_name

get_clock_info -property property_name_tcl_list
```

Returns

The "-list" switch returns a signal list.

Detailed Description

Use this command to launch the Clock Viewer GUI or list flops and latches in the fanout of a declared clock.

-gui
    Launch the Clock Viewer window.

    The Clock Viewer displays all the clock information in an easy-to-read manner. Use this window to get detailed information regarding your clock configuration and environment. This window presents information regarding the clocking signals and their connected drivers, related registers, related properties, and aliases. It also reports clock characteristics and runs a sanity check on each clock. The window also provides information

```
    about declared clocks, including their reference and derived clocks and
    inputs rated by them.

    The window also provides access to simplified clock tree visualizations
    with the Schematic Viewer and Graph GUIs.

declared_clock
    -list {[flop] [latch] [load] [connected_driver]}+
    [-transitive] [-silent]
    Return a Tcl list of the specified signal types in the fanout of the
    specified declared clock.

    . This command takes a list of enums.

    . Use "-transitive" to list signals in the transitive fanout of the
      specified clock.

    . Use "-silent" to turn off the output to the screen and return information
      with a Tcl return value, which is helpful in Tcl scripts.

-fastest_clocks
    Return a Tcl list with all the user-declared fastest clocks.
    The fastest clocks are the clocks with the smallest factor. Neither
    "-both_edges" nor a clock's phase impact the determination for the fastest
    clock.

-load_all_related_info
    Populate all the related properties and registers of all the clocking
    signals shown in the Clock Viewer.

    By default, the Clock Viewer only populates the columns of related info
    when you select a row in the Clocking Signals pane of the Clock Viewer.

-csv file_name
    Generate a report in CSV format with the current Clock Viewer information.

-property property_name_tcl_list
    Returns a list of pairs consisting of the clock and the edge that triggers
    the property.

    NOTE:
    . If the tool's fastest clock triggers the property, the clock and edge
      information are omitted from the Tcl return.
```

```
    . Properties with various SVA temporal operators might include logic
      triggered by the tool's fastest clock, such as disable_iff, $past, $rose,
      $fell, and so forth.
```

Example

```
// Launch Clock Viewer
get_clock_info -gui

get_clock_info clk -list {flop latch}
f1.q f2.q l1.q l2.q

get_clock_info -load_all_related_info
```

See Also

clock,
sanity_check

get_decomposed_expression

Returns a list containing the sub-expressions of the provided argument.

Syntax

```
get_decomposed_expression (signal_name | property_name | literal_expression)
                          [-max_depth N]
                          [-reverse]
                          [-disable_sub_products]
```

Returns

Returns a list of expression strings corresponding to the sub-expressions
of the provided argument.

Detailed Description

Use this command to get the sub-expressions of the provided argument.

NOTE: List of decomposable Verilog operators:
. Logical operators: &&, ||, !
. Bitwise operators: &, |, ^, ~, ~&, ~|, ~^
. Sequential operators: ##, |=>, |->
. Conditional (ternary '?') operator: COND_EXPR ? THEN_EXPR : ELSE_EXPR
. Comparison operators: ==, !=, ===, !==, >, <=, <, >=

The remaining Verilog operators can appear in a given expression, but the tool
does not decompose them.

signal_name | property_name | literal_expression
    [-max_depth N] [-reverse]
    [-disable_sub_products]
    Get the sub-expressions of the provided argument.
    . signal_name: The command returns the sub-expressions of the specified
      signal's driving expression.

    . property_name: The command returns the sub-expressions of the specified

property's expression.
NOTE: This command does not decompose embedded property expressions. In this case, the command returns the specified property's name.

. literal_expression: The command returns the sub-expressions of the specified expression, whose operands must be design signals. See the examples below.
NOTE: VHDL expressions are not supported.

. Use "-max_depth" to get all sub-expressions up to the specified depth, where N is an unbounded non-negative integer (0, 1, 2, ..., $).
NOTE: the default depth value is "$" (returns all sub-expressions in all depths).

. Use "-reverse" to get sub-expressions in the reverse order, that is, starting from the leaf (non-decomposable) sub-expressions toward the full expression you specified.

. Use "-disable_sub_products" to disable product operator decomposition. This might help speed up the decomposition of large products (expressions involving several consecutive operators && or &), for example, product expression (f1 & f2 & ... & f10) has a total of 1024 sub-product expressions. If you use "-disable_sub_products", the tool returns just the 10 factors (f1, f2, ..., f10).

NOTE: To prevent undesired long decompositions, by default, the argument expression is limited to 100 sub-expressions. You can alter this limit with the command "set_max_num_sub_expressions <max_num_sub_expressions>".

Example

```
# In the examples below, "a", "b", "c", and "d" represent design signals

get_decomposed_expression { a ##2 b ##[5:7] c } -max_depth 0
# Returns a list containing the following expressions:
#    a ##2 b ##[5:7] c

get_decomposed_expression { a ##2 b ##[5:7] c } -max_depth 1
# Returns a list containing the following expressions:
#    a ##2 b ##[5:7] c
#    a ##2 b
#    c
```

```
get_decomposed_expression { a ##2 b ##[5:7] c } -max_depth 2
# OR
get_decomposed_expression { a ##2 b ##[5:7] c } -max_depth 3
# OR
get_decomposed_expression { a ##2 b ##[5:7] c } -max_depth $
# OR
get_decomposed_expression { a ##2 b ##[5:7] c }
# Returns a list containing the following expressions:
#    a ##2 b ##[5:7] c
#    a ##2 b
#    c
#    a
#    b

get_decomposed_expression { a & ~( b | ( c & ~d ) ) } -max_depth 0
# Returns a list containing the following expressions:
#    (a & ~b & (~c | d))

get_decomposed_expression { a & ~( b | ( c & ~d ) ) } -max_depth 1
# Returns a list containing the following expressions:
#    (a & ~b & (~c | d))
#    (a & ~b)
#    (a & (~c | d))
#    {(~b & (~c | d))

get_decomposed_expression { a & ~( b | ( c & ~d ) ) } -max_depth 2
# Returns a list containing the following expressions:
#    (a & ~b & (~c | d))
#    (a & ~b)
#    (a & (~c | d))
#    {(~b & (~c | d))
#    a
#    ~b
#    a
#    (~c | d)
#    ~b
#    (~c | d)

get_decomposed_expression { a & ~( b | ( c & ~d ) ) } -max_depth 3
# OR
get_decomposed_expression { a & ~( b | ( c & ~d ) ) } -max_depth 4
# OR
```

```
get_decomposed_expression { a & ~( b | ( c & ~d ) ) } -max_depth $
# OR
get_decomposed_expression { a & ~( b | ( c & ~d ) ) }
# Returns a list containing the following expressions:
#    (a & ~b & (~c | d))
#    (a & ~b)
#    (a & (~c | d))
#    {(~b & (~c | d))
#    a
#    ~b
#    a
#    (~c | d)
#    ~b
#    (~c | d)
#    ~c
#    d
#    ~c
#    d
```

get_design_info

Prints out design information.

Syntax

```
Top-Instance Scope
------------------
get_design_info (no arguments)

get_design_info -list
                    ([parameter] [local_parameter] [interface_port_parameter]
                     [interface_port_local_parameter] [constant] [enum]
                     [output] [input] [inout] [stopat]
                     [undriven] [undriven_internal] [loop_index] [multiple_driven]
                     [bbox_mod] [bbox_inst] [bbox_in] [bbox_out] [bbox_inout]
                     [bbox_in_instance_port] [bbox_out_instance_port]
                     [bbox_inout_instance_port]
                     [basic] [counter] [fsm] [fifo] [array [-no_aggregate]]
                     [register] [flop] [latch] [gate]
                     [multiplier] [divider] [modulus]
                     [module] [module_no_param] [module_name] [module_name_no_param]
                     [instance [-depth N]]
                     [bind_instance [-depth N]]
                     [interface] [interface_no_param] [interface_instance]
[interface_port]
                     [prim_inst] [prim_mod]
                     [celldefine]
                     [package] [compilation_unit]
                     [architecture] [signal] [wire] [property]
                     [assert] [assume] [cover] [macro]
                    )+
                    [-verbosity (high | low | silent)]
                    [-typed_list]
                    [-silent]
                    [-file file_name [-force]]
                    [-filter filter_expression [-regexp] [-case_sensitive]]
                    [-gui]
                    [-include_hier_path]
                    [-pattern (sv_packed | sv_assignment)]
```

```
                    [-transitive]

get_design_info -include_config [-instance instance_name]

get_design_info -enum enum_name

"-instance" Scope
-----------------
get_design_info -instance instance_name
                [-list
                      ([parameter] [local_parameter] [interface_port_parameter]
                       [interface_port_local_parameter] [constant] [enum]
                       [output] [input] [inout] [stopat]
                       [undriven] [undriven_internal] [multiple_driven]
                       [bbox_mod] [bbox_inst] [bbox_in] [bbox_out] [bbox_inout]
                       [bbox_in_instance_port] [bbox_out_instance_port]
                       [bbox_inout_instance_port]
                       [basic] [counter] [fsm] [fifo] [array [-no_aggregate]]
                       [register [-depth N]] [flop [-depth N]] [latch [-depth N]]
                       [gate]
                       [multiplier] [divider] [modulus]
                       [module] [module_no_param] [module_name] [module_name_no_param]
                       [instance [-depth N]]
                       [bind_instance [-depth N]]
                       [interface] [interface_no_param] [interface_instance]
[interface_port]
                       [prim_inst] [prim_mod]
                       [celldefine]
                       [architecture] [signal]
                       [wire] [property] [assert] [assume] [cover]
                       [parent] [empty_inst]
                      )+
                ]
                      [-verbosity (high | low | silent)]
                      [-typed_list]
                      [-silent]
                      [-file file_name [-force]]
                      [-filter filter_expression [-regexp] [-case_sensitive]]
                      [-gui]
                      [-include_hier_path]
                      [-pattern (sv_packed | sv_assignment)]
                      [-transitive]
```

```
"-module" Scope
---------------
get_design_info -module module_name
                [-list
                      ([parameter] [interface_port_parameter]
[interface_port_local_parameter]
                       [constant] [enum] [output] [input] [inout] [stopat]
                       [undriven] [undriven_internal] [loop_index] [multiple_driven]
                       [bbox_mod] [bbox_inst] [bbox_in] [bbox_out] [bbox_inout]
                       [bbox_in_instance_port] [bbox_out_instance_port]
                       [bbox_inout_instance_port]
                       [basic] [counter] [fsm] [fifo] [array [-no_aggregate]]
                       [register] [flop] [latch] [gate]
                       [multiplier] [divider] [modulus]
                       [module] [module_no_param] [module_name] [module_name_no_param]
                       [instance [-no_param_match_default_only]]
                       [instance_include_bbox [-no_param_match_default_only]]
                       [bind_instance]
                       [interface] [interface_no_param] [interface_instance]
[interface_port]
                       [prim_inst] [prim_mod]
                       [celldefine]
                       [architecture] [signal] [wire]
                       [property] [assert] [assume] [cover]
                      )+
                ]
                      [-verbosity (high | low | silent)]
                      [-typed_list]
                      [-silent]
                      [-file file_name [-force]]
                      [-filter filter_expression [-regexp] [-case_sensitive]]
                      [-gui]
                      [-include_hier_path]
                      [-pattern (sv_packed | sv_assignment)]

"-interface_instance" Scope
---------------------------
get_design_info -interface_instance interface_instance_name
                -list ([interface_name] [interface_name_no_param]
                                [parameter] [local_parameter]
                               )+
                      [-typed_list]
                      [-silent]
```

```
                    [-file file_name [-force]]
                    [-pattern (sv_packed | sv_assignment)]
                    [-transitive]

"-interface_port" Scope
-----------------------
get_design_info -interface_port interface_port_name
                 -list ([interface_name] [interface_name_no_param]
                                [parameter] [local_parameter]
                               )+
                    [-typed_list]
                    [-silent]
                    [-file file_name [-force]]
                    [-pattern (sv_packed | sv_assignment)]
                    [-transitive]

"-interface" Scope
------------------
get_design_info -interface interface_name
                 -list
                      ([interface_instance] [interface_port]
                       [parameter]
                      )+
                      [-typed_list]
                      [-silent]
                      [-file file_name [-force]]
                      [-filter filter_expression [-regexp] [-case_sensitive]]
                      [-pattern (sv_packed | sv_assignment)]
"-package" Scope
----------------
get_design_info -package package_name
                [-list
                      ([constant] [enum] [local_parameter] [signal]
                      )+
                ]
                      [-silent]

"-compilation_unit" Scope
-------------------------
get_design_info -compilation_unit compilation_unit_name
                -list
                      ([parameter] [local_parameter])+
                      [-typed_list]
```

```
                    [-silent]
                    [-file file_name [-force]]
                    [-filter filter_expression [-regexp] [-case_sensitive]]
                    [-pattern (sv_packed | sv_assignment)]

"-signal" Scope
---------------
get_design_info -signal signal_list
                [-list
                      ([attribute] [basic] [counter] [fsm] [fifo] [array [-
no_aggregate]]
                       [register [-fanout]] [flop [-fanout]] [latch [-fanout]]
                       [gate] [input]
                       [undriven] [undriven_internal] [loop_index]
                       [bbox_inout] [bbox_out]
                       [special_values] [multiple_driven]
                       [module] [instance] [bind_instance] [signal] [wire]
                       [parent]
                      )+
                ]
                      [-verbosity (high | low | silent)]
                      [-typed_list]
                      [-silent]
                      [-file file_name [-force]]
                      [-filter filter_expression [-regexp] [-case_sensitive]]
                      [-gui]

"-property" Scope
-----------------
get_design_info -property property_name+ [-regexp]
                [-list
                      ([attribute] [basic] [counter] [fsm] [fifo] [array [-
no_aggregate]]
                       [register] [flop] [latch] [gate] [input]
                       [undriven] [undriven_internal] [loop_index]
                       [multiplier] [divider] [modulus]
                       [bbox_inout] [bbox_out] [multiple_driven]
                       [module] [instance] [signal] [wire]
                       [property] [assert] [assume] [cover] [stopat]
                       [property_flop_bit]
                      )+
                ]
                      [-verbosity (high | low | silent)]
```

```
                    [-typed_list]
                    [-silent]
                    [-file file_name [-force]]
                    [-filter filter_expression [-regexp] [-case_sensitive]]
                    [-gui]

"-task" Scope
-------------
get_design_info -task task_name+ [-compute_all]
                [-list
                      ([basic] [counter] [fsm] [fifo] [array [-no_aggregate]]
                       [register] [flop] [latch] [gate] [input]
                       [undriven] [undriven_internal] [loop_index]
                       [module] [instance] [signal] [wire]
                       [property] [assert] [assume] [cover]
                      )+
                ]
                      [-verbosity (high | low | silent)]
                      [-typed_list]
                      [-silent]
                      [-file file_name [-force]]
                      [-filter filter_expression [-regexp] [-case_sensitive]]
                      [-gui]

Counter, FSM, and FIFO Detection
--------------------------------
get_design_info -signal signal_list
                 -is_counter | -is_fsm | -is_fifo
                  [-verbosity high]

Cached-Info Clearing
--------------------
get_design_info -clear

Depth Calculation
-----------------
get_design_info (-signal name+ |-property name+)
                  -list (flop | latch | register)
                  -depth N
                  [-boundary]
                  [-silent]
```

Returns

An integer, signal list, or true/false.

Detailed Description

Design information is a feature that gives insight into the formal computational complexity of the design. Access this feature through the GUI interface or the "get_design_info" Tcl command.

This command provides various structural and functional information for a specific scope within the given design. It provides:

- Structural metrics - Including the number of gates, flops, and so forth

- Functional metrics - Including the set of counters, any special values of the counters, the set of finite-state machines, and so forth

In addition, for modules and instances, this command provides:

- Module source information - Including the number of RTL lines, instances, and embedded properties.

- Current task information - Including the number of stopats in the current task.

See the command reference manual appendix "get_design_info Transitivity" for a list of the transitivity of "get_design_info" scopes: top instance (no scope switch), "-instance", and "-module". This appendix also identifies limitations and exceptions. Access the manual from the menu option Help - Command Reference.

  NOTE: This command reports stopats that meet these requirements:
  . In the current task
  . In the specified scope
  . In the COI of properties in the current task and scope
  For a more general list of stopats, use the "stopat -list" command (help stopat).

Important information about this command:
. You can specify the scope (focus) for which this information is provided. For example, you might want to get the list of flops in the transitive fanin of all the properties in a task or the list of counters feeding to an output of

```
  the design. You can also control the exact set of information provided
  through options.
. If you do not specify arguments, this command returns information for the top
  instance.
. The "-list" switch supports multiple enum types in a single Tcl command, but
  you must supply at least one enum. For example, you can specify:
  get_design_info -instance myInstance -list output input
  In this example, the command returns the analysis information for outputs and
  inputs. And the return value (the single line at the end) is a list of output
  names followed by input names. Use "-typed_list" to add a type element before
  each of the requested lists.
. Use the command "set_design_exploration_include_clock_logic" to control the
  logic the tool traverses when gathering information.
. See the section "Customizing Lists" for switches that refine your list, for
  example, for scripting.


Top-Instance Scope
==================
no arguments
    If you do not specify arguments, this command returns the following
    statistics for the top instance:
    . Instance source info, such as the number of RTL instances, embedded
      properties by type, and the number of lines of code
    . Instance structural metrics such as the number of ports, registers, and
      gates
    The output of this command is the sum of all registers (flops and
    latches).

-list ([parameter] [local_parameter] [interface_port_parameter]
       [interface_port_local_parameter] [constant] [enum]
       [output] [input] [inout] [stopat]
       [undriven] [undriven_internal] [loop_index] [multiple_driven]
       [bbox_mod] [bbox_inst] [bbox_in] [bbox_out] [bbox_inout]
       [bbox_in_instance_port] [bbox_out_instance_port]
       [bbox_inout_instance_port]
       [basic] [counter] [fsm] [fifo] [array [-no_aggregate]]
       [register] [flop] [latch] [gate]
       [multiplier] [divider] [modulus]
       [module] [module_no_param] [module_name] [module_name_no_param]
       [instance [-depth N]]
       [bind_instance [-depth N]]
       [interface] [interface_no_param] [interface_instance] [interface_port]
```

```
   [prim_inst] [prim_mod]
   [celldefine]
   [package] [compilation_unit]
   [architecture] [signal] [wire] [property]
   [assert] [assume] [cover] [macro])+
      [-verbosity (high | low | silent)]
      [-typed_list]
      [-silent]
      [-file file_name [-force]]
      [-filter filter_expression [-regexp] [-case_sensitive]]
      [-gui]
      [-pattern (sv_packed | sv_assignment)]
      [-transitive]
Get design information for the top instance.

Important things to know:
. This analysis ignores any configurations such as stopats,
  abstractions, and replace drivers.
. Below you will find definitions of the supported "-list" enums for the
  top-instance scope.
. The top-instance scope includes the top-instance itself or the top
  instance's hierarchy (the top instance and its subinstances) depending on
  the selected enum.
. See the section "Customizing Lists" for switches that refine your list,
  for example, for scripting.

Supported "-list" Enums for the Top-Instance Scope
--------------------------------------------------
. parameter: Get the parameters listed in the top instance.
    This command returns a string of parameter-value pairs.
    Example: [param1] [value1] [param2] [value2].

. local_parameter: List the local parameters in the top instance. The
    tool returns a string of parameter-value pairs. Example:
    [param1] [value1] [param2] [value2].

. interface_port_parameter: List the interface port parameters in the top
    instance. The tool returns a string of parameter-value pairs. Example:
    [param1] [value1] [param2] [value2].

. interface_port_local_parameter: List the interface port local parameters
    in the top instance. The tool returns a string of parameter-value pairs.
    Example: [param1] [value1] [param2] [value2].
```

. constant: Get the VHDL constants declared in the top instance. The tool
  returns a string of constant-value pairs. Example:
  [const1] [value1] [const2] [value2].

. enum: List the enum variables declared under the top instance's hierarchy,
  displaying their names followed by their mnemonics and their values.
  This command returns a list of space-separated enum names.
  Example: [enum1] [enum2] [enum3].

. output: List the names of outputs of the top instance and their sizes.
  . This command returns a list of space-separated output names.
  . Use "-verbosity high" to also list the number of flops and gates in
    their COI and minimum and maximum depth. (For this command, depth
    means the number of visible gates the tool must traverse to arrive
    from the output into a primary input.)

. input: List the names of inputs of the top instance and their sizes.
  . Inputs can be PIs, signals with hard stopats, or output
    signals of instance stopats.
  . This command returns a list of space-separated input names.

. inout: List the names of inouts of the top instance and their sizes.
  This command returns a list of space-separated inout names.

. stopat: List all the stopats under the top instance hierarchy and in
  the current task.
  NOTE: This command reports stopats that meet these requirements:
  . In the current task
  . In the COI of properties in the current task and under the top
    instance scope
  For a more general list of stopats, use the "stopat -list" command
  (help stopat -gui).

. undriven: List the names and sizes of all undriven signals under the top
  instance's hierarchy.

  This command returns a list of space-separated names of undriven
  signals.

. undriven_internal: List the names and sizes of all undriven signals under
  the top instance's hierarchy that are not primary inputs.

This command returns a list of space-separated names of undriven internal signals.

. loop_index: List the names and sizes of all indexes of generate for loops under the top instance's hierarchy.

  This command returns a list of space-separated names of loop index signals.

. multiple_driven: List the names and sizes of all multiple-driven signals or high impedance sources that are under the top instance's hierarchy.

  Not all signals returned by this command have more than one driver. They can be a source of high impedance such as z-assignments (1'bz), tri-states, bufifs, and mos gates (nmos, pmos, cmos).

  This command returns a list of space-separated names of multiple-driven or high impedance source signals.

. bbox_mod: List all modules that have been blackboxed under the top instance's hierarchy.

. bbox_inst: List all instances that have been blackboxed under the top instance's hierarchy.

  NOTE: The list contains other blackboxed types, for example, arrays and multipliers, but it does not identify the instance type.

. bbox_in
  bbox_out
  bbox_inout
  List the names and sizes of the signals connected to the specified kind of port (input, output, or inout) of blackboxed modules under the top instance's hierarchy.
  (Ports that behave as both input and output ports are inout ports.)
  This command returns a list of space-separated signal names.

. bbox_in_instance_port
  bbox_out_instance_port
  bbox_inout_instance_port
  List triples of signals connected to the specified kind of blackboxed ports (input, output, or inout), blackboxed instance names, and blackboxed connection ports for blackboxed modules under the top

```
    instance's hierarchy.
    This command returns a list of triples.

. basic: List the "default" set of information for the top-instance scope.
    . The Tcl command returns the number of flops in the top-instance
      scope.
    . The command "get_design_info -list basic" has the same effect as
      "get_design_info" with no additional switches.

. counter: Identify counters in the vector of flops detected under the top
    instance's hierarchy.
    . In addition to listing the names of flops, this command can print the
      following information when you use the "-verbosity high" switch:
      . Number of bits - flop width.
      . Size logic - number of gates inside the flop's COI.
      . Max depth - maximum number of flops from the original flop to a
        primary input.
      . Special values - values that are critical to the counter.
      . Direct flops - list of flops directly connected to the original
        flop. This list contains flop types (FSM, COUNTER, or OTHER).
      . COI flops - list of flops in the COI of the original flop.
    . This command returns a list of space-separated counter names.

. fsm: Identify FSMs in the vector of flops detected under the top
    instance's hierarchy.
    . In addition to listing the names of flops, this command can print the
      following information when you use the "-verbosity high" switch:
      . Number of bits - flop width.
      . Size logic - number of gates inside the flop's COI.
      . Max depth - maximum number of flops from the original flop to a
        primary input.
      . Special values - values that are critical to the FSM.
      . Direct flops - list of flops directly connected to the original
        flop. This list contains flop types (FSM, COUNTER, or OTHER).
      . COI flops - list of flops in the COI of the original flop.
    . This command returns a list of space-separated FSM names.

. fifo: Identify FIFO memories (multidimensional arrays) under the top
    instance's hierarchy.
    . In addition to listing the names of FIFO memories, this command can
      print the following information when you use the "-verbosity high"
      switch:
      . Number of bits - flop width.
```

```
      . Size logic - number of gates inside the flop's COI.
      . Max depth - maximum number of flops from the original flop to a
        primary input.
      . FIFO interface - clocks, resets, enables, access pointers, and data
        signals on input (write) and output (read) sides of the FIFO,
        in addition to associated full and empty signals.
      . Direct flops - list of flops directly connected to the original
        flop. This list contains flop types (FSM, COUNTER, or OTHER).
      . COI flops - list of flops in the COI of the original flop.
    . This command returns a list of space-separated FIFO memory names.

. array [-no_aggregate]: Identify multidimensional arrays (including FIFOs)
    in the vector of flops detected under the top instance's hierarchy.
    . Use "-no_aggregate" to get a bit-blasted list.
    . In addition to listing the names of flops, this command can print the
      following information when you use the "-verbosity high" switch:
      . Number of bits - flop width.
      . Size logic - number of gates inside the flop's COI.
      . Max depth - maximum number of flops from the original flop to a
        primary input.
      . Direct flops - list of flops directly connected to the original
        flop. This list contains flop types (FSM, COUNTER, or OTHER).
      . COI flops - list of flops in the COI of the original flop.
    . This command returns a list of space-separated array names.

. register: List all flops and latches under the top instance's hierarchy,
    displaying their names and sizes (in bits).
    This command returns a list of space-separated flop and latch names.

. flop: List all flops under the top instance's hierarchy, displaying their
    names and sizes (in bits).
    . The list contains flop numbers shown as x(y)(z property flop bits).
      . x is the total number of flops.
      . y is the sum of bits of the flops, not including property flop
        bits.
      . z is the sum of bits of property flops, which are generated as
        observers for SVA and PSL properties.
    . This command returns a list of space-separated flop names.

. latch: List all latches under the top instance's hierarchy, displaying
    their names and sizes (in bits).
    This command returns a list of space-separated latch names.
```

```
. gate: List the number of gates and gate bits under the top instance's
    hierarchy.
    This command returns the number of gates.

    Extended information:
    Gate count relates to combinational logic. Flops and latches are not
    included in the gate count. The count is signal level, and it takes
    into account wide gates.

    Gate bits are bit level, and it is the sum of all bits of all
    combinational gates.

    Usually, for estimating size, you will be interested in bit level
    because it is a better way to estimate proof complexity, and it is
    closer to the real synthesis size (bitwise).

    Example:
    In the example below, we have 1 flop, width 3 bits. Also, we have 2
    combinational gates, width 2. In this case, the sum is 4 bits:

    Gates = 2 and Gate bits = 4

    module reg_example(input wire clk,
             input wire rst,
             input wire [2:0] d,
             output reg [2:0] q);

      always @ (posedge clk or negedge rst)
      begin
        if (~rst) begin
            q <= 3'b0;
        end
        else begin
            q <= d;
        end
      end

    endmodule

    [<embedded>] get_design_info
    Statistics [for instance "reg_example"]
    ---------------------------
    # Flops:         1 (3) (0 property flop bits)
```

```
    # Latches:       0 (0)
    # Gates:         2 (4)
    # Nets:          13
    # Ports:         4
    # RTL Lines:     16
    # RTL Instances: 1
    # Embedded Assumptions: 0
    # Embedded Assertions:  0
    # Embedded Covers:      0

. multiplier
  divider
  modulus
    List all multipliers, dividers, or modulus under the top instance's
    hierarchy.
    This command lists two kinds of multipliers, dividers, or modulus:
    1. Blackboxed - each multiplier, divider, or modulus is described by
       three signals:
       {out_internal left_internal right_internal}
       The right and left internal signals are the operands, and
       "out_internal" is the resulting signal.
    2. Cell-defined - each multiplier, divider, or modulus that has not
       been blackboxed is cell defined. By default, all multipliers,
       dividers, and modulus are blackboxed; however, you can use
       "elaborate -bbox_mul", "-bbox_div", and "-bbox_mod" to avoid black
       boxing them.
    . Each multiplier, divider, and modulus is described by three signals:
      {out_internal left_internal right_internal}
    . The right and left internal signals are the operands, and
      "out_internal" is the resulting signal.
    . This functionality is useful for manual abstraction.
    . This command returns a list of space-separated triples of signals.

. module: List all non-blackboxed modules under the top instance's
    hierarchy, displaying their names. This command returns a list of
    space-separated module names.

. module_no_param: List all non-blackboxed modules under the top instance's
    hierarchy, displaying their names without the non-default parameter
    values. This command returns a list of space-separated module names.

. module_name: Return the module name for the top instance.
```

. module_name_no_param: Return the module name for the top instance without the non-default parameter values.

. instance: List all instances under the top instance's hierarchy, displaying their names. This command returns a list of space-separated instance names. The list does not contain blackboxed instances (see "bbox_inst").

  Use "-depth" to filter the list of instances using the depth level, listing the instances until the chosen depth. Depth 0 is the top instance.

. bind_instance: List all bind instances under the top instance's hierarchy, displaying their names. This command returns a list of space-separated bind instance names. The list does not contain blackboxed instances (see "bbox_inst").

  Use "-depth" to filter the list of bind instances using the depth level, listing the bind instances until the chosen depth. Depth 0 is the top bind instance.

. interface: Use this command to list all SystemVerilog Interfaces used under the top instance's hierarchy, displaying their names. This command returns a list of space-separated interface identifier names.

. interface_no_param: Use this command to list all SystemVerilog Interfaces used under the top instance's hierarchy, displaying their names without the non-default parameter values. This command returns a list of space-separated interface identifier names.

. interface_instance: Use this command to list all SystemVerilog Interface instances under the top instance's hierarchy, displaying their names. This command returns a list of space-separated interface instance names.

. interface_port: Use this command to list all SystemVerilog Interfaces used as ports under the top instance's hierarchy, displaying their names. This command returns a list of space-separated interface port names.

. prim_inst: List all user-declared primitive instances under the top instance's hierarchy, displaying their names. This command returns a list of space-separated instance names.

  NOTE: This enum requires "set_library_cell_detection 0".

```
. prim_mod: List all user-declared primitive modules under the top
    instance's hierarchy, displaying their names. This command returns a
    list of space-separated module names.

    NOTE: This enum requires "set_library_cell_detection 0".

. celldefine: List all modules declared as "celldefine" under  the top
    instance's hierarchy, displaying their names. This command returns a
    list of space-separated module names.

. package: List used packages, displaying their names.
    This command returns a list of space-separated package names.

. compilation_unit: List all compilation units, displaying their names.
    This command returns a list of space-separated compilation unit names.

    IMPORTANT: This command does not return compilation units that only
    contain module declarations.

. architecture: List all architectures under the top instance's hierarchy,
    displaying their names. This command returns a list of space-separated
    architecture names.

. signal: List all signals under the top instance's hierarchy, displaying
    their names and sizes in bits.
    . Signals are the set of all registers, primary inputs, primary
      outputs, and wires (signals = registers + PIs + POs + wires).
    . This command returns a list of space-separated signal names.

. wire: List all wires under the top instance's hierarchy, displaying their
    names and sizes in bits.
    . Wires are all the signals not classified as registers, primary
      inputs, or primary outputs (wires = signals - registers - PIs - POs).
    . This command returns a list of space-separated signal names.

. property
  assert
  assume
  cover
    List the current task's embedded and Tcl properties of the specified
    type.
```

```
        NOTE:
        . The list includes embedded and Tcl properties, embedded and
          Tcl assertions, embedded and Tcl assumptions, or embedded and Tcl
          cover directives in the top-instance scope.
        . The list does not include assertions extracted by the apps, for
          example, X-Prop and Connectivity assertions.
        . The list does not include global assumptions specified with "assume
          -env".

        This command lists the number and names of properties and returns a
        list of space-separated property names.

    . macro: List macros in the elaborated design.

        NOTE: This enum includes macros defined outside of the top instance.

        This command returns a list of space-separated macro names with their
        values.

-include_config [-instance instance_name]
    Get design information, taking into account changes made to the design
    such as hard stopats and instance stopats.

    Use "-instance" to filter the statistics for the specified instance.

    The output of this command is the sum of all registers (flops and
    latches) not disabled by stopats.

    NOTE:
    . This command traverses clock logic by default, even without running
      "set_design_exploration_include_clock_logic on".
    . This command takes into account instances connected to the main design
      using "connect".

-enum enum_name
    Get a list of all labels and values for the specified enum variable. This
    command lists labels and their values in pairs with single-space separation
    as shown below:

    label_1 value_1 ... label_n value_n

"-instance" Scope
=================
```

```
-instance instance_name
               [-list
                     ([parameter] [local_parameter] [interface_port_parameter]
                      [interface_port_local_parameter] [constant] [enum]
                      [output] [input] [inout] [stopat]
                      [undriven] [undriven_internal] [loop_index] [multiple_driven]
                      [bbox_mod] [bbox_inst] [bbox_in] [bbox_out] [bbox_inout]
                      [bbox_in_instance_port] [bbox_out_instance_port]
                      [bbox_inout_instance_port]
                      [basic] [counter] [fsm] [fifo] [array [-no_aggregate]]
                      [register [-depth N]] [flop [-depth N]] [latch [-depth N]]
                      [gate]
                      [multiplier] [divider] [modulus]
                      [module] [module_no_param] [module_name] [module_name_no_param]
                      [instance [-depth N]] [bind_instance [-depth N]]
                      [interface] [interface_no_param] [interface_instance]
[interface_port]
                      [prim_inst] [prim_mod]
                      [celldefine]
                      [architecture] [signal]
                      [wire] [property] [assert] [assume] [cover]
                      [parent] [empty_inst]
                     )+
               ]
                     [-verbosity (high | low | silent)]
                     [-typed_list]
                     [-silent]
                     [-file file_name [-force]]
                     [-filter filter_expression [-regexp] [-case_sensitive]]
                     [-gui]
                     [-pattern (sv_packed | sv_assignment)]
                     [-transitive]
```

Get design information for the specified instance.

Important things to know:

- This analysis ignores any configurations such as stopats, abstractions, and replace drivers.
- Below you will find definitions of the supported "-list" enums for the "-instance" scope.
- The "-instance" scope includes the selected instance itself or the selected instance's hierarchy (the instance and its subinstances) depending on the selected enum.
- See the section "Customizing Lists" for switches that refine your list,

```
  for example, for scripting.

Supported "-list" Enums for the "-instance" Scope
-------------------------------------------------
. parameter: Get the parameters listed in the specified instance.
    This command returns a string of parameter-value pairs.
    Example: [param1] [value1] [param2] [value2].

. local_parameter: Get the local parameters listed in the specified instance.
    The tool returns a string of parameter-value pairs.
    Example: [param1] [value1] [param2] [value2].

. interface_port_parameter: List the interface port parameters in the
    specified instance. The tool returns a string of parameter-value pairs.
    Example: [param1] [value1] [param2] [value2].

. interface_port_local_parameter: List the interface port local parameters
    in the specified instance. The tool returns a string of parameter-value
    pairs. Example: [param1] [value1] [param2] [value2].

. constant: Get the VHDL constants declared in the specified instance. The
    tool returns a string of constant-value pairs. Example:
    [const1] [value1] [const2] [value2].

. enum: List the enum variables declared in the specified instance's scope,
    displaying their names followed by their mnemonics and their values.
    This command returns a list of space-separated enum names.
    Example: [enum1] [enum2] [enum3].

. output: List the names of outputs of the selected instance and their sizes.
    . This command returns a list of space-separated output names.
    . Use "-verbosity high" to also list the number of flops and gates in
      their COI and minimum and maximum depth. (For this command, depth
      means the number of visible gates the tool must traverse to arrive
      from the output into a primary input.)

. input: List the names of inputs of the selected instance and their sizes.
    . Inputs can be PIs, signals with hard stopats, or output
      signals of instance stopats.
    . This command returns a list of space-separated input names.

. inout: List the names of inouts of the selected instance and their sizes.
    This command returns a list of space-separated inout names.
```

```
. stopat: List all the stopats that are under the selected instance's
    hierarchy and in the current task.
    NOTE: This command reports stopats that meet these requirements:
    . In the current task
    . In the COI of properties in the current task and under the selected
      instance's scope
    For a more general list of stopats, use the "stopat -list" command
    (help stopat -gui).

. undriven: List the names and sizes of all undriven signals under the
    specified instance's hierarchy.

    This command returns a list of space-separated names of undriven
    signals.

. undriven_internal: List the names and sizes of all undriven signals under
    the specified instance's hierarchy that are not primary inputs.

    This command returns a list of space-separated names of undriven
    internal signals.

. loop_index: List the names and sizes of all indexes of generate for loops
    under the specified instance's hierarchy.

    This command returns a list of space-separated names of loop index
    signals.

. multiple_driven: List the names and sizes of all multiple-driven signals
    or high impedance sources under the specified instance's hierarchy.

    Not all signals returned by this command have more than one driver.
    They can be a source of high impedance such as z-assignments (1'bz),
    tri-states, bufifs, and mos gates (nmos, pmos, cmos).

    This command returns a list of space-separated names of multiple-driven
    or high impedance source signals.

. bbox_mod: List all modules that have been blackboxed under the
    specified instance's hierarchy.

. bbox_inst: List all instances that have been blackboxed under the
    specified instance's hierarchy.
```

NOTE: The list includes other blackboxed types, for example, arrays and multipliers, but it does not identify the instance type.

. bbox_in
bbox_out
bbox_inout
List the names and sizes of the signals under the specified instance's hierarchy connected to the specified kind of port (input, output, or inout) of blackboxed modules.
(Ports that behave as both input and output ports are inout ports.)
. This command returns a list of space-separated signal names.
. This analysis ignores any configurations such as stopats, abstractions, and replace drivers.

. bbox_in_instance_port
bbox_out_instance_port
bbox_inout_instance_port
List triples of signals connected to the specified kind of blackboxed ports (input, output, or inout), blackboxed instance names, and blackboxed connection ports for blackboxed modules under the specified instance's hierarchy.
This command returns a list of triples.

. basic: List the "default" set of information for the "-instance" scope.
. The Tcl command returns the number of flops in the "-instance" scope.
. The command "get_design_info -instance instance_name -list basic" has the same effect as "get_design_info -instance instance_name" with no additional switches.

. counter: Identify counters in the vector of flops detected under the selected instance's hierarchy.
. In addition to listing the names of flops, this command can print the following information when you use the "-verbosity high" switch:
. Number of bits - flop width.
. Size logic - number of gates inside the flop's COI.
. Max depth - maximum number of flops from the original flop to a primary input.
. Special values - values that are critical to the counter.
. Direct flops - list of flops directly connected to the original flop. This list includes flop types (FSM, COUNTER, or OTHER).
. COI flops - list of flops in the COI of the original flop.

```
    . The Tcl command returns a list of space-separated counter names.

. fsm: Identify FSMs in the vector of flops detected under the selected
    instance's hierarchy.
    . In addition to listing the names of flops, this command can print the
      following information when you use the "-verbosity high" switch:
      . Number of bits - flop width.
      . Size logic - number of gates inside the flop's COI.
      . Max depth - maximum number of flops from the original flop to a
        primary input.
      . Special values - values that are critical to the FSM.
      . Direct flops - list of flops directly connected to the original
        flop. This list includes flop types (FSM, COUNTER, or OTHER).
      . COI flops - list of flops in the COI of the original flop.
    . This command returns a list of space-separated FSM names.

. fifo: Identify FIFO memories (multidimensional arrays) under the selected
    instance's hierarchy.
    . In addition to listing the names of FIFO memories, this command can
      print the following information when you use the "-verbosity high"
      switch:
      . Number of bits - flop width.
      . Size logic - number of gates inside the flop's COI.
      . Max depth - maximum number of flops from the original flop to a
        primary input.
      . FIFO interface - clocks, resets, enables, access pointers, and data
        signals on input (write) and output (read) sides of the FIFO,
        in addition to associated full and empty signals.
      . Direct flops - list of flops directly connected to the original
        flop. This list includes flop types (FSM, COUNTER, or OTHER).
      . COI flops - list of flops in the COI of the original flop.
    . This command returns a list of space-separated FIFO memory names.

. array [-no_aggregate]: Identify multidimensional arrays (including FIFOs)
    in the vector of flops detected under the selected instance's hierarchy.
    . Use "-no_aggregate" to get a bit-blasted list.
    . In addition to listing the names of flops, this command can print the
      following information when you use the "-verbosity high" switch:
      . Number of bits - flop width.
      . Size logic - number of gates inside the flop's COI.
      . Max depth - maximum number of flops from the original flop to a
        primary input.
      . Direct flops - list of flops directly connected to the original
```

```
        flop. This list includes flop types (FSM, COUNTER, or OTHER).
      . COI flops - list of flops in the COI of the original flop.
    . This command returns a list of space-separated array names.

. register [-depth N]: List all flops and latches under the selected
    instance's hierarchy, displaying their names and sizes (in bits).

    Use "-depth" to filter the list of registers using the instance depth
    level, listing the registers until the chosen depth. Depth 0 is the top
    instance, and depth $ means all levels.

    This command returns a list of space-separated flop and latch names.

. flop: List all flops under the selected instance's hierarchy, displaying
    their names and sizes (in bits).

    Use "-depth" to filter the list of flops using the instance depth
    level, listing the flops until the chosen depth. Depth 0 is the top
    instance, and depth $ means all levels.

    . The list includes flop numbers shown as x(y)(z property flop bits).
      . x is the total number of flops.
      . y is the sum of bits of the flops, not including property flop
        bits.
      . z is the sum of bits of property flops, which are generated as
        observers for SVA and PSL properties.
    . This command returns a list of space-separated flop names.

. latch: List all latches under the selected instance's hierarchy,
    displaying their names and sizes (in bits).

    Use "-depth" to filter the list of latches using the instance depth
    level, listing the latches until the chosen depth. Depth 0 is the top
    instance, and depth $ means all levels.

    This command returns a list of space-separated latch names.

. gate: List the number of gates and gate bits under the selected instance's
    hierarchy. This command returns the number of gates.

. multiplier
  divider
  modulus
```

```
    List all multipliers, dividers, or modulus under the selected instance's
    hierarchy.
    This command lists two kinds of multipliers, dividers, or modulus:
    1. Blackboxed - each multiplier, dividers, or modulus is described by
       three signals:
       {out_internal left_internal right_internal}
       The right and left internal signals are the operands, and
       "out_internal" is the resulting signal.
    2. Cell-defined - each multiplier, divider, or modulus that has not
       been blackboxed is cell defined. By default, all multipliers,
       dividers, and modulus are blackboxed; however, you can use
       "elaborate -bbox_mul", "-bbox_div", and "-bbox_mod" to avoid black
       boxing them.
    . Each multiplier, divider, and modulus is described by three signals:
      {out_internal left_internal right_internal}
    . The right and left internal signals are the operands, and
      "out_internal" is the resulting signal.
    . This functionality is useful for manual abstraction.
    . This command returns a list of space-separated triples of signals.

. module: List all non-blackboxed modules under the selected instance's
    hierarchy, displaying their names. This command returns a list of
    space-separated module names.

. module_no_param: List all non-blackboxed modules under the selected
    instance's hierarchy, displaying their names without the non-default
    parameter values. This command returns a list of space-separated module
    names.

. module_name: Return the module name for the specified instance.

. module_name_no_param: Return the module name for the specified instance
    without the non-default parameter values.

. instance: List all instances under the selected instance's hierarchy,
    displaying their names.

    This command returns a list of space-separated
    instance names. The list does not include blackboxed instances
    (see "bbox_inst").

    Use "-depth" to filter the list of instances using the depth level,
    listing the instances until the chosen depth. Depth 0 is the selected
```

instance.

. bind_instance: List all bind instances under the selected instance's hierarchy, displaying their names.

  This command returns a list of space-separated bind instance names. The list does not include blackboxed instances (see "bbox_inst").

  Use "-depth" to filter the list of bind instances using the depth level, listing the bind instances until the chosen depth. Depth 0 is the top bind instance.

. interface: Use this command to list all SystemVerilog Interfaces used under the selected instance's hierarchy, displaying their names. This command returns a list of space-separated interface identifier names.

. interface_no_param: Use this command to list all SystemVerilog Interfaces used under the selected instance's hierarchy, displaying their names without the non-default parameter values. This command returns a list of space-separated interface identifier names.

. interface_instance: Use this command to list all SystemVerilog Interface instances under the selected instance's hierarchy, displaying their names. This command returns a list of space-separated interface instance names.

. interface_port: Use this command to list all SystemVerilog Interfaces used as ports under the selected instance's hierarchy, displaying their names. This command returns a list of space-separated interface port names.

. prim_inst: List all user-declared primitive instances under the selected instance's hierarchy, displaying their names. This command returns a list of space-separated instance names.

  NOTE: This enum requires "set_library_cell_detection 0".

. prim_mod: List all user-declared primitive modules under the selected instance's hierarchy, displaying their names. This command returns a list of space-separated module names.

  NOTE: This enum requires "set_library_cell_detection 0".

. celldefine: List all modules declared as "celldefine" under the selected instance's hierarchy, displaying their names. This command returns a

```
      list of space-separated module names.

   . architecture: List all architectures under the selected instance's
      hierarchy, displaying their names. This command returns a list of
      space-separated architecture names.

   . signal: List all signals under the selected instance's hierarchy,
      displaying their names and sizes in bits.
      . Signals are the set of all registers, primary inputs, primary
        outputs, and wires (signals = registers + PIs + POs + wires).
      . This command returns a list of space-separated signal names.

   . wire: List all wires under the selected instance's hierarchy, displaying
      their names and sizes in bits.
      . Wires are all the signals not classified as registers, primary
        inputs, or primary outputs (wires = signals - registers - PIs - POs).
      . This command returns a list of space-separated signal names.

   . property
     assert
     assume
     cover
       List the current task's embedded and Tcl properties of the specified
       type in the "-instance" scope.

       NOTE:
       . The list includes embedded and Tcl properties, embedded and
         Tcl assertions, embedded and Tcl assumptions, or embedded and Tcl
         cover directives in the "-instance" scope.
       . The list does not include assertions extracted by the apps, for
         example, X-Prop and Connectivity assertions.
       . The list does not include global assumptions specified with "assume
         -env".

       This command lists the number and names of properties and returns a
       list of space-separated property names.

   . parent: List the parent of the specified instance.

   . empty_inst: List the subinstances of the specified instance that have no
      internal logic, just ports.

"-module" Scope
```

```
===============
-module module_name
               [-list
                     ([parameter] [constant] [interface_port_parameter]
                      [interface_port_local_parameter] [enum]
                      [output] [input] [inout] [stopat]
                      [undriven] [undriven_internal] [loop_index] [multiple_driven]
                      [bbox_mod] [bbox_inst] [bbox_in] [bbox_out] [bbox_inout]
                      [bbox_in_instance_port] [bbox_out_instance_port]
                      [bbox_inout_instance_port]
                      [basic] [counter] [fsm] [fifo] [array [-no_aggregate]]
                      [register] [flop] [latch] [gate]
                      [multiplier] [divider] [modulus]
                      [module] [module_no_param] [module_name] [module_name_no_param]
                      [instance [-no_param_match_default_only]]
                      [instance_include_bbox [-no_param_match_default_only]]
                      [bind_instance]
                      [interface] [interface_no_param] [interface_instance]
[interface_port]
                      [prim_inst] [prim_mod]
                      [celldefine]
                      [architecture] [signal] [wire]
                      [property] [assert] [assume] [cover]
                     )+
               ]
                     [-verbosity (high | low | silent)]
                     [-typed_list]
                     [-silent]
                     [-file file_name [-force]]
                     [-filter filter_expression [-regexp] [-case_sensitive]]
                     [-gui]
                     [-pattern (sv_packed | sv_assignment)]
```

Get design information for the specified module.

Important things to know:

- This analysis ignores any configurations such as stopats, abstractions, and replace drivers.
- Below you will find definitions of the supported "-list" enums for the "-module" switch.
- See the section "Customizing Lists" for switches that refine your list, for example, for scripting.

Supported "-list" Enums for the "-module" Switch

```
-----------------------------------------------
. parameter: Get the parameters of the specified module. The tool returns
    a string of parameter-value pairs. If you pass the module's name to
    "-module" without parameters, the tool returns default values.
    Example: [param1] [value1] [param2] [value2].

. interface_port_parameter: Get the interface port parameters of the
    specified module. The tool returns a string of parameter-value pairs.
    Example: [param1] [value1] [param2] [value2].

. interface_port_local_parameter: Get the interface port local parameters of
    the specified module. The tool returns a string of parameter-value
    pairs. Example: [param1] [value1] [param2] [value2].

. constant: Get the VHDL constants declared in the specified module. The
    tool returns a string of constant-value pairs. Example:
    [const1] [value1] [const2] [value2].

. enum: List all enum variables declared in an instance of the specified
    module, displaying their names followed by their mnemonics and their
    values. This command returns a list of space-separated enum names.
    Example: [enum1] [enum2] [enum3].

. output: List the names of outputs and their sizes.
    . This command returns a list of space-separated output names.
    . Use "-verbosity high" to also list the number of flops and gates in
      their COI and minimum and maximum depth. (For this command, depth
      means the number of visible gates the tool must traverse to arrive
      from the output into a primary input.)

. input: List the number of inputs followed by their names and
    sizes.
    . Inputs can be PIs, signals with hard stopats, or output
      signals of instance stopats.
    . This command returns a list of space-separated input names.

. inout: List the number of inouts followed by their names and
    sizes.
    This command returns a list of space-separated inout names.

. stopat: List all the stopats that are in an instance of the specified
  module.
    NOTE: This command reports stopats that meet these requirements:
```

```
    . In the current task
    . In the specified module scope
    . In the COI of properties in the current task and in the specified
      module
    For a more general list of stopats, use the "stopat -list" command
    (help stopat -gui).

. undriven: List the names and sizes of all undriven signals in the scope
    of an instance of the specified module, including its subinstances.

    This command returns a list of space-separated names of undriven
    signals.

. undriven_internal: List the names and sizes of all undriven signals that
    are not primary inputs and that are under the hierarchy of an instance
    of the specified module.

    This command returns a list of space-separated names of undriven
    internal signals.

. loop_index: List the names and sizes of all indexes of generate for loops
    under the hierarchy of an instance of the specified module.

    This command returns a list of space-separated names of loop index
    signals.

. multiple_driven: List the names and sizes of all multiple-driven signals
    or high impedance sources under the hierarchy of an instance of the
    specified module.

    Not all signals returned by this command have more than one driver.
    They can be a source of high impedance such as z-assignments (1'bz),
    tri-states, bufifs, and mos gates (nmos, pmos, cmos).

    This command returns a list of space-separated names of multiple-driven
    or high impedance source signals.

. bbox_mod: List all modules that have been blackboxed under instances of
    the specified module.

. bbox_inst: List all instances that have been blackboxed under instances
    of the specified module.
```

NOTE: The list includes other blackboxed types, for example, arrays and multipliers, but it does not identify the instance type.

. bbox_in
bbox_out
bbox_inout
List the names and sizes of the signals connected to the specified kind of port (input, output, or inout) of blackboxed modules under instances of the specified module. (Ports that behave as both input and output ports are inout ports.)
This command returns a list of space-separated signal names.

. bbox_in_instance_port
bbox_out_instance_port
bbox_inout_instance_port
List triples of signals connected to the specified kind of blackboxed ports (input, output, or inout), blackboxed instance names, and blackboxed connection ports for blackboxed modules under instances of the specified module.
This command returns a list of triples

. basic: List the "default" set of information for the specified module.
. The Tcl command returns the number of flops in the specified module.
. The command "get_design_info -module module_name -list basic" has the same effect as "get_design_info -module module_name" with no additional switches.

. counter: Identify counters in the vector of flops detected under an instance of the specified module.
. In addition to listing the names of flops, this command can print the following information when you use the "-verbosity high" switch:
. Number of bits - flop width.
. Size logic - number of gates inside the flop's COI.
. Max depth - maximum number of flops from the original flop to a primary input.
. Special values - values that are critical to the counter.
. Direct flops - list of flops directly connected to the original flop. This list includes flop types (FSM, COUNTER, or OTHER).
. COI flops - list of flops in the COI of the original flop.
. The Tcl command returns a list of space-separated counter names.

. fsm: Identify FSMs in the vector of flops detected under an instance of the specified module.

```
      . In addition to listing the names of flops, this command can print the
        following information when you use the "-verbosity high" switch:
        . Number of bits - flop width.
        . Size logic - number of gates inside the flop's COI.
        . Max depth - maximum number of flops from the original flop to a
          primary input.
        . Special values - values that are critical to the FSM.
        . Direct flops - list of flops directly connected to the original
          flop. This list includes flop types (FSM, COUNTER, or OTHER).
        . COI flops - list of flops in the COI of the original flop.
      . The Tcl command returns a list of space-separated FSM names.

  . fifo: Identify FIFO memories (multidimensional arrays) under an instance
      of the specified module.
      . In addition to listing the names of FIFO memories, this command can
        print the following information when you use the "-verbosity high"
        switch:
        . Number of bits - flop width.
        . Size logic - number of gates inside the flop's COI.
        . Max depth - maximum number of flops from the original flop to a
          primary input.
        . FIFO interface - clocks, resets, enables, access pointers, and data
          signals on input (write) and output (read) sides of the FIFO,
          in addition to associated full and empty signals.
        . Direct flops - list of flops directly connected to the original
          flop. This list includes flop types (FSM, COUNTER, or OTHER).
        . COI flops - list of flops in the COI of the original flop.
      . This command returns a list of space-separated FIFO memory names.

  . array [-no_aggregate]: Identify multidimensional arrays (including FIFOs)
      in the vector of flops detected under an instance of the specified
      module.
      . Use "-no_aggregate" to get a bit-blasted list.
      . In addition to listing the names of flops, this command can print the
        following information when you use the "-verbosity high" switch:
        . Number of bits - flop width.
        . Size logic - number of gates inside the flop's COI.
        . Max depth - maximum number of flops from the original flop to a
          primary input.
        . Direct flops - list of flops directly connected to the original
          flop. This list includes flop types (FSM, COUNTER, or OTHER).
        . COI flops - list of flops in the COI of the original flop.
      . The Tcl command returns a list of space-separated array names.
```

```
. register: List all flops and latches under an instance of the specified
    module, displaying their names and sizes (in bits).
    The Tcl command returns a list of space-separated flop and latch names.

. flop: List all flops under an instance of the specified module, displaying
    their names and sizes (in bits).
    . The list includes flop numbers shown as x(y)(z property flop bits).
      . x is the total number of flops.
      . y is the sum of bits of the flops, not including property flop
        bits.
      . z is the sum of bits of property flops, which are generated as
        observers for SVA and PSL properties.
    . The Tcl command returns a list of space-separated flop names.

. latch: List all latches under an instance of the specified module,
    displaying their names and sizes (in bits).
    The Tcl command returns a list of space-separated latch names.

. gate: List the number of gates and gate bits under an instance of the
    specified module.
    This command returns the number of gates.

. multiplier
  divider
  modulus
    List all multipliers, dividers, or modulus in the specified module.
    This command lists two kinds of multipliers, dividers, or modulus:
    1. Blackboxed - each multiplier, dividers, or modulus is described by
       three signals:
       {out_internal left_internal right_internal}
       The right and left internal signals are the operands, and
       "out_internal" is the resulting signal.
    2. Cell-defined - each multiplier, divider, or modulus that has not
       been blackboxed is cell defined. By default, all multipliers,
       dividers, and modulus are blackboxed; however, you can use
       "elaborate -bbox_mul", "-bbox_div", and "-bbox_mod" to avoid black
       boxing them.
    . Each multiplier, divider, and modulus is described by three signals:
      {out_internal left_internal right_internal}
    . The right and left internal signals are the operands, and
      "out_internal" is the resulting signal.
    . This functionality is useful for manual abstraction.
```

```
    . This command returns a list of space-separated triples of signals.

. module: List all non-blackboxed modules under the instances of the
    specified module, displaying their names. This command returns a list of
    space-separated module names.

. module_no_param: List all non-blackboxed modules under the instances of
    the specified module, displaying their names without the non-default
    parameter values. This command returns a list of space-separated module
    names.

. module_name: Return the specified module name.

. module_name_no_param: Return the specified module name without the
    non-default parameter values.

. instance [-no_param_match_default_only]: List all non-blackboxed
    instances of the specified module, displaying their names.
    . If you use the switch "-no_param_match_default_only", the argument
      to the "-module" switch cannot have parameters, and the command matches
      only the instantiations of the given module in which parameters were not
      redefined.
    . The default behavior for a module name without parameters is to match
      all the instantiations of the given module.
    . This switch has no effect if the argument for "get_design_info
      -module" has parameters.
    . This command returns a list of space-separated instance names.
    . The list does not include blackboxed instances
      (see "instance_include_bbox").

. instance_include_bbox [-no_param_match_default_only]: List all instances
    of the specified module whether they are blackboxed or not, displaying
    their names.
    . If you use the switch "-no_param_match_default_only", the argument
      to the "-module" switch cannot have parameters, and the command matches
      only the instantiations of the given module in which parameters were not
      redefined.
    . The default behavior for a module name without parameters is to match
      all the instantiations of the given module.
    . This switch has no effect if the argument for "get_design_info
      -module" has parameters.
    . This command returns a list of space-separated instance names.
```

. bind_instance: List all bind instances of the specified module, displaying their names. This command returns a list of space-separated bind instance names. The list does not include blackboxed bind instances.

. interface: Use this command to list all SystemVerilog Interfaces used under the specified module, displaying their names. This command returns a list of space-separated interface identifier names.

. interface_no_param: Use this command to list all SystemVerilog Interfaces used under the specified module, displaying their names without the non-default parameter values. This command returns a list of space-separated interface identifier names.

. interface_instance: Use this command to list all SystemVerilog Interface instances under the specified module, displaying their names. This command returns a list of space-separated interface instance names.

. interface_port: Use this command to list all SystemVerilog Interfaces used as ports under the specified module, displaying their names. This command returns a list of space-separated interface port names.

. prim_inst: List all user-declared primitive instances under the instances of the specified module, displaying their names. This command returns a list of space-separated instance names.

  NOTE: This enum requires "set_library_cell_detection 0".

. prim_mod: List all user-declared primitive modules under the instances of the specified module, displaying their names. This command returns a list of space-separated module names.

  NOTE: This enum requires "set_library_cell_detection 0".

. celldefine: List all modules declared as "celldefine" under the instances of the specified module, displaying their names. This command returns a list of space-separated module names.

. architecture: List all architectures of the specified entity, displaying their names. This command returns a list of space-separated architecture names.

. signal: List all signals under an instance of the specified module,

```
        displaying their names and sizes in bits.
       . Signals are the set of all registers, primary inputs, primary
         outputs, and wires (signals = registers + PIs + POs + wires).
       . This command returns a list of space-separated signal names.

   . wire: List all wires under an instance of the specified module, displaying
       their names and sizes in bits.
       . Wires are all the signals not classified as registers, primary
         inputs, or primary outputs (wires = signals - registers - PIs - POs).
       . This command returns a list of space-separated signal names.

   . property
     assert
     assume
     cover
       List the current task's embedded properties of the specified type in
       the specified module.

       NOTE:
       . The list includes embedded and Tcl properties, embedded and
         Tcl assertions, embedded and Tcl assumptions, or embedded and Tcl
         cover directives in the "-module" scope.
       . The list does not include assertions extracted by the apps, for
         example, X-Prop and Connectivity assertions.
       . The list does not include global assumptions specified with "assume
         -env".

       This command lists the number and names of properties and returns a
       list of space-separated property names.

"-interface_instance" Scope
===========================
-interface_instance interface_instance_name
                -list ([interface_name] [interface_name_no_param]
                       [parameter] [local_parameter]
                      )+
                     [-typed_list]
                     [-silent]
                     [-file file_name [-force]]
                     [-pattern (sv_packed | sv_assignment)]
                     [-transitive]

   Get design information for the specified SystemVerilog Interface instance.
```

```
    Supported "-list" Enums for the "-interface_instance" Scope
    -----------------------------------------------------------
    . interface_name: Return the interface name for the specified SystemVerilog
        Interface instance.

    . interface_name_no_param: Return the interface name for the specified
        SystemVerilog Interface instance without the non-default parameter
        values.

    . parameter: Get the parameters listed in the specified SystemVerilog
        Interface instance. This command returns a string of parameter-value
        pairs.
        Example: [param1] [value1] [param2] [value2].

    . local_parameter: Get the local parameters listed in the specified
        SystemVerilog Interface instance. This command returns a string of
        parameter-value pairs.
        Example: [param1] [value1] [param2] [value2].

"-interface_port" Scope
=======================
-interface_port interface_port_name
                -list ([interface_name] [interface_name_no_param]
                       [parameter] [local_parameter]
                      )+
                     [-typed_list]
                     [-silent]
                     [-file file_name [-force]]
                     [-pattern (sv_packed | sv_assignment)]
                     [-transitive]

    Get design information for the specified SystemVerilog Interface port.

    Supported "-list" Enums for the "-interface_port" Scope
    -------------------------------------------------------
    . interface_name: Return the interface name for the specified SystemVerilog
        Interface port.

    . interface_name_no_param: Return the interface name for the specified
        SystemVerilog Interface port without the non-default parameter
        values.
```

```
    . parameter: Get the parameters listed in the specified SystemVerilog
       Interface port. This command returns a string of parameter-value pairs.
       Example: [param1] [value1] [param2] [value2].

    . local_parameter: Get the local parameters listed in the specified
       SystemVerilog Interface port. This command returns a string of
       parameter-value pairs.
       Example: [param1] [value1] [param2] [value2].

"-interface" Scope
==================
-interface interface_name
                -list
                     ([interface_instance] [interface_port]
                      [parameter]
                     )+
                     [-typed_list]
                     [-silent]
                     [-file file_name [-force]]
                     [-filter filter_expression [-regexp] [-case_sensitive]]
                     [-pattern (sv_packed | sv_assignment)]

    Get design information for the specified SystemVerilog Interface.

    NOTE: The tool supports SystemVerilog Interfaces that are only used as a
    top instance's port. However, unlike interfaces that are explicitly
    instantiated, in this context, the interface has no interface_instance.

    Supported "-list" Enums for the "-interface" Scope
    --------------------------------------------------
    . interface_instance: List all instances of the specified SystemVerilog
        Interface that were created by explicit instantiation, displaying their
        names.
        This command returns a list of space-separated interface instance names.

    . interface_port: List all instances of the specified SystemVerilog
        Interface that were created by using an interface instance as a port,
        displaying their names.
        This command returns a list of space-separated interface port names.

    . parameter: Get the parameters of the specified SystemVerilog Interface.
        This command returns a string of parameter-value pairs. If you pass the
        interface's name to "-interface" without parameters, the tool returns
```

```
        default values.
        Example: [param1] [value1] [param2] [value2].

"-package" Scope
================
-package package_name
               [-list
                     ([constant] [enum] [local_parameter] [signal]
                     )+
               ]
                     [-silent]
                     [-pattern (sv_packed | sv_assignment)]
    Get design information for the specified package.

    NOTE: By default, for all apps except SPV and SEC, design compilation
    supports signals in packages and the global area (that is, outside modules
    in SystemVerilog). SPV and SEC remove packages by default.

    Naming Rules:
    . Packages are named just as they are declared in the RTL.
    . If a package is inside a library, its name is prefixed with the library
      name (lib.pkg). Since this is not a valid design unit name, the tool
      escapes it according to the language mode ("\lib.pkg " for Verilog and
      "\lib.pkg\" for VHDL).
    . If the package name collides with an instance name under the "top" unit,
      the tool renames it with an incremental natural positive integer suffix
      (_1, _2, _3, and so forth).

    Supported "-list" Enums for the Package Scope
    ---------------------------------------------
    . constant: Get the VHDL constants declared in the specified package. The
        tool returns a string of constant-value pairs. Example:
        [const1] [value1] [const2] [value2].

    . enum: List all enum variables declared in the specified package's scope,
        displaying their names followed by their mnemonics and their values.
        This command returns a list of space-separated enum names.
        Example: [enum1] [enum2] [enum3].

    . local_parameter: Get the local parameters listed in the specified
        package. The tool returns a string of parameter-value pairs. Example:
        [param1] [value1] [param2] [value2].
```

```
    . signal: List all signals in the "-package" scope, displaying their names
        and sizes in bits.
        . Signals are the set of all registers, primary inputs, primary
          outputs, and wires (signals = registers + PIs + POs + wires).
        . This command returns a list of space-separated signal names.

"-compilation_unit" Scope
-------------------------
-compilation_unit compilation_unit_name
               -list
                     ([parameter] [local_parameter])+
                     [-typed_list]
                     [-silent]
                     [-file file_name [-force]]
                     [-filter filter_expression [-regexp] [-case_sensitive]]
                     [-pattern (sv_packed | sv_assignment)]

    Get design information for the specified compilation unit.

    Supported "-list" Enums for the "-compilation_unit" Scope
    ---------------------------------------------------------
    . parameter: Get the local parameters listed in the specified compilation
        unit. This command returns a string of parameter-value pairs. Example:
        [param1] [value1] [param2] [value2].

        NOTE: The compilation unit scope cannot have non-local parameters.
        That is, all parameters declared in this scope are local regardless of
        the use of the keyword "parameter" or "localparam". You can use this
        enum interchangeably with "local_parameter".

    . local_parameter: Get the local parameters listed in the specified
        compilation unit. This command returns a string of parameter-value
        pairs. Example:
        [param1] [value1] [param2] [value2].

"-signal" Scope
===============
-signal signal_list
               [-list
                     ([attribute] [basic] [counter] [fsm] [fifo] [array [-
no_aggregate]]
                      [register [-fanout]] [flop] [-fanout]] [latch [-fanout]]
                      [gate] [input]
```

```
                  [undriven] [undriven_internal] [loop_index]
                  [bbox_inout] [bbox_out]
                  [special_values] [multiple_driven] [module]
                  [instance] [bind_instance] [signal] [wire]
                  [parent]
                )+
          ]
                [-verbosity (high | low | silent)]
                [-typed_list]
                [-silent]
                [-file file_name [-force]]
                [-filter filter_expression [-regexp] [-case_sensitive]]
                [-gui]
Get design information for one or more specified signals.

Important things to know:
. Below you will find definitions of the supported "-list" enums for the
  "-signal" scope.
. See the section "Customizing Lists" for switches that refine your list,
  for example, for scripting.
. This scope supports multiple signals except in the case of
  "special_values", which supports a single signal.

Supported "-list" Enums for the "-signal" Scope
-----------------------------------------------
. attribute: List all SystemVerilog attributes in the "-signal" scope.
    . This command lists only SystemVerilog attributes.
    . This command returns a string of attribute-value pairs. Example:
      [attribute1] [value1] [attribute2] [value2].

. basic: List the "default" set of information for the "-signal" scope.
    . The Tcl command returns the number of flops in the "-signal"
      scope.
    . The command "get_design_info -signal signal_list -list basic" has the
      same effect as "get_design_info -signal signal_list" with no
      additional switches.

. counter: Identify counters in the vector of flops detected in the
    "-signal" scope.
    . In addition to listing the names of flops, this command can print the
      following information when you use the "-verbosity high" switch:
      . Number of bits - flop width.
      . Size logic - number of gates inside the flop's COI.
```

```
        . Max depth - maximum number of flops from the original flop to a
          primary input.
        . Special values - values that are critical to the counter.
        . Direct flops - list of flops directly connected to the original
          flop. This list includes flop types (FSM, COUNTER, or OTHER).
        . COI flops - list of flops in the COI of the original flop.
      . This command returns a list of space-separated counter names.

  . fsm: Identify FSMs in the vector of flops detected in the "-signal"
      scope.
      . In addition to listing the names of flops, this command can print the
        following information when you use the "-verbosity high" switch:
        . Number of bits - flop width.
        . Size logic - number of gates inside the flop's COI.
        . Max depth - maximum number of flops from the original flop to a
          primary input.
        . Special values - values that are critical to the FSM.
        . Direct flops - list of flops directly connected to the original
          flop. This list includes flop types (FSM, COUNTER, or OTHER).
        . COI flops - list of flops in the COI of the original flop.
      . This command returns a list of space-separated FSM names.

  . fifo: Identify FIFO memories (multidimensional arrays) in the "-signal"
      scope.
      . In addition to listing the names of FIFO memories, this command can
        print the following information when you use the "-verbosity high"
        switch:
        . Number of bits - flop width.
        . Size logic - number of gates inside the flop's COI.
        . Max depth - maximum number of flops from the original flop to a
          primary input.
        . FIFO interface - clocks, resets, enables, access pointers, and data
          signals on input (write) and output (read) sides of the FIFO,
          in addition to associated full and empty signals.
        . Direct flops - list of flops directly connected to the original
          flop. This list includes flop types (FSM, COUNTER, or OTHER).
        . COI flops - list of flops in the COI of the original flop.
      . This command returns a list of space-separated FIFO memory names.

  . array [-no_aggregate]: Identify multidimensional arrays (including FIFOs)
      in the vector of flops detected in the "-signal" scope.
      . Use "-no_aggregate" to get a bit-blasted list.
      . In addition to listing the names of flops, this command can print the
```

```
      following information when you use the "-verbosity high" switch:
      . Number of bits - flop width.
      . Size logic - number of gates inside the flop's COI.
      . Max depth - maximum number of flops from the original flop to a
        primary input.
      . Direct flops - list of flops directly connected to the original
        flop. This list includes flop types (FSM, COUNTER, or OTHER).
      . COI flops - list of flops in the COI of the original flop.
    . This command returns a list of space-separated array names.

. register: List all flops and latches in the "-signal" scope,
    displaying their names and sizes (in bits).
    . Use the "-fanout" switch with this argument to list all flops and
      latches in the fanout of the specified signal.
    . This command returns a list of space-separated flop and latch names.

. flop: List all flops in the "-signal" scope, displaying their names
    and sizes (in bits).
    . The list includes flop numbers shown as x(y)(z property flop bits).
      . x is the total number of flops.
      . y is the sum of flop bits connected to the specified signals,
        not including property flop bits.
      . z is the sum of bits of property flops, which are generated as
        observers for SVA and PSL properties.
    . Use the "-fanout" switch with this argument to list all flops in the
      fanout of the specified signal.
    . The list includes flop numbers shown as x(y).
      . x is the total number of flops.
      . y is the sum of bits connected to the specified signals.
    . This command returns a list of space-separated flop names.

. latch: List all latches in the "-signal" scope, displaying their
    names and sizes (in bits).
    . Use the "-fanout" switch with this argument to list all latches in
      the fanout of the specified signal.
    . This command returns a list of space-separated latch names.

. gate: List the number of gates and gate bits for the specified signal.
    This command returns the number of gates.

. input: List the number of inputs followed by their names and
    sizes.
    . Inputs can be PIs, signals with hard stopats, or output
```

```
      signals of instance stopats.
    . This command returns a list of space-separated input names.

. undriven: List the names and sizes of all undriven signals in the scope
    of the specified signals.

    This command returns a list of space-separated names of undriven
    signals.

. undriven_internal: List the names and sizes of all undriven signals that
    are not primary inputs and that are in the scope of the specified
    signals.

    This command returns a list of space-separated names of undriven
    internal signals.

. loop_index: List the names and sizes of all indexes of generate for loops
    in the scope of the specified signals.

    This command returns a list of space-separated names of loop index
    signals.

. multiple_driven: List the names and sizes of all multiple-driven signals
    or high impedance sources in the scope of the specified signals.

    Not all signals returned by this command have more than one driver.
    They can be a source of high impedance such as z-assignments (1'bz),
    tri-states, bufifs, and mos gates (nmos, pmos, cmos).

    This command returns a list of space-separated names of multiple-driven
    or high impedance source signals.

. bbox_inout: List the names and sizes of the inout ports of blackboxed
    instances that are in the scope of the specified signals.

    This command returns a list of space-separated signal names.

. bbox_out: List the names and sizes of the output ports of blackboxed
    instances that are in the scope of the specified signals.

    This command returns a list of space-separated signal names.

. special_values: List all counter and FSM special values detected.
```

```
      . Limitation: This argument applies to a single signal.
      . Each special value is listed as an integer between parentheses and
        a textual description, if one exists, for example, "(0) SOFA".
      . This command returns a list of space-separated special values.

    . module: List all modules in the "-signal" scope, displaying their
      names. This command returns a list of space-separated module names.

    . instance: List all instances in the "-signal" scope, displaying their
      names. This command returns a list of space-separated instance
      names. The list does not include blackboxed instances.

    . bind_instance: List all bind instances of the "-signal" scope, displaying
      their names. This command returns a list of space-separated bind
      instance names. The list does not include blackboxed bind instances

    . signal: List the specified signals.
      This command returns a list of space-separated signal names.

    . wire: List all wires in the "-signal" scope, displaying their names
      and sizes in bits.
      . Wires are all the signals not classified as registers, primary
        inputs, or primary outputs (wires = signals - registers - PIs - POs).
      . This command returns a list of space-separated signal names.

    . parent: List the parent instance of the specified signals.

"-property" Scope
=================
-property property_name+ [-regexp]
              [-list
                    ([attribute ][basic] [counter] [fsm] [fifo] [array [-
no_aggregate]]
                     [register] [flop] [latch] [gate] [input]
                     [undriven] [undriven_internal] [loop_index]
                     [bbox_inout] [bbox_out] [multiple_driven]
                     [module] [instance] [signal] [wire]
                     [property] [assert] [assume] [cover] [stopat]
                     [property_flop_bit]
                    )+
              ]
                    [-verbosity (high | low | silent)]
                    [-typed_list]
```

```
                  [-silent]
                  [-file file_name [-force]]
                  [-filter filter_expression [-regexp] [-case_sensitive]]
                  [-gui]
Get design information for one or more specified properties.

Important things to know about this command:
. This analysis considers hard stopats and instance stopats
  in the corresponding task.
. This switch supports wildcards and regular expressions in property names.
  When using a regular expression, use the "-regexp" switch after the
  pattern:
  get_design_info -property {pattern} -regexp
. Information available for more than one property is: number and list
  of flops, latches, registers and inputs (PIs and stopats).
. Below you will find definitions of the supported "-list" enums for the
  "-property" scope.
. See the section "Customizing Lists" for switches that refine your list,
  for example, for scripting.

Supported "-list" Enums for the "-property" Scope
-------------------------------------------------
. attribute: List all SystemVerilog attributes in the "-property" scope.
    . This command lists only SystemVerilog attributes.
    . This command returns a string of attribute-value pairs. Example:
      [attribute1] [value1] [attribute2] [value2].

. basic: List the "default" set of information for the "-property" scope.
    . The Tcl command returns the number of flops in the "-property"
      scope.
    . The command "get_design_info -property property_name -list basic" has
      the same effect as "get_design_info -property property_name" with no
      additional switches.

. counter: Identify counters in the vector of flops detected in the
    "-property" scope.
    . In addition to listing the names of flops, this command can print the
      following information when you use the "-verbosity high" switch:
      . Number of bits - flop width.
      . Size logic - number of gates inside the flop's COI.
      . Max depth - maximum number of flops from the original flop to a
        primary input.
      . Special values - values that are critical to the counter.
```

```
      . Direct flops - list of flops directly connected to the original
        flop. This list includes flop types (FSM, COUNTER, or OTHER).
      . COI flops - list of flops in the COI of the original flop.
    . This command returns a list of space-separated counter names.

. fsm: Identify FSMs in the vector of flops detected in the "-property"
    scope.
    . In addition to listing the names of flops, this command can print the
      following information when you use the "-verbosity high" switch:
      . Number of bits - flop width.
      . Size logic - number of gates inside the flop's COI.
      . Max depth - maximum number of flops from the original flop to a
        primary input.
      . Special values - values that are critical to the FSM.
      . Direct flops - list of flops directly connected to the original
        flop. This list includes flop types (FSM, COUNTER, or OTHER).
      . COI flops - list of flops in the COI of the original flop.
    . This command returns a list of space-separated FSM names.

. fifo: Identify FIFO memories (multidimensional arrays) in the "-property"
    scope.
    . In addition to listing the names of FIFO memories, this command can
      print the following information when you use the "-verbosity high"
      switch:
      . Number of bits - flop width.
      . Size logic - number of gates inside the flop's COI.
      . Max depth - maximum number of flops from the original flop to a
        primary input.
      . FIFO interface - clocks, resets, enables, access pointers, and data
        signals on input (write) and output (read) sides of the FIFO,
        in addition to associated full and empty signals.
      . Direct flops - list of flops directly connected to the original
        flop. This list includes flop types (FSM, COUNTER, or OTHER).
      . COI flops - list of flops in the COI of the original flop.
    . This command returns a list of space-separated FIFO memory names.

. array [-no_aggregate]: Identify multidimensional arrays (including FIFOs)
    in the vector of flops detected in the "-property" scope.
    . Use "-no_aggregate" to get a bit-blasted list.
    . In addition to listing the names of flops, this command can print the
      following information when you use the "-verbosity high" switch:
      . Number of bits - flop width.
      . Size logic - number of gates inside the flop's COI.
```

```
      . Max depth - maximum number of flops from the original flop to a
        primary input.
      . Direct flops - list of flops directly connected to the original
        flop. This list includes flop types (FSM, COUNTER, or OTHER).
      . COI flops - list of flops in the COI of the original flop.
    . This command returns a list of space-separated array names.

. register: List all flops and latches in the "-property" scope,
    displaying their names and sizes (in bits).
    This command returns a list of space-separated flop and latch names.

. flop: List all flops in the "-property" scope, displaying their names
    and sizes (in bits).
    . The list includes flop numbers shown as x(y)(z property flop bits).
      . x is the total number of flops.
      . y is the sum of flop bits connected to the specified property,
        not including property flop bits.
      . z is the sum of bits of property flops, which are generated as
        observers for SVA and PSL properties.
    . This command returns a list of space-separated flop names.

. latch: List all latches in the "-property" scope, displaying their
    names and sizes (in bits).
    This command returns a list of space-separated latch names.

. gate: List the number of gates and gate bits for the specified property.
    This command returns the number of gates.

. input: List the number of inputs followed by their names and
    sizes.
    . Inputs can be PIs, signals with hard stopats, or output
      signals of instance stopats.
    . This command returns a list of space-separated input names.

. undriven: List the names and sizes of all undriven signals in the scope
    of the specified properties.

    This command returns a list of space-separated names of undriven
    signals.

. undriven_internal: List the names and sizes of all undriven signals that
    are not primary inputs and that are in the scope of the specified
    properties.
```

```
    This command returns a list of space-separated names of undriven
    internal signals.

. loop_index: List the names and sizes of all indexes of generate for loops
    in the scope of the specified properties.

    This command returns a list of space-separated names of loop index
    signals.

. multiple_driven: List the names and sizes of all multiple-driven signals
    or high impedance sources in the scope of the specified properties.

    Not all signals returned by this command have more than one driver.
    They can be a source of high impedance such as z-assignments (1'bz),
    tri-states, bufifs, and mos gates (nmos, pmos, cmos).

    This command returns a list of space-separated names of multiple-driven
    or high impedance source signals.

. multiplier
  divider
  modulus
    List all cell-defined multipliers, dividers, or modulus in the scope
    of the specified properties
    "-property" scope.
    . Each multiplier, divider, or modulus that has not
      been blackboxed is cell defined. By default, all multipliers,
      dividers, and modulus are blackboxed; however, you can use
      "elaborate -bbox_mul", "-bbox_div", and "-bbox_mod" to avoid
      blackboxing them.
    . Each multiplier, divider, and modulus is described by three signals:
      {out_internal left_internal right_internal}
    . The right and left internal signals are the operands, and
      "out_internal" is the resulting signal.
    . This functionality is useful for manual abstraction.
    . This command returns a list of space-separated triples of signals.

  NOTE:
  . Blackboxed multipliers, dividers and modulus are not listed
      by this command.
  . Blackboxed multipliers, dividers and modulus will prevent this
      command from listing non-blackboxed multipliers dividers and
```

modulus that precede them in the "-property" scope, unless these non-blackboxed operators are enocountered through other routes in the "-property" scope.

. bbox_inout: List the names and sizes of the inout ports of blackboxed instances that are in the scope of the specified properties.

  This command returns a list of space-separated signal names.

. bbox_out: List the names and sizes of the output ports of blackboxed instances that are in the scope of the specified properties.

  This command returns a list of space-separated signal names.

. module: List all modules in the "-property" scope, displaying their names. This command returns a list of space-separated module names.

. instance: List all instances in the "-property" scope, displaying their names. This command returns a list of space-separated instance names. The list does not include blackboxed instances.

. signal: List all signals in the "-property" scope, displaying their names and sizes in bits.
  . Signals are the set of all registers, primary inputs, primary outputs, and wires (signals = registers + PIs + POs + wires).
  . This command returns a list of space-separated signal names.

. wire: List all wires in the "-property" scope, displaying their names and sizes in bits.
  . Wires are all the signals not classified as registers, primary inputs, or primary outputs (wires = signals - registers - PIs - POs).
  . This command returns a list of space-separated signal names.

. property
  assert
  assume
  cover
  List one or more of the current task's embedded or Tcl properties of the specified type.

  NOTE:
  . This command supports wildcards. If you do not use wildcards, it returns the specified property.

```
        . The list includes embedded and Tcl properties, embedded and
          Tcl assertions, embedded and Tcl assumptions, or embedded and Tcl
          cover directives.
        . The list does not include assertions extracted by the apps, for
          example, X-Prop and Connectivity assertions.
        . The list does not include global assumptions specified with "assume
          -env".

        The list includes the number and names of properties the return is a
        list of space-separated property names.

    . stopat: List all the stopats in the "-property" scope, displaying their
      names and sizes in bits. This command returns a list of space-separated
      signal names.

    . property_flop_bit: Return the number of property flop bits for the
      specified property.

"-task" Scope
=============
-task task_name+ [-compute_all]
               [-list
                     ([basic] [counter] [fsm] [fifo] [array [-no_aggregate]]
                      [register] [flop] [latch] [gate] [input]
                      [undriven] [undriven_internal] [loop_index]
                      [module] [instance] [signal] [wire]
                      [property] [assert] [assume] [cover]
                     )+
               ]
                     [-verbosity (high | low | silent)]
                     [-typed_list]
                     [-silent]
                     [-file file_name [-force]]
                     [-filter filter_expression [-regexp] [-case_sensitive]]
                     [-gui]

-task task_name+ [-compute_all]
    Get design information for one or more specified tasks.

    Important things to know about this command:
    . This analysis considers hard stopats and instance
      stopats in this task.
    . Use the optional "-compute_all" switch to request the computation of the
```

```
  list of registers for the specified task and for all properties within
  this task. The computation of registers for properties in a task can be
  done more efficiently if they are all done at the same time, and
  therefore, you will save time.
. This switch supports wildcards (* and ?) in task names:
  get_design_info -task t*
  get_design_info -task task1 {task2} t*
  get_design_info -task task?
. Information available for more than one task is: number and list
  of flops, latches, registers and inputs (PIs and stopats).
. Below you will find definitions of the supported "-list" enums for the
  "-task" scope.
. See the section "Customizing Lists" for switches that refine your list,
  for example, for scripting.
. This command provides only a basic subset of the available information
  about properties in a task. If you want more detailed information, use
  the command "get_property_list" instead.

NOTE: If you run the "reset" command prior to this command, the reset
signal is listed as a task input signal. This is the behavior
regardless of whether the task has properties.

Supported "-list" Enums for the "-task" Scope
---------------------------------------------
. basic: List the "default" set of information for the "-task" scope.
    . The Tcl command returns the number of flops in the "-task"
      scope.
    . The command "get_design_info -task task_name -list basic" has the
      same effect as "get_design_info -task task_name" with no additional
      switches.

. counter: Identify counters in the vector of flops detected in the
    "-task" scope.
    . In addition to listing the names of flops, this command can print the
      following information when you use the "-verbosity high" switch:
      . Number of bits - flop width.
      . Size logic - number of gates inside the flop's COI.
      . Max depth - maximum number of flops from the original flop to a
        primary input.
      . Special values - values that are critical to the counter.
      . Direct flops - list of flops directly connected to the original
        flop. This list includes flop types (FSM, COUNTER, or OTHER).
      . COI flops - list of flops in the COI of the original flop.
```

```
    . This command returns a list of space-separated counter names.

. fsm: Identify FSMs in the vector of flops detected in the "-task"
    scope.
    . In addition to listing the names of flops, this command can print the
      following information when you use the "-verbosity high" switch:
      . Number of bits - flop width.
      . Size logic - number of gates inside the flop's COI.
      . Max depth - maximum number of flops from the original flop to a
        primary input.
      . Special values - values that are critical to the FSM.
      . Direct flops - list of flops directly connected to the original
        flop. This list includes flop types (FSM, COUNTER, or OTHER).
      . COI flops - list of flops in the COI of the original flop.
    . This command returns a list of space-separated FSM names.

. fifo: Identify FIFO memories (multidimensional arrays) in the "-task"
    scope.
    . In addition to listing the names of FIFO memories, this command can
      print the following information when you use the "-verbosity high"
      switch:
      . Number of bits - flop width.
      . Size logic - number of gates inside the flop's COI.
      . Max depth - maximum number of flops from the original flop to a
        primary input.
      . FIFO interface - clocks, resets, enables, access pointers, and data
        signals on input (write) and output (read) sides of the FIFO,
        in addition to associated full and empty signals.
      . Direct flops - list of flops directly connected to the original
        flop. This list includes flop types (FSM, COUNTER, or OTHER).
      . COI flops - list of flops in the COI of the original flop.
    . This command returns a list of space-separated FIFO memory names.

. array [-no_aggregate]: Identify multidimensional arrays (including FIFOs)
    in the vector of flops detected in the "-task" scope.
    . Use "-no_aggregate" to get a bit-blasted list.
    . In addition to listing the names of flops, this command can print the
      following information when you use the "-verbosity high" switch:
      . Number of bits - flop width.
      . Size logic - number of gates inside the flop's COI.
      . Max depth - maximum number of flops from the original flop to a
        primary input.
      . Direct flops - list of flops directly connected to the original
```

```
        flop. This list includes flop types (FSM, COUNTER, or OTHER).
      . COI flops - list of flops in the COI of the original flop.
    . This command returns a list of space-separated array names.

. register: List all flops and latches in the "-task" scope,
    displaying their names and sizes (in bits).
    This command returns a list of space-separated flop and latch names.

. flop: List all flops in the "-task" scope, displaying their names
    and sizes (in bits).
    . The list includes flop numbers shown as x(y)(z property flop bits).
      . x is the total number of flops.
      . y is the sum of bits of the flops, not including property flop
        bits.
      . z is the sum of bits of property flops, which are generated as
        observers for SVA and PSL properties.
    . This command returns a list of space-separated flop names.

. latch: List all latches in the "-task" scope, displaying their
    names and sizes (in bits).
    This command returns a list of space-separated latch names.

. gate: List the number of gates and gate bits for the specified task.
    This command returns the number of gates.

. input: List the number of inputs followed by their names and sizes.
    . Inputs can be PIs, signals with hard stopats, or output
      signals of instance stopats.
    . This command returns a list of space-separated input names.

    NOTE: If you run the "reset" command prior to this command, the reset
    signal is listed as a task input signal. This is the behavior
    regardless of whether the task has properties.

. undriven: List the names and sizes of all undriven signals in the scope
    of the specified tasks.

    This command returns a list of space-separated names of undriven
    signals.

. undriven_internal: List the names and sizes of all undriven signals that
    are not primary inputs and that are in the scope of the specified
    tasks.
```

```
    This command returns a list of space-separated names of undriven
    internal signals.

. loop_index: List the names and sizes of all indexes of generate for loops
    in the scope of the specified tasks.

    This command returns a list of space-separated names of loop index
    signals.

. module: List all modules in the "-task" scope, displaying their
    names. This command returns a list of space-separated module names.

. instance: List all instances in the "-task" scope, displaying their
    names. This command returns a list of space-separated instance names.
    The list does not include blackboxed instances.

. signal: List all signals in the "-task" scope, displaying their
    names and sizes in bits.
    . Signals are the set of all registers, primary inputs, primary
      outputs, and wires (signals = registers + PIs + POs + wires).
    . This command returns a list of space-separated signal names.

. wire: List all wires in the "-task" scope, displaying their names
    and sizes in bits.
    . Wires are all the signals not classified as registers, primary
      inputs, or primary outputs (wires = signals - registers - PIs - POs).
    . This command returns a list of space-separated signal names.

. property
  assert
  assume
  cover
    List embedded and Tcl properties of the specified type in the specified
    task.

    NOTE:
    . The list includes embedded and Tcl properties, embedded and
      Tcl assertions, embedded and Tcl assumptions, or embedded and Tcl
      cover directives.
    . The list does not include assertions extracted by the apps, for
      example, X-Prop and Connectivity assertions.
    . The list does not include global assumptions specified with "assume
```

```
        -env".
      . This command provides only a basic subset of the available information
        about properties in a task. If you want more detailed information, use
        the command "get_property_list" instead.

      This command lists the number and names of properties and returns a
      list of space-separated property names.

Customizing Lists
=================
    Use the following switches with one or more of the enums listed above.

    -verbosity (high | low | silent)
      Use the specified level of verbosity to provide information.
      . high - Print all available information.
        . Example: When used with "-list flop", this command prints:
          . The total number of flops and property flops (flops generated as
            observers for SVA and PSL properties).
          . A list of all the signal names with their sizes shown in
            parentheses.
        . The return value is the sum of bits of all flops.
      . low - Print minimal information.
        . Example: When used with "-list flop", this command prints the
          space-separated flop names.
        . The "low" verbosity level is the default.
      . silent - Turn off output to the screen and return design info with a
        Tcl return value.
        The "silent" option is helpful in Tcl scripts.

    -typed_list
      Change the return format for the "-list" switch by adding a type element
      before each of the requested lists.
      . The list format is as follows:
        { input { <input_signal>+ } output { <output_signal>+ } }
      . Refer to the "Example" section.

    -silent
      Turn off output to the screen and return design info with a Tcl return
      value.
      . The "silent" option is helpful in Tcl scripts.
      . This switch has the same effect as "-verbosity silent".

    -file file_name [-force]
```

```
  Print information to the specified "file_name".

  Use "-force" to overwrite an existing file.

-filter filter_expression [-regexp] [-case_sensitive]
  Filters the returned list using case-insensitive wildcard strings
 (default).
  . Use "-regexp" to change the filter expression to a case-insensitive
    regular expression.
  . Use "-case_sensitive" to filter output using case-sensitive wildcard
    strings or regular expressions.

-gui
  Display the requested information in the Design Information GUI.

-include_hier_path
  Display signal names with the instance path. Use this switch with the
  following arguments:
  . output
  . input
  . inout

-pattern (sv_packed | sv_assignment)
  Format the list of parameter values.
  The "sv_packed" option formats parameter values as a string of bits.
  The "sv_assignment" option formats parameter values as expanded struct
  members.
  The default option is "sv_packed".

-transitive
  Return a list under the specified scope instance hierarchy.
  The return names display their hierarchical path.
  Use this switch with at least one of the following arguments:
  Top-Instance Scope
  . parameter
  "-instance" scope
  . parameter
  . interface
  . interface_no_param
  . interface_instance
  . interface_port
  "-interface_instance" scope
  . parameter
```

```
      "-interface_port" scope
      . parameter

Counter, FSM, and FIFO Detection
================================
-signal signal_list
                -is_counter |-is_fsm |-is_fifo
                [-verbosity high]
    Get design information for one or more specified signals.

    Important things to know about this command:
    . This analysis only considers hard stopats and instance stopats specified
      with "-env".
    . This command supports wildcards (* and ?) in the signal name.
    . "-is_counter" returns true if the signal is a counter or false, if not.
    . "-is_fsm" returns true if the signal is a finite state machine (FSM) or
      false, if not.
    . "-is_fifo" returns true if the signal is the output of a
      first-in-first-out (FIFO) memory or false, if it is not.
    . If you use "-verbosity high", and the signal is a counter, FSM, or FIFO,
      the command prints additional details about the detected structure:
      counter's critical values, FSM's states, and FIFO's interface.

Cached-Info Clearing
====================
-clear
    Remove all cached information for all "get_design_info" commands.
    (Use this command to free memory.)

Depth Calculation
=================
-signal name+ |-property name+
                -list (flop | latch | register)
                -depth N [-boundary] [-silent]
    List all the state elements that are within N layers of sequential logic
    from one or more target signals or properties.

    NOTE: This switch supports wildcards (* and ?) in signal and property
    names.

    . -list flop: run the depth calculation in terms of flop layers.
    . -list latch: run the depth calculation in terms of latch layers.
    . -list register: run the depth calculation in terms of both flops and
```

```
    latches.

  If the target property or signal is a state element itself, it is
  considered to be at depth 0.

  . Use "-boundary" to list only the state elements at the outer layer
    corresponding to depth N.
  . Use "-silent" to get just the signal names list, not a printout of the
    output of each signal with bit-width annotations.
```

Example

```
get_design_info -instance ing2 -list stopat
get_design_info -module pci_rst_int
get_design_info -module pci_bridge32 -list output
get_design_info -list counter
get_design_info -list bbox_in_instance_port
{in bbinst bbinst.in}

// List the flops and latches that are within distance 1 (in terms of layers of
// state elements) from the target signal.
get_design_info -signal sigName1 -list register -depth 1

// List the flops on the second layer of flops from either one of the two
// target signals.
get_design_info -signal sigName1 sigName2 -list flop -depth 2 -boundary

// List the flops that are on the 3rd layer of flops away from the target
// property.
get_design_info -property propName1 -list flop -depth 3 -boundary

// List the state elements (flops or latches) that are 2 layers of state
// elements from the target properties.
get_design_info -property propName1 propName2 -list register -depth 2 -boundary

// List FSMs and outputs for instance "arb", and return a typed list.
get_design_info -instance arb -list fsm output -typed_list
fsm {arb_core.arb_state arb_core.pri_enc.curr_chan tag_mgr.tag_poolA tag_mgr.tag_poolB
tag_mgr.tag_poolC} output {dat_out dat_val full0 full1 full2 full3 wb0_ack_o wb0_data_o
wb0_err_o wb1_ack_o wb1_data_o wb1_err_o wb2_ack_o wb2_data_o wb2_err_o wb3_ack_o
wb3_data_o wb3_err_o}
```

See Also

get_flop_info,
get_latch_info,
get_signal_info,
get_property_list,
jforeach,
set_design_exploration_include_clock_logic,
set_use_low_power_port_definition

get_fanin

Returns the signals in the fanin of a signal or property.

Syntax

```
get_fanin (signal_name | property_name | expression) [-add_clock_logic]
   [-ignore_weak_driver] [-env]
   [-avoid_pin (flop_async_reset | latch_enable | mux_enable)+]
   [-show_schematic]

get_fanin -go_into_cell_define (signal_name | property_name | expression)
   [-add_clock_logic] [-ignore_weak_driver] [-env]

get_fanin -transitive (signal_name | property_name | expression)
   [(-task task_name |-env)] [-ignore_weak_driver]
   [-avoid_pin (flop_async_reset | flop_clock | latch_enable | mux_enable)+]
   [-filter_out (boundary | non_boundary | user_reset | (data | control))+]
   [-barrier (signal_name | signal_list | reg_expr | reg_expr_list)] [-silent]
   [-show_schematic]

get_fanin -show_expr signal_name [-no_constant_propagation]
   [-ignore_weak_driver]
```

Returns

Returns a string containing a list of the names of the fanin signals.

Detailed Description

Use this command to find the signals in the fanin (after skipping buffers) of the specified signal, property, or expression.

```
(signal_name | property_name | expression) [-add_clock_logic]
             [-ignore_weak_driver] [-env]
             [-avoid_pin (flop_async_reset | latch_enable | mux_enable)+]
             [-show_schematic]
    Find the signals in the fanin of the specified signal, property, or
```

```
    expression.

    NOTE: If you specify an expression, the command returns the operands, that
    is, the signals that drive the expression.

-add_clock_logic
    . Use "-add_clock_logic" to traverse the clock pin logic.

-ignore_weak_driver
    . Use "-ignore_weak_driver" to specify that you want the tool to consider
      driver strengths and report only the possible active drivers of
      multiple-driven nets.

      NOTE: The tool does not take into account constant values (propagated or
        not). Analysis is based entirely on the driver strengths declared in
        the RTL.

-env
    . Use "-env" to run analysis based on the entire design region ignoring the
      task context, such as assumptions and stopats.

-avoid_pin (flop_async_reset | latch_enable | mux_enable)+
    . Use "-avoid_pin" with one or more of the following options to prevent
      the "get_fanin" traversal from going through the specified pins:
      . flop_async_reset
      . latch_enable
      . mux_enable

-show_schematic
    . Use "-show_schematic" to open the schematic viewer showing the specifed
      signal and its fanins.

-go_into_cell_define (signal_name | property_name | expression)
                     [-add_clock_logic] [-ignore_weak_driver] [-env]
    Include signals in cell defines in the results.

    Important things to know:
    . If the target "signal" is in a cell define, the tool automatically
      includes fanin signals in that cell define.
    . This switch can provide a considerable performance gain. Weigh this gain
      against ignoring task-based assumptions and stopats.

-add_clock_logic
```

```
    . Use "-add_clock_logic" to traverse the clock pin logic.

-ignore_weak_driver
    . Use "-ignore_weak_driver" to specify that you want the tool to consider
      driver strengths and report only the possible active drivers of
      multiple-driven nets.

      NOTE: The tool does not take into account constant values (propagated or
        not). Analysis is based entirely on the driver strengths declared in
        the RTL.

-env
    . Use "-env" to run analysis based on the entire design region ignoring the
      task context, such as assumptions and stopats.

-transitive (signal_name | property_name | expression) [(-task task_name |-env)]
            [-ignore_weak_driver]
            [-avoid_pin (flop_async_reset | flop_clock | latch_enable
                         | mux_enable)+]
            [-filter_out (boundary | non_boundary | user_reset
                          | (data | control))+]
            [-barrier (signal_name | signal_list | reg_expr | reg_expr_list)]
            [-silent] [-show_schematic]
    Run the command recursively until all results are primary inputs or stopats
    that are in the fanin cone of the specified signal, property, or
    expression. By default, the transitive "get_fanin" traverses clock pins and
    returns the full path.

-ignore_weak_driver
    . Use "-ignore_weak_driver" to specify that you want the tool to consider
      driver strengths and report only the possible active drivers of
      multiple-driven nets.

      NOTE: The tool does not take into account constant values (propagated or
        not). Analysis is based entirely on the driver strengths declared in
        the RTL.

-task task_name
    . Use "-task" to run analysis based on the task design region.

-env
    . Use "-env" to run analysis based on the entire design region ignoring the
      task context, such as assumptions and stopats.
```

```
-avoid_pin (flop_async_reset | flop_clock | latch_enable | mux_enable)+
    . Use "-avoid_pin" with one or more of the following options to prevent
      the "get_fanin" traversal from going through the specified pins:
      . flop_async_reset
      . flop_clock
      . latch_enable
      . mux_enable

-filter_out (boundary | non_boundary | user_reset | (data | control))+
    . Use "-filter_out" with one or more of the following options to avoid
      showing them in the results:
      . boundary - Filter out boundary signals from results.
      . non_boundary - Filter out non-boundary signals from results.
      . user_reset - Filter out user-specified reset signals.
      . data - Filter out data signals from results.
      . control - Filter out control (latch enable, mux enable and asynchronous
          reset condition) signals from results.
      NOTE: Combining "-filter_out data" and "-filter_out control" in a single
      command is not allowed since every returned signal is a control or data
      signal or both.

-barrier (signal_name | signal_list | reg_expr | reg_expr_list)
    . Use "-barrier" to allow the transitive "get_fanin" to stop at the
      specified signals as if they were boundaries.
      NOTE: If you also use the switch "-filter_out", the barrier signals will
      be seen as boundary signals.

-silent
    . Use "-silent" to turn off the output to the screen and return
      information with a Tcl return value, which is helpful in Tcl scripts.

-show_schematic
    . Use "-show_schematic" to open the schematic viewer showing the specifed
      signal and its fanins.

-show_expr signal_name [-no_constant_propagation] [-ignore_weak_driver]
    Return the combinational expression of the specified signal in terms of its
    immediate fanin signals rather than a simple list of the signals.

    Important things to know:
    . This switch does not support `celldefine.
    . This switch does not support the following, and in these cases, the
```

```
      command is not able to return the correct expression:
      . Hierarchical references
      . Escaped signals
      . Expression slicing, for example,
          {1'b0, 1'b0, 1'b0, 1'b0, 1'b0, 1'b0, 1'b0,
          o2p_queue_blk.pre_queue_valid}[o2p_queue_blk.pci_ping]

-no_constant_propagation
    . Use "-no_constant_propagation" to avoid the expression simplification due
      to constant propagation.

-ignore_weak_driver
    . Use "-ignore_weak_driver" to specify that you want the tool to consider
      driver strengths and report only the possible active drivers of
      multiple-driven nets.

      NOTE: The tool does not take into account constant values (propagated or
        not). Analysis is based entirely on the driver strengths declared in
        the RTL.
```

Example

```
get_fanin A
{A}

get_fanin "a && b"
{a b}

get_fanin "~a"
{a}

get_fanin -transitive arb_core.tag_req
{clk rst input selector}

# The following command tells the tool not to traverse the clock pins and to
# return only the boundary signals
get_fanin -transitive signalA -avoid_pin flop_clock \
          -filter_out non_boundary -silent
{signalB signalC signalD signalE}

get_fanin -show_expr signalB
{signalA && signalC}
```

See Also

get_fanout,
get_signal_info

get_fanout

Returns the signals in the immediate fanout of the specified signal.

Syntax

```
get_fanout signal_name [(-env |-task task_name)] [-connected_properties]

get_fanout -transitive signal_name [(-env |-task task_name)]
   [-avoid_pin (flop_async_reset | flop_clock | latch_enable | mux_enable)+]
   [-filter_out (boundary | non_boundary | user_reset)+]
   [-barrier (signal_name | signal_list | reg_expr | reg_expr_list)] [-silent]
```

Returns

Returns a string containing the names of the fanout signals.

Detailed Description

Use this command to find the signals in the immediate fanout of the
specified signal.

```
signal_name [(-env |-task task_name)] [-connected_properties]
    Find the signals in the immediate fanout of the specified signal.

-env
    . Use "-env" to run analysis based on the entire design region ignoring the
       task context, such as assumptions and stopats.

-task task_name
    . Use "-task" to run analysis based on the task design region.

-connected_properties
    . Use "-connected_properties" to report properties connected to the
      immediate fanout of "signal_name". A connected property is a property
      that contains "signal_name" as a direct driver. For example, in
      "assert {a == b} -name prop_name", "prop_name" is a connected property
      for "a" and "b".
```

```
      LIMITATION: The tool does not report reset or soft assumptions because
        these are not supported.

-transitive signal_name [(-env |-task task_name)]
            [-avoid_pin (flop_async_reset | flop_clock | latch_enable
                         | mux_enable)+]
            [-filter_out (boundary | non_boundary | user_reset)+]
            [-barrier (signal_name | signal_list | reg_expr | reg_expr_list)]
            [-silent]
    Run the command recursively until all results are boundary signals that are
    in the fanout cone of the specified signal.

-env
    . Use "-env" to run analysis based on the entire design region ignoring the
       task context, such as assumptions and stopats.

-task task_name
    . Use "-task" to run analysis based on the task design region.

-avoid_pin (flop_async_reset | flop_clock | latch_enable | mux_enable)+
    . Use "-avoid_pin" with one or more of the following options to prevent
      the "get_fanout" traversal from going through the specified pin types:
      . flop_async_reset
      . flop_clock
      . latch_enable
      . mux_enable

-filter_out (boundary | non_boundary | user_reset)+
    . Use "-filter_out" with one or more of the following options to avoid
      showing them in the results:
      . boundary - Filter out boundary signals from results.
      . non_boundary - Filter out non-boundary signals from results.
      . user_reset - Filter out user-specified reset signals.

-barrier (signal_name | signal_list | reg_expr | reg_expr_list)
    . Use "-barrier" to allow the transitive "get_fanout" to stop at the
      specified signals as if they were boundary signals.
      NOTE:
      . When "reg_expr" does not match any signal name, you can downgrade the
        resulting error (EPM044) to a warning, and in that case, the command
        runs with no barrier.
      . If you also use the switch "-filter_out", the barrier signals are seen
        as boundary signals.
```

```
-silent
    . Use "-silent" to turn off the output to the screen.
```

Example

```
get_fanout A
{B}

# The following command tells the tool not to traverse the clock pins and to return
only the boundary signals
get_fanout -transitive A -avoid_pin flop_clock -filter_out non_boundary -silent
{B C D E}
```

See Also

get_fanin,
get_signal_info

get_filename

Locates the filename for a specified module, instance, or entity.

Syntax

```
get_filename (-module module_name |-inst instance_name |-entity entity_name)
```

Returns

Returns a string containing the name of the file associated with the specified module, instance, or entity name.

Detailed Description

Use this command to find the filename for a specified module, instance, or entity.

-module module_name
    Report the associated filename for the specified module.

-inst instance_name
    Report the associated filename for the specified instance.

-entity entity_name
    Report the associated filename for the specified VHDL entity.

Example

```
get_filename -module pci_core
/path/where/module/pci_core_is_located
```

get_flop_info

Returns information about signals related to one or more input flops.

Syntax

```
get_flop_info [-clock]
              [-edge]
              [-data]
              [-reset_type]
              [-reset_pin
               |-sync_reset_pin
               |-async_reset_pin]
              [ -reset_value_pin
               |-sync_reset_value_pin
               |-async_reset_value_pin]
              [-file_name]
              [-line]
              [-bit_blast |-group_all ]
              [-silent]
              [-typed_list |-compact]
              [-power |-show_expr]
              [-defined_clock]
               flop_name+
```

Returns

Returns different lists depending on the switches used. Switches can be combined to include multiple fields (for example, "-clock -data" returns flop name, clock signal, and data signals).

Switches	Returned List Description
no switch	List per flop with: flop name, clock signal, data signal, reset pin signal, reset value pin signal, reset type (async, sync, mixed, none), file location (file name and line numbers).
-clock	List per flop with: flop name and clock signal.

-edge	List per flop with: flop name and clock edge (posedge, negedge, or mixed).
-data	List per flop with: flop name and data pin signals.
-reset_type	List per flop with: flop name and reset type (async, sync, mixed, or none).
-reset_pin	List per flop with: flop name and reset pin signals.
-sync_reset_pin	List per flop with: flop name and synchronous reset condition signal.
-async_reset_pin	List per flop with: flop name and asynchronous reset condition signal.
-reset_value_pin	List per flop with: flop name and reset value pin signals.
-sync_reset_value_pin	List per flop with: flop name and synchronous reset value pin signal.
-async_reset_value_pin	List per flop with: flop name and asynchronous reset value pin signal.
-file_name	List per flop with: flop name and file name where the flop is instantiated.
-line	List per flop with: flop name and line number(s) of the flop instantiation.
-bit_blast	Returns separate lists per individual bit of each flop.
-group_all	Groups all information in a single list instead of separate lists per flop.
-typed_list	Returned list includes type label before each element.
-compact	Returned list does not include the flop name from each list.
-power	List per flop with: flop name and primary supply net

	name. Empty net name if the flop is not power aware.
-show_expr	Return a list with combinational expressions instead of signal lists for all fanin signals (as data pin and reset pin).
-defined_clock	List per flop with: flop name and user-defined clocks connected to the flop.

Detailed Description

Use this command to get information about one or more flops, including reset information. When there is no asynchronous reset connected to specified flops, the tool attempts to find a synchronous reset signal.

When a flop has both an asynchronous reset and a synchronous reset (according to Jasper Apps heuristics) the command returns the asynchronous one unless you specify one of the "-sync_*" switches.

By default, "get_flop_info" returns one list for each flop gate driving the input signal. Each of these lists contains signals related to clock, data, power, reset, reset value pins, and reset type for the specified flop. In addition, the lists provide location (file name and line numbers) and indicate whether the reset signal found is asynchronous (async) or synchronous (sync). If there is no reset signal, the reset type is "none". If the flop has an abstracted reset value (abstract -reset_value), the reset value pin includes the notation "abstracted".

NOTE: This command accepts one or more flop names.

-clock
    Report the clock signal that drives the specified flop.

-edge
    Report the sensitivity edge of the specified flop.

-data
    List signals related to the data pin in the result.

-reset_type
    Report the type of the reset condition.

```
    Reset type can be synchronous, asynchronous, or mixed. Mixed is applicable
    for wide flops that contain a mixture of reset types (async, sync, none).

    NOTE: This switch is not supported with "-reset_pin".

-reset_pin
    List signals related to the reset pin.

    NOTE: This switch is not supported with "-reset_type".

-sync_reset_pin
    Report the sync reset condition, if there is one, even if there is also an
    asynchronous reset.

-async_reset_pin
    Report the asynchronous reset condition.

-reset_value_pin
    List signals related to the reset value pin, and if the reset value is
    abstracted, also include the notation "abstracted".

-sync_reset_value_pin
    Report the synchronous reset value pin, if there is one, even if there is
    also an asynchronous reset value pin.

-async_reset_value_pin
    Report the asynchronous reset value pin, if there is one.

-file_name
    Report the name of the file where the flop is instantiated.

    By default, "get_flop_info" reports the file name and one or more line
    numbers or ranges of numbers.

-line
    Report the line number where the flop is instantiated.

    By default, "get_flop_info" reports the file name and one or more line
    numbers or ranges of numbers.

-bit_blast
    Return the individual bits information of a flop.
```

```
-group_all
    Group all requested information in a single list.

-silent
    Avoid printing user-friendly information on the screen.

-typed_list
    Report the type information before each list in the result.

-compact
    Exclude the flop name from the result.

-power
    Report the name of the primary supply net.

    NOTE: If the specified flop is not power aware, the report prints with an
    empty "Power" field and the return has an empty Tcl list.

-show_expr
    Return the combinational expression of the specified signal in terms of its
    immediate fanin signals rather than a simple list of the signals.

    Important things to know:
    . This switch does not support `celldefine.
    . This switch does not support the following, and in these cases, the
      command is not able to return the correct expression:
      . Hierarchical references
      . Escaped signals
      . Expression slicing, for example,
        {1'b0, 1'b0, 1'b0, 1'b0, 1'b0, 1'b0, 1'b0, o2p_queue_blk.pre_queue_valid}
[o2p_queue_blk.pci_ping]

-defined_clock
     Return the list of user-defined clocks connected to the specified flop.

     NOTE:
     . If you declared an inverted clock, this command returns the name
       with the edge.
     . If you declared the clock as both edges, this command returns the
       edge with the clock name if the specified flop is sampled by the
       negedge of that clock.
```

Example

```
// Get fanin from data, clock, reset, and reset value pins.
[<embedded>] get_flop_info arb_core.arb_state
Flop information:
Flop: arb_core.arb_state
    Clock: arb_core.clk
    Data: arb_core.one_chan_rdy arb_core.one_more_chan_rdy arb_core.curr_chan_eof
arb_core.cred_avail arb_core.curr_chan_vld arb_core.rst arb_core.arb_state
    Reset pin: rst
    Reset value pin:
    Reset type: sync
    Location: ./design/arb_core.v: 108-177,1-4

{arb_core.arb_state arb_core.clk {arb_core.one_chan_rdy arb_core.one_more_chan_rdy
arb_core.curr_chan_eof arb_core.cred_avail arb_core.curr_chan_vld arb_core.rst
arb_core.arb_state} rst {} sync {./design/arb_core.v 108-177,1-4}}

// Return only clock and data pins fanin.
get_flop_info -clock -data flop1

// Return clock information. (More than one list if multiFlop is related to
// more than one flop gate.)
get_flop_info -clock multiFlop

// Return clock information. (Group all clocks in a single list.)
get_flop_info -clock -group_all multiFlop

// Get info on a flop with an abstracted reset value
get_flop_info q4
Flop information:
Flop: q4
    Clock: clk
    Data: q3
    Reset pin: rst
    Reset value pin:
    Reset type: async
    Location: floplatchline.v: 13-27,1-4

{q4 clk q3 rst {} async {floplatchline.v 13-27,1-4}}

abstract -reset_value q4
get_flop_info -show_expr q4
```

```
Flop information:
Flop: q4
    Clock: clk
    Data: q3
    Reset pin: {rst, rst, rst}
    Reset value pin: 3'b000 abstracted
    Reset type: async
    Location: floplatchline.v: 13-27,1-4

{q4 clk q3 {{{rst, rst, rst}}} {3'b000 abstracted} async {floplatchline.v 13-27,1-4}}

// Get info on a "mixed" signal. In the following example, the signal "q" is
// actually two different flops: q[0] and q[1], and one is sensitive to the
// rising edge of the clock, while the other is sensitive to the falling edge.
get_flop_info q -edge -group_all
Flop information:
Flop: q
    Clock: clk1 clk2 mixed
    Data: d
    Reset pin: rst
    Reset value pin:
    Reset type: async
    Location: edge.v: 3-8,1-4  10-15,1-4

{q {mixed clk1 clk2} d rst {} async {edge.v {3-8,1-4 10-15,1-4}}}
```

See Also

get_design_info,
get_latch_info,
get_signal_info,

get_inst_top

Gets the top instance path to be analyzed formally.

Syntax

```
get_inst_top
```

Returns

Returns a string (identifier name) for the top hierarchical instance found in the design. When you launch Jasper using "xrun" and "-inst_top", the command returns the instance path defined with "-inst_top"; otherwise, it returns the top module.

Detailed Description

Use this command to get the top instance that will be analyzed formally. When you launch Jasper using "xrun" and "-inst_top", the top instance is the one defined with "-inst_top". If you do not use "-inst_top", the command returns the top module.

Example

```
get_inst_top
top
```

See Also

get_top_module

get_install_dir

Returns the full path of the Jasper installation directory.

Syntax

```
get_install_dir
```

Returns

The full path of the Jasper installation directory.

Detailed Description

Use this command to get the full path of the Jasper installation directory.

Example

```
get_install_dir
/jasper/install/dir
```

See Also

get_proj_dir

get_latch_info

Returns information about signals related to the input latch.

Syntax

```
get_latch_info [-enable]
               [-data]
               [-power]
               [-file_name]
               [-line]
               [-bit_blast |-group_all ]
               [-silent]
               [-typed_list]
               [-show_expr]
               latch_name
```

Returns

Returns lists of signal name lists.

Detailed Description

Use this command to get information about a latch, excluding reset information.

By default, "get_latch_info" returns one list for each latch gate driving the input signal. Each of these lists contains signals related to enable and data pins of the latch. In addition, the lists provide location (file name and line numbers). Use the optional switches to refine the return information as shown in the examples.

-enable
    Include the enable signal that drives the specified latch in the result.

-data
    Include a list of signals related to the data pin in the result.

-power

Include the name of the primary supply net.

NOTE: If the specified latch is not power aware, the report prints with an empty "Power" field and the return includes an empty Tcl list.

-file_name
Include the name of the file where the latch is instantiated.

By default, "get_latch_info" includes the file name and one or more line numbers or ranges of numbers.

-line
Include the line number where the latch is instantiated.

By default, "get_latch_info" includes the file name and one or more line numbers or ranges of numbers.

-group_all
Group all requested information in a single list.

-bit_blast
Return the individual bits information of a latch.

-silent
Avoid printing user-friendly information on the screen.

-typed_list
Include the type information before each list in the result.

-show_expr
Return the combinational expression of the signals in terms of their immediate fanin signals rather than a simple list of the signals.

Example

```
// Get fanin from data and enable pins.
[<embedded>] get_latch_info latch1
Latch information:
Latch: latch1
    Data: rst1 in1
    Enable: en1 rst1
    Location: test_latch_info.v: 11-15,5-8

{latch1 {rst1 in1} {en1 rst1} {test_latch_info.v 11-15,5-8}}

// Return only enable and data pins' fanin.
get_latch_info -enable -data latch1

// Return enable information. (More than one list if "multiLatch" is related to
// more than one latch gate.)
get_latch_info -enable multiLatch

// Return enable information. (Group all enables in a single list.)
get_latch_info -enable -group_all multiLatch
```

See Also

get_design_info,
get_flop_info,
get_signal_info,

get_needed_assumptions

Finds the irreducible set of assumptions to achieve a target status.

Syntax

```
get_needed_assumptions -property property_name
                        [-status target_status_tcl_list]
                        [-assumes assumption_tcl_list]
                        [-include_env_assumes]
                        [-include_liveness_assumes]
                        [-include_clock]
                        [-include_reset]
                        [-include_properties]
                        [-required_assumes assumption_tcl_list]
                        [-attempts number_of_prove_attempts]
                        [-attempts_time_limit time_limit]
                        [-max_trace_length trace_length]
                        [-stop_status target_status_tcl_list]
                        [-stop_on_remaining remaining_percentage]
                        [-engine_mode engine_tcl_list]
                        [-proofgrid_mode (local | shell | lsf | oge | nc | cluster)]
                        [-max_parallel_jobs num_parallel_jobs]
                        [-from property_name [-trace_id N] [-cycle N]]
                        [-bg]
                        [-verbose]
                        [-silent]

get_needed_assumptions -result

get_needed_assumptions -run_status
```

Returns

The command returns a Tcl dictionary.
The Tcl dictionary can contain the following key/value pairs:

Key	Value

run_status	Run Status (See below for additional explanation.)
exit_status	Exit Status (See below for additional explanation.)
prove_attempts	The number of prove attempts
needed	A set of needed assumptions
under_approx	Any needed assumptions the tool identified prior to stopping (applies if the command stops before finding the final result)
needed_reset	A set of reset constraints (applies when using "-include_reset")
needed_clock	A set of clock constraints (applies when using "-include_clock")
needed_properties	A set of assertions or covers (applies when using "-include_properties")
result_task	The task created by the command that demonstrates the result.

Interpret the run status as follows:

Run Status	Definition
not_started	The run has not yet started.
completed	The run completed.
running	The run is in progress.

Interpret the exit status as follows:

Exit Status	Definition
completed	A minimal set of assumptions has been found.
attempts_limit	The command was stopped because the attempts limit was reached. The set of assumptions returned

```
                    is an overapproximation of the minimal needed set.
                    If any assumptions are determined to be definitely
                    needed, the return value includes these as an
                    additional last element.

attempts_time_limit The command was stopped because the attempts
                    time limit was reached. In this case, none
                    of the other elements is returned.

max_trace_length    The command was stopped because max_trace_length was
                    reached. In this case, none of the other elements is
                    returned.

stopped_by_user     The command was stopped by the user. The set of
                    assumptions returned is an overapproximation of the
                    minimal needed set. If any assumptions are
                    determined to be definitely needed, the return value
                    includes these as an additional last element.

stopped_by_status   The command was stopped because the status specified
                    by "-stop_status" was reached.

stopped_by_remaining The command stopped because the percentage of remaining
                    assumptions (namely, assumptions that have not been
                    determined yet as either needed or not needed) reached
                    the "-stop_on_remaining" threshold.

aborted             The command stopped due to an error. In this case,
                    none of the other elements is returned.
```

Default

```
The defaults are as follows:
-status: If the property has been processed and has a proof
         result (proven, unreachable, or error) the tool uses that status.
         If it does not have a proof result, the default is "proven" for
         assertions and "unreachable" for covers.
-assumes: all enabled assumptions
-attempts: 0
-attempts_time_limit: 120s
-max_trace_length: 0
-max_parallel_jobs: 1
```

Detailed Description

This command systematically disables and enables assumptions to eventually find an irreducible set of assumptions needed to achieve the target prove status for the target property. That is, if you enable only the irreducible set of assumptions, and then you disable any one of them, you will not achieve the target prove status for the target property.

The command creates a task called NDD_ASSUMES and copies the current task's assumptions, stopats, initial value abstractions, and target property to the new task. Then, it iteratively uses bisection of the set of assumptions to find the needed assumptions (the irreducible set of assumptions).

Use this command to:
. Find conflicting assumptions when Jasper Apps reports an overconstraint.
. Find conflicting assumptions when a cover is unexpectedly unreachable.
. Find the minimal set of assumptions to prove a property, potentially
  speeding up the proof.

When trying to identify assumptions using this command, it is often useful to take note of which engine produced the result and ensure this engine is enabled with the "-engine_mode" switch. You can also specify it with "set_engine_mode" before running "get_needed_assumptions".

In the overconstrained case, indicated by a :noConflict cex, also consider using "check_assumptions -minimize" instead.

NOTE:
. By default, the tool displays the following in the console:
  . Notification when the check begins and ends
  . Incremental messages to report needed assumptions as they are discovered
  . A final report of results
  Use "-silent" to turn off these messages.

. You can run this command while "prove" is running in the
  background provided you do not use any of the following
  "get_needed_assumptions" switches:
  . -include_env_assumes
  . -include_clock
  . -include_reset

. When running the "get_needed_assumptions" command with
  "-include_env_assumes", "-include_clock", or "-include_reset", the

```
  environment at the end of the run might differ from that before the run
  because the commands might disable some constraints.

-property property_name
    Find needed assumptions for the specified property.

-status target_status_tcl_list
    Try to achieve one of the specified statuses.

    NOTE:
    . If the property has been processed and has a proof result (proven,
      unreachable, or error) the tool uses that status. If it does not have
      a proof result, the default is "proven" for assertions and "unreachable"
      for covers.
    . This switch accepts a Tcl list.
    . For undetermined status, this switch also accepts the values "time_limit"
      and "max_trace_length" to specify undetermined status due to these
      parameters.

-assumes assumption_tcl_list
    Use the specified assumptions only. The default is to use all enabled
    assumptions.

-include_env_assumes
    Include environment assumptions in the analysis. Environment assumptions
    affect all tasks and the reset analysis.

    NOTE: This switch invalidates all proof results in all tasks.

-include_liveness_assumes
    Consider liveness assumptions for safety properties. By default, the tool
    does not consider liveness assumptions when the specified property is a
    safety property, but it does consider liveness assumptions with a
    safety component, also known as mixed liveness assumptions.

-include_clock
    Compute the needed set of clock constraints. For "clock -rate" constraints,
    these will not be minimized but applied if and only if the corresponding
    clock signal is part of the needed set of clock constraints.

    NOTE: This switch can invalidate all proof results in all tasks.

-include_reset
```

```
    Compute the needed set of reset expressions. "reset -vcd", "reset -fsdb",
    "reset -shm", "reset -vwdb", and "reset -sequence" are not supported.

    NOTE: This switch can invalidate all proof results in all tasks.

-include_properties
    Include assertions and covers in the analysis.
    In some cases, if the logic of a task assertion or cover includes an
    unstable latch loop, the property can have an assumption-like effect.

-required_assumes assumption_tcl_list
    Include the specified assumptions in the set of needed assumptions.
    These will be assumed to be needed and included in the result.

-attempts number_of_prove_attempts
    Stop after the specified number of proof attempts. The default is 0
    (unlimited).

-attempts_time_limit time_limit
    Time limit per iteration, measured in wall clock time

    The time limit is a positive integer that takes an optional trailing unit:
    "s" for seconds, "m" for minutes, "h" for hours or "d" for days. If you do
    not specify a unit, the unit is seconds. The default is 120s.

-max_trace_length trace_length
    Limit the length of traces to search for. The default is 0 (unlimited).

-stop_status target_status_tcl_list
    Stop immediately whenever one of the specified statuses is reached.
    The resulting list of assumptions are the assumptions used to reach
    that status, and they are not necessarily a minimal set.

-stop_on_remaining remaining_percentage
    Stop immediately when the percentage of remaining assumptions (namely,
    assumptions that have not yet been determined as either needed or not
    needed) reaches the specified remaining percentage.

-engine_mode engine_tcl_list
    Use the specified engines for this command.

-proofgrid_mode (local | shell | lsf | oge | nc | cluster)
    Use the specified ProofGrid mode for this command.
```

```
    See "set_proofgrid_mode" for additional information about this setting.

-max_parallel_jobs num_parallel_jobs
    Use the specified parallelism level (that is, the number of jobs run in
    parallel) for the needed assumptions computations.

-from property_name [-trace_id N] [-cycle N]
    Use an existing trace as the initial value for the analysis.

    . Use "-trace_id" to specify a trace other than the default. The default
      property trace is the last shortest trace. See "assert/cover
      -set_store_trace" for the commands to configure the  number of traces
      generated and stored by the tool.

    . Use "-cycle" to specify the state of the trace at a specific cycle as the
      reset value.
      .  If you do not specify a cycle, or if you specify 0, the tool
         uses the cycle in which the property was triggered. This is often, but
         not always, the last cycle of the trace. See "set_trace_extension".
      .  If you specify $, or if you specify -1, the tool uses the last cycle of
         the existing trace.
      .  A negative N selects the cycle relative to the end of the trace.

    NOTE: When using "-from", the target status, see "-status", is a status from the
    selected start state (not the reset state). The statuses "proven" and "unreachable"
    mean there is no trace from this state that violates or covers the property.
    See also "prove -from".

-bg
    Run the "get_needed_assumptions" command in the background and continue to
    accept and run new commands.

    NOTE:
    . When running "get_needed_assumptions" in the background, it is not
      possible to use any of the following "get_needed_assumptions" switches:
        . -include_env_assumes
        . -include_clock
        . -include_reset
        . -from
    . It is not possible to run another "get_needed_assumptions" command while
      a "get_needed_assumptions" command is running in the background.

-verbose
```

```
    Output progress information.

    Progress information includes:
    . Iteration number
    . Number of assumptions determined as required
    . Number of assumptions being evaluated in the current iteration
    . Excluded assumptions

-silent
    Do not print messages to the log and do not report progress and results in
    the console.

-result
    Get the result of the last "get_needed_assumptions" run.
    If a "get_needed_assumptions" run is in progress, it will return
    just the run_status (either "not_started" or "running").

-run_status
    Get the run status of the last "get_needed_assumptions" run.
    . If "get_needed_assumptions" is in progress, the tool returns the
      run_status (either "not_started" or "running").
    . If the last "get_needed_assumptions" run completed, the tool returns
      "completed".
    . If "get_needed_assumptions" did not run, the tool returns "not_started".

NOTE: The "set_cmd_time_limit" command has no effect on the
"get_needed_assumptions" command.
```

Example

```
get_needed_assumptions -property cover_property -status unreachable

Completed successfully.
============================================================
SUMMARY
============================================================
        Resulting task             : NDD_ASSUMES1
             Prove attempts        : 17
             Achieved status       : unreachable
             Assumptions
                - needed           : assume:8 assume:0 assume:1
```

```
status completed prove_attempts 17 needed {assume:8 assume:0 assume:1} result_task
NDD_ASSUMES1

Example code to save and restore environment assumptions in case of running
"get_needed_assumptions" with "-include_env_assumes".

# Helper procs to save off original assume configuration and restore it after
# get_needed_assumptions command invocation.
proc save_assumes {} {
    global assumes_config_array
    # get list of env assumes
    set env_assumes [assume -list -env -silent]
    foreach assume $env_assumes {
     set disabled [dict get [get_property_info $assume] disabled]
     # Save off flag indicating it is disabled
     set assumes_config_array($assume) $disabled
    }
}

proc restore_assumes {} {
    global disabled assumes_config_array
    set assume_list [array names assumes_config_array]
    # Iterate through assumes and restore assume state based on disable flag
    # Value. 0 - enabled, 1 - disabled
    foreach assume $assume_list {
     if { $assumes_config_array($asssume) == 0 } {
        # Originally enabled so enable it regardless of current state.
        assume -enable $assume
        puts "Re-enabling environmental assume: $assume"
     }
    }
}

## Example showing saving off and restoration of assumes
save_assumes
get_needed_assumptions -include_env_assumes -property cover1
# Restore original assumes
restore_assumes
```

See Also

assume,
check_assumptions,
prove

get_proj_dir

Gets the location of the project directory.

Syntax

```
get_proj_dir
```

Returns

Returns the location of the project directory.

Detailed Description

Use this command to extract the location of the project directory.

NOTE: This command is helpful when you launch the tool with the "-proj" switch to change the default location.

Example

```
get_proj_dir
/path/to/jgproject
```

See Also

get_install_dir

get_property_info

Returns information about a property.

Syntax

```
get_property_info property_name
get_property_info -list { [name] [task] [source_task] [type] [clock] [edge] [command]
                                    [expression] [precondition]
                                    [related_covers] [liveness] [converted] [helper]
[sst]
                                    [app] [file] [pa] [tcl] [error] [disabled]
                                    [deprecated] [status] [related_cover_status]
                                    [pseudo_constant] [assume_bound]
                                    [time] [proof_timestamp] [engine]
                                    [min_length] [max_length] [sst_max_length]
[stem_length] [loop_length] [trace_length]
                                    [proof_effort] [engineL_trail] [eq_prop]
                                    [store_trace] [trace_extension] [intermediate]
                                    [latest_trace_engine]
                                    [num_traces] [trace_id]
                                    [validity_status] [run_status] [target_bound]
[auto_target_bound]
                                    [precondition_expression]
                                    [postcondition_expression]
                                    [evaluate_in_trace] [c_property_type]
                                    [related_property_type] [is_related_property]
                                    [is_related_cover] [is_related_assert]
                                    [related_properties] [related_covers]
                                    [related_asserts] [function_call_stack]}
                  property_name
```

Returns

Returns a Tcl dictionary with information about the property.

The "-list" switch returns a Tcl list of specified values where each element in the returned list is the value of the corresponding attribute in the attribute list.

Detailed Description

```
Use this command to get information about a property. You can run the command
in either of two ways, with or without the "-list" switch.

property_name
    Return information in the form of a Tcl dictionary. The keys in the
    dictionary are names of attributes and the values in the dictionary are the
    values of the corresponding attributes.

    If an attribute has no value for the specified property, the tool omits it
    from the returned dictionary.

    You can use the returned dictionary to create a Tcl array with the help of
    the "array set" command. See the example below.

    NOTE: Do not rely on the order of the keys in the dictionary. New keys
    might be inserted in the dictionary at any position in a new version of
    Jasper Apps. We strongly advise that you access the returned value from
    this command through Tcl's "dict" command, or by creating an array with
    "array set" (see above).

-list { [name] [task] [source_task] [type] [clock] [edge] [command]
        [expression] [precondition]
        [related_covers] [liveness] [converted] [helper] [sst]
        [app] [file] [pa] [tcl] [error] [disabled]
        [deprecated] [status] [related_cover_status]
        [pseudo_constant] [assume_bound]
        [time] [proof_timestamp] [engine]
        [min_length] [max_length] [sst_max_length] [stem_length] [loop_length]
[trace_length]
        [proof_effort] [engineL_trail] [eq_prop] [store_trace]
        [trace_extension] [intermediate] [latest_trace_engine]
        [num_traces] [trace_id]
        [validity_status] [run_status] [target_bound] [auto_target_bound]
        [precondition_expression] [postcondition_expression]
        [evaluate_in_trace] [c_property_type]
        [related_property_type]
        [is_related_cover] [is_related_assert]
        [related_covers] [related_asserts] [function_call_stack]
      } property_name
    Return a Tcl list of specified values.
```

```
Each element in the returned list is the value of the corresponding
attribute in the attribute list.

If you specify a property that has no value for the specified attribute,
the command fails with an error.

Attributes Available for All Properties
---------------------------------------
. name: The name of the property

. task: The name of the task in which the property resides

. source_task: The name of the task from which the property was copied.

        NOTE: If you remove the source task, this command returns the
        current task as the source task.

. type: The type of the property
        Possible values:
        . assert
        . assert(spv)
        . assert(xprop)
        . assume
        . assume(reset)
        . assume(soft)
        . assume(soft not to)
        . assume(soft to)
        . assume(xprop)
        . assume(cag)
        . assert(noConflict)
        . assert(noDeadEnd)
        . assert(noLoop)
        . cover
        . cover(path)
        . cover(spv)
        . cover(live)

. clock: The names of the clocks that trigger the property

        NOTE: If the tool's fastest clock triggers the property, this
        clock is omitted from the Tcl return.

. edge: A list of pairs consisting of the clock and the edge that triggers
```

the property.

NOTE: If the tool's fastest clock triggers the property, the clock and edge information are omitted from the Tcl return.

. command: The name of the Tcl command that shall be used to operate on the property, for example, to disable it
(possible values: "assert", "assume", and "cover")

. expression: The expression of the property

NOTE: For embedded properties, the tool lists the internal signal that represents the property inside the tool. You can use this signal to refer to the property, for example, in commands such as "get_fanin <signal>".

. precondition: "1" if the property is a precondition, "0" otherwise

. related_covers: A list of covers related to the property
(see "elaborate -create_related_covers")

. related_asserts: A list of assertions related to the property
(see "elaborate -create_related_covers")

. related_properties: A list of assertions and covers related to the property (see "elaborate -create_related_covers")

. liveness: "1" if the property is a liveness property, "0" otherwise

NOTE: Non-liveness properties become liveness when trace extension is "$". See "trace_extension".

. converted: "1" if the property was converted from another property, "0" otherwise

. helper: "1" if the property is a helper assertion

. sst: "1" if the property is an SST target assertion

. app: The short name of the app the property belongs to

. file: a list containing the file name, start and end lines of the property, if the property stems from a source file, or an empty list otherwise

```
. pa: "1" if the property stems from a Proof Accelerator, "0" otherwise

. tcl: "1" if the property was defined using a Tcl command, "0" otherwise
       . This includes properties extracted by Apps.
       . For converted properties, this attribute is copied from
         the original property.

. error: "1" if  the property failed to compile, "0" otherwise

. disabled: "0" if the property is enabled, "1" if it is disabled

. deprecated: "0" if the property is not deprecated, "1" if it is deprecated


       NOTE: Deprecated properties are properties that contain references
       to signals that no longer exist in the design. Signals can be
       removed when you run the "connect" or "elaborate -req" command.

       Deprecated properties behave like disabled properties, that is,
       they are not evaluated in proofs or other flows. Running a new
       "connect" or "elaborate -req" command that re-instates the missing
       signals automatically re-enables the deprecated properties.

. status: The current status of the property
       . Possible values for assertions:
         . proven
         . marked_proven
         . cex
         . ar_cex
         . undetermined
         . unprocessed
         . processing
         . error
       . Possible values for covers:
         . unreachable
         . covered
         . ar_covered
         . undetermined
         . unprocessed
         . processing
         . error
       . Possible values for assumptions:
```

```
          . temporary
          . approved

        NOTE: This argument exists only for backward compatibility, we recommend
        that you use validity_status and run_status instead of this argument.

. related_cover_status: The aggregated proof status of the related covers.
        The returned value is one of the following:
        . red      - At least one related cover has the status
                     "unreachable".
        . yellow   - The condition for "red" is not fulfilled, and at least
                     one related cover has the status "undetermined",
                     "unprocessed", "processing" or "error".
        . ar_green - The condition for "yellow" is not fulfilled, and at
                     least one related cover has the status "ar_covered".
        . green    - The condition for "ar_green" is not fulfilled, and
                     there is at least one related cover. That is, all
                     related covers have the status "covered".
        . white    - There are no related covers.

. pseudo_constant: "1" if the property is a pseudo constant, "0" otherwise

. assume_bound: The length of the bound of the assumption, "0" for
        unbounded assumptions and other kinds of properties

. function_call_stack: The C/C++ function call stack for the property.

Additional Attributes Available for Assertions and Covers
---------------------------------------------------------
. time: The time spent on the property (in seconds)

. proof_timestamp: The wall-clock time and date at which the proof result
  for the property was determined.

  NOTE:
  . The timestamp is returned in a human-readable log-print format, for
    example, "2025-07-11 14:23:45". If you desire, you can convert it to
    epoch-seconds using Tcl's clock scan command. See the example below.
  . The visibilty of this attribute can be controlled. See
    "set_property_info_proof_timestamp" for additional information.

. engine: The engine that delivered the best currently available
        information about the property, or the first engine to
```

```
        determine the property. The engine can be one of the
        selected engines, see "set_engine_mode", or a special (virtual)
        engine:
        . "User" - See "assert -mark_proven" for additional information
        . "PRE"  - When the property was solved during preprocessing, see
                   "set_proof_simplification" for additional information.

. min_length: The minimal length of a possible trace for the property in
        the form of a positive integer or the string "infinite".  This
        information tells you that shorter traces do not exist, which is
        also known as the "bounded proof". The tool displays this number in
        the Property Table in the "Bound" column.

        NOTE: This attribute has no meaning for liveness properties.

. max_length: If a trace exists, the shortest trace is no longer than
        max_length. "infinite" indicates that no upper bound for a possible
        minimal trace has been found. The tool displays this number (or the
        word "infinite") in the Property Table in the "Bound" column unless
        "sst_max_length" is smaller, see "sst_max_length". If a trace
        exists, min_length is equal to max_length when the trace is known to
        be minimal.

        NOTE: This attribute has no meaning for liveness properties.

. trace_bound: The set of possible lengths of the shortest trace represented
       as a list of intervals of lengths. An interval {a b} means the
       shortest trace may have a length a <= l <= b. Intervals with a
       single value {a a} are abbreviated as a, and an unlimited value is
       represented with $. For example, the set of intervals {1 3 {5 8} 10}
      means the shortest trace is 1, 3, 5-8, or 10 cycles long.

. sst_max_length: If a trace exists for any of the helper assertions in the
        task, the shortest trace is no longer than its
        "sst_max_length". "infinite" indicates that no upper bound for a
        possible trace has been found. The tool displays the
        "sst_max_length" number if smaller that "max_length" followed by
        "(SST)" in the Property Table in the "Bound" column.

        NOTE: This attribute is generated by "prove -sst". It has no meaning
        for non-helper properties.

. stem_length: The length of stem portion of the trace associated with the property
```

for liveness properties, "0" if there is no trace.

. loop_length: The length of loop portion of the trace associated with the property
for liveness properties, "0" if there is no trace.

. trace_length: The length of the trace associated with the property, "0"
if there is no trace

. proof_effort: The Proof Effort is a rough indication of how much effort
the engine has spent establishing a proven or unreachable result.
Different engines use different scales. The value is also
contingent on the full proof settings, which other properties were
proven together with this, and on indeterministic ProofGrid timing
variations. A very low value indicates that this engine did not
have to spend a lot of effort proving this property. A very low
value does not necessarily mean that the property is low value or
trivial.

NOTE: For liveness properties, the value is the maximum trace
attempts run on the property as can be seen in the ProofGrid
Manager. You can think of this representation as "the tool has
considered traces this long (stem + loop) for this property." And
you can use this information as a weak kind of engine-specific
progress indicator.

. engineL_trail: The sequence of properties used to find a trace by engine
L; "{}" if another engine found the trace or no trace has been
found. See "set_engineL_generate_trails".

. eq_prop: If the property is known to semantically have the same status
as other properties, the tool lists the next property in a circular
chain of equivalent properties. Otherwise, it displays {}.

NOTE: If two properties are listed as equivalent here, it means
that a bound, or a minimal trace, or a proof of the absence of
traces ("proven" or "unreachable") for one also applies to the
other. The tool does not guarantee that all mentioned properties at
all times will show the same actual status.

. store_trace: See "-set_store_trace" in the "assert" and "cover" commands.

. trace_extension: See "-set_trace_extension" and "-clear_trace_extension"
in the "assert" and "cover" commands, and the "set_trace_extension"

```
        command.

. intermediate: "1" if intermediate is set; otherwise, "0"

. latest_trace_engine: The engine that generated the latest trace of the
        property.

. num_traces: The number of traces stored for the property.

. trace_id: The list of trace IDs for all traces stored for the property.

. validity_status: Current validity status of the property.
        . Possible values for assertions:
          . proven
          . marked_proven
          . cex
          . ar_cex
          . undetermined
          . bounded_proven_auto
          . bounded_proven_user
          . unknown
          . error
        . Possible values for covers:
          . unreachable
          . covered
          . ar_covered
          . undetermined
          . bounded_unreachable_user
          . unknown
          . error
        . Possible values for assumptions:
          . temporary
          . approved
. run_status: Current run status of the property.
       . Possible values:
          . unprocessed
          . queued
          . processing
          . processed

. target_bound: Current user-defined target bound of the property.

  NOTE: Use "assert -set_target_bound" or "cover -set_target_bound"
```

commands to configure the target bound of the property. A return value of "0" means infinite/not set.

. auto_target_bound: The target bound that is automatically inferred by the tool. The return value "0" means infinite/not set.

NOTE: This is automatically set by the tool.

. precondition_expression: The expression specified as precondition for extended covers.

. postcondition_expression: The expression specified as postcondition for extended covers.

. evaluate_in_trace: Return "1" if the property was processed by the "prove" or "hunt" command due to the "evaluate_in_trace" switch and the result was obtained from that evaluation; otherwise, return "0".

. Possible values (if "1"):
. covered
. cex
. undetermined

. c_property_type: Return the type of C++/C assertions in the c2rtl app.

. Possible values:
. none
. c_sanity
. c_user

. related_property_type: The related property type of the property

. Possible values:
. none
. precondition
. contrapositive
. witness
. precondition_infinite
. precondition_late
. infinite_contrapositive
. safety_in_liveness_assert

. is_related_cover: Return "1" if the property is a related cover;
  otherwise, return "0".

. is_related_assert: Return "1" if the property is a related assertion;
  otherwise, return "0".

. is_related_property: Return "1" if the property is a related assertion or
  a related cover; otherwise, return "0".
. function_call_stack: Return the call stack information. Call stack is
  available for C/C++ assertions and sanity checks.

Example

```
get_property_info c0
name c0 task <embedded> type cover command cover expression {##5 e==8'hFF} precondition
0 related_covers {} liveness 0 converted 0 helper 0 sst 0 intermediate 1 app fpv file
{} pa 0 tcl 1 error 0 deprecated 0 disabled 0 status unprocessed validity_status
unknown run_status unprocessed target_bound 4 related_cover_status white
pseudo_constant 0 assume_bound 0 time 0.0 engine ? min_length 1 max_length infinite
trace_length 0 latest_trace_engine ? proof_effort 0 x_interpretation default_x eq_prop
{} store_trace 1 engineL_trail {} x_sources {} trace_extension {} num_traces 0 trace_id
{} proof_timestamp undefined

array set info [get_property_info <embedded>::c0];
foreach attr [lsort [array names info]] { puts "$attr: $info($attr)" }
app: fpv
assume_bound: 0
command: cover
converted: 0
deprecated: 0
disabled: 0
engine: N
engineL_trail:
eq_prop:
error: 0
expression: ##5 e==8'hFF
helper: 0
sst: 0
intermediate: 1
latest_trace_engine: N
liveness: 0
max_length: 6
```

```
min_length: 6
name: c0
num_traces: 1
file: {}
pa: 0
tcl: 1
precondition: 0
proof_effort: 0
proof_timestamp: 2025-07-11 14:23:45
pseudo_constant: 0
related_cover_status: white
related_covers:
run_status: processed
status: covered
store_trace: 1
target_bound: 4
task: <embedded>
time: 0.101876020432
trace_extension:
trace_id: 1
trace_length: 6
type: cover
validity_status: covered
x_interpretation: default_x
x_sources:


get_property_info -list {status engine time min_length trace_length} prop
cex L 12.3456 9 10
[get_property_info -list command $propName] -disable $propName

# Listing related_covers with multiple attributes
# -----------------------------------------------
get_property_info -list {status engine time min_length trace_length related_covers}
prop
cex L 12.3456 9 10 {prop:precondition1 prop:precondition2 prop:precondition3}
...

# Listing related_covers as a single attribute
# --------------------------------------------
get_property_info -list related_covers prop
{prop:precondition1 prop:precondition2 prop:precondition3}
```

```
# Manipulating proof_timestamp attribute
# ----------------------------------------
set formatted_time [lindex [get_property_info c0 -list proof_timestamp] 0]
tcl_clock scan $formatted_time -format {%Y-%m-%d %H:%M:%S}
1752240431
```

See Also

assert,
assume,
cover,
elaborate,
get_status,
get_trace_info

get_property_list

Returns a list of properties whose attributes satisfy specified value criteria.

Syntax

```
get_property_list [-include {attribute_1 value_list_1 attribute_2 value_list_2 ...}]
                  [-exclude {attribute_1 value_list_1 attribute_2 value_list_2 ...}]
                  [-regexp]
                  [-task task_name+]
                  [-exclude_env]
                  [-no_task_prefix]
```

Returns

Returns a list of properties whose attributes satisfy specified value criteria.

Detailed Description

Use this command to get a list of properties whose attributes satisfy value criteria specified by the "-include" and "-exclude" switches.

-include {attribute_1 value_list_1 attribute_2 value_list_2 ...}
    Include in the returned Tcl list the names of properties where ALL of the specified "attribute_N" have values in their corresponding "value_list_N". A property is included only if it satisfies every attribute condition specified (AND logic).

-exclude {attribute_1 value_list_1 attribute_2 value_list_2 ...}
    Exclude from the returned Tcl list the names of properties where AT LEAST ONE of the specified "attribute_N" has a value in specified "value_list_N". A property is excluded if it matches any single attribute condition specified (OR logic).

    NOTE: Exclusion takes precedence over inclusion, which means that if you use an attribute value with both the "-include" and "-exclude" switches, any property having this attribute value is excluded from the returned Tcl list.

The following is the list of attributes supported by "-include" and "-exclude". Refer to the TclHelp for "get_property_info" for more details, including possible values.

. app
. assume_bound
. command
. converted
. disabled
. error
. helper
. sst
. engine
. eq_prop
. expression
. intermediate
. is_related_cover
. is_related_assert
. latest_trace_engine
. liveness
. max_length
. min_length
. name
. pa
. precondition
. proof_effort
. pseudo_constant
. related_cover_status
. related_property_type
. run_status
. status
. store_trace
. target_bound
. task
. tcl
. trace_id
. sst_max_length
. stem_length
. loop_length
. trace_length
. type
. time

```
    . validity_status
    . c_property_type

    NOTE:
    . The arguments for the "-include" and "-exclude" switches are lists
      with an even number of elements. Those elements must come in attribute/value
      pairs.
    . Values specified for numeric attributes like "trace_length" must be
      preceded by one of the following comparison operators: <, <=, >, >=, ==, !=.
      In the case of the equal operator, you have the option of specifying ==
      or not.
    . "-include" and "-exclude" switches support regular expressions in
      attributes values. Refer to the examples.
    . The "value_list_N" elements are lists. This makes it possible to specify
      more than one condition for which the attribute should match. For instance,
      the command "get_property_list -include {status {covered proven}}" will
      return properties whose status is either "covered" or "proven".
    . The attributes received by "-include" and "-exclude" that are compatible
      with the "-regexp" switch (see the list under "-regexp" below) interpret
      wildcards by default when you use the wildcard characters "*" or "?".
      When you use wildcards, you must escape all characters with special
      meaning in a wildcard expression, that is, "?", "*", "[", and "]".

-regexp
    Use "-regexp" to specify that you are using a regular expression to specify
    attributes for "-include", "-exclude", and "-task". "-regexp" applies to the
    "-task" switch and for the following attributes of "-include" and "-exclude":

      . app
      . command
      . converted
      . disabled
      . error
      . helper
      . sst
      . engine
      . eq_prop
      . expression
      . intermediate
      . latest_trace_engine
      . liveness
      . name
      . pa
```

```
      . precondition
      . pseudo_constant
      . related_cover_status
      . related_property_type
      . run_status
      . status
      . store_trace
      . task
      . tcl
      . type
      . validity_status
      . c_property_type

    NOTE:
    . Because many regular expressions contain characters that have special
      meaning in Tcl, a regular expression often includes characters that need
      to be escaped. Escape all characters that are not alphabetic or numeric
      in the content of the "value_list_N" of the attributes above. One way to
      escape an expression is to surround it with a pair of curly braces.

-task task_name+
    Use "-task" to apply this command to the specified tasks instead of the
    current task.

    NOTE: "-task" is an alias for the task attribute of "-include". If both are
    used, "-include" takes precedence.
    Example:
    get_property_info -task a
    a.prop

    get_property_info -task aa
    aa.prop

    get_property_info -include {task aa} -task a
    aa.prop

    NOTE:
    "-task" interprets wildcards on the content of "task_name" by default. Use
    the special wildcards "*" or "?" to access this functionality. Using
    wildcards makes it necessary to escape all characters that are not
    alphabetic or numeric.

-exclude_env
```

```
    Use "-exclude_env" to suppress environment properties.

-no_task_prefix
    Use "-no_task_prefix" to suppress task name prefixes on property names.
```

Example

```
# The following command lists all enabled assertions and covers
# with status different from proven, cex, unreachable, and cover.
get_property_list -include {type {assert cover} disabled {0}} \
                  -exclude {status {proven cex unreachable covered}}
# The following commands specify numeric attributes
get_property_list -include {trace_length {<20}}
get_property_list -include {trace_id {==2}}
get_property_list -include {trace_id {2}}
# The following command uses wildcard to get "top.inst01[_vld|_valid]"
# and "top.inst02[_vld|_valid]" properties
get_property_list -include {name {{top.inst0*\[_vld|_valid\]}}}
# The following command uses "-regexp" so it needs to escape "assert(noConflict)"
get_property_list -include {name {{top.inst0.*\(_vld|_valid\)}}} \
                  -exclude {type {{assert\(noConflict\)}{assume\(reset\)}}} -regexp
```

See Also

get_property_info

get_reset_info

Gets a reset VCD or customized report and saves reset values.

Syntax

```
get_reset_info [-all | (-x_value [-with_reset_pin])]
               [-bit_blast]
               [-cycle N]
               [-list
                ([type] [cycle] [module] [instance] [configuration])+
                [-sort_by (type | cycle | module | instance)]
                [-silent]]
                [(-filter_in |-filter_out) pattern_list [-regexp |-exact]]
                |-get_value (signal_name [cycle | cycle:cycle]
                  [-radix ( 2 | 8 | 10 | 16 | bin | oct | dec
                            | signed_dec | hex | signed_hex | bits_set
                            | floating_point | string)])
               |-get_length
               |-get_unstable
               |-save_reset_vcd file_name
                [-include_internal]
                [-last_cycle cycle]
                [-force] [-visible_only]
               |-save_reset_fsdb file_name
                 [-last_cycle cycle]
                 [-force] [-visible_only]
               |-save_reset_shm directory_name
                 [-last_cycle cycle]
                 [-force] [-visible_only]
               |-save_reset_vwdb directory_name
                 [-last_cycle cycle]
                 [-force] [-visible_only]
               |-save_values file_name [-all |-x_value] [-bit_blast] [-comments]
                   [-include_internal] [-include_encrypted]
               |-constant_registers
               |-formal -suggest_bound
```

Returns

```
Returns the list of flops and latches and their values as Tcl return values.
The format is:
. Without "module" or "instance":
  {sig1 value1} {sig2 value2}
. With "module" and "instance":
  {sig1 value1 module1 instance1} {sig2 value2 module2 instance2}
. With "instance" but not "module":
  {sig1 value1 instance1} {sig2 value2 instance2}
. With "type" and "cycle" but not "instance" or "module":
  {register_type_1 sig1 value1 cycle1} {register_type_2 sig2 value2 cycle2}
```

Detailed Description

```
Use this command to return reset information. If you do not supply any
switches, the tool lists flops and latches with defined values (0 or 1). Use
the optional switches to customize a return that you can then process to build
a report. Use the "save_*" switches to save a file that can be used to specify
the reset state.

-all
    List all design flops and latches and their values.

    If a register does not have a reset value, the tool prints value X.

-x_value
    List only the uninitialized flops and latches (X).

    Use "-with_reset_pin" to list only the uninitialized flops that have
    asynchronous reset. This command does not report flops abstracted with
    "abstract -init_value", "stopat -env", or "stopat -reset".

-bit_blast
    List each bit of a flop or latch separately.

    If all bits are 0, the tool aggregates them. For example, 10'b0000000000
    prints as 10'b0.

-cycle N
    Return flop and latch reset values for the specified cycle.
```

```
    If you do not specify "-cycle", the last cycle of the reset trace is the
    default.

-list ([type] [cycle] [module] [instance] [configuration])+
      [-sort_by (type | cycle | module | instance)]
      [-silent]
    List the specified additional information for each register.

    . Use "type" to include the type of register, for example, D flop.

    . Use "cycle" to include the reset cycle.
      NOTE:
      . If you use this argument, and you specified reset with "reset
        -init_state", the tool issues a warning because there is no reset
        cycle for this case.
      . For reset expression mode, reset analysis calculates flop values that
        have their output changed due to some clock activity during the
        transition between reset and proof. "get_reset_info" reports the last
        cycle of the reset trace, but the first cycle of formal analysis will
        reflect the potential changes that can happen during the transition
        between reset and formal analysis.

    . Use "module" to include the module where the register is instantiated.

    . Use "instance" to include the name of the instance where the register is
      instantiated.

    . Use "configuration" to include the reset configuration that was specified
      with the "reset" command.

    . Use "-sort_by" with "type", "cycle", "module", or "instance" to specify
      the sorting scheme.
      NOTE: When sorting by "cycle", the list is from highest to lowest.

    . Use "-silent" to return the list as Tcl return values without printing
      the list on the screen.

-filter_in pattern_list [-regexp |-exact]
    Filters in the returned list based on a list of wildcard strings (default).

    Use "-regexp" to change the filter expression to a regular expression or
    "-exact" for literal matching.
```

```
-filter_out pattern_list [-regexp |-exact]
    Filters out the returned list based on a list of wildcard strings
    (default).

    Use "-regexp" to change the filter expression to a regular expression or
    "-exact" for literal matching.

-get_value (signal_name [cycle | cycle:cycle]
             [-radix ( 2 | 8 | 10 | 16 | bin | oct | dec | signed_dec | hex
                      | signed_hex | bits_set | floating_point | string)])
    Return the value for a specified signal from a simulated reset trace.

    . Specifying cycles is optional.
      . If you do not specify a cycle or cycle range, the tool returns
        the signal values of all cycles.
      . If you specify a single cycle (for example, 7), you are asking the
        tool to get the value for the specified signal in cycle seven.
      . You can specify a range of cycles; for example, 2:3.
      . Use $ to mean the last cycle. For example, 1:$ is equivalent to always
        (which is the default when you do not specify cycles) and 4:$ means
        from cycle four until the end of the trace.
    . Use "-radix" to return values in the specified radix.

    NOTE: This command is not supported when using a reset file in conjunction
    with reset expressions.

-get_length
    Get the number of cycles reset analysis required for convergence.

    Use "get_reset_info -get_length" without additional switches.

-get_unstable
    List the registers whose values did not converge at the end of the
    reset analysis.

    During the reset computation, the tool samples all flops and latches at
    specific cycles to check whether their values converged, and
    "-get_unstable" returns the ones that did not pass this convergence test.
    The convergence test for a register is successful when the tool detects
    the absence of value changes between the cycles where the sampling occurs.

    Use "get_reset_info -get_unstable" without additional switches.
```

```
-save_reset_vcd file_name [-include_internal] [-last_cycle cycle] [-force]
    [-visible_only]
    Create a VCD file from a simulated reset trace.

    . Use "file_name" to specify the VCD file name.
    . Use "-include_internal" to include internal signals in the VCD file.
    . Use "-last_cycle" to define the last cycle of the saved trace.
    . Use "-force" if the specified VCD file already exists and you want
      to overwrite it.
    . Use "-visible_only" to write only RTL-defined signals to the file.

    CAUTION: Loading files with the switch "-include_internal" carries a risk
    of a mismatch in internal signals that can lead to false proof results in
    some cases. Internal signals in reset files might not be the same as those
    generated by a different Jasper release version or a different version
    of the design or a different design setup and configuration. (Design setup
    and configuration differences include the order in which files are passed
    to "analyze" and "elaborate" and the order in which assumptions and
    assertions are created.)

-save_reset_fsdb file_name [-last_cycle cycle] [-force] [-visible_only]
    Create an FSDB file from a simulated reset trace.

    . Use "file_name" to specify the FSDB file name
    . Use "-last_cycle" to define the last cycle of the saved trace..
    . Use "-force" if the specified FSDB file already exists and you want
      to overwrite it.
    . Use "-visible_only" to write only RTL-defined signals to the file.

    IMPORTANT:
    . Jasper Apps writes 6.3 FSDB traces and reads version 6.6 FSDB
      traces. FSDB Reader 6.6 was introduced by Verdi version X-2025.06.
    . FSDBs you generate with the latest Jasper are backwards compatible
      with all Jasper versions from 2025.03 FCS and newer. In addition,
      they are backwards compatible with older versions of Verdi released
      during that same period.
    . Jasper Apps supports backwards compatibility, that is, the tool can
      read FSDBs generated from older versions of Jasper Apps and
      Synopsys Verdi.

    LIMITATION: Compound blackboxed output values are saved as instances
    hierarchy.
```

```
-save_reset_shm directory_name [-last_cycle cycle] [-force] [-visible_only]
    Create an SHM directory from a simulated reset trace.

    . Use "directory_name" to specify the SHM directory name.
    . Use "-last_cycle" to define the last cycle of the saved trace.
    . Use "-force" if the specified SHM directory already exists and you want
      to overwrite it.
    . Use "-visible_only" to write only RTL-defined signals to the file.

    LIMITATION: Compound blackboxed output values are saved as instances
    hierarchy.

-save_reset_vwdb directory_name [-last_cycle cycle] [-force] [-visible_only]
    Create a VWDB directory from a simulated reset trace.

    . Use "directory_name" to specify the VWDB directory name.
    . Use "-last_cycle" to define the last cycle of the saved trace.
    . Use "-force" if the specified VWDB directory already exists and you want
      to overwrite it.
    . Use "-visible_only" to write only RTL-defined signals to the file.

    LIMITATION: Compound blackboxed output values are saved as instances
    hierarchy.

-save_values file_name [-all |-x_value] [-bit_blast] [-comments]
    [-include_internal] [-include_encrypted]
    Generate a file that can be used with "reset -init_state".

    If you do not supply any additional switches, the tool only saves
    flops and latches with defined values.

    . Use "-save_values" to do the following:
      . Save the registers and their values to a file that you can read
        with the "reset -init_state" command
      . Save the results of a complex "reset" command for use in another
        session

    . Use "-all" to save values of all design flops and latches. If a
      register does not have a reset value, it saves as value X.

    . Use "-x_value" to save only the uninitialized flops and latches (X).

    . Use "-bit_blast" to list each bit of a register separately.
```

```
      If all bits are 0, the tool aggregates them. For example, 10'b0000000000
      prints as 10'b0.

    . Use "-comments" to list comments for each register listed in the output
      file.

    . Use "-include_internal" to save values of internal flops.

    . Use "-include_encrypted" to capture reset values of encrypted flops in an
      encrypted file.

    CAUTION: Loading files with the switch "-include_internal" carries a risk
    of a mismatch in internal signals that can lead to false proof results in
    some cases. Internal signals in reset files might not be the same as those
    generated by a different Jasper release version or a different version
    of the design or a different design setup and configuration. (Design setup
    and configuration differences include the order in which files are passed
    to "analyze" and "elaborate" and the order in which assumptions and
    assertions are created.)

-constant_registers
    Return the list of flops and latches that have a constant value after reset
    analysis due to propagating design constants, environmental assumptions,
    and reset.

-formal -suggest_bound
    Return relevant information from the formal reset mode configuration.

    Use "-suggest_bound" to get a bound estimate you can use with "reset
    -formal -bound".
```

Example

```
# Example for capturing information needed for customized reports:
analyze -sv my_design.v
elaborate
clock clk
reset {~resetn}
get_reset_info -bit_blast -list type cycle

get_reset_info -silent
{sig1 value1} {sig2 value2}

get_reset_info -list type cycle module instance -silent
{sig1 value1 type1 cycle1 module1 instance1} {sig2 value2 type2 cycle2 module2
instance2}
```

See Also

jforeach

get_root_name

Gets the root of the instance tree.

Syntax

```
get_root_name
```

Returns

Returns a string (identifier name) for the root of the instance tree of the design.

Detailed Description

Use this command to get the root of the instance tree.

Example

```
get_root_name
top
```

See Also

elaborate,
get_top_module

get_signal_info

Extracts information about a specified signal.

Syntax

```
get_signal_info signal_name

get_signal_info -type signal_name

get_signal_info -logic signal_name

get_signal_info -port_type signal_name

get_signal_info -indexes [-show_unpacked] signal_name

get_signal_info -bit_blast signal_name

get_signal_info -width signal_name

get_signal_info -equiv [-forward] signal_name

get_signal_info -get_module signal_name

get_signal_info -get_instance [-split] signal_name

get_signal_info -get_signal [-full] signal_name

get_signal_info -strength signal_name

Declaration Source Location
---------------------------
get_signal_info -declaration signal_name+
               (-file_name |-line)+ [-silent]

Driver Source Location
----------------------
get_signal_info -driver signal_name+
               (-file_name |-line )+ [-ignore_stopats] [-silent]

User-Defined Type
-----------------
get_signal_info -type_definition signal_name
               [-typed_list] [-silent]
```

Returns

Returns the signal type, logic, port direction, indexes, expanded name into its bits, or width.

Detailed Description

Use this command to extract information about a signal.

NOTE: If the signal belongs to an instance of a module, you must include the hierarchy in the signal name. If you do not include the hierarchy, the command applies to the top-level module.

signal_name
    The tool uses "-type" by default if you do not specify any switch.

-type signal_name
    Return the signal type for the specified signal.

    The signal type will be one of the following:
    . output   - signal is a top module output
    . input   - signal is a top module input
    . inout   - signal is both a top module input and output
    . undriven - signal has no driving logic
    . MPRAM   - signal is a RAM net
    . logic   - signal is none of the above

-logic signal_name
    Return the signal logic type for the specified signal.

    The return will be one of the following:
    . latch
    . flop
    . wire
    . MPRAM

-port_type signal_name
    Return the port type for the specified signal.

    The return will be one of the following:
    . input - signal is a module input
    . output - signal is a module output

```
    . inout  - signal is both a module input and output
    . undef  - signal is not a module port

-indexes [-show_unpacked] signal_name
    Return details about the signal: MSB, LSB, fields (for structs and unions),
    and number of read and write ports (for multiport RAMs).

    The command returns one of the following:
    . "Bit" (in cases where the specified signal is not indexed)
    . "1D_Array" followed by the MSB and LSB indexes
    . "Struct" followed by a list of fields
    . "Union" followed by a list of fields
    . "MPRAM" followed by the number of read, write, and asynchronous write
      ports
    . "MD_Array" followed by each dimension's MSB and LSB indexes
      NOTE: Use "-show_unpacked" to include the string "unpacked" in the return
      for any unpacked range in an MD_Array result.
    . "Enum" followed by all enum values.

-bit_blast signal_name
    Return the individual bits of a signal, most significant bit first.

    Use this switch to print the individual bits of a signal within its
    specified range. For example, consider the following signal A:
    wire [2:0] A;

    If you run "get_signal_info -bit_blast A", the result is the following:
    A[2] A[1] A[0]

    You can also specify a range within a signal, for example, "get_signal_info
    -bit_blast A[1:0]" returns the following:
    A[1] A[0]

-width signal_name
    Return the width of a signal.

    For example, consider the following signal A:
    wire [2:0] A;

    If you run "get_signal_info -width A", the result is the following:
    3

-equiv [-forward] signal_name
```

```
    Return a list of signals that are equivalent to the specified signal.
    The signal_name can be a signal or a bit.
    NOTE: Compound signals are not supported.
    . Use "-equiv signal_name" with no additional switches to print an ordered
      list of equivalent drivers.
      NOTE: This command is unidirectional (backwards).
    . Use "-equiv signal_name -forward" to print a set of equivalent loads.
      NOTE: The set that "-forward" returns is not ordered.

    For example, consider the following signals A, B, C and D:
    wire A, B, C, D;

    assign A=B;
    assign B=C;
    assign D=B;

    If you run "get_signal_info -equiv A", the result is the following:
    A B C

    If you run "get_signal_info -equiv C -forward", the result is the following:
    B A D

-get_module signal_name
    Return the name of the module in which the signal is declared.

    For example, consider the following module:
    module top ();
         wire A;
    endmodule

    If you run "get_signal_info -get_module A", the result is the following:
    top

-get_instance [-split] signal_name
    Return the name of the instance in which the signal is instantiated.

    Use "-split" to print the instance path list

    For example, consider the following code:
    module top();
        mod1 inst();
    endmodule
    module mod1();
```

```
        wire A;
    endmodule;

    If you run "get_signal_info -get_instance inst.A", the result is the
    following:
    inst1

    If the signal is in deeper hierarchy, the result can look like the
    following:
    inst1.inst2.inst3

    If you run "get_signal_info -get_instance inst1.inst2.inst3.A -split", the
    result is the following:
    inst1 inst2 inst3

-get_signal [-full] signal_name
    Return the name of the signal within the instance hierarchy.

    Use "-full" to get the same signal name, but without any slice operator.

    For example, consider the following code:
    module top();
        mod1 inst();
    endmodule
    module mod1();
        wire A[3:0];
    endmodule;

    If you run "get_signal_info -get_signal inst.A", the result is the
    following:
    A

    If you run "get_signal_info -get_signal inst.A[1:0]", the result is the
    following:
    A

    If you run "get_signal_info -get_signal inst.A[1:0] -full", the result is
    the following:
    inst.A

-strength signal_name
    Returns the strength of a signal.
```

```
    For example, consider the following signals A, B, C and D:
    wire A;
    supply1 B;
    pullup (C);
    pulldown (D);

    "get_signal_info -strength A" returns:
    {strong0, strong1}

    "get_signal_info -strength B" returns:
    {highz0, supply1}

    "get_signal_info -strength C" returns:
    {highz0, pull1}

    "get_signal_info -strength D" returns:
    {pull0, highz1}

    NOTE: This switch does not work with tri-state gates, since their strength
    is conditional. For this scenario, the strength strong0 and strong1 are
    returned.
    For instance:

    module top;
        wire a, b, c, cond, out;
        and (c, a, b); // strong0, strong1
        nmos (out, c, cond); // strong or highz
    endmodule

    "get_signal_info -strength out" returns:
    {strong0, strong1}

Declaration Source Location
===========================
-declaration signal_name+
        (-file_name |-line)+ [-silent]
    List the source location of the declaration of the signal.
    . Use "-file_name" to get the name of the file where the signal is
      declared.
    . Use "-line" to get the line number where the signal is declared.
    . Use "-silent" to turn off output to the screen and return design info
      with a Tcl return value.
```

```
      Output:
      Location: <left line num-right line num, left column num-right column num>

      Return value:
      Location: <left line num-right line num, left column num-right column num>

Driver Source Location
======================
-driver signal_name+
        (-file_name |-line)+ [-ignore_stopats] [-silent]
    List the source location of the drivers of the signal.
    . Use "-file_name" to get the name of the file where the signal is
      assigned.
    . Use "-line" to get the line number where the signal is assigned.
    . Use "-ignore_stopats" to get the driver source location information even
      if the signal has a stopat.
    . Use "-silent" to turn off output to the screen and return design info
      with a Tcl return value.

      Output:
      Location: <left line num-right line num, left column num-right column num>

      Return value:
      Location: <left line num-right line num, left column num-right column num>

User-Defined Type
=================
-type_definition signal_name
                [-typed_list] [-silent]
    List the user-defined type of the specified signal or compound, followed by
    its information.

    This command provides various information for a specified user-defined
    type. It provides the following:

    . Type name - the name of the user-defined type.

    . File - the path of the file where the type is declared.

    . Source Location - the lines and columns where the type is declared.

    . Scope - the scope where the type is declared. The scope can be a module
      or package.
```

```
    . Library - the library where the scope is, if it exists.

    . Use "-typed_list" to change the return format by adding a type element
      before each of the requested lists.

      The list format is as follows:
      { signal { <signal_name>+ } type_name { <user_defined_type_name>+ } file {
<file_name>+ } source_location { <(start_line, start_column), (end_line, end_column))>+
} scope { <scope_name>+ } library { <library_name>+ } }

    . Use "-silent" to turn off output to the screen and return design info
      with a Tcl return value.
```

Example

```
get_signal_info -type s1
undriven

get_signal_info -indexes a -show_unpacked
MD_Array unpacked 3 0 6 0

get_signal_info -driver wb3_data_o -file_name -line
    Location: ./design/arb_wb.v: 26-26,16-25
```

See Also

get_design_info,
get_flop_info,
get_latch_info,
get_fanin,
get_fanout

get_signal_list

Gets a list of unique signals.

Syntax

```
get_signal_list signal_list
    [-union |-intersection |-subtraction] signal_list
    [-pattern_filter_in pattern -regexp]
    [-pattern_filter_out pattern -regexp]
    [-bit_blast]
    [-case_sensitive]
    [-force]
    [-aggregate]
```

Returns

Returns a list of signals.

Detailed Description

Use this command to get a list of signals.

-union signal_list
    Return the union of signals in the first and second list.

    For example, consider the following signals A and B:
    wire A;
    wire B;

    If you run "get_signal_list {A} -union {B}", the result is the following:
    A B

-intersection signal_list
    Return the intersection of signals in the first and second list.

    For example, consider the following signals A and B:
    wire A;

```
    wire B;

    If you run "get_signal_list {A} -intersection {A B}", the result is the
    following:
    A

-subtraction signal_list
    Return the subtraction of signals in the first and second list.

    For example, consider the following signals A and B:
    wire A;
    wire B;

    If you run "get_signal_list {A B} -subtraction {A}", the result is the
    following:
    B

-pattern_filter_in pattern
    Return the result list being filtered by the pattern. The results will be a
    list of signals that match the specified pattern.

    For example, consider the following signals A and B:
    wire A;
    wire B;

    If you run "get_signal_list {A B A} -pattern_filter_in A* -regexp", the
    result is the following:
    A

-pattern_filter_out pattern
    Return the result list being filtered by the pattern. The result will be a
    list of signals that do not match the specified pattern.

    For example, consider the following signals A and B:
    wire A;
    wire B;

    If you run "get_signal_list {A B B} -pattern_filter_out A* -regexp", the
    result is the following:
    B

-bit_blast signal_name
    Return the individual bits of a signal with the most significant bit first.
```

```
    Use this switch to print the individual bits of a signal within its
    specified range. For example, consider the following signals A and B:
    wire [2:0] A;
    wire [2:0] B;

    If you run "get_signal_list {A B} -bit_blast", the result is the following:
    A[2] A[1] A[0] B[2] B[1] B[0]

-force
    Do not perform name validation.

    If you use this switch, an invalid signal name does not trigger an error.

-aggregate
    Return the list of the signals aggregated if they are part of the same structure.

    For example, consider the following signals S.a, S.b:
    struct {
        bit a;
        bit b;
    } S;

    If you run "get_signal_list {S.a S.b} -aggregate", the result is the following:
    S
```

Example

```
get_signal_list {A B} -union {C D}
```

get_status

Gets the aggregated proof status of a set of properties.

Syntax

```
get_status property_name_tcl_list [-validity] [-run] [-regexp]
```

Returns

The aggregated proof status of the specified properties (proven, cex, covered, unreachable, unprocessed, undetermined, etc.) or the validity/run status, depending on the options provided.

Detailed Description

Use this command to see the proof status of one or more assertions or cover properties.

NOTE:
. To get the status of properties not in the current task, prepend the property names with their task name.
. It is possible to combine properties from several tasks in the same list.
. This command supports wildcards and regular expressions. The default is wildcards. Use the "-regexp" switch to use regular expressions instead.

If you use neither "-validity" nor "-run", the possible return values are those listed below, according to property type:

. assertions
  . proven
  . marked_proven
  . cex
  . ar_cex
  . undetermined
  . unprocessed
  . processing
  . error

```
. covers
  . unreachable
  . covered
  . ar_covered
  . undetermined
  . unprocessed
  . processing
  . error

. mix of assertions and covers
  . determined
  . ar_determined
  . undetermined
  . unprocessed
  . processing
  . error
  . no_properties

NOTE:
. If you run "get_status" on two or more properties that have a status of
  error, unprocessed, undetermined, or processing, the tool prioritizes the
  statuses in that order. That is, if there is one property that has status
  "error" and another one that has status "unprocessed", the result of the
  command will be "error".
. If you specify a mix of assertions and covers, the return is "determined" if
  all assertions have status "proven" or "cex" and all covers have status
  "unreachable" or "covered".
. If you specify a mix of assertions and covers and the conditions for
  returning "determined" are not fulfilled, the return is "ar_determined" if
  all assertions have status "proven", "cex", or "ar_cex" and all covers have
  status "unreachable", "covered", or "ar_covered".

If the property_name_tcl_list contains two or more properties, and you use
"-validity", "-run", or both switches, Jasper Apps prioritizes the return
based on the following order.

. Validity status:
  . error
  . undetermined
  . determined

. Run Status:
```

```
  . unprocessed
  . queued
  . processing
  . processed

-validity
    Return the aggregated validity status of a set of properties.

-run
    Return the aggregated run status of a set of properties.

-regexp
    Use regular expressions instead of wildcards for property name matching.
```

Example

```
get_status {p1}
proven

get_status {p2}
covered

get_status {p1 p2}
determined

get_status {p3}
unprocessed

get_status {*}
unprocessed

get_status -validity -run {p1 p2 p3}
determined

get_status -run {p1 p2 p3}
processed
```

See Also

get_property_info,
prove

get_tool

Returns the requested tool, application(s), or mode.

Syntax

get_tool [(-batch |-app |-start_app |-list_apps |-design_loaded)]

Returns

Without arguments: abbreviation for the tool running your script. With arguments: true, false, or an abbreviation for the current app.

Detailed Description

Use this command to determine which tool is running the script. This command can be useful for applying different settings depending on the tool that is running the script.

Use the individual switches for this command in Jasper Apps to access how the tool has been configured.

-batch
    Return "true" if the tool was launched in batch mode.

-app
    Return the current app.

-start_app
    Return the app that was active when the tool launched.

-list_apps
    Return the list of available apps.

-design_loaded
    Return "true" if a design is loaded.

Example

```
get_tool –app
```

get_top_module

Gets the top module or entity to be analyzed formally.

Syntax

```
get_top_module [-no_parameters]
```

Returns

Returns a string (identifier name) for the top hierarchical module/entity found in the design.

Detailed Description

Use this command to get the top module or entity that will be analyzed formally. The top module name contains any parameters used during elaboration, including their values.

-no_parameters
    Return the top module name without parameters.

Example

```
get_top_module
top
```

See Also

elaborate,
get_root_name

get_trace_info

Returns information about a trace.

Syntax

```
get_trace_info trace_id
```

Returns

Returns a list of key/value pairs with information about the trace.

Detailed Description

Use this command to get information about a trace.

trace_id
: Return information in the form of a list with an even number of elements. The first element in each pair is the name of an attribute, and the second element in each pair is the value of the previous attribute.

  If an attribute has no value for the specified property, the tool omits it from the returned list.

  You can use the returned list to create a Tcl array with the help of the "array set" command. See the example below.

  NOTE: Do not rely on the order of the pairs in the list. New pairs might be inserted in the list at any position in a new version of Jasper Apps. We strongly advise that you access the returned value from this command through an array created with "array set" (see above).

Example

```
get_property_info -list trace_id assert_1 # prove
{1 3 4}
get_property_info -list trace_id assert_2 # hunt -run
{2 3 6 7 9}
get_trace_info 1
length 6 engine L time 0.4 job 0.0.L tag {}
get_trace_info 6
length 75 engine Ht time 8.5 job 0.1.Ht tag s1:0
```

See Also

get_property_info

get_uxdb_status

Gets the value of the "uxdb_status" variable, "on" or "off".

Syntax

```
get_uxdb_status
```

Returns

Returns the value of the "uxdb_status" variable.

Detailed Description

Use this command to check whether Jasper's connection to the Information Server is "on" or "off". If it is "on", then Jasper is connected to the Information Server, which enables the Jasper Expert System to collect data about the run and provide recommendations that are dependent on this data. If it is "off", then Jasper is not connected to the Information Server.

NOTE: To enable connection to the Jasper Information System, you must define the JASPER_UXDB_PATH environment variable before loading Jasper. Define this variable as follows:
setenv JASPER_UXDB_PATH port@server_name

Example

```
get_uxdb_status
```

get_version

Gets the version or build date of the tool.

Syntax

```
get_version [-build_date]
```

Returns

Returns a string containing the tool version release number or build date.

Detailed Description

Use this command to display the current Jasper Apps release number or build date.

-build_date
    Use "-build_date" to return the date of the build instead of the release number.

Example

```
get_version
2026.03 FCS 64 bits

get_version -build_date
2026.03.26 12:48:29 UTC
```

graph

Graphs generation and algorithms computation.

Syntax

```
Creating Graphs
--------------------------------
graph -create
    ( -name graph_name [-from signal]  [-to signal]
     |-property complete_property_name
     |-instance instance_name)
    [-type (data | signals | bitwise_info | bbox)]
    [-filter (full | register | interface)]

graph -add graph_name
    ( -node string_list
     |-edge string_list
     |-color color_name (-node string_list))

graph -remove graph_name
    ( -node string_list
     |-edge string_list)

Calculating Shortest Path
-------------------------
graph -shortest_path
    ( -from signal
      -to signal
      -type (all | register | hierarchy | logic))
    [-complete_path]
```

Returns

Multiple returns.

Detailed Description

```
Use this command to create visual graphs or to calculate the shortest path
between two design signals.
NOTE: Visual graphs can either be based on the netlist (design-based)
or independent of the netlist (non-design-based).

Creating a Graph
---------------------------------
-create
   ( -name graph_name [-from signal] [-to signal]
    |-property complete_property_name
    |-instance instance_name
   [-type (data | signals | bitwise_info | bbox)]
   [-filter (full | register | interface)]

     Create a graph that is either design-based or non-design-based.
     . Use "-name" to create an empty graph with the specified name that is non-design-
based.
       NOTE: If you specify -from and/or -to, a design-based graph is created from
       the COI of the specified signals.
     . Use "-property" to create a graph with the COI of the specified property.
       The graph name will be that of the specified property.
     . Use "-instance" to create a graph from the specified instance. The graph name
       will be that of the specified instance.
     . Use "-type" to create a graph of the specified type. The default is "bbox".
       . bbox    - Adds all visible signals and black boxes, connects all black boxed
                   inputs with black boxed outputs, and provides information about
                   bit connections
       . bitwise - Adds all visible signals and provides information about bit
                   connections.
       . signals - Adds all visible signals
       . data    - Add only visible data signals
     . Use "-filter" to filter the graph nodes. The default is "full".
       . full      - Complete graph
       . register  - Shows only register nodes
       . interface - Shows only interface nodes

-add graph_name
    ( -node string_list
     |-edge string_list
     |-color color_name (-node string_list))

     Add nodes, edges, or color annotations to the specified graph.
     Edges and nodes can be added only in non-design-based graphs.
```

```
     . Use "-node" to specify the node names.
     . Use "-edge" to specify which nodes are connected by an edge.
       NOTE: Each edge is represented by two nodes.
     . Use "-color" to add color to the nodes.
       NOTE: Each node supports more than one color annotation.

-remove graph_name
    ( -node string_list
     |-edge string_list)

     Remove nodes, edges from the specified graph.
     Nodes and edges can be removed only from non-design-based graphs.

     . Use "-node" to specify which nodes you want to remove.
       NOTE: Removing a node also removes it edges.
     . Use "-edge" to specify which edges are going to be removed.
       NOTE: Each edge is represented by two nodes.

Calculating the Shortest Path
-----------------------------

-shortest_path
    ( -from signal
      -to signal
      -type (all | register | hierarchy | logic))
     [-complete_path]
    Calculates the shortest path between two signals.

    . Use "-from" to specify the source signal.
    . Use "-to" to specify the destination signal.
    . Use "-type" to specify the length of the path between the "-from" signal
      and the "-to" signal.
      . Use "all" to consider all signals in the path.
      . Use "register" to consider the number of registers in the path.
      . Use "hierarchy" to consider the number of hierarchy changes in the path.
      . Use "logic" to consider the number of gates in the path.
    . Use "-complete_path" to return the complete path.

    NOTE: The default signal list return is the source, destination, and
    the signals along the path based on the specified "-type".  With the
    "-complete_path" switch, you can get all of the signals on the path.
```

Example

```
graph -create -name family
graph -add family -node {jacky paul ann}
graph -add family -edge {jacky ann ann paul}
graph -add family -color black -node {jacky paul}
graph -create -instance top
graph -shortest_path -from instA.a -to instB.b -type all
graph -shortest_path -from instA.a -to instB.b -type register -complete_path
```

gui

Provides interactivity with GUIs through custom widgets, dialogs, and filters.

Syntax

```
Opening a Dialog
----------------
gui -open_dialog
            ( -file
              (-open |-save)
              -type file_types_allowed
              [-dir_or_filename dir_path_or_file_name]
             |-input
              [-label_text label_text]
              [-default_text default_input_text]
              [-option_list {option_list+} (-check_box |-drop_down)
                   [-default_option {default_option_list+}]]
             |-message
              [-text message_text]
              [-type (information | warning | critical | question)])
            [-title title_string]
            [-target (session | console)]

Querying Widgets
----------------
gui -query
            -target
                ( -app_view ( afv | conn | cov | csr
                                  | fpv | fsv | lpv | sec | spv
                                  | superlint | unr | xprop )
                 |-visualize visualize_window_name)
            -widget (signal_name | property_table)
            [(-selection |-all)
             |-item_limit maximum_number_of_items]

Specifying and Managing Filters
-------------------------------
gui -filter
            -capture
```

```
        [config_name]
        ( -app_view ( all | afv | c2rtl | caf | cdc |  cfpv
                              | conn | cov | csr | fpv | fsv | hls | lpv
                              | rvv | sec | spv | superlint
                              | unr | xprop ) )
        [-widget (property_table)]
        [-force]
    |-apply
        (config_name_list | config_name)
        [-widget (property_table)]
    |-remove
        {config_name_list+}
        [-widget (property_table)]
    |-clear [-widget (property_table)]
    |-list [-widget (property_table)]
        ( -app_view ( all | afv | c2rtl | caf | cdc | cfpv
                              | conn | cov | csr | fpv | fsv | hls | lpv
                              | rvv | sec | spv | superlint
                              | unr | xprop ) )
    |-import filter_file.xml
    |-export filter_file.xml
```

Returns

No return values

Detailed Description

Use this command to provide interactivity to Jasper Apps views or Visualize windows with the ability to query specific GUI widgets, open dialogs for user input, and create filter configurations.

```
Opening a Dialog
----------------
-open_dialog
    ( -file
        (-open |-save)
        -type file_types_allowed
        [-dir_or_filename dir_path_or_file_name]
     |-input
        [-label_text label_text]
```

```
        [-default_text default_input_text]
        [-option_list option_list (-check_box | -drop_down) [-default_option
default_option_list]]
    |-message
        [-text message_text]
        [-type (information | warning | critical | question)])
    [-title title_string]
    [-target (session | console)]

   Open a file or text input dialog for the main or Visualize window(s).

   . Use "-file" to specify a file dialog.
     . Use "-open" to specify a file open dialog where the user can select an
       existing file and click an "Open" button.
     . Use "-save" to specify a file save dialog where the user can select an
       existing file or type a new filename and click a "Save" button.
     . Use "-type" to specify the types of files the dialog accepts.
     . Use "-dir_or_filename" to specify either the target directory or filename
       for the dialog.
   . Use "-input" to specify a text input dialog.
     . Use "-label_text" to specify the text for the label field in the dialog.
     . Use "-default_text" to specify the text written in the text input field
       in the dialog.
     . Use "-option_list" to specify an input dialog to present an options
       dialog.
     . Use "-drop_down" to specify the options to be shown in a drop-down list.
     . Use "-check_box" to specify the options to be shown as a group of check
       boxes.
     . Use "-default_option" to set the default option.
       This switch accepts a list of indexes to the options specified with
       "-option_list". (The first option is index 0.) If used with
       "-drop_down", you can only specify one index, which indicates the
       selected option in the drop-down list. If used with "-check_box", you
       can specify multiple lists of indexes, and the check boxes that
       correspond to the specified indexes will be checked. (The others will
       be unchecked.)
       . Return values for a "-drop_down" dialog:
         Tcl list with the index and text of the chosen option.
       . Return values for a "-check_box" dialog:
         Tcl list with one element for each check box. Each element consists
         of the index, text, and state of the check box.
         In the following example, the first and last check boxes were
         checked by the user:
```

```
        {{0} {P1} {on}} {{1} {P2} {off}} {{2} {P3} {on}}

    . Use "-message" to specify a message dialog.
      . Use "-text" to specify the text for the message dialog.
      . Use "-type" to specify the type for the message dialog: "information",
        "warning", "critical", or "question".
        If you do not specify this option, the default is "information".
        Return values for a "question" message dialog:
        . 1 if the selected answer is yes
        . 0 if the selected answer is no
        For other message dialog types, the return value is 1.

    . Use "-title" to specify the text to be displayed in the title bar of the
      dialog.
    . Use "-target" to specify the parent of the dialog. Specify "console" for
      the main window (default if nothing is specified) or "session" for
      Visualize windows.

Querying Widgets
----------------
-query
    -target
         ( -app_view ( afv | conn | cov |csr | fpv
                     | fsv | lpv | sec | spv | superlint | unr
                     | xprop)
          |-visualize visualize_window_name)
        -widget (signal_name | property_table)
        [(-selection |-all) |-item_limit maximum_number_of_items]
    Query the specified widgets in the specified window. If there are multiple
    columns in the widget, the text returned is from the "Name" column.

    . Use "-target" to specify the app view or Visualize window to be queried.
    . Use "-app_view" to specify the app view to query.
      . afv       - Architectural Formal Verification App
      . conn      - Connectivity Verification App
      . cov       - Coverage App
      . csr       - CSR Verification App
      . fpv       - Formal Property Verification App
      . fsv       - Functional Safety Verification App
      . lpv       - Low Power Verification App
      . sec       - Sequential Equivalence Checking App
      . spv       - Security Path Verification App
      . superlint - Superlint App
```

```
      . unr       - UNR Analysis App
      . xprop     - X-Propagation Verification App
    . Use "-visualize" to specify the Visualize window to be queried.
    . Use "-widget" to specify the widget to be queried. The currently
      supported widgets are:
      . signal_name       - Signal name pane in the specified Visualize window
      . property_table    - Property table in the specified app view
    . Use "-selection" to return only the selected items in the widget.
      This is the default if nothing is specified.
    . Use "-all" to return all items in the widget.
    . Use "-item_limit" to define the maximum number of items to be returned.
      NOTE: If not defining an item limit, the tool attempts to return all of
      the items matching the options specified, which can negatively impact
      performance.

Specifying and Managing Filters
-------------------------------
-filter
    -capture
        [config_name]
        (-app_view ( all | afv | c2rtl | caf | cdc | cfpv | conn | cov | csr |fpv
                   | fsv | hls | lpv | rvv | sec | spv
                   | superlint | unr | xprop ) )
        [-widget (property_table)]
        [-force]
    Capture the currently applied filters to the specified widget as a filter
    configuration. If you do not specify a config_name, the tool generates a
    default one.

    NOTE:
    . If you do not specify "-app_view", the filter configuration is
      created for the current app view.
    . To apply the filter configuration for all Jasper apps use
      "-app_view all".

    . Use "-force" if the specified "config_name" already exists and you want
      to overwrite it.
    . Use "-widget" to specify the widget. If you do not specify this option,
      the default is "property_table".

    The return value is the internal ID of the created filter configuration.

-filter -apply config_name
```

```
        [-widget (property_table)]
    Apply the specified filter configuration for the specified widget in the
    current app view.

-filter -remove (config_name_list | config_name)
        [-widget (property_table)]
    Remove the specified filter configurations from the specified widget in the
    current app view.

-filter -clear [-widget (property_table)]
    Remove all filter configurations for the specified widget in all app views.

-filter -list [-widget (property_table)]
        (-app_view ( all | afv | c2rtl | caf | cdc | cfpv | conn | cov | csr |fpv
                   | fsv | hls | lpv | rvv | sec | spv
                   | superlint | unr | xprop ) )
    List the filter configurations as a Tcl list.

    The return value is a Tcl list where each element of the list is a Tcl list
    with elements - filter configuration name, internal ID, app short name, and
    widget name.

    NOTE:
    . If you do not specify "-app_view", the filter configurations returned are
      for the current app view.
    . To return the filter configurations for all Jasper apps use
      "-app_view all".

-filter -import filter_file.xml
    Import filter configurations from the specified "filter_file", that is, a
    valid filter configuration XML file created by Jasper.

-filter -export filter_file.xml
    Export the filter configuration to the specified "filter_file.xml".

    NOTE: Your filter file must include the ".xml" file extension.
```

Example

```
gui -open_dialog -file -type "vcd" -save -title "Save VCD file" -target session
gui -open_dialog -file -type "tcl itcl" -open -title "Open tcl file" -target console
gui -open_dialog -input -title "Instance name" -label_text "Enter the name of the
instance" -default_text "top"
gui -query -target -app_view fpv -widget property_table -selection
gui -query -target -visualize visualize:0 -widget signal_name -all
```

See Also

custom_gui_action

help

Prints information about commands or messages.

Syntax

```
help [-usage] [command_name]

help -message [message_id]

help -apropos keyword

help -gui [tcl | command_name]
```

Returns

No return values

Detailed Description

Use this command to display help information, including the following:

- List of all commands (use "help" with no arguments)

- Details of a specified command

- List of all Message Help (use "help -message" with no arguments)

- Details about a specified Error, Warning, or Info message

The "help" command supports tab completion, which behaves similar to a standard bash shell. That is, the first time you press the Tab key, nothing happens unless there is a unique completion for the command.

Example: If we have the commands cmd1 and cmd2 and you type "help c"

```
and press Tab, the text will be expanded to "help cmd". The second
time you press Tab, the tool lists (in the log pane) all available
commands that start with the characters to the left of the cursor.
To distinguish this text from other output in this pane, the text is
blue. In this example, the tool lists cmd1 and cmd2. (You might need
to scroll to the bottom of the log pane to see the return.)


[-usage] [command_name]
    Display the command syntax and details for the specified
    "command_name".

    If you do not specify "command_name", the tool lists all
    available command help.

    When you specify a non-existent command as an argument to the "help"
    command, the tool lists all commands that contain the argument as part of
    their names. For example, "help get" lists all commands that contain
    "get" in their names at any position.

    If you specify the switch "-usage" the detailed description and
    the example sections of the command help will not be displayed.

-message [message_id]
    Display message help for the specified "message_id".

    If you do not specify a "message_id", the tool lists all
    available message help.

-apropos keyword
    List available commands with a brief description containing the keyword.

-gui [tcl | command_name]
    Open the Jasper Help Browser window to show the Tcl command
    descriptions.

    . Use "tcl" to launch the window with the Tcl command descriptions in the
      foreground.

    . Use the name of a command in Jasper to directly open its help
      content.
```

Example

```
help
help stopat
help -usage visualize
help -message
help -message ICD003
help -apropos counter
help -gui analyze
```

hunt

Use formal verification to increase the chances of finding CEXs (no proof).

Syntax

```
Configuring Strategies to Help Find Bugs
----------------------------------------
hunt -config -strategy strategy_name
             [-mode (cycle_swarm | bound_swarm | formal | guidepoint |
                             loop_swarm | simulation | state_swarm | trace_swarm |
                             trace_search)]
             [-engine_mode {engine+}]
             [-time_limit time_limit]
             [-max_jobs N]
             [-per_engine_max_jobs N]
             [-max_trace_length {N+}]
             [-target_depth N]
             [-interval_cycles N]
             [-interval_time_limit (auto | time_limit)]
             [-no_cover_traces [(true | false)]]
             [-from property_name [-trace_id N] [-cycle N]]
             [-target_type {(cover | assert | assert_liveness)+}]
             [-unset {param+}]
             [-per_property_time_limit time_limit]
             [-per_property_time_limit_factor N]
             [-max_first_trace_attempt N]
             [-trace_attempt_time_limit time_limit]
             [-trace_attempt_time_limit_factor N]
             [-first_trace_attempt {N+}]
             [-deeper_cycles_earlier (true | false)]
             [-start_state_count {number_of_start_states+}]
             [-tail_length {number_of_cycles_to_backtrack+}]
             [-segment_time_limit {time_limit+}]
             [-max_segment_length {segment_length+}]
             [-random_diversification_factor {0.0 - 1.0+}]
             [-auto_helper_num N]
             [-auto_helper_type {cover_type+}]
             [-auto_keep_helpers]
             [-auto_cleanup_time_limit time_limit]
```

```
              [-auto_partition {type partition_type num N}]
              [-stagnation_time_limit time_limit
               [-stagnation_action (stop | restart)]]
              [-helper_set_store_trace (1 | unlimited)]
              [-path {cover+}]
              [-refresh N]
              [-seed N]
              [-per_trace_time_limit per_trace_time_limit]
              [-on_determined tcl_callback]
              [-dump_trace
               [-dump_trace_type (shm | vcd | fsdb | vwdb)]
               [-dump_trace_dir dir_name]]

Adding and Removing Strategy's Constraints
------------------------------------------
hunt -config -strategy strategy_name
             ( -add_constraint expression [-name constraint_name] [-bound N]
              |-remove_constraint constraint_name)

Saving and Loading Hunt
-----------------------
hunt -save file_name.cfg [-strategy strategy_name] [-force]

hunt -load file_name.cfg

Running Hunt
------------
hunt -run [-strategy strategy_name]
          (-property property_name_tcl_list [-regexp]
           |-task task_name)
          [-evaluate_in_trace property_name_tcl_list]
          [-tag tag_name]
          [-use_strategy strategy_name]
          [-auto
           [-auto_helper_num N]
           [-auto_helper_type {cover_type+}]
           [-auto_keep_helpers]
           [-auto_cleanup_time_limit time_limit]
           [-disable_trace_swarm]
           [-auto_partition {type partition_type num N}]
           [-stagnation_time_limit time_limit
            [-stagnation_action (stop | restart)]]]
          [-engine_mode {engine+}]
```

```
          [-time_limit time_limit]
          [-max_jobs N
            [-trace_swarm_ratio N]]
          [-per_engine_max_jobs N]
          [-max_trace_length N]
          [-no_cover_traces [(true | false)]]
          [-helper_set_store_trace (1 | unlimited)]
          [-from property_name [-trace_id N] [-cycle N]]
          [-seed N]
          [-rerun_job (job_index | job_name)]
          [-on_determined tcl_callback]
          [-dump_trace
           [-dump_trace_type (shm | vcd | fsdb | vwdb)]
           [-dump_trace_dir dir_name]]
          [-verbosity N]
          [-force]
          [-bg]
```

Viewing Hunt

```
hunt -list (strategy | tag)

hunt -show -strategy [strategy_name] [-silent]
```

Reporting Hunt

```
hunt -report [-status]
                      [-detailed]
                      [-start_time]
                      (([-engine_config]
                        [-engine_stats]
                        [-job_score])
                        | [-from_info])
                      [-sort_by (  cex | covered | first_trace_attempt | job
                            | local_overconstraint | max_segment_length
                            | max_trace_length | mode | random_diversification_factor
                            | rarity_job_score | restarts | results_job_score
                            | searches_from_reset | segment_length_exceeded
                            | segment_time_limit | segment_time_out | start_state_count
                            | start_time | tail_length | total_searches | unprocessed )
]

hunt -report -trace_swarm
```

```
        [-tag tag_name]
        [-pending]
        [-running]
        [-completed]
        [-silent]
```

```
Clearing Hunt
-------------
hunt -clear [ -strategy [strategy_name] [-force]
                      |-tag [tag_name]]
```

Returns

When hunting without "-bg", these are the return values:

. "hunt -config -strategy strategy_name" returns the name of the strategy
  if it was successfully created or modified; otherwise, it returns an error.

. "hunt -run" returns information about the settings used for the hunt
   operation, and you can view results in the Property Table.

. "hunt -report" and "hunt -report -status" return the list of tags along with
  their respective specific data such as mode, strategy, and the number of
  properties with each status. Each tag list also contains internal information
  for their respective job runs. Job lists have their own counters of
  properties and status and can also have information about the job
  configuration (when you use the switch "-engine_config") and a list of the
  properties for which the job reached an answer (when you use the switch
  "-detailed"). This information is the same shown in the table printed in the
  console.

. "hunt -report -trace_swarm" returns a list of dictionaries for tags related
  to trace_swarm strategies along with information such as status, time_limit,
  and a summary of pending, completed, and running traces related to the
  applicable hunt run. If you specify a tag, you can also specify a filter for
  a trace's status, such as "-pending", "-completed", and "-running". If you
  specify a trace status, the report displays further details, including either
  threads and jobs that have consumed traces or a list of pending traces in the
  queue. The report also presents the same detailed information with a Tcl
  dictionary.

Detailed Description

```
Use this command to hunt counterexamples using formal verification. This
feature supports various strategies. Each strategy comprises a formal
verification flow.

NOTE:
. "hunt" does not support FSV properties when using the "trace_swarm"
  or "trace_search" strategies.
. The following switches are initial releases to gather feedback from early
  adopters and finalize implementation for an upcoming release:
  . -dump_trace
  . -job_score
  . -load
  . -on_determined
  . -save

Contact support@cadence.com to provide feedback or comments targeting the
upcoming production version.

Configuring Strategies to Help Find Bugs
----------------------------------------
-config -strategy strategy_name
          [-mode (  cycle_swarm | bound_swarm | formal | guidepoint | loop_swarm
                  | simulation | state_swarm | trace_swarm | trace_search)]
        [-engine_mode {engine+}]
        [-time_limit time_limit]
        [-max_jobs N]
        [-max_trace_length {N+}]
        [-no_cover_traces [(true | false)]]
        [-from property_name [-trace_id N] [-cycle N]]
        [-unset {param+}]
        [-on_determined tcl_callback]
        [-dump_trace
             [-dump_trace_type (shm | vcd | fsdb | vwdb)]
             [-dump_trace_dir dir_name]]

    Use the specified strategy to hunt for bugs.

    . Use "-mode" to set a strategy mode different from the default
      "state_swarm".

    . Use "-engine_mode", "-time_limit", "-max_jobs", "-max_trace_length", and
```

"-no_cover_traces" to customize any strategy mode:

. "-engine_mode" specifies engines used by the specified strategy when applicable. It is not applicable to "state_swarm", which assumes engine L.

. "-time_limit" specifies the "hunt" time limit.

. "-max_jobs" specifies the maximum number of engine jobs that run in parallel in a hunt run. If you have enough licenses, "max_jobs" engine jobs will run in parallel. See additional semantic for this value and for "per_engine_max_jobs" in the sections for each strategy configuration.

NOTE: "max_jobs" must be a limited value greater than 0 in "hunt -auto", "trace_swarm", and "trace_search" strategy runs. For other hunt strategies, see the semantic adopted when "max_jobs" is 0 (unlimited) in each strategy configuration section.

. "-max_trace_length" specifies the limit for the length of traces the tool searches for. You can specify a natural value, a list of naturals, or a distribution of naturals. Specifying a list or distribution is only valid for the simulation strategy. Other strategies ignore lists and use only the first value defined inside them.
It is set to 0 by default for "state_swarm", "cycle_swarm", "loop_swarm", and "trace_swarm" strategies.

. "-no_cover_traces" specifies that no cover traces will be generated. Use this switch to avoid the overhead of generating and storing cover traces.
It is on by default for "state_swarm" strategies only. To turn it on or off, specify the optional argument "true" or "false".

. Use "-from" to specify an existing trace as the initial value for the hunt run.

. Use "-trace_id" to specify a trace other than the default. The default property trace is the last shortest trace. See "assert/cover -set_store_trace" for the commands to configure the number of traces generated and stored by the tool.

. Use "-cycle" to specify the state of the trace at a specific cycle as the reset value. If you do not specify a cycle, or if you specify 0,

the tool uses the cycle in which the property was triggered. This is often, but not always, the last cycle of the trace. See "set_trace_extension".

. Use "-unset" to clear configuration of the specified parameters so that "hunt" inherits the parameters from the proof environment. The following parameters can be specified with "-unset": "time_limit", "max_trace_length", "engine_mode", "max_jobs", "per_engine_max_jobs", "no_cover_traces", "seed".

Hunt priority for parameters, in descending order:
. Use the parameter value specified in "hunt -run";
. If parameter was not defined with "hunt -run", use the value defined for the strategy with "hunt -config";
. If "hunt -config" was not specified or the parameter was not defined with "hunt -config", use the strategy's default value for the parameter;
. If "-unset" is specified for a parameter, use proof environment values ("hunt -run", "hunt -config", and strategy default values are ignored).

Example:

```
set_max_trace_length 15
hunt -config -strategy s1 -mode state_swarm
hunt -run -strategy s1 ;# uses max_trace_length from state_swarm default
                       ;# value(0)
hunt -config -strategy s1 -mode state_swarm -max_trace_length 10
hunt -run -strategy s1 ;# uses max_trace_length strategy value(10)
hunt -run -strategy s1 -max_trace_length 5 ;# uses run max_trace_length
                                           ;# value(5)
hunt -config -strategy s1 -unset max_trace_length
hunt -run -strategy s1 ;# uses proof environment max_trace_length
                       ;# value(15)
```

. Use "-on_determined" to set a Tcl proc to call when "hunt" determines a property.

The tool calls the specified command or procedure when a "hunt" command determines a property. The tool provides three arguments: the event, property name, and "prove" thread ID. Currently, the event is always "determined", but it is possible that we will introduce more events in a future version.

CAUTION: "on_determined" procedures are very powerful, but they can be unsafe as they are hard to debug, and it is possible to use procedures that make the tool loop.

```
  . Use "-dump_trace" to dump traces to the disk as soon as the engines find
    them. The tool uses the SHM format as the default and writes files in the
    project directory. The trace has values for all signals in the analysis
    region of the proof target.

    . Use "-dump_trace_type" with "shm", "vcd", "fsdb", or "vwdb" to
      specify the file dump type. VCD files are dumped as compressed files to
      save disk space.

      IMPORTANT:
      . Jasper writes 6.3 FSDB traces and reads version 6.6 FSDB
        traces. FSDB Reader 6.6 was introduced by Verdi version X-2025.06.
      . FSDBs you generate with the latest Jasper are backwards compatible
        with all Jasper versions from 2025.03 FCS and newer. In addition,
        they are backwards compatible with older versions of Verdi released
        during that same period.
      . Jasper supports backwards compatibility, that is, the tool
        can read FSDBs generated from older versions of Jasper and
        Synopsys Verdi.

    . Use "-dump_trace_dir" to specify the dump directory where the files
      will be written.

  NOTE:
  . If you reconfigure with new value parameters for previously defined
    parameters, the tool overwrites the definitions and reports what changed.
  . By default, the tool creates a new strategy if a strategy with the same
    name does not exist yet.
  . Not all strategies work with liveness or infinite properties due to engine
    limitations. Refer to the following list:
    . "cycle_swarm": Always able to handle liveness or infinite.
    . "bound_swarm": Always able to handle liveness or infinite.
    . "formal": Can handle liveness or infinite with the default engines.
    . "guidepoint": Can handle liveness or infinite with the default engines.
    . "loop_swarm": Only handle liveness or infinite.
    . "simulation": Can handle liveness or infinite when using engine U.
    . "state_swarm": Unable to handle liveness or infinite.
    . "trace_swarm": Can handle liveness or infinite with the default engines.
    . "trace_search": Can handle liveness or infinite with the default engines.

Configuring Parameters for "cycle_swarm" Mode
---------------------------------------------
```

```
-config -strategy strategy_name
        -mode cycle_swarm
        [-engine_mode (B | B4 | Bm | Hts | Ht)]
        [-max_jobs N]
        [-first_trace_attempt {N+}]
        [-max_trace_length N]
        [-deeper_cycles_earlier (true | false)]
        [-max_first_trace_attempt N]
        [-trace_attempt_time_limit time_limit]
        [-per_property_time_limit time_limit]
        [-per_property_time_limit_factor N]
        [-seed N]
    Use the specified configuration for the "cycle_swarm" strategy.
    The strategy parallelizes different groups of trace attempts processing.

    NOTE: If you do not specify parameters, the tool uses predefined default
    values when hunting. This strategy is always able to handle liveness or infinite
    properties.

    Important things to know about "cycle_swarm":
    ---------------------------------------------
    . Bound aggregation is done from the parallel trace attempts processing.
    . "cycle_swarm" ignores "per_engine_max_jobs" even if you specify it with
      "hunt -config" or "hunt -run".
    . "cycle_swarm" also ignores "set_proofgrid_per_engine_max_local_jobs" and
      "set_proofgrid_per_engine_max_jobs".
    . For "cycle_swarm" with engine Bm or Ht:
      . The tool does not support safety or finite properties mixed with
        liveness or infinite properties in the target:
        . If the target contains both safety or finite and liveness or infinite
          properties, the tool ignores the liveness or infinite properties.
        . If the target contains liveness or infinite properties only, the tool
          processes them.

    . Use "-strategy strategy_name -mode cycle_swarm" to create a new
      strategy for "cycle_swarm" mode.

    . Use "-engine_mode" to specify the engine. The possible engines for this
      strategy are B, B4, Bm, Hts or Ht. Specify only one of them.
      If you do not specify this switch, the tool defaults to engine Bm.

    . Use "-max_jobs" to specify the number of engine jobs for the hunt search.
```

For multiproperty engine modes (Bm or Ht), one engine job is instantiated for each group of trace attempts.

For single-property engine modes (B, B4 or Hts), N engine jobs are instantiated for each group of trace attempts. N is equal to "max_jobs" divided by "max_first_trace_attempt". By adjusting these settings, N properties are processed in parallel for each group of trace attempts.

NOTE:
. If you specify the value 0 for the "max_jobs" switch, the tool uses the number of all possible trace attempts groups as the value for "max_jobs".
. If you unset this switch, the tool uses the value set for "proofgrid_max_local_jobs" if in local mode, or "proofgrid_max_jobs" otherwise.
. If you do not specify this switch, the default value is 6.

See also "cycle_swarm" strategy examples in the examples section.
If you do not specify this switch, the default value is 6.

. Use "-first_trace_attempt" to specify the cycle at which the engine will make its first trace attempt on each engine job. You can specify a list of values or a distribution. The percentage symbol is optional. You can also set distributions by weight. For example, setting first_trace_attempt to {20%:[1] 80%:[2..4]} is equivalent to setting it to {1:[1] 4:[2..4]}. If you do not specify this switch, the tool computes a sequential list of values taking the targets' bounds into account. In this case, the first element is the minimum bound among the targets, and the number of first_trace_attempts is limited by the value of "max_first_trace_attempt".

Example:
Engine: Bm (default)
Properties' minimum bound: 7

```
hunt -config -strategy cs -mode cycle_swarm \
     -max_jobs 5 -max_first_trace_attempt 3
hunt -run -strategy cs -task .
# first_trace_attempts: {7 8 9}

hunt -config -strategy cs \
     -max_jobs 5 -max_first_trace_attempt 10
hunt -run -strategy cs -task .
# first_trace_attempts: {7 8 9 10 11}
```

```
  # ("max_first_trace_attempt" is limited by "max_jobs")

  hunt -config -strategy cs \
       -max_jobs 5 -first_trace_attempt {1 4 15}
  hunt -run -strategy cs -task .
  # first_trace_attempts: {1 4 15} (considering user's input)

  NOTE:
  . The tool might adjust "first_trace_attempt" values to prevent
    multiple engine B, B4 or Hts jobs from attempting to find traces on the
    same cycle. To avoid that possibility, the tool automatically sets
    "engineB_trace_attempt_increment" to the number of jobs to be
    dispatched and tunes each first_trace_attempt value to ensure they have
    a different "first_trace_attempt mod max_jobs". For example:
    max_jobs = 3 and first_trace_attempt = {2, 5, 7} => 2 mod 3 = 2,
    5 mod 3 = 2 and 7 mod 3 = 1. Therefore, the tool tunes
    first_trace_attempt 5 to 6 so that 6 mod 3 = 0 and the final
    first_trace_attempt = {2, 6, 7}.
  . All values specified for "first_trace_attempt" must be smaller than the
    value specified for "max_trace_length".

. Use "-max_trace_length" to specify the limit for the length of traces the
  tool searches for. You can specify a natural value for "cycle_swarm".
  If you do not specify this switch, the tool defaults to 0.

. Use "-deeper_cycles_earlier" to control how the tool distributes trace
  attempts among groups.
  If the value is "false", the groups will explore alternating trace
  attempts.
  If "-max_trace_length" is not specified, groups will keep alternating
  trace attempts until the proof ends.
  If the value is "true", "-max_trace_length" is a required switch and the
  specified trace attempt interval is distributed so that some groups can
  start exploring deeper cycles since the beginning of the run.
  For the "cycle_swarm" strategy mode, this value is "false" by default.

  Example:
  # deeper_cycles_earlier: false (default)
  hunt -config -strategy cs -mode cycle_swarm \
       -max_jobs 5 \
       -first_trace_attempt 11
  hunt -run -strategy cs -task .
  # Trace attempts explored by each group:
```

```
# group1: {11, 16, 21, 26, 31, ...}
# group2: {12, 17, 22, 27, 32, ...}
# group3: {13, 18, 23, 28, 33, ...}
# group4: {14, 19, 24, 29, 34, ...}
# group5: {15, 20, 25, 30, 35, ...}
# The first trace attempts explored by the tool will be
# {11, 12, 13, 14, 15}, in parallel.

# deeper_cycles_earlier: true
hunt -config -strategy cs -mode cycle_swarm \
     -max_jobs 10 -max_first_trace_attempt 5 -engine_mode B \
     -first_trace_attempt 11 -max_trace_length 110 \
     -deeper_cycles_earlier true
hunt -run -strategy cs -task .
# Trace attempts explored by each group:
# group1: {11, 12, 13, ... 29, 30}
# group2: {31, 32, 33, ... 49, 50}
# group3: {51, 52, 53, ... 69, 70}
# group4: {71, 72, 73, ... 89, 90}
# group5: {91, 92, 93, ... 109, 110}
# The first trace attempts explored by the tool will be
# {11, 31, 51, 71, 91}, in parallel.
```

. Use "-max_first_trace_attempt" to determine the number of groups of trace attempts to run in parallel.

  For single-property engine modes (B, B4 or Hts), N engine jobs are instantiated for each group of trace attempts. N is equal to "max_jobs" divided by "max_first_trace_attempt". By adjusting these settings, N properties are processed in parallel for each group of trace attempts.

  Note that the value of "max_jobs" must be a multiple of "max_first_trace_attempt" for maximizing job instances of single-property engines.

  If you do not specify this switch, its default value is "max_jobs".

. Use "-trace_attempt_time_limit" to specify a time limit for each trace attempt. Default value is "100%:[300..600]", ensuring that each run can use a different random value.

. For engines B, B4 and Hts, use "-per_property_time_limit" to specify the per-property time limit for the engine jobs. If you do not specify this

switch, the tool uses the current proof environment configuration values. See "set_prove_per_property_time_limit".

. For engines B, B4 and Hts, use "-per_property_time_limit_factor" to control iterative geometric progression of the time limit imposed per property. If you do not specify this switch, the tool uses the current proof environment configuration values. See "set_prove_per_property_time_limit_factor".

. Use "-seed" for hunt reproducibility. Specify a hexadecimal value to be used by the tool as a seed to determine a strategy's parameters randomly selected that will be used in a "hunt" run. The parameters randomly selected by the tool are those specified in a list or distribution format in "cycle_swarm", "state_swarm", and "simulation" strategies. If you do not specify this switch, the default value is 0, which is a special value that tells the tool to select a nondeterministic seed. Other values give deterministic behavior. When using the same seed together with the same parameters and distributions, the tool generates identical values for parameters.

  NOTE:
  . The tool uses the hunt seed to set values for configurable run attributes, such as "first_trace_attempt" for "state_swarm" and "trace_attempt_time_limit" for "cycle_swarm". If you run the command without an explicit seed, the tool selects a random seed and uses it to define the values of any attributes that do not have an explicit setting either in the configuration or with run-time options. The tool reports auto-generated seeds, and you can use them with subsequent "hunt" runs as an explicit seed setting, thus ensuring the tool applies the same attribute values.

    Using the same seed does not guarantee the same results due to many sources for non-determinism that affect engine processing from run to run. For example, if you use a heavily loaded machine, a time limit might prevent the tool from detecting a trace that it might otherwise detect. In particular, the engine L search is sensitive to the traces it detects. Such a nuance can result in different results.

  . If you unset this switch (hunt -config -strategy... -unset seed), specifically for "cycle_swarm" mode, the tool uses a random seed value to determine configuration parameters for distributions. This is equivalent to setting "cycle_swarm" seed to 0.

```
    See the examples section.

Configuring Parameters for "bound_swarm" Mode
---------------------------------------------
-config -strategy strategy_name
        -mode bound_swarm
        [-first_trace_attempt N]
        [-max_trace_length N]
        [-deeper_cycles_earlier (true | false)]
        [-max_first_trace_attempt N]
        [-trace_attempt_time_limit time_limit]
        [-trace_attempt_time_limit_factor N]
        [-engine_mode (B | B4 | Bm | Hts | Ht)]
        [-max_jobs N]
    Use the specified configuration for the "bound_swarm" strategy.
    The stratregy parallelizes different groups of trace attempts processing.

    NOTE: If you do not specify parameters, the tool uses predefined default
    values when hunting. This strategy is always able to handle liveness or infinite
    properties.

  Important things to know about "bound_swarm":
  ---------------------------------------------
    . "bound_swarm" is an enhanced version of "cycle_swarm" for a bounded
      interval proof. It is designed to iterate over the same set of cycles,
      from "first_trace_attempt" to "max_trace_length", automatically
      increasing "trace_attempt_time_limit" by factor
      "trace_attempt_time_limit_factor" as explained below.
    . Bound aggregation is done from the parallel trace attempts processing.
    . "bound_swarm" ignores "per_engine_max_jobs", "-per_property_time_limit",
      and "-per_property_time_limit_factor" even if you specify them with
      "hunt -config".
    . "bound_swarm" also ignores "set_proofgrid_per_engine_max_local_jobs"
      and "set_proofgrid_per_engine_max_jobs".
    . For "bound_swarm" with engine Bm or Ht:
      . The tool does not support safety or finite properties mixed with
        liveness or infinite properties in the target:
        . If the target contains both safety or finite and liveness or infinite
          properties, the tool ignores the liveness or infinite properties.
        . If the target contains liveness or infinite properties only, the tool
          processes them.

    . Use "-strategy strategy_name -mode bound_swarm" to create a new
```

```
  strategy for "bound_swarm" mode.

. Use "-first_trace_attempt" to specify the lower bound for the
  interval. It represents the cycle at which the engine will make
  its first trace attempt. The tool computes a sequential list of
  trace attempts, with the first element being the specified value.
  the number of different "first_trace_attempt" is limited by the
  value of "max_first_trace_attempt".

  NOTE:
  . If you do not specify this switch, the tool takes the targets'
    bounds into account. In this case, the first element is the
    minimum bound among the targets.
  . If you specify a value for this switch and this value is smaller
    than the minimum bound already found for the targets, the tool
    issues a message and uses the minimum bound as the first
    element of the list.
  . The value specified for "first_trace_attempt" must be smaller
    than the value specified for "max_trace_length".

. Use "-max_trace_length" to specify the upper bound for the interval
  the tool searches for. You can specify a natural value for it. If
  you do not specify this switch, the tool computes a value that is the
  minimum value for "first_trace_attempt" added to an offset of 100 cycles.

. Use "-deeper_cycles_earlier" to control how the tool distributes trace
  attempts among groups.
  If the value is "false", the groups will explore alternating trace
  attempts.
  If the value is "true", the specified trace attempt interval is
  distributed so that some groups can start exploring deeper_cycles since
  the beginning of the run.
  For the "bound_swarm" strategy mode, this value is "true" by default.

  Example:
  # deeper_cycles_earlier: false
  hunt -config -strategy bs -mode bound_swarm \
       -max_jobs 5 \
       -first_trace_attempt 21 -max_trace_length 70 \
       -deeper_cycles_earlier false
  hunt -run -strategy bs -task .
  # Trace attempts explored by each group:
  # group1: {21, 26, 31, ... 61, 66}
```

```
  # group2: {22, 27, 33, ... 62, 67}
  # group3: {23, 28, 33, ... 63, 68}
  # group4: {24, 29, 34, ... 64, 69}
  # group5: {25, 30, 35, ... 65, 70}
  # The first trace attempts explored by the tool will be
  # {21, 22, 23, 24, 25}, in parallel.

  # deeper_cycles_earlier: true (default)
  hunt -config -strategy bs -mode bound_swarm \
       -max_jobs 10 -max_first_trace_attempt 5 -engine_mode B \
       -first_trace_attempt 21 -max_trace_length 70
  # Properties current min_length: 1
  hunt -run -strategy bs -task .
  # Trace attempts explored by each group:
  # group1: {21, 22, 23, ... 29, 30}
  # group2: {31, 32, 33, ... 39, 40}
  # group3: {41, 42, 43, ... 49, 50}
  # group4: {51, 52, 53, ... 59, 60}
  # group5: {61, 62, 63, ... 69, 70}
  # The first trace attempts explored by the tool will be
  # {21, 31, 41, 51, 61}, in parallel.
```

. Use "-trace_attempt_time_limit" to specify a time limit for each trace attempt. Default value is 1, ensuring that first iteration runs quicker, and subsequent runs have this value increased.

. Use "-trace_attempt_time_limit_factor" to set the common ratio used for a geometric progression of trace_attempt_time_limit. The effective trace_attempt_time_limit will be:

  ```
  trace_attempt_time_limit * factor ^ scan
  ```

  Where "scan" starts at zero and increments after each complete sweep over the properties and all "first_trace_attempts" in each group of trace attempts.

. Use "-engine_mode" to specify the engine. The possible engines for this strategy are B, B4, Bm, Hts or Ht. Specify only one of them.
If you do not specify this switch, the tool defaults to engine Bm.

. Use "-max_jobs" to specify the number of engine jobs for the hunt search.

  For multiproperty engine modes (Bm or Ht), one engine job is instantiated

```
  for each group of trace attempts.

  For single-property engine modes (B, B4 or Hts), N engine jobs are
  instantiated for each group of trace attempts. N is equal to "max_jobs"
  divided by "max_first_trace_attempt". By adjusting these settings, N
  properties are processed in parallel for each group of trace attempts.

  NOTE:
  . If you specify the value 0 for the "max_jobs" switch, the tool uses the
    number of all possible trace attempts groups as the value for
    "max_jobs".
  . If you unset this switch, the tool uses the value set for
    "proofgrid_max_local_jobs" if in local mode, or "proofgrid_max_jobs"
    otherwise.
  . If you do not specify this switch, the default value is 6.

. Use "-max_first_trace_attempt" to determine the number of groups of trace
  attempts to run in parallel.

  For single-property engine modes (B, B4 or Hts), N engine jobs are
  instantiated for each group of trace attempts. N is equal to "max_jobs"
  divided by "max_first_trace_attempt". By adjusting these settings, N
  properties are processed in parallel for each group of trace attempts.

  Note that the value of "max_jobs" must be a multiple of
  "max_first_trace_attempt" for maximizing job instances of single-property
  engines.

  If you do not specify this switch, its default value is "max_jobs".

Examples:
. Default "bound_swarm":
  # trace_attempt_time_limit: 1s (1st scan), 10s (2nd scan), 100s (3rd scan)...
  # max_jobs: 6; first_trace_attempt: min bound;
  # max_trace_length: min bound + 100; engine_mode: Bm

    hunt -config -strategy BS -mode bound_swarm
    hunt -run -strategy BS (-property | -task )

. Custom factor, interval, jobs, multi-property
  # trace_attempt_time_limit: 1s (1st scan), 5s (2nd scan), 25s (3rd scan)
  # max_jobs: 10; first_trace_attempt: 65; max_trace_length: 200
```

```
        hunt -config -strategy BS -mode bound_swarm \
             -first_trace_attempt 65 -max_trace_length 200 \
             -trace_attempt_time_limit_factor 5 -max_jobs 10
        hunt -run -strategy BS (-property | -task )

    . Property parallelism, dispatches 5 groups of trace attemps with 2 engine
      jobs each
      # max_jobs: 10; max_first_trace_attempt 5
      # first_trace_attempt: {10 11 12 13 14}

        hunt -config -strategy BS -mode bound_swarm -engine_mode (B | B4) \
             -max_first_trace_attempt 5 -max_jobs 10 \
             -first_trace_attempt 10
        hunt -run -strategy BS (-property | -task )

Configuring Parameters for "formal" Mode
----------------------------------------
-config -strategy strategy_name
        -mode formal
        [-max_jobs N]
        [-per_engine_max_jobs N]
    Use the specified configuration for the "formal" strategy. This strategy is
    able to handle liveness or infinite properties with the default engine setup.

    NOTE: If you do not specify parameters, the tool uses the proof
    configuration environment values when hunting.

    . Use "-strategy strategy_name -mode formal" to create a new
      strategy for the "formal" mode.

      The "formal" mode consists of running "hunt" with the formal proof
      configuration environment. You can use "hunt" common switches available
      for all hunt modes to overwrite formal proof configuration, and you can
      add and remove hunt local constraints.

      If you do not specify the "-engine_mode" switch, default engines are
      {Ht Bm Hts B}.

    . Use "-max_jobs" to specify the maximum number of jobs to dispatch.
      If you do not specify this switch, the default value is 6.

    . Use "-per_engine_max_jobs" to configure the number of engine_jobs per
      engine to dispatch. If you do not specify this switch, the default value
```

```
      is 2.

      Example:
      hunt -config -strategy s1 -mode formal -time_limit 2h -max_jobs 20
      hunt -config -strategy s1 -per_engine_max_jobs 5
      # dispatches hunt with up to 20 jobs in parallel, up to 5 jobs per engine:
      hunt -run -strategy s1
      # dispatches hunt with up to 20 jobs in parallel, up to 3 jobs per engine:
      hunt -config -strategy s1 -per_engine_max_jobs 3
      hunt -run -strategy s1

Configuring Parameters for "guidepoint" Mode
--------------------------------------------
-config -strategy strategy_name
        -mode guidepoint
        [-engine_mode {engine+}]
        [-path {start_state+}]
        [-per_engine_max_jobs N]
        [-tail_length {number_of_cycles_to_backtrack+}]
    Use the specified configuration for the "guidepoint" strategy. This
    strategy is able to handle liveness or infinite properties with the default engine
    setup.

    NOTE: If you do not specify parameters, the tool uses predefined default
    values when hunting.

    . Use "-strategy strategy_name -mode guidepoint" to create a new
      strategy for the "guidepoint" mode.

      The "guidepoint" mode requires a list of covers that are to be
      reached in the specified order so that a formal prove is dispatched
      starting from the last cover.

    . Use "-engine_mode" to specify the engines to be used in both the covers
      path and in the final step target. The default is engines {Hp Ht B L}.

    . Use "-path" to specify the list of covers. The tool targets the covers in
      the specified order.

    . Use "-per_engine_max_jobs" to configure the number of engine_jobs per
      engine during the last phase of "guidepoint".

    . Use "-tail_length" to specify the starting point for a new search that is
```

```
      N cycles before the triggered at cycle of each cover trace in covers path.
      If you do not specify this switch, default value is 0.

      NOTE: If you do not specify this switch, the default value is the value
      set for "proofgrid_per_engine_max_local_jobs" if in local mode, or
      "proofgrid_per_engine_max_jobs", otherwise. See
      "set_proofgrid_per_engine_max_jobs",
      "set_proofgrid_per_engine_max_local_jobs", and the example in the section
      for "formal" strategy configuration.

Configuring Parameters for "loop_swarm" Mode
--------------------------------------------
-config -strategy strategy_name
        -mode loop_swarm
        [-engine_mode {engine+}]
        [-max_jobs N]
        [-loop_length {N+}]
        [-loop_length_incr N]
        [-max_loop_length N]
        [-max_trace_length N]
        [-seed N]
    Use the specified configuration for the "loop_swarm" strategy.

    Use the "loop_swarm" strategy to skip or partially analyze loop lengths in
    liveness or infinite properties checks. This strategy supports only liveness
    or infinite properties.

    The strategy parallelizes jobs with different "loop_length" values

    NOTE: If you do not specify parameters, the tool uses predefined default
    values when hunting.

    Important things to know about "loop_swarm":
    --------------------------------------------
    . "loop_swarm" ignores "per_engine_max_jobs" even if you specify it with
      "hunt -config" or "hunt -run".
    . "loop_swarm" also ignores "set_proofgrid_per_engine_max_local_jobs" and
      "set_proofgrid_per_engine_max_jobs".
    . "per_engine_max_jobs" is automatically computed from "max_jobs" and the
      total of loop length attempts.
    . For "loop_length" in distribution format, the
      number of loop length attempts in parallel is equal to
      "max_jobs" / number of engine modes and "per_engine_max_jobs" is 1.
```

. Use "-strategy strategy_name -mode loop_swarm" to create a new strategy
  for "loop_swarm" mode.

. Use "-engine_mode" to specify the engine. The possible engines for this
  strategy are B and B4. If you do not specify this switch, the tool defaults
  to engine B.

. Use "-max_jobs" to specify the maximum number of jobs to dispatch.
  If you do not specify this switch, the default value is 6.

  NOTE:
  . If you unset this switch, the tool uses the value set for
    "proofgrid_max_local_jobs" if in local mode, or "proofgrid_max_jobs"
    otherwise.

. Use "-loop_length" to specify the loop lengths at which the engine makes
  its trace attempts.
  You can specify a list of loop lengths, a distribution, or a single value
  to be the initial loop length of a sequence determined by "-loop_length_incr".
  If you do not specify this switch, the tool defaults to 1.

. Use "-loop_length_incr" to specify the increment the tool uses between
  a loop length and the next one. This switch is only effective if "loop_length"
  contains a single value. The tool increments values until "max_loop_length"
  is reached or "max_jobs" loop lengths are generated.
  If this switch is set to 0, only the specified loop length is attempted.
  If you do not specify this switch, the tool defaults to 1.

. Use "-max_loop_length" to specify the maximum value for generated loop
  lengths values when "-loop_length_incr" is valid.
  If this is set to 0, the tool generates "max_jobs" loop lengths.
  If you do not specify this switch, the tool defaults to 0.

. Use "-max_trace_length" to specify the limit for the length of traces the
  tool searches for. You can specify a natural value for "loop_swarm".
  If you do not specify this switch, the tool defaults to 0.

. Use "-seed" for hunt reproducibility. Specify a hexadecimal value to be
  used by the tool as a seed to determine a strategy's randomly selected
  parameters that will be used in a "hunt" run. The parameters randomly
  selected by the tool are those specified in a list or distribution format
  in "loop_swarm", "cycle_swarm", "state_swarm", and "simulation" strategies.

```
      If you do not specify this switch, the default value is 0, which is a
      special value that tells the tool to select a nondeterministic seed.
      Other values give deterministic behavior. When using the same seed
      together with the same parameters and distributions, the tool generates
      identical values for parameters.

      NOTE:
        Using the same seed does not guarantee the same results due to many
        sources for non-determinism that affect engine processing from run to
        run. For example, if you use a heavily loaded machine, a time limit
        might prevent the tool from detecting a trace that it might otherwise
        detect.

      Example:

      # Default: loop_length: 1, loop_length_incr: 1, max_jobs: 6
      hunt -strategy ls -mode loop_swarm
      hunt -run -strategy ls -task .
      # loop length attempts: {1 2 3 4 5 6}

      hunt -config -strategy ls -max_jobs 5 -loop_length 10 -loop_length_incr 5
      hunt -run -strategy ls -task.
      # loop length attempts: {10 15 20 25 30}

      hunt -config -strategy ls -max_jobs 5 -loop_length {20%:[20..40] 80%:[50..80]}
      hunt -run -strategy cs -task . -seed 100
      # loop length attempts: {22 51 65 69 77}

      hunt -config -strategy ls -max_jobs 6 -loop_length {5 9 23}
      hunt -run -strategy cs -task . -seed 100
      # loop length attempts: {5 9 23}
      # per_engine_max_jobs: 2 # max_jobs / num loop lengths

Configuring Parameters for "simulation" Mode
--------------------------------------------
-config -strategy strategy_name
        -mode simulation
        [-engine_mode {engine+}]
        [-max_jobs N]
        [-max_trace_length {N+}]
        [-restarts N]
        [-seed N]
    Use the specified configuration for the "simulation" strategy.
```

The strategy parallelizes jobs with different "max_trace_length" values.

. Use "-strategy strategy_name -mode simulation" to create a new strategy for the "simulation" mode.

. Use "-engine_mode" to specify the engines used by this strategy. Valid engines are {U U2 J Q3}. If you do not specify this switch, the default value is {U2}.

. Use "-max_jobs" to specify the number of engine jobs for the hunt search.

  NOTE:
  . If you do not specify this switch, the default value is 6.
  . If you specify the value 0 for this switch, the tool uses the number of all possible "max_trace_length" values as the value for "max_jobs".
  . If you unset this switch, the tool uses the value set for "proofgrid_max_local_jobs" if in local mode, or "proofgrid_max_jobs" otherwise.
  . For the "simulation" strategy, the tool sets "per_engine_max_jobs" to value 1. Hunt creates an engine job for each possible configuration and dispatches "max_jobs" different engine jobs, each with a different engine seed.

. Use "-max_trace_length" to specify the limit for the length of traces the tool searches for. You can specify a list of values or a distribution as the argument. If you do not specify this switch, the default distribution is {50%:[50..200] 50%:[200..1000]}.

. Use "-restarts" to specify the number of restarts the engine makes before abandoning the proof.

  If you do not specify this switch, the default value is 1000.
  See also the documentation for the configuration commands "set_engineJ_restarts", "set_engineQ3_restarts", "set_proofgrid_restarts".

. Use "-seed" for hunt reproducibility. Specify a hexadecimal value to be used by the tool as a seed to determine a strategy's parameters randomly selected and engine seeds that will be used in a "hunt" run. The parameters randomly selected by the tool are those specified in a list or distribution format in "cycle_swarm", "loop_swarm", "state_swarm", and "simulation" strategies. If you do not specify this switch, the default value is 0, which is a special value that tells the tool to select a

nondeterministic seed. Other values give deterministic behavior. When using the same seed together with the same parameters and distributions, the tool generates identical values for parameters.

NOTE:
. The tool uses the hunt seed to set values for configurable run attributes, such as "first_trace_attempt" for "state_swarm" and "trace_attempt_time_limit" for "cycle_swarm". If you run the command without an explicit seed, the tool selects a random seed and uses it to define the values of any attributes that do not have an explicit setting either in the configuration or with run-time options. The tool reports auto-generated seeds, and you can use them with subsequent "hunt" runs as an explicit seed setting, thus ensuring the tool applies the same attribute values.

  Using the same seed does not guarantee the same results due to many sources for non-determinism that affect engine processing from run to run. For example, if you use a heavily loaded machine, a time limit might prevent the tool from detecting a trace that it might otherwise detect. In particular, the engine L search is sensitive to the traces it detects. Such a nuance can result in different results.

. If you unset this switch (hunt -config -strategy... -unset seed), for "simulation" mode, the tool uses seed values set in the session for the engine modes being used. See also the documentation for configuration commands "set_engineJ_seed", "set_engineU_seed", and "set_engineQ3_seed".

NOTE:
. This strategy is able to handle liveness properties only when using engine U or U2.
. If you do not specify parameters, the tool uses predefined default values when hunting.

Configuring Parameters for "state_swarm" Mode

```
-config -strategy strategy_name
        -mode state_swarm
        [-max_jobs N]
        [-max_trace_length N]
        [-first_trace_attempt {N+}]
        [-start_state_count {number_of_start_states+}]
        [-tail_length {number_of_cycles_to_backtrack+}]
```

```
    [-segment_time_limit {time_limit+}]
    [-max_segment_length {segment_length+}]
    [-random_diversification_factor {segment_length+}]
    [-target_type {(cover | assert | assert_liveness)+}]
    [-auto_helper_num N]
    [-auto_helper_type {cover_type+}]
    [-auto_keep_helpers]
    [-auto_cleanup_time_limit time_limit]
    [-auto_partition {type partition_type num N}]
    [-stagnation_time_limit time_limit
     [-stagnation_action (stop | restart)]]
    [-helper_set_store_trace (1 | unlimited)]
    [-seed N]
Use the specified configuration for the "state_swarm" strategy. This
strategy is not able to handle liveness or infinite properties.

NOTE: If you do not specify parameters, the tool uses predefined default
values when hunting.

. This strategy uses engine L.

. Use "-strategy strategy_name [-mode state_swarm]" to create a new
  strategy for "state_swarm" mode.

  The "state_swarm" mode creates a set of L engines with different
  configurations and dispatches one parallel job for each configuration.

  Engine L needs helper traces as start states to make proper progress
  while searching for new traces, that is, counterexamples (bugs) or cover
  traces. When running a "state_swarm" strategy, specify the assertion
  properties along with relevant helper cover properties as the target.

. Use "-max_jobs" to specify the number of engine L jobs to be dispatched,
  each job with specific computed values for
  "first_trace_attempt", "start_state_count", "tail_length",
  "segment_time_limit", "max_segment_length", and
  "random_diversification_factor".

  NOTE:
  . If you do not specify this switch, the default value is 20.
  . If you specify the value 0 for this switch, the tool automatically
    determines the number of engine L jobs based on the maximum size of
    the user-specified configuration lists.
```

  . If you define a configuration list with a size bigger than the value
    of "max_jobs", the tool removes its leftover elements.
  . If you define a configuration list with a size smaller than the value
    of "max_jobs", the tool replicates its elements from left to right
    until it reaches "max_jobs".
  . If you unset this switch, the tool uses the value set for
    "proofgrid_max_local_jobs" if in local mode, or "proofgrid_max_jobs"
    otherwise.
  . For the "state_swarm" strategy, the tool sets "per_engine_max_jobs" to
    the "max_jobs" value.

  Example:
```
hunt -config -strategy s1 -mode state_swarm -time_limit 2h
set_proofgrid_max_jobs 10
set_proofgrid_max_local_jobs 10
# Dispatches hunt with up to 20 engine L jobs in parallel. The tool
# ignores set_proofgrid_max_jobs / set_proofgrid_max_local_jobs above and
# assumes the default max_jobs value for both max_jobs and
# per_engine_max_jobs
hunt -run -strategy s1 -task . time_limit 1h
# Dispatches hunt with up to 10 engine L jobs in parallel. The tool
# assumes set_proofgrid_max_jobs / set_proofgrid_max_local_jobs above for
# both max_jobs and per_engine_max_jobs.
hunt -config -strategy s1 -unset max_jobs
hunt -run -strategy s1 -task . -time_limit 1h
```

. Use "-max_trace_length" to specify the limit for the length of traces the
  tool searches for. You can specify a natural value for state swarm.
  If you do not specify this switch, the tool defaults to 0.

. Use "-first_trace_attempt" to specify the cycle at which the engine will
  make its first trace attempt. You can specify a list of values or a
  distribution as the argument. If you do not specify this switch, the
  default distribution is {100%:[3..15]}.

. Use "-start_state_count" to specify how many parallel start states each
  engine L job will use to search for new traces. Each different helper
  cover trace is a new start state. You can specify a single value, a list
  of values, or a distribution as the argument. If you do not specify this
  switch, the tool uses the default single value 1, that is, 1 start state
  for each engine L job. Consider configuring more than 1 start state if
  you have many traces to explore and only a few of them might be valuable.

. Use "-tail_length" to specify the starting point for a new search that is N cycles before the end of found traces. You can specify a list of values or a distribution as the argument. If you do not specify this switch, the default distribution is {50%:[1] 30%:[2] 20%:[3..5]}.

. Use "-segment_time_limit" to specify the amount of time the tool spends searching from each start state.

  Specify the time limit as an integer followed by one of the following (no spaces):
  . s = seconds
  . m = minutes
  . h = hours

  To remove the limit, use 0s. You can specify a list of values or a distribution as the argument. If you do not specify this switch and time_limit is 0, the default distribution is {100%:[100..600]}. If you do not specify this switch and time_limit is not 0, the default distribution will be based on time_limit value.

. Use "-max_segment_length" to specify the number of trace attempts the tool makes from each start state. You can specify a list of values or a distribution as the argument. If you do not specify this switch, the default distribution is {100%:[50..300]}.

. Use "-random_diversification_factor" to specify the random diversification value used by engine L. A high value gives more diversification, but potentially, much slower progress. You can specify a list of values or a distribution as the argument. If you do not specify this switch, the default distribution is {50%:[0.03..0.15] 45%:[0.15..0.50] 5%:[0.50..0.90]}.

. Use "-target_type" to define the run target by filtering the list of properties specified in "hunt -run" by property type.
  Possible values are: "cover", "assert", or "assert_liveness".
  This feature is available for state_swarm and trace_swarm strategies.

  Example:
  hunt -config -strategy ss -mode state_swarm -target_type {cover assert}
  hunt -run -task . -strategy ss -time_limit 1h

  In the above example, although the "-run" command specifies the target is all properties in the current task, the "-config" command applies a

"filter by type," which means covers and assertions in the current task are targets for "hunt -run".

. Use "-auto_helper_num" to specify the number of helper covers the tool is expected to generate automatically and append to the targets list in the "hunt -auto" flow. Default value is 300. Helper covers are essential to "state_swarm".

. Use "-auto_helper_type" to specify the type of helper covers the tool is expected to generate automatically and append to the targets list in the "hunt -auto" flow. Available types are "fsm", "pattern", and "enable". Default value is {fsm pattern}.

  NOTE: Computation of "enable" signals may be time consuming for some designs and is disabled by default. However, using "-auto_helper_type {enable fsm pattern}" to specify this helper type might improve hunt performance for some designs.

. Use "-auto_keep_helpers" if you want to have the generated helpers visible in the property table.

. Use "-auto_cleanup_time_limit" to specify the amount of time the tool should spend cleaning up unreachable and easy helper covers in the "hunt -auto" flow. Cleanup of unreachable and easy helper covers might improve the performance of "state_swarm".
  Default value is 120s for the "state_swarm" strategy mode.

. Use "-seed" for hunt reproducibility. Specify a hexadecimal value to be used by the tool as a seed to determine a strategy's parameters randomly selected and engine seeds that will be used in a "hunt" run. The parameters randomly selected by the tool are those specified in a list or distribution format in "cycle_swarm", "loop_swarm", "state_swarm", and "simulation" strategies. If you do not specify this switch, the default value is 0, which is a special value that tells the tool to select a nondeterministic seed. Other values give deterministic behavior. When using the same seed together with the same parameters and distributions, the tool generates identical values for parameters.

  NOTE:
  . The tool uses the hunt seed to set values for configurable run attributes, such as "first_trace_attempt" for "state_swarm" and "trace_attempt_time_limit" for "cycle_swarm". If you run the command without an explicit seed, the tool selects a random seed and uses it to

define the values of any attributes that do not have an explicit setting either in the configuration or with run-time options. The tool reports auto-generated seeds, and you can use them with subsequent "hunt" runs as an explicit seed setting, thus ensuring the tool applies the same attribute values.

Using the same seed does not guarantee the same results due to many sources for non-determinism that affect engine processing from run to run. For example, if you use a heavily loaded machine, a time limit might prevent the tool from detecting a trace that it might otherwise detect. In particular, the engine L search is sensitive to the traces it detects. Such a nuance can result in different results.

. If you unset this switch (hunt -config -strategy... -unset seed), specifically for "state_swarm" mode, the tool uses the seed value set in the session for engine L. See also the documentation for the configuration command "set_engineL_seed".

. Use "-auto_partition" to partition properties into groups and run the "hunt -auto" flow sequentially for each partition. A dedicated set of helper covers will be generated for the COI of each group, and the total run time will be divided equally among all groups.

. Use "-stagnation_time_limit time_limit" to define the maximum time interval to wait with no determined result in the "hunt -auto" flow. When the tool reaches stagnation, it completes the stagnation action. Default is 0, disabling stagnation monitoring.

. Use "-stagnation_action" to specify the action to take when the tool reaches the stagnation time limit. Action options are:
  . "stop" - Interrupt the current proof
  . "restart" - Interrupt the current partition proof and start a new "hunt -auto" flow for the undetermined properties. The tool re-partitions remaining undetermined properties after one round with all partitions and restarts the "hunt -auto" flow with new targets.

  Default value is "stop".

. Use "-helper_set_store_trace" to save multiple traces for internally generated helper covers for the "hunt -auto" flow.
  Default value is 1. Set to "unlimited" to save multiple traces.

NOTE: "-no_cover_traces" is on by default for "state_swarm" strategies only. To turn it on or off, specify the optional argument "true" or

```
    "false".


Configuring Parameters for "trace_swarm" Mode
---------------------------------------------
-config -strategy strategy_name
        -mode trace_swarm
        [-engine_mode {engine+}]
        [-max_jobs N]
        [-per_engine_max_jobs N]
        [-max_trace_length N]
        [-tail_length N]
        [-per_trace_time_limit per_trace_time_limit]
        [-target_type {(cover | assert | assert_liveness )+}]
    Use the specified configuration for the "trace_swarm" strategy. This
    strategy is able to handle liveness or infinite properties with the default
    engine setup.

    . Use "-strategy strategy_name -mode trace_swarm" to create a new strategy
      for the "trace_swarm" mode.

    . Use "-target_type" to define run target by filtering the list of
      properties specified in "hunt -run" by property type.
      Possible values are: cover, assert or assert_liveness.
      Feature available for state_swarm and trace_swarm strategies.

      When you run "hunt" with the "trace_swarm" strategy, the tool dispatches
      multiple hunt runs for the specified target from each trace
      available in the run traces queue. The run traces queue is populated with
      either all traces available in the Property Table or traces found by
      the "trace_swarm" strategy or traces found by some other strategy running
      in parallel. See "-use_strategy" switch in "hunt -run".

      This strategy extends bug hunting from traces found by any "prove"
      and/or any "hunt" strategy using any specified engine.

      Example:
      hunt -config -strategy ts -mode trace_swarm -target_type assert_liveness
                   -per_trace_time_limit 10m
      hunt -run -task . -strategy ts

      In the above example, for each trace in the Property Table, the tool does a
      hunt on the liveness assertions in the current task from the last cycle of
```

the trace.

. Use "-engine_mode" to specify the engines used by this strategy.
  If you do not specify this switch, the default value is
  {Ht B}.

. Use "-max_jobs" to specify the number of engine jobs to be used for bug
  hunting from traces. If you do not specify this switch, the default value
  is 20. You might see fewer engine jobs than the specified "max_jobs" if
  an insufficient number of traces are available.

  NOTE:
  . If you unset this switch, the tool uses the value set for
    "proofgrid_max_local_jobs" if in local mode, or "proofgrid_max_jobs"
    otherwise.
  . "max_jobs" must be a limited value greater than 0 for "trace_swarm"
    strategy.

. Use "-per_engine_max_jobs" to configure the number of engine_jobs per
  engine to dispatch. If you do not specify this switch, the default
  value is 1.

  NOTE: If you unset this switch, the tool uses the value set for
  "proofgrid_per_engine_max_local_jobs" if in local mode, or
  "proofgrid_per_engine_max_jobs", otherwise.

  Example:
```
  set_proofgrid_mode local
  set_proofgrid_per_engine_max_local_jobs 15
  hunt -config -strategy t1 -mode trace_swarm
  hunt -run -strategy t1 ;# uses per_engine_max_jobs from state_swarm default
                         ;# value(1)
  hunt -config -strategy t1 -mode trace_swarm -per_engine_max_jobs 10
  hunt -run -strategy t1 ;# uses per_engine_max_jobs strategy value(10)
  hunt -run -strategy t1 -per_engine_max_jobs 5 ;# uses hunt -run
                                                ;# value(5)
  hunt -config -strategy t1 -unset per_engine_max_jobs
  hunt -run -strategy t1 ;# uses proof environment per_engine_max_local_jobs
                         ;# value(15)
```

. Use "-max_trace_length" to specify the limit for the length of traces the
  tool searches for. You can specify a natural value for trace swarm.
  If you do not specify this switch, the tool uses the current proof

```
      environment configuration values for simulation engines and 0 for
      other engines.

    . Use "-tail_length" to specify the starting point for a new search that is
      N cycles before the triggered at cycle of found traces. If you do not specify
this
      switch, default value is 0.

    . Use "-per_trace_time_limit" to specify the time limit for hunting from
      each trace. If you do not specify this switch, the default value
      is the "time_limit" divided by 10, assuming 1m as the lower limit and
      20m as the upper limit.

      The tool continuously identifies available proof licenses, so that there
      are always the maximum number of proof jobs running in parallel.
      As hunting from each trace finishes, new proof licenses become available,
      and the tool automatically dispatches more jobs. This process continues
      until hunt reaches "time_limit" or all properties are determined.

Configuring Parameters for "trace_search" Mode
----------------------------------------------
-config -strategy strategy_name
        -mode trace_search
        [-engine_mode {engine+}]
        [-max_jobs N]
        [-target_depth N]
        [-interval_cycles N]
        [-interval_time_limit (auto | time_limit)]
    Use the specified configuration for the "trace_search" strategy. This
    strategy explores to a specified depth around a trace. Users specify the
    target depth they wish to explore, and the tool launches searches at regular
    intervals along the trace to explore to the specified depth. This strategy
    is able to handle liveness or infinite properties with the default engine
    setup.

    . Use "-strategy strategy_name -mode trace_search" to create a new strategy
      for the "trace_search" mode.

    . Use "-engine_mode" to specify the engines.
      If you do not specify this switch, the tool sets it by default to
      {B Bm}.

    . Use "-max_jobs" to specify the number of engine jobs you want to use for
```

bug hunting from traces. If you do not specify this switch, the default value is 20. You might see fewer engine jobs than the specified "max_jobs" if an insufficient number of traces and cycle intervals are available.

. Use "-target_depth" to specify the target depth to search from any point along the trace. The value must be greater than 0. If you do not specify this switch, the tool sets it by default to 10.

. Use "-interval_cycles" to specify the number of cycles between subsequent search points. The value must be greater than 0. If you do not specify this switch, the tool sets it by default to the target_depth.

. Use "-interval_time_limit" to specify the time limit to each interval search. If you specify 0, any new search will take the remaining time limit as the search time limit. If you specify any value greater than zero, any new search time limit will be that value. If you specify "auto", an estimated value based on the total time limit and the total number of searches to be launched will be used as the interval search time limit. If you do not specify this switch, the tool sets it to "auto".

Adding and Removing Strategy's Constraints

```
-config -strategy strategy_name
        ( -add_constraint expression [-name constraint_name] [-bound N]
         |-remove_constraint constraint_name)
```

Add or remove constraints to or from strategies.

NOTE: These constraints will be local to the hunt run, that is, they are not part of the task environment.

. Use "-strategy" to specify the strategy you are adding the constraint to or removing the constraint from. This switch is mandatory.

. Use "-name" to specify a custom name for a constraint. Names must be unique in the strategy scope. By default, the tool automatically generates a name following the format <strategy_name>:constraint:<number>. Use "hunt -show -strategy strategy_name" to see strategy constraints.

. Use "-bound" to apply the specified constraint for N number of cycles (after reset). By default, a constraint is applicable for all cycles.

Examples:

```
      . Add constraint s1:constraint:0 (default name) to strategy s1:
        hunt -config -strategy s1 -add_constraint "a==b"
      . Add constraint enable1, applied on the first 10 cycles, to strategy s1:
        hunt -config -strategy s1 -add_constraint "en1" -name enable1 -bound 10
      . Add constraint b_1 to strategy s1:
        hunt -config -strategy s1 -add_constraint "b==a" -name b_1
      . Remove constraint b_1 from strategy s1:
        hunt -config -strategy s1 -remove_constraint b_1

Saving and Loading Hunt
-----------------------
-save file_name.cfg [-strategy strategy_name] [-force]
    Save strategy configurations to the specified file. The file extension must
    be ".cfg".

    . Use "-strategy" to select a specific strategy to save.

    . Use "-force" to save to an existing file.

-load file_name.cfg
    Load strategy configurations from a file saved by "hunt -save". The file
    extension must be ".cfg".

NOTE: Jasper "save" and "restore" commands also save and load hunt strategies.
For additional details, also see the documentation for these commands.

Running Hunt
------------
-run [-strategy strategy_name]
     ( -property property_name_tcl_list [-regexp]
      |-task task_name)
     [-with_proven]
     [-evaluate_in_trace property_name_tcl_list]
     [-tag tag_name]
     [-use_strategy strategy_name]
     [-auto
       [-auto_helper_num N]
       [-auto_helper_type {cover_type+}]
       [-auto_keep_helpers]
       [-auto_cleanup_time_limit time_limit]
       [-disable_trace_swarm]
       [-auto_partition {type partition_type num N}]
       [-stagnation_time_limit time_limit
```

```
    [-stagnation_action (stop | restart)]]]
 [-engine_mode {engine+}]
 [-time_limit time_limit]
 [-max_jobs N
   [-trace_swarm_ratio N]]
 [-per_engine_max_jobs N]
 [-max_trace_length N]
 [-no_cover_traces [(true | false)]]
 [-helper_set_store_trace (1 | unlimited)]
 [-from property_name [-trace_id N] [-cycle N]]
 [-seed N]
 [-rerun_job (job_index | job_name)]
 [-on_determined tcl_callback]
 [-dump_trace
     [-dump_trace_type (shm | vcd | fsdb | vwdb)]
     [-dump_trace_dir dir_name]]
 [-verbosity N]
 [-bg]
Run the specified or default bug-hunting strategy to search for
counterexamples (bugs).

. Use "-strategy" to specify the strategy you want to run. If you do not
  specify a strategy, the tool uses the default "state_swarm" strategy.

. Use "-property" or "-task" to specify the target(s) of the hunt.

  NOTE:
  . "-property" supports wildcards and regular expressions in property
    names. When using a regular expression, use the "-regexp" switch after
    the pattern, for example:

    hunt -run -property {pattern} -regexp

  . Include as part of the target(s) all the properties that are proven or
    unreachable that you want to be used as assumptions during the hunt run,
    and make "set_proven_directive true". If you want the tool to use all
    proven assertions in the same task as assumptions, you can skip adding
    them as part of the target property list and just specify "-with_proven"
    (see below).

. Use "-with_proven" to use all proven assertions in the same task as
  assumptions.
  This switch offers a convenient way to specify that you want the tool to
```

use all proven assertions as assumptions without having to include them as part of the target property list and to rely on the "set_proven_directive" command.

. Use "-evaluate_in_trace" to include a set of properties that are evaluated in the trace whenever the tool finds a cex or cover for the target property. The proof results for these included properties will mainly be discovered by checking the traces found on the target properties, that is, without running a hunt on them.

. Use "-tag" to add a tag to the specific "hunt" run.
If you do not specify "-tag", the tool generates one automatically for the run consisting of the strategy name appended with ":" and a sequential number.

. Use "-use_strategy" to join a "trace_swarm" strategy with a different strategy in the same hunt run. The strategy specified with "-strategy" runs first, and the "trace_swarm" strategy specified with "-use_strategy" is triggered by some condition raised by the first strategy. Currently, "trace_swarm" only supports joining it with "state_swarm", "simulation", and "formal".

Each trace found by the first strategy run triggers the command "hunt -run -from" in the "trace_swarm" run. "-use_strategy" enables hunting for liveness or infinite properties with "state_swarm", which is not possible with a regular "state_swarm" hunt run.

Example:
(1)
```
hunt -config -strategy ss -mode state_swarm -target_type cover
hunt -config -strategy ts -mode trace_swarm -target_type assert_liveness
           -per_trace_time_limit 10m
hunt -run -task . -strategy ss -use_strategy ts -time_limit 1h
```

In the above example, the tool runs "ss" strategy over the cover properties in the current task. In parallel, for each trace found by "ss", it does a hunt on the liveness assertions in the current task from the last cycle of the trace for 10 minutes.

(2)
```
hunt -config -strategy sim -mode simulation -target_type cover
hunt -run -task . -strategy sim -use_strategy ts -time_limit 1h
```

These commands result in the same flow described by (1) except that it uses "simulation" as the first strategy instead of "state_swarm". The same can be done with "formal"; that is, you can use "formal" as the first strategy.

. Use "-auto" to dispatch the bug hunting recommended flow. It comprises the following steps:

(1) Generating helper covers and cleaning up easy ones until reaching the number of undetermined helper covers specified by "-auto_helper_num".
This cycle might be repeated for a maximum of three times (check "-auto_helper_num" and "-auto_cleanup_time_limit").
Quality of helper covers is essential to the good performance of "state_swarm". Although configurable for any strategy allowed by "hunt -auto", this step is enabled by default only for "state_swarm".

(2) Dispatching the first strategy on specified targets plus helper covers automatically generated in the previous step (when applicable).
By default, the "hunt -auto" flow dispatches the default "state_swarm" strategy. You can create a customized "state_swarm", "simulation", or "formal" strategy with
"hunt -config" and apply it by specifying
"hunt -run -auto -strategy strategy_name".

(3) Dispatching "trace_swarm" for all traces found in steps (2) and (3).
By default, the "hunt -auto" flow dispatches the default "trace_swarm" strategy, but it assumes the first strategy's "max_jobs" instead of the default "trace_swarm" "max_jobs".
You can customize a "trace_swarm" strategy with "hunt -config" and apply it by specifying "hunt -run -auto -use_strategy strategy_name". In this case, the tool honors the customized "trace_swarm" "max_jobs".

NOTE: "max_jobs" must be a limited value greater than 0 when you specify "-auto".

. Use "-auto_helper_num" to specify the number of helper covers the tool is expected to generate automatically and append to the targets list in the "hunt -auto" flow. Helper covers are essential to "state_swarm"; thus, the default value is 300 for this strategy. For other strategies,

the default is set to 0.

. Use "-auto_helper_type" to specify the type of helper covers the tool is expected to generate automatically and append to the targets list in the "hunt -auto" flow. Available types are "fsm", "pattern", and "enable". Default value is {fsm pattern}.

  NOTE: Computation of "enable" signals may be time consuming for some designs and is disabled by default. However, using "-auto_helper_type {enable fsm pattern}" to specify this helper type might improve hunt performance for some designs.

. Use "-auto_keep_helpers" if you want to have the generated helpers visible in the property table.

. Use "-auto_cleanup_time_limit" to specify the amount of time the tool spends cleaning up easy helper targets in the "hunt -auto" flow. Default value is 120s for "state_swarm" strategy mode and 0 for other strategies supported by the "-auto" flow.
  Cleanup of easy targets might improve the performance of "state_swarm".

. Use "-disable_trace_swarm" to avoid running "trace_swarm" in the "hunt -auto" flow.

. Use "-auto_partition" to partition properties into groups and run the hunt auto flow sequentially in each partition.

  NOTE:
  . A dedicated set of helper covers is generated for the COI of each group, and the total run time is divided equally between all groups.
  . Define grouping either by specifying the number of properties in each group or by the number of groups. The default runs all properties together in a single group ({type group num 1}). To run the hunt auto flow in each property separately, use {type property num 1}.

. Use "-stagnation_time_limit time_limit" to define the maximum time interval to wait with no determined result in the "hunt -auto" flow. When the tool reaches stagnation, it completes the stagnation action. Default is 0, disabling stagnation monitoring.

. Use "-stagnation_action" to specify the action to take when the tool reaches the stagnation time limit. Action options are:

```
. "stop" - Interrupt the current proof
. "restart" - Interrupt the current partition proof and start a new
  "hunt -auto" flow for the undetermined properties. The tool
  re-partitions remaining undetermined properties after one round with
  all partitions and restarts the "hunt -auto" flow with new targets.
Default value is "stop".

Examples:

. Auto-generate helper covers, clean them up and run default strategies
  for "state_swarm" and "trace_swarm": 40 total jobs = 20SS + 20TS

  hunt -run -auto -property|-task

. Do not run trace_swarm: total jobs = 20SS

  hunt -run -auto -property|-task -disable_trace_swarm

. 50 total jobs - scaled as per ratio in default strategies:
  50% "trace_swarm" = 25SS + 25TS

  hunt -run -auto -property|-task -max_jobs 50

. 50 total jobs, scaled as per explicit 40% "trace_swarm" = 30SS + 20TS

  hunt -run -auto -property|-task -max_jobs 50 -trace_swarm_ratio 40

. Increase the number of helper covers to 200 and allocate 1m
  clean-up/optimization time

  hunt -run -auto -property|-task -auto_helper_num 200
       -auto_cleanup_time_limit 1m

. Enable stagnation monitoring, default action is to stop after
  stagnation

  hunt -run -auto -property|-task -stagnation_time_limit 10m

. Change the default grouping of a single group to 5 groups

  hunt -run -auto -property|-task -auto_partition {type group num 5}

. Change the default grouping of a single group to N groups with 10
```

properties each

hunt -run -auto -property|-task -auto_partition {type property num 10}

. Change the default grouping of a single group to a single property per group

hunt -run -auto -property|-task -auto_partition {type properties num 1}

. Use a custom strategy for "state_swarm" with the default "trace_swarm" strategy

```
hunt -config -strategy SS -mode state_swarm ...
hunt -run -auto -property|-task -strategy SS
```

. Use a default strategy for "state_swarm" with custom "trace_swarm" strategy

```
hunt -config -strategy TS -mode trace_swarm ...
hunt -run -auto -property|-task -use_strategy TS
```

. Use custom strategies for both "state_swarm" and "trace_swarm"

hunt -run -auto -property|-task -strategy SS -use_strategy TS

. Use "-engine_mode", "-time_limit", "-max_jobs", "-per_engine_max_jobs", "-max_trace_length", "-no_cover_traces", and "-seed" to customize a specific hunt run without needing to change the strategy configuration you want to use:

. "-engine_mode" specifies engines used by the hunt when applicable.

NOTE: If you use more than one strategy in a hunt run (for example, you specify the "-use_strategy" switch or a "hunt -auto" flow), this switch determines only the engines for the first strategy (excluding "state_swarm", in which only engine L is allowed). Use the "hunt -config" command to modify engine_mode for the secondary strategy.

. "-time_limit" specifies the hunt time limit.

. "-max_jobs" specifies the maximum number of engine jobs that run in parallel in a hunt run. If you have enough licenses, "max_jobs" engine jobs will run in parallel.

It replaces the max_jobs previously defined by a strategy. If you specify "-use_strategy", max_jobs is distributed to both "-strategy" and "-use_strategy" according to the ratio of jobs specified by "-strategy" and jobs specified by "-use_strategy".
For example: "hunt -auto -max_jobs 60" means 30 jobs for "state_swarm" and 30 jobs for "trace_swarm", since both default "state_swarm" strategy and default "trace_swarm" strategy define max_jobs as 20.

. You can use "-trace_swarm_ratio" together with "-max_jobs" to distribute max_jobs between "-strategy" and "-use_strategy" according to the specified ratio "N". "N" is a natural number 0 - 100, inclusive. For example, for "hunt -run -auto -max_jobs 80 -trace_swarm_ratio 75", the first strategy runs 20 jobs while "trace_swarm" strategy runs 60 (75% of 80).

NOTE: "max_jobs" must be a limited value in "hunt -auto" and "trace_swarm" strategy runs. In these cases, the tool reports an error if "max_jobs" is 0 (unlimited).

. "-per_engine_max_jobs" specifies the number of engine_jobs per engine to dispatch.

NOTE:
. The formal, guidepoint, and trace_swarm strategies support this switch.
. The "bound_swarm", "cycle_swarm", "loop_swarm", "state_swarm", and "simulation" strategies DO NOT accept this switch.
. For the strategies that support "per_engine_max_jobs", this switch overrides proofgrid_per_engine_max_jobs and proofgrid_per_engine_max_local_jobs. See "set_proofgrid_per_engine_max_jobs" and "set_proofgrid_per_engine_max_local_jobs". Also see specific semantic for values set for "per_engine_max_jobs" combined with "max_jobs" in the sections for each strategy configuration.

. "-max_trace_length" specifies the limit for the length of traces the tool searches for.

. "-no_cover_traces" specifies that no cover traces will be generated. Use this switch to avoid the overhead of generating and storing cover traces.
It is on by default for "state_swarm" strategies only. To turn it on

```
      or off, specify the optional argument "true" or "false".

    . "-helper_set_store_trace" is an option to save multiple traces for
      internally generated helper covers (for example, in the hunt auto flow).
      Default value is 1. Set to "unlimited" to save multiple traces.

    . "-seed" specifies a hexadecimal value for the seed that determines
      parameters set as a list or distributions and engine seeds when
      applicable. This switch only affects "cycle_swarm", "loop_swarm",
      "simulation", and "state_swarm" modes.

  . Use "-from" to specify an existing trace as the reset value for the
    hunt run.

    . Use "-trace_id" to specify a trace other than the default property
      trace. The default one is the last shortest trace. See "assert/cover
      -set_store_trace" for the commands to configure the number of traces
      generated and stored by the tool.

    . Use "-cycle" to specify the state of the trace at a specific cycle as
      the reset value. If you do not specify a cycle, or if you specify 0,
      the tool uses the cycle in which the property was triggered. This is
      often, but not always, the last cycle of the trace. See
      "set_trace_extension".

    . If the running strategy is a trace_swarm or trace_search strategy,
      "-from" accepts a Tcl list as the argument. The tool checks all properties
      specified in the list, filtering out the ones that do not have a trace
available.
      The tool adds all the existing traces to the traces queue and those that
      are common to different properties are added only once.

      NOTE:
      . If the tool does not find a trace for any of the properties on the
        list, the command errors out.
      . If you also specify "-trace_id" for trace_swarm or trace_search, "-from"
accepts
        only one property, and the specified trace must belong to the
        specified property.
      . If you do not specify "-from" for "trace_swarm" or "trace_search",
        the tool populates the run traces queue with the traces available
        in the Property Table.
```

. Use "-rerun_job" to reproduce a single job from a previous hunt run. This switch assures that, if you use the same set of configurations from a previous run, you are able to re-run a job that might have produced good results (see also "hunt -report").

  You can use either of two ways to specify which job you want to run again: specify the job index or job name. You can get job indexes from messages that print in the proof log. For example, you can specify a job named "5.8.L" that prints a message "5.8.L: Job index 8" by either job index 8 or job name "5.8.L". See the examples section.

  If you do not re-run a job inside the same session using its name as an argument ("5.8.L" in our example), you must use the same configurations (strategy and seed).

  NOTE:
  . This switch only affects "state_swarm" mode.
  . Re-running a job does not guarantee the same results as a previous run. Results can be influenced by many factors, including changes in design constraints, machine resources, license availability, and so forth. However, it does improve the chance of achieving those results again.
  . When using the "-rerun_jobs" switch, the tool automatically overrides "-max_jobs" and sets it to 1.

. Use "-on_determined" to set a Tcl proc to call when "hunt" determines a property.

  The tool calls the specified command or procedure when a "hunt" command determines a property. The tool provides three arguments: the event, the property name, and the "prove" thread ID. Currently, the event is always "determined", but it is possible that we will introduce more events in a future version.

  CAUTION: "on_determined" procedures are very powerful, but they can be unsafe as they are hard to debug, and it is possible to use procedures that make the tool loop.

. Use "-dump_trace" to dump traces to the disk as soon as the engines find them. The tool uses the SHM format as the default and writes files in the project directory. The trace has values for all signals in the analysis region of the proof target.

  . Use "-dump_trace_type" with "shm", "vcd", "fsdb", or "vwdb" to

```
      specify the file dump type. VCD files are dumped as compressed files to
      save disk space.

      IMPORTANT:
      . Jasper writes 6.3 FSDB traces and reads version 6.6 FSDB
        traces. FSDB Reader 6.6 was introduced by Verdi version X-2025.06.
      . FSDBs you generate with the latest Jasper are backwards compatible
        with all Jasper versions from 2025.03 FCS and newer. In addition,
        they are  backwards compatible with older versions of Verdi released
        during that same period.
      . Jasper supports backwards compatibility, that is, the tool
        can read FSDBs generated from older versions of Jasper and
        Synopsys Verdi.

    . Use "-dump_trace_dir" to specify the dump directory where the files
      will be written.

  . Use "-verbosity" to change the verbosity level for messages. The default
    level is 1.

  . Use "-force" to run the "hunt" command for the selected properties,
    regardless of cex or covered status. In these cases, the tool tries to
    obtain more traces for any property that is configured to store multiple
    traces.

    See "assert/cover -set_store_trace" for the commands to configure the
    number of traces generated and stored by the tool.

    As opposed to "prove -force", "hunt -run -force" does not clear
    existing proof results.

    NOTE: If you use "-force" with the "state_swarm" strategy, the tool uses
    all covers (including determined covers) in the proof target as helper
    covers for engine L. In addition, it is not necessary to explicitly
    configure the tool to store multiple traces.

  . Use "-bg" to run the "hunt" command in the background.

Viewing Hunt
------------
-list (strategy | tag)
    List hunt strategies or tags.
```

```
    . Use "strategy" to list all strategies.
    . Use "tag" to list all tags.

-show -strategy [strategy_name] [-silent]
    Show strategy details.

    . Use "-strategy" to show details about the specified strategy. If you do
      not specify the strategy name, the command shows details about all
      strategies.

    . Use "-silent" to suppress output on the console.

    NOTE: In this release, use "hunt -report" to see tag details.

Reporting Hunt
--------------
-report [-status]
        [-detailed]
        (([-engine_config]
          [-engine_stats]
          [-job_score])
         | [-from_info])
        [-start_time]
        [-sort_by (  cex | covered | first_trace_attempt | job
                   | local_overconstraint | max_segment_length
                   | max_trace_length | mode | random_diversification_factor
                   | rarity_job_score | restarts | results_job_score
                   | searches_from_reset | segment_length_exceeded
                   | segment_time_limit | segment_time_out | start_state_count
                   | start_time | tail_length | total_searches | unprocessed ) ]
    Report results of "hunt -run" executions. The report lists the number of
    properties for each tag along with their respective status. It also prints
    the list of jobs that were fired during its execution.

    . Use "-status" to report results for background "hunt -run" instances that
      are still running. With "-status", the tool presents results for tags for
      which the run has not finished yet. Without "-status", the tool only
      lists tags for which the run has already finished.

    . Use "-detailed" to include the list of properties that each job found a
      status for.

    . Use "-engine_config" to see the specific configurations of each job.
```

NOTE: Configurations are only shown if you use one of the following engines: L, B, B4, or Bm.

. Use "-engine_stats" to see the specific statistics of each job.

NOTE: Statistics are only shown if you use engine L.

. Use "-job_score" to get two kinds of scores for each job: "Results Job Score" and "Rarity Job Score." The former is based on the number of determined properties by each job. The latter is based on the rarity of properties found by each job. Properties that are found by many jobs are considered "easy" targets, while properties that are found by just one or a few jobs have bigger weights and thus elevate the rarity job score. The higher the scores, the better.

NOTE: You can use scores to compare jobs within the same tag. Do not use scores to compare jobs with different tags.

. Use "-from_info" to show information about "hunt" runs that use an existing trace as the initial value, such as runs with the trace_swarm strategy and other runs that use the "-from" switch.

NOTE: You cannot use "-from_info" with "-engine_config", "-engine_stats", or "-job_score".

. Use "-start_time" to see the date and time in which tags were started.

. Use "-sort_by" to specify the sorting column of the output table.

By default, the tool lists tags in the order they were executed and sorts jobs of the same tag in ascending order. When sorting by "start_time" or "mode", you are changing the way tags are listed. Otherwise, you are choosing columns with job-related information. Some columns only appear when you use the switches "-engine_stats" and "-engine_config" along with some modes. Others will only be available when using switches "-job_score" and "-start_time". When you try to sort by a column that is not available in the output table, the tool issues a warning message and uses the default sorting.

. Meaning of each column in the report:

Default Columns

```
---------------
. Mode: strategy mode. Example: "state_swarm", "cycle_swarm".
. Strategy: Strategy name. If you do not configure a name, the tool
  generates it. Example: "<default>".
. Tag: Run name. If you do not configure a name, the tool generates it.
  Example: "<default>:0".
. Properties: Number of properties considered by the run/job.
. Processing: Number of properties that are being processed by the
  run/job. This column is only available if you use the switch "-status".
. Unproc: Number of properties unprocessed by the run/job.
. Total CEX: Number of assertion properties for which the job/tag found a
  counterexample. See also "-detailed" switch.
. Total Covered: Number of cover properties for which the job/tag found a
  cover trace. See also "-detailed" switch.
. Seed: Seed value used in the run. See also "-seed".
. Result: Result string returned by the Tcl command at the end of the
  run. This column is not available if you use the switch "-status".
. Start Time: Date and time in which this tag started. This column is only
  available if you use the "-start_time" switch.

Engine Configuration Columns
----------------------------
These columns are available when using the switch "-engine_config".
. Start State Count: Engine L job start state used to search for new
  traces. See also "-start_state_count".
. Tail Length: Engine L job starting point for a new search that is N
  cycles before the end of found traces. See also "-tail_length".
. Max Segment Length: Maximum number of trace attempts to be done by an
  engine L job from its start state. The trace attempts number
  corresponds to the job segment length. See also "-max_segment_length".
. Segment Time Limit: Amount of time the engine L job spends searching
  from each start state. See also "-segment_time_limit".
. Random Diver. Factor: Diversification value used by the engine L job.
  See also "-random_diversification_factor".
. First Trace Attempt: The cycle at which jobs from engines L, B, B4, or
  Bm will make their first trace attempt. See also "-first_trace_attempt".
. Per Prop. Time Limit: Amount of time allowed for the engine B job to
  spend on each property. See also "-per_property_time_limit" and
  "set_prove_per_property_time_limit".
. Per Prop. Time Limit Fact.: Engine B job factor used to grow the time
  shown in "Per Prop. Time Limit" column. See also
  "-per_property_time_limit_factor" and
  "set_prove_per_property_time_limit_factor".
```

```
    NOTE: Engine configurations are used to configure the jobs in the
    beginning of the run.

    Engine Statistics Columns
    -------------------------
    These columns are available when using the switch "-engine_stats".
    . Segment Time Out: Number of times the engine L job timed out searching
      from the current start state before switching to a new unused start
      state.
    . Segment Length Exceeded: Number of times the engine L job reached a
      segment length limit.
    . Searches From Reset: Number of times the engine L job took "reset" as
      the start state.
    . Total Searches: Total number of start states used by the engine L job.
      NOTE: "Total CEX" and "Total Covered" might be greater than the total
      number of searches due to incidental traces.
    . Local Overcon.: Number of times the engine L job found a dead-end
      situation because of overcontraints.
    . Restarts: Number of times the engine L job was restarted by ProofGrid
      or by the user.
    . Max Trace Length: Max trace length reached by the engine L job.

    NOTE: Engine statistics are collected during the run and reflect each
    job behavior, which is influenced directly by the job configurations.

    Job Score Columns
    -----------------
    These columns are available when using the switch "-job_score".
    . Results Job Score: Score value based on the number of determined
      properties by each job.
    . Rarity Job Score: Score value based on the rarity of properties found by
      each job. Properties that are found by many jobs are considered "easy"
      targets, while properties that are found by just one or a few jobs have
      bigger weights and thus elevate this score.

-report -trace_swarm
        [-tag tag_name]
        [-pending]
        [-completed]
        [-running]
        [-silent]
```

Report results of "hunt -run" for trace_swarm strategies specified with either the "-strategy" or "use_strategy" switch. If you do not specify any options, the report presents a high-level summary for each tag.

. Use "-tag" to report results for a specific trace_swarm tag. If you specify tags for strategies that are not trace_swarm or are related to a run that does not involve a trace_swarm strategy, the command returns a blank report.

. Use "-pending" to report details about the state of the queue, displaying all traces that have not been consumed for the specified tag.

. Use "-completed" to report details about the state of completed runs for the specified tag, displaying threads and jobs that have consumed a trace in the queue and the number of properties and property status for each job.

. Use "-running" to report details about the state of active runs for the specified tag, displaying threads and jobs that have consumed a trace in the queue and the number of properties and property status for each job.

. Use "-silent" to display only Tcl return data without printing formatted report text in the console.

Clearing Hunt

```
-clear [ -strategy [strategy_name] [-force]
        |-tag [tag_name]]
```

Clear hunt data.

. Use "-strategy" to remove strategy configurations.

. Use "-force" to force the removal of the strategy's tags when there are tags for the strategy.

. Use "-tag" to remove tags.

. Specify "-clear", without additional switches, to clear all hunt data, that is, strategies, tags, and their reporting results.

NOTE: Removing a tag clears its "hunt" reporting results but not the proof results in the Property Table. To reset property status in the Property

```
    Table, you must run "clear -result".
```

Example

```
Hunting with the Default Strategy
---------------------------------
Bug hunting supports a default strategy that does not require any
configuration.

The use model is:

analyze requirement.v
elaborate
reset rst
clock clk
hunt -run -task . -time_limit 1h

This command hunts using the default "state_swarm" strategy with default
parameters.

Customizing the Default Strategy
--------------------------------
You can customize the default strategy by creating a new strategy with the
default mode and then customizing it.

The use model is:

analyze requirement.v
elaborate
reset rst
clock clk
hunt -config -strategy ST1   // Creates new strategy ST1 with default mode
                               // (state_swarm)
hunt -config -strategy ST1 -max_jobs 30   // Customizes number of max jobs
set helper_covers [get_property_list -include {type cover}] // helper covers
set asserts {a1 a2 a3}     // target asserts
set target_properties [concat $asserts $helper_covers] // Define target
                             // properties to include helper covers for engine L
hunt -run -strategy ST1 -property $target_properties -time_limit 1h

This command hunts using newly created strategy ST1, which has been customized
to change the max number of jobs.
```

```
Creating a Strategy with a Non-Default Mode
-------------------------------------------
You can create strategies with different modes.

The use model is:

analyze requirement.v
elaborate
reset rst
clock clk
hunt -config -strategy ST1 -mode guidepoint  // Creates new strategy
                                               // ST1 and sets its mode
                                               // to "guidepoint"
hunt -config -strategy ST1 -path {C1 C2}     // Sets ordered list of covers
hunt -run -strategy ST1 -property prop:0

This command hunts using strategy ST1. First, it tries to reach C1. If it
succeeds, it tries to reach C2. If it reaches C2, then a formal prove of the
target property begins, starting from the C2 trace.

Creating a "cycle_swarm" Strategy
---------------------------------
NOTE: You can also combine all switches in one "hunt" command.

hunt -config -strategy cs1 -mode cycle_swarm -engine_mode B
hunt -config -strategy cs1 -per_property_time_limit 30
hunt -config -strategy cs1 -per_property_time_limit_factor 10
hunt -config -strategy cs1 -max_jobs 6 -first_trace_attempt {10 12 14 16 18 20 24 28} -
max_first_trace_attempt 3
hunt -run -task t1 -strategy cs1
# Dispatch 3 trace attempts groups, each one with 2 engine B jobs and 1
# first_trace_attempt value randomly selected from the list.
hunt -config -strategy cs1 -first_trace_attempt {100%:[10..12]} -engine_mode Bm
hunt -run -task t1 -strategy cs1
# Tool ignores per_property_time_limit, per_property_time_limit_factor,
# max_first_trace_attempt since they are not valid for engine Bm and assumes
# the value 3 for max_jobs since there are only 3 possible values for
# first_trace_attempt.
hunt -config -strategy cs1 -max_jobs 12 -first_trace_attempt {14 16 18 20 24 28} -
max_first_trace_attempt 0
hunt -run -task t1 -strategy cs1
# Dispatch 6 trace attempts groups with each specified first_trace_attempt,
```

```
# with value 2 as the per_engine_max_jobs.

Reproducing a job from a previous hunt run
------------------------------------------
# First hunt run
hunt -config -strategy ss1 -mode state_swarm -seed ABCD
hunt -run -strategy ss1 -task .
# Check results for each tag/job
hunt -report
# Decide to re-run job 5.8.L
hunt -run -strategy ss1 -rerun_job 5.8.L -task .
```

See Also

set_hunt_single_proof_thread

include

Reads and executes a script file of Tcl/Jasper Apps commands.

Syntax

```
include file_name
```

Returns

Same as Tcl's built-in source command.

Detailed Description

Use the "include" command to read and execute a script file "file_name" of Tcl and/or Jasper Apps commands. From the Tcl interpreter's point of view, this command behaves exactly like the "source" command. The difference between the two commands is that the "include" command echoes executed commands and their results to the log file and to the console, which makes it easier to debug problems in script files.

NOTE: For additional error information, check the content of the $errorInfo variable by running the following command:
set $errorInfo

Example

```
include script.tcl
```

See Also

restore,
save

incremental_elaborate_wrapper_mode

Sets the "elaborate" command incremental wrapper flow mode.

Syntax

```
incremental_elaborate_wrapper_mode ( -enable dut_name
                                   |-disable)
```

Returns

No return values

Detailed Description

Use Incremental Wrapper Elaborate Mode if you adopted the "Wrapper Mode Formal Environment." This kind of environment consists of one wrapper module with only the DUT and the testbench instantiated. The incremental wrapper elaborate makes it possible to update the testbench part without re-elaborating the DUT.

-enable dut_name
    Enable the use of "elaborate -reload_tb" to re-elaborate the testbench.

    Use "-dut" to specify the DUT's top module. "elaborate -reload_tb" re-elaborates only the testbenches.

    NOTE:
    . Use this switch to specify the DUT's top module. The tool does not re-elaborate the specified module when you run "elaborate -reload_tb". Changes to the DUT are not handled by this flow.
    . The tool automatically analyzes all your files when you run "elaborate -reload_tb".

    WARNING: You must run this command before "analyze" to successfully run incremental wrapper mode.

-disable

```
    Disable the use incremental elaborate wrapper mode flow.

LIMITATIONS: The following features are not supported by the incremental
wrapper mode flow.
. Coverage flow
. Low power flow
. Hierarchical reference in PSL designs
. Unsupported commands:
    . analyze -save
    . analyze -restore
    . analyze -req
    . compilation -config
    . connect -repository
    . restore -elaborated_design
    . save -elaborated_design
    . set_library_cell_optimization

The following apps automatically disable "incremental_elaborate_wrapper_mode":
. SEC (Excluding the hierarchical flow)
. SPV
. XPROP
. FSV
```

Example

```
incremental_elaborate_wrapper_mode -enable top
analyze -sv dut.v
elaborate
elaborate -reload_tb
```

See Also

elaborate

info_query

Returns information from the connected Jasper Information System server.

Syntax

```
info_query
   [(-job [job_id_tcl_list]
             |-property [property_name_tcl_list] [-regexp]
             |-message [message_id_tcl_list] [-regexp]
             |-command [command_tcl_list] [-regexp]
             |-license [license_type_tcl_list] [-regexp]
             |-job_diff -job_1 job1_id -job_2 job2_id -diff_option (commands |
              properties | messages) -diff_view (all | different | empty |
              identical))]
   [(-order_by field_tcl_list |-limit max_records_number)]
   [-custom_query custom_query] [-tags tag_tcl_list] [-fields field_tcl_list]
   [-last_days days_number] [-no_enclosing] [-silent]
```

Returns

A list of lists

Detailed Description

Use this command to get information about verification activity from the connected Jasper Information System server. To retrieve data from the server using this command, both environment variables below must be configured and pointing to an available Jasper Information System server with a valid DAAPI access token:

- JASPER_UXDB_PATH
- JASPER_UXDB_TOKEN

Refer to the DAAPI documentation in the "Jasper Expert System User Guide" for more information (Help - Application Guides - Expert System User Guide).

NOTE: The switches "-job", "-property", "-message", "-command", "-license", and "-job_diff" are optional. The tool accepts only one of these switches in

```
a single command. If you do not specify a switch, this command retrieves data
by job.

NOTE: The switches "-order_by" and "-limit" are mutually exclusive.

-job job_id_tcl_list
    Use "-job" to get information on all jobs.
    Provide ID list to get information on the specified jobs.

-property property_name_tcl_list [-regexp]
    Use "-property" to get information on all properties.
    . Provide property name list to get information on the specified properties.
    . Use "-regexp" to get information on properties that satisfy the specified
      regular expressions.

-message message_id_tcl_list [-regexp]
    Use "-message" to get information on all messages.
    . Provide message ID list to get information on the specified messages.
    . Use "-regexp" to get information on messages that satisfy the specified
      regular expressions.

-command command_tcl_list [-regexp]
    Use "-command" to get information on all commands.
    . Provide command list to get information on the specified commands.
    . Use "-regexp" to get information on commands that satisfy the specified
      regular expressions.

-license license_type_tcl_list [-regexp]
    Use "-license" to get information on all licenses.
    . Provide license type list to get information on the specified licenses.
    . Use "-regexp" to get information on licenses that satisfy the specified
      regular expressions.

-job_diff (-job_1 job1_id
           -job_2 job2_id
           -diff_option (commands | properties | messages)
           -diff_view (all | different | empty | identical))
    Use "-job_diff" to get the diff between two jobs. You must provide
    "-job_1", "-job_2", "-diff_option", and "-diff_view".
    . Use "-job_1" to specify the ID of the first job in the diff.
    . Use "-job_2" to specify the ID of the second job in the diff.
    . Use "-diff_option" to specify the diff option, which can be
      "commands", "properties", or "messages".
```

```
    . Use "-diff_view" to specify the diff view, which can be "all",
      "different", "empty", or "identical".

-order_by field_tcl_list
    Use "-order_by" to get information ordered as specified in the field list.

-limit max_records_number
    Use "-limit" to limit records returned to a maximum number of rows.
    The tool ignores the "-limit" if the "max" argument is equal to zero (0).

-custom_query custom_query
    Use "-custom_query" to get data that satisfies the query.
    The DAAPI query syntax can be found in the "Jasper Expert System User
    Guide" (Help - Application Guides - Expert System User Guide).

-tags tag_tcl_list
    Use "-tags" to get information on all records with the specified tags.

-fields field_tcl_list
    Use "-fields" to get information for the specified fields. You can find the
    fields list in the "Jasper Expert System User Guide"
    (Help - Application Guides - Expert System User Guide).

-last_days days_number
    Use "-last_days" to get records for the specified number of days.
    The tool ignores "-last_days" if the "days" argument is equal to zero (0).

-no_enclosing
    Use "-no_enclosing" to get information without double quotes surrounding
    each field.

-silent
    Use "-silent" to turn off the output to the screen and return information
    with a Tcl return value, which is helpful in Tcl scripts.
```

Example

```
info_query -fields {start_time tags user job_status} -tags {reg72 CEX_DEBUG}

info_query -job 19 -fields {start_time tool_long_version job_status}

info_query -property bus.read_valid \
             -fields {status engine min_length time} \
             -last_days 7 \
             -order_by {min_length time}

info_query -command -last_days 15

info_query -message EES024

info_query -license -last_days 1 -fields {job_id start_day user}

info_query -property {^bus.read.* prop.*} -regexp -last_days 10

info_query -job_diff -job_1 3 -job_2 4 -diff_option properties -diff_view identical
```

ipk

Analyzes an IPK and extracts or loads proof kit files.

Syntax

```
ipk -jdb jdb
         [ -analyze [-analyze_opts analyze_switches_tcl_list]
          |-extract_pk_files
            (jaspergold | sim) [-dir output_dir]
          |-extract_wrapper [-dir  output_dir]
          |-load_wrapper wrapper_file_tcl_list
            [-analyze_opts analyze_switches_tcl_list]
         ]
ipk -list [-silent]
```

Returns

No return values

Detailed Description

```
Use the "ipk" command to analyze an Intelligent Proof Kit (IPK) for use with
formal verification, extract proof kit files for use with simulation, or load
wrapper files.

-jdb jdb [ -analyze [-analyze_opts analyze_switches_tcl_list]
          |-extract_pk_files
            (jaspergold | sim) [-dir output_dir]
          |-extract_wrapper [-dir output_dir]
          |-load_wrapper wrapper_file_tcl_list
            [-analyze_opts analyze_switches_tcl_list]
         ]
    Load the specified IPK database.
    . Use "-jdb" to specify the IPK you want to load.
      NOTE:
      . If you do not supply additional switches, the tool analyzes the
        specified IPK.
      . If you do not provide the path, then Jasper Apps loads the
        built-in IPK. Otherwise, it loads the IPK at the specified path,
        which can come from an earlier or later Jasper Apps release.
        You can find the .jdb files in the IPK subdirectory of the Jasper
        Apps installation.
    . Use "-analyze" to analyze the IPK for use with formal verification. Use
      "-analyze_opts" to specify which switches you want to set in the analyze
      step.
    . Use "-extract_pk_files" to extract files suitable for verification with
      simulation and use "-dir" to specify the output directory for extracted
      files. The default directory is pwd. The supported arguments are:
      . jaspergold: Jasper Apps
      . sim       : Simulation encrypted with IEEE P1735
    . Use "-extract_wrapper" to extract a default wrapper for the IPK and use
      "-dir" to specify the output directory for extracted files. The default
      directory is pwd.
    . Use "-load_wrapper" to analyze a proof kit using one or more specified
      wrappers. Use "-analyze_opts" to specify which switches you want to set
      in the analyze step.

-list [-silent]
    Show the list of IPK files installed in Jasper

    Use "-silent" to return the list as Tcl return values without
    printing the list on the screen.
```

Example

```
# Load DFI 3.0 proof kit in Jasper using a wrapper:
ipk -jdb jasper_dfi3.jdb -load_wrapper jasper_dfi3_wrapper_1.sv
ipk -jdb jasper_dfi3.jdb.gz -load_wrapper jasper_dfi3_wrapper_1.sv
```

See Also

analyze,
elaborate

ipxact

Prints the requested IP-XACT information or executes the requested action.

Associated App(s)

Connectivity Verification, CSR Verification

Syntax

```
Action Switches
---------------
ipxact -load xml_file_name.xml+
       [-lib_dir ipxact_directory]*
       [-rtl_dir rtl_directory]*
       [-ignore_missing_definition]
       [-ignore_missing_portmap]
       [-recursive]
       [-setup (design | abvip | csr | all)+
         [-view ipxact_view_name]
         [-top ipxact_top_name]
         [-analyze_opts analyze_options]
         [-elaborate_opts elaborate_options]
         [-csr_opts csr_options]]

ipxact -check_rtl_consistency [-strict]
                                      [-depth depth_value]

ipxact -clear

List Switches
-------------
ipxact -list
             ( file | design | component | interface
              |-design spirit:design_name
                 (  instance | component | interconnection | hier_connection
                  | ad_hoc_connection)
              |-component spirit:component_name
                 (port | interface | design | memory_map)
```

```
              |-component spirit:component_name
                  -interface spirit:busInterface_name
                  (port_map | interface_type)
              |-interface spirit:busType_type signal
             )

Interface Connectivity Map
--------------------------
ipxact -generate_connectivity_map csv_file_name.csv
             [spirit:design_name
                [-instance instance_name]
                [-module module_name]
             ]
             [-interfaces]
             [-recursive]
             [-infer_interface_definitions]
             [-no_debug]

CSR Map
-------
ipxact -generate_csr_map csv_file_name.csv
       [-component spirit:component_name]
       [-maps maps]
       [-no_debug]
       [-volatile (read_undefined | write_undefined | ignore)]
```

Returns

A list of requested objects or the output of a requested action.

Detailed Description

Use this command to get information from an IP-XACT specification or to request actions related to this specification.

```
Action Switches
---------------
-load xml_file_name.xml+
    [-lib_dir ipxact_directory]*
    [-rtl_dir rtl_directory]*
    [-ignore_missing_definition]
```

```
[-ignore_missing_portmap]
[-recursive]
[-setup (design | abvip | csr | all)+
   [-view ipxact_view_name]
   [-top ipxact_top_name]
   [-analyze_opts analyze_options]
   [-elaborate_opts elaborate_options]
   [-csr_opts csr_options]]
Load the IP-XACT specification contained in xml_file_name.xml files and the
referenced IP-XACT elements.

. Use "-lib_dir" to search the specified directory for IP-XACT files for
  information referenced by the IP-XACT files loaded.
. Use "-rtl_dir" to search the specified directory for files specified by
  <spirit:component> <spirit:fileSets>.
. Use "-ignore_missing_definition" to ignore references to <spirit:busDefinition>
  and/or <spirit:abstractionDefinition> elements. This is recommended when using
  IP-XACT files in CSR App.
. The tool issues an error if any missing <spirit:portMap> is defined
  as required in the related <spirit:abstractionDefinition>. Use
  "-ignore_missing_portmap" to suppress the error.
  NOTE: This switch is recommended when using IP-XACT files for the CSR App.
  CAUTION: If you use this switch, Jasper might not be able to correctly
  extract all of the missing portMap information.
. Use "-recursive" to enable recursive search in the specified directories.
. Use "-setup" to automate design setup operations using data extracted
  from IP-XACT files.
  . Use "design" to compile the design (analyze and elaborate RTL files)
    based on the data extracted from the <spirit:fileSets> element.
  . Use "abvip" generate a bind file that connects AMBA protocols to their
    corresponding ABVIP and run "elaborate". The ABVIP is selected based on
    the <spirit:busInterfaces> element. The generated bind file is
    included in compilation.
  . Use "csr" to run elaborate and activate the correct ABVIP instance as a
    CSR checker. The instance is selected based on the <spirit:memoryMapRef>
    element.
  . Use "all" to compile the design, generate the ABVIP bind file, and
    activate the correct ABVIP instance as a CSR checker. The instance is
    selected based on the <spirit:memoryMapRef> element.
  . Use "-view" to pass the specified IP-XACT view name to use on
    compilation. If you do not specify "-view", the tool chooses the view
    with name "RTL". If the "RTL" view is absent, the first view without
    the element <spirit:hierarchyRef> is used.
```

```
      . Use "-top" to pass the specified IP-XACT component name related to
        design's top. It must be used if the argument of "-load" is a list of
        files. If "-top" is not specified, the top component is the single
        file passed on "-load".
      . Use "-analyze_opts" to pass specific arguments to the "analyze"
        command.
      . Use "-elaborate_opts" to pass specific arguments to the "elaborate"
        command.
      . Use "-csr_opts" to pass specific arguments to the "check_csr" command
        when setup mode used is "csr".

        NOTE: "abvip" and "csr" modes require ABVIP to be configured with
        "abvip -set_location". Supported protocols are AHB Lite, APB, and AXI.

-check_rtl_consistency [-strict]
                       [-depth depth_value]
    Compares the structure of the elaborated design against the structure of
    the loaded IP-XACT document and reports inconsistencies.

    This command prints a report highlighting the following inconsistencies:
    . Missing structures: structures specified in the IP-XACT but not
      implemented in the design.
    . Different structures: structures with the same name but different
      attributes. Applies to ports and parameters only.

    For each instance of the tree, which is formed by the instances present in both
    the design and IP-XACT, the tool compares the following structures:
    . Instances below (instance and module names)
    . Ports (name, direction, and width)
    . Parameters (name and value)

    . Use "-strict" to also report structures that are implemented in the design
      but not specified in the IP-XACT (additional structures).
    . Use "-depth" to stop the algorithm at the specified instance depth where
      "1" is the depth of the top instance. Specify "0" to run the command
      without performing any check.

-clear
    Clear all previously loaded IP-XACT elements from memory.

List Switches
-------------
-list (file | design | component | interface)
```

```
    List the information that matches the specified criteria.

    NOTE: The following switches are valid for the global IP-XACT scope.

    . Use "file" to list the IP-XACT files that have been loaded.
    . Use "design" to list the <spirit:design> elements.
    . Use "component" to list <spirit:component> elements.
    . Use "interface" to list all <spirit:busInterface> element names.

-design spirit:design_name
    -list (  instance | component | interconnection | hier_connection
           | ad_hoc_connection)
    List the information that matches the specified criteria.

    NOTE: The following switches are valid for the spirit:design_name scope.

    . Use "instance" to list spirit:component_name instances.
    . Use "component" to list components instantiated.
    . Use "interconnection" to list <spirit:interconnection> elements.
      The result is a list of pairs. The first pair is formed by an instance
      name and the name of an interface implemented by that instance. The
      second element of the pair might also be a description of instance and
      interface, or a list of such descriptions. In the first case, the two
      interfaces are connected, while in the second case, the interfaces from
      the list monitor the interface described by the first element of the
      pair.
    . Use "hier_connection" to list <spirit:hierConnection> elements.
      The result is a list of tuples. The first element of the tuple is the
      name of a spirit:busInterface_name implemented by the component
      corresponding to spirit:design_name. The second and third elements of the
      tuple define an instance and a spirit:busInterface_name. The interface
      and the instance's bus interface form a connection.
    . Use "ad_hoc_connection" to list <spirit:adHocConnection> elements.
      The result is a list of pairs. Each pair describes two ports. If a port
      is described by a single element, then that element is the port name and
      the port belongs to the component associated with spirit:design_name. If
      the description has two elements, then the first element is the name of
      the instance that contains the port and the second element is the port
      name.

-component spirit:component_name
    -list (port | interface | design | memory_map)
    List the information that matches the specified criteria.
```

```
    NOTE: The following switches are valid for the spirit:component_name scope.

    . Use "port" to list ports present in the component.
    . Use "interface" to list interfaces implemented by the specified
      spirit:component_name.
    . Use "design" to get the <spirit:design> element associated with the
      specified spirit:component_name.
    . Use "memory_map" to list the address maps in the component.

-component spirit:component_name -interface spirit:busInterface_name
    -list (port_map | interface_type)
    List the information that matches the specified criteria.

    . Use "port_map" to list the maps between logical and physical ports in the
      spirit:busInterface_name implementation of the spirit:component_name. The
      result is a list of pairs with the logical and physical names.
    . Use "interface_type" to list the spirit:busType_type and the
      spirit:abstractionType_type of the spirit:busInterface_name
      implementation of spirit:component_name.

-interface spirit:busType_type
    -list signal
    List the signals of the <spirit:abstractionDefinition> element for
    spirit:busType_type.

Interface Connectivity Map
--------------------------
-generate_connectivity_map csv_file_name.csv
    [spirit:design_name [-instance instance_name] [-module module_name]]
    [-interfaces] [-recursive] [-infer_interface_definitions] [-no_debug]
    Generate a Connectivity map based on connections found in
    spirit:design_name.

    The resulting map has interface definitions and interface connections.

    . Use "-instance" when spirit:design_name is implemented by instance_name.
      Otherwise, the tool assumes that the top module implements
      spirit:design_name.
    . Use "-module" when spirit:design_name is implemented by module_name.
      Otherwise, if the instance name is provided, the tool searches for the
      module name in the current elaboration, or assumes that
      spirit:design_name is the module name if the instance is not specified.
```

```
    . Use "-interfaces" to generate a map without connections. The map
      contains only interface definitions and port maps.
    . Use "-recursive" to generate connectivity information for all IP-XACT
      elements found by recursively exploring the instance hierarchy rooted in
      spirit:design_name. By default, the command generates connectivity
      information only for spirit:design_name.
    . Use "-infer_interface_definitions" to generate INTERFACE_MAP definitions
      for missing <spirit:abstractionDefinition> elements.
    . Use "-no_debug" to generate the map without any reference to the IP-XACT
      file names and line numbers.

    NOTE: You must elaborate the design before using this command.

CSR Map
-------
-generate_csr_map csv_file_name.csv
    -component spirit:component_name [-maps maps] [-no_debug]
    -volatile (read_undefined | write_undefined | ignore)
    Generate a CSV file for "check_csr -load" based on the description of
    spirit:component_name.
    . Use "-component" to specify the spirit:component_name. This switch is not
      required if only one spirit:component is loaded.
    . Use "-maps" to limit the CSV file content to the specified maps.
    . Use "-no_debug" to generate the map without any reference to the IP-XACT
      file names and line numbers.
    . Use "-volatile" to configure how the tool handles the IP-XACT specific
      volatile element.
      . Use "write_undefined" to generate the attribute BUS_WRITE_CHECK =
        write_undefined.
      . Use "read_undefined" to generate the attribute BUS_READ_CHECK =
        read_undefined.
      . Use "ignore" to treat the container as if the volatile attribute was not
        set.
      NOTE:
      . You can find more information about the attributes in the "Jasper
        Control and Status Register Verification App User Guide" (Help -
        Application Guides - CSR Verification App User Guide).
      . According to Section 6 of the IEEE 1685(TM), "volatile (optional) when
        true indicates in the case of a write followed by read, or in the case
        of two consecutive reads, there is no guarantee as to what is returned
        by the read on the second transaction or that this return value is
        consistent with the write or read of the first transaction."
```

NOTE: You must elaborate the design before using this command.

Example

```
ipxact -load main_ixpact_design.xml -lib_dir components -lib_dir interfaces
ipxact -load main_ixpact_design.xml -rlt_dir design_rtl -recursive
ipxact -list component
ipxact -list port_map -component bridge -interface clock
ipxact -list signals -interface ingress
ipxact -list port -component bridge
ipxact -list instance -design bridge

# Interface Connectivity Map Generator
# ------------------------
# The following commands create a Connectivity map with the connections
# generated from an IP-XACT specification.
ipxact -load bridge.xml -lib_dir ../ipxact_library -rtl_dir .
ipxact -generate_connectivity_map interface_connections.csv bridge -instance bridge0

# CSR Map Generator
# ----------------
# The following commands create a CSR map from the description of two
# memory maps of an IP-XACT component.
ipxact -load data_banks.xml -lib_dir ../ipxact_library -rtl_dir .
ipxact -generate_csr_map csr_map.csv -component data_bank -maps "data0 data1"

# IP-XACT automation
# ----------------
# The following commands uses IP-XACT data to compile the design, generate ABVIP
# bind, and activate the correct ABVIP instance as a CSR checker using the switch
# "-partitioned_checks", copying assumes and related covers from embedded task
# into CSR task.
ipxact -load main_component.xml -lib_dir ./lib -setup csr -csr_opts "-
partitioned_checks"
```

See Also

check_conn,
check_csr

jasper_history

Queries information about Jasper commands' run history.

Syntax

```
jasper_history -command jg_command_name
               [-switch switch_name]
               [-no_switch excluded_switch_name]
               [  -message_id id
                | -message_type (info | warning | error)]
               [-search_mode (lazy | last_only)]
               [-result_dict variable_name]

jasper_history -clear
```

Returns

If "-command" is specified returns "1" if there is a match and "0" otherwise.
If "-clear" is specified nothing is returned.

Detailed Description

Use this command to query information about Jasper commands that have been run. Unlike the standard "history" Tcl command, this command does not show the history of commands run by Jasper but returns whether a command matching certain conditions has been run.

-command jg_command_name
    [-switch switch_name]
    [-no_switch switch_name]
    [-message_id id | -message_type (info | warning | error)]
    [-search_mode (lazy | last_only)]
    [-result_dict variable_name]

    Use this command to query if a specified command matching the specified conditions has been run in this analysis session. The tool searches from the most recent command to the oldest and returns as soon as it finds a

```
    match.

    Available conditions are:
    . -switch:       Matches runs in which the specified switch was present
    . -no_switch:    Matches runs in which the specified switch was not present
    . -message_id:   Matches runs that issued the message with the specified ID.
    . -message_type: Matches runs that issued a message of the type specified
    NOTE: You can specify the "switch_name" arguments with or without a leading
    "-". A leading "-" is automatically assumed, if not specified, unless the
    name starts with a "+".

    . Use "-search_mode" to specify one of the following values to control
      how the search is performed. If you do not specify "-search_mode", the
      default is "lazy":
      . lazy:      Starting from the most recently issued command look for the
                   first command that matches the specified conditions.
      . last_only: Check only the most recent occurrence of the specified
                   command for the specified conditions.

    . Use "-result_dict" to store relevant data about the query as a Tcl
      dictionary in the variable specified by "variable_name".
      . Entries in the Tcl dictionary created when you use the switch
        "-result_dict":
        . command_line: The complete line of the command that matched.
                        This key is always present when there is a match.
        . message_text: The text of the message matched if you used
                        "-message_id" or "-message_type".
        NOTE: The tool automatically erases old messages when it reaches the
        message history limit. Use "set_message_history_limit" to control this
        limit.

    NOTE:
    . The "jasper_history_per_command_limit" configuration controls the number
      of issued commands the tool stores. The purpose of this limit is to avoid
      excessive memory consumption when you have run a large number of
      commands.
    . The tool stores commands in history with their original name even if you
      rename them.

-clear
    Clears all the stored command runnings.
```

Example

```
# Simple example:
jasper_history -command analyze
1

# Example showing how to get the command line that matched:
jasper_history -command analyze -switch sv -result_dict result
1
dict get $result command_line
analyze -sv top.sv

# Example with no match:
jasper_history -command analyze -no_switch vhdl
0

# Example where both "command_line" and "message_text" are present. The
# search queries the last "clock" command only, even if a previous one
  triggered WCK001:
jasper_history -command clock -message_id WCK001 -result_dict result -search_mode
last_only
1
dict get $result command_line
clock clk
dict get $result message_text
WARNING (WCK001): Clock "clk" is already declared as the fastest clock.

# Specifying a switch that starts with "+" instead of "-"
analyze -sv top.sv +libnonamehide
jasper_history -command analyze -switch +libnonamehide
1
```

See Also

set_jasper_history_per_command_limit,
set_message_history_limit

jasper_model_divider

Manages the jasper_model_divider PA connection and configuration.

Syntax

```
jasper_model_divider
        [-abstract signal_name [-parameter param_name param_value]*
        [-connect port_name signal_name]*
        [-auto]
        ]
        [-init [-preload operand_1 operand_2]* signal_name]
        [-justify [-precise] signal_name]
        [-suggest]
```

Returns

When you use "-suggest", the return value is a list of divider instances that fall into one of the following two categories:
. Instances that have been abstracted by jasper_model_divider
. Instances that are returning abstract values in the current trace (hence, these instances might benefit from the application of "jasper_model_divider -justify").

Default

No default

Detailed Description

Use the jasper_model_divider command to instantiate and configure the jasper_model_divider PA. This PA provides a means of achieving full, unbounded proofs on properties that involve logic containing dividers (properties that are otherwise generally intractable for formal tools).

To learn more about the jasper_model_divider PA, see the following resources:
. jasper_model_divider data sheet: To see the data sheet from the Help menu,

```
  choose Proof Accelerator Datasheets - jasper_model_divider.

  CAUTION: Automatically extracted Proof Accelerator tasks can contain implicit
  assumptions that are not visible to the user, and constraining tasks with
  assumptions that are not visible makes it impossible to evaluate their
  impact.

  If you are using Verification Component Proof Accelerators (for
  example, jasper_scoreboard_2) or Abstraction Component Proof Accelerators
  (for example, jasper_model_fifo), do not attempt to link a Proof Accelerator
  task to a non-Proof Accelerator task. Instead, you should do the following:
  task -link my_task -to <pa_task>

-abstract signal_name
    Replace the specified divider signal with an instance of the
    jasper_model_divider PA.

    Run this command after elaboration and before specifying the clock
    and reset conditions.

-parameter param_name param_value
    Use the parameters specified by the "param_name" and "param_value"
    pairs.

    Important things to know about this command:
    . Use this switch with the "-abstract" switch.
    . If you do not supply parameters, the tool
      uses the default parameter values. (Refer to the
      jasper_model_divider datasheet.)

-connect port_name signal_name
    Connect the port (port_name) on the boundary of the
    jasper_model_divider PA to a specified signal (signal_name) in
    the design.

    Specify "signal_name" in one of two ways:
    . As a port on the boundary of the module containing the divider
      being replaced.
    . As an absolute path to a signal anywhere in the verification
      environment.

    Use this switch with the "-abstract" switch.
```

```
-auto
    Automatically connect the input port of the jasper_model_divider
    PA to the design signal with the same name.

    Important things to know about this command:
    . Use this switch with the "-abstract" switch.
    . If you also use the "-connect" switch, the explicit connections
      specified with that switch take precedence over the name-based
      matching specified by this "-auto" switch.
    . The tool does not autoconnect signals of different widths.

    NOTE: For proper functioning, take special care with the "clk"
    signal. Use one of the following three options:

           1) Ensure that the module containing the divider that is
              being abstracted contains a port named "clk", and allow
              the connection of the jasper_model_divider "clk" signal
              to be handled with the "-auto" switch.

           2) Ensure that the module containing the divider that is
              being abstracted contains a clock port, and then use
              an explicit "-connect" switch to make the connection.
              If the clock port on the module containing the divider
              is called "clock", then the syntax is:

                  -connect clk clock

           3) Use an explicit "clock" command to define the clock
              behavior for the jasper_model_divider. If the path to
              the divider being abstracted is "hash_i.quotient", then
              the clock command is:

                   clock <clock> hash_i.jasper_divider_i_quotient.clk

              In this command, <clock> is a clock previously declared with
              the Jasper Apps "clock" command.

-init signal_name
    Set "signal_name" to the maximum level of abstraction.

    Run this command after elaboration and before specifying the clock
    and reset conditions.
```

-preload operand_1 operand_2
    Preload the abstraction for the specified signal with the results of a specific division.

-justify
    Decrease the abstraction level of the specified instance by one degree.

-precise
    As a part of the "justify" operation, preload the abstraction for the specified signal with the first abstract division encountered in the trace.

-suggest
    Analyze the current trace to determine whether justifying the specified divider will provide any benefit.

jasper_model_mpram

Manages jasper_model_mpram Proof Accelerator connection and configuration.

Syntax

```
jasper_model_mpram
[-abstract instance_name [-parameter param_name param_value]*
                         [-connect port_name signal_name]*
                         [-auto]
]
[-init [-preload addr [data]]* instance_name]
[-justify instance_name]
[-suggest]
[-clear]
```

Returns

When you use "-suggest", the return value is a list of memory instances
that fall into one of the following two categories:
. Instances that have been abstracted by jasper_model_mpram
. Instances that are returning abstract values in the current trace
  (hence, these instances might benefit from the application of
 "jasper_model_mpram -justify").

Default

No default

Detailed Description

Use the "jasper_model_mpram" command to instantiate and configure the
jasper_model_mpram PA. The jasper_model_mpram PA is an encrypted macro that,
along with a supporting Tcl command, provides a means of achieving full,
unbounded proofs on properties that involve logic containing memories
(properties that are otherwise generally intractable for formal tools).

The jasper_model_mpram PA achieves this goal by providing a flexible abstraction that enables the formal engines to abstract the majority of the specified memory while preserving those parts of its behavior that are needed to achieve the proof.

To learn more about the jasper_model_mpram PA, access the following resources:
. jasper_model_mpram data sheet. To see the data sheet from the Help menu, choose Proof Accelerator Datasheets - jasper_model_mpram.

  CAUTION: Automatically extracted Proof Accelerator tasks can contain implicit assumptions that are not visible to the user, and constraining tasks with assumptions that are not visible makes it impossible to evaluate their impact.

  If you are using Verification Component Proof Accelerators (for example, jasper_scoreboard_2) or Abstraction Component Proof Accelerators (for example, jasper_model_fifo), do not attempt to link a Proof Accelerator task to a non-Proof Accelerator task. Instead, you should do the following: task -link my_task -to <pa_task>

-abstract instance_name
    Replace the specified memory instance and all other like instances with an instance of the jasper_model_mpram PA.

    Important things to know:
    . To replace a single memory instance, use one of the following workarounds:
      . Create a requirements file that manually instantiates jasper_model_mpram, then bind it to the specific instance with "connect -bind".
      . Elaborate the given memory instance with a unique parameter, then use the jasper_model_mpram command on the specific instance.
    . Run this command after elaboration and before specifying the clock and reset conditions.

-parameter param_name param_value
    Use the parameters specified by the param_name and param_value pairs.

    Important things to know about this command:
    . Use this switch with the "-abstract" switch.
    . If you do not supply parameters, Jasper Apps uses the default parameter values. (Refer to the jasper_model_mpram datasheet.)

-connect port_name signal_name

```
    Connect the port (port_name) on the boundary of the jasper_model_mpram PA
    to a specified signal (signal_name) in the design.

    Specify signal_name in one of two ways:
    . As a port on the boundary of the memory instance being replaced
    . As an absolute path to a signal anywhere in the verification environment

    Use this switch with the "-abstract" switch.

-auto
    Automatically connect the input port of the jasper_model_mpram PA to the
    design signal with the same name.

    Important things to know about this command:
    . Use this switch with the "-abstract" switch.
    . If you also use the "-connect" switch, the explicit connections specified
      with that switch take precedence over the name-based matching specified
      by this "-auto" switch.
    . Jasper Apps does not autoconnect signals of different widths.

    NOTE: For proper functioning, take special care with the "clk" signal. Use
    one of the following three options:
    . Ensure that the RAM that is being abstracted contains a port named "clk",
      and allow the connection of the jasper_model_mpram "clk" signal to be
      handled with the "-auto" switch.
    . Ensure that the RAM that is being abstracted contains a clock port, and
      then use an explicit "-connect" switch to make the connection. If the
      clock port on the RAM is called "clock", then the syntax is as follows:

      -connect clk clock

    . Use an explicit "clock" command to define the clock behavior for the
      jasper_model_mpram. If the path to the RAM being abstracted is EXT_MEM,
      then the "clock" command is as follows:

      clock <clock> EXT_MEM.jasper_mpram_i.clk

      In this command, <clock> is a clock previously declared with the
      Jasper Apps "clock" command.

-init instance_name
    Set instance_name to the maximum level of abstraction.
```

```
-preload addr [data]
    Treat the contents of the specified address (addr) as non-abstracted data
    and the data value (data) as the reset value for the specified
    memory location.

    Use this switch with the "-init" switch.

-justify
    Decrease the abstraction level of the specified instance by one memory
    location.

    NOTE: This Proof Accelerator models the RAM being abstracted by selecting
    certain memory locations and treating them concretely. Generally, this is
    done on a first-come, first-served basis; however, in the event of multiple
    simultaneous accesses, ports are serviced in the following order:
    . Write ports before read ports
    . Lower numbered ports before higher numbered ports

-suggest
    Analyze the current trace to determine whether justifying any of the
    instances of the jasper_model_mpram will provide any benefit.

-clear
    Clear the jasper_model_mpram data structures.

    NOTE: The command "clear -all" does not clear Proof Accelerator data
    structures.
```

Example

```
// Abstract the EXT_MEM:
jasper_model_mpram -abstract EXT_MEM -auto                              \
                   -connect   write           {MEM_Req && !MEM_RnW}     \
                   -connect   read            {MEM_Req && MEM_RnW}     \
                   -connect   addr_rd         {MEM_Addr}                \
                   -connect   addr_wr         {MEM_Addr}                \
                   -connect   data_rd         {MEM_Read_data}           \
                   -connect   data_wr         {MEM_Write_data}          \
                   -connect   read_allocate   {(MEM_Addr == test_addr)} \
                   -connect   write_allocate  {(MEM_Addr == test_addr)} \
                   -connect   concrete        {concrete}                \
                   -parameter ADDR_WIDTH 32                             \
                   -parameter DATA_WIDTH 3

assume {~MEM_Req |-> ~MEM_Ready)


// Constrain the MEM_Ready signal such that it cannot be asserted unless the
// MEM_Req signal is also asserted:

assume {~MEM_Req |-> ~MEM_Ready)


// Initialize the EXT_MEM abstraction:

jasper_model_mpram -init EXT_MEM


// Set the depth of the EXT_MEM abstraction (in this case, only one location is
// required):

jasper_model_mpram -justify EXT_MEM
```

jasper_model_multiplier

Manages the jasper_model_multiplier PA connection and configuration.

Syntax

```
jasper_model_multiplier
        [-abstract signal_name [-parameter param_name param_value]*
        [-connect port_name signal_name]*
        [-auto]
        ]
        [-init [-preload operand_1 operand_2]* signal_name]
        [-justify [-precise] signal_name]
        [-suggest]
```

Returns

When you use "-suggest", the return value is a list of multiplier instances that fall into one of the following two categories:
. Instances that have been abstracted by jasper_model_multiplier
. Instances that are returning abstract values in the current trace (hence, these instances might benefit from the application of "jasper_model_multiplier -justify").

Default

No default

Detailed Description

Use the jasper_model_multiplier command to instantiate and configure the jasper_model_multiplier PA. This PA provides a means of achieving full, unbounded proofs on properties that involve logic containing multipliers (properties that are otherwise generally intractable for formal tools).

To learn more about the jasper_model_multiplier PA, see the following resources:
. jasper_model_multiplier data sheet: To see the data sheet from the Help menu,

```
  choose Proof Accelerator Datasheets - jasper_model_multiplier.

  CAUTION: Automatically extracted Proof Accelerator tasks can contain implicit
  assumptions that are not visible to the user, and constraining tasks with
  assumptions that are not visible makes it impossible to evaluate their
  impact.

  If you are using Verification Component Proof Accelerators (for
  example, jasper_scoreboard_2) or Abstraction Component Proof Accelerators
  (for example, jasper_model_fifo), do not attempt to link a Proof Accelerator
  task to a non-Proof Accelerator task. Instead, you should do the following:
  task -link my_task -to <pa_task>

-abstract signal_name
    Replace the specified multiplier signal with an instance of the
    jasper_model_multiplier PA.

    Run this command after elaboration and before specifying the clock
    and reset conditions.

-parameter param_name param_value
    Use the parameters specified by the "param_name" and "param_value"
    pairs.

    Important things to know about this command:
    . Use this switch with the "-abstract" switch.
    . If you do not supply parameters, the tool
      uses the default parameter values. (Refer to the
      jasper_model_multiplier datasheet.)

-connect port_name signal_name
    Connect the port (port_name) on the boundary of the
    jasper_model_multiplier PA to a specified signal (signal_name) in
    the design.

    Specify "signal_name" in one of two ways:
    . As a port on the boundary of the module containing the multiplier
      being replaced.
    . As an absolute path to a signal anywhere in the verification
      environment.

    Use this switch with the "-abstract" switch.
```

```
-auto
    Automatically connect the input port of the jasper_model_multiplier
    PA to the design signal with the same name.

    Important things to know about this command:
    . Use this switch with the "-abstract" switch.
    . If you also use the "-connect" switch, the explicit connections
      specified with that switch take precedence over the name-based
      matching specified by this "-auto" switch.
    . The tool does not autoconnect signals of different widths.

    NOTE: For proper functioning, take special care with the "clk"
    signal. Use one of the following three options:

           1) Ensure that the module containing the multiplier that is
              being abstracted contains a port named "clk", and allow
              the connection of the jasper_model_multiplier "clk" signal
              to be handled with the "-auto" switch.

           2) Ensure that the module containing the multiplier that is
              being abstracted contains a clock port, and  then use
              an explicit "-connect" switch to make the connection.
              If the clock port on the module containing the multiplier
              is called "clock", then the syntax is:

                  -connect clk clock

           3) Use an explicit "clock" command to define the clock
              behavior for the jasper_model_multiplier. If the path to
              the multiplier being abstracted is "hash_i.product", then
              the clock command is:

                   clock <clock> hash_i.jasper_multiplier_i_product.clk

              In this command, <clock> is a clock previously declared with
              the Jasper Apps "clock" command.

-init signal_name
    Set "signal_name" to the maximum level of abstraction.

    Run this command after elaboration and before specifying the clock
    and reset conditions.
```

```
-preload operand_1 operand_2
    Treat the product of the specified operands as non-abstracted
    data.

    Use this switch with the "-init" switch. Refer to the example section
    below.

-justify
    Decrease the abstraction level of the specified instance by one multiply
    operation.

-precise
    Specify that first abstracted operation in the current trace should
    be preloaded into the abstraction as a part of the "justify" operation.

-suggest
    Analyze the current trace to determine whether justifying any of the
    instances of the jasper_model_multiplier will provide any benefit.
```

Example

```
// Abstract the multiplier in a hash function:
jasper_model_multiplier -abstract hash_i.product           \
                        -connect a   hash_i.address          \
                        -connect b   hash_i.m                \
                        -connect y   hash_i.product          \
                        -auto                                \
                        -parameter A_WIDTH $cache_addr_width \
                        -parameter B_WIDTH 6                 \
                        -parameter Y_WIDTH [expr $cache_addr_width + 6]

// Initialize the abstraction:
jasper_model_multiplier -init hash_i.product

// Set the depth of the abstraction to one operation:
jasper_model_multiplier -justify hash_i.product


// Specify that the PA should start from reset knowing to track
// the result of "8'hff * 4'hB"

jasper_model_multiplier -init -preload 8'hFF 4'hB

// Specify the same product, but with decimal notation

jasper_model_multiplier -init -preload 8'd255 4'd11
```

jasper_model_ram

Manages the jasper_model_ram PA connection and configuration.

Syntax

```
jasper_model_ram
        [-abstract instance_name [-parameter param_name param_value]*
        [-connect port_name signal_name]*
        [-auto]
        ]
        [-init [-preload addr [data]]* instance_name]
        [-justify instance_name]
        [-suggest]
```

Returns

When you use "-suggest", the return value is a list of memory instances
that fall into one of the following two categories:
. Instances that have been abstracted by jasper_model_ram
. Instances that are returning abstract values in the current trace
  (hence, these instances might benefit from the application of
  "jasper_model_ram -justify").

Default

No default

Detailed Description

Use the jasper_model_ram command to instantiate and configure the
jasper_model_ram PA. This PA provides a means of achieving full,
unbounded proofs on properties that involve logic containing memories
(properties that are otherwise generally intractable for formal tools).

To learn more about the jasper_model_ram PA, see the following resources:
. jasper_model_ram data sheet: To see the data sheet from Help menu, choose

```
  Proof Accelerator Datasheets - jasper_model_ram.

CAUTION: Automatically extracted Proof Accelerator tasks can contain implicit
assumptions that are not visible to the user, and constraining tasks with
assumptions that are not visible makes it impossible to evaluate their impact.

If you are using Verification Component Proof Accelerators (for
example, jasper_scoreboard_2) or Abstraction Component Proof Accelerators (for
example, jasper_model_fifo), do not attempt to link a Proof Accelerator task to
a non-Proof Accelerator task. Instead, you should do the following:
task -link my_task -to <pa_task>

-abstract instance_name
    Replace the specified memory instance and all other like instances
    with an instance of the jasper_model_ram Proof Accelerator.

    Important things to know:
    . To replace a single memory instance, use one of the following
      workarounds:
      . Create a requirements file that manually instantiates
        jasper_model_ram, then bind it to the specific instance with
        "connect -bind".
      . Elaborate the given memory instance with a unique parameter, then
        use the "jasper_model_ram" command on the specific instance.
    . Run this command after elaboration and before specifying the clock
      and reset conditions.
    . When you use this command, the tool elaborates the design again and
      erases the effect of some commands, such as "reset".

-parameter param_name param_value
    Use the parameters specified by the "param_name" and "param_value"
    pairs.

    Important things to know about this command:
    . Use this switch with the "-abstract" switch.
    . If you do not supply parameters, the tool
      uses the default parameter values. (Refer to the
      jasper_model_ram datasheet.)

-connect port_name signal_name
    Connect the port (port_name) on the boundary of the
    jasper_model_ram PA to a specified signal (signal_name) in
    the design.
```

```
    Specify "signal_name" in one of two ways:
    . As a port on the boundary of the memory instance being replaced.
    . As an absolute path to a signal anywhere in the verification
      environment.

    Use this switch with the "-abstract" switch.

    NOTE: If you are connecting to a signal or port of an instance, you cannot
    specify "signal_name" signals that are in the top module.

-auto
    Automatically connect the input port of the jasper_model_ram
    PA to the design signal with the same name.

    Important things to know about this command:
    . Use this switch with the "-abstract" switch.
    . If you also use the "-connect" switch, the explicit connections
      specified with that switch take precedence over the name-based
      matching specified by this "-auto" switch.
    . The tool does not autoconnect signals of
      different widths.

    NOTE: For proper functioning, take special care with the "clk" signal.
          Use one of the following three options:

           1) Ensure that the RAM that is being abstracted contains a
              port named "clk", and allow the connection of the
              jasper_model_ram "clk" signal to be handled with the
              "-auto" switch.

           2) Ensure that the RAM that is being abstracted contains a
              clock port, and  then use an explicit "-connect" switch
              to make the connection. If the clock port on the
              RAM is called "clock", then the syntax is as follows:

                  -connect clk clock

           3) Use an explicit "clock" command to define the clock
              behavior for the jasper_model_ram. If the path to
              the RAM being abstracted is "EXT_MEM", then
              the "clock" command is:
```

```
                clock <clock> EXT_MEM.jasper_ram_i.clk

            In this command,  <clock> is a clock previously declared with the
            Jasper Apps "clock" command.

-init instance_name
    Set "instance_name" to the maximum level of abstraction.

    Run this command after elaboration and before specifying the clock
    and reset conditions.

    NOTE: "maximum level of abstraction" means that every access returns an
    abstract value (that is, nothing is justified). The default value is 32,
    which is also the maximum concrete values jasper_model_ram can store. Thus,
    there is no benefit to running "jasper_model_ram -justify <instance_name>"
    more than 32 times.

-preload addr [data]
    Treat the contents of the specified address (addr) as non-abstracted
    data, and when provided, use the optional "data" value as the reset
    value for the specified memory location.

    Use this switch with the "-init" switch. Refer to the example section
    below

-justify instance_name
    Decrease the abstraction level of the specified instance by one memory
    location.

    NOTE:
    . This Proof Accelerator models the RAM being abstracted by selecting
      certain memory locations and treating them concretely. Generally, this is
      done on a first-come, first-served basis; however, in the event of
      multiple simultaneous accesses, ports are serviced in the following
      order:
      . Write ports before read ports
      . Lower numbered ports before higher numbered ports
    . The maximum number of concrete values jasper_model_ram can store is 32;
      therefore, there is no benefit to running "jasper_model_ram -justify
      <instance_name>" more than 32 times.

-suggest
    Analyze the current trace to determine whether justifying any of the
```

instances of the jasper_model_ram will provide any benefit.

Example

```
// Connect the ram PA (shown here for an example with two read ports and one
// write port):
jasper_model_ram -abstract  my_ram                                  \
                 -parameter ADDR_WIDTH                           64  \
                 -parameter DATA_WIDTH                           16  \
                 -parameter MEM_STYLE                             0  \
                 -parameter RESET_STYLE                           1  \
                 -connect   clk                                 clk  \
                 -connect   rstN                               rstN  \
                 -connect   write                           myWrite  \
                 -connect   addr_wr                     myWriteAddr  \
                 -connect   data_wr                     myWriteData  \
                 -connect   read                             myRead  \
                 -connect   addr_rd                      myReadAddr  \
                 -connect   data_rd                      myReadData

// Initialize the abstraction (using the jasper_model_ram -init command):
jasper_model_ram -init               \
                   -preload   ...      \
                   my_ram
// NOTE:
// a. If the PA is instantiated directly, the instance name used in the command
//    is the instance name of the PA
// b. If the PA is instantiated via Tcl, the instance name used in the command
//    is the instance name of the RAM being abstracted.

// Tune the abstraction (using the jasper_model_ram -justify command):
jasper_model_ram -justify my_ram

// NOTE:
// a. If the PA is instantiated directly, the instance name used in the command
//    is the instance name of the PA
// b. If the PA is instantiated via Tcl, the instance name used in the command
//    is the instance name of the RAM being abstracted.
```

jasper_scoreboard_3

Loads definitions to configure jasper_scoreboard_3 PA.

Syntax

```
jasper_scoreboard_3 -init
```

Returns

No return values

Default

No default

Detailed Description

Use the "jasper_scoreboard_3" command to load basic configuration definitions to be used with jasper_scoreboard_3 instantiations. This PA provides a means to verify the integrity of data packets transferred through a design.

To learn more about the jasper_scoreboard_3 PA, do one of the following to access the jasper_scoreboard_3 data sheet:
. From the Help menu, choose Proof Accelerator Datasheets - jasper_scoreboard_3.

-init
    Loads jasper_scoreboard_3 parameter types and definition helpers. As shown in the jasper_scoreboard_3 data sheet, all parameter types and values can be found within the jasper_scoreboard_3_pkg package or in the global scope with macros prefixed by the "JS3_" string.

    Run this command before analyzing your design files.

    NOTE: These definitions are stored together with "analyze" data; thus, invoking "analyze -clear" or "clear -all" unloads jasper_scoreboard_3 definitions.

Example

```
// Load parameter definitions
jasper_scoreboard_3 -init
// Setup design
analyze -sv my_design.v
elaborate
```

jforeach

Optimizes memory for large Tcl results by iterating through results.

Syntax

```
jforeach variable_list command body
```

Returns

No return values

Detailed Description

Use this command to iterate over a command that has a large result in an optimized mode, returning results on demand into the variables in variable_list. Use this command when you expect the command's Tcl result to take near or more than 2GB of space.

The "jforeach" command supports the following commands:

- check_superlint -list
- check_cdc -list
- check_cov -list
- get_design_info (following switches only):
  - -list
  - -instance [-list]
  - -module [-list]
  - -entity [-list]
  - -interface -list
  - -interface_instance -list
  - -interface_port -list
  - -property [-list]
  - -signal [-list]
  - -package [-list]
  - -compilation_unit -list
  - -task -list
- get_fanin -transitive
- get_fanout -transitive
- get_reset_info (following switches only):
  - -all
  - -x_value
  - -bit_blast
  - -list
  - -constant_registers
  - -get_unstable
- get_signal_list

NOTE: The "jforeach" command turns off the output to the screen.

Example

```
jforeach item {get_reset_info} {
      puts $item
  }
```

See Also

get_design_info,
get_reset_info

liberty

Loads specified Synopsys(R) Liberty(TM) files and reports cell information.

Syntax

```
liberty -load (file_name)+ [-repository repository_id]

liberty -info (cell_module_name)+ [-repository repository_id]

liberty -list [-include {attribute_1 value_list_1 attribute_2 value_list_2 ...}]
                        [-exclude {attribute_1 value_list_1 attribute_2 value_list_2
...}]

liberty -model_timing_arc
        -lib lib_name
        -cell cell_name
        (-pin pin_name | -bus bus_name)
        -related_pin related_pin_name
        -when condition_expression
        -timing_type (rising_edge | setup_timing)
```

Returns

"-info" returns the name of a Synopsys Liberty cell, its source location, line file, and attributes.

"-list" returns a list of all Synopsys Liberty cells used in the design. Or if you use "-include" or "-exclude" it filters the list and returns the names of cells that satisfy the value criteria you specified.

Detailed Description

Use the "liberty" command to load Synopsys(R) Liberty(TM) library modeling standard files into Jasper Apps, report information on a specified cell, and list the names of all cells that satisfy specified values.

-load (file_name)+ [-repository repository_id]

```
    Loads the specified Synopsys Liberty files. This command accepts multiple
    Liberty files or a file listing Liberty files as arguments.

    You can use this command to make pg_pins defined in the Synopsys Liberty
    files accessible by the UPF "connect_supply_net" command. In addition, the
    Clock Domain Crossing App can use the files during analysis
    (check_cdc -clock_domain -find...).

    . Use "-repository" to load the specified files inside a specified
      repository, which is outside of the main design scope. The argument
      "repository_id" is an integer specifying the repository inside which
      the Liberty files will be loaded. Currently, two repositories are
      available: 0 and 1.

-info (cell_module_name)+ [-repository repository_id]
    This command returns a Tcl dictionary containing Synopsys Liberty cell
    source location and its attributes. In addition, you can use
    the "-repository" switch to specify the repository ID.

    NOTE:
    . Dictionary keys are "source_location", "always_on", "level_shifter",
      "isolation", "memory", "power_switch", "retention",
      "isolation_level_shifter", and "pad".
    . Possible values for "always_on", "level_shifter", "isolation", "memory",
      "power_switch", "retention", "isolation_level_shifter", and "pad" are 0
      or 1.

-list [-include {attribute_1 value_list_1 attribute_2 value_list_2 ...}]
      [-exclude {attribute_1 value_list_1 attribute_2 value_list_2 ...}]
    This command returns a list of all Synopsys Liberty cells used in the
    design. In addition, you can use the "-include" and "-exclude" switches to
    return a list of Synopsys Liberty cells whose attributes satisfy value
    criteria specified by the switches.

    . Use "-include" with a list of attributes to specify what you want the
      tool to include in the returned Tcl list. The list includes cells that
      match the criteria specified by "attribute_N" with "value_list_N".

    . Use "-exclude" with a list of attributes to specify what you want to
      exclude from the returned Tcl list. The list excludes cells that
      match the criteria specified by "attribute_N" with "value_list_N".

    NOTE: "-exclude" takes precedence over "-include", which means that if you
```

```
    use an attribute value with both the "-include" and "-exclude" switches,
    the tool excludes any attribute that has the specified attribute value from
    the returned Tcl list.

    The following is the list of attributes supported by "-include" and
    "-exclude". Refer to the TclHelp for "liberty -info" for more details,
    including possible values.

    . always_on
    . isolation
    . isolation_level_shifter
    . level_shifter
    . memory
    . power_switch
    . retention
    . pad

    NOTE: "-include" and "-exclude" switches values must be 0 or 1. Refer
    to the examples.

-model_timing_arc
    -lib lib_name
    -cell cell_name
    (-pin pin_name | -bus bus_name)
    -related_pin related_pin_name
    -when condition_expression
    -timing_type (rising_edge | setup_timing)

    Adds or overrides a Liberty's timing arc for MCP purposes.

    . Use "-lib" to specify the Liberty's library name.
    . Use "-cell" to specify the cell name.
    . Use "-pin" or "-bus" to specify the pin/bus name.
    . use "-related_pin" to specify timing arc's "related_pin" name.
    . Use "-when" to specify timing arc's "when" condition expression.
    . Use "-timing_type" to specify between timing arc's "timing_type". Use
      "rising_edge" for START point and "setup_timing" for END point.

    NOTE: All switches are required. "-bus" and "-pin" are mutually exclusive.
    NOTE: For "-when" condition, check for expression consistency with Verilog
    syntax.
    NOTE: Confirm that library, cell and pin/bus names match exactly in the
    Liberty model file.
```

NOTE: The tool issues a warning on success of a Liberty model override or if a new arc is added.

Example

```
# The following command loads the "top.lib" and "example.lib" libraries.
liberty -load top.lib example.lib

# The following command returns the source location, line number, and attributes
information
# if flop is a Synopsys Liberty cell.
liberty -info flop

# The following command returns a list of all Synopsys Liberty cells used in the
design.
liberty -list

# The following command returns a list of all isolation Synopsys Liberty cells used in
the design.
liberty -list -include {isolation 1}

# The following command returns a list of all Synopsys Liberty cells used in the design
that
# are not retention.
liberty -list -include {retention 0}

# The following command returns a list of all Synopsys Liberty cells used in the design
that
# are isolation and not always_on.
liberty -list -include {isolation 1 always_on 0}

# The following command returns a list of all Synopsys Liberty cells used in the design
that
# are not power_switch.
liberty -list -exclude {power_switch 1}

# The following command returns a list of all power_switch Synopsys Liberty cells used
in the design.
liberty -list -exclude {power_switch 0}

# The following command loads libraries listed in libs.txt file.
liberty -load libs.txt
```

```
# The following command adds a rising edge timing arc for MCP purposes.
liberty -model_timing_arc -lib m_lib -cell m_cell -bus Q -related_pin CL -timing_type
rising_edge -when {!REN & WEN}

# The following command accesses the information from liberties loaded on repository 1.
liberty -info A -repository 1
```

load_radix_file

Loads a radix file.

Syntax

```
load_radix_file file_name+
```

Returns

Returns an error message if file cannot be loaded.

Detailed Description

```
Use this command to load one or more radix files before a trace
becomes available.

When you use this command, radix definitions are loaded
and use clauses are only executed if a trace is available.
When a trace becomes available, use clauses are applied
automatically.

NOTE:
. If a Visualize window is available, this command is equivalent to "visualize
  -load_radix file_name".
. A new custom radix file can overwrite the current signal radix defined by the
  user or by another custom radix file.

file_name+
    Load and apply radixes defined in the specified file_name(s).
    You can list one or more files.

    "file_name" file format consists of radix definition clauses
    and radix use clauses.

    Radix definition clauses syntax:
    ================================
```

```
radix <radix_name>  // radix name to be displayed in GUI
size 8                    // number of bits
base b|o|d|h              // optional. Default base is binary
begin
  TAG1 <value>      // value specified in defined base
  TAG2 <value>      // "x" is allowed as a wildcard. It is the only
  .                       // supported wildcard. No wildcards are supported
  .                       // in decimal base.
  .
end
```

Notes:

. Values will be zero extended if you do not completely specify size.
  Example: 10 is represented by 00000010 if size is 8 and
  base is binary.

. If you do not specify a tag for certain values, the Visualize window
  displays the default radix.

. A number must be matched by only one tag.

Radix definition clauses with color syntax:

The tool supports a color field in definition clauses. Use this field to highlight specified values in Visualize waveforms.

```
radix <radix_name>
size N
base b|o|d|h
begin
   TAG1 <value> <color1>
   TAG2 <value> <color2>
   ...
end
```

Definitions can be mixed to allow applying highlights for a subset of the radix values as shown below.

```
radix <radix_name>
size N
base b|o|d|h
begin
   TAG1 <value>
```

```
   TAG2 <value> <color> // This value is highlighted in waveforms
   ...
end

Jasper uses the Qt predefined color service. You can supply either the
Qt color or the hex color code.

Qt Color         Hex Color Name and Code
--------         ----------------------
white            White        #ffffff
black            Black        #000000
red              Red          #ff0000
darkRed          Dark red     #800000
green            Green        #00ff00
darkGreen        Dark green   #008000
blue             Blue         #0000ff
darkBlue         Dark blue    #000080
cyan             Cyan         #00ffff
darkCyan         Dark cyan    #008080
magenta          Magenta      #ff00ff
darkMagenta      Dark magenta #800080
yellow           Yellow       #ffff00
darkYellow       Dark yellow  #808000
gray             Gray         #a0a0a4
darkGray         Dark gray    #808080
lightGray        Light gray   #c0c0c0

Radix use clauses syntax:
=========================

use [radix <radix_name> | radix_signal <signal_name>
begin
  signal <signal_name>, ... , <signal_name>
  signal <signal_name>
end

Notes:
. You can specify the radix you want to apply to listed signals with
  a radix name or signal name:
  ("use radix <radix_name>" clause) or
  ("use radix_signal <signal_name>" clause).

. When using a radix name ("use radix <radix_name>"), the
```

```
  radix you want to apply must be defined previously, in the same
  file.

. Do not use Wildcards in signal names.

Custom Radix JSON File:
=======================
Use a Custom Radix JSON File to define custom radices and radices usage
using JSON format.

NOTE: If the command fails to recognize the file as a JSON format, the file
will be parsed as a regular Custom Radix File.

Header object:
==============
The header object contains two mandatory values, the type value and the
version value as shown below:

"header" : {
    "type" : "visualize_radix_file",
    "version" : 1
}

NOTE:
. The header object is mandatory.
. The type value differentiates this type of JSON file from others JSON
  files. The single possible value for type is "visualize_radix_file".
. The version value differentiates this JSON file from other future
  versions.

Radices object:
===============
Radices objects contain all defined radices where the key is the name of the
radix and the value contains the radix definition.

"radices" : {
    "fp_radix_a" : {
        "type" : "floating_point",
        "mapping" : {
            "16" : {"exponent" : 5, "mantissa" : 10},
            "32" : {"exponent" : 8, "mantissa" : 23},
            "64" : {"exponent" : 11, "mantissa" : 52}
        }
```

```
    },
    "fp_radix_b" : {
        "type" : "floating_point",
        "mapping" : {
            "18" : {"exponent" : 6, "mantissa" : 11},
            "34" : {"exponent" : 9, "mantissa" : 24}
        }
    },
    ...
}
```

NOTE: The radices object is optional.

Floating-Point radix:
====================
Floating-point radix lets you create radices with specific configurations following the IEEE 754 standard.
You can specify different signal widths with different exponent and mantissa widths. It has two mandatory objects:
. type: It holds the type of the radix, and it is "floating_point" for the floating-point radix.
. mapping: Contains the floating-point definition for each signal size.

```
          16 bits signal
+--+---------------+-------------+
|  |               |             |
+--+---------------+-------------+
|15|14            7|6           0
|  |               +---mantissa (7bits)
|  +--------exponent (8bits)
+----sign (1bit)
```

```
"fp_radix_b" : {
    "type" : "floating_point",
    "mapping" : {
        "16" : {"exponent" : 8, "mantissa" : 7},
        "34" : {"exponent" : 9, "mantissa" : 24}
    }
}
```

Each element in the mapping is an object with a signal width as a key and an object containing the exponent width and mantissa width.
A single floating-point radix can hold several mapping values, one for each

```
size.

NOTE:
. Mapping values must respect the following constraint:
  size = exponent + mantissa + 1
. The maximum supported signal size is 64.

You can also define a custom bias to be subtracted from the exponent.

"fp_radix_b" : {
  "type" : "floating_point",
  "mapping" : {
      "16" : {"exponent" : 8, "mantissa" : 7, "bias": 32},
      "34" : {"exponent" : 9, "mantissa" : 24, "bias": 1024}
  }
}

NOTE: The radix is still subject to denormalization shift if exponent is 0.

Fixed-Point radix:
====================
Fixed-point radix lets you create radices with specific configurations.
You can specify different signal widths with different fractional widths
and sign. It has two mandatory objects:
. type: It holds the type of the radix, and it is "fixed_point" for the
  fixed-point radix.
. mapping: Contains the fixed-point definition for each signal size.

          16 bits signal
+--+---------------+-------------+
|  |               |             |
+--+---------------+-------------+
|15|14            7|6           0
|                  +---fractional (7bits)
|
+----sign (1bit)

"fp_radix_b" : {
    "type" : "fixed_point",
    "mapping" : {
        "16" : {"fractional" : 8, "sign" : false},
        "34" : {"fractional" : 9, "sign" : true}
    }
```

```
}

Each element in the mapping is an object with a signal width as a key and an
object containing the fractional width and whether the radix is signed.
A single fixed-point radix can hold several mapping values, one for each
size.

NOTE:
. Mapping values must respect the following constraints:
  . signed: size >= fractional + 1
  . unsigned: size >= fractional
. The maximum supported signal size is 64.

Usages object:
==============
The usages object maps a signal to a specific radix. This radix can be any
radix already defined or a built-in radix such as dec, hex, bin, and so on.

"usages" : {
    "signal_a" : "hex",
    "signal_b" : "fp_radix_a",
    "signal_c" : "dec",
    ...
    "signal_z" : "fp_radix_b"
}

NOTE:
. The usages object is optional.
. Use the full hierarchical path when specifying a signal.
```

Example

```
load_radix_file radix_D.rdx
```

See Also

visualize -load_radix

pretty_print

Displays a Tcl list in an easy-to-read format.

Syntax

pretty_print

Returns

No return values

Detailed Description

Use this command to display any Tcl list in a more easy-to-read format. With this command, the tool prints each element of a list on a separate line with no indentation. "{}" indicates null elements. Nested lists begin with "{", end with "}", and print with a two-space indentation.

Example

```
// List without "pretty_print"
puts $someList
a {} c {d.1 d.2} e {f.1 {} {f.3.1 f.3.2} f.4} g

// List with "pretty_print"
pretty_print $someList
a
{}
c
{
  d.1
  d.2
}
e
{
  f.1
  {}
  {
    f.3.1
    f.3.2
  }
  f.4
}
g
```

proof_structure

Sets up formal methodologies using Jasper tasks.

Syntax

```
proof_structure -init root_node_name
                [-from task_name]
                [(-copy_all |-copy_assumes |-copy_asserts |-copy_covers)+ [-
exclude_disabled_properties]]
                [-copy_abstractions (all | counter | init_value | reset_value)+]
                [-copy_related_covers]
                [-copy_related_asserts]
                [-copy_stopats [-copy_ratings]]
                [-copy property_name_list [-regexp]]
                [-no_copy_env]

proof_structure -create assume_guarantee
                [-op_name operation_name]
                [-imp_name node_names_list]
                [-from node_name]
                -property list_of_property_lists
                [-remaining]
                [-non_cumulative]
                [-fail_if missing_property (assert | cover)+]
                [-copy_related_covers]
                [-copy_related_asserts]
                [-disable_operation_limit]

proof_structure -create partition
                [-op_name operation_name]
                [-imp_name node_names_list]
                [-from node_name]
                [-copy list_of_property_lists
                  [-remaining]
                  [-fail_if missing_property (assert | cover)+]
                  [-fail_if duplicated_property (assert | cover)+]
                  [-num_copies N]]
                [-copy_related_covers]
                [-copy_related_asserts]
```

```
                [-disable_operation_limit]

proof_structure -create stopat
                [-op_name operation_name]
                [-imp_name stopat_node_name]
                [-from node_name]
                (-add (signal_name+ | instance_name+) | -remove (signal_name+ |
instance_name+))

proof_structure -create edit_node
               [-op_name operation_name]
               [-imp_name edit_node_name]
               [-from node_name]
               [-property property_list]

proof_structure -create case_split
               [-op_name operation_name]
               [-imp_name node_names_list]
               [-from node_name]
               -condition condition_list
               [-property property_list]
               [-disable_operation_limit]
               [-case_name cases_names_list]
               [-compress]

proof_structure -create hard_case_split
               [-validity (combinational | invariant | exhaustive)]
               [-op_name operation_name]
               [-imp_name node_names_list]
               [-from node_name]
               -condition condition_list
               [-property property_list]
               [-disable_operation_limit]

proof_structure -create compositional_assume_guarantee
               [-op_name operation_name]
               [-imp_name node_names_list]
               [-from node_name]
               [-property property_list]
               [-copy property_list]
               [-copy_related_covers]
               [-copy_related_asserts]
```

```
proof_structure -create underconstraint
                [-op_name operation_name]
                [-imp_name node_names_list]
                [-from node_name]
                [-property property_list]

proof_structure -create overconstraint
                [-op_name operation_name]
                [-imp_name node_names_list]
                [-from node_name]
                [-property property_list]

proof_structure -get_property_info property
                 [-list (status | min_length | max_length)+]
                 [-node node_name]

proof_structure -get_node_info node_name
                 [-list (parent | children | class | type | propagation_status |
condition | configuration)+]

proof_structure -get_node_list

proof_structure -set_visible_results (local | proof_structure)

proof_structure -propagate

proof_structure -remove node_name

proof_structure -export node_name
                -create new_task_name

proof_structure -edit target_node_name
                -copy property_name_list
                -source_task source_task_name

proof_structure -unify_results
```

Returns

Returns a list of created/removed nodes or a property status dictionary.

Detailed Description

Detailed Description

Use this command to set up formal methodologies in Jasper with a graphic viewer showing the relationship between environments and Proof Structure result propagation.

NOTE: "proof_structure" does not support differential analysis properties (for example, X-Prop properties, SPV properties, cover -path). The tool blocks the command and issues an error.

```
-init root_node_name
      [-from task_name]
      [( -copy_all |-copy_assumes |-copy_asserts
        |-copy_covers)+ [-exclude_disabled_properties]]
      [-copy_abstractions (all | counter | init_value | reset_value)+]
      [-copy_related_covers]
      [-copy_related_asserts]
      [-copy_stopats [-copy_ratings]]
      [-copy property_name_list [-regexp]]
      [-no_copy_env]
```

Initialize a Proof Structure tree by importing a Jasper task into the Proof Structure environment, creating a root node. This root node includes all the abstractions, assumptions and hard stopats from the source task by default.

NOTE: You must specify a "root_node_name" for the root node that Proof Structure creates.

. Use "-from task_name" to specify the source task. If you do not specify a task, the tool uses the current task.

. Use "-copy_all" to copy all abstractions, assertions, assumptions, covers, and hard stopats from the source task to the Proof Structure node.

. Use "-copy_abstractions" to copy one or more abstraction types from the source task to the Proof Structure node:
  . all - counter, init value, and reset value abstractions.
  . counter - counter abstractions. See "abstract -counter".
  . init_value - initial value abstractions. See "abstract -init_value".
  . reset_value - reset value abstractions. See "abstract -reset_value".

. Use "-copy_assumes" to copy all assumptions from the source task to the

Proof Structure node.

. Use "-copy_asserts" to copy all assertions from the source task to the Proof Structure node.

. Use "-copy_covers" to copy all cover directives from the source task to the Proof Structure node.

. Use "-copy_related_covers" to copy all automatically extracted related covers for the properties that are being copied.

  NOTE: For additional information about related covers, see "elaborate -create_related_covers" and the command reference manual appendix "Vacuity Checking with Automatically Extracted Covers."

. Use "-copy_related_asserts" to copy all automatically extracted related assertions for the properties that are being copied.

. Use "-copy_stopats" to copy all stopats from the source task to the Proof Structure node. And use "-copy_ratings" with "-copy_stopats" to copy all the task ratings on hard stopats.

. Use "-copy" with a Tcl list to copy specified assertions, assumptions, or covers from the source task to the Proof Structure node.

  NOTE:
  . The "-copy" switch supports wild cards (* and ?) and regular expressions for property names.
  . When using a regular expression, use the "-regexp" switch after the pattern.

. Use "-no_copy_env" to prevent copying all abstractions, assumptions and hard stopats from the source task to the Proof Structure node.
  NOTE:
  . The effects of the switches initiated with "-copy" prevail over the effect of "-no_copy_env".
  . Use both switches "-copy_all" and "-no_copy_env" to copy only the covers and assertions from the source task to the Proof Structure node.

. Use "-exclude_disabled_properties" to ignore disabled covers, assumptions and assertions when specifying "-copy_all", "-copy_asserts", "-copy_assumes", or "-copy_covers".
  NOTE: This option limits only "-copy_all", "-copy_asserts",

"-copy_assumes", and "-copy_covers". Other copy options, including "-copy" with a Tcl property list, have higher precedence than "-exclude_disabled_properties", so disabled properties selected by those options will be copied.

```
-create assume_guarantee [-op_name operation_name]
                         [-imp_name node_names_list]
                         [-from node_name]
                         [-property list_of_property_lists]
                         [-remaining]
                         [-non_cumulative]
                         [-fail_if missing_property (assert | cover)+]
                         [-copy_related_covers]
                         [-copy_related_asserts]
                         [-disable_operation_limit]
```

Create an Assume-Guarantee operation using the task or node specified with "-from" as the source for the operation.

The Assume-Guarantee operation consists of partitioning the problem into multiple contexts based on the decomposition groups specified with "-property", using a set of targets as assumptions to prove another set of properties, and guaranteeing that the targets used as assumptions are valid in the environment. For each group N starting with N=2, we create a node with the properties in N, and use the properties in all groups [1...(N-1)] as assumptions.

The results from the implementation nodes are propagated to their parent, respecting propagation rules, depending on how the environments for children and the parent relate. The result from the implementation nodes in groups [1...(N-1)] are also propagated to implementation node N.

The implementation nodes support further editing as long as no operation is created from them. The supported changes include:

. Inserting stopats and init value abstractions: use commands "stopat" and "abstract -init_value".
. Removing and clearing assumptions: use commands "assume -remove" and "assume -clear".
. Adding reset value abstractions and counter abstractions: use commands "abstract -reset_value" and "abstract -counter".
  NOTE: The assume node does not accept the "abstract -reset_value" command switches "condition_expression", "expression" or "constant".

NOTE: The editing of the assume node might change the applicable propagation rules applicable.

NOTE: Find additional information on propagation rules in the "Proof Structure" chapter of the Jasper Platform and Formal Property Verification App User Guide.

WARNING: The propagated results for the Proof Structure are not shown in the Property Table. Use "proof_structure -get_property_info" to retrieve the results.

The command returns the operation name and a dictionary with parent node and implementation nodes.

. Use "-op_name operation_name" to specify the operation name (name for the Assume-Guarantee operation).

  NOTE: If you do not specify "-op_name", the tool automatically generates a name for the operation. The name is prefixed with "AG_".

. Use "-imp_name node_names_list" to specify the names of the nodes the tool creates.

  NOTE: If you do not specify "-imp_name", the tool automatically generates names for the nodes. The nodes are prefixed with the name of the operation node.

. Use "-from" to specify the source node for the operation.

  NOTE: If you do not specify "-from", the tool uses the current task as the source node.

. Use "-property" to specify the property groups. The order of declaration determines the node relationship.

  NOTE: Covers can be specified only in the last property group.

. Use "-remaining" to create the last property group with the properties not specified by the "-property" command.

. Use "-non_cumulative" to create a non-cumulative version of the Assume-Guarantee operation. In cumulative mode (default), targets proven in each node are used as assumptions in all subsequent nodes. With "-non_cumulative", only the targets of the immediate previous node are used as assumptions in the current node rather than on all previous ones.

. Use "-fail_if missing_property" to specify the type of properties you want the tool to check. This check triggers an error if properties in the "-from" node matching the specified type are not present in one of the implementation nodes.

. Use "-copy_related_covers" to copy all automatically extracted related covers for the properties that are being copied.

. Use "-copy_related_asserts" to copy all automatically extracted related assertions for the properties that are being copied.

. Use "-disable_operation_limit" to force the tool to create the operation with a number of implementation nodes greater than the global limit of 100.

NOTE: Creating more than 100 implementation nodes can make the GUI temporarily unresponsive.

```
-create partition [-op_name operation_name]
                  [-imp_name node_names_list]
                  [-from node_name]
                  [-copy list_of_property_lists
                    [-remaining]
                    [-fail_if missing_property (assert | cover)+]
                    [-fail_if duplicated_property (assert | cover)+]
                    [-num_copies N]]
                  [-copy_related_covers]
                  [-copy_related_asserts]
                  [-disable_operation_limit]
```

Create a "partition" operation using the node specified with "-from" as the source for the operation.
The command creates N nodes, breaking the properties of the source node into different nodes. Each child node includes all the abstractions, assumptions, and hard stopats from the source node by default. The number of nodes is defined by the number of partitions listed in "-copy" or the argument of "-num_copies".

NOTE: By default, the command uses "-num_copies 1".

The command returns the operation name and a dictionary with the parent node and the names of the created nodes.

. Use "-op_name operation_name" to specify the operation name (name for

the partition operation).

NOTE: If you do not specify "-op_name", the tool automatically generates a name for the operation. The name is prefixed with "P_".

. Use "-imp_name node_names_list" to specify the names of the nodes the tool creates.

NOTE: If you do not specify "-imp_name", the tool automatically generates names for the nodes. The nodes are prefixed with the name of the operation node.

. Use "-from" to specify the node for the operation.

NOTE: If you do not specify "-from", the tool uses the current task as the source node.

. Use "-copy" to specify the target properties that will be included in each partition.

NOTE: "-copy" accepts a list of asserts and covers only.

. Use "-remaining" with "-copy" to create a new partition with the properties not specified by the "-copy" command.

. Use "-fail_if duplicated_property" to specify the type of properties that will be checked in the mutually exclusive check. This check makes the command accept only partitions in which no properties are duplicated.

. Use "-fail_if missing_property" to specify the type of properties you want the tool to check. This check triggers an error if properties in the "-from" node matching the specified type are not present in one of the partitions.

. Use "-num_copies" to specify the number of copies you want to create for the node specified by "-from".

. Use "-copy_related_covers" to copy all automatically extracted related covers for the properties that are being copied.

. Use "-copy_related_asserts" to copy all automatically extracted related assertions for the properties that are being copied.

```
    . Use "-disable_operation_limit" to force the tool to create the operation
      with a number of implementation nodes greater than the global limit of
      100.

      NOTE: Creating more than 100 implementation nodes can make the GUI
      temporarily unresponsive.

-create stopat [-op_name operation_name]
               [-imp_name stopat_node_name]
               [-from node_name]
               (-add (signal_name+ | instance_name+)
               |-remove (signal_name+ | instance_name+))
    Create a "stopat" operation using the node specified with "-from" as the
    source for the operation.
    The command creates one node with copies of all properties and abstractions
    from the target node but with a stopat environment change as specified by
    the "-add" or "-remove" switch.

    NOTE:
    . The "-add" and "-remove" switches support Tcl lists as arguments, which
      is the preferred usage.

    . Adding a stopat to an instance adds stopats to its outputs and read-only
      hierarchical references except its undriven outputs, which are already
      free and do not need stopats.

    . Find additional information on propagation rules in the "Proof
      Structure" chapter of the Jasper Platform and Formal Property
      Verification App User Guide.

    The command returns the operation name and a dictionary with the parent
    node and the name of the created node.

    . Use "-op_name operation_name" to specify the operation name (name for
      the partition operation).

      NOTE: If you do not specify "-op_name", the tool automatically generates
      a name for the operation. The name is prefixed with "S_".

    . Use "-imp_name stopat_node_name" to specify the name of the node the tool
      creates.

      NOTE: If you do not specify "-imp_name", the tool automatically generates
```

```
      names for the nodes. The nodes are prefixed with the name of the
      operation node.

    . Use "-from" to specify the source node for the operation.

      NOTE: If you do not specify "-from", the tool uses the current task as
      the source node.

-create edit_node [-op_name operation_name]
                  [-imp_name edit_node_name]
                  [-from node_name]
                  [-property property_list]
    Create an "edit" operation using the node specified with "-from" as the
    source for the operation.
    The command creates one node with copies of all properties and abstractions
    from the target node and allows changes to the created node. The supported
    changes include inserting, removing, disabling, enabling, and converting
    properties.

    NOTE:
    . The newly created node supports further editing as long as no
      operation is created from it.
    . Find additional information on propagation rules in the "Proof
      Structure" chapter of the Jasper Platform and Formal Property
      Verification App User Guide.

    The command returns the operation name and a dictionary with the parent
    node and the name of the created node.

    . Use "-op_name operation_name" to specify the operation name (name for
      the partition operation).

      NOTE: If you do not specify "-op_name", the tool automatically generates
      a name for the operation. The name is prefixed with "E_".

    . Use "-imp_name edit_node_name" to specify the name of the node the tool
      creates.

      NOTE: If you do not specify "-imp_name", the tool automatically generates
      names for the nodes. The nodes are prefixed with the name of the
      operation node.

    . Use "-from" to specify the source node for the operation.
```

```
      NOTE: If you do not specify "-from", the tool uses the current task as
      the source node.

    . Use "-property" to specify the target properties to be included in
      the edit operation.

      NOTE: If you do not specify "-property", all properties from the source
      node are considered targets of the operation.

-create case_split [-op_name operation_name]
  [-imp_name node_names_list]
  [-from node_name]
  -condition condition_list
  [-property property_list]
  [-disable_operation_limit]
  [-case_name cases_names_list]
  [-compress]
    Create a "Case Split" operation using the node specified with "-from" as
    the source for the operation.
    This command creates N + 1 nodes as follows. You will use "-condition" to
    specify conditions, that is, boolean expressions, which define the case
    splits the tool creates.
    For example, "-condition {{a == 0} {a == 1} {a == 2}}" creates three
    case condition nodes, one to run proofs for each of the three specified
    values of signal a. The relevant condition is added as a precondition to
    the proofs of target properties in each case node:
    if property P is among the proof targets, the property to be proven becomes
    "(a==0) |-> P" in the first node, "(a==1) |-> P" in the second, and
    "(a==2) |-> P" in the third.
    In addition to these N nodes, the tool creates one more, which is the
    completeness check node, that is, the node with the completeness check that
    validates the conditions you specified. The completeness check node
    validates that at least one of the expressions specified with "-condition"
    is always true; otherwise this check prevents the propagation of proven
    results.

    The results from the nodes are propagated to their parent, respecting
    propagation rules.

    NOTE:
    . The propagated results also depend on how the environments for the
      children and their parent relate and the status of the completeness check.
```

```
. Find additional information on propagation rules in the "Proof
  Structure" chapter of the Jasper Platform and Formal Property
  Verification App User Guide.

The command returns a dictionary composed of the operation name
("op_name"), the type of operation ("operation"), the parent node
("source_node"), and the implementation nodes ("imp_nodes"). Check the
example section for more information.

WARNING:
. The Case Split operation only natively supports Tcl properties.
  To use embedded properties, see the help of command
  "set_proof_structure_support_embedded_on_case_split".

  NOTE: Embedded liveness properties are not supported in Proof Structure,
  even with "set_proof_structure_support_embedded_on_case_split".

. The Case Split operation does not propagate related cover results.

. Use "-op_name operation_name" to specify the operation name (name for
  the case split operation).

  NOTE: If you do not specify "-op_name", the tool automatically generates
  a name for the operation. The name is prefixed with "CS_".

. Use "-imp_name node_names_list" to specify the names of the nodes the
  tool creates.

  NOTE:
  . If you do not specify "-imp_name", the tool automatically generates
    names for the nodes. The nodes are prefixed with the name of the
    operation node.
  . The name of the completeness check node is auto-generated. The name is
    prefixed with the operation name plus ".completeness_check_".

. Use "-from" to specify the node for the operation.

  NOTE: If you do not specify "-from", the tool uses the current task as
  the source node.

. Use "-condition" to specify the condition to be used in each case split
  node.
```

```
      NOTE: The conditions are required to be combinatorial Boolean
      expressions.

    . Use "-property" to specify the target assertions that will be included in
      the case split operation.

      NOTE: If you do not specify "-property", all assertions from the source
      node are considered targets of the operation.

    . Use "-disable_operation_limit" to force the tool to create the operation
      with a number of implementation nodes greater than the global limit of
      100.

      NOTE: Creating more than 100 implementation nodes can make the GUI
      temporarily unresponsive.

    . Use "-case_name" to specify a name for each case. The case name acts as a
      prefix for the property names. Each case's properties are named as
      "prefix_name", where "prefix" is the case's name and "name" is the
      original property's name.

    . Use "-compress" to create a single implementation node containing all case
      properties and the completeness check.

      NOTE: If you use "-compress" without "-case_name", the tool automatically
      prefixes the properties with "case_n", where "n" is the case's
      corresponding position in the list specified with "-condition".

-create hard_case_split
  [-validity (combinational | invariant | exhaustive)]
  [-op_name operation_name]
  [-imp_name node_names_list]
  [-from node_name]
  -condition condition_list
  [-property property_list]
  [-disable_operation_limit]
    Create a "Hard Case Split" operation using the node specified with "-from"
    as the source for the operation. This command creates N + 2 nodes as
    follows:
    Use "-condition" to specify conditions, that is, boolean expressions,
    which define the case splits the tool creates.
    For example, "-condition {{a == 0} {a == 1} {a == 2}}" creates three
    case condition nodes, one to run proofs for each of the three specified
```

values of signal a. The relevant condition is added as an assumption in each case node: the first case node adds "assume {a==0}", the second adds "assume {a==1}", and the third adds "assume {a==2}".

In addition to these N nodes, the tool creates two more. The first is the completeness check node, that is, the node with the completeness check that validates the conditions you specified. The completeness check node validates that at least one of the expressions specified with "-condition" is always true; otherwise this check prevents the propagation of proven results.

The other additional node is the validity node, which allows the user to specify one of three validity types to enable correctness of the overall case splitting proof:

. combinational: The validity node must prove that the logic is purely combinational with no state elements.
. invariant: The validity node must prove that the case condition in any trace is constant.
. exhaustive: The validity node assumes that at least one case condition is true at all times, and uses this to prove the proof targets.

If you do not specify a mode, the tool assumes the "exhaustive" mode for the validity check.

The results from the nodes are propagated to their parent, respecting propagation rules.

NOTE:

. The propagated results also depend on how the environments for the children and their parent relate and the status of the completeness check.
. Find additional information on propagation rules in the "Proof Structure" chapter of the Jasper Platform and Formal Property Verification App User Guide.

The command returns a dictionary composed of the operation name ("op_name"), the type of operation ("operation"), the parent node ("source_node"), and the implementation nodes ("imp_nodes"). Check the example section for more information.

WARNING: The Hard Case Split operation does not propagate related cover results.

. Use "-validity" to select a mode for the validity check node:
  . combinational - The validity node must prove that the logic is purely combinational with no state elements. There is no expression on the validity check property.
  . invariant - The validity node must prove that the case condition in any trace is constant.
  . exhaustive - The validity node assumes that at least one case condition is true at all times, and uses this to prove the proof targets.

  NOTE: If you do not specify a mode, the tool assumes the "exhaustive" mode for the validity check.

. Use "-op_name operation_name" to specify the operation name (name for the case split operation).

  NOTE: If you do not specify "-op_name", the tool automatically generates a name for the operation. The name is prefixed with "HCS_".

. Use "-imp_name node_names_list" to specify the names of the nodes the tool creates.

  NOTE:
  . If you do not specify "-imp_name", the tool automatically generates names for the nodes. The nodes are prefixed with the name of the operation node.
  . The name of the completeness check node is auto-generated. The name is prefixed with the operation name plus ".completeness_check_".
  . The name of the validity check node is auto-generated. The name is prefixed with the operation name plus ".validity_check_".

. Use "-from" to specify the node for the operation.

  NOTE: If you do not specify "-from", the tool uses the current task as the source node.

. Use "-condition" to specify the condition to be used in each case split node.

  NOTE: The conditions are required to be combinatorial Boolean expressions.

. Use "-property" to specify the target assertions that will be included in

```
      the case split operation.

      NOTE: If you do not specify "-property", all assertions from the source
      node are considered targets of the operation.

    . Use "-disable_operation_limit" to force the tool to create the operation
      with a number of implementation nodes greater than the global limit of
      100.

      NOTE: Creating more than 100 implementation nodes can make the GUI
      temporarily unresponsive.


-create compositional_assume_guarantee [-op_name operation_name]
  [-imp_name node_names_list]
  [-from node_name]
  [-property property_list]
  [-copy property_list]
  [-copy_related_covers]
  [-copy_related_asserts]
    Create a "Compositional Assume-Guarantee" operation using the node specified
    with "-from" as the source for the operation.
    This command creates a new node using the properties passed in "-property"
    as targets of the Compositional Assume-Guarantee. This operation uses an
    inductive method which, for any cycle N, assumes all the target properties
    are true up to bound N, and uses this to prove each target at bound N+1.
    The propagation algorithm ensures that only sound results are propagated
    up the structure.

    NOTE: The propagated results also depend on how the environments for the
    children and their parent relate and the status of all target properties.

    The command returns a dictionary composed of the operation name
    ("op_name"), the type of operation ("operation"), the parent node
    ("source_node"), and the implementation nodes ("imp_nodes"). Check the
    "Example" section for more information.

    WARNING: The Compositional Assume-Guarantee operation currently supports only
    SVA embedded safety properties. Tcl properties, liveness, auto-generated
    properties, PSL, deadlock, and complex-clock properties are not supported.

    . Use "-op_name operation_name" to specify the operation name (name for
      the case split operation).
```

NOTE: If you do not specify "-op_name", the tool automatically generates a name for the operation. The name is prefixed with "CAG_".

. Use "-imp_name node_names_list" to specify the names of the nodes the tool creates.

NOTE: If you do not specify "-imp_name", the tool automatically generates names for the nodes. The nodes are prefixed with the name of the operation node.

. Use "-from" to specify the node for the operation.

NOTE: If you do not specify "-from", the tool uses the current task as the source node.

. Use "-property" to specify the target assertions to be used in the operation.

NOTE: If you do not specify "-property", all assertions from the source node are considered targets of the operation.

. Use "-copy" to specify the extra properties to be included in the operation.
NOTE:
. Extra properties can be assertions and covers, and they are not subject to the same limitations as those specified with the "-property" switch.
. The extra properties specified with "-copy" exist in the same node as the target properties, and their result propagation follows the same rules as the target properties, but the status of the extra properties does not impact the propagation of the target properties.

. Use "-copy_related_covers" to copy all automatically extracted related covers for the properties that are being copied.

NOTE: For additional information about related covers, see "elaborate -create_related_covers" and the command reference manual appendix "Vacuity Checking with Automatically Extracted Covers."

. Use "-copy_related_asserts" to copy all automatically extracted related assertions for the properties that are being copied.

-create underconstraint [-op_name operation_name]

```
                    [-imp_name edit_node_name]
                    [-from node_name]
                    [-property property_list]
Create a node where just underconstraint operations can be performed using
the node specified with "-from" as the source for the operation.
The command creates one node with copies of the properties and proof
environment from the target node and allows changes to the created node. The
supported changes include:
. Inserting stopats and init value abstractions: use commands "stopat" and
  "abstract -init_value".
. Removing and clearing assumptions: use commands "assume -remove" and
  "assume -clear".
. Adding reset value abstractions and counter abstractions: use commands
  "abstract -reset_value" and "abstract -counter".
  NOTE: The switches "condition_expression", "expression" and "constant" are
  not allowed for the command "abstract -reset_value".

NOTE:
. The newly created node supports further editing as long as no
  operation is created from it.
. Find additional information on propagation rules in the "Proof
  Structure" chapter of the Jasper Platform and Formal Property
  Verification App User Guide.

The command returns the operation name and a dictionary with the parent
node and the name of the created node.

. Use "-op_name operation_name" to specify the operation name (name for
  the underconstraint operation).

  NOTE: If you do not specify "-op_name", the tool automatically generates
  a name for the operation. The name is prefixed with "U_".

. Use "-imp_name underconstraint_node_name" to specify the name of the node
  the tool creates.

  NOTE: If you do not specify "-imp_name", the tool automatically generates
  names for the nodes. The nodes are prefixed with the name of the
  operation node.

. Use "-from" to specify the source node for the operation.

  NOTE: If you do not specify "-from", the tool uses the current task as
```

```
      the source node.

    . Use "-property" to specify the target properties to be included in
      the underconstraint operation.

      NOTE: If you do not specify "-property", all properties from the source
      node are considered targets of the operation.

-create overconstraint [-op_name operation_name]
                       [-imp_name edit_node_name]
                       [-from node_name]
                       [-property property_list]
    Create a node where just overconstraint operations can be performed using
    the node specified with "-from" as the source for the operation.
    The command creates one node with copies of the properties and proof
    environment from the target node and allows changes to the created node. The
    supported changes include:
    . Inserting and converting assumptions: Use commands "assume" and "assume
      -from_assert".
    . Removing and clearing init value abstractions and stopats: use commands
      "stopat -remove", "abstract -init_value -remove", "stopat -clear" and
      "abstract -init_value -clear".
    . Removing reset value abstractions and counter abstractions: use commands
      "abstract -reset_value -remove" and "abstract -counter -remove".
      NOTE: If the switches "condition_expression", "expression" or "constant"
      were used when creating the reset value abstraction, these cannot be
      removed from the node.

    NOTE:
    . The newly created node supports further editing as long as no
      operation is created from it.
    . Find additional information on propagation rules in the "Proof
      Structure" chapter of the Jasper Platform and Formal Property
      Verification App User Guide.

    The command returns the operation name and a dictionary with the parent
    node and the name of the created node.

    . Use "-op_name operation_name" to specify the operation name (name for
      the partition operation).

      NOTE: If you do not specify "-op_name", the tool automatically generates
      a name for the operation. The name is prefixed with "E_".
```

```
    . Use "-imp_name overconstraint_node_name" to specify the name of the node
      the tool creates.

      NOTE: If you do not specify "-imp_name", the tool automatically generates
      names for the nodes. The nodes are prefixed with the name of the
      operation node.

    . Use "-from" to specify the source node for the operation.

      NOTE: If you do not specify "-from", the tool uses the current task as
      the source node.

    . Use "-property" to specify the target properties to be included in
      the overconstraint operation.

      NOTE: If you do not specify "-property", all properties from the source
      node are considered targets of the operation.

-get_property_info property [-list (status | min_length | max_length)+]
                            [-node node_name]
    Retrieve Proof Structure information for the specified property.
    The command returns a list of key/value pairs with information about the
    property.
    The "-list" switch returns a Tcl list of specified values where each
    element in the returned list is the value of the corresponding attribute
    in the attribute list.

    The Proof Structure information is a set of:
    . status: Current validity status of the property. Possible values:
              . proven
              . cex
              . covered
              . unreachable
              . undetermined

    . min_length: The minimal length of a possible trace for the property in
                  the form of a positive integer or the string "infinite". This
                  information tells you that shorter traces do not exist,
                  which is also known as the "bounded proof".

                  NOTE: This attribute has no meaning for liveness or infinite
                  properties.
```

```
    . max_length: If a trace exists, the shortest trace is no longer than
                  max_length. "infinite" indicates that no upper bound for a
                  possible minimal trace has been found. If a trace exists,
                  min_length is equal to max_length when the trace is known to
                  be minimal.

                  NOTE: This attribute has no meaning for liveness or infinite
                  properties.

    . Use "-node" to specify the scope for the property search.

-get_node_info node_name
    [-list (parent | children | class | type | propagation_status|
            condition | configuration)+]
    Retrieve information on a Proof Structure node.
    This command returns a dictionary that includes all available information
    for the node. Use the optional "-list" switch to return only the information
    requested. When using "-list" the return is a list with the requested
    information.

    . parent: Parent of the node.
      NOTE: If the node is a root node, the parent is the target task.

    . children: All the child nodes.
      NOTE: Since this return value is a list (even if it contains only 0 or
      1 elements), the overall return value of the "proof_structure
      -get_node_info" command becomes a list of lists.

    . class: Primary node classification
      Possible values:
      . root
      . operation
      . implementation

    . type: Specific node type
      Example of possible values:
      . Assume Guarantee
      . imp(assume)
      . imp(guarantee)
      . Partition
      . imp(partition #)
      . Stopat
```

```
      . imp(stopat)

    . propagation_status: Type of results that can be propagated from the node
      to its parent

    . condition: The local condition used to create the node if it is a Case
      Split implementation node.

    . configuration: The configuration used to create the node. You can use each
      element in the list as an argument to the "proof_structure" command
      to recreate the node configuration.

  NOTE: If you do not specify one or more list values, the tool returns a
  dictionary for all.

-get_node_list
    Returns a list of all Proof Structure nodes.

-set_visible_results (local | proof_structure)
    Set which results are visible on Proof Structure pane.

-propagate
    Propagate Proof Structure results from nodes to their parents.

-remove node_name
    Remove the specified node and all of its children from the Proof Structure.
    The command returns all the removed nodes.

    NOTE:
    . If you are removing a node from an Assume-Guarantee operation, this
      command removes all of the operation and its children.

    . It is not possible to remove Proof Structure nodes while a proof is
      running on the background.

-export node_name -create new_task_name
    Export the Proof Structure Node to a standard task.
    The command returns the name of the created task.

  . The specified "node_name" will be exported to the
    specified task.

  . Use "-create new_task_name" to specify the name of the created task.
```

```
-edit target_node_name
    -copy property_name_list
    -source_task source_task_name
    Edit the target proof structure node by copying properties from the source
    task.

    . Use "-copy" with a Tcl list to copy specified assertions or covers from
      the specified task.
    . Use "-source_task" to copy contents from the specified task instead of
      the current task.

    NOTE: The switch "-edit" is valid for any Proof Structure node that is an
    implementation node with no child nodes (no Proof Structure operation
    based on it).

-unify_results
    This command unifies Proof Structure results and local results by copying
    the propagated results from Proof Structure to the Property Table.
    After running the command, the "get_property_info" command will be able to
    report the results obtained from Proof Structure propagation.

    NOTE: With the 2025.12 release, the unification between Proof
    Structure results and local results happens automatically during
    propagation. Thus, this command is obsolete, and it will be removed
    in a future release.
```

Example

```
proof_structure -init ROOT -copy_all -from MY_TASK
ROOT

proof_structure -create assume_guarantee -from ROOT -property {p1 {p2 p3} p4 p5}
op_name AG_0 operation assume_guarantee source_node ROOT imp_nodes {AG_0.0 AG_0.1
AG_0.2 AG_0.3}

proof_structure -create assume_guarantee -from ROOT -property {p1 {p2 p3} p4 p5} -
op_name MY_AG
op_name MY_AG operation assume_guarantee source_node ROOT imp_nodes {MY_AG.0 MY_AG.1
MY_AG.2 MY_AG.3}

proof_structure -create assume_guarantee -from ROOT -property {p1 {p2 p3} p4 p5} -
```

```
imp_name {AG_1 AG_2 AG_3 AG_4} -op_name MY_AG2
op_name MY_AG2 operation assume_guarantee source_node ROOT imp_nodes {AG_1 AG_2 AG_3
AG_4}

proof_structure -create partition -from AG_2 -copy {{p2} {p3}}
op_name P_0 operation partition source_node AG_2 imp_nodes {P_0.0 P_0.1}

proof_structure -create partition -from AG_4 -num_copies 2
op_name P_1 operation partition source_node AG_4 imp_nodes {P_1.0 P_1.1}

catch {proof_structure -create partition -from AG_2 -copy {{p2} {p2}} -fail_if
duplicated_property assert}
1

catch {proof_structure -create partition -from AG_2 -copy {{p2} {p2}} -fail_if
missing_property assert}
1

proof_structure -create stopat -from AG_3 -add {a b}
op_name S_0 operation stopat source_node AG_3 imp_nodes S_0.0

proof_structure -create stopat -from S_0 -remove {a}
op_name S_1 operation stopat source_node S_0 imp_nodes S_1.0

proof_structure -create edit_node -from AG_3.0
op_name E_1 operation stopat source_node AG_3.0 imp_nodes E_1.0
# Further changes are allowed in E_1.0
assert -remove E_1.0::p1
assume $expr -name E_1.0::my_new_expr

proof_structure -create case_split -from ROOT -condition {{sig == 1'b0} {sig == 1'b1}}
op_name CS_0 operation case_split source_node ROOT imp_nodes {C_0.0 CS_0.1
CS_0.completeness_check_0}

proof_structure -create case_split -from ROOT -condition {{cnt == 0} {cnt > 0} {cnt <
64}} -property {p1 p2}
op_name CS_1 operation case_split source_node ROOT imp_nodes {C_1.0 CS_1.1 CS_1.2
CS_1.completeness_check_0}

# Specifying only properties accepted on Case Split
proof_structure -create case_split -condition {cond1 cond2} -property
[get_property_list -include {tcl 1 task ROOT} -exclude {type assume}]
op_name CS_2 operation case_split source_node ROOT imp_nodes {C_2.0 CS_2.1
```

```
CS_2.completeness_check_0}

proof_structure -create compositional_assume_guarantee -from ROOT -property {top.p1
top.p2}
op_name CAG_0 operation compositional_assume_guarantee source_node ROOT imp_nodes
{CAG_0.0}

proof_structure -create compositional_assume_guarantee -from ROOT -property {top.p1
top.p2} -copy {p3 c1}
op_name CAG_1 operation compositional_assume_guarantee source_node ROOT imp_nodes
{CAG_1.0}

proof_structure -get_property_info ROOT::p1
name ROOT::p1 status undetermined min_length 1 max_length infinite

proof_structure -get_property_info p1 -node ROOT -list status
undetermined

proof_structure -get_node_info ROOT -list children
{AG_0 MY_AG2 MY_AG}

proof_structure -get_node_info ROOT -list configuration
{-init ROOT -copy_all -from MY_TASK}

proof_structure -get_node_list
{AG_0 AG_0.0 AG_0.1 ROOT}

proof_structure -export AG_4 -create SOME_NEW_TASK
SOME_NEW_TASK

proof_structure -remove MY_AG
MY_AG.0 MY_AG.1 MY_AG.2 MY_AG MY_AG.3

proof_structure -unify_results
```

See Also

assert,
assume,
prove,
task,
set_proof_structure_support_embedded_on_case_split

proofmaster

Controls the ProofMaster settings and queries ProofMaster cache data.

Syntax

```
Configuring ProofMaster settings
----------------------------------------
proofmaster -config
             [-keep_traces (none | asserts | all | default)]
             [-cache_traces (none | all | default | asserts)]
             [-max_data_age time_limit]
             [-prove_cache_max_data_age time_limit]
             [-trace_replay_max_data_age time_limit]
             [-prove_cache_min_trace_time time_limit]
             [-trace_replay_min_trace_time time_limit]
             [-prove_cache_time_limit time_limit]
             [-max_dir_size size_limit]
             [-stagnation_mode (stop | default | bounds | off)]
             [-stagnation_trigger_type (relaxed | default | aggressive)]
             [-stagnation_trigger_max_time time]
             [-use_case (individual | full_signoff | bounds_signoff
               | bug_hunting) [-show] [-silent]]
             [-with_soft (none | asserts | covers | all | default)]
             [-trace_replay_cross_task (on | off)]
             [-list [switch_name] [-silent]]
             [-list_changed [-silent]]

Querying ProofMaster cache data
----------------------------------------
proofmaster -list
             [-type (all | prove_cache | trace_replay | ppd)]
             [-property [property_name]]
             [-cache_results property_name_list
              |-all_cache_results]
             [-first_match]
             [-file [file_name]]
             [-silent]
             [field_name...]
```

Returns

```
For "proofmaster -config": No return values.
For "proofmaster -list": Returns a Tcl dictionary with cache information for
the specified property or properties.
```

Default

```
-keep_traces  all
-cache_traces  all
-max_data_age and -prove_cache_max_data_age and -trace_replay_max_data_age 0
 (unlimited), by default, the tool never automatically deletes proof cache data
 and trace replay data.
-prove_cache_min_trace_time 0s
-trace_replay_min_trace_time 180s
-prove_cache_time_limit 0s
-max_dir_size 0 (unlimited)
-stagnation_mode default
-stagnation_trigger_type default
-stagnation_trigger_max_time 0s
-use_case individual
-trace_replay_cross_task off
```

Detailed Description

```
NOTE:
    . All configuration changes apply only to the current use case.
    . Each use case maintains its own independent set of configuration values.

-keep_traces (none | asserts | all | default)
    Use the "-keep_traces" switch to specify how you want proofs to control
    the overhead of generating and storing traces:
    . all | default: Find traces for both assertions and covers.
    . none: Disable trace generation during proofs to avoid the overhead
      of creating and storing traces. Properties still show CEX/covered
      status, but no traces are available for inspection.
    . asserts: Find traces for assertions only.

    NOTE:
    . When using "-keep_traces asserts," the tool finds a trace for an
      assertion. If the trace also includes a witness for a cover property,
```

```
      you can open the trace for the cover property despite specifying
      "-keep_traces asserts".
    . This setting does not affect the "visualize" command.

-cache_traces (none | all | default | asserts)
    Use the "-cache_traces" switch to control trace caching during proofs.
    . all | default: Enable caching of all traces.
    . none: Disable caching of all traces.
    . asserts: Enable caching of assertion traces only.

    NOTE: This setting applies to both prove cache and trace replay traces.

-max_data_age time_limit
-prove_cache_max_data_age time_limit
-trace_replay_max_data_age time_limit
    Use these commands to automatically delete unused proof cache data
    or trace replay data from the ProofMaster directory after N days.
    This feature prevents proof cache data or trace replay data from
    growing indefinitely, resulting in faster operations and smaller
    ProofMaster databases.

    NOTE: Using "-max_data_age" sets both "-prove_cache_max_data_age" and
          "-trace_replay_max_data_age" to the same value.

-prove_cache_min_trace_time time_limit
    Write prove cache trace files that took longer than the specified time
    using "time_limit" to generate.
    The default value for "time_limit" is zero. If it is set to zero,
    all trace files are cached.

-trace_replay_min_trace_time time_limit
    Write trace replay trace files that took longer than the specified time
    using "time_limit" to generate.
    The default value for "time_limit" is 180s, if it is set to zero,
    all trace files will be cached.

    NOTE: This is the only ProofMaster configuration supported in SEC
    for the time being.

-prove_cache_time_limit time_limit
    Use this command to set the time limit of property signatures computation in
    proof caching.
```

```
    NOTE:
    . The tool re-starts the time_limit with each new task.
    . To remove the limit, use 0s.

-max_dir_size size_limit
    Use this command to set the maximum disk space limit for the ProofMaster
    directory to manage cache storage efficiently. The "size_limit" can be
    specified in KB, MB, GB, or TB.
    When the cache reaches its set limit, ProofMaster follows these rules:
    . It does not store new cache data until the existing data decreases.
    . Cache types are prioritized in order: PPD, trace replay, and proof cache.

    NOTE:
    . The default value is 0 (unlimited cache size).
    . Numeric values without units are interpreted as KB (e.g., 1000 = 1000KB).
    . All size units use binary calculations (1K = 1024 bytes, 1M = 1048576 bytes).
    . Use "-max_data_age" in conjunction with this option to automatically
      delete old data and create an eviction policy.
    . Initial directory data does not respect the cache limit.
    . Cache data can be stored partially (some components without others).

-stagnation_mode (stop | default | bounds | off)
    Use "-stagnation_mode" to specify the type of stagnation algorithm to use
    when the tool reaches stagnation.
    . stop: Stop the current proof completely.
    . default: Modify engine selection.
    . bounds: Optimize for extending bounded proofs.
    . off: Continue with proof effort.

-stagnation_trigger_type (relaxed | default | aggressive)
    Use the "-stagnation_trigger_type" switch to control how soon ProofMaster
    decides that a proof has stagnated.
    . relaxed: Trigger stagnation more slowly.
    . default: Trigger stagnation at a normal rate.
    . aggressive: Trigger stagnation more quickly.

-stagnation_trigger_max_time time
    Use this switch to specify the maximum time ProofMaster should wait after
    starting a run to trigger stagnation.

    NOTE:
    . The default value is 0s (disabled). In this case, the stagnation trigger
      point is affected solely by the value of "stagnation_trigger_type".
```

```
    . The value specified for "stagnation_trigger_max_time" must be smaller
      than the value specified for "prove_time_limit".

-use_case (individual | full_signoff | bounds_signoff
            | bug_hunting) [-show] [-silent]
    Use this switch to specify the ProofMaster use case to use.
    . individual: Intended for use during individual development of new proofs.
      This default use case is tuned similarly to standard orchestration proofs.
    . full_signoff: Intended for late-stage project regressions, prioritizes
      proofs of assertions and of precondition covers.

      NOTE:
      . The tool automatically overrides the "-keep_traces" and
        "-cache_traces" settings to "asserts" if they are not explicitly set
        with another value by you when defining the "full_signoff" use case.
      . The tool automatically overrides the "-stagnation_trigger_type" setting
        to "aggressive" if it is not explicitly set with another value by you
        when defining the "full_signoff" use case.

    . bounds_signoff: Intended for use in late-stage project regressions
      where achieving full proofs for all properties is no longer feasible.
      The intent is to sign off with some bounded proofs by focusing on a subset
      of proof engines (such as parallel SAT) that are best at extending bounds.
      This works on covers as well as assertions to minimize vacuity dangers.

      NOTE:
      . The tool automatically overrides the "-keep_traces" and
        "-cache_traces" settings to "asserts" if they are not explicitly set
        with another value by you when defining the "bounds_signoff" use case.
      . The tool automatically overrides the "-stagnation_trigger_type" setting
        to "aggressive" if it is not explicitly set with another value by you
        when defining the "bounds_signoff" use case.
      . The tool automatically overrides the "-stagnation_mode" setting to
        "bounds" if it is not explicitly set with another value by you
        when defining the "bounds_signoff" use case.

    . bug_hunting: Intended for use in regular regressions, prioritizes
      finding counterexamples for failing assertions.

    NOTE:
    . Use "-show" to return a Tcl dictionary with the ProofMaster
      configuration used for the use case without executing it.
    . Use "-silent" to suppress the informational message about the changed
```

```
      ProofMaster configuration.
    . When using "bug_hunting" or "full_signoff" use cases, the engine mode
      automatically overrides to "auto" if it is not already set.

-with_soft (none | asserts | covers | all | default)
    Use this switch to control whether proofs attempt to apply any available
    user-created soft constraints, and on which property types.
    . all: Apply soft constraints on all properties.
    . covers: Apply soft constraints on cover properties only.
    . asserts: Apply soft constraints on assertion properties only.
    . none: Do not apply soft constraints.
    . default: Use the default behavior (same as "all").

    This controls the "prove_with_soft_mode" setting. See
    "set_prove_with_soft_mode".

-trace_replay_cross_task (on | off)
    Lets trace replay use traces from other tasks when proving the same
    property. When proving a property in a new task, trace replay normally
    cannot access traces generated in other tasks, even for the same
    property. This option removes that restriction, allowing trace replay to
    reuse proof traces from any task. This can improve verification
    efficiency by leveraging previous proof work rather than starting from
    scratch each time.

-list [switch_name] [-silent]
    Use this command to get ProofMaster configuration settings values.
    If "switch_name" is not specified:
    . Displays all switches along with their current values.
    If "switch_name" is specified:
    . For each switch in the list, displays the most recently set value via
      "proofmaster -config".
    . If no value has been explicitly set, it displays the default value.

    NOTE: Use the "-silent" switch to suppress the pretty-printed output

-list_changed [-silent]
    Use this command to display all switches that have been updated from their
    default values via "proofmaster -config", along with their current values.

    NOTE: Use the "-silent" switch to suppress the pretty-printed output
```

```
Querying ProofMaster cache data (proofmaster -list)
---------------------------------------------------
Use "proofmaster -list" to query information about ProofMaster cache data
for properties. This subcommand lets you inspect what data ProofMaster
has cached, including prove cache, trace replay, and PPD data.

-type (all | prove_cache | trace_replay | ppd)
    Specify the type of cache data to query.
    . all: Query all cache types (default).
    . prove_cache: Query only prove cache data.
    . trace_replay: Query only trace replay data.
    . ppd: Query only PPD data.

-property [property_name]
    Query cache data for the specified property. When used with
    "-cache_results" or "-all_cache_results", performs cache validation
    instead of a standard query.

-cache_results property_name_list
    Perform cache validation for the specified property against the listed
    properties. The tool checks whether the cache data for the listed
    properties is compatible with the specified property (given via
    "-property"). The result is a Tcl dictionary indicating how many
    properties matched and which ones.

-all_cache_results
    Perform cache validation for the specified property (given via
    "-property") against all properties stored in ProofMaster. This is
    mutually exclusive with "-cache_results".

-first_match
    When performing cache validation with "-cache_results" or
    "-all_cache_results", stop after the first matching property is found.

    NOTE: This option is only supported with "-type prove_cache".

-file [file_name]
    Write the results to a file instead of returning them. If no file name
    is specified, the tool writes to "proofmaster_results.tcl".

-silent
    Suppress the output to the screen.
```

```
field_name...
    Optionally, specify one or more field names to include in the result.
    The valid fields are "has_data," "size," and "creation timestamp." If
    no fields are specified, it returns all available information.
```

Example

```
set_proofmaster on
proofmaster -config -keep_traces none
proofmaster -config -cache_traces none
proofmaster -config -max_data_age 10 -trace_replay_min_trace_time 60s
proofmaster -config -prove_cache_time_limit 2m
proofmaster -config -max_dir_size 500M
proofmaster -config -with_soft all
proofmaster -config -with_soft covers
proofmaster -config -stagnation_mode bounds
proofmaster -config -stagnation_trigger_type aggressive
proofmaster -config -use_case individual -show
proofmaster -config -list
proofmaster -config -list {keep_traces cache_traces}
proofmaster -config -list_changed

# Query ProofMaster cache data for a specific property
proofmaster -list -property {my_task::my_property} -type prove_cache

# Query cache data and return only specific fields
proofmaster -list -property {my_task::my_property} "has_data" "size"

# Validate cache compatibility for a property against all cached properties
proofmaster -list -property {my_task::my_property} -all_cache_results

# Validate cache compatibility against specific properties, stop on first match
proofmaster -list -type prove_cache -property {my_task::my_property} -cache_results
{task::propA task::propB} -first_match

# Write results to a file
proofmaster -list -property {my_task::my_property} -file my_results.tcl
```

property_triage

Groups similar properties with CEX traces into buckets.

Syntax

```
property_triage -run
                [-effort (low | medium | high)]
                [-property property_name_list | -task task_name]
                [-silent]
                [-file filename [ -force]]

property_triage -result
                [-silent]
                [-file filename [ -force]]
```

Returns

```
-run, -result:
  Returns a Tcl dictionary with the clustering results. Each bucket is
  identified by a sequential number, the number of properties in that group,
  and a list of the property names.
```

Detailed Description

```
Use this command to group assertions properties with similarities using
clustering algorithms. The command provides options to run the triage and export
results, and retrieve the last clustering result.

NOTE: "property_triage" does not support FSV properties.

-run [-effort (low | medium | high)]
      [-property property_name_list | -task task_name]
      [-silent]
      [-file filename [-force]]
    Runs property triage to group properties in a task or a specified list.
    . Use "-effort" to define the extent of information to extract and process
      to identify similarities between properties. If you do not specify an
      effort, the tool uses the effort "medium" by default. Each piece of
      extracted information is assigned a weight based on its impact on the
      relationship between the properties. By using more information, the tool
      can identify similar features with greater flexibility when comparing
      information from two properties.
      The following modes are available:
      . low: uses basic property and trace information and is the fastest mode
        to identify groups. This level of effort creates a larger number of
        groups.
      . medium: includes trace analysis and more property information in the
        clustering process. This level of effort is less restricted when
        comparing properties.
      . high: includes further trace and property analysis. This mode is less
        strict when comparing properties and produces fewer groups.
    . Use "-property" to apply the command to the specified properties.
    . Use "-task" to apply the command to the specified task instead of the
      current task.
    . Use "-silent" to turn off the output to the screen and return information
      with a Tcl return value.
    . Use "-file" to dump the clustering results to a .csv file. The
      resulting file lists the cluster ID for each property name.
    . Use "-force" to overwrite an existing file.

-result [-silent] [-file filename [-force]]
    Displays or dumps the results of the most recent triage clustering.
    . Use "-silent" to suppress console output and return only the Tcl result.
    . Use "-file" to dump the last clustering results to a .csv file.
    . Use "-force" to overwrite an existing file.
```

Example

```
property_triage -run
property_triage -run -effort low
property_triage -run -effort high -task task_a
property_triage -run -property {prop1 prop2 prop3} -file triage.csv -force
property_triage -result
property_triage -result -file triage.csv -silent
```

See Also

get_property_info,
visualize,

prove

Proves properties, or controls a "prove" command running in the background.

Associated App(s)

Depending on the property to be proven.

Syntax

```
prove ( -all [-cex_limit number_of_failures]
       |-task [task_name_tcl_list]
         [-cex_limit number_of_failures]
       |-property property_name_tcl_list [-regexp]
         [-evaluate_in_trace  property_name_tcl_list]
         [-cex_limit number_of_failures]
       |-instance instance_name
         [-except subinstance_name+]
       | expression
      )
      [-from property_name [-trace_id N]
        [-cycle N]]
      [-on_determined proc ]
      [-with_helpers]
      [-with_proven]
      [-bg]
      [-force]
      [-asserts | -covers]
      [-quiet]
      [-soft_constraints signal_name_tcl_list [-extend_region]]
      [-soft_low_constraints signal_name_tcl_list [-extend_region]]
      [-engine_mode engine_mode_tcl_list]
      [-iter N]
      [-first_trace_attempt N]
      [-per_property_max_time_limit time_limit]
      [-per_property_time_limit time_limit]
      [-per_property_time_limit_factor N]
      [-time_limit time_limit]
      [-verbosity N |-silent]
```

```
      [-per_engine_max_jobs N]
      [-max_jobs N]
      [-max_partitions N]
      [-no_traces]
      [-no_cover_traces]
      [-prefer_shortest]
      [-prefer_quiet]
      [-with_soft]
      [-simplification_only]
      [-dry_run]
      [-dump_trace
       [-dump_trace_type (shm | vcd | fsdb)]
       [-dump_trace_dir dir_name]]
      [-with_ppd
       [path_to_ppd_file]]
      [-save_ppd
       [path_to_ppd_file]]
      [-orchestration (on | off)]
      [-sst [N]]
      [-tag tag]

When a proof is running in the background:

prove ( -stop [[index.]engine_mode]
       |-kill index.engine_mode
       |-wait
       |-status [-silent]
       |-per_engine_max_jobs N
       |-add engine_mode
       |-restart [index.]engine_mode
       |-ignore_property property_name_tcl_list
      )
      [-thread thread_id | -tag tag]
```

Returns

When proving properties without "-bg" these are the return values:

Return Value	Definition
proven, covered	All properties have the same determined status.
cex, unreachable	The return is one of the values listed.

ar_covered, ar_cex	
determined	All properties are fully determined, but they have different statuses ("proven", "cex", "unreachable", "covered").
determined_or_skipped	Some properties were skipped (are ignored by the selected proof setting) or left undetermined. See "prove -from". All other properties are fully determined.
ar_determined	All properties are determined or "ar_covered" or "ar_cex", and at least one property is "ar_covered" or "ar_cex".
overconstrained	At least one task has inconsistent assumptions or clock and reset.
unprocessed	At least one property was left unprocessed.
per_property_time_limit	
time_limit	time_limit or per_property_time_limit has expired.
max_trace_length	max_trace_length (iter) is reached.
stopped_by_user	The prove command was stopped by the user.
out_of_memory	At least one engine ran out of memory.
internal_limit	Internal limit exceeded.
engineJ_attempts	Engine J failed to satisfy all constraints, see "set_engineJ_attempts", "set_engineJ_restarts".
no_properties	The given selection contained no properties.
background	The prove command is now running in the background. NOTE: You can use the Application menu to put a foreground proof into the background (Application - Proof - Run Job in Background). When you do this, the tool returns "background" to the console. The

```
                  Application menus of the following Apps do not have
                  this option: CDC, Coverage, FSV, SPV, Superlint
-----------------  ------------------------------------------------
background thread_id
                  The "prove" command is running in the background
                  and is associated with the specified "thread_id".
-----------------  ------------------------------------------------
aborted           The prove command stopped due to a ProofGrid error.
-----------------  ------------------------------------------------
failed            The prove command stopped due to an error.
-----------------  ------------------------------------------------
marked_proven     The property was created with "assert -mark_proven".
```

Detailed Description

Use this command to determine if assertions are "proven" or "cex" (or "ar_cex") and cover properties are "unreachable" or "covered" (or "ar_covered").

Important information about this command:

- "prove" does not support FSV. The tool issues an error in this case. Use "check_fsv -prove" instead.
- "prove" does not support SPV or X-Prop properties. The tool issues a warning in these cases and ignores the SPV or X-Prop properties respectively. Use "check_spv -prove" or "check_xprop -prove" instead.
- When a property result becomes "cex", "ar_cex", "covered", or "ar_covered", this command generates a trace "triggering" the property. See "visualize".
- The main switches control which properties to prove. Properties that are disabled are accepted but ignored. See "assert -disable", "cover -disable".
- A number of commands, most notably "set_engine_mode" and "set_proofgrid_mode" control _how_ property results are determined.
- By default, all engines leverage proven properties as assumptions for the proof of other properties proven together.
- You can monitor the proof progress by watching the Properties pane, Console tab, Proof Messages tab, and the ProofGrid Manager window.
- Also use this command to control a "prove" command running in the background (see "prove -bg").

NOTE: Determining property results can consume considerable time and memory.

-all [-cex_limit number_of_failures]

```
    Prove all assertions and cover properties in all tasks.
    Use "-all" with "-cex_limit" to specify that you want the prove
    flow to stop the proof job after it reaches a certain threshold of
    failures on FPV properties. If the prove flow stops on cex_limit,
    it returns "determined_or_skipped".
    NOTE: "prove -all" issues an error message if used in a environment
    with Proof Structure nodes as it includes nodes that might not require
    proof. In such cases, do one of the following:
    . Use "prove -task" and specify the tasks/nodes instead
    . Use the utility from the Tcl library to prove necessary nodes
    (::jasper::psu::prove_all or ::jasper::psu::prove_all_serial)
    . Use "set_message -warning EPST052" to downgrade the message to a warning

-task [task_name_tcl_list] [-cex_limit number_of_failures]
    Prove all assertions and cover properties in the specified tasks.
    Use "-task" with "-cex_limit" to specify that you want the prove
    flow to stop the proof job after it reaches a certain threshold of
    failures on FPV properties. If the prove flow stops on cex_limit,
    it returns "determined_or_skipped".

    NOTE:
    . If you do not supply a task name, the tool proves all assertions and
      covers in the current task.
    . When specifying a list of task names, enclose the list in curly braces.
    . This switch supports wildcards (* and ?).

-property property_name_tcl_list [-regexp]
  [-evaluate_in_trace property_name_tcl_list]
  [-cex_limit number_of_failures]
    Prove the specified properties.
    Use "-property" with "-cex_limit" to specify that you want the prove
    flow to stop the proof job after it reaches a certain threshold of
    failures on FPV properties. If the prove flow stops on cex_limit,
    it returns "determined_or_skipped".

    NOTE:
    . To get full control over which proven properties are used as
      assumptions, use the default "set_proven_directive true", do not use
      "-with_proven" (see definition below), and do include the proven and
      unreachable properties you want to use in the list of properties.
      In this case, the tool uses only the proven and unreachable properties
      that are in your list as assumptions.
    . To prove properties not in the current task prepend the property
```

```
      names with "<task>::".
    . All listed properties must belong to the same task.
    . This switch supports wildcards and regular expressions; for example:
      prove -property {expression} -regexp
    . Use "-evaluate_in_trace" to include a set of properties for the tool to
      evaluate when it finds a cex or a cover for one or more specified target
      properties. In general, the tool discovers the status of these evaluated
      properties by checking the traces found for the primary target
      properties. That is, the formal proof does not explicitly target them.

-instance instance_name [-except subinstance_name+]
    Prove all assertions and cover properties in the specified module
    instance and its associated subinstances, except the specified
    subinstances. The module subinstance names are relative to the
    module instance name; that is, you will not specify a full path.

    NOTE: Embedded properties, which are declared in the RTL, belong to a
    specific instance, and the tool extracts them to the task <embedded> during
    elaboration. For this reason, "prove -instance" only applies to
    properties that belong to the <embedded> task, which must be the
    current task ("task -set <embedded>").

expression
    If an assertion with the specified expression already exists, prove that;
    otherwise, use the same behavior as "prove -property [assert expression]".
    See "assert".

-from property_name [-trace_id N] [-cycle N]
    Use an existing trace as the initial value for another formal proof.

    . Use "-trace_id" to specify a trace other than the default. The default
      property trace is the last shortest trace. See "assert/cover
      -set_store_trace" for the commands to configure the  number of traces
      generated and stored by the tool.

    . Use "-cycle" to specify the state of the trace at a specific cycle as the
      reset value. If you do not specify a cycle, or if you specify 0, the tool
      uses the cycle in which the property was triggered. This is often, but
      not always, the last cycle of the trace. See "set_trace_extension".

      If you specify $, or if you specify -1, the tool uses the last cycle of
      the existing trace. A negative N selects the cycle relative to the end
      of the trace.
```

```
    Use this feature when a formal proof finds an interesting scenario (for
    example, a counterexample or complex cover) and you want to perform another
    search from that point. By doing so, you might find scenarios that are
    deeper than what you could reach with other formal proofs.

    NOTE:
    . This is a bug-hunting feature. The tool is not able to prove assertions
      or determine that covers are unreachable when you use "prove -from".
    . Found traces will start from reset and include the trace to the
      specified property as a prefix.
    . If undetermined properties are triggered in the existing trace, the trace
      will be used as the result for those properties. (The first time a
      property is triggered along a trace is the most significant event.)

-on_determined proc
    Override the current "on_determined" for this "prove"
    command. See "set_prove_on_determined".

-with_helpers
    Prove with helper assertions.

    You can prioritize helper assertions over regular assertions by submitting
    them first to the engines with "assert -helper...".

-with_proven
    Use all proven assertions in the same task as assumptions.

    This switch offers a convenient way to specify that you want the tool to
    use all proven assertions as assumptions without having to include them as
    part of the target property list and to rely on the "set_proven_directive"
    command.

-bg
    Run the "prove" command in the background and continue to accept
    and run new commands.

-force
    Clear existing proof results for the selected properties before
    beginning. See "clear -result".

-asserts
    Prove only assertions, not covers.
```

```
-covers
    Prove only covers, not assertions.

-quiet
    Use all undriven signals as extra soft low constraints. This is a
    complement to "assume -soft", "prove -soft_constraints", and "prove
    -soft_low_constraints" that offers more convenience but less control.
    Jasper only attempts to apply soft constraints in the current proof
    if "set_prove_prefer_quiet on" was executed, "-prefer_quiet" was added
    to the "prove" command, or engine QT was added to the engine list.

-soft_constraints signal_name_tcl_list [-extend_region]
    Use the selected signals (those matching any of the list of
    wildcards) as extra soft constraints.

    Important things to know:
    . Use this switch for signals you prefer to be high in traces, see "assume
      -soft".
    . Use "-extend_region" to prevent the tool from ignoring out-of-region
      signals and issuing a warning.
    . This switch supports wildcards (* and ?) in signal names.
    . This switch supports Tcl lists as arguments.

-soft_low_constraints signal_name_tcl_list [-extend_region]
    Use the selected signals (those matching any of the list of
    wildcards) as extra soft low constraints.

    Important things to know:
    . Use this switch for signals you prefer to be low in traces, see "assume
      -soft".
    . Use "-extend_region" to prevent the tool from ignoring out-of-region
      signals and issuing a warning.
    . This switch supports wildcards (* and ?) in signal names.
    . This switch supports Tcl lists as arguments.

-engine_mode engine_mode_tcl_list
    Override the current "engine_mode" for this "prove"
    command. See "set_engine_mode".

-iter N
    Override the current "max_trace_length" for this "prove"
    command. See "set_max_trace_length".
```

```
    NOTE: You can also use this command to specify a "bounded proof", that is,
    a proof that terminates after reaching the specified iteration.

-first_trace_attempt
    Override the current "first_trace_attempt" for this "prove"
    command. See "set_first_trace_attempt".

-per_property_max_time_limit time_limit
    Override the current "prove_per_property_max_time_limit" for this
    "prove" command. See "set_prove_per_property_max_time_limit".

-per_property_time_limit time_limit
    Override the current "prove_per_property_time_limit" for this
    "prove" command. See "set_prove_per_property_time_limit".

-per_property_time_limit_factor N
    Override the current "prove_per_property_time_limit_factor" for this
    "prove" command. See "set_prove_per_property_time_limit_factor".

-time_limit time_limit
    Override the current "prove_time_limit" for this "prove"
    command. See "set_prove_time_limit".

-verbosity N
    Override the current "prove_verbosity" for this "prove"
    command. See "set_prove_verbosity".

-silent
    Alias for -verbosity 0.

-per_engine_max_jobs N
    Run the specified number of engine_jobs per engine.

    NOTE: This switch overrides proofgrid_per_engine_max_jobs and
    proofgrid_per_engine_max_local_jobs. See
    "set_proofgrid_per_engine_max_jobs" and
    "set_proofgrid_per_engine_max_local_jobs".

-max_jobs N
    Run the specified number of engine_jobs.

    NOTE: This switch overrides "proofgrid_max_jobs" and
```

```
    "proofgrid_max_local_jobs". See "set_proofgrid_max_jobs" and
    "set_proofgrid_max_local_jobs".

-max_partitions N
    Run the specified number of partitions when orchestration is off.

    NOTE: This switch overrides proofgrid_max_partitions. See
    "set_proofgrid_max_partitions".

-no_traces
-no_cover_traces
    Do not generate any traces or cover traces specifically.

    Use this switch to avoid the overhead of generating and storing traces.
    The properties will show cex/covered statuses as usual, but there will be
    no traces to inspect. You must re-do the proof to get traces. Use this
    switch when the trace data is not important.

    NOTE: If the tool finds a trace for an assertion, and the trace also
    contains a witness for a cover property, you will be able to open the trace
    for the cover property despite specifying "-no_cover_traces".

-prefer_shortest
    Override the current "prove_prefer_shortest" for this "prove"
    command. See "set_prove_prefer_shortest".

-prefer_quiet
    Override the current "prove_prefer_quiet" for this "prove"
    command. See "set_prove_prefer_quiet".

-with_soft
    Override the current "prove_with_soft" for this "prove"
    command. See "set_prove_with_soft".

-simplification_only
    Run only preprocessing and simplification without invoking the regular
    formal engines.

    NOTE:
    . You can use this switch heuristically, in parallel with the regular
      engines, to determine properties that are trivial for simplification.
    . "prove -simplification_only" and "prove -engine_mode" are mutually
      exclusive (within the same command).
```

```
    . "prove -simplification_only" overrides the current "proof_simplification"
      setting. See "set_proof_simplification".
    . "prove -simplification_only" restores results from the proof cache if
      "prove_cache" is on. You can prevent this behavior by running
      "set_prove_cache off". (The default setting is "off".)

-dry_run
    Compute an upper bound on the number of engine jobs that could be
    launched by the proof if it was run.

    The return value is a list of name/value pairs of the bound of the total
    number of jobs as well as a bound for each engine in the proof. The return
    is suitable for use with Tcl "dict".

    NOTE: This feature does not work with proof orchestration. See
    "set_prove_orchestration".

-dump_trace
    [-dump_trace_type (shm | vcd | fsdb)]
    [-dump_trace_dir dir_name]
    Dump traces to the disk as soon as the engines find them. The tool uses
    the SHM format as the default and writes files in the project directory.

    . Use "-dump_trace_type" with "shm", "vcd", or "fsdb" to
      specify the file dump type. VCD files are dumped as compressed files to
      save disk space.

      IMPORTANT:
      . Jasper Apps writes 6.3 FSDB traces and reads version 6.6 FSDB
        traces. FSDB Reader 6.6 was introduced by Verdi version X-2025.06.
      . FSDBs you generate with the latest Jasper are backwards compatible
        with all Jasper versions from 2025.03 FCS and newer. In addition,
        they are backwards compatible with older versions of Verdi released
        during that same period.
      . Jasper Apps supports backwards compatibility, that is, the tool can
        read FSDBs generated from older versions of Jasper Apps and
        Synopsys Verdi.

    . Use "-dump_trace_dir" to specify the dump directory where the files will
      be written.

    NOTE: If you have also used the set_trace_optimization command, the trace
    file reflects those settings, for example:
```

```
    . If trace optimization is ar, then only the analysis region is saved into
      the file. This option is the default.
    . If you use set_trace_optimization standard, then the entire design is
      dumped into the file. However, you should be aware that this setting can
      increase the time it takes to generate a trace.

-with_ppd
    [path_to_ppd_file]
    Read proof profiling data (PPD) previously saved with "-save_ppd".
    The tool uses this information to make future runs faster and consume fewer
    resources.

    NOTE:
    . If you use "-with_ppd" without the path_to_ppd_file argument, the tool
      uses the "sessionPPD.ppd" file in the session directory.
    . If you use "-with_ppd", and the ppd_file_name contains information from
      multiple runs, the tool attempts to find the most relevant information
      for the running proof, considering information from all previous runs.
    . If you use "-with_ppd" and Proof Orchestration is disabled, the tool
      enables it and prints a warning. However, if you explicitly disable Proof
      Orchestration, the tool issues an error.

-save_ppd
    [path_to_ppd_file]
    Save the PPD for use with future runs to potentially make them run faster
    and consume fewer resources.

    If you use "-save_ppd" without the path_to_ppd_file argument, the tool
    writes the information to a default PPD file named "sessionPPD.ppd" in the
    session directory.

    NOTE:
    . If you use "-save_ppd", and the ppd_file_name is not empty, the tool
      appends new information to the existing information.
    . If you use "-save_ppd" and Proof Orchestration is disabled, the tool
      enables it and prints a warning. However, if you explicitly disable Proof
      Orchestration, the tool issues an error.

-orchestration (on | off)
    Override the current "prove_orchestration" setting for this "prove"
    command. See "set_prove_orchestration".

-sst [N]
```

Assuming that all enabled helper- and sst-properties in the task pass the first few cycles, run the proof on the helpers without reset and without bounded assumptions (see "assume -bound"). Properties are allowed to fail at the cycle specified by "N", which represents the minimum trace length. If "N" is not provided, properties are allowed to fail starting at the cycle specified by the "set_sst_default_trace_length" command.

For properties with a trace extension, properties are allowed to fail earlier in the SST trace (see "assert -set_trace_extension" and "set_trace_extension"). This means that such properties are required to pass for fewer cycles. In particular, if the extension exceeds the SST trace length, the property is allowed to fail anywhere and does not effectively constrain the trace.

In all cases, the traces generated might be longer than the minimum specified.

If a trace is found for a property, it is delivered as an SST trace, and the property remains undetermined. By inspecting the SST trace and basing another helper assertion on it, you might learn the different failure modes on rerunning the proof. This could result in no trace possible.
Use "visualize -violation -sst" to open an SST trace generated by the "prove -sst" command in Visualize.

Resets are kept as soft constraints, so with "-prefer_quiet", SST traces starting close to reset are generated.

If a proof is found, it is reported as an "sst_max_length" bound, see "get_property_info -list sst_max_length". If all properties have a sufficient "min_length" bound, a full proven result is established. If only a single property is being assumed, then a normal single property "max_length" bound is reported. If an "sst_max_length" is achieved for all helper properties then a full proof can be achieved by first running a normal non-sst proof establishing "min_length" bounds surpassing the "sst_max_length", and then finally run another "prove -sst" drawing the final conclusion. The final proof is trivial.

NOTE:
. Properties can be made helper properties using "assert -set_helper".
. Selected properties that are not already helpers become sst-properties.
. The cost and effectiveness of a SST proof depend on "set_sst_default_trace_length" and the full set of enabled helper properties in the task. You can explicitly include helper properties

```
      using one of the following:
        . "-with_helpers" switch
        . "-property <property name>"
        . "-with_proven" switch
        . "-task" or "-all"
    . Helper and sst properties are always included in the proof, but will play
      an active or passive role depending on whether they are explicitly included
      in the property list or included using "with_helpers". The roles:
        . Active: The property is assumed in the first few cycles, but also
          proven.
        . Passive: The property is assumed in the first few cycles, but then
          evaluated in trace.
    . SST proofs do not support formal reset.

     See also "get_property_info", "get_property_list", "visualize -violation",
     "set_sst_default_trace_length", "prove -with_helpers",
     "prove -with_proven", and "prove -evaluate_in_trace".

-tag tag
    Attach the tag to the proof you are launching. You can later use this tag
    to reference the proof threads created by this command. For example, use
    "prove -status -tag," "prove -stop -tag," "prove -kill -tag," or
    "prove -restart -tag." You can also use "get_run_info -list tag" to list
    all tags of the currently running threads.

    NOTE:
    . Each tag must be unique. The tool issues an error if another proof is
      already using the specified tag.
    . "-tag" and "-thread" are mutually exclusive.

-stop [engine_mode | index.engine_mode] [-thread thread_id | -tag tag]
    Stop the entire "prove" command running in the background or a single
    engine_mode or engine_job (index.engine_mode).

    Use "-thread" to stop a proof thread or an engine_mode or engine_job that
    is associated with a specified thread.
    Use "-tag" to stop all proof threads associated with the specified tag.

    NOTE:
    . "-stop" is not blocking and might return before the proof has actually
      stopped. If you want blocking behavior, use "prove -stop; prove -wait",
      see below.
    . "prove -stop" stops all running proof threads.
```

```
    . Stopping all engines in a proof is equivalent to stopping the proof.

-kill index.engine_mode [-thread thread_id | -tag tag]
    Kill the proofgrid_shell process for the specified engine job.

    Use "-thread" to kill a proof thread or an engine_mode or engine_job that
    is associated with a specified thread.
    Use "-tag" to kill engine jobs in all proof threads for the specified
    tag.

    NOTE: Depending on the nature of the proofgrid_shell you are using (see
    "set_proofgrid_shell"), killing a job can have drawbacks. Therefore, you
    should only use this command as a last resort when
    "-stop index.engine_mode" does not appear to be working.

-wait [-thread thread_id]
    Wait for the currently running "prove" command to complete before running
    additional commands.

    Use "-thread" to tell the tool to wait for the "prove" command running
    under the specified thread to complete before running additional commands.

    NOTE:
    . "-wait" without "-thread" waits for all running background proofs to
      complete.
    . "-wait -thread" waits for the specified thread to complete.

    This option is useful when "prove" is running in the background (-bg). In
    effect, it moves "prove" back to the foreground, and the tool accepts no
    additional commands until the proof completes.

    "prove -wait" returns the value the "prove" command would have
    returned if it had run without "-bg" (see above). It returns
    this value even if the previously started proof has already
    stopped or never ran in the background.

-status [-thread thread_id | -tag tag] [-silent]
    List the running engine jobs and their statuses.

    . Use "-thread" to list the status of a specified thread's running engine
      jobs.
    . Use "-tag" to list the status of all proof threads for the specified
      tag.
```

```
    . Use "-silent" to turn off the output to the screen and return information
      with a Tcl return value, which is helpful in Tcl scripts.

    The following list shows prove statuses.
    . "NeverStarted": Nothing has run in this session.
    . "Pending": There is currently a prove about to start.
    . "Simplify": A prove is currently running proof simplification.
    . "Running engineJob*": A prove is running with the listed number of
      engineJobs currently running.
    . "Running threadStatus*": A prove is running with the listed threads, where
      threadStatus is the status of a thread in the list.
    . "Finishing": Job is about to finish. Interactive commands sent to this
      job are ignored.
    . "Completed": No job is currently running. The last prove has completed.
    . "licenseStarved": A job stopped because, for example, it lost a license.

    NOTE: "prove -status" lists the status for all running engine jobs for all
    proof threads. The tool prefixes statuses with the associated thread ID.

-per_engine_max_jobs N [-thread thread_id]
    Dynamically change the per_engine_max_jobs setting for the running
    "prove" command.

    Use "-thread" to change the setting for the "prove" that is associated with
    a specified thread.

-add engine_mode [-thread thread_id]
    Add the specified engine_mode to the running "prove" command.

    Use "-thread" to add the specified engine_mode to the "prove" that is
    associated with a specified thread.

-restart (engine_mode | index.engine_mode) [-thread thread_id | -tag tag]
    Stop and restart the specified engine or engine job.

    Use "-thread" to stop and restart the proof thread or an engine_mode or
    engine_job that is associated with a specified thread.
    . Use "-tag" to restart engine jobs in all proof threads for the specified
      tag.

-ignore_property property_name_tcl_list [-thread thread_id]
    Instruct engines in the running "prove" command to stop processing the
    specified properties.
```

```
NOTE:
. Use "-thread" to ignore properties only in the specified thread;
  otherwise, "prove -ignore_property" affects all running proof threads.
. This switch supports wildcards and regular expressions, for example:
  prove -ignore_property {precondition1$} -regexp
. Wildcard matching on this command only matches properties in the current
  task. To match multiple tasks at once, use the scope separator "::", for
  example:
  prove -ignore_property {*my_task*::*my_prop*}
```

Example

```
prove -all
prove -task taskA -engine_mode D
prove -task { taskA taskB taskC }
prove  { mon.master_error == 1'b0 }
prove -instance instA -except subinstB
prove -task taskA -save_ppd -with_ppd

# Specify multiple engines for parallel processing for faster run time
prove -task taskA -engine_mode { d g h }

# Instruct all running single-property engines to stop processing propertyA
prove -ignore_property { propertyA }

# Launch a proof with a tag and then control it using the tag
prove -all -bg -tag my_proof_run
prove -status -tag my_proof_run
prove -stop -tag my_proof_run
```

See Also

get_status,
set_engine_mode,
set_max_trace_length,
set_proof_simplification,
set_proofgrid_mode,
set_proofgrid_per_engine_max_jobs,
set_proofgrid_per_engine_max_local_jobs,
set_prove_orchestration,
set_prove_per_property_time_limit,
set_prove_time_limit,
set_prove_with_soft,
set_prove_stop_on_cex_limit,
set_proven_directive,
visualize

put_message

Adds a user-specified message to Jasper logging system.

Syntax

```
put_message message_text (-info |-warning |-error) [-suppress_id]
   [-file file_name] [-line line_number] [-tag tag_text]
```

Returns

No return values

Detailed Description

Use this command to add a user-specified info, warning, or error message to Jasper logging system, which is reflected in both the log file and GUI console pane.

```
message_text
        (-info |-warning |-error)
        [-suppress_id]
        [-file file_name]
        [-line line_number]
        [-tag tag_text]
```

Use one of the switches, "-info", "-warning", or "-error" to add a message of info, warning, or error type to the Jasper log system. The message identifiers corresponding to the switches are IUSR001, WUSR001, and EUSR001.

. Specify any string as the "message_text" and surround the text with double quotes. The text forms the body of the message.
. Use "-suppress_id" to omit the message identifier.
When you use "-suppress_id", the "get_message -message message_id" command does not include messages with a suppressed ID in the count.
. Use "-file" and "-line" to specify a "file_name" and "line_number" to create a clickable hyperlink when the message is shown in the GUI

```
      console pane.
    . Use "-tag" to specify a "tag_text" to add a unique tag text for the
      message. The text prints after the message identifier in square
      brackets. Jasper Information System can read the tag to generate
      reports.

    NOTE: Use "set_message -disable" with a message identifier to disable a
    message.

-info
    Add an info message. Message identifier: IUSR001.

-warning
    Add a warning message. Message identifier: WUSR001.

-error
    Add an error message. Message identifier: EUSR001.

-suppress_id
    Omit the message identifier. When you use "-suppress_id",
    the "get_message -message message_id" command does not include
    messages with a suppressed ID in the count.

-file file_name
    Specify a "file_name" to create a clickable hyperlink when
    the message is shown in the GUI console pane.

-line line_number
    Specify a "line_number" to create a clickable hyperlink
    when the message is shown in the GUI console pane.

-tag tag_text
    Specify a unique "tag_text" for the message. The text
    prints after the message identifier in square brackets.
    Jasper Information System can read the tag to generate
    reports.
```

Example

```
put_message -info "this is an information message"
INFO (IUSR001): this is an information message

put_message -info "this is an information message without identifier" -suppress_id
this is an information message without identifier

put_message -warning "this is a warning message" -file top.v -line 87
WARNING (WUSR001): top.v(87):this is a warning message

put_message -error "this is an error message" -file top.v -tag "TAG001"
ERROR (EUSR001): top.v(0): [TAG001] this is an error message
```

See Also

set_message

read_sdc

Loads and configures specified Synopsys(R) Design Constraints (SDC) files.

Syntax

```
SDC Files Management
--------------------
read_sdc (sdc_file_tcl_list |-clear)

Configuring Custom Naming Styles
--------------------------------
read_sdc -config (configurations_dictionary |-clear)
```

Returns

No return values

Detailed Description

```
SDC Files Management
--------------------
sdc_file_tcl_list |-clear
    Use "read_sdc <sdc_file_tcl_list>" to load Synopsys(R) Design
    Constraints (SDC) files, parse the files, and populate an internal database
    with the SDC constraints information. SDC constraints are not applied
    automatically to any context. Apps or flows that need them interpret the
    information in the database to apply the constraints properly. The Common
    Timing Engine (CTE) parses the SDC files.

    Use "-clear" to clear SDC constraint information from previously read SDC file.

    The "read_sdc" command supports the following SDC commands. Commands not
    included on this list are skipped and the tool prints a warning to "cte.log".

    . all_clocks
    . all_inputs
    . all_instances
```

. all_outputs
. compare_collections
. copy_collection
. create_clock
. create_generated_clock
. current_design
. current_instance
. foreach_in_collection
. remove_from_collection
. add_to_collection
. sizeof_collection
. index_collection
. append_to_collection
. filter_collection
. get_arcs
. get_cells
. get_clocks
. get_design
. get_designs
. get_generated_clocks
. get_lib_cells
. get_lib_pins
. get_nets
. get_object_name
. get_pins
. get_ports
. get_property
. query_objects
. range_collection
. remove_case_analysis
. remove_clock
. remove_clock_groups
. remove_clock_sense
. remove_disable_timing
. remove_generated_clock
. remove_input_delay
. remove_output_delay
. reset_case_analysis
. reset_clock
. reset_clock_groups
. reset_clock_sense
. reset_disable_timing
. reset_generated_clock

```
    . reset_input_delay
    . reset_mode
    . reset_output_delay
    . reset_path_exception
    . reset_sense
    . set_case_analysis
    . set_clock_groups
    . set_clock_sense
    . set_disable_timing
    . set_false_path
    . set_global
    . set_input_delay
    . set_mode
    . set_multicycle_path
    . set_output_delay
    . set_sense
    . sort_collection

    NOTE:
    . Ensure you elaborate a design before using this command.
    . After loading, the CTE log file can be found at
      "<jgproject>/sessionLogs/session_N/cte.log"
    . You can load files only once. Use "read_sdc -clear" before performing
      another load operation. Each new load overwrites the CTE log file.
    . The tool loads and parses files in the order they appear in the command's
      argument list.
    . The tool stops parsing the files at the first occurence of a blocking
      error. You must manually clear the database with "read_sdc -clear" after
      resolving the error and before reloading files.

Configuring Custom Naming Styles
--------------------------------
-config (configurations_dictionary |-clear)
    Use "read_sdc -config <configurations_dictionary>" to set up custom
    naming style rules that will be applied to the related RTL elements of the
    design.

    The custom naming styles are effective only in the context of SDC files read
    by Jasper through "read_sdc". The configurations set here do not affect
    the names of RTL elements in other parts of Jasper

    The "<configurations_dictionary>" argument consists of one or more
    pairs of rules, where each rule consists of the configuration name followed
```

by its desired value. The values are classified by types, and each rule is associated with and expects a single value type. Providing a value of incorrect type or an ill-formed value causes to tool to ignore the current rule pair being processed.

Use "read_sdc -config -clear" to clear and disable all naming style rules.

The value types are classified as follows:

- String:
  The value must be a regular string.
- Pattern:
  The value must be a string, but must include '%s' and/or '%d' placeholders, where the relevant information of the RTL element is inserted.
- List of Pairs:
  The value must be a Tcl list of Tcl lists, where the last dimension list should be of size 2. Example: {{A B} {C D} {E F}}
- Boolean:
  The value must be inferred as a boolean. Examples of accepted values are:
  0, 1, on, off, true, false.

Jasper supports the following rules, including their value types and default value:

Configuration Name	Type	Default Value
hdl_generate_index_style	Pattern	%s[%d]
hdl_generate_separator	String	.
hdl_array_naming_style	Pattern	%s[%d]
hdl_bus_wire_naming_style	Pattern	%s[%d]
hdl_instance_array_naming_style	Pattern	%s[%d]
hdl_reg_naming_style	Pattern	%s_reg%s
hdl_record_naming_style	Pattern	%s.%s
hdl_bit_blasted_port_style	Pattern	%s[%d]

```
 hdl_bus_naming_style             | Pattern | %s[%d]
----------------------------------+---------+------------------------
 hdl_rename_cdn_flop_pins         | List of | {{CLK CLK} {D D} {Q Q}
                                  | Pairs   |  {RV RV} {RC RC}}
----------------------------------+---------+------------------------
 hdl_parameter_naming_style       | Pattern | *
----------------------------------+---------+------------------------
 hdl_use_if_generate_prefix       | Boolean | true
----------------------------------+---------+------------------------
 hdl_use_block_prefix             | Boolean | true
----------------------------------+---------+------------------------
 hdl_use_for_generate_prefix      | Boolean | true
----------------------------------+---------+------------------------
 hdl_flatten_complex_port         | Boolean | false
----------------------------------+---------+------------------------
*The default value of the rule "hdl_parameter_naming_style" cannot be
 expressed by a single pattern string. You should clear the rules with
 "read_sdc -config -clear" if you desire this configuration to return to its
 default behavior. Refer to the detailed description of
 "hdl_parameter_naming_style" for more information.

. hdl_generate_index_style:
    Expects 1 '%s' and 1 '%d'.
    Specifies how the information of for-generate blocks appears on instance
    and signal names. %s represents the block label, while %d represents the
    index. If the generate for block is unnamed, it will be named based on
    the rules defined in the Verilog-2005 LRM (Section 12.4.3 of IEEE Std
    1364-2005). This configuration affects only instances and signals inside
    for-generate blocks. It does not affect instances and signals inside
    if-generate or case-generate statements.

. hdl_generate_separator:
    Specifies the separator string that appears after generate block labels.
    This configuration is applicable to instances and signals inside the
    following generate blocks: for-generate, if-generate, and case-generate.

. hdl_array_naming_style:
    Expects 1 '%s' and 1 '%d'.
    Specifies the naming format for individual bits of array signals in RTL.
    This configuration is only applicable to unpacked dimensions. %s
    represents the signal name, while %d represents the index. The rule is
    applied recursively to each unpacked dimension of a signal. Example:
      design.sv:
```

```
      ...
        logic [1:0] array_signal [1:0][1:0];
      ...
      read_sdc -config {hdl_array_naming_style A_%s_B_%d_C}
    Considering the LRM-compliant signal array_signal[0][1][0], the
    changed name will be:
      A_A_array_signal_B_0_C_B_1_C[0]

. hdl_bus_wire_naming_style:
    Expects 1 '%s' and 1 '%d'.
    Specifies the naming format for individual packed-dimension bits of
    signals in RTL. %s represents the signal name, while %d represents the
    index. The rule is only applied to the last dimension of a multibit
    signal if it's a packed dimension. Considering the design of the
    previous example:
      read_sdc -config {hdl_bus_wire_naming_style A_%s_B_%d_C}
    The signal array_signal[0][1][0] will be named:
      A_array_signal[0][1]_B_0_C

. hdl_instance_array_naming_style:
    Expects 1 '%s' and 1 '%d'.
    Specifies the naming format for each dimension of a multidimensional
    instance in RTL. %s represents the instance name, while %d represents
    the index. The rule is applied recursively to each dimension of the
    instance.

. hdl_record_naming_style:
    Expects 2 '%s'.
    Specifies the format used to depict how members of a record variable are
    accessed. The first %s represents the name of the record variable, while
    the second %s represents the record member. Record, in this sense, means
    Struct and Union in Verilog, and Record Types in VHDL. The rule is
    applied recursively to each individual access of record members in a
    variable.

. hdl_bit_blasted_port_style:
    Expects 1 '%s' and 1 '%d'.
    Specifies the format used to name the top module ports that are multibit
    packed-dimension signals. %s represents the port name, while %d
    represents the index of the individual bit. The rule is only applied to
    the last dimension of a multibit top-module port if it's a packed
    dimension.
```

. hdl_bus_naming_style:
Expects 1 '%s' and 1 '%d'.
Specifies the format used to name the ports of non-top modules that are multibit packed-dimension signals. %s represents the port name, while %d represents the index of the individual bit. The rule is only applied to the last dimension of a multibit non-top module port if it's a packed dimension.

. hdl_rename_cdn_flop_pins:
Accepted flop pins to be renamed: CLK, D, Q, RV, RC.
Use this configuration to rename the pins of generated flop instances. Each pair of the List of Pairs represents a mapping, where the first element of the pair is one of the pins listed above and the second element is the desired new name.

. hdl_parameter_naming_style:
Expects 1 '%s' and 1 '%d'.
Specifies the format shown for individual parameters and their respective values in the names of parameterized modules. For each parameter, %s represents the parameter name, while %d represents the parameter value. This rule, if enabled, has an impact on the naming of parameterized modules. Consider the following module:

```
design.sv:
...
module design #(parameter A=1, B=2) ();
...
```

Jasper default naming style for this module is:

```
design:(A=1,B=2)
```

However, if 'hdl_parameter_naming_style' rule is applied as follows:

```
read_sdc -config {hdl_parameter_naming_style %s=%d_}
```

The tool produces:

```
designA=1_B=2_
```

In other words, the encapsulation ":(...)" originally present on the module name, as well as the "," separator, are removed when the rule is enabled. This means that it is not possible to maintain the original naming convention while using this rule.

. hdl_reg_naming_style:
Expects 2 '%s'.
Specifies the format for naming created register instances in SDC. The first %s represents the deepest component of the type hierarchy of the signal, if the signal is of compound type. If the signal is a scalar, the first %s is the name of the signal itself. The second %s represents

```
the remaining information of the compound signal not included in the
first %s, and is empty if the signal is a scalar. For example, consider
the following RTL:
  design.sv:
  ...
  typedef struct {
      logic register_in_record;
  } RECORD;
  ...
      RECORD rec;
      genvar i;
      generate
          for (i = 0; i < 2; i = i + 1) begin
              logic [2:0] multibit_register;
              logic single_bit_register;
          end
      endgenerate
  ...
Where "rec.register_in_record", "multibit_register", and
"single_bit_register" are registers. Also considering the following rule
configuration:
  read_sdc -config {hdl_reg_naming_style %s_REG_%s}
Following are examples of register instances that would be generated:
  rec_REG_register_in_record
  genblk1.multibit_register_REG_[2]
  genblk1.single_bit_register_REG_
It must be noted that all other naming style rules are applied to the
signal that generates the register, and, therefore, the register name
will also contain those modifications to its name. Moreover, if the
signal is of compound type and any rule caused the second %s to have a
prefix in comparison to the first %s, the second %s will contain the
full, altered version of the name of the signal. If we consider the RTL
of the previous example, but with the following configurations:
  read_sdc -config {hdl_reg_naming_style %s_REG_%s}
  read_sdc -config {hdl_array_naming_style %s_A_%d_B}
  read_sdc -config {hdl_record_naming_style C_%s_D_%s_E}
  read_sdc -config {hdl_generate_index_style F_%s_G_%s_H}
Following are examples of register instances that would be generated in
this case:
  rec_REG_C_rec_D_register_in_record_E
  F_genblk_G_1_H.multibit_register_REG__A_2_B
  F_genblk_G_1_H.single_bit_register_REG_
Additionally, registers generated from packed-dimension signals will not
```

have packed-dimension rules applied to their names, even if those rules are enabled. All multibit signals are treated as unpacked dimensions when generating their register instance and, thus, get the "hdl_array_naming_style" rule applied to them if that rule is enabled.

. hdl_use_if_generate_prefix:
Controls whether if-generate labels appear on the names of signals and instances. If disabled, both the label and the separator are excluded from the RTL element name.

. hdl_use_block_prefix:
Controls whether block labels appear on the names of signals and instances. If disabled, both the label and the separator are excluded from the RTL element name.

. hdl_use_for_generate_prefix:
Controls whether for-generate labels appear on the names of signals and instances. If disabled, the label of the for-generate block is removed from the RTL element name, but the index and the separator are kept. Moreover, the separator and the index switch positions, and both are sent to the end of the RTL element base name. The configurations of "hdl_generate_index_style" are carried away to this operation as well. This means that if you set a custom for-generate index naming style with a custom prefix and/or a custom indexer, the custom prefix/indexer is moved to the end of the RTL element base name. The same applies if the "hdl_generate_separator" configuration is set. For example, consider the following RTL:

```
design.sv:
...
module top();
    genvar i, j;
    generate
    for (i = 0; i < 2; i = i + 1) begin : FOR_BLK_1
      for (j = 0; j < 2; j = j + 1) begin : FOR_BLK_2
        logic [2:0] multibit_signal [1:0];
        MOD md_instance [1:0] ();
      end
    end
    endgenerate
endmodule
...
```

Without any naming style configurations applied, the following elements could be fetched, for example:

```
    FOR_BLK_1[0].FOR_BLK_2[0].multibit_signal[0][2]
    FOR_BLK_1[1].FOR_BLK_2[1].multibit_signal[1][0]
    FOR_BLK_1[0].FOR_BLK_2[0].md_instance[0]
    FOR_BLK_1[1].FOR_BLK_2[0].md_instance[1]
  Now, consider the same design shown above, but With the following naming
  style configurations applied:
    read_sdc -config {hdl_use_for_generate_prefix false}
    read_sdc -config {hdl_generate_index_style A(%s)B(%d)C}
    read_sdc -config {hdl_generate_separator _}
  The RTL elements listed above would now look as follows:
    multibit_signal_A()B(0)_A()B(0)[0][2]
    multibit_signal_A()B(1)_A()B(1)[1][0]
    md_instance_A()B(0)_A()B(0)[0]
    md_instance_A()B(1)_A()B(0)[1]

. hdl_flatten_complex_port:
  Controls whether complex top module ports are represented as packed
  vector ports. The bits in the vector port are set according to how
  fields/dimensions are ordered in the definition of the complex port.
  A port is considered complex when it is not declared as a simple vector
  of bits or as a single bit scalar. For example, ports declared using a
  structure typedef or multi-dimensional array ports are complex ports.
  When enabled, the complex ports are considered multibit packed-dimension
  signals. For example, consider the following RTL:
    design.sv:
    ...
    typedef struct
    {
      bit a;
      bit b[1:0];
    } struct_t;
    ...
    module top(input struct_t str, logic md_array[1:0][1:0]);
    endmodule
    ...
  Without any naming style configurations applied, the following elements
  could be retrieved, for example:
    str.b[0] str.b[1] str.a
    md_array[0][0] md_array[0][1] md_array[1][0] md_array[1][1]
  Now, consider the same design shown above, but with the following naming
  style configuration applied:
    read_sdc -config {hdl_flatten_complex_port true}
  The RTL elements listed above would now look as follows:
```

```
      str[0] str[1] str[2]
      md_array[0] md_array[1] md_array[2] md_array[3]
    Also, you could maintain the configuration above and add a rule that
    applies to multibit packed-dimension signals, for example:
      read_sdc -config {hdl_bit_blasted_port_style %s(%d)}
    This would result in the following elements:
      str(0) str(1) str(2)
      md_array(0) md_array(1) md_array(2) md_array(3)

NOTE:
. Do not use "read_sdc -config" if the provided value is equal to the
  default as this impacts performance unnecessarily.
. You can configure naming style rules only before reading an SDC file with
  "read_sdc". If an SDC file has been read, use "read_sdc -clear" before
  configuring any additional rules.
. If an error occur while processing the configurations, the related rule
  is ignored and remains disabled. If you pass more than one configuration
  in configuration_dictionary, the tool tries to process the remaining
  configurations.
. "read_sdc -clear" does not clear the naming style rules. It is still
  necessary to call "read_sdc -config -clear" to disable the rules. "clear
  -all", on the other hand, does clear the rules.
. You can perform multiple calls to "read_sdc -config" configuring the same
  rule. The rule value is overwritten at each call.
. Where it is applicable, the tool merges the values passed across multiple
  calls to the command if the values do not intersect, but the tool still
  overwrites them if there is an intersection. Example:
    read_sdc -config {hdl_rename_cdn_flop_pins {{CLK ck} {D in} {Q out1}}}
    read_sdc -config {hdl_rename_cdn_flop_pins {{Q out2} {RV r_val} {RC r_cond}}}
    The rule will have stored: {{CLK ck} {D in} {Q out2} {RV r_val} {RC r_cond}}
```

Example

```
SDC Files Management
--------------------
read_sdc file.sdc
read_sdc file1.sdc file2.sdc file3.sdc
read_sdc {file1.sdc file2.sdc file3.sdc}
read_sdc -clear

Configuring Custom Naming Styles
--------------------------------
read_sdc -config {hdl_generate_index_style %s_MID_GEN_%d_END_GEN}
read_sdc -config {hdl_generate_index_style %s_%d hdl_array_naming_style {%s_[%d]_}}
read_sdc -config -clear
```

redirect

Redirects stdout to a file or Tcl variable.

Syntax

```
redirect (-file file_name |-variable var_name)
          [-force |-append] [-tee] command
```

Returns

The value returned by command.

Detailed Description

Use this command to capture output written to stdout by a Tcl command. You can redirect the output into either a file or a Tcl variable.

NOTE: Output written to stdout by background jobs is also redirected. Therefore, avoid using this command while background jobs are running, for example, "prove -bg" and "visualize -bg".

-file file_name [-force |-append]
    Direct the output to the specified file.

    . Use "-force" to overwrite an existing file.
    . Use "-append" to append to an existing file.

-variable var_name [-append]
    Redirect the output to the specified Tcl variable.

    Use "-append" to append output to an existing Tcl variable. If the variable already exists and you do not use "-append", the tool replaces the existing value with the new value.

-tee
    In addition to redirecting output to a file or Tcl variable, print the output on the Jasper Apps console.

Example

```
redirect -file prove-log.txt -force { prove -all }
redirect -file visualize-log.txt -force { visualize -violation -property a }
```

replace_driver

Disconnects the original driver of a signal and replaces it with a new one.

Syntax

```
replace_driver signal_name+ -expression new_driver
               [-env |-task task_name]
               [-condition expression]
               [-silent]

replace_driver -remove signal_name+
               [-env |-task task_name]
               [-silent]

replace_driver -list
               [-env |-task task_name]
               [-silent]

replace_driver -clear
               [-env |-task task_name]
               [-silent]
```

Returns

"-list" returns a list of target-replacement pairs.

Detailed Description

Use this command to specify that an internal design signal should be treated as if it were disconnected from its original driving logic, and use the newly-specified expression to act as its driver instead.

This command essentially modifies the design, meaning that design exploration commands, such as "get_fanin", consider the new driver instead of the original one.

CAUTION: Proof results must be interpreted with care after using this command,

since they are based on behavior that may not exist in the original design.

You can use this command in the following example scenarios:
. When you want to test what would be the behavior of a signal if its driving logic were modified in the design, without having to actually modify the design files and elaborate it again.
. When you want to provide a driving expression for blackbox output ports.
. When the driving logic of a signal is too complex and you want to abstract it by replacing it with simpler logic to speed up proofs.

signal_name+ -expression new_driver
    [-env |-task task_name]
    [-condition expression]
    [-silent]
    Disconnect the original driving logic from the target signals and connect them to the new driving expression specified.
    . Use "-env" to replace the driver globally.
    . Use "-task" to replace the driver on the specified task.
    . If you do not use any of the switches above, the tool applies the Replace Driver on the current task.
    . Use "-condition" to specify that the driver must be replaced only when the specified expression is true, retainig the original driving logic when it is false.
    . Use "-silent" to replace the driver without printing any messages on the screen.

    NOTE:
    . The driving expression and condition expression can be of any kind supported by Jasper, including $dff, $dlatch, $past, |->, ##n. For more details, see the "Jasper Tcl Operators" section in the "SVA Support" appendix of the Jasper Apps Command Reference Manual.
    . When the target is a System Verilog Interface signal, the Replace Driver will be applied to its actual driver.

-remove signal_name+
    [-env |-task task_name]
    [-silent]
    Remove the Replace Driver operation from the target signals, restoring their original driving logic.
    . Use "-env" to remove a global Replace Driver.
    . Use "-task" to remove a Replace Driver from the specified task.
    . If you do not use any of the switches above, the tool removes the Replace Driver from the current task.

```
    . Use "-silent" to remove the Replace Driver without printing any messages
      on the screen.

-list
    [-env |-task task_name]
    [-silent]
    List all the signals that are target of a Replace Driver operation and their
    respective replacement expressions.
    . Use "-env" to list the global Replace Drivers.
    . Use "-task" to list Replace Drivers on the specified task.
    . If you do not use any of the switches above, the tool lists the Replace
      Drivers on the current task.
    . Use "-silent" to return the list as Tcl return values without printing the
      list on the screen.

-clear
    [-env |-task task_name]
    [-silent]
    Remove the specified set of Replace Driver operations.
    . Use "-env" to remove all global Replace Drivers.
    . Use "-task" to remove all Replace Drivers from the specified task.
    . If you do not use any of the switches above, the tool removes all Replace
      Drivers from the current task.
    . Use "-silent" to remove the Replace Drivers without printing any messages
      on the screen.
```

Example

```
replace_driver signalA -expression {sel ? in1 : in2}
replace_driver signalB -expression data -condition enable
replace_driver inst1.clk inst2.clk inst3.clk -expression clk -env

virtual_net counter -width 32
virtual_net counter_next -expression {counter+1} -width 32
replace_driver counter -expression {$dff(counter_next,clk,rst,32'b0)} -silent

replace_driver -remove signalA
replace_driver -list -task my_task
replace_driver -clear
```

See Also

stopat

report

Reports verification results.

Syntax

```
report [-all [-include_type]]
       [-file file_name [-force |-append]]
       [(-task task_name_tcl_list
         |-property property_tcl_list)]

report [-info] [-summary]
       [-results [-include_type]]
       [-detailed]
       [-file file_name [-force |-append]]
       [(-task task_name_tcl_list
         |-property property_tcl_list)]

report [-csv [-include_type]]
       [-file file_name [-force |-append]]
       [(-task task_name_tcl_list
         |-property property_tcl_list)]

report [-json -config -fpv]
       [ (-task task_name_tcl_list
        |-property property_tcl_list)]

report [-json -config -cov]
       [-task task_name_tcl_list]
       [-include_cover_item_detailed]
       [-include_assert_detailed]
       [-include_module_coverage]
       [-type (formal | stimuli | checker | bounded)]
       [-exclude (  const | assume | reset | waived | disabled
                  | unreachable | deadcode | synthesis_optimized)]

report [-json -file file_name]
       [-force]
```

Returns

No return values

Detailed Description

Use this command to report verification results. These reports contain lists of all properties and their proof status (proven, cex, covered, and so forth). If you do not include any options with this command, the tool prints out a header, a table of results and a table of assumptions, unless the "-json" subcommand is used, in which case you will be required to redirect the output to a file.

```
[-all [-include_type]]
    [-file file_name [-force |-append]]
    [(-task task_name_tcl_list | -property property_tcl_list)]
    Generate a report with all sections, that is, summary, info, results,
    assumptions, and details.
    . Use "-include_type" to include the property type in the report table in a
      "Type" column.
    . Use "-file" to save the report to a specified file instead of displaying
      it on the screen.
      . Use "-force" if the specified report file already exists, and you want
        to overwrite it.
      . Use "-append" to add the report to the end of the specified file.
    . Use "-task" to generate a report on the specified tasks only.
      NOTE:
      . Replace "task_name_list" with a period (.) to specify the current task.
      . When specifying a list of task names, enclose the list in curly braces.
      . If you do not specify "-task" or "-property", the report includes all
        properties in all tasks.
    . Use "-property" to generate a report including only the specified
      list of properties. Refer to the examples to see how you can use the
      "get_property_list" command to select properties.

[-info] [-summary]
    [-results [-include_type]]
    [-detailed]
    [-file file_name [-force |-append]]
    [(-task task_name_tcl_list | -property property_tcl_list)]
    . Use "-info" to generate the report with design info: hard stopats, global
      assumptions, resets, and clocks.
    . Use "-summary" to generate the results summary.
```

```
    . Use "-results" to generate the table of results. And use "-include_type"
      to include the property type in the report table in a "Type" column.
    . Use "-detailed" to generate the table of detailed results, including
      proof bound and proof engine for each property.
    . Use "-file" to save the report to a specified file instead of displaying
      it on the screen.
      . Use "-force" if the specified report file already exists, and you want
        to overwrite it.
      . Use "-append" to add the report to the end of the specified file.
    . Use "-task" to generate a report on the specified tasks only.
      NOTE:
      . Replace "task_name_list" with a period (.) to specify the current task.
      . When specifying a list of task names, enclose the list in curly braces.
      . If you do not specify "-task" or "-property", the report includes all
        properties in all tasks.
    . Use "-property" to generate a report including only the specified
      list of properties. Refer to the examples to see how you can use the
      "get_property_list" command to select properties.

[-csv [-include_type]]
    [-file file_name [-force |-append]]
    [(-task task_name_tcl_list | -property property_tcl_list)]
    Generate a table of results.
    . Use "-csv" to generate the report as comma-separated values (CSV).
    . Use "-include_type" to include the property type in the report table in a
      "Type" column.
    . Use "-file" to save the report to a specified file instead of displaying
      it on the screen.
      . Use "-force" if the specified report file already exists, and you want
        to overwrite it.
      . Use "-append" to add the report to the end of the specified file.
    . Use "-task" to generate a report on the specified tasks only.
      NOTE:
      . Replace "task_name_list" with a period (.) to specify the current task.
      . When specifying a list of task names, enclose the list in curly braces.
      . If you do not specify "-task" or "-property", the report includes all
        properties in all tasks.
    . Use "-property" to generate a report including only the specified
      list of properties. Refer to the examples to see how you can use the
      "get_property_list" command to select properties.

[-json -config -fpv]
    [(-task task_name_tcl_list |-property property_tcl_list)]
```

```
    Configure the fields of the JSON file generated by "report -json -file" for
    the FPV App.
    . Use "-config -fpv" to configure the fields of the report supported by the
      FPV App.
      NOTE: This command overwrites previous report configurations for the FPV
      App.
    . Use "-task" to generate the JSON report for the selected tasks.
      NOTE: If "-task" is not specified, all tasks are included in the report.
    . Use "-property" to generate a report including only the specified list of
      properties. Refer to the examples to see how the "get_property_list"
      command can be used to select properties.
      NOTE: If "-property" is not specified, the report includes all properties
      of the selected tasks.

[-json -config -cov]
    [-task task_name_tcl_list]
    [-include_cover_item_detailed]
    [-include_assert_detailed]
    [-include_module_coverage]
    [-type (formal | stimuli | checker | bounded)]
    [-exclude (  const | assume | reset | waived | disabled
               | unreachable | deadcode | synthesis_optimized)]
    Configure the fields of the JSON file generated by "report -json -file" for
    the COV App.
    . Use "-config -cov" to configure the fields of the report supported by the
      COV App.
      NOTE: This command overwrites previous report configurations for the COV
      App.
    . Use "-task" to generate the JSON report for the selected tasks.
      NOTE: If "-task" is not specified, all tasks are included in the report.
    . Use "-include_cover_item_detailed" to include the detailed field of the
      "coverage_summary" section of the COV report. The detailed field of the
      "coverage_summary" section provides information of each cover item of an
      instance, considering proof results for the selected tasks.
      NOTE: By default, the detailed field of the "coverage_summary" is empty.
    . Use "-include_assert_detailed" to include the detailed field of the
      "assert_summary" section of the COV report. The detailed field of the
      "assert_summary" section provides the cover item ids in bound, out bound,
      unprocessed, and excluded for each assertion of each selected task.
      NOTE: By default, the detailed field of the "assert_summary" is empty.
    . Use "-include_module_coverage" to include the module coverage information
      in the JSON report for the COV App.
      The module coverage information includes: module, types, and their
```

```
      detailed information under the "module_coverage_summary" section of
      the COV report. This option is equivalent to "Coverage View: Module"
      in the Coverage Analysis GUI window.
      NOTE: This switch respects the "-include_cover_item_detailed" switch,
      which means that detailed information is not included if
      "-include_cover_item_detailed" is not present.
    . Use "-type" to filter which coverage proof type information should be
      available in the JSON report for the COV App.
      NOTE:
      . All options are visible by default. The "-type" switch is used as a
        filter.
      . For example, if just "-type formal" is issued, only formal proof type
        information is reported.
    . Use "-exclude" to filter which cover items will be marked as "excluded" in
      the JSON report.
      NOTE:
      . By default, all cover items are included, meaning that no cover item
        should have the "Excluded" field as "True" for the detailed view of the
        "coverage_summary" section and no instance should have an "excluded"
        count for the "coverage_summary" field.
      . For example, if just "-exclude deadcode" is issued, all cover items
        with a proof result of deadcode on the selected task should be marked
        as excluded in the detailed view, as well as ignored in the
        "coverage_summary" of the instance.

[-json -file file_name]
    [-force]
    Generate a JSON with the results.
    . Use "-json" to generate the report in JSON format.
    . You must use "-file" to save the report to a specified file.
      . Use "-force" if the specified report file already exists, and you want
        to overwrite it.
    NOTE: Use "-config" to configure the supported app sections of the report.
```

Example

```
report
report -summary -task {taskA taskB}
report -file report.txt -force -detailed

# Generate a report for the current task in CSV format:
report -task . -csv -file report_[task -set .].csv

# Generate a report including only cover properties.
report -property [get_property_list -include {type cover}]

# Generate a JSON report for the FPV App selecting only taskA
report -json -config -fpv -task {taskA}
report -json -file file.json

# Generate a JSON report for the Coverage App with only type formal
report -json -config -cov -type formal
report -json -file file.json
```

See Also

get_property_list

reset

Specifies design reset condition.

Syntax

```
reset  ([-expression] (pin_constraint_list | pin_constraint+) [-max_iterations N [-
clock clock_signal]])
       [(-init_state file_name [-postfix postfix_file_name]
                                                [-disable_wildcard] [-
include_internal])
       |(-vcd file_name [-include_internal])
       |(-fsdb file_name)
       |(-shm file_name)
       |(-vwdb file_name)]
           [[-postfix postfix_file_name] [-disable_wildcard]]
           [-time time [-time_scale (ms | us | ns | ps)] [-exact]]
           [ -hier_path path_difference
            |-hier_map -from source_hier -to destination_hier
           ]
       [-non_resettable_regs value]

reset -formal (pin_constraint_list | pin_constraint+ |-none)
                      [-bound N] [-force] [-max_iterations N [-clock clock_signal]]

reset -sequence (seq_file_name [-disable_wildcard] [-include_internal])
                         |(-vcd file_name [-include_internal])
                         |(-shm file_name [-time time [-exact]])
                         |(-vwdb file_name [-time time [-exact]])
                         |(-fsdb file_name)
                         [[-postfix postfix_file_name] [-disable_wildcard]]
                         [-start_formal_cycle cycle [-from_end]]
                         [-start_condition expression -end_condition expression]
                         [-start_time time] [-time time]
                         [-time_scale (ms | us | ns | ps)]
                         [ -hier_path path_difference
                           |-hier_map -from source_hier -to destination_hier
                         ]
                         [-non_resettable_regs value]
```

```
reset -analyze [-synchronous] [-list (flop | signal)] [-silent]
                          [-task task_name] [-conflict]

reset -none [-max_iterations N [-clock clock_signal]]
                       [-non_resettable_regs value]

reset -list [signal] [-typed_list] [-silent]

reset -clear
```

Returns

No return values

Detailed Description

Use this command to specify the reset condition for the design under verification. Reset conditions are always *globally* applied. And by default, they are noncumulative. For cumulative reset specification, see "set_cumulative_reset".

The "reset" analysis only considers global stopats and assumptions (specified with "-env"). Task-specific information (including assumption and stopat information) does not affect reset values. The tool fully propagates assumptions that directly force signals to constant values but does not guarantee that more complex assumptions are enforced during reset analysis. In the default mode, the tool runs an implicit clear for each "reset" command.

Important things to know about this command:

. Embedded SVA and PSL assumptions are not used during reset analysis unless you elaborate with "-global_embedded_assumes". When using this switch, reset analysis considers these assumptions provided they are written on primary inputs (including the clock).
. Before you can run reset analysis, including the reset analysis that runs for "prove" and "visualize", you must specify reset. Use one of the following commands:
  . "reset ~signalName" to declare a reset pin
  . "reset -expression" to use a well-defined expression
  . "reset -sequence" to set a reset sequence file
  . "reset -init_state" to set a reset snapshot file

```
  . "reset -none" to specify a free-running reset environment
  NOTE:
  . If you use a signal name or expression to declare reset, specify the
    condition under which the reset is active. For example, if the reset signal
    is active low rst_n, use the command "reset ~rst_n" (or for VHDL, "reset
    {not rst_n}").
  . If you use "reset -none", the tool runs reset analysis, so if you do not
    want to run the analysis, use an empty "reset -init_state" file.
  . If you are not ready to run one of the specified commands, use "reset
    -analyze" to help set up the reset environment properly.
. During reset analysis simulation, the tool prints a message after
  advancing 10 percent of the iterations, and it continues to print messages at
  every successive 10 percent increment. For example, if the maximum number of
  iterations in simulation is 100, a message prints each 10 iterations until
  the iteration of convergence (that is, less than or equal to maximum number
  of iterations).

There are four primary ways to specify reset conditions:

1. A set of pin constraint expressions
2. A reset sequence in a file (internal Jasper format, VCD, FSDB, SHM, VWDB).
   For external formats you can specify a time range you want the tool to read.
3. An initial state file with the reset values for a specific set of registers
   (internal Jasper format, VCD, FSDB, SHM, VWDB).
   For external file formats, you can choose the time at which to extract the
   reset values.
4. Free reset environment (reset -none)

You can also adjust the values for specific registers by providing
the following commands:

5. reset -non_resettable_regs: all undefined initial values (but not those
   from "abstract -init_value") will assume the specified initial value
6. abstract -init_value: force the registers to have undefined initial values

Options 1 and 3 can be combined. At least one of the options from 1 to 4
must be present with two exceptions.

Exceptions:
. Options 1 and 2 above are mutually exclusive; however, you can postfix
  expressions to the sequence file. (Refer to the "-sequence" definition
  below.)
. The options "-fsdb", "-vcd", "-shm" and "-vwdb" are mutually exclusive.
```

```
[-expression] (pin_constraint_list | pin_constraint+)
    [-max_iterations N [-clock clock_signal]]
    A set of pins (input or global stopat net) that are asserted
    when the design is being reset by the tool and de-asserted
    when the design is being formally verified.

    NOTE:
    . "reset -expression" only supports pin constraints.
    . The "-expression" switch is optional. That is, the following command pairs
      have the same effect:
      Example 1:
      reset -expression ~rst1N rst2
      reset ~rst1N rst2

      Example 2:
      reset -expression {~rst1N rst2}
      reset {~rst1N rst2}

    Reset analysis runs until register values stop changing or until it
    reaches 100 iterations.

    . Use "-max_iterations" with an integer to change the default. And use
      "-clock" to specify that you want the number of iterations based on the
      specified clock.

      NOTE:
      . The specified clock must be a declared clock or a propagated clock
        signal.
      . This switch does not accept zero.

    . Use "-non_resettable_regs" to assign the specified value to any register
      with an undefined reset value that is not the result of the "abstract
      -init_value" command.

      NOTE: "value" must be 0 or 1.

    ATTENTION: Jasper Apps currently only supports pin
    constraints that force an input to have a constant value,
    for example, of the form "rst", "~rst", "a==1'b1", "a==1'b0", and
    so forth. If you have more than one reset pin, you can use a list of
    pins: {rst1 ~rst2 rst3==1'b1}. When arrays are declared as reset pins,
    the tool ensures that all bits of the array are asserted during the reset
```

```
    analysis. Outside of the reset analysis, all other states are possible for
    these bits.
    Example:
      wire rst[2];
      reset rst;
    During reset: rst = 2'b11
    After reset: rst != 2'b11 (so reset can be 2'b00, 2'b01, 2'b10)

    NOTE: For reset expression mode, reset analysis calculates flop values that
    have their output changed due to some clock activity during the
    transition between reset and proof. The first cycle of formal analysis
    will reflect the potential changes that can happen during the transition
    between reset and formal analysis.

-init_state file_name [-postfix postfix_file_name]
    [-disable_wildcard] [-include_internal]
    Get reset values of registers from the specified file.

    . Use "-postfix" to add global assumptions. The postfix assumptions are
      asserted during the proof and have no effect on the reset initial state,
      even if they are listed as reset expressions when using the command
      "reset -list". The postfix files have the same syntax as reset initial
      state files.

    . Use "-disable_wildcard" to turn off the wildcard matching feature when
      reading initial state files to (potentially) read files faster.

    . Use "-include_internal" to load internal signals from the reset file.

    CAUTION: "reset -init_state -include_internal" carries a risk of a mismatch
    in internal signals that can lead to false proof results in some cases.
    Internal signals in reset files might not be the same as those generated by
    a different Jasper release version or a different version of the design
    or a different design setup and configuration. (Design setup and
    configuration differences include the order in which files are passed to
    "analyze" and "elaborate" and the order in which assumptions and assertions
    are created.)

    Any register with an undefined value not resulting from the command
    "abstract -init_value" takes the value provided by the init state
    file. If the init state file defines a value for a register and
    the command "abstract -init_value" defines another value, the tool
    issues a warning and discards the value in the file.
```

```
Format your file as follows:
Line 1: <Signal_A>
Line 2: <Value>
Line 3: <Signal_B>
Line 4: <Value>
...

Example:
wb_dat_o
8'b11000001
prer
16'b10101010_10101010
...

OR

Line 1: <Signal_A> <Value>
Line 2: <Signal_B> <Value>
...

Example:
wb_dat_o 8'b11000001
prer 16'b10101010_10101010
...

Important things to know about this command:
. The tool supports both Verilog and VHDL constant values except for VHDL
  decimal.
. The tool supports encrypted files saved with "get_reset_info -save_values
  -include_encrypted" and "visualize -save -init_state -include_encrypted".
. The tool supports the IEEE definition of variable names, including
  escaped names.
. The tool supports wildcards (* and ?) in "reset -init_state" files (see
  "Example" section). However, do not attempt to use wildcards for escaped
  names. The tool interprets them as literals.
  NOTE: Use "-disable_wildcard" to disable this feature.
. A "Value" can be either a numeric constant or an enum value.
. Prefix your comments with "#". The tool recognizes the end of the
  line as the end of the comment.
. Signals with an unspecified bus size are interpreted as decimal, for
  example:
    signal_A[3:0]
```

```
        10
      signal_A has 4 bits, "10" is interpreted as 4'd10 (4'b1010).
    . Signals with an incorrect width are truncated/extended, for example:
        signal_A[1:0]
        4'b1001
        signal_B[3:0]
        1'b1
      signal_A takes the value 2'b01
      signal_B takes the value 4'b0001

[(-vcd file_name [-include_internal]) |(-fsdb file_name) | (-shm file_name)
 | (-vwdb file_name)
    [-postfix postfix_file_name] [-disable_wildcard]
    [-time time [-time_scale  (ms | us | ns | ps)] [-exact]]
    [-hier_path path_difference |-hier_map -from source_hier -to destination_hier]]

    Get reset values of registers from the specified file.
    . Use "-vcd", "-fsdb", "-shm" or "-vwdb" to specify the type of trace to be
      interpreted.

      IMPORTANT:
      . Jasper Apps writes 6.3 FSDB traces and reads version 6.6 FSDB
        traces. FSDB Reader 6.6 was introduced by Verdi version X-2025.06.
      . FSDBs you generate with the latest Jasper are backwards compatible
        with all Jasper versions from 2025.03 FCS and newer. In addition,
        they are backwards compatible with older versions of Verdi released
        during that same period.
      . Jasper Apps supports backwards compatibility, that is, the tool can
        read FSDBs generated from older versions of Jasper Apps and
        Synopsys Verdi.

    . Use "-postfix" to add global assumptions. The postfix assumptions are
      asserted during the proof and have no effect on the reset initial state,
      even if they are listed as reset expressions when using the command
      "reset -list". The postfix files have the same syntax as reset initial
      state files.

    . Use "-disable_wildcard" to turn off the wildcard matching feature when
      reading initial state and postfix files to (potentially) read files
      faster.

    . Use "-time" to specify an integer representing the time point where the
      signal values are sampled. The time is proportional to the timescale
```

defined with `timescale in the testbench. If you do not specify a time, the tool uses the last time in the file.

. Use "-time_scale" with the unit of measure (ms, us, ns, or ps) to specify the time scale. If you do not specify a time scale, the tool uses the default from the trace file.

. Use "reset (-shm | -vwdb) file_name -time time -exact" to prevent the tool from using the clock's waveform when defining the sampling points of the trace file. This switch forces the tool to use the values of the defined timestamp as the reset state.
NOTE: Only "reset... -shm" and "reset... -vwdb" support the "-exact" switch.

. Trace files are usually generated during simulation where the testbench module is the top module for the verification run, and consequently, for the trace file too. When using a file generated by simulation to reset your design in Jasper Apps, use the optional "-hier_path" switch to specify the hierarchical path difference from the testbench module to the design top module.

For example, if the design top module is "dut_top", which is instantiated as the top module in the testbench "testbench_top" as "i_dut_top", use the following command to reset the design with the trace file (replace "file_name" and "time" with actual values):

```
reset -vcd  <file_name> -time <time> -hier_path testbench_top.i_dut_top
reset -fsdb <file_name> -time <time> -hier_path testbench_top.i_dut_top
reset -shm  <file_name> -time <time> -hier_path testbench_top.i_dut_top
reset -vwdb <file_name> -time <time> -hier_path testbench_top.i_dut_top
```

. Use "-hier_map" for trace files generated during simulation if you have a wrapper module.
  . Use "-from" to specify the source (simulation environment) hierarchy.
  . Use "-to" to specify the destination (formal environment) hierarchy.

. Use "reset (-vcd |-fsdb |-shm |-vwdb) trace_filename" to use the inputs from the specified trace file for reset simulation.

. Use "reset -vcd -include_internal" to load internal signals from the reset file.

CAUTION: "reset -vcd -include_internal" carries a risk of a mismatch in

```
      internal signals that can lead to false proof results in some cases.
      Internal signals in reset files might not be the same as those generated
      by a different Jasper release version or a different version of the
      design or a different design setup and configuration. (Design setup and
      configuration differences include the order in which files are passed to
      "analyze" and "elaborate" and the order in which assumptions and
      assertions are created.)

    LIMITATIONS:
    . Compound black-boxed output values will not be restored.
    . FSDB files are not supported on Arm Achitecture.

-formal (pin_constraint_list | pin_constraint+ |-none) [-bound N] [-force]
    [-max_iterations N [-clock clock_signal]]

    Compute reset simulation during the formal analysis.
    . Use "-none" to set up formal reset without specifying pin constraints.
    . Use "-bound" to specify the number of cycles the declared set of pins are
      asserted during the formal analysis. After the specified bound, the tool
      assumes that the design has been reset, and the declared set of pins are
      de-asserted. If this switch is not specified, the bound is automatically
      inferred by the tool.
    . Use "-force" to run the analysis ignoring all stopats and abstractions.

    NOTE:
    . Standard reset analysis also runs before the formal analysis; however,
      its results are not meaningful since there will be a reset phase during
      the formal analysis. Hence, results from the whole process involving
      reset analysis will be available after the specified bound. Then, proof
      traces will also contain outputs from the reset analysis. This approach
      might result in more concrete values.
    . Standard reset analysis runs until register values stop changing or until
      it reaches 100 iterations. Use "-max_iterations" with an integer to
      change the default. This switch does not accept zero.
    . Use "-clock" to specify that you want the number of iterations based on
      the specified clock, which must be a declared clock or a propagated clock
      signal.

    LIMITATIONS:
    . The following commands do not work when reset is in formal mode:
      . visualize -reset, -sst
      . get_reset_info
      . abstract -init_value
```

```
    . If you declare the abstractions or stopats (listed above) before running
      "reset -formal", the tool does not allow the reset setup. In that case,
      remove those abstractions and stopats or use "-force" to run "reset
      -formal" disregarding all restrictions.
    . This command is exclusively intended for the Formal Property Verification
      App, and it might not work for apps such as Security Path Verification
      App, X-Propagation Verification App, and Sequential Equivalence Checking
      App.

    ATTENTION: You can get unexpected CEXs and covers during the reset phase in
    the formal analysis as the properties will be evaluated then as opposed to
    what happens in a standard Jasper reset where property analysis starts
    after reset has finished. Due to this behavior, you must use "disable iff"
    clauses if you want to suppress property evaluation during formal reset. As
    an alternative, you can use "set_evaluate_properties_on_formal_reset off"
    before running "elaborate". For additional details, run
    "help set_evaluate_properties_on_formal_reset".


-sequence (seq_file_name [-disable_wildcard] [-include_internal])
    Get reset values of registers from the specified sequence file.

    NOTE:
    . To accelerate run times for reset sequence simulation, use
      "set_trace_show_reset false".
    . When the sequence file contains values for flops or latches that do
      not have stopats, it triggers the error (EFL058), which you can
      downgrade to a warning with "set_message -warning EFL058".

    . Use "-disable_wildcard" to turn off the wildcard matching feature when
      reading sequence files to (potentially) read files faster.
    . Use "-include_internal" to load internal signals from the reset file.

      CAUTION: "reset -sequence -include_internal" carries a risk of a
      mismatch in internal signals that can lead to false proof results in some
      cases. Internal signals in reset files might not be the same as those
      generated by a different Jasper release version or a different
      version of the design or a different design setup and configuration.
      (Design setup and configuration differences include the order in which
      files are passed to "analyze" and "elaborate" and the order in which
      assumptions and assertions are created.)

    . "-expression" and "-sequence" are mutually exclusive; however, you can
```

```
  add expressions to the sequence file as "postfix" information. This
  feature makes it possible to specify the inputs that should be forced to
  a certain value (to "de-assert" the reset signal) during "prove" and
  "visualize" analysis.
. No sampling occurs at the boundary between the reset and formal analysis,
  that is, between cycle 0 and cycle 1. You must assume responsibility for
  setting the correct number of cycles to reach the desired initial state
  after reset.
. For reset sequence mode, reset analysis calculates flop values that
  have their output changed due to some clock activity during the
  transition between reset and proof if the reset trace stops at a clock
  state that is consistent with the initial clock state of the formal
  analysis. Then, the first cycle of formal analysis will reflect the
  potential changes that can happen during the transition between reset and
  formal analysis.

All the reset values will be defined by simulating this sequence
for the specified number of iterations. If you use the $past/prev
operators, this simulation will be used as the pre-verification history to
define their initial values.

Format your file as follows:
Line 1: Signal_Name Value
Line 2: Signal_Name Value
Line 3: Number of clock ticks
Line 4: Signal_Name Value
...
Line n: Number of clock ticks
#### (Optional) Postfix information
#### Postfix information is the section that contains a list of signal
#### names and values between the last clock ticks specified and $.
Line n+1: Signal_Name Value
Line n+2: Signal_Name Value
...
Line n+m: $

Important things to know:
. The initial definition of signals (before any clock tick definition)
  might contain both input values (PI or hard stopats) and register
  reset values.

. Clock ticks are relative to the fastest clock.
```

. If the signal appears in one iteration and not in the following iterations, its value will be maintained in the interval.

. Jasper Apps supports two directives for use in sequence files: $wait_all and $wait_for_one. You can add these directives within an input change scope/context. They are a prerequisite for simulating the next input changes. You will close a block with $end_wait followed by the "number of clock ticks," which is required as it serves as a timeout for the wait block.
  . $wait_all defines a block with conditions that must be met during a given input change described between two reference cycles or one reference cycle and the end of the reset sequence.
  . $wait_for_one defines a block with a set of conditions, and at least one condition must be met during a given input change described between two reference cycles or one reference cycle and the end of the reset sequence.

  Example:

```
...
rst 1'b1
10
rst 1'b1
$wait_all
out_st 1'b1
out_nd 1'b1
$end_wait
50
...
```

  You can use $wait_until_convergence instead of specifying the number of clock ticks. If you include this directive, it means that reset analysis runs until it reaches convergence for a given set of input value changes. The number of iterations executed respects the clock configurations.

  Example:

```
...
rst 1'b1
10
rst 1'b0
$wait_until_convergence
out_st 1'b1
out_nd 1'b1
15
...
```

. The simulation will iterate until the final iteration number. If there is no such definition, it will iterate until the last iteration where signals were defined.

. The format of a line that contains a signal and a value must be strictly <Signal_Name> <Value>.

. A Value can be either a numeric constant or an enum value.

. The maximum size of a line in a reset file should be 64k characters.

. The tool supports wildcards (* and ?) in "reset -sequence" files. However, do not attempt to use wildcards for escaped names. The tool interprets them as literals.
NOTE: Use "-disable_wildcard" to disable this feature.

. The postfix signal values are used as global assumptions to be observed during the proof phase. The tool supports single-bit and wide signals. The following is an example of wide signals in the postfix section:
WReset 2'b10
2
WReset 2'b11
2
WReset 2'b00
$

. Prefix your comments with "#". The tool recognizes the end of the line as the end of the comment.

Example:

```
resetN 1'b1
input_a 2'b10
input_b 2'b11
1
input_a 8'b00
2
resetN 1'b0
2
#postfix information
resetN 1'b1
input_b 2'b01
$
```

```
   The simulation for the example above takes five iterations, and
   the values are:

   Iteration    1      2     3     4    5   Proof cycles
   resetN       1      1     1     0    0        1
   input_a     10     00    00    00   00       ??
   input_b     11     11    11    11   11       01

   The tool supports binary, octal, hexadecimal, decimal, and signed decimal
   values. It is possible to specify more than one radix in a file, as shown
   below.

   in_9bit 9'b001101001
   # 9'b1101001 is interpreted as 9'b001101001
   in_9bit 9'b1101001
   in_9bit 9'h6_9
   rstN 3'o0 # octal number
   in_9bit 9'o151
   in_9bit 9'd105 # decimal number
   in_9bit 9'b001x01001
   # 9'bx1010x1 is interpreted as 9'bxxx1010x1
   in_9bit 9'bx1010x1

   You can also mix radixes in one signal:

   # signal_16bits[0:15]
   signal_16bits[0:7] 8'b010101xx
   signal_16bits[8:15] 8'd5

   The parser only accepts unsigned numbers and it will not
   accept definitions where:

   - A constant is bigger than the signal (signal_16bits 17'd0)
   - A signal is in different order from the design definition:
     # signal_16bits[0:15]
     signal_16bits[12:5] 8'd0

-sequence
    (-vcd file_name [-include_internal])
   |(-shm file_name [-time time [-exact]])
   |(-vwdb file_name [-time time [-exact]])
   |(-fsdb file_name)
```

```
   [[-postfix postfix_file_name] [-disable_wildcard]]
   [-start_formal_cycle [-from_end]]
   [-start_condition expression -end_condition expression]
   [-start_time time] [-time time] [-time_scale (ms | us | ns | ps)]
   [ -hier_path path_difference
     |-hier_map -from source_hier -to destination_hier
   ]
   [-non_resettable_regs value]
Get reset values of registers from the specified sequence file.

All the reset values will be defined by simulating this sequence
for the specified interval or for the entire trace length. This is done
by reading values for all PIs, undriven signals, and initial values
for registers and applying these as inputs to the full design simulation.

No sampling occurs at the boundary between the reset and formal analysis,
that is, between cycle 0 and cycle 1. You must assume responsibility for
setting the correct number of cycles to reach the desired initial state
after reset.

NOTE: To accelerate run times for reset sequence simulation, use
"set_trace_show_reset false".

. Use "-vcd", "-fsdb", "-shm" or "-vwdb" to specify the type of trace to be
  interpreted.

  IMPORTANT:
  . Jasper Apps writes 6.3 FSDB traces and reads version 6.6 FSDB
    traces. FSDB Reader 6.6 was introduced by Verdi version X-2025.06.
  . FSDBs you generate with the latest Jasper are backwards compatible
    with all Jasper versions from 2025.03 FCS and newer. In addition,
    they are backwards compatible with older versions of Verdi released
    during that same period.
  . Jasper Apps supports backwards compatibility, that is, the tool can
    read FSDBs generated from older versions of Jasper Apps and
    Synopsys Verdi.

  . Use "-postfix" to add global assumptions. The postfix assumptions are
    asserted during the proof and have no effect on the reset initial state,
    even if they are listed as reset expressions when using the command
    "reset -list". The postfix files have the same syntax as reset initial
    state files.
. Use "-start_formal_cycle" to specify the cycle of the reset sequence where
```

the formal analysis starts. Running part of the sequence through formal analysis instead of simulation can be useful since X-propagation in simulation is pessimistic and formal analysis can propagate more constants.

. Use "reset -sequence -start_formal_cycle cycle -from_end" to count the start cycle for formal reset downward from the end of the reset sequence.

. Use "-start_time" to specify an integer representing the time point where the tool should start reading the trace. The time is proportional to the timescale defined with `timescale in the testbench. If you do not specify a time, the tool starts from time 0.

. Use "-time" to specify an integer representing the time point where the tool should stop reading the trace. The time is proportional to the timescale defined with `timescale in the testbench. If you do not specify a time, the tool will read the file to the end of the trace.

. Use "reset -sequence (-shm | -vwdb) file_name -time time -exact" to prevent the tool from using the clock's waveform when defining the sampling points of the trace file. This switch forces the tool to use the values of the defined timestamp as the reset state.
NOTE: Only "reset... -shm" and "reset... -vwdb" support the "-exact" switch.

. Use "-time_scale" with the unit of measure (ms, us, ns, or ps) to specify the time scale. If you do not specify a time scale, the tool uses the default from the trace file.

. Trace files are usually generated during simulation where the testbench module is the top module for the verification run, and consequently, for the trace file too. When using a file generated by simulation to reset your design in Jasper Apps, use the optional "-hier_path" switch to specify the hierarchical path difference from the testbench module to the design top module.

  For example, if the design top module is "dut_top", which is instantiated as the top module in the testbench "testbench_top" as "i_dut_top", use the following command to reset the design with the trace file (replace "file_name" with actual value):

```
reset -sequence -vcd  <file_name> -hier_path testbench_top.i_dut_top
reset -sequence -fsdb <file_name> -hier_path testbench_top.i_dut_top
reset -sequence -shm  <file_name> -hier_path testbench_top.i_dut_top
```

```
      reset -sequence -vwdb <file_name> -hier_path testbench_top.i_dut_top

    . Use "-hier_map" for trace files generated during simulation if you have a
      wrapper module.
      . Use "-from" to specify the source (simulation environment) hierarchy.
      . Use "-to" to specify the destination (formal environment) hierarchy.

    . Use "reset -sequence -vcd -include_internal" to load internal signals
      from a VCD file.

      CAUTION: "reset -sequence -vcd -include_internal" carries a risk of a
      mismatch in internal signals that can lead to false proof results in some
      cases. Internal signals in reset files might not be the same as those
      generated by a different Jasper release version or a different
      version of the design or a different design setup and configuration.
      (Design setup and configuration differences include the order in which
      files are passed to "analyze" and "elaborate" and the order in which
      assumptions and assertions are created.)

    . Use "-disable_wildcard" to turn off the wildcard matching feature when
      reading postfix files to (potentially) read files faster.

    . Use the switches "-start_condition" and "-end_condition" to pick the start
      and end times based on the respective conditions.

      NOTE:
      . If a start or end condition is not unique in the trace, the tool
        considers its first occurrence.
      . You can use "-start_condition" and "-end_condition" independently.
      . If you do not specify "-start_condition" the tool uses cycle 0 as the
        initial cycle. If you do not specify "-end_condition", the tool uses
        the last cycle reported in the trace generated from the specified SHM,
        VWDB, FSDB, or VCD file.

    . Use "-non_resettable_regs" to assign the specified value to any register
      with an undefined reset value that is not the result of the "abstract
      -init_value" command.

      NOTE: "value" must be 0 or 1.

    LIMITATION: Compound black-boxed output values will not be restored.

-analyze [-synchronous] [-list (flop | signal)] [-silent] [-task task_name] [-
```

```
conflict]
    Analyze the design's reset logic and return information about
    simple asynchronous reset signals (driven by signal) and complex
    asynchronous reset signals if they are driven by an AND-gated tree.

    . Use "-synchronous" to also return information about synchronous signals.
    . Use "-list flop" to list the design flops that are connected to a
      reset signal.
    . Use "-list signal" to limit the return to a list of reset signals.
    . Use "-silent" to return the list as Tcl return values without
      printing the list on the screen.
    . Use "-task" to apply the command to the specified task instead of the
      current task.
    . Use "-conflict" to identify conflicts in RTL's reset.

    The return value is a list of the reset signals (negated if they are
    active-low). If you use "-list flop", the return inserts a list
    containing the design flops connected to each reset signal with the number
    of bits, for example:

    {~rst Async Simple 6 {{a[1:0]} {a[3]} b} 0} {~rst2 Async Simple 1 c 0}

    In this example, 6 is the total number of bits (3 from "a" and 3 from "b") for
    ~rst, and 1 is the total number of bits (1 from "c") for ~rst2.

    If you use "-list signal", the return is a list of reset signals, for example:

    ~rst ~rst2

    The reset report contains the following information:
    . Signal name - if the inferred reset is driven by an expression
      and represented by an internal signal, the report displays
      the internal signal's fanin instead of the internal signal's name.
    . Type
      . Simple reset - driven by signal.
      . Complex reset - driven by an AND-gated tree.
    . Design flops connected - Number of flops in the design that are affected
      by this signal. For example:

      Design flops connected 2 (6) : a[1:0] a[3] b

      In this example, 2 is the number of nets ("a" and "b") connected to the
      reset signal, and 6 is the total number of bits (3 from "a" and 3 from
```

```
      "b").
    . Number of PSL/SVA flops connected - Number of flops generated as
      observers for SVA and PSL properties that are affected by this signal.

-none [-max_iterations N [-clock clock_signal]] [-non_resettable_regs value]
    Run the reset flow with an empty list of reset signals. This means:

    1) Run constant propagation: design constants and environment assumptions
    2) Run a simulation until it converges (at most 100 iterations)

    . Use "-max_iterations" with an integer to change the default. And use
      "-clock" to specify that you want the number of iterations based on the
      specified clock.

      NOTE:
      . The specified clock must be a declared clock or a propagated clock
        signal.
      . This switch does not accept zero.

    . Use "-non_resettable_regs" to assign the specified value to any register
      with an undefined reset value that is not the result of the "abstract
      -init_value" command.

      NOTE: "value" must be 0 or 1.

    NOTE:
    . This command can be helpful when you intend to Visualize and debug
      reset logic.
    . Be aware that if you use "reset -none"...
      . The tool runs a basic reset simulation and combinational constant
        propagation (which takes into account the global assumptions, global
        stopats, and design constants). And as a result, there will be at least
        one clock iteration of reset prior to the formal analysis, and any
        non-reset registers that are tied to a constant will take on that
        constant value.
      . Reset simulation considers all user-defined clocks and uses the
        propagated constants derived from the global environment and design
        constants. If you do not define clocks, the simulation will be clocked
        only by the internal fastest clock.
      . Defining the clocks can impact reset analysis since they can trigger
        many registers during reset simulation.

-list [signal] [-typed_list] [-silent]
```

```
    Show the current list of reset conditions on the screen.
    The tool displays a list of absolute path file names.

    . Use "signal" to list only reset signal names associated with the reset
      expression(s) in the Tcl return.
    . Use "-typed_list" to add a type element before each of the requested
      lists.
      . The list format is as follows:

        $resetConfig(none) {true/false <is reset none>}
        $resetConfig(expression) {<list of reset expressions>}
        $resetConfig(file_type) {<file type>}
        $resetConfig(sequence) {true/false <is simulated reset trace from file>}
        $resetConfig(file_name) {<file name>}
        $resetConfig(time) {<vcd/fsdb time>}
        $resetConfig(start_time) {<simulated vcd/fsdb start_time>}
        $resetConfig(end_time) {<simulated vcd/fsdb end_time>}
        $resetConfig(hier_path) {<vcd/fsdb hier path>}
        $resetConfig(time_scale) {vcd/fsdb time scale>}
        $resetConfig(max_iterations) {<max iteration config>}
        $resetConfig(non_resettable_regs) {<resettable regs value>}
        $resetConfig(abstract_value) {<abstract bits>}
        $resetConfig(resetDeclared) {true/false <any reset declared>}

        Each of these fields is empty by default ( {} ) except for
        $resetConfig(none) and $resetConfig(reset_declared), which are false or
        true, depending on the reset configuration.
      . Refer to the "Example" section.
    . Use "-silent" to return the list as Tcl return values without printing
      the list on the screen.

    . The "reset -list" command shows postfix assumptions added with the
      "-postfix" switch. Those assumptions are asserted during the proof and
      have no effect on the reset initial state.

-clear
    Remove previously specified reset conditions globally.
```

Example

```
reset -expression ~rst1N rst2
reset -expression {~rst1N rst2}
```

```
reset -expression ~rst1N rst2 -max_iterations 150

reset -list -typed_list
reset expressions:
----------
0:  rst
----------

expression rst max_iterations {} file_type {} sequence {} file_name {} hier_path {}
time {} start_time {} end_time {} time_scale {} non_resettable_regs {} abstract_value
{} none false reset_declared true

// Using wildcards in init_state files. Consider the following init_state
// declaration:

    sigA 4'b0
    sigB 4'b0
    sigC 4'b0

//  You can declare the following:

    sig* 4'b0

//  And if you want to set a value for every signal/input in the design, do the
//  following:

    * 0

# Specifying reset from a VCD file generated by simulation when you have a
# wrapper module
#
# Simulation environment:
# SimulationEnv.TestBench.<Instances_I_want_to_verify>
#
# Formal environment:
# Formal_wrapper.<Instances_I_want_to_verify>
#
# Use the following command:

reset -vcd  my_vcd  -hier_map -from SimulationEnv.TestBench -to Formal_wrapper
reset -fsdb my_fsdb -hier_map -from SimulationEnv.TestBench -to Formal_wrapper
reset -shm  my_shm  -hier_map -from SimulationEnv.TestBench -to Formal_wrapper
reset -vwdb my_vwdb  -hier_map -from SimulationEnv.TestBench -to Formal_wrapper
```

See Also

assume,
clock,
get_reset_info,
prove,
sanity_check,
set_cumulative_reset,
set_reset_effort,
set_trace_show_reset,
stopat

restore

Restores previous sessions given a database.

Syntax

```
restore database
        [-light |-clean]
        [-include ( elaborated_design | rtl_commands | rtl_files
                  | session_data | app_data | jasper_configurations
                  | cache | custom_buttons | sourced_files
                  | tcl_environment | visualize_windows)+]
        [-exclude ( elaborated_design | rtl_commands | rtl_files
                  | session_data | app_data | jasper_configurations
                  | cache | custom_buttons | sourced_files
                  | tcl_environment | visualize_windows)+]
        [-show |-list_designs |-design [-repository id]]
        [-session session_name |-all_sessions]
restore -visualize
        ([-list [-silent]] |
            [-property [-window name])
```

Returns

No return values

Detailed Description

Use this command to restore a previous session saved by the "save" command.

NOTE:
. This command stops periodic updates to existing databases.
. UNR does not save the Xcelium coverage database. Thus, you must ensure
  that you preserve this database between save/restore sessions.

no-argument
    If you do not specify arguments, the tool attempts to restore
    [get_proj_dir]/auto_saved_database.

```
    For apps that don't support automatic saving, the tool only saves data
    before the proof starts.

database [-light |-clean]
         [-include ( elaborated_design | rtl_commands | rtl_files
                   | session_data | app_data | jasper_configurations
                   | cache | custom_buttons | sourced_files
                   | tcl_environment| visualize_windows)+]
         [-exclude ( elaborated_design | rtl_commands | rtl_files
                   | session_data | app_data | jasper_configurations
                   | cache | custom_buttons | sourced_files
                   | tcl_environment| visualize_windows)+]
         [-show |-list_designs |-design [-repository id]]
         [-session session_name |-all_sessions]

    Restores a Jasper database given a specification of components. If no
    switch is provided, the "default" mode is used and the tool tries to load
    all existing data inside the database besides "cache". The tool prints a
    report detailing what was restored from the database.

    . Use "-light" or "-clean" to change the built-in mode. If no switch is
      provided, the tool attempts to restore the default mode.

    . Use "-light" to restore the bare minimum required to restart a new
      session. This mode attempts to restore session data, RTL commands and
      Jasper configurations, expecting that you run the new session within a
      network that has the RTL files in the same locations.

    . Use "-clean" to disable all categories. This mode does not restore any
      data; it is a foundation mode to help when just a few categories are
      desired.

    . Use "-include" and "-exclude" to add or remove a list of categories to be
      restored following the same structure presented in the "save" command.
      The categories "pre_restore_script" and "post_restore_script" are also
      added or removed by these switches.

    . Use "-show" to query save/restore information for all the sessions in a
      database. The tool will list, for each session, the components included
      and their description.

    . Use "-list_designs" to list the available designs. For databases saved in
```

```
      the context of SEC, designs are presented as "SPEC" and "IMP".

    . Use "-design design_name" to load a design in database as a main design.
      Only the "elaborated_design" supports this partial load operation. Do not
      load properties, traces or app data for just one side of the design.

    . Use "-repository id" to restore a design to a specific target respoistory
      in Jasper. If this switch is omitted, the tool loads the design in to the
      main design repository.

    . Use "-session" to restore a specified session from the database. If you
      do not use the "-session" switch, and either there is only one session
      saved in the database or there is one matching the current session name,
      the tool restores the current session. If there is more than one session
      saved in the database, and none of them match the current session name,
      the tool issues an error.

    . Use "-all_sessions" to restore all the sessions saved in the database.

-visualize
    ( [-list [-silent]]
     |[-property [-window window_name])

    Use this command to iterate over restored Visualize configurations from the
   last "restore" command.

    . Use "-list" to list properties that have one or more Visualize traces
      saved. The return is a Tcl dictionary. Use "-silent" to suppress console
      standard output.
    . Use "-property" to open a saved trace for a property. Optionally use
      "-window" to specify a Window name (see "restore -visualize -list").
```

Example

```
restore
restore mydb.jdb -session session_1
restore mydb.jdb -session all
restore mydb.jdb -light -include {app_data}
restore mydb.jdb -exclude {pre_restore_script post_restore_script}
restore mydb.jdb -clean -include {elaborated_design} -design repository_1
restore mydb.jdb -clean -include {elaborated_design} -design repository_1 -repository 0
```

See Also

save,
source_lock

sanity_check

Facilitates the successful setup of the global environment.

Syntax

```
sanity_check (no arguments)

sanity_check -analyze
  (  simple_clk | inactive_clk | gated_clk | complex_clk
   | simple_reset | all)
  [-task task_name]
  [-verbose |-silent ]

sanity_check -waiver
  ( ( -add |-remove) signal_name
        (-type (  simple_clk | inactive_clk | gated_clk | complex_clk
                          | simple_reset | all)
        )
     |-list
     |-clear
  )

sanity_check -auto
  (simple_clk | inactive_clk | gated_clk | complex_clk | all)
  -set_severity_level (ignore | info | warning | error)

sanity_check -auto -disable
  (simple_clk | inactive_clk | gated_clk | complex_clk | all)

sanity_check -auto
  (simple_clk | inactive_clk | gated_clk | complex_clk | all)
  -list
```

Returns

. For "-analyze", this command returns the problematic signals followed by
  suggested actions.
. For "-waiver", this command returns the user-specified waiver type.
. For "-auto", there is no return value.

Detailed Description

Use this command to help set up the global environment. "sanity_check" is an interactive command; that is, you will potentially run several iterations of it. Use each run to clean up certain issues in clock or reset setup. When the command no longer returns problematic signals and suggestions, you have successfully set up your environment.

A clock-related warning indicates that a signal being used as the clock input pin of a flop is connected to one or more primary inputs or undriven signals and none of them is declared as a clock.

A simple reset-related warning indicates that there is an unary expression that can reset the flop and should be declared.

A complex reset-related warning indicates that the tool cannot give the exact expression to reset the flop, but there are declarable primary inputs or undriven signals connected to the flop and none of them are declared as a reset.

This command covers simple clock, unintended constant clock, gated clock, complex clock, D-flop clocks, and simple reset configurations. You can also use this command to waive intended constant clocks, gated clocks, D-flop clocks, and undeclared clocks and resets.

no arguments
    If you do not specify arguments, this command runs "sanity_check -analyze all".

-analyze (simple_clk | inactive_clk | gated_clk | complex_clk | simple_reset
          | all)
         [-task task_name]
         [-verbose |-silent]
    Return a list of problematic items that were not appropriately treated during the environment setup.

```
    . Use "simple_clk" to direct the analysis to consider undeclared simple
      clock signals. These include primary inputs, undriven signals due to
      stopats, inconsistent clock edge declarations, and clocking signals that
      come from a black-box output.
    . Use "inactive_clk" to direct the analysis to consider unintended constant
      clocks.
    . Use "gated_clk" to direct the analysis to consider undefined gated clocks,
      that is, AND/OR gated clocks without a declared clock in their inputs.
    . Use "complex_clk" to direct the analysis to consider complex clock
      structures.
    . Use "simple_reset" to direct the analysis to consider undefined simple
      reset signals, undefined simple AND reset expressions, and undefined
      reset polarities, including asynchronous and synchronous resets.
      Each reset expression returned is followed by its type, which can be any
      of the following:
      . Sync      : Synchronous
      . Async     : Asynchronous
      . Async/Sync: Both asynchronous and synchronous
      NOTE: "simple_reset" can warn you about some types of complex reset.
    . Use "all" to direct the analysis to consider all of the types listed
      above.
      NOTE: This command returns the result for each clock analysis type
      separately, in an iterative way, in the following sequence: simple reset,
      simple clock, inactive clock, gated clock, and complex clock.
    . Use "-task task_name" to run the analysis for the specified task. If you
      do not specify a task, "sanity_check" runs the analysis for the global
      environment configuration.
    . Use "-verbose" to report the problematic signals followed by suggested
      actions.
    . Use "-silent" to hide the reporting information and issue only the Tcl
      return that includes the problematic signals.

-waiver (( -add |-remove) signal_name
             (-type (  simple_clk | inactive_clk | gated_clk | complex_clk
                     | simple_reset | all))
         |-list
         |-clear)
    Run the specified action on waivers or get the specified information.
    . Use "-add" to add the specified signal with the specified waiver type to
      the waiver list.
    . Use "-remove" to remove the specified signal with the specified waiver
      type from the waiver list.
    . Use "-list" to return a list of all waived signals and their types.
```

```
    . Use "-clear" to remove all waivers.

-auto (simple_clk | inactive_clk | gated_clk | complex_clk | all)
         -set_severity_level (ignore | info | warning | error)
    Run the clock sanity check analysis during the proof flow (that is, while
    running "prove" and "visualize" commands) and set the message severity
    level for the issues found.

    . Use one of the following to specify the type of clock analysis you want
      "sanity_check" to run:
      . simple_clk
      . inactive_clk
      . gated_clk
      . complex_clk
      . all

    . Use "-set_severity_level" to set the severity level for messages
      generated by the sanity check.
      NOTE: You can also use the command "set_message" to change the severity
      levels for the related messages.

-auto -disable (  simple_clk | inactive_clk | gated_clk | complex_clk
                | all)
    Disable the specified sanity check analysis.

-auto (simple_clk | inactive_clk | gated_clk | complex_clk | all)
        -list
    List the severity level set for the specified type of clock analysis.

    NOTE: Use "all" to list all levels.
```

Example

```
sanity_check -analyze inactive_clk

# Adding an intended constant clock to the waivers list:
sanity_check -waiver -add sigConst -type inactive_clk

sanity_check -waiver -remove sigConst -type inactive_clk
sanity_check -waiver -list

# Interactive Example
```

```
# --------------------
# Reading in the design and using "sanity_check" runs to
# identify and resolve issues
analyze -sv test.v
elaborate
reset rst

# No clock has been specified, so the following check fails
sanity_check -analyze gated_clk -verbose
Clocks with mux logic found: 3

Clock: reg_flop_mux_input_clk
Undriven signals in the fanin of the clock: sel

Clock: reg_pi_flop_mux_input_clk
Undriven signals in the fanin of the clock: clk sel

Clock: reg_pi_flop_mux_input_two_clk
Undriven signals in the fanin of the clock: clk clk4 sel
Use the "clock" command to declare one of the undriven signals as a clock or
use "sanity_check -waiver -add" to ignore this error.
For mux clocks, you can also run "assume -env" or "assume -constant" on the
selector signal.
{reg_flop_mux_input_clk sel} {reg_pi_flop_mux_input_clk {clk sel}}
{reg_pi_flop_mux_input_two_clk {clk clk4 sel}}

# Declaring two clocks to fix two issues
clock clk4
clock clk

# Running "assume -env" on the selector signal.
assume -env {sel==1}

INFO (IPM005): The name "assume:0" is assigned to assumption "sel==1".
assume:0

# Re-running "gated_clk" analysis
sanity_check -analyze gated_clk
No errors found in gated clock analysis.
```

See Also

clock,
clock -analyze,
reset,
reset -analyze,
set_message

save

Saves the database, session, or script.

Syntax

```
save database
     [-light |-clean]
     [-include ( elaborated_design | rtl_commands | rtl_files
                | session_data | app_data | jasper_configurations
                | cache | custom_buttons | sourced_files
                | tcl_environment | visualize_windows)+]
     [-exclude ( elaborated_design | rtl_commands | rtl_files
                | session_data | app_data | jasper_configurations
                | cache | custom_buttons | sourced_files
                | tcl_environment | visualize_windows)+]
     [-pre_restore_script script_path]
     [-post_restore_script script_path]
     [-dry_run]
     [-all_sessions]
     [-periodic [-period time] |-status]
     [-force]
save -periodic -stop
save -script file_name
          [-force]
          [-check]
          [-task task_name]
          [-ar]
          [-prove |-prove_task task_name |-prove_property property_name]
          [-sst]
          [-elaborated_design dir_name [-force]]
```

Returns

No return values

Detailed Description

Use this command to save the current session or generate a Tcl script that can be used for regression proofs.

When you use the "save" command, the tool runs a checksum of all opened files. When you attempt to restore a database, the tool checks opened files and files in the checksum file against the saved checksum.

NOTE:
. UNR does not save the Xcelium coverage database. Thus, you must ensure that you preserve this database between save/restore sessions.

```
database [-light |-clean]
         [-include ( elaborated_design | rtl_commands | rtl_files
                   | session_data | app_data | jasper_configurations
                   | cache | custom_buttons | sourced_files
                   | tcl_environment| visualize_windows)+]
         [-exclude ( elaborated_design | rtl_commands | rtl_files
                   | session_data | app_data | jasper_configurations
                   | cache | custom_buttons | sourced_files
                   | tcl_environment| visualize_windows)+]
         [-pre_restore_script script_path]
         [-post_restore_script script_path]
         [-dry_run]
         [-all_sessions]
         [-periodic [-period time] |-status]
         [-force]
```

Saves a Jasper database given a specification of components to be saved. If no switch is provided, the tool uses the default mode and saves all components aside from "cache" are saved. The data included in the database depends on the specified components, configurations in the current session and the built-in mode chosen to filter out components.

Use the built-in modes "-clean" or "-light" to filter out categories automatically and reduce the number of arguments passed to "-exclude". In the default mode, the tool saves more detailed data information about each mode as described below.

Built-in modes:

. default: Saves most common save/restore data required to restart work in a new session with minimal changes. This includes the elaborated design, session data, and all environment data.

. light: Saves the bare minimum required to restart a new session, aiming for a lightweight database file. This mode saves session data, Jasper configurations and RTL commands, expecting that you will run the new session within a network that has the RTL files in the same locations. However, different apps might customize this based on their own needs and save more information.

. clean: This mode does not save any data. This is a foundation mode to help when just a few components are needed.

The description of the available components captured by this command and their default values for the default mode are presented in the following list:

Available Components and Possible Values

. elaborated_design (on | off): Include the elaborated design, which makes it possible to skip elaboration during "restore". This component also includes LPV and Liberty databases.

. rtl_commands (on | off): Include the commands used to compile the elaborated design. You can rerun them on the restored session instead of using the "elaborated_design" file.

. rtl_files (on | off): Include the RTL and Liberty files if Source Lock is enabled in the current session. See "source lock" for additional details.

. session_data (on | off): Include data relative to the current session:
  . Property results and traces
  . checksum (see information about checksum above)

. app_data (on | off): Specific App and flows data, such as C2RTL, CDC, Combo Loop, CONN, CSR, Expert System, FSV, SEC, SPV, and X-Prop.

. jasper_configurations (on | off): Include configurations set for the current session:
  . Configuration commands
  . Message severities
  . Compilation Configuration "compilation -config"

. cache (off | on): Include the folders specified by "set_proofmaster_dir" and "prove_cache_path". During "restore", the folders are copied to the

```
  default value of these configurations (inside the project directory).
  This is the only component disabled by default.

. custom_buttons (on | off): Include custom GUI actions set for the GUI of
  this current session. If running in batch mode, no data is included and
  this is ignored.

. sourced_files (on | off): Include the input Tcl script and its sourced/
  included files. During "restore", the files are copied back to a folder
  inside the session's project directory.

. tcl_environment (on | off): All Tcl user data, including namespaces,
  procs, rename operations, aliases, and variables.

. visualize_windows (on | off): Saves the recipe of all Visualize windows
  opened. During "restore", the scripts are copied back to a folder inside
  the session's project directory.

When the save operation is concluded, the tool prints a report detailing
what was saved on the database. To preview the components included given
the switches, the description of each component and a list of sessions
included in the database, use the option "-dry_run".

For Miter apps (C2RTL, SEC, SPV, and XPROP), the minimum valid "save"
configuration must have "app_data", "session_data", and "elaborated_design"
enabled.

For the FSV app, "elaborated_design" can be saved on its own, but "app_data"
must be saved together with "session_data" and "elaborated_design", and
"session_data" must be saved with "app_data" and "elaborated_design".

The database file is a multi-session file. When saving a session with a
name that already exists within an existing database, the tool generates an
error message. Otherwise, it appends the current session database to the
file. Use the "-force" option to allow overwriting and the "-all_sessions"
option to save all sessions simultaneously.

NOTE:
. For the "rtl_files" mode config to work, you must enable Source Lock
  with "source_lock -enable" before "analyze" and "liberty" commands.
. Use "-pre_restore_script" and "-post_restore_script" to include custom
  Tcl files that execute before or after the "restore" command.
. Use "-periodic" to enable Jasper to update proof results periodically in
```

```
      the database. Only one periodically updated database must exist. This
      feature requires "session_data" to be "on". The default period of time is
      equal to 30 minutes.
    . When you use "-periodic", data is periodically saved through the activity
      of the next "prove" command only. After that proof completes, there are
      no further automatic saves, and you must invoke the "save" command again
      if you want to save an updated Jasper state later.
    . "-periodic" is not supported in all Miter apps (C2RTL, FSV, SEC, SPV,
      and XPROP).
    . Use "-period" to specify an interval of time between two consecutive
      updates in database. The value must be higher than 30 minutes.

-periodic -stop
    Stops periodic updates in any periodically updated database.

    NOTE: The following commands also stop periodic updates:
    . "clear"
    . "restore"
    . "check_conn -clear

-periodic -status
    Returns the status of periodic saves. If a periodically updated database
    exists, returns its status, the session it is associated with, its path,
    and the period. If no periodically updated database exists or if periodic
    save was deactivated, the status indicates that as well.

-script file_name [-force] [-check] [-task task_name] [-ar]
                  [-prove |-prove_task task_name |-prove_property property_name]
                  [-sst] [-elaborated_design dir_name [-force]]
    Create a Tcl file for restoring the analysis state and checking
    proofs.

    NOTE: You can read in the Tcl file with the Tcl "source" or "include"
    command. The script also expands the "prove" command into separate
    justifications, if applicable, for regression testing. The tool writes the
    file to the directory in which you invoked it.

    . Use "-force" to overwrite any existing file that has the specified name.

    . Use "-check" when using one of the following options, " -prove",
      "-prove_task", or "-prove_property", to add a "check_code" command for
      each property that has a valid result.
```

. Use "-task" to only generate Tcl commands to recreate the specified task.

  NOTE:
  . If you use this option with "-prove", this command only generates
    "prove" commands for the specified "task_name".
  . Replace "task_name" with a period (.) to specify the current task.

. Use "-ar" to only generate Tcl commands to set up analysis region
  information.

  NOTE: If you use this option, the tool does not save commands for reading
  in HDL files.

. Use "-prove" to add a "prove" command that proves all properties.

. Use "-prove_task" to add a "prove" command that proves the properties in
  the specified task.

. Use "-prove_property" to add a "prove" command that proves the specified
  property.

. Use "-sst" to save an SST script that will reproduce the current state
  with helper assertions and so forth.

. Use "-elaborated_design" to copy the elaborated design to the specified
  directory. And use "-force" to overwrite any existing directory that has
  the specified name.

  "-elaborated_design" makes it possible to restore an elaborated design
  without re-elaborating it.

  NOTE: If Source Lock is enabled, RTL Files are included by default in the
  save directory. See "set_save_restore_capture_source_lock_rtl" for
  additional details.

Example

```
save -script my_proof

# ------------------------------------------------------------------------------
# Use the following command to save your database to a specified directory
# (my_db).

save my_db

# ------------------------------------------------------------------------------
# Use the following command to save your database to a specified directory
# (my_db) and include "cache" for ProofMaster and prove cache.

save my_db -include {cache}

# ------------------------------------------------------------------------------
# Use the following command to save just the elaborated design.

save -clean my_db -include {elaborated_design}

# ------------------------------------------------------------------------------
# For Miter apps (C2RTL, SEC, SPV, and XPROP), the minimum valid "save"
# configuration must have "app_data", "session_data", and "elaborated_design"
# enabled.
#
# For the FSV app, "elaborated_design" can be saved on its own, but "app_data"
# must be saved together with "session_data" and "elaborated_design", and
# "session_data" must be saved with "app_data" and "elaborated_design".

save my_db -clean -include {app_data session_data elaborated_design}
```

See Also

restore,
set_proofmaster_dir,
set_prove_cache_path,
source_lock

schematic_viewer

Draws a schematic of a portion of or an entire loaded design.

Syntax

```
schematic_viewer -draw [-instance instance_name]
                       [-task task_name]
                       [ -window window_name
                        |-new_window [new_window_name]]
                       [-visualize_window visualize_window_name]

schematic_viewer -signal signal_list [-all |-driver |-port]
                                     [-full_hierarchy]
                                     [-task task_name]
                                     [ -window window_name
                                      |-new_window [new_window_name]]
                                     [-visualize_window visualize_window_name]

schematic_viewer -path
                 -from from_signal
                 -to to_signal
                 [-through through_signal_list]
                 [-task task_name]
                 [ -window window_name
                  |-new_window [new_window_name]]
                 [-visualize_window visualize_window_name]

schematic_viewer -open [-task task_name]
                       [ -window window_name
                        |-new_window [new_window_name]]

schematic_viewer -export -file filename
                 [-force]
                 [-save_highlights]
                 [-window window_name]
                 [-format [svg | pdf]]
                 [-color  [  original | monochromatic | inverted_luminance
                                      | inverted_colors]]
                 [-scale [whole | visible_portion]]
                 [-context_info]

schematic_viewer -list

schematic_viewer -clear [-window window_name]

schematic_viewer -delete [-window window_name]
```

Detailed Description

```
Use this command to open a schematic viewer window and draw the top module
schematic, its instances, or specific signals.

-draw [-instance instance_name]
      [-task task_name]
      [-window window_name |-new_window [new_window_name]]
      [-visualize_window visualize_window_name]

    Draw the schematic of the top module on the schematic viewer window.
    . Use "-instance" to draw a specified instance instead of the top module
      on the schematic viewer window.
    . Use "-task" to draw the schematic of the specified task on the schematic
      viewer window.
    . By default, the tool applies your "schematic_viewer" command to the
      current window. Use "-window" to apply your command to a window other
      than the current window.
    . Use "-new_window" to show the "schematic_viewer" in a new window with
      a name specified by the optional new_window_name argument.
      NOTE: If you do not provide the new_window_name argument,
      a tool-generated name is used.
    . Use "-visualize_window" to draw the schematic coupled to a specific
      Visualize window.
      NOTE: The Schematic Viewer uses constant annotation based on the
      Visualize red current marker.

-signal signal_list [-all |-driver |-port]
                    [-full_hierarchy]
                    [-task task_name]
                    [-window window_name |-new_window [new_window_name]]
                    [-visualize_window visualize_window_name]

    Draw the schematic of the specified signal list on the schematic viewer
    window.

    . Use "-all" to draw all gates connected to the signals specified by the
      signal list.
    . Use "-driver" to draw the gates that drive the signals specified by the
      signal list.
    . Use "-port" to draw the ports of the signals specified by the signal
      list.
    . Use "-full_hierarchy" to draw all instance levels related to the signals
```

```
      specified by the signal list.
    . Use "-task" to draw the schematic in relation to the specified task.
    . Use "-window" to specify a window other than the default window.
    . Use "-new_window" to show the "schematic_viewer" in a new window with
      a name specified by the optional new_window_name argument.
      NOTE: If you do not provide the new_window_name argument,
      a tool-generated name is used.
    . Use "-visualize_window" to draw the schematic coupled to a specific
      Visualize window.
      NOTE: The Schematic Viewer uses constant annotation based on the
      Visualize red current marker.

-path -from from_signal -to to_signal
      [-through through_signal_list]
      [-task task_name]
      [-window window_name |-new_window [new_window_name]]
      [-visualize_window visualize_window_name]

    Show one path between two signals.

    If there is no path between the signals, the viewer displays the
    signals themselves and their driving logic.

    . Use "-through" to draw one schematic path between the from and
      to signals going through the "through_signal_list" sequence of signals.
    . Use "-task" to draw the schematic in relation to the specified task.
    . Use "-window" to specify a window other than the default window.
    . Use "-new_window" to show the "schematic_viewer" in a new window with
      a name specified by the optional new_window_name argument.
      NOTE: If you do not provide the new_window_name argument,
      a tool-generated name is used.
    . Use "-visualize_window" to draw the schematic coupled to a specific
      Visualize window.
      NOTE: The Schematic Viewer uses constant annotation based on the
      Visualize red current marker.

-open [-task task_name] [-window window_name |-new_window [new_window_name]]

    Open the specified schematic viewer window.

    . Use "-task" to open a task other than the current task.
    . Use "-window" to specify a window other than the default window.
    . Use "-new_window" to show the "schematic_viewer" in a new window with
```

```
      a name specified by the optional new_window_name argument.
      NOTE: If you do not provide the new_window_name argument,
      a tool-generated name is used.

-export -file filename [-force]
      [-save_highlights]
      [-window window_name]
      [-format (svg | pdf) ]
      [-color (   original | monochromatic | inverted_luminance
                | inverted_colors)]
      [-scale (whole | visible_portion)]
      [-context_info]

    Export the specified schematic viewer window.

    . Use "-file" to export to the specified file. And use "-force" to
      overwrite an existing file.
    . Use "-save_highlights" to save the schematic highlights.
    . Use "-window" to export a window other than the default window.
    . Use "-format" to define the file format.
    . Use "-color" to define the file color mode.
    . Use "-scale" to specify how much of the schematic to export, the whole
      schematic or the visible portion only.
    . Use "-context_info" to export schematic with window context information
      if it exists.
      NOTE: This switch applies to the CDC App only.

-list

    List the names of all schematic viewer windows. The order of the elements
    in the list is not defined.

-clear [-window window_name]

    Clear the contents of the specified schematic viewer window.

    Use "-window" to clear a window other than the default window.

-delete [-window window_name]

    Close the specified schematic viewer window.

    Use "-window" to close a window other than the default window.
```

Example

```
schematic_viewer -draw -instance f1
schematic_viewer -signal top_i.v5 -all -window {my_window}
```

See Also

set_schematic_viewer_collapse_combo_logic,
set_schematic_viewer_show_collapsed_logic_as_cloud,
set_schematic_viewer_show_function_nets,
set_schematic_viewer_show_instance_unconnected_pins,
set_schematic_viewer_show_register_name

session

Manipulates analysis sessions.

Syntax

```
General Session Manipulation
----------------------------
session -refresh
session -new [session_name] [-remote]
        (-define tcl_variable tcl_variable_value)*
        [-callback_on_close cmds]
        [-dont_echo_callback_on_close]
        [-dont_use_as_current_session]
        [-reuse_elaborated_design]
session -close [session_name]
session -run_cmds [session_name] commands
        [-use_hier_path
        |-add_hier_path path
        |-remove_hier_path path]
        [-callback cmds]
        [-dont_echo]
        [-dont_echo_callback]
session -run_script tcl_script+
        [-use_hier_path
        |-add_hier_path path
        |-remove_hier_path path]
session -clear_msg

Support for Exporting Session Configuration
-------------------------------------------
session -set_name session_name
session -get_name
session -get_setup
session -get_proj_dir
session (-export_sva [-keep_hierarchy] |-export_psl)
        [(-task task_name) |-env]
        (-file file_name) [-force]

Support for Setting Up the Database for a Different Design Hierarchy
```

```
-------------------------------------------------------------------
session -clear_hier_path
session -set_hier_path [-add |-remove] path

Support for Configuring How a Remote Session Should be Launched
---------------------------------------------------------------
session -set_remote_mode (local | shell | lsf | oge)
session -set_remote_shell full_path_to_shell_plus_args
session -get_remote_mode
session -get_remote_shell

Plugin Configuration
--------------------
session -load_plugin plugin_list -plugin_dir plugin_dir
```

Returns

Various

Detailed Description

Use this command to manipulate analysis sessions.

```
Session Identifiers and Names
-----------------------------
The tool uses session identifiers in the form session_N and stores their
associated log and command files in the project directory, for example,
jgproject/sessionLogs/session_0. For convenience, you can specify a session
name for the current session (session -set_name mySession) and launch a new
session with a specified name (session -new myOtherSession), but the session
identifiers in the project directory remain unchanged.

General Session Manipulation
----------------------------
-refresh
    Refresh the analysis session with the default setup in the database and
    reload the RTL.

-new [session_name] [-remote] (-define tcl_variable tcl_variable_value)*
    [-callback_on_close cmds] [-dont_echo_callback_on_close]
    [-dont_use_as_current_session] [-reuse_elaborated_design]
```

```
Launch a new analysis session with the specified name.

If you do not specify a name, the tool names the new session "session_n",
where "n" is a number greater than 0. This command returns the name of the
new session.

. Use "-remote" to start a new session with the user-specified
  launcher that you specified with the "-remote_mode" switch at
  startup or the "session -set_remote_mode" command. If you use mode
  lsf or oge, or have a shell script that calls your server farm, the
  tool can support multiple analysis sessions starting on different
  machines.

. Use "-define" to start a new session and use the specified values for the
  specified Tcl variables.

  Implementation guidance:
  . The "-define" switch is especially useful in launching two analysis
    sessions, each with a different version of the RTL so you can bring
    up the corresponding waveforms for comparison.
  . Define multiple variables by using one "-define" switch for each
    definition.

. Use "-callback_on_close" to register commands to be executed in the
  current analysis session when the launched session closes or exits
  unexpectedly.

. Use "-dont_echo_callback_on_close" to suppress command echoing for
  the commands registered with "-callback_on_close". Without this switch,
  the tool echoes the commands on the console of the analysis session that
  executed the "session -new" command.

  NOTE: "-callback_on_close" is a prototype and has limitations.

. Use "-dont_use_as_current_session" to prevent switching to the created
  session and keep the analysis session executing "session -new" command as
  the current session.

. Use "-reuse_elaborated_design" to automatically load the design that was
  elaborated in the current session without re-elaborating it.

  NOTE: "session -reuse_elaborated_design" is an initial release to gather
```

```
      feedback from early adopters and finalize implementation for an upcoming
      release. Contact support@cadence.com to provide feedback or comments.

-close [session_name]
    Close the session with the user-specified session name.

    Important things to know about this command:
    . If multiple sessions with the same name are present, this command
      closes the one launched earliest.
    . If you do not provide a session name, the session that executed this
      command closes.

-run_cmds [session_name] commands
    [-use_hier_path |-add_hier_path path |-remove_hier_path path]
    [-callback cmds] [-dont_echo] [-dont_echo_callback]

    Run the specified commands in the session with the specified name.
    . Return "true" if the specified session exists.
    . Return "false" if the specified session does not exist.

    . Use "-use_hier_path", "-add_hier_path", or "-remove_hier_path" to
      execute scripts generated for an RTL with a different hierarchy.
      See "session -set_hier_path" for determining the value of the path
      argument.

    . Use "-dont_echo" to suppress command echoing on the console of the
      specified analysis session.

    . Use "-callback" to register commands to be executed in the
      current analysis session when the session with the matching name
      has completed the specified commands.

    . Use "-dont_echo_callback" to suppress command echoing for the callback
      commands. Without this switch, the tool echoes the commands on the console
      of the current analysis session.

      NOTE: "-callback" is a prototype and has limitations.

-run_script tcl_script+ [ -use_hier_path |-add_hier_path path
                        |-remove_hier_path path]
    Run the Tcl script in batch so that "assume", "assert", and "cover"
    commands do not attempt to populate the database.
```

```
    NOTE: Run a structural analysis after the script if you want to
    populate the database with the assert, assume, and cover, elements.

    Use "-use_hier_path", "-add_hier_path", or "-remove_hier_path" to
    execute scripts generated for an RTL with a different hierarchy.
    See "session -set_hier_path" for determining the value of the path argument.

-clear_msg
    Clear all message log panes (all tabs) for the active session.

Support for Exporting Session Configuration
-------------------------------------------
-set_name session_name
    Set the name of the current analysis session.

-get_name
    Return the name of the current analysis session.

-get_setup
    Return the content of the setup script used to set up the current
    session.

-get_proj_dir
    Return the project directory specific to the current session.

    Compare this command to the "get_proj_dir" command, which returns the
    project directory specific to the tool.

-export_sva [-keep_hierarchy] |-export_psl
    [-task task_name |-env] -file file_name
    [-force]

    Translate the session's Tcl properties into SVA or PSL and
    save them in the specified file.

    Important things to know about this command:
    . Tcl properties are created with Jasper "assume", "assert",
      and "cover" commands.
    . If you specify "-export_sva", bind the resulting SVA to the RTL file
      with SVA "bind".
    . Use "-keep_hierarchy" to export SVA that directly references signals
      through hierarchy rather than mapping SVA signals through ports binding,
      which is the default behavior.
```

```
    . If you specify "-export_psl", analyze the new PSL file along with the
      design to see the extracted properties in their environment.
    . By default, this command applies to the current task.
      Use the optional "-task" switch to translate properties that belong
      to a task other than the current task.
    . Use "-env" to extract only the assumptions created with "assume -env".
      NOTE: Environmental (global) assumptions are not included by default.
    . Use "-force" to overwrite any existing file that has the specified name.


Support for Setting Up the Database for a Different Design Hierarchy
--------------------------------------------------------------------
-clear_hier_path
    Clear the hierarchical path for this session.
    The tool automatically runs this command right before running the
    Default Setup script in the database.

-set_hier_path [-add |-remove] path
    Set the hierarchical path for this session. This path represents the
    difference between the data captured in the Behavioral Indexing Database
    and the hierarchy of the RTL in the current session. For example,

    . If an embedded property has task::top.a.b.c, and the current session
      only uses task::top2.c, use the command "session -set_hier_path -remove
      top.a.b".

    . If an embedded property has task::top.c, and the current session
      only uses task::top2.a.b.c, use the command "session -set_hier_path -add
      top2.a.b".

    Add this command to the default setup script of the database,
    depending on how the RTL is elaborated according to the Tcl variables
    that can be configured by the "session -define" commands.

    NOTE: If you do not specify either "-add" or "-remove", the default is
    "-add".

Support for Configuring How a Remote Session Should be Launched
---------------------------------------------------------------
-set_remote_mode (local | shell | lsf | oge)
    Set up the configuration for "session -remote -new". This is similar to
    "set_proofgrid_mode", but it applies to session management, rather than
    ProofGrid management.
```

```
    local
        Launch remote analysis session on your local machine (the machine
        running the Jasper Apps session).

    shell
        Use a custom command or script to spawn the jobs, see
        "session -set_remote_shell".

    lsf
        Launch remote analysis sessions on the Platform LSF(R) cluster that is
        expected to be available using the LSF_BINDIR environment variable with
        LSF_BINDIR/bsub.

    oge
        Launch remote analysis session on the Oracle(R) Grid Engine (OGE) that
        is expected to be available using the SGE_ROOT environment variable,
        with SGE_ROOT/bin/arch/qrsh.

        NOTE: This option was previously named "sge", which stands for Sun(R)
        Grid Engine.

-set_remote_shell full_path_to_shell_plus_args
    The argument must be a string beginning with the full path to
    an executable. As an option, that string can continue with a
    space-separated set of extra arguments that will be given to the
    executable. You can use double and single quotes to form an argument
    that contains spaces. The executable must take the job to start and its
    arguments as further arguments. See also "set_proofgrid_socket_communication".

-get_remote_mode
    Return the current setting from the "-set_remote_mode" switch.

-get_remote_shell
    Return the current setting from the "-set_remote_shell" switch.

Plugin Configuration
--------------------
-load_plugin plugin_list -plugin_dir plugin_dir
    Load the plugins (provided by Cadence) from the plugin directory, including
    but not limited to "new proof accelerators" to be instantiated and connected
    to the RTL and "new tcl commands" to be called from the Tcl interface.
```

Example

```
session -new session2
session -run_cmds session2 "source my.tcl"

// Running a new session in remote mode with LSF:
session -set_remote_mode lsf
session -remote -new session2

// Running a new session in remote mode with a shell script:
session -set_remote_mode shell
session -set_remote_shell "my.script -arg abcd"
session -remote -new session2
```

See Also

database

set_current_gui

Set the current GUI view to the specified app view.

Syntax

```
set_current_gui (  afv | c2rtl | caf | cdc | cfpv | conn | cov | csr | fpv | fsv
                 | hls | lpv | mcp | rvv | sec | spv | superlint | unr | xprop)
```

Returns

No return values.

Detailed Description

Use this command to change the current GUI view. When an app has already been launched, this command is essentially the same as clicking its tab in the main window. If the specified app has not been launched, this command launches it.

```
. afv       - Architectural Formal Verification App
. c2rtl     - C to RTL Equivalence Checking App
. caf       - C Autoformal App
. cdc       - Clock Domain Crossing Verification App
. cfpv      - C Formal Property Verification App
. conn      - Connectivity Verification App
. cov       - Coverage App
. csr       - CSR Verification App
. fpv       - Formal Property Verification App
. fsv       - Functional Safety Verification App
. hls       - HLS Equivalence Checking App
. lpv       - Low Power Verification App
. mcp       - Multicycle Path Verification App
. rvv       - RISC-V Verification App
. sec       - Sequential Equivalence Checking App
. spv       - Security Path Verification App
. superlint - Superlint App
. unr       - Coverage Unreachability App
. xprop     - X-Propagation Verification App
```

See Also

get_tool

show_list

Formats list values returned by a Tcl or Jasper command.

Syntax

```
show_list '[' command ']' [-unsorted]
```

Returns

No return values

Detailed Description

Use this command to format the list of values returned by a Tcl or Jasper command with a carriage return after each item in the list.

This command is useful for formatting a list of values that is going to be printed to a log file.

command [-unsorted]
    Format the list of values returned by the specified command.

    Use "-unsorted" to return the list in its original order.

Example

```
show_list [ dglob ]
```

show_source_browser

Shows the source browser window.

Syntax

```
show_source_browser [(-instance instance_name
                     |-property property_name
                     |-signal signal_list)]
```

Returns

No return values

Detailed Description

Use this command to show the source for a specified instance, property, or set of signals in the Source Browser window. If you do not specify an instance, property, or signal, the tool opens the Source Browser window in its current state.

NOTE: This command has no effect in batch mode.

Example

```
show_source_browser
show_source_browser -instance instanceA
show_source_browser -property propertyA
show_source_browser -signal {signalA signalB signalC}
```

source_lock

Enables a Source Lock on loaded design files.

Syntax

```
source_lock -enable [lock_dir]

source_lock -disable

source_lock -status
```

Returns

The output of a requested action.

Detailed Description

Use the "source_lock -enable" command before the "analyze" and "liberty" commands to enable a Source Lock on loaded design files. Every analyzed file is copied to a lock directory, and debugging features will refer to the locked version of the file. In addition, Visualize and Source Browser features will refer to the same original compiled file, even if the files are modified after compilation.

NOTE: This command works as expected in the default Jasper source browser window, and in all windows of FPV, SPV, X-Prop, and SEC.

-enable [lock_dir]
    Enable the Source Lock feature for analyzed files.
    If you do not specify the lock_dir, the default is the project directory.

      NOTE:
      . You must use this command before the "analyze" and "liberty" commands.
      . If you run the command after running "analyze", the tool issues a warning to inform you that Source Lock will be enabled on the next compilation run.
      . When enabled, a padlock icon on the bottom right of the Source Browser window indicates that the file being read is the locked version of that file.
      . When enabled, "analyze -save" and "analyze -restore" commands do not work. You must disable Source Lock before using these commands.
      . Only RTL, Liberty files, and files included by "-f" and "-f_relative_to_file_location" that are visible in the Source Browser are locked. UPF, IP-XACT, and other files not part of the RTL source or loaded Liberty are not locked.

-disable
    Disable the Source Lock feature for the next compilation run.

-status
    Return a Tcl dictionary containing the status of Source Lock and the current lock directory.

    NOTE:
    . Dictionary keys are "status" and "lock_dir".
    . Possible values for the status are "on" and "off".

Example

```
source_lock -enable
source_lock -enable [get_proj_dir]/my_lock_dir
source_lock -disable
source_lock -status

// Before enabling Source Lock
source_lock -status
status off lock_dir /proj_dir/sessionLogs/session_0/source_lock

// After enabling Source Lock
source_lock -enable [get_proj_dir]/my_lock_dir
source_lock -status
status on lock_dir /proj_dir/my_lock_dir/source_lock
```

See Also

set_save_restore_capture_source_lock_rtl

sst

Checks that SST traces are still valid or searches for an invariant.

Syntax

```
sst ( -check -property property_name
            [-trace_id trace_id] [-clear]
            [-cycle N]
     |-generate -signals signals
            [-name name]
            [-file file_name]
            [-precondition expression]
            [-reset expression]
            [-load |-clear]
            [-bg]
            [-time_limit time_limit]
            [-no_helper] [-decompose]
    )
```

Returns

For "-check", this command returns a Tcl dictionary where the key is the invalidated trace ID and the value is a list with the helper assertions causing the invalidation.

For "-generate", this command returns a Tcl dictionary with the following possible keys: "filename", "thread_id", "status", and "properties". The "filename" and "thread_id" keys occur when the generation was launched in the background using "-bg". Any properties that are created will be added as a list for "properties" key. The "status" key gives a generic result summary for the command.

Detailed Description

```
-check -property property_name
       [-trace_id trace_id]
       [-clear]
```

```
       [-cycle N]
    Use this command to check whether new helper assertions invalidate an
    existing SST trace. The trace is invalid if it violates a helper assertion.
    . Use "-property" to specify the property to check if the SST trace violates
      any helper assertion. By default, the command checks all SST traces from
      the property.
      A trace can be related to multiple properties.
    . Use "-trace_id" to check a specific trace.
    . Use "-clear" to clear all invalid traces from the property status.
    . Use "-cycle" to set a cycle threshold for failure checks. If a violation
      occurs in a cycle before the specified value, the trace becomes invalid.
      If no value is provided, the tool uses the default value from
      "get_sst_default_trace_length".
    NOTE: The command does not support liveness properties, and cannot report if
    a trace is invalid due to a liveness helper violation.

-generate -signals signals
       [-name name]
       [-file file_name]
       [-precondition expr]
       [-reset expr]
       [-load |-clear]
       [-bg]
       [-time_limit time_limit]
       [-no_helper]
       [-decompose]
    Use this command to search for a state invariant (a property that must hold
    true for specified "signals" in every reachable state) expressed over
    "signals" without having to specify the exact expression that makes up the
    invariant.
    While the search is ongoing, the information collected is incrementally
    stored to a file. The default path and name used for the storage is
    {project}/work/helpers/{name}.dat. The invariants are created as assertions
    and will need to be proven along with any other user properties.
    . Use "-name name" to specify the base property name to be used.
    . Use "-file file_name" to override the path and name of the file used to
      store and retrieve the captured values.
    . Use "-precondition" to specify an optional precondition (the invariant
      only needs to hold when the precondition is true).
    . Use "-reset" to specify a "disable iff (expr)" condition to be used.
    . Use "-load" to create properties from the contents currently in "file".
    . Use "-clear" to discard all previously stored information in "file".
    . Use "-bg" to run the search in the background. The status can be checked
```

```
    with "prove -status thread_id".
  . Use "-time_limit time_limit" to specify the time limit for the search.
  . Use "-no_helper" to ensure that the created properties are not tagged as
    "helper" properties.
  . Use "-decompose" to attempt to create several smaller properties each
    capturing a part of the invariant.
```

Example

```
sst -check -property my_prop1
sst -check -property my_prop1 -trace_id 2 -clear

sst -generate -bg -name my_prop2 -signals {SIG1 SIG2 SIG3}
prove -wait
sst -generate -load -name my_prop2 -signals {SIG1 SIG2 SIG3}
```

See Also

prove -sst
assert -helper
assert -set_helper
get_sst_default_trace_length
set_sst_default_trace_length

stopat

Stops traversing a netlist at the specified signals/instances.

Syntax

```
stopat [-env |(-task task_name) |-reset]
       (signal_name+ | instance_name+)
       [-condition expression]

stopat [-env |(-task task_name) |-reset]
       (-list |-list_user |-list_all) [-include_condition]
       [-silent]

stopat [-env |(-task task_name) |-reset]
       -remove [-silent]
       (signal_name+ | instance_name+)

stopat [-env |(-task task_name) |-reset]
       -clear
```

Returns

"-remove" returns "true" or "false".

Detailed Description

Use this command to specify that an internal design signal should be treated as a free input. In other words, it can have any possible value according to its sample configuration.

CAUTION:

. By default, stopat signals are sampled as standard free inputs, which is according to the "clock -rate -default" configuration. To change this behavior, see the "clock -rate" command.
. The tool errors out in the proof setup if you place a stopat on a signal whose original driver is as fast as the tool's fastest clock.

The "stopat" command forces the tool to stop traversing a netlist at a specified signal or instance name. As a result, the logic driving the signal or instance is not included in the current analysis.

The specified signals are essentially disconnected from their driving logic. You can use this command effectively when the logic driving a signal is very large (for example, a 64-bit address comparison), and it is known that this signal can take any possible value (for example, address matches certain values or it does not).

This command is context-sensitive; that is, it applies to the current task unless you use the "-env" switch to apply it globally or the "-task" switch to specify a task other than the current task.

NOTE:
. The "stopat" command supports wildcards (* and ?) in signal names, including signals with a bit range.
. The "stopat" command supports escaped names, including those that contain "*" and "?".
. When signal is a System Verilog Interface signal, the "stopat" command will be applied to its actual driver.

[-env |(-task task_name) |-reset]
    (signal_name+ | instance_name+)
    [-condition expression]
    Apply a stopat on one or more specified signals or instances.
    . Use "-env" to apply the stopat globally.
    . Use "-task" to apply the stopat to the specified task.
    . Use "-reset" to apply the stopat during reset only.
    . If you do not use any of these switches, the stopat applies to the current task.
    . Use "-condition" to specify that the stopat is only effective when the specified expression is true, retainig the original driving logic when it is false.

    Important things to know about this command:
    . The "reset" command considers global (-env) and reset (-reset) stopats.
    . If you specify more than one signal or instance, use curly braces for each name. Refer to the example section below.
    . This switch supports Tcl lists as arguments.
    . If you add a stopat to an instance, its outputs and read-only hierarchical references will receive stopats, except its undriven outputs, which are already free and do not need stopats.

```
    . You can only provide a condition for a stopat on a signal, not on an
      instance.
    . The condition expression can be of any kind supported by Jasper, including
      $dff, $dlatch, $past, |->, ##n. For more details, see the "Jasper Tcl
      Operators" section in the "SVA Support" appendix of the Jasper Apps
      Command Reference Manual.
    . For power-aware designs, if you use "set_use_low_power_port_definition
      on" and apply a stopat to an instance, the net that is the output of the
      isolation cell receives the stopat, that is, "<output_port_name>_UPF_ISO".

[-env |(-task task_name) |-reset]
    (-list |-list_user |-list_all) [-include_condition]
    [-silent]
    List the specified set of stopats.
    . Use "-env" to list the global stopats.
    . Use "-task" to list stopats that impact the specified task, that is, task
      and global stopats.
    . Use "-reset" to list the stopats that apply to reset only.
    . Use "-list" with no additional switches to list all stopats with driven
      nets in the current task plus the global stopats.
    . Use "-list_user" for a list of user-defined stopats.
    . Use "-list_all" for a list of all stopats, including undriven signals.
    . Use "-include_condition" when there are conditional stopats to display
      their respective condition expressions.
    . Use "-silent"  to return the list as Tcl return values without printing
      the list on the screen.

[-env |(-task task_name) |-reset]
    -remove [-silent]
    (signal_name+ | instance_name+)
    Remove the stopat from the specified signal.
    . Use "-env" to remove a global stopat.
    . Use "-task" to remove a stopat from the specified task.
    . Use "-reset" to remove a reset stopat.
    . If you do not use any of these switches, the tool removes the stopat from
      the current task.
    . Use "-silent" to return Tcl return values without printing to the screen.

    NOTE: The tool returns "true" if it removed the stopat and "false" if it
    did not.

[-env |(-task task_name) |-reset]
    -clear
```

```
Remove the specified set of stopats.
. Use "-env" to remove all global stopats.
. Use "-task" to remove all stopats from the specified task.
. Use "-reset" to remove all reset stopats.
. If you do not use any of these switches, the tool removes all stopats
  from the current task.

NOTE: This command does not affect SEC properties or abstractions.
```

Example

```
stopat signalA
stopat {ing0} {ing1} {ing2} {ing3}
stopat {sig*[3:0]}
stopat {sig*[7]}
stopat inst0
stopat signalB -condition {!enable}
stopat -remove signalA
stopat -remove inst0
```

See Also

clock
replace_driver

task

Creates, sets, removes, edits, links, and reports on a task.

Syntax

```
Getting the Current Task's Name
-------------------------------
task (no arguments)


Creating Tasks
--------------
task -create new_task_name
     [-set]
     [-copy_all]
     [-copy_abstractions (all | counter | init_value | reset_value)+]
     [-copy_assumes]
     [-copy_asserts]
     [-copy_covers]
     [-copy_related_covers]
     [-copy_covergroups]
     [-copy_stopats [-copy_ratings]]
     [-copy property_name_list [-regexp]]
     [-source_task source_task_name]


Editing Tasks
-------------
task -edit target_task_name
     [-copy_all]
     [-copy_abstractions (all | counter | init_value | reset_value)+]
     [-copy_assumes]
     [-copy_asserts]
     [-copy_covers]
     [-copy_related_covers]
     [-copy_covergroups]
     [-copy_stopats [-copy_ratings]]
     [-copy property_name_list [-regexp]]
     [-source_task source_task_name]
```

```
Linking Tasks (Inheritance)
---------------------------
task -link (task_name)+ [-to inheriting_task_name] [-silent]

task -unlink (task_name)+ [-to inheriting_task_name] [-silent]


Setting and Removing Tasks
--------------------------
task -set task_name [-update_gui]

task -remove [task_name]

Utility Switches
----------------
task  -list [-silent]
     |-show [task_name] [-silent]
     |-num_asserts [task_name]
     |-num_covers [task_name]
     |-num_assumes [task_name]
```

Returns

Various. With no arguments, returns current task name.

Detailed Description

Use this command to create, edit, link, copy, and get information about tasks or to set a task as the current task.

```
Reserved Task Names
-------------------
Do not use the following reserved task names when creating new tasks:
. <*_MODEL_*>
. <*_PACKET_INTEGRITY>
. <Superlint_*>
. <CDC_*>
. <constraints>
. <embedded>*
```

```
. <fsv_task_N>
. <LowPower>
. <synthesized>
. Connectivity
. CSR*
. NDD_ASSUMES
. NDD_CONSTRAINTS
. unr*
. XP_*
. XPROP

Prohibited Task Names
---------------------
Jasper Apps does not support task names that include escaped identifiers or
special characters that need to be escaped.

Getting the Current Task's Name
-------------------------------
no arguments
    Return the name of the current task.

Creating Tasks
--------------
-create new_task_name
     [-set]
     [-copy_all]
     [-copy_abstractions (all | counter | init_value | reset_value)+]
     [-copy_assumes]
     [-copy_asserts]
     [-copy_covers]
     [-copy_related_covers]
     [-copy_covergroups]
     [-copy_stopats [-copy_ratings]]
     [-copy property_name_list [-regexp]]
     [-source_task source_task_name]
    Create a new interactive task with the specified name.

    NOTE:
    . Refer to the list of reserved task names above.
    . Do not use curly braces "{" "}" in task names.
    . With one exception, you can specify the new_task_name at any point
      after the "-create" switch. You cannot specify the new_task_name after
      "-copy".
```

. Use "-set" to set the newly created task as the current task.

. Use "-copy_all" to copy all abstractions, assertions, assumptions, covers, hard stopats, and ratings from the current or specified task.

. Use "-copy_abstractions" to copy one or more abstraction types from the current or specified task:
  . all   - counter, init value, and reset value abstractions.
  . counter   - counter abstractions. See "abstract -counter".
  . init_value - initial value abstractions. See "abstract -init_value".
  . reset_value - reset value abstractions. See "abstract -reset_value".

. Use "-copy_assumes" to copy all assumptions from the current or specified task.

. Use "-copy_asserts" to copy all assertions from the current or specified task.

. Use "-copy_covers" to copy all cover directives from the current or specified task.

. Use "-copy_related_covers" to copy all automatically extracted related covers for the properties that are being copied.

  NOTE: For additional information about related covers, see "elaborate -create_related_covers" and the command reference manual appendix "Vacuity Checking with Automatically Extracted Covers".

. Use "-copy_covergroups" to copy all covergroups structure and related properties from the current or specified task.

. Use "-copy_stopats" to copy all hard stopats from the current or specified task. And use "-copy_ratings" with "-copy_stopats" to copy all the task ratings on hard stopats.

. Use "-copy" with a Tcl list to copy specified assertions, assumptions, or covers from the current or specified task.

  NOTE:
  . The "-copy" switch supports wild cards (* and ?) and regular expressions for property names.
  . When using a regular expression, use the "-regexp" switch after the

```
        pattern. Refer to the examples.

    . Use "-source_task" to copy contents from the specified task instead of
      the current task.

      NOTE: The tool ignores "-source_task" if you do not also specify a
      "-copy*" switch.

Editing Tasks
--------------
-edit target_task_name
     [-copy_all]
     [-copy_abstractions (all | counter | init_value | reset_value)+]
     [-copy_assumes]
     [-copy_asserts]
     [-copy_covers]
     [-copy_related_covers]
     [-copy_covergroups]
     [-copy_stopats [-copy_ratings]]
     [-copy property_name_list [-regexp]]
     [-source_task source_task_name]
    Edit the target task by copying configurations from the current or source
    task.

    . Use "-copy_all" to copy all abstractions, assertions, assumptions,
      covers, hard stopats, and ratings from the current or specified task.

    . Use "-copy_abstractions" to copy one or more abstraction types from the
      current or specified task:
      . all         - counter, init value, and reset value abstractions.
      . counter     - counter abstractions. See "abstract -counter".
      . init_value  - initial value abstractions. See "abstract -init_value".
      . reset_value - reset value abstractions. See "abstract -reset_value".

    . Use "-copy_assumes" to copy all assumptions from the current or specified
      task.

    . Use "-copy_asserts" to copy all assertions from the current or specified
      task.

    . Use "-copy_covers" to copy all cover directives from the current or
      specified task.
```

```
    . Use "-copy_related_covers" to copy all automatically extracted related
      covers for the properties that are being copied.

      NOTE: For additional information about related covers, see "elaborate
      -create_related_covers" and the command reference manual appendix
      "Vacuity Checking with Automatically Extracted Covers."

    . Use "-copy_covergroups" to copy all covergroups structure and related
      properties from the current or specified task.

    . Use "-copy_stopats" to copy all hard stopats from the current or
      specified task. And use "-copy_ratings" with "-copy_stopats" to copy all
      the task ratings on hard stopats.

    . Use "-copy" with a Tcl list to copy specified assertions, assumptions, or
      covers from the current or specified task.

      NOTE:
      . The "-copy" switch supports wild cards (* and ?) and regular
        expressions for property names.
      . When using a regular expression, use the "-regexp" switch after the
        pattern. Refer to the examples.

    . Use "-source_task" to copy contents from the specified task instead of
      the current task.

      NOTE:
      . The tool ignores "-source_task" if you do not also specify a
        "-copy*" switch.

      . You cannot edit an FSV task.

Linking Tasks (Inheritance)
---------------------------
Task inheritance promotes hierarchical building of verification tasks to assist
"divide and conquer" strategies in task development by creating live links
between tasks. A task "inherits" the contents of one or more supporting tasks,
and the inherited contents are treated as part of the inheriting task. The
"contents" are: assumptions, assertions, cover directives, and hard stopats.

NOTE: Child properties mirror their parent's status.
. When you remove a property in a parent task, it is removed in the linked
  (child) task.
```

```
. To disable or re-enable child properties, you must disable or re-enable the
  parent property in the parent task.
. To convert a child assumption to an assertion or assertion to an assumption,
  run "assert -from_assume" or "assume -from_assert" on the parent property in
  the parent task.

-link (task_name)+ [-to inheriting_task_name] [-silent]
    Link the specified tasks to the current task.

    Use the optional "-to" switch to create a link to an inheriting task that
    is not the current task.

    CAUTION: Automatically extracted Proof Accelerator tasks can contain
    implicit assumptions that are not visible to the user, and constraining
    tasks with assumptions that are not visible makes it impossible to evaluate
    their impact.

    If you are using Verification Component Proof Accelerators (for
    example, jasper_scoreboard_2) or Abstraction Component Proof Accelerators
    (for example, jasper_model_fifo), do not attempt to link a Proof
    Accelerator task to a non-Proof Accelerator task. Instead, you should do
    the following:
    task -link my_task -to <pa_task>

    Use "-silent" to turn off info messages. This switch is helpful for
    scripting.

-unlink (task_name)+ [-to inheriting_task_name] [-silent]
    Remove an existing link between the specified tasks and the current task.

    Use the optional "-to" switch to remove a link to an inheriting task that
    is not the current task.

    Use "-silent" to turn off info messages. This switch is helpful for
    scripting.

Setting and Removing Tasks
--------------------------
-set task_name [-update_gui]
    Set the specified "task_name" as the current task.

    Use the optional "-update_gui" switch to select the specified "task_name"
    on the active GUI task tree, if there is one.
```

```
-remove [task_name]
    Remove the specified "task_name" (or the current task if the task
    name is omitted).

    NOTE: You can only remove user-created tasks.

Utility Switches
----------------
The tool accepts the following commands when a "prove" or "visualize" process
is running in the background.

-list [-silent]
    Return all tasks.

    Use "-silent" to return the list as Tcl return values without printing the
    list on the screen.

-show [task_name] [-silent]
    Print property types and names and return property names for the current or
    specified task.

    Use "-silent" to return the list as Tcl return values without printing the
    list on the screen.

-num_asserts [task_name]
    Return the number of assertions in the current or specified task.

-num_covers [task_name]
    Return the number of covers in the current or specified task.

-num_assumes [task_name]
    Return the number of assumptions in the current or specified task.
```

Example

```
task
task -create -set newTask assert:0
task -show
task -list
task -set taskB

# Pattern-matching with regular expressions:
# ------------------------------------------
# Use the "-regexp" switch after the pattern.
task -create "gui_copy_3" -set -source_task "<embedded>" -copy
"overflow_i.arithmetic_overflow_poi_98_prop_98" -regexp

# Copy all assertions and related covers from the current task to the new
# task named "my_task".
task -create my_task -copy_asserts -copy_related_covers
```

view

Views the contents of a file.

Syntax

```
view file_name
```

Returns

No return values

Detailed Description

Use this command to view the contents of a file.

This command is particularly useful when viewing encrypted requirement files in pre-built kits designed for Jasper Apps.

file_name
    The name of the file to be viewed.

Example

```
view requirement.v
```

virtual_net

Creates a new virtual net, optionally driven by a specified expression.

Syntax

```
virtual_net ( net_name [-width net_width]
                               [-expression driving_expression]
             |net_name -remove
             |-list
             |-clear
             |-show { [expr] [width] }
            )
```

Returns

No return values.

Detailed Description

Use this command to create a new virtual net, optionally driven by a specified expression. These virtual nets are created in the "environment" context, which makes them accessible to all tasks. Once you create a virtual net, both "-remove" and "-clear" remove only the driving logic of the nets, and they remain usable as signals for other commands. To completely remove them, use "clear -all".

NOTE:
. Adding a virtual net does not invalidate proof results.
  See "set_enable_preserve_property_status" for more information.
. If a Visualize window is open, adding a virtual net forces the window to
  reprocess the trace, except for traces with frozen cycles, which are cleared
  from the window.

net_name -width net_width -expression driving_expression
    Creates a new virtual net named "net_name".

    . Use "-width" to force a specific width for the new virtual net,

```
      which defaults to the width of "driving_expression"
      or 1 if you do not specify "-expression".
      NOTE: The width of the virtual net must be smaller than 2**24 bits.

    . Use "-expression" to specify the logic driving the new virtual net,
      which defaults to an undriven net.

      NOTE: The expression can be of any kind supported by Jasper, including
      $dff, $dlatch, $past, |->, ##n. For more details, see the "Jasper Tcl
      Operators" section in the "SVA Support" appendix of the Jasper Apps
      Command Reference Manual.

net_name -remove
    Removes the logic driving the specified virtual net.

-list
    Returns a list of virtual net names.

-clear
    Removes all logic driving virtual nets.

-show { [expr] [width] }
    Returns the specified value.

    If a different value is specified, the command fails with an error.

    If no value is specified, the command returns both expression and
    width value.

    Attributes Available for All Virtual Nets
    -----------------------------------------
    . expr: The driving logic expression
    . width: The driving logic width
```

Example

```
virtual_net c -expression a|b
virtual_net d -expression {{a,c}}
virtual_net -list
{c d}

virtual_net c -show expr
a|b
virtual_net c -show width
1
virtual_net d -show expr
{{a,c}}
virtual_net d -show width
2
virtual_net d -remove
virtual_net -list
{c}

virtual_net -clear
virtual_net -list
{}
```

visualize

Generates a trace to Visualize a requirement or design scenario.

Associated App(s)

Depending on the visualize recipes to be visualized.

Syntax

```
Generating Initial Visualize Waveforms
--------------------------------------
visualize [-cover]
          (expression |(-property property_name) [-regexp])
          [ -new_window [new_window_name]
           |-window window_name]
          [-engine (engine_mode | {engine_mode+})]
          [-proof_time time_limit]
          [-annotation  {annotation}]
          [-batch] [-silent] [-bg]
          [(-sig_order sig_order_file_name)|-no_signals]
          [-task task_name] [-trace_id id]

visualize -violation
          (expression |(-property property_name) [-regexp])
          [ -new_window [new_window_name]
           |-window window_name]
          [-engine (engine_mode | {engine_mode+})]
          [-proof_time time_limit]
          [-annotation  {annotation}]
          [-batch] [-silent] [-bg]
          [(-sig_order sig_order_file_name)|-no_signals]
          [-task task_name] [-trace_id id]
          [-sst]

Setting a New Target
--------------------
visualize -set_target [-cover |-violation]
          [-window window_name]
```

```
          (expression |(-property property_name [-regexp]))

Configuring Waveforms
---------------------
visualize -force -soft expression [-name name]
          [-config_group_name config_group_name]
          [-annotation  {annotation}]
          [cycle | cycle:cycle]
          [-loop]
          [-window window_name]

visualize -force (expression |(-property property_name [-regexp]))
          [cycle | cycle:cycle] [-name name]
          [  (-after expression_1)
           | (-after -after_property property_name [-regexp])
           | (-before expression_1)]
          [-config_group_name config_group_name]
          [-annotation  {annotation}] [-loop] [-window window_name]

visualize -at_least_once (expression |(-property property_name [-regexp]))
          [(-after expression_1) | (-after -after_property property_name [-regexp])]
          [-before expression_1]
          [cycle | cycle:cycle] [-name name]  [-config_group_name config_group_name]
          [-annotation  {annotation}] [-loop] [-window window_name]

visualize -not -soft expression [-name name]
          [-config_group_name config_group_name]
          [-annotation  {annotation}] [cycle | cycle:cycle]
          [-window window_name]

visualize -not (expression |(-property property_name [-regexp]))
          [cycle | cycle:cycle] [-name name]
          [  (-after expression_1)
           | (-after -after_property property_name [-regexp])
           | (-before expression_1)]
          [-config_group_name config_group_name]
          [-annotation {annotation}] [-window window_name]

visualize -min_length length [-window window_name]

visualize -max_length length [-window window_name]

visualize -freeze cycle [-window window_name]
```

```
visualize -replot [-new_window [new_window_name]] [-force]
         [-engine (engine_mode | {engine_mode+})] [-proof_time time_limit]
         [-quiet]
         [-min_variation]
         [-prompt] [-batch] [-silent] [-bg]
         [(-sig_order sig_order_file_name)|-no_signals] [-window window_name]

visualize -go_backward [-new_window [new_window_name]] [-force]
         [(-sig_order sig_order_file_name)|-no_signals]
         [-proof_time time_limit] [-min_variation]
         [-batch] [-silent] [-bg] [-window window_name]

visualize -go_forward [-new_window [new_window_name]] [-force]
         [(-sig_order sig_order_file_name)|-no_signals]
         [-proof_time time_limit] [-min_variation]
         [-batch] [-silent] [-bg] [-window window_name]

visualize -loop_unroll N [-batch]
         [(-sig_order sig_order_file_name) |-no_signals] [-window window_name]

visualize -get_loop_unroll [-window window_name]

visualize -get_show_reset_cycles [-window window_name]

visualize -set_show_reset_cycles (on | off)
          [-window window_name]

visualize -set_infinite_extension (true | false) [-window window_name]

visualize -get_infinite_extension [-window window_name]

visualize -get_zoom_range [-window window_name]

visualize -set_zoom_range [M:N]
          [-window window_name]

Utility Switches - For Configurations
-------------------------------------
visualize -rename_conf old_conf_name new_conf_name [-window window_name]
        |-remove_conf conf_name [-window window_name]
        |-disable_conf conf_name [-window window_name]
        |-enable_conf conf_name [-window window_name]
```

```
        |-offset_conf conf_name offset_value [-window window_name]
        |-clear_conf [-window window_name]

visualize -replace_conf (-force -soft |-not -soft) expression
         [cycle | cycle:cycle] -name conf_name [-window window_name]

visualize -replace_conf (-force |-not |-at_least_once)
         (expression |-property property_name)
         [(-after expression_1) | (-after -after_property property_name [-regexp])]
         [-before expression_1]
         [cycle | cycle:cycle]
         -name conf_name
         [-loop]
         [-window window_name]

visualize -replace_conf -signal signal_name -sequence
         ({value[{cycles}][ value[{cycles}]]*} [cycle])
         -name conf_name [-window window_name]

Utility Switches - General
--------------------------
visualize -list [-silent] [-all | config_group_name] [-window window_name]

visualize -list -all_windows [-silent]

visualize -save [-script] [-force]
    [-annotation] [-marker] [-custom_button]
          [ -setup_only [-window window_name]
           |-config_only  [-window window_name]]
            [ -add_hier_path path_name
             |-remove_hier_path path_name
            ]
          file_name

visualize -save -init_state file_name [-cycle N] [-force]
          [-include_internal] [-include_encrypted] [-bit_blast]
          [(-window window_name) |-reset_window]

visualize -save -reset_sequence [-force] file_name
          [-include_internal] [-include_encrypted] [-bit_blast]
          [(-window window_name) |-reset_window]

visualize -save -vcd [-imp |-spec] vcd_file_name [-force]
```

```
          [-sig_order sig_order_file_name]
          [-last_cycle cycle] [-visible_only]
          [(-window window_name) |-reset_window]

visualize -save -fsdb [-imp |-spec] fsdb_file_name [-force]
          [-time_scale [N][time_unit]]
          [-sig_order sig_order_file_name]
          [-last_cycle cycle] [-visible_only]
          [-hier_map -from source_hier -to destination_hier]
          [(-window window_name) |-reset_window]
          [-include_reset_cycles]

visualize -save -shm [-imp |-spec] shm_directory_name [-force]
          [-time_scale [N][time_unit]]
          [-sig_order sig_order_file_name]
          [-last_cycle cycle] [-visible_only]
          [-hier_map -from source_hier -to destination_hier]
          [(-window window_name) |-reset_window]
          [-include_reset_cycles]

visualize -save -vwdb [-imp |-spec] vwdb_directory_name [-force]
          [-time_scale [N][time_unit]]
          [-sig_order sig_order_file_name]
          [-last_cycle cycle] [-visible_only]
          [-hier_map -from source_hier -to destination_hier]
          [(-window window_name) |-reset_window]
          [-include_reset_cycles]

visualize -save -image image_file_name [-window window_name]

visualize -load (-vcd |-fsdb |-shm |-vwdb) trace_file_name
          [-sampling (-infer |-clock_period N[time_unit])]
          [-time_scale [N][time_unit]]
          [-start_time N] [-end_time N]
          [ -hier_path path_difference
           |-hier_map -from source_hier -to destination_hier]
          [-two_valued_simulation]
          [ -new_window [new_window_name]
           |-window window_name]
          [-batch]

visualize -confirm (-vcd |-fsdb |-shm |-vwdb) trace_file_name
          [-sampling (-infer |-clock_period N[time_unit])]
```

```
          [-time_scale [N][time_unit]]
          [-start_time N] [-end_time N]
          [ -hier_path path_difference
           |-hier_map -from source_hier -to destination_hier]
          [-two_valued_simulation]
          [-use_reset_analysis]
          [ -new_window [new_window_name]
           |-window window_name]
          [-batch]

visualize -get_sim_time cycle [-window window_name]

visualize -reset

visualize -show ( max_length | min_length | frozen_cycles | task | type
                        | trace_length | quiet )
                        [-window window_name]

visualize -trace -list (length | engine) [-window window_name]

visualize -save_sig_order file_name [(-window window_name) |-reset_window] [-
save_highlight]

visualize -set_silent_mode (true | false)

visualize -get_silent_mode

visualize -set_annotate_clock_events (true | false) [-window window_name]

visualize -get_annotate_clock_events [-window window_name]

visualize -undo [-window window_name]

visualize -redo [-window window_name]

visualize -highlight -diff target_window_name
                                              [-target_window_start_cycle N]
                                              [-source_window_start_cycle N]
                                              [-window source_window_name]

visualize -highlight -diff (-prev_plot |-next_plot)
                                              [-window window_name]
```

```
visualize -relevant_diff signal source_cycle target_cycle target_window_name
        [-configuration ([control_path | data_path] [undriven_only]
                         [filter_async_reset] [show_clock] [show_equal]
                         [instance_ports_only])+
        [-iteration_depth N |-num_whys N |-unlimited_depth]
        [(-window source_window_name)]

visualize -relevant_logic signal cycle
        [-configuration ([control_path | data_path] [undriven_only]
                         [filter_async_reset] [show_clock] [show_equal]
                         [show_twins_diff] [instance_ports_only])+
        [-iteration_depth N |-num_whys N |-unlimited_depth]
        [(-window window_name) |-reset_window]

visualize -clear_all [-window window_name]

visualize -sig_order sig_order_file_name
        [(-window window_name) |-reset_window]

visualize -no_signals [-window window_name]

visualize -new_window [new_window_name] [-copy [-window old_window_name]]

visualize -rename_window old_name new_name

visualize -set_current_window window_name

visualize -get_current_window

visualize -get_last_active_window

visualize -remove_window (window_name |-reset_window)

visualize -remove_current_window

visualize  (-get_type
            |-get_value signal_name
             [cycle | cycle:cycle]
             [-radix ( 2 | 8 | 10 | 16 | bin | oct | dec | signed_dec
                      | hex | signed_hex | bits_set | floating_point | string)]
            |-get_length
            |-get_stem_length
            |-get_loop_length
```

```
            |-get_looping_flops
           )
           [-window window_name]

visualize -is_looping [-window window_name]

visualize -get_trace_id [-window window_name]

visualize (-load_radix file_name) [-window window_name]

visualize -set_radix (signal_name | expression)
          ( -bin |-oct |-dec |-signed_dec
           |-hex |-signed_hex |-bits_set
           |-floating_point |-string |-default
           |-user-defined radix_name)
    [-regexp |-occurrence N |-all]
          [(-window window_name) |-reset_window]

visualize -marker marker_name -cycle (N | N:M)
          [-color color]
          [-row row_number]
          [(-window window_name) |-reset_window]

visualize -marker (-list |-show marker_name_list
                           |-remove marker_name_list |-clear)
          [(-window window_name) |-reset_window]

visualize -marker [marker_name] -callback_on_moved procedure_name
          [(-window window_name) |-reset_window]

visualize -get_target [-window window_name]

Signal List Manipulation
------------------------
visualize -add_sig
           (signal_name | expression) [row]
          |-file file_name)
          [(-window window_name) |-reset_window]

visualize -remove_sig
           (signal_name | expression)
           [-regexp | -occurrence N | -all]
           [(-window window_name) |-reset_window]
```

```
visualize -signal_group
          ( -expand (signal_group_name | array | struct | union)+
                [-all]
           |-collapse (signal_group_name | array | struct | union)+
                [-all]
           |-create signal_group_name
                [-sig  (signal | signal_group_name)+]
           |-remove signal_group_name [-keep_signal]
           |-add_signal signal_group_name
                -sig  (signal | signal_group_name)+
           |-remove_signal signal_group_name
                -sig  (signal | signal_group_name)+
                [-keep_signal]
           |-add_spacer signal_group_name
          )
          [(-window window_name) |-reset_window]

visualize -add_spacer
           [row]
           [(-window window_name) |-reset_window]

visualize -plot_row -list
          ([signal] [group])+
          [-group signal_group_name]
          [-include_id]
          [-target_pane (anchor | default | preview)]
          [-window window_name]

visualize -tracking_signal
          ( -add tracking_signal [row_id]
                -child_signal signal_list
           |-remove tracking_signal [row_id]
                -child_signal signal_list
          )
          [(-window window_name) |-reset_window]

Color and Comment Annotation Switches
-------------------------------------
visualize -define_color_annotation { {signal_name time_step color}+ }
          annotation_name [-replace |-append]
          [(-window window_name) |-reset_window]
```

```
visualize -comment {signal_name time_step color [row_id]}
         annotation_name -text comment
         [-replace |-append |-remove |-position (top_left | top_right | bottom_left |
bottom_right)]
         [(-window window_name) |-reset_window]

visualize -comment -arrow {from_signal_name time_step color [row_id]}
         {to_signal_name time_step color [row_id]} annotation_name
         [-text comment]
         [-replace |-append |-remove |-position (top | bottom)]
         [(-window window_name) |-reset_window]

visualize -comment (-collapse |-expand)
         [-window window_name]

visualize -activate_color_annotation annotation_name+
         [(-window window_name) |-reset_window]

Signal Highlight Switches
-------------------------
visualize -define_signal_highlight { {signal_or_property_name color}+ }
         [(-window window_name) |-reset_window]
         [-add |-remove]

Interactive Constraint Extraction
---------------------------------
visualize -extract_conf [-reason [-depth depth_value] [-transitive]] [-value]
              { {signal_name [cycle | cycle:cycle] }+ }
              [cycle | cycle:cycle]
              [-name conf_name] [-window window_name]

visualize -concretize_conf {conf_name} [-window window_name]

Waveform Analysis
-----------------
visualize -check_props
          [-cover |-violation |-assumption]
          [[-bg] -update_gui]
          [-names_only]
          [-x_only]
          [-filter property_name_filter_list [-regexp]]
          [-stop_on { [cover_reached] [assertion_violated] [assumption_violated] }
                   [-start_cycle N] [-first_only]]
```

```
          [-silent]
          [-window window_name]

visualize -check_props (-wait |-stop |-status)

visualize -compare signalA_name signalB_name [-shift_cycles cycles]
            [-ignore_x] [-ignore_values value+] [-window window_name]

Directed Test
-------------
visualize -directed_test
          -signal signal_name
          -value value_list
          -range range_list
          [-name test_name]
          [-window window_name]

visualize -directed_test -append test_point
          -value value_list
          -range range_list
          [-window window_name]

visualize -directed_test -remove test_point_list
          [-window window_name]

visualize -directed_test -rename old_name new_name
          [-window window_name]

visualize -directed_test -replace test_point
          [-signal signal_name]
          [-value value_list
           -range range_list]
          [-window window_name]

visualize -directed_test -list
          [-filter (name | signal | value | range | status) filter_string [-regexp]]
          [-window window_name]

visualize -directed_test -show
          test_point (signal | value | range | status)
          [-window window_name]

visualize -directed_test -report
```

```
          [-name test_point_list]
          [-file file_name [ -force]]
          [-silent]
          [-window window_name]

visualize -directed_test -check [test_point_list]
          [-silent] [-stop_on_failure]
          [-window window_name]
```

QuietTrace

```
----------
visualize -quiet ((true [cycle:cycle]) | false | -signal signal_list)
          [-attempt_time_limit time_limit] [-window window_name]

visualize -get_quiet [-attempt_time_limit] [-window window_name]
```

Configuration Grouping

```
----------------------
visualize -config_group
        ('{'conf_name+'}' [-name config_group_name]
        |-expand config_group_name
        |-remove config_group_name
        |-rename old_config_group_name new_config_group_name
        |-replace '{'conf_name+'}' -name config_group_name
        |-add_conf conf_name config_group_name
        |-remove_conf conf_name
        |-merge config_group_name_1 config_group_name_2 [-name final_config_group_name]
        ) [-window window_name]
```

Annotating Targets and Configurations

```
-----------------------------------
visualize -set_annotation (-target |-conf conf_name)
        {annotation}] [-window window_name]

visualize -get_annotation (-target |-conf conf_name) [-window window_name]
```

Specifying a Waveform Sequence in a Single Command

```
-------------------------------------------------
visualize -signal signal_name -sequence
          ('{'value['{'cycles'}'][ value['{'cycles'}']]*'}' [cycle])
          [-name name] [-config_group_name config_group_name] [-window window_name]
```

Visualize Task Manipulation

```
--------------------------
visualize -set_task task_name [-window window_name]

visualize -get_task [-window window_name]

Switches for Background  Visualize
----------------------------------
visualize -stop [-thread thread_id]

visualize -wait

visualize -status [-silent]

Switches for Task Context Visualize
-----------------------------------
visualize  -assume ( [-bound N] expression |-constant signal_name )
                [-name name]
                [-annotation '{'annotation'}']
                [-replace] [-window window_name]

visualize  -stopat signal_or_instance_name+
              [-name name]
              [-annotation '{'annotation'}']
              [-replace]  [-window window_name]

visualize  -set_as_AR [-window window_name]

Trace Log Switches
------------------
visualize -trace_log -add (-display_expr display_expr
                 [-log_group log_group_name]
                 [-module module_name |-instance instance_list])
                 [-condition ( -expression expr
                                        |-cycle_factor N
                                        |-value_change sig_list)+]
                 [-set_verbosity N]

visualize -trace_log -run [log_group_list]
             [-verbosity N]
             [(-window window_name) |-reset_window]
             [-file file_name [-force]]
             [-include sub]
             [-filter -coi]
```

```
visualize -trace_log -list (log_group | display_expr) [-silent]

visualize -trace_log -show
            ( -log_group log_group_name
             |-display_expr display_expr)
    [-silent]

visualize -trace_log (-enable |-disable |-remove)
              ( -log_group log_group_list
               |-display_expr display_expr_list)

visualize -trace_log -clear

Why
---
visualize  -why signal_name cycle
                [-transitive]
                [-ignore_connected_assumptions]
                [-simple_buffer_transitive]
                [-stop_at_hierarchy_change]
                [-include_non_ticking_gated_clocks]
                [-gui]
                [-silent]
                [(-window window_name) |-reset_window]

Non-Linear Cycle Operations
---------------------------
visualize -cycle -hide cycle:cycle
                [-window window_name]

visualize -cycle -unhide [target_cycle]
                [-window window_name]
```

Returns

When visualizing properties without "-bg" these are the return values:

. "visualize -violation" returns "cex", "ar_cex", or "proven".

. "visualize -cover" returns "covered", "ar_covered", or "unreachable".

. "visualize -violation" and "visualize -cover" can return the following:

  . "undetermined" - No trace was found because the tool reached the maximum trace length.

  . "error" - The tool detected overconstraint.

    NOTE: Not all engines can detect overconstraint; therefore, it is possible for the same configuration to return a value of "unreachable"/"proven" or a return value of "error", depending on the engine.

Detailed Description

Use this command to Visualize a requirement or design scenario in the form of a violation trace. You can also Visualize how the requirement description might reach a certain coverage point in the form of a cover trace.

IMPORTANT: Using signal order files, databases, and drag-and-drop or copy-paste operations between different versions of the tool is not recommended. We cannot guarantee backwards compatibility in these areas as we implement continuous enhancements.

When you use the "set_engine_mode" command at any point prior to using Visualize, the tool uses the specified engine to Visualize your target. If you do not specify an engine mode during your session, the tool uses the default engine set, {Hp Ht N B} (Use "help set_engine_mode -gui" for additional information about the default engine set.)

NOTE: By default, Visualize does not display struct values for all intermediate struct nodes; however, you can use the "set_visualize_show_full_struct on" command to display them.

```
The use model is:

analyze requirement.v
elaborate
reset rst
visualize -violation -property requirement

This command supports regular expressions for property names.

You will typically use the GUI Visualize window and access additional switches
from a menu.

Jasper Apps supports SVA expressions for
"visualize". Important things to know about support:

- Expressions can be added incrementally to the waveform
  window, as long as their inputs are inside the analysis region.
- Refer to your command reference manual for limitations and clarification.

NOTE: To use "visualize" commands with FSV properties, you must initially run
"check_fsv -visualize".

Generating Initial Visualize Waveforms
--------------------------------------
no-argument
    If Visualize is in progress, this is equivalent
    to "visualize -replot".

[-cover] (expression |(-property property_name [-regexp]))
         [ -new_window [new_window_name]
          |-window window_name]
         [-engine (engine_mode | {engine_mode+})]
         [-proof_time time_limit] [-annotation {annotation}]
         [-batch] [-silent] [-bg]
         [(-sig_order sig_order_file_name)|-no_signals]
         [-task task_name]

    Generate an initial waveform to Visualize how a design might
    cause the requirement description to enter a certain state.
    (Implicitly, Jasper Apps also uses a "-clear_all" on previous
    Visualize data if the target differs from the previous target.)

    . Replace "expression" with an expression for a cover point or use the
```

```
  "-property" switch to Visualize a property in the current task.

  NOTE:
  . The "-property" switch supports wildcards and regular expressions.
    An error results unless there is exactly one matching property. When
    using a regular expression, use the "-regexp" switch after the pattern:
    visualize... -property {pattern} -regexp
  . The "-property" switch only accepts cover properties.

. Use "-new_window" to generate the waveform in a new window with a
  name specified by the optional new_window_name argument.
  NOTE:
  . If you do not provide the new_window_name argument, a tool-generated
    name is used.
  . The new window becomes the "current window."

. By default, the tool applies your "visualize" command to the current
  window. Use "-window" to apply your command to a window other
  than the current window.
  . If you specify a name that does not exist yet, this switch renames the
    current window with the specified name.
  . If there are no Visualize windows, a new window opens with the
    specified name.
  NOTE: "-window" and "-new_window" are mutually exclusive.

. Use "-engine" to specify one or more engine modes you want Visualize to
  use. Specify a single engine or use curly braces to specify
  parallel-engine processing.

. Use "-proof_time" to set a limit on the time for this command. See
  "set_prove_time_limit".

. Use "-annotation" to annotate the target with
  the description enclosed by the curly braces.

  View annotations in the Visualize window by using the mouse
  to hover over the target name and reveal a tooltip. You can print
  annotations to the screen with the "-get_annotation" switch.

. Use "-batch" to prevent the tool from displaying the
  waveform it generates, which improves performance.

. Use "-silent" to turn off the output to the screen and return information
```

```
      with a Tcl return value.

      NOTE: "-silent" does not stop the trace from being shown in the Visualize
      window. You must use "-batch" to suppress the trace.

    . Use "-bg" to run the command in the background and continue to accept
      and run new commands.

    . Use "-sig_order" to generate a trace that displays only the signals in
      the specified signal order file.

    . Use "-no_signals" to load a trace without displaying signals.

    . Use "-task" to apply the command to the specified task instead of the
      current task. Do not use the "-task" and "-property" switches in the same
      command. If you need to specify both a "property_name" and "task_name",
      use the "-property" switch and prepend the property name with "<task>::".
      (Replace <task> with an actual task name.)

    . Use "-trace_id" to display the specified trace.
      NOTE: See "assert -set_store_trace" or "cover -set_store_trace" for
      details on how to configure the number of traces generated and stored
      by the tool.

-violation (expression |(-property property_name [-regexp]))
           [ -new_window [new_window_name]
            |-window window_name]
           [-engine (engine_mode | {engine_mode+})]
           [-proof_time time_limit] [-annotation {annotation}]
           [-batch] [-silent] [-bg]
           [(-sig_order sig_order_file_name)|-no_signals]
           [-task task_name] [-trace_id id] [-sst]
    Generate an initial waveform to Visualize how a design with
    bugs might fail the requirement. (Implicitly, Jasper Apps
    also uses a -clear_all on previous Visualize data if the target
    differs from the previous target.)

    . Replace "expression" with an expression for a requirement or use the
      "-property" switch to Visualize a property in the current task.

      NOTE:
      . The "-property" switch supports wildcards and regular expressions.
        An error results unless there is exactly one matching property. When
```

```
    using a regular expression, use the "-regexp" switch after the pattern:
    visualize... -property {pattern} -regexp
  . The "-property" switch only accepts assertions.

. Use "-new_window" to generate the waveform in a new window with a
  name specified by the optional new_window_name argument.
  NOTE:
  . If you do not provide the new_window_name argument, a tool-generated
    name is used.
  . The new window becomes the "current window."

. By default, the tool applies your "visualize" command to the current
  window. Use "-window" to apply your command to a window other
  than the current window.
  . If you specify a name that does not exist yet, this switch renames the
    current window with the specified name.
  . If there are no Visualize windows, a new window opens with the
    specified name.
  NOTE: "-window" and "-new_window" are mutually exclusive.

. Use "-engine" to specify one or more engine modes you want Visualize to
  use. Specify a single engine or use curly braces to specify
  parallel-engine processing.

. Use "-proof_time" to set a limit on the time for this command. See
  "set_prove_time_limit".

. Use "-annotation" to annotate the target with
  the description enclosed by the curly braces.

  View annotations in the Visualize window by using the mouse
  to hover over the target name and reveal a tooltip. You can print
  annotations to the screen with the "-get_annotation" switch.

. Use "-batch" to prevent the tool from displaying the
  trace it generates, which improves performance.

. Use "-silent" to turn off the output to the screen and return information
  with a Tcl return value.

  NOTE: "-silent" does not stop the trace from being shown in the Visualize
  window. You must use "-batch" to suppress the trace.
```

```
    . Use "-bg" to run the command in the background and continue to accept
      and run new commands.

    . Use "-sig_order" to generate a trace that displays only the signals in
      the specified signal order file.

    . Use "-no_signals" to load a trace without displaying signals.

    . Use "-task" to apply the command to the specified task instead of the
      current task. Do not use the "-task" and "-property" switches in the same
      command. If you need to specify both a "property_name" and "task_name",
      use the "-property" switch and prepend the property name with "<task>::".
      (Replace <task> with an actual task name.)

    . Use "-trace_id" to display the specified trace.
      NOTE: See "assert -set_store_trace" or "cover -set_store_trace" for
      details on how to configure the number of traces generated and stored
      by the tool.

    . Use "-sst" to run SST mode for the specified property. Visualize replots
      any constraint and operation using the same mode. SST traces do not
      support operations such as freeze and extend or formal reset.
      NOTE: See "prove -sst" for details about SST proof.

Setting a New Target
--------------------
-set_target [-cover |-violation]
            (expression |(-property property_name [-regexp]))
            [-window window_name]
    Replace the target for the current Visualize (or set an initial
    target if no Visualize is currently in progress), but do not
    replot.

    The default is "-cover" if you do not specify a switch (-cover or
    -violation).

    By default, the tool applies your "visualize" command to the current
    window. Use "-window" to apply your command to a window other
    than the current window.
    . If you specify a name that does not exist yet, this switch renames the
      current window with the specified name.
    . If there are no Visualize windows, a new window opens with the
      specified name.
```

```
    NOTE:
    . If some of the trace is frozen and you attempt to target an X-Prop
      property, the tool generates an error.
    . The "-property" switch supports wildcards and regular expressions.
      An error results unless there is exactly one matching property. When
      using a regular expression, use the "-regexp" switch after the pattern:
      visualize... -property {pattern} -regexp

    This command preserves:
    . All previously added configurations (force, at_least_once,
      min_length, max_length)
    . The Visualize stack (modified and replotted versions of the current
      trace).

Configuring Waveforms
---------------------
-force -soft expression [-name name]
      [-config_group_name config_group_name] [-annotation {annotation}]
      [cycle | cycle_1:cycle_2]
      [-loop]
      [-window window_name]
    Prefer traces where the specified expression is satisfied in as many
    cycles as possible.

    NOTE: When there are multiple "-force -soft" configurations, they might
    conflict with each other. The trace given is guaranteed to present a
    local maxima satisfying as many as possible in as many cycles as
    possible; however, soft constraints are optional, and will never affect
    (extend) trace length.

    Implementation guidance:
    . Use "-name" to specify a name for this configuration.
    . Use "-config_group_name" to specify a group name for this configuration.
    . Use "-annotation" to annotate the configuration with the
      description enclosed in the curly braces.
    . View annotations in the Visualize window by using the mouse to hover
      over the configuration name and reveal a tooltip. You can print
      annotations to the screen with the "-get_annotation" switch.
    . Specifying a cycle is optional. If you do not specify a cycle,
      that is equivalent to "always".
      . If you specify a single cycle (for example, 7), you are
        asking the tool to prefer the specified expression in cycle seven.
```

```
      . Use $ to mean "the last cycle" and $-N to mean "N cycles before
        the last cycle".
      . When cycle_1=1, the tool will give preference to showing the
        expression being covered between the first cycle of the trace
        (the first cycle after reset) and cycle_2; when cycle_2=$,
        the tool gives preference to showing the expression being
        covered between cycle_1 and the last cycle of the trace.
    . Use "-loop" when adding the constraint to an infinite trace.
    . By default, the tool applies your "visualize" command to the current
      window. Use "-window" to apply your command to a window other
      than the current window.
      . If you specify a name that does not exist yet, this switch renames the
        current window with the specified name.
      . If there are no Visualize windows, a new window opens with the
        specified name.

    NOTE: If you use "-force -soft" to create a soft constraint,
    QuietTrace(TM) will not artificially display the specified signal as
    constant low.

-force (expression |(-property property_name [-regexp]))
    [cycle | cycle_1:cycle_2] [-name name]
    [  (-after expression_1)
     | (-after -after_property property_name [-regexp])
     | (-before expression_1)]
    [-config_group_name config_group_name] [-annotation {annotation}]
    [-loop]
    [-window window_name]
    Ensure the specified expression or property is true from cycle_1
    to cycle_2 of the trace.

    The generated trace will have at least as many
    cycles as cycle_1.

    Implementation guidance:
    . Specify an expression or use "-property" for this
      configuration.
    . Use "-after" or "-after -after_property" to configure a trace that
      shows one scenario occurring after another has occurred.
      When using these after switches, the cycle count described below
      begins after "expression_1" (or "property_name") instead of after reset.
    . The "-property" and "-after -after_property" switches support wildcards
      and regular expressions. An error results unless there is exactly one
```

```
      matching property. When using a regular expression, use the "-regexp"
      switch after the pattern:
      visualize... -property {pattern} -regexp
    . Use "-before" to configure a trace that shows one scenario occurring
      before another has occurred. When using -before, the cycle count
      described below begins before "expression_1" instead of after reset.
    . When cycle_1=1, the expression will be true starting in the first
      cycle of the trace (the first cycle after reset); when
      cycle_2=$, the expression will be true until the last cycle of
      the trace.
    . Use the cycle range $:$ to force the expression or property at the
      last cycle of the generated trace. This range is not supported with
      "-after" or "-before".
    . Use negative cycle numbers to refer to a range that counts back from
      the end of the trace, for example:
      . -7:-5 refers to a range from 7 cycles before the end of the trace
        through the cycle that occurs 5 cycles before the end of the
        trace.
      . 1:-5 refers to a range from cycle 1 through the cycle that
        occurs 5 cycles before the end of the trace.
      . -5:$ refers to a range from 5 cycles before the end of the trace to
        the end of the trace.
    . For infinite traces, do not specify cycles or ranges.
      You must specify the full range {1:$} or omit cycle specification.
    . Use "-name" to specify a name for this configuration.
    . Use "-config_group_name" to specify a group name for this configuration.
    . Use "-annotation" to annotate the configuration
      with the description enclosed by curly braces.
    . View annotations in the Visualize window by using the mouse to hover
      over the configuration name and reveal a tooltip. You can print
      annotations to the screen with the "-get_annotation" switch.
    . Use "-loop" when adding the constraint to an infinite trace.

-at_least_once (expression |(-property property_name [-regexp]))
    [(-after expression_1) | (-after -after_property property_name [-regexp])]
    [-before expression_1]
    [cycle | cycle_1:cycle_2] [-name name] [-config_group_name config_group_name]
    [-annotation {annotation}] [-loop] [-window window_name]

    Ensure the specified expression or property is true at least once
    between cycle_1 and cycle_2 of the trace.

    The generated trace will have at least as many
```

```
cycles as cycle_1.

Implementation guidance:
. Specify an expression or use "-property" for this
  configuration.
. Use "-after" or "-after -after_property" to configure a trace that
  shows one scenario occurring after another has occurred.
  When using these after switches, the cycle count described below
  begins after "expression_1" (or "property_name") instead of after reset.
. The "-property" and "-after -after_property" switches support wildcards
  and regular expressions. An error results unless there is exactly one
  matching property. When using a regular expression, use the "-regexp"
  switch after the pattern:
  visualize... -property {pattern} -regexp
. Use "-before" to configure a trace that shows one scenario occurring
  before another has occurred. When using "-before", the cycle count
  described below begins before "expression_1" instead of after reset.
. Use "-after" together with "-before" to configure a trace that
  shows one scenario occurring between two others. When using both
  "-after" and "-before" the cycle count described below actually
  determines the number of times the primary behavior will occur.
. When cycle_1=1, the expression will be covered at least once between
  the first cycle of the trace (the first cycle after reset) and
  cycle_2; when cycle_2=$, the expression will be covered at least
  once between cycle_1 and the last cycle of the trace.
. Use the cycle range $:$ to see the expression covered at the last
  cycle of the generated trace. This range is not supported with
  "-after" or "-before".
. Use negative cycle numbers to refer to a range that counts back from
  the end of the trace, for example:
  . -7:-5 refers to a range from 7 cycles before the end of the trace
    through the cycle that occurs 5 cycles before the end of the
    trace.
  . 1:-5 refers to a range from cycle 1 through the cycle that
    occurs 5 cycles before the end of the trace.
  . -5:$ refers to a range from 5 cycles before the end of the trace to
    the end of the trace.
. For infinite traces, do not specify cycles or ranges.
  You must specify the full range {1:$} or omit cycle specification.
. Use "-name" to specify a name for this configuration.
. Use "-config_group_name" to specify a group name for this configuration.
. Use "-annotation" to annotate the configuration
  with the description enclosed by curly braces.
```

```
    . View annotations in the Visualize window by using the mouse
      to hover over the configuration name and reveal a tooltip.
      You can print annotations to the screen with the "-get_annotation"
      switch.
    . Use "-loop" when adding the constraint to an infinite trace.
    . By default, the tool applies your "visualize" command to the current
      window. Use "-window" to apply your command to a window other
      than the current window.
      . If you specify a name that does not exist yet, this switch renames the
        current window with the specified name.
      . If there are no Visualize windows, a new window opens with the
        specified name.

-not -soft expression [-name name]
    [-config_group_name config_group_name] [-annotation {annotation}]
    [cycle | cycle_1:cycle_2]
    [-window window_name]

    Prefer traces where the opposite of the specified expression is satisfied
    in as many cycles as possible.

    NOTE: When there are multiple "-not -soft" configurations, they might
    conflict with each other. The trace given is guaranteed to present a
    local maxima satisfying as many as possible in as many cycles as
    possible; however, soft constraints are optional, and will never affect
    (extend) trace length.

    Implementation guidance:
    . Use "-name" to specify a name for this configuration.
    . Use "-config_group_name" to specify a group name for this configuration.
    . Use "-annotation" to annotate the configuration with the
      description enclosed in the curly braces.
    . View annotations in the Visualize window by using the mouse to hover
      over the configuration name and reveal a tooltip. You can print
      annotations to the screen with the "-get_annotation" switch.
    . Specifying a cycle is optional. If you do not specify a cycle,
      that is equivalent to "always".
      . If you specify a single cycle (for example, 7), you are
        asking the tool to prefer the opposite of the specified expression in
        cycle seven.
      . Use $ to mean "the last cycle" and $-N to mean "N cycles before
        the last cycle".
      . When cycle_1=1, the tool will give preference to showing the
```

```
        opposite of the expression being covered between the first cycle of
        the trace (the first cycle after reset) and cycle_2; when
        cycle_2=$, the tool gives preference to showing the opposite of the
        expression being covered between cycle_1 and the last cycle of
        the trace.
    . By default, the tool applies your "visualize" command to the current
      window. Use "-window" to apply your command to a window other
      than the current window.
      . If you specify a name that does not exist yet, this switch renames the
        current window with the specified name.
      . If there are no Visualize windows, a new window opens with the
        specified name.

    NOTE: If you use "-not -soft" to create a soft constraint,
    QuietTrace will not artificially display the specified signal as
    constant low.

-not (expression |(-property property_name [-regexp]))
    [cycle | cycle_1:cycle_2] [-name name]
    [  (-after expression_1)
     | (-after -after_property property_name [-regexp])
     | (-before expression_1)]
    [-config_group_name config_group_name] [-annotation {annotation}]
    [-window window_name]

    Ensure the opposite of the specified expression or property is true for the
    specified cycle or range of cycles of the trace.

    The generated trace will have at least as many cycles as cycle_1.
    If you do not specify cycles, that is equivalent to "always".

    Implementation guidance:
    . Specify an expression or use "-property" for this
      configuration.
    . Use "-after" or "-after -after_property" to configure a trace that
      shows one scenario occurring after another has occurred.
      When using these after switches, the cycle count described below
      begins after "expression_1" (or "property_name") instead of after reset.
    . The "-property" and "-after -after_property" switches support wildcards
      and regular expressions. An error results unless there is exactly one
      matching property. When using a regular expression, use the "-regexp"
      switch after the pattern:
      visualize... -property {pattern} -regexp
```

```
    . Use "-before" to configure a trace that shows one scenario occurring
      before another has occurred. When using "-before", the cycle count
      described below begins before "expression_1" instead of after reset.
    . When cycle_1=1, the opposite of the expression will be covered between
      the first cycle of the trace (the first cycle after reset) and
      cycle_2; when cycle_2=$, the opposite of the expression will be
      covered between cycle_1 and the last cycle of the trace.
    . Use the cycle range $:$ to force the opposite of the expression or
      property at the last cycle of the generated trace. This range is not
      supported with "-after" or "-before".
    . Use "-name" to specify a name for this configuration.
    . Use "-config_group_name" to specify a group name for this configuration.
    . Use "-annotation" to annotate the configuration
      with the description enclosed by curly braces.
    . View annotations in the Visualize window by using the mouse
      to hover over the configuration name and reveal a tooltip.
      You can print annotations to the screen with the "-get_annotation"
      switch.
    . By default, the tool applies your "visualize" command to the current
      window.
    . Use "-window" to apply your command to a window other
      than the current window.
      . If you specify a name that does not exist yet, this switch renames the
        current window with the specified name.
      . If there are no Visualize windows, a new window opens with the
        specified name.

-min_length length [-window window_name]
    Extend the trace to have a minimum length.

    By default, the tool applies your "visualize" command to the current
    window. Use "-window" to apply your command to a window other
    than the current window.
    . If you specify a name that does not exist yet, this switch renames the
      current window with the specified name.
    . If there are no Visualize windows, a new window opens with the
      specified name.

-max_length length [-window window_name]
    Set the specified length as the maximum trace length the
    Visualize engine will attempt to generate (default $).

    To remove the upper bound on the length of the trace,
```

```
    use $ for "length".

    By default, the tool applies your "visualize" command to the current
    window. Use "-window" to apply your command to a window other
    than the current window.
    . If you specify a name that does not exist yet, this switch renames the
      current window with the specified name.
    . If there are no Visualize windows, a new window opens with the
      specified name.

-freeze cycle [-window window_name]
    Lock the values of all the inputs and the initial values
    of the registers (even those that are not shown in the
    Visualize window) from the beginning of the trace through the
    specified cycle.

    The "-freeze" switch accepts negative numbers for the "cycle" argument.
    Using a negative number indicates that you want to count back from the end
    of the trace and freeze all cycles up to that point. The following
    examples use a 10-cycle trace:
    . -4 freezes the first six cycles of the trace.
    . 4 freezes the first four cycles of the trace.
    . $ freezes all 10 cycles.

    By default, the tool applies your "visualize" command to the current
    window. Use "-window" to apply your command to a window other
    than the current window.
    . If you specify a name that does not exist yet, this switch renames the
      current window with the specified name.
    . If there are no Visualize windows, a new window opens with the
      specified name.

    NOTE:
    . Freezing is not supported with X-Prop properties.
    . If an assertion can be violated, different engines can generate different
      traces, but no engine will fail to generate a trace except when using the
      Visualize freeze-and-extend mode of analysis. In these cases, it is
      possible that, using the same command set that successfully generated a
      trace with one engine will fail to generate a trace with another engine
      because the frozen portion can be different for different engines.
    . Traces with frozen cycle are cleared when using the "virtual_net" command.

-replot [-new_window [new_window_name]] [-force]
```

```
    [-engine (engine_mode | {engine_mode+})] [-proof_time time_limit]
    [-quiet] [-min_variation]
    [-prompt] [-batch] [-silent] [-bg]
    [(-sig_order sig_order_file_name)|-no_signals]
    [-window window_name]
Regenerate a trace that satisfies the current
configuration (that is, update the trace based on reconfiguration).

. Use "-new_window" to view the results of the replot operation in a new
  window while leaving the original window intact. If another replot
  operation is running in the background for the original window or the
  switches "-engine", "-quiet", and "-min_variation" are not used, the new
  window will plot the last finished replot configuration.

  To specify the new window's name, use the optional new_window_name
  argument.
  NOTE:
  . If you do not provide the new_window_name argument, a tool-generated
    name is used.
  . The new window becomes the "current window."

. Use "-force" to ignore any cached trace core and
  call the engine to generate a new full trace.

. Use "-force" in conjunction with the "-engine" switch to
  replot using a different engine. Using "-engine" without
  "-force" only works if there is no cached trace core or you use the
  "-new_window" switch. In  addition, "-engine" only works for the current
  "-replot" command. For future "-replot" commands, the default engine
  applies unless you specify "-engine".

. Use "-engine" to specify one or more engine modes you want Visualize to
  use. Specify a single engine or use curly braces to specify
  parallel-engine processing.

. Use "-proof_time" to set a limit on the time for this command. See
  "set_prove_time_limit".

. Use "-quiet" to enable QuietTrace.

. Use "-min_variation" to replot with as little change as possible in the
  trace. This switch helps you understand the new trace by suppressing
  differences that are unrelated to changes you made prior to replotting.
```

```
    . Use "-batch" to prevent the tool from displaying the
      trace it generates, which improves performance.

    . Use "-prompt" to generate the new trace without affecting the Visualize
      window. When the new trace is ready, the tool prompts you and offers
      to display the new trace.

    . Use "-silent" to turn off the output to the screen and return information
      with a Tcl return value.

      NOTE: "-silent" does not stop the trace from being shown in the Visualize
      window. You must use "-batch" to suppress the trace.

    . Use "-bg" to run the command in the background and continue to accept
      and run new commands.

    . Use "-sig_order" to generate a trace that displays only the signals in
      the specified signal order file.

    . Use "-no_signals" to load a trace without displaying signals.

    . By default, the tool applies your "visualize" command to the current
      window. Use "-window" to apply your command to a window other
      than the current window.

    . The SPV and X-Prop Apps do not support this command.

-go_backward [-new_window [new_window_name]] [-force] [-min_variation] [-batch]
        [-silent] [-bg] [-proof_time time_limit]
        [(-sig_order sig_order_file_name)|-no_signals]
        [-window window_name]
    If there are new configurations, remove them and
    keep the current trace. Otherwise, go back to the
    previous trace.

    Returns "false" if it is not possible to go backward
    (because we are already at the first trace) or if
    the set of configurations we reach when we go backward
    is one where no trace is possible. Otherwise, it returns
    "true".

    . Use "-new_window" to display the results of the "-go_backward" operation
```

```
      in a new window while leaving the original window intact.

      To specify the new window's name, use the optional new_window_name
      argument.
      NOTE:
      . If you do not provide the new_window_name argument, a tool-generated
        name is used.
      . The new window becomes the "current window."

    . Use "-force" to ignore any cached trace core and call the engine to
      generate a new full trace.

    . Use "-min_variation" to replot with as little change as possible in the
      trace. This switch helps you understand the new trace by suppressing
      differences that are unrelated to changes you made prior to replotting.

    . Use "-batch" to prevent the tool from displaying the trace it generates,
      which improves performance.

    . Use "-silent" to turn off the output to the screen and return information
      with a Tcl return value.

    . Use "-bg" to run the command in the background and continue to accept
      and run new commands.

    . Use "-proof_time" to set a limit on the time for this command. See
      "set_prove_time_limit".

    . Use "-sig_order" to generate a trace that displays only the signals in
      the specified signal order file.

    . Use "-no_signals" to load a trace without displaying signals.

    . By default, the tool applies your "visualize" command to the current
      window. Use "-window" to apply your command to a window other
      than the current window.

-go_forward [-new_window [new_window_name]] [-force] [-min_variation] [-batch]
        [-silent] [-bg] [-proof_time time_limit]
        [(-sig_order sig_order_file_name)|-no_signals]
        [-window window_name]
    Go forward to the next trace.
```

```
Returns "false" if it is not possible to go forward
(because we are already at the last trace) or if
the set of configurations we reach when we go forward
is one where no trace is possible. Otherwise, it returns
"true".

. Use "-new_window" to display the results of the "-go_forward" operation
  in a new window while leaving the original window intact.

  To specify the new window's name, use the optional new_window_name
  argument.
  NOTE:
  . If you do not provide the new_window_name argument, a tool-generated
    name is used.
  . The new window becomes the "current window."

. Use "-force" to ignore any cached trace core and call the engine to
  generate a new full trace.

. Use "-min_variation" to replot with as little change as possible in the
  trace. This switch helps you understand the new trace by suppressing
  differences that are unrelated to changes you made prior to replotting.

. Use "-batch" to prevent the tool from displaying the trace it generates,
  which improves performance.

. Use "-silent" to turn off the output to the screen and return information
  with a Tcl return value.

. Use "-bg" to run the command in the background and continue to accept
  and run new commands.

. Use "-proof_time" to set a limit on the time for this command. See
  "set_prove_time_limit".

. Use "-sig_order" to generate a trace that displays only the signals in
  the specified signal order file.

. Use "-no_signals" to load a trace without displaying signals.

. By default, the tool applies your "visualize" command to the current
  window. Use "-window" to apply your command to a window other
  than the current window.
```

```
-loop_unroll N [-batch]
        [(-sig_order sig_order_file_name) |-no_signals]
        [-window window_name]
    Expand the trace by unrolling the loop N times (that is, repeat the looped
    portion of the trace). This command only applies to looping traces.

    . Use "-batch" to prevent the tool from displaying the trace it generates,
      which improves performance.

    . Use "-sig_order" to generate a trace that displays only the signals in
      the specified signal order file.

    . Use "-no_signals" to load a trace without displaying signals.

    . By default, the tool applies your "visualize" command to the current
      window. Use "-window" to apply your command to a window other
      than the current window.

-get_loop_unroll [-window window_name]
    Report how many times the loop was unrolled.

    By default, the tool applies your "visualize" command to the current
    window. Use "-window" to apply your command to a window other
    than the current window.

-get_show_reset_cycles [-window window_name]
    Report whether reset cycles are visible in the specified window.

    Use "-window" to apply your command to a window other than the current
    window.

-set_show_reset_cycles (on | off)
    [-window window_name]
    By default, Visualize does not display reset cycles in the waveforms pane.
    Use this command to change the current setting.

    . Use "on" to display reset cycles.
    . Use "off" to hide reset cycles.
    . Use "-window" to apply your command to a window other than the current
      window.

    NOTE: "set_trace_show_reset" must be set to "true" (default) before reset
```

```
    analysis. Check the current setting with "get_trace_show_reset".

-set_infinite_extension (true | false) [-window window_name]
    Enable generation of traces with an infinite extension for safety or finite
    properties.
    When enabled, the tool generates a looping trace for a safety or finite
    property, which is useful for determining that there are no dead ends for a
    trace previously found by the tool. This switch has limitations, and it does
    not work under the following conditions:
    . If the target is liveness assertion, infinite cover, X-Prop property,
      or SPV property.
    . If you have enabled the following Visualize configurations: min/max
      length and the user-specified Visualize constraint "-not".

    NOTE: The target must be defined in some task.

    For more information, refer to "set_trace_extension".

    . Use "true" to turn infinite extension on.

    . Use "false" to turn infinite extension off.

    . By default, the tool applies your "visualize" command to the current
      window. Use "-window" to apply your command to a window other
      than the current window.

-get_infinite_extension [-window window_name]
    Report the status of infinite extension.
    The tool returns 1 if it is enabled and 0 if not.
    For more information, refer to "visualize -set_infinite_extension".

    NOTE:
    . This command being enabled does not mean the trace found has an
      infinite extension. Use "visualize -is_looping" to determine if the trace
      has an infinite extension.
    . By default, the tool applies your "visualize" command to the current
      window. Use "-window" to apply your command to a window other
      than the current window.

-get_zoom_range [-window window_name]
    Report the range of clock cycles visible in the specified window.

    Use "-window" to apply your command to a window other than the current
```

```
    window.

-set_zoom_range [M:N]
    [-window window_name]
    Change the range of clock cycles visible in the specified window.

    . Specify a range of cycles, for example, 1:5.
    . Use $ to mean "the last cycle".
    . Use "-window" to apply your command to a window other than the current
      window.

    NOTE: For zoom fit, omit cycle specification.

Utility Switches - For Configurations
-------------------------------------
-rename_conf old_conf_name new_conf_name [-window window_name]
    Rename the specified configuration (-force|-at_least_once).

    By default, the tool applies your "visualize" command to the current
    window. Use "-window" to apply your command to a window other
    than the current window.
    . If you specify a name that does not exist yet, this switch renames the
      current window with the specified name.
    . If there are no Visualize windows, a new window opens with the
      specified name.

-remove_conf conf_name [-window window_name]
    Remove the specified configuration (-force|-at_least_once).

    By default, the tool applies your "visualize" command to the current
    window. Use "-window" to apply your command to a window other
    than the current window.
    . If you specify a name that does not exist yet, this switch renames the
      current window with the specified name.
    . If there are no Visualize windows, a new window opens with the
      specified name.

    This switch supports wildcards (* and ?) in the name.

-disable_conf conf_name [-window window_name]
    Keep the configuration but do not pass the information to
    the proof engines or include it in any replot.
```

```
    By default, the tool applies your "visualize" command to the current
    window. Use "-window" to apply your command to a window other
    than the current window.
    . If you specify a name that does not exist yet, this switch renames the
      current window with the specified name.
    . If there are no Visualize windows, a new window opens with the
      specified name.

    NOTE: Wildcards are supported.

-enable_conf conf_name [-window window_name]
    Re-enable the specified (previously disabled) configuration.

    By default, the tool applies your "visualize" command to the current
    window. Use "-window" to apply your command to a window other
    than the current window.
    . If you specify a name that does not exist yet, this switch renames the
      current window with the specified name.
    . If there are no Visualize windows, a new window opens with the
      specified name.

    NOTE: Wildcards are supported.

-offset_conf conf_name offset_value [-window window_name]
    Add offset_value to both cycles of the configuration
    with the given name.

    By default, the tool applies your "visualize" command to the current
    window. Use "-window" to apply your command to a window other
    than the current window.
    . If you specify a name that does not exist yet, this switch renames the
      current window with the specified name.
    . If there are no Visualize windows, a new window opens with the
      specified name.

    This command supports wildcards (* and ?) in the name.
    "offset_value" must be a positive integer.

    Refer to the examples below.

-clear_conf [-window window_name]
    Remove all existing configurations for the next trace
    without clearing previously generated traces.
```

```
    By default, the tool applies your "visualize" command to the current
    window. Use "-window" to apply your command to a window other
    than the current window.
    . If you specify a name that does not exist yet, this switch renames the
      current window with the specified name.
    . If there are no Visualize windows, a new window opens with the
      specified name.

-replace_conf (-force -soft |-not -soft) expression
         [cycle | cycle:cycle] -name conf_name [-window window_name]
    Replace the existing configuration (-name conf_name) with
    a configuration specified by the remaining "-replace_conf"
    parameters.

    By default, the tool applies your "visualize" command to the current
    window. Use "-window" to apply your command to a window other
    than the current window.
    . If you specify a name that does not exist yet, this switch renames the
      current window with the specified name.
    . If there are no Visualize windows, a new window opens with the
      specified name.

    NOTE:
    . These switches mirror the GUI procedure for editing
      an existing configuration. You will typically use the GUI
      for this purpose.
    . "-replace_conf" does not support "-group_name".

-replace_conf (-force |-not |-at_least_once)
         (expression |-property property_name)
         [(-after expression_1) | (-after -after_property property_name [-regexp])]
         [-before expression_1]
         [cycle | cycle:cycle]
         -name conf_name
         [-window window_name]
         [-loop]
    Replace the existing configuration (-name conf_name) with
    a configuration specified by the remaining "-replace_conf"
    parameters.

    By default, the tool applies your "visualize" command to the current
    window. Use "-window" to apply your command to a window other
```

```
    than the current window.
    . If you specify a name that does not exist yet, this switch renames the
      current window with the specified name.
    . If there are no Visualize windows, a new window opens with the
      specified name.
    . Use -loop when replacing a force or at_least_once constraint in an
      infinite trace.

    NOTE:
    . These switches mirror the GUI procedure for editing
      an existing configuration. You will typically use the GUI
      for this purpose.
    . The "-property" switch supports wildcards and regular expressions. An
      error results unless there is exactly one matching property. When using a
      regular expression, use the "-regexp" switch after the pattern:
      visualize... -property {pattern} -regexp
    . "-replace_conf" does not support "-group_name".

-replace_conf -signal signal_name -sequence
         ('{'value['{'cycles'}'][ value['{'cycles'}']]*'}' [cycle])
         -name conf_name [-window window_name]
    Replace the specified configuration (-name conf_name) with a configuration
    specified by the remaining "-replace_conf" parameters.

    By default, the tool applies your "visualize" command to the current
    window. Use "-window" to apply your command to a window other
    than the current window.
    . If you specify a name that does not exist yet, this switch renames the
      current window with the specified name.
    . If there are no Visualize windows, a new window opens with the
      specified name.

    NOTE:
    . These switches mirror the GUI procedure for editing
      an existing configuration. You will typically use the GUI
      for this purpose.
    . "-replace_conf" does not support "-group_name".

Utility Switches - General
--------------------------
-list [-silent] [-all | config_group_name] [-window window_name]
    List the configuration that is currently specified.
```

```
    The tool lists the Visualize configurations you specified thus far for
    ungrouped (top-level) configurations.
    . Use "-silent" to turn off the output to the screen and return information
      with a Tcl return value.
    . Use "-all" to view all configurations, grouped or ungrouped.
    . Provide a group_name to view configurations in the specified group.
    . By default, the tool applies your "visualize" command to the current
      window. Use "-window" to apply your command to a window other
      than the current window.

-list -all_windows [-silent]
    List the names of all Visualize windows.

    Use "-silent" to turn off the output to the screen and return information
    with a Tcl return value.

-save [-script] [-force]
      [-annotation [-user_defined_only]] [-marker] [-custom_button]
      [ -setup_only [-window window_name]
       |-config_only [-window window_name]
      ]
       [ -add_hier_path path_name
        |-remove_hier_path path_name]
      file_name
    Save the configurations for the current trace into
    the specified script file.

    Pending (blue) configurations added since the last "replot"
    are saved as pending configurations.

    If "file_name" already exists, the command returns an error. To overwrite
    the existing file, use the "-force" switch.

    The script contains both setup (analyze, elaborate, etc.)
    and configuration. To limit the contents of the script:

    . Use "-setup_only" to save only the setup portion of the
      script, not the actual Visualize configuration.
    . Use "-config_only" to save only the Visualize configuration
      portion of the script, not the setup.
      . Use "-add_hier_path" or "-remove_hier_path" to run scripts generated
        for RTL with a different hierarchy.
      . See "session -set_hier_path" for determining the value of the path
```

```
        argument.
    . Use "-annotation" to save annotations to the script file.
      . Use "-user_defined_only" to save custom annotations only.
      NOTE: If your design changes, Visualize might not be able to
      successfully restore annotations.
    . Use "-marker" to save custom markers to the script file.
    . Use "-custom_button" to save custom buttons defined by the
      "custom_gui_action" command for the Visualize window.
      NOTE: The custom buttons can malfunction if the restoring environment
      is not the same as the capturing environment, for example, a different
      path for the icon image file or Tcl-dependent files.
    . By default, the tool applies your "visualize" command to the current
      window. Use "-window" to apply your command to a window other
      than the current window.

    NOTE: Specifying "visualize -save file_name" is the same as
    "visualize -save -script file_name."

-save -init_state file_name [-cycle N] [-force]
    [-include_internal] [-include_encrypted] [-bit_blast]
    [(-window window_name) |-reset_window]
    Save the signal values from the last or specified cycle of the trace in a
    format suitable for use with "reset -init_state".

    This command only saves values for registers that are part of the AR/COI of
    the current Visualize window. Run "set_trace_optimization standard" before
    the initial trace generation to dump values for all design registers
    instead.
    NOTE:
    . The following apps do not honor "set_trace_optimization" in this flow:
      SEC, SPV, X-Prop, C2RTL, and HLS.

    The saved file reflects the configurations you specified prior to the last
    time you used "visualize -replot". Pending configurations added since the
    last "replot" are not used.

    . Use "-cycle" to specify a cycle other than the last cycle.

    . If the specified file name already exists, use "-force" to replace the
      existing file. If you do not use "-force", the tool issues an error
      message.

    . Use "-include_internal" to save the internal state based on assumptions,
```

```
      assertions, temporal operators, and so forth.

    . Use "-include_encrypted" to capture the values of encrypted flops and
      the internal state of Proof Accelerators in an encrypted file.

    . Use "-bit_blast" to save each bit of a signal separately.

    . By default, the tool applies your "visualize" command to the current
      window. Use "-window" to apply your command to a window other
      than the current window.
      . If you specify a name that does not exist yet, this switch renames the
        current window with the specified name.
      . If there are no Visualize windows, a new window opens with the
        specified name.
      . If you have already launched a Reset Analysis window, use
        "-reset_window" to apply your command to the Reset Analysis window.

    CAUTION: Loading files with the switch "-include_internal" carries a risk
    of a mismatch in internal signals that can lead to false proof results in
    some cases. Internal signals in reset files might not be the same as those
    generated by a different Jasper release version or a different version
    of the design or a different design setup and configuration. (Design setup
    and configuration differences include the order in which files are passed
    to "analyze" and "elaborate" and the order in which assumptions and
    assertions are created.)

-save -reset_sequence [-force] file_name
    [-include_internal] [-include_encrypted] [-bit_blast]
    [(-window window_name) |-reset_window]
    Save the register values from the first cycle of the trace and
    the input values from all cycles of the trace in a format
    suitable for use with "reset -sequence".

    This command only saves values for registers that are part of the AR/COI of
    the current Visualize window. Run "set_trace_optimization standard" before
    the initial trace generation to dump values for all design registers and
    inputs instead.
    NOTE:
    . The following apps do not honor "set_trace_optimization" in this flow:
      SEC, SPV, X-Prop, C2RTL, and HLS.

    The saved file reflects the configurations you specified
    prior to the last time you used "visualize -replot".
```

```
    Pending configurations added since the last "replot" are
    not used.

    If the specified file name already exists, use "-force" to replace the
    existing file. If you do not use "-force", the tool issues an error
    message.

    . Use "-include_internal" to save the internal state based on assumptions,
      assertions, temporal operators, and so forth.

    . Use "-include_encrypted" to capture sequence values for encrypted signals
      and the internal state of Proof Accelerators in an encrypted file.

    . Use "-bit_blast" to save each bit of a signal separately.

    . By default, the tool applies your "visualize" command to the current
      window. Use "-window" to apply your command to a window other
      than the current window.
      . If you specify a name that does not exist yet, this switch renames the
        current window with the specified name.
      . If there are no Visualize windows, a new window opens with the
        specified name.
      . If you have already launched a Reset Analysis window, use
        "-reset_window" to apply your command to the Reset Analysis window.

    CAUTION: Loading files with the switch "-include_internal" carries a risk
    of a mismatch in internal signals that can lead to false proof results in
    some cases. Internal signals in reset files might not be the same as those
    generated by a different Jasper release version or a different version
    of the design or a different design setup and configuration. (Design setup
    and configuration differences include the order in which files are passed
    to "analyze" and "elaborate" and the order in which assumptions and
    assertions are created.)

-save -vcd [-imp |-spec] vcd_file_name [-force]
           [-sig_order sig_order_file_name]
           [-last_cycle cycle] [-visible_only]
           [(-window window_name) |-reset_window]

    Save the current trace values in VCD format to the specified
    "vcd_file_name".

  . Use "-imp" to save only SEC imp signals.
```

```
  . Use "-spec" to save only SEC spec signals.
    NOTE: "-imp" and "-spec" are mutually exclusive. If you do not use
    "-imp" or "-spec", the tool saves both imp and spec signals by default.

    . Use "-force" if the specified VCD file already exists and you want to
      overwrite it.
    . Use "-sig_order" and specify a "sig_order_file_name" to
      save only the values of signals that are in "sig_order_file_name".
    . Use "-last_cycle" to define the last cycle of the saved trace.
    . Use "-visible_only" to write only RTL-defined signals to the file.
      NOTE: This switch is recommended only if the exported trace will be used
      on a tool that is not Jasper.
    . By default, the tool applies your "visualize" command to the current
      window.
      . Use "-window" to apply your command to a window other than the current
        window.
      . If you have already launched a Reset Analysis window, use
        "-reset_window" to apply your command to the Reset Analysis window.
      . If you specify a name that does not exist yet, this switch renames the
        current window with the specified name.
      . If there are no Visualize windows, a new window opens with the
        specified name.

    The VCD file preserves design hierarchy.

    NOTE:
    . If you have also used the "set_trace_optimization"
      command, the VCD file reflects those settings, for example:
      . If trace optimization is "ar", then only the analysis region is
        saved into the VCD file. (This is the default setting.)
      . If trace optimization is "standard", then the entire design
        is dumped into the VCD. However, this setting can increase the time
        it takes to generate a trace.

    . Run the "set_trace_optimization" command before generating a trace.
      NOTE: The following apps do not honor "set_trace_optimization" in this
      flow: SEC, SPV, X-Prop, C2RTL, and HLS.

    LIMITATION: The command does not support properties with local variables.

-save -fsdb [-imp |-spec] fsdb_file_name [-force]
            [-time_scale [N][time_unit]]
            [-sig_order sig_order_file_name]
```

```
        [-hier_map -from source_hier -to destination_hier]
        [-last_cycle cycle] [-visible_only]
        [(-window window_name) |-reset_window]
        [-include_reset_cycles]
Save the current trace values in FSDB format to the specified
"fsdb_file_name".

IMPORTANT:
. Jasper Apps writes 6.3 FSDB traces and reads version 6.6 FSDB
  traces. FSDB Reader 6.6 was introduced by Verdi version X-2025.06.
. FSDBs you generate with the latest Jasper are backwards compatible
  with all Jasper versions from 2025.03 FCS and newer. In addition,
  they are backwards compatible with older versions of Verdi released
  during that same period.
. Jasper Apps supports backwards compatibility, that is, the tool can
  read FSDBs generated from older versions of Jasper Apps and
  Synopsys Verdi.

. Use "-imp" to save only SEC imp signals.
. Use "-spec" to save only SEC spec signals.
  NOTE: "-imp" and "-spec" are mutually exclusive. If you do not use
  "-imp" or "-spec", the tool saves both imp and spec signals by default.

. Use "-force" if the specified FSDB file already exists, and you want to
  overwrite it.
. Use "-sig_order" and specify a "sig_order_file_name" to
  save only the values of signals that are in "sig_order_file_name".
. Use "-last_cycle" to define the last cycle of the saved trace.
. Use "-hier_map" for trace files that must map a design hierarchy in the
  simulation environment that differs from the formal environment.
  . Use "-from" to specify the source (formal environment) hierarchy.
  . Use "-to" to specify the destination (simulation environment) hierarchy.
. Use "-visible_only" to write only RTL-defined signals to the file.
  NOTE: This switch is recommended only if the exported trace will be used
  on a tool that is not Jasper.
. Use "-time_scale" to specify the time scale:
  . N is mandatory if you do not specify "time_unit".
    . If you use N and do not specify "time_unit", the default is
      nanoseconds (ns).
    . N is a natural non-zero number.
  . "time_unit" is mandatory if you do not specify N.
    . If you use "time_unit" and do not specify N, the default is 1.
    . "time_unit" is s, ms, us, ns, ps, or fs.
```

```
    . By default, the tool applies your "visualize" command to the current
      window.
      . Use "-window" to apply your command to a window other than the current
        window.
      . If you have already launched a Reset Analysis window, use
        "-reset_window" to apply your command to the Reset Analysis window.
      . If you specify a name that does not exist yet, this switch renames the
        current window with the specified name.
      . If there are no Visualize windows, a new window opens with the
        specified name.
    . Use "-include_reset_cycles" to save the reset cycles prepended on the
      beginning of the saved waveform.

    The FSDB file preserves design hierarchy.

    NOTE:
    . If you have also used the "set_trace_optimization" command, the FSDB file
      reflects those settings, for example:
      . If trace optimization is "ar", then only the analysis region is
        saved into the FSDB file. (This is the default setting.)
      . If trace optimization is "standard", then the entire design
        is dumped into the FSDB. However, this setting can increase the time
        it takes to generate a trace.

    . Run the "set_trace_optimization" command before generating a trace.
      NOTE: The following apps do not honor "set_trace_optimization" in this
      flow: SEC, SPV, X-Prop, C2RTL, and HLS.

    LIMITATIONS:
    . Compound blackboxed output values are saved as instances.
    . The command does not support properties with local variables.

-save -shm [-imp |-spec] shm_directory_name [-force]
           [-time_scale [N][time_unit]]
           [-sig_order sig_order_file_name]
           [-hier_map -from source_hier -to destination_hier]
           [-last_cycle cycle] [-visible_only]
           [(-window window_name) |-reset_window]
           [-include_reset_cycles]
    Save the current trace values in SST2/SHM format to the specified
    "shm_directory_name".

    . Use "-imp" to save only SEC imp signals.
```

```
. Use "-spec" to save only SEC spec signals.
  NOTE: "-imp" and "-spec" are mutually exclusive. If you do not use
  "-imp" or "-spec", the tool saves both imp and spec signals by default.

. Use "-force" if the specified SHM directory already exists and you want
  to overwrite it.
. Use "-time_scale" to specify the time scale:
  . N is mandatory if you do not specify "time_unit".
    . If you use N and do not specify "time_unit", the default is
      nanoseconds (ns).
    . N is a natural non-zero number.
  . "time_unit" is mandatory if you do not specify N.
    . If you use "time_unit" and do not specify N, the default is 1.
    . "time_unit" is s, ms, us, ns, ps, or fs.
. Use "-sig_order" and specify a "sig_order_file_name" to save only the
  values of signals that are in "sig_order_file_name".
. Use "-last_cycle" to define the last cycle of the saved trace.
. Use "-hier_map" for trace files that must map a design hierarchy in the
  simulation environment that differs from the formal environment.
  . Use "-from" to specify the source (formal environment) hierarchy.
  . Use "-to" to specify the destination (simulation environment) hierarchy.
. Use "-visible_only" to write only RTL-defined signals to the file.
  NOTE: This switch is recommended only if the exported trace will be used
  on a tool that is not Jasper.
. By default, the tool applies your "visualize" command to the current
  window.
  . Use "-window" to apply your command to a window other than the current
    window.
  . If you have already launched a Reset Analysis window, use
    "-reset_window" to apply your command to the Reset Analysis window.
  . If you specify a name that does not exist yet, this switch renames the
    current window with the specified name.
  . If there are no Visualize windows, a new window opens with the
    specified name.
. Use "-include_reset_cycles" to save the reset cycles prepended on the
  beginning of the saved waveform.

NOTE:
. The SHM directory preserves design hierarchy.

. If you have also used the "set_trace_optimization" command, the SHM
  directory reflects those settings, for example:
  . If trace optimization is "ar", then only the analysis region is
```

```
        saved into the SHM directory. (This is the default setting.)
      . If trace optimization is "standard", then the entire design
        is dumped into the SHM. However, this setting can increase the time
        it takes to generate a trace.

    . Run the "set_trace_optimization" command before generating a trace.
      NOTE: The following apps do not honor "set_trace_optimization" in this
      flow: SEC, SPV, X-Prop, C2RTL, and HLS.

    LIMITATIONS:
    . Compound blackboxed output values are saved as instances.
    . The command does not support properties with local variables.

-save -vwdb [-imp |-spec] vwdb_directory_name [-force]
           [-time_scale [N][time_unit]]
           [-sig_order sig_order_file_name]
           [-hier_map -from source_hier -to destination_hier]
           [-last_cycle cycle] [-visible_only]
           [(-window window_name) |-reset_window]
           [-include_reset_cycles]
    Save the current trace values in VWDB format to the specified
    "vwdb_directory_name".

    . Use "-imp" to save only SEC imp signals.
    . Use "-spec" to save only SEC spec signals.
      NOTE: "-imp" and "-spec" are mutually exclusive. If you do not use
      "-imp" or "-spec", the tool saves both imp and spec signals by default.

    . Use "-force" if the specified VWDB directory already exists and you want
      to overwrite it.
    . Use "-time_scale" to specify the time scale:
      . N is mandatory if you do not specify "time_unit".
        . If you use N and do not specify "time_unit", the default is
          nanoseconds (ns).
        . N is a natural non-zero number.
      . "time_unit" is mandatory if you do not specify N.
        . If you use "time_unit" and do not specify N, the default is 1.
        . "time_unit" is s, ms, us, ns, ps, or fs.
    . Use "-sig_order" and specify a "sig_order_file_name" to save only the
      values of signals that are in "sig_order_file_name".
    . Use "-last_cycle" to define the last cycle of the saved trace.
    . Use "-hier_map" for trace files that must map a design hierarchy in the
      simulation environment that differs from the formal environment.
```

```
      . Use "-from" to specify the source (formal environment) hierarchy.
      . Use "-to" to specify the destination (simulation environment) hierarchy.
    . Use "-visible_only" to write only RTL-defined signals to the file.
      NOTE: This switch is recommended only if the exported trace will be used
      on a tool that is not Jasper.
    . By default, the tool applies your "visualize" command to the current
      window.
      . Use "-window" to apply your command to a window other than the current
        window.
      . If you have already launched a Reset Analysis window, use
        "-reset_window" to apply your command to the Reset Analysis window.
      . If you specify a name that does not exist yet, this switch renames the
        current window with the specified name.
      . If there are no Visualize windows, a new window opens with the
        specified name.
    . Use "-include_reset_cycles" to save the reset cycles prepended on the
      beginning of the saved waveform.

    NOTE:
    . If you have also used the "set_trace_optimization" command, the VWDB
      directory reflects those settings, for example:
      . If trace optimization is "ar", then only the analysis region is
        saved into the VWDB directory. (This is the default setting.)
      . If trace optimization is "standard", then the entire design
        is dumped into the VWDB. However, this setting can increase the time it
        takes to generate a trace.

    . Run the "set_trace_optimization" command before generating a trace.
      NOTE: The following apps do not honor "set_trace_optimization" in this
      flow: SEC, SPV, X-Prop, C2RTL, and HLS.

    LIMITATION: This command does not support VHDL designs.

-save -image image_file_name [-window window_name]
    Save the current Visualize window waveform view as a PNG file with
    the specified image_file_name.

    NOTE: Use "-window" to apply your command to a window other than the current
    window.

-load (-vcd |-fsdb |-shm |-vwdb) trace_file_name
       [-sampling (-infer |-clock_period N[time_unit])]
       [-time_scale [N][time_unit]]
```

```
   [-start_time N] [-end_time N]
   [ -hier_path path_difference
    |-hier_map -from source_hier -to destination_hier]
   [-two_valued_simulation]
   [ -new_window [new_window_name]
    |-window window_name]
   [-batch]
Load the trace from the specified file as a Visualize trace, using the
fastest declared clock to calculate the number of formal cycles.

NOTE:
.  The fastest declared clock must be in the specified trace file;
   otherwise, the tool errors out with a message advising that it is
   unable to load the file unless you use the switch
   "-sampling -clock_period".
. "visualize -save" and "visualize -load" do not preserve X values.

Visualize loads the trace in read-only mode, which means it loads values
for all design signals in memory without running any simulation. This
method is usually much faster than using "visualize -confirm".

. Use "-vcd", "-fsdb", "-shm", or "-vwdb" to specify the type of the
  trace to be interpreted.

  IMPORTANT:
  . Jasper Apps writes 6.3 FSDB traces and reads version 6.6 FSDB
    traces. FSDB Reader 6.6 was introduced by Verdi version X-2025.06.
  . FSDBs you generate with the latest Jasper are backwards compatible
    with all Jasper versions from 2025.03 FCS and newer. In addition,
    they are backwards compatible with older versions of Verdi released
    during that same period.
  . Jasper Apps supports backwards compatibility, that is, the tool can
    read FSDBs generated from older versions of Jasper Apps and
    Synopsys Verdi.

. Use "-sampling" to specify how the tool will sample the trace. By
  default, the tool samples the trace exclusively on the edges of the main
  declared clock.
  . Use "-infer" to determine the trace sampling period by inspecting the
    behavior of clocking signals in the trace. These signals are determined
    by a structural analysis of the design.
  . Use "-clock_period" to define the sampling period explicitly. The
    generated trace will ignore any clock definitions, and the trace cycles
```

will align with the defined period.
NOTE:
. Supported units of measurement are ms, us, ns, and ps.
. The argument is in the form of a constant plus a unit of measurement, for example, "-clock_period 10ns".

. Use "-time_scale" with the unit of measurement (ms, us, ns, or ps) to specify the time scale. If you do not specify a time scale, the tool uses the default from the trace file.
. Use "-hier_path" when the trace file contains a hierarchy that is different from the one in the design loaded in Jasper. For example, if the design top module is "dut_top", which is instantiated as the top module in the testbench "testbench_top" as "i_dut_top", use the following command (replace "file_name" with an actual value):
visualize -load -vcd <file_name> -hier_path testbench_top.i_dut_top
visualize -load -fsdb <file_name> -hier_path testbench_top.i_dut_top
visualize -load -shm <file_name> -hier_path testbench_top.i_dut_top
visualize -load -vwdb <file_name> -hier_path testbench_top.i_dut_top
. Use "-start_time" to skip the initial chunk of simulation data. Typically, you will use this switch to skip the reset sequence and so forth.
. Use "-end_time" to ignore the last chunk of simulation data.
. Use "-hier_map" for trace files generated during simulation if you have a wrapper module.
. Use "-from" to specify the source (simulation environment) hierarchy.
. Use "-to" to specify the destination (formal environment) hierarchy.
. By default, the tool applies three-valued logic semantics (1, 0, X) when loading the trace. In some cases, Visualize supports plotting four-value logic semantics (1, 0, X, Z) for multiple-driven nets. To change to two-valued logic semantics (1, 0), use "-two_valued_simulation".
For differential analysis properties (X-Prop, SPV, and path), four-valued logic semantics are used, and the tooltip displays the original and tainted values.
. By default, the tool applies your "visualize" command to the current window. Use "-window" to apply your command to a window other than the current window.
. If you specify a name that does not exist yet, this switch renames the current window with the specified name.
. If there are no Visualize windows, a new window opens with the specified name.
. Use "-new_window" to generate the waveform in a new window with a name specified by the optional window_name argument.
NOTE:
. If you do not provide the window_name argument, a tool-generated

```
        name is used.
      . The new window becomes the "current window."
    . Use "-batch" to prevent the tool from displaying the
      trace it generates, which improves performance.

    LIMITATIONS:
    . Compound blackboxed output values will not be restored.
    . The command does not support properties with local variables.

-confirm (-vcd |-fsdb |-shm |-vwdb) trace_file_name
       [-sampling (-infer |-clock_period N[time_unit])]
       [-time_scale [N][time_unit]]
       [-start_time N] [-end_time N]
       [ -hier_path path_difference
        |-hier_map -from source_hier -to destination_hier]
       [-two_valued_simulation]
       [-use_reset_analysis]
       [ -new_window [new_window_name]
        |-window window_name]
       [-batch]
    Load the trace from the specified file as a Visualize trace, using the
    fastest declared clock to calculate the number of formal cycles.

    NOTE: The fastest declared clock must be in the specified trace file;
    otherwise, the tool errors out with a message advising that it is
    unable to load the file unless you use the switch "-sampling -clock_period".

    Visualize loads the trace by running a simulation and propagating values
    from inputs to outputs. This command respects the "set_trace_optimization"
    setting when running the simulation. At the end of the simulation,
    Visualize checks for inconsistencies between simulated values and the
    original trace values and reports them in the Visualize window's "Confirmed
    Inconsistencies" tab, which displays the cycle and signal for the
    inconsistency. Refer to the Visualize chapter of the user guide for
    additional information (Help - User Guide).
    NOTE:
    . The following apps do not honor "set_trace_optimization" in this flow:
      SEC, SPV, X-Prop, C2RTL, and HLS.

    The Visualize "-confirm" flow does not support liveness or infinite
    properties. The tool does not draw conclusions for these properties.
    That is, the tool does not report whether a liveness or infinite property
    passed or failed confirmation.
```

. Use "-vcd", "-fsdb", "-shm", or "-vwdb" to specify the type of the trace to be interpreted.

  IMPORTANT:
  . Jasper Apps writes 6.3 FSDB traces and reads version 6.6 FSDB traces. FSDB Reader 6.6 was introduced by Verdi version X-2025.06.
  . FSDBs you generate with the latest Jasper are backwards compatible with all Jasper versions from 2025.03 FCS and newer. In addition, they are backwards compatible with older versions of Verdi released during that same period.
  . Jasper Apps supports backwards compatibility, that is, the tool can read FSDBs generated from older versions of Jasper Apps and Synopsys Verdi.

. Use "-sampling" to specify how the tool will sample the trace. By default, the tool samples the trace exclusively on the edges of the main declared clock.
  . Use "-infer" to determine the trace sampling period by inspecting the behavior of clocking signals in the trace. These signals are determined by a structural analysis of the design.
  . Use "-clock_period" to define the sampling period explicitly. The generated trace will ignore any clock definitions, and the trace cycles will align with the defined period.
    NOTE:
    . Supported units of measurement are ms, us, ns, and ps.
    . The argument is in the form of a constant plus a unit of measurement, for example, "-clock_period 10ns".
. Use "-time_scale" with the unit of measurement (ms, us, ns, or ps) to specify the time scale. If you do not specify a time scale, the tool uses the default from the trace file.
. Use "-hier_path" when the trace file contains a hierarchy that is different from the one in the design loaded in Jasper. For example, if the design top module is "dut_top", which is instantiated as the top module in the testbench "testbench_top" as "i_dut_top", use the following command (replace "file_name" with an actual value):
  visualize -confirm -vcd <file_name> -hier_path testbench_top.i_dut_top
  visualize -confirm -fsdb <file_name> -hier_path testbench_top.i_dut_top
  visualize -confirm -shm <file_name> -hier_path testbench_top.i_dut_top
  visualize -confirm -vwdb <file_name> -hier_path testbench_top.i_dut_top
. Use "-start_time" to skip the initial chunk of simulation data. Typically, you will use this switch to skip the reset sequence and so forth.

```
. Use "-end_time" to ignore the last chunk of simulation data.
. Use "-hier_map" for trace files generated during simulation if you have a
  wrapper module.
  . Use "-from" to specify the source (simulation environment) hierarchy.
  . Use "-to" to specify the destination (formal environment) hierarchy.
. By default, the tool applies three-valued logic semantics (1, 0, X) when
  loading the trace. In some cases, Visualize supports plotting four-value
  logic semantics (1, 0, X, Z) for multiple-driven nets. To change to
  two-valued logic semantics (1, 0), use "-two_valued_simulation".
  For differential analysis properties (X-Prop, SPV, and path), four-valued
  logic semantics are used, and the tooltip displays the original and
  tainted values.
. By default, the tool uses register values from the trace file. Use the
  switch "-use_reset_analysis" if you want to use values from the
  Jasper reset analysis.
  NOTE: This switch is recommended only if the trace comes from Jasper
  and the current session uses the same configurations.
. By default, the tool applies your "visualize" command to the current
  window. Use "-window" to apply your command to a window other
  than the current window.
  . If you specify a name that does not exist yet, this switch renames the
    current window with the specified name.
  . If there are no Visualize windows, a new window opens with the
    specified name.
. Use "-new_window" to generate the waveform in a new window with a
  name specified by the optional window_name argument.
  NOTE:
  . If you do not provide the window_name argument, a tool-generated
    name is used.
  . The new window becomes the "current window."
. Use "-batch" to prevent the tool from displaying the
  trace it generates, which improves performance.

LIMITATIONS:
. Compound black-boxed output values will not be restored.
. The command does not work in designs with name collisions: declaring a
  variable with the same name in multiple unnamed processes or always
  blocks.
. The command might not work when the trace has a property sampled by gated
  clock.
. The command does not support properties with local variables. These
  properties are not saved and restored, as a result, the command reports
  inconsistencies.
```

```
-get_sim_time cycle [-window window_name]
    Return the simulation time related to a Visualize cycle.

    NOTE: The returned simulation time refers to the beginning of the cycle.

    Use this command after you have loaded an external trace using "visualize
    -load" or "visualize -confirm".

    By default, the tool applies your "visualize" command to the current
    window. Use "-window" to apply your command to a window other
    than the current window.

-reset
    Load the trace from reset as a Visualize trace.

    Important things to know:
    . Run this command after you use "analyze", "elaborate", and commands
      that set up your environment (for example, "clock", "reset", "assume",
      and so forth).
    . This command opens a new window with the reset trace. The window name is
      "Reset Analysis".

-show ( max_length | min_length | frozen_cycles | task | type
      | trace_length | quiet )
      [-window window_name]
    Show the Visualize options for the current or specified window.

    Use "-window" to apply your command to a window other than the current
    window.

-trace -list (length | engine) [-window window_name]
    List the trace settings used for the current or specified window.
    Use "-window" to apply your command to a window other than the current
    window.

-save_sig_order file_name [(-window window_name) |-reset_window]
                [-save_highlight]
    Save the current signal list to the specified file.

    NOTE: If the file name already exists, this command replaces the file's
    contents with the current signal list.
```

```
    . By default, the tool applies your "visualize" command to the current
      window.
      . Use "-window" to apply your command to a window other than the current
        window.
      . Use "-reset_window" to apply your command to the Reset Analysis window.
    . Use "-save_highlight" if you want to save the row highlight status into
      the file also.

-set_silent_mode (true | false)
    Turn off the output to the screen and return information
    with a Tcl return value.

    NOTE: This setting persists for "visualize" commands until you change it
    with another "-set_silent_mode" command. When silent mode is set, the tool
    suppresses all info messages and proof messages that would normally be
    generated by successive "visualize" commands, and the tool does not
    generate an error when Visualize fails to find a trace.

-get_silent_mode
    Report whether silent mode is enabled for "visualize" commands.

-set_annotate_clock_events (true | false) [-window window_name]
    Turn the clock events annotation on or off for the Visualize window
    "window_name".

    By default, the tool applies your "visualize" command to the current
    window. Use "-window" to apply your command to a window other
    than the current window.

-get_annotate_clock_events [-window window_name]
    Report whether the clock events annotations are enabled for the Visualize
    window "window_name".

    By default, the tool applies your "visualize" command to the current
    window. Use "-window" to apply your command to a window other
    than the current window.

-undo [-window window_name]
    Undo the previous configuration.

    By default, the tool applies your "visualize" command to the current
    window. Use "-window" to apply your command to a window other
    than the current window.
```

```
    NOTE:
    . Run the command multiple times to step back through configurations.
    . This command has no effect after using "-replot".

-redo [-window window_name]
    Redo the previous configuration.

    By default, the tool applies your "visualize" command to the current
    window. Use "-window" to apply your command to a window other
    than the current window.

    NOTE:
    . Run the command multiple times to reinstate multiple configurations
      you previously removed.
    . This command has no effect after using "-replot".

-highlight -diff target_window_name [-target_window_start_cycle N]
                                    [-source_window_start_cycle N]
                                    [-window source_window_name]
    Apply highlights to show the differences between the current
    Visualize window and the specified Visualize window.

    . Use "-target_window_start_cycle" to specify a start cycle for the
      analysis on the specified Visualize window.
    . Use "-source_window_start_cycle" to specify a start cycle for the
      analysis on the Visualize window specified by "-window" (or the current
      Visualize window, if you did not specify "-window").
    . Use "-window" to specify a window other than the current window.

    NOTE: The tool applies yellow highlights to signals that have different
    values. If the current trace has more cycles than the one you are comparing
    it to, the tool applies an orange highlight to the additional cycles.

-highlight -diff (-prev_plot |-next_plot) [-window window_name]
    Apply highlights to show the differences between two contiguous plots in
    the current Visualize stack.

    . Use "-prev_plot" to create highlights that compare the current window's
      trace and the previous trace in the stack.
    . Use "-next_plot" to create highlights that compare the current window's
      trace and the next trace in the stack.
```

```
    NOTE: The tool applies yellow highlights to signals that have different
    values. If the current trace has more cycles than the one you are comparing
    it to, the tool applies an orange highlight to the additional cycles.

-relevant_diff signal source_cycle target_cycle target_window_name
    [-configuration ([control_path | data_path] [undriven_only]
                     [filter_async_reset] [show_clock] [show_equal]
                     [instance_ports_only])+
    [-iteration_depth N |-num_whys N |-unlimited_depth]
    [-window source_window_name]
    Highlight Relevant Difference combines the Highlight Relevant Logic and
    Highlight Difference features to highlight signals that are both relevant
    in one trace or the other and different between the two traces.

    Use this command to highlight logic that is relevant and different between
    source and target windows.
    . Use "-configuration" with one or more of the following arguments to
      configure the relevant logic trace:
      . data_path: Highlight only the data path of the design.
      . control_path: Highlight only the control path of the design.
      . undriven_only: Highlight only undriven signals.
      . filter_async_reset: Do not highlight reset-related signals.
      . show_clock: Highlight clock-related signals.
      . show_equal: Highlight equivalent (buffer) signals.
      . instance_ports_only: Highlight only signals that are instance ports.
    . Use "-iteration_depth" to specify the trace length (number of
      iterations). The default is a 2-iteration trace.
      NOTE: Specifying a large number of iterations can degrade performance in
      terms of run time.
    . Use "-num_whys" to specify the number of Whys you want to call.
    . Use "-unlimited_depth" to specify unlimited iterations.
    . Use "-window" to specify a source window other than the current window.

-relevant_logic signal cycle
    [-configuration ([control_path | data_path] [undriven_only]
                     [filter_async_reset] [show_clock] [show_equal]
                     [show_twins_diff] [instance_ports_only])+
    [-iteration_depth N |-num_whys N |-unlimited_depth]
    [(-window window_name) |-reset_window]
    Highlight the transitive "Why" of the specified signal/iteration.

    Use this command to check the logic that generated a certain value in the
    trace.
```

```
    . Use "-configuration" with one or more of the following arguments to
      configure the relevant logic trace:
      . data_path: Highlight only the data path of the design.
      . control_path: Highlight only the control path of the design.
      . undriven_only: Highlight only undriven signals.
      . filter_async_reset: Do not highlight reset-related signals.
      . show_clock: Highlight clock-related signals.
      . show_equal: Highlight equivalent (buffer) signals.
      . show_twins_diff: Highlight SEC twin signals that have different values.
      . instance_ports_only: Highlight only signals that are instance ports.
    . Use "-iteration_depth" to specify the trace length (number of
      iterations). The default is a 2-iteration trace.
      NOTE: Specifying a large number of iterations can degrade performance in
      terms of run time.
    . Use "-num_whys" to specify the number of Whys you want to call.
      NOTE: This behavior is consistent with the pre-2014.03 behavior for
      "-depth". The "-depth" switch is being deprecated.
    . Use "-unlimited_depth" to specify unlimited iterations.
    . Use "-window" to specify a window other than the current window.
    . Use "-reset_window" to specify the Reset Analysis window.

-clear_all [-window window_name]
    Clear all configurations and all previously generated
    traces.

    By default, the tool applies your "visualize" command to the current
    window. Use "-window" to apply your command to a window other
    than the current window.

-sig_order sig_order_file_name [(-window window_name) |-reset_window]
    Launch a Visualize window containing the signals in the specified signal
    order file.

    By default, the tool applies your "visualize" command to the current
    window.
    . Use "-window" to apply your command to a window other than the current
      window.
    . Use "-reset_window" to apply your command to the Reset Analysis window.

-no_signals [-window window_name]
    Clear all signals from the current trace.

    By default, the tool applies your "visualize" command to the current
```

```
    window. Use "-window" to apply your command to a window other
    than the current window.
    . If you specify a name that does not exist yet, this switch renames the
      current window with the specified name.
    . If there are no Visualize windows, a new window opens with the
      specified name.

-new_window [new_window_name] [-copy [-window old_window_name]]
    Launch a new Visualize window.

    If you do not use additional switches, the new window is empty.
    . Use the optional argument to "-new_window" to specify a name for the
      new window.
      NOTE:
      . If you do not provide the new_window_name argument, a tool-generated
        name is used.
      . The new window becomes the "current window."
    . Use "-copy" to clone the current Visualize window. The default name is
      the name of the copied window with "_copy" appended, and the new window
      becomes the new current window.
      NOTE:
      . If a replot operation is running in the background for the current
        window, the new window will plot the last finished replot
        configuration.
    . Use "-window" to specify an original Visualize window other than the
      current one.

-rename_window old_name new_name
    Rename the specified Visualize window.

-set_current_window window_name
    Set the specified Visualize name (window) as the current Visualize name.

-get_current_window
    Report the current Visualize name (window).

-get_last_active_window
    Report the last activated Visualize name (window).

-remove_window (window_name |-reset_window)
    Remove the specified Visualize window or the Reset Analysis window.

-remove_current_window
```

```
    Remove the current Visualize name (window).

-get_type [-window window_name]
    Show the Visualize type (violation, cover, conflict, or
    dead-end) for the current or specified window.

    By default, the tool applies your "visualize" command to the current
    window. Use "-window" to apply your command to a window other than the
    current window.

-get_value signal_name [cycle | cycle:cycle]
    [-radix ( 2 | 8 | 10 | 16 | bin | oct | dec | signed_dec | hex | signed_hex
             | bits_set | floating_point | string)]
    [-window window_name]
    Get the value of the specified signal.

    . Specifying cycles is optional. If you do not specify a cycle or cycle
      range, the default is "1:$", which reports the value at every cycle of
      the trace.

      You can specify cycles in the following ways:
      . Use "$" to specify the last cycle of the trace in the Visualize window.
      . Use "$-N" to specify "N cycles before the last cycle".
        Example: 3 cycles before the last cycle is "$-3".
      . Use "N:$" to specify a range of cycles beginning with N and ending with
        the last cycle of the trace.
        Example: From cycle 4 to the last cycle is "4:$".
      . Use "N" to specify a single cycle.
      . Use "N:M" to specify a range of cycles.
        Example: From cycle 2 to cycle 3, inclusive is "2:3".

    . Use "-radix" to return values in the specified radix.

      NOTE:
      . Refer to the command reference manual appendix "Custom Radix Files" for
        additional information about radix handling (Help - Command Reference).
      . For "bits_set", the value and the information displayed in the
        Visualize waveforms pane and in reports are as follows:

        Value      Display or Report
        -----      -----------------
        4'b0010    1
        4'b1101    0, 2-3
```

```
        4'b0000    -

    . By default, the tool applies your "visualize" command to the current
      window. Use "-window" to apply your command to a window other
      than the current window.

-get_length [(-window window_name) |-reset_window]
    Get the length of the current trace.

    By default, the tool applies your "visualize" command to the current
    window.
    . Use "-window" to apply your command to a window other than the current
      window.
    . Use "-reset_window" to apply your command to the Reset Analysis window.


-get_stem_length [-window window_name]
    Get the length of the stem of the current trace. The stem is the
    part of the trace that is not part of the loop.

    By default, the tool applies your "visualize" command to the current
    window. Use "-window" to apply your command to a window other
    than the current window.

-get_loop_length [-window window_name]
    Get the length of the loop of the current trace. For looping
    traces this value is greater than 0, for other traces it is 0.

    By default, the tool applies your "visualize" command to the current
    window. Use "-window" to apply your command to a window other
    than the current window.

-get_looping_flops [-window window_name]
    Return a list of all the flops with looping behavior in the current
    infinite trace and return an empty list if there is no such trace.

    By default, the tool applies your "visualize" command to the current
    window. Use "-window" to apply your command to a window other
    than the current window.

-is_looping [-window window_name]
    Return "true" if the current trace has a loop. Only infinite traces
    can have a loop.
```

```
    By default, the tool applies your "visualize" command to the current
    window. Use "-window" to apply your command to a window other
    than the current window.

-get_trace_id [-window window_name]
    Get the trace ID of the current trace.

    NOTE:
    . The ID changes on every replot operation.
    . When using "-go_backward" or "-go_forward", the ID retrieved is
      consistent with what was calculated for the corresponding replot
      operation that initially generated the trace being displayed.

    By default, the tool applies your "visualize" command to the current
    window. Use "-window" to apply your command to a window other
    than the current window.

-load_radix file_name [-window window_name]
    Load and apply radixes defined in the specified "file_name".

    "file_name" format consists of radix definition clauses and radix use
    clauses. Refer to the command reference manual for radix file format
    guidance.

    By default, the tool applies your "visualize" command to the current
    window. Use "-window" to apply your command to a window other
    than the current window.

-set_radix (signal_name | expression)
           ( -bin |-oct |-dec |-signed_dec |-hex |-signed_hex |-bits_set
            |-floating_point |-string |-default |-user-defined radix_name)
    [-regexp |-occurrence N |-all]
                  [(-window window_name) |-reset_window]
    Change the radix on the specified signal or expression in the trace.
    . Use either a full signal name or use "-regexp" to apply the radix
      setting to multiple signals.
    . If the trace has duplicated signal instances, use "-occurrence" to
      specify which instance you want to change or use "-all" to make the
      change on all duplicated instances.
      NOTE: If you use neither switch, the tool applies the radix setting to
      the first instance of the duplicated signal.
```

```
    By default, the tool applies your "visualize" command to the current
    window.
    . Use "-window" to apply your command to a window other than the current
      window.
    . Use "-reset_window" to apply your command to the Reset Analysis window.

-marker marker_name -cycle (N | N:M)
    [-color color]
    [-row row_number]
    [(-window window_name) |-reset_window]
    Create a custom marker.

    . Use "-cycle" to specify the location of the marker. Use integer N to
      create a regular marker. Use a range N:M to create a range marker.
    . Use "-color" to specify the color of the marker. The color string can
      be a recognizable color name, such as "red" or "blue". Or you can use
      the color code, such as "#FF0000". The initial default color is blue.
      The tool chooses subsequent default colors by cycling through available
      colors.
    . Use "-row" to specify the row of the marker. Row 0 is the primary row of
      the marker area. You can place a marker in the extension rows 1-9 so
      they display clearly without overlapping.
    . By default, the tool applies your "visualize" command to the current
      window.
      . Use "-window" to apply your command to a window other than the current
        window.
      . Use "-reset_window" to apply your command to the Reset Analysis window.

-marker (-list |-show marker_name_list |-remove marker_name_list |-clear)
    [(-window window_name) |-reset_window]

    . Use "-list" to get the names of all the markers in the specified
      Visualize window.
    . Use "-show" to get a Tcl dictionary of detailed information about the
      specified markers.
    . Use "-remove" to remove the specified markers.
      NOTE: You cannot remove the "Current" marker because it is a system
      marker.
    . Use "-clear" to remove all custom markers.
      NOTE: You cannot remove the "Current" marker because it is a system
      marker.
    . By default, the tool applies your "visualize" command to the current
      window.
```

```
      . Use "-window" to apply your command to a window other than the current
        window.
      . Use "-reset_window" to apply your command to the Reset Analysis window.

-marker [marker_name] -callback_on_moved procedure_name
    [(-window window_name) |-reset_window]

    Add a callback procedure to the current or specified marker.

    . Use the "marker_name" argument to specify a marker other than the
      current marker.
    . Use "-callback_on_moved" to specify the name of the procedure to be called
      after a marker is moved.
    . By default, the tool applies your "visualize" command to the current
      window.
      . Use "-window" to apply your command to a window other than the current
        window.
      . Use "-reset_window" to apply your command to the Reset Analysis window.

-get_target [-window window_name]
    Report the target expression or property name for the specified Visualize window.

Signal List Manipulation
------------------------
Use the following switches to add and remove signals and groups, expand and
collapse groups, and manage the signals inside the groups.

-add_sig
     (signal_name | expression) [row]
    |-file file_name
    [(-window window_name) |-reset_window]
    Add the specified signal or expression to the trace at the specified row.
    The top of the waveform is row 0.

    Use "-file" to add the signals or expressions in the specified file to the
    trace.

    By default, the tool applies your "visualize" command to the current
    window.
    . Use "-window" to apply your command to a window other than the current
      window.
    . Use "-reset_window" to apply your command to the Reset Analysis window.
```

```
-remove_sig
     (signal_name | expression)
    [-regexp |-occurrence N |-all]
    [(-window window_name) |-reset_window]
    Remove the specified signal or expression from the trace.
    You can either use a full signal name or use a regular expression (with the
    "-regexp" switch) to remove multiple signals.
    . If the trace has duplicated signal instances, use "-occurrence" to
      specify which instance you want to remove.
    . Use "-all" to remove all duplicated signal instances.
    NOTE: If you provide neither "-occurrence" nor "-all", the tool removes the
    first instance of duplicated signals.

    By default, the tool applies your "visualize" command to the current
    window.
    . Use "-window" to apply your command to a window other than the current
      window.
    . Use "-reset_window" to apply your command to the Reset Analysis window.

-signal_group
    ( -expand (signal_group_name | array | struct | union)+ [-all]
     |-collapse (signal_group_name | array | struct | union)+ [-all]
     |-create signal_group_name [-sig (signal | signal_group_name)+ ]
     |-remove signal_group_name [-keep_signal]
     |-add_signal signal_group_name -sig (signal | signal_group_name)+
     |-remove_signal signal_group_name
        -sig (signal | signal_group_name)+ [-keep_signal]
     |-add_spacer signal_group_name
    )
    [(-window window_name) |-reset_window]
    Use following switches to manipulate signal groups or arrays.
    . Use "-expand" to expand the specified group, array, struct, or union
      items in the trace. Or use "-all" to expand all child groups recursively.
    . Use "-collapse" to collapse the specified group, array, struct, or union
      items in the trace. Or use "-all" to collapse all child groups
      recursively.
    . Use "-create" to create a new user-defined group in the trace. You can
      use "-sig" to add specified signals or groups into the new group.
    . Use "-remove" to remove a specified user-defined group from the trace.
      By default, the tool removes the group's contents, but you can use
      "-keep_signal" to preserve the contents.
    . Use "-add_signal" to add specified signals or groups into the specified
      user-defined group.
```

```
    . Use "-remove_signal" to remove signals or groups from the specified
      group.
      By default, the tool removes the specified items from the trace, but you
      can use "-keep_signal" to preserve the removed items.
    . Use "-add_spacer" to add a spacer row to the waveforms pane.
      If the insertion bar is on a group row, the tool adds the spacer at this
      position. If the insertion bar is not on a group row, the tool adds the
      spacer at the end of the specified group.

    By default, the tool applies your "visualize" command to the current
    window.
    . Use "-window" to apply your command to a window other than the current
      window.
    . Use "-reset_window" to apply your command to the Reset Analysis window.

-add_spacer
    [row]
    [(-window window_name) |-reset_window]
    Add a spacer row to the waveforms pane.

    . Use a row number to add the spacer at the specified row. The top of the
      waveform is row 0. If you do not specify a row, the tool adds the spacer
      at the insertion bar position.

    By default, the tool applies your "visualize" command to the current
    window.
    . Use "-window" to apply your command to a window other than the current
      window.
    . Use "-reset_window" to apply your command to the Reset Analysis window.

-plot_row -list
    ( [signal] [group] )+
    [-signal_group signal_group_name]
    [-include_id]
    [-target_pane (anchor | default | preview)]
    [-window window_name]
    Return a list of rows of signals and groups that are visible in the Visualize
    window.

    . Use "signal" to list signal rows or "group" to list group rows.
      If you want to plot signal and group rows at the same time, use "signal
      group".
    . Use "-signal_group" to list only immediate descendants rows of the specified
```

```
      signal group.
    . Use "-include_id" to also show the row_id for each of the listed signal or
      "group" rows.
    . Use "-target_pane" to list a signal or group in the specified area of the
      waveform pane. The target can be "anchor", "default", or "preview". By
      default, "-target_pane" applies to the main waveform pane.
    . Use "-window" to apply your command to a window other than the current
      window.
    NOTE:
    . This command returns a list of all signals and all groups at all
      levels.
    . If you use "-list {signal group}", the return is a list of names. Thus,
      it is not possible to differentiate signals from groups. To
      differentiate, use "-list signal" or "-list group".

-tracking_signal
    ( -add tracking_signal [row_id] -child_signal signal_list
    | -remove tracking_signal [row_id] -child_signal signal_list )
    [( -window window_name ) | -reset_window ]

    Manipulate the specified tracking signals or arrays.
    . Use "-add" to transform a signal into a tracking signal or specify an
      already existing tracking signal (for example, when adding a child to a
      tracking signal).
      . Specify a row_id to differentiate between signals with the same name.
        Run "visualize -plot_row -list {signal group} -include_id -target_pane
        default" to find the row_id.
      . Use "-child_signal" to specify one or more signals you want to add
        to the tracking signal.
    . Use "-remove" to specify a tracking signal you want to remove signals
      from.
      . Specify a row_id to differentiate between signals with the same name.
        Run "visualize -plot_row -list {signal group} -include_id -target_pane
        default" to find the row_id.
      . Use "-child_signal" to specify one or more signals you want to remove
        from the tracking signal.
    . By default, the tool applies your "visualize" command to the current
      window.
      . Use "-window" to apply your command to a window other than the current
        window.
      . Use "-reset_window" to apply your command to the Reset Analysis window.

Color and Comment Annotation Switches
```

```
-------------------------------------
Use the following switches and arguments to define, activate, and
name a color or comment annotation for the Visualize window.
. Each color or comment annotation consists of a set of one or more triplets.
. Each triplet consists of a signal_name-time_step-color.

By default, the tool applies your "visualize" command to the current window.
Use "-window" to apply your command to a window other than the current window.

Refer to the example section below.

-define_color_annotation { {signal_name time_step color}+ } annotation_name
    [-replace |-append] [(-window window_name) |-reset_window]
    Define the color annotation for the specified "signal_name" at the
    specified "time_step" point.
    NOTE: The tool accepts a list of lists for {signal_name time_step color}.

    The tool lists annotation names in the Trace Exploration combo
    box located on the Visualize window toolbar.

    . signal_name time_step color: Specify this first-argument triplet as
      follows:
      . signal_name: The name of the signal to which the color annotation
        applies.
      . time_step: The time point when the color annotation applies.
      . color: The annotation color. Possible values are purple, magenta, navy,
        blue, cyan, green, olive, yellow, orange, brown, red, grey, and dark
        grey.

      To delete an annotation, specify an empty argument by including
      the {} with nothing enclosed.

    . (Optional) annotation_name: The name of the color annotation. You cannot
      modify predefined annotations, such as those from ATD and Visualize. If
      you do not specify a name, the default name is "User Defined."

    . By default, the tool replaces an existing annotation definition with the
      definition given by this command. But you can use the "-append" switch to
      append the annotation to an existing annotation instead. The "-append"
      switch also allows you to build an annotation definition incrementally.

    . By default, the tool applies your "visualize" command to the current
      window.
```

. Use "-window" to apply your command to a window other than the current window.
. Use "-reset_window" to apply your command to the Reset Analysis window.

The tool returns the name of the annotation to show you have successfully specified it.

This command is helpful for highlighting the path of an X-assignment to the signal to which it propagates.

-comment {signal_name time_step color [row_id]} annotation_name -text comment [-replace |-append |-remove |-position (top_left | top_right | bottom_left | bottom_right)] [(-window window_name) |-reset_window]
Add a comment to the specified "signal_name" at the specified "time_step" point.

The tool lists annotation names in the Trace Exploration combo box located on the Visualize window toolbar.

. signal_name time_step color: Specify this first-argument triplet as follows:
  . signal_name: The name of the signal to which the comment annotation applies.
  . time_step: The time point when the comment annotation applies.
  . color: The annotation color. Possible values are purple, magenta, navy, blue, cyan, green, olive, yellow, pale yellow, bright yellow, orange, brown, red, grey, and dark grey.
  . row_id: The row ID is an optional argument. This argument is used by tool-generated commands. The tool uses the ID to ensure the comment is assigned to the right signal row when you have duplicated signals in the waveform signal list. The tool always generates the ID and uses it automatically; however, you do not need to use row ID to add a comment. NOTE: The row ID is only valid in the same Visualize session. It cannot be applied to different Visualize sessions.

. annotation_name: The name of the comment annotation. You cannot add a comment to predefined annotations, such as those from ATD and Visualize.

. Use "-text" to specify the comment text you want to add to the annotation. It can be multiple lines.

. By default, the tool appends the new comment to the specified annotation.

```
      You can use the "-replace" switch to replace all existing comments in the
      annotation with the new one. You can use the "-append" switch to
      build an annotation incrementally.

    . Use "-remove" to remove the specified comment from a specified point. The
      comment and color must be identical to an existing comment.

    . Use "-position" to change the position of the comment. By default, the
      position is "auto" (the tool positions the comment). You can use
      "top_left", "top_right", "bottom_left", or "bottom_right" to place the
      comment at the specific location (relative to the specified point).

    . By default, the tool applies your "visualize" command to the current
      window.
      . Use "-window" to apply your command to a window other than the current
        window.
      . Use "-reset_window" to apply your command to the Reset Analysis window.

    This command is helpful for explaining or documenting the trace so multiple
    users can work on the trace together.

-comment -arrow {from_signal_name time_step color [row_id]}
    {to_signal_name time_step color [row_id]} annotation_name
    [-text comment]
    [-replace |-append |-remove |-position (top | bottom)]
    [(-window window_name) |-reset_window]
    Add an arrow between two specified signal-cycle points and include a
    comment.

    The tool lists annotation names in the Trace Exploration combo
    box located on the Visualize window toolbar.

    . from_signal_name time_step color: Specify this first-argument triplet as
      the start point of the arrow.
    . to_signal_name time_step color: Specify this second-argument triplet as
      the end point of the arrow.
      . signal_name: The name of the signal to which the arrow comment applies.
      . time_step: The time point when the arrow comment applies.
      . color: The annotation color. Possible values are purple, magenta, navy,
        blue, cyan, green, olive, yellow, pale yellow, bright yellow, orange,
        brown, red, grey, and dark grey.
      . row_id: The row ID is an optional argument. This argument is used by
        tool-generated commands. The tool uses the ID to ensure the comment is
```

assigned to the right signal row when you have duplicated signals in the waveform signal list. The tool always generates the ID and uses it automatically; however, you do not need to use row ID to add a comment. NOTE: The row ID is only valid in the same Visualize session. It cannot be applied to different Visualize sessions.

. annotation_name: The name of the comment annotation. You cannot add a comment to predefined annotations, such as those from ATD and Visualize.

. Use "-text" to specify the comment text you want to add to the arrow. It can be multiple lines. If you do not provide the switch, the tool creates an empty comment (arrow only).

. By default, the tool appends the new comment to the specified annotation. You can use the "-replace" switch to replace all existing comments in the annotation with the new one. You can use the "-append" switch to build an annotation incrementally.

. Use "-remove" to remove the specified comment from specified points. The comment text and color must be identical to an existing comment.

. Use "-position" to change the position of the comment. By default, the position is "auto" (the tool positions the comment). You can use "top" and "bottom" to specify an upward curve or downward curve in the lines between specified points.

. By default, the tool applies your "visualize" command to the current window.
  . Use "-window" to apply your command to a window other than the current window.
  . Use "-reset_window" to apply your command to the Reset Analysis window.

This command is helpful for explaining or documenting the trace so multiple users can work on the trace together.

-comment (-collapse |-expand) [-window window_name]
Collapse or expand the comment text in waveform annotations.
You can subsequently view annotations text in the Visualize window by using the mouse to hover over the collapsed text box and reveal a tooltip.

By default, the tool applies your "visualize" command to the current window. Use "-window" to apply your command to a window other than the

```
    current window.

-activate_color_annotation annotation_name+
    [(-window window_name) |-reset_window]
    Apply the specified color "annotation_name".

    . annotation_name: Any of the annotations in the Trace Explanation combo box
      (both user-defined and predefined). Use the default annotation named "Off"
      to clear the current annotation from the trace.
    . By default, the tool applies your "visualize" command to the current
      window.
      . Use "-window" to apply your command to a window other than the current
        window.
      . Use "-reset_window" to apply your command to the Reset Analysis window.

-define_signal_highlight { {signal_or_property_name color}+ }
    [(-window window_name) |-reset_window] [-add |-remove]
    Define the row highlighting for the specified signal or property name
    using the specified color.
    . signal_or_property_name color: Specify this first-argument pair as
      follows:
      . signal_or_property_name: The name of the signal or property to which
        the highlight applies.
      . color: The highlight color. Possible values are blue, green,
        yellow, orange, and red.
    . By default, the tool applies your "visualize" command to the current
      window.
      . Use "-window" to apply your command to a window other than the current
        window.
      . Use "-reset_window" to apply your command to the Reset Analysis window.
    . Use "-add" to add highlight signals or properties incrementally.
    . Use "-remove" to remove highlights from specified signals or properties.
    . If you do not specify either "-add" or "-remove", the specified highlight
      list replaces the current highlight list.
    . To delete all signal highlights, specify an empty argument by including
      the {} with nothing enclosed.

Interactive Constraint Extraction
---------------------------------
-extract_conf [-reason [-depth depth_value] [-transitive]] [-value]
              {{signal [cycle | cycle:cycle]}+}
              [cycle | cycle:cycle] [-name conf_name]
              [-window window_name]
```

```
    Find a new trace that shows one or more of the specified signals
    exhibiting:

    . A value that was seen on one of the cycles specified for that signal,
      but with different values on the immediate fanin (-reason). This is
      the default switch.

    . A value that is different from those seen on any of the cycles
      specified for that signal (-value).

    . Either different values on the immediate fanin or a
      different value on the signal itself (-reason -value).

    . Use "-depth" with "-reason" to tell how many levels of registers
      to look through for the reason for a signal."depth_value" must be
      a positive integer.

    . Use "-transitive" with "-reason" if one of the signals returned
      as part of the reason is an internal wire. (The tool will then
      recursively do "Why" on that signal until all the signals reported
      as part of the reason are either inputs or registers.)

    . Specifying cycles is optional. If you do not specify cycles,
      that is equivalent to "always".
      . If you specify a single cycle (for example, 7), you are
        asking the tool to ensure the expression generated by
        "-extract_conf" is true in cycle seven.
      . You can specify a range of cycles; for example, 2:3.
      . Use $ to mean "the last cycle". For example, 1:$ is
        equivalent to "always" and 4:$ is equivalent to "from
        cycle four until the end of the trace".

    . Use "-name" to specify a conf_name for this configuration.

    . By default, the tool applies your "visualize" command to the current
      window. Use "-window" to apply your command to a window other
      than the current window.

    Implementation guidance: Use curly braces around each signal-cycle set.

    Refer to the example section below.

-concretize_conf {conf_name} [-window window_name]
```

```
    Make the specified, extracted configuration concrete.

    Use this command to replace the specified extracted configuration
    with a configuration that captures how the extracted configuration
    is satisfied in the current trace.

    By default, the tool applies your "visualize" command to the current
    window. Use "-window" to apply your command to a window other
    than the current window. If you specify a name that does not exist yet,
    this switch renames the current window with the specified name.

Waveform Analysis
-----------------
-check_props [-cover |-violation |-assumption] [[-bg] -update_gui] [-names_only]
             [-x_only] [-filter property_name_filter_list [-regexp]]
             [-stop_on { [cover_reached] [assertion_violated] [assumption_violated] }
                       [-start_cycle N] [-first_only]]
             [-silent] [-window window_name]
    Analyze exercised covers and violated assertions in the current waveform.

    . Use "-cover", "-violation", or "-assumption" to check only for covers,
      assertion violations, or assumption violations.
    . Use "-bg" to run the command in the background. All other commands will
      be available except for "visualize -replot". Currently, multiple
      simultaneous jobs are not allowed. This switch returns the ID of the new
      background job. This switch requires the "-update_gui" switch.
    . Use "-update_gui" to refresh the Visualize window with the current
      analysis results.
    . Use "-names_only" to only show the names of properties. By default, the
      tool also shows the properties' type and the cycles in which they are
      covered or violated.
    . Use "-x_only" to only return the cycles in which the trace is X for the
      given properties. By default, the tool does not classify X as a violation
      or cover.
    . Use "-filter" to specify one or more properties to be checked. And use
      "-regexp" to enable regular expressions.
      NOTE: This switch supports wildcards and Tcl lists.
    . Use "-stop_on" to stop the analysis once a specific event happens. The
      event can be a cover being reached, an assertion being violated, or an
      assumption being violated.
    . Use "-start_cycle" to specify the cycle at which Visualize starts
      watching for events to stop the analysis.
    . Use "-first_only" to stop the analysis at most once for each property.
```

```
    . Use "-silent" to turn off output to the screen.
    . By default, the tool applies your "visualize" command to the current
      window. Use "-window" to apply your command to a window other than the
      current window.

    NOTE:
    . This operation is typically conducted with the Indexed Behaviors
      table "Analyze" button.
    . The FSV App does not support "visualize -check_props".
    . DA traces do not support "visualize -check_props".

    3-Value Simulation for "check_props" on Incidental Traces
    ---------------------------------------------------------
    Visualize clearly differentiates between signals in the COI and out of
    the COI when running "check_props" on incidental traces (that is, traces
    that came from the formal proof of other properties in the task). As a
    result, you can expect to see the following:

    . "visualize -check_props" shows a 3-value plot for simulation when based
      on incidental traces.

      . Signals that are out of the COI of the incidental trace used in the
        simulation appear as X in the waveform.

      . When running "check_props" on incidental traces, signals that are out
        of the COI clearly show that the signal does not have a defined value.
        And a property might have an undefined state (shown as X in the
        waveform) due to undefined values in its COI.

    . When running "check_props" on incidental traces, because of the 3-value
      simulation, the tool might issue the warning WVS031 to alert you to
      undefined values at some cycles.

-check_props (-wait |-stop |-status)
    Manage "visualize -check_props" commands running in the background.

    . Use "-wait" to block the Tcl console until the running job stops.
    . Use "-stop" to cancel and stop any running job.
    . Use "-status" to get the status of "-check_props" jobs. The result is a
      Tcl dictionary in which the key is the job ID, and the value is the job
      status. The possible statuses are "NeverStarted", "Running", and
      "Completed".
```

```
-compare signalA_name signalB_name [-shift_cycles cycles]
    [-ignore_x] [-ignore_values value+]
    [-window window_name]
    Compare the value of two signals cycle-by-cycle.

    This command generates a comparison results row in the Visualize window to
    show cycles that match and cycles that do not match.

    . Use "-shift_cycles" to offset the cycles on compared signals.
    . Use "ignore_x" if you do not care about x value. If you do not use this
      switch, any x value shows as "mismatch" in the comparison results.
    . Use "ignore_values" and specify a list of values that should not be
      considered mismatches. The values in the list produce a "match" result
      when doing the value comparison.
    . By default, the tool applies your "visualize" command to the current
      window. Use "-window" to apply your command to a window other
      than the current window.

Directed Test
-------------
-directed_test -signal signal_name -value value_list -range range_list
               [-name test_name] [-window window_name]
    Add a new test point to Visualize.

    You can check this test point against the trace with "visualize
    -directed_test -check". This command returns the name of the new test
    point.

    . Use "-signal" to specify the signal to check.
    . Use "-value" to specify a list with the expected values.
    . Use "-range" to specify a list with the cycle ranges in which the signal
      should be tested. A cycle range takes the form beginCycle:endCycle.
    . Use "-name" to specify the name of the test point. If you do not specify
      a name, the tool-generated name is "testpoint:<N>".
    . Use "-silent" to turn off the output to the screen.
    . By default, the tool applies your "visualize" command to the current
      window. Use "-window" to apply your command to a window other
      than the current window.
      . If you specify a name that does not exist yet, this switch renames the
        current window with the specified name.
      . If there are no Visualize windows, a new window opens with the
        specified name.
```

```
-directed_test -append test_point_name -value value_list -range range_list
     [-window window_name]
    Append a list of values and ranges to a test point.

    . Use "-value" to specify the new expected value list.
    . Use "-range" to specify a list with the cycle ranges in which the signal
      should be tested. A cycle range takes the form beginCycle:endCycle.
    . By default, the tool applies your "visualize" command to the current
      window. Use "-window" to apply your command to a window other
      than the current window.

-directed_test -remove test_point_list [-window window_name]
    Remove a list of test points from Visualize.

    By default, the tool applies your "visualize" command to the current
    window. Use "-window" to apply your command to a window other than the
    current window.
    . If you specify a name that does not exist yet, this switch renames the
      current window with the specified name.
    . If there are no Visualize windows, a new window opens with the specified
      name.

-directed_test -rename old_name new_name [-window window_name]
    Rename a test point.

    By default, the tool applies your "visualize" command to the current
    window. Use "-window" to apply your command to a window other than the
    current window.
    . If you specify a name that does not exist yet, this switch renames the
      current window with the specified name.
    . If there are no Visualize windows, a new window opens with the specified
      name.

-directed_test -replace test_point
           [-signal signal_name]
           [-value value_list
            -range range_list]
          [-window window_name]
    Replace one or more attributes of a test point.

    NOTE: You must specify at least one attribute.

    . Use "-signal" to specify a new signal to check.
```

```
    . Use "-value" to specify the new expected value list.
    . Use "-range" to specify a list with the cycle ranges in which the signal
      should be tested. A cycle range takes the form beginCycle:endCycle.
    . By default, the tool applies your "visualize" command to the current
      window. Use "-window" to apply your command to a window other
      than the current window.
      . If you specify a name that does not exist yet, this switch renames the
        current window with the specified name.
      . If there are no Visualize windows, a new window opens with the
        specified name.

-directed_test -list
     [-filter (name | signal | value | range | status) filter_string [-regexp]]
     [-window window_name]
    List the configured test points.

    . Use "-filter" to refine the results by searching for a specific attribute
      value. This switch supports wildcards.
    . Use "-regexp" to match the attributes using regular expression semantics.
    . By default, the tool applies your "visualize" command to the current
      window. Use "-window" to apply your command to a window other
      than the current window.
      . If you specify a name that does not exist yet, this switch renames the
        current window with the specified name.
      . If there are no Visualize windows, a new window opens with the
        specified name.

-directed_test -show test_point (signal | value | range | status)
     [-window window_name]
    Show one of the attributes of a test point.

    By default, the tool applies your "visualize" command to the current
    window. Use "-window" to apply your command to a window other than the
    current window.
    . If you specify a name that does not exist yet, this switch renames the
      current window with the specified name.
    . If there are no Visualize windows, a new window opens with the specified
      name.

-directed_test -report [-name test_point_list] [-file file_name [-force]]
     [-silent] [-window window_name]
    Print a report of a list of test points, including all attributes.
```

```
    . Use "-name" to specify a test point list to report. Without it, the tool
      reports all test points.
    . Use "-file" to save the report to a file.
    . Use "-force" to overwrite any existing file.
    . Use "-silent" to turn off the output to the screen and return information
      with a Tcl list of each test point.
    . By default, the tool applies your "visualize" command to the current
      window. Use "-window" to apply your command to a window other
      than the current window.
      . If you specify a name that does not exist yet, this switch renames the
        current window with the specified name.
      . If there are no Visualize windows, a new window opens with the
        specified name.

-directed_test -check [test_point_list] [-silent] [-stop_on_failure]
     [-window window_name]
    Check a list of test points against the trace.

    . Use "-silent" to turn off the output to the screen and return the status
      of the check.
    . Use "-stop_on_failure" if you want a test point failure to stop the check
      and trigger an error.

    . By default, the tool applies your "visualize" command to the current
      window. Use "-window" to apply your command to a window other than the
      current window.
      . If you specify a name that does not exist yet, this switch renames the
        current window with the specified name.
      . If there are no Visualize windows, a new window opens with the specified
        name.

    This command prints a report of the results and returns the aggregate test
    status. Results include the following:
    . Result "passed" means all tests passed.
    . Result "failed" means at least one test failed.
    . Result "n/a" means that at least one test could not be checked because
      the trace lacks information for the tested signal.
    . Result "no_testpoints" means that no test points have been configured.

    NOTE:
    . If you do not specify a test point list, the tool checks all test
      points.
    . If you save a configuration with visualize -save filename.tcl or the GUI
```

```
      "File" menu option "Save Visualize Configuration," the tool adds
      "visualize -directed_test -check" to the saved recipe.

QuietTrace
----------
-quiet ((true [cycle:cycle]) | false | -signal signal_list)
       [-attempt_time_limit time_limit] [-window window_name]
    Generate quieted waveforms to decrease the unnecessary distraction that
    commonly comes from looking at waveforms with many signals toggling at the
    boundary and in the internal logic that are of no consequence to the
    behavior being illustrated.

    The QuietTrace goal is to provide a faster and better understanding of the
    waveforms. The Visualize Configuration pane in the Visualize window
    includes a tree node for "Quiet" data. When you enable the feature, it
    displays figures that show, out of the total number of soft constraints or
    preferences that were possible, how many were satisfied and how many were
    not satisfied in the final waveform.

    . Use "true" to turn QuietTrace on, and if you want to apply QuietTrace to
      a subset of the trace, specify a cycle range. Refer to the examples
      below.
    . Use "false" to turn off QuietTrace.
    . Use "-signal" to attempt to quiet one or more signals by adding soft
      constraints.
    . Use "-attempt_time_limit" to set a time limit for each QuietTrace engine
      iteration. The default value is 0s (unlimited). For more
      information on this setting, check the "set_engineQT_attempt_time_limit"
      command.
    . By default, the tool applies your "visualize" command to the current
      window. Use "-window" to apply your command to a window other
      than the current window.
      . If you specify a name that does not exist yet, this switch renames the
        current window with the specified name.
      . If there are no Visualize windows, a new window opens with the
        specified name.

    NOTE: If you specify multiple commands with different configurations, the
    tool only takes the last one into consideration; that is, the final
    configuration overrides any previous ones.

-get_quiet [-attempt_time_limit] [-window window_name]
    Returns whether QuietTrace mode is enabled in the specified window.
```

```
    Use "-attempt_time_limit" to return the current setting for the engine QT
    trace attempt time limit. For more information, see the
    "get_engineQT_attempt_time_limit" command.

    Use "-window" to specify a window. If you do not specify a window, the
    command returns the QuietTrace mode of the current window.

Configuration Grouping
----------------------
-config_group
     {conf_name+} [-name config_group_name] [-window window_name]
     |-expand config_group_name
     |-remove config_group_name
     |-rename old_config_group_name new_config_group_name
     |-replace {conf_name+} -name config_group_name
     |-add_conf {conf_name+} config_group_name
     |-remove_conf {conf_name+}
     |-merge config_group_name_1 config_group_name_2
      -name final_config_group_name
      [-window window_name]

    Add or edit configuration groups.
    . Use "-name" to place one or more configurations in a "group" with the
      specified group_name. Use curly braces around the set of configuration
      names.
    . Use "-expand" to ungroup the specified group_name.
    . Use "-remove" to remove the specified group_name and all its contents.
    . Use "-rename" to rename the specified group.
    . Use "-replace" with "-name" to edit the specified group by replacing its
      current configurations with one or more specified configurations. Use
      curly braces around the set of configuration names.
    . Use "-add_conf" to edit the specified group_name by adding the specified
      configuration to the current configurations. Use curly braces around the
      set of configuration names.
    . Use "-remove_conf" to edit a group by removing the specified
      configuration from the group. Use curly braces around the set of
      configuration names.
    . Use "-merge" to merge the configurations from group_name_1 and
      group_name_2 into a new group. Use "-name" to specify a preferred name.

    By default, the tool applies your "visualize" command to the current
    window. Use "-window" to apply your command to a window other
```

```
    than the current window.

    Attempting to specify a window name that does not exist triggers an error.

Annotating Targets and Configurations
-------------------------------------
-set_annotation (-target |-conf conf_name)
        {annotation} [-window window_name]
    Annotate the specified Visualize target or configuration or modify
    the existing annotation with the specified description that is
    enclosed by curly braces.

    To remove an annotation, use empty curly braces.

    By default, the tool applies your "visualize" command to the current
    window. Use "-window" to apply your command to a window other
    than the current window. If you specify a name that does not exist yet,
    this switch renames the current window with the specified name.

-get_annotation (-target |-conf conf_name) [-window window_name]
    Print the annotation for the specified target or configuration to the
    screen.

    By default, the tool applies your "visualize" command to the current
    window. Use "-window" to apply your command to a window other
    than the current window.

Specifying a Waveform Sequence in a Single Command
--------------------------------------------------
-signal signal_name -sequence {value[{iterations}][ value[{iterations}]]...}
 [iteration]
 [-name name] [-config_group_name config_group_name] [-window window_name]
    Display a waveform sequence for the specified signal that shows the
    specified values.

    Important things to know about this command:
    . Use one blank space as the delimiter between values.
    . Use "x" or "X" to show a don't care value that does not constrain
      the signal.
    . Use "{iterations}" after a value, with no space between the value and
      curly braces, to specify the number of times the value is repeated.
    . Use "cycle" to specify the cycle in which the first specified
      "value" applies. If you do not specify "cycle", the first "value"
```

```
      applies in cycle 1.
    . Use "-name" to specify a "name" for this configuration.
    . Use "-config_group_name" to specify a "config_group_name" for this
      configuration.
    . By default, the tool applies your "visualize" command to the current
      window. Use "-window" to apply your command to a window other
      than the current window. If you specify a name that does not exist yet,
      this switch renames the current window with the specified name.

    Refer to the example section below.

Visualize Task Manipulation
---------------------------
-set_task task_name [-window window_name]
    Set the specified task as the current Visualize task.

    Important things to know about this command:
    . If you have one or more current Visualize windows when you run this
      command, the target of the current window must use only properties that
      are present in the new task.
    . By default, the tool applies your "visualize" command to the current
      window. Use "-window" to apply your command to a window other
      than the current window.

-get_task [-window window_name]
    Report the name of the current Visualize task.

    By default, the tool applies your "visualize" command to the current
    window. Use "-window" to apply your command to a window other
    than the current window.

Switches for Background  Visualize
----------------------------------
-stop [-thread thread_id]
    Stop the "visualize" command running in the background or a single thread
    of parallel jobs.

    NOTE:
    . The command returns the string "background (thread <thread_id>)".
    . Use "visualize -stop" to stop the currently running background Visualize.
    . Use "visualize -thread thread_id" to stop a specified Visualize thread.

-wait
```

```
    Wait for the running "visualize -bg" command.

    This option is useful when "visualize" is running in the background (-bg).
    In effect, it moves Visualize back to the foreground, and the tool
    accepts no additional commands until Visualize completes.

-status [-silent]
    List the running engine jobs and their statuses.

    Use "-silent" to turn off the output to the screen and return information
    with a Tcl return value, which is helpful in Tcl scripts.

    The following list shows visualize statuses.
    . "NeverStarted": Nothing has run in this session.
    . "Pending": There is currently a Visualize about to start.
    . "Simplify": A Visualize is currently running proof simplification.
    . "Running engineJob*": A Visualize is running with the listed number of
      engineJobs currently running.
    . "Processing results": The proof has been completed and Visualize is
      processing the proof results.
    . "Completed": No job is currently running. The last Visualize has
      completed.

Switches for Task Context Visualize
-----------------------------------
-assume ( [-bound N] expression |-constant signal_name )
            [-name name] [-annotation {annotation}] [-replace]
            [-window window_name]
    Create a task context assumption for the specified Boolean expression or
    signal.
    . Use "-bound" with an expression to apply the specified assumption for N
      number of cycles after reset. By default an assumption is applicable
      for all cycles.
    . Use "-constant" and a signal name to assume that the signal is pseudo
      constant; that is, the signal has a fixed but unknown value throughout
      the analysis.
    . Use "-name" to label the newly created assumption so it can be reviewed
      or referenced in the future.
    . Use "-annotation" to annotate the specified property with the
      description enclosed by the curly braces.
    . Use "-replace" to replace the expression for the specified assumption or
      to change the task context constraint type.
      . Use "-name" to specify the assumption you are editing.
```

```
      . Use "-annotation" to add or change the assumption's annotation.
    . By default, the tool applies your "visualize" command to the current
      window. Use "-window" to apply your command to a window other
      than the current window.

-stopat signal_or_instance_name+ [-name name] [-annotation {annotation}]
                                 [-replace] [-window window_name]
    Stop traversing a netlist at the specified signals or instances.
    This command returns the label of the stopat it added.

    . Use "-name" to label the newly created stopat so it can be reviewed or
      referenced in the future.
    . Use "-annotation" to annotate the specified stopat with the
      description enclosed by the curly braces.
    . Use "-replace" to replace the signal or instance name or the task context
      constraint type for the specified stopat; for example:
      visualize -stopat {out0} -name justify0 -replace
      . Use "-name" to specify the stopat you are editing.
      . Use "-annotation" to add or change the stopat's annotation.
    . By default, the tool applies your "visualize" command to the current
      window. Use "-window" to apply your command to a window other
      than the current window.

-set_as_AR [-window window_name]
    Modify the AR to reflect the changes made with task context constraints.

    By default, the tool applies your "visualize" command to the current
    window. Use "-window" to apply your command to a window other
    than the current window.

Trace Log Switches
------------------
The following list identifies limitations in the trace log feature.

  . Unsupported format specifiers
    . %u: Unformatted 2-value data
    . %z: Unformatted 4-value data
    . %e: Display "real" in exponential
    . %f: Display "real" in decimal
    . %g: Display "real" in exponential or decimal, whichever has shortened
          printed output
    . Display library binding information (Does not apply to Jasper Apps)
```

```
  . Unsupported: $monitor, $fatal, $error, $warning, and $info.

  . Not possible to save and re-load display definitions

  . Unsupported regex/wildcards: Currently, it is not possible to define
    display expressions with wildcards.

  . By default, the $displays are not synthesized by the tool. Specify
    "set_dollar_display_synthesis on" before running "analyze" and
    "elaborate" to enable this synthesis.

  . A log_group named "rtl_display" stores the $displays.
    It is not possible to remove the group.

-trace_log -add (-display_expr display_expr
    [-log_group log_group_name]
    [-module module_name |-instance instance_list])
    [-condition (-expression expr |-cycle_factor N |-value_change sig_list)+]
    [-set_verbosity N]
    Add the specified display expression to the trace log database.
    The display expression format is a list. The first element is a format
    message (for example, "My name is %d"), and the following elements are the
    arguments of the message. This command returns the name of the added
    display_expr.
    . Use "-log_group" to define a custom group for the display expression. By
      default, the group is "default_group".
    . Use "-module" to add the display_expr to all instances of a specified
      module.
    . Use "-instance" to add the display_expr to the listed instances only.
    . Use "-condition" to define a triggering condition that will display the
      message:
      . Use "-expression" to trigger the message when the defined expr is true
        in the trace.
      . Use "-cycle_factor" to trigger the message on every N cycles.
      . Use "-value_change" to trigger the message when any signal in the list
        changes value.
        NOTE: If one of the conditions is met, the message prints.
    . Use "-set_verbosity" to define the verbosity of a display_expr.
      By default, the verbosity is 1, which means that the message will always
      be printed by the "-run" command.

-trace_log -run [log_group_list]
    [-verbosity N]
```

```
    [(-window window_name) |-reset_window]
    [-file file_name [-force]]
    [-include sub]
    [-filter -coi]
    Run the trace log analysis.
    NOTE:
    . If you do not define the log_group_list, the analysis runs for all
      trace logs.

    . By default, the tool applies your "visualize" command to the current
      window.
      . Use "-window" to apply your command to a window other than the current
        window.
      . If you have already launched a Reset Analysis window, use
        "-reset_window" to apply your command to the Reset Analysis window.

    . Use "-verbosity" to define the level of verbosity for the analysis. This
      level will match with the "-set_verbosity" you defined with the "-add"
      command.
      NOTE: The tool prints messages at the specified verbosity level and lower.
    . Use "-file" to redirect the output of the analysis to a file and use
      "-force" to overwrite an already existing file.

    . Use "-include sub" to report the sub type of each message on the output.
    . Use "-filter -coi" to print only signals that are in the target's cone of
      influence.

-trace_log -list (log_group | display_expr) [-silent]
    Return the already defined trace_log_names.
    Use "-silent" to prevent messages from printing in the console.

-trace_log -show (-log_group log_group_name |-display_expr display_expr)
    [-silent]
    Return detailed information about trace_log groups or display expressions.
    Use "-silent" to prevent messages from printing in the console.

-trace_log (-enable |-disable |-remove)
    (-log_group log_group_list |-display_expr display_expr_list)
    Enable, disable, or remove the specified log group or display expression.

-trace_log -clear
    Clear the database.
```

```
Why
---
-why signal_name cycle+
     [-transitive]
     [-ignore_connected_assumptions]
     [-simple_buffer_transitive]
     [-stop_at_hierarchy_change]
     [-include_non_ticking_gated_clocks]
     [-gui]
     [-silent]
     [-window window_name) |-reset_window]
    Find signals in the immediate fanin of the specified signal that affect its
    value at the specified cycle.

    By default, the tool applies your "visualize" command to the current
    window.
    . Use "-window" to specify a window other than the current window.
    . If you have already launched a Reset Analysis window, use "-reset_window"
      to apply your command to the Reset Analysis window.
    . Use "-transitive" to run transitive Why analysis and find the root
      cause of the signal value.
    . Use "-ignore_connected_assumptions" to prevent the search for assumptions
      that might force values of primary input targets (Why on boundary).
    . Use "-simple_buffer_transitive" to enable transitivity through "simple"
      buffers (inverters and netmixes are NOT considered "simple" buffers).
    . Use "-stop_at_hierarchy_change" to disable "default" transitivity
      through the "first" hierarchy change (instance port crossing).
    . Use "-include_non_ticking_gated_clocks" to include non-ticking gated
      clocks in the "Why" results for flops.
    . Use "-gui" to display the result in the Visualize Source Pane.
    . Use "-silent" to turn off the output to the screen and return information
      with a Tcl return value.

Non-Linear Cycle Operations
---------------------------
-cycle -hide cycle:cycle
    [-window window_name]
    Remove the specified cycles from view for better access to cycles of
    interest. The collapsed cycles cannot include a zoom group.
    . Specify a range of cycles, for example, 10:15.
    . By default, the tool applies your "visualize" command to the current
      window. Use "-window" to apply your command to a window other than the
      current window.
```

```
-cycle -unhide [target_cycle]
    [-window window_name]
    Reinstate previously hidden cycles that were specified with "-hide".
    . Specify a target cycle for the previously hidden cycles. If you do not
      specify a target, the tool reinstates all hidden cycles.
    . By default, the tool applies your "visualize" command to the current
      window. Use "-window" to apply your command to a window other than the
      current window.
```

Example

```
visualize -violation -property requirement
-----------------------

// Color annotation examples follow.
// Define a new color annotation
visualize -define_color_annotation {{a 1 red} {b 2 blue}} red_and_blue

// Activate the new color annotation
visualize -activate_color_annotation red_and_blue

// Delete the color annotation
visualize -define_color_annotation {} red_and_blue

-----------------------

// Example for "-replace_conf":
// Specifying a configuration
visualize -at_least_once {(foo == 1'b01)} 1:1 -name myConf

// Editing the "myConf" configuration
visualize -replace_conf -force {(foo == 1'b01)} 1:2 -name myConf
-----------------------

// Example for "-offset_conf":
// In the following example,we specify "conf1".
// We then use "-offset_conf" to change the cycle range from 3:5 to 5:7.

// Specify conf1
visualize -force exprA 3:5 -name conf1
```

```
// Offset the range for conf1
visualize -offset_conf conf1 2

// The resulting conf1 is the equivalent of the following command:
visualize -force exprA 5:7 -name conf1
-----------------------

// Example for Interactive Constraint Extraction:
// If we have a trace where my_input == 2'b00 for
// the first two cycles and then 2'b01 for the third and fourth cycles,
// we can say:
visualize -extract_conf {my_input {1} my_input {3:4} } -value -name myconfName 5:$

// This is equivalent to:
visualize -at_least_once {my_input != 2'b00 %amp;%amp; my_input !=2'b01} -name
myConfName 5:$
-----------------------

// Examples for specifying a waveform in a single command:

visualize -signal A -sequence {0 1 1 0 0 1 1 1 0}

// The {3} shows the number of times the value 1 is repeated.
visualize -signal B -sequence {0 1{3} 0 1 0}

visualize -signal C -sequence {0 1 1 1 0 0 x x 0}

visualize -signal D -sequence {0 1{3} 0{2} x{3} 0}

visualize -signal busA -sequence {8'b0001 0000 8'hffff 8'h0 8'd0}

visualize -signal busB -sequence {8'b0001 0000 8'hffff 8'h0{2}}
-----------------------

// Examples for applying QuietTrace to a cycle range
// The second command overrides (and erases) the first one. That is, the
// commands are not cumulative.
visualize -quiet true 4:9
visualize -quiet true 12:100

// Loading a VCD file generated by simulation when you have a wrapper module
//
// Simulation environment:
```

```
// SimulationEnv.TestBench.<Instances_I_want_to_verify>
//
// Formal environment:
// Formal_wrapper.Instances_I_want_to_verify>
//
// Use the following command:

visualize -load -vcd  my_vcd  -hier_map -from SimulationEnv.TestBench -to
Formal_wrapper
visualize -load -fsdb my_fsdb -hier_map -from SimulationEnv.TestBench -to
Formal_wrapper
visualize -load -shm  my_shm  -hier_map -from SimulationEnv.TestBench -to
Formal_wrapper
visualize -load -vwdb my_vwdb -hier_map -from SimulationEnv.TestBench -to
Formal_wrapper
-----------------------

// Examples for listing rows of signals and groups that are visible in the
// Visualize window.
visualize -plot_row -list
visualize -plot_row -list signal
visualize -plot_row -list group
visualize -plot_row -list signal -include_id
visualize -plot_row -list signal -target_pane preview
visualize -plot_row -list group -window windowName
visualize -plot_row -list -target_pane anchor -window windowName
-----------------------

// Examples for manipulating tracking signals in Visualize window.
visualize -tracking_signal -add signal_a -child_signal {signal_b signal_c signal_d}
visualize -tracking_signal -add signal_a 8 -child_signal {signal_b signal_d}
visualize -tracking_signal -add signal_a -child_signal signal_b
visualize -tracking_signal -remove signal_a -child_signal {signal_b signal_c signal_d}
visualize -tracking_signal -remove signal_a 8 -child_signal {signal_b signal_d}
visualize -tracking_signal -remove signal_a -child_signal signal_b
```

See Also

set_trace_optimization

5

Set Commands

This chapter includes documentation for the configuration commands. Every configuration command has a complementary `get_*` command to report the current setting. For example, the `get_prove_with_soft` command returns `on` or `off` depending on the current setting of the `set_prove_with_soft (on | off)` configuration.

> For details related to the `get_*` commands, access the Jasper TclHelp Browser from the tool (*Help – Tcl Command Help*).

set_afv_generate_auto_deadlock

Specifies if AFV should automatically generate a deadlock assertion.

Syntax

```
set_afv_generate_auto_deadlock (true | false)
```

Returns

No return values

Default

false

Detailed Description

Use the "set_afv_generate_auto_deadlock" command to specify whether
AFV should automatically generate a deadlock assertion on all "can" wires.

true:
    AFV generates a deadlock assertion on all "can" wires automatically.

false:
    AFV does not automatically generate a deadlock assertion on all "can"
    wires.

Example

```
set_afv_generate_auto_deadlock true
```

set_afv_input_conditions_nested

Specifies input as single-line conditional or multiple nested if-statements.

Syntax

```
set_afv_input_conditions_nested (true | false)
```

Returns

No return values

Default

false

Detailed Description

Use the "set_afv_input_conditions_nested" command to specify whether Arch table input conditions are grouped into a single-line conditional if-statement or broken into multiple nested conditional if-statements.

true:
    Break input condition statements into multiple nested conditional if-statements.

false:
    Group input condition statements into a single conditional if-statement. This is the default.

Example

```
set_afv_input_conditions_nested true
```

set_afv_prll_property_consolidation

Specifies number of rows at which the tool generates only one parallel property.

Syntax

```
set_afv_prll_property_consolidation N
```

Returns

No return values

Default

50

Detailed Description

Use the command "set_afv_prll_property_consolidation" to generate one parallel property for all cases instead of generating separate properties for unique row pairs in a table. This can be helpful when dealing with tables that have a large number of rows.

With this feature, you can check parallel properties on large tables with a single generated property that is both easy to understand and easy to debug. The default value is 50; thus, if you do not change the default value and a table has 50 rows or less, the tool generates properties for unique row pairs in the table. If your table has more than 50 rows, the tool creates one property that covers all cases.

Example

```
set_afv_prll_property_consolidation 40
```

See Also

check_afv

set_afv_strict_declarations

Specifies strict declarations for AFV signals.

Syntax

```
set_afv_strict_declarations (true | false)
```

Returns

No return values

Default

false

Detailed Description

Use the "set_afv_strict_declarations" command to specify whether AFV should error out on undeclared signals used in AFV tables.

true:
    AFV errors out on undeclared signals.

false:
    AFV does not error out on undeclared signals.

Example

```
set_afv_strict_declarations true
```

set_allow_latch_enable_loops

Assume all combinational loops through latch-enable pins are stable.

Syntax

```
set_allow_latch_enable_loops (true | false)
```

Returns

No return values

Default

true

Detailed Description

Use this command to specify how you want the tool to handle combinational loops that go through latch enable pins.

. true: Assume all combinational loops through latch enable pins in the design are stable.
WARNING: Unstable combinational loops handled by the "true" option might lead to wrong proof results.
NOTE:
. A stable loop might have an arbitrary value at each cycle it is open (that is, when nothing is forcing its value).
. This command overrides "set_loop_handling" on loops through latch enable pins.

. false: Let "set_loop_handling" handle all combinational loops.

Combinational loop handling operations are environment operations. Any changes to this setting will invalidate proof and plot results.

CAUTION: Using the "true" approach with properties that have functionally true combinational loops going through latch-enable pins can lead to false positive proof results or an overconstrained environment.

NOTE:
. By default (if you keep the setting as "true"), the tool assumes all combinational loops going through latch-enable pins are stable.
. "set_allow_latch_enable_loops" is included as an initial releases to gather feedback from early adopters and finalize implementation for an upcoming release. Contact support@cadence.com to provide feedback or comments.

To learn more about the combo loop flow, view the Jasper "Combo Loop Handling Flow" presentation available on Cadence Online Support (https://support.cadence.com/apex/ArticleAttachmentPortal?id=a1Od0000007MVwiEAG%amp;pageName=ArticleContent).

Example

```
set_allow_latch_enable_loops true

Using Global Loop Handling as default handler
---------------------------------------------
set_allow_latch_enable_loops false
prove -property prop
```

See Also

set_loop_handling

set_analyze_bbox_parse_mode

Controls parsing for black-boxed modules found by the "analyze -y" directive.

Syntax

```
set_analyze_bbox_parse_mode (no_header | read_header)
```

Returns

No return values

Default

no_header

Detailed Description

Use this command to control whether the tool will parse header (ANSI ports and parameters) of a blackboxed module discovered by the "analyze" command's "-y" directive. With the "no_header" value, "analyze" skips the module, while with "read_header", the tool reads ANSI ports and parameters.

Example

```
set_analyze_bbox_parse_mode read_header
```

See Also

analyze

set_analyze_libnonamehide

Enables or disables storing read modules in a temporary directory.

Syntax

```
set_analyze_libnonamehide (default | on | off)
```

Returns

No return values.

Default

on when in global file compilation unit (GFCU) analyze mode
off when in single- or multi-file compilation unit (SFCU or MFCU) analyze mode

Detailed Description

Use this command to control the "+libnonamehide" switch in the "analyze" command. When you enable this feature, the tool keeps all read modules in a temporary library during compilation.

Important things to know:

- For the default global file compilation unit (GFCU) mode of analyze, this setting is on by default.
- For multi- and single-file compilation unit (MFCU and SFCU) mode, this setting is off by default.
- Use "set_analyze_libnonamehide default" to return the tool to the GFCU, MFCU, and SFCU default settings.

Example

```
set_analyze_libnonamehide on
```

See Also

analyze,
set_analyze_librescan

set_analyze_librescan

Enables or disables directory rescanning to find Verilog modules.

Syntax

```
set_analyze_librescan (on | off)
```

Returns

No return values.

Default

off

Detailed Description

Use this command to control the "+librescan" switch in the "analyze" command. When you enable this feature, the tool always scans all Verilog directories in the directory list when looking for a new module.

Example

```
set_analyze_librescan on
```

See Also

analyze,
set_analyze_libnonamehide

set_analyze_libunboundsearch

Enables or disables the cross-file compilation unit "-L" option.

Syntax

```
set_analyze_libunboundsearch (on | off)
```

Returns

No return values.

Default

off

Detailed Description

Use this command to control the "+libunboundsearch" switch in the "analyze" command. When you enable this feature, the tool does not clear "-L" options when analyzing in multi- and single-file compilation unit (MFCU and SFCU) mode.

Example

```
set_analyze_libunboundsearch on
```

See Also

analyze,
set_analyze_librescan

set_analyze_y_and_v_command_line_order

Modifies the command line order of -y and -v arguments for the analyze command.

Syntax

```
set_analyze_y_and_v_command_line_order (keep | y_first)
```

Returns

No return values

Default

keep

Detailed Description

Use this command to specify the order the "analyze" command respects when
reading "-y" and "-v" arguments from the command line.

keep
    Respect the order used on the command line.

y_first
    Analyze "-y" arguments before "-v" arguments.

NOTE:
. This command affects "-y" and "-v" order on the command line only.
  It does not affect "-y" and "-v" definitions accumulated across multiple
  "analyze" commands.
. This is an initial release to gather feedback from
  early adopters and finalize implementation for an upcoming release.
  Contact support@cadence.com to provide feedback or comments.

Example

```
set_analyze_y_and_v_command_line_order keep

set_analyze_y_and_v_command_line_order y_first
```

See Also

analyze

set_annotation

Annotates the specified property or signal.

Syntax

```
set_annotation ((-property property_name [-regexp])
    |-signal signal_name) '{'annotation'}'
```

Returns

No return values

Default

No default

Detailed Description

```
Use this command to annotate a property or signal or to modify an existing
annotation. View annotations in the Visualize window waveforms pane by
using the mouse to hover over the property or signal and reveal the tool tip.

-property property_name [-regexp]
    Annotate the specified property.

    NOTE:
    . This switch supports wildcards and regular expressions; for example:
      set_annotation -property {expression} -regexp {annotation}
    . When the tool finds a wildcard or regular expression match for the
      property names, it applies the corresponding annotations to all matching
      properties.

-signal signal_name
    Annotate the specified signal.

{annotation}
    Annotate the specified property or signal with the specified description
    that is enclosed by curly braces.

    To remove an annotation, use empty curly braces.
```

Example

```
set_annotation -signal clk {Global clock}

# Remove annotation
set_annotation -signal clk {}
```

set_auto_disable_related_covers

Configures the tool to automatically disable related covers

Syntax

```
set_auto_disable_related_covers (true | false)
```

Returns

No return values

Default

true

Detailed Description

By default, the tool automatically disables all enabled related covers when you disable their main properties. Use this command to turn this feature on or off.

Example

```
set_auto_disable_related_covers false
```

set_automatic_library_search

Enables "analyze" and "elaborate" to search modules in user-declared libraries.

Syntax

```
set_automatic_library_search (on | self_first | off)
```

Returns

No return values

Default

off

Detailed Description

By default, "analyze" and "elaborate" only search for modules in the default library "work". However, you can explicitly specify additional user-declared Verilog packages or a VHDL or Verilog library with the "analyze -L" and "elaborate -L" commands.

CAUTION: If you specify "on" or "self_first", and the same name exists for multiple definitions of a module in different libraries, the tool chooses any of these definitions during "elaborate", and it does not generate a warning message for this situation.

on
    Enable future "analyze" and "elaborate" commands to automatically search for modules in user-defined packages or libraries. This behavior is equivalent to adding an "-L" switch to "analyze" and "elaborate" commands for each user-declared package or library.

self_first
    Search the current working library (library named "work" or defined by "-lib" directive, if present) for an unresolved object before looking into the other libraries listed with the "-L" directive as described in the "on" mode.

off
    Search for modules in the default library "work". This is the default setting.

Example

```
set_automatic_library_search on
# Analyzes dut module inside my_lib library.
analyze -sv -lib my_lib dut.sv
# Analyzes top module, which instantiates dut.
# "set_automatic_library_search" implicitly adds
# "-L my_lib" switch to the following commands:
analyze -sv top.sv
elaborate -top top
```

See Also

analyze,
elaborate

set_blackbox_assistant_require_clock_paths_in_connections

Controls whether blackbox assistant requires clock paths in connections

Syntax

```
set_blackbox_assistant_require_clock_paths_in_connections (true | false)
```

Returns

No return values

Default

false

Detailed Description

Use this command to enable blackbox assistant to require clock paths on connections tied to constant values. On connections tied to constants the entire COI of data paths of the destination signal will be required for elaboration. When this mode is enabled, the paths of clock signals inside the COI of the destination signal will also be required for elaboration.

Example

```
set_blackbox_assistant_require_clock_paths_in_connections true
```

set_blackbox_assistant_require_control_paths_in_connections

Controls whether blackbox assistant requires control paths in connections

Syntax

```
set_blackbox_assistant_require_control_paths_in_connections (true | false)
```

Returns

No return values

Default

false

Detailed Description

Use this command to enable blackbox assistant to require control paths on connections tied to constant values. On connections tied to constants the entire COI of data paths of the destination signal will be required for elaboration. When this mode is enabled, the paths of control signals (MUX selector, flip-flop reset value/condition and latch enable) inside the COI of the destination signal will also be required for elaboration.

Example

```
set_blackbox_assistant_require_control_paths_in_connections true
```

set_c2rtl_disable_imp_assumption

Specifies whether to automatically disable imp embedded assumptions.

Syntax

```
set_c2rtl_disable_imp_assumption (none | assumptions_embedded)
```

Returns

No return values

Default

none

Detailed Description

Use this command to specify whether to disable imp side embedded assumptions.
during the C2RTL generate verification process.

```
none                 - Do now disable embedded assumptions. This is the
                       default.
assumptions_embedded - Disable embedded assumptions.
```

NOTE: You must run this command before C2RTL setup.

Example

```
set_c2rtl_disable_imp_assumption assumptions_embedded
```

set_c2rtl_max_signals_to_plot

Sets the maximum number of signals plotted in a C/C++ plot without user prompt.

Syntax

```
set_c2rtl_max_signals_to_plot N
```

Returns

No return values

Default

10

Detailed Description

When multiple signals are pasted or dragged into the waveform in C/C++ mode, the tool issues a message that shows the number of signals that will be plotted so you can specify whether to plot them all. Use this command to set the maximum number of signals that can be pasted or dragged without this message appearing.

NOTE: A single C/C++ variable can match multiple signals.

Example

```
set_c2rtl_max_signals_to_plot 50
```

set_cache_proof_simplification

Turns on (or off) proof simplification caching.

Syntax

```
set_cache_proof_simplification (on | off)
```

Returns

No return value

Default

off

Detailed Description

This command enables or disables caching of the result of the simplification of the verification task, which is controlled by "set_proof_simplification". Use this command for a faster proof startup when running "prove" or "visualize" on the same task multiple times. Proof simplification caching is off by default.

Important things to know:

. The tool stores the cache in the project directory, and these files store the entire netlist, which can be some megabytes per cached result on larger designs.
. You can clear the cache with "clear -cache_proof_simplification".
. Proof simplification normally uses proven properties to simplify the netlist, but if you use caching, the tool cannot take advantage of newly proven properties. As a consequence, proof performance can be affected.
. To take advantage of the cache, a task must be identical to a previous prove or Visualize run except for property status. Additional proven properties should not affect the cache hit rate.
. You will know the tool is using a cached result by checking the log for a message such as the following:

```
Performing Proof Simplification...
0.Hp: Loaded cached simplification result
Proof Simplification completed in 1.07 s
```

Example

```
set_cache_proof_simplification on
```

See Also

clear -cache_proof_simplification,
prove,
set_proof_simplification,
visualize

set_caf_engine_mode

Controls the engine mode used by CAF.

Syntax

```
set_caf_engine_mode
        ( B | B4 | Bm | C | C2 | D | EC | G | G2 | H | Hp | Hps | Ht
        | Hts | I | J | K | L | M | N | AB | AD | AG | AM | Oh | Q3
        | R | Tri | U | U2 | TM | QT)+
```

Returns

No return values

Default

Ht J N I L

Detailed Description

Use this command to set the engine mode for CAF.

Example

```
set_caf_engine_mode Tri
set_caf_engine_mode {Ht Hp M}
```

See Also

check_caf,
set_caf_prove_parallel_tasks,
set_engine_mode

set_caf_prove_keep_traces

Specifies whether "check_caf -prove" keeps traces.

Syntax

```
set_caf_prove_keep_traces (assert | all | none)
```

Returns

No return values

Default

assert

Detailed Description

Use this command to specify whether "check_caf -prove" generates traces
for assertions only, for all properties, or for no properties.
By default, "check_caf -prove" generates traces for assertions only ("assert").

Example

```
set_caf_prove_keep_traces none
```

See Also

autoprove,
check_caf

set_caf_prove_parallel_tasks

Controls task parallelism during CAF proof stage.

Syntax

```
set_caf_prove_parallel_tasks (on | off | auto)
```

Returns

No return values

Default

auto

Detailed Description

Use this command to enable task parallelism during CAF proof stage.
By default, CAF proof stage issues a single "autoprove" command during
a local execution. When you set "caf_prove_parallel_tasks" to "on"
(or "auto" if the "proofgrid_mode" is not "local"), CAF attempts to run
tasks in parallel, that is, it runs a single "autoprove" command for each
task in parallel.

NOTE: Memory, CPU, and license resource allocation could be high.

on
    Runs CAF proof stage in parallel for all tasks.

off
    Disables CAF task parallelism during proof stage.

auto
    Runs CAF proof stage for all tasks in parallel when "proofgrid_mode"
    is not "local".

Example

```
set_caf_prove_parallel_tasks on
check_caf -prove
check_caf -report
```

See Also

check_caf,
set_engine_mode,
set_proofgrid_mode

set_caf_prove_time_limit

Specifies the time limit for the "check_caf -prove" command.

Syntax

set_caf_prove_time_limit time_limit

Returns

None

Default

2h

Detailed Description

Use this command to specify the time limit for the "check_caf -prove" command. The time limit is an integer ending in "s" for seconds, "m" for minutes, or "h" for hours. The default limit for the "check_caf -prove" command is "2h".

Example

set_caf_prove_time_limit 1h

See Also

autoprove,
check_caf

set_caf_signal_type

Enables signal extraction for the specified signal types only.

Syntax

```
set_caf_signal_type {all | input | output | custom}+
```

Returns

No return value

Default

output

Detailed Description

Use this command to specify which types of signals CAF extracts when you enable signal checks such as HLS_BT_STCK.

Example

```
set_caf_signal_type {input output custom}
```

See Also

check_caf,
set_caf_signals_filter_wide_signal,
set_caf_signals_extraction_heuristic

set_caf_signals_customize

Specifies signals to be used for CAF HLS_BT_STCK checks with custom types.

Syntax

```
set_caf_signals_customize {signal_name_list}
```

Returns

No return values

Detailed Description

Specifies a list of signals to be used for CAF (C Auto Formal) HLS_BT_STCK check with custom signal types option. The selected signals are extracted in addition to all the other enabled signals.

NOTE: Each time you use this command, it replaces the signals previously provided with this command.

Example

```
set_caf_signals_customize {id1 id2}
```

See Also

check_caf,
set_caf_signals_filter_wide_signal

set_caf_signals_extraction_heuristic

Configures heuristic for extracting signal checks in CAF.

Syntax

```
set_caf_signals_extraction_heuristic (all | auto)
```

Returns

No return values

Default

auto

Detailed Description

Use "set_caf_signals_extraction_heuristic" to specify the heuristic
CAF uses during the extraction of signal checks.

. all - Extract signal checks for all wide signals.
. auto - Extract wide-signal checks on signals that are in the sensitivity list
         or have critical values. This is the default.

Example

```
set_caf_signals_extraction_heuristic all
```

See Also

check_caf,
set_caf_signals_filter_wide_signal

set_caf_signals_filter_connected

Controls filtering of connected signals for CAF signal checks.

Syntax

```
set_caf_signals_filter_connected (true | false)
```

Returns

No return values

Default

true

Detailed Description

Use "set_caf_signals_filter_connected" to specify whether CAF filters
connected signals when signal checks are enabled:

. true - Connected signals are filtered and represented by one property only.
          This is the default.
. false - Connected signals are not filtered.

Example

```
set_caf_signals_filter_connected false
```

See Also

check_caf,
set_caf_signals_filter_wide_signal,
set_caf_signals_extraction_heuristic

set_caf_signals_filter_constant

Controls filtering of constant signals for CAF signal checks.

Syntax

```
set_caf_signals_filter_constant (true | false)
```

Returns

No return values

Default

true

Detailed Description

Use "set_caf_signals_filter_constant" to specify whether CAF filters out design constants when signal checks are enabled:

. true - Constant signals are filtered out and no properties are generated for them. This is the default.
. false - Constant signals are not filtered, and bit stuck-at properties are generated for constant outputs.

NOTE: This command considers design constants and not functional constants.

Example

```
set_caf_signals_filter_constant false
```

See Also

check_caf,
set_caf_signals_filter_connected,
set_caf_signals_filter_wide_signal

set_caf_signals_filter_wide_signal

Configures whether CAF extracts checks on wide signals.

Syntax

```
set_caf_signals_filter_wide_signal (on | off)
```

Returns

No return values

Default

on

Detailed Description

Use "set_caf_signals_filter_wide_signal" to specify whether CAF extracts wide signals.

. on - Consider single-bit signals only. This is the default.
. off - Consider wide signals for all signal checks.

Example

```
set_caf_signals_filter_wide_signal off
```

See Also

check_caf,
set_caf_signals_extraction_heuristic

set_caf_signals_max_bits

Limits the number of bits for stuckat property generation in CAF.

Syntax

```
set_caf_signals_max_bits N
```

Returns

No return values

Default

1

Detailed Description

Use "caf_signals_max_bits" to limit the number of bits on which the tool generates toggle or stuckat properties when "caf_stuck_at_bitblasted" is set to true. Use a non-positive value to unlimit max bits.

Example

```
set_caf_signals_max_bits 4
```

See Also

check_caf,
set_caf_stuck_at_bitblasted

set_caf_stuck_at_bitblasted

Controls the bit blasting of the stuckat properties in CAF.

Syntax

```
set_caf_stuck_at_bitblasted (true | false)
```

Returns

No return values

Default

false

Detailed Description

Use "set_caf_stuck_at_bitblasted" to control the bit blasting of stuckat properties in CAF.

Example

```
set_caf_stuck_at_bitblasted true
```

See Also

check_caf

set_capture_elaborated_design

The "set_capture_elaborated_design" command has no effect.

Syntax

```
set_capture_elaborated_design (on | off)
```

Returns

No return values

Default

off

Detailed Description

With the current save/restore flow, this configuration is obsolete and can be removed from your scripts.

See Also

elaborate,
restore,
save

set_capture_property_expression

Enables or disables extraction of expressions used in embedded properties.

Syntax

```
set_capture_property_expression (on | off)
```

Returns

No return values

Default

on

Detailed Description

Enables or disables extraction of expressions used in embedded properties.

NOTE:
. Use this command before "elaborate".
. You must enable this configuration to use the "cover -extend" feature.

Example

```
set_capture_property_expression off
```

See Also

cover -extend

set_cdc_calculate_boundary_data_during_extract

Enables boundary data calculation during the CDC extraction phase.

Syntax

```
set_cdc_calculate_boundary_data_during_extract (true | false)
```

Returns

No return values

Default

false

Detailed Description

Use this command to enable or disable the calculation of boundary data
during the CDC structural analysis extraction phase.

true
    The tool calculates boundary data during extraction. This enables a regular
    CDC database to be used as a hierarchical database without explicitly
    calling "check_cdc -signoff -export" command.
false
    The tool does not calculate boundary data during extraction.

Example

```
set_cdc_calculate_boundary_data_during_extract true
```

See Also

check_cdc

set_cdc_clock_period_max_factor

Specifies the maximum normalized period of a CDC clock.

Syntax

```
set_cdc_clock_period_max_factor N
```

Returns

No return values

Default

10

Detailed Description

Use this command to specify the maximum normalized period of a CDC clock. This period affects the normalization algorithm and sets all periods to 1 in case normalized factors exceed the value specified.

Example

```
set_cdc_clock_period_max_factor 30
```

set_cdc_conditional_waiver_property_limit

Sets the maximum number of properties CDC generates for conditional waivers.

Syntax

```
set_cdc_conditional_waiver_property_limit threads_quantity
```

Returns

No return values

Default

200000

Detailed Description

Use this command to change the maximum number of properties the CDC App will generate for conditional waivers.

Example

```
set_cdc_conditional_waiver_property_limit 500000
```

set_cdc_gate_level_analysis_mode

Enables or disables CDC gate-level analysis mode.

Syntax

```
set_cdc_gate_level_analysis_mode (on | off)
```

Returns

No return values

Default

off

Detailed Description

Use this command to enable or disable CDC gate-level analysis. When enabled, CDC analysis reports violations specific for gate-level designs.

Example

```
set_cdc_gate_level_analysis_mode true
```

set_cdc_machine_learning_flow

Enables the machine-learning (ML) flow for suggested waivers in the CDC domain.

Syntax

```
set_cdc_machine_learning_flow (true | false)
```

Returns

No return values

Default

false

Detailed Description

Use this command to enable the machine learning (ML) flow for ML-suggested waivers for CDC checks.

Example

```
set_cdc_machine_learning_flow true
```

set_cdc_parallel_extraction

Sets the number of threads for CDC structural analysis.

Syntax

```
set_cdc_parallel_extraction threads_quantity
```

Returns

No return values

Default

1

Detailed Description

Use this command to enable or disable CDC multithread analysis by changing the number of threads.

Example

```
set_cdc_parallel_extraction 5
```

set_cell_define_debug_visibility

Goes into cell define blocks during design traversals.

Syntax

```
set_cell_define_debug_visibility (on | off)
```

Returns

No return values

Default

off

Detailed Description

Use this command to specify that you want the tool to go inside cell define blocks during design traversals, that is, fanin, fanout, driver, and load. If you set this configuration variable to on, both the Tcl and GUI design traversals consider results that are inside cell defines as valid results.

off
    Do not go inside cell define blocks during design traversals, that is, fanin, fanout, driver, and load. This is the default setting.

on
    Consider results that are inside cell defines as valid results.

Example

```
set_cell_define_debug_visibility on
```

See Also

get_fanin,
get_fanout,
set_function_debug_visibility

set_cfe_compile_pedantic_errors

Controls whether the CFE compiler enforces strict C/C++ standard compliance.

Syntax

```
set_cfe_compile_pedantic_errors (true | false)
```

Returns

No return values

Default

false

Detailed Description

This command configures the strictness level of the CFE compiler. When set to "true", the compiler enforces strict ISO C/C++ standard compliance.
When set to "false", the compiler accepts GCC C/C++ extensions.

. true: Enforces strict ISO C/C++ standard compliance, rejecting compiler-specific extensions and non-standard language features.
. false: The compiler accepts GCC C/C++ extensions. This is the default.

NOTE: This setting must be configured before running "check_c2rtl -compile".

Example

```
set_cfe_compile_pedantic_errors true
```

See Also

check_c2rtl

set_cfe_profiler_default_function_name

Sets the default function name display format in the CFE profiler.

Syntax

```
set_cfe_profiler_default_function_name (short | reduced | full)
```

Returns

No return values

Default

full

Detailed Description

This command configures the default function name display format used in the CFE profiler widget and profiler data dump.

. full: Displays the full function name including namespaces, classes, and template parameters.
. reduced: Displays the reduced function name with template parameters removed.
. short: Displays the short function name without namespace or class prefixes.

NOTE: This setting affects both the profiler GUI and the profiler data dump.

Example

```
set_cfe_profiler_default_function_name reduced
```

See Also

check_c2rtl

set_clock_loop_model

Controls how Jasper handles loops through clock pins during proofs.

Syntax

```
set_clock_loop_model (none | assume_no_loop | allow_loop)
```

Returns

No return value

Default

assume_no_loop

Detailed Description

Loops through clock pins can cause glitches in design logic. Flops can disable their own clock causing a glitch on their clock pin. See below when d is 0:

```
  reg   q;
  assign gclk = q & clk;
  always @ (posedge gclk) q <= d;
```

Use this command to control how the tool handles structural clock loops during the proof.

none
    Exit with ECB100/ECB101 if there are structural loops through clock pins. The flop, or part of the loop, will be listed in the error message.

assume_no_loop
    Assume that structural clock loops cannot be functional. The flops will be listed in a WCB100/WCB101 warning. See "check_loop -formal" for how to check if functional clock loops are possible, and if they are, how they can be analyzed.

allow_loop
    If loops through clock pins are present, attempt to build a model that correctly handles them. In complex cases, this can fail with ECK113. See "check_loop -formal" and/or "check_loop -include_clock" for how to identify and understand the loop.

See "check_loop -formal" for how to check if functional clock loops are possible. See "check_loop -include_clock" for how to identify and understand structural loops.

Example

```
set_clock_loop_model none
```

See Also

check_loop,
set_loop_handling

set_cmd_time_limit

Sets the time limit for most Tcl commands.

Syntax

```
set_cmd_time_limit time_limit
```

Returns

No return values

Default

7200s

Detailed Description

Use this command to set the time limit for any command except those listed below. The time limit is an integer ending in "s" for seconds, "m" for minutes, or "h" for hours.

. "get_needed_assumptions" does not honor "set_cmd_time_limit".
. The following commands do not honor "set_cmd_time_limit", but they do honor "prove_time_limit". (Refer to the "See also" section.)
  . autoprove
  . check_afv -prove
  . check_assumptions
  . check_cdc -metastability -prove
  . check_cdc -protocol_check -prove
  . check_cdc -signal_config -prove
  . check_cdc -waiver -prove
  . check_conn -prove
  . check_cov -measure
  . check_csr -prove
  . check_fsv -prove
  . check_sec -prove
  . check_spv -prove
  . check_unr -prove
  . check_xprop -prove
  . hunt
  . prove
  . visualize

time_limit
    Use the specified time limit for each command.

    NOTE: To remove the limit, use 0s.

Example

```
set_cmd_time_limit 2m
```

See Also

configure,
set_elab_time_limit,
set_prove_per_property_time_limit,
set_prove_per_property_time_limit_factor,
set_prove_time_limit

set_command_license_wait_interval

Sets the time interval between license checkout attempts.

Syntax

```
set_command_license_wait_interval wait_interval
```

Returns

No return values

Default

1s

Detailed Description

Use this command to set the time interval between license checkout attempts.

NOTE: This setting only affects checkouts requested by Tcl commands.

Example

```
set_command_license_wait_interval 2s
```

See Also

set_command_license_wait_retries

set_command_license_wait_retries

Sets the number of checkout retries the tool should attempt.

Syntax

```
set_command_license_wait_retries N
```

Returns

No return values

Default

0

Detailed Description

Use this command to set the number of checkout retries the tool should attempt before failing.

NOTE: This setting only affects license checkouts requested by Tcl commands.

Example

```
set_command_license_wait_retries 1
```

See Also

set_command_license_wait_interval

set_compilation_cache

Enables or disables compilation caching for "elaborate".

Syntax

```
set_compilation_cache (on | off)
```

Returns

No return values

Default

off

Detailed Description

Use this command to enable or disable compilation caching.

When enabled, the tool caches Verilog modules during elaboration, so subsequent invocations of the "elaborate" command do not need to re-elaborate them. Modules are automatically reloaded from cache.

NOTE: The results of the analysis are not cached, so subsequent invocations re-analyze all files.

on: Enable compilation caching flow.
off: Disable compilation caching flow. This is the default.

In overwrite scenarios, the cache stores the final definition.

LIMITATIONS: The following apps/flows are not supported by Compilation Cache:
. blackbox_assistant
. elaborate -req
. save/restore
. MCP
. Superlint
. UNR (P2P)
. PSL inheritance with default clock
. Cpi2Jetlist
. C2RTL

Example

```
set_compilation_cache on
```

See Also

set_compilation_cache_path

set_compilation_cache_path

Defines a location for "compilation_cache".

Syntax

```
set_compilation_cache_path path
```

Returns

No return values

Default

/work

Detailed Description

Use this command to control the "compilation_cache" path.
When "compilation_cache" is enabled, the compilation cache files are stored in the path defined with this command. If no path is specified, the default location is used.

Example

```
set_compilation_cache on
set_compilation_cache_path myCache
```

See Also

set_compilation_cache

set_compilation_command

Provides a compilation command to compile C/C++.

Syntax

```
set_compilation_command "command"
```

Returns

No return values

Detailed Description

Use this command to specify a compilation command for C/C++ compilation. The "cfe_debug" command uses the value you set to compile the C/C++ source code if needed.

Example

```
set_compilation_command "g++ main.cpp -o main -I include/"
```

See Also

cfe_debug

set_compilation_skip_errors_in_filelist

Defines if invalid "-f" file entries raise errors or are ignored with warnings.

Syntax

```
set_compilation_skip_errors_in_filelist (on | off)
```

Returns

No return values

Default

on

Detailed Description

By default, the tool ignores invalid or unknown entries in "-f" files and issues a warning. Use this command to change the behavior so that invalid entries trigger an error instead.

on
Invalid or unknown entries in "-f" files trigger an error.

off
Invalid or unknown entries in "-f" files are ignored, and a warning message is displayed.

Example

```
set_compilation_skip_errors_in_filelist off
analyze -sv -f file.f
```

set_compilation_unit_naming_mode

Sets the automatic naming mode for Compilation Units.

Syntax

```
set_compilation_unit_naming_mode (file | incremental | none)
```

Returns

No return values

Default

"file"

Detailed Description

Use this command to define how to automatically name auto-generated Compilation Units.

Naming modes:

- **file**
  Automatically name an auto-generated Compilation Unit with the first analyzed filename as the suffix. All dots in the filename are changed to underscores.

- **incremental**
  The automatically generated suffix name of a Compilation Unit is an incremental number starting with the number 0.

- **none**
  Disable automatic naming for Compilation Units. Use "analyze -cu_name" to define the Compilation Unit name. If you do not define the Compilation Unit name, the tool issues the VERI-9096 error message.

NOTE: Run this command before "analyze".

Example

```
set_compilation_unit_naming_mode file
set_compilation_unit_naming_mode incremental
set_compilation_unit_naming_mode none
```

See Also

analyze

set_compilation_unit_naming_prefix

Sets the auto-naming prefix for Compilation Units.

Syntax

```
set_compilation_unit_naming_prefix prefix_name
```

Returns

No return values

Default

```
"\$unit"
```

Detailed Description

Use this command to define the prefix for automatic naming of Compilation Units. You can use any valid Verilog Identifier as a custom Compilation Unit prefix.

NOTE:
. Run this command before "analyze".
. SystemVerilog custom prefixes ($unit, $top, and $root) are not allowed.

Example

```
set_compilation_unit_naming_prefix my_prefix_name
```

set_conn_prove_loop_handling

Configures the default loop handling method for Connectivity proof strategies.

Syntax

```
set_conn_prove_loop_handling (full | partial | none)
```

Returns

No return values

Default

full

Detailed Description

```
Use this command to specify how you want the tool to handle loops in the COI of
properties proven by the proof strategies executed during "check_conn -prove":

none
    Apply no Connectivity loop handling strategy. Instead, apply
    "set_loop_handling" strategies.

partial
    Handle combinational loops by automatically adapting the proof model, and
    accept only valid proof results.

full
    Apply the partial strategy, and if combinational loops remain, apply
    automatic stopats in the COI of properties to remove loops. This
    option is the default setting.

    NOTE: Using this approach can lead to false counterexamples. If any
    properties fall into this category, the tool issues ICON010 during the
    proof and provides debugging instructions.

NOTE:
    . "set_loop_handling" still applies for proofs run after Connectivity
      proof strategies, and for any other "prove" command run on properties
      generated by Connectivity.
    . Connectivity toggle checks are not included in this setting and are
      covered by "set_loop_handling".

To learn more about the combo loop flow, go to https://support.cadence.com and
use the following search string:

Jasper Combo Loop Handling Flow
```

Example

```
set_conn_prove_loop_handling partial
```

See Also

set_loop_handling,
set_clock_loop_model

set_counter_speculative_detection

Turns on (or off) the speculative counter detection algorithm.

Syntax

```
set_counter_speculative_detection (on | off)
```

Returns

No return value

Default

off

Detailed Description

Use this command to enable the speculative counter detection algorithm. With this algorithm, the tool employs more permissive rules that allow it to detect more counters than the default algorithm. This new algorithm is useful when counter signals or signals within the path back to the counter flop are sliced. By default, this feature is off.

NOTE: This feature carries the risk of identifying false counters.

Example

```
set_counter_speculative_detection on
```

set_covergroup_bin_size_limit

Sets a limit for the size of covergroup bins.

Syntax

```
set_covergroup_bin_size_limit N
```

Returns

No return values

Default

5000000

Detailed Description

Use this command to specify a limit for the size of covergroup bins.

Bins inside covergroups can have variable sizes. Each bin generates a cover property, and using extremely large values can impact both performance and usability of the tool. Any bin that may exceed this value is ignored with the warning message WCG049 and is not considered by the tool.

NOTE: If the limit is set to "0", the tool treats it as "infinite".

Example

```
set_covergroup_bin_size_limit 0
set_covergroup_bin_size_limit 100000
```

set_covergroup_cross_product_limit

Sets a limit for the product of cross coverage inside covergroups.

Syntax

```
set_covergroup_cross_product_limit N
```

Returns

No return values

Default

5000000

Detailed Description

Use this command to specify a limit for the product of cross coverage.

The number of products of a covergroup cross is defined by all possible combinations of its related coverpoints. Since the tool needs to process every combination during synthesis to generate the actual covers, this is useful to avoid generating an unrealistic number of properties.

NOTE:
. If the limit is set to "0", the tool treats it as "infinite".
. The product count of a covergroup cross is defined only by its related coverpoints. The tool does not consider "ignore_bins".

Example

```
set_covergroup_cross_product_limit 0
set_covergroup_cross_product_limit 100000
```

set_cte_enable_session_tcl_env_restore

Determines if SDC processing considers Jasper's Tcl environment.

Syntax

```
set_cte_enable_session_tcl_env_restore (true | false)
```

Returns

No return values

Default

true

Detailed Description

If set to "false", SDC processing does not consider Jasper's Tcl environment when processing the SDC file.
If set to "true", Jasper's Tcl environment (variables, procs, namespaces, and so on) is restored before processing the SDC file.

NOTE: Use this command before "read_sdc".

Example

```
set_cte_enable_session_tcl_env_restore false
```

See Also

read_sdc

set_cumulative_reset

Enables reset cumulative mode.

Syntax

```
set_cumulative_reset (on | off)
```

Returns

No return values

Default

off

Detailed Description

Use "set_cumulative_reset" to enable or disable the reset cumulative mode. Turn on this mode for incremental reset specification to prevent subsequent "reset" commands from erasing the previous configuration.

NOTE: If you use an illegal combination of "reset" commands, the tool issues an error. For example, the tool issues an error for the following combination:

```
reset rst_a
reset -sequence seq_file
```

Example

```
set_cumulative_reset on
```

set_database_auto_save_restore

Enables automatic saves for the Jasper database.

Syntax

```
set_database_auto_save_restore (on | off)
```

Returns

No return values

Default

off

Detailed Description

Use this command before running "analyze" and "elaborate" to enable Jasper to save the database automatically. The tool saves the database in the Jasper project directory when you run the "prove" command. To restore the automatically saved database, re-run Jasper Apps with the same project directory and run the "restore" command.

Example

```
set_database_auto_save_restore on
```

See Also

restore

set_database_session_save_restore

Enables session-specific Jasper database save and restore.

Syntax

```
set_database_session_save_restore (on | off)
```

Returns

No return values

Default

off

Detailed Description

Use this command to enable session-specific save and restore to the Jasper database. In this mode, every session saves its data to a separate directory in the database.

Example

```
set_database_session_save_restore on
```

See Also

save,
restore

set_debug_handoff_source_lock

Specifies whether Debug Handoff uses Source Lock as reference for the bundle.

Syntax

```
set_debug_handoff_source_lock (on | off)
```

Returns

No return values

Default

on

Detailed Description

Use this command to control whether Debug Handoff uses Source Lock when saving the analyzed RTL files. By default, Debug Handoff uses Source Lock.

on: Debug Handoff reads the analyzed files from Source Lock.
    This is the default.
off: Debug Handoff reads the analyzed files from the user's file structure
    directly.

 NOTE: If Source Lock is not enabled, Debug Handoff will read from the user's
    filesystem regardless of this setting.

Example

```
set_debug_handoff_source_lock off
```

See Also

debug_handoff
get_debug_handoff_source_lock
source_lock

set_design_exploration_include_clock_logic

Includes clock logic in design info computations.

Syntax

```
set_design_exploration_include_clock_logic (on | off)
```

Returns

No return values

Default

off

Detailed Description

When on, Design Information includes clock logic in its computations. This configuration command enables or disables traversing through the clock pin of flip-flops.

For example:

```
module top (input clk, d, output logic q);
  always @(posedge clk) q <= d;
endmodule

set_design_exploration_include_clock_logic off
get_design_info -signal q -list signal -silent
d q

set_design_exploration_include_clock_logic on
get_design_info -signal q -list signal -silent
clk d q
```

Example

```
set_design_exploration_include_clock_logic on
```

set_dollar_display_synthesis

Enables $display synthesis in elaboration.

Syntax

```
set_dollar_display_synthesis (on | off)
```

Returns

No return values

Default

off

Detailed Description

Use "set_dollar_display_synthesis" to enable or disable the feature that synthesizes $display system I/O tasks. Use this command before running "analyze" and "elaborate" commands.

Example

```
set_dollar_display_synthesis on
```

set_elab_time_limit

Sets the time limit for elaboration.

Syntax

```
set_elab_time_limit time_limit
```

Returns

No return values

Default

0s (unlimited)

Detailed Description

Use this command to set the time limit for the "elaborate"
command. The time limit is an integer ending in "s"
for seconds, "m" for minutes, or "h" for hours.

time_limit
    Use the specified limit for each "elaborate" command.

    NOTE: To reset the default (unlimited), use 0s.

Example

```
set_elab_time_limit 3h
```

See Also

configure,
set_cmd_time_limit,
set_property_compile_time_limit,
set_prove_per_property_time_limit,
set_prove_time_limit,
set_task_compile_time_limit

set_elaborate_bbox_inout_simplification

Configures inout port simplification for blackboxed modules.

Syntax

```
set_elaborate_bbox_inout_simplification (off | on)
```

Returns

No return values

Default

on

Detailed Description

This command configures inout port modelling for blackboxed modules.

off
    Configures inout ports of blackboxed modules to be modelled as two ports, input and output.

on
    Configures inout ports of blackboxed modules to be modelled as a single input or output port.

NOTE: Disabling the simplification might add drivers to clock signals connected to inout ports of blackboxed modules, which might trigger error ECK025 on issuing the "clock" command. To avoid this error, add an environmental stopat on the clock signals.

Example

```
set_elaborate_bbox_inout_simplification off
```

set_elaborate_bbox_output_driver

Configures output strength of ports in blackboxed modules.

Syntax

```
set_elaborate_bbox_output_driver (strong | highz)
```

Returns

No return values

Default

strong

Detailed Description

This command configures the output strength of ports in blackboxed modules.

highz
    Configures ports of blackboxed modules to output a weak driver (z).

strong
    Configures ports of blackboxed modules to output a strong driver (x).

Example

```
set_elaborate_bbox_output_driver highz
```

set_elaborate_generate_sage_database

Controls whether Verisium Debug Database (Sage) is generated during elaboration.

Syntax

```
set_elaborate_generate_sage_database (on | off)
```

Returns

No return values

Default

off

Detailed Description

When enabled, the Verisium Debug Database (Sage) is populated with information during the elaboration phase, letting you load a Jasper-elaborated design in the Verisium Debug tool for debugging and analysis.

Example

```
set_elaborate_generate_sage_database on
```

set_elaborate_interrupt_on_error

Enables or disables stopping "elaborate" earlier on errors.

Syntax

```
set_elaborate_interrupt_on_error (on | off)
```

Returns

No return values

Default

off

Detailed Description

By default, "elaborate" collects as many errors as possible and reports them when it can make no further progress. Use this command to specify that you want to stop "elaborate" as soon as possible when it encounters a synthesis error.

NOTE: Resilient compilation and incremental flows do not support interrupting "elaborate" with this command.

Example

```
set_elaborate_interrupt_on_error on
```

See Also

elaborate

set_elaborate_limited_unrolled_loop_precondition

Enables the creation of precondition property for limited unrolled loops.

Syntax

```
set_elaborate_limited_unrolled_loop_precondition (on | off)
```

Returns

No return values

Default

off

Detailed Description

Use this command to enable the creation of a precondition property whenever a non-constant loop is limited unrolled during synthesis with "elaborate -enable_limited_unrolled_loops". This property helps verify whether the unrolled loop can have any scenario for an early exit.

on: Create a precondition property for limited unrolled loops.
off: Do not create a precondition property for limited unrolled loops.
     This is the default.

Example

```
set_elaborate_limited_unrolled_loop_precondition on
```

See Also

elaborate -enable_limited_unrolled_loops

set_elaborate_limited_unrolled_loop_property

Enables the creation of an automatic property for limited unrolled loops.

Syntax

```
set_elaborate_limited_unrolled_loop_property (on | off)
```

Returns

No return values

Default

on

Detailed Description

Use this command to enable the creation of an automatic property whenever a non-constant loop is limited unrolled during synthesis with "elaborate -enable_limited_unrolled_loops". This property helps verify whether the loop has been unrolled a sufficient number of times to avoid an early exit when run.

on: Create an automatic property. This is the default.
off: Do not create an automatic property.

Example

```
set_elaborate_limited_unrolled_loop_property off
```

See Also

elaborate -enable_limited_unrolled_loops

set_elaborate_procedural_loop_limit

Sets the procedural loop limit for elaborate.

Syntax

```
set_elaborate_procedural_loop_limit loop_count
```

Returns

No return values

Default

0

Detailed Description

Use this command to control the maximum number of iterations allowed in a procedural loop. The tool issues an error during synthesis if it reaches the configured limit.

This is a persistent setting for "elaborate -procedural_loop_limit". The real value used for the limit is the minimum between the procedural limit and the standard loop limit configured by "elaborate -loop_limit", which has a default of 5000.

The UNR app has a default procedural loop limit of 30. You can use this command to override the UNR default.

Example

```
set_elaborate_procedural_loop_limit 50
```

See Also

elaborate -loop_limit,
elaborate -procedural_loop_limit

set_elaborate_quiet_trace

Controls whether elaborate generates global QuietTrace soft constraints.

Syntax

```
set_elaborate_quiet_trace  (on | off)
```

Returns

No return values

Default

off

Detailed Description

When "set_elaborate_quiet_trace" is "on", "elaborate" adds QuietTrace(TM) soft constraints to the global environment. If you want QuietTrace soft constraints only in specific tasks, or you want to add them after elaboration, you can use "assume -soft -quiet" instead. Jasper only attempts to apply soft constraints in a proof if "set_prove_prefer_quiet on" was executed, "-prefer_quiet" was added to the "prove" command, or engine QT was added to the engine list.
NOTE: You must run this command before elaborating the design.

Example

```
set_elaborate_quiet_trace on
```

See Also

assume -soft -quiet,
assume -env -list,
elaborate

set_elaborate_real_mode

Sets how real and wreal signals are handled during elaboration.

Syntax

```
set_elaborate_real_mode (legacy | integer)
```

Returns

No return values

Default

legacy

Detailed Description

Use this command to control how real and wreal signals are handled during elaboration.

"set_elaborate_real_mode legacy" produces a downgradable VERI-1139 error for designs containing real data types. If downgraded, the tool ignores the port or signal during synthesis. When compiling Verilog AMS, the wreal data type is accepted but synthesized to an unsigned 64-bit vector. This is the default behavior.

"set_elaborate_real_mode integer" models both real and wreal data types as 64-bit integers, providing a more realistic signed representation.

In both configurations, the tool rounds hardcoded real values to the nearest 64-bit integer.

Example

```
set_elaborate_real_mode integer
```

See Also

set_message -warning

set_elaborate_single_run_mode

Enables or disables elaborate single run mode.

Syntax

```
set_elaborate_single_run_mode (on | off)
```

Returns

No return values

Default

on

Detailed Description

Use this command to control the "elaborate" single run mode. By default, single run mode is enabled, and in this mode, the tool uses less memory, but it only supports a single "elaborate" command. Therefore, any flow or feature that issues a second "elaborate" command after the design is already compiled will not work.

The following unsupported app and feature automatically disable single run mode and issue a warning:

. auto_setup
. DLD App

The following unsupported features issue an error and require manually disabling the single run mode:

. Multiple elaborate commands
. connect -bind
. connect -elaborate
. jasper_model_mpram
. jasper_model_divider
. jasper_model_multiplier

Example

```
set_elaborate_single_run_mode off
```

set_elaborate_use_disk

Specifies how the elaborated design is transferred to the current session.

Syntax

```
set_elaborate_use_disk (on | off)
```

Returns

No return values

Default

on

Detailed Description

Use this command to specify how the elaborated design is serialized to the current session.

on
  The tool uses the disk to serialize the elaborated design to the session. This option is the default.

off
  The tool uses the memory to serialize the elaborated design to the session.

NOTE:
. "elaborate_use_disk" must be set before you run the "elaborate" command.

CAUTION:
. Disabling this configuration avoids using storage space and can improve elaboration time, but it might increase the elaborate memory peak.

See Also

elaborate

set_enable_preserve_property_status

Sets whether property status is preserved if valid after environment changes.

Syntax

```
set_enable_preserve_property_status (true | false)
```

Returns

No return values

Default

true

Detailed Description

By default, any change in the environment invalidates all the results. When this configuration is enabled, the following should no longer invalidate all the results:

. Adding an assumption
. Enabling an assumption
. Converting a property to assumption ("-bound" or not)
. Adding a virtual net

NOTE:

. Adding an X-Prop assumption still invalidates all the results, even if this setting is enabled.
. This setting does not support properties that are non-linear, built-in, or X-Prop.

Example

```
set_enable_preserve_property_status true
```

set_engine_algorithm_selection

Enables engine algorithm selection.

Syntax

```
set_engine_algorithm_selection (off | train | infer | auto)
```

Returns

No return values

Default

off

Detailed Description

Use this command to direct the "prove" command to either train or infer engine configurations automatically for the set of properties processed by the proof.

. In "train" mode, the proof trains a machine learning (ML) model for the set of properties processed by the proof. You can use this trained model in subsequent "prove" invocations to direct the engines to automatically select engine configurations.

. In "infer" mode, the proof uses a previously trained model to infer automatically engine configurations.

. In "auto" mode, the proof combines "train" and "infer" modes.

NOTE:
. The set of engines for which configurations are trained and inferred are engines B, N, AM, and M.

Example

```
set_engine_algorithm_selection train
```

See Also

prove,
set_engine_algorithm_selection_dir

set_engine_algorithm_selection_dir

Sets the directory to save models created by engine algorithm selection.

Syntax

```
set_engine_algorithm_selection_dir full_path_to_directory
```

Returns

No return values

Default

Jasper project directory

Detailed Description

Use this command to specify the directory to save models created
by engine algorithm selection.

NOTE:
. The tool creates all parent directories as needed.
. If you do not specify a directory, the tool stores the ML models in the
  project directory. These models are not deleted when reusing the same project
  directory across multiple tool runs.

Example

```
set_engine_algorithm_selection_dir ~/my_dir
```

See Also

set_engine_algorithm_selection

set_engine_distributed_mode

Enables running portfolios of distributed engines.

Syntax

```
set_engine_distributed_mode (on | off)
```

Returns

No return values

Default

off

Detailed Description

Set "engine_distributed_mode" to "on" to direct the tool to run a portfolio of engines in accordance with the values of the configuration variables "engine_processes", "engineB_processes", and "engineBm_processes".

Example

```
set_engine_distributed_mode on
set_engineB_processes 32
prove -all -engine_mode {B}
```

See Also

set_engine_processes,
set_engineB_processes,
set_engineBm_processes

set_engine_mode

Optimizes the proof for different types of designs and requirements.

Syntax

```
set_engine_mode
    auto
  | -auto N
  | default
  | (B | B4 | Bm | C | C2 | D | G | G2 | H | Hs | Hp | Hps | Ht | Hts | I | J
  | K | L | M | Mp | N | AB | AD | AG | AM | Oh | Q3 | R | Tri
  | U | U2 | TM | QT)+

C2RTL Word-Level Engines
------------------------
    (WHp | WHt | WHps | WHs | WB | WB1 | WB2 | WB3 | WI | WAM | WECX1 | WECX2 | WG2 |
WA1 | WA2 | WA3)+
```

NOTE: "set_engine_mode" accepts engine names in the short form noted above and in the long form, which uses the prefix "engine", for example, "engineB".

Returns

No return values

Default

auto

Detailed Description

By default, Jasper Apps uses Proof Orchestration if you use the default engine mode (set_engine_mode auto). Proof Orchestration removes the burden of choosing engines and fine tuning proofs, so you can focus on verification. This feature dynamically adjusts engine selection, time limits, and so forth during

a proof while respecting user-defined max jobs/licenses and global time limits.

Orchestration always respects the engine_mode you set. However, if you set engine_mode to "auto", Orchestration can choose to run any set of engines. If you run a proof with Proof Profiling Data (see "prove -save_ppd" and "prove -with_ppd"), and you also specified engine_mode "auto", Orchestration runs any useful engine (showed determinations) based on the data in the PPD. Otherwise, even if the engine was useful but it is not part of the engine mode, Orchestration does not run it. Therefore, unless you know specifically which engines are useful to your proof, the advice is to let Orchestration choose the set of engines ("set_engine_mode auto").

NOTE: The flow running PPD without Orchestration uses the engine_mode you specify. And in the case of engine_mode "auto", it runs the default engine mode: {Ht Hp N B}.

Using Proof Orchestration is preferred in most cases; however, in some cases, you might want to disable Proof Orchestration and use the "set_engine_mode" command to specify engines for "prove" and "visualize" that are tuned for different purposes. Specify a single engine or use curly braces to specify multiple engines. In general, using multiple engines leads to overall faster results.

Using multiple engines (or multiple jobs per engine) is most effective when running the jobs on remote machines (see "set_proofgrid_mode" and "set_proofgrid_per_engine_max_jobs"). Using many engines (or jobs) in local mode might overload your machine and lead to poor performance or even tool termination due to an out-of-memory situation.

Many (but not all) engines target properties serially. The first engine mode or process to converge on a proof or find a counterexample ends the proof of the target property.

Important things to know about this command:
. You cannot specify H with either Hp or Ht. For example, { Hp Ht } is
  acceptable, but the tool issues an error when you attempt to use { H Hp }.
. You can use "prove -engine" to override the engine mode for individual proofs.

Refer to the "Jasper Engine Selection Guide" for details (Help > Application Guides).

auto
    Use with "prove_orchestration on" to direct the "prove" command to select

engines automatically.

When prove_orchestration is "off", the tool uses the default engine mode: {Ht Hp N B}.

See also "set_prove_orchestration".

-auto N
Use the specified number of automatically selected engines.

Use this command in conjunction with "set_proofgrid_per_engine_max_jobs" or "set_proofgrid_per_engine_max_local_jobs" to increase proof power. Use "get_engine_mode" to see which engines are currently specified.

default
Use the default engine mode, {Ht Hp N B}.

B
Use engine B, which only searches for traces and will generally not find proofs.

NOTE: Engine B is included in the default engine mode.

B4
Use engine B4, which only searches for traces and will generally not find proofs.

NOTE:
. Engine B4 tries to infer better-than-default engine B configuration on each given property based on a machine learning model trained on our internal benchmarks across multiple customer designs using supervised learning.
. Engine B4 respects all configurations affecting engine B except "engine_threads".

Bm
Use engine Bm, which finds counterexamples for multiple safety properties in the same task concurrently.

NOTE: When you ask engine Bm to prove a mix of safety and liveness assertions, the tool first proves all safety properties concurrently. Then, it proves the liveness properties one-by-one. (Other engines conduct a sequential proof of each property, both safety and liveness.)

C

Use engine C, which can work well with a combination of C, C2, I, and N when the analysis region becomes too complex for convergence with the default engine mode.

NOTE: While other engines use all the logic in the target's COI, engine C incrementally includes logic and generally converges with a subset of the analysis region.

See also: "set_engineC_optimization"

C2

Use engine C2, which can work well with a combination of C, C2, I, and N when the analysis region becomes too complex for convergence with the default engine mode.

NOTE: While other engines use all the logic in the target's COI, this engine incrementally includes logic and generally converges with a subset of the analysis region.

See also: "set_engineC_optimization"

D

Use engine D, which proves or finds counterexamples for one property at a time (sequential proof of all properties in each task).

See also: set_engineD_optimization

G

Use engine G, which proves or finds counterexamples for one property at a time.

NOTE: Engines G and G2 are often better suited for verifying complex sequential properties such as for data path credit or token management units.

See also: "set_engineG_optimization"

G2

Use engine G2, which proves or finds counterexamples for one property at a time.

NOTE: Engines G and G2 are often better suited for verifying complex sequential properties such as for data path credit or token management units.

See also: "set_engineG_optimization"

H

Use engine H, which proves or finds counterexamples for multiple safety properties in the same task concurrently.

NOTE:
. When you ask engine H to prove a mix of safety and liveness assertions, the tool first proves all safety properties concurrently. Then, it proves the liveness properties one-by-one. (Other engines conduct a sequential proof of each property, both safety and liveness.)
. Running engine H with engines Ht or Hp is not supported.

Hs

Use engine Hs, which proves or finds counterexamples for one property at a time.

NOTE:
. Engine Hs can work on safety or liveness properties.
. Running engine Hs with engines Hts or Hps is not supported.

Hp

Use engine Hp, which is a variant of engine H that is focused on finding proofs for multiple safety properties in the same task concurrently.

NOTE:
. Although engine Hp focuses on finding proofs, it is also capable of finding counterexamples.
. In combination with engine Ht, the two engines collaborate in finding both proofs and counterexamples.
. Running engine Hp with engine H is not supported.

Hps

Use engine Hps, which is a variant of engine H that is focused on finding proofs.

NOTE:
. This engine works on one property at a time (sequential proof of all properties in each task).

. It is a good idea to combine Hps with at least one other engine that is good at finding traces.
. Running engine Hps with engine Hs is not supported.

Ht

Use engine Ht, which is a variant of engine H that is focused on finding counterexamples for multiple safety properties in the same task concurrently.

NOTE:
. In combination with Hp, the two engines collaborate in finding both proofs and counterexamples.
. Running engine Ht with engine H is not supported.

Hts

Use engine Hts, which is a variant of engine H that is focused on finding counterexamples.

NOTE:
. This engine works on one property at a time (sequential proof of all properties in each task).
. Running engine Hts with engine Hs is not supported.

I

Use engine I, which works well with a combination of C, C2, I, and N when the analysis region becomes too complex for convergence with the default engine mode.

NOTE: While other engines use all the logic in the target's COI, this engine incrementally includes logic and generally converges with a subset of the analysis region.

See also: "set_engineD_optimization"

J

Use engine J, which only searches for traces, and will generally not find proofs.

NOTE:
. Engine J searches for traces for multiple safety properties in the same task concurrently, just like engine H.
. Engine J ignores liveness assertions or infinite covers.
. Use engine J in combination with "set_proofgrid_per_engine_max_jobs" or

"set_proofgrid_per_engine_max_local_jobs" to spawn multiple instances of engine J for potentially better performance.
. Unlike some other engines, when engine J's search for traces is successful, it will not necessarily generate the shortest possible traces.
. Engine J might give up during its search. See "set_engineJ_attempts".

See also: "set_engineJ_attempts", "set_engineJ_restarts", "set_engineJ_max_trace_length", and "set_engineJ_seed"

K

Use engine K to find a bounded proof.

NOTE:
. Engine K only searches for traces.
. It will generally not find proofs.
. It proceeds one property at a time.
. It exchanges information with engines C, I, and N during proofs of the same property to speed up the process.
. It iteratively includes logic from the COI, minimizing the amount of logic used while searching.

L

Use engine L to leverage reached cover properties to try to reach otherwise hard-to-reach expressions.

NOTE:
. Engine L ignores liveness assertions or infinite covers.
. As with engines J, M, and N, if engine L finds a trace, it will not necessarily be the shortest possible trace.
. An example application of engine L is to find a deep error state, such as those that occur in post-silicon debugging (PSD) exercises.
. You can also use engine L to increase the coverage percentage of a task with multiple targets. In this case, the tool can use multiple engine L instances (and licenses) to increase coverage.
. Use engine L in combination with "set_proofgrid_per_engine_max_jobs" or "set_proofgrid_per_engine_max_local_jobs" to spawn multiple instances of engine L.

M

Use engine M to find a full proof.

NOTE:

. This engine works best on properties with a small COI and without many complex (non-pin) constraints.
. With this engine, the emphasis is on proving properties that are valid (assertions) or unreachable (covers). It can find traces; however, it is not well suited for that purpose.
. Engine M proves properties sequentially.
. It works well on liveness assertions or infinite covers.
. As with engines J, L, and N, if engine M finds a trace, it will not necessarily be the shortest possible trace.

Mp

Use engine Mp, which is a variant of engine M that is focused on finding proofs for multiple safety properties in the same task concurrently.

NOTE:
. Engine Mp runs multiple rounds, with each round increasing the proof effort.
. Engine Mp works best with a combination of engines M, N, and AM. These engines exchange information to speed up proofs of related properties.
. Although engine Mp focuses on finding proofs, it is also capable of finding counterexamples. However, it is not well suited for that purpose.
- As with engine M, if engine Mp finds a trace, it will not necessarily be the shortest possible trace.

N

Use engine N to find a full proof.

NOTE:
. As with engine M, this engine works best on properties with a small COI and without many complex (non-pin) constraints; however, engine N is not as limited as engine M in this respect.
. As with engine M, the emphasis is on proving properties that are valid (assertions) or unreachable (covers). It can find traces; however, it is not well suited for that purpose.
. Engine N proves properties sequentially.
. It works well on liveness assertions or infinite covers.
. As with engines J, L, and M, if engine N finds a trace, it will not necessarily be the shortest possible trace.
. Engine N is part of the CIK group of engines and will take part in the CIK cooperation.
. Engine N is included in the default engine mode.

AB

AD

AG
AM
Use the specified abstraction engine to realize potentially better results than an engine it is based on.

. AB is a bounded-proof abstraction engine.
  Use this engine mode for cases where you might normally use engine B.
. AD is a proof-search abstraction engine.
  Use this engine mode for cases where you might normally use engine D or I.
. AG is a proof-search, data path abstraction engine.
  Use this engine mode for cases where you might normally use engine G or C.
. AM is a proof-search abstraction engine.
  Use this engine mode for cases where you might normally use engine M or N.

Oh
Use engine Oh, which is focused on finding proofs for all safety properties concurrently.

NOTE:
. Engine Oh can find proofs, and min_length and max_length bounds, but never traces.
. Engine Oh might give up during its search, similar to engine J.
. Engine Oh ignores liveness assertions or infinite covers.

Q3
Use engine Q3, which only searches for traces, and will generally not find proofs.

NOTE:
. Engine Q3 ignores liveness assertions or infinite covers
. Engine Q3 cannot prove that a trace does not exist.
. Engine Q3 searches for traces for multiple safety properties in the same task concurrently, just like engine J.
. Engine Q3 always works on the "check_assumptions" assertions, which you can see in the ProofGrid Manager when they are not shown in the Property Table. See "check_assumptions".
. Use engine Q3 with "random_diversification" "on" in combination with "set_proofgrid_per_engine_max_jobs" or "set_proofgrid_per_engine_max_local_jobs" to spawn multiple instances of engine Q3 for potentially better performance.

```
    . Unlike some other engines, when engine Q3's search for traces is
      successful, it will not necessarily generate the shortest possible
      traces.

    See also: "set_engineQ3_max_trace_length",
    "set_engineQ3_random_diversification", "set_engineQ3_restarts", and
    "set_engineQ3_seed"

R
    Use engine R, which is focused on finding proofs for all safety properties
    concurrently.

    NOTE:
    . Engine R is better at finding proofs (like Hp) than traces.
    . It provides trace attempts, proof attempts, and min_length updates.
    . Engine R uses a proof strategy similar to engine M. It limits resources
      with a timeout and automatically restarts with longer timeouts.
    . This engine can find non-minimal traces.

Tri
    Use engine Tri, which uses a proof strategy similar to the IFV Trident
    engine.

    NOTE:
    . Engine Tri uses several processes at once, default 8, to process one
      property.If you use ProofGrid mode "shell", you will likely need to
      update your script to address this behavior by using the variable
      "%jasper_engine_job_threads" in the command string. For example:

      set_proofgrid_shell
{ /.../bsub -n %jasper_engine_job_threads -R "span[hosts=1]" ..}

    . It provides only trace attempts and min_length updates.

    . It can find non-minimal traces.

U
    Use engine U, which only searches for traces and will generally not find
    proofs.

    NOTE:
    . Engine U searches for traces for multiple safety properties in the same
      task concurrently, just like engine J.
```

```
    . Use engine U in combination with "set_proofgrid_per_engine_max_jobs" or
      "set_proofgrid_per_engine_max_local_jobs" to spawn multiple instances of
      engine U for potentially better performance.
    . Unlike some other engines, when engine U's search for traces is
      successful, it will not necessarily generate the shortest possible
      traces.
    . Engine U ignores liveness assertions or infinite covers.

    See also: "set_engineU_max_trace_length", "set_engineU_seed", and
    "set_proofgrid_engineU_max_jobs".

U2
    Use engine U2, which only searches for traces and will generally not find
    proofs.

    NOTE:
    . Engine U2 cannot prove that a trace does not exist.
    . Engine U2 searches for traces for multiple safety properties in the same
      task concurrently, just like engine J.
    . Use engine U2 in combination with "set_proofgrid_per_engine_max_jobs" or
      "set_proofgrid_per_engine_max_local_jobs" to spawn multiple instances of
      engine U2 for potentially better performance.
    . Unlike some other engines, when engine U2's search for traces is
      successful, it will not necessarily generate the shortest possible
      traces.
    . Engine U2 ignores liveness assertions or infinite covers.

    See also: "set_engineU2_max_trace_length", "set_engineU2_seed",
     "set_engineU2_restarts", and "set_proofgrid_engineU2_max_jobs".

TM
    Use engine TM, which only establishes minimal traces for safety properties
    that have non-minimal traces.

    NOTE:
    . Use TM, preferably together with prefer_shortest to improve results
      achieved by other engines. See also "prove -prefer_shortest" and
      "set_prove_prefer_shortest".
    . See also "check_assumptions" to learn how to establish minimal traces for
      its properties. Engine TM ignores them.

QT
    Use engine QT, which establishes quiet traces for liveness and safety
```

properties that have traces.

NOTE:
. Use QT with "assume -soft -quiet/-noisy/-quiet_from/-max_from", "assume -xprop -soft -quiet/-noisy/-quiet_from/-max_from", and "assume -soft signal/wide_signal/{$stable(signal)}.
. See also "prove -quiet -property" and "prove -soft_constraints/ -soft_low_constraints" for using QT in the prove flow.
. See also "visualize -property" and "visualize -quiet true/-force -soft" for using QT in the Visualize flow.
. See also "check_assumptions" to learn how to establish minimal traces for its properties. Engine QT ignores them.

WHp
Use engine WHp, which is a variant of engine Hp that is focused on finding proofs. WHp uses word-level abstraction techniques.

WHt
Use engine WHt, which is a variant of engine Ht that is focused on finding counterexamples. WHt uses word-level abstraction techniques.

WHps
Use engine WHps, which is a variant of engine Hps that is focused on finding proofs. WHps uses word-level abstraction techniques.

WHs
Use engine WHs, which is a variant of engine H that is focused on finding proofs.

WB
Use engine WB, which is a variant of engine B that is focused on finding counterexamples. WB uses word-level abstraction techniques.

WB1
Use engine WB1, which is a word-level engine focused on finding counterexamples.

WB2
Use engine WB2, which is a word-level engine focused on finding counterexamples.

WB3
Use engine WB3, which is a word-level engine focused on finding counterexamples.

```
WI
    Use engine WI, which is a word-level engine focused on finding
    proofs.

WAM
    Use engine WAM, which is a variant of AM and uses word-level abstraction
    techniques.

WECX1
WECX2
    Use engine WECX1 and WECX2 when comparing 2 designs which are structurally
    similar and there are high number of internal matching nodes.These are word
    level engines focused on finding proofs.

WG2
    Use engine WG2, which is a variant of G2 and uses word-level abstraction
    techniques.

WA1
    Use engine WA1, which is focused on finding proofs for Datapath properties.

WA2
    Use engine WA2, which is focused on finding proofs for Datapath properties
specifically
    around Multipliers

WA3
    Use engine WA3, which is focused on finding proofs for Datapath properties
```

Example

```
set_engine_mode b
prove expr1

set_engine_mode {d g h}
prove -task my_task

// In the following scenario, the tool launches three engine L instances, and
// in addition to the session license, the tool checks out two additional
// licenses. First, engine L picks start=0. Second, engine L picks start=1.
// And third, engine L picks start=3. (The default state start is 0, 1, 3,
// 5, 7, ....)
set_engine_mode {L}
set_proofgrid_per_engine_max_jobs 3
prove -task ...
```

See Also

check_assumptions,
prove,
visualize

set_engine_processes

Specifies the number of distributed engines for engine B and Bm.

Syntax

```
set_engine_processes N
```

Returns

No return values

Default

1

Detailed Description

When "engine_distributed_mode" is on, use "engine_processes" to direct
the "prove" command to replace engines B and Bm each with a portfolio
of N distributed engines, each being a custom variant of engine B or Bm.
In addition, when N>1, the proof runs a distributed manager engine,
DistMgr, used to manage the N custom engines of each portfolio.

NOTE:
. This configuration has no effect if "engine_distributed_mode" is off.
. The portfolio of engines this command enables can be beneficial when
  trying to prove deeper min length bounds or when searching for hard
  to find counterexamples.
. Use "prove -dry_run" to get an upper bound on the number of jobs that
  could be launched by a proof when this configuration is used.
. The DistMgr engine uses up to two cores on the same host.
. The tool does not let you run a portfolio of engine B or Bm together
  with any engine other than B or Bm.
. The configurations "engineB_processes" and "engineBm_processes" take
  precedence over this configuration.

Example

```
set_engine_distributed_mode on
set_engine_processes 32
prove -all -engine_mode {B}
```

See Also

set_engineB_processes,
set_engineBm_processes,
set_engine_distributed_mode

set_engine_solver

Sets the solver used in the Boolean satisfiability decision procedure.

Syntax

```
set_engine_solver (default | solverE | solverF)
```

Returns

No return values

Default

default

Detailed Description

A solver is a decision procedure for the Boolean satisfiability problem. Use this command to specify the solver you want the engines to use.

NOTE:
. This command does not affect all engines. In particular, engines G*, J, R, and Tri are not affected. The tool also ignores this setting when you specify "set_engine_threads" with more than one thread.
. We generally do not recommend you use this command directly. Proof Orchestration under ProofMaster automatically selects among these solvers as part of its orchestration heuristic, and successful ones are reused via ProofMaster PPD files. A solver selection is also embedded in the custom engines reported by orchestration and optionally invoked by the "custom_engine" command.

default
    Select a suitable solver for each engine.

solverE
    Set the standard solver. This is the default solver for many engines.

solverF
    Set solverF, which can be helpful in some cases, particularly when encountering "Internal Limit" from the engines.

Example

```
set_engine_solver solverF
```

See Also

custom_engine,
set_engine_mode,
set_engine_threads

set_engine_threads

Places an upper limit on the number of threads used by each engine.

Syntax

```
set_engine_threads N
```

Returns

No return values

Default

1

Detailed Description

Use this command to place an upper limit on the number of threads used by each engine. Using more than one engine thread might be especially beneficial when working with engines B, Ht, or L on properties that otherwise do not converge; that is, multiple threads can be especially useful with these engines when you are in a bug-hunting mode.

N
: Use at most N threads per engine, where N is an integer number between 1 and 16, inclusive.

  Setting the "engine_threads" limit to an integer larger than 1 might improve proof convergence, but it might also cause some engines to consume more memory than when the limit is set to 1.

Example

```
set_engine_threads 5
```

See Also

set_engine_mode

set_engineB_before_first_mode

Specifies how engine B will treat any cycles before its first trace attempt.

Syntax

```
set_engineB_before_first_mode (skip | check | assume)
```

Returns

No return values

Default

skip

Detailed Description

Use this command to specify how engine B will treat any cycles before its first trace attempt.

skip
: Do not make any assumptions or checks about the existence of traces shorter than "engineB_first_trace_attempt" in any way.

check
: Embed the check for traces shorter than "engineB_first_trace_attempt" into the first trace attempt. This means that if there exist traces shorter than "engineB_first_trace_attempt" which are extendable to that length, the first trace attempt will find a trace, but it is not necessarily the shortest one possible.

assume
: Allow engine B to assume that there is no trace to be found before "engineB_first_trace_attempt". If this assumption holds, this mode is to be preferred over "skip" or "check" because it gives the engine more information. However, if the assumption does not hold, the trace attempts might become overconstrained and might fail to find longer traces even though they exist.

Example

```
set_engineB_before_first_mode assume
```

See Also

set_engineB_before_increment_mode,
set_engineB_first_trace_attempt

set_engineB_before_increment_mode

Specifies how engine B treats any cycles between consecutive trace attempts.

Syntax

```
set_engineB_before_increment_mode (skip | check | assume)
```

Returns

No return values

Default

skip

Detailed Description

Use this command to specify how engine B treats any cycles between consecutive trace attempts.

skip
: Do not make any assumptions or checks about the existence of traces with lengths between trace attempts.

check
: Embed the check for traces shorter than a particular trace attempt "K" but longer than the previous trace attempt into the trace attempt "K". This means that if such a trace exists, trace attempt "K" will find a trace, but it is not necessarily the shortest one possible.

assume
: Allow engine B to assume that there is no trace to be found between the trace attempts that have run. If this assumption holds, this mode is to be preferred over "skip" or "check" because it gives the engine more information. However, if the assumption does not hold, the trace attempts might become overconstrained and might fail to find longer traces even though they exist.

Example

```
set_engineB_before_increment_mode assume
```

See Also

set_engineB_before_first_mode,
set_engineB_first_trace_attempt,
set_engineB_trace_attempt_increment

set_engineB_first_trace_attempt

Specifies the cycle at which engine B will make its first trace attempt.

Syntax

```
set_engineB_first_trace_attempt N
```

Returns

No return values

Default

1

Detailed Description

By default, engine B (like all engines) searches for traces starting with traces of length 1 ("trace attempt 1"). Sometimes, you can achieve better results by skipping a few early attempts, and instead, start at a higher trace attempt. With this command, you can directly specify the first trace attempt engine B will make.

The following scenarios are examples of times when a higher trace attempt might be warranted:

. When slightly longer traces can be found much faster. (Sometimes trace attempt 1 might even be intractable, while at the same time, trace attempt 2 finds a trace immediately.)

. When you want to see a trace that is not too short. This might not work for trivial properties. "assert/cover -set_trace_extension", and "visualize" freeze-and-extend are more reliable ways to see more cycles than what you get by default.

. When prior proofs have established that there are no traces to be found shorter than a certain bound--the min_length bound--then, it is safe to start at a higher attempt in the sense that a minimal trace will still be found.

NOTE:
. Increasing the first trace attempt _can_ shave off some proof time, but it can also have the opposite effect. Perhaps counterintuitively, it might be faster to make all trace attempts from 1 to 50 than it is to make all trace attempts from 20 to 50.

. When skipping the first few trace attempts, it is no longer guaranteed that the shortest possible trace is found.

Example

```
set_engineB_first_trace_attempt 9
```

See Also

set_engineB_before_first_mode,
set_engineB_trace_attempt_increment,
set_first_trace_attempt

set_engineB_processes

Specifies the number of distributed engines for engine B.

Syntax

```
set_engineB_processes N
```

Returns

No return values

Default

1

Detailed Description

When "engine_distributed_mode" is on, use "engineB_processes" to direct the "prove" command to replace engine B with a portfolio of N distributed engines, each being a custom variant of engine B. In addition, when N>1, the proof runs a distributed manager engine, DistMgr, used to manage these N custom engines.

NOTE:
. This configuration has no effect if "engine_distributed_mode" is off.
. The portfolio of engines this command enables can be beneficial when trying to prove deeper min length bounds or when searching for hard to find counterexamples.
. Use "prove -dry_run" to get an upper bound on the number of jobs that can be launched by a proof when this configuration is used.
. The DistMgr engine uses up to two cores on the same host.
. The tool does not let you run a portfolio of engine B together with any other engine except for engine Bm.
. This configuration takes precedence over the configuration "engine_processes".

Example

```
set_engine_distributed_mode on
set_engineB_processes 32
prove -all -engine_mode {B}
```

See Also

set_engineBm_processes,
set_engine_processes,
set_engine_distributed_mode

set_engineB_trace_attempt_increment

Specifies the increment engine B uses between one trace attempt and the next.

Syntax

```
set_engineB_trace_attempt_increment N
```

Returns

No return values

Default

At every consecutive length (1)

Detailed Description

By default, engine B (like all engines) searches for traces at every consecutive length (trace attempt). Sometimes, you can achieve better results by incrementing the trace length more than 1 between every trace attempt. Use this command to specify the increment engine B uses between one trace attempt and the next.

The following scenarios are examples of times when a higher trace attempt increment might be warranted:

. When the design is partly clocked by a regular slow clock and can only trigger the property every k'th cycle.

. When, for hard to understand reasons, the overall progress will be faster using a higher increment.

NOTE:
. As with engineB_first_trace_attempt, there is an element of speculation here with respect to which setting will be fastest. The best way to find out is through observation.

. When skipping trace attempts, it is no longer guaranteed that engine B will find the shortest possible trace.

Example

```
set_engineB_trace_attempt_increment 9
```

See Also

set_engineB_before_first_mode,
set_engineB_first_trace_attempt

set_engineB_trace_attempt_time_limit

Sets a trace attempt time limit for engine B.

Syntax

```
set_engineB_trace_attempt_time_limit time_limit
```

Returns

No return values

Default

0 (unlimited)

Detailed Description

Use this command to specify a trace attempt time limit for engine B.

The tool supports three formats for the time limit:
. ISO8601:   hh':'[mm[':'[ss['.'[fff]]]]]
  Examples: 2:30, 2:, 0:0:23, 2:30:05.235
. Jasper: [dd'd'][hh'h'][mm'm'][ss[.fff]'s']
  Examples: 2h30m, 2h, 23s, 2h30m05.235s
. Seconds: sssss[.fff]['s']
  Examples: 9000s, 7200, 23s, 9005.235s

time_limit
    Use the specified time limit for each engine B trace attempt.

    NOTE:
    . If you do not specify a time unit, the default unit is seconds.
    . To remove the limit, use 0s.

Example

```
set_engineB_trace_attempt_time_limit 20s
```

See Also

set_engineB_trace_attempt_time_limit_incr

set_engineB_trace_attempt_time_limit_incr

Sets a trace attempt time limit increment for engine B.

Syntax

```
set_engineB_trace_attempt_time_limit_incr N
```

Returns

No return values

Default

By default, engine B trace attempt time is unlimited.

Detailed Description

Use this command to specify the time limit increment for engine B when the specified limit for trace attempts expires.

The tool supports three formats for the time limit:

```
. ISO8601:    hh':'[mm[':'[ss['.'[fff]]]]]
  Examples:   2:30, 2:, 0:0:23, 2:30:05.235
. Jasper: [dd'd'][hh'h'][mm'm'][ss[.fff]'s']
  Examples:   2h30m, 2h, 23s, 2h30m05.235s
. Seconds:    sssss[.fff]['s']
  Examples:   9000s, 7200, 23s, 9005.235s
```

N

Use the specified time limit increment when the time limit expires.

NOTE: If you do not specify a time unit, the default unit is seconds.

Example

```
set_engineB_trace_attempt_time_limit_incr 20s
```

See Also

set_engineB_trace_attempt_time_limit

set_engineBm_first_trace_attempt

Specifies the cycle at which engine Bm will make its first trace attempt.

Syntax

```
set_engineBm_first_trace_attempt N
```

Returns

No return values

Default

1

Detailed Description

By default, engine Bm (like all engines) searches for traces starting with traces of length 1 ("trace attempt 1"). Sometimes, you can achieve better results by skipping a few early attempts, and instead, start at a higher trace attempt. With this command, you can directly specify the first trace attempt engine Bm will make.

The following scenarios are examples of times when a higher trace attempt might be warranted:

. When slightly longer traces can be found much faster. (Sometimes trace attempt 1 might even be intractable, while at the same time, trace attempt 2 finds a trace immediately.)

. When you want to see a trace that is not too short. This might not work for trivial properties. "assert/cover -set_trace_extension", and "visualize" freeze-and-extend are more reliable ways to see more cycles than what you get by default.

. When prior proofs have established that there are no traces to be found shorter than a certain bound--the min_length bound--then, it is safe to start at a higher attempt in the sense that a minimal trace will still be found.

NOTE:
. Increasing the first trace attempt _can_ shave off some proof time, but it can also have the opposite effect. Perhaps counterintuitively, it might be faster to make all trace attempts from 1 to 50 than it is to make all trace attempts from 20 to 50.

. When skipping the first few trace attempts, it is no longer guaranteed that the shortest possible trace is found.

Example

```
set_engineBm_first_trace_attempt 9
```

See Also

set_engineBm_trace_attempt_increment,
set_first_trace_attempt

set_engineBm_processes

Specifies the number of distributed engines for engine Bm.

Syntax

```
set_engineBm_processes N
```

Returns

No return values

Default

1

Detailed Description

When "engine_distributed_mode" is on, use "engineBm_processes" to direct the "prove" command to replace engine Bm with a portfolio of N distributed engines, each being a custom variant of engine Bm. In addition, when N>1, the proof will run a distributed manager engine, DistMgr, used to manage these N custom engines.

NOTE:
. This configuration has no effect if "engine_distributed_mode" is off.
. The portfolio of engines this command enables can be beneficial when trying to prove deeper min length bounds or when searching for hard to find counterexamples.
. Use "prove -dry_run" to get an upper bound on the number of jobs that can be launched by a proof when this configuration is used.
. The DistMgr engine uses up to two cores on the same host.
. The tool does not let you run a portfolio of engine Bm together with any other engine except for engine B.
. This configuration takes precedence over the configuration "engine_processes".

Example

```
set_engine_distributed_mode on
set_engineBm_processes 32
prove -all -engine_mode {Bm}
```

See Also

set_engineB_processes,
set_engine_processes,
set_engine_distributed_mode

set_engineBm_trace_attempt_increment

Specifies the increment engine Bm uses between one trace attempt and the next.

Syntax

```
set_engineBm_trace_attempt_increment N
```

Returns

No return values

Default

At every consecutive length (1)

Detailed Description

By default, engine Bm (like all engines) searches for traces at every consecutive length (trace attempt). Sometimes, you can achieve better results by incrementing the trace length more than 1 between every trace attempt. Use this command to specify the increment engine Bm uses between one trace attempt and the next.

The following scenarios are examples of times when a higher trace attempt increment might be warranted:

. When the design is partly clocked by a regular slow clock and can only trigger the property every k'th cycle.

. When, for hard to understand reasons, the overall progress will be faster using a higher increment.

NOTE:
. As with engineBm_first_trace_attempt, there is an element of speculation here with respect to which setting will be fastest. The best way to find out is through observation.

. When skipping trace attempts, it is no longer guaranteed that engine Bm will find the shortest possible trace.

Example

```
set_engineBm_trace_attempt_increment 9
```

See Also

set_engineBm_first_trace_attempt

set_engineBm_trace_attempt_time_limit

Sets a trace attempt time limit for engine Bm.

Syntax

```
set_engineBm_trace_attempt_time_limit time_limit
```

Returns

No return values

Default

0 (unlimited)

Detailed Description

Use this command to specify a trace attempt time limit for engine Bm.

The tool supports three formats for the time limit:

```
. ISO8601:    hh':'[mm[':'[ss['.'[fff]]]]]
  Examples:   2:30, 2:, 0:0:23, 2:30:05.235
. Jasper: [dd'd'][hh'h'][mm'm'][ss[.fff]'s']
  Examples:   2h30m, 2h, 23s, 2h30m05.235s
. Seconds:    sssss[.fff]['s']
  Examples:   9000s, 7200, 23s, 9005.235s
```

time_limit
    Use the specified time limit for each engine Bm trace attempt.

    NOTE:
    . If you do not specify a time unit, the default unit is seconds.
    . To remove the limit, use 0s.

Example

```
set_engineBm_trace_attempt_time_limit 20s
```

See Also

set_engineBm_trace_attempt_time_limit_incr

set_engineBm_trace_attempt_time_limit_incr

Sets a trace attempt time limit increment for engine Bm.

Syntax

```
set_engineBm_trace_attempt_time_limit_incr N
```

Returns

No return values

Default

By default, engine Bm trace attempt time is unlimited.

Detailed Description

Use this command to specify the time limit increment for engine Bm when the specified limit for trace attempts expires.

The tool supports three formats for the time limit:

```
. ISO8601:    hh':'[mm[':'[ss['.'[fff]]]]]
  Examples:   2:30, 2:, 0:0:23, 2:30:05.235
. Jasper: [dd'd'][hh'h'][mm'm'][ss[.fff]'s']
  Examples:   2h30m, 2h, 23s, 2h30m05.235s
. Seconds:    sssss[.fff]['s']
  Examples:   9000s, 7200, 23s, 9005.235s
```

N

Use the specified time limit increment when the time limit expires.

NOTE: If you do not specify a time unit, the default unit is seconds.

Example

```
set_engineBm_trace_attempt_time_limit_incr 20s
```

See Also

set_engineBm_trace_attempt_time_limit

set_engineC_optimization

Sets the optimization mode for engine C.

Syntax

```
set_engineC_optimization (static | dynamic | adaptive)
```

Returns

No return values

Default

static

Detailed Description

Use this command to set the optimization mode for engine C.

static
   Use the standard engine C ordering. This is the default setting.

dynamic
   Use a different engine C ordering.

adaptive
   Predict the best ordering mechanism for the current design and
   use it.

Example

```
set_engineC_optimization adaptive
```

See Also

set_engine_mode

set_engineCG_max_mem

Sets a memory limit for engine C and G data structures.

Syntax

```
set_engineCG_max_mem memory_limit
```

Returns

No return values

Default

4096MB

Detailed Description

Use this command to set a memory limit (in megabytes) for engine C and G data structures. The default engineCG_max_mem limit is 4096MB.

Important things to know:

. In most cases, it is best to set the memory limit to a value greater than 100.
. Keep in mind that total memory consumption for a job will be greater than engineCG_max_mem. Consider setting a limit lower than the total amount of memory you want to use for a job, for example, 20-30 less.

Example

```
set_engineCG_max_mem 500
```

set_engineD_optimization

Specifies the optimization mode for engines D and I.

Syntax

```
set_engineD_optimization (standard | high)
```

Returns

No return values

Default

standard

Detailed Description

Specify the optimization mode for engines D and I.

standard
    Do not optimize engines D and I. This is the default setting.

high
    For every proof attempt made by engines D and I, simplify the task of verification to benefit successive attempts.

Example

```
set_engineD_optimization high
```

set_engineG_optimization

Specifies the optimization mode for engine G.

Syntax

```
set_engineG_optimization (static | dynamic | adaptive)
```

Returns

No return values

Default

static

Detailed Description

Use this command to set the optimization mode for engine G.

static
   Use the standard engineG ordering. This is the default setting.

dynamic
   Use a different engineG ordering.

adaptive
   Predict the best ordering mechanism for the current design and
   use it.

Example

```
set_engineG_optimization adaptive
```

See Also

set_engine_mode

set_engineH_before_first_mode

Sets how engine H treats undetermined cycles before its first trace attempt.

Syntax

```
set_engineH_before_first_mode (skip | assume)
```

Returns

No return values

Default

skip

Detailed Description

Use this command to set how engine H treats any undetermined cycles before its first trace attempt.

skip
    Do not make any assumptions or checks about the existence of traces shorter than "engineH_first_trace_attempt" in any way.

assume
    Allow engine H to assume that there is no trace to be found before "engineH_first_trace_attempt". If this assumption holds, or if this is going to be independently checked, this mode is to be preferred over "skip" because it gives the engine more information. However, if the assumption does not hold, the trace attempts might become overconstrained and might fail to find longer traces even though they exist - but only when the property must be violated (or covered) also in an earlier, now assumed, cycle.

NOTE:
. Even with "assume", all results, including bounds, will always be accurate.
. This configuration applies only to Ht and Hts. Other H* variants are unaffected.
. This configuration is inconsequential for cycles already known (via min_length bound) not to have traces.

Example

```
set_engineH_before_first_mode skip
```

See Also

set_engineB_before_first_mode
set_engineH_before_increment_mode,
set_engineH_first_trace_attempt

set_engineH_before_increment_mode

Sets how engine H treats undetermined cycles between consecutive trace attempts.

Syntax

```
set_engineH_before_increment_mode (skip | assume)
```

Returns

No return values

Default

skip

Detailed Description

Use this command to specify how engine H treats any undetermined cycles between consecutive trace attempts.

skip
    Do not make any assumptions or checks about the existence of traces with lengths between trace attempts.

assume
    Allow engine H to assume that there is no trace to be found between the trace attempts that have run. If this assumption holds, or if this is going to be independently checked, this mode is to be preferred over "skip" because it gives the engine more information. However, if the assumption does not hold, the trace attempts might become overconstrained and might fail to find longer traces even though they exist - but only when the property must be violated (or covered) also in an earlier, now assumed, cycle.

NOTE:
. Even with "assume", all results, including bounds, will always be accurate.
. This configuration applies only to Ht and Hts. Other H* variants are unaffected.
. This configuration is inconsequential for cycles already known (via min_length bound) not to have traces.

Example

```
set_engineH_before_increment_mode skip
```

See Also

set_engineB_before_increment_mode,
set_engineH_before_first_mode,
set_engineH_first_trace_attempt,
set_engineH_trace_attempt_increment

set_engineH_first_proof_attempt

Specifies the cycle at which engines will make their first proof attempt.

Syntax

```
set_engineH_first_proof_attempt N
```

Returns

No return values

Default

1

Detailed Description

By default, engines search for proof starting at attempt 1 ("proof attempt 1"). Sometimes, you can achieve better results by skipping a few early attempts, and instead, start at a higher attempt. With this command, you can directly specify the first proof attempt engines will make.

NOTE:

. The command only affects engines H, Hp, and Hps.

. The effective proof attempt depends on many factors, and it is generally hard to predict.

Example

```
set_engineH_first_proof_attempt 3
```

set_engineH_first_trace_attempt

Specifies the cycle at which engine Ht* and Hps make the first trace attempt.

Syntax

```
set_engineH_first_trace_attempt N
```

Returns

No return values

Default

1

Detailed Description

By default, engine H* (like all engines) searches for traces starting with traces of length 1 ("trace attempt 1"). Sometimes, you can achieve better results by skipping a few early attempts, and instead, start at a higher trace attempt. With this command, you can directly specify the first trace attempt engine Ht* and Hps will make.

The following scenarios are examples of times when a higher trace attempt might be warranted:

. When slightly longer traces can be found much faster. (Sometimes trace attempt 1 might even be intractable, while at the same time, trace attempt 2 finds a trace immediately.)

. When you want to see a trace that is not too short. This may not work for trivial properties. "assert/cover -set_trace_extension", and "visualize freeze-and-extend" are more reliable ways to see more cycles than what you may get by default.

. When prior proofs have established that there are no traces to be found shorter than a certain bound--the min_length bound--then, it is safe to start at a higher attempt in the sense that a minimal trace will still be found.

NOTE:
. Increasing the first trace attempt _can_ shave off some proof time, but it can also have the opposite effect. Perhaps counterintuitively, it might be faster to make all trace attempts from 1 to 50 than it is to make all trace attempts from 20 to 50.

. When skipping the first few trace attempts, it is no longer guaranteed that the shortest possible trace is found.

. This command is effective with engines Ht, Hts, and Hps. Other engines, including other H* engines, ignore this setting.

. This configuration is inconsequential for cycles already known (via min_length bound) not to have traces.

Example

```
set_engineH_first_trace_attempt 9
```

See Also

,

set_engineB_first_trace_attempt,
set_engineBm_first_trace_attempt,
set_first_trace_attempt

set_engineH_last_proof_attempt

Specifies the cycle at which engines will make their last proof attempt.

Syntax

```
set_engineH_last_proof_attempt N
```

Returns

No return values

Default

0

Detailed Description

Use this command to set the limit for proof attempts made.

NOTE:

. The command only affects engines H, Hp, and Hps.

. The effective proof attempt depends on many factors, and it is generally hard to predict.

Example

```
set_engineH_last_proof_attempt 3
```

See Also

set_engineH_first_proof_attempt,
set_first_trace_attempt,
set_max_trace_length

set_engineH_restart

Restarts engine H's max length search when another engine reports a proof.

Syntax

```
set_engineH_restart (on | off)
```

Returns

No return value

Default

on

Detailed Description

While engine H attempts to find a max length for a property, other engines might report findings that would be beneficial to its search. For example, another engine's findings might let engine H discover a max length at a lower depth. "engineH_restart" enables engine H to restart its search for max lengths whenever another engine reports a proven property.

NOTE: Restarting might lead to less optimal results for easier properties.

Example

```
set_engineH_restart off
```

set_engineH_trace_attempt_increment

Specifies the increment engine H uses between one trace attempt and the next.

Syntax

```
set_engineH_trace_attempt_increment N
```

Returns

No return values

Default

At every consecutive length (1)

Detailed Description

By default, engine H (like all engines) searches for traces at every consecutive length (trace attempt). Sometimes you can achieve better results by incrementing the trace length more than 1 between every trace attempt. Use this command to specify the increment engine H uses between one trace attempt and the next.

The following scenarios are examples of times when a higher trace attempt increment might be warranted:

. When the design is partly clocked by a regular slow clock and can only trigger the property every k'th cycle.

. When, for hard to understand reasons, the overall progress will be faster using a higher increment.

NOTE:
. This configuration applies only to Ht and Hts. Other H* variants are unaffected.

. As with engineH_first_trace_attempt, there is an element of speculation here with respect to which setting will be fastest. The best way to find out is through observation.

. When skipping trace attempts, it is no longer guaranteed that engine H will find the shortest possible trace.

. This configuration is inconsequential for cycles already known (via min_length bound) not to have traces.

Example

```
set_engineH_trace_attempt_increment 9
```

See Also

set_engineH_before_first_mode,
set_engineH_first_trace_attempt

set_engineH_trace_attempt_time_limit

Sets a trace attempt time limit for engine H.

Syntax

```
set_engineH_trace_attempt_time_limit time_limit
```

Returns

No return values

Default

0 (unlimited)

Detailed Description

Use this command to specify a trace attempt time limit for engine H.

When the limit expires, engineH will proceed to the next trace attempt. If "prove_verbosity" is 8 or above, a proof message will indicate the trace attempt time-out, and will include the currently effective limit, see "set_engineH_trace_attempt_time_limit_incr".

The tool supports three formats for the time limit:
. ISO8601: hh':'[mm[':'[ss['.'[fff]]]]]
  Examples: 2:30, 2:, 0:0:23, 2:30:05.235
. Jasper: [dd'd'][hh'h'][mm'm'][ss[.fff]'s']
  Examples: 2h30m, 2h, 23s, 2h30m05.235s
. Seconds: sssss[.fff]['s']
  Examples: 9000s, 7200, 23s, 9005.235s

time_limit
    Use the specified time limit for each engine H trace attempt.

    NOTE:
    . If you do not specify a time unit, the default unit is seconds.
    . To remove the limit, use 0s.

NOTE:
. This configuration applies only to Ht and Hts. Other H* variants are unaffected.

Example

```
set_engineH_trace_attempt_time_limit 20s
```

See Also

set_engineH_trace_attempt_time_limit_incr,
set_prove_verbosity

set_engineH_trace_attempt_time_limit_incr

Sets a trace attempt time limit increment for engine H.

Syntax

```
set_engineH_trace_attempt_time_limit_incr N
```

Returns

No return values

Default

0s

Detailed Description

Use this command to specify the time limit increment for engine H when the specified limit for trace attempts expires.

The tool supports three formats for the time limit:
. ISO8601: hh':'[mm[':'[ss['.'[fff]]]]]
  Examples: 2:30, 2:, 0:0:23, 2:30:05.235
. Jasper: [dd'd'][hh'h'][mm'm'][ss[.fff]'s']
  Examples: 2h30m, 2h, 23s, 2h30m05.235s
. Seconds: sssss[.fff]['s']
  Examples: 9000s, 7200, 23s, 9005.235s

N
    Use the specified time limit increment when the time limit expires.

    NOTE: If you do not specify a time unit, the default unit is seconds.

NOTE:
. This configuration applies only to Ht and Hts. Other H* variants are unaffected.
. This configuration has an effect only when "engineH_trace_attempt_time_limit" is not 0 (unlimited).

Example

```
set_engineH_trace_attempt_time_limit_incr 20s
```

See Also

set_engineH_trace_attempt_time_limit

set_engineJ_attempts

Sets number of attempts to solve constraints at each cycle before restarting.

Syntax

```
set_engineJ_attempts N
```

Returns

No return values

Default

1000

Detailed Description

Use this command to define the maximum number of attempts engine J will make at every cycle to satisfy all assumptions, Visualize configurations, and latch equations. If engine J fails to satisfy all assumptions within this bound, it restarts.

N
    Make the specified positive integer attempts at every cycle to satisfy all constraints.

Note: Use this command with "set_engineJ_restarts" to get a bit further when you have nontrivial assumptions, configurations, or latches and you get "WARNING (WAS019): Assumptions too hard to satisfy, gave up." However, this command will not help if the assumptions or Visualize configurations or latches are too complex. The only choice then is to switch to another engine mode.

Example

```
prove -engine J -all
WARNING (WAS019): Assumptions too hard for engine J to satisfy; gave up.

set_engineJ_attempts 200
set_engineJ_restarts 200

prove -engine J -all
```

See Also

set_engine_mode,
set_engineJ_restarts

set_engineJ_max_trace_length

Specifies an upper limit on the length of traces searched for by engine J.

Syntax

```
set_engineJ_max_trace_length N
```

Returns

No return values

Default

200

Detailed Description

By default, the maximum trace length for engine J proofs is 200 cycle.
Use this command to change the default. Use 0 to impose no limit.

N
    Limit engine J traces to the specified nonnegative integer cycle.

    NOTE: To remove the limit, use 0.

NOTE:

. Engine J also honors the max_trace_length as an upper
  limit on the length of the traces searched for. The effective
  max_length for engine J is thus the minimum (max_trace_length or
  engineJ_max_trace_length).

. Engine J does not make an exhaustive search for traces at every cycle
  and might generate very long traces. Or it might continue to search
  deeper and deeper even though there are shorter traces to be found. Use this
  command with "set_engineJ_restarts" to ensure that engine J
  spends the time searching for traces with lengths you want to deal
  with. Keep in mind that shorter traces are often easier to interpret.

Example

```
prove -engine J -all
Max trace length reached.

get_engineJ_max_trace_length

set_engineJ_max_trace_length 5000
get_engineJ_seed

set_engineJ_seed 52243c71906b2166
prove -engine J -all
```

See Also

set_engine_mode,
set_engineJ_restarts,
set_max_trace_length

set_engineJ_migrate

Enables engine job handover to Q3 if the engineJ_attempts limit is exceeded.

Syntax

```
set_engineJ_migrate (on | off)
```

Returns

No return values

Default

off

Detailed Description

Use this command to instruct the tool to change engine jobs from engine J to Q3 upon reaching the engineJ_attempts limit instead of exiting. Engine J tends to fail due to its weak handling of constraints, but when it works, it tends to be much faster than Q3. Q3, on the other hand, has the ability to reliably solve constraints. The migration (that is, handover) provides a convenient way to get the best of both.

NOTE:
. If you run engine J with migrate=on, do not run it in parallel with Q3. It would be wasteful.
. The reported identity of the engine job does not change when it migrates to Q3. That is, the engine job keeps its x.J label, and the report shows that the results are from J.

Example

```
prove -engine J -all
engineJ_attempts
set_engineJ_migrate on
prove  -engine J -all
cex
```

See Also

set_engine_mode,
set_engineJ_attempts,
set_engineJ_restarts

set_engineJ_restarts

Specifies the number of restarts engine J makes before abandoning the proof.

Syntax

```
set_engineJ_restarts N
```

Returns

No return values

Default

1000

Detailed Description

Use this command to define the number of restarts engine J makes before abandoning the proof. A restart means starting again with trace attempt 1 and upwards re-searching for short traces. Two events trigger restart:

- Failing to satisfy all constraints at a cycle within the specified number of attempts
- Reaching max_trace_length

At every restart, the tool issues a message in the Proof messages tab that reports the current seed, which is 16 hexadecimal characters.

N
    Limit engine J restarts to the specified nonnegative integer attempts.

NOTE: You can reuse the reported seed to start from that state in a later trace search; however, using the seed only works as long as the proof setup is identical, including the number of undetermined properties.

Example

```
prove -engine J -all
J: Restart 1000, Seed b45785b0fdf4a5ce
// no trace found. Retry with changed settings:

get_engineJ_restarts
set_engingJ_restarts 2000

get_engineJ_seed
set_engineJ_seed b45785b0fdf4a5ce

prove -engine J -all
```

See Also

set_engine_mode,
set_engineJ_attempts,
set_engineJ_max_trace_length,
set_engineJ_seed,
set_max_trace_length

set_engineJ_seed

Specifies a hexadecimal setting for the seed used by engine J.

Syntax

```
set_engineJ_seed hex_string
```

Returns

No return values

Default

000000000000000e

Detailed Description

Use this command to supply a hexadecimal setting for the seed that determines how engine J searches for traces. 0 is a special value that allows engine J to pick an indeterministic seed. Other values give deterministic behavior. If you want the tool to issue a message in the Proof messages tab that reports the current seed at every restart, you must specify a prove_verbosity >=7. (See "set_prove_verbosity" or "prove...-verbosity".) The example shows how you can use seed information.

hex_string
    Use "hex_string" as the seed for engine J.

Example

In the example below, engine J finds a hard trace after a long time and after 950 restarts. Now you want to replay the proof. By picking the seed that was used at the last restart as the initial seed, you will find the trace again without a single restart.

```
prove -engine J -all
J: Restart 950, Seed be9f1153b0d0fa36

set_engineJ_seed be9f1153b0d0fa36
prove  -engine J -all
```

See Also

set_engine_mode,
set_engineJ_restarts

set_engineJ_single_property

Selects multi-/single- property mode for engine J.

Syntax

```
set_engineJ_single_property (true | false)
```

Returns

No return values

Default

false

Detailed Description

Use this command to control whether engine J should prove all properties within a task in parallel (false) or a single property at a time (true). Use this command if engine J does not show good performance proving tasks with multiple properties. It might improve performance.

Example

```
set_engineJ_single_property true
```

See Also

set_engine_mode

set_engineL_asserts_are_intermediate

Specifies whether asserts should be treated as sources of intermediate states.

Syntax

```
set_engineL_asserts_are_intermediate (on | off)
```

Returns

No return values

Default

on

Detailed Description

Use this command to specify whether engine L will treat assertions as potential sources of intermediate targets along with covers.

Example

```
set_engineL_asserts_are_intermediate off
```

set_engineL_cache

Controls the use of engine L cache.

Syntax

```
set_engineL_cache (on | off)
```

Returns

No return values

Default

off

Detailed Description

Use this command to specify whether engine L will use a cover point cache when you run "prove" or "visualize".

When "engineL_cache" is on, the tool retrieves cached cover points from earlier engine L runs that had the same set of properties and stores all intermediate cover points found by engine L in the current run. The cache is located in the project directory.

NOTE: If the set of properties in the current run differs from earlier runs, the tool does not retrieve any cached information.

When the tool retrieves cached information, it prints a log message as follows:

```
    0.L: Loaded 42 cached start states
```

When the tool stores information in the cache, it prints a log message as follows:

```
    0.L: Cached 17 start states
```

Use proof simplification caching (set_cache_proof_simplification on) to increase the impact of engine L caching. To clear the cache, use the command "clear -engineL_cache".

Example

```
set_engineL_cache on
```

See Also

prove,
set_cache_proof_simplification,
visualize

set_engineL_generate_trails

Enables trail generation.

Syntax

```
set_engineL_generate_trails (on | off)
```

Returns

No return value

Default

on

Detailed Description

Enables generation of trails by engine L. A trail is the sequence of intermediate properties used to find a trace for a particular property. You can later retrieve the trail for a property by running the command "get_property_info -list engineL_trail".

Example

```
set_engineL_generate_trails off
```

See Also

get_property_info

set_engineL_max_segment_length

Sets the segment length limit.

Syntax

set_engineL_max_segment_length list_of_natural

Returns

No return values

Default

No limit

Detailed Description

By default, engine L searches from a selection of the determined start states for new traces. The max_segment_length variable controls how many trace attempts the tool makes from these start states before replacing the start states. If the engine cycles through all start states without finding new traces within the limit, the tool automatically increases the limit.

NOTE: This command accepts a list of segment length limits, one for each engine L job. If the list is shorter than the current proofgrid_per_engine_max_jobs, the last engine jobs will all use the last value in engineL_max_segment_length. In particular, if you provide a single value to the command, all engine jobs use that segment length limit.

Example

set_engineL_max_segment_length 5

set_engineL_random_diversification

Diversifies engine L instances by introducing randomness.

Syntax

```
set_engineL_random_diversification (on | off)
```

Returns

No return values

Default

on

Detailed Description

When using several engine L instances (multiple licenses), this setting can increase the chance of reaching deep covers by introducing more diversification in the search between the different engine L instances.

Example

```
set_engineL_random_diversification off
```

set_engineL_search_mode

Specifies whether to target properties in order or in parallel.

Syntax

```
set_engineL_search_mode (ordered | parallel | N)
```

Returns

No return values

Default

parallel

Detailed Description

Use this command to specify how engine L will target properties.

ordered
: Target the specified undetermined properties left to right, according to their order in the "prove" command.

parallel
: Target the specified undetermined properties in parallel. This is the default setting.

N
: Target the specified number of undetermined properties in parallel at each attempt. "N" must be a positive number.

Example

```
// With the following commands, the tool tries to find a trace to p1,
// then to p5, and finally, to p3.
set_engine_mode L
set_engineL_search_mode ordered
prove -property p1 p5 p3

// With the following commands, the tool tries to find a trace to p1 or
// c1, and depending on which one of them is found first, it continues
// with p1 and c2 or c1 and c2.
set_engine_mode L
set_engineL_search_mode 2
prove -property p1 c1 c2
```

set_engineL_seed

Specifies a hexadecimal setting for the seed used by engine L.

Syntax

```
set_engineL_seed hex_string
```

Returns

No return values

Default

00000001

Detailed Description

Use this command to supply a hexadecimal setting for the seed that determines how engine L searches for traces when random_diversification is on. 0 is a special value that allows engine L to pick an indeterministic seed. Other values give deterministic behavior. If you want the tool to issue a message in the Proof Messages tab that reports the current seed at every restart, you must specify a prove_verbosity >=5. (See "set_prove_verbosity" or "prove... -verbosity".) The example shows how you can use seed information.

hex_string
    Use "hex_string" as the seed for engine L.

Example

```
set_engineL_seed be9f1153
prove  -engine L -all
```

See Also

set_engine_mode,
set_engineL_random_diversification

set_engineL_segment_time_limit

Sets the segment time limit.

Syntax

```
set_engineL_segment_time_limit list_of_time_limits
```

Returns

No return values

Default

2h

Detailed Description

By default, engine L searches from a selection of the determined start states for new traces. The "segment_time_limit variable" controls how much time the tool spends searching from these start states before replacing the start states. If the engine cycles through all start states without finding new traces within the limit, the tool automatically increases the limit.

The tool supports three formats for the time limit:

```
. ISO8601:    hh':'[mm[':'[ss['.'[fff]]]]]
  Examples:   2:30, 2:, 0:0:23, 2:30:05.235
. Jasper: [dd'd'][hh'h'][mm'm'][ss[.fff]'s']
  Examples:   2h30m, 2h, 23s, 2h30m05.235s
. Seconds:    sssss[.fff]['s']
  Examples:   9000s, 7200, 23s, 9005.235s
```

NOTE: This command accepts a list of segment time limits, one for each engine L job. If the list is shorter than the current proofgrid_per_engine_max_jobs, the last engine jobs will all use the last value in engineL_segment_time_limit. In particular, if you provide a single value to the command, all engine jobs use that segment time limit.

Example

```
set_engineL_segment_time_limit 10m
```

set_engineL_state_removal

Controls start state use.

Syntax

```
set_engineL_state_removal (fifo | lru)
```

Returns

No return values

Default

lru

Detailed Description

Use this command to change the heuristics for start state selection in engine L. From every trace, the engine picks a start state, and this command controls the heuristics used to decide which subset of the discovered start states to use. With "fifo", engine L uses the last found start states, while with "lru", it chooses the start states that have been the most fruitful in the past. This setting has no effect unless you are using more than one license.

Example

```
set_engineL_state_removal fifo
```

See Also

set_engineL_tail_length

set_engineL_tail_length

Specifies a starting point that is N cycles before the end of found traces.

Syntax

```
set_engineL_tail_length N
```

Returns

No return values

Default

1

Detailed Description

By default, engine L uses one cycle before the last cycle of a trace as the start state for a new search. Use this command to specify a start state that is N number of cycles before the end of an existing trace. A higher value can help with dead-end states, where it is not possible to respect all the assumptions moving forward. This command helps make the search "wider" as the tool can use more than the concrete state from the trace. However, higher values cost more in terms of the time they take as the engine will potentially have to do N extra trace attempts.

NOTE: This command accepts a list of tail lengths, one for each engine L job. If the list is shorter than the current proofgrid_per_engine_max_jobs, the last engine jobs will all use the last value in engineL_tail_length. In particular, if you provide a single value for the tail length, all engine jobs use that tail length.

Example

```
set_engineL_tail_length 2

# Vary tail lengths for multiple ProofGrid engine jobs
set_proofgrid_per_engine_max_jobs 4
set_engineL_tail_length "1 2 1 2"
```

See Also

set_engineL_state_removal

set_engineM_optimization

Specifies the optimization mode for engine M.

Syntax

```
set_engineM_optimization (on | off)
```

Returns

No return values

Default

on

Detailed Description

With this setting on, which is the default, engine M converges faster on proofs in some cases. However, this setting can make engine M slower to find counterexamples, particularly long and non-minimal ones.

Example

```
set_engineM_optimization off
```

set_engineQ3_max_trace_length

Specifies upper trace length limit for searches by Q3 with diversification on.

Syntax

```
set_engineQ3_max_trace_length N
```

Returns

No return values

Default

200

Detailed Description

By default, the maximum trace length (when diversification is on) for engine Q3 is 200 cycles. Use this command to change the default. Use 0 to impose no limit.

N
    Limit engine Q3 traces to the specified nonnegative integer cycle.

    NOTE: To remove the limit, use 0.

NOTE:

. When engineQ3_random_diversification is off, this setting has no effect.

. Engine Q3 also honors the max_trace_length as an upper limit on the length of traces searched for. The effective max_length for engine Q3 is thus the minimum (max_trace_length or engineQ3_max_trace_length).

. Engine Q3 does not make an exhaustive search for traces at every cycle and might generate very long traces. Or it might continue to search deeper and deeper even though there are shorter traces to be found. Use this command with "set_engineQ3_restarts" to ensure that engine Q3 spends the time searching for traces with lengths you want to deal with. Keep in mind that shorter traces are often easier to interpret.

Example

```
set_engineQ3_random_diversification on
prove -engine Q3 -all
Max trace length reached.

get_engineQ3_max_trace_length

set_engineQ3_max_trace_length 5000
get_engineQ3_seed

set_engineQ3_seed 52243c71
prove -engine Q3 -all
```

See Also

set_engine_mode,
set_engineQ3_random_diversification,
set_engineQ3_restarts,
set_max_trace_length

set_engineQ3_random_diversification

Enables or disables random diversification for engine Q3.

Syntax

```
set_engineQ3_random_diversification (on | off)
```

Returns

No return values

Default

on

Detailed Description

Use this command to enable or disable random diversification for engine Q3. When random diversification is on, engine Q3 can be further controlled using the following settings:

. engineQ3_random_diversification_factor
. engineQ3_seed
. engineQ3_max_trace_length
. engineQ3_restarts

These settings have no effect when diversification is off. When diversification is on, multiple engine Q3 instances can run in parallel.

Example

```
set_engineQ3_random_diversification on
prove -engine Q3 -per_engine_max_jobs 10 -all
2.Q3: Restart 950, Seed be9f1153

set_engineQ3_random_diversification be9f1153
prove  -engine Q3 -all
```

See Also

set_engine_mode,
set_engineQ3_random_diversification,
set_engineQ3_restarts

set_engineQ3_random_diversification_factor

Specifies the random_diversification_factor used by engine Q3.

Syntax

set_engineQ3_random_diversification_factor 0.0 - 1.0

Returns

No return values

Default

0.5

Detailed Description

Use this command to set the random_diversification_factor used by engine Q3 when random_diversification is on. A high value gives more diversification, but potentially much slower progress. A very low value is probably faster, but will perhaps give too little diversification for restarts and multiple jobs to make sense. Compare the State Signatures between different jobs, and/or different restarts to see how much divergence the diversification accomplishes.

Example

```
set_engineQ3_random_diversification on
set_engineQ3_random_diversification_factor 0.001
prove -engine Q3 -all
0.Q3: Restart 12, Seed 7, 0.1%
0.Q3: Cycle 106, State Sig: 17a49d99
1.Q3: Cycle 106, State Sig: 17a49d99
2.Q3: Cycle 106, State Sig: 17a49d99

set_engineQ3_random_diversification_factor 0.4
prove -engine Q3 -all
0.Q3: Restart 12, Seed 7, 40.0%
0.Q3: Cycle 106, State Sig: 17a49d99
1.Q3: Cycle 106, State Sig: 17ba23fe
2.Q3: Cycle 106, State Sig: 13235696
```

See Also

set_engine_mode,
set_engineQ3_random_diversification,
set_engineQ3_restarts

set_engineQ3_restarts

Specifies the number of restarts engine Q3 makes before abandoning the proof.

Syntax

```
set_engineQ3_restarts N
```

Returns

No return values

Default

1000

Detailed Description

Use this command to define the number of restarts engine Q3 makes when random_diversification is on before abandoning the proof. A restart means starting again with trace attempt 1 and upwards re-searching for short traces. One event triggers restart:

Reaching max_trace_length

At every restart, the tool issues a message in the Proof Messages tab that reports the current seed, which is 8 hexadecimal characters.

N
    Limit engine Q3 restarts to the specified nonnegative integer attempts.

NOTE: You can reuse the reported seed to start from that state in a later trace search; however, using the seed only works as long as the proof setup is identical, including the number of undetermined properties.

Example

```
set_engineQ3_random_diversification on
prove -engine Q3 -all
Q3: Restart 1000, Seed b45785b0
// No trace found. Retry with changed settings:

get_engineQ3_restarts
set_engingQ3_restarts 2000

get_engineQ3_seed
set_engineQ3_seed b45785b0

prove -engine Q3 -all
```

See Also

set_engine_mode,
set_engineQ3_max_trace_length,
set_engineQ3_random_diversification,
set_engineQ3_seed,
set_max_trace_length

set_engineQ3_seed

Specifies a hexadecimal setting for the seed used by engine Q3.

Syntax

```
set_engineQ3_seed hex_string
```

Returns

No return values

Default

00000007

Detailed Description

Use this command to supply a hexadecimal setting for the seed that determines how engine Q3 searches for traces when random_diversification is on. 0 is a special value that allows engine Q3 to pick an indeterministic seed. Other values give deterministic behavior. If you want the tool to issue a message in the Proof Messages tab that reports the current seed at every restart, you must specify a prove_verbosity >=5. (See "set_prove_verbosity" or "prove... -verbosity".) The example shows how you can use seed information.

hex_string
    Use "hex_string" as the seed for engine Q3.

Example

In the example below, engine Q3 finds a hard trace after a long time and after 950 restarts. Now you want to replay the proof. By picking the seed that was used at the last restart as the initial seed, you will find the trace again without a single restart.

```
set_engineQ3_random_diversification on
prove -engine Q3 -all
Q3: Restart 950, Seed be9f1153

set_engineQ3_seed be9f1153
prove  -engine Q3 -all
```

See Also

set_engine_mode,
set_engineQ3_random_diversification,
set_engineQ3_restarts

set_engineQT_attempt_time_limit

Sets a quiet attempt time limit for engine QT.

Syntax

```
set_engineQT_attempt_time_limit time_limit
```

Returns

No return values

Default

0 (unlimited)

Detailed Description

Use this command to specify a quiet attempt time limit for engine QT.

The tool supports three formats for the time limit:
. ISO8601: hh':'[mm[':'[ss['.'[fff]]]]]
  Examples: 2:30, 2:, 0:0:23, 2:30:05.235
. Jasper: [dd'd'][hh'h'][mm'm'][ss[.fff]'s']
  Examples: 2h30m, 2h, 23s, 2h30m05.235s
. Seconds: sssss[.fff]['s']
  Examples: 9000s, 7200, 23s, 9005.235s

time_limit
    Use the specified time limit for each engine QT quiet attempt.

    NOTE:
    . If you do not specify a time unit, the default unit is seconds.
    . To remove the limit, use 0s.
    . By specifying a time limit on a quiet attempt, some soft constraints are
      not fully considered for satisfaction; hence, some soft constraints might
      not be satisfied due to the time limit.

Example

```
set_engineQT_attempt_time_limit 20s
```

set_engineQT_optimization

Specify whether to enable the optimization for engine QT.

Syntax

```
set_engineQT_optimization (on | off)
```

Returns

No return values

Default

on

Detailed Description

With this setting on, which is the default, engine QT simplifies the tasks of QT iterations to benefit successive attempts.

Example

```
set_engineQT_optimization on
```

set_engineU2_max_trace_length

Specifies an upper limit for the trace length for searches by engine U2.

Syntax

```
set_engineU2_max_trace_length N
```

Returns

No return values

Default

200

Detailed Description

By default, the maximum trace length for engine U2 is 200 cycles. Use this command to change the default. Use 0 to impose no limit.

N
    Limit engine U2 traces to the specified nonnegative integer cycle.

    NOTE: To remove the limit, use 0.

NOTE:

. Engine U2 also honors the max_trace_length as an upper limit on the length of traces searched for. The effective max_length for engine U2 is thus the minimum (max_trace_length or engineU2_max_trace_length).

. Engine U2 does not make an exhaustive search for traces at every cycle and might generate very long traces. Or it might continue to search deeper and deeper even though there are shorter traces to be found. Use this command to ensure that engine U2 spends the time searching for traces with lengths you want to deal with. Keep in mind that shorter traces are often easier to interpret.

Example

```
set_engineU2_max_trace_length 50
```

See Also

set_max_trace_length

set_engineU2_restarts

Specifies the number of restarts engine U2 makes before abandoning the proof.

Syntax

```
set_engineU2_restarts N
```

Returns

No return values

Default

1000

Detailed Description

Use this command to define the number of restarts engine U2 makes before abandoning the proof. A restart means starting again with trace attempt 1 and upwards re-searching for traces. One event triggers restart:

Reaching max_trace_length

At every restart, the tool issues a message in the Proof Messages tab that reports the current seed, which is 8 hexadecimal characters.

N
: Limit engine U2 restarts to the specified nonnegative integer attempts.

  NOTE: You can reuse the reported seed to start from that state in a later trace search; however, using the seed only works if the proof setup is identical, including the number of undetermined properties.

  NOTE: The seed might not be reported in some trace searches (because it cannot be reused); however, the seed can be used (and will be reported always) at the first upward search.

Example

```
set_engineU2_restarts 10000
prove -engine U2 -all
U2: Restarted 10000 times.
// No trace found. Retry with changed settings:

set_engineU2_restarts 20000
prove -engine U2 -all
```

See Also

set_engineU2_max_trace_length,
set_engineU2_seed,
set_max_trace_length

set_engineU2_seed

Specifies a hexadecimal setting for the seed used by engine U2.

Syntax

```
set_engineU2_seed hex_string
```

Returns

No return values

Default

00000001

Detailed Description

Use this command to supply a hexadecimal setting for the seed that determines how engine U2 searches for traces. 0 is a special value that allows engine U2 to pick an indeterministic seed. Other values give deterministic behavior.

With the default prove verbosity, which is 6, the tool issues a message in the Proof Messages tab that reports the current seed at every restart. The example shows how you can use seed information.

NOTE: If you run "set_engineU2_seed" and specify multiple jobs, each job uses a different seed, which is chosen deterministically as a function of the seed you specify and the engine job number.

hex_string
    Use "hex_string" as the seed for engine U2.

Example

```
prove -engine_mode U2 -all

// No trace found for some asserts or covers. Retry with changed
// settings:

get_engineU2_seed

set_engineU2_seed 52243c71
prove -engine_mode U2 -all
```

set_engineU_max_trace_length

Specifies an upper limit for the trace length for searches by engine U.

Syntax

```
set_engineU_max_trace_length N
```

Returns

No return values

Default

100

Detailed Description

By default, the maximum trace length for engine U is 100 cycles. Use this command to change the default. Use 0 to impose no limit.

N
    Limit engine U traces to the specified nonnegative integer cycle.

    NOTE: To remove the limit, use 0.

NOTE:

. Engine U also honors the max_trace_length as an upper limit on the length of traces searched for. The effective max_length for engine U is thus the minimum (max_trace_length or engineU_max_trace_length).

. Engine U does not make an exhaustive search for traces at every cycle and might generate very long traces. Or it might continue to search deeper and deeper even though there are shorter traces to be found. Use this command to ensure that engine U spends the time searching for traces with lengths you want to deal with. Keep in mind that shorter traces are often easier to interpret.

Example

```
set_engineU_max_trace_length 50
```

See Also

set_max_trace_length

set_engineU_seed

Specifies a hexadecimal setting for the seed used by engine U.

Syntax

```
set_engineU_seed hex_string
```

Returns

No return values

Default

00000001

Detailed Description

Use this command to supply a hexadecimal setting for the seed that determines how engine U searches for traces. 0 is a special value that allows engine U to pick an indeterministic seed. Other values give deterministic behavior.

With the default prove verbosity, which is 6, the tool issues a message in the Proof Messages tab that reports the current seed at every restart. The example shows how you can use seed information.

NOTE: If you run "set_engineU_seed" and specify multiple jobs, each job uses a different seed, which is chosen deterministically as a function of the seed you specify and the engine job number.

hex_string
    Use "hex_string" as the seed for engine U.

Example

```
prove -engine_mode U -all

// No trace found for some asserts or covers. Retry with changed
// settings:

get_engineU_seed

set_engineU_seed 52243c71
prove -engine_mode U -all
```

set_engineU_trace_attempt_time_limit

Sets a trace attempt time limit for engine U.

Syntax

```
set_engineU_trace_attempt_time_limit time_limit
```

Returns

No return values

Default

0 (unlimited)

Detailed Description

Use this command to specify a trace attempt time limit for engine U. When a trace attempt exceeds the trace attempt time limit, the engine restarts processing with a new seed.

Use this command to direct engine U to restart when it is making no progress.

The tool supports three formats for the time limit:
. ISO8601: hh':'[mm[':'[ss['.'[fff]]]]]
  Examples: 2:30, 2:, 0:0:23, 2:30:05.235
. Jasper: [dd'd'][hh'h'][mm'm'][ss[.fff]'s']
  Examples: 2h30m, 2h, 23s, 2h30m05.235s
. Seconds: sssss[.fff]['s']
  Examples: 9000s, 7200, 23s, 9005.235s

time_limit
    Use the specified time limit for each engine U trace attempt.

    NOTE:
    . If you do not specify a time unit, the default unit is seconds.
    . To remove the limit, use 0s.

Example

```
set_engineU_trace_attempt_time_limit 1m
```

set_engineWL_config

Sets the Word-Level engine configuration profile.

Syntax

```
set_engineWL_config WL_config_name
```

Returns

No return values

Default

Base

Detailed Description

Use this command to select a predefined Word-Level (WL) engine configuration profile by name.

Configuration names are case-insensitive.

Examples of valid configuration names include:
Base, Base0, AM0, AM1, CAM0, CAPP1.

Use "get_engineWL_config" to report the current WL configuration.

Example

```
set_engineWL_config AM0
```

set_evaluate_properties_on_formal_reset

Controls property evaluation during the reset phase in the formal analysis.

Syntax

```
set_evaluate_properties_on_formal_reset (on | off)
```

Returns

No return values

Default

on

Detailed Description

Enables evaluation of properties during the reset phase in the formal analysis.

When using "reset -formal" to compute the reset phase during the formal analysis, it is possible to avoid evaluating property results. Use this command to control the evaluation of properties during the reset proof phase.

NOTE:
. As this command affects the creation of the compiled properties for embedded and Tcl properties, you must run it before "elaborate".
. Setting this command to "off" prevents CEX "noise" coming from reset conditions when assertions are not modeled to handle reset scenarios.
. "set_evaluate_properties_on_formal_reset" is an initial release to gather feedback from early adopters and finalize implementation for an upcoming release. Contact support@cadence.com to provide feedback or comments.

LIMITATIONS: The Superlint App and X-Propagation Verification App do not support this command.

Example

```
set_evaluate_properties_on_formal_reset on
```

set_executable_path

Provides path to the executable that will be used to debug C/C++ code with GDB.

Syntax

```
set_executable_path "path"
```

Returns

No return values

Detailed Description

Use this command to provide the executable path that the "cfe_debug" command uses to open the desired executable with GDB.

Example

```
set_executable_path /home/user1/example/main
```

See Also

cfe_debug

set_exit_handler

Sets a Tcl proc to run when an analysis session exits for predefined reasons.

Syntax

```
set_exit_handler tcl_proc_name
```

Returns

No return values

Default

""

Detailed Description

The tool calls the specified Tcl procedure when the analysis session is
going to exit due to the following reasons:

```
. Termination signal: The analysis session catches a termination signal
                      (SIGTERM, SIGHUP)
. Exit command      : The interpreter is processing the Tcl command "exit".
. Auto-exit on error: The interpreter encountered an error and it was configured
                      to exit automatically in such a case (-batch).
. User exit from GUI: The user requests an exit from the GUI.
```

NOTE: More reasons might be added in the future.

Pass a Tcl dictionary as an argument with information about the reason the
session was asked to exit. The entries in the dictionary depend on the value of
the "reason" key, which must be present always.

Dictionary fields according to reason:

```
. Termination signal:
  . reason: SIGNAL
```

```
  . status: <integer>
  . signal: (SIGTERM | SIGHUP)
    signal is the name of the signal caught
. Exit command:
  . reason: EXIT_CMD
  . status: <integer>
. Auto-exit on error:
  . reason: AUTO_EXIT_ON_ERROR
  . status: <integer>
. User exit from GUI:
  . reason: GUI_USER
  . status: <integer>

The "status" field corresponds to the exit status for the exiting analysis
session.

NOTE:
. Issuing the "exit" command from inside the exit handler unwinds the
  evaluation stack, effectively finishing the handler. If a new exit status is
  supplied, it replaces the one previously set.
. Issuing the "exit" command from inside the exit handler does not
  recursively trigger the handler.
. In future versions of the tool, the contents of the dictionary might be
  extended to include additional information on a given exit reason.
. If the Tcl interpreter is busy evaluating a script when the analysis
  session is set to exit, the tool tries to interrupt and cancel all
  evaluations taking place before running the defined procedure. The time it
  takes to begin evaluating the procedure depends on how fast the script being
  evaluated responds to the interruption request.
```

Example

```
proc exit_handler {args_dict} {
    set reason [dict get $args_dict reason]
    if {$reason == "SIGNAL"} {
        set signal [dict get $args_dict signal]
        # $signal is either : SIGTERM, SIGHUP
        puts "Exiting the analysis session because signal $signal was caught in the
analysis session."
    } elseif {$reason == "EXIT_CMD"} {
        puts "Exiting the analysis session because the exit command was issued."
    } elseif {$reason == "AUTO_EXIT_ON_ERROR"} {
        puts "Exiting the analysis session because it encountered an error."
    } elseif {$reason == "GUI_USER"} {
        puts "Exiting the analysis session because of a user exit in GUI."
    }
}
set_exit_handler exit_handler
```

set_expert_system_property_send_limit

Specify the number of property status updates to send to the information server.

Syntax

```
set_expert_system_property_send_limit N
```

Returns

No return values

Default

20000

Detailed Description

Use this command to set the limit on property status updates sent to the Jasper Information System server. The default is "20,000". Use "0" for for unlimited updates.

Example

```
set_expert_system_property_send_limit 40000
```

set_expert_system_regression_model_version

Sets the value of the "expert_system_regression_model_version" variable.

Syntax

```
set_expert_system_regression_model_version version
```

Returns

No return values

Default

Empty string

Detailed Description

Use this command to set the regression model version to be used with jobs sent to the Jasper Information System server.

Example

```
set_expert_system_regression_model_version ver12
```

set_expert_system_regression_tag

Sets the value of the "expert_system_regression_tag" variable.

Syntax

```
set_expert_system_regression_tag tag
```

Returns

No return values

Default

Empty string

Detailed Description

Use this command to set the regression tag to be used with jobs sent to the Jasper Information System server.

Example

```
set_expert_system_regression_tag reg001
```

set_expert_system_status

Enables turning Jasper Expert System "on" or "off".

Syntax

```
set_expert_system_status (on | off)
```

Returns

No return values

Default

on

Detailed Description

Use this command to disable or enable Jasper Expert System, which is "on" by default".

Example

```
set_expert_system_status off
```

set_expert_system_superlint_tasks_filter

Sets whether Superlint tasks are sent to the Information System server.

Syntax

```
set_expert_system_superlint_tasks_filter (true | false)
```

Returns

No return values

Default

true

Detailed Description

Use this command to specify whether Superlint tasks are filtered from the content sent to the Jasper Information System server. If the value is "true", Superlint tasks are filtered. To send Superlint tasks to the server, set this variable to "false".

NOTE: Use this command before loading the design.

Example

```
set_expert_system_superlint_tasks_filter true
set_expert_system_superlint_tasks_filter false
```

set_expert_system_tcl_design_name

Specifies the design name to be sent to the Information System server.

Syntax

```
set_expert_system_tcl_design_name design_name
```

Returns

No return values

Default

Empty string

Detailed Description

Use this command to set a user-defined design name for the current job to be sent to the Jasper Information System server. By default, the design name sent to the Information System server is the top module name.

NOTE: This command must be used prior to connecting Jasper to the Information System server and before any design has been loaded.

Example

```
set_expert_system_tcl_design_name top_rev0
```

set_expert_system_unprocessed_property_status

Specifies whether to include properties with "unprocessed" status in updates.

Syntax

```
set_expert_system_unprocessed_property_status (true | false)
```

Returns

true or false

Default

false

Detailed Description

Use this command to specify whether the tool includes properties with "unprocessed" status in updates sent to the Jasper Information System server. The default is "false".

Example

```
set_expert_system_unprocessed_property_status true
```

set_export_sva_bind_enable_directive

Specifies a conditional compilation directive for the SVA bind statement.

Syntax

```
set_export_sva_bind_enable_directive directive
```

Returns

No return values

Detailed Description

Exported SVA files contain a bind statement, which binds the SVA module to the RTL design. When exporting to SVA files from the property synthesis apps, use this command to add a conditional compilation guard, for example, "`ifdef XXX" or "`ifndef XXX", around the bind statement.

Example

```
set_export_sva_bind_enable_directive "`ifdef SVA_ON"
```

See Also

check_superlint,
export

set_export_sva_embed_lib_modules

Specifies whether library modules should be embedded in exported SVA output.

Syntax

```
set_export_sva_embed_lib_modules (true | false)
```

Returns

No return values

Default

true

Detailed Description

Exported SVA files might contain Jasper library module instantiations.
When exporting to SVA files from the property synthesis apps,
use this command to turn library modules embedding on or off.

Example

```
set_export_sva_embed_lib_modules false
```

See Also

export

set_export_sva_handshake_delay_threshold

Specifies the threshold for the delay in the handshake properties.

Syntax

```
set_export_sva_handshake_delay_threshold threshold
```

Returns

No return values

Detailed Description

When exporting to SVA from the property synthesis apps, use this command to set a threshold for converting the maximum delay in the handshake property to '$'.

For example, if the "export_sva_handshake_delay_threshold" variable is set to 100, then the tool converts the max delay to $ when exporting to SVA if the difference between the min delay and max delay is more than 100. Thus, $handshake(s1, s2, 1, 1800, 0, 0) would create an SVA property that matches to $handshake(s1, s2, 1, $, 0, 0)

Example

```
set_export_sva_handshake_delay_threshold 200
```

set_export_sva_hierarchy_separator

Specifies the RTL hierarchy separator character.

Syntax

```
set_export_sva_hierarchy_separator hierarchy_separator
```

Returns

No return values

Default

.

Detailed Description

During an SVA export, the tool uses the hierarchy separator character when exporting SVA to create appropriate binding for exported properties. Use this command to modify the RTL hierarchy separator character.

Example

```
set_export_sva_hierarchy_separator /
```

set_export_sva_hierarchy_separator_escape_string

Specifies the string used to replace the RTL hierarchy separator character.

Syntax

```
set_export_sva_hierarchy_separator_escape_string escape_string
```

Returns

No return values

Default

__

Detailed Description

During an SVA export, the tool replaces the hierarchy separator with an alternate string (known as an "escape string"), so you can later bind exported SVA as if all signals reside in one hierarchy. Use this command to modify the default escape string used for the RTL hierarchy separator.

Example

```
set_export_sva_hierarchy_separator_escape_string _X_
```

set_export_sva_target_environment

Specifies the target validation environment for SVA export.

Syntax

```
set_export_sva_target_environment (formal | simulation | emulation)
```

Returns

No return values

Default

formal

Detailed Description

Use this command to specify the target validation environment for SVA export. With this command, the tool applies optimizations specific to one or more validation environments during SVA export.

Example

```
set_export_sva_target_environment simulation
```

set_export_sva_trace_suffix_cycles

Specifies the number of cycles that will be added to the cover property.

Syntax

```
set_export_sva_trace_suffix_cycles N
```

Returns

No return values

Detailed Description

When exporting to SVA from the property synthesis apps, use this command to debug N cycles after the event, specified as a cover property, occurred.

For example, if the "export_sva_trace_suffix_cycles" variable is set to 5, the tool generates the cover property with a suffix of the SVA delay operator as 5 additional cycles: "cover property (fsm == IDLE_STATE ##5 1'b1);"

Example

```
set_export_sva_trace_suffix_cycles 5
```

See Also

cover

set_expr_32bit_constants

Extend expressions with unsized numbers to 32-bit constants.

Syntax

```
set_expr_32bit_constants (on | off)
```

Returns

No return values

Default

on

Detailed Description

By default, the tool extends expressions with unsized numbers to 32-bit constants. Use this command to disable the feature or re-enable it.

NOTE: An unsized number is a simple decimal number or a number with a base specifier but no size specification.

Example

```
set_expr_32bit_constants off
```

set_extended_inout_port_alias_handling

Enables or disables extended elaborate handling of alias through inout ports.

Syntax

```
set_extended_inout_port_alias_handling (on | off)
```

Returns

No return values

Default

off

Detailed Description

Use this command to control the extended "elaborate" handling of alias through inout ports. On the example below, the inout_model module makes A1 and A2 nets be aliases.

Example:

```
    # test.v
    module top;
        logic A1, A2;
        inout_model i(.a(A1), .b(A2));
    endmodule

    module inout_model (.a(signal), .b(signal));
        inout signal;
    endmodule

    # run.tcl
    set_extended_inout_port_alias_handling on
    analyze -sv test.v
    elaborate -top top
```

Example

```
set_extended_inout_port_alias_handling on
```

set_file_access_time_limit

Sets the time limit used by "analyze" command while waiting for file access.

Syntax

```
set_file_access_time_limit time_limit
```

Returns

No return values

Default

0s (unlimited)

Detailed Description

If the access time for a specific file read during the "analyze" command
exceeds the configured limit, the WFL033 warning will be issued.

time_limit
    Time limit while waiting for file access during "analyze" command.

    The tool supports three formats for the time limit:
    . ISO8601: hh':'[mm[':'[ss['.'[fff]]]]]
        Examples: 2:30, 2:, 0:0:23, 2:30:05.235
    . Jasper: [dd'd'][hh'h'][mm'm'][ss[.fff]'s']
        Examples: 2h30m, 2h, 23s, 2h30m05.235s
    . Seconds: sssss[.fff]['s']
        Examples: 9000s, 7200, 23s, 9005.235s

    To reset the default (unlimited), use 0s.

Example

```
# configures file access time threshold used by the "analyze" command to 1 second
set_file_access_time_limit 1s
```

set_first_trace_attempt

Specifies the cycle at which engines will make their first trace attempt.

Syntax

```
set_first_trace_attempt N
```

Returns

No return values

Default

1

Detailed Description

By default, engines search for traces starting with traces of length 1 ("trace attempt 1"). Sometimes, you can achieve better results by skipping a few early attempts, and instead, start at a higher trace attempt. With this command, you can directly specify the first trace attempt engines will make.

The following scenarios are examples of times when a higher trace attempt might be warranted:

. When slightly longer traces can be found much faster. (Sometimes trace attempt 1 might even be intractable, while at the same time, trace attempt 2 finds a trace immediately.)

. When you want to see a trace that is not too short.

. When prior proofs have established that there are no traces to be found shorter than a certain bound--the min_length bound--then, it is safe to start at a higher attempt in the sense that a minimal trace will still be found.

. When engines focused on concluding that no trace is to be found might be led away from unfruitful analysis using this command.

NOTE:
. Increasing the first trace attempt _can_ shave off some proof time, but it can also have the opposite effect. Perhaps counterintuitively, it might be faster to make all trace attempts from 1 to 50 than it is to make all trace attempts from 20 to 50.

. When skipping the first few trace attempts, it is no longer guaranteed that engine B will find the shortest possible trace.

. Some engines do not have the capability to start at something other than trace attempt 1. Such engines will ignore this setting.

. This command is effective with engines L, B, Bm, Ht*, Hps, D, K, I, N, and C only. However, for engines B and L, it is possible to receive a trace shorter than the specified length.

Example

```
set_first_trace_attempt 9
```

See Also

set_max_trace_length,
set_engineB_first_trace_attempt,
set_engineBm_first_trace_attempt,
set_engineH_first_trace_attempt

set_force_property_removal

Controls removal of converted embedded properties when using "-remove_original".

Syntax

```
set_force_property_removal (true | false)
```

Returns

No return values

Default

true

Detailed Description

By default, the commands "assume -from_assert -remove_original" and "assert -from_assume -remove_original" remove embedded properties. Use "set_force_property_removal false" to prevent the tool from removing embedded properties.

NOTE: Use this command before running the "assert" or "assume" commands.

Example

```
set_force_property_removal false
```

See Also

assert,
assume

set_fsv_abstract_result_mode

Specifies a global fault abstract result mode for FSV.

Syntax

```
set_fsv_abstract_result_mode (none | default | constrained)
```

Returns

No return values

Default

"none"

Detailed Description

Use this command to specify the fault abstract result mode for FSV.
. Use "default" to consider only sound results.
. Use "constrained" to consider some unsound results.
. Use "none" to clear the abstract result mode.

Example

```
set_fsv_abstract_result_mode default
```

set_fsv_allow_faults_in_bbox

Specifies whether FSV imports faults that are injected in blackbox instances.

Syntax

```
set_fsv_allow_faults_in_bbox (true | false)
```

Returns

No return values

Default

false

Detailed Description

Set this variable to "true" before issuing the "check_fsv -fault_work -import" command to import faults that are injected in blackbox instances. blackboxed. The default is "false".

Example

```
set_fsv_allow_faults_in_bbox true
```

See Also

check_fsv

set_fsv_bbox_abstraction_level

Control the abstraction level of black boxes during generate.

Syntax

```
set_fsv_bbox_abstraction_level (0 | 1 | 2)
```

Returns

No return values

Default

1

Detailed Description

Use this command to control the abstraction level of black boxes during generate.

Level 0: Disable black box abstraction.
Level 1: Abstract black boxes so that faults propagate through them.
Level 2: Treat black-boxed inputs in the COI of strobes as strobe points.

The tool abstracts multipliers, dividers, and modulo black boxes with a combinatorial abstraction. Other black boxes are abstracted with a mixed abstraction.

Example

```
set_fsv_bbox_abstraction_level 0
```

See Also

check_fsv

set_fsv_break_loop_mode

Specifies how FSV handles breaking combo loops.

Syntax

```
set_fsv_break_loop_mode (default | sync)
```

Returns

No return values

Default

default

Detailed Description

Use this command to specify how you want the tool to handle breaking loops under "set_loop_handling break".

default
    The break point might lead to spurious observability.

sync
    The break point is not an observability source.
    Note: This is the equivalent of adding a stopat that might mask fault propagations.

NOTE: Changing the value of this configuration clears the proof results.

Example

```
set_fsv_break_loop_mode sync
```

set_fsv_bus_fault_partition

Sets the chunks a multi-bit signal is divided into when adding a bus fault.

Syntax

```
set_fsv_bus_fault_partition N
```

Returns

No return values

Default

1

Detailed Description

Use "set_fsv_bus_fault_partition" to specify the value of the parameter "fsv_bus_fault_partition" used when adding faults. This value determines the number of chunks a multi-bit signal is divided into when adding the corresponding bus fault.
NOTE:
. "N" must be greater than or equal to 1.
. The default is 1.

Example

```
set_fsv_bus_fault_partition 2
```

See Also

check_fsv

set_fsv_bus_fault_width

Sets the threshold at which a multi-bit signal is considered a bus.

Syntax

```
set_fsv_bus_fault_width N
```

Returns

No return values

Default

0

Detailed Description

Use "set_fsv_bus_fault_width" to specify the value of the parameter "fsv_bus_fault_width" that is used when adding faults. This value determines the threshold at which a multi-bit signal is considered a bus.
NOTE:
. "N" must be greater than or equal to 0.
. The default is 0.

Example

```
set_fsv_bus_fault_width 4
```

See Also

check_fsv

set_fsv_classification_scheme

Sets the scheme used to determine faults classification.

Syntax

```
set_fsv_classification_scheme (iso26262 | diagnostic)
```

Returns

No return values

Default

iso26262

Detailed Description

Use this command to set the scheme FSV uses to determine faults classification.

When you specify "diagnostic", "always detected" faults are considered "safe" regardless of whether they are propagatable. When you specify "iso26262", propagatable faults are always considered "dangerous" regardless of whether they are always detected by a checker. The default is "iso26262".

Example

```
set_fsv_classification_scheme diagnostic
```

See Also

check_fsv

set_fsv_clock_cycle_time

Sets the clock cycle time.

Syntax

```
set_fsv_clock_cycle_time time
```

Returns

No return values

Detailed Description

Use "set_fsv_clock_cycle_time" to convert simulation time to formal clock cycles. The clock cycle time is relevant to the SET fault type and/or the fault injection time window when you specify the SET hold time or the injection time without clock cycle time.

NOTE:
. The format for time is [scale][unit], and unit can be any of the following: fs, ps, ns (default), us, ms, or s.
. You can overwrite this value with "check_fsv -generate [-clock_cycle_time time]".

Example

```
set_fsv_clock_cycle_time 50ns
```

See Also

check_fsv

set_fsv_constant_propagation_analysis_optimization

Enables optimizations during structural phase constant propagation analysis.

Syntax

```
set_fsv_constant_propagation_analysis_optimization (0 | 1 | 2)
```

Returns

No return values

Default

1

Detailed Description

Use this command to enable optimizations during the computation of constant propagation analysis in the structural phase.

. Level 0: FSV executes no optimization.
. Level 1: FSV executes a non-aggressive optimization, which computes a pruned transitive fanin of specific gates.
. Level 2: FSV executes an aggressive optimization, which computes the full transitive fanin of specific gates.
  NOTE: This option might add a significant amount of run time to the structural analysis phase. Consider using this optimization when the percentage of constant signals in the design is high.

Example

```
set_fsv_constant_propagation_analysis_optimization 1
```

See Also

check_fsv

set_fsv_engine_mode

Sets the AutoProve engine mode for the FSV App.

Syntax

```
set_fsv_engine_mode
    (  B | B4 | Bm | C | C2 | D | EC | G | G2 | H | Hp | Hps | Ht | Hts | I | J
     | K | L | M | N | AB | AD | AG | AM | Oh | Q3 | R | Tri | U | TM | QT)+
```

Returns

No return values

Default

Tri U EC Ht Hp I Bm N AM D

Detailed Description

Use this command to set the engines run by the FSV "autoprove" command.

Example

```
set_fsv_engine_mode {EC N Hp Ht Bm}
```

See Also

check_fsv

set_fsv_export_fault_relations

Specifies whether FSV exports fault relations into a UCM file.

Syntax

```
set_fsv_export_fault_relations (true | false)
```

Returns

No return values

Default

false

Detailed Description

Set this variable to "true" before issuing "check_fsv -fault_work -export"
to export fault relations to UCM. The default is "false".

Example

```
set_fsv_export_fault_relations true
```

See Also

check_fsv

set_fsv_export_propagated_constants

Specifies whether FSV exports propagated constants to the .csv file.

Syntax

```
set_fsv_export_propagated_constants (true | false)
```

Returns

No return values

Default

false

Detailed Description

Set this variable to "true" before issuing "check_fsv -fault_work -export"
to export propagated constants to the .csv file. The default is "false".

Example

```
set_fsv_export_propagated_constants true
```

See Also

check_fsv

set_fsv_fault_tw

Sets the fault injection time window.

Syntax

```
set_fsv_fault_tw (auto | start_time[:end_time])
```

Returns

No return values

Default

auto

Detailed Description

Use "set_fsv_fault_tw" to specify the fault injection time window or "anytime".
The FSV App injects the fault within the specified time window.

NOTE:
. Specify the time interval (start:end) with integers followed by one of the
  following units (no spaces): fs, ps, ns (default), us, ms, or s.
. Use "$" to specify infinite end time.
. This command specifies the fault time window for "check_fsv -fault -add
  -from_file" and "check_fsv -fault_work -import".
. Use 0:$ to lift all restrictions and inject faults at any time.
. The default time window is "auto", which is "0:$" for SEU and SET faults
  and "0" otherwise.

Example

```
set_fsv_fault_tw 0:$
set_fsv_fault_tw 3fs:5fs
set_fsv_fault_tw 1:6
set_fsv_fault_tw 50ns
```

See Also

check_fsv

set_fsv_generate_activatability

Enables activatability analysis pre-generate.

Syntax

```
set_fsv_generate_activatability (on | off)
```

Returns

No return values

Default

on

Detailed Description

Use this command to generate nets for checking fault activatability. If you run "check_fsv -generate" with this command set to "on", the tool generates special nets in the FSV task for checking faults activatability.

Example

```
set_fsv_generate_activatability off
```

See Also

check_fsv

set_fsv_generate_always_detected

Enables always_detected analysis pre-generate.

Syntax

set_fsv_generate_always_detected (on | off)

Returns

No return values

Default

off

Detailed Description

Use this command to generate nets for checking "always_detected" faults. If you run "check_fsv -generate" with this command set to "on", the tool generates special nets in the FSV task for checking "always_detected" faults.

Example

set_fsv_generate_always_detected on

See Also

check_fsv

set_fsv_generate_always_propagated

Enables "always_propagated" analysis pre-generate.

Syntax

```
set_fsv_generate_always_propagated (on | off)
```

Returns

No return values

Default

off

Detailed Description

Use this command to generate nets for checking "always_propagated" faults. If you run "check_fsv -generate" with this command set to "on", the tool generates special nets in the FSV task for checking "always_propagated" faults.

Example

```
set_fsv_generate_always_propagated on
```

See Also

check_fsv

set_fsv_generate_co_max_delay_time

Sets the max number of cycles to check for detection after fault injection.

Syntax

```
set_fsv_generate_co_max_delay_time time
```

Returns

No return values

Default

$

Detailed Description

Use "set_fsv_generate_co_max_delay_time" to specify the maximum number of clock cycles to check for detection after a fault was injected.

NOTE:
. This command affects the "detectability" and "always_detected" checks.
. The format for time is [scale][unit], and unit can be any of the following:
  cc, fs, ps, ns, us, ms, or s.
. If no unit is specified, the default is "cc".
. This command also accepts "$" to indicate an infinite max delay time.
. Setting the max delay time to a finite value will make the "always_detected"
  property a safety or finite property instead of liveness or infinite.

Example

```
set_fsv_generate_co_max_delay_time 2
```

See Also

check_fsv

set_fsv_generate_copy_env

Enables copying the environment from the current task to new tasks.

Syntax

```
set_fsv_generate_copy_env (on | off)
```

Returns

No return values

Default

off

Detailed Description

Use this command to copy the environment from the current task to all future tasks.

Example

```
set_fsv_generate_copy_env on
```

See Also

check_fsv

set_fsv_generate_detectability

Enables detectability analysis pre-generate.

Syntax

```
set_fsv_generate_detectability (on | off)
```

Returns

No return values

Default

off

Detailed Description

Use this command to generate nets for checking fault detectability. If you run "check_fsv -generate" with this command set to "on", the tool generates special nets in the FSV task for checking fault detectability.

Example

```
set_fsv_generate_detectability on
```

See Also

check_fsv

set_fsv_generate_fo_co_max_delay_time

Sets the max number of cycles to check for detection after fault propagation.

Syntax

```
set_fsv_generate_fo_co_max_delay_time time
```

Returns

No return values

Default

```
$
```

Detailed Description

Use "set_fsv_generate_fo_co_max_delay_time" to specfiy the maximum number of clock cycles to check for detection after a propagation was found during the proof of the "propagated_always_detected" check.

NOTE:
. This command affects the "propagated_always_detected" check.
. The format for time is [scale][unit], and unit can be any of the following: cc, fs, ps, ns, us, ms, or s.
. If no unit is specified, the default is "cc".
. This command also accepts "$" to indicate an infinite max delay time.
. Setting the max delay time to a finite value will result in creating a safety property for the "always_detected" check instead of a liveness property.

Example

```
set_fsv_generate_fo_co_max_delay_time 2
```

See Also

check_fsv

set_fsv_generate_fo_max_delay_time

Sets the max number of cycles to check for propagation after fault injection.

Syntax

```
set_fsv_generate_fo_max_delay_time time
```

Returns

No return values

Default

```
$
```

Detailed Description

Use "set_fsv_generate_fo_max_delay_time" to set the maximum number of cycles to check for propagation after a fault was injected.

NOTE:

- This command affects the "propagatability" and "always_propagated" checks.
- The format for time is [scale][unit], and unit can be any of the following: cc, fs, ps, ns, us, ms, or s.
- If no unit is specified, the default is "cc".
- This command also accepts "$" to indicate an infinite max delay time.
- Setting the max delay time to a finite value will result in creating a safety property for the "always_propagated" check instead of a liveness property.

Example

```
set_fsv_generate_fo_max_delay_time 2
```

See Also

check_fsv

set_fsv_generate_propagatability

Enables propagatability analysis pre-generate.

Syntax

```
set_fsv_generate_propagatability (on | off)
```

Returns

No return values

Default

on

Detailed Description

Use this command to generate nets for checking fault propagatability. If you run "check_fsv -generate" with this command set to "on", the tool generates special nets in the FSV task for checking fault propagatability.

Example

```
set_fsv_generate_propagatability off
```

See Also

check_fsv

set_fsv_generate_propagated_always_detected

Enables "propagated_always_detected" analysis pre-generate.

Syntax

```
set_fsv_generate_propagated_always_detected (on | off)
```

Returns

No return values

Default

off

Detailed Description

Use this command to generate nets for checking "propagated_always_detected" faults. If you run "check_fsv -generate" with this command set to "on", the tool generates special nets in the FSV task for checking "propagated_always_detected" faults.

Example

```
set_fsv_generate_propagated_always_detected on
```

See Also

check_fsv

set_fsv_generate_use_bus_faults

Enables generation of bus faults instead of bit faults that are part of the bus.

Syntax

```
set_fsv_generate_use_bus_faults (on | off)
```

Returns

No return values

Default

off

Detailed Description

Use this command to enable the automatic generation of bus faults instead of bit faults that are part of that bus. The bus fault represents an "or" grouping of the individual bit faults.

Example

```
set_fsv_use_bus_faults on
```

See Also

check_fsv

set_fsv_generate_use_memory_faults

Enables generation of memory faults instead of bit faults as part of the memory.

Syntax

```
set_fsv_generate_use_memory_faults (on | off)
```

Returns

No return values

Default

off

Detailed Description

Use this command to enable the automatic generation of memory faults instead of bit faults that are part of that memory. The memory fault represents an "or" grouping of the individual bit faults.

Example

```
set_fsv_use_memory_faults on
```

See Also

check_fsv

set_fsv_import_include_ut

Specifies whether FSV import includes untestable faults.

Syntax

```
set_fsv_import_include_ut (true | false)
```

Returns

No return values

Default

false

Detailed Description

Set this variable to "true" before issuing "check_fsv -fault_work -import"
to include untestable faults. The default is "false"".

Example

```
set_fsv_import_include_ut true
```

See Also

check_fsv

set_fsv_import_prime_only

Specifies whether FSV imports all faults or primary faults only.

Syntax

```
set_fsv_import_prime_only (true | false)
```

Returns

No return values

Default

true

Detailed Description

Set this variable to "true" before issuing "check_fsv -fault_work -import"
to import primary faults only. The default is "true"".

Example

```
set_fsv_import_prime_only false
```

See Also

check_fsv

set_fsv_max_checks_per_task

Sets the maximum number of checks that FSV generates per task.

Syntax

```
set_fsv_max_checks_per_task N
```

Returns

No return values

Default

150000

Detailed Description

Use this command to set the maximum number of checks that FSV generates per task.
Note: Use 0 to generate an unlimited number of checks per task.

Example

```
set_fsv_max_checks_per_task 0
```

See Also

check_fsv

set_fsv_max_faults_per_task

Sets the maximum number of faults per task.

Syntax

```
set_fsv_max_faults_per_task N
```

Returns

No return values

Detailed Description

Use this command to specify the maximum number of faults per task the FSV App generates when you run "check_fsv -generate". The FSV App generates tasks and distributes the fault nodes based on the specified threshold. You can overwrite this value with the command "check_fsv -generate [-fsv_max_faults_per_task N]".

Example

```
set_fsv_max_faults_per_task 32
```

See Also

check_fsv

set_fsv_memory_fault_width

Sets the threshold at which a multi-bit signal is considered a memory.

Syntax

```
set_fsv_memory_fault_width N
```

Returns

No return values

Default

0

Detailed Description

Use "set_fsv_memory_fault_width" to specify the value of the parameter "fsv_memory_fault_width" that is used when adding faults. This value determines the threshold at which a multi-bit signal is considered a memory.
NOTE:
. "N" must be greater than or equal to 0.
. The default is 0.

Example

```
set_fsv_memory_fault_width 4
```

See Also

check_fsv

set_fsv_proof_min_cluster_size

Control the cluster size for grouping by instance.

Syntax

```
set_fsv_proof_min_cluster_size cluster_size
```

Returns

No return values

Default

0

Detailed Description

Use this command to set the minimum amount of faults in a cluster.
When you do not specify a minimum size, which is equivalent to using the default value, every instance becomes a group.

NOTE: Changing this setting might cause faults of different instances to be grouped together
if the number of faults in an instance is less than the specified minimum amount of faults.

Example

```
set_fsv_proof_min_cluster_size 64
```

See Also

check_fsv

set_fsv_proof_sanity_check

Specifies how FSV handles sanity properties.

Syntax

```
set_fsv_proof_sanity_check (on | auto | off)
```

Returns

No return values.

Default

auto

Detailed Description

Use this command to set whether sanity properties should be skipped, handled automatically by FSV, or included when using a regular "prove" command.

. Use "set_fsv_proof_sanity_check off" to disable the proof of sanity properties.
. Use "set_fsv_proof_sanity_check auto" to enable automatic handling of sanity properties. This is the default option.
. Use "set_fsv_proof_sanity_check on" to include the sanity properties when calling "check_fsv -prove" without any other filter.

NOTE: When checker strobes with checker mode "assert" exist, FSV automatically generates and proves the required sanity properties.

Example

```
set_fsv_proof_sanity_check off
```

See Also

check_fsv

set_fsv_prove_keep_traces

Disable trace suppression in all modes.

Syntax

```
set_fsv_prove_keep_traces (On | Off)
```

Returns

No return values.

Default

off

Detailed Description

Use this command if you want the tool to generate traces.

Example

```
set_fsv_prove_keep_traces off
```

See Also

check_fsv

set_fsv_prove_strategy

Sets the FSV prove strategy.

Syntax

```
set_fsv_prove_strategy (basic | clustering)
```

Returns

No return values

Default

basic

Detailed Description

Use this command to set the strategy for the "check_fsv -prove -strategy" command.

. basic: Run the proof on specified faults in one group. This is the default.
. clustering: Leverage clustering of faults for better convergence.

NOTE: See the "check_fsv -prove -strategy" command help for additional details.

Example

```
set_fsv_prove_strategy clustering
```

See Also

check_fsv

set_fsv_set_hold_time

Sets the SET fault hold time.

Syntax

```
set_fsv_set_hold_time time
```

Returns

No return values

Default

1cc

Detailed Description

Use "-set_hold_time" to specify the hold time for SET faults.

NOTE:
. The format for time is [scale][unit] or [scale][unit]:[scale][unit], and unit
  can be any of the following: cc (default), fs, ps, ns, us, ms, or s.
. The default hold time is "1cc".
. You can overwrite this value with "check_fsv -generate -set_hold_time time".
. The specified hold time applies to "-fault -add", "-fault -add -from_file",
  and "-fault_work -import".

Example

```
set_fsv_set_hold_time 50ns
```

See Also

check_fsv

set_fsv_source_browser_max_line_count

Sets the maximum number of lines the FSV source browser processes.

Syntax

```
set_fsv_source_browser_max_line_count N
```

Returns

No return values

Default

300000

Detailed Description

Use this command to set the maximum number of lines the FSV source browser processes for large files. Large files are files that reach the file size threshold specified with "set_fsv_source_browser_size_threshold".

Example

```
set_fsv_source_browser_max_line_count 100000
```

See Also

check_fsv,
set_fsv_source_browser_size_threshold

set_fsv_source_browser_size_threshold

Sets the file size threshold for the FSV source browser.

Syntax

```
set_fsv_source_browser_size_threshold N
```

Returns

No return values

Default

50

Detailed Description

Use this command to set the file size threshold (in MB) for the FSV source browser. The FSV source browser truncates files that exceed this threshold to show only the maximum number of lines specified with "set_fsv_source_browser_max_line_count".

Example

```
set_fsv_source_browser_size_threshold 20
```

See Also

check_fsv,
set_fsv_source_browser_max_line_count

set_fsv_source_browser_updating

Specify whether the source browser is updated during the proof.

Syntax

```
set_fsv_source_browser_updating (on | off | auto)
```

Returns

No return values

Default

auto

Detailed Description

Use this command to specify whether the source browser is updated during the proof.

on - Update the source browser during the proof.
off - Do not update the source browser during the proof.
auto - Update the source browser automatically during the proof.

Example

```
set_fsv_source_browser_updating off
```

See Also

check_fsv

set_fsv_stopat_abstraction_level

Control the abstraction level of stopats in FSV.

Syntax

```
set_fsv_stopat_abstraction_level (0 | 1)
```

Returns

No return values

Default

0

Detailed Description

Use this command to control the abstraction level of stopats during structural analysis and during the proof.

Level 0: FSV blocks fault propagation at stopat signals.
Level 1: FSV abstracts stopat signals so that faults propagate through them.

Example

```
set_fsv_stopat_abstraction_level 1
```

See Also

check_fsv

set_fsv_strobe_checker_mode

Sets the checker mode for check strobes.

Syntax

```
set_fsv_strobe_checker_mode (diff | assert)
```

Returns

No return values

Default

diff

Detailed Description

Use this command to specify the checker mode that determines when the tool considers the fault "detected" at the checker strobe:

. "diff" - The fault is detected when the strobe signals of the good machine and bad machine differ.
. "assert"" - The fault is detected when the bad machine strobe signal is asserted. Use "-assert_value" to control whether the strobe is asserted with the value 0 or 1.

Example

```
set_fsv_strobe_checker_mode assert
```

See Also

check_fsv,
set_fsv_strobe_checker_mode_assert_value

set_fsv_strobe_checker_mode_assert_value

Specifies the value at which the bad machine strobe signal is asserted

Syntax

```
set_fsv_strobe_checker_mode_assert_value (0 | 1)
```

Returns

No return values

Default

1

Detailed Description

Use this command to specify the value at which the bad machine strobe signal is asserted when the "fsv_strobe_checker_mode" value is "assert".

Example

```
set_fsv_strobe_checker_mode_assert_value 0
```

See Also

check_fsv

set_fsv_strobe_optimization

Enables or disables optimizing strobes observer nets.

Syntax

```
set_fsv_strobe_optimization (on | off)
```

Returns

No return values

Default

on

Detailed Description

Use this command to enable or disable the generation of strobes observer nets based on the strobes in the fanout of the fault only.

Example

```
set_fsv_strobe_optimization off
```

See Also

check_fsv

set_fsv_structural_coi

Enables COI analysis on strobes when performing structural analysis.

Syntax

```
set_fsv_structural_coi (on | fo | co | off)
```

Returns

No return values

Default

on

Detailed Description

Use this command to enable or disable COI analysis.
Specify "fo" or "co" to enable COI analysis for FO or CO
strobes only. Specify "on" to run COI analysis for all
current strobes.

Example

```
set_fsv_structural_coi off
```

See Also

check_fsv

set_fsv_structural_constant

Enables activatability analysis when performing structural analysis.

Syntax

```
set_fsv_structural_constant (on | off)
```

Returns

No return values

Default

on

Detailed Description

Use this command to run the activatability analysis, skipping non-activatable faults. If you run "check_fsv -generate" with this command set to "on", the tool runs activatability analysis before generating any other faults.

Example

```
set_fsv_structural_constant off
```

See Also

check_fsv

set_fsv_structural_constant_propagation_analysis

Enables constant propagation analysis when performing structural analysis.

Syntax

```
set_fsv_structural_constant_propagation_analysis (on | fo | co | off)
```

Returns

No return values

Default

on

Detailed Description

Use this command to enable or disable structural constant propagation analysis. Specify "fo" or "co" to enable constant propagation analysis for FO or CO strobes only. Specify "on" to enable constant propagation analysis for all current strobes.

Example

```
set_fsv_structural_constant_propagation_analysis off
```

See Also

check_fsv

set_fsv_structural_distance

Enables distance calculation when performing structural analysis.

Syntax

```
set_fsv_structural_distance (on | fo | co | off)
```

Returns

No return values

Default

off

Detailed Description

Use this command to enable or disable structural distance calculation. Specify "fo" or "co" to enable distance calculation for FO or CO strobes only. Specify "on" to enable distance calculation for all current strobes.

Example

```
set_fsv_structural_distance on
```

See Also

check_fsv

set_fsv_structural_fault_relations

Enables fault relations analysis when performing structural analysis.

Syntax

```
set_fsv_structural_fault_relations (all | on | fo | co | off)
```

Returns

No return values

Default

on

Detailed Description

Use "-fault_relations" to run dominance and equivalence analysis to generate a fault relations graph.
NOTE:

- The dominance direction is from gates' output towards inputs, that is, observed dominance.
- Use "fo" or "co" to run fault relations analysis from the FO or CO strobes only. Use "on" to run fault relations analysis from both the FO and CO strobes.
- Use "all" to analyze fault relations for all faults, regardless of the strobes' COI.

Example

```
set_fsv_structural_fault_relations off
```

See Also

check_fsv

set_fsv_structural_propagation_analysis

Enables propagation analysis when performing structural analysis.

Syntax

```
set_fsv_structural_propagation_analysis (on | fo | co | off)
```

Returns

No return values

Default

on

Detailed Description

Use this command to enable or disable structural propagation analysis. Specify "fo" or "co" to enable propagation analysis for FO or CO strobes only. Specify "on" to enable propagation analysis for all current strobes.

Example

```
set_fsv_structural_propagation_analysis off
```

See Also

check_fsv

set_fsv_structural_seq_constants_engine_mode

Sets the engine mode for the FSV sequential constants propagation proof.

Syntax

```
set_fsv_structural_seq_constants_engine_mode
    (  B | B4 | Bm | C | C2 | D | EC | G | G2 | H | Hp | Hps | Ht | Hts | I | J
     | K | L | M | N | AB | AD | AG | AM | Oh | Q3 | R | Tri | U | TM | QT)+
```

Returns

No return values

Default

Hp Ht N B

Detailed Description

Use this command to specify the engine mode for the "check_fsv -structural" command during sequential constants propagation formal analysis.

Example

```
set_fsv_structural_seq_constants_engine_mode {EC N Hp Ht Bm}
```

See Also

check_fsv

set_fsv_structural_seq_constants_propagation

Enables sequential constant propagation analysis for structural analysis.

Syntax

```
set_fsv_structural_seq_constants_propagation (simple | formal | off)
```

Returns

No return values

Default

simple

Detailed Description

Use this command to enable or disable structural sequential constant propagation analysis.

. Specify "simple" for proving sequential constants analysis using proof simplifications only.
. Specify "formal" for proving design sequential constants using regular "prove" and associated time limits.

The propagated sequential constants will be used during FSV structural and formal analysis, and by the Schematic Viewer and Visualize.

Example

```
set_fsv_structural_seq_constants_propagation off
```

See Also

check_fsv

set_fsv_structural_seq_constants_prove_time_limit

Sets the time limit for FSV sequential constants propagation analysis proof.

Syntax

```
set_fsv_structural_seq_constants_prove_time_limit time_limit
```

Returns

No return values

Default

86400s (24h)

Detailed Description

Use this command to set the time limit for the proof run by the "check_fsv -structural" command during sequential constants propagation flow.

The tool supports three formats for the time limit:

```
. ISO8601:    hh':'[mm[':'[ss['.'[fff]]]]]
  Examples:   2:30, 2:, 0:0:23, 2:30:05.235
. Jasper: [dd'd'][hh'h'][mm'm'][ss[.fff]'s']
  Examples:   2h30m, 2h, 23s, 2h30m05.235s
. Seconds:    sssss[.fff]['s']
  Examples:   9000s, 7200, 23s, 9005.235s
```

Example

```
set_fsv_structural_seq_constants_prove_time_limit 2h
```

See Also

check_fsv

set_fsv_unsafe_inference

Enables unsafe inference of check results from bus faults.

Syntax

```
set_fsv_unsafe_inference (on | off)
```

Returns

No return values

Default

off

Detailed Description

Use this command to enable unsafe inference of results from bus faults and memory faults to their inferred single-bit faults. When this variable is "on", all single-bit faults are marked according to the following rules:

. group fault is activated => all single-bit faults are activated
. group fault is propagated => all single-bit faults are activated
. group fault is detected => all single-bit faults are activated
. group fault is not always detected => all single-bit faults are not always detected

Example

```
set_fsv_unsafe_inference on
```

See Also

check_fsv

set_function_debug_visibility

Enables debugging for functions.

Syntax

```
set_function_debug_visibility (on | off)
```

Returns

No return values

Default

on

Detailed Description

Use this command to enable debugging for functions. With this command, you have access to any signal inside a function. That is, the signals are visible and you can operate on them with debugging features. For example, if you do a "Why," the source pane displays the code related to the function with the function signals highlighted. When this feature is off, the tool highlights entire functions instead of signals within functions.

Example

```
set_function_debug_visibility on
```

set_functional_loop_time

Sets a time limit for functional loop analysis in the break_functional flow.

Syntax

```
set_functional_loop_time time_limit
```

Returns

No return values

Default

600s

Detailed Description

```
The "set_loop_handling break_functional" flow has two main steps:

.  The first step enumerates all combo loops in the design.
.  The second step classifies the loops as functionally true or false loops.

Use "set_functional_loop_time" to set the time limit for the step that
classifies combo loops as functionally true loops. When the tool reaches the
time-out, it handles unclassified loops according to the value of the
"set_loop_handling_fallback" command.

time_limit
    Use the specified time limit in the functional combo loop analysis step in
    the "set_loop_handling break_functional" flow.

    The tool supports three formats for the time limit:
    . ISO8601: hh':'[mm[':'[ss['.'[fff]]]]]
      Examples: 2:30, 2:, 0:0:23, 2:30:05.235
    . Jasper: [dd'd'][hh'h'][mm'm'][ss[.fff]'s']
      Examples: 2h30m, 2h, 23s, 2h30m05.235s
    . Seconds: sssss[.fff]['s']
      Examples: 9000s, 7200, 23s, 9005.235s

    To remove the limit, use 0s.

NOTE:
. The "autoprove" command and AutoProve-related commands also respect the loop
  handling method specified by this command.
```

Example

```
set_functional_loop_time 2h
```

See Also

check_loop,
set_loop_handling,
set_loop_handling_fallback

set_gate_level_compilation_mode

Enables or disables gate-level compilation mode.

Syntax

```
set_gate_level_compilation_mode (on | off)
```

Returns

No return values

Default

off

Detailed Description

Use this command to control optimizations during gate-level design compilation. When enabled, messages with the same ID are limited to 20 repetitions during elaboration.

Example

```
set_gate_level_compilation_mode on
```

See Also

elaborate

set_gdb_path

Provides path to GDB that will be used to debug C/C++ code.

Syntax

```
set_gdb_path "path"
```

Returns

No return values

Default

gdb

Detailed Description

Use this command to provide the path that the "cfe_debug" command uses to open GDB.

Example

```
set_gdb_path /bin/usr/gdb
```

See Also

cfe_debug

set_hunt_single_proof_thread

Specifies whether hunt properties will be proven in a single proof thread.

Syntax

```
set_hunt_single_proof_thread (on | off)
```

Returns

No return values

Default

on

Detailed Description

When running hunt strategies without this feature, the engine instances with the specific strategy settings are launched in multiple proof threads. When you enable this feature, all the engine instances run under a single proof thread. Also, a custom engine is created for each strategy setting, and you see the custom engines running instead of the base engine.

. Use "on" to enable this feature.
. Use "off" to disable this feature.

NOTE: This mode is applicable only to "cycle_swarm", "bound_swarm", and "simulation" hunt strategies in standalone mode (without "-use_strategy" and "-auto"), and this setting is ignored when not applicable.

Example

```
set_hunt_single_proof_thread on
```

See Also

hunt

set_incremental_compilation_mode

Enables or disables advanced incremental compilation mode.

Syntax

```
set_incremental_compilation_mode (on | off)
```

Returns

No return values

Default

off

Detailed Description

Use this command to control the advanced incremental compilation mode. When enabled, multiple elaborate commands can be used on Jasper. The first elaborate will compile the entire design as a regular elaborate command. All subsequent elaborate commands will synthesize only modules or units inside touched files between two elaborate commands. Also, dependencies for modified modules/units are synthesized.

DUT and/or TB changes are supported on the incremental flow.

The Advanced Incremental Compilation flow supports the parallel compilation flow integration for additional performance gains. To enable parallel flow optimization along with advanced incremental, call "set_parallel_synthesis_mode on" before the first analyze command.

IMPORTANT:
. The flow supports incremental compilation flow during the same session.
. Re-analyze all files inside the same compilation unit if you have made changes that might impact any modules or units inside these files. This ensures the accuracy of the incremental elaboration results.
. All macros and -v/-y module definitions from the compilation unit are cleared

```
  and re-analyzed on every new compilation's first analyze command.
. We cannot change elaborate switches/subcommands during the advanced incremental
  flow. Only "bbox_m", "bbox_i", and "parameter" switches can change during the
  elaborate commands.
  . A warning message will be thrown if any switch changes and complete synthesis
    with the new switches will be executed.
. We cannot change any configuration command values between the elaborations.
  . An error message (ECD037) will be thrown if any configuration has its values
    changed after the first elaborate command.
. Avoid using "analyze -clear", "elaborate -clear", and "clear -all" between the
  elaborations when the advanced incremental compilation flow is enabled. These
  commands will remove the flow optimization, which makes it necessary to do the
  complete synthesis again.
. Environment configuration and proof results are not stored during the incremental
  compilations when the flow is enabled.
. "Save/Restore" flow is still not integrated with the Advanced Incremental
  Compilation. However, there is no block error being thrown when we use
  "save/restore" commands with the advanced incremental flow.
  . The last elaboration result will be saved once the "save" command is called.
  . If the "restore" command is called and after it, there is another "elaborate"
    command, this elaboration will not take advantage of advanced incremental
    flow, taking the same time of a elaboration without the Advanced Incremental
    Compilation enabled.

NOTE: The cases in which a module or unit depends on another are listed below:
. Packages: Changes in packages trigger the synthesis of all modules or units
  that use the package.
. Hierarchical references (HRs): HRs to constants (parameters and system
  function call, for example, $bits) create a dependency between two modules
  or units.
. SV Interfaces: Modules that instantiate interfaces or have interface ports
  are re-synthesized if the interface definition changes.

LIMITATIONS: The following apps/flows are not supported by Advanced Incremental
Compilation:
. blackbox_assistant
. elaborate -req
. save/restore
. MCP
. Superlint
. UNR (P2P)
. PSL inheritance with default clock
. Cpi2Jetlist
```

. C2RTL

Example

```
set_incremental_compilation_mode on
```

See Also

analyze,
elaborate,
clear

set_internal_from_mode

Sets the X-injection mode for the modeling of internal "-from" signals.

Syntax

```
set_internal_from_mode (multi | mono)
```

Returns

No return values

Default

multi

Detailed Description

Use this command to set the X-injection mode for the modeling of internal "-from" signals.

. Use "multi" to allow Xs to be injected an infinite number of times. This option is the default.
. Use "mono" to allow the X to be injected only once.

Example

```
set_internal_from_mode mono
```

set_jasper_history_per_command_limit

Sets the limit of Jasper command entries to keep in history per command name.

Syntax

```
set_jasper_history_per_command_limit limit
```

Returns

No return values.

Default

0

Detailed Description

Use this command to change the limit of Jasper command entries to keep in history for queries by "jasper_history". This limit applies per command name.

Example

```
set_jasper_history_per_command_limit 100
```

See Also

jasper_history

set_jaspersim_max_jobs

Defines the maximum number of parallel jobs for the parallel simulator.

Syntax

```
set_jaspersim_max_jobs N
```

Returns

No return value

Default

machine defined

Detailed Description

Use this command to create a limit for the maximum number of parallel jobs for the parallel simulator run. By default, it returns the number of jobs supported by the system.

Example

```
set_jaspersim_max_jobs 8
```

See Also

set_jaspersim_mode

set_jaspersim_mode

Specifies the Jasper simulation method.

Syntax

```
set_jaspersim_mode (single | parallel)
```

Returns

No return value

Default

single

Detailed Description

Reset Analysis and Visualize run simulations to concretize values for signals. This is a high effort operation that might take a long time to run depending on the size of the design and trace. It is possible to boost the simulation performance by running it in parallel. Use this command to enable the parallel simulation feature.

NOTE: Parallel simulation is only effective on the full design region. This means that if it is not a reset analysis or if "get_trace_optimization" is not standard, parallel simulation will be turned off.

Example

```
set_jaspersim_mode parallel
set_jaspersim_partition_method default
set_jaspersim_partition_depth 2

set_jaspersim_mode parallel
set_jaspersim_partition_method file
set_jaspersim_partition_depth Partition.txt
```

See Also

set_jaspersim_max_jobs,
set_jaspersim_partition_depth,
set_jaspersim_partition_file,
set_jaspersim_partition_method,
set_trace_optimization

set_jaspersim_partition_depth

Defines a value for default parallel simulator partitioning.

Syntax

```
set_jaspersim_partition_depth N
```

Returns

No return value

Default

2

Detailed Description

By default, Jasper partitions a design by creating partitions tied to the design instances. The partitioning splits the design according to its hierarchy, where the top instance is the first depth. Use this command to define the depth of the instances that will become partitions.

Important things to know about this command:
- The partition depth must be an integer greater than 1.
- Setting a depth of 1 or less is not allowed and results in an error.

Example

```
set_jaspersim_default_depth 4

set_jaspersim_mode parallel
set_jaspersim_partition_method default
set_jaspersim_partition_depth 2

set_jaspersim_mode parallel
set_jaspersim_partition_method file
set_jaspersim_partition_depth Partition.txt
```

See Also

set_jaspersim_mode,
set_jaspersim_partition_file

set_jaspersim_partition_file

Defines a partition file for parallel simulator partitioning.

Syntax

```
set_jaspersim_partition_file file_name
```

Returns

No return value

Default

""

Detailed Description

```
The Jasper partition file defines a list of instances that will become
partitions for the parallel simulation. Format the file as follows:
Line 1: slaves:
Line 2: instance_list
Line 3: instance_list
...
Line n: master:
Line n+1: instance
Line n+2: instance

Important things to know:
. Jaspergold has two partition types: "slave", which runs in parallel and
"master", which runs sequentially.
. Everything that is not a "slave" is a "master".
. When an instance is defined as "slave", all its children are included.
. Each line under "slaves:" is a "slave" partition.
. Each line under "master:" is a "slave" child that is forced to live in the
  "master".
. It is not possible to define nested instances as different partitions.
  For example: If "inst.a" and "inst.b" are defined as partitions, "inst"
  cannot be another partition. But if "inst" is a partition, "inst.a" and
  "inst.b" can be defined as "master".
. instance_list is a list of instances separated by a comma (for example:
  inst.a, inst.b).
. instance_list supports wildcards (* and ?).
. When using wildcards, if a matched instance is already in another partition,
  this instance is not added. Example:

  slaves:
  inst.a
  inst.*

  The first partition contains "inst.a" and the second partition contains all
  other child instances of "inst".
```

Example

```
set_jaspersim_partition_file my_partition_file
```

set_jaspersim_partition_method

Enables configuring partitions for the Jasper parallel simulator.

Syntax

```
set_jaspersim_partition_method (default | file)
```

Returns

No return value

Default

default

Detailed Description

The Jasper parallel simulator partitions the design to dispatch parallel jobs. Use this command to define the partitioning method.

Example

```
set_jaspersim_partition_method file

set_jaspersim_mode parallel
set_jaspersim_partition_method default
set_jaspersim_partition_depth 2

set_jaspersim_mode parallel
set_jaspersim_partition_method file
set_jaspersim_partition_depth Partition.txt
```

See Also

set_jaspersim_partition_depth,
set_jaspersim_partition_file

set_jaspersim_partition_min_size

Defines the minimum size of a partition in parallel simulator partitioning.

Syntax

```
set_jaspersim_partition_min_size N
```

Returns

No return value

Default

1024

Detailed Description

By default, Jasper devides a design by creating partitions tied to the design instances. This command enables control of the minimum size of a partition. If a partition does not match the criteria, it is automatically merged into the controller partition.

Example

```
set_jaspersim_mode parallel
set_jaspersim_partition_min_size 64
```

See Also

set_jaspersim_mode,
set_jaspersim_partition_depth

set_legacy_expression_parser

Sets the Tcl expression parser for design compilation, legacy or improved.

Syntax

```
set_legacy_expression_parser (on | off)
```

Returns

No return values

Default

off

Detailed Description

```
Sets the Tcl expression parser used during elaboration:
. The legacy expression parser, if command value is "on"
. The improved Tcl parser, if the command value is "off"

Compared to the legacy parser, the improved parser includes improvements in the
following four major areas:
. Complies with the SystemVerilog LRM, including the following improvements:
    . Extension of size and sign for all operators shall respect all
      LRM-defined rules
    . Correct precedence for not operator:
      The tool now parses "not a |=> b" as "(not a) |=> b" and not "not
      (a |=> b)"
    . Strict compliance with name escaping: Incorrectly escaped names are no
      longer supported

. Supports all SystemVerilog operators and constructions defined in the LRM,
  including the following:
    . Unary and binary operators such as "inside", "unpack", (>>), and others
    . Conditional edge event: @(posedge clk iff ~rst)
    . Part-select indexing (+:, -:)
    . SystemVerilog system functions such as $unsigned, $signed, and $clog2

. Closes gaps between embedded property support and Tcl expressions that were
  present in the legacy parser.

. Aligns tool messages with common Jasper elaboration flow, resulting in
  more meaningful error and warning messages.

NOTE:
. The parser does not currently support VHDL syntax mode ("elaborate -mode
  vhdl"). Jasper uses the legacy parser in this case.
. References to RTL functions and tasks are not supported.
. Macro references use the final value assigned during compilation.
. Operand name-lookup remains unchanged with the same limitations present in
  the legacy expression parser, including but not restricted to:
    . Names cannot refer to functions or tasks.
    . Names cannot access specific macro definitions from specific scopes.
. Do not attempt to use this command after design elaboration. In that case,
  Jasper issues an error.
```

Example

```
# Enabling the legacy Tcl parser.
set_legacy_expression_parser on

# Enabling the improved Tcl parser.
set_legacy_expression_parser off
```

set_liberty_cells_auto_blackbox

Configures the tool to blackbox all instances of Liberty(TM) cells.

Syntax

```
set_liberty_cells_auto_blackbox (on | off)
```

Returns

No return values

Default

off

Detailed Description

Use this command to ignore any definition for Liberty(TM) cells and
blackbox all related instantiations.

on: Force the blackboxing of all Liberty cells.
off: Do not force the blackboxing Liberty cells. This is the default.

NOTE: Enabling this command ensures that all Liberty cells are blackboxed.
However, disabling this command (or accepting the default, which is "off") does
not guarantee that Liberty cells will not be blackboxed for some other reason.

Example

```
set_liberty_cells_auto_blackbox on
```

See Also

liberty

set_liberty_cells_override_rtl_definition

Configures the tool to prioritize Liberty cell definitions over RTL.

Syntax

```
set_liberty_cells_override_rtl_definition (on | off)
```

Returns

No return values

Default

off

Detailed Description

Use this command to control whether the tool prioritizes Liberty(TM) or RTL definition of modules. When the configuration is "on", the tool ignores the RTL definition of modules that have a cell defined in libraries loaded using "liberty -load".

Example

```
set_liberty_cells_override_rtl_definition on
```

See Also

liberty

set_liberty_enable_cell_define

Encapsulates generated Verilog model for Liberty(TM) cells by "`celldefine".

Syntax

```
set_liberty_enable_cell_define (on | off)
```

Returns

No return values

Default

on

Detailed Description

Use this command to report whether the generated Verilog model for Liberty cells are encapsulated by pragma "`celldefine" and "`endcelldefine". This setting is valid only when you enable the Liberty function synthesis using "set_liberty_enable_function_synthesis on".

```
off: Generated Verilog models for Liberty cells are not encapsulated. This
     is the default.
on:  Generated Verilog models for Liberty cells are encapsulated by
     "`celldefine".
```

Example

```
set_liberty_enable_cell_define off
```

See Also

set_liberty_enable_function_synthesis

set_liberty_enable_function_synthesis

Specifies whether the tool synthesizes Liberty cell functions.

Syntax

```
set_liberty_enable_function_synthesis (on | off)
```

Returns

No return values

Default

off

Detailed Description

Use this command to specify whether the tool generates a functional Verilog model for Liberty cells during elaboration.
When this configuration is enabled, standard cells such as combinational gates, flops, and latches are no longer blackboxed, and have a functional model synthesized based on their Liberty description.

Example

```
set_liberty_enable_function_synthesis on
```

set_liberty_enable_multithread_load

Enables or disables multithread library loading.

Syntax

```
set_liberty_enable_multithread_load (on | off)
```

Returns

No return values

Default

on

Detailed Description

Use this command to enable or disable using multiple threads when loading libraries with "liberty -load".

on: Use 8 threads on "liberty -load". This is the default.
off: Use only 1 thread on "liberty -load".

Example

```
set_liberty_enable_multithread_load off
```

See Also

liberty

set_liberty_enable_timing_model_synthesis

Configures Liberty enable timing model synthesis.

Syntax

```
set_liberty_enable_timing_model_synthesis (on | off)
```

Returns

No return values

Default

off

Detailed Description

Use this command to enable timing propagation through memory cells in Multicycle Path Verification App.
When this configuration is enabled, Liberty memory cells are no longer blackboxed in the MCP analysis and will have a timing model synthesized based on their Liberty description.
Liberty memory models must have a complete definition of "memory_read" and "memory_write" conditions to be correctly analyzed. If information is not correct or is incomplete, it must be adjusted using the command "liberty -model_timing_arc".

Example

```
set_liberty_enable_timing_model_synthesis on
```

set_library_cell_detection

Enables automatic detection of library cells.

Associated App(s)

Formal Property Verification

Syntax

```
set_library_cell_detection (on | off)
```

Returns

No return values

Default

off

Detailed Description

By default, the tool recognizes library cells as any user-defined primitives or modules enclosed by the "celldefine" pragma. However, you can expand the tool's definition with this command, which applies a heuristic to classify as library cells any module that contains only a few single-bit inputs and outputs.

on
    Enable the automatic detection of library cells.

off
    Disable the automatic detection of library cells. This option is default.

NOTE: "set_library_cell_detection" is offered as an initial release to gather feedback from early adopters and finalize implementation for an upcoming release. Contact support@cadence.com to provide feedback or comments targeting the upcoming production version.

Example

```
set_library_cell_optimization 2
set_library_cell_detection on
analyze -verilog my_design.v -v my_lib.v
elaborate -top my_design
```

See Also

set_library_cell_optimization

set_library_cell_optimization

Enables optimizations for synthesis of library cells.

Associated App(s)

Formal Property Verification

Syntax

```
set_library_cell_optimization (0 | 1 | 2)
```

Returns

No return values

Default

1

Detailed Description

```
This command enables the simplification of the synthesis containing
library cells. The tool simplifies by inlining library cells into
the parent instances and converting the library cell logic into internal
logic.

The tool identifies library cells as any user-defined primitives or modules
enclosed by the "celldefine" pragma. Refer to "set_library_cell_detection" to
extend the library cell definition.

0
    Do not optimize library cells.

1
    Apply optimizations to combinational or unnamed UDP instantiations.

2
    Apply optimizations to any UDP instantiation.

NOTE:
. This command has no effect on library cells black boxed with "elaborate
  -bbox_m".
. This command does not honor "elaborate -bbox_i". Instead, it optimizes the
  library cells and does not black box them.
```

Example

```
set_library_cell_optimization 1
analyze -verilog my_design.v -v my_lib.v
elaborate -top my_design
```

See Also

set_library_cell_detection

set_log_timestamp_format

Sets the format for the timestamp of logging messages.

Syntax

```
set_log_timestamp_format format
```

Returns

No return values

Default

```
"%Y-%m-%d %H:%M:%S: "
```

Detailed Description

Use this command to specify the format for the timestamps displayed in logging messages. The format accepts a string containing a combination of regular characters and format specifiers. The format specifiers include those accepted by the "strftime" C function, in addition to the specifiers "%3" and "%6", for milliseconds and microseconds, respectively.

NOTES:

. The milliseconds and microseconds format specifiers are mutually exclusive.
. All characters after the milliseconds and microseconds format specifiers are treated as plain text.

Example

```
set_log_timestamp_format "%Y-%m-%d %H:%M:%S.%6: "

# The conversion specification for UNIX (Epoch) time gives the time in seconds
# that has elapsed since a specific date. This format is useful for calculating
# the time difference between two log messages:

set_log_timestamp_format "%s: "
```

See Also

set_log_timestamp_mode

set_log_timestamp_mode

Turns on/off the timestamp display for logging messages.

Syntax

```
set_log_timestamp_mode (on | off)
```

Returns

No return values

Default

off

Detailed Description

Use this command to specify whether logging messages are prepended by a timestamp.

NOTE: This command does not apply to non-Jasper Apps frontend messages that begin with VERI-, VHDL-, or VDB-.

Example

```
set_log_timestamp_mode on
```

See Also

set_log_timestamp_format

set_loop_filter_latch_patterns

Automatically filters loops that go through latch patterns causing false loops.

Syntax

```
set_loop_filter_latch_patterns (on | off)
```

Returns

No return values

Default

on

Detailed Description

This setting activates a structural pattern check for latches that cause combinational loops to become functionally false during loop detection with the "check_loop -latch_data" command.

The "check_loop... -compress_inner_loops" command does not support this setting because the tool performs pattern matches against unit combinational loops, and "-compress_inner_loops" returns a list of strongly connected components, which might include more than one combinational loop.

on
    Activate structural pattern matching.
    This option is the default.

off
    Deactivate structural pattern matching.

Example

```
set_loop_filter_latch_patterns off
```

See Also

check_loop

set_loop_filter_startpoints

Automatically filters startpoints in the design not connected to loops.

Syntax

```
set_loop_filter_startpoints (on | off)
```

Returns

No return values

Default

off

Detailed Description

Use this command to run the algorithm that removes startpoints that are not connected to any loops. This filtering occurs during the loop-detection preprocessing stage. We recommend using this option to reduce memory consumption by the "check_loop" command. It might add some overhead in time; hence, you should weigh the advantages of using it in an interactive flow.

on
    Activate the loop preprocessing startpoints filter.

off
    Deactivate the loop preprocessing startpoints filter.
    This option is the default.

Example

```
set_loop_filter_startpoints on
```

See Also

check_loop

set_loop_handling

Configures the default loop handling method.

Syntax

```
set_loop_handling (none | skip | assume_stable | break_functional | break)
```

Returns

No return values

Default

none

Detailed Description

Use this command to specify how you want the tool to handle loops in the COI of properties:

```
. none:        Do not handle loops.
. skip:        Do not attempt to prove properties that contain combinational
               loops in their cone of influence.
. assume_stable: Assume all loops in the design are stable.
               WARNING: Unstable loops handled by the "assume_stable" option
               might lead to wrong proof results.
               NOTE: Stable loops might have an arbitrary value at each cycle
               it is open (that is, when nothing is forcing its value).
. break_functional: Check the functionality of all combo loops and use the
               result to break them with the best approach. The flow has
               two steps. The first step enumerates all combo loops, and
               the second step analyzes whether the loops are functionally
               true loops.

               Use "set_loop_handling_fallback" to specify handling for
               unclassified loops.
```

NOTE: The functional loop analysis step uses the global environment, and it is not affected by task-specific configurations.

. break: Break combinational loops in the design. Combinational loops are non-sequential loops that go strictly through combinatorial gates or through combinatorial parts of sequential gates (for example, through the clock and reset pins of flops, or through the enable and data pins of latches).

This mode breaks loops through the async reset pin of flops, but does not break loops through clock/enable pins or through the data pin of latches. Use "check_loop -include_clock" or "check_loop -formal" to find clock/enable loops, and "check_loop -latch_data" to find latch data loops.

Breakpoint Information:
- Breakpoints are added per proof (by thread ID). Each proof setup might result in different breakpoint placements.
- When breakpoints are added, INFO message ICB009 reports the count of breakpoints and lists those on visible signals.
- To display all breakpoints associated with a specific property, including internal signals, use "check_loop -breakpoints".
- In Visualize, breakpoints are displayed as green waveforms. The "Why" action on breakpoint signals displays INFO message ICB008, which explains that the signal value was obtained by disconnecting its driving logic due to breakpoint combo loop.
- In Schematic Viewer, launched from Visualize, breakpoints are indicated by an orange "X!" mark on the affected signals.

Loop handling operations are environment operations. This means that they can invalidate proof and plot results when you change the handling. The following changes in loop handling clear proof results:

. Changing break to assume_stable
. Changing assume_stable to break
. Changing assume_stable or break to skip or none

The following changes to loop handling settings do not clear proof and plot results:

. Changing skip to assume_stable, break, or none
. Changing none to assume_stable, break, or skip

The command "set_loop_handling_fallback" has the same set of rules, but it does not support the "none" setting and it is only effective if loop handling is set

```
to "break".

CAUTION:
. Using the "assume_stable" approach with properties that have functionally
  true loops can lead to false positive proof results or an overconstrained
  environment.
. Using the "break" approach with properties that have functionally
  false loops can lead to false negative proof results.

NOTE:
. By default (if you keep the setting as "none"), the tool issues an error if
  a property has a loop in its COI. However, in some cases, Jasper Apps can
  safely optimize away loops, so you will only see the error if loops remain
  after the optimization. For example, constants can lead to loop removal in
  the case of a constant selector in a mux or a 0 going into an AND gate.
. The Coverage Unreachability (UNR) App ignores this command and automatically
  cuts loops it encounters.
. The FSV App properties do not support the "skip" or "assume_stable" arguments.
. The SEC, SPV, and X-Prop App properties do not support the "assume_stable"
  argument.
. The "autoprove" command and AutoProve-related commands also respect the loop
  handling method specified by this command.
. By default, the tool will assume all loops going through latch-enable pins
  are stable. To change this behavior, see "set_allow_latch_enable_loops".

To learn more about the combo loop flow, go to https://support.cadence.com and
use the following search string:

Jasper Combo Loop Handling Flow
```

Example

```
set_loop_handling skip

Setting Loop Handling with a Fallback Method
--------------------------------------------
set_loop_handling break_functional
set_loop_handling_fallback break
set_prove_check_loop_time 1
set_functional_loop_time 1
prove -property prop
```

See Also

set_allow_latch_enable_loops,
set_functional_loop_time,
set_loop_handling_fallback,
set_prove_check_loop_time,
check_loop

set_loop_handling_fallback

Configures the default loop handling fallback method.

Syntax

```
set_loop_handling_fallback (skip | assume_stable | break)
```

Returns

No return values

Default

skip

Detailed Description

Use this command to specify how you want the tool to handle unclassified loops in the COI of properties when using "set_loop_handling break_functional":

. skip: Do not attempt to prove properties that contain unclassified combinational loops in their cone of influence.
. assume_stable: Handle the loops as if they were not functionally true loops.
. break: Break the loops as if they were functionally true loops.

CAUTION: Using the "assume_stable" approach with properties that have functionally true loops can lead to false positive proof results or an overconstrained environment.

NOTE:
. The "autoprove" command and AutoProve-related commands also respect the loop handling method specified by this command.
. The SEC, SPV, FSV, and X-Prop App properties do not respect the "set_loop_handling_fallback" configuration modifying.
Instead, those apps will set the value of "set_loop_handling_fallback" as "break".

Example

```
set_loop_handling_fallback break
```

See Also

check_loop,
set_functional_loop_time,
set_loop_handling,
set_prove_check_loop_time

set_lpv_enable_corrupt_annotation

Enables or disables corrupt annotations in Visualize.

Syntax

```
set_lpv_enable_corrupt_annotation (on | off)
```

Returns

No return values

Default

on

Detailed Description

Enables or disables corrupt annotation for signals in Visualize.
When the configuration is enabled, the signals have a red cross-hatch annotation for the cycles where their related supplies is off.

Example

```
set_lpv_enable_corrupt_annotation off
```

See Also

check_lpv

set_lpv_enable_pin_corruption_in_liberty_bbox

Enable pin corruption on blackboxed instances of Liberty(TM) cells.

Syntax

```
set_lpv_enable_pin_corruption_in_liberty_bbox (on | off)
```

Returns

No return values

Default

off

Detailed Description

Use this command to disable the insertion of corrupt gates by LPV App in data pins on blackboxed instances of Liberty(TM) cells.

on: Insert corrupt gates.
off: Do not insert corrupt gates. This is the default.

Example

```
set_lpv_enable_pin_corruption_in_liberty_bbox on
```

See Also

liberty,
check_lpv

set_lpv_enable_upf_path_format

Configures the path format used by the UPF query commands.

Syntax

```
set_lpv_enable_upf_path_format (on | off)
```

Returns

No return values

Default

off

Detailed Description

Use this command to specify the format UPF queries should follow for the signal and instance paths they return.

- on: The format is the standard UPF format for hierarchical paths.
- off: The returned paths are in Verilog format.

Example

```
set_lpv_enable_upf_path_format on
```

See Also

check_lpv

set_lpv_enable_upf_signals

Enables or disables UPF signals.

Syntax

```
set_lpv_enable_upf_signals (on | off)
```

Returns

No return values

Default

off

Detailed Description

Enables or disables signals that are extracted from UPF representation for local handles, status, or values. These signals are identified by the prefix ":UPF", and the tool currently supports the following case: Simstate values of primary supply set handles of power domains.

Example

```
set_lpv_enable_upf_signals on
```

See Also

check_lpv

set_lpv_xprop_mode

Adds the power net of target to the precondition of X-Prop targets in LPV+XPROP.

Syntax

```
set_lpv_xprop_mode (on | off)
```

Returns

No return values

Default

off

Detailed Description

This command is useful only in LPV+XPROP flow. Use this command to avoid failures on targets when the power net of a target is down.

on
: Do not allow X-Prop targets to fail when a block is powered down.

  The power net of a target net is added as a precondition of X-Prop targets. For control and data targets that have internal preconditions, the power net is conjuncted to the internal precondition. The power net is also conjuncted with the precondition expression specified with the "-precond" option, for example, "check_xprop -create ... -precond expr". The command "assert -show..." shows the conjuncted power net in the precondition. In non-power-aware designs, this command does not have any effect.

off
: Do not add any power net to the precondition.
  This is the default.

Example

```
analyze -sv $RTL_DIR/dut.sv
elaborate -top dut

check_lpv -load_upf lowpower.upf
check_lpv -generate_power_design

clock clk
reset rst

set_lpv_xprop_mode on

check_xprop -create ....
check_xprop -create ....
check_xprop -prove -all
```

set_max_num_sub_expressions

Limits the number of sub-expressions for "get_decomposed_expression" signals.

Syntax

```
set_max_num_sub_expressions max_num_sub_expressions
```

Returns

No return value.

Detailed Description

Use this command to limit the number of sub-expressions in the driving expression of the signal argument provided to the "get_decomposed_expression" command.

NOTE: The default value for this property is 100.

Example

```
set_max_num_sub_expressions 50
get_max_num_sub_expressions
50
```

See Also

get_decomposed_expression

set_max_trace_length

Sets the limit for the length of traces to search for.

Syntax

```
set_max_trace_length [-task task_name] N
```

Returns

No return values

Default

0

Detailed Description

Use this command to set the limit for the length of traces to search for. When "max_trace_length" is set, if one engine reaches the length limit for some property, all other engines will stop working on that property.

NOTE: You can also use this command when specifying a "bounded proof", that is, a proof that terminates after reaching the specified maximum length. However, this command does not change how Jasper reports the result. A proof that reaches the max trace length is still shown as "undetermined". If you want Jasper to report that a proof to a specified bound should be considered "bounded proven" or "bounded unreachable" rather than "undetermined", you must also use "set_prove_target_bound".

-task task_name
    Set the maximum trace length for the specified task.

    NOTE: Replace "task_name" with a period (.) to specify the current task.

N
    Use the specified maximum length for traces.

NOTE:
    . To remove the limit, use 0.

    . This option is useful for detecting hidden counters.

    . Depending on engine_mode, a possible side effect is that this will also put a lower limit on how hard proofs ("proven" and "unreachable" results) are searched for.

    . This setting limits the Trace Attempt numbers. However, in some cases, Trace Attempt number K can generate a trace longer than K cycles. Examples include engines L and M and "prove -from".

Example

```
set_max_trace_length 25
```

See Also

prove,
set_engineJ_max_trace_length,
set_prove_target_bound,
visualize

set_memory_aware_export_memory_report

Enables or disables exporting the Memory Aware Proofs (MAP) memory report.

Syntax

```
set_memory_aware_export_memory_report (on | off)
```

Returns

No return values

Default

off

Detailed Description

Use this command to enable the MAP debug report for formal proof runs.
This report provides:
. Insights into engine jobs dispatched during runs, including machine sizes
  requested by ProofGrid for each job.
. Useful information to track and analyze memory-aware job dispatch behavior,
  helping you understand how resources are allocated during MAP runs.
By default, the MAP debug report is saved in the project directory:
jgproject/sessionLogs/session_0/map_report.csv.gz
To save the MAP debug report to a specific path, use the following command:
"set_memory_aware_memory_report_path <path>".

Example

```
set_memory_aware_proofs on
set_memory_aware_export_memory_report on
```

See Also

set_memory_aware_memory_report_path,
set_memory_aware_proofs,
set_memory_aware_initial_mem,
set_memory_aware_mem_factor,
set_memory_aware_min_mem,
set_memory_aware_max_mem,
set_proofmaster

set_memory_aware_initial_mem

Defines the initial memory limit for Memory Aware Proofs (MAP).

Syntax

```
set_memory_aware_initial_mem N
```

Returns

No return values

Default

0

Detailed Description

Use this command to set the initial memory prediction for engine job dispatch during MAP runs.
This prediction is used during the first proof run, when no historical memory usage data is available in the Proof Profiling Database (PPD).
The initial memory value helps determine the starting memory allocation size for engine jobs, ensuring adequate resources are allocated even in the absence of prior data.

NOTE:
. By default, the value is interpreted in Kibibytes.
. You can specify other units explicitly using:
  . M for Mebibytes
  . G for Gibibytes

Example

```
set_memory_aware_initial_mem 1024    # 1024 KiB
set_memory_aware_initial_mem 1M      # 1 Mebibyte
set_memory_aware_initial_mem 1G      # 1 Gibibyte
```

See Also

set_memory_aware_proofs,
set_memory_aware_mem_factor,
set_memory_aware_min_mem,
set_memory_aware_max_mem,
set_proofmaster

set_memory_aware_max_mem

Defines the maximum memory limit for Memory Aware Proofs (MAP).

Syntax

```
set_memory_aware_max_mem N
```

Returns

No return values

Default

0

Detailed Description

Use this command to set the maximum memory limit for engine job dispatch during MAP runs. This limit corresponds to the largest allocation size permitted by the farm for formal engine job dispatch.
Once the maximum memory limit is reached, ProofGrid stops launching engine jobs for the specific target property and notifies you that the memory limit has been hit. This helps prevent unnecessary resource allocation and ensures controlled scaling.

NOTE:
. The maximum memory limit must be greater than or equal to the initial and minimum memory limits defined in MAP.
. By default, values are interpreted in Kibibytes.
. You can specify other units explicitly using:
  . M for Mebibytes
  . G for Gibibytes

Example

```
set_memory_aware_max_mem 67108864K     # 64 Gibibytes
set_memory_aware_max_mem 65536M        # 64 Gibibytes
set_memory_aware_max_mem 64G           # 64 Gibibytes
```

See Also

set_memory_aware_proofs,
set_memory_aware_initial_mem,
set_memory_aware_mem_factor,
set_memory_aware_min_mem,
set_proofmaster

set_memory_aware_mem_factor

Defines the memory scaling factor for Memory Aware Proofs (MAP).

Syntax

```
set_memory_aware_mem_factor N
```

Returns

No return values

Default

0

Detailed Description

Use this command to define the Memory Aware Proofs factor, which adjusts the memory limit for engine job dispatch during MAP runs.
This factor is applied for scaling the memory budget for engine jobs when a running engine job exceeds the available allocated memory and terminates. In such cases, the next job is automatically dispatched with a larger memory allocation, scaled according to the specified factor.
The Memory Aware Proofs factor helps optimize resource allocation for memory-intensive jobs by proactively increasing memory limits.

NOTE:
. The factor accepts fractional values (for example: 1.2, 1.5, 2.5).
. The value must be greater than 1.

Example

```
set_memory_aware_mem_factor 1.5
```

See Also

set_memory_aware_proofs,
set_memory_aware_initial_mem,
set_memory_aware_min_mem,
set_memory_aware_max_mem,
set_proofmaster

set_memory_aware_memory_report_path

Sets the path for the Memory Aware Proofs (MAP) memory report.

Syntax

```
set_memory_aware_memory_report_path <path>
```

Returns

No return values

Default

off

Detailed Description

Use this command to specify the location for saving the MAP debug report when the "set_memory_aware_export_memory_report" option is enabled.

Example

```
set_memory_aware_proofs on
set_memory_aware_export_memory_report on
set_memory_aware_memory_report_path ~/memory_reports/
```

See Also

set_memory_aware_export_memory_report,
set_memory_aware_proofs,
set_memory_aware_initial_mem,
set_memory_aware_mem_factor,
set_memory_aware_min_mem,
set_memory_aware_max_mem,
set_proofmaster

set_memory_aware_min_mem

Defines the minimum memory limit for Memory Aware Proofs (MAP).

Syntax

```
set_memory_aware_min_mem N
```

Returns

No return values

Default

0

Detailed Description

Use this command to set the minimum memory limit for engine job dispatch during MAP runs. This limit corresponds to the smallest allocation size permitted by the farm for formal engine job dispatch.
Setting a minimum memory limit ensures that the allocated memory for the dispatched engine jobs does not fall below this limit, maintaining consistency and reliability in proof runs.

NOTE:
. The minimum memory limit must be less than or equal to the initial memory limit defined for MAP.
. By default, values are interpreted in Kibibytes.
. You can specify other units explicitly using:
  . M for Mebibytes
  . G for Gibibytes

Example

```
set_memory_aware_min_mem 10            # 10 Kibibytes
set_memory_aware_min_mem 100K          # 100 Kibibytes
set_memory_aware_min_mem 1M            # 1 Mebibyte
set_memory_aware_min_mem 1G            # 1 Gibibyte
```

See Also

set_memory_aware_proofs,
set_memory_aware_initial_mem,
set_memory_aware_mem_factor,
set_memory_aware_max_mem,
set_proofmaster

set_memory_aware_proofs

Enables or disables Memory Aware Proofs (MAP).

Syntax

```
set_memory_aware_proofs (on | off)
```

Returns

No return values

Default

off

Detailed Description

Use this command to enable or disable the Memory Aware Proofs (MAP) flow.
The MAP flow predicts the memory footprint for engine jobs
by using historic memory usage data available from the previous runs. The first proof run with MAP collects information about the memory consumption of engine jobs on various properties into the PPD database. In subsequent proof runs MAP uses the historic data as memory predictions to optimize engine job dispatch through PPD Replay, ensuring efficient resource allocation.
As the design and proof evolve, MAP continuously refines and updates the memory predictions in subsequent proof runs on similar design/properties.

Note: The MAP flow is currently supported by the following Jasper Apps: FPV (orchestration proof only), SEC, C2RTL, Superlint, Connectivity, CSR, LPV, and RVV. Other Jasper Apps do not support MAP.

Additionally, MAP is not supported with non-orchestration proofs or Hunt.

When MAP is enabled with non-orchestration proof flows, Hunt, or other Jasper Apps that do not support MAP, a fallback mechanism is triggered. This fallback uses a memory scan loop that scales memory for engine jobs based on the user-

```
defined initial, minimum, maximum, and scaling factor limits for MAP.

Note: When using MAP, memory predictions for engine jobs are stored in the
Proof Profiling Database (PPD) component of ProofMaster. To enable MAP,
ProofMaster must be explicitly enabled using the command "set_proofmaster on".

Note: When running MAP, the ProofGrid configuration must be updated to use
the appropriate template argument or environment-variable for specifying the
memory request (see the examples for different ProofGrid modes below).
The following three environment-variables, along with their corresponding
template arguments, have been added to capture the effective memory request
for engine jobs in KiB, MiB, and GiB, respectively:
1. $JASPER_ENGINE_JOB_MEMORY_REQUEST_KiB (%jasper_engine_job_memory_request_kib)
2. $JASPER_ENGINE_JOB_MEMORY_REQUEST_MiB (%jasper_engine_job_memory_request_mib)
3. $JASPER_ENGINE_JOB_MEMORY_REQUEST_GiB (%jasper_engine_job_memory_request_gib)

You can choose any one of these environment variables or the corresponding
template argument based on your ProofGrid configuration.

Examples:

ProofGrid Mode Shell:

 set_proofgrid_mode shell
 set_proofgrid_shell {/grid/sfi/farm/bin/bsub -q lnx64 -o /dev/null -K -P
  JG:2026.03:FAV:test -W 200:00 -R "(OSNAME == Linux && (OSREL==EE80
  ||OSREL==SLES15) && OSBIT==64)
  rusage[mem=%jasper_engine_job_memory_request_mib] span[hosts=1]"}
Or
 set_proofgrid_shell shell.sh
 shell.sh
 #!/bin/sh
 exec /grid/sfi/farm/bin/bsub -q lnx64 -o /dev/null -K -P JG:2026.03:FAV:test
  -W 200:00 -R "(OSNAME == Linux && (OSREL==EE80||OSREL==SLES15) &&
   OSBIT==64) rusage[mem=$JASPER_ENGINE_JOB_MEMORY_REQUEST_MiB]
   span[hosts=1]" $@

ProofGrid Mode LSF:

 set_proofgrid_mode lsf
 set_proofgrid_queue lnx64
 set_proofgrid_extra_args { -W 24:00 -P JG:2603:JG:PE -R "(OSNAME ==
  Linux && (OSREL==EE80||OSREL==SLES15) && OSBIT==64)
```

```
  rusage[mem=%jasper_engine_job_memory_request_mib] span[hosts=1]"}

Note: ProofGrid configuration can vary depending on your environment,
whether it is an on-prem farm, a cloud setup, or another custom infrastructure.
However, the MAP setup remains consistent when running with Jasper Cloud
Solution (JCS). When running MAP with JCS, you must specify one of the three
provided template arguments in your ProofGrid configuration within your LSF/NC
setup.

set_proofgrid_extra_args { -W 24:00 -P JG:2603:JG:PE -R "(OSNAME ==
 Linux && (OSREL==EE80||OSREL==SLES15) && OSBIT==64)
 rusage[mem=%jasper_engine_job_memory_request_mib] span[hosts=1]"}
```

Example

```
set_proofmaster on
set_memory_aware_proofs on
```

See Also

set_memory_aware_initial_mem,
set_memory_aware_mem_factor,
set_memory_aware_min_mem,
set_memory_aware_max_mem,
set_proofmaster

set_memory_aware_wastage_factor

Sets the memory wastage factor for ProofGrid workers.

Syntax

```
set_memory_aware_wastage_factor value
```

Returns

No return values

Default

0.5

Detailed Description

This command sets the memory wastage factor for ProofGrid workers. The value must be between 0 and 1 inclusive, with a default of 0.5.

The wastage factor controls how much of an idle worker's memory may go unused when it is reused for a new job. Given M1 (the memory of the existing idle worker) and M2 (the memory requested by the new incoming job), the waste fraction is calculated as (M1 - M2) / M1. The wastage factor caps this maximum allowable waste.

When set to 0, no waste is allowed: the worker's memory must exactly match the job's request (M1 must equal M2).

When set to 0.5, up to half of the worker's memory may go unused. For example, a worker with 8 GB of memory can run jobs requesting between 4 and 8 GB (since M1 can be at most 2 x M2).

When set to 1, any idle worker with enough memory can be reused regardless of how much memory is wasted (any M1 >= M2 qualifies).

Note: If Bridge is disabled, the value of this Tcl command is ignored.

Example

```
set_memory_aware_wastage_factor 0.5
set_memory_aware_wastage_factor 0
set_memory_aware_wastage_factor 1.0
```

See Also

set_proofgrid_bridge,
set_memory_aware_proofs

set_message

Manipulates error, warning, or info message or changes message severity.

Syntax

```
set_message -disable message_id
set_message -enable message_id
set_message -error message_id
set_message -warning message_id
set_message -info message_id
set_message -default (message_id |-all)
set_message  -unlimit_repeated_message message_id
            |-limit_repeated_message message_id
            |-limit_all_repeated_message
set_message  -limit_beta_warning_message
            |-unlimit_beta_warning_message
set_message  -limit_initial_release_warning
            |-unlimit_initial_release_warning
set_message -limit (number | none) [message_id]
```

Returns

. For warning and info messages (-disable, -enable, -error, -limit),
  this command triggers an error if the specified ID is not valid.
. For "-error" with front-end messages, the tool does not validate IDs, and
  this command only triggers an error if the ID does not start with VERI-,
  VHDL-, or VDB-.
. For "-warning", this command triggers an error if the specified ID is not
  valid.

Detailed Description

Use this command to manipulate and change severity for Jasper and front-end messages. This command is helpful when you want to filter out specific info and warning messages that are not meaningful to you or change the severity of messages to suit your needs.

```
NOTE: For ABVIP checkers, "elaborate" ignores all user-specified message
severity level changes, and "analyze" ignores all except the ones related to
ports of top ABVIP modules.

Command guidelines:
. Specify a single message ID with the command. (Use the command multiple times
  if you want to manipulate multiple messages.)
. Message IDs are case sensitive.

-disable message_id
    Disable the specified info or warning message.

    Important things to know:
    . You cannot specify an error message ID.
    . This command issues a special info message that appears in the
      "Warnings/Errors" message tab in the console.

-enable message_id
    Enable the specified info or warning message.

    NOTE: This command issues a special info message that appears in the
    "Warnings/Errors" message tab in the console.

-error message_id
    Change the severity of the specified message to "ERROR".

    Important things to know:
    . This switch applies broadly to messages beginning with VERI-, VHDL-, VDB-,
      or VNLR.
    . This switch applies broadly to Jasper IP-XACT warnings, which begin
      with WIPX.
    . This switch applies to the following Jasper messages:
      . ICK010, ICK011, ICK012, ICK013
      . WAP021
      . WAS020
      . WAS022
      . WCC001, WCC017
      . WCD036
      . WCDC096-WCDC133, WCDC201-WCDC217
      . WCK006, WCK013
      . WCON001, WCON002, WCON012, WCON015
      . WCOV015
      . WCSR001-WCSR006
```

```
      . WCTR001, WCTR032, WCTR033, WCTR035
      . WDE002
      . WFL008, WFL012
      . WG013
      . WLPV030, WLPV031, WLPV036, WLPV050, WLPV051, WLPV058, WLPV071, WLPV118
      . WMTR023, WMTR149
      . WNL006, WNL010, WNL015, WNL018, WNL021, WNL023, WNL032, WNL033, WNL038,
      . WNL051, WNL066, WNL073, WNL074, WNL084
      . WOBS001-WOBS004
      . WPF002
      . WPM018, WPM021, WPM022, WPM032, WPM033
      . WRA001
      . WRPT002
      . WRS014
      . WRS034
      . WRTL009-WRTL010
      . WSEC015
      . WSW008, WSW010
      . WTR034
      . WUNR001-WUNR013
      . WWA004, WWA005, WWA018
      . WWB001-WWB005, WWB007
      . WCG*: All WCG warnings can be upgraded to errors.
    . The tool generates an error message when the specified message ID is not
      supported.
    . If you upgrade a warning to an error, you can revert it to a warning with
      "set_message -warning".

-warning message_id
    Change the severity of the specified message to "WARNING".

    Important things to know:
    . This switch applies to all Jasper and front-end info messages.
    . This switch applies to a limited set of front-end error messages,
      including the following:
      . VDB-1010, VDB-1011, VDB-1041, VDB-9018
      . VERI-1010, VERI-1011, VERI-1042, VERI-1070, VERI-1084, VERI-1122,
        VERI-1139, VERI-1148, VERI-1158, VERI-1167, VERI-1180, VERI-1187,
        VERI-1216, VERI-1240, VERI-1241, VERI-1245, VERI-1264, VERI-1273,
        VERI-1274, VERI-1276, VERI-1289, VERI-1348, VERI-1349, VERI-1352,
        VERI-1370, VERI-1401, VERI-1419, VERI-1420, VERI-1437, VERI-1438,
        VERI-1439, VERI-1479, VERI-1556, VERI-1557, VERI-1558, VERI-1575,
        VERI-1576, VERI-1620, VERI-1691, VERI-1749, VERI-1760, VERI-1797,
```

VERI-1893, VERI-1895, VERI-1896, VERI-1906, VERI-1928, VERI-1952, VERI-1974, VERI-1976, VERI-1990, VERI-1991, VERI-2048, VERI-2058, VERI-2072, VERI-2079, VERI-2089, VERI-2094, VERI-2162, VERI-2228, VERI-2269, VERI-2326, VERI-2366, VERI-2367, VERI-2386, VERI-2468, VERI-2489, VERI-2524, VERI-2540, VERI-2558, VERI-2568, VERI-2575, VERI-2607, VERI-2624, VERI-2635, VERI-2660, VERI-2662, VERI-2669, VERI-2695, VERI-2705, VERI-2706, VERI-8002, VERI-8014, VERI-8021, VERI-8029, VERI-9021, VERI-9022, VERI-9024, VERI-9087, VERI-9099, VERI-9104

- VHDL-1160, VHDL-1291, VHDL-1454, VHDL-1591, VHDL-1669, VHDL-1704, VHDL-8010, VHDL-8011
- VNLR-1000, VNLR-1001, VNLR-1002, VNLR-1003, VNLR-1004, VNLR-1005, VNLR-1006, VNLR-1007, VNLR-1008, VNLR-1054

CAUTION:

- Downgrading VERI-1070 can result in an undetected synthesis-simulation mismatch.
- Although you can downgrade VERI-1419 (illegal operand for operator), it is not recommended for most cases because the triggering issue affects synthesis. That is, if you downgrade the error to a warning, the tool ignores the expression in which the operator occurs and everything that relies on that expression.
- Downgrading VERI-1557 to a warning (elements must both be signed or both be unsigned) can lead to different proof results.
- For VERI-1760, depending on the design and the usage of the union, you might lose some data or have undriven nets.
- For the UNR App, VERI-2367 (missing signal for implicit port connection) is a warning by default. For all other flows, it is an error by default.
- For VERI-2524, the default value of the input port might be discarded during synthesis.

. This switch applies to a limited set of Jasper error messages, including the following:
  - EAD019
  - ECC031, ECC032, ECC035, ECC036, ECC037, ECC038, ECC040, ECC041, ECC044, ECC051, ECC057, ECC064
  - ECDC100-ECDC105, ECDC110, ECDC141
  - ECON001-ECON018, ECONO20, ECON021
  - ECOV040, ECOV073, ECOV081
  - ECSR001-ECSR029
  - ECTR013, ECTR070
  - EFL058

```
      . EFSV017, EFSV040, EFSV051, EFSV054, EFSV072
      . EG017
      . EGI031
      . EIPX024, EIPX025
      . EJT009, EJT015
      . ELIB021-ELIB033, ELIB036, ELIB042, ELIB043
      . ELPV004, ELPV006, ELPV113, ELPV021, ELPV055, ELPV056, ELPV057, ELPV102,
        ELPV132, ELPV159, ELPV167, ELPV168, ELPV169, ELPV170, ELPV171, ELPV172,
        ELPV180, ELPV501, ELPV507, ELPV521, ELPV524
      . ENL098, ENL141, ENL169, ENL173, ENL175, ENL187
      . EOBS* messages (all)
      . EPC311
      . EPF020, EPF060, EPF080
      . EPM016, EPM029, EPM031, EPM044
      . EPST038
      . ERS034, ERS059, ERS060, ERS063
      . ESA010, ESA011
      . ESEC093, ESEC154, ESEC155, ESEC156, ESEC157
      . ESPV026, ESPV326
      . ETR055, ETR056, ETR058
      . EWB001-EWB056
    . The tool generates an error message when the specified message ID is not
      supported.
    . This switch enables a limited set of Jasper and front-end messages that
      the tool does not print by default. Refer to the command reference manual
      appendix "Disabled Front-End Messages" for a list of messages that can be
      warnings and a list of messages that can be either warnings or info
      messages.
    . Downgrading message severity is not recommended because it can produce
      unexpected results and adversely affect debugging.

-info message_id
    Change the severity of the specified message to "INFO" or enable specified
    front-end messages.

    NOTE:
    . This switch applies to all Jasper and front-end warning messages.
    . This switch applies to a limited set of front-end messages, including
      the following:
      . VERI-1010, VERI-1070, VERI-1084, VERI-1122, VERI-1148, VERI-1158,
        VERI-1187, VERI-1216, VERI-1240, VERI-1241, VERI-1264, VERI-1276,
        VERI-1348, VERI-1349, VERI-1437, VERI-1438, VERI-1439, VERI-1557,
        VERI-1558, VERI-1620, VERI-1760, VERI-1797, VERI-1896, VERI-1906,
```

```
        VERI-2058, VERI-2094, VERI-2162, VERI-2228, VERI-2269, VERI-9087
      . VHDL-1454, VHDL-1591, VHDL-1669, VHDL-1704
      . VNLR-1000, VNLR-1001, VNLR-1002, VNLR-1003, VNLR-1004, VNLR-1005,
        VNLR-1006, VNLR-1007, VNLR-1008, VNLR-1054
    . This switch applies to a limited set of Jasper error messages,
      including the following:
      . EFL058
      . EIPX024, EIPX025
      . ELIB022
      . ELPV055, ELPV056, ELPV057, ELPV102
      . ENL098, ENL141
      . EOBS001, EOBS002, EOBS003, EOBS004, EOBS005, EOBS006, EOBS007, EOBS008,
        EOBS009, EOBS010, EOBS011, EOBS012, EOBS013, EOBS014, EOBS015, EOBS016
      . ERS063
    . This switch enables a limited set of Jasper and front-end messages that
      the tool does not print by default. Refer to the command reference manual
      appendix "Disabled Front-End Messages" for a list of messages that can be
      either warnings or info messages.

-default (message_id |-all)
    Re-instate the default message severity level for the specified message or
    for all messages.

-unlimit_repeated_message message_id
    |-limit_repeated_message message_id
    |-limit_all_repeated_message
    Turn on or off specified repetitive front-end messages.

    NOTE:
    . By default, the tool does not display repetitive front-end messages.
    . Specify your preferences for front-end messages that have the same source
      location before elaborating the design.
    . Use "-unlimit_repeated_message" if you want the tool to print all
      front-end messages that have the same source location.
    . Use "-limit_repeated_message" to specify a message you do not want the
      tool to repeat.
    . Use "-limit_all_repeated_message" to specify the default handling. With
      this switch, the tool does not repeat front-end messages that have the
      same source location.

-limit_beta_warning_message
    |-unlimit_beta_warning_message
    Turn off repetitive beta warning messages or reinstate the default.
```

```
    NOTE:
    . By default, the tool prints a beta warning message each time you run a
      beta command or use a beta feature.
    . Use "-limit_beta_warning_message" to print the warning only the first time
      you run a beta command or use a beta feature.
    . Use "-unlimit_beta_warning_message" to print the warning every time you
      run a beta command or use a beta feature. This is the default.

-limit_initial_release_warning
    |-unlimit_initial_release_warning
    Turn off repetitive initial release messages or reinstate the default.

    NOTE:
    . By default, the tool prints an initial release message each time you run
      an initial release command or use an initial release feature.
    . Use "-limit_initial_release_warning" to print the warning only the first
      time you run an initial release command or use an initial release
      feature.
    . Use "-unlimit_initial_release_warning" to print the warning every time
      you run an initial release command or use an initial release feature.
      This is the default.

-limit (number | none) [message_id]
    This command enables you to set the maximum number of repetitions for all
    Jasper warning and info messages. If a specific "message_id" is provided,
    the command adjusts the repetition limit for that message only.

    NOTE:
    . This command is not allowed with error messages and messages upgraded to
      error by "set_message -error" command.
    . Use "none" argument to remove message limitations.
```

Example

```
set_message -disable ICD001
set_message -error VERI-1173
set_message -error WNL038
set_message -warning VERI-1158
set_message -limit 5 VNLR-1026
set_message -limit 20
```

See Also

set_log_timestamp_format,
set_log_timestamp_mode

set_message_history_limit

Sets the limit of stored user messages issued in the current session.

Syntax

```
set_message_history_limit limit
```

Default

100

Detailed Description

Use this command to change the limit of user messages that can be kept in storage for "get_message -list".
When the limit is surpassed, the oldest issued messages are erased from memory and, therefore, will not be listed by the aforementioned command's switch.

NOTE: To remove the limit, use $.

Example

```
set_message_history_limit 1000
```

set_missing_entries_as_x_in_comb_udp

Sets how missing entries in combinational UDPs are handled during elaboration.

Syntax

```
set_missing_entries_as_x_in_comb_udp (on | off)
```

Returns

No return values

Default

on

Detailed Description

Use this command to control how missing entries in combinational UDPs are handled during elaboration. Starting with Jasper 2024.03, missing entries drive the output state to the unknown value X by default, as defined in SV LRM.

off: Return to the pre-Jasper 2024.03 behavior, where missing entries in combinational UDPs are ignored during UDP synthesis.

on: Drive the output state of missing entries in combinational UDPs to the unknown value X. This is the default.

Example

```
set_missing_entries_as_x_in_comb_udp off
```

See Also

set_message -warning

set_module_elaboration_memory_limit

Sets the maximum memory footprint that a module can reach during elaboration.

Syntax

```
set_module_elaboration_memory_limit value
```

Returns

No return values

Default

0 (unlimited)

Detailed Description

Sets a memory limit (RSS), in MB, for modules while running the "elaborate" command.

When you set a limit, the tool periodically measures the memory footprint of the module it is elaborating. If it exceeds the limit, the tool interrupts the module elaboration and automatically black boxes it before resuming the elaboration flow.

This configuration is useful when elaborating designs with memory-heavy modules in memory-constrained environments, that is, machines that do not have enough free memory to elaborate the entire design. We recommend setting this configuration for applications where module black boxing is not an issue.

If the tool applies an automatic black box, it prints a warning message (VERI-9082 or VHDL-9058).

NOTE:
. To remove the limit, use 0.
. Run this command before "elaborate".
. This setting polls the memory footprint every 5 seconds.

Example

```
set_module_elaboration_memory_limit 35000; # MB
```

See Also

elaborate

set_no_fallback_in_use_clause

Specify whether fallbacks are active when handling use clauses in SystemVerilog.

Syntax

```
set_no_fallback_in_use_clause (on | off)
```

Returns

No return values

Default

off

Detailed Description

Use this command to control the fallback mechanisms when processing use clauses in SystemVerilog configurations during elaboration.

off: Use default clauses to resolve use clauses in SystemVerilog configurations when they failed during elaboratation. This is the default.

on: Skip default clauses when resolving use clauses in SystemVerilog configurations during elaboration. When active, this setting leads to an error if the use clause fails.

Example

```
set_no_fallback_in_use_clause on
```

See Also

set_message -warning

set_optimize_cell_netlists

Enables optimization for cell netlists that behave as a single gate.

Syntax

```
set_optimize_cell_netlists (all | celldefine | none)
```

Returns

No return values

Default

none

Detailed Description

This command enables the simplification of the synthesis containing simple modules or entities. The tool simplifies by replacing them by equivalent gates.

The tool identifies modules/entities that are equivalent to a gate and replaces their instantiations by gates.

For instance, consider the following module:

```
    module my_and_4(output [3:0] out, input [3:0] in1, in2);
        and (out[3], in1[3], in2[3]);
        and (out[2], in1[2], in2[2]);
        and (out[1], in1[1], in2[1]);
        and (out[0], in1[0], in2[0]);
    endmodule
```

    All instances of "my_and_4" are replaced by a wide AND gate.

none
    Do not optimize simple modules or entities.

celldefine
    Try to apply optimizations only to modules marked as celldefine.

all
    Try to apply optimizations to any module or entity.

NOTE:
. Signal names inside optimized cells are lost.
. Registers inside optimized cells are preserved.

Example

```
set_optimize_cell_netlists celldefine
analyze -sv my_design.v -v my_lib.v
elaborate -top my_design
```

set_parallel_synthesis_directory

Sets the directory where parallel synthesis can write temporary files.

Syntax

```
set_parallel_synthesis_directory dir_name
```

Returns

No return values

Default

When "set_parallel_synthesis_dispatch_mode" is "local", default value is "/tmp/"; otherwise, the default value is the project directory.

Detailed Description

Parallel synthesis writes temporary files as part of its process communication. Use this command to specify where you want the tool to write these files.

By default, the flow writes into "/tmp/" directory, unless the "set_parallel_synthesis_dispatch_mode" is not local, in which case, the flow writes its files into the project directory.

NOTE: When "set_parallel_synthesis_dispatch_mode" is not local, use the "set_parallel_synthesis_directory" command to specify a path that all cluster machines can access.

See Also

elaborate,
set_parallel_synthesis_dispatch_mode,
set_parallel_synthesis_mode,
set_parallel_synthesis_shell

set_parallel_synthesis_dispatch_mode

Specifies how to spawn multiple synthesis processes, locally or on a shell grid.

Syntax

```
set_parallel_synthesis_dispatch_mode (local | shell)
```

Returns

No return values

Default

local

Detailed Description

```
Parallel synthesis enables the "elaborate" command to spawn multiple
synthesis processes (jg_frontend). This command specifies how the tool spawns
jobs.

Helpful information about using Runtime NetworkComputer(TM) with Jasper
Apps:

. Consult your NetworkComputer documentation for installation, configuration,
  and limitations.
. You might need to set up authentication such that you can log in to
  all machines in the cluster without a password. See for example
  "ssh-keygen" and consult your NetworkComputer documentation for
  details.

local
    Run the processes on your local machine (the machine running the
    Jasper Apps session).

shell
    Use a custom command or script to spawn the jobs, see
    "set_parallel_synthesis_shell". Ensure that the
    "set_parallel_synthesis_directory" command specifies a path that
    all cluster machines can access.
```

Example

```
set_parallel_synthesis_dispatch_mode shell
set_parallel_synthesis_shell "[get_proofgrid_shell]"
```

See Also

elaborate,
set_parallel_synthesis_directory,
set_parallel_synthesis_mode,
set_parallel_synthesis_num_process,
set_parallel_synthesis_shell

set_parallel_synthesis_dispatch_timeout

Sets the time that the flow waits for jobs to be launched in "shell" mode.

Syntax

```
set_parallel_synthesis_dispatch_timeout time_limit
```

Returns

No return values

Default

0s (unlimited)

Detailed Description

Parallel synthesis enables the "elaborate" command to spawn multiple compilation processes. This command specifies how much time the flow waits for jobs to be launched according to "parallel_synthesis_shell" when "parallel_synthesis_dispatch_mode" is set to "shell" mode.

time_limit
    Time limit while waiting for compilation jobs to be launched.

    The tool supports three formats for the time limit:
    . ISO8601: hh':'[mm[':'[ss['.'[fff]]]]]
        Examples: 2:30, 2:, 0:0:23, 2:30:05.235
    . Jasper: [dd'd'][hh'h'][mm'm'][ss[.fff]'s']
        Examples: 2h30m, 2h, 23s, 2h30m05.235s
    . Seconds: sssss[.fff]['s']
        Examples: 9000s, 7200, 23s, 9005.235s

    To reset the default (unlimited), use 0s.

Example

```
# configures parallel synthesis to wait for up to 5 min for jobs to be launched
set_parallel_synthesis_dispatch_timeout 5m
```

See Also

set_parallel_synthesis_dispatch_mode,
set_parallel_synthesis_shell

set_parallel_synthesis_manual_partition_file

Sets the path to the file that specifies how to partition the design.

Syntax

```
set_parallel_synthesis_manual_partition_file full_path_to_partition_file
```

Returns

No return values

Default

""

Detailed Description

Parallel synthesis enables the "elaborate" command to spawn multiple synthesis processes (jg_frontend). This setting specifies what part of the design each job handles.

The tool partitions the design based on a user-created file. Use the command "set_parallel_synthesis_manual_partition_file" to set the file path.

Each line in the file you create represents a job. Add a semicolon-separated list of modules for each job.

Example:
1) Partition file with 2 jobs:
   A;C
   B;D
2) Partition file with 4 jobs:
   subA1;d3
   subA2;d4
   d1
   d2

NOTE:
. If the partition file has more jobs than processes configured with "set_parallel_synthesis_num_process", the tool waits until a job finishes to start the next job.
. The partition file should have only modules. Instances are not valid inputs.

Example

```
set_parallel_synthesis_manual_partition_file /home/user/workspace/partitionFile
```

See Also

set_parallel_synthesis_mode,
set_parallel_synthesis_num_process,
set_parallel_synthesis_partition_method

set_parallel_synthesis_memory_limit

Sets the memory limit for parallel synthesis jobs.

Syntax

```
set_parallel_synthesis_memory_limit memory_limit
```

Returns

No return values

Default

0 (unlimited)

Detailed Description

Parallel synthesis enables the "elaborate" command to spawn multiple compilation processes. This command specifies the maximum amount of memory that each parallel synthesis job can use.

memory_limit
    Memory limit in MB for each parallel synthesis job.

    To reset the default (unlimited), use 0.

Example

```
# configures parallel synthesis to use a maximum of 1024 MB of memory per job
set_parallel_synthesis_memory_limit 1024
```

See Also

set_parallel_synthesis_mode

set_parallel_synthesis_mode

Sets the parallel synthesis mode for the "elaborate" command.

Syntax

```
set_parallel_synthesis_mode (on | off)
```

Returns

No return values

Default

off

Detailed Description

Parallel synthesis enables the "elaborate" command to spawn multiple synthesis processes (jg_frontend). Use this command to control whether the feature is on or off. If parallel synthesis is off, the tool ignores all other parallel synthesis configuration commands.

By default, processes are launched on the local machine. To run processes on different machines, use the command "set_parallel_synthesis_dispatch_mode" to configure your farm.

NOTE:
. To configure how the design is partitioned for each process, see "set_parallel_synthesis_partition_method".
. At the end of sequential elaboration, the tool might print info message IPS010, which suggests a parallel synthesis setup when sequential elaboration time is at least 60 seconds and the expected time saving is at least 5%.

WARNING: To successfully run parallel synthesis, turn this command on before the "analyze" command.

LIMITATIONS:
. parallel synthesis issues warning message WPS002 and is automatically disabled for any of the following unsupported flows:
  . PSL inheritance with default clock
  . set_elaborate_single_run_mode off
  . Cpi2Jetlist

See Also

analyze,
elaborate,
set_parallel_synthesis_dispatch_mode,
set_parallel_synthesis_num_process,
set_parallel_synthesis_partition_method

set_parallel_synthesis_num_process

Sets the number of processes spawned in parallel synthesis mode.

Syntax

```
set_parallel_synthesis_num_process N
```

Returns

No return values

Default

6

Detailed Description

Use this command to set the number of processes launched by the parallel synthesis flow.

Example

```
set_paralllel_synthesis_num_process 3
```

See Also

set_parallel_synthesis_mode,
set_parallel_synthesis_num_process,
set_parallel_synthesis_shell

elaborate,
set_parallel_synthesis_dispatch_mode,
set_parallel_synthesis_manual_partition_file,
set_parallel_synthesis_mode,
set_parallel_synthesis_partition_method,
set_parallel_synthesis_shell,
set_parallel_synthesis_shell_stop

set_parallel_synthesis_partition_method

Sets the method for parallel synthesis partitioning for the design.

Syntax

```
set_parallel_synthesis_partition_method (auto | manual | dynamic)
```

Returns

No return values

Default

dynamic

Detailed Description

Parallel synthesis enables the "elaborate" command to spawn multiple synthesis processes (jg_frontend). This command specifies the partitions each process handles.

Consider the following instance tree of a design for further explanation:

```
    * top (top)
    |   * a (A)
    |   |   * subA1 (subA1)
    |   |   * subA2 (subA2)
    |   * a1 (A)
    |   |   * subA1 (subA1)
    |   |   * subA2 (subA2)
    |   * b1 (B)
    |   * c1 (C)
    |   * d(D)
    |   |   * d1 (subD1)
    |   |   * d2 (subD2)
    |   |   * d3 (subD3)
    |   |   * d4 (subD4)
```

A partition for parallel synthesis is a group of sub-hierarchies of the design that will be synthesized in a different process, and each partition consists of a group of modules that represent the starting points of the sub-hierarchies.

Examples: Consider the partitions {A}, {B, subD1}, {C, subD2, subD3}, and {subD4}.
{A} will synthesize the modules A, subA1, subA2, top, and D.
{B, subD1} will synthesize B and subD1, top, and D.
{C, subD2, subD3} will synthesize C, subD2 and subD3, top, and D.
And {subD4} will synthesize subD4, top, and D.

NOTE: All partitions will synthesize top and D, because they are not part of any partition.

WARNING:
- The top module cannot be part of any partition.
- If one module is in a partition, its children should not be added to any other partition.
  Example: You cannot add A and subA1 in different partitions.
- The modules that are not part of any sub-hierarchy are synthesized by all partitions.

auto
The tool automatically partitions your design by equally distributing top's children among the number of processes configured with "set_parallel_synthesis_num_process". The max number of partitions generated is equal to the number of processes.
Example:
1) set_parallel_synthesis_num_process 2
   Partitions are: {A, C} and {B, D}.
2) set_parallel_synthesis_num_process 3
   Partitions are: {A, D}, {B}, and {C}.
3) set_parallel_synthesis_num_process 4
   Partitions are: {A}, {B}, {C}, and {D}.

NOTE:
- If the top module has only one child, the tool goes deeper in the tree until it finds a module with more than one child.
- Automatic partitioning DOES NOT analyze which modules consume more resources, which means that the generated partitions might not be optimal.

```
manual
    The tool partitions the design based on a user-created file. Set the file
    path with the "set_parallel_synthesis_manual_partition_file" command. Each
    line in the file you create represents a job. Add a semicolon-separated
    list of modules for each job.

    Example:
    1) Partition file with 2 jobs:
       A;C
       B;D
    2) Partition file with 4 jobs:
       subA1;d3
       subA2;d4
       d1
       d2

    NOTE: If the partition file has more jobs than processes configured with
    "set_parallel_synthesis_num_process", the tool waits until a job finishes
    to start the next job.

dynamic
    The tool partitions the design based on process availability. All design
    modules are queued and each one is assigned to the first process that
    becomes available. This optimizes resource use by minimizing idle time.

    NOTE:
    . Restrictions of previous methods DO NOT apply to dynamic partitioning.
      Any module can be assigned to any process regardless of its hierarchical
      depth.
    . Current limitations include modules that have interface ports or type
      parameters. These modules are always assigned to the same process
      handling the parent modules.
```

See Also

set_parallel_synthesis_manual_partition_file,
set_parallel_synthesis_mode,
set_parallel_synthesis_num_process

set_parallel_synthesis_shell

Sets the command to launch parallel jobs when the dispatch mode is "shell".

Syntax

```
set_parallel_synthesis_shell full_path_to_shell_plus_args
```

Returns

No return values

Default

""

Detailed Description

Parallel synthesis enables the "elaborate" command to spawn multiple synthesis processes (jg_frontend). This command specifies how the jobs are spawned when "parallel_synthesis_dispatch_mode" is set to "shell".

Important things to know:

. The parallel_synthesis_shell must be a string beginning with the full path to an executable. As an option, that string can continue with a space-separated set of extra arguments that will be given to the executable. You can use double and single quotes to form an argument that contains spaces. The unquoted token "%job_name" will be replaced with a unique Jasper front end ID. This can be useful to ensure that jobs still in a batch queue are killed as "elaborate" exits (see "set_parallel_synthesis_shell_stop").
. The executable must propagate (that is, tunnel) stdin/stdout reliably from and to the job.
. The parallel_synthesis_shell executable must have a lifetime that is as long as the jg_frontend process it started. For example, when using Platform LSF(R) mode, use non-interactive queues, and use "-K" to cause bsub to wait for the jg_frontend job to complete before exiting. If you do not use "-K", you might receive numerous LSF messages stating that the jg_frontend jobs

```
  failed and "Exited with exit code 1" due to (for example) the following:
  jg_frontend: connect: Connection refused
  jg_frontend: failed to connect
. If parallel_synthesis_mode is "shell", you must update the value of
  parallel_synthesis_shell because the default value is an empty string.
. parallel_synthesis does not robustly support automatic re-queuing of killed
  jobs, but you can disable the feature with "bsub -rn" and rely on the
  built-in parallel_synthesis restart mechanism.

NOTE: The jobs do not have any wait states, which can sometimes be the
case with other tools that require interactive mode. The executable can
use stderr for its own output, which will be sent to the terminal starting
the tool. The executable might also issue messages on stdout under very
limited circumstances:

. The messages must come before and/or after all messages the job produces.
. Use the "set_parallel_synthesis_directory" command to specify a path that all
  cluster machines can access.

If these rules are not respected, Jasper parallel_synthesis will not work
reliably.

Troubleshooting:
To get more insight into why your parallel_synthesis jobs fail to launch,
use the built-in debugging features of the programming language you use for your
parallel_synthesis_shell as shown in the bash examples below.

1. Create the following executable file:

   debug.sh
   #!/bin/bash
   exec /bin/bash -xv "$@" 2>&1

2. Prefix your parallel_synthesis_shell setting with "./debug.sh":

   set_parallel_synthesis_shell
   {myscripttodebug.sh}

   becomes

   set_parallel_synthesis_shell
   {./debug.sh myscripttodebug.sh}
```

With this addition to the command, the exact UNIX command executed will be echoed to the proof messages; however, if you prefer to have the exact UNIX command executed being echoed on stderr instead, use the following command:

```
set_parallel_synthesis_shell
{/bin/bash -xv myscripttodebug.sh}
```

As an alternative, you can add the following to the beginning of myscripttodebug.sh:

```
#!/bin/bash -xv
```

If your parallel_synthesis_shell is written in another language (for example, C, Tcl, or Java) use the debugging features of that language.

Example

```
set_parallel_synthesis_shell {/usr/bin/ssh -x myhost}

set_parallel_synthesis_shell {bsub -J %job_name -K -q lnx64 -R "OSNAME==Linux &&
OSREL==EE50 rusage[mem=4000] span[hosts=1]"}
set_parallel_synthesis_shell_stop {bkill -J %job_name }

# In your wrapper file, "exec" the bsub command
set_parallel_synthesis_shell {./my_bsub_wrapper.sh}

set_parallel_synthesis_shell {/usr/local/lsf/bin/bsub -J %job_name -Ip -q normal -R
"select[type==X86_64  && mem>2000] rusage[mem=1000] span[hosts=1]"}
set_parallel_synthesis_shell_stop {/usr/local/lsf/bin/bkill -J %job_name }
```

See Also

set_parallel_synthesis_directory,
set_parallel_synthesis_dispatch_mode,
set_parallel_synthesis_num_process,
set_parallel_synthesis_shell_stop

set_parallel_synthesis_shell_stop

Sets the jg_frontend jobs clean-up command when the dispatch mode is "shell".

Syntax

```
set_parallel_synthesis_shell_stop full_path_to_shell_plus_args
```

Returns

No return values

Detailed Description

Parallel synthesis enables the "elaborate" command to spawn multiple synthesis processes (jg_frontend). Use this command to specify a stop hook that will be executed when the "elaborate" command is about to exit. The token "%job_name" is replaced with a unique jg_frontend ID. This can be useful to ensure that jobs still in a batch queue are killed as "elaborate" exits.

Example

```
set_parallel_synthesis_shell {/usr/local/lsf/bin/bsub -J %job_name }
set_parallel_synthesis_shell_stop {/usr/local/lsf/bin/bkill -J %job_name }
```

See Also

set_parallel_synthesis_shell

set_per_property_simplification

Controls whether to run a property-specific simplification step during "prove".

Syntax

```
set_per_property_simplification (on | off)
```

Returns

No return values

Default

off

Detailed Description

With "per_property_simplification on" most single-property engines run an additional simplification step as part of starting a proof for a property.

NOTE:

- The setting only apply to engines: B, D, M, G, G2, K, I, N, C, C2, AB, AD, AM, and AG. It has no direct effect on other engines.
- per_property_simplification is expected to run fast and have modest memory requirements (much cheaper than proof_simplification). There can, however, be exceptions to this expectation.
- per_property_simplification can, on its own, sometimes prove properties or provide min_length and/or max_length bounds. Sometimes, it can leave a simplified problem for the proof engine to prove. In other cases, it fails to provide any benefit.

Example

```
set_per_property_simplification on
```

See Also

prove

set_persistent_proofgrid_timeout

Sets the persistency timeout value for idle ProofGrid workers.

Syntax

```
set_persistent_proofgrid_timeout value
```

Returns

No return values

Default

30s

Detailed Description

This command lets you configure the idle-timeout period for persistent ProofGrid workers. If a worker remains idle longer than the specified timeout, it will send a termination notification to the Bridge and self-terminate.

Example

```
set_persistent_proofgrid_timeout 60
set_persistent_proofgrid_timeout 60s
set_persistent_proofgrid_timeout 5m
set_persistent_proofgrid_timeout 6h
set_persistent_proofgrid_timeout 1h05m25s
set_persistent_proofgrid_timeout 00:15
set_persistent_proofgrid_timeout 20:07:05
```

set_preference

Sets the preference to the specified value.

Syntax

```
set_preference {preference_name} preference_value
```

Returns

No return values.

Detailed Description

Use this command to set the value for a preference. The values defined using this command have precedence over values configured using the Preferences dialog (menu "Edit > Preferences").

Methods for Changing Preferences
==========
Jasper supports the following ways to set preferences.
. Run the "set_preference" command on the command line or include it in a setup Tcl file.
. Modify the "~/.config/cadence/jasper.conf" file.
. Use the GUI Preferences dialog, which you can access from the Edit menu.

Preference-Setting on the Command Line

Jasper supports setting preferences with the "set_preference" command. The "Preference Mapping" section below lists supported preferences. Possible values for the settings are: Boolean, integer, text, and color (hex format).

The tool stores "set_preference" command settings in the Jasper project directory. Thus, you do not need to reissue the "set_preference" command in a new session if you use the same project directory.

Preference-Setting in the "jasper.conf" File

You can modify the "~/.config/cadence/jasper.conf" file with any of the key/value pairs listed in the "Preference Mapping" section below using the "jasper.conf" formatting.

Example:

The "Visualize Tab" subsection of the "Preference Mapping" section below lists the following entry.

. Key   : Visualize/WhySettings/IsStopAtHierarchyChange
  . Type: Bool
  . Name: Visualize > Stop at hierarchy change

Format the content in the "jasper.conf" file as shown below.

```
[Visualize]
WhySettings\IsStopAtHierarchyChange=false
```

Formatting notes:
. The first element in the key string becomes the header, and you must enclose it with square brackets.
. The separator changes from forward to backward slash.
. The "=" separates the key and its value.

The settings you specify with the Preferences dialog are stored in the "jasper.conf" file, but some preferences can only be specified in the "jasper.conf" file, which is the case for temporary solutions for specific issues. We retain support for these preferences for backwards compatibility.

Example: The following code highlighting preferences can improve performance in terms of run times for Why and Driver operations on large designs.

```
[SourceCodeBrowser]
IncrHighlight_Disabled=(true|false)
IncrHighlight_Threshold=<N>
```

Refer to the "Jasper Platform and Formal Property Verification App User Guide" for additional information about settings (Help > User Guide).

NOTE: You can find examples of the configuration files in the installation directory in the "etc/res/settings/jasper.conf.example" file.

Preference-Setting with the GUI

```
-------------------------------
The key/value pairs listed in the "Preference Mapping" section below represent
preferences you can specify with the GUI. The tool captures modified defaults
in the "~/.config/cadence/jasper.conf" file, and they become the defaults
either immediately or in future Jasper sessions. (The tool issues a
message if a change requires a restart.)

Refer to the "Jasper Platform and Formal Property Verification App User
Guide" for additional information about the configuration file (Help > User
Guide).

Preference Mapping
==================
Each of the keys below maps to an option in the Jasper Preferences dialog.
For example, the Visualize/ShowValuePane key maps to the Visualize tab. The
value is Boolean (true or false). And the option name in the dialog is "Show
trace values pane".

Visualize Tab
-------------
. Key   : Visualize/ShowValuePane
  . Type: Bool
  . Name: Visualize > Show trace values pane

. Key   : Visualize/ShowSourceNavigationPane
  . Type: Bool
  . Name: Visualize > Show navigation pane in the Source Pane

. Key   : Visualize/ShowWaveformResetCycles
  . Type: Bool
  . Name: Visualize > Show reset cycles in waveform viewer

. Key   : Visualize/DisplayHierNameByDefault
  . Type: Bool
  . Name: Visualize > Display hierarchical name in the waveform viewer by default

. Key   : Visualize/StructuralNameAsHierName
  . Type: Bool
  . Name: Visualize > Treat structural name of struct/union/interface/record as
          part of hierarchical name

. Key   : Visualize/UndoRedoSettings/HighlightDifferences
  . Type: Bool
```

```
  . Name: Visualize > Navigate on stack by highlighting trace differences

. Key   : Visualize/ReplotSettings/ShowToolBox
  . Type: Bool
  . Name: Visualize > Show replot toolbox when replot is required

. Key   : Visualize/WhySettings/IsIgnoreConnectedAssumptions
  . Type: Bool
  . Name: Visualize > Ignore connected assumptions

. Key   : Visualize/WhySettings/IsSimpleBufferTransitive
  . Type: Bool
  . Name: Visualize > Simple buffer transitive

. Key   : Visualize/WhySettings/IsStopAtHierarchyChange
  . Type: Bool
  . Name: Visualize > Stop at hierarchy change

. Key   : Visualize/WhySettings/IncludeNonTickingGatedClocks
  . Type: Bool
  . Name: Visualize > Include non-ticking gated clocks

. Key   : Visualize/WhySettings/GateLevelMuxIgnoreDeassertedBits
  . Type: Bool
  . Name: Visualize > Only follow asserted bits in Gate Level Mux

. Key   : Windows/SimplifyRaisingWindowsThroughSocket
  . Type: Bool
  . Name: Visualize > Avoid closing the Visualize window before raising

. Key   : Visualize/WaveEdit/UseForceConstraints
  . Type: Bool
  . Name: Visualize > Reinforce iteration position by "-force" and iteration
          number when generating constraints

. Key   : Visualize/PlotAllowDuplicateSignal
  . Type: Bool
  . Name: Visualize > Allow plotting duplicate signals

. Key   : Visualize/ZoomFitAfterPlot
  . Type: Bool
  . Name: Visualize > Apply zoom-fit on waveform viewer after plotting new trace
          data
```

```
. Key   : Visualize/TraceExtensionCollapsed
  . Type: Bool
  . Name: Visualize > Collapse the trace extension cycles

. Key   : Visualize/ShowWhyOnCEXAutomatically
  . Type: Bool
  . Name: Visualize > Show the Why result automatically when opening a trace

. Key   : Visualize/PlotAllBitsOfWhyResult
  . Type: Bool
  . Name: Visualize > Plot all bits of Why result

. Key   : Visualize/EnablePreviewWhyResults
  . Type: Bool
  . Name: Visualize > Enable display of Preview Why Results

. Key   : Visualize/ShowSignalValueUnderscores
  . Type: Bool
  . Name: Visualize > Show signal value underscores

. Key   : Visualize/SignalValueDecimalPlaces
  . Type: Int
  . Name: Visualize > Set signal value decimal places

. Key   : Visualize/SignalLineColors/AbstractedCounter
  . Type: Color
  . Name: Visualize > Plot Colors > Abstracted Counter

. Key   : Visualize/SignalLineColors/BoundaryRegisterOrWire
  . Type: Color
  . Name: Visualize > Plot Colors > Boundary Register or wire

. Key   : Visualize/SignalLineColors/ConeOfInfluence
  . Type: Color
  . Name: Visualize > Plot Colors > Cone Of Influence

. Key   : Visualize/SignalLineColors/FreeNet
  . Type: Color
  . Name: Visualize > Plot Colors > Free Net

. Key   : Visualize/SignalLineColors/FreeWireOrFlop
  . Type: Color
```

```
  . Name: Visualize > Plot Colors > Free Wire Or Flop

. Key   : Visualize/SignalLineColors/InputClock
  . Type: Color
  . Name: Visualize > Plot Colors > Input (Clock) \ Wire (Gated Clock)

. Key   : Visualize/SignalLineColors/InternalWire
  . Type: Color
  . Name: Visualize > Plot Colors > Internal Wire

. Key   : Visualize/SignalLineColors/Latch
  . Type: Color
  . Name: Visualize > Plot Colors > Latch

. Key   : Visualize/SignalLineColors/MixedSignal
  . Type: Color
  . Name: Visualize > Plot Colors > Mixed Signal

. Key   : Visualize/SignalLineColors/MixedSignalPartiallyJustified
  . Type: Color
  . Name: Visualize > Plot Colors > Mixed Signal (partially justified)

. Key   : Visualize/SignalLineColors/StateRegister
  . Type: Color
  . Name: Visualize > Plot Colors > State Register

. Key   : Visualize/ShowAllValueAnnotations
  . Type: Text
  . Name: Visualize > Annotations mode for signals in the Source Pane
  . Possible Values: Off, Relevant, COI

. Key   : Visualize/Theme
  . Type: Text
  . Name: Visualize > Visualize Color Theme
  . Possible Values: "White background and colorful style"
                     "White background and simple style"
                     "Black background and colorful style"
                     "Black background and simple style"

. Key   : Visualize/ReplotSettings
  . Type: Text
  . Name: Visualize > Replot Settings > Replot Results Target Window
  . Possible Values: AskMe, AskMeOnlyWhenMultipleTraces, UseSameWindow
```

```
. Key   : Visualize/TrackingSignalsSettings
  . Type: Text
  . Name: Visualize > Tracking Signals
  . Possible Values: Always, Never, Prompt

Console Tab
-----------
. Key   : Console/OpenHyperlinksInExternalEditor
  . Type: Bool
  . Name: Open message hyperlinks in external editor

. Key   : Console/EscapeDraggedExternalText
  . Type: Bool
  . Name: Escape text dropped on console

. Key   : Console/CommandColor
  . Type: Color
  . Name: Text color for logged commands on console

. Key   : Console/DebugMessageColor
  . Type: Color
  . Name: Text color for debug messages on console

. Key   : Console/ErrorMessageColor
  . Type: Color
  . Name: Text color for error messages on console

. Key   : Console/ErrorResultColor
  . Type: Color
  . Name: Text color for error returns on console

. Key   : Console/HyperlinkColor
  . Type: Color
  . Name: Text color for hyperlinks on console

. Key   : Console/IncludedCmdMessageColor
  . Type: Color
  . Name: Text color for include messages on console

. Key   : Console/InfoMessageColor
  . Type: Color
  . Name: Text color for info messages on console
```

```
. Key   : Console/ResultColor
  . Type: Color
  . Name: Text color for command returns on console

. Key   : Console/TabCompletionColor
  . Type: Color
  . Name: Text color for tab completions on console

. Key   : Console/WarningMessageColor
  . Type: Color
  . Name: Text color for warning messages on console

Dialogs Tab
-----------
. Key   : ConfirmExit
  . Type: Bool
  . Name: Dialogs > Show "Confirm Exit" warning dialog when exiting the tool

. Key   : ConfirmExitWhileBusy
  . Type: Bool
  . Name: Dialogs > Show "Session Busy" warning dialog when exiting the tool

. Key   : Dialogs/ShowSaveCurrentWorkDlg
  . Type: Bool
  . Name: Dialogs > Show "Database Modified" warning dialog when exiting the tool

. Key   : ConfirmTerminateSessions
  . Type: Bool
  . Name: Dialogs > Show "Confirm Sessions Termination" warning dialog when
          exiting the tool

. Key   : ConfirmSessionClose
  . Type: Bool
  . Name: Dialogs > Show "Confirm Sessions Close" warning dialog when closing
          the sessions

. Key   : AskForSignalOrder
  . Type: Bool
  . Name: Dialogs > Show "Save Signal List" warning dialog when exiting
          Visualize window

. Key   : Warning/ShowSuggestAnalysisSetupWarning
```

```
  . Type: Bool
  . Name: Dialogs > Show warning dialog when using "Suggest Analysis Setup"

. Key   : ConfirmResetInfer
  . Type: Bool
  . Name: Dialogs > Show warning dialog when using reset infer button

. Key   : Dialogs/AskForAddDirectedTestPoint
  . Type: Bool
  . Name: Dialogs > Show "Add Directed Test Point" dialog when adding directed
          test point in Visualize window

. Key   : Dialogs/ShowUserDefinedColorInLowColorDepthWarning
  . Type: Bool
  . Name: Dialogs > Show color depth warning dialog when highlight with
          user-defined color in Visualize window

. Key   : Dialogs/showTclEvalHardInterruptConfirmation
  . Type: Bool
  . Name: Dialogs > Show Tcl interpreter unwind stack (forced interruption)

. Key   : Dialogs/showKillAllTclExecProcessesConfirmation
  . Type: Bool
  . Name: Dialogs > Show kill Tcl processes spawned by exec confirmation dialog

. Key   : Dialogs/showSkipCellDefineWarning
  . Type: Bool
  . Name: Dialogs > Show warning dialog when driver trace skips cell defines

. Key   : Dialogs/showPlotMultipleSignalsWarning
  . Type: Bool
  . Name: Dialogs > Show warning dialog when adding multiple C/C++ signals
    to the waveform

External Tools Tab
------------------
. Key   : ExternalApplications/dif.command
  . Type: Text
  . Name: External Tools > DIFF

. Key   : ExternalApplications/htm.command
  . Type: Text
  . Name: External Tools > HTM/HTML
```

```
. Key   : ExternalApplications/pdf.command
  . Type: Text
  . Name: External Tools > PDF

. Key   : ExternalApplications/spreadsheet.command
  . Type: spreadsheet
  . Name: External Tools > Spreadsheet

. Key   : ExternalApplications/txt.command
  . Type: Text
  . Name: External Tools > TXT

. Key   : ExternalApplications/vcd.command
  . Type: Text
  . Name: External Tools > VCD

Source Code Browser Tab
-----------------------
. Key   : SourceCodeBrowser/SyntaxHighlighting/Verilog/NonRelevantCode
  . Type: Color
  . Name: Source Browser > Verilog Syntax Highlight > No relevant code

. Key   : SourceCodeBrowser/SyntaxHighlighting/Verilog/BlockComment
  . Type: Color
  . Name: Source Browser > Verilog Syntax Highlight > Block comment

. Key   : SourceCodeBrowser/SyntaxHighlighting/Verilog/LineComment
  . Type: Color
  . Name: Source Browser > Verilog Syntax Highlight > Line comment

. Key   : SourceCodeBrowser/SyntaxHighlighting/Verilog/StringConstant
  . Type: Color
  . Name: Source Browser > Verilog Syntax Highlight > String constant

. Key   : SourceCodeBrowser/SyntaxHighlighting/Verilog/NumericConstant
  . Type: Color
  . Name: Source Browser > Verilog Syntax Highlight > Numeric constant

. Key   : SourceCodeBrowser/SyntaxHighlighting/Verilog/CompilerDirective
  . Type: Color
  . Name: Source Browser > Verilog Syntax Highlight > Compiler directive
```

```
. Key   : SourceCodeBrowser/SyntaxHighlighting/Verilog/Keyword
  . Type: Color
  . Name: Source Browser > Verilog Syntax Highlight > Keyword

. Key   : SourceCodeBrowser/SyntaxHighlighting/Verilog/SystemTask
  . Type: Color
  . Name: Source Browser > Verilog Syntax Highlight > System task

. Key   : SourceCodeBrowser/SyntaxHighlighting/Verilog/Identifier
  . Type: Color
  . Name: Source Browser > Verilog Syntax Highlight > Identifier

. Key   : SourceCodeBrowser/SyntaxHighlighting/Verilog/MacroCall
  . Type: Color
  . Name: Source Browser > Verilog Syntax Highlight > Macro identifier

. Key   : SourceCodeBrowser/SyntaxHighlighting/VHDL/NonRelevantCode
  . Type: Color
  . Name: Source Browser > VHDL Syntax Highlight > No relevant code

. Key   : SourceCodeBrowser/SyntaxHighlighting/VHDL/LineComment
  . Type: Color
  . Name: Source Browser > VHDL Syntax Highlight > Line comment

. Key   : SourceCodeBrowser/SyntaxHighlighting/VHDL/StringLiteral
  . Type: Color
  . Name: Source Browser > VHDL Syntax Highlight > String literal

. Key   : SourceCodeBrowser/SyntaxHighlighting/VHDL/CharacterLiteral
  . Type: Color
  . Name: Source Browser > VHDL Syntax Highlight > Character literal

. Key   : SourceCodeBrowser/SyntaxHighlighting/VHDL/BitStringLiteral
  . Type: Color
  . Name: Source Browser > VHDL Syntax Highlight > Bit string literal

. Key   : SourceCodeBrowser/SyntaxHighlighting/VHDL/NumericLiteral
  . Type: Color
  . Name: Source Browser > VHDL Syntax Highlight > Numeric literal

. Key   : SourceCodeBrowser/SyntaxHighlighting/VHDL/Keyword
  . Type: Color
  . Name: Source Browser > VHDL Syntax Highlight > Keyword
```

```
. Key   : SourceCodeBrowser/SyntaxHighlighting/VHDL/Identifier
  . Type: Color
  . Name: Source Browser > VHDL Syntax Highlight > Identifier

. Key   : SourceCodeBrowser/SyntaxHighlighting/HighlightIdentifier
  . Type: Color
  . Name: Source Browser > HighlightIdentifier

. Key   : SourceCodeBrowser/SyntaxHighlighting/HighlightCursorLineColor
  . Type: Color
  . Name: Source Browser > Highlight cursor line with color

. Key   : SourceCodeBrowser/SyntaxHighlighting/Background
  . Type: Color
  . Name: Source Browser > Background

. Key   : SourceCodeBrowser/SyntaxHighlighting/DefaultText
  . Type: Color
  . Name: Source Browser > Default Text
  . Type: Color

. Key   : SourceCodeBrowser/AllowDebugOnChangedFiles
  . Type: Bool
  . Name: Source Browser > Allow debugging on changed files

. Key   : SourceCodeBrowser/LargeFileSizeThresholdMB
  . Type: Int
  . Name: Source Browser > Large file size threshold in MB

. Key   : SourceCodeBrowser/LargeFileMaxLineCount
  . Type: Int
  . Name: Source Browser > Large file maximum line count

. Key   : SourceCodeBrowser/FileLinesPerIncrement
  . Type: Int
  . Name: Source Browser > Number of lines per file increment loaded

. Key   : SourceCodeBrowser/DesignExplorationResultOptimizationThreshold
  . Type: Int
  . Name: Source Browser > Threshold of Design Exploration results to enable
          performance optimization that limits highligthing and inline value
          annotation to current Target only.
```

```
General Tab
-----------
. Key   : AnalysisSessionTableFilterSharing
  . Type: Bool
  . Name: General > Tables > Apply same filter settings to property tables in
          all analysis sessions

. Key   : PropertyView/DynamicSorting
  . Type: Bool
  . Name: General > Tables > Enable dynamic sorting on property tables

. Key   : PropertyView/ShowRelatedCoversInPropertyTable
  . Type: Bool
  . Name: General > Tables > Show related properties in the property tables

. Key   : PropertyView/ShowCovergroupCoversInPropertyTable
  . Type: Bool
  . Name: General > Tables > Show covergroup covers in the property tables

. Key   : Search/OnFlySearching
  . Type: Bool
  . Name: General > Search and Text Filters > Enable immediate searching

. Key   : TextFilters/OnFlyFiltering
  . Type: Bool
  . Name: General > Search and Text Filters > Enable immediate filtering

. Key   : FilterInterruptThreshold
  . Type: Int
  . Name: General > Search and Text Filters > Filtering interrupt threshold

. Key   : Help/UseNewHelpBrowser
  . Type: Bool
  . Name: General > Help > Use new Help Browser

. Key   : ProofStructure/CollapseOperationNodes
  . Type: Bool
  . Name: General > Proof Structure > Collapse operation nodes in Proof
          Structure tree

. Key   : GUI/ForegroundColor
  . Type: Color
```

```
  . Name: General GUI foreground color

. Key   : GUI/BackgroundColor
  . Type: Color
  . Name: General GUI background color
```

Example

```
set_preference {Visualize/PlotAllowDuplicateSignal} false
set_preference {Visualize/SignalValueDecimalPlaces} 2
set_preference {Visualize/SignalLineColors/Latch} #F3F412
set_preference {Visualize/Theme} {White background and colorful style}
```

set_prompt1

Sets the Tcl script used to define the Tcl prompt.

Syntax

```
set_prompt1 tcl_script
```

Returns

No return value

Default

```
if {[get_tool -design_loaded]} { return "\[[task]\] % " } else { return "% " }
```

Detailed Description

Use this command to define the normal Tcl prompt. Every time the tool is about to show the prompt, it evaluates the specified Tcl script and uses its result as the prompt.

NOTE: If you set a Tcl variable in the script it will be visible in the Tcl environment.

Example

```
# Add a timestamp to the command prompt
set_prompt1 {
    if {[catch {task}]} {
        return "[tcl_clock format [tcl_clock seconds] -format {%Y-%m-%d %H:%M:%S}] "
    } else {
        return "[tcl_clock format [tcl_clock seconds] -format {%Y-%m-%d %H:%M:%S}] \
[[task]\] "
    }
}
```

See Also

set_prompt2

set_prompt2

Sets the Tcl script used to define the Tcl prompt for incomplete commands.

Syntax

```
set_prompt2 tcl_script
```

Returns

No return value

Default

```
return "> "
```

Detailed Description

Use this command to define the Tcl prompt for commands that are not yet complete. Every time the tool is about to show this prompt, it evaluates the specified Tcl script and uses its result as the prompt.

NOTE: If you set a Tcl variable in the script it will be visible in the Tcl environment.

See Also

set_prompt1

set_proof_simplification

Controls simplification during proof.

Syntax

```
set_proof_simplification (on | off)
```

Returns

No return value

Default

on

Detailed Description

During proof and trace visualization, the tool simplifies verification tasks before the actual verification starts. Simplification reduces the size of the netlist processed by the engines to achieve better overall performance. Use this command to control simplification during proof. View the size of the proof netlist by clicking on the Proof Messages tab of the main Jasper Apps GUI.

NOTE: In some cases, disabling simplification leads to faster operation, particularly with engines I, C, and C2. Simplification increases the likelihood that properties are solved during preprocessing ("PRE"), see "get_property_info" for additional information.

Example

```
set_proof_simplification off
```

See Also

prove,
set_cache_proof_simplification,
visualize

set_proof_simplification_max_iterations

Sets the maximum number of proof simplification iterations.

Syntax

```
set_proof_simplification_max_iterations N
```

Returns

No return value.

Default

100

Detailed Description

Use this command to configure the maximum number of proof simplification iterations. In a standard, unconstrained proof flow, the default value of 100 iterations is designed to allow proof simplification to fully contribute to the main proof before completion.

However, when operating under strict time constraints, it might be advisable to reduce the maximum number of iterations. Lowering this limit prevents the proof simplification stage from consuming a disproportionate share of the total proof time budget, helping to preserve sufficient time for the proof engines.

set_proof_structure_auto_propagation

Sets whether Proof Structure auto propagation is enabled.

Syntax

```
set_proof_structure_auto_propagation (on | off)
```

Returns

No return values

Default

on

Detailed Description

Use this command to specify whether proof results in Proof Structure nodes are automatically propagated when the proof ends or manually propagated using the "propagate" button.

Example

```
set_proof_structure_auto_propagation on
```

See Also

proof_structure

set_proof_structure_support_embedded_on_case_split

Sets if Proof Structure supports embedded properties for case split nodes.

Syntax

```
set_proof_structure_support_embedded_on_case_split (on | off)
```

Returns

No return values

Default

off

Detailed Description

```
Use "set_proof_structure_support_embedded_on_case_split" before design
elaboration to specify whether Proof Structure supports embedded properties
to create case split nodes.
. on   - Use embedded properties when creating Proof Structure case split
         nodes.
. off  - Do not use embedded properties to create Proof Structure case split
         nodes.
```

Example

```
set_proof_structure_support_embedded_on_case_split on
analyze -sv design.v
elaborate
```

See Also

proof_structure

set_proofgrid_bridge

Enables or disables the use of Advanced Job Dispatch and Management feature.

Syntax

```
set_proofgrid_bridge (on | off)
```

Returns

No return values

Default

on

Detailed Description

This command controls activation of Advanced Job Dispatch and Management feature. When enabled, it provides advanced scheduling and optimization capabilities, including job array dispatch, integration of custom start/stop scripts, and persistent ProofGrid jobs.

```
on:  Provide advanced scheduling and optimization of ProofGrid jobs. This is
     the default.
off: Do not provide advanced scheduling and optimization of ProofGrid jobs.
```

Example

```
set_proofgrid_bridge off
```

See Also

set_proofgrid_cloud_jobarray,
set_proofgrid_cloud_job_start_template,
set_proofgrid_cloud_job_status_template,
set_proofgrid_cloud_job_stop_template,
set_proofgrid_engine_file_path,
set_proofgrid_cloud_file_upload,
set_proofgrid_cloud_upload_storage,
set_persistent_proofgrid_timeout

set_proofgrid_cloud_file_upload

Enables file uploading mechanism with Bridge.

Syntax

```
set_proofgrid_cloud_file_upload (auto | true | false)
```

Returns

No return values

Default

auto

Detailed Description

Enables or disables file uploading mechanism with Bridge.

```
. auto:  Enable upload if "proofgrid_mode" is set to "clound_connector".
  This is the default.
. true:  Permanently enable file upload.
. false: Permanently disable file upload.

NOTE:
. In the default setting, engine processes access the files through the shared
  file system.
. If you set the command to an invalid value, the tool issues an error message.
. If Bridge is disabled, the value of this Tcl command is ignored.
```

Example

```
set_proofgrid_cloud_file_upload true
```

set_proofgrid_cloud_job_start_template

Specifies the executable binary path used in job start command.

Syntax

```
set_proofgrid_cloud_job_start_template binary_path
```

Returns

No return values

Default

""

Detailed Description

This command specifies the executable binary path used in job start command.
If it is set to a valid executable binary, it overrides
the default executable grid binary.
When set to an invalid executable binary, the tool issues an error message.
If Bridge is disabled, the value of this Tcl command is ignored.

Example

```
set_proofgrid_cloud_job_start_template /home/user/mybsub
```

set_proofgrid_cloud_job_status_template

Specifies the executable binary path used in job status command.

Syntax

```
set_proofgrid_cloud_job_status_template binary_path
```

Returns

No return values

Default

""

Detailed Description

This command specifies the executable binary path used in job status command.
If it is set to a valid executable binary, it overrides the default executable grid binary.
When set to an invalid executable binary, Jasper issues an error message.
If Bridge is disabled, the value of this Tcl command is ignored.

Example

```
set_proofgrid_cloud_job_status_template /home/user/mybsub
```

set_proofgrid_cloud_job_stop_template

Specifies the executable binary path used in job stop command.

Syntax

```
set_proofgrid_cloud_job_stop_template binary_path
```

Returns

No return values

Default

""

Detailed Description

This command specifies the executable binary path used in job stop command.
If it is set to a valid executable binary, it overrides the default executable grid binary.
When set to an invalid executable binary, Jasper issues an error message.
If Bridge is disabled, the value of this Tcl command is ignored.

Example

```
set_proofgrid_cloud_job_stop_template /home/user/mybsub
```

set_proofgrid_cloud_jobarray

Controls the use of job arrays when dispatching ProofGrid jobs.

Syntax

```
set_proofgrid_cloud_jobarray (on | off | auto)
```

Returns

No return values

Default

auto

Detailed Description

This command controls whether job arrays are used when ProofGrid submits jobs to a supported grid system (for example, LSF or NC). When job arrays are enabled, the ProofGrid issues a single job array submission, which might improve scheduling efficiency and resource utilization.

When the option is set to "auto", the tool selects the appropriate behavior based on the active grid system. For instance, job arrays are enabled by default under LSF, while they might be disabled for other systems.

To enable job arrays, Advanced Job Dispatch and Management feature must be enabled using "set_proofgrid_bridge on".

on: Issue a single job array submission when dispatching ProofGrid jobs.
off: Do not issue any job array submission.
auto: Select the appropriate behavior based on the active grid system. This is the default.

Example

```
set_proofgrid_cloud_jobarray off
set_proofgrid_cloud_jobarray on
```

See Also

set_proofgrid_bridge

set_proofgrid_cloud_upload_storage

Specifies the storage type used in file uploading.

Syntax

```
set_proofgrid_cloud_upload_storage (filesystem | S3)
```

Returns

No return values

Default

filesystem

Detailed Description

This command specifies the storage type used in file uploading.
It is only used when you set "proofgrid_cloud_file_upload" to "true".
The default value is "filesystem". The Bridge uploads
the engine files through webSocket.
When this command is set to "S3", the Bridge uses "S3" upload storage type.
This type requires the Tcl commands "set_proofgrid_bridge_s3_bucket" and
"set_proofgrid_bridge_s3_endpoint" to be set. Optionally, the object key,
scheme, and virtual addressing style can be configured using the
corresponding "set_proofgrid_bridge_s3_*" Tcl commands.
If Bridge is disabled, the value of this Tcl command is ignored.

Example

```
set_proofgrid_cloud_upload_storage S3
set_proofgrid_cloud_upload_storage filesystem
```

set_proofgrid_cluster

Sets the sequence of hosts on which to spawn proof engine jobs.

Syntax

```
set_proofgrid_cluster hostname_list
```

Returns

No return values

Default

localhost

Detailed Description

Use this command to set the sequence of hosts on which to spawn proof engine jobs. During prove (when proofgrid=cluster) ProofGrid spawns the first job on the first host, the second job on the second host, and so on. When the list of hosts ends, ProofGrid picks hosts from the beginning of the list again. You can specify hosts with IP numbers or host names (full or local). The hostname list accepts as delimiters " " (space), "," (comma), and ":" (colon).

As a simple balancing facility for the cluster, for example, when some hosts have higher capacity than others, consider the following guidelines:
* You can repeat hosts in the list.
* You can precede a host with a multiplier: M*hostname.

You need to set up authentication such that you can log in to all machines in the cluster without a password. See for example "ssh-keygen" for details.

Example

```
set_proofgrid_mode cluster
set_proofgrid_cluster {2*a b}
prove -all
# ProofGrid creates every third job on machine b, and the rest on machine a.
```

See Also

set_proofgrid_mode

set_proofgrid_engine_file_path

Specifies the custom engine file path.

Syntax

```
set_proofgrid_engine_file_path file_path
```

Returns

No return values

Default

""

Detailed Description

This command specifies the custom engine file path. When not set,
the engine file is located under the Jasper project directory.
When you set this variable to an invalid path, Jasper issues an error message.
If Bridge is disabled, the value of this Tcl command is ignored.

Example

```
set_proofgrid_engine_file_path /home/user/engine_file.bs
```

set_proofgrid_engineJ_max_jobs

Controls the number of engine J jobs.

Syntax

```
set_proofgrid_engineJ_max_jobs (N | follow | start_cap)
```

Returns

No return value

Default

start_cap

Detailed Description

```
Use this setting together with proofgrid_per_engine_max_jobs or
proofgrid_per_engine_max_local_jobs to control the number of engine J jobs.
This command supports the three "modes" described below.

N
   Use a positive integer N to limit the number of engine J jobs below
   the number of max_jobs. The number of started engine J jobs will
   be the minimum of N and max_jobs.

follow
    Make engine J reduce the number of engine jobs in the same way
    other engines (apart from engines Q3, L, J, and U) do when the number
    of remaining undetermined properties drops below max_jobs. The
    ProofGrid will not run more engine J jobs than there are undetermined
    properties being worked on in the current proof.

start_cap
    Let the number of engine jobs be determined by the global max_jobs
    setting. The difference between this mode and the "follow" mode is that the
    number of jobs does not decrease as properties become determined.
```

Example

```
# Start at most 7 engine J jobs
set_proofgrid_engineJ_max_jobs 7
```

See Also

set_proofgrid_per_engine_max_local_jobs,
set_proofgrid_per_engine_max_jobs,
set_proofgrid_engineL_max_jobs

set_proofgrid_engineL_max_jobs

Controls the number of engine L jobs.

Syntax

```
set_proofgrid_engineL_max_jobs (N | follow | start_cap)
```

Returns

No return value

Default

start_cap

Detailed Description

```
Use this setting together with proofgrid_per_engine_max_jobs or
proofgrid_per_engine_max_local_jobs to control the number of engine L jobs.
This command supports the three "modes" described below.

N
   Use a positive integer N to limit the number of engine L jobs below
   the number of max_jobs. The number of started engine L jobs will
   be the minimum of N and max_jobs.

follow
    Make engine L reduce the number of engine jobs in the same way
    other engines (apart from engines Q3, L, J, and U) do when the number
    of remaining undetermined properties drops below max_jobs. The
    ProofGrid will not run more engine L jobs than there are undetermined
    properties being worked on in the current proof.

start_cap
    Let the number of engine jobs be determined by the global max_jobs
    setting. The difference between this mode and the "follow" mode is that the
    number of jobs does not decrease as properties become determined.
```

Example

```
# Start at most 7 engine L jobs
set_proofgrid_engineL_max_jobs 7
```

See Also

set_proofgrid_per_engine_max_local_jobs,
set_proofgrid_per_engine_max_jobs,
set_proofgrid_engineJ_max_jobs

set_proofgrid_engineQ3_max_jobs

Controls the number of engine Q3 jobs.

Syntax

```
set_proofgrid_engineQ3_max_jobs (N | follow | start_cap)
```

Returns

No return value

Default

start_cap

Detailed Description

```
Use this setting together with proofgrid_per_engine_max_jobs or
proofgrid_per_engine_max_local_jobs to control the number of engine Q3 jobs.
This command supports the three "modes" described below.

N
   Use a positive integer N to limit the number of engine Q3 jobs below
   the number of max_jobs. The number of started engine Q3 jobs will
   be the minimum of N and max_jobs.

follow
    Make engine Q3 reduce the number of engine jobs in the same way
    other engines (apart from engines Q3, L, J, and U) do when the number
    of remaining undetermined properties drops below max_jobs. The
    ProofGrid will not run more engine Q3 jobs than there are undetermined
    properties being worked on in the current proof.

start_cap
    Let the number of engine jobs be determined by the global max_jobs
    setting. The difference between this mode and the "follow" mode is that the
    number of jobs does not decrease as properties become determined.
```

Example

```
# Start at most 7 engine Q3 jobs
set_proofgrid_engineQ3_max_jobs 7
```

See Also

set_proofgrid_engineJ_max_jobs,
set_proofgrid_per_engine_max_jobs,
set_proofgrid_per_engine_max_local_jobs

set_proofgrid_engineU2_max_jobs

Controls the number of engine U2 jobs.

Syntax

```
set_proofgrid_engineU2_max_jobs (N | follow | start_cap)
```

Returns

No return value

Default

start_cap

Detailed Description

```
Use this setting together with proofgrid_per_engine_max_jobs or
proofgrid_per_engine_max_local_jobs to control the number of engine U2 jobs.
This command supports the three "modes" described below.

N
   Use a positive integer N to limit the number of engine U2 jobs below
   the number of max_jobs. The number of started engine U2 jobs will
   be the minimum of N and max_jobs.

follow
    Make engine U2 reduce the number of engine jobs in the same way
    other engines (apart from engines Q3, L, and J) do when the number
    of remaining undetermined properties drops below max_jobs. The
    ProofGrid will not run more engine U2 jobs than there are undetermined
    properties being worked on in the current proof.

start_cap
    Let the number of engine jobs be determined by the global max_jobs
    setting. The difference between this mode and the "follow" mode is that the
    number of jobs does not decrease as properties become determined.
```

Example

```
# Start at most 7 engine U2 jobs
set_proofgrid_engineU2_max_jobs 7
```

See Also

set_proofgrid_engineJ_max_jobs,
set_proofgrid_engineQ3_max_jobs,
set_proofgrid_engineU_max_jobs,
set_proofgrid_per_engine_max_jobs,
set_proofgrid_per_engine_max_local_jobs

set_proofgrid_engineU_max_jobs

Controls the number of engine U jobs.

Syntax

```
set_proofgrid_engineU_max_jobs (N | follow | start_cap)
```

Returns

No return value

Default

start_cap

Detailed Description

```
Use this setting together with proofgrid_per_engine_max_jobs or
proofgrid_per_engine_max_local_jobs to control the number of engine U jobs.
This command supports the three "modes" described below.

N
   Use a positive integer N to limit the number of engine U jobs below
   the number of max_jobs. The number of started engine U jobs will
   be the minimum of N and max_jobs.

follow
    Make engine U reduce the number of engine jobs in the same way
    other engines (apart from engines Q3, L, and J) do when the number
    of remaining undetermined properties drops below max_jobs. The
    ProofGrid will not run more engine U jobs than there are undetermined
    properties being worked on in the current proof.

start_cap
    Let the number of engine jobs be determined by the global max_jobs
    setting. The difference between this mode and the "follow" mode is that the
    number of jobs does not decrease as properties become determined.
```

Example

```
# Start at most 7 engine U jobs
set_proofgrid_engineU_max_jobs 7
```

See Also

set_proofgrid_engineJ_max_jobs,
set_proofgrid_engineQ3_max_jobs,
set_proofgrid_per_engine_max_jobs,
set_proofgrid_per_engine_max_local_jobs

set_proofgrid_extra_args

Sets extra_args that will be supplied to the active "proofgrid_mode".

Syntax

```
set_proofgrid_extra_args args
```

Returns

No return values

Default

""

Detailed Description

Use this command to supply extra arguments to a specified "proofgrid_mode" and thereby have additional control over how the jobs are spawned. This feature allows you to use the built-in modes for LSF, OGE, or NetworkComputer, even if you need to adhere to certain resource attributes, constraints, or policies when using the grid.

The ProofGrid mode "shell" continues to be the mode that gives you maximum flexibility, but it also demands the most of you in terms of ensuring that the jobs are spawned in a correct way. "proofgrid_extra_args" adds some flexibility to the built-in ProofGrid modes, in that they can be used more often, but it also demands some additional caution to ensure that you use extra arguments that make sense and work.

NOTE: For proper parsing, you must prefix the first argument with a space.

Notes per proofgrid_mode:

lsf	The tool passes args to bsub. Example: { -P proj -R "OSNAME==Linux && OSREL==EE50 rusage[mem=4000]"} Consult your LSF documentation for more information.
oge	The tool passes args to qsub (or qrsh if "proofgrid_socket_communication=off"). Consult your Grid Engine documentation for more information.
nc	The tool passes args to nc run. Consult your NetworkComputer documentation for more information.
local|cluster	The tool ignores extra_args you specify.
shell	The specified extra_args will be available in the environment variable JASPER_EXTRA_ARGS for the shell to use (see "set_proofgrid_shell").

Example

```
set_proofgrid_extra_args { -R "select[type==LINUX64 && mem>8000] rusage[mem=8000]"}
```

See Also

set_proofgrid_shell

set_proofgrid_manager

Turns on or off the ProofGrid Manager feature.

Syntax

```
set_proofgrid_manager (on | off)
```

Returns

No return values

Default

off

Detailed Description

Use this command to turn on or off the ProofGrid Manager feature to orchestrate parallel proofs. By default, the feature is off.

NOTE:
. You can enable ProofGrid Manager at any time during a proof or Visualize, and it will show results from that point on. To capture complete results, enable ProofGrid Manager before starting the proof or Visualize.
. For additional information, refer to the ProofGrid Manager manual and the License Usage and Control guide (Help > Application Guides).
. The proofgrid manager can consume high memory in X-Prop, SEC, and SPV applications when executing the proof strategy.

on
    Turn on the ProofGrid Manager feature.

off
    Turn off the ProofGrid Manager feature.

Example

```
set_proofgrid_manager on
```

set_proofgrid_max_jobs

Sets the limit for jobs used by ProofGrid (when not in local mode).

Syntax

```
set_proofgrid_max_jobs N
```

Returns

No return values

Default

0 (unlimited)

Detailed Description

Use this command to set the maximum number of jobs ProofGrid can use to deliver quick results. This command, the "proofgrid_per_engine_max_jobs" variable, and the number of available licenses limit the number of jobs ProofGrid can start (see "set_proofgrid_per_engine_max_jobs").

Also see the ProofGrid Manager manual and the License Usage and Control guide (Help > Application Guides).

Example

```
set_proofgrid_max_jobs 5
```

See Also

set_proofgrid_per_engine_max_jobs

set_proofgrid_max_local_jobs

Sets the limit for jobs used by ProofGrid when "proofgrid_mode" is "local".

Syntax

```
set_proofgrid_max_local_jobs N
```

Returns

No return values

Default

0 (unlimited)

Detailed Description

Use this command to set the number of jobs ProofGrid can use when "proofgrid_mode" is "local". This command, the "proofgrid_per_engine_max_jobs" variable, and the number of available licenses limit the number of jobs ProofGrid can start (see "set_proofgrid_per_engine_max_jobs").

Also see the ProofGrid Manager manual and the License Usage and Control guide (Help > Application Guides).

Example

```
set_proofgrid_max_local_jobs 8
```

See Also

set_proofgrid_max_jobs,
set_proofgrid_per_engine_max_local_jobs

set_proofgrid_max_partitions

Sets the maximum number of partitions for partitionable engines.

Syntax

```
set_proofgrid_max_partitions (none | N)
```

Returns

No return values

Default

none

Detailed Description

Use this command to set the maximum number of partitions ProofGrid can use for engines Ht, Bm, Mp, and R. With partitioning, ProofGrid splits the set of properties in a proof into a number of partitions and allocates one engine job of each partitionable engine for each partition.

This setting takes effect only for proofs using the "prove" command when orchestration is turned off. See "set_prove_orchestration".

Example

```
set_proofgrid_max_partitions 5
```

See Also

set_proofgrid_per_engine_max_jobs,
set_prove_orchestration

set_proofgrid_mem_scan

Turn on memory scan.

Syntax

```
set_proofgrid_mem_scan (on | off)
```

Returns

No return values

Default

off

Detailed Description

During proof, engine jobs for single-property engines jump from property to property (see "set_prove_per_property_time_limit"). Engine jobs might also exhaust the available memory and terminate, only to have ProofGrid restart them (see "set_proofgrid_restarts"). A ProofGrid shell might, based on information in environment variables, submit the restarting job to a host with more memory available (see "set_proofgrid_shell"). The memory consumption of an engine job depends on which property it is working on, so when the job terminates on a specific property, it might be missing the opportunity to prove other properties within the available memory.

The mem_scan mode provides a way to get more proof results out of memory-limited machines (that is, a way to delay the migration to bigger hosts) when a proof has many properties, and some of them will run out of memory and trigger an engine job restart. The environment variable "JASPER_ENGINE_JOB_BUDGET" contains a number that indicates if ProofGrid determines that this job can make more progress with the same amount of memory as before or if it requires more memory. Initially, the number is 0. The "budget" retains its current value until ProofGrid has an indication that no remaining target can make progress without more memory. When that happens, the

budget increments to 1 (and then 2, 3, and so on). When the current budget remains because other single property engines are still working on a property this engine has restarted on, the engine job waits until the other engines are ready to move to a new property.

To get the intended value out of mem_scan the proofgrid_shell should take JASPER_ENGINE_JOB_BUDGET into account when allocating a host. The mapping from the relative budget numbers to concrete resource requirements is left for the user to decide based on the relative cost and availability of the different grid resources, and perhaps, based on experience concerning the memory consumption of the engines on the set of properties in practice.

NOTE:
. "set_proofgrid_mem_scan" is an initial release to gather feedback from early adopters and finalize implementation for an upcoming release. Contact support@cadence.com to provide feedback or comments targeting the upcoming production version.
. Budget is tracked per engine, so technically, it is possible to map the budget number onto concrete resource requirements differently for different engines.
. It is possible that an engine job terminates unexpectedly for other reasons, but mem_scan interprets it as out of memory.
. JASPER_ENGINE_JOB_BUDGET is set for all engines, whether mem_scan is on or off. Only single-property engines get wait states with "mem_scan on".
. Set a high restart value to allow ProofGrid to continue multiple engine job restarts (see "set_proofgrid_restarts").
. This feature does not work well with proof orchestration (see "set_prove_orchestration").

Example

```
set_proofgrid_mem_scan on
set_proofgrid_restarts 100
set_proofgrid_mode shell
set_proofgrid_shell {./my_bsub_wrapper.sh
```

See Also

set_proofgrid_restarts,
set_proofgrid_shell,
set_prove_orchestration,
set_prove_per_property_time_limit

set_proofgrid_mode

Sets the grid on which to run parallel proofs.

Syntax

```
set_proofgrid_mode (local | shell | lsf | oge | nc | cluster)
```

Returns

No return values

Default

local

Detailed Description

The "prove" and "visualize" commands often spawn multiple proof engine jobs (jg_proof processes). This command specifies how the tool spawns jobs.

Also see the ProofGrid Manager manual (Help > Application Guides).

Helpful information about using Platform LSF(R), Oracle(R) Grid Engine (OGE), and Runtime NetworkComputer(TM) with Jasper Apps:

. Consult your LSF, OGE, or NetworkComputer documentation for installation, configuration, and limitations.
. The project directory must be accessible using the same path from all machines in the cluster.
. You might need to set up authentication such that you can log in to all machines in the cluster without a password. See for example "ssh-keygen" and consult your LSF, OGE, or NetworkComputer documentation for details.
. You might get X11 forwarding warnings. You can ignore them.
. See "set_proofgrid_socket_communication" for more information.

local
    Run the processes on your local machine (the machine running the
    Jasper Apps session).

shell
    Use a custom command or script to spawn the jobs, see
    "set_proofgrid_shell".

lsf
    Run the processes on the LSF cluster that is expected to be
    available.

oge
    Run the processes on the OGE that is expected to be available.

    NOTE: When using engine Tri or "set_engine_threads", you must specify a
    parallel environment [PE], which will run all slots on the same host. For
    example:
    set_proofgrid_extra_args { -pe [PE] %jasper_engine_job_threads}

nc
    Run the processes on the NetworkComputer that is expected to be available.

    NOTE: Refer to the Runtime Design Automation website for additional
    information about NetworkComputer: rtda.com.

cluster
    Spread the processes over the machines defined by "set_proofgrid_cluster".

Advanced Job Dispatch and Management features are supported in local,
lsf and nc modes only. Other modes like shell, cluster and oge use
the legacy ProofGrid dispatch system.

See Also

prove,
set_proofgrid_per_engine_max_jobs,
set_proofgrid_per_engine_max_local_jobs,
set_proofgrid_shell,
set_proofgrid_cluster,
set_proofgrid_socket_communication,
visualize,
set_proofgrid_bridge

set_proofgrid_pending_time_limit

Sets the limit for a "job pending" warning.

Syntax

```
set_proofgrid_pending_time_limit time_limit
```

Returns

No return values

Default

600s

Detailed Description

Use this command to enable a warning when engine jobs spend more than a certain amount of time pending before being scheduled. You can use this warning as an indication that the ProofGrid is not able to use machine resources effectively.

The tool supports three formats for the time limit:

```
. ISO8601:    hh':'[mm[':'[ss['.'[fff]]]]]
  Examples:   2:30, 2:, 0:0:23, 2:30:05.235
. Jasper: [dd'd'][hh'h'][mm'm'][ss[.fff]'s']
  Examples:   2h30m, 2h, 23s, 2h30m05.235s
. Seconds:    sssss[.fff]['s']
  Examples:   9000s, 7200, 23s, 9005.235s
```

Example

```
set_proofgrid_pending_time_limit 10m
```

See Also

set_proofgrid_mode,
set_proofgrid_shell

set_proofgrid_per_engine_max_jobs

Sets the limit for engine_jobs started by ProofGrid per engine.

Syntax

set_proofgrid_per_engine_max_jobs N

Returns

No return values

Default

2

Detailed Description

Use this command to set the number engine_jobs ProofGrid can start per engine to deliver quick results.

When ProofGrid runs, it attempts to start the specified number of jobs per engine and use them for parallel jobs to deliver results faster. 0 means unlimited.

NOTE: When setting the limit for maximum jobs per engine, consider your operating system's default limit for file descriptors and the number of file descriptors required. Each per engine job requires 1 file descriptor, each proof job requires 1 additional file descriptor (2 if socket_communication off), and additional file descriptors are required for other purposes. For example, {Hp Ht B K I D N} with 100 jobs per engine and socket_communication off requires greater than 1104 file descriptors; however, the default limit for Linux is 1024.

The actual number of jobs per engine managed by ProofGrid might be different from what you have specified and can change over time for a number of reasons, including the following:

```
. ProofGrid can determine an upper limit on the level of parallelism it
  can put to good use. This number can vary over time while the proof
  is running. In particular, it is common for this number to drop when
  there are only a few properties remaining to prove. The number of
  jobs actually running will dynamically drop accordingly.

. Fewer licenses might be available from the license server.

. The number of jobs per engine can increase, up to the limit you set, as
  licenses become available. The tool attempts license checkout once every 60
  seconds. It will try to check out as many licenses as possible until it
  reaches the specified limit.

Important information about this command:
. Increasing the number of jobs per engine can speed up the "prove" command,
  but it can also increase the load on ProofGrid. For best results, you might
  need to monitor the memory and CPU load of the grid to ensure that it does
  not get out of bounds.
. This command has no effect on proofs already in progress, you must run it
  before running the "prove" command.

Also see ProofGrid Manager manual and the License Usage and Control guide
(Help > Application Guides).
```

See Also

prove,
set_engine_mode,
set_proofgrid_max_jobs,
set_proofgrid_per_engine_max_local_jobs,
set_proofgrid_mode,
set_proofgrid_socket_communication

set_proofgrid_per_engine_max_jobs_per_session

Specifies an upper limit for number of per-engine jobs in analysis sessions.

Syntax

```
set_proofgrid_per_engine_max_jobs_per_session n
```

Returns

No return values

Default

0

Detailed Description

Use this command to specify an upper limit on the total number of per-engine jobs used at any one time in an analysis session. In most cases, one per-engine job will correspond to one license.

Use the value "0" to remove the limit.

Implementation Guidance:
To specify that you want new proofs to wait for a license when they reach the limit set by this command, set "command_license_wait_interval" and "command_license_wait_retries" to non-zero values. The proof jobs will then wait until you increase the limit, another job finishes so that a "slot" opens, you interrupt the proof (using the GUI), or the tool reaches the time limit.

NOTE:
. The tool computes the time limit by taking the product of these two configurations, and since this product by default is zero, you must set both to non-zero values.

. When the tool enters the waiting period, you will see an INFO message.

. The "prove" command is blocked during the wait even if you specified the "-bg" switch.

See Also

prove,
set_proofgrid_per_engine_max_jobs,
set_proofgrid_per_engine_max_local_jobs

set_proofgrid_per_engine_max_local_jobs

Sets the limit for engine jobs per engine ProofGrid starts when in "local" mode.

Syntax

```
set_proofgrid_per_engine_max_local_jobs N
```

Returns

No return values

Default

1

Detailed Description

Use this command to set the number of engine jobs ProofGrid can start per engine when proofgrid_mode is "local".

This command sets a lower limit for the maximum number of jobs per engine in effect when proofgrid_mode is local. This command can be useful when switching between local and distributed ProofGrid modes when it is not suitable to start as many jobs on your local machine as can be started in the distributed setting.

The actual number of jobs per engine managed by ProofGrid might be different from what you have specified for a number of reasons, including the following:

. ProofGrid can determine an upper limit on the level of parallelism it can put to good use. This number can vary over time while the proof or Visualize is running. In particular, it is common for this number to drop when there are only a few properties remaining. The number of jobs actually running dynamically drops accordingly.

. Fewer licenses might be available from the license server.

. The number of jobs per engine can increase, up to the limit you set, as licenses become available. The tool attempts license checkout once every 60 seconds. It will try to check out as many licenses as possible until it reaches the specified limit.

NOTE: This command has no effect on proofs or Visualize already in progress, you must run it before running the "prove" or "visualize" command.

Also see the ProofGrid Manager manual and the License Usage and Control guide (Help > Application Guides).

Example

```
set_proofgrid_per_engine_max_local_jobs 2
```

See Also

set_proofgrid_per_engine_max_jobs,
set_proofgrid_max_local_jobs

set_proofgrid_queue

Sets the queue name on which to run parallel proofs.

Syntax

```
set_proofgrid_queue queue_name
```

Returns

No return values

Default

""

Detailed Description

Use this command to send a queue name, when not "", to the selected "proofgrid_mode" and thereby control where the jobs will be spawned. Some grid systems might permit specifying multiple queues. Consult the documentation of your grid system for more information.

Notes per proofgrid_mode:

local|cluster	The queue name will be ignored
shell	The queue name will be available in the environment variable JASPER_QUEUE for the shell to use, see "set_proofgrid_shell".
lsf|oge|nc	The queue name will be specified using "-q queue_name". You can find available queues with "exec bqueues" or "exec qstat" respectively. Consult your LSF, OGE, or NetworkComputer documentation for more information.

See Also

set_proofgrid_shell

set_proofgrid_restarts

Specifies the number of restarts ProofGrid allows per engine job.

Syntax

```
set_proofgrid_restarts N
```

Returns

No return values.

Default

10

Detailed Description

By default, ProofGrid allows 10 restarts per engine job before abandoning a job. Use this command to control the number of restarts.

Example

```
set_proofgrid_restarts 20
```

See Also

set_proofgrid_mode

set_proofgrid_resubmit_time_limit

Sets the limit for job resubmission.

Syntax

```
set_proofgrid_resubmit_time_limit time_limit
```

Returns

No return values

Default

0s (unlimited)

Detailed Description

Use this command to enable regular resubmission of engine jobs. If set, the ProofGrid terminates and resubmits engine jobs regularly.

Important: Engine modes that operate on all properties in parallel (for example, H, J, L) are not affected by this setting.

The tool supports three formats for the time limit:

```
. ISO8601:    hh':'[mm[':'[ss['.'[fff]]]]]
  Examples:   2:30, 2:, 0:0:23, 2:30:05.235
. Jasper: [dd'd'][hh'h'][mm'm'][ss[.fff]'s']
  Examples:   2h30m, 2h, 23s, 2h30m05.235s
. Seconds:    sssss[.fff]['s']
  Examples:   9000s, 7200, 23s, 9005.235s
```

Example

```
set_proofgrid_resubmit_time_limit 1h
```

set_proofgrid_shell

Sets the command used to launch ProofGrid jobs when "proofgrid_mode" is "shell".

Syntax

```
set_proofgrid_shell full_path_to_shell_plus_args
```

Returns

No return values

Default

```
"/bin/sh -c 'echo =proofgrid_shell is not defined; false'"
```

Detailed Description

During prove and Visualize, often multiple proof engine jobs (jg_proof processes) are spawned. This setting specifies how the jobs are spawned when "proofgrid_mode" is set to "shell".

Important things to know:

. The proofgrid_shell must be a string beginning with the full path to an executable. As an option, that string can continue with a space-separated set of extra arguments that will be given to the executable. You can use double and single quotes to form an argument that contains spaces. The unquoted token "%jasper_session_id" will be replaced with a unique Jasper Apps session ID (see JASPER_SESSION_ID). This can be useful to ensure that jobs still in a batch queue are killed as the "prove" or "visualize" exits, see "set_proofgrid_shell_stop". The unquoted token "%jasper_engine_job_threads" will be replaced with the effective number of cores required by the engine job being launched by the ProofGrid. This setting is relevant when using engine Tri (which uses eight threads by default) or when using more than one engine thread (see "get_engine_threads").
. The executable must take the job to start and its arguments as further arguments.

. The executable must propagate (that is, tunnel) stdin/stdout reliably from and to the job unless using socket communication (see "set_proofgrid_socket_communication"). This arrangement is sometimes called interactive mode.
. The proofgrid_shell executable must have a lifetime that is as long as the jg_proof process it started. For example, when using Platform LSF(R) mode, use non-interactive queues, and use "-K" to cause bsub to wait for the ProofGrid job to complete before exiting. If you do not use "-K", you might receive numerous LSF messages stating that the proof jobs failed and "Exited with exit code 1" due to (for example) the following:
  jg_proof: connect: Connection refused
  jg_proof: failed to connect
. Consider giving all jobs in a proof thread a common name and use this name in the "set_proofgrid_shell_stop" command to ensure all jobs are stopped at the end of the proof. JASPER_SESSION_ID can provide a suitable name, see below.
. If proofgrid_mode is "shell", you must update the value of proofgrid_shell because the default value is merely a placeholder that indicates you need to modify it. The default value will cause the engine jobs to issue a message to the console and then fail; that is, it is not useful for running proofs.
. ProofGrid does not robustly support automatic re-queuing of killed jobs, but you can disable the feature with "bsub -rn" and rely on the built-in ProofGrid restart mechanism.
. If you are using LSF, the tool runs a sanity check and then issues a warning if you have not requested enough grid resources to run an engine job with the intended number of threads:

  Adjusting job to use N slots.

  See the description of the environment variable JASPER_ENGINE_JOB_THREADS below or the examples for a way to request the correct number of slots.
. If you are running Memory Aware Proof (MAP) flow, the proofgrid shell configuration needs to modified to use the template argument %jasper_engine_job_memory_request_mib for memory request. See example below.

The following environment variables are available to the shell:

. JASPER_INSTALL_DIR: The location of the installation.
. JASPER_SESSION_ID: A string unique to this session and prove invocation (see "set_proofgrid_shell_stop").
. JASPER_PROOF_THREAD_ID: The proof thread ID of this proof.
. JASPER_ENGINE_MODE: The engine mode this job will run. Ex: "engineHt" See the Examples section to learn how you might use this variable.
. JASPER_ENGINE_JOB_ID: The ID of this job. Ex: "0.Ht"

. JASPER_ENGINE_JOB_RESTART: The number of times this job ID has been restarted during this prove invocation. "0" means this is the first time this job ID was started during this prove invocation.
. JASPER_ENGINE_JOB_UNEXPECTED_TERMINATION: The number of times this job ID has been restarted unexpectedly, for example, by being killed by the grid system.
. JASPER_ENGINE_JOB_MAX_RESTART: The number of restarts ProofGrid allows per engine job.
. JASPER_ENGINE_JOB_THREADS: The number of "threads" this job is expected to run. This corresponds to numbers of processor cores the shell must allow the engine job to consume (see, for example, "bsub -n", "qsub -pe [PE]", or "nc run -dp").
. JASPER_ENGINE_JOB_CAN_MORPH: Indicates if the engine job can morph or if it remains the engine JASPER_ENGINE_MODE throughout its lifetime.
. JASPER_ENGINE_JOB_BUDGET: A number indicating the relative amount of memory this job is expected to consume. The initial budget is "0". When an engine job restarts, the budget increments to indicate that more memory is needed (based on indications that previous jobs have run out of memory) or stays the same to indicate that more memory is not needed. See "set_proofgrid_mem_scan".
. JASPER_PROJECT_DIR: The project directory specific to the current session (see "session -get_proj_dir").
. JASPER_PROVE_TIME_LIMIT: If the current proof has a time limit, this variable contains the number of seconds; otherwise, the tool does not set this variable. This value can safely be used as a hard wall-clock limit for the job (for example, to format the time for bsub -W).
  NOTE:
  . For commands that set a time limit, see "set_prove_time_limit" and "prove -time_limit".
  . By default, the prove time limit is 86400 seconds (24 hours).
. JASPER_SOCKET_COMMUNICATION: Whether communication happens over a TCP/IP socket or standard in/out. "1" if "proofgrid_socket_communication" is on; otherwise, "0".
. JASPER_QUEUE: The name of the queue engine jobs should be spawned to according to the "proofgrid_queue" (see "get_proofgrid_queue").
. JASPER_EXTRA_ARGS: Extra arguments according to "proofgrid_extra_args" (see "get_proofgrid_extra_args").

All environment variables except JASPER_EXTRA_ARGS are also available as template arguments "%jasper_<name>" where <name> is the name of the corresponding environment variable in lower case. For example, the variable JASPER_ENGINE_JOB_THREADS is available as %jasper_engine_job_threads.
Any argument in the shell that matches a template is replaced with the value of

that variable. And if a variable has no value, it is replaced with an empty string.

NOTE: The jobs do not have any wait states, which can sometimes be the case with other tools that require interactive mode. The executable can use stderr for its own output, which will be sent to the terminal starting the tool. The executable might also issue messages on stdout under very limited circumstances:

. The messages must come before and/or after all messages the job will produce.
. When socket_communication=off the messages must not contain the SOH
  - Start of heading control character "\1"
. The messages will appear as Proof Messages in the designated tab and in the log. Messages beginning with "=" will print to the console instead of the Proof Messages tab.
. The project directory must be accessible using the same path from all machines in the cluster.

If these rules are not respected, Jasper ProofGrid will not work reliably.

Troubleshooting:
To get more insight into why your ProofGrid jobs fail to launch, use the built-in debugging features of the programming language you use for your proofgrid_shell as shown in the bash examples below.

1. Create the following executable file:

```
debug.sh
#!/bin/bash
exec /bin/bash -xv "$@" 2>&1
```

2. Prefix your proofgrid_shell setting with "./debug.sh":

```
set_proofgrid_shell
{myscripttodebug.sh}

becomes

set_proofgrid_shell
{./debug.sh myscripttodebug.sh}
```

With this addition to the command, the exact UNIX command executed will be

echoed to the proof messages; however, if you prefer to have the exact UNIX command executed being echoed on stderr instead, use the following command:

```
set_proofgrid_shell
{/bin/bash -xv myscripttodebug.sh}
```

As an alternative, you can add the following to the beginning of myscripttodebug.sh:

```
#!/bin/bash -xv
```

If your proofgrid_shell is written in another language (for example, C, Tcl, or Java) use the debugging features of that language.

Example

```
set_proofgrid_shell {/usr/bin/ssh -x myhost}

set_proofgrid_shell {bsub -J %jasper_session_id -n %jasper_engine_job_threads -K -q
lnx64 -R "OSNAME==Linux && OSREL==EE50 rusage[mem=4000] span[hosts=1]"}
set_proofgrid_shell_stop {bkill -J %jasper_session_id }

# In your wrapper file, "exec" the bsub command
set_proofgrid_shell {./my_bsub_wrapper.sh}

set_proofgrid_shell {/usr/local/lsf/bin/bsub -J %jasper_session_id -n
%jasper_engine_job_threads -Ip -q normal -R "select[type==X86_64  && mem>2000]
rusage[mem=1000] span[hosts=1]"}
set_proofgrid_shell_stop {/usr/local/lsf/bin/bkill -J %jasper_session_id }

set_proofgrid_shell {/usr/local/lsf/bin/bsub -q lnx64 -K -W 200:00 -R " (OSNAME ==
Linux && (OSREL==EE80||OSREL==SLES12))
rusage[mem=%jasper_engine_job_memory_request_mib] span[hosts=1]"}

# Example script
# --------------
# This is an example of how you can access the JASPER_ENGINE_MODE variable from
# the proofgrid_shell script. You can then use information from
# JASPER_ENGINE_MODE to decide how to submit a grid job

echo_engine.sh:
                #!/bin/sh
                # output must begin with < or be directed to stderr to not
                # confuse the proofgrid
                echo \< $$: $JASPER_ENGINE_MODE
                #echo $$: $JASPER_ENGINE_MODE 1>&2
                exec $@

echo_engine.tcl:
                set_proofgrid_mode shell
                set_proofgrid_shell {./echo_engine.sh}
                set_engine_mode {D B J}
                prove -all
```

See Also

set_proofgrid_mode,
set_proofgrid_restarts,
set_proofgrid_shell_stop,
set_proofgrid_socket_communication

set_proofgrid_shell_stop

Sets the command used to clean ProofGrid jobs when "proofgrid_mode" is "shell".

Syntax

```
set_proofgrid_shell_stop full_path_to_shell_plus_args
```

Returns

No return values

Detailed Description

During prove and Visualize the tool often spawns multiple proof engine jobs (jg_proof processes). Use this setting to specify a stop hook that will be run when the "prove" or "visualize" command is about to exit if engine jobs do not terminate at the end of a proof or are still pending in a batch queue. The token "%jasper_session_id" will be replaced with a unique session ID. This can be useful to ensure that jobs still in a batch queue are killed as the "prove" or "visualize" exits.

NOTE: The command is expected to execute to completion quickly and it will block the proof thread during its execution.

Example

```
set_proofgrid_shell {/usr/local/lsf/bin/bsub -J %jasper_session_id }
set_proofgrid_shell_stop {/usr/local/lsf/bin/bkill -J %jasper_session_id }
```

See Also

set_proofgrid_shell

set_proofgrid_socket_communication

Specifies whether communication happens over a TCP/IP socket or standard in/out.

Syntax

```
set_proofgrid_socket_communication (on | off)
```

Returns

No return values

Default

on

Detailed Description

When "on", engines exchange information during proofs using a TCP/IP socket.

Important information about this command (when "on"):
. For Platform LSF(R) mode, use non-interactive queues, and use "-K" to cause bsub to wait for the ProofGrid job to complete before exiting.
. For Oracle(R) Grid Engine (OGE) mode, use qsub instead of qrsh, and use "-sync y" to cause qsub to wait for the ProofGrid job to complete before exiting.
. For shell mode, make user-specified ProofGrid shells synchronous (wait to exit until the proof or Visualize job is done).

Important information about this command (when "off"):
. When proofgrid_mode=shell the shell must propagate (that is, tunnel) stdin/stdout reliably from and to the job. This arrangement is sometimes called interactive mode. For other ProofGrid modes, this is taken care of for you.
. Your LSF setup and policies must allow jobs started with bsub -I. The jobs will communicate using stdin/stdout, but will have no wait states.

Also see the ProofGrid Manager manual and the License Usage and Control guide (Help > Application Guides).

Example

```
set_proofgrid_socket_communication on
set_proofgrid_shell {/tools/vendors/bin/nc run -v 0 -w -C jg}

set_proofgrid_socket_communication off
set_proofgrid_shell {/tools/vendors/bin/nc run -v 0 -Ir -C jg}
```

See Also

set_proofgrid_mode,
set_proofgrid_shell

set_proofmaster

Enables or disables ProofMaster.

Syntax

```
set_proofmaster (on | off)
```

Returns

No return values

Default

off

Detailed Description

Use this command to enable or disable ProofMaster. ProofMaster is a push-button solution for achieving better convergence, faster results, and improved bounds during the development life cycle of a design. It does so by using machine learning to dynamically orchestrate engine technology during proof (Proof Orchestration) and leveraging historical knowledge to better use computational resources (proof caching, Trace Replay and PPD).

The targeted flow for ProofMaster value is consecutive runs on stable or changing designs. All users can benefit from using ProofMaster.

NOTE:
. ProofMaster is compatible with the following commands only:
  . prove
  . check_conn -prove
  . check_cov -measure
  . check_csr -prove
  . check_hls -prove
  . check_sec -prove
  . check_spv -prove

  . check_unr -prove
  . check_xprop -prove
  . hunt -run
. When using the "check_spv -prove" command and ProofMaster is "on",
  only the SPV prove cache, Proof orchestration, and PPD work (without
  Trace Replay).
. When using the "check_xprop -prove" command and ProofMaster is "on",
  only the X-Prop prove cache, Proof orchestration, and PPD work (without
  Trace Replay).
. When using the "hunt -run" command and ProofMaster is "on",
  only Prove Cache and Trace Replay works (without PPD or Proof Orchestration).
  The following strategy modes are supported: "state_swarm", "cycle_swarm",
  "bound_swarm", "simulation", "formal", "trace_swarm"*, "trace_search"*.
  * The "trace_swarm" and "trace_replay" strategy modes support
  only Trace Replay.
  With the considerations above, hunt auto flow is also supported.
. ProofMaster has limited functionality with properties of types
  path (cover -path) and helper covers (cover -generate and hunt -auto).
. The COV App does not support proof cache when running checker coverage.
. As part of its heuristics, ProofMaster caches proof results based on
  property signatures (hashes). By default, cache signature computation
  under ProofMaster is run in parallel with the main proof, running one cache
  job on the local host. You can speed up the signature computation for your
  proof by increasing the number of cache jobs run in parallel, potentially
  restoring proof results faster (see "set_prove_cache_max_jobs"). Cache jobs
  run by ProofMaster can also be launched according to your ProofGrid settings
  (see "set_prove_cache_job_mode").
. You should not expect to see 100% cached results restoration when you have
  changes to the design, environment, or properties. Although you might
  perceive some changes as small, they can be big for the formal engines.
. As a more resilient caching mechanism, ProofMaster also saves traces based on
  property names, and tries to replay (simulate) these traces in future proofs.
  This feature is called Trace Replay and it is an extension of cache, so
  "Cache" is displayed in the property table whenever Trace Replay successfully
  retrieves cached results (traces).

Example

```
set_proofmaster on
prove -task <embedded>    # Proof Orchestration, enabled by machine learning,
                            # already has an effect on first proof triggered
                            # with ProofMaster
clear -result
prove -task <embedded>    # From the second proof onward, ProofMaster can
                            # leverage historical knowledge to reach results
                            # faster. You can expect to see results restored by
                            # proof caching, and proof strategies to be picked
                            # up while reading the PPD file.
```

See Also

set_proofmaster_dir,
set_proofmaster_initial_dir,
set_proofmaster_max_data_age,
set_prove_cache_job_mode,
set_prove_cache_max_jobs

set_proofmaster_dir

Specifies a location for ProofMaster data.

Syntax

```
set_proofmaster_dir dir_name
```

Returns

No return values

Default

jgproject

Detailed Description

Use this command to specify where you want to store ProofMaster data when "set_proofmaster" is on.

NOTE:
. The tool creates all parent directories as needed.
. A ProofMaster directory can be shared by multiple users. Read and write operations are enabled for all ProofMaster data.
. When you configure a ProofMaster directory, the tool creates local copies of the ProofMaster files in the project directory to avoid concurrency issues when reading and writing data during a session. At the end of each "prove" command invocation, the tool writes ProofMaster data to the specified ProofMaster directory.
. If you do not specify a ProofMaster directory, the tool stores ProofMaster data in the project directory by default. ProofMaster data is not deleted when reusing the same project directory across multiple invocations of the tool.

Example

```
set_proofmaster_dir /home/usr/proofmaster/proofMaster_123
```

See Also

set_proofmaster

set_proofmaster_initial_dir

Imports ProofMaster data from an existing dir and uses it immediately.

Syntax

```
set_proofmaster_initial_dir dir_name
```

Returns

No return values

Default

No default value

Detailed Description

Use this command to populate a new ProofMaster directory with data from an existing one and immediately use it during proof. This feature is useful when you want to leverage ProofMaster data from a centralized repository without writing back to it.

NOTE:
. The tool ignores the data in the initial ProofMaster directory if the directory configured by "set_proofmaster_dir" is already populated.
. The tool never writes back to the initial ProofMaster directory.

Example

```
set_proofmaster on
set_proofmaster_dir local_PM
set_proofmaster_initial_dir central_regression_PM
prove -task <embedded>      # This proof copies data from
                              # "central_regression_PM" to "local_PM", and uses
                              # it in this proof. No changes made to
                              # "central_regression_PM". New results are stored
                              # in "local_PM" only.
clear -result
prove -task <embedded>      # Since "local_PM" is now populated with data,
                              # data from "central_regression_PM" is ignored.
```

See Also

set_proofmaster,
set_proofmaster_dir,

set_proofmaster_max_data_age

Deletes unmatched proof cache data from the ProofMaster dir after N days.

Syntax

```
set_proofmaster_max_data_age N
```

Returns

No return values

Default

0 (unlimited). That is, by default, the tool never deletes proof cache data automatically.

Detailed Description

Use this command to specify that you want "prove" commands to automatically delete proof cache data from the ProofMaster directory that has not been matched by the tool in the last N days. This feature is useful for preventing proof cache data to grow indefinitely, enabling faster operations during proof and smaller ProofMaster databases.

Example

```
set_proofmaster on
set_proofmaster_dir local_PM
set_proofmaster_max_data_age 15
prove -task <embedded> # If any proof cache results not matched by the tool
                       # within the last 15 days exist in "local_PM", the tool
                       # permanently deletes them during proof
```

set_property_compile_time_limit

Sets the per-property time limit for compiling properties during elaboration.

Syntax

```
set_property_compile_time_limit time_limit
```

Returns

No return values

Default

5s

Detailed Description

Use this command to set the time limit for compiling each property during synthesis if you get the message "property compile time limit exceeded." Property compilation runs during elaboration. The time limit is an integer ending in "s" for seconds, "m" for minutes, or "h" for hours.

time_limit
    Use the specified limit for each "elaborate" command.

    NOTE: Jasper Apps generally recognizes "0s" as "unlimited". In the case of this command, "0s" sets the highest supported limit, which is 10000 seconds.

Example

```
set_property_compile_time_limit 15s
```

See Also

set_elab_time_limit,
set_task_compile_time_limit

set_property_delay_cycles

Adds a specified delay to properties.

Syntax

```
set_property_delay_cycles N
```

Returns

No return values

Default

0 cycles

Detailed Description

Use this command to specify the number of cycles you would like to delay properties.

NOTE:
. As this command affects the creation of the compiled properties for embedded and Tcl properties, you must run it before "elaborate" (or before creating the Tcl properties).
. "set_property_delay_cycles" is an initial release to gather feedback from early adopters and finalize implementation for an upcoming release. Contact support@cadence.com to provide feedback or comments.
. This configuration is not supported in X-Prop

N
    Delay properties by the specified number of cycles.

Example

```
set_property_delay_cycles 15
```

set_property_info_proof_timestamp

Controls the visibility of the "proof_timestamp" property attribute.

Syntax

```
set_property_info_proof_timestamp (true | false)
```

Returns

No return value

Default

true

Detailed Description

Use this command to control the visibility of the "proof_timestamp" property attribute. When "property_info_proof_timestamp" is true, the "proof_timestamp" property attribute value is returned by the "get_property_info" command.

The "proof_timestamp" property attribute is defined as follows. If the "prove" command changes the status of the property from undetermined to determined, then the "proof_timestamp" property attribute is updated with the wall clock time and date at which this determination occurs.

NOTE
. A "prove" command in this context is any command that invokes one proof thread or a set of proof threads. Examples include "prove", "hunt", and "check_assumptions".
. The "proof_timestamp" attribute of a property is the string "undefined" if the property is undetermined and has only trivial "min_length" and "max_length" bounds.

Example

```
set_property_info_proof_timestamp true
```

See Also

get_property_info

set_property_table_dynamic_sort

Sets your dynamic sorting preference for property tables.

Syntax

```
set_property_table_dynamic_sort (preference | on |off)
```

Returns

No return values

Default

preference

Detailed Description

Use this command to set your dynamic sorting preference for property tables. This setting takes precedence over the current preferences setting (Edit - Preferences).

Dynamic sorting updates the order of properties as a proof progresses and cell contents change. This command is particularly helpful if you have an extensive set of properties and you want to temporarily disable dynamic sorting for a faster GUI response time.

preference
: Dynamically sort the Property Table using the current preference setting (Edit - Preferences - General - Tables). This is the default.

on
: Turn dynamic sorting on, regardless of the current preference setting.

off
: Turn dynamic sorting off, regardless of the current preference setting.

Example

```
set_property_table_dynamic_sort off
```

set_property_triage_max_threads

Specifies the maximum number of threads for "property_triage" analysis.

Syntax

```
set_property_triage_max_threads num_threads
```

Returns

No return values

Default

1

Detailed Description

Use this command to specify the maximum number of threads to be used for "property_triage" analysis.

The "num_threads" argument is an integer greater than or equal to 1. Setting "num_threads" to 1 means that only one thread is used, while setting it to a higher value allows for parallel execution of the operations, potentially improving performance on multi-core systems.

Example

```
set_property_triage_max_threads 4
```

See Also

property_triage

set_prove_advanced_simplification

Performs property-independent model simplification before running proof command.

Syntax

```
set_prove_advanced_simplification (on | off)
```

Returns

No return values

Default

off

Detailed Description

Performs property-independent model simplification before running the actual proof command.

NOTE: This feature does not support the SEC App, the SPV App, or the X-Prop App. If you attempt to run a prove command for one of these apps when the feature is enabled, the tool issues a warning message and disables the feature.

Example

```
set_prove_advanced_simplification on
```

See Also

set_prove_advanced_simplification_time_limit

set_prove_advanced_simplification_time_limit

Specifies the wall-clock time limit for "prove_advanced_simplification".

Syntax

```
set_prove_advanced_simplification_time_limit  time_limit
```

Returns

No return values

Default

300s

Detailed Description

Use this command to change the maximum time spent on the
"prove_advanced_simplification" run.

The tool supports three formats for the time limit:
. ISO8601:   hh':'[mm[':'[ss['.'[fff]]]]]
  Examples: 2:30, 2:, 0:0:23, 2:30:05.235
. Jasper: [dd'd'][hh'h'][mm'm'][ss[.fff]'s']
  Examples: 2h30m, 2h, 23s, 2h30m05.235s
. Seconds: sssss[.fff]['s']
  Examples: 9000s, 7200, 23s, 9005.235s

Example

```
set_prove_advanced_simplification_time_limit 10m
```

See Also

set_prove_advanced_simplification

set_prove_cache

Controls the use of property proof results caching.

Syntax

```
set_prove_cache (on | off | load_only | save_only)
```

Returns

No return values

Default

off

Detailed Description

Use this command to specify the use of property proof results
caching (used when running "prove" or "visualize").

on
    Use previously cached property proof results and store new ones.

off
    Disable caching and re-using property proof results.

load_only
    Use previously cached property proof results.

save_only
    Cache new property proof results.

By default, the tool identifies relevant cache results based on the proof task.
Use "set_prove_cache_mode" to control it.

By default, the cache resides in the session project directory. Use

"set_prove_cache_path" to specify a different location.

By default, trace caching and retrieving are disabled. Use
"set_prove_cache_traces" to control it.

NOTE:

. When proof results come from the proof cache, they are marked with the virtual engine "Cache". On a cache hit for a previously undetermined result, the tool issues a proof message: "Retrieving proof results from cache."

. You can retain the cache across tool sessions by reusing the same project directory.

. Multiple properties can share the same cache file if the tool proves that they are structurally equivalent. This means that cached results might be leveraged for a property you are proving for the first time if its signature matches an existing cached property.

. "prove -simplification_only" restores results from the proof cache if "prove_cache" is on. You can prevent this behavior by running "set_prove_cache off". (The default setting is "off".)

. "cover -path" properties are not supported by the proof caching feature. The tool will not cache or retrieve results of these properties when proving a task that contains them.

Example

```
set_prove_cache on

# Using caching in your environment
# ---------------------------------
# The first time you run the following set of commands, the proof runs as usual
# because no cache exists. On subsequent runs, once the "prove" command
# starts but before engines start, Jasper evaluates the proof cache and
# restores proof results that can be restored. For example, a property was
# previously proven, and nothing changed in its COI since the last run, so it
# is marked proven without spending engine time on it. If the tool is unable to
# restore the status for a property, a normal proof runs on that property.

set_prove_cache on
set_prove_cache_path my_cache_path
...
prove...
...
exit
```

See Also

prove,
set_prove_cache_in_parallel,
set_prove_cache_job_mode,
set_prove_cache_max_jobs,
set_prove_cache_mode,
set_prove_cache_path,
set_prove_cache_traces,
visualize

set_prove_cache_in_parallel

Controls running proof caching in parallel.

Syntax

```
set_prove_cache_in_parallel (true | false)
```

Returns

No return values

Default

false

Detailed Description

Use this command to control running proof cache in parallel to the regular proof.

NOTE:
. This setting is applicable only when "prove_cache_mode" is "coi".

. Proof caching in parallel is not applicable when using "prove_clock_optimization". The tool automatically disables the feature.

Example

```
set_prove_cache_in_parallel true
```

See Also

set_prove_cache,
set_prove_cache_job_mode,
set_prove_cache_max_jobs,
set_prove_cache_mode

set_prove_cache_job_mode

Sets the grid on which to run the parallel proof caching.

Syntax

```
set_prove_cache_job_mode (local | follow_proofgrid)
```

Returns

No return values

Default

local

Detailed Description

Use this command to set the grid on which to run the parallel proof caching jobs.

local
    Run the jobs on your local machine (the machine running the Jasper Apps session).

follow_proofgrid
    Follow and run the jobs as specified in "proofgrid_mode".

NOTE: This setting is applicable only when you also enable "prove_cache_in_parallel".

Example

```
set_prove_cache_job_mode follow_proofgrid
```

See Also

set_proofgrid_mode,
set_prove_cache,
set_prove_cache_in_parallel,
set_prove_cache_max_jobs,
set_prove_cache_mode

set_prove_cache_max_jobs

Sets the limit for proof cache parallel jobs.

Syntax

```
set_prove_cache_max_jobs N
```

Returns

No return values

Default

1

Detailed Description

Use this command to set the limit for proof cache jobs running in parallel.

NOTE:
. This setting is applicable only when you also enable
  "prove_cache_in_parallel".

. Using "0" sets the limit to the highest supported value, which is the number
  of per_engine_max_jobs configured for the main proof. The reason for this
  bound is that prove cache shares licenses with the main proof. That is, it
  does not check out additional licenses.

Example

```
set_prove_cache_max_jobs 0
```

See Also

set_proofgrid_per_engine_max_jobs,
set_proofgrid_per_engine_max_local_jobs,
set_prove_cache,
set_prove_cache_in_parallel,
set_prove_cache_job_mode,
set_prove_cache_mode

set_prove_cache_mode

Controls the mode of property proof results caching.

Syntax

```
set_prove_cache_mode (task | coi)
```

Returns

No return values

Default

task

Detailed Description

Use this command to specify the mode of property proof results caching. The tool uses this configuration to identify a relevant cache result. The cache is based on structural equivalence, which means that for the tool to identify a relevant cache, it must determine that the property is structurally identical to a previously proven property.

task
    Base caching on the property's proof task. This mode is faster but less resilient against changes in the RTL, semantics, environment, and the exact sequence of properties included in the proof.

coi
    Base caching on a two-step approach to benefit from broader caching. Under this mode, the caching is based on the property's proof task (task mode). Moreover, it's based on the property's cone of influence and all assumptions, which is more resilient against changes in the RTL outside the COI of the property, semantics, environment, and the exact sequence of properties included in the proof. However, it takes longer to compute. By default, proof caching runs on the same proof thread. Use "set_prove_cache_in_parallel" to control it under this mode.

NOTE:
. Many subtle changes you make can undermine the structural equivalence even though you still perceive it to be the same property.

. When using "visualize", the tool automatically overrides this setting and sets it to "task".

. When an SEC proof is running, the SEC prove cache always uses COI mode.

Example

```
set_prove_cache_mode coi
```

See Also

set_prove_cache,
set_prove_cache_in_parallel,
set_prove_cache_job_mode,
set_prove_cache_max_jobs

set_prove_cache_path

Sets the property proof results cache directory.

Syntax

```
set_prove_cache_path full_path_to_prove_cache_directory
```

Returns

No return values

Detailed Description

Use this command to specify the property proof results cache directory.

NOTE:
. The tool creates all parent directories as needed.

. Confirm that the directory has the right access permissions for the
  mode specified by the "set_prove_cache" command.

Example

```
set_prove_cache_path ~/my_prove_cache
```

See Also

set_prove_cache

set_prove_cache_traces

Controls trace caching and retrieving from the cache during proofs.

Syntax

```
set_prove_cache_traces (true | false)
```

Returns

No return values

Default

false

Detailed Description

Use this command to control trace caching and retrieving cached traces during proofs.

Example

```
set_prove_cache_traces true
```

See Also

set_prove_cache

set_prove_check_loop_time

Sets time limit for the combo loop enumeration step in break functional flow.

Syntax

```
set_prove_check_loop_time time_limit
```

Returns

No return values

Default

600s

Detailed Description

```
The "set_loop_handling break_functional" flow has two main steps: the first step
enumerates all combo loops in the design, and the second one classifies the
loops as functionally true loops or as false loops. Use this command to set the
time limit for the combo loop enumeration step. When the tool reaches the
time-out, it switches to a faster enumeration algorithm that is not as
accurate as the default algorithm.

time_limit
    Use the specified time limit in the loop enumeration step in the
    "set_loop_handling break_functional" flow.

    The tool supports three formats for the time limit:
    . ISO8601: hh':'[mm[':'[ss['.'[fff]]]]]
      Examples: 2:30, 2:, 0:0:23, 2:30:05.235
    . Jasper: [dd'd'][hh'h'][mm'm'][ss[.fff]'s']
      Examples: 2h30m, 2h, 23s, 2h30m05.235s
    . Seconds: sssss[.fff]['s']
      Examples: 9000s, 7200, 23s, 9005.235s
    To remove the limit, use 0s.

NOTE:
. To remove the limit, use 0s.
. The "autoprove" command and AutoProve-related commands also respect the loop
  handling method specified by this command.
```

Example

```
set_prove_check_loop_time 2h
```

See Also

check_loop,
set_loop_handling,
set_loop_handling_fallback

set_prove_clock_optimization

Configures the tool to use proof clock optimization.

Syntax

```
set_prove_clock_optimization (true | false)
```

Returns

No return values

Default

false

Detailed Description

Use this command to turn on clock optimization during proof.

Defining multiple clock inputs with different clock rates, or defining the fastest clock with "-both_edges", can impact proof performance. If it is not possible to avoid multiple clock ratios or "-both_edges", then turning on clock optimization might improve proof performance.

NOTE:
. No engine, including engine TM, is guaranteed to generate minimal traces when "prove_clock_optimization" is on.
. When clock optimization is on, engines might produce traces that are longer than the specified max_trace_length.
. When clock optimization is on, the tool might produce counterexamples to :noConflict; however, it does not produce counterexample traces.
. Clock optimization is not applicable to properties of type SPV, XPROP, and "cover -path".

Example

```
set_prove_clock_optimization true
```

See Also

clock

set_prove_dump_trace_type

Specifies which types of traces to dump with "prove -dump_trace".

Syntax

```
set_prove_dump_trace_type (all | assert | cover)
```

Returns

No return values

Default

all

Detailed Description

Use this command to specify which trace types are dumped with "prove -dump_trace". By default, the tool dumps all traces generated by "prove -dump_trace".

all
    Dump all traces generated by "prove -dump_trace". This is the default.

assert
    Dump traces for assertions only.

cover
    Dump traces for covers only.

Example

```
set_prove_dump_trace_type assert
```

See Also

prove

set_prove_evaluate_in_trace_mode

Controls "prove -evaluate_in_trace" for partial or full trace evaluation.

Syntax

```
set_prove_evaluate_in_trace_mode (partial | full)
```

Returns

No return values

Default

partial

Detailed Description

Use this command to specify whether the tool evaluates part of the trace or the full trace when you use "prove -evaluate_in_trace".

NOTE: "set_prove_evaluate_in_trace_mode" is an initial release to gather feedback from early adopters and finalize implementation for an upcoming release. Contact support@cadence.com to provide feedback or comments targeting the upcoming production version.

partial
    Evaluate the properties specified by "prove -evaluate_in_trace" to find any traces shorter than the shortest known traces for the properties.

full
    Evaluate the properties specified by "prove -evaluate_in_trace" on the trace found by the proof regardless of the status of the properties or the length of the shortest known traces for them.

    NOTE: Selecting "full" mode can impact the run-time performance of the tool.

Example

```
set_prove_evaluate_in_trace_mode full
```

See Also

prove

set_prove_inferred_target_bound

Configure the tool to automatically infer target bounds for FPV assertions.

Syntax

```
set_prove_inferred_target_bound (auto | all | off)
```

Returns

No return values

Default

auto

Detailed Description

Use this command to let the tool automatically infer the target bound for assertions when Coverage App is enabled with bound type in its settings. The bounds are set only when "check_cov -measure" is executed with type bound. A target bound of an assertion is set only when all considered cover items in its COI have converged (that is, they are determined). Disabled and waived cover items are not considered.

The inferred target bound of an assertion is the maximum stimuli bound of all cover items in its COI. The cover items considered are:

auto: code coverage (Toggle, FSM, Branch, Statement, Expression) and related covers. This is the default.
all: code coverage and functional coverage, which includes both user covers and covergroups.
off: the feature is disabled and the tool does not infer target bounds for any assertion.

IMPORTANT: The status of properties reaching the inferred bound will change to "Bounded Proven (Auto)"; however, this command does not affect proofs in any way. That is, it only changes how the results are reported.

NOTE:
. You can use auto target bound together with a user-specified bound ("set_prove_target_bound").
. The statuses display if the proof reaches their respective auto target bounds.
. The tool displays the yellow bounded proof status indicator ("Bounded Proven (auto)") if it reaches the auto target bound and then green ("Bounded Proven (user)") if it reaches the user-specified target bound (set with "assert -set_target_bound" or "set_prove_target_bound").
. If the auto target bound is greater than the user-specified target bound for a specific property, the tool never displays the yellow bounded proof indicator for this property.

Example

```
set_prove_inferred_target_bound all
set_prove_inferred_target_bound off
```

See Also

assert -set_target_bound,
cover -set_target_bound,
set_prove_target_bound

set_prove_invariants_import

Enables import of invariants as implicit assumptions.

Syntax

```
set_prove_invariants_import (on | off | auto)
```

Returns

No return values

Default

auto

Detailed Description

Some engines can produce invariants during the proof. Invariants that supported the proof of one property are likely to help in proving additional properties. Use this command to enable invariant reuse by turning the invariants collected so far into implicit assumptions.

Invariant import is restricted to the scope of the current "prove" command. There is no invariant sharing between different "prove" commands.

on
    Enable invariant reuse in all proofs, with or without Proof Orchestration.

off
    Disable invariant reuse in all proofs, with or without Proof Orchestration.

auto
    Enable invariant reuse in Proof Orchestration proofs only.
    This is the default setting.

NOTE:
. Currently, only engines Mp, M, N, and AM are able to produce invariants.
  You must run at least one of these engines to benefit from invariant import.
. Mp, M, N, AM, H, Hs, Hp, and Hps are the engines able to import invariants.
. Invariant import is a continuous process as engines keep polling for
  the newly arrived invariants.

set_prove_no_cover_traces

Controls cover trace generation during proofs.

Syntax

```
set_prove_no_cover_traces (true | false)
```

Returns

No return values

Default

false

Detailed Description

Use this command to suppress or reactivate cover trace generation during proofs. With this command, you can avoid the overhead of generating and storing cover traces. The properties show covered statuses as usual, but there are no traces to inspect for covers. You must re-do the proof to get traces. Use this command when the trace data for covers is not important.

NOTE:
. This setting does not affect the "visualize" command.
. If the tool finds a trace for an assertion, and the trace also contains a witness for a cover property, you will be able to open the trace for the cover property despite specifying "set_prove_no_cover_traces true".

Example

```
set_prove_no_cover_traces true
```

See Also

prove,
set_prove_no_traces

set_prove_no_traces

Controls trace generation during proofs.

Syntax

```
set_prove_no_traces (true | false)
```

Returns

No return values

Default

false

Detailed Description

Use this command to suppress or reactivate trace generation during proofs. With this command, you can avoid the overhead of generating and storing traces. The properties show cex/covered statuses as usual, but there are no traces to inspect. You must re-do the proof to get traces. Use this command when the trace data is not important.

NOTE: This setting does not affect the "visualize" command.

Example

```
set_prove_no_traces true
```

See Also

prove,
set_prove_no_cover_traces

set_prove_on_determined

Sets a Tcl proc to call when a prove command determines a property.

Syntax

```
set_prove_on_determined proc
```

Returns

No return values

Default

""

Detailed Description

The tool calls the specified Tcl proc when a "prove" command determines a property. The tool provides three arguments: the event, property name, and prove thread ID. Currently, the event is always "determined", but it is possible that we will introduce more events in a future version.

When implementing the Tcl proc, take into account that it will run while the "prove" command is running. This means that all Jasper Tcl commands that are prohibited while a proof is running will fail with a Tcl error if you include them in the proc. The "clear" command is one such command.

CAUTION: on_determined Tcl procs are very powerful, but they can be unsafe as they are hard to debug, and it is possible to use Tcl procs that make the tool loop. We recommend using "prove -on_determined" to register a prove-command-specific Tcl proc since the global setting is the default for all proofs, and it might result in consequences you did not consider.

Example

```
set n 0
proc print_determined {event property_name thread_id} {
    global n
    puts "determined property [incr n]: $property_name,  thread $thread_id"
}
prove -all -bg -on_determined print_determined
```

See Also

prove -on_determined

set_prove_orchestration

Controls orchestration of proof engines.

Syntax

```
set_prove_orchestration (on | off)
```

Returns

No return values

Default

on

Detailed Description

Use proof orchestration to specify that you want the tool to optimize the proof dynamically, by applying learning techniques to tune the list of engines, property order, and per-property time limits. This includes:
. Adding and stopping engines dynamically while the proof is running
. Dynamically changing the order in which properties are scanned
. Dynamically changing the per-property time limit

Important things to know about this command:
. If "engine_mode" is "auto" and "proofgrid_max_jobs" is 0, orchestration uses 10 jobs when "proofgrid_mode" is not "local", and 4 jobs when it is "local".
. This setting only affects proofs run with the "prove" or "visualize" commands. It does not affect proofs run with the "hunt" command.
. Use "prove -orchestration" to override this setting for individual proofs.
. Orchestration does not support "per_property_time_limit_factor" 1 or 0. In these cases, if you do not specify "set_prove_orchestration off", the tool issues an error.
. If "per_property_time_limit" is 0, all engines run without a time limit. Orchestration can add engines if resources are available, but no longer stops engines. This mode is recommended when running one or a few properties only.

on
    Enable orchestration, which is the default.

    The tool considers user settings in general, but it might not strictly respect them. In particular, if you use "set_engine_mode" with a list of engines, orchestration chooses engines from the list dynamically. That is, it does not use engines outside the list. If you specify "set_engine_mode auto", orchestration determines the set of engines on its own.

off
    Disable orchestration.

    The tool uses the pre-2016.12 proof behavior.

Example

```
set_prove_orchestration off
```

See Also

set_prove_orchestration_with_trace_swarm,
prove,
set_engine_mode

set_prove_orchestration_hunt_trace_swarm_jobs_ratio

Controls the percentage of proof jobs to be shifted to "trace_swarm".

Syntax

set_prove_orchestration_hunt_trace_swarm_jobs_ratio ratio

Returns

No return values

Default

0.1

Detailed Description

Use this command to determine the ratio of proof jobs that are shifted to "trace_swarm" when it starts running.

The ratio is defined as a floating-point value in the open interval (0,1)

NOTE: If the resulting "trace_swarm" jobs exceeds the "trace_swarm" licenses limit, the tool reduces "trace_swarm" jobs and issues a warning.

Example

set_prove_orchestration_hunt_trace_swarm_jobs_ratio 0.5

See Also

set_prove_orchestration_with_trace_swarm,
set_prove_orchestration_hunt_trace_swarm_start_time_ratio

set_prove_orchestration_hunt_trace_swarm_start_time_ratio

Controls the ratio of proof time after which "trace_swarm" runs.

Syntax

```
set_prove_orchestration_hunt_trace_swarm_start_time_ratio ratio
```

Returns

No return values

Default

0.25

Detailed Description

Use this command to define the percentage of proof orchestration run time after which "trace_swarm" is initiated.

The ratio is defined as a floating-point value in the range [0, 1), where 0 ≥ ratio < 1.

NOTE: "trace_swarm" starts running after a time corresponding to (ratio * proof total time limit) passes.

Example

```
set_prove_orchestration_hunt_trace_swarm_start_time_ratio 0.3
```

See Also

set_prove_orchestration_with_trace_swarm

set_prove_orchestration_hunt_trace_swarm_strategy

Sets the "trace_swarm" strategy that runs along with proof orchestration.

Syntax

```
set_prove_orchestration_hunt_trace_swarm_strategy strategy
```

Returns

No return values

Default

Detailed Description

Use this command to set the hunt "trace_swarm" strategy that runs along with proof orchestration.

The strategy must be predefined using the "hunt -config -mode trace_swarm" command.

NOTE: The "max_jobs" and "time_limit" configurations defined for the "trace_swarm" strategies are ignored when integrated to proof orchestration. To define these values, use "set_prove_orchestration_hunt_trace_swarm_jobs_ratio" and "set_prove_orchestration_hunt_trace_swarm_start_time_ratio", respectively.

Example

```
hunt -config -strategy ts -mode trace_swarm -engine_mode {Ht B}
set_prove_orchestration_hunt_trace_swarm_strategy ts
```

See Also

set_prove_orchestration_with_trace_swarm,
set_prove_orchestration_hunt_trace_swarm_jobs_ratio,
set_prove_orchestration_hunt_trace_swarm_start_time_ratio,
hunt -config

set_prove_orchestration_stagnation

Controls stagnation monitoring in orchestration proofs.

Syntax

```
set_prove_orchestration_stagnation (true | false)
```

Returns

No return values

Default

true

Detailed Description

Use this command to control stagnation monitoring in orchestration proofs.

true: Monitor proof stagnation.
false: Do not monitor proof stagnation.

Example

```
set_prove_orchestration_stagnation false
```

See Also

set_prove_orchestration

set_prove_orchestration_with_trace_swarm

Controls whether or not "trace_swarm" runs along with proof orchestration.

Syntax

```
set_prove_orchestration_with_trace_swarm (true | false)
```

Returns

No return values

Default

false

Detailed Description

Use this command to control whether or not "trace_swarm" runs along with proof orchestration.

true: Launch "trace_swarm" after a given time running proof orchestration and shift resources from prove to "trace_swarm", accordingly.
false: Do not launch "trace_swarm" with proof orchestration.

NOTE:
. Use the "set_prove_orchestration_hunt_trace_swarm_start_time_ratio" command to define the percentage of proof orchestration run time after which "trace_swarm" will be initiated.
. Use "set_prove_orchestration_hunt_trace_swarm_jobs_ratio" to specify the percentage of proof orchestration jobs to be shifted to "trace_swarm"

Example

```
set_prove_orchestration_with_trace_swarm false
```

See Also

set_prove_orchestration_hunt_trace_swarm_strategy,
set_prove_orchestration_hunt_trace_swarm_start_time_ratio,
set_prove_orchestration_hunt_trace_swarm_jobs_ratio,
set_prove_orchestration

set_prove_per_property_max_time_limit

Specifies the single-property engine time limit for the proof of each property.

Syntax

```
set_prove_per_property_max_time_limit time_limit
```

Returns

No return values

Default

0s

Detailed Description

```
Use this command to set a wall clock time limit for the proof of each
individual property. After at least one engine works consecutively on a
property for a duration greater than the user-specified limit, all
single-property engines will stop processing that property.

The tool supports three formats for the time limit:
. ISO8601:    hh':'[mm[':'[ss['.'[fff]]]]]
  Examples:   2:30, 2:, 0:0:23, 2:30:05.235
. Jasper: [dd'd'][hh'h'][mm'm'][ss[.fff]'s']
  Examples:   2h30m, 2h, 23s, 2h30m05.235s
. Seconds:    ssssss[.fff]['s']
  Examples:   9000s, 7200, 23s, 9005.235s

Important things to know about this command:
. This command is useful when the value of the variable
  "prove_per_property_time_limit_factor" is greater than one.
  It allows you to limit the growth of the "per_property_time_limit" variable.

. Engine modes that operate on all properties in parallel (for example,
  H, J, L) are not affected by this command.

NOTE: To remove the limit, use 0s.
```

Example

```
set_prove_per_property_max_time_limit 15m
```

See Also

set_prove_per_property_time_limit,
set_prove_per_property_time_limit_factor,
set_prove_time_limit

set_prove_per_property_time_limit

Specifies the wall clock time limit for the proof of each property.

Syntax

```
set_prove_per_property_time_limit time_limit
```

Returns

No return values

Default

1s

Detailed Description

Use this command to set a wall clock time limit for the proof of each individual property. The tool supports three formats for the time limit:

```
. ISO8601:    hh':'[mm[':'[ss['.'[fff]]]]]
  Examples:   2:30, 2:, 0:0:23, 2:30:05.235
. Jasper: [dd'd'][hh'h'][mm'm'][ss[.fff]'s']
  Examples:   2h30m, 2h, 23s, 2h30m05.235s
. Seconds:    sssss[.fff]['s']
  Examples:   9000s, 7200, 23s, 9005.235s
```

This command is useful for allowing a "prove" command to spend a limited amount of time on every property. Without this command, there is a risk that all the "prove_time_limit" time is spent on the first hard property even though there might be many easy properties following. (This risk does not apply to engine H, J, or L, since they run all properties in parallel.)

By combining this command with "set_prove_per_property_time_limit_factor" the "prove" command does not stop when the time limit has expired for all properties; rather, it restarts with an effective "per_property_time_limit" grown by the factor. The goal of this feature is to allow the "prove" command

```
to quickly determine the result for simple properties while giving harder
properties the time they need.

In the following example, if we have 10 properties to be proven with
the "prove" command, the tool allocates no more than 10 seconds
for each property and 120 wall clock seconds for the "prove"
command as a whole.

set_prove_per_property_time_limit_factor 0
set_prove_per_property_time_limit 10s
set_prove_time_limit 2m

time_limit
    Use the specified time limit for the proof of each property.

    NOTE: To remove the limit, use 0s.

Important things to know about this command:
. Unlike "set_prove_time_limit", the time this setting limits starts when an
  engine job actually starts proving a particular property. Thus, it
  excludes time spent doing reset analysis and proof simplification,
  waiting in batch queues, and so forth.

. This limit is individually respected by each concurrent proof
  job. Different jobs might have different startup latencies, so they
  are not necessarily synchronized with respect to this time limit.

. Engine modes that operate on all properties in parallel (for example,
  H, J, L) support this setting only in the sense that the total time
  is limited by this time multiplied by the number of properties.

. Using "set_prove_per_property_time_limit_factor", the expiration of this
  limit, for all properties, triggers a restart with a geometrically
  increased factor rather than stopping the proof. In this case,
  "per_property_time_limit" will never be a reason for the "prove" command
  to stop. The "prove_time_limit" configuration works as usual.
```

Example

```
set_prove_per_property_time_limit 5m
```

See Also

set_cmd_time_limit,
set_elab_time_limit,
set_prove_per_property_time_limit_factor,
set_prove_time_limit

set_prove_per_property_time_limit_factor

Controls iterative geometric progression of prove_per_property_time_limit.

Syntax

```
set_prove_per_property_time_limit_factor factor
```

Returns

No return values

Default

10

Detailed Description

Use this command to set the common ratio used for a geometric progression of prove_per_property_time_limit or to turn off geometric progression by setting the factor to "0". The effective per_property_time_limit will be:

per_property_time_limit * factor ^ scan

Where "scan" starts at zero and increments after each complete sweep over the properties in the current proof.

This command is useful for allowing individual engine modes within a proof to quickly prove simple properties while giving harder properties sufficient time to prove. The setting does not apply to inherently parallel modes like H, J, or L on safety properties.

At the boundary between one scan and the next a proof message is issued indicating the effective per_property_time_limit for the coming scan.

factor
    Use the specified factor to generate the geometric progression.

    NOTE:
    . To disable geometric progression, use 0.
    . The tool automatically disables geometric progression when the number of remaining properties is less than or equal to the number of held licenses.
    . When geometric progression is automatically disabled, per-property time becomes unlimited, that is, "0s".

Example

```
set_prove_per_property_time_limit 1s
set_prove_per_property_time_limit_factor 30
# effective time limits: 1s, 30s, 15m, 7.5h
```

See Also

set_prove_per_property_time_limit

set_prove_prefer_shortest

Controls whether engines that can continue searching for a shorter trace.

Syntax

```
set_prove_prefer_shortest (on | off)
```

Returns

No return values

Default

off

Detailed Description

By default, a proof will consider a property to be solved as soon as it has been established whether it has a trace. Turn this setting on to make proofs consider a property to be fully solved only when a shortest trace is established. When a shortest trace is established, the "min_length" and "max_length" bounds are the same (see "get_property_info"), and the property table displays a single number in the "Bound" column.

NOTE:

. If you are using orchestration, which is the default if "set_proofmaster" is "on", this setting helps orchestration choose appropriate engines, including the specialized "trace minimization" engine (TM), that enable continued searches for a minimal trace after the cex/covered status has been achieved and a non-shortest trace has been found.
. If you are not using orchestration but are enabling "prove_prefer_shortest", you should ensure that the engine list provided to "set_engine_mode" includes engine TM.
. This setting also applies to "check_assumptions".
. This setting is supported by SEC only when you use the "User" strategy or the "-no_auto" switch.
. This setting is not supported by SPV or X-Prop.

Example

```
set_prove_prefer_shortest on
```

See Also

prove,
check_assumptions,
get_property_info

set_prove_report_mem

Controls the generation of frequent engine job memory consumption messages.

Syntax

```
set_prove_report_mem (on | off)
```

Returns

No return values

Default

off

Detailed Description

Use this command to turn on or off the generation of memory consumption proof messages in the "Proof Messages" tab. Refer to the following example:

```
2017-09-17 11:45:55: 0.13.N: Mem: 3.36MiB [0.00 s]
```

NOTE: The numbers given correspond to the Resident Set Size reported by the operating system and might be misleading.

Example

```
set_prove_report_mem on
```

set_prove_report_period

Sets the sampling period for requesting memory reports from proof processes.

Syntax

```
set_prove_report_period period
```

Returns

No return values

Default

10

Detailed Description

Use this command to control the period with which the engine jobs receive requests for sampling their memory consumption.

The argument corresponds to the number of seconds between consecutive requests, and it must be a positive integer.

NOTE: If you change this value while proof jobs are running in the background, the new value will only be effective after the next sample request.

Example

```
set_prove_report_period 15
```

See Also

set_prove_report_mem

set_prove_report_task

Controls the inclusion of the task name in proof messages.

Syntax

```
set_prove_report_task (on | off)
```

Returns

No return values

Default

off

Detailed Description

Use "set_prove_report_task" to control whether Jasper Apps prefixes property names with the task name in proof messages. This feature is useful for distinguishing between multiple tasks that include properties with the same name.

Example

```
set_prove_report_task on
...
prove -all
...
INFO (IPF057): 1.0.N: The property "foo0::top.eg.i_egress.a_wvalid_2cycle" was proven
in 0.01 s.
INFO (IPF057): 1.0.N: The property "foo0::top.brdg.i_bridge.a_fifo_no_overflow" was
proven in 0.02 s.
INFO (IPF047): 1.0.N: The cover property
"foo0::top.brdg.i_bridge.a_fifo_no_overflow:precondition1" was covered in 17 cycles in
4.06 s.
INFO (IPF057): 1.0.N: The property "foo1::top.eg.i_egress.a_wvalid_2cycle" was proven
in 0.01 s.
...
```

set_prove_start_timer_on_engine_start

Starts prove command timer only after the first proof job starts.

Syntax

```
set_prove_start_timer_on_engine_start (on | off)
```

Returns

No return values

Default

on

Detailed Description

Use this command to make the "prove" command timer start counting only after at least one engine has started.

Example

```
set_prove_start_timer_on_engine_start on
```

See Also

set_prove_per_property_time_limit,
set_prove_time_limit

set_prove_stop_on_cex_limit

Sets the value of the variable "prove_stop_on_cex_limit".

Syntax

```
set_prove_stop_on_cex_limit N
```

Returns

No return values

Default

0 (unlimited)

Detailed Description

Use this command to direct the "prove" command to stop right after it reports the first "prove_stop_on_cex_limit" counterexamples.

Example

```
set_prove_stop_on_cex_limit 1
```

See Also

prove

set_prove_stop_on_target_bound

Sets the value of the variable "prove_stop_on_target_bound".

Syntax

```
set_prove_stop_on_target_bound (on | off)
```

Returns

No return values

Default

off

Detailed Description

Use this command to direct the engines to stop working on a property when the target bound of the property is reached.

NOTE: This command is supported only for orchestration proofs, hunt "cycle_swarm", SEC, SPV and XPROP.

Example

```
set_prove_stop_on_target_bound on
```

See Also

assert -set_target_bound,
cover -set_target_bound

set_prove_stop_on_unreachable_limit

Sets the value of the variable "prove_stop_on_unreachable_limit".

Syntax

```
set_prove_stop_on_unreachable_limit N
```

Returns

No return values

Default

0 (unlimited)

Detailed Description

Use this command to direct the "prove" command to stop immediately after reporting the first "prove_stop_on_unreachable_limit" unreachable covers.

Example

```
set_prove_stop_on_unreachable_limit 1
```

See Also

prove

set_prove_summary

Hides the SUMMARY table at the end of the "prove" command.

Syntax

```
set_prove_summary (on | off)
```

Returns

No return values

Default

on

Detailed Description

By default, when the "prove" command completes, the tool displays a proof "SUMMARY" table. Use this command to suppress the table or reinstate the default behavior.

Example

```
set_prove_summary off
```

See Also

prove

set_prove_target_bound

Specifies a global target bound for all properties.

Syntax

```
set_prove_target_bound (N)
```

Returns

No return values

Default

0

Detailed Description

Use this command to specify a global target bound for all properties.

IMPORTANT: The status of properties reaching the specified bound will change to "Bounded Proven (User)" or "Bounded Unreachable (User)"; however, this command does not affect proofs in any way. That is, it only changes how the results are reported. If you also want engines to stop further analysis once they reach the target bound, you must also use "set_max_trace_length."

NOTE:
. If you want proofs to stop when this target bound is reached, the "set_prove_stop_on_target_bound" configuration must be "true". Otherwise, Jasper continues running proofs after reaching the target bound.
. Liveness properties are not supported and ignored by this command.
. "clear -result" does not affect the global target bound.
. This command has no effect on properties for which a finite target bound (> 0) has been set using "assert -set_target_bound" or "cover -set_target_bound".
. If you restore a jasper_database using the "restore -jdb" command, the status of an "Undetermined" property does not automatically change to a bounded status according to the current target bound. You must specify the target bound again after restoring the database.

N
    Apply the specified bound as the global target bound.
    Default is 0 with a special meaning that the global target bound is infinite.

Example

```
set_prove_target_bound 15
```

See Also

assert -set_target_bound,
cover -set_target_bound,
set_prove_stop_on_target_bound,
set_max_trace_length,
set_prove_inferred_target_bound

set_prove_time_limit

Sets the time limit for Visualize and proofs.

Syntax

```
set_prove_time_limit time_limit
```

Returns

No return values

Default

86400s (24h)

Detailed Description

Use this command to set the time limit for the following commands:

. autoprove
. check_afv -prove
. check_assumptions
. check_cdc -metastability -prove
. check_cdc -protocol_check -prove
. check_cdc -signal_config -prove
. check_cdc -waiver -prove
. check_conn -prove
. check_cov -measure
. check_csr -prove
. check_fsv -prove
. check_sec -prove
. check_spv -prove
. check_unr -prove
. check_xprop -prove
. formal_profiler
. hunt
. prove

```
. visualize

The time excludes preprocessing steps such as reset analysis, which runs
during the first "prove" but not during subsequent "prove" calls.

The tool supports three formats for the time limit:
. ISO8601:    hh':'[mm[':'[ss['.'[fff]]]]]
  Examples:   2:30, 2:, 0:0:23, 2:30:05.235
. Jasper: [dd'd'][hh'h'][mm'm'][ss[.fff]'s']
  Examples:   2h30m, 2h, 23s, 2h30m05.235s
. Seconds:    sssss[.fff]['s']
  Examples:   9000s, 7200, 23s, 9005.235s

NOTE:
. The tool re-starts the time_limit with each new task.
. The command "set_cmd_time_limit" has no effect on the commands that honor
  this command.
. The command "set_prove_start_timer_on_engine_start" controls whether the time
  limit includes the time spent in batch queue when engine jobs are first
  launched (see "set_proofgrid_mode").

time_limit
    Use the specified time limit for each "prove" or "visualize" command.

    NOTE: To remove the limit, use 0s.
```

Example

```
set_prove_time_limit 2h
```

See Also

set_cmd_time_limit,
set_elab_time_limit,
set_prove_per_property_time_limit,
set_prove_start_timer_on_engine_start

set_prove_verbosity

Sets the verbosity level for the "prove" command.

Syntax

```
set_prove_verbosity 0-11
```

Returns

No return values

Default

6

Detailed Description

Use this command to control the amount of information printed to the log, the console, and the proof messages tab. The higher the value, the more output.

Value	Description
0	Silent
1	Adds IPF029 assigning name to proof target
2	Adds IPF031 proof settings
3	Adds IPF035 task list message
4	Adds proof summary and generic proof error messages
5	Adds the most basic, high-level proof message tab messages
6	Adds proof simplification and brief trace attempt messages (default verbosity)
7	Adds most low-level proof message tab messages
8	Adds trace attempt time-out and brief size messages
9	Adds fine-grained design size messages and engine L- and G-specific messages
11	Adds X transform messages

NOTE: Responsiveness for the main Jasper GUI becomes slow when it is flooded by proof engine messages. As a workaround, you can reduce the prove_verbosity. Consider reducing it to less than 6, and if that does not solve the sluggishness, you can go as low as 0.

Example

```
set_prove_verbosity 0
```

See Also

prove

set_prove_with_soft

Instructs proofs to apply any available user-created soft constraints.

Syntax

```
set_prove_with_soft (on | off)
```

Returns

No return values

Default

off

Detailed Description

Proofs attempt to apply any available user-created soft constraints to generate an improved trace for properties with the status "cex" or "covered".
If "assume -soft -quiet" has been used, or elaboration was done after "set_elaborate_quiet_trace on", this effectively enables QuietTrace(TM).
By default, a proof considers a property to be solved as soon as it has a trace, without considering any soft constraints. Turn this setting on to make proofs attempt to also satisfy all soft constraints.

NOTE:

. Engine QT has the ability to continue working on a property after finding a trace. Turning this variable on automatically adds engine QT to the engine mode.
. Engine QT only establishes quiet traces for properties with soft constraints in their COI. That is, it searches for quiet traces after it receives the status cex or covered.
. When engine QT establishes a quiet trace, a "QT" tag appears next to the engine in the Engine column of the property table. The Bound column is not affected by the result of engine QT.

Example

```
set_prove_with_soft on
```

See Also

assume -soft,
assume -soft -max_from,
assume -soft -noisy,
assume -soft -quiet,
assume -soft -quiet_from,
assume -xprop -soft -max_from,
assume -xprop -soft -noisy,
assume -xprop -soft -quiet,
assume -xprop -soft -quiet_from,
get_property_info,
prove,
prove -soft_low_constraints,
prove -soft_constraints,
prove -with_soft,
set_prove_with_soft_mode,
set_prove_prefer_shortest

set_prove_with_soft_mode

Controls which property types have soft constraints applied during proofs.

Syntax

```
set_prove_with_soft_mode (none | asserts | covers | all | default)
```

Returns

No return values

Default

default

Detailed Description

Use this command to control whether proofs attempt to apply any available user-created soft constraints, and which property types are targeted.

. all: Apply soft constraints to all properties (assertions and covers).
. covers: Apply soft constraints to cover properties only.
. asserts: Apply soft constraints to assertion properties only.
. none: Do not apply soft constraints to any properties.
. default: Use the default behavior (same as "all").

This command provides finer-grained control compared to "set_prove_with_soft", which only supports "on" or "off". "set_prove_with_soft on" is equivalent to "set_prove_with_soft_mode all", and "set_prove_with_soft off" is equivalent to "set_prove_with_soft_mode none".

NOTE:
. Engine QT has the ability to continue working on a property after finding a trace. Enabling this setting automatically adds engine QT to the engine mode.
. Engine QT only establishes quiet traces for properties with soft constraints in their COI. That is, it searches for quiet traces after it receives the status cex or covered.
. When engine QT establishes a quiet trace, a "QT" tag appears next to the engine in the Engine column of the property table. The Bound column is not affected by the result of engine QT.

Example

```
# Apply soft constraints to cover properties only
set_prove_with_soft_mode covers

# Apply soft constraints to all properties
set_prove_with_soft_mode all

# Disable soft constraints
set_prove_with_soft_mode none
```

See Also

`assume -soft`,
`assume -soft -quiet`,
`proofmaster -config -with_soft`,
`prove -with_soft`,
`set_prove_with_soft`,
`set_prove_prefer_shortest`

set_proven_directive

Enables the tool to use proven assertions and unreachable covers as assumptions.

Syntax

```
set_proven_directive (true | false)
```

Returns

No return values

Default

true

Detailed Description

By default, the tool uses proven assertions and unreachable covers as assumptions.
Use this command to turn this feature "on" or "off".

Important things to know about this command:

. Using proven or unreachable properties as assumptions can help the proof to determine other properties. However, this behavior is not guaranteed, so it is also possible for this feature to deteriorate the performance of a formal proof.

. Properties that are already proven or unreachable before the "prove" command are guaranteed to be used as assumptions when this feature is "on". These properties must be included in the target, either by being included in the property list or by using "-with_proven".

. Properties proven during the execution of the proof might not be used as assumptions by all formal engines, even when this feature is "on". To ensure that all formal engines use proven and unreachable properties as assumptions, you must restart the proof.

```
  Due to this, restarting the proof after some properties are proven
  might lead to better results.

  Example:

  set_proven_directive true
  prove -property P1              # P1 is proven
  prove -property P2              # P1 is not used as assumption (not
                                  # included in property list)
  prove -property P3 -with_proven # P1 is used as an assumption (included by
                                  # "-with_proven")
  prove -property {P1 P4}         # P1 is used as assumption (included in
                                  # property list)
  prove -task my_task             # P1 is used as an assumption (in my_task)

. Due to this feature, it is possible to see better performance when stopping
  and then restarting a formal proof. The reason is that properties proven
  during a proof may not be used as assumptions by all formal engines. However,
  after restarting the proof, all formal engines will use proven properties as
  assumptions, and only then, is performance boosted.

  Example:

  prove -property {P1 P2} # P1 is proven during formal proof and not used as
                          # an assumption for P2 (see bullet above)
                          # As a result, P2 remains undetermined
  prove -property {P1 P2} # P1 was proven before this formal proof, so it is
                          # used as assumption for P2 (see bullet above)
                          # As a result P2 is now proven
```

Example

```
set_proven_directive false
```

See Also

prove -with_proven
hunt -run -with_proven

set_read_sdc_debug

Enables SDC Debug Analysis.

Syntax

```
set_read_sdc_debug ( on | off )
```

Returns

No return values.

Default

off

Detailed Description

Use this command to enable SDC Debug Analysis during read_sdc. By enabling this command, cte.log file shows additional details of SDC issues found during SDC Parsing and SDC Analysis. It also enables "SDC Constraints Extraction" section in MCP Audit Report (generated with check_timing_exception -audit) command.

Example

```
set_read_sdc_debug on
```

See Also

read_sdc
check_timing_exception

set_regexp_mode

Changes Jasper regular expression semantics.

Syntax

```
set_regexp_mode (legacy | tcl | perl)
```

Returns

No return values

Default

legacy

Detailed Description

When "set_regexp_mode" is legacy, each Tcl command uses the regular expression semantics it has historically used for backward compatibility.

The "perl" and "tcl" mode of unified regular expression behavior are partial string match and case sensitive enabled.

Enabling this command affects the following commands:

. assert -clear_store_trace
. assert -clear_trace_extension
. assert -disable
. assert -enable
. assert -from_assume
. assert -remove
. assert -set_store_trace
. assert -set_trace_extension
. assume -disable
. assume -enable
. assume -from_assert

. assume -remove
. assume -reset -disable
. assume -reset -enable
. assume -reset -remove
. assume -set_type
. autoprove -property
. check_c2rtl -disable_mapping
. check_c2rtl -enable_mapping
. check_c2rtl -import_property
. check_c2rtl -list
. check_c2rtl -map
. check_c2rtl -set_context
. check_c2rtl -unmap
. check_cdc -filter -add
. check_cdc -protocol_check -export
. check_cdc -report
. check_conn -load
. check_cov -init
. check_cov -list
. check_cov -measure
. check_cov -report
. check_fsv -barrier -add
. check_fsv -fault -list
. check_fsv -fault -remove
. check_fsv -fault_work -import
. check_fsv -strobe -add
. check_fsv -strobe -list
. check_fsv -strobe -remove
. check_reset -breakpoint -list
. check_sec -auto_map_reset_x_values
. check_sec -clear_context_of_mapping
. check_sec -disable_mapping
. check_sec -enable_mapping
. check_sec -get_twin_of_signal
. check_sec -import_property
. check_sec -list
. check_sec -map
. check_sec -mapping_info
. check_sec -prove
. check_sec -related_verification
. check_sec -set_context_of_mapping
. check_sec -setup
. check_sec -show_schematic

. check_sec -signoff
. check_sec -unmap
. check_superlint -list
. check_superlint -report
. check_superlint -waiver -add
. check_unr -disable_property
. check_unr -enable_property
. check_unr -list
. check_unr -prove
. check_xprop -abstract
. check_xprop -highlight_failure_path
. check_xprop -import_cover
. check_xprop -propagation
. check_xprop -prove
. check_xprop -waiver -add
. config_rtlds -no_scan
. config_rtlds -rule
. compilation -config
. cover -clear_store_trace
. cover -clear_trace_extension
. cover -disable
. cover -enable
. cover -extend
. cover -remove
. cover -set_store_trace
. cover -set_trace_extension
. expert_system -waiver -add
. get_analyze_info -location
. get_design_info -compilation_unit
. get_design_info -entity
. get_design_info -instance
. get_design_info -interface
. get_design_info -list
. get_design_info -module
. get_design_info -property
. get_design_info -signal
. get_design_info -task
. get_property_list
. get_signal_list
. get_status
. hunt -run
. proof_structure -init
. prove -ignore_property

```
. prove -property
. set_annotation
. task -create
. task -edit
. visualize
. visualize -at_least_once
. visualize -check_props
. visualize -directed_test -list
. visualize -force
. visualize -not
. visualize -replace_conf
. visualize -set_target
. visualize -violation

The "set_sec_hierarchical_regular_expression" command is a known exception to
"set_regexp_mode".
    This command uses Tcl semantics by default and does not support Perl mode.
```

Example

```
set_regexp_mode tcl
```

set_reset_effort

Defines algorithm-effort level used during reset analysis.

Syntax

```
set_reset_effort (low | medium | high)
```

Returns

No return values

Default

low

Detailed Description

Use this command to configure the algorithm-effort level applied during reset analysis. Since there is a trade-off between performance and effectiveness of constant propagation during reset analysis, this configuration provides a way to choose the amount of algorithm effort used in constant propagation according to your performance requirements.

Example

```
set_reset_effort high
analyze -verilog my_design.v
elaborate -top my_design
clock clk
reset -sequence reset.seq
get_reset_info
```

set_reset_extended_mode

Defines when reset analysis will use information related to "check_reset".

Syntax

```
set_reset_extended_mode (on | off)
```

Returns

No return values

Default

on

Detailed Description

Use this command to configure whether reset analysis invoked by commands that require it will use configuration and verification settings specified with "check_reset".

See Also

check_reset

set_reset_max_iterations

Sets the max number of iterations used for reset analysis.

Syntax

```
set_reset_max_iterations N
```

Returns

No return values

Default

100

Detailed Description

Use this command to set the max number of iterations in the reset analysis.

If you use the command "reset -max_iterations" to set a value, the tool ignores this configuration variable during reset analysis.

Example

```
set_reset_max_iterations 4000
```

See Also

reset

set_resilient_compilation

Enables or disables resilient compilation.

Syntax

```
set_resilient_compilation (on | off)
```

Returns

No return values

Default

"on" for the UNR App with the P2P flow
"off" for all other apps and flows

Detailed Description

Enables or disables resilient compilation while running "analyze" and "elaborate" commands.

Resilient compilation automatically handles errors that occur during design analysis and elaboration such as unsynthesizable constructs, syntax errors, and undeclared references. By default, modules that contain errors are blackboxed and a list of messages is relaxed when you use "set_resilient_compilation on". You can refine this approach with the "set_resilient_compilation_mode" command.

NOTE: You cannot use this command after design elaboration.

Example

```
set_resilient_compilation on
```

See Also

set_resilient_compilation_mode

set_resilient_compilation_mode

Sets the resilient compilation mode.

Syntax

```
set_resilient_compilation_mode (  module | always_process_block
                                | continuous_assignment | module_instantiation
                                | property | relaxed_checking | all)+
```

Returns

No return values

Default

"all" for UNR App with the P2P flow
"module" and "relaxed_checking" for all other apps and flows

Detailed Description

Use this command to define the level of granularity or strategy used to automatically handle errors during resilient compilation.

Supported resilient compilation modes:

. module: Set this mode to have modules automatically black boxed by the tool when it finds errors in their contents during analysis or elaboration. This mode is active by default and cannot be deactivated.

. always_process_block: Set this mode to have errors handled at the always block level (Verilog and SystemVerilog) and process block level (VHDL). Multiple statements can be contained within always and process blocks. The tool only discards synthesis of the ones that contain errors.

. continuous_assignment: Set this mode to have errors handled at the

continuous assignment level (Verilog and SystemVerilog). The language allows multiple assignments in a single continuous assignment construct. The tool only discards synthesis of the ones that contain errors, unless the error is related to the overall continuous assignment. In that case, it also discards all child assignments.

. module_instantiation: Set this mode to handle errors at the module instantiation level (Verilog and SystemVerilog) and component instantiation level (VHDL). If the tool finds errors in the contents of an instantiation, it automatically black boxes the related instance. At this level, it is not necessary to black box the containing module or unit, and as result, the tool preserves more design logic.

. property: Set this mode to have errors handled at the embedded property level. For property-related errors having type:
  . "VERI-XXXX": the synthesis of the property is fully discarded (currently only supported in SystemVerilog).
  . "EOBSXXX": the property is still visible after elaboration but in the disabled state.

. relaxed_checking: Set this mode to automatically change the default severity of some messages from error to warning. This is a strategy that relaxes some syntactic and semantic checks defined in the VHDL, Verilog, and SystemVerilog LRMs and some checks related to limitations in the tool, while maintaining well-defined behavior.

  The tool applies "relaxed_checking" before attempting to apply any of the other resilient compilation modes. In some cases, "relaxed_checking" allows resilient compilation to make progress without depending on other specified modes (for example, "module", "always_process_block", and so forth). And as a result, the tool can preserve more design logic. This mode is active by default.

  Below is the list of relaxed messages:
  . VERI-1010, VERI-1011, VERI-1014, VERI-1015, VERI-1016, VERI-1042, VERI-1066, VERI-1070, VERI-1071, VERI-1084, VERI-1122, VERI-1139, VERI-1148, VERI-1158, VERI-1165, VERI-1180, VERI-1184, VERI-1187, VERI-1196, VERI-1216, VERI-1225, VERI-1226, VERI-1240, VERI-1241, VERI-1262, VERI-1264, VERI-1265, VERI-1273, VERI-1276, VERI-1289, VERI-1293, VERI-1313, VERI-1314, VERI-1315, VERI-1323, VERI-1339, VERI-1343, VERI-1347, VERI-1348, VERI-1349, VERI-1350, VERI-1352, VERI-1353, VERI-1370, VERI-1377, VERI-1401, VERI-1403, VERI-1419,

```
        VERI-1420, VERI-1427, VERI-1428, VERI-1430, VERI-1437, VERI-1438,
        VERI-1439, VERI-1442, VERI-1444, VERI-1461, VERI-1463, VERI-1524,
        VERI-1532, VERI-1533, VERI-1535, VERI-1539, VERI-1542, VERI-1556,
        VERI-1557, VERI-1558, VERI-1564, VERI-1575, VERI-1576, VERI-1580,
        VERI-1597, VERI-1615, VERI-1620, VERI-1622, VERI-1624, VERI-1636,
        VERI-1648, VERI-1655, VERI-1659, VERI-1663, VERI-1683, VERI-1684,
        VERI-1691, VERI-1716, VERI-1736, VERI-1740, VERI-1746, VERI-1749,
        VERI-1750, VERI-1760, VERI-1797, VERI-1809, VERI-1815, VERI-1868,
        VERI-1893, VERI-1895, VERI-1896, VERI-1906, VERI-1928, VERI-1952,
        VERI-1974, VERI-1976, VERI-1980, VERI-1991, VERI-2048, VERI-2058,
        VERI-2065, VERI-2072, VERI-2079, VERI-2085, VERI-2089, VERI-2091,
        VERI-2094, VERI-2097, VERI-2136, VERI-2162, VERI-2196, VERI-2205,
        VERI-2228, VERI-2247, VERI-2266, VERI-2268, VERI-2269, VERI-2326,
        VERI-2367, VERI-2369, VERI-2386, VERI-2489, VERI-2524, VERI-2540,
        VERI-2558, VERI-2575, VERI-2607, VERI-2617, VERI-2622, VERI-2623,
        VERI-2624, VERI-2626, VERI-2628, VERI-2630, VERI-2631, VERI-2635,
        VERI-2640, VERI-2660, VERI-2662, VERI-2669, VERI-2672, VERI-2685,
        VERI-2695, VERI-2705, VERI-2706, VERI-2777, VERI-2779, VERI-2780,
        VERI-2783, VERI-2802, VERI-2810, VERI-2823, VERI-2839, VERI-2840,
        VERI-2841, VERI-2842, VERI-2843, VERI-2851, VERI-2860, VERI-2912,
        VERI-8002, VERI-8014, VERI-8021, VERI-8029, VERI-9022, VERI-9024,
        VERI-9087, VERI-9099, VERI-9104, VHDL-1160, VHDL-1454, VHDL-1591,
        VHDL-1669, VHDL-1704, VHDL-1769, VHDL-8010.

      NOTE: Use the following command to print the body of specified
      messages with placeholders for relevant details:

      get_message -text <VERI-id>

      NOTE: Some apps might remove messages from the resilient relaxed list.
      Check the app help to determine the behavior with relaxed mode.

  . all: Set this mode to enable all available modes.
      NOTE: Enabling finer levels of granularity does not disable the
      automatic black boxing mode. That means the tool applies the finer
      approach wherever possible, but it falls back to the "module" mode
      when it cannot apply other modes.

NOTE:
. The original driver of signals that are targets of discarded assignments
  ("always_process_block" and "continuous_assignment" modes) is automatically
  replaced by a free value. If the error is on the left-hand side of the
  assignment, the tool does not synthesize it.
```

```
. Syntax errors during analysis are not supported by the
  "always_process_block", "continuous_assignment" and "property" modes.
. You cannot use this command after design elaboration.
```

Example

```
# In the following example, resilient compilation is enabled,
# then the default mode is changed.
set_resilient_compilation on
set_resilient_compilation_mode always_process_block
```

See Also

set_resilient_compilation

set_restore_resolve_file_path

Specifies the method for resolving paths for "restore -elaborated_design".

Syntax

```
set_restore_resolve_file_path (absolute | relative)
```

Returns

No return values

Default

absolute

Detailed Description

Use this command to specify how "restore -elaborated_design" resolves file paths.

absolute
    Resolve file paths as if the "restore" command took place in the same directory
    in which the files were first analyzed.
    This option is the default.

relative
    Resolve file paths with respect to the current directory. That is, use the
    relative path of files as they appeared on the "analyze" command line.
    If the relative path does not point to an existing file, the tool uses the
    "absolute" path.

The file path information is used when displaying files in the Source Code Browser.

Example

```
set_restore_resolve_file_path relative
```

See Also

analyze,
restore

set_rtlds_auto_translate_rtl_to_gate_level_signals

Enables or disables automatic translation of signals from RTL to gate-level.

Syntax

```
set_rtlds_auto_translate_rtl_to_gate_level_signals (on | off)
```

Returns

No return values

Default

on

Detailed Description

Use this command to enable or disable automatic translation of signals from RTL to gate-level. When enabled, the tool tries to translate signals used in the script to a gate-level signal naming.

on: Translate signals from RTL to gate-level. This is the default.
off: Do not translate signals from RTL to gate-level.

NOTE: This setting only applies when "set_cdc_gate_level_analysis_mode" is enabled.

Example

```
set_rtlds_auto_translate_rtl_to_gate_level_signals off
```

set_save_compress_database

Enables Jasper to compress generated databases.

Syntax

```
set_save_compress_database (on | off)
```

Returns

No return values

Default

off

Detailed Description

Use this command to control whether a session must compress the files generated when saving session databases. This feature reduces file sizes if possible.

NOTE:
. This feature is supported by "save -jdb" and "save -elaborated_design" only and does not affect "save -dir" or "save -script".
. In "save -elaborated_design" each file is compressed individually inside the output folder.

on
    Compresses databases.

off
    Does not compress databases.
    This option is default.

Example

```
set_save_compress_database on
```

See Also

save,
restore

set_save_restore_capture_source_lock_rtl

With Source Lock active, includes source-locked RTL in save -elaborated_design.

Syntax

```
set_save_restore_capture_source_lock_rtl (on | off)
```

Returns

No return values

Default

on

Detailed Description

Use this command to control if, in a session with Source Lock active, the RTL that was locked by Source Lock will be included in the package saved by the command "save -elaborated_design". All the files that are locked by Source Lock will be compressed along with the Source Lock database into a file in the folder specified in "save -elaborated_design".
If Source Lock is not active, this configuration does not apply.

NOTE:
. When restoring a package that included the RTL from Source Lock, this configuration must be active to restore the RTL and the Source Lock database. Otherwise, only the elaborated design is restored.
. The tool only includes files if this configuration command is on before you run the "analyze" command.
. Turning off this configuration after "elaborate" prevents "save -elaborated_design" from including the Source Lock files.

on
    Includes source-locked RTL in "save -elaborated_design".
    This option is the default.

off
    Always avoids including source-locked RTL in "save -elaborated_design".

Example

```
set_save_restore_capture_source_lock_rtl on
```

See Also

save,
source_lock

set_schematic_viewer_character_limit

Specifies the maximum number of characters in a label before it is truncated.

Syntax

```
set_schematic_viewer_character_limit N
```

Returns

No return values

Default

20

Detailed Description

Use this command to configure the maximum number of characters in Jasper Schematic Viewer labels. The tool automatically truncates labels by omitting those characters that exceed the limit. By default, the tool truncates all labels longer than 20 characters.

NOTE: This option affects only schematic windows that open after you run this command.

0
    Show all characters in all labels.

N
    Automatically truncate labels with more than N characters.

Example

```
set_schematic_viewer_character_limit 0
```

set_schematic_viewer_collapse_combo_logic

Specifies if combinational logic should be collapsed into clouds.

Syntax

```
set_schematic_viewer_collapse_combo_logic (true | false)
```

Returns

No return values

Default

false

Detailed Description

Use this command if you want to collapse all combinational logic between flops, primary inputs, and outputs into a cloud for ease of visualization.

NOTE: This option affects only schematic windows that are opened after the execution of this command.

true
    Combinational logic is collapsed into clouds.

false
    No clouds will be shown. All gates are still present.

NOTE: "set_schematic_viewer_collapse_combo_logic" is included as an initial release to gather feedback from early adopters and finalize implementation for an upcoming release. Contact support@cadence.com to provide feedback or comments targeting the upcoming production version.

Example

```
set_schematic_viewer_collapse_combo_logic true
```

See Also

set_schematic_viewer_show_collapsed_logic_as_cloud
get_schematic_viewer_collapse_combo_logic

set_schematic_viewer_collapse_constants

Specifies whether collapsed constants are shown and their format.

Syntax

```
set_schematic_viewer_collapse_constants (bin | hex | false)
```

Returns

No return values

Default

bin

Detailed Description

Use this command to specify how the Jasper Schematic Viewer shows power and ground constants. By default, the Jasper Schematic Viewer collapses constants and displays them in binary format.

NOTE: This option affects only schematic windows that you open after you run this command.

bin
: Collapse constants and display them in binary format. This is the default setting.

hex
: Collapse constants and display them in hexadecimal format.

false
: Do not collapse ground and power symbols into constant labels.

Example

```
set_schematic_viewer_collapse_constants hex
```

set_schematic_viewer_info_pane_area

Specifies where in a schematic viewer window to show the information pane.

Syntax

```
set_schematic_viewer_info_pane_area (left | right | top | bottom)
```

Returns

No return values

Default

top

Detailed Description

Use this command to specify where in a schematic viewer its
information pane will be shown.

NOTE:
. The setting "schematic_viewer_show_info_pane" must be set to "true"
  in order for this setting to have any effect.
. This setting only applies to new schematic viewer windows.
. Not all schematic viewer windows have an information pane. If a
  window does not have an information pane, this setting has no effect
  on that window.

left
    Show the information pane to the left of the schematic

right
    Show the information pane to the right of the schematic

top
    Show the information pane on the top of the schematic

bottom
    Show the information pane below the schematic

Example

```
set_schematic_viewer_info_pane_area right
```

See Also

set_schematic_viewer_show_info_pane

set_schematic_viewer_show_collapsed_logic_as_cloud

Specifies whether collapsed combinational logic displays as a cloud or box.

Syntax

```
set_schematic_viewer_show_collapsed_logic_as_cloud (true | false)
```

Returns

No return values

Default

true

Detailed Description

Use this command to specify your preferred shape for collapsed combinational logic. By default, the Jasper Schematic Viewer displays collapsed combinational logic as a cloud.

NOTE:
. This option affects only schematic windows that you open after you run this command.
. This feature is an initial release to gather feedback from early adopters and finalize implementation for an upcoming release. Contact support@cadence.com to provide feedback or comments targeting the upcoming production version.

true
    Display collapsed combinational logic as a cloud. This is the default setting.

false
    Display collapsed combinational logic as a box.

Example

```
set_schematic_viewer_show_collapsed_logic_as_cloud false
```

See Also

set_schematic_viewer_collapse_combo_logic

set_schematic_viewer_show_function_nets

Specifies the schematic viewer display type.

Syntax

```
set_schematic_viewer_show_function_nets (box | flatten | false)
```

Returns

No return values

Default

flatten

Detailed Description

Use this command to specify the type schematic the tool displays when you run the command "schematic_viewer".

box
    Show functions and tasks as boxed entities on schematics.

flatten
    Show the full netlist of functions and tasks expanded on schematics. This is the default setting.

false
    Do not show function and task netlists on schematics.

Example

```
set_schematic_viewer_show_function_nets false
```

See Also

schematic_viewer

set_schematic_viewer_show_gate_cell_names

Specifies whether the gate cell names will be displayed.

Syntax

```
set_schematic_viewer_show_gate_cell_names (true | false)
```

Returns

No return values

Default

true

Detailed Description

Use this command to specify whether the Jasper Schematic Viewer displays gate cell names.
By default, the Jasper Schematic Viewer displays gate cell names.

NOTE:
. Gate cell names display beside the gate symbol.
. Register names are preferred over gate cell names.
  See "set_schematic_viewer_show_register_name" for more details.
. This option affects only schematic windows that you open after you run this command.

Example

```
set_schematic_viewer_show_gate_cell_names false
```

See Also

schematic_viewer,
set_schematic_viewer_show_register_name

set_schematic_viewer_show_info_pane

Specifies whether the schematic viewer information pane is displayed by default.

Syntax

```
set_schematic_viewer_show_info_pane (true | false)
```

Returns

No return values

Default

true

Detailed Description

Use this command to specify whether the information pane that is available in some schematic viewer windows will be displayed by default when the window is created.

NOTE:
. This setting only applies to new schematic viewer windows.
. Not all schematic viewer windows have an information pane. If a window does not have an information pane, this setting has no effect on that window.

true
    The pane will be shown (only applicable to schematic viewer windows that actually create an info pane).

false
    The pane will not be shown. (If the pane is available, you can still open it by invoking a menu item on the "View" drop down menu.)

Example

```
set_schematic_viewer_show_info_pane false
```

set_schematic_viewer_show_instance_unconnected_pins

Specifies whether the schematic shows unconnected instance pins.

Syntax

```
set_schematic_viewer_show_instance_unconnected_pins (true | false)
```

Returns

No return values

Default

true

Detailed Description

Use this command to specify whether the schematic shows unconnected instance pins. By default, the tool shows unconnected instance pins (true).

NOTE: This command affects only schematics opened after its execution.

Example

```
set_schematic_viewer_show_instance_unconnected_pins false
```

set_schematic_viewer_show_register_name

Specifies whether to show the visible net name below the flop or latch symbol.

Syntax

```
set_schematic_viewer_show_register_name (true | false)
```

Returns

No return values

Default

false (for all apps except CDC)

Detailed Description

Use this command to specify whether the schematics show the visible net names below the flop or latch symbol. By default, nothing is shown below the flop or latch symbol (false).

NOTE:
. This command affects only schematics opened after its execution.
. For the CDC App, the default for this variable is "true".

Example

```
set_schematic_viewer_show_register_name on
```

set_schematic_viewer_show_strength_information

Specifies whether strength information is displayed on the schematic.

Syntax

```
set_schematic_viewer_show_strength_information (true | false)
```

Returns

No return values

Default

true

Detailed Description

Use this command to specify whether the Schematic Viewer shows strength information in elements' tooltips.

The information might be available for gates and nets.

NOTE:
. This setting only applies to new Schematic Viewer windows.
. By default (that is, when this setting is "true"), the schematic always displays gate strength.
. The strength information for nets is only available if the Schematic Viewer window is opened with a Visualize window using the "-visualize_window" option.

Example

```
set_schematic_viewer_show_strength_information false
```

See Also

schematic_viewer

set_schematic_viewer_simplify_case_statements

Specifies whether case statements are simplified into a single gate.

Syntax

```
set_schematic_viewer_simplify_case_statements (true | false)
```

Returns

No return values

Default

true

Detailed Description

Use this command to specify how the Jasper Schematic Viewer draws case statements, in a simplified manner as a single gate or as a combination of comparisons and multiplexers. By default, the Jasper Schematic Viewer simplifies case statements.

NOTE: This option affects only schematic windows that you open after you run this command.

Example

```
set_schematic_viewer_simplify_case_statements false
```

set_schematic_viewer_simplify_concatenations

Specifies whether concatenations of multidimensional arrays should be collapsed.

Syntax

```
set_schematic_viewer_simplify_concatenations (true | false)
```

Returns

No return values

Default

true

Detailed Description

Use this command to specify how the Jasper Schematic Viewer draws multidimensional arrays, in a simplified manner as a single wire or as multiple ones.
By default, the Jasper Schematic Viewer simplifies multidimensional arrays.

NOTE: This option affects only schematic windows that you open after you run this command.

Example

```
set_schematic_viewer_simplify_concatenations false
```

set_sdc_vhdl_expr_mode

Enables VHDL format processing of SDC files.

Syntax

```
set_sdc_vhdl_expr_mode (on | off)
```

Returns

No return values

Default

off

Detailed Description

```
By default, SDC files are processed in Verilog format. Use this command to
interpret SDC file arguments in VHDL format. Example:
    design.vhd:
    ...
        d:   in std_logic_vector(1 downto 0);
    ...
    sdc.sdc:
        create_clock -period 1 {d(0) d(1)}

WARNING: To successfully process the SDC file in VHDL, enable this command
before issuing the "read_sdc" command.

LIMITATIONS:
 . VHDL is case insensitive, but the SDC files are processed as case sensitive.
 . Escaped names in VHDL style ( "\" + name + "\") are not supported in SDC
   files. Though you can have escaped names in the design file, you must refer
   to them in the SDC files without the escaped identifiers or using Verilog
   escaped identifiers ("\" + name + " "). Example:
        design.vhd:
        ...
            \d$_\:   in std_logic_vector(1 downto 0);
        ...
        sdc.sdc:
            create_clock -period 1 {d$_(0)}
            create_clock -period 1 {\d$_(1) }
```

Example

```
set_sdc_vhdl_expr_mode on
```

See Also

read_sdc

set_sec_auto_disable_violating_mappings

Specifies whether SEC sanity check automatically disables violating mappings.

Syntax

```
set_sec_auto_disable_violating_mappings (true | false)
```

Returns

No return values

Default

false

Detailed Description

Use this command to specify whether you want the Sequential Equivalence Checking (SEC) App to automatically disable mappings that violate sanity checking.

The violating reasons are of the following:
clock_signal, init_diff, init_mapping_ranges_diff, init_mapping_with_context, init_mapping_with_stopat, assumption_comb_coi, context_both_edge, dscp_on_constant, fast_rated_signal.

For more info about the violating reasons see "check_sec" help.

Example

```
set_sec_auto_disable_violating_mappings true
```

See Also

check_sec

set_sec_automap_aggregate_mappings

Specifies whether SEC auto map will aggregate the compound mappings.

Syntax

```
set_sec_automap_aggregate_mappings (true | false)
```

Returns

No return values

Default

false

Detailed Description

Use this command to specify that you want the SEC App auto map feature to map multiple signals related to the same compound signal (such as MD arrays, structs, and combinations of signal types) using a single mapping. When set to "true", the tool attempts to create a single mapping by aggregating all the signals that share the same root node.

For example, the outcome of the auto map command:
check_sec -map -auto -signal_type input

on the input compound:
input [3:0] in [1:0];

is:
{in[1:0] -> top_imp.in[1:0]}

Example

```
set_sec_automap_aggregate_mappings true
```

See Also

check_sec

set_sec_autoprove

Specifies the proof flow the SEC App uses, "autoprove" or "prove".

Syntax

```
set_sec_autoprove (true | false)
```

Returns

No return values

Default

true

Detailed Description

Use this command to specify the proof flow the SEC App uses, "autoprove" or "prove". If you use "set_sec_autoprove false", the command "check_sec -prove" runs a low-level "prove" command rather than the default "autoprove".

Example

```
set_sec_autoprove false
```

See Also

autoprove,
check_sec

set_sec_autoprove_expose_internal_cex

Shows identified counterexamples in the internal CEX task.

Syntax

```
set_sec_autoprove_expose_internal_cex (true | false)
```

Returns

No return values

Default

false

Detailed Description

Use this command to specify that you want SEC "autoprove" to show the counterexamples (CEXs) it found on internal signals. This command can be helpful for early debugging of failures. For example, if you do not expect to find a CEX on any internal signals, or if the outputs are not determined yet, this command can help you investigate the reason for the failures. The internal CEX traces are located in the "<SEC_work_task>_internal_cex" task, where <SEC_work_task> is the task from which your SEC proof was run.

NOTE: Prove commands, such as "check_sec -prove", cannot be run on the internal CEX task.

Example

```
set_sec_autoprove_expose_internal_cex true
```

See Also

check_sec,
set_sec_autoprove_minimal_max_length_of_internal_cex,
set_sec_autoprove_max_number_of_internal_cex

set_sec_autoprove_expose_status_progress

Specifies whether SEC displays a task for internal statuses.

Syntax

```
set_sec_autoprove_expose_status_progress (on | off)
```

Returns

No return values

Default

off

Detailed Description

Use this command to specify that SEC "autoprove" shows the status of internal properties created on storages. This command can be helpful for seeing the progress of the proof. For example, if the proof is running and you do not know how it is progressing, you can see which storages have a Cex, Proven, Conditionally Proven, or Unknown status.

NOTE:

. "<SEC_work_task>_status_progress" is the status progress task, where <SEC_work_task> is the task from which your SEC proof was run.

. This flow has limited functionality with parallel proofs. Only the first proof can generate a status progress task.
. Prove commands, such as "check_sec -prove", cannot be run on the status progress task.

Example

```
set_sec_autoprove_expose_status_progress on
```

See Also

check_sec

set_sec_autoprove_include_related_properties

Includes related verification tasks of the processed targets.

Syntax

```
set_sec_autoprove_include_related_properties (true | false)
```

Returns

No return values

Default

false

Detailed Description

By default, this setting is "false". And in that case, SEC AutoProve evaluates only the specified properties or the properties derived from the specified mapping IDs. This setting might provide reduced visibility into the actual status of the verification.

When this setting is "true", SEC AutoProve expands the proof process invoked by the following commands:
. check_sec -prove -property
. check_sec -prove -mapping_id
. check_sec -prove -mapping_type

The expanded proof process includes associated verification targets in the COI of specified targets, including:
. related covers of specified targets
. related covers of assumptions
. bbox_input assertions
. clock gating covers
. abstractions and their related covers

Example

```
set_sec_autoprove_include_related_properties true
```

See Also

check_sec

set_sec_autoprove_max_number_of_internal_cex

Sets the max number of counterexamples shown in the internal CEX task.

Syntax

```
set_sec_autoprove_max_number_of_internal_cex N
```

Returns

No return values

Default

100

Detailed Description

Use this command to define the maximal amount of counterexamples found on internal signals that can populate the internal_cex task per SEC proof.
The internal CEX traces are located in the "<SEC_work_task>_internal_cex" task, where <SEC_work_task> is the task from which your SEC proof was run.

Example

```
set_sec_autoprove_max_number_of_internal_cex 50
```

See Also

check_sec,
set_sec_autoprove_expose_internal_cex,
set_sec_autoprove_minimal_max_length_of_internal_cex

set_sec_autoprove_minimal_max_length_of_internal_cex

Sets the minimal max length of counterexamples in the internal CEX task.

Syntax

```
set_sec_autoprove_minimal_max_length_of_internal_cex N
```

Returns

No return values

Default

2

Detailed Description

Use this command to specify that you want SEC "autoprove" to exclude counterexample (CEX) traces with a max length smaller than N from the internal_cex task. This command can be helpful for early debugging of failures with the specified trace length.

The internal CEX traces are located in the "<SEC_work_task>_internal_cex" task, where <SEC_work_task> is the task from which your SEC proof was run.

Example

```
set_sec_autoprove_minimal_max_length_of_internal_cex 7
```

See Also

check_sec,
set_sec_autoprove_expose_internal_cex,
set_sec_autoprove_max_number_of_internal_cex

set_sec_autoprove_post_proof_activities_after_stop

Specifies whether to continue app-specific activity after stopping a proof.

Syntax

```
set_sec_autoprove_post_proof_activities_after_stop (on | off)
```

Returns

No return values

Default

off

Detailed Description

Some app logic runs after a proof completes and prolongs the proof time. Use this command to specify that you want the post-proof activities to continue if you manually stop the proof either with the GUI stop button or the "prove -stop" command.

Post-proof activities, which currently include only the SEC proof-cache-writing phase, extend the proof time. If you manually stop the proof, the app tries to interactively stop the proof as fast as possible and skips the post-proof activities.

Use the "set_sec_autoprove_post_proof_activities_after_stop on" command to signify the following:

. You do not want the tool to skip post-proof activities if you manually stop the proof.
. You understand that stopping the proof will take some time due to these activities, and you want to wait for them.

NOTE: If you run this command, you will not be able to interrupt cache writing.

Example

```
set_sec_autoprove_post_proof_activities_after_stop on
```

See Also

check_sec,
prove

set_sec_autoprove_skip_prequalification

Skip the SEC "autoprove" prequalification step.

Syntax

```
set_sec_autoprove_skip_prequalification (true | false)
```

Returns

No return values

Default

false

Detailed Description

Use this command to specify that you want the SEC App AutoProve to skip the prequalification step, the step in which the SEC App tries to prove the helper assertions that were automatically generated by the "autoprove" command.

Example

```
set_sec_autoprove_skip_prequalification true
```

See Also

check_sec

set_sec_autoprove_strategy

Configures SEC "autoprove" to use the specified proof strategy.

Syntax

```
set_sec_autoprove_strategy (basic | proof | clock_gating | state_matching)
```

Returns

No return values

Default

proof

Detailed Description

Use this command to globally specify the proof strategy for SEC "autoprove".

. Use "basic" to run basic "autoprove" without adding any cutpoints or helpers.
. Use "proof" to run "autoprove" with cutpoint refinement loop handling. This option is the default.
. Use "clock_gating" to tune the settings for "autoprove" to match designs with clock gating, where the gating is disabled for one side (spec/imp) and enabled for the other side.
. Use "state_matching" for any change in the RTL that does not change the assignment to the state holding variables (flops/latches).

Example

```
set_sec_autoprove_strategy basic
```

See Also

check_sec,
set_proofgrid_per_engine_max_local_jobs,
set_proofgrid_max_local_jobs

set_sec_break_loop_mode

Specifies how SEC handles breaking combo loops.

Syntax

```
set_sec_break_loop_mode (default | sync)
```

Returns

No return values

Default

default

Detailed Description

Use this command to specify how you want the tool to handle breaking loops under "set_loop_handling break".

default
    Breaking a loop using a specific signal tries to break its twin signal if it exists. This is the equivalent of adding a stopat on both sides.

sync
    Breaking a loop using a specific signal tries to break its twin signal if it exists, and they have the same value. This is the equivalent of adding a stopat on both sides and an assumption that both signals are equal.

NOTE: Changing the value of this configuration clears the proof results.

Example

```
set_sec_break_loop_mode sync
```

See Also

set_loop_handling

set_sec_disable_imp_assumption

Disables the imp side assumptions automatically.

Syntax

set_sec_disable_imp_assumption (none | assumptions_embedded)

Returns

No return values

Default

assumptions_embedded

Detailed Description

Use this command to specify which imp assumptions you want the SEC generate verification process to disable. The default is to disable all imp side embedded assumptions (assumptions_embedded) because they might lead to false positive results and/or degraded performance of the SEC proof strategies. However, you can avoid disabling imp side embedded assumptions automatically by running "set_sec_disable_imp_assumption none".

IMPORTANT: You must run this command before SEC setup.

Example

set_sec_disable_imp_assumption none

See Also

check_sec

set_sec_elaborate_mode

Configure elaboration to run with SEC configurations and modeling.

Syntax

```
set_sec_elaborate_mode (on | off)
```

Returns

No return values

Default

off

Detailed Description

Use this command to specify that the "elaborate" command runs with SEC configurations and modeling.

. on: Run "elaborate" with SEC configurations and modeling. The elaborated design is identical to that generated "check_sec -elaborate".
. off: Do not run "elaborate" with SEC configurations and modeling. This is the default.

Example

```
set_sec_elaborate_mode on
set_sec_elaborate_mode off
```

See Also

check_sec,
restore

set_sec_expose_non_app_related_properties

Specifies whether the Property Table and Task Tree include non-SEC properties.

Syntax

```
set_sec_expose_non_app_related_properties (true | false)
```

Returns

No return values

Default

false

Detailed Description

Use this command to specify whether the Property Table displays non-SEC-related properties and whether they affect the status bar of the Task Tree in the SEC GUI. Set the configuration to "false" to focus only on SEC-related properties by removing the distracting non-related properties.

Example

```
set_sec_expose_non_app_related_properties true
```

See Also

check_sec

set_sec_fractional_verification_mode

Specifies whether SEC fractional verification mode is enabled.

Syntax

```
set_sec_fractional_verification_mode (on | off)
```

Returns

No return values

Default

on

Detailed Description

Use this command to specify whether the SEC App avoids generating assertions that are outside a verification target's cone of influence. Intuitively, such properties are irrelevant to the proof process and therefore can be disregarded. The fractional verification flow takes place as part of the "check_sec -interface" command and works by disabling mappings between signals that are outside a verification target's cone of influence.

```
on:  Avoid generating assertions that are outside a target's cone of
     influence. This is the default behavior.
off: Generate assertions that are outside a target's cone of influence.
```

Example

```
set_sec_fractional_verification_mode on
```

See Also

check_sec

set_sec_hierarchical_blackbox_failing_instance

Controls black boxing during hierarchical verification.

Syntax

```
set_sec_hierarchical_blackbox_failing_instance (true | false)
```

Returns

No return values

Default

true

Detailed Description

Use this command to define whether hierarchical verification black boxes already verified instances even if their verification failed. The tool reports failing instances and considers them in the final verification result. If the value of this variable is "false", the tool includes failing instances for the verification at higher hierarchy levels. The script generated by "check_sec -setup -hierarchical" queries this value.

Example

```
set_sec_hierarchical_blackbox_failing_instance false
```

See Also

check_sec -setup -hierarchical

set_sec_hierarchical_check_assumption

Sets the sec_hierarchical_check_assumption variable.

Syntax

```
set_sec_hierarchical_check_assumption (true | false)
```

Returns

No return values

Default

false

Detailed Description

Use this command to include the assumptions_conflict properties in proofs. These properties are special properties built into every task to check the effects of assumptions.

NOTE: If the "sec_hierarchical_check_assumption" value is "true", the assumptions_conflict signoff category is not waived.

Example

```
set_sec_hierarchical_check_assumption true
```

set_sec_hierarchical_force_job

Sets the job results database update option value.

Syntax

```
set_sec_hierarchical_force_job (true | false)
```

Returns

No return values

Default

true

Detailed Description

Use this command to update the job results database.

Value	Description
true	Rerun the job for new results and update the database.
false	Use the result stored in the database without rerunning the job.

Example

```
set_sec_hierarchical_force_job false
```

set_sec_hierarchical_generate_script

Sets the value of the "sec_hierarchical_generate_script" variable.

Syntax

```
set_sec_hierarchical_generate_script (true | false)
```

Returns

No return values

Default

true

Detailed Description

Use this command to skip HSEC script generation when called with the "check_sec -setup -hierarchical" command.

Value	Description
true	Generate the HSEC script when called with the "check_sec -setup -hierarchical" command.
false	Skip HSEC script generation when called with the "check_sec -setup -hierarchical" command.

Example

```
set_sec_hierarchical_generate_script false
```

See Also

check_sec -setup -hierarchical

set_sec_hierarchical_match_port_number

Sets the value of the "sec_hierarchical_match_port_number" variable.

Syntax

```
set_sec_hierarchical_match_port_number (true | false)
```

Returns

No return values

Default

true

Detailed Description

Use this command to force top-level mapping regardless of whether the same number of ports exists.

Value	Description
true	Map instances with the same name only if the number of ports is the same.
false	Map instances with the same name even if the number of ports is not the same.

Example

```
set_sec_hierarchical_match_port_number false
```

See Also

check_sec -setup -hierarchical

set_sec_hierarchical_module_average_size

Sets the average size of the partition.

Syntax

```
set_sec_hierarchical_module_average_size N
```

Returns

No return values

Default

50000

Detailed Description

Use this command to specify the average size of the partition and ensure that the size of each partition complies with the given value.

NOTE: The size of the partition is a function of the number of storage elements and the number of nets.

Example

```
set_sec_hierarchical_module_average_size 10000
```

set_sec_hierarchical_module_excludelist

Sets a list of patterns for excluding modules for hierarchical verification.

Syntax

```
set_sec_hierarchical_module_excludelist pattern_list
```

Returns

No return values

Default

```
{}
```

Detailed Description

Use this command to specify the list of patterns used to match modules that will be excluded from partitioning during hierarchical verification setup. These patterns are in the style of Tcl's "string match" command. The SEC App uses the patterns during "check_sec -setup -hierarchical" to skip modules that should not be used for hierarchy decomposition.

Example

```
# Do not decompose hierarchy at modules that match the pattern *clkgate*
set_sec_hierarchical_module_excludelist { *clkgate* }
```

See Also

check_sec -setup -hierarchical

set_sec_hierarchical_module_includelist

Sets a list of patterns to match modules for hierarchy decomposition points.

Syntax

```
set_sec_hierarchical_module_includelist pattern_list
```

Returns

No return values

Default

```
{}
```

Detailed Description

Use this command to set the list of patterns used to match modules that will be included as hierarchy decomposition points during hierarchical verification setup. These patterns are in the style of Tcl's "string match" command. The SEC App uses the patterns during "check_sec -setup -hierarchical" to force modules that should be used for hierarchy decomposition, given that the respective instance names match.

Example

```
# Include modules matching pattern *datapath* in hierarchy decomposition
# (given that their instance names are matching):
set_sec_hierarchical_module_includelist { *datapath* }
```

See Also

check_sec -setup -hierarchical

set_sec_hierarchical_module_min_size

Sets the minimum size of the partition.

Syntax

```
set_sec_hierarchical_module_min_size N
```

Returns

No return values

Default

5000

Detailed Description

Use this command to specify the minimum size of the partition. If the partition size is smaller than the specified value, the tool merges the partition into its parent partition.

Example

```
set_sec_hierarchical_module_min_size 1000
```

set_sec_hierarchical_num_session

Sets the number of sessions used for hierarchical verification to N.

Syntax

```
set_sec_hierarchical_num_session N
```

Returns

No return values

Default

0

Detailed Description

Use this command to set the number of parallel sessions the tool uses for hierarchical verification. This value is queried by the script generated by "check_sec -setup -hierarchical".

Example

```
set_sec_hierarchical_num_session 4
```

See Also

check_sec -setup -hierarchical

set_sec_hierarchical_regular_expression

Sets a list of regular expressions for instance matching.

Syntax

```
set_sec_hierarchical_regular_expression pattern_list
```

Returns

No return values

Default

```
{}
```

Detailed Description

Use this command to specify the list of regular expression for instance matching in the HSEC flow when the spec and imp names are different. "sec_hierarchical_regular_expression" takes a list of spec and imp mapping pair regular expression.

Example

```
# Do not decompose the hierarchy at modules that match the pattern *clkgate*
set_sec_hierarchical_regular_expression { *clkgate* }
```

See Also

check_sec -setup -hierarchical

set_sec_hierarchical_report_incremental_status

Sets the value of the "sec_hierarchical_report_incremental_status" variable.

Syntax

```
set_sec_hierarchical_report_incremental_status (true | false)
```

Returns

No return values

Default

true

Detailed Description

Use this command to write status updates between verification steps and re-read when running scriptFile again.

Value	Description
true	Write status updates between verification steps and continue from the last step when running scriptFile again.
false	Do not write status updates between verification steps.

Example

```
set_sec_hierarchical_report_incremental_status false
```

See Also

check_sec -setup -hierarchical

set_sec_hierarchical_run_time_limit

Sets the restart time-out period for unresponsive worker sessions in HSEC.

Syntax

```
set_sec_hierarchical_run_time_limit 0
```

Returns

No return values

Default

0 (unlimited)

Detailed Description

Use this command to specify the restart time-out period for unresponsive worker sessions in HSEC. If the session does not update its log file within the specified period, the tool restarts it.

The tool handles this scenario as if the timeout had occurred inside the session itself; that is, it includes the error module at the parent level again.

The argument must be a value from "0" to "2147483647", inclusive.

Example

```
set_sec_hierarchical_run_time_limit 100
```

set_sec_hierarchical_save_partition_database

Drives database file generation for the partition process.

Syntax

```
set_sec_hierarchical_save_partition_database (none | failing | all)
```

Returns

No return values

Default

none

Detailed Description

Use this command to drive database file generation for the partitioning process.

Value	Description
none	The process does not generate any database files.
failing	Drives database file generation for failing partitions.
all	Drives database file generation for all partitions.

Example

```
set_sec_hierarchical_save_partition_database all
```

set_sec_hierarchical_signoff_waiver_list

Sets the sec_hierarchical_signoff_waiver_list variable.

Syntax

```
set_sec_hierarchical_signoff_waiver_list pattern_list
```

Returns

No return values

Default

```
{x_signals_and_undrivens assumptions_conflict mapped_stopats
assumptions_on_driven_signals}
```

Detailed Description

Use this command to specify the list of signoff waived categories. By default, the following categories are waived, that is, they are not included in the signoff report: "x_signals_and_undrivens", "assumptions_conflict", "mapped_stopats", "assumptions_on_driven_signals".

Example

```
set_sec_hierarchical_signoff_waiver_list {mapped_stopats assumptions_on_driven_signals}
```

set_sec_hierarchical_use_analyze_snapshot

Sets the value of the "sec_hierarchical_use_analyze_snapshot" variable.

Syntax

```
set_sec_hierarchical_use_analyze_snapshot (true | false)
```

Returns

No return values

Default

false

Detailed Description

Use this command to avoid running the "analyze" command when a previously loaded "analyze" snapshot is available.

Value	Description
true	Re-analyze designs for every module.
false	Do not re-analyze designs with analyzed modules. This is the default.

Example

```
set_sec_hierarchical_use_analyze_snapshot true
```

set_sec_mapping_max_word_size

Splits the created mapping entries into chunks of size up to N bits.

Syntax

```
set_sec_mapping_max_word_size N
```

Returns

No return values

Default

0

Detailed Description

Use "set_sec_mapping_max_word_size" to specify a maximum size for mapping entries created by the "check_sec -map" command. Mapping of vector signals or bit ranges that are wider than the specified value splits the entries into small "chunks" with a width up to "max_word_size" each. Specify the value 0 to use the full range of bits.

Example

```
set_sec_mapping_max_word_size 8
```

See Also

check_sec

set_sec_mpram_compound_port_mapping

Sets whether the tool maps MPRam ports as compounds.

Syntax

```
set_sec_mpram_compound_port_mapping (on | off)
```

Returns

No return values

Default

on

Detailed Description

Use this command to specify whether the tool maps MPRam ports as compounds. When this configuration is enabled, MPRam ports of the same type generate a unique mapping entry and, thus, a unique verification target or property. When this configuration is disabled, the tool generates one target or property per port.

Example

```
set_sec_mpram_compound_port_mapping off
```

set_sec_proofgrid_utilization_warning_percentage_threshold

Specifies Proofgrid utilization threshold in percentage for getting warnings

Syntax

```
set_sec_proofgrid_utilization_warning_percentage_threshold N
```

Returns

No return values

Default

70

Detailed Description

Use this command to specify the threshold in percentage for getting warnings when Proofgrid utilization is lower then the threshold.

NOTE: When setting the threshold to 0 no warnings will be shown.

Example

```
set_sec_proofgrid_utilization_warning_percentage_threshold 90
```

set_sec_return_mapping_dictionary_from_gui

Controls whether SEC GUI commands use mapping dictionaries or mapping IDs.

Syntax

```
set_sec_return_mapping_dictionary_from_gui (true | false)
```

Returns

No return values

Default

false

Detailed Description

Use this command to specify that you want SEC commands that are issued from the GUI to include mapping dictionaries instead of mapping IDs. A mapping dictionary is a list of mapping attributes that helps identify a particular list of mapping pairs. Use "help check_sec -gui" and see the section "Mapping Dictionary" for more info.

This command can be helpful if you want to include SEC commands issued by the GUI in your Tcl files. Without this setting, the commands issued by the GUI include mapping IDs, which change from session to session and make it difficult to re-use Tcl scripts. Instead, with this command, the GUI issues commands that include dictionaries that can be used to consistently identify mapping pairs across sessions.
Example:

. With this configuration set to "false", commands issued from the GUI include mapping IDs.
  check_sec -disable -mapping -id 1
. With this configuration set to "true", commands issued from the GUI include mapping dictionaries.
  check_sec -disable -mapping -dict {{spec_signal {out2} imp_signal {top_imp.out2} type {ver_target_primary_output} context {} map_to_constant {-1} is_inverse {false} is_global {true} tag {}}}

NOTE: "set_sec_return_mapping_dictionary_from_gui" is included as an initial release to gather feedback from early adopters and finalize implementation for an upcoming release. Contact support@cadence.com to provide feedback or comments.

Example

```
set_sec_return_mapping_dictionary_from_gui true
```

See Also

check_sec

set_sec_run_source_browser_diff

Specifies whether SEC highlights differences between Spec and Imp designs.

Syntax

```
set_sec_run_source_browser_diff (true | false)
```

Returns

No return values

Default

true

Detailed Description

This feature enables the SEC "Source Browser Diff" for highlighting
differences in the source code under the source browser tab.
It enables you to specifically highlight differences in target instances in
two different spec and imp files and provides a user-friendly interface for
visualizing and navigating these differences. This can help with early check
of RTL source code differences before investing in an expensive SEC proof.
For example, if the interface check failed and you need to know if
the root cause is changes in the design, you can quickly debug the highlighted
scopes in the design with source code changes.

Differences Highlights Navigation:
. Nodes Arrows: to navigate across next and previous differences in
  nodes/scopes boundaries.
. Differences Arrows: to navigate across text blocks differences.
. Highlighted source code differences synced with navigation arrows clicks.
. Manual instance selection in the instance tree synchronizes the navigation
  arrows data and source code highlights will be updated automatically to the
  current selected instance.
. By clicking on line number of source code that has differences, source code
  highlights are synchronized automatically on the second design source code.

Differences Highlights Color Types:
. green: Identical instances.
. red: Instances with difference in the source code.
. gray: Spec instance missing in Imp side.
. gray: Imp instance missing in Spec side.
. red with green frame: Identical instances with difference in the children
  scope.
. green with red frame: Identical instances with difference in parameters.

NOTE:
. You must use this setting before SEC setup.
. Differences in spaces and empty lines are ignored.
. Differences in comments are ignored.
. Blackboxed and encrypted modules are compared based only on instance parameters
  inputs.
. Diff is disabled when compile context is set to “both” (same design).
. VHDL is not currently supported.
. Hierarchical SEC (HSEC) flow is not supported.

Example

```
set_sec_run_source_browser_diff false
```

set_sec_shallow_elaboration

Specifies whether SEC is configured to run shallow elaboration.

Syntax

```
set_sec_shallow_elaboration (on | off)
```

Returns

No return values

Default

off

Detailed Description

Use this configuration before the elaboration in conjunction with the Hierarchical SEC flow. When enabled, the "check_sec -elaborate" command performs a partial elaboration that considers only the module hierarchy of the design.

Example

```
set_sec_shallow_elaboration on
```

See Also

check_sec

set_sec_show_bound_progress_in_log

Specifies whether SEC shows bound progress in the proof log.

Syntax

```
set_sec_show_bound_progress_in_log (on | off)
```

Returns

No return values

Default

off

Detailed Description

Use this command to specify whether you want SEC to show bound progress in the proof log.

Example

```
set_sec_show_bound_progress_in_log on
```

set_sec_show_proof_log

Specifies whether SEC shows the proof log.

Syntax

```
set_sec_show_proof_log (true | false)
```

Returns

No return values

Default

false

Detailed Description

Use this command to specify whether you want the Sequential Equivalence Checking (SEC) App to show the proof log.
NOTE: Proof logs for no_auto are always shown.

When setting this configuration to "true", the tool prints more information in log:
1. Accumlative ProofGrid utilization data for the all internal proofs
2. Utilization report for each internal prove:
    a. Prove runtime
    b. ProofGrid slot seconds - accumulative runtime of all engines in the prove
    c. Average parallel running jobs - average number of jobs ran in practice (slot seconds / prove runtime)
    d. Average max jobs limit - average jobs limit to the specific prove (actual limit is dynamic during the proof)
    e. Running to limit jobs ratio - the ratio between average parallel running jobs to average max jobs limit
    (Average parallel running jobs / Average max jobs limit)
    f. ProofGrid jobs runtime in percentage - the percentage of the engines' actual runtime since ProofGrid launched their requests

    NOTE: First, look at the ProofGrid jobs run time in percentage (#f above). If it is low, it might show environment issues. If it is high, look at Average parallel running jobs (#b above), and analyze whether this number makes sense compared to number of properties and number of jobs you have submitted to the proof.

Example

```
# Example for accumlative ProofGrid utilization data:
INFO (IPF180): 0: ProofGrid utilization: 99.05slot-seconds: 576.64

# Example for utilization report of internal prove:
INFO (ISEC134): Utilization: Action run time: 20 seconds.
INFO (ISEC134): Utilization: Proof grid slot seconds: 18.87.
INFO (ISEC134): Utilization: Average parallel running jobs: 10.94332.
INFO (ISEC134): Utilization: Average max jobs limit: 10.95.
INFO (ISEC134): Utilization: Running to limit jobs ratio: 99.93%.
INFO (ISEC134): Utilization: Proof grid jobs run time in percentage: 99.66.

Example for using the setting:
set_sec_show_proof_log on
```

set_sec_task_based_auto_mapping_on_setup

Enables generating task-based mappings for verification targets in SEC setup.

Syntax

```
set_sec_task_based_auto_mapping_on_setup (on | off)
```

Returns

No return values

Default

off

Detailed Description

Use this command to specify whether you want the SEC setup auto-mapping to generate task-based mapping for verification targets and toggle covers or global mappings.

. on: Generated auto-mappings in setup for verification targets and toggle covers are task-based.
. off: Generated auto-mappings in setup for verification targets and toggle covers are global. This is the default.

NOTE: This command must be called before running the setup.

Example

```
set_sec_task_based_auto_mapping_on_setup on
```

See Also

check_sec

set_sec_time_step_for_status_progress_log_entries

Specifies how frequently SEC status progress task log entries are printed.

Syntax

```
set_sec_time_step_for_status_progress_log_entries N
```

Returns

No return values

Default

10m

Detailed Description

Use this command to specify how frequently "SEC autoprove" prints a message with the current status of its status progress tasks. This command can be useful for easy detection of general proof progression and possible stagnation by a simple search in the proof log. The message alerts of possible stagnation if the task's status stayed the same between two consecutive messages.
Note the following notation: P: Proven. CP: Conditionally Proven. U: Unknown, UC: Unconfirmed Cex, C: Cex, S: Skipped.

Example

```
set_sec_time_step_for_status_progress_log_entries 20
```

set_sec_twin_debug_mode

Specifies whether debugging tools open with the SEC twin signal mode enabled.

Syntax

```
set_sec_twin_debug_mode (on | off)
```

Returns

No return values

Default

on

Detailed Description

Use this command to specify whether you want the Sequential Equivalence Checking (SEC) App to enable all twin debugging capabilities. For finer control over various twin debugging capabilities, use the "Sequential Equivalence Checking" toolbar in the Visualize window.

Example

```
set_sec_twin_debug_mode off
```

set_sec_use_uif_modeling

Sets whether UIF is used (on) or not (off) to model blackboxed operators.

Syntax

```
set_sec_use_uif_modeling
```

Returns

No return values

Default

on

Detailed Description

Use this command to set whether UIF is used to model blackboxed operators.

Using UIF (uninterpreted functions) avoids creating verification targets/properties for every blackbox input. Instead, the tool models blackbox outputs considering the inputs, and as long as they are equal between spec and imp designs, the output reflects it.

Currently, the following blackbox operators are supported:
- MPRam read ports: For these, the considered input is the memory read address.
- Complex arithmetic operators (multiplier, divider, power, modulo, remainder): For these binary operators, the considered inputs are both left and right operands.

NOTE: When UIF is enabled, SEC no longer treats the inputs of the handled blackboxed operators as blackbox inputs. Thus, "check_sec -interface" does not complain if these inputs are not directly mapped.

Example

```
set_sec_use_uif_modeling on
```

set_sequence_wait_timeout

Defines the lower boundary time out for $wait_until_convergence.

Syntax

```
set_sequence_wait_timeout cycles
```

Returns

No return values

Default

10000

Detailed Description

Use this command to configure the lower boundary time out for $wait_until_convergence in terms of cycles. $wait_until_convergence is a directive that specifies that a given set of signal values will remain unchanged until the reset convergence. And this configuration specifies the lower boundary of the maximum number of cycles the simulation runs to reach reset convergence. The value is adjusted so that the total number of cycles is the next multiple of clock factors, to respect clock configurations. The reference for a cycle is the tool's fastest clock.

Example

```
set_sequence_wait_timeout 1000
reset -sequence reset.seq
get_reset_info
```

See Also

reset

set_show_checker_on_hierarchy_tree

Displays SystemVerilog checkers on the Design Hierarchy Tree.

Syntax

```
set_show_checker_on_hierarchy_tree (on | off)
```

Returns

No return values

Default

on

Detailed Description

Use this command to enable displaying SystemVerilog checkers on the Design Hierarchy Tree displayed in the main Jasper window, Source Code Browser, and Visualize window.

Example

```
set_show_checker_on_hierarchy_tree on
```

set_show_disabled_property_count

Specifies whether the disabled property count displays in the GUI.

Syntax

```
set_show_disabled_property_count (on | off)
```

Returns

No return values

Default

on

Detailed Description

Use this command to enable displaying the disabled property count in:
. FPV Property Table counter widget (at the bottom of the table) and tooltips
. Design Hierarchy pane summarized properties icon for each instance
. Task Tree progress bar and tooltips

NOTE: You cannot use this command after design elaboration.

Example

```
set_show_disabled_property_count off
```

set_show_interface_on_hierarchy_tree

Displays SystemVerilog interfaces on the Design Hierarchy Tree.

Syntax

```
set_show_interface_on_hierarchy_tree (on | off)
```

Returns

No return values

Default

on

Detailed Description

Use this command to enable displaying SystemVerilog interfaces on the Design Hierarchy Tree displayed in the main Jasper window, Source Code Browser, and Visualize window.

Example

```
set_show_interface_on_hierarchy_tree on
```

set_show_proven_directive

Displays the proven properties assumed at the start of the current proof.

Syntax

```
set_show_proven_directive (on | off)
```

Returns

No return values

Default

off

Detailed Description

Use this command to display a full list of the proven properties assumed in the current proof. The tool displays the list as part of IPF036.

NOTE:

- Since the list can include thousands of properties, this command might significantly increase the size of the log file.
- The list does not include new proven properties that will be added dynamically during the proof.

Example

```
set_show_proven_directive on
```

set_simulation_max_loop_limit

Limits the times the simulation can loop over the same net without converging.

Syntax

```
set_simulation_max_loop_limit max_loop_limit
```

Returns

No return values

Default

1000

Detailed Description

Use this command to set the maximum number of times that the simulation can try to find a value for a net without converging before issuing an error.

Example

```
set_simulation_max_loop_limit 2000
```

set_single_property_liveness

Selects multi-/single- property mode for liveness properties.

Syntax

```
set_single_property_liveness (on | off)
```

Returns

No return values

Default

on

Detailed Description

Use this command to control whether multi-property engines prove all liveness properties within a task in parallel.

. off: Prove all liveness properties in parallel with all safety properties.
. on: Prove a single liveness property at a time.

NOTES:
. The engines that support this are Ht, Hp, H, and J. Other multi-property engines do not prove liveness properties in parallel.
. The value "off" modifies the behaviour described in "set_engine_mode".
. Potential advantages, and drawbacks, of "off" include:
  . Parallel proof is better at establishing cheap proof results for liveness properties fast when there are many properties in total, and sequential behavior is not controlled by "set_per_property_time_limit".
  . Parallel proof is better at sharing work during Trace and Proof attempts between multiple liveness properties, reducing overall CPU consumption for the same work.
  . Get incidental :live cover results. See "check_assumptions".
  . In a mixed safety and liveness property proof, there is slower attempt progress for the safety properties. The liveness properties are also handled in parallel.
  . There is startup cost that might be noticeable when there are many liveness properties and the design is big.

Example

```
set_single_property_liveness off
```

See Also

, set_engine_mode

set_spv_autoprove_basic_engine_mode

Sets the AutoProve engine mode for the SPV App when using "basic" strategy.

Syntax

```
set_spv_autoprove_basic_engine_mode
    (  B | B4 | Bm | C | C2 | D | EC | G | G2 | H | Hp | Hps | Ht | Hts | I | J
     | K | L | M | N | AB | AD | AG | AM | Oh | Q3 | R | Tri | U | TM | QT)+
```

Returns

No return values

Default

N AM EC Hp Ht Tri B L

Detailed Description

Use this command to set the engines run by the SPV "prove" command.

Example

```
set_spv_autoprove_basic_engine_mode {EC N Hp Ht Bm}
```

See Also

check_spv

set_spv_autoprove_expose_internal_cex

Specifies whether SPV displays a task for internal counterexamples.

Syntax

```
set_spv_autoprove_expose_internal_cex (on | off)
```

Returns

No return values

Default

false

Detailed Description

Use this command to specify that SPV "autoprove" shows the counterexamples found on internal signals. This command can be helpful for early debugging of failures. For example, if you did not expect to find a CEX on specific internal signals or targets are not yet determined, this command can help you investigate the reason for the failures.

The internal CEX traces are located in the "<SPV_work_task>_<SPV_target_property>_internal_cex" task, where <SPV_work_task> is the task from which your SPV proof was run and <SPV_target_property> is the property sent to the SPV proof.

NOTE:
. If multiple properties are sent to the same proof, one is arbitrarily chosen as the <SPV_target_property>.
. This flow has limited functionality with parallel proofs. Only the first proof can generate a status progress task.
. Prove commands, such as "check_spv -prove", cannot be run on the internal CEX task.

Example

```
set_spv_autoprove_expose_internal_cex on
```

See Also

check_spv

set_spv_autoprove_expose_status_progress

Specifies whether SPV displays a task for internal statuses.

Syntax

```
set_spv_autoprove_expose_status_progress (on | off)
```

Returns

No return values

Default

off

Detailed Description

Use this command to specify that SPV "autoprove" shows the status of internal properties created on storages. This command can be helpful for seeing the progress of the proof. For example, if the proof is running and you do not know how it is progressing, you can see which storages have a Cex, Proven, Conditionally Proven, or Unknown status.

NOTE: "<SPV_work_task>_<SPV_target_property>_status_progress" is the status progress task, where <SPV_work_task> is the task from which your SPV proof was run and <SPV_target_property> is the property sent to the SPV proof.

NOTE:
. If multiple properties are sent to the same proof, one is arbitrarily chosen as the <SPV_target_property>.
. This flow has limited functionality with parallel proofs. Only the first proof can generate a status progress task.
. Prove commands, such as "check_spv -prove", cannot be run on the status progress task.

Example

```
set_spv_autoprove_expose_status_progress on
```

See Also

check_spv

set_spv_autoprove_max_number_of_internal_cex

Sets the max number of counterexamples shown in the internal CEX task.

Syntax

```
set_spv_autoprove_max_number_of_internal_cex N
```

Returns

No return values

Default

100

Detailed Description

Use this command to define the maximum number of counterexamples found on internal signals that can be reported in the SPV internal CEX task. The internal CEX traces are located in the "<SPV_work_task>_<SPV_target_property>_internal_cex" task, where <SPV_work_task> is the task from which your SPV proof was run and <SPV_target_property> is the property sent to the SPV proof.

Example

```
set_spv_autoprove_max_number_of_internal_cex 50
```

See Also

check_spv,
set_spv_autoprove_expose_internal_cex,

set_spv_autoprove_minimal_max_length_of_internal_cex

Sets the minimal max length of counterexamples in the internal CEX task.

Syntax

```
set_spv_autoprove_minimal_max_length_of_internal_cex N
```

Returns

No return values

Default

2

Detailed Description

Use this command to specify that you want SPV "autoprove" to exclude counterexample (CEX) traces with a max length smaller than N from the "internal_cex" task. This command can be helpful for early debugging of failures with the specified trace length.

The internal CEX traces are located in the "<SPV_work_task>_internal_cex" task, where <SPV_work_task> is the task from which your SPV proof was run.

Example

```
set_spv_autoprove_minimal_max_length_of_internal_cex 7
```

See Also

check_spv,
set_spv_autoprove_expose_internal_cex,
set_spv_autoprove_max_number_of_internal_cex

set_spv_autoprove_skip_prequalification

Specifies whether the tool skips the SPV "autoprove" prequalification step.

Syntax

```
set_spv_autoprove_skip_prequalification (on | off)
```

Returns

No return values

Default

off

Detailed Description

Use this command to specify that you want the SPV App AutoProve to skip the prequalification step, the step in which the SPV App tries to prove the helper assertions that were automatically generated by the "autoprove" command.

Example

```
set_spv_autoprove_skip_prequalification on
```

See Also

check_sec

set_spv_break_loop_mode

Specifies how SPV handles breaking combo loops.

Syntax

```
set_spv_break_loop_mode (default | sync)
```

Returns

No return values

Default

default

Detailed Description

Use this command to specify how you want the tool to handle breaking loops under "set_loop_handling break".

default
    The break point is a taint source, similar to a "from" signal.

sync
    The break point is not a taint source, and it does not allow taint to pass through it. This is equivalent to adding a stopat on the signal.

NOTE: Changing the value of this configuration clears the proof results.

Example

```
set_spv_break_loop_mode sync
```

See Also

set_loop_handling

set_spv_create_combined_precondition

Specifies whether a combined precondition is created with "check_spv -create".

Syntax

```
set_spv_create_combined_precondition (true | false)
```

Returns

No return values

Default

true

Detailed Description

You can create SPV properties with From and To preconditions. If you specify both preconditions, the tool can automatically create a third precondition to specify that the To precondition be reached at the same cycle as the From precondition or later. SPV automatically generates this third precondition only if "set_spv_create_combined_precondition" is "true", and the From and To preconditions have been created.

Example

```
set_spv_create_combined_precondition false
```

See Also

check_spv

set_spv_graph_show_alternative_paths

Specifies whether the SPV graph colors alternative paths in light red.

Syntax

```
set_spv_graph_show_alternative_paths (on | off)
```

Returns

No return values

Default

off

Detailed Description

By default, the SPV graph of a trace colors only nodes relevant in the path shown in Visualize (Dark Red). Use this command to also color alternative paths existing in the trace (Light Red).

off
    SPV graph of a trace colors only nodes relevant in the path shown in Visualize (Dark Red).
on
    SPV graph also colors alternative paths (Light Red).

NOTE: With this variable set to "on", Graph coloring takes much more time to finish.

Example

```
set_spv_graph_show_alternative_paths on
```

See Also

check_spv

set_spv_include_control_logic

Specifies whether SPV properties analyze control logic during the proof.

Syntax

```
set_spv_include_control_logic (on | off)
```

Returns

No return values

Default

on

Detailed Description

By default, SPV properties consider all logic between source and destination nodes to check for data propagation. Use this command to include or exclude control logic from the data propagation analysis.

on
    SPV properties are created to also consider control logic.
    This option is the default.

off
    SPV properties are created to exclude control logic from the analysis.

NOTE:
. With this variable set to "off", SPV excludes the following:
  . Mux selectors
  . Latch enables
  . Flop clock or reset pins
  . Output pin of comparison gates (equal, not-equal, greater-than, less-than)
  . Shift or rotate control pins
  . Reduce gates outputs (reduce OR, NOR, AND, XOR, XNOR)
. This setting affects SPV properties only if it was disabled ("off") when SPV properties were created.

Example

```
analyze -sv $RTL_DIR/dut.sv
elaborate -top dut

clock clk
reset rst

set_spv_include_control_logic off
check_spv -create -from in -to out -name prop_not_through_control_logic

set_spv_include_control_logic on
set prop [check_spv -create -from in -to out -name prop_through_all_logic]

check_spv -prove -property $prop
```

See Also

check_spv

set_spv_keep_driving_logic

Specify whether SPV keeps driving logic of source signals from original design.

Syntax

```
set_spv_keep_driving_logic (on | off)
```

Returns

No return values

Default

off

Detailed Description

When injecting a taint, SPV discards the original driving logic of the source signals by default.

on
    SPV keeps the original driving logic of the source signals driven from the original design.

off
    SPV discards the original driving logic of the source signals.

NOTE:
. This setting causes justified values for the "-from" signal instead of random values and is mandatory if the property or the design depends on the "-from" signal value.
. Enabling this setting disables an optimization, so it might cause performance degradation.
. This setting affects SPV properties only if it was enabled ("on") when SPV properties were created.

Example

```
set_spv_keep_driving_logic on
```

See Also

check_spv

set_spv_proofgrid_utilization_warning_percentage_threshold

Specifies Proofgrid utilization threshold in percentage for getting warnings

Syntax

```
set_spv_proofgrid_utilization_warning_percentage_threshold N
```

Returns

No return values

Default

70

Detailed Description

Use this command to specify the threshold in percentage for getting warnings when Proofgrid utilization is lower then the threshold.

NOTE: When setting the threshold to 0 no warnings will be shown.

Example

```
set_spv_proofgrid_utilization_warning_percentage_threshold 90
```

set_spv_prove_single_thread

Specifies whether SPV properties will be proven in a single proof thread.

Syntax

```
set_spv_prove_single_thread (on | off | auto)
```

Returns

No return values

Default

auto

Detailed Description

When using the basic proof strategy, SPV proves all properties in a single proof thread when you send multiple SPV properties to the "prove" command.

. Use "auto" to let SPV automatically decide whether to use this feature. This is the default.
. Use "on" to enable this feature.
. Use "off" to disable this feature.

NOTE:
. Only the basic proof strategy supports this configuration.
. Properties with taint feedback loop enabled are not supported and will be skipped.

Example

```
set_spv_prove_single_thread off
```

See Also

check_spv

set_spv_show_bound_progress_in_log

Specifies whether SPV shows bound progress in the proof log.

Syntax

```
set_spv_show_bound_progress_in_log (on | off)
```

Returns

No return values

Default

off

Detailed Description

Use this command to specify whether you want SPV to show bound progress in the proof log.

Example

```
set_spv_show_bound_progress_in_log on
```

set_spv_show_proof_log

Specifies whether SPV shows the proof log.

Syntax

```
set_spv_show_proof_log (on | off)
```

Returns

No return values

Default

off

Detailed Description

Use this command to specify whether you want the SPV App to show the proof log.

When setting this configuration to "true", the tool prints more information in log:

1. Accumlative ProofGrid utilization data for the all internal proofs
2. Utilization report for each internal prove:
    a. Prove runtime
    b. ProofGrid slot seconds – accumulative runtime of all engines in the prove
    c. Average parallel running jobs – average number of jobs ran in practice (slot seconds / prove runtime)
    d. Average max jobs limit – average jobs limit to the specific prove (actual limit is dynamic during the proof)
    e. Running to limit jobs ratio – the ratio between average parallel running jobs to average max jobs limit
    (Average parallel running jobs / Average max jobs limit)
    f. ProofGrid jobs runtime in percentage – the percentage of the engines' actual runtime since ProofGrid launched their requests

    NOTE: First, look at the ProofGrid jobs run time in percentage (#f above). If it is low, it might show environment issues. If it is high, look at Average parallel running jobs (#b above), and analyze whether this number makes sense compared to number of properties and number of jobs you have submitted to the proof.

Example

```
# Example for accumlative ProofGrid utilization data:
INFO (IPF180): 0: ProofGrid utilization: 99.05slot-seconds: 576.64

# Example for utilization report of internal prove:
INFO (ISEC134): Utilization: Action run time: 20 seconds.
INFO (ISEC134): Utilization: Proof grid slot seconds: 18.87.
INFO (ISEC134): Utilization: Average parallel running jobs: 10.94332.
INFO (ISEC134): Utilization: Average max jobs limit: 10.95.
INFO (ISEC134): Utilization: Running to limit jobs ratio: 99.93%.
INFO (ISEC134): Utilization: Proof grid jobs run time in percentage: 99.66.

Example for using the setting:
set_spv_show_proof_log on
```

set_spv_taint_feedback_loop

Specifies whether SPV applies a taint barrier on the "-from" signal.

Syntax

```
set_spv_taint_feedback_loop (on | off)
```

Returns

No return values

Default

off

Detailed Description

By default, this command applies a taint barrier on the "-from" signal.
This prevents taint from looping back to the "-from" signal through the design.

on
    SPV enables taint to be reinjected if it arrives at the "-from" signal
    again. Use this option to enable taint feedback loops.

off
    SPV applies a taint barrier on the "-from" signal.

NOTE:
. This does not impact the concrete values or edges on the "-from" signal.
. See "keep_driving_logic".
. This setting affects SPV properties only if it was enabled ("on") when SPV
  properties were created.

Example

```
set_spv_taint_feedback_loop on
```

See Also

check_spv

set_spv_time_step_for_status_progress_log_entries

Specifies how frequently SPV status progress task log entries are printed.

Syntax

```
set_spv_time_step_for_status_progress_log_entries N
```

Returns

No return values

Default

10m

Detailed Description

Use this command to specify how frequently "SPV autoprove" prints a message with the current status of its status progress tasks. This command can be useful for easy detection of general proof progression and possible stagnation by a simple search in the proof log. The message alerts of possible stagnation if the task's status stayed the same between two consecutive messages.
Note the following notation: P: Proven. CP: Conditionally Proven. U: Unknown, UC: Unconfirmed Cex, C: Cex, S: Skipped.

Example

```
set_spv_time_step_for_status_progress_log_entries 20
```

set_src_inline_macro_expansion

Use this command to enable or disable the inline macro expansion feature.

Syntax

```
set_src_inline_macro_expansion (full | text | off)
```

Returns

No return values.

Default

full

Detailed Description

Use the inline macro expansion feature to expand macro calls inline in the Source Browser window. To expand a macro inline, hover over it and click the + icon next to it. Expanding a macro call replaces its text with the macro body with all formal arguments substituted with actual arguments. This setting is enabled by default. You can disable this feature by setting it to "off".

full
: The text shown is the first step of the macro expansion, that is, nested macros are not expanded. The identifiers in the expanded text are clickable and all design exploration operations, including Driver, Why, and Declaration, are supported. This mode requires calculating and storing extra debugging information compared to the "text" mode.

text
: The text shown is the last step of the macro expansion, that is, all nested macros are expanded. The identifiers in the expanded text are not clickable because less debugging information is calculated and stored in this mode. Use this mode to avoid potential runtime and peak memory increase for specific test cases.

off
: Disable the inline macro expansion feature.

Example

```
Consider the following RTL:
`define  A2  a
`define  A1 `A2
`define  A0 `A1

`define B b
`define SUM(x, y) (x + y)

assign result = `SUM(`A0 , `B);

set_src_inline_macro_expansion text

Expanding `SUM changes the line to (a and b are not clickable):
assign result = -(a + b);

set_src_inline_macro_expansion full

Expanding `SUM changes the line to :
assign result = -(`A0 + `B);

One possible sequence of subsequent expansions (a and b are clickable) :
assign result = -(-`A1 + `B);
assign result = -(--`A2 + `B);
assign result = -(---a + `B);
assign result = -(---a + -b);
```

set_src_loop_iteration_debug

Enables advanced for loop debugging features in the Source Pane.

Syntax

```
set_src_loop_iteration_debug (on | off)
```

Returns

No return values

Default

on

Detailed Description

When this setting is enabled and the result of a Why operation is inside a loop construct (procedural for loop, generate block, and so forth), the looping index value is annotated with the current loop iteration. In addition, this configuration enables enhanced manual index selection, which lets you select any loop iteration to debug, updating the annotations, tooltips and signal resolution when you drag them from the Source Pane to the waveform. The manual selection operation behaves as a regular design exploration operation, showing up as a new entry on the History Pane and letting you navigate through the past of manual index changes with the "Backward" and "Forward" arrows.

on: Enable the advanced for loop debugging features in the Source Pane.
     This is the default.
off: Do not enable the advanced for loop debugging features in the Source Pane.

NOTE:
. The setting "set_src_loop_iteration_debug" must be set before elaboration.
. Enabling this setting can impact the run-time performance of elaboration
  phase.

set_sst_default_trace_length

Sets the default SST trace length.

Syntax

```
set_sst_default_trace_length N
```

Returns

No return values

Default

15

Detailed Description

Use this command to set the default SST trace length. This length will be the length of the SST trace if you do not set the maximum or minimum length for Visualize.

Example

```
set_sst_default_trace_length 10
```

See Also

visualize

set_stable_name_in_case_assert

Specify whether the names for automatic case assertions are stable.

Syntax

```
set_stable_name_in_case_assert (on | off)
```

Returns

No return values

Default

on

Detailed Description

Use this command to control how the names for automatic case assertions are created.

on: Stable names for case assertions will be based on the case statement structure as shown below:

```
    test._automatic_priority_case_3CA27A
    test._automatic_unique_case_59E443
```

off: Names will be created based on the relative order of the case statements:

```
    test._automatic_priority_case_0
    test._automatic_unique_case_1
```

Example

```
set_stable_name_in_case_assert off
```

See Also

set_message -warning

set_superlint_add_automatic_task_assumption_bitblasted

Enables bitblasting of automatic task assumptions.

Syntax

```
set_superlint_add_automatic_task_assumption_bitblasted (true | false)
```

Returns

No return values

Default

false

Detailed Description

Use "superlint_add_automatic_task_assumption_bitblasted" to enable bitblasting of automatic task assumptions.
NOTE: For "superlint_add_automatic_task_assumption_bitblasted" to take effect, either "superlint_fsm_automatic_assumptions_type" or "superlint_signals_automatic_assumptions_type" must have a value other than "none".

Example

```
set_superlint_add_automatic_task_assumption_bitblasted true
```

See Also

check_superlint,
set_superlint_fsm_automatic_assumptions_type,
set_superlint_signals_automatic_assumptions_type

set_superlint_arithmetic_overflow_counter_filter

Enables filtering arithmetic overflow of counters checks.

Syntax

```
set_superlint_arithmetic_overflow_counter_filter (none | non_critical | all)
```

Returns

No return values

Default

non_critical

Detailed Description

Use "superlint_arithmetic_overflow_counter_filter" to enable filtering arithmetic overflow of non-critical-valued or all counters checks in Superlint.

Use "none" to disable filtering arithmetic overflow of any counters checks in Superlint.

Use "non_critical" to enable filtering arithmetic overflow of non-critical-valued counters checks in Superlint.

Use "all" to enable filtering arithmetic overflow of all types of counters checks in Superlint.

Example

```
set_superlint_arithmetic_overflow_counter_filter none
```

See Also

check_superlint

set_superlint_arithmetic_overflow_enable_exponentiation

Enables checking overflow on power operation.

Syntax

```
set_superlint_arithmetic_overflow_enable_exponentiation (true | false)
```

Returns

No return values

Default

false

Detailed Description

Use this command to enable the overflow checks on the power operations.

NOTES:
- The command "set_superlint_arithmetic_overflow_enable_exponentiation" must be used before elaboration to have any effect.
- To get the accurate results of power overflow check, disable power operator blackboxing by adding "-disable_auto_bbox" or "-bbox_pow" value switches to the "elaborate" command.

Example

```
set_superlint_arithmetic_overflow_enable_exponentiation true
```

See Also

check_superlint

set_superlint_arithmetic_overflow_enable_multiplication

Enables checking overflow on the expression of each multiplication operation.

Syntax

```
set_superlint_arithmetic_overflow_enable_multiplication (true | false)
```

Returns

No return values

Default

false

Detailed Description

By default, Superlint checks for both "statement" and "expression" arithmetic overflow as follows:

. "statement" - generates checks on the assignment
. "expression" - generates checks on each of the "+" and "-" operations

Use this command to enable the "expression" mode to generate checks on the multiplication operations as well.

Example

```
set_superlint_arithmetic_overflow_enable_multiplication on
```

See Also

check_superlint

set_superlint_conflicting_assertions_remove_design_constraints

Specifies whether ASR_IS_CNFL considers design constraints.

Syntax

```
set_superlint_conflicting_assertions_remove_design_constraints (true | false)
```

Returns

No return values

Default

true

Detailed Description

Use this command to specify whether ASR_IS_CNFL checks consider design constraints.

True
    Do not consider design constraints. This is the default.

False
    Consider design constraints.

Example

```
set_superlint_conflicting_assertions_remove_design_constraints false
```

set_superlint_deadcode_full_case_filter

Controls the generation of dead code checks in default of case.

Syntax

```
set_superlint_deadcode_full_case_filter (true | false)
```

Returns

No return values

Default

false

Detailed Description

Use this command to prevent generation of dead code checks for defaults of case if the case is already full (all cases specified).

Example

```
set_superlint_deadcode_full_case_filter true
```

See Also

check_superlint

set_superlint_deadcode_unified_loop

Controls the generation of dead code checks on rows inside FOR loops.

Syntax

```
set_superlint_deadcode_unified_loop (true | false)
```

Returns

No return values

Default

false

Detailed Description

Use this command to control the generation of dead code checks on rows inside FOR loops. When set to "false", the tool generates a dead code property for all possible values of the loop variables on each line. When set to "true", the tool generates only one dead code property for each line with OR expression of all possible values of the loop variables, and the property is proven if the row is reachable by at least one possible value of the loop variables.

Note: The variable "superlint_deadcode_unified_loop" must be set before elaboration.

Example

```
set_superlint_deadcode_unified_loop true
```

set_superlint_debug_unreachable

Enables computation of root causes for unreachable properties.

Syntax

```
set_superlint_debug_unreachable (true | false)
```

Returns

No return values

Default

false

Detailed Description

Use this command to specify whether Superlint enables the computation of root causes for unreachable properties after the "check_superlint -prove" command finishes.

true - Enables computation of root causes.
false - Disables computation of root causes. This is the default.

Example

```
set_superlint_debug_unreachable true
```

See Also

set_superlint_debug_unreachable_per_property_time_limit,
set_superlint_debug_unreachable_time_limit

set_superlint_debug_unreachable_per_property_time_limit

Specifies the time limit for per property root causes computation.

Syntax

```
set_superlint_debug_unreachable_per_property_time_limit time_limit
```

Returns

No return values

Default

5s

Detailed Description

Use this command to specify the per property time limit for root causes computation in the first iteration after the "check_superlint -prove" command.

Example

```
set_superlint_debug_unreachable_per_property_time_limit 1m
```

See Also

set_superlint_debug_unreachable,
set_superlint_debug_unreachable_time_limit

set_superlint_debug_unreachable_time_limit

Specifies the overall time limit for root causes computation.

Syntax

```
set_superlint_debug_unreachable_time_limit time_limit
```

Returns

No return values

Default

120m

Detailed Description

Use this command to specify the overall time limit for root causes computation after the "check_superlint -prove" command finishes.
The time limit is an integer ending in "s" for seconds, "m" for minutes, or "h" for hours. The default limit is "5s".

Example

```
set_superlint_debug_unreachable_time_limit 50m
```

See Also

set_superlint_debug_unreachable,
set_superlint_debug_unreachable_per_property_time_limit

set_superlint_enable_compile_messages

Configures the tool to package compilation messages as Superlint messages.

Syntax

```
set_superlint_enable_compile_messages (true | false)
```

Returns

No return values

Default

true

Detailed Description

Use this command to enable or disable the packaging of "analyze" and "elaborate" messages as Superlint checks. When the value is set to "true", the tool packages these messages as Superlint checks.

NOTE: "set_superlint_enable_compile_messages" is included as an initial release to gather feedback from early adopters and finalize implementation for an upcoming release. Contact support@cadence.com to provide feedback or comments.

Example

```
set_superlint_enable_compile_messages false
```

set_superlint_enable_hier_lint

Specifies whether Superlint enables the hiearchical flow in the LINT domain.

Syntax

```
set_superlint_enable_hier_lint (true | false)
```

Returns

No return values

Default

false

Detailed Description

Use this command to enable the hierarchical analysis flow for LINT checks in Superlint.

true - Superlint builds and saves the hierarchical database with the "save" command.
false - Superlint does not build or save a hierarchical database. This option is the default.

Example

```
set_superlint_enable_hier_lint true
```

set_superlint_enable_observability_tags

Enables the filtering of Superlint checks by observability expression.

Syntax

```
set_superlint_enable_observability_tags
    {(  ASG_AR_OVFL | EXP_AR_OVFL | ARY_IS_OOBI | BUS_IS_FLOT | BUS_IS_CONT
      | ASG_IS_XRCH | EXP_IS_DVBZ | SHF_IS_OVFL | SHF_IS_ABSO | VAR_IS_CTOL
      | all | none)+}
```

Returns

No return values

Default

all

Detailed Description

"set_superlint_enable_observability_tags" enables the filtering of specified checks by observability expression even if the checks have not been observed. When you specify one or more checks using the "superlint_enable_observability_tags" variable, the tool generates an observability expression for these checks, which is then observed to a next flop, latch, or primary output.

NOTE: Observability expressions are internal and available in formal only.

Example

```
set_superlint_enable_observability_tags {ASG_AR_OVFL EXP_AR_OVFL ARY_IS_OOBI}
```

See Also

check_superlint

set_superlint_enable_rtl_inline_waiver

Configures the tool to process lint pragmas.

Syntax

```
set_superlint_enable_rtl_inline_waiver (true | false)
```

Returns

No return values

Default

false

Detailed Description

Use this command to enable or disable the processing of RTL waiver pragmas for Superlint checks. When the value is set to "true", the tool processes and applies lint pragma waivers.

Example

```
set_superlint_enable_rtl_inline_waiver true
```

set_superlint_engine_mode

Controls the engine mode used by Superlint.

Syntax

```
set_superlint_engine_mode
        ( B | B4 | Bm | C | C2 | D | EC | G | G2 | H | Hp | Hps | Ht | Hts | I | J
        | K | L | M | N | AB | AD | AG | AM | Oh | Q3 | R | Tri | U | U2 | TM | QT)+
```

Returns

No return values

Default

Ht J N I L

Detailed Description

Use this command to set the engine mode for Superlint.

Example

```
set_superlint_engine_mode Tri
set_superlint_engine_mode {Ht Hp M}
```

See Also

check_superlint,
set_engine_mode,

set_superlint_engine_mode_fsm_deadlock_livelock

Controls the engine mode used by Superlint for FSM Deadlock/Livelock checks

Syntax

```
set_superlint_engine_mode_fsm_deadlock_livelock ( Tri | M)+
```

Returns

No return values

Default

```
Tri M
```

Detailed Description

Use this command to set the engine mode of Superlint for FSM Deadlock/Livelock checks (FSM_IS_DDLK/FSM_IS_LVLK)

Example

```
set_superlint_engine_mode_fsm_deadlock_livelock Tri
set_superlint_engine_mode_fsm_deadlock_livelock {Tri M}
```

See Also

check_superlint,
set_superlint_engine_mode,
set_engine_mode

set_superlint_extract_default_case_x_assignment_checks

Controls the generation of x assignemnt checks in default of case.

Syntax

```
set_superlint_extract_default_case_x_assignment_checks (true | false)
```

Returns

No return values

Default

true

Detailed Description

Use this command to prevent generation of x assignment checks for defaults of case.
true: Generate X assignment checks. This is the default.
false: Do not generate X assignment checks.

Example

```
set_superlint_extract_default_case_x_assignment_checks false
```

See Also

check_superlint

set_superlint_extract_exclude_fsm

Disables extraction of specified FSMs.

Syntax

```
set_superlint_extract_exclude_fsm signal_name+
```

Default

No default values

Detailed Description

To exclude an FSM from extraction, set the "superlint_extract_exclude_fsm" variable to a specific FSM signal name and extract.

Example

```
set_superlint_extract_exclude_fsm state
```

See Also

check_superlint

set_superlint_extract_low_level_fsm

Enables specific low-level FSM extraction.

Syntax

```
set_superlint_extract_low_level_fsm signal_name
```

Default

No default values

Detailed Description

FSM detection in the Superlint App uses a higher level of heuristics
than the FSM detection used in "get_design_info". This might cause some FSM POIs
to be filtered from your results on extraction, which triggers a warning message.
To extract a specific FSM from the list of filtered FSM POIs, set the
"superlint_extract_low_level_fsm" variable to a specific FSM signal name and re-
extract.

Example

```
set_superlint_extract_low_level_fsm state
```

See Also

check_superlint

set_superlint_filter_covered_reset

Specifies whether Superlint reports dead code properties covered during reset.

Syntax

```
set_superlint_filter_covered_reset (true | false)
```

Returns

No return values

Default

true

Detailed Description

Use this command to specify whether "check_superlint -extract" reports dead code properties that are covered during reset.

Example

```
set_superlint_filter_covered_reset false
```

See Also

check_superlint

set_superlint_fsm_automatic_assumptions_type

Specifies types of signals on which to generate automatic fairness assumptions.

Syntax

```
set_superlint_fsm_automatic_assumptions_type (  none | all | primary_inputs
                                              | internal_undrivens
                                              | bbox_outputs
                                              | cutpoints | coi)+
```

Returns

None

Default

none

Detailed Description

Specify types of signals on which Superlint generates automatic assumptions for the FSM tasks.

Example

```
set_superlint_fsm_automatic_assumptions_type all
set_superlint_fsm_automatic_assumptions_type {primary_inputs cutpoints}
```

See Also

check_superlint,
set_superlint_add_automatic_task_assumption_bitblasted,
set_superlint_signals_automatic_assumptions_type

set_superlint_fsm_ignore_self_transitions

Specifies whether the tool generates self-transitions checks.

Syntax

```
set_superlint_fsm_ignore_self_transitions (true | false)
```

Returns

No return values

Default

false

Detailed Description

Use this command to specify whether Superlint disables the generation of AUTO_FORMAL_FSM_REACHABILITY checks for self-state transitions.

true - Do not generate FSM reachability checks for self-state transitions.

false - Generate FSM reachability checks for self-state transitions. This is the default.

Example

```
set_superlint_fsm_ignore_self_transitions true
```

set_superlint_fsm_max_state

Configure maximum number of states in FSM.

Syntax

```
set_superlint_fsm_max_state n
```

Returns

No return values

Default

100

Detailed Description

Use this command to set the maximum limit on states allowed for FSM candidates in the FSM detection process. The tool stops FSM detection for candidates that have more states than the number specified.

Example

```
set_superlint_fsm_max_state 5
```

See Also

check_superlint

set_superlint_fsm_no_mtrn_max_state

Configure maximum number of states in FSM for extracting FSM_NO_MTRN check.

Syntax

```
set_superlint_fsm_no_mtrn_max_state n
```

Returns

No return values

Default

16

Detailed Description

Use this command to set the maximum limit on states allowed for FSM candidates in the extraction of FSM_NO_MTRN check. The tool stops FSM_NO_MTRN extraction for candidates that have more states than the number specified.

Example

```
set_superlint_fsm_no_mtrn_max_state 16
```

See Also

check_superlint

set_superlint_fsm_strategy_per_level_time_limit

Specifies the time limit for each hierarchical level in the FSM divide-and-conquer strategy.

Syntax

```
set_superlint_fsm_strategy_per_level_time_limit time_limit
```

Returns

No return values

Default

1h

Detailed Description

Use this command to specify the time limit allocated for each hierarchical level when using the FSM divide-and-conquer strategy with "check_superlint -prove -fsm".

The FSM divide-and-conquer strategy divides the design into hierarchical levels and adds stop-ats at instance boundaries to prove FSM deadlock and livelock properties. This setting controls how much time is allocated for proving properties at each level.

The time limit is an integer, ending with "s" for seconds, "m" for minutes, or "h" for hours. The default is "1h."

Example

```
set_superlint_fsm_strategy_per_level_time_limit 2h
check_superlint -prove -fsm
```

See Also

check_superlint,
set_superlint_fsm_strategy_prove_time_limit

set_superlint_fsm_strategy_prove_time_limit

Specifies the total time limit for the FSM divide-and-conquer strategy.

Syntax

```
set_superlint_fsm_strategy_prove_time_limit time_limit
```

Returns

No return values

Default

12h

Detailed Description

Use this command to specify the total time limit for the entire FSM divide-and-conquer strategy when using "check_superlint -prove -fsm".

The FSM divide-and-conquer strategy divides the design into hierarchical levels to prove FSM deadlock and livelock properties. This setting controls the overall time budget for the entire strategy across all hierarchical levels.

The time limit is an integer, ending with "s" for seconds, "m" for minutes, or "h" for hours. The default is "8h."

NOTE: This total time limit is distributed across all hierarchical levels. The time allocation for each level is controlled by "set_superlint_fsm_strategy_per_level_time_limit".

You can also override this setting by using the "-time_limit" switch with the "check_superlint -prove -fsm" command.

Example

```
set_superlint_fsm_strategy_prove_time_limit 8h
check_superlint -prove -fsm
```

See Also

check_superlint,
set_superlint_fsm_strategy_per_level_time_limit

set_superlint_leverage_formal

Enables the leverage of formal engines to refine results and reduce noise.

Syntax

```
set_superlint_leverage_formal (true | false)
```

Default

```
true
```

Detailed Description

Use this command to enable the leverage of formal engines, which can refine the results from lint engines to improve the quality of results and reduce noise.
Note: Enable this command before extracting checks and running the proof.

Example

```
set_superlint_leverage_formal true
```

set_superlint_limit_observable_boundary_signal

Sets the maximum number of boundary signals to be added to the Visualize window.

Syntax

```
set_superlint_limit_observable_boundary_signal N
```

Returns

No return values

Default

10

Detailed Description

Use this command to specify the maximum number of boundaries you want added to trace when debugging observable violations.

. Use "-1" to specify an unlimited number of boundary signals.
. Use "0" to disable observability-based debugging.
. Use any positive number to limit the number of boundary signals to the specified maximum.

Example

```
set_superlint_limit_observable_boundary_signal 5
```

See Also

check_superlint

set_superlint_loop_count_limit

Specifies the maximum number of loops for which Superlint extracts properties.

Syntax

```
set_superlint_loop_count_limit n
```

Returns

No return values

Default

10

Detailed Description

Specifies the maximum number of loops in the design for which Superlint extracts properties. When the value is 0, Superlint issues properties for all loops in the design.
Note: The proof status of previously extracted combo-loop checks will be invalidated.

Example

```
set_superlint_loop_count_limit 11
```

See Also

check_superlint,
check_loop

set_superlint_loop_filter_startpoints

Automatically filters startpoints not connected to loops in Superlint extract.

Syntax

```
set_superlint_loop_filter_startpoints (on | off)
```

Returns

No return values

Default

on

Detailed Description

Use this command to run the algorithm that removes startpoints that are not connected to any loops. This filtering occurs during the loop-detection preprocessing stage. Use this option to reduce memory consumption by the "check_superlint -extract" command. This option might add some overhead, so you should weigh the advantages of using it in an interactive flow.

on
    Activate the loop preprocessing startpoints filter in Superlint extract. This option is the default.
off
    Deactivate the loop preprocessing startpoints filter in Superlint extract.

Example

```
set_superlint_loop_filter_startpoints on
```

See Also

check_superlint -extract

set_superlint_machine_learning_flow

Enables the machine-learning (ML) flow for suggested waivers in the LINT domain.

Syntax

```
set_superlint_machine_learning_flow (true | false)
```

Returns

No return values

Default

false

Detailed Description

Use this command to enable the machine learning (ML) flow for ML-suggested waivers for LINT checks in Superlint.

Example

```
set_superlint_machine_learning_flow true
```

set_superlint_observability_local_expression_levels

Specifies the maximum number of levels searched for local observability gates.

Syntax

```
set_superlint_observability_local_expression_levels N
```

Returns

No return values

Default

2

Detailed Description

By default, the tool collects all the local observability gates until it reaches the second user-visible signal.

Use this command to extend the number of user-visible signal levels for which the tool attempts to collect local observability gates.

Example

```
set_superlint_observability_local_expression_levels 3
```

See Also

check_superlint

set_superlint_parallel_extraction

Controls the number of threads used during extraction.

Syntax

```
set_superlint_parallel_extraction N
```

Returns

No return values

Default

1

Detailed Description

Use this command to configure the number of threads used during extraction.
 .8 threads is a suggested value
 .0 removes the limit on the number of threads. Superlint uses as many threads as there are cores on the CPU

Example

```
set_superlint_parallel_extraction 8
```

See Also

check_superlint
get_superlint_parallel_extraction

set_superlint_parallel_synthesis

Specifies whether Superlint enables the parallel synthesis in the LINT domain.

Syntax

```
set_superlint_parallel_synthesis (true | false)
```

Returns

No return values

Default

false

Detailed Description

Use this command to enable the parallel synthesis/ elaboration for LINT checks in Superlint.
If parallel synthesis is "false", the tool ignores all other parallel synthesis configuration commands.

You can set the number of "jg_frontend" processes based on the design needs.
The default value for the number of "jg_frontend" processes is 6.
To change configuration settings for parallel synthesis, use the following FPV commands: "set_parallel_synthesis_num_process N" and "set_parallel_synthesis_partition_method".

Example

```
set_superlint_parallel_synthesis true
```

See Also

set_parallel_synthesis_dispatch_mode,
set_parallel_synthesis_num_process,
set_parallel_synthesis_partition_method

set_superlint_prove_keep_traces

Specifies whether "check_superlint -prove" keeps traces.

Syntax

```
set_superlint_prove_keep_traces (assert | all | none)
```

Returns

No return values

Default

assert

Detailed Description

Use this command to specify whether "check_superlint -prove" will generate traces for assertions only, for all properties, or for no properties. By default, "check_superlint -prove" generates traces for assertions only ("assert").

Example

```
set_superlint_prove_keep_traces none
```

See Also

autoprove,
check_superlint

set_superlint_prove_memory_limit

Sets a memory limit for engine C and G when used in "check_superlint -prove".

Syntax

```
set_superlint_prove_memory_limit memory_limit
```

Returns

No return values

Default

2048

Detailed Description

Use this command to set a memory limit (in megabytes) for engine data structures used during "check_superlint -prove".

Important things to know:
. In most cases, it is best to set the memory limit to a value greater than 100.
. Total memory consumption for a job will be greater than "superlint_prove_memory_limit", so consider setting a limit lower than the total amount of memory you want to use for a job, for example, 20-30 MB less.

Example

```
set_superlint_prove_memory_limit 500
```

See Also

autoprove,
check_superlint

set_superlint_prove_time_limit

Specifies the time limit for the "check_superlint -prove" command.

Syntax

```
set_superlint_prove_time_limit time_limit
```

Returns

None

Default

2h

Detailed Description

Use this command to specify the time limit for the "check_superlint -prove" command. The time limit is an integer ending in "s" for seconds, "m" for minutes, or "h" for hours. The default limit for the "check_superlint -prove" command is "2h".

Example

```
set_superlint_prove_time_limit 1h
```

See Also

autoprove,
check_superlint

set_superlint_report_negative_result_overflow

Controls reporting of overflow due to assigning negative results to variables.

Syntax

```
set_superlint_report_negative_result_overflow (true | false)
```

Returns

No return values

Default

false

Detailed Description

Use this command to control whether Superlint reports arithmetic overflow violations caused by assigning negative results to unsigned variables. For example, when this variable is enabled, the tool reports overflow for the following:
.u1[4:0] = u2[3:0] - u3[3:0] where u1, u2 and u3 are unsigned.
.u1[4:0] = s1[3:0] where u1 is unsigned and s1 is signed .

NOTE: The command "set_superlint_report_negative_result_overflow" must be used before elaboration to have any effect.

true - Superlint reports overflow that is a result of assigning a negative result to an unsigned variable.
false - Superlint does not report overflow that is a result of assigning a negative result to an unsigned variable. This is the default.

Example

```
set_superlint_report_negative_result_overflow true
```

See Also

check_superlint

set_superlint_report_same_drivers_multiple_assignment

Specifies whether to report a SIG_IS_MLAS property when drivers are the same.

Syntax

```
set_superlint_report_same_drivers_multiple_assignment (true | false)
```

Returns

No return values

Default

false

Detailed Description

When extracting properties of tag SIG_IS_MLAS, the tool can either discard multiple assignments by the same driver or consider them.
Set the "superlint_report_same_drivers_multiple_assignment" variable to "true" to consider same drivers when generating SIG_IS_MLAS properties.

. "true": Superlint considers same drivers when generating SIG_IS_MLAS properties.
. "false": Superlint does not consider same drivers when generating SIG_IS_MLAS properties. This is the default.

Example

```
set_superlint_report_same_drivers_multiple_assignment true
```

set_superlint_report_unsigned_casting

Controls reporting of overflow due to unsigned casting.

Syntax

```
set_superlint_report_unsigned_casting (true | false)
```

Returns

No return values

Default

true

Detailed Description

Use this command to control whether Superlint reports arithmetic overflow violations caused by unsigned casting done on signed variables.
For example, when this variable is enabled, the tool reports overflow for the following:
.u3 = $unsigned(s3) where u3 is a 3-bits unsigned register and s3 is a 3-bits signed register.

NOTE: The command "set_superlint_report_unsigned_casting" must be used before elaboration to have any effect.

true - Superlint reports overflow that is a result of unsigned casting done on signed variables.
false - Superlint does not report overflow that is a result of unsigned casting done on signed variables.

Example

```
set_superlint_report_unsigned_casting false
```

See Also

check_superlint

set_superlint_rule_profiler

Enables profiling for Superlint rules.

Syntax

```
set_superlint_rule_profiler (on | off)
```

Returns

none

Default

off

Detailed Description

When "superlint_rule_profiler" is enabled, Superlint measures the overall run time and memory use for each rule, module, or instance. The profiler is disabled by default.
NOTE: Enabling the "superlint_rule_profiler" command might negatively impact performance.

Example

```
set_superlint_rule_profiler on
```

set_superlint_signal_type

Enables signal extraction for the specified signal types only.

Syntax

```
set_superlint_signal_type {all | wire | flop | latch | input | output | custom}+
```

Returns

No return value

Default

```
flop output custom
```

Detailed Description

Use this command to specify which types of signals Superlint extracts when you enable one or more of the following tags: SIG_NO_TGRS, SIG_NO_TGFL, SIG_NO_TGST, SIG_IS_DLCK, or SIG_NO_TGRS.

Example

```
set_superlint_signal_type {wire input}
```

See Also

check_superlint

set_superlint_signals_automatic_assumptions_type

Specifies signal types for which to generate automatic fairness assumptions.

Syntax

```
set_superlint_signals_automatic_assumptions_type (  none | all | primary_inputs
                                                  | internal_undrivens
                                                  | bbox_outputs
                                                  | cutpoints)+
```

Returns

None

Default

none

Detailed Description

Specify types of signals on which Superlint generates automatic assumptions for deadlock signals tasks.

Example

```
set_superlint_signals_automatic_assumptions_type all
set_superlint_signals_automatic_assumptions_type {primary_inputs cutpoints}
```

See Also

check_superlint,
set_superlint_add_automatic_task_assumption_bitblasted,
set_superlint_fsm_automatic_assumptions_type

set_superlint_signals_customize

Enables signal extraction for the specified signals.

Syntax

```
set_superlint_signals_customize {signal_name_list}
```

Returns

None

Default

```
{}
```

Detailed Description

Use this command to add signals you want Superlint to extract. The selected signals are extracted in addition to all the other enabled signals.
NOTE: Each time you use this command, it replaces the signals previously provided with this command.

Example

```
set_superlint_signals_customize {id1 id2}
```

See Also

check_superlint

set_superlint_signals_deadlock_bitblasted

Controls the bit blasting of the deadlock signals properties in Superlint.

Syntax

```
set_superlint_signals_deadlock_bitblasted (true | false)
```

Returns

No return values

Default

true

Detailed Description

Use "superlint_signals_deadlock_bitblasted" to control the bit blasting of deadlock signals properties in Superlint.
. Use "-true" to enable bit blasting. This is the default.
. Use "-false" to disable bit blasting.

Example

```
set_superlint_signals_deadlock_bitblasted false
```

See Also

check_superlint

set_superlint_signals_deadlock_max_width

Sets the value of the variable "superlint_signals_deadlock_max_width".

Syntax

```
set_superlint_signals_deadlock_max_width N
```

Returns

No return values

Default

10

Detailed Description

Use this command to specify the maximum width for which signal deadlock checks are created when the "superlint_signals_deadlock_bitblasted" variable is "off".

Example

```
set_superlint_signals_deadlock_max_width 12
```

See Also

check_superlint,
set_superlint_signals_deadlock_bitblasted

set_superlint_signals_extraction_heuristic

Configures heuristic for extracting signal checks.

Syntax

```
set_superlint_signals_extraction_heuristic (all | auto)
```

Returns

No return values

Default

auto

Detailed Description

Use "set_superlint_signals_extraction_heuristic" to specify the heuristic Superlint uses during the extraction of signal checks.

. all - Extract signal checks for all wide signals.
. auto - Extract wide-signal checks on signals that are in the sensitivity list or have critical values.

Example

```
set_superlint_signals_extraction_heuristic all
```

See Also

check_superlint,
set_superlint_signals_filter_wide_signal

set_superlint_signals_filter_connected

Controls extraction of connected signals.

Syntax

```
set_superlint_signals_filter_connected (true | false)
```

Default

```
true
```

Detailed Description

Use this command to specify whether Superlint filters connected signals when signal checks are enabled:

. true - Connected signals are filtered and represented by one property only
. false - Connected signals are not filtered

Example

```
set_superlint_signals_filter_connected true
```

See Also

check_superlint

set_superlint_signals_filter_constant

Configures whether Superlint extracts checks on design constants.

Syntax

```
set_superlint_signals_filter_constant (true | false)
```

Returns

No return values

Default

true

Detailed Description

Use "set_superlint_signals_filter_constant" to specify whether Superlint extracts checks on design constants.
. Use "true" if you want Superlint to extract checks on design constants.
. Use "false" if you do not want Superlint to extract checks on design constants.

NOTE: This command considers design constants and not functional constants.

Example

```
set_superlint_signals_filter_constant false
```

See Also

check_superlint

set_superlint_signals_filter_wide_signal

Configures whether Superlint extracts checks on wide signals.

Syntax

```
set_superlint_signals_filter_wide_signal (on | off | auto)
```

Returns

No return values

Default

auto

Detailed Description

```
Use "set_superlint_signals_filter_wide_signal" to specify whether Superlint
extracts wide signals.
. on   - Consider single-bit signals only.
. off  - Consider wide signals for all signal checks.
. auto - Consider wide signals for SIG_AT_STCK checks only.
```

Example

```
set_superlint_signals_filter_wide_signal false
```

See Also

check_superlint,
set_superlint_signals_extraction_heuristic

set_superlint_signals_max_bits

Limits the number of bits on which to generate toggle or stuckat properties.

Syntax

```
set_superlint_signals_max_bits N
```

Returns

No return values

Default

1

Detailed Description

Use "superlint_signals_max_bits" to limit the number of bits on which the tool generates toggle or stuckat properties when "superlint_stuck_at_bitblasted" is set to true or on which the tool generates deadlock properties when "superlint_signal_deadlock_bitblasted" is set to true. Use a non-positive value to unlimit max bits.

Example

```
set_superlint_signals_max_bits 4
```

See Also

check_superlint,
set_superlint_stuck_at_bitblasted

set_superlint_stuck_at_bitblasted

Controls the bit blasting of the stuck-at properties in Superlint.

Syntax

```
set_superlint_stuck_at_bitblasted (true | false)
```

Returns

No return values

Default

false

Detailed Description

Use "superlint_stuck_at_bitblasted" to control the bit blasting of stuck-at properties in Superlint.

Example

```
set_superlint_stuck_at_bitblasted true
```

See Also

check_superlint

set_superlint_sva_max_depth

Sets the value of the variable "superlint_sva_max_depth".

Syntax

```
set_superlint_sva_max_depth N
```

Returns

No return values

Default

10

Detailed Description

Use this command to set the maximum depth (in terms of gate complexity) for SVA expressions.

NOTE: If properties are already extracted with unavailable SVA expressions, you must run "check_superlint -extract" to re-extract with new results and avoid duplication.

Example

```
set_superlint_sva_max_depth 13
```

See Also

check_superlint

set_superlint_sva_max_length

Sets the value of the variable "superlint_sva_max_length".

Syntax

```
set_superlint_sva_max_length N
```

Returns

No return values

Default

1000

Detailed Description

Use this command to specify the length (in terms of the number of characters allowed) for SVA property expressions.

NOTE: If properties are already extracted with unavailable SVA expressions, you must run "check_superlint -extract" to re-extract with new results and avoid duplication.

Example

```
set_superlint_sva_max_length 100
```

See Also

check_superlint

set_superlint_unified_deadcode_checks

Controls the generation of dead code checks on the same source line.

Syntax

```
set_superlint_unified_deadcode_checks (all | none | instance)
```

Returns

No return values

Default

all

Detailed Description

The setting controls unifying dead code checks on the same source line with different options as follows:

all: The tool generates only one dead code property for each line with OR expression of all possible values. The property is covered (reachable) if at least one value of the loop variables, instance, or function call can reach the row. The property is unreachable if all the values of the loop variables, instance, or function call cannot reach the row and it shows one violation. This is the default.

none: The tool generates a dead code property for every possible value of the loop variables for each line of code, for each line in each instance, and for each line in the function for all the calls.

instance: The tool generates only one dead code property for each line with OR expression of all possible values of the loop variables or function calls within the same instance.
The property is covered (reachable) if at least one value of the loop variables or function call within the same instance can reach the row. The property is unreachable if all the values of the loop variables or function call within the same instance cannot reach the row and it shows one violation.

Note: The variable "superlint_unified_deadcode_checks" must be set before elaboration.

Example

```
set_superlint_unified_deadcode_checks none
```

set_superlint_verbosity_level

Configures verbosity level of Superlint extraction.

Syntax

```
set_superlint_verbosity_level (0 | 1 | 2)
```

Returns

No return values

Default

1

Detailed Description

Use this command to specify the verbosity level during extraction. Options include the following:

. 0 - No additional messages are printed
. 1 - Default verbosity level. Prints messages on starting new stage during extract
. 2 - Prints message for each property added

Example

```
set_superlint_verbosity_level 0
```

See Also

check_superlint

set_superlint_waiver_export_version

Includes tool version in the header of the exported Superlint waivers file.

Syntax

```
set_superlint_waiver_export_version (true | false)
```

Returns

No return values

Default

false

Detailed Description

Use "set_superlint_waiver_export_version true" to include tool version information in the header of the file specified with the "check_superlint -waiver -export" command. By default, the header does not include the tool version.

Example

```
set_superlint_waiver_export_version true
```

See Also

check_superlint

set_tag_irrelevant_values

Tags irrelevant values in the Visualize window as X.

Syntax

```
set_tag_irrelevant_values (true | false)
```

Returns

No return values

Default

false

Detailed Description

Use this command to set the mode for tagging irrelevant values in the Visualize window, showing irrelevant parts of the trace as X. This command applies to all future traces during your session.

Tagging irrelevant values provides additional debugging information, allowing you to focus on the signals that actually affect the analysis result. To use this feature, you must first turn on the irrelevant values computation with "set_trace_optimization -irrelevant_value_computation true".

true
   The tool shows inputs and other signals irrelevant to the related assumptions and targets as X.

false
   The tool assigns the value 0 to inputs that are irrelevant to the related assumptions and targets so that there are no X values in the trace.

Example

```
set_tag_irrelevant_values true
```

See Also

set_trace_optimization

set_task_compile_time_limit

Sets the per-task time limit for compiling properties during elaboration.

Syntax

```
set_task_compile_time_limit time_limit
```

Returns

No return values

Default

120s

Detailed Description

Use this command to set the per-task time limit for compiling properties during synthesis. This process runs during elaboration. The time limit is an integer ending in "s" for seconds, "m" for minutes, or "h" for hours.

time_limit
    Use the specified limit for each "elaborate" command.

    NOTE: Jasper Apps generally recognizes "0s" as "unlimited". In the case of this command, "0s" sets the highest supported limit, which is 10000 seconds.

Example

```
set_task_compile_time_limit 45s
```

See Also

set_elab_time_limit,
set_property_compile_time_limit

set_tcl_auto_show_error_info

Specifies whether Jasper outputs the contents of the "errorInfo" variable.

Syntax

```
set_tcl_auto_show_error_info (on | off)
```

Returns

No return values

Default

off

Detailed Description

Use this command to tell Jasper whether to output the contents of the "errorInfo" Tcl variable instead of the error result when a Tcl script exits with an error. The value of this variable contains a stack trace showing the nested Tcl commands that had been invoked at the time of the error.

Example

```
set_tcl_auto_show_error_info on
```

set_tcl_no_precondition

Specifies whether Jasper generates precondition covers for liveness assertions.

Syntax

```
set_tcl_no_precondition (on | off)
```

Returns

No return values

Default

off

Detailed Description

By default, the tool automatically generates precondition covers for liveness assertions you create with the "assert" command. Use "set_tcl_no_precondition on" if you do not want automatically generated covers.

Example

```
set_tcl_no_precondition on
```

set_tcl_related_covers

Enables or disables the precondition check for Tcl properties.

Syntax

```
set_tcl_related_covers (on | off)
```

Returns

No return values

Default

off

Detailed Description

Use this command to specify that you want Jasper Apps to create precondition covers for properties that you create with Tcl commands.

Example

```
set_tcl_related_covers on
```

set_terminal_binary_path

Path to the terminal that will be opened with GDB.

Syntax

```
set_terminal_binary_path "path"
```

Returns

No return values

Default

gdb

Detailed Description

Use this command to provide the path to the terminal that the "cfe_debug" command uses to open GDB.

Example

```
set_terminal_binary_path /bin/usr/xterm
```

See Also

cfe_debug

set_time_format

Configures the time format in messages.

Syntax

```
set_time_format (second | minute | hour)
```

Returns

No return values

Default

second

Detailed Description

Use this command to change the time format in messages to display in seconds, minutes, or hours.

Example

```
set_time_format second
```

set_trace_extension

Extends a trace with N extra cycles beyond the cycle that triggers a property.

Syntax

```
set_trace_extension N
```

Default

```
0
```

Detailed Description

Use this command to extend a trace by a specified number of cycles that continue to satisfy assumptions.

Important things to know:

. Trace extensions apply only to finite (non-looping) traces; therefore, this command has no effect on traces for liveness or infinite properties.

. A trace extension is part of the global property environment (just like clock and reset), and it will alter the meaning of your properties. When you change the trace extension, the tool clears proof results that are no longer guaranteed. Running a new proof after extending the trace can change the property status from cex or covered to proven or unreachable.

. A trace extension forces the proof engines to look for longer traces, which will most likely slow them down whether a trace exists or not.

. "$" is a special value that means infinite. If you use "$", you will get looping traces for safety or finite properties.

. The extension will be visible in the waveforms as additional cycles in which all assumptions are respected, but the property "signal" itself can have arbitrary behavior.

. Trace extension is not supported by the C2RTL App, SEC App, SPV App, or X-Prop App.

Usually, the full trace length and the cycle in which a property is triggered coincide. However, when you use this command to extend the trace, that is no longer the case. The following list describes how the distinction manifests in the tool:

. The property bounds, as seen in the "Bound" column in the main window property table, and as accessible using "get_property_info -list {min_length max_length}", will always refer to the cycle triggering the property, that is, the length excluding any trace extension. This means that as long as we do not suffer from non-extendable "dead-end" traces, the "bound" numbers will be comparable regardless of extension.

. The "Trace Attempts" and the trace length will refer to the full length including any trace extension.
  NOTE:
  . You can see "Trace Attempts" in the ProofGrid Manager window and the "Proof Messages" tab.
  . You can see the trace length with "get_property_info -list trace_length" or "visualize -get_length".

. When using engine L, you should consider adding the trace extension to the "engineL_tail_length" (see "set_engineL_tail_length"). Doing so instructs engine L to extract new start states at (or before) the actual property violation rather than a few cycles after.

set_trace_optimization

Configures optimization in trace computation.

Syntax

```
set_trace_optimization (ar | coi | standard)
set_trace_optimization -irrelevant_value_computation (true | false)
```

Returns

No return values

Default

ar and false

Detailed Description

Use this command to change the optimization in trace computations.

Things you should know about usage:

- In addition to their impact on performance, trace optimization
  settings are reflected in saved trace files:
  visualize -save (-vcd |-fsdb).
  For example,
  . If trace optimization is "ar", then only the analysis region is
    saved into the trace file. (This is the default setting.)

  . If trace optimization is "standard", then the entire design is
    dumped into the trace file. However, you should be aware that this
    setting can increase the time it takes to generate a trace.
- To tag irrelevant values, that is, show signal values that do not
  affect the analysis result as don't cares with x or X, you must
  first turn on the irrelevant values computation with
  "set_trace_optimization -irrelevant_value_computation true". (Tagging

```
  provides additional debugging information, allowing you to focus
  on the signals that actually affect the analysis result.)

ar | coi | standard
    Change trace optimization as specified.
    . Use "ar" to use the analysis region for optimizations.

      If tunnel mode is off, this is the same as "coi". This setting is the
      default option and is faster than the standard setting because,
      initially, the tool simulates only the analysis region. As
      you add logic from outside the AR, the tool simulates it automatically.

    . Use "coi" to use the cone of influence for optimizations.

      This setting is faster than the standard setting, but the values of the
      signals outside the cone of influence will be shown as Xs.

    . Use "standard" to use the standard set of optimizations for the full
      design.

      This setting is intended for full design exploration mode, with good time
      and memory performance in trace generation, Why computations, and so
      forth.

    NOTE:
      The following apps do not honor "set_trace_optimization" in this flow:
      SEC, SPV, X-Prop, C2RTL, and HLS.

-irrelevant_value_computation (true | false)
    Turn on or off the trace computation to detect irrelevant values.
    . true:  Turn on trace computation to detect irrelevant values.
    . false: Turn off trace computation to detect irrelevant values. This is the
      default.

    NOTE: The "set_tag_irrelevant_values" feature uses the computation result.
```

Example

```
set_trace_optimization standard
```

See Also

set_tag_irrelevant_values, visualize

set_trace_show_reset

Enables the tool to prefix traces with the reset sequence.

Syntax

```
set_trace_show_reset (true | false)
```

Returns

No return values

Default

true

Detailed Description

Use the "set_trace_show_reset" command to disable or re-enable the feature that makes it possible to display the reset sequence as a prefix in the Visualize window.

Important things to know about this command:
. By default, the reset prefix feature is enabled.
. The tool only displays the register values of a trace file specified with "reset -vcd", "reset -fsdb", "reset -shm", or "reset -vwdb" and only if you do not use "reset -expression" or "reset -sequence".
. If you use "reset -expression" or "reset -sequence" with other "reset" options (-init_state, -vcd, -fsdb, -non_resettable_regs) or "abstract -init_value", it is possible to have inconsistencies in the trace prefix (the portion that shows the reset values). This situation occurs because the trace prefix shows the results of the reset simulation (-expression or -sequence), and the tool applies value changes the other commands impose after the reset simulation. Therefore, the Visualize window uses the final reset values imposed by other commands beginning at cycle 1.
. By default, the Visualize window does not display the reset sequence. To display the prefix, use the menu View > Show Reset Cycles or the "visualize -set_show_reset_cycles on" command.
. To accelerate run times for reset sequence simulation, use "set_trace_show_reset false".
. When "trace_show_reset" is set to "false", flows that use incremental reset analysis error out, displaying "Incremental reset analysis works only when 'trace_show_reset' is 'true'".

Example

```
analyze -verilog my_design.v
elaborate -top my_design
clock clk
reset {resetN==1'b0}
prove -property {<embedded>::my_property}
visualize -violation -property {<embedded>::my_property}
// Use the View menu to display the reset sequence.
```

set_unr_break_clock_loop

Applies stopats on clock loops for the UNR flow.

Syntax

```
set_unr_break_clock_loop (on | off)
```

Returns

No return values

Default

on

Detailed Description

By default, the UNR App applies stopats on clock loops. Use this command to avoid applying stopats on clock loops or to re-enable the default behavior.

Example

```
set_unr_break_clock_loop off
```

See Also

check_unr

set_unr_functional_coverage

Enable functional coverage for the UNR flow.

Syntax

```
set_unr_functional_coverage (true | false)
```

Returns

No return values

Default

false

Detailed Description

By default, the UNR App does not enable functional coverage. Use this command to enable functional coverage for the UNR analysis or to restore the default behavior.

Example

```
set_unr_functional_coverage true
```

See Also

check_unr

set_unr_functional_coverage_mode

Select mode of functional coverage for the UNR flow.

Syntax

```
set_unr_functional_coverage_mode (assertion |covergroup |all)
```

Returns

No return values

Default

all

Detailed Description

By default, the UNR App select all mode for functional coverage. Use this command to select other modes of functional coverage for the UNR analysis.

Example

```
set_unr_functional_coverage_mode assertion
```

See Also

check_unr,
set_unr_functional_coverage

set_unr_partition_max_jobs

Sets a limit on the number of partitions that run in parallel.

Syntax

```
set_unr_partition_max_jobs N
```

Returns

No return values

Default

3

Detailed Description

By default, the UNR Auto partition runs 3 partitions in parallel.
Use this command to specify a limit on the number of partitions that run in parallel.

Example

```
set_unr_partition_max_jobs 2
```

set_unr_prove_max_chunks

Sets the maximum number of chunks for dividing proof properties in the UNR flow.

Syntax

```
set_unr_prove_max_chunks N
```

Returns

No return values

Default

6

Detailed Description

By default, the UNR App divides proofs with more than 100000 properties into chunks, optimizing the verification process for complex designs by avoiding excessive memory use. Use this command to set the current maximum number of chunks into which proof properties can be divided.

Example

```
set_unr_prove_max_chunks 4
```

See Also

check_unr,
set_unr_prove_max_properties_per_chunk

set_unr_prove_max_properties_per_chunk

Sets the maximum number of properties per proof chunk in the UNR flow.

Syntax

```
set_unr_prove_max_properties_per_chunk N
```

Returns

No return values

Default

100000

Detailed Description

By default, the UNR App divides proofs with more than 100000 properties into chunks, optimizing the verification process for complex designs by avoiding excessive memory use. Use this command to set the current maximum number of properties allowed per chunk, with the number of needed chunks being calculated based on this value up to the maximum limit set by the system.

Example

```
set_unr_prove_max_properties_per_chunk 50000
```

See Also

check_unr,
set_unr_prove_max_chunks

set_use_low_power_port_definition

Specifies whether output is the new UPF_ISO port.

Syntax

```
set_use_low_power_port_definition (on | off)
```

Returns

No return values

Default

off

Detailed Description

Use this command to enable "get_design_info -list output" to show the new UPF_ISO output port when an isolation cell has been inserted or to enable "stopat instance_name" to stop at the new UPF_ISO output port.

Example

```
set_use_low_power_port_definition on
analyze -sv test.v
elaborate
check_lpv -load_upf test.upf
check_lpv -generate_power_design
get_design_info -list output

List of outputs: 2 (2)
------------------
out2 (1)
out_UPF_ISO (1)

out2 out_UPF_ISO
```

See Also

get_design_info,
stopat

set_visualize_annotate_clock_events

Draws slow or gated clocks events on Visualize waveforms.

Syntax

```
set_visualize_annotate_clock_events (on | off)
```

Returns

No return values

Default

on

Detailed Description

Use "set_visualize_annotate_clock_events" to disable or re-enable the feature that adds gray triangles to the background of a Visualize plot to show where a signal might have its value changed by slow or gated clock edges.

Important things to know about this command:
. If the clock logic is too complex, plotting performance might be affected.
. Visualize adds clock event triangles for flop outputs and flop buffers only.

Example

```
set_visualize_annotate_clock_events off
```

set_visualize_auto_check_props

Computes exercised covers and violated assertions.

Syntax

```
set_visualize_auto_check_props (on | off)
```

Returns

No return values.

Default

off

Detailed Description

By default, the tool does not automatically compute exercised covers and violated assertions. Use this command to turn on the computation. If the computation is turned "on", the properties in the Indexed Behaviors (Visualize window) display the cycles in which covers are exercised and assertions are failing. Find this display in the "At" column.

NOTE: Enabling this computation can impact run-time performance.

Example

```
set_visualize_auto_check_props on
```

set_visualize_auto_load_debugging_tables

Loads debugging tables for all applicable waveforms.

Syntax

```
set_visualize_auto_load_debugging_tables (on | off)
```

Returns

No return values

Default

off

Detailed Description

By default, the tool does not automatically load debugging tables. Use this command to set the tool to automatically load debugging tables for all applicable waveforms.

NOTE: Enabling debugging tables can impact run-time performance.

Example

```
set_visualize_auto_load_debugging_tables on
```

set_visualize_default_layout

Specifies the classic or simplified Visualize window layout.

Syntax

```
set_visualize_default_layout (classic | simplified | auto)
```

Returns

No return values

Default

auto

Detailed Description

Use this command to configure the default layout of the Visualize window. The "layout" of the Visualize window is a collection of settings in a Visualize window. The settings include toolbars, views, and so forth.

If you specify the "auto" option, Visualize uses the "classic" layout for general Jasper sessions and the "simplified" layout for the Debug Handoff mode.

set_visualize_highlight_limit

Specifies a cover trace length threshold to suppress highlight calculation.

Syntax

```
set_visualize_highlight_limit cycles
```

Returns

No return values

Default

10000 cycles

Detailed Description

By default, if a cover trace is longer than 10,000 cycles, Visualize suppresses the highlight computation for the entire trace. Use "set_visualize_highlight_limit" to modify the upper limit.

NOTE:
. To remove the limit, use $.
. To suppress all highlight calculations, use 0.

Important: Computing highlights for long traces can adversely affect Visualize performance in terms of run times.

Example

```
set_visualize_highlight_limit 25000
```

See Also

visualize

set_visualize_log_default_auto_sync

Specifies whether Visualize Log pane updates the data automatically.

Syntax

```
set_visualize_log_default_auto_sync (on | off)
```

Returns

No return values

Default

off

Detailed Description

Use this command to specify whether the Visualize Log pane updates the data automatically when the check result of the property, directed test, or $display changes.

set_visualize_multiple_traces

Enables the capture of intermediary traces when running Visualize QuietTrace.

Syntax

```
set_visualize_multiple_traces (on | off | preference)
```

Returns

No return values

Default

preference

Detailed Description

Use "set_visualize_multiple_traces" to enable or disable the capture of intermediary traces when running Visualize QuietTrace. When this feature is disabled, only the final trace is captured. By default, the tool respects the Visualize Preferences setting (Edit – Preferences – Visualize).

Example

```
set_visualize_multiple_traces on
```

set_visualize_preload_signal_list

Specifies the signal list file the tool loads in each new Visualize window.

Syntax

```
set_visualize_preload_signal_list [-task task_name] file_name
```

Returns

No return values

Detailed Description

Use this command to specify a signal list file that you want the tool to load in each new Visualize window. By default, no preload signal list file exists.

You can generate a sig_order file in the following ways.
1. Use the command "visualize -save_sig_order"
2. Use the Visualize menu option:
   File - Save Signal List

NOTE: This setting applies to all Visualize windows opened after this point. If you want to apply a different setting to a specific Visualize window, use the "visualize -sig_order" command.

-task task_name
    Set the signal file for the specified task.

    NOTE: Replace "task_name" with a period (.) to specify the current task.

Example

```
set_visualize_preload_signal_list /home/mydir/mylist.sig
set_visualize_preload_signal_list t1list.sig -task t1
```

See Also

visualize

set_visualize_show_default_signals

Specifies whether Visualize window loads default signals when opening a trace.

Syntax

```
set_visualize_show_default_signals (on | off)
```

Returns

No return values

Default

on

Detailed Description

Use this command to specify whether the Visualize window loads the default signals when it opens a trace. The default signals can be the system clock, immediate fanin signals of the target property or expression, or app-specific signals.

See Also

set_visualize_preload_signal_list

set_visualize_show_full_struct

Enables value generation for all struct nodes.

Syntax

```
set_visualize_show_full_struct (on | off)
```

Returns

No return values

Default

off

Detailed Description

By default, Visualize does not display struct values for all intermediate struct nodes. Use "set_visualize_show_full_struct" to enable this feature.

Example

```
set_visualize_show_full_struct on
```

See Also

visualize

set_visualize_show_strength_information

Specifies whether strength information is displayed on the Visualize window.

Syntax

```
set_visualize_show_strength_information (on | off)
```

Returns

No return values

Default

on

Detailed Description

Use this command to specify whether the Visualize window shows strength information in elements' tooltips. The information displays for each signal when you hover the pointer over a trace value in a given cycle.

Example

```
set_visualize_show_strength_information off
```

set_visualize_waveedit_transparency

Sets the WaveEdit background transparency.

Syntax

```
set_visualize_waveedit_transparency N
```

Returns

No return values

Default

60

Detailed Description

Use this command to set the transparency percentage of the WaveEdit background.

Example

```
// Set the WaveEdit background to 0 transparency, so the trace looks as it would
// outside the WaveEdit mode
set_visualize_waveedit_transparency 0

// Set the WaveEdit background to 80 transparency, so the trace looks 80%
// fainter than it would outside the WaveEdit mode
set_visualize_waveedit_transparency 80
```

set_why_include_non_ticking_gated_clocks

Turns on (or off) the inclusion of non-ticking gated clocks in "Why" results.

Syntax

```
set_why_include_non_ticking_gated_clocks (on | off)
```

Returns

No return value.

Default

off

Detailed Description

Use this command to include gated clocks in the results of the "Why" feature when called over flops whose primary clock has been masked/disabled.

Example

```
set_why_include_non_ticking_gated_clocks on
```

set_word_level_reduction

Enables word-level transformations/reductions during the proof flow.

Syntax

```
set_word_level_reduction (on | off)
```

Returns

No return values

Default

off

Detailed Description

Use this command to enable the word-level proof flow with transformations/reductions that optimize the design before calling the proof engines.

NOTE:
. Using word-level reduction can degrade performance in terms of run time; however, the word-level flow model generation step has a default wall clock time limit of 300 seconds. Use the command "set_word_level_time_limit" to set a different limit.
. The tool reverts to bit-level flow if it detects design constructs that are not supported by the word-level flow (for example: divider, modulus, power operators).
. The word-level flow is not supported in the following cases:
  . When using SEC, SPV, X-Prop, and C2RTL Apps.
  . When using the Coverage App with the proof core calculation enabled
  . When using the LPV App with a power-aware design
  In addition, Formal Profiler does not support the word-level flow.

Example

```
set_word_level_reduction on
```

See Also

set_word_level_time_limit

set_word_level_time_limit

Specifies the wall clock time limit for word-level flow model generation step.

Syntax

```
set_word_level_time_limit  time_limit
```

Returns

No return values

Default

300s

Detailed Description

Use this command to change the time limit for word-level flow model generation step.

The tool supports three formats for the time limit:

```
. ISO8601:    hh':'[mm[':'[ss['.'[fff]]]]]
  Examples:   2:30, 2:, 0:0:23, 2:30:05.235
. Jasper: [dd'd'][hh'h'][mm'm'][ss[.fff]'s']
  Examples:   2h30m, 2h, 23s, 2h30m05.235s
. Seconds:    sssss[.fff]['s']
  Examples:   9000s, 7200, 23s, 9005.235s
```

Example

```
set_word_level_time_limit 10m
```

See Also

set_word_level_reduction

set_xprop_autoprove_expose_internal_cex

Specifies whether XPROP displays a task for internal counterexamples.

Syntax

```
set_xprop_autoprove_expose_internal_cex (on | off)
```

Returns

No return values

Default

false

Detailed Description

Use this command to specify that XPROP "autoprove" shows the counterexamples found on internal signals. This command can be helpful for early debugging of failures. For example, if you did not expect to find a CEX on specific internal signals or targets are not yet determined, this command can help you investigate the reason for the failures.

The internal CEX traces are located in the
"<XPROP_work_task>_<XPROP_target_property>_internal_cex" task,
where <XPROP_work_task> is the task from which your XPROP proof was run and
<XPROP_target_property> is the property sent to the XPROP proof.

NOTE:
. If multiple properties are sent to the same proof, one is arbitrarily
  chosen as the <XPROP_target_property>.
. Prove commands, such as "check_xprop -prove", cannot be run on
  the internal CEX task.

Example

```
set_xprop_autoprove_expose_internal_cex on
```

See Also

check_xprop

set_xprop_autoprove_max_number_of_internal_cex

Sets the max number of counterexamples shown in the internal CEX task.

Syntax

```
set_xprop_autoprove_max_number_of_internal_cex N
```

Returns

No return values

Default

100

Detailed Description

Use this command to define the maximum number of counterexamples found on internal signals that can be reported in the XPROP internal CEX task. The internal CEX traces are located in the

"<XPROP_work_task>_<XPROP_target_property>_internal_cex" task, where <XPROP_work_task> is the task from which your XPROP proof was run and <XPROP_target_property> is the property sent to the XPROP proof.

Example

```
set_xprop_autoprove_max_number_of_internal_cex 50
```

See Also

check_xprop,
set_xprop_autoprove_expose_internal_cex

set_xprop_autoprove_minimal_max_length_of_internal_cex

Sets the minimal max length of counterexamples in the internal CEX task.

Syntax

```
set_xprop_autoprove_minimal_max_length_of_internal_cex N
```

Returns

No return values

Default

2

Detailed Description

Use this command to specify that you want XPROP "autoprove" to exclude counterexample (CEX) traces with a max length smaller than N from the "internal_cex" task. This command can be helpful for early debugging of failures with the specified trace length.

The internal CEX traces are located in the "<XPROP_work_task>_internal_cex" task, where <XPROP_work_task> is the task from which your XPROP proof was run.

Example

```
set_xprop_autoprove_minimal_max_length_of_internal_cex 7
```

See Also

check_xprop,
set_xprop_autoprove_expose_internal_cex,
set_xprop_autoprove_max_number_of_internal_cex

set_xprop_break_loop_mode

Specifies how X-Prop handles breaking combo loops.

Syntax

```
set_xprop_break_loop_mode (default | sync)
```

Returns

No return values

Default

default

Detailed Description

Use this command to specify how you want the tool to handle breaking loops under "set_loop_handling break".

default
    The break point is an X-source.

sync
    The break point is not an X-source.

NOTE: Changing the value of this configuration clears the proof results.

Example

```
set_xprop_break_loop_mode sync
```

See Also

set_loop_handling

set_xprop_ignore_non_resetting_flops_as_source

Consider only uninitialized flops that are part of a reset block as X-sources.

Syntax

```
set_xprop_ignore_non_resetting_flops_as_source (on | off) [-task task_name]
```

Returns

No return values

Default

off

Detailed Description

When considering the initial value of uninitialized flops as an X-source ("xprop_use_reset_state" is "on"), only flops that are part of a reset block but not assigned a concrete initial value will have their initial value considered an X-source.

[-task task_name]
    It is possible to change the sources of X just for a specific task instead of for all tasks. In this case, only task_name's properties get their proof status cleared.

    When a specific task behavior is set, it has higher priority over global behaviors. When a global behavior is set, all task-specific behaviors previously set are cleared.

See Also

assert,
elaborate

set_xprop_proofgrid_utilization_warning_percentage_threshold

Specifies Proofgrid utilization threshold in percentage for getting warnings

Syntax

```
set_xprop_proofgrid_utilization_warning_percentage_threshold N
```

Returns

No return values

Default

70

Detailed Description

Use this command to specify the threshold in percentage for getting warnings when Proofgrid utilization is lower then the threshold.

NOTE: When setting the threshold to 0 no warnings will be shown.

Example

```
set_xprop_proofgrid_utilization_warning_percentage_threshold 90
```

set_xprop_show_proof_log

Specifies whether XPROP shows the proof log.

Syntax

```
set_xprop_show_proof_log (true | false)
```

Returns

No return values

Default

false

Detailed Description

Use this command to specify whether you want the X-Prop App to show the proof log.

When setting this configuration to "true", the tool prints more information in log:

1. Accumlative ProofGrid utilization data for the all internal proofs
2. Utilization report for each internal prove (ISEC134):
    a. Prove runtime
    b. ProofGrid slot seconds – accumulative runtime of all engines in the prove
    c. Average parallel running jobs – average number of jobs ran in practice (slot seconds / prove runtime)
    d. Average max jobs limit – average jobs limit to the specific prove (actual limit is dynamic during the proof)
    e. Running to limit jobs ratio – the ratio between average parallel running jobs to average max jobs limit (Average parallel running jobs / Average max jobs limit)
    f. ProofGrid jobs runtime in percentage – the percentage of the engines' actual runtime since ProofGrid launched their requests

    NOTE: First, look at the ProofGrid jobs run time in percentage (#f above). If it is low, it might show environment issues. If it is high, look at Average parallel running jobs (#b above), and analyze whether this number makes sense compared to number of properties and number of jobs you have submitted to the proof.

Example

```
# Example for accumlative ProofGrid utilization data:
INFO (IPF180): 0: ProofGrid utilization: 99.05slot-seconds: 576.64

# Example for utilization report of internal prove:
INFO (ISEC134): Utilization: Action run time: 20 seconds.
INFO (ISEC134): Utilization: Proof grid slot seconds: 18.87.
INFO (ISEC134): Utilization: Average parallel running jobs: 10.94332.
INFO (ISEC134): Utilization: Average max jobs limit: 10.95.
INFO (ISEC134): Utilization: Running to limit jobs ratio: 99.93%.
INFO (ISEC134): Utilization: Proof grid jobs run time in percentage: 99.66.

Example for using the setting:
set_xprop_show_proof_log on
```

set_xprop_use_all_undriven

Treats all undriven nets as sources of X during X-propagation checks.

Syntax

```
set_xprop_use_all_undriven (on | off) [-task task_name]
```

Returns

No return values

Default

off and all tasks

Detailed Description

```
By default, the tool treats only black-boxed outputs as sources of
X. Use this command to change the default behavior and treat
all undriven nets (black-boxed outputs, inputs, stopat nets, and internal
undriven nets) as sources of X during X-propagation checks.

Important information about this command: Using the command
"set_xprop_use_all_undriven on" has the same effect as the following commands
combined:
. set_xprop_use_bbox_outputs on
. set_xprop_use_inputs on
. set_xprop_use_stopats on
. set_xprop_use_internal_undriven on

CAUTION:
. If "xprop_use_all_undriven" is on, it supersedes any prior *and* future
  instances of the commands listed above.
. If "xprop_use_all_undriven" is off, the tool *obeys* any prior and future
  instances of the commands listed above.

on
    Treat all undriven nets as sources of X.

off
    Do not treat all undriven nets at once as sources of X.
    This option is the default.

[-task task_name]
    Change the sources of X for the specific task instead of for all tasks.

    NOTE:
    . If you use the "-task" switch, the tool clears the proof status for
      properties in the specified task.
    . If a specific task behavior is set, it has higher priority over global
      behaviors. However, if you set a global behavior, the tool clears all
      previously set task-specific behaviors.
```

See Also

set_xprop_use_bbox_outputs,
set_xprop_use_bus_contention,
set_xprop_use_bus_floating,
set_xprop_use_inputs,
set_xprop_use_internal_undriven,
set_xprop_use_reset_state,
set_xprop_use_stopats,

set_xprop_use_bbox_outputs

Treats black-boxed outputs as sources of X during X-propagation checks.

Syntax

```
set_xprop_use_bbox_outputs (on | off) [-task task_name]
```

Returns

No return values

Default

on and all tasks

Detailed Description

```
By default, the tool treats black-boxed outputs as sources of X during
X-propagation checks. Use this command to change the default behavior so that
the tool does not treat the black-boxed outputs as sources of X during
X-propagation checks.

Important information about this command:
. If blackboxed outputs are not treated as X-sources, the tool propagates
  Xs through the blackbox (that is, if an X reaches an input of the blackbox,
  all outputs might be considered as X).
. This command is intended for fine-grained control over what the tool
  considers as sources of X during X-propagation. If you want to treat all
  undriven nets (blackboxed outputs, inputs, stopat nets, and internal undriven
  nets) as sources of X, use the command "set_xprop_use_all_undriven on"
  instead of this command.

CAUTION:
. If "xprop_use_all_undriven" is on, it supersedes any prior *and* future
  instances of "set_xprop_use_bbox_outputs".
. If "xprop_use_all_undriven" is off, the tool *obeys* any prior and future
  instances of "set_xprop_use_bbox_outputs".

on
    Treat black-boxed outputs as sources of X.
    This option is the default.

off
    Do not treat black-boxed outputs as sources of X.

[-task task_name]
    Change the sources of X for the specific task instead of for all tasks.

    NOTE:
    . If you use the "-task" switch, the tool clears the proof status for
      properties in the specified task.
    . If a specific task behavior is set, it has higher priority over global
      behaviors. However, if you set a global behavior, the tool clears all
      previously set task-specific behaviors.
```

See Also

set_xprop_use_all_undriven,
set_xprop_use_inputs,
set_xprop_use_internal_undriven,
set_xprop_use_reset_state,
set_xprop_use_stopats,
set_xprop_use_bus_contention,
set_xprop_use_bus_floating

set_xprop_use_bus_contention

Treats all multiple driven buses as sources of X during X-propagation checks.

Syntax

```
set_xprop_use_bus_contention (on | off) [-task task_name]
```

Returns

No return values

Default

on and all tasks

Detailed Description

```
By default, the tool handles multiple driven buses in the design as sources of
X during X-propagation checks. Use "elaborate -disable_x_handling" if you do not
want multiple driven buses treated as sources of X during X-propagation checks.
After elaborating the design, use "set_xprop_use_bus_contention off" to ignore
Xs in the design during X-propagation checks.

on
    Treat all multiple driven buses in the design as sources of X.
    This option is the default.

off
    Do not treat multiple driven buses in the design as sources of X.

[-task task_name]
    Change the sources of X for the specific task instead of for all tasks.

    NOTE:
    . If you use the "-task" switch, the tool clears the proof status for
      properties in the specified task.
    . If a specific task behavior is set, it has higher priority over global
      behaviors. However, if you set a global behavior, the tool clears all
      previously set task-specific behaviors.
```

Example

```
analyze -v2k my_hdl_file.v
elaborate
get_xprop_use_bus_contention
set_xprop_use_bus_contention off // Ignore Xs during X-Prop checks.
```

See Also

set_xprop_use_all_undriven,
set_xprop_use_bbox_outputs,
set_xprop_use_inputs,
set_xprop_use_internal_undriven,
set_xprop_use_reset_state,
set_xprop_use_stopats

set_xprop_use_bus_floating

Treats all undriven buses as sources of X during X-propagation checks.

Syntax

```
set_xprop_use_bus_floating (on | off) [-task task_name]
```

Returns

No return values

Default

on and all tasks

Detailed Description

```
By default, the tool handles undriven buses in the design as sources of X during
X-propagation checks. Use "elaborate -disable_x_handling" if you do not want
undriven buses treated as sources of X during X-propagation checks.
After elaborating the design, use "set_xprop_use_bus_floating off" to ignore
Xs in the design during X-propagation checks.

on
    Treat all undriven buses in the design as sources of X.
    This option is the default.

off
    Do not treat undriven buses in the design as sources of X.

[-task task_name]
    Change the sources of X for the specific task instead of for all tasks.

    NOTE:
    . If you use the "-task" switch, the tool clears the proof status for
      properties in the specified task.
    . If a specific task behavior is set, it has higher priority over global
      behaviors. However, if you set a global behavior, the tool clears all
      previously set task-specific behaviors.
```

Example

```
analyze -v2k my_hdl_file.v
elaborate
get_xprop_use_bus_floating
set_xprop_use_bus_floating off // Ignore Xs during X-Prop checks.
```

See Also

set_xprop_use_all_undriven,
set_xprop_use_bbox_outputs,
set_xprop_use_inputs,
set_xprop_use_internal_undriven,
set_xprop_use_reset_state,
set_xprop_use_stopats

set_xprop_use_init_abstraction

Treats flop init abstractions as sources of X during X-propagation checks.

Syntax

```
set_xprop_use_init_abstraction (on | off) [-task task_name]
```

Returns

No return values

Default

off and all tasks

Detailed Description

```
By default, X-propagation checks do not treat flop init abstractions as
sources of X. Use this command to enable flop init abstractions as sources of
X or reinstate the default.

on
    Use flop init abstractions as sources of X.

off
    Ignore flop init abstractions as sources of X.
    This option is the default.

[-task task_name]
    Apply this command to the specified task.

    NOTE:
    . If you specify a task, the tool only clears proof status for the
      specified task's properties.
    . A task configuration value has priority over global configuration values,
      but if you subsequently apply the command to the global environment, the
      tool clears the previously specified task-specific configuration.
    . When using "xprop_use_reset_state off", the init value of uninitialized
      flops with init value abstraction is considered a source of X.
```

Example

```
analyze -sv $RTL_DIR/dut.sv
elaborate -top dut

clock clk
reset -expression ~rst_n

set_xprop_use_init_abstraction on

abstract -init a_out
assert -xprop -to a_out

prove -task
```

See Also

abstract,
assert,
reset

set_xprop_use_inputs

Treats primary inputs as sources of X during X-propagation checks.

Syntax

```
set_xprop_use_inputs (on | off) [-task task_name]
```

Returns

No return values

Default

off and all tasks

Detailed Description

By default, the tool does not treat primary inputs as sources of X during X-propagation checks. Use this command to change the default behavior and treat primary inputs as sources of X during x-propagation checks.

Important information about this command:
This command is intended for fine-grained control over what the tool considers as sources of X during X-propagation. If you want to treat all undriven nets (black-boxed outputs, inputs, stopat nets, and internal undriven nets) as sources of X, use the command "set_xprop_use_all_undriven on" instead of this command.

CAUTION:
. If "xprop_use_all_undriven" is on, it supersedes any prior *and* future instances of "set_xprop_use_inputs".
. If "xprop_use_all_undriven" is off, the tool *obeys* any prior and future instances of "set_xprop_use_inputs".

on
    Treat primary inputs as sources of X.

off
    Do not treat primary inputs as sources of X.
    This option is the default.

[-task task_name]
    Change the sources of X for the specific task instead of for all tasks.

    NOTE:
    . If you use the "-task" switch, the tool clears the proof status for properties in the specified task.
    . If a specific task behavior is set, it has higher priority over global behaviors. However, if you set a global behavior, the tool clears all previously set task-specific behaviors.

See Also

set_xprop_use_all_undriven,
set_xprop_use_bbox_outputs,
set_xprop_use_internal_undriven,
set_xprop_use_reset_state,
set_xprop_use_stopats,
set_xprop_use_bus_contention
set_xprop_use_bus_floating

set_xprop_use_internal_undriven

Sets whether internal undriven nets/unconnected input ports are sources of X.

Syntax

```
set_xprop_use_internal_undriven (on | off) [-task task_name]
```

Returns

No return values

Default

off and all tasks

Detailed Description

By default, the tool does not treat internal undriven nets and unconnected input ports as sources of X during X-propagation checks. Use this command to change the default behavior and treat internal undriven nets and unconnected input ports (other than black-boxed outputs, inputs, and stopat nets) as sources of X during X-propagation checks.

Important information about this command:
This command is intended for fine-grained control over what the tool considers as sources of X during X-propagation. If you want to treat all undriven nets (black-boxed outputs, inputs, stopat nets, unconnected input ports and internal undriven nets) as sources of X, use the command "set_xprop_use_all_undriven on" instead of this command.

CAUTION:
. If "xprop_use_all_undriven" is on, it supersedes any prior *and* future instances of "set_xprop_use_internal_undriven".
. If "xprop_use_all_undriven" is off, the tool *obeys* any prior and future instances of "set_xprop_use_internal_undriven".

on
    Treat internal undriven nets and unconnected input ports as sources of X.

off
    Do not treat internal undriven nets and unconnected input ports as sources of X. This option is the default.

[-task task_name]
    Change the sources of X for the specific task instead of for all tasks.

    NOTE:
    . If you use the "-task" switch, the tool clears the proof status for properties in the specified task.
    . If a specific task behavior is set, it has higher priority over global behaviors. However, if you set a global behavior, the tool clears all previously set task-specific behaviors.

See Also

set_xprop_use_all_undriven,
set_xprop_use_bbox_outputs,
set_xprop_use_inputs,
set_xprop_use_reset_state,
set_xprop_use_stopats,
set_xprop_use_bus_contention,
set_xprop_use_bus_floating

set_xprop_use_reset_abstraction

Treats flop reset abstractions as sources of X during X-propagation checks.

Syntax

```
set_xprop_use_reset_abstraction (on | off) [-task task_name]
```

Returns

No return values

Default

off and all tasks

Detailed Description

```
By default, X-propagation checks do not treat flop reset abstractions as
sources of X. Use this command to enable flop reset abstractions as sources of
X or reinstate the default.

on
    Use flop reset abstractions as sources of X.

off
    Ignore flop reset abstractions as sources of X.
    This option is the default.

[-task task_name]
    Apply this command to the specified task.

    NOTE:
    . If you specify a task, the tool only clears proof status for the
      specified task's properties.
    . A task configuration value has priority over global configuration values,
      but if you subsequently apply the command to the global environment, the
      tool clears the previously specified task-specific configuration.
```

Example

```
analyze -sv $RTL_DIR/dut.sv
elaborate -top dut

clock clk
reset -expression ~rst_n

set_xprop_use_reset_abstraction on

abstract -reset_value a_out
assert -xprop -to a_out

prove -task
```

See Also

assert,
reset

set_xprop_use_reset_state

Treats uninitialized registers as sources of X during X-propagation checks.

Syntax

```
set_xprop_use_reset_state (on | off) [-task task_name]
```

Returns

No return values

Default

on and all tasks

Detailed Description

By default, X-propagation checks treat uninitialized registers as sources of X. Use this command to ignore uninitialized registers or reinstate the default.

on
    Use uninitialized registers as sources of X.
    This option is the default.

off
    Ignore uninitialized registers.

[-task task_name]
    It is possible to change the sources of X just for a specific task, instead of for all tasks. In this case just task_name's properties get their proof status cleared.

    When a specific task behavior is set, it has higher priority over global behaviors. When a global behavior is set, all task specific behaviors previously set are cleared.

Example

```
analyze -sv $RTL_DIR/dut.sv
elaborate -top dut

clock clk
reset -expression ~rst_n

get_reset_info -x_value

assert -xprop -to a_out
assert -xprop -to b_out

set_xprop_use_reset_state on
prove -task
```

See Also

assert,
elaborate

set_xprop_use_stopats

Treats stopat nets as sources of X during X-propagation checks.

Syntax

```
set_xprop_use_stopats (on | off) [-task task_name]
```

Returns

No return values

Default

off and all tasks

Detailed Description

By default, the tool does not treat stopat nets as sources of X during
X-propagation checks. Use this command to change the default behavior
and treat stopat nets (including instance outputs after instance stopat)
as sources of X during X-propagation checks.

Important information about this command:
This command is intended for fine-grained control over what the tool considers
as sources of X during X-propagation. If you want to treat all undriven nets
(black-boxed outputs, inputs, stopat nets, and internal undriven nets) as
sources of X, use the command "set_xprop_use_all_undriven on" instead of this
command.

CAUTION:
. If "xprop_use_all_undriven" is on, it supersedes any prior *and* future
  instances of "set_xprop_use_stopats".
. If "xprop_use_all_undriven" is off, the tool *obeys* any prior and future
  instances of "set_xprop_use_stopats".

on
    Treat stopat nets as sources of X.

off
    Do not treat stopat nets as sources of X.
    This option is the default.

[-task task_name]
    Change the sources of X for the specific task instead of for all tasks.

    NOTE:
    . If you use the "-task" switch, the tool clears the proof status for
      properties in the specified task.
    . If a specific task behavior is set, it has higher priority over global
      behaviors. However, if you set a global behavior, the tool clears all
      previously set task-specific behaviors.

See Also

set_xprop_use_all_undriven,
set_xprop_use_bbox_outputs,
set_xprop_use_inputs,
set_xprop_use_internal_undriven,
set_xprop_use_reset_state,
set_xprop_use_bus_contention,
set_xprop_use_bus_floating

A

Appendix A: PSL Support

This appendix lists Jasper Apps support for IEEE® Std 1850™-2005 *Standard for Property Specification Language (PSL)*. By default, Jasper Apps analyzes embedded PSL. The table below lists various clarifications, exceptions, and limitations.

- Jasper Apps supports SystemVerilog properties in PSL code. Use the `analyze` switch `-change_hdl_mode_inside_vunit` to enable the compiler to change the parser mode to the HDL language specified by the `analyze` command when compiling a PSL verification unit.
- To learn more about PSL, download the application note "Introduction to PSL" from the Jasper Apps landing page of the Cadence Application Support and Knowledge website (support.cadence.com). The goal of this application note is to provide you with sufficient knowledge of PSL so that you can write your own design-specific set of properties to be formally verified.

Item	Supported	Description
`'event`, `rising_edge` and `falling_edge` (VHDL)		
	Yes* *In clock expressions only	Jasper Apps does not support `'event` in regular properties in the VHDL flavor of PSL. As a workaround, use `rose(a)` or `fell(a)` instead of `a'event` in the case of a Boolean signal `a`.
Binding verification units of an HDL flavor different from the design		
	No	It is not possible to bind a verification unit written in the VHDL flavor to a Verilog or SystemVerilog design (and vice versa).
Binding verification units to design instances		
	No	Binding verification units to design instances is not supported.

Binding verification units to modules analyzed with `analyze -repository`		

	No	Binding verification units to modules analyzed with the `analyze -repository` command is not supported. However, you can use embedded PSL in these modules.
		
		If you want to bind a PSL verification unit to a verification module that contains modeling code to be used in the properties, read the module as part of the design with the `analyze` command instead of `analyze -repository` and use `connect -bind` to instantiate it in the design hierarchy.
		
		Example of verification module with `-repository`:
		
		`# Analyze the DUV files`
		
		`analyze...`
		
		`# Analyze the verification files`
		
		`analyze -repository...`
		
		`# Elaborate design hierarchy`
		
		`elaborate...`
		
		`# Elaborate verification module`
		
		`elaborate -repository...`
		
		`# Instantiate verification module`
		
		`connect -repository...`
		
		Example of verification module usage for PSL vunits:
		
		`# Analyze the DUV files`
		
		`analyze...`
		
		`# Analyze the verification files`
		
		`analyze...`
		
		`# Analyze the PSL vunit files`
		
		`analyze -psl...`
		
		`# Elaborate design hierarchy`

		`elaborate...`
		
		`# Instantiate verification module`
		
		`connect -bind... -elaborate`

Binding verification units to modules retrieved through the `analyze -y` and `-v` mechanisms		
	Yes	If there are multiple modules with the name specified in the PSL binding, the tool uses the first one.
`Compound_SERE` operators `&&`, `&`, and `|`		
	Yes* *See description for restrictions.	The `Compound_SERE` operators `&&`, `&`, and `|` do not parse with the correct precedences. This can lead to erroneous properties or strange parse error messages. As a workaround, use braces `{ }` to control the operands to these operators.
Design wire overriding (modeling layer)		
	No	Design wire overriding (modeling layer) is not supported. This feature allows design nets to be overridden if the same signal name is specified in the modeling layer (having the same type). As a workaround, consider the following scenario. If the name of a modeling layer signal is the same as the name for a signal in the module to which the verification unit binds (but the intention was not to override it), change the name of the modeling layer signal to something that does not clash with a design signal name.
GDL		
	No	The GDL flavor, which is also known as EDL, is not supported.
Inheritance of symbols		
	No	Inheritance of symbols is not supported. For example, consider a case where we declare a property `p` in a vunit `v` that is inherited by vunit `w`. In this case, we cannot access `p` within `w`. The workaround in this case is to move (that is, copy) the definition of `p` to `w`.

`isunknown()`		
	Yes* *See description for restrictions.	• `isunknown(<constant>)` reports true (`1'b1`) if the constant contains `x` or `z`. `isunknown(<constant>)` is supported in Verilog only. • `isunknown(<signal>)` always reports false (`1'b0`).
Local variables		
	No	Local variables in PSL properties are not supported.
Macro layer		
	Yes* *See description for restrictions.	With the following exceptions, the macro layer is supported inside vunits: • String expressions involving `%for` variables are not supported. • The VHDL flavor `#if` and `#elif` pre-processing directives are not supported.
modeling layer `next()` function		
	Yes* *See description for restrictions	The modeling layer `next()` function is supported; however, it is not allowed directly in properties. It is a construct used in modeling code. Refer to Section 5.2.3.2 of the IEEE 1850 PSL LRM for additional limitations. This item is not related to the `next` keyword for properties.
Multiple clocks		
	Yes* *See description for restrictions.	The tool elaborates properties sampled by more than one clock, provided you use the `-multiple_clock` switch. Using this switch can, in some cases, increase the complexity of a prove or Visualize job.

Native PSL (VHDL 2008)		
	Yes* *See description for restrictions	Support is limited to the following VHDL 2008 native PSL items: • PSL `property`, `sequence`, and `clock` declaration • PSL directives (`assume`, `cover`, and `assert`) • PSL vunits support hierarchical references; however, external names cannot reference packages.
Non-determinism		
	Yes* *See description for restrictions.	The `union` keyword is supported; however, the built-in functions `nondet()` and `nondet_vector()` are not supported.
Optional branching extension layer		
	No	This CTL part of PSL is not supported. It deals with branching tree logic. This means there is no support for the operators `AX`, `AG`, `AF`, `EX`, `EG`, `EF`, `A [ U ]`, and `E [ U ]`.
Parameterized properties		
	No	Parameterized properties are not supported.
Parameterized SERE		
	No	Parameterized SEREs are not supported.
Replicated properties		
	Yes* *See description for restrictions	With the following limitations, the specification of replicated properties using the `forall` keyword is supported within the VHDL flavor only: • The optional `Index_Range` field is not supported. • The use of replicated properties is not allowed in embedded PSL code. For more information, refer to Section 6.2.3 of the IEEE-1850 2005 LRM, *IEEE Standard for Property Specification Language (PSL)*.

SystemVerilog (modeling layer)		
	Yes* *See description for restrictions	Verilog data types, for example, `reg`, `integer`, and `time`, are supported, but SystemVerilog data types, for example, `logic` and `enum`, are not supported.
Temporal parameters		
	Yes* *See description for restrictions	Passing temporal parameters, sequences to sequences or sequences and properties to properties, is only supported in the VHDL flavor.
Unbound verification units		
	No	There is no support for the inheritance or elaboration of unbound verification units for use as independent property definitions. For more information, refer to Section 7.2.3 of the IEEE-1850 2010 LRM, *IEEE Standard for Property Specification Language (PSL)*.
Using keywords `vunit`, `vmode`, and `vprop` in VHDL code		
	No	The PSL keywords `vunit`, `vmode`, and `vprop` are always treated as keywords in "normal" VHDL code; however, since PSL keywords are case sensitive, it is acceptable to use, for example, `vUnit` as a normal identifier within both VHDL and PSL code. In general, if an identifier is one of the PSL keywords listed above, then you cannot directly reference that identifier from within PSL expressions. Instead, use an extended identifier, for example, `\vunit\`.
Verilog constructs specified in the IEEE-1850 2010 LRM, Section 8.1		

	Yes* *See description for restrictions	The Verilog `structures` construct, which is specified in Section 8.1 of the IEEE-1850 2010 LRM, *IEEE Standard for Property Specification Language (PSL)*, is supported if, and only if, you use the `analyze` switch `-change_hdl_mode_inside_vunit`. Integer ranges of type `(<constant expression> : <constant expression>)`, defined in the IEEE-1850 2010 LRM, are **not** supported, since they are not part of the Verilog's syntax.
`vprop` cover directives		
	Yes	The tool accepts cover directives in `vprop` verification units and issues a warning.

B

Appendix B: SVA Support

This appendix lists IEEE Std 1800™-2005, IEEE Std 1800-2009, IEEE Std 1800-2012, and IEEE Std 1800-2017 SystemVerilog Assertions (SVA) support status for Jasper Apps. It also gives a brief introduction to the use flow.

> SystemVerilog 2012 and SystemVerilog 2017 have only minor differences, so the tool treats them as if they are the same. Furthermore, for SVA purposes, there are only minor differences between them and SystemVerilog 2009. Therefore, the information about SystemVerilog 2009 in this appendix is also valid for SystemVerilog 2012.

This appendix includes the following sections:

- SVA Support Status
- Jasper Tcl Operators
- Additional Support for SystemVerilog 2009 and 2012

SVA Support Status

Jasper Apps supports IEEE 1800-2005, IEEE Std 1800-2009, IEEE Std 1800-2012, and IEEE Std 1800-2017 SystemVerilog Assertions. You can embed SVA in the design files or in a separate requirements file.

The primary use flow for SVA is:

```
# If you are using SV 2009, replace "analyze -sv" with "analyze -sv09".

# If you are using SV 2012, replace "analyze -sv" with "analyze -sv12".

analyze -sv file.v

elaborate

# SVA properties will now appear in the <embedded> task.
```

```
prove -all
```

The table "SVA Support Status" below lists various clarifications, exceptions, and limitations in SVA support. And "Additional Support for SystemVerilog 2009 and SystemVerilog 2012" at the end of this chapter provides information on additional SystemVerilog 2009 and SystemVerilog 2012 items the tool supports if you use the command `analyze -sv09` or `analyze -sv12`.

To learn more about SVA, download the application note "Introduction to SVA" and the "SVA Quick Reference Guide." To access the documents, go to the Jasper ASK page, look for the *Search Jasper Apps* search box, and type the titles there. The goal of these publications is to provide you with sufficient knowledge of SVA so that you can write your own design-specific set of properties to be formally verified.

SVA Support Status

Item	Supported	Additional Information
`===` and `!==`		
	Yes* * See the next column for restrictions.	• If you do not use `elaborate -triple_equal`, the tool treats the operators `===` and `!==` as `==` and `!=`. That is, by default, the tool does not respect case equality semantics. • Jasper supports triple equal for `0`, `1`, and Z, but support for X signals is limited.
`$isunknown()`		
	Yes* * See the next column for restrictions.	• `$isunknown(<constant>)` reports true `(1'b1)` if the constant contains x or z. • `$isunknown(<signal>)` and `$isunknown(<expression>)` always report false `(1'b0)`. • The tool does not support clocking statements in SVA assumptions that use `$isunknown`. See the following example: `assume property (@(posedge clk2) $isunknown(<signal>));)`
`action_block`		

	No	`action_block` specifies what actions are taken upon success or failure of the assertion. It is not applicable to formal verification. The tool ignores it and does not issue a warning.
`bind` statement		
	Yes* * See the next column for restrictions and guidelines.	Jasper Apps supports binding to modules and instances; however, the `bind` statement requires you to analyze the file as SystemVerilog (`-sv*`). The tool adds properties to the `<embedded>` task. The `bind` statement can be located in the following places: • Inside a module • Inside an interface • In the compilation-unit scope *Property files must be read as part of the design, not as requirements files.* If you read a property file with `analyze -req`, you will encounter the following: • If `bind` is in the requirements file, the tool generates an error message (design module or instance of module not found). • If `bind` is not in the requirements file, the elaboration does not find the property file, and it automatically black boxes the module or instance. Parameters are supported in module instantiations. It is a good practice to write the `bind` statement outside modules.
Binding to modules retrieved through the `analyze -y` and `-v` mechanisms		
	Yes	If there are multiple modules with the name specified in the SVA binding, the tool uses the first one.
Binding SVA checker modules to VHDL entities		

	Yes* * See the next column for restrictions and guidelines.	• The tool accepts the following in the bind statement: ◦ Operators `%`, `+`, `-`, `*`, `/`, `>`, `<`, `>=`, `<=`, `==`, `!=`, `&&`, `||`, `<<`, `>>`, `<<<`, `>>>`, `**`, `!` ◦ Concatenation ◦ Type casts ◦ Integer constants ◦ Function calls ◦ Array indexing ◦ Array slicing ◦ Selected names • The tool supports binding to entities and architectures. To bind to an architecture, specify the name as `\entity(arch)`. This format is required by Verilog lexical rules. If you bind to an entity name only, it will bind to all architectures of that entity. • The tool supports binding by position and name. Use the `.*` syntax to bind to all ports of the checker module matching names in the architecture. • The tool supports the use of constants and parameters in bind statements. • Bind statements are global and you cannot undo them. After the tool analyzes a bind statement, the only way to remove it is with `clear -all`. If elaboration fails, you must use `analyze -clear` to clear the parse tree; otherwise, the tool ignores the bind statement that failed and elaboration might finish without generating an error. The tool supports VHDL record fields in bind statement port declarations.
`cover property`		

	Yes*
	* See the next column for restrictions.
		`cover property` is supported for sequences and properties. If possible, `cover property` will identify a span of cycles where an instance of the property or sequence is attempted (exercised) and succeeds. This trace will be annotated with advanced debugging indicators.
		Example:
		`assert property (F);`
		`cover property (F);`
		• If the assertion is proven and it is not vacuously true, then, in most cases, the cover will produce an interesting sanity trace for `F`.
		• If the assertion is vacuously true, the cover will not be satisfied.
		• If the assertion gives a counterexample, the cover might still produce a cover trace. Although not all instances are satisfied, this trace indicates an instance where the property is satisfied.

`dist`		
	No	`dist` targets randomized simulation. It is not applicable to formal verification. The tool ignores it and does not issue a warning.
Event expressions		
	Yes* * See the next column for restrictions.	Event operators are fully supported inside always blocks. The operator `iff` is supported if you analyze the file as SystemVerilog (`-sv*`).
`expect`		
	No	`expect` is not applicable to formal verification. The tool ignores it and does not issue a warning.
`first_match(...)`		

	Yes* * See the next column for restrictions.	The following are not supported within an operand sequence of `first_match`: • The operators `##0`, `[=n:m]`, `[=n:$]`, `[->n:m]`, and `[->n:$]` • Local variable assignments
Immediate assertions		
	Yes	Immediate assertions are also known as in-line assertions.
Liveness properties		
	Yes* * See the next column for restrictions.	• Visualize supports liveness properties as targets. Refer to the documentation for `visualize -force` and `visualize -at_least_once` for additional details about configuring liveness constraints. To learn about liveness properties, see Appendix C, "Liveness vs. Safety."
Local variable assignments		

	Yes*
	*See stipulations in the next column.
		The `sequence_match_item` (`operator_assignment` or `inc_or_dec_expression`) is also known as a local variable assignment. SVA local variable assignments are supported under the following conditions:
		• Assignments are allowed in sequences that are in a negative context such as in the left-hand side of a suffix implication or in the scope of negation.
		• The tool does not support multi-clocked properties.
		• Assumptions without a fixed or finite-ranged cycle distance have a variable size limit.
		Examples of local variables used in a sequence and properties:
		`sequence data_check;`
		`int x;`

```
    a ##1 (!a, x = data_in) ##1
    !b[*0:$] ##1 b &&
    (data_out == x);
endsequence
property data_check_p;
    int x;
    a ##1 (!a, x = data_in) ##1
    !b[*0:$] ##1 b |->
    (data_out == x);
endproperty
property fifo_overflow;
    int cnt;
    not ((empty, cnt=0) ##0 (1'b1,
    cnt = cnt+push-pop)[*0:$] ##1
    cnt==8)
endproperty
```

> Local variable assignments used along with sequence methods (`matched`, `triggered`) are not supported.

Property gated clock

	Yes*
	*See stipulations in the next column.
		Defining a property gated clock using a wire declaration can result in a clock glitch that can produce a wrong proof result. Use the clock event operator `iff` instead.
		Example:
		Produces clock glitch:
		`wire local_clk = clk && clock_en;`
		Does not produce clock glitch:
		`gated_clk: assert property ( @(posedge clk iff clock_en) disable iff`
		`( !clock_en ) x && !y |-> strong( ##[1:$] y ) );`

Recursive properties		
	No	The tool does not support directly or mutually recursive properties.
Sequence properties		
	Yes	In SystemVerilog 2009 and SystemVerilog 2012, sequences can be either weakly embedded or strongly embedded. For example, in `r|=>weak(s)`, `s` is weakly embedded, and in `r|=>strong(s)`, `s` is strongly embedded. By default, without the operators `weak` and `strong`, a sequence is weakly embedded in an assertion or an assumption and strongly embedded in a cover. In SystemVerilog 2005 only strong embedding is available. If you intend to use properties written for 2005 with `analyze -sv09`, you can use `elaborate -sv09_strong_embedding` for backward compatibility between 2009/2012 and 2005 versions. The tool converts SystemVerilog 2005 strong sequence expressions into weak ones with `elaborate -weaken_embedding`, which is the opposite of `-sv09_strong_embedding`.
Verification statements in initial blocks		

	No	The tool ignores verification statements (`assert`, `assume`, or `cover`) in initial blocks along with everything else in initial blocks, and it issues a warning.

Jasper Tcl Operators

The table below clarifies SVA temporal operators support in Tcl commands.

Item	Description/Additional Information
`$dff(data, clock, reset_condition = 0, reset_value = 0)`	

	This expression creates a flip-flop, the width of which is determined by the size of the `data` expression.
	• `data`: Any expression.
	• `clock`: Expression with 1 bit.
	The tool errors out if the expression size > 1.
	• `reset_condition`: Expression with 1 bit. Default value is `1'b0`. If expression size is larger than 1, the expression is `or-reduced` to 1 bit.
	• `reset_value`: Expression with *N* bits. *N* is the width of the `data` expression. Default value is 0. If the expression is larger than *N*, it is truncated to *N*. If the expression is shorter than *N*, it is `sign-expanded` to *N*.
	It is not possible to specify the reset condition without a reset value.
	Examples:
	`assert {$dff(d, clk, rst, 1'b0) == q}`
	`cover {$dff(d, clk, {1'b0, rst, 1'b0}, 1'b0) == q}`
	`assume {$dff({d1,d2,d4}, clk, rst, 3'b000) == q}`

`$dlatch(data, enable)`	

	This expression creates a latch, the width of which is determined by the size of the `data` expression. • `data`: Any expression. • `enable`: Expression with 1 bit. If the expression size is larger than 1, the expression is `or-reduced` to 1 bit. Example: `assert {en1 ##0 1[*] |-> l1 == $dlatch(ld1, en1) }`

Additional Support for SystemVerilog 2009 and 2012

The following table lists various clarifications, exceptions, and limitations in SVA support. In this section, you can find a list of the additional support available if you use `analyze -sv09` or `analyze -sv12`.

Item	Additional Information
`case` statements inside properties	

For example:

```
property P (logic [1:0] delay);
  @(posedge clk)
  case (delay)
    2'd0: a |-> b;
    2'd1: a |-> ##2 b;
    2'd2: a |-> ##4 b;
    2'd3: a |-> ##8 b;
    default: 0;
  endcase
endproperty
```

`restrict` statements

`.triggered`

`let`

`##[*]` and `##[+]`

```
expr implies expr
expr iff expr
```

`nexttime`
`s_nexttime`
`s_always`
`until`
`s_until`
`until_with`
`s_until_with`
`s_eventually`
`eventually`
`reject_on`
`sync_reject_on`
`accept_on`
`sync_accept_on`

`default disable iff`
Sequence properties

In SystemVerilog 2009 and SystemVerilog 2012, sequences can be either weakly embedded or strongly embedded. For example, in `r|=>weak(s)`, `s` is weakly embedded, and in `r|=>strong(s)`, `s` is strongly embedded. By default, without the operators `weak` and `strong`, a sequence is weakly embedded in an assertion or an assumption and strongly embedded in a cover.

In SystemVerilog 2005 only strong embedding is available. If you intend to use properties written for 2005 with `analyze -sv09`, you can use `elaborate -sv09_strong_embedding` for backward compatibility between 2009 or 2012 and 2005 versions.

> The tool converts SystemVerilog 2005 strong sequence expressions into weak ones with `elaborate -weaken_embedding`, which is the opposite of `-sv09_strong_embedding`.

C

Appendix C: Liveness vs. Safety

Jasper supports both safety and liveness properties. Safety properties are common, stating that some condition must always be fulfilled or that some condition must never happen in some finite time window. A liveness property uses SystemVerilog Assertions (SVA) operators like `s_eventually` or `s_until` to state that some condition must occur at some unspecified time in the future, thus requiring the tool to analyze potentially infinite traces.

Look at the following two properties:

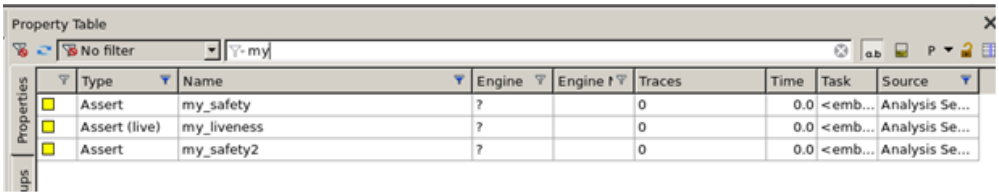
- Property A (safety): After start is active, busy must be active for three consecutive cycles.
  ```
  start |=> busy[*3]
  ```
- Property B: After start is active, done must eventually become active.
  ```
  start |=> s_eventually done
  ```

Property A is a safety property since it states a condition that must always be fulfilled within a finite 3-cycle window. Property B, on the other hand, is a liveness property since it states that the end signal must be active at some unspecified time in the future.

In general, safety properties are much easier for Jasper engines to handle efficiently, and for you to debug, so we recommend that you emphasize the use of safety rather than liveness properties.

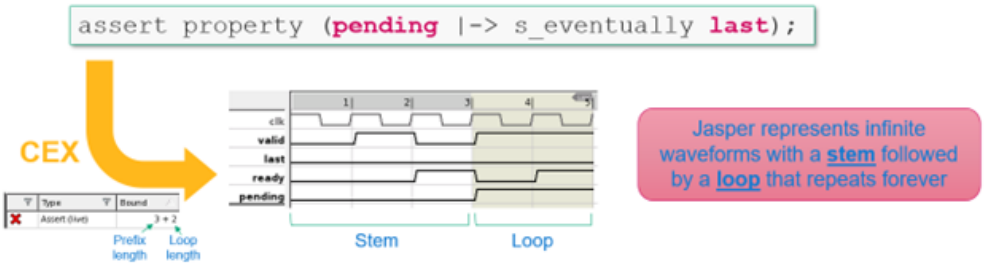
In the Jasper property table, you can always recognize liveness assertions because their type is shown as `Assert (live)`, rather than just `Assert` (see below).

Liveness Assertion in the Property Table



An important difference between liveness and safety properties is that when a safety property is false, each counterexample is finite, but a liveness counterexample must be infinite. This is because, to show something will never eventually happen, the tool must show how to enter an infinite loop in which that event does not occur. In the Visualize GUI, this is represented by a counterexample that contains a "stem", an unshaded waveform that occurs once, followed by a "loop", a shaded area that is understood to repeat forever (see below).

Liveness Counterexample



The Visualize GUI also offers the right-click option to "Unroll Loop" in case you want to see more iterations of the repetition, to better understand the trace, though logically these additional iterations are redundant.

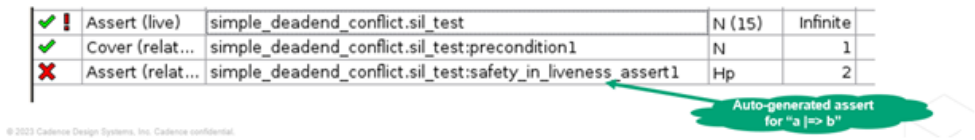
For inconclusive proofs that do not converge or generate a counterexample, the analysis depth shown in the Property Table for a liveness property will always be in parentheses, showing that it does not indicate a bounded proof. Unlike safety properties, an engine exploring to depth `<n>` cannot definitively prove there is no counterexample that can be displayed within `<n>` cycles. This is another reason why liveness properties are more difficult to work with than safety.

It is also possible to state an assertion that combines safety and liveness, such as:

```
sil_test : assert {a |=> b #-# s_eventually c};
```

Here the `a |=> b` component is a safety property, as it can be falsified by a finite trace that proceeds one clock cycle beyond `a`. However, if that aspect is not violated, an infinite counterexample would be needed to disprove the eventuality condition. (We recommend separating such properties into distinct safety and liveness ones rather than writing them in this style, for more efficient verification and debug.) If such a combined assertion is used, Jasper generates a `safety_in_liveness` related assertion, to enable quick discovery and debug of a safety counterexample (see below).

Example of `safety_in_liveness` Related Assertion



Note that the table entry for the assertion itself, with the `Assert (live)` annotation, is just checking the liveness component of the assertion. Success or failure of the safety component is shown only in the `safety_in_liveness` related assertion.

If using liveness assertions, you should be aware of liveness-related options that should be added to several other commonly used commands:

- `elaborate … -create_related_covers {… infinite_precondition}`: This adds related covers to check that an infinite trace is possible after an assertion precondition. This is critical to check, because if an infinite trace is not possible, a liveness assertion is considered vacuously `true`.
- `elaborate … -create_related_covers {… infinite_contrapositive}`: Contrapositive related covers are another type of related cover, which is less commonly used but can reveal some different types of vacuity traces. If you are using the `-create_related_covers {… contrapositive}` flag and also have liveness assertions, you will usually want to add `infinite_contrapositive` as well. See the elaborate command for more information on the contrapositives.
- `check_assumptions … -live`: This checks that every liveness assumption (also called "fairness" assumptions) can be true infinitely often. Again, this is critical for liveness to avoid possible overall vacuity in the proof environment.
- `hunt`: If using the `hunt` command, there are certain strategies that specifically support liveness: `loop_swarm` and `cycle_swarm`. Using `hunt -auto` automatically includes liveness-supporting engines in the default configuration.

One other consideration related to liveness is that pre-2009 SystemVerilog standards allowed liveness properties to be expressed as open time windows, such as `##[0:$]`. For example:

```
liveness_prop_2005: assert property (start |=> ##[0:$] done)
```

Starting in the 2009 standard, such expressions are not evaluated as liveness by default. During elaboration, Jasper generates a WOBS002 warning, "Weakly embedded unbounded sequence in an ASSERT directive." If you see this warning, you should rewrite the property to use an `s_eventually` operator, as in our example at the start of this section:

```
liveness_prop_2005: start |=> s_eventually done
```

Alternatively, you can use the `elaborate` option

```
-sv09_strong_embedding
```

to transform all such open time window properties into liveness.

In general, if you are using liveness properties, there are many methodology considerations that are slightly different than the usual methods for safety properties. You can learn more about tools and methodology for liveness in the Jasper Liveness Rapid Adoption Kit (RAK), available at the Cadence Application Support and Knowledge portal.

D

Appendix D: Formal Property Verification Proof Setup

This appendix includes a summary of proof setup commands to guide you as you initiate your verification session. For each command listed, you will find a short description and the following information:

- **Task column** – Indicates context sensitivity; for example, only applies to the current task. In some cases, this column provides a way to make the command context sensitive.
- **Global column** – Indicates whether the command applies to the global environment. In some cases, this column provides a way to make the command global.

To set the task using the command line, refer to "task".

- For more help with a command, click on the hyperlinked cross-reference, which takes you to the complete command reference.
- Also see the *Jasper Engine Selection Guide* (*Help – Application Guides*).
- Also see the *Jasper ProofGrid and ProofGrid Manager Guide* (*Help – Application Guides*).
- Also see `set_prove_orchestration` to specify that you want the tool to optimize the proof dynamically, by applying learning techniques to tune the list of engines, property order, and per-property time limits.
- Also see `hunt` to learn how you can hunt for counterexamples using formal verification. And refer to "Deep Bug Hunting" for an introduction to the bug hunting feature, strategies, and the command flow.

Command		Task		Global
abstract				

Create a counter, initial-value, register-value, reset-value, or CDC abstraction.		Default		Use the `-env` switch.
`assert`				
Add an assertion about the signals in the design to the current task.		Only		N/A
`assume`				
Add an assumption about the signals in the design to the current task or all tasks. The `clock` and `reset` analyses only consider global assumptions.		Default		Use the `-env` switch.
`clock`				
Specify a clock-related configuration. The `clock` command only considers global assumptions and stopats.		N/A		Only
`configure`				
List or unlimit the current configuration or change it to default values.		N/A		Only
`cover`				
Add a cover directive for signals in the design to the current task.		Only		N/A
`custom_engine`				
Manage custom engines in the current session.		N/A		Only
`get_needed_assumptions`				

Find the irreducible set of assumptions to achieve a target status.		Only		N/A
`prove`				
Prove a single property or multiple properties with respect to a task.		Default when you use `prove` *<expr>*		Use the `-all` switch to prove all assertions and covers in all tasks.
`reset`				
Specify the design reset condition. The `reset` command only considers global assumptions and stopats.		N/A		Only
`sanity_check`				
Facilitate the successful setup of the global environment. You might need multiple iterations of this interactive command to reach a proper setup.		Use the `-task` *<task_name>* switch.		Default
`set_allow_latch_enable_loops`				
Assume all combinational loops through latch-enable pins are stable. Unstable combinational loops handled by the `true` option might lead to wrong proof results. Default: `true`		N/A		Only
`set_auto_disable_related_covers`				

Configure the tool to automatically disable related covers. Default: `true`		N/A		Only
`set_cache_proof_simplification`				
Enable or disable proof simplification caching. Use the `clear -cache_proof_simplification` command to clear the cache. Default: `off`		N/A		Only
`set_elaborate_quiet_trace`				
Specify whether `elaborate` automatically generates global QuietTrace soft constraints. Default: `off` See `assume` (`-soft -quiet`, `-soft -noisy`, `-soft -quiet_from`, and `-soft -max_from`).		N/A		Only
`set_engine_algorithm_selection`				
Direct the `prove` command to either train or infer engine configurations automatically (for engines B, N, and AM) for the set of properties processed by the proof. Default: `off`		N/A		Only
`set_engine_algorithm_selection_dir`				
Sets the directory to save models created by engine algorithm selection.		N/A		Only
`set_engine_mode`				
Optimize the proof process for different types of designs and requirements. Specify a single engine or use curly braces to specify parallel-engine processing. Default: `{Hp Ht N B}`		N/A		Only
`set_engine_solver`				

Specify the solver you want the engines to use in the decision procedure for the Boolean satisfiability problem. Default: `default`		N/A		Only
`set_engine_threads`				
Specify the upper bound on the number of threads used by each engine. Using more than one engine thread can be especially beneficial when working with engines B, Ht, or L on properties that otherwise do not converge; that is, multiple threads can be especially useful with these engines when you are in a bug-hunting mode. Default: `1`		N/A		Only
`set_engineB_before_first_mode`				
Specify how engine B will treat any cycles before its first trace attempt. Default: `skip`		N/A		Only
`set_engineB_before_increment_mode`				
Specify how engine B will treat any cycles between consecutive trace attempts. Default: `skip`		N/A		Only
`set_engineB_first_trace_attempt`				
Specify the cycle at which engine B will make its first trace attempt. Default: `1`		N/A		Only
`set_engineB_trace_attempt_increment`				
Specify the increment engine B uses between one trace attempt and the next. Default: at every consecutive length (`1`)		N/A		Only
`set_engineB_trace_attempt_time_limit`				

Specify a trace attempt time limit for engine B. Default: `0` (unlimited)		N/A		Only
`set_engineB_trace_attempt_time_limit_incr`				
Specify the time limit increment for engine B when the specified limit for trace attempts expires. Default: `0` (unlimited)		N/A		Only
`set_engineBm_first_trace_attempt`				
Specify the cycle at which engine Bm will make its first trace attempt. Default: `1`		N/A		Only
`set_engineBm_trace_attempt_increment`				
Specify the increment engine Bm uses between one trace attempt and the next. Default: at every consecutive length (`1`)		N/A		Only
`set_engineBm_trace_attempt_time_limit`				
Specify a trace attempt time limit for engine Bm. Default: `0` (unlimited)		N/A		Only
`set_engineBm_trace_attempt_time_limit_incr`				
Specify the time limit increment for engine Bm when the specified limit for trace attempts expires. Default: `0` (unlimited)		N/A		Only
`set_engineC_optimization`				

Specify the optimization mode for engine C: • `static` – Use the standard engine C ordering. • `dynamic` – Use engine C2, which has a different engine ordering scheme. • `adaptive` – Predict the best engine (engine C or C2) for the current design and use it. Default: `static`		N/A		Only
`set_engineCG_max_mem`				
Specify the memory limit for engines C and G data structures. • It is best to set the memory limit to a value greater than 100. • Engines C and G jobs will consume more memory than `engineCG_max_mem`. Default: `4096MB`.		N/A		Only
`set_engineD_optimization`				
Specify the optimization mode for engines D and I. • `standard` – Do not optimize engines D and I. • `high` – For every proof attempt made by engines D and I, simplify the task of verification to benefit successive attempts. Default: `standard`		N/A		Only
`set_engineG_optimization`				

Specify the optimization mode for engine G: • `static` – Use the standard engine G ordering. • `dynamic` – Use engine G2, which has a different engine ordering scheme. • `adaptive` – Predict the best engine (engine G or G2) for the current design and use it. Default: `static`		N/A		Only
`set_engineH_before_first_mode`				
Set how engine H treats undetermined cycles before its first trace attempt. Default: `skip`		N/A		Only
`set_engineH_before_increment_mode`				
Set how engine H treats undetermined cycles between consecutive trace attempts. Default: `skip`		N/A		Only
`set_engineH_trace_attempt_increment`				
Specify the increment engine H uses between one trace attempt and the next. Default: `1`		N/A		Only
`set_engineH_trace_attempt_time_limit`				
Set a trace attempt time limit for engine H. Default: `0`		N/A		Only
`set_engineH_trace_attempt_time_limit_incr`				
Set a trace attempt time limit increment for engine H. Default: `0s`		N/A		Only
`set_engineH_first_proof_attempt`				

Specify the cycle at which engines H, Hp, and Hps make their first proof attempt. Default: `1`		N/A		Only
`set_engineH_first_trace_attempt`				
Specify the cycle at which engine Ht, Hts, and Hps will make their first trace attempt. Default: `1`		N/A		Only
`set_engineH_last_proof_attempt`				
Specify the cycle at which engines H, Hp, and Hps make their last proof attempt. Default: `0` (unlimited)		N/A		Only
`set_engineJ_attempts`				
Define the maximum number of attempts engine J will make at every cycle to satisfy all assumptions, Visualize configurations, and latch equations. If engine J fails to satisfy all assumptions within this bound, it restarts. Default: 1000 attempts per cycle		N/A		Only
`set_engineJ_max_trace_length`				
Change the default maximum trace length for engine J proofs. Default: 200 cycles		N/A		Only
`set_engineJ_migrate`				
Enable (or disable) engine job handover to engine Q3 if the engineJ_attempts limit is exceeded. Default: `off`		N/A		Only
`set_engineJ_restarts`				

Define the number of restarts engine J makes before abandoning the proof. A restart means starting again with trace attempt 1 and upwards re-searching for short traces. Two events trigger restart: • Failing to satisfy all constraints at a cycle within the specified number of attempts • Reaching `max_trace_length` Default: 1000 times		N/A		Only
`set_engineJ_seed`				
Supply a hexadecimal setting for the seed that determines how engine J searches for traces. 0 allows engine J to pick an indeterministic seed. Default: `000000000000000e`		N/A		Only
`set_engineJ_single_property`				
Control whether engine J should prove all properties within a task in parallel (`false`) or one property at a time (`true`). Default: `false`		N/A		Only
`set_engineL_asserts_are_intermediate`				
Specify whether asserts should be treated as sources of intermediate states. Default: `on`		N/A		Only
`set_engineL_cache`				
Specify whether to cache cover points for engine L. Use the `clear -engineL_cache` command to clear the cache. Default: `off` See also `clear`.		N/A		Only
`set_engineL_generate_trails`				

Specify whether engine L will generate trails, that is, the sequence of intermediate properties used to find a trace for a particular property. Default: `on`		N/A		Only
`set_engineL_max_segment_length`				
Specify the segment length limit. Default: `0` (unlimited)		N/A		Only
`set_engineL_random_diversification`				
Specify whether engine L will find different traces for the same property to diversify start states for other properties Default: `on`		N/A		Only
`set_engineL_search_mode`				
Specify whether engine L will target properties in order or in parallel. Default: `parallel`		N/A		Only
`set_engineL_seed`				
Specify a hexadecimal setting as the seed for engine L. Default: `00000001`		N/A		Only
`set_engineL_segment_time_limit`				
Specify the engine L segment time limit. Default: `2h`		N/A		Only
`set_engineL_state_removal`				
Specify whether engine L should pick start states based on the last-found start states (`fifo`) or the start states that have been most fruitful (`lru`). Default: `lru`		N/A		Only

`set_engineL_tail_length`				
Specify how many cycles before the end of the found traces the new search will start. Default: 1 cycle		N/A		Only
`set_engineM_optimization`				
Specify whether to enable engine M optimization. Default: `on`		N/A		Only
`set_engineQ3_max_trace_length`				
Specify the trace length upper limit for searches by engine Q3 when diversification is on. Default: 200		N/A		Only
`set_engineQ3_random_diversification`				
Enable random diversification for engine Q3. Default: `on`		N/A		Only
`set_engineQ3_random_diversification_factor`				
Specify the random diversification factor used by engine Q3. Default: `.5`		N/A		Only
`set_engineQ3_restarts`				
Limit engine Q3 restarts to the specified non-negative integer attempts. Default: 1000		N/A		Only
`set_engineQ3_seed`				
Specify a hexadecimal setting as the seed for engine Q3. Default: `00000007`		N/A		Only
`set_engineQT_attempt_time_limit`				

Specify the time limit for each engine QT quiet attempt. Default: `0` (unlimited)		N/A		Only
`set_engineQT_optimization`				
Specify whether to enable engine QT optimization. Default: `on`		N/A		Only
`set_engineU_max_trace_length`				
Specify the trace length upper limit for searches by engine U when diversification is `on`. Default: `100`		N/A		Only
`set_engineU_seed`				
Specify a hexadecimal setting as the seed for engine U. Default: `00000001`		N/A		Only
`set_engineU_trace_attempt_time_limit`				
Specify a trace attempt time limit for engine U. Default: `0` (unlimited)		N/A		Only
`set_engineU2_max_trace_length`				
Specify the trace length upper limit for searches by engine U2. Default: `200`		N/A		Only
`set_engineU2_restarts`				
Limit engine U2 restarts to the specified non-negative integer attempts. Default: `1000`		N/A		Only
`set_engineU2_seed`				
Specify a hexadecimal setting as the seed for engine U2. Default: `00000001`		N/A		Only

`set_first_trace_attempt`				
Specify the cycle at which engines will make their first trace attempt. Default: `1`		N/A		Only
`set_functional_loop_time`				
Specify the time limit for the functional loop analysis step in the `break functional` flow. Default: `600s`		N/A		Only
`set_loop_handling`				
Specify how you want the tool to handle loops in the COI of properties. Default: `none`		N/A		Only
`set_loop_handling_fallback`				
Configure the default loop handling fallback method. Default: `skip`		N/A		Only
`set_max_trace_length`				
Specify the maximum length for traces in the Jasper Visualize window. Default: `0` (unlimited)		Use the `-task <task_name>` switch.		Default
`set_memory_aware_proofs`				
Enable or disable Memory Aware Proofs (MAP). The MAP flow is currently supported by the following Jasper Apps: C2RTL, FPV (proof orchestration only), SEC, Superlint, Connectivity, CSR, LPV, RVV		N/A		Only
`set_memory_aware_export_memory_report`				

Enable or disable exporting the Memory Aware Proofs (MAP) memory report. To save the MAP debug report to a specific path, use the following command: `set_memory_aware_memory_report_path <path>.`		N/A		Only
`set_memory_aware_memory_report_path`				
Set the path for the Memory Aware Proofs (MAP) memory report. Use this command when the `set_memory_aware_export_memory_report` option is enabled.		N/A		Only
`set_memory_aware_initial_mem`				
Define the initial memory limit for Memory Aware Proofs (MAP). By default, the value is interpreted in Kibibytes.		N/A		Only
`set_memory_aware_max_mem`				
Define the maximum memory limit for Memory Aware Proofs (MAP). The maximum memory limit must be greater than or equal to the initial and minimum memory limits defined in MAP.		N/A		Only
`set_memory_aware_mem_factor`				
Define the memory scaling factor for Memory Aware Proofs (MAP). The value must be greater than 1.		N/A		Only
`set_memory_aware_min_mem`				
Define the minimum memory limit for Memory Aware Proofs (MAP). The minimum memory limit must be less than or equal to the initial memory limit defined for MAP.		N/A		Only
`set_per_property_simplification`				

Specify whether a property-specific simplification step should run during prove. Default: `off`		N/A		Only
`set_proof_simplification`				
Disable (or re-enable) simplification. Simplification reduces the size of the netlist processed by the engines to achieve better overall performance, particularly with engines I, C, and C2. Default: `on`		N/A		Only
`set_proofgrid_cluster`				
Specify the sequence of hosts on which to spawn proof engine jobs. Default: none		N/A		Only
`set_proofgrid_engineJ_max_jobs`				
Control the number of engine J jobs. • `N` – Use a positive integer `N` to limit the number of engine J jobs below the number of `max_jobs`. • `follow` – Make engine J reduce the number of engine jobs in the same way as other engines do when the number of remaining undetermined properties drops below `max_jobs`. • `start_cap` – Let the number of engine jobs be determined by the global `max_jobs` setting. Default: `start_cap`		N/A		Only
`set_proofgrid_engineL_max_jobs`				

Control the number of engine L jobs. • `N` – Use a positive integer `N` to limit the number of engine L jobs below the number of `max_jobs`. • `follow` – Make engine L reduce the number of engine jobs in the same way as other engines do when the number of remaining undetermined properties drops below `max_jobs`. • `start_cap` – Let the number of engine jobs be determined by the global `max_jobs` setting. Default: `start_cap`		N/A		Only
`set_proofgrid_engineQ3_max_jobs`				
Control the number of engine Q3 jobs. • `N` – Use a positive integer `N` to limit the number of engine Q3 jobs below the number of `max_jobs`. • `follow` – Make engine Q3 reduce the number of engine jobs in the same way as other engines do when the number of remaining undetermined properties drops below `max_jobs`. • `start_cap` – Let the number of engine jobs be determined by the global `max_jobs` setting. Default: `start_cap`		N/A		Only
`set_proofgrid_engineU_max_jobs`				

Control the number of engine U jobs. • `N` – Use a positive integer `N` to limit the number of engine U jobs below the number of `max_jobs`. • `follow` – Make engine U reduce the number of engine jobs in the same way as other engines do when the number of remaining undetermined properties drops below `max_jobs`. • `start_cap` – Let the number of engine jobs be determined by the global `max_jobs` setting. Default: `start_cap`		N/A		Only
`set_proofgrid_engineU2_max_jobs`				
Control the number of engine U2 jobs. • `N` – Use a positive integer `N` to limit the number of engine U2 jobs below the number of `max_jobs`. • `follow` – Make engine U2 reduce the number of engine jobs in the same way as other engines do when the number of remaining undetermined properties drops below `max_jobs`. • `start_cap` – Let the number of engine jobs be determined by the global `max_jobs` setting. Default: `start_cap`		N/A		Only
`set_proofgrid_extra_args`				
Set extra arguments that will be supplied to the active `proofgrid_mode`. Default: `""`		N/A		Only
`set_proofgrid_manager`				

Turn `on` (or `off`) the ProofGrid Manager feature. ProofGrid Manager begins collecting data when you enable it. Activity prior to that point is not captured. For complete data collection, enable it prior to running a prove or Visualize. Default: `off`		N/A		Only
`set_proofgrid_max_jobs`				
Set the limit for jobs used by ProofGrid (when not in local mode). Default: `0` (unlimited)		N/A		Only
`set_proofgrid_max_local_jobs`				
Set the limit for jobs used by ProofGrid when in `local` mode. Default: `0` (unlimited)		N/A		Only
`set_proofgrid_mem_scan`				
Turn `on` memory scan to manage memory budgets. Default: `off`		N/A		Only
`set_proofgrid_mode`				
Specify how multiple proof engine jobs are spawned for `prove` and `visualize`. Default: `local`		N/A		Only
`set_proofgrid_pending_time_limit`				
Set the time limit for a `job pending` warning. Default: `600s`		N/A		Only
`set_proofgrid_per_engine_max_jobs`				

Specify the limit for engine jobs per engine started by ProofGrid when not in `local` mode. Default: `2`		N/A		Only
`set_proofgrid_per_engine_max_jobs_per_session`				
Specify the upper limit for the number of per-engine jobs in analysis sessions. Default: `0` (unlimited)		N/A		Only
`set_proofgrid_per_engine_max_local_jobs`				
Set the limit for engine jobs per engine started by ProofGrid when in `local` mode. Default: `1`		N/A		Only
`set_proofgrid_queue`				
Set the queue name on which to run parallel proofs. Default: `""`		N/A		Only
`set_proofgrid_restarts`				
Specify the number of restarts ProofGrid allows per engine job. Default: `10`		N/A		Only
`set_proofgrid_resubmit_time_limit`				
Specify the time limit for job resubmission. Default: `0` (unlimited)		N/A		Only
`set_proofgrid_shell`				

Set the command used to launch ProofGrid jobs. Default: `"/bin/sh -c 'echo =proofgrid_shell is not defined; false'"` If `proofgrid_mode` is `shell`, you must update the value of `proofgrid_shell` because the default value is merely a placeholder that indicates you need to modify it. The default value will cause the engine jobs to issue a message to the console and then fail; that is, it is not useful for running proofs.		N/A		Only
`set_proofgrid_shell_stop`				
Set the command used to clean up ProofGrid jobs when `proofgrid_mode` is `shell`. Default: `none`		N/A		Only
`set_proofgrid_skip_abandoned`				
Specify whether a restarted job will return to its abandoned target. Default: `false`		N/A		Only
`set_proofgrid_socket_communication`				
Specify whether communication happens over a TCP/IP socket or standard in/out. Default: `on`		N/A		Only
`set_proofmaster`				
Specify whether ProofMaster is enabled. ProofMaster uses machine learning to dynamically orchestrate engine technology during proof (Proof Orchestration) and leverages historical knowledge to better use computational resources (proof caching and PPD). Default: `off`		N/A		Only

`set_proofmaster_dir`				
Specify a location for ProofMaster data. Default: `jgproject`		N/A		Only
`set_proofmaster_initial_dir`				
Import ProofMaster data from an existing directory and uses it immediately. Default: `none`		N/A		Only
`set_proofmaster_max_data_age`				
Delete unmatched proof cache data from the ProofMaster dir after N days. Default: `0` (The tool never deletes proof cache data automatically.)		N/A		Only
`set_property_delay_cycles`				
Specify the number of cycles you would like to delay properties. Default: `0` cycles		N/A		Only
`set_prove_advanced_simplification`				
Perform property-independent model simplification before running the proof command. Default: `off`		N/A		Only
`set_prove_advanced_simplification_time_limit`				
Specify the maximum time spent on the `prove_advanced_simplification` run. Default: `300s`		N/A		Only
`set_prove_cache`				

Specify the use of property proof results caching (used when running `prove` or `visualize`). Default: `off`		N/A		Only
`set_prove_cache_in_parallel`				
Control running proof caching in parallel. Default: `false`		N/A		Only
`set_prove_cache_job_mode`				
Set the grid on which to run the parallel proof caching. Default: `local`		N/A		Only
`set_prove_cache_max_jobs`				
Set the limit for proof cache parallel jobs. Default: `1`		N/A		Only
`set_prove_cache_mode`				
Control the mode of property proof results caching. Default: `task`		N/A		Only
`set_prove_cache_path`				
Specify the property proof results cache directory Default: `none`		N/A		Only
`set_prove_cache_traces`				
Control trace caching and retrieving cached traces during proofs. Default: `false`		N/A		Only
`set_prove_check_loop_time`				
Specify the time limit for the combo loop enumeration step in the realistic flow. Default: `600s`		N/A		Only

`set_prove_clock_optimization`				
Control whether clock optimization is enabled during the proof. Default: `off`		N/A		Only
`set_prove_evaluate_in_trace_mode`				
Specify whether the tool evaluates part of the trace or the full trace when you use `prove -evaluate_in_trace`. Default: `partial`		N/A		Only
`set_prove_invariants_import`				
Enable import of invariants as implicit assumptions. Default: `off`		N/A		Only
`set_prove_no_cover_traces`				
Specify whether to generate cover traces during proofs. Default: `false`		N/A		Only
`set_prove_no_traces`				
Specify whether to generate traces during proofs. Default: `false`		N/A		Only
`set_prove_on_determined`				
Specify a Tcl proc to call when a `prove` command determines a property. Default: `""`		N/A		Only
`set_prove_per_property_max_time_limit`				

Specify the single-property engine time limit for the proof of each property. Default: `0` (unlimited)		N/A		Only
`set_prove_per_property_time_limit`				
Specify the maximum time allowed for proving each property. Default: `1s`		N/A		Only
`set_prove_per_property_time_limit_factor`				
Set the common ratio used for a geometric progression of `prove_per_property_time_limit` or turn off geometric progression by setting the factor to `0`. Default: `10` When used with `set_per_property_time_limit`, the effective limit is the following: `per_property_time_limit * factor ^ iter` See also `set_prove_per_property_time_limit`		N/A		Only
`set_prove_with_soft`				
Instruct proofs to apply any available user-created soft constraints. Default: `off`		N/A		Only
`set_prove_prefer_shortest`				
Control whether `prove` will continue searching for a shorter trace after identifying a non-minimal trace. Default: `off`		N/A		Only
`set_prove_report_mem`				

Specify whether to generate frequent engine job memory consumption messages. Default: `off`		N/A		Only
`set_prove_report_period`				
Set the sampling period (in seconds) for requesting memory reports from proof processes. Default: `10`		N/A		Only
`set_prove_report_task`				
Specify whether the tool includes task names in proof messages. Default: `off`		N/A		Only
`set_prove_start_timer_on_engine_start`				
Start the `prove` command timer only after the first proof job starts. Default: `on`		N/A		Only
`set_prove_target_bound`				
Specify the global target bound for all property proofs. Default: `0` (infinite)		N/A		Only
`set_prove_time_limit`				
Specify the maximum time allowed for a `prove` command. Default: `86400` seconds (24h)		N/A		Only
`set_prove_verbosity`				
Control the amount of information printed to the log file, the GUI *Console* pane, and the GUI *Proof messages* pane. Default: `6`		N/A		Only
`set_proven_directive`				

Specify whether the tool uses proven directives as assumptions. Default: `true`		N/A		Only
`set_prove_stop_on_cex_limit`				
Set the value of the `prove_stop_on_cex_limit` variable. Default: `0` (unlimited)		N/A		Only
`set_prove_stop_on_unreachable_limit`				
Set the value of the variable `prove_stop_on_unreachable_limit`. Default: `0` (unlimited)		N/A		Only
`set_tcl_no_precondition`				
Specify whether the tool generates precondition covers for liveness assertions. Default: `off`		N/A		Only
`set_tcl_related_covers`				
Enable the precondition check for on-the-fly (Tcl) properties. Default: `off`		N/A		Only
`set_trace_extension`				
Extend a trace *N* extra cycles beyond the cycle that triggered a property. Default: `0`		N/A		Only
`set_word_level_reduction`				
Enable word-level transformations or reductions during the proof flow. Default: `off`		N/A		Only
`set_word_level_time_limit`				

Specify the wall clock time limit for the word-level flow model generation step. Default: `300s`		N/A		Only
`stopat`				
Stop traversing a netlist at the specified signals/instances. The `clock` and `reset` analyses only consider global stopats.		Default		Use the `-env` switch.

E

Appendix E: Deep Bug Hunting

While classic formal verification focuses on exhaustively analyzing the state space with a wide search that analyzes one cycle at a time, bug hunting focuses on finding bugs with a deep but sparse search of the state space. It skips cycles and states; thus, it is not an exhaustive search, that is, it is not a full proof.

In general, you will precede bug hunting with your established verification approach (create assertions, add constraints, find bugs, apply abstractions). You will move to bug hunting when you have enough assertions and you are no longer progressing with the proof, that is, you are not getting more proven, cex, or covered properties. Instead, the number of undetermined properties remains the same.

Refer to the following topics in this appendix:

- Overview
- Formal Strategy with Overconstraining
- Bound Swarm Strategy
- Loop Swarm Strategy
- Cycle Swarm Strategy
- State Swarm Strategy
- Trace Swarm Strategy
- Trace Search Strategy
- Guidepointing Strategy
- Simulation Strategy
- "hunt" Command Flow
- Hunt Limitations

- Deadlock Hunting
- Additional Resources

Overview

Bug hunting with the `hunt` command is similar to using the `prove` command in that it runs a formal proof on properties or a task and it honors some of the same configurations, for example, `-task` or `-property`, `-time_limit`, `-bg`, `-from`, and so forth. However, it is different in that it takes a strategy as input, which you can configure separately with the command `hunt -config`, and it allows local overconstraining (similar to Visualize constraints). Another difference relates to the command `hunt -run -force`. You can use this command on selected properties that already have cex or covered status. In these cases, the tool tries to find more traces for any property that is configured to store multiple traces. And unlike `prove -force`, `hunt -run -force` does not clear existing proof results.

> See `assert -set_store_trace` ("Utility Switches") and `cover -set_store_trace` ("Utility Switches") for the commands to configure the number of traces generated and stored by the tool.

The `hunt` command supports a variety of strategies. Each strategy comprises a formal verification flow with its own parameters and settings, including default settings, but you can also customize the settings and use multiple strategies in parallel. The proof and environment configurations are local to each strategy, and any configurations not defined by the strategy are inherited from the task and global environment. This feature also supports save and restore functions so you can re-use strategies.

The `hunt` command also supports distributions for various configurations.

In addition to the strategies described in this appendix, the tool supports an automatic, recommended flow (`hunt -run -auto`) that simplifies the deployment of State Swarm by including helper cover generation and optimization, partitioning, stagnation monitoring, and running Trace Swarm. This feature simplifies running an effective State Swarm strategy with minimal input from you and improved out-of-the-box results. Refer to the command documentation for switch definitions and examples: "Running Hunt".

Improve your `hunt` results using suggestions provided by Expert System. Expert System runs in the background and monitors your normal activity. It provides recommendations for improving your work flow. Begin with the default settings and monitor progress with the ProofGrid Manager. Locate the Expert System buttons in the top right corner of the main window, Click the *Show Me* button for recommendation details and vary your `hunt` configuration based on recommendations from Expert System.

In concert with the `hunt` command, Jasper provides deadlock hunting with the command `check_dlh`. Apply deadlock hunting on blocks inside designs that are known to be deadlock prone. This feature is effective at reproducing known deadlocks, and with some design knowledge, you can expose unknown deadlocks.

The deadlock hunting flow generates helper covers for one or more instances where you want to hunt for deadlocks and generates trap properties that might expose deadlock candidates. From this point, you can run the traditional `hunt` command with the goal of generating counterexamples for trap assertions. If the tool finds counterexamples, you will use `check_dlh -verify` to evaluate whether they are real deadlocks.

Formal Strategy with Overconstraining

The Formal strategy conducts a search using trace-finding engines. The default engine mode is `{Ht B Bm Hts}`, or you can specify a different engine mode with `hunt -run... -engine_mode`. With the Formal strategy, you can take advantage of local overconstraining, that is, an overconstrained proof. This strategy only looks for counterexamples and covered properties. Due to overconstraining, proven and unreachable properties might be invalid.

Formal with Overconstraining reduces the input space with overconstraints to improve performance; however, only cases under constraint are tested. Some common overconstraints include the following:

- Disable agents/masters
- Limit packet/transaction length
- Constrain data bus to a set of values (for example, 0000 or 1111 only)
- Pick values for configuration registers instead of `stopat` plus `assume -constant`

> Overconstraining is also allowed with other strategies, that is, `-add_constraint` is not restricted to the Formal strategy.

Example

The following example commands configure a Formal strategy, incrementally add constraints, and then run the strategy on a specified task to hunt for bugs.

```
# Configure formal strategy

hunt -config -strategy oc -mode formal


# Add constraints to strategy

hunt -config -strategy oc -add_constraint {!req1}

hunt -config -strategy oc -add_constraint {len==4}


# Run proof

hunt -run -strategy oc -task <embedded>

INFO (IHT002): Settings used for this hunt:

    strategy                        = oc

    mode                            = formal

    time_limit                      = 86400s

    engine_mode                     = Ht Bm B Hts
```

Bound Swarm Strategy

The Bound Swarm strategy can search for bugs that you anticipate within a certain bound range, achieving coverage closure or simply increasing confidence that no more bugs exist within a specified range. This strategy is an enhanced version of Cycle Swarm that reaches deep design states with threads that have different starting points. You can run Bound Swarm with multiple engine jobs to work on multiple properties in parallel using single property engines or create multiple proof threads, where each thread works on different cycles in parallel to reduce the overall

scan time. The threads do not have overlapping analyzed cycles, and they communicate determined results to other threads.

Bound swarm is an iterative search mode that starts by analyzing the first cycle in the range until that cycle has either been fully analyzed or the `trace_attempt_time_limit` has expired. After either event, analysis moves to the next cycle. After the tool analyzes the last cycle in the range, the process restarts from the first cycle. The `trace_attempt_time_limit` is initially set low and automatically increases by a set factor with each subsequent iteration.

You can typically achieve the best results using the single-property engine B or B4 when a 1:1 job:property ratio can be maintained within each proof thread.

The Bound Swarm mode accepts engine B, Bm, or B4. The default engine is Bm. Additional default settings include the following:

- `first_trace_attempt`: Sets the lower bound. The tool calculates a sequential list defined by the number of engine jobs/threads. The lower bound is the minimum existing proof bound of the target properties.
- `max_trace_length`: Sets the upper bound. The tool sets it to `first_trace_attempt` +100.

Use the Bound Swarm in your flow as follows:

- At a later verification stage, that is, after broader flows with `prove` or `hunt -auto`
- In a guided flow, after some design analysis

Objectives you might target with this strategy include the following:

- Test RTL updates before check-in
- Run "deep soak" test regressions, for example, weekend runs
- Find paths to deep cover targets for signoff

It is possible to tune the configurations for a bounded cycle analysis, for example, if you know which cycles you want to target, specify `first_trace_attempt` for the lower bound and `max_trace_length` for the upper bound. The run is most efficient with a precise interval.

Example:

```
first_trace_attempt 10max_trace_length 180
```

This bounded cycle analysis enables quick scans over every cycle in the interval to ensure that it scans all cycles and possibly exposes bugs quicker in deeper but less complex cycles. The bounded analysis is able to re-scan high-complexity cycles by iteratively increasing the per cycle time limit and exposing bugs from complex cycles. In addition, the tool explores all cycles exhaustively in the specified interval, interrupting processing only if properties are determined, cycles are fully explored, or it reaches a time limit.

This strategy is able to handle liveness properties.

Example

Refer to the following examples.

```
# Default bound_swarm (BS)

# trace_attempt_time_limit: 1s (1st scan), 10s (2nd scan), 100s (3rd scan)...

# max_jobs: 6; first_trace_attempt: min bound + 1; max_trace_length: min bound + 101

hunt -config -strategy BS -mode bound_swarm


#----------------------

# Bounded interval analysis with incremental exploration. The hunt iteratively

# increases the trace_attempt_time_limit, re-starting with the same configuration

# but giving more time to explore cycles.


hunt -config -strategy bs -mode bound_swarm \

     -max_jobs  4 \

     -first_trace_attempt  1  -max_trace_length  100 \

     -trace_attempt_time_limit 1s  -trace_attempt_time_limit_factor 10


#----------------------

# Custom factor

# trace_attempt_time_limit: 1s (1st scan), 5s (2nd scan), 25s (3rd scan)


hunt -config -strategy BS -mode bound_swarm -trace_attempt_time_limit_factor 5


#----------------------

# Custom interval analysis


hunt -config -strategy BS -mode bound_swarm \
```

```
-first_trace_attempt 65 -max_trace_length 200
```

The following table lists relevant Bound Swarm switches and their valid and default values.

Switch	Valid Values	Default Values	Description
`-trace_attempt_time_limit`			
	Time	1s	Defines the per cycle time limit for the first Bound Swarm iteration.
`-trace_attempt_time_limit_factor`			
	N > 1	10	Defines a factor by which, at each iteration, the value for per cycle time limit is increased.
`-first_trace_attempt`			
	N	Tool calculated	Defines the lower bound that threads use as the starting cycle for exploration. Each thread must have a different `first_trace_attempt`. Based on the specified value, the tool calculates a sequential list based on the number of threads. If you do not specify this switch, the lower bound is the minimum bound already found for the targets.
`-max_trace_length`			
	N	Tool calculated	Defines the upper bound that threads must respect while exploring cycles. If you do not specify this switch, the tool calculates the default as `first_trace_attempt + 100`.
`-engine_mode`			
	B, Bm, B4	Bm	Defines the `engine_mode`. This switch accepts only one engine.
`-max_jobs`			

	N	*	If `max_jobs` is greater than the number of all possible `first_trace_attempt` values or if `max_first_trace_attempt` is smaller than `max_jobs`, the tool computes `per_engine_max_jobs` for each thread and takes `max_jobs` as the max total of spawned engine jobs among all threads. * Memory consumed by engine jobs running with Bound Swarm can increase over time. The rate of increase is influenced by the `max_jobs` setting. Consider reducing `max_jobs` to reduce the rate of memory consumption at deeper cycles.
`-max_first_trace_attempts`			
	N	Max jobs	Defines the number of `first_trace_attempts` that are explored in parallel, each one in a separate proof thread. It allows parallel property processing by the single-property engine jobs.

Loop Swarm Strategy

The Loop Swarm Strategy enables multiple proofs to run in parallel, each one working on different loop lengths. A significant performance boost can often be achieved by considering multiple possibilities running in parallel.

In the default mode, the Loop Swarm strategy uses a predefined built-in configuration which runs properties using a separate loop length for each engine job starting at 1, 2, 3, and so on as follows:

```
hunt -run -strategy <loop_swarm> -property|-task -max_jobs 10
```

In this example, the default strategy runs 10 jobs in parallel, each one considering a different loop length in the range 1, 2, ..., 9, 10.

You can also use a customized strategy in the following way:

```
hunt -config -strategy <name> -mode loop_swarm other_options
```

You can define the loop lengths in three ways using *`other_options`*:

- A fixed list of specific loop lengths: `-loop_length {5 7 12 17 23}`
- A regular sequence: `-loop_length 5 -loop_length_incr 3` → `5, 8, 11, ...`

- A random seed-based distribution: `-loop_length {40%:[10..20] 60%:[50..75]} [-seed $S]`

The following table lists relevant Loop Swarm switches and their valid and default values.

Switch	Valid Values	Default Values	Description
`-engine_mode`			
	B4, B	B	Specifies the engine.
`-max_jobs`			
	N	*	Specifies the number of engine jobs for the hunt search.
`-loop_length`			
	N+	`1`	Specifies the loop length at which the engine will make its trace attempt on each engine job. You can specify a single value, a list of values, or a distribution. When `-loop_length_incr` is defined, `-loop_length` must specify the first loop length to be attempted. When `-loop_length_incr` is undefined, `-loop_length` alone determines the list of all loop lengths to be attempted. If `-loop_length` contains a list of two or more values, or a distribution, `loop_length_incr` is assumed `0`.
`-loop_length_incr`			
	N	`1`	Specifies the increment the tool uses between a loop length and the next one. If it is set to `0`, the list of loop lengths to be attempted is determined directly from `-loop_length`.
`-max_loop_length`			
	N	*	Specifies the maximum value for `loop_length` when `-loop_length_incr` is greater than `0`.
`-max_trace_length`			
	N	`0`	Specifies the limit for the length of traces the tool searches for. You can specify a natural value for `loop_swarm`.

`-seed`	N	`0`	Specifies a hexadecimal value to be used by the tool as a seed to determine a strategy's randomly selected parameters that will be used in a `hunt` run. The parameters randomly selected by the tool are those specified in a list or distribution format in `loop_swarm`, `cycle_swarm`, `state_swarm`, and `simulation` strategies.

See "hunt" for more details and other options associated with this mode

Cycle Swarm Strategy

The Cycle Swarm strategy is also known as B-swarm. This strategy dispatches multiple formal searches over a range of cycles, which you will specify. The Cycle Swarm strategy can use engine B (single-property engine) or Bm (multi-property engine) to scan properties and run formal analysis at advanced cycles, that is, with a different first trace attempt picked for each job.

Example

The following example commands configure a Cycle Swarm strategy and then run the strategy on a specified task to hunt for bugs.

```
# Configure cycle swarm cycles

hunt -config -strategy cs -mode cycle_swarm \

    -max_jobs 6 \

    -first_trace_attempt {30:[5..20] 70:[50..100]}


# Run cycle swarm# Uses engine Bm to look at all properties at the same time

hunt -run -strategy cs -property {P1 P2 P3}


INFO (IHT002): Settings used for this hunt:

    strategy                    = cs

    mode                        = cycle_swarm
```

```
    time_limit                    = 86400s
    max_jobs                      = 6
    engine_mode                   = Bm
    first_trace_attempt           = 6 58 98 17 66 88

# Optional: Reconfigure cycle swarm to use engine B
hunt -config -strategy cs -engine_mode B \
    -per_property_time_limit 1m \
    -per_property_time_limit_factor 10

# Run cycle swarm again# Now using engine B to work on each property at a time
hunt -run -strategy cs -property {P1 P2 P3}
INFO (IHT002): Settings used for this hunt:
    strategy                      = cs
    mode                          = cycle_swarm
    time_limit                    = 86400s
    max_jobs                      = 6
    engine_mode                   = B
    per_property_time_limit       = 60s
    per_property_time_limit_factor = 10
    first_trace_attempt           = 77 52 71 69 5 90
```

The following table lists relevant Cycle Swarm switches and their valid and default values.

Switch	Valid Values	Default Values	Description
-engine_mode			
	B, B4, Bm	Bm	Specifies the engine.

`-max_jobs`			
	N > 0	6	Specifies the number of engine jobs for the hunt search. If you specify the value `0` for this switch, the tool uses the number of all possible `first_trace_attempt` as the value.
`-first_trace_attempt`			
	N > 1	*	Specifies the cycle at which the engine will make its first trace attempt on each engine job. If you do not specify this switch, the tool computes a sequential list of values taking the targets' bounds into account. * All values specified for `first_trace_attempt` must be smaller than the value specified for `max_trace_length`.
`-max_first_trace_attempt`			
	N	*	Determines the maximum number of `first_trace_attempts` to consider in a `hunt -run` (used exclusively or `cycle_swarm` with engine B or B4). * The tool limits the value for `max_first_trace_attempt` to the number of all possible `first_trace_attempt` values and to the `max_jobs` value.
`-trace_attempt_time_limit`			
	Time	*	Specifies a time limit for each trace attempt. * If you do not specify this switch, the default distribution is 100%:[300..600].
`-per_property_time_limit` `-per_property_time_limit_factor`			

	Time N	Tool defined	Specifies the per-property time limit for the engine jobs (for engines B and B4). If you do not specify these switches, the tool uses the current proof environment configuration values. The `hunt` command does not prevent you from independently setting these options. In most cases, if you set the `-per_property_time_limit_factor` option in a strategy without specifying `-per_property_time_limit`, the tool respects the task setting for `per_property_time_limit` (as defined by the configuration command for the `prove_per_property_time_limit` variable. That is, the settings are related but no dependency exists.
`-seed`	N	Tool defined	Specifies a hexadecimal value to be used by the tool as a seed to determine a strategy's parameters randomly selected that will be used in a `hunt` run. If you do not specify this switch, the default value is `0`, which is a special value that tells the tool to select a nondeterministic seed.

State Swarm Strategy

The State Swarm strategy is also known as L-swarm. This strategy dispatches multiple engine L jobs with different configurations. It is possible to tune the configurations, but it is usually not necessary. State Swarm also supports an automatic, recommended flow (`hunt -run -auto`) that simplifies the deployment of State Swarm by including helper cover generation and optimization, partitioning, stagnation monitoring, and running Trace Swarm. This feature simplifies running an effective State Swarm strategy with minimal input from you and improved out-of-the-box results. Refer to the command documentation for switch definitions and examples: "Running Hunt".

To get the best results, you will specify helper covers that expose relevant states of the design and are close to each other, so engine L can go from one cover to the next. In addition, create covers that are diverse, so engine L can increase its chance of reaching one scenario by exploring different ways. The best helper covers we have seen to date are based on event counters and "seen" signals. For example, create covers that count how many packets are outstanding (packets in – packets out) and covers that keep track of which packet types have been seen.

You can use `cover -generate -auto` to automatically generate covers for State Swarm. This

command automatically generates covers based on the RTL structure. By default, the command automatically determines how many covers to generate, but you can use the `-num` switch if you prefer to specify the number of covers. The tool generates covers for FSM signals and based on pattern matching, but you can use the `-type` switch to select only one type of cover, that is covers for FSMs, enables (that is, critical flop and clock enable signals), or for signals matching pre-defined string patterns. Examples of pre-defined patterns are: full, empty, valid, hit, and so forth.

- If you do not specify types, the tool generates covers for all types.
- The naming convention for automatically generated covers is `fsm:*`, `pattern:*`, or `enable:*`.

Example

The following example commands generate covers, configure a State Swarm strategy, configure engine L, and then run the strategy on a specified task to hunt for bugs.

```
# Automatically generate 100 covers

cover -generate -auto -num 100

INFO: Generating 100 covers of type(s): [fsm, pattern] in task <embedded>.

# Configure hunt

hunt -config -strategy ss -mode state_swarm \

     -max_jobs 10 \

     -time_limit 12h


# Optional: Adjust specific engine L configuration

hunt -config -strategy ss \

     -first_trace_attempt {20%:[1] 80%:[2..40]}


# Run hunt

hunt -run -strategy ss -bg -task <embedded>

INFO (IHT002): Settings used for this hunt:

    strategy                      = ss
```

```
mode                            = state_swarm
time_limit                      = 86400s
max_jobs                        = 10
engine_mode                     = L
start_state_count               = 1 1 1 1 1 1 1 1 1 1
tail_length                     = 1 4 2 2 1 1 2 1 2 1
first_trace_attempt             = 1 35 39 19 3 21 2 9 23 1
segment_time_limit              = 1982s 410s 4815s  ...
max_segment_length              = 7 8 50 6 32 36 45 48 9 6
random_diversification_factor = 0.256 0.86 0.36 0.15 ...
```

The following table lists relevant State Swarm switches and their values.

Switch	Valid Values	Default Values	Description
`-max_jobs`			
	N > 0	20	Specifies the number of engine L jobs to be dispatched, each job with specific computed values for `first_trace_attempt`, `start_state_count`, `tail_length`, `segment_time_limit`, `max_segment_length`, and `random_diversification_factor`. If you specify the value 0 for this switch, the tool uses the number of all possible parameter combinations as the value for `max_jobs`.
`-first_trace_attempt`			
	N	*	Specifies the cycle at which the engine will make its first trace attempt. You can specify a list of values or a distribution as the argument. * If you do not specify this switch, the default distribution is {100%:[3..15]}.
`-start_state_count`			

	Start state	1	Specifies the list of start states engine L uses to search for new traces.
`-tail_length`			
	Number of cycles to backtrack	*	Specifies the starting point for a new search that is N cycles before the end of found traces. You can specify a list of values or a distribution as the argument. * If you do not specify this switch, the default distribution is {50%:[1] 30%:[2] 20%:[3..5]}.
`-segment_time_limit`			
	Time	*	Specifies the amount of time the tool spends searching from each start state. The time limit is an integer followed by one of the following (no spaces): s (seconds), m (minutes), h (hours). To remove the limit, use 0s. You can specify a list of values or a distribution as the argument. * If you do not specify this switch and `time_limit` is 0, the default distribution is `{100%:[100..600]}`. If you do not specify this switch and `time_limit` is not 0, the default distribution will be based on `time_limit` value.
`-max_segment_length`			
	Segment length	*	Specifies the number of trace attempts the tool makes from each start state. You can specify a list of values or a distribution as the argument. * If you do not specify this switch, the default distribution is {100%:[50..300]}.
`-random_diversification_factor`			
	Segment length	*	Specifies the random diversification value used by engine L. You can specify a list of values or a distribution as the argument. * If you do not specify this switch, the default distribution is {50%:[0.03..0.15] 45%:[0.15..0.50] 5%:[0.50..0.90]}
`-target_type`			

	Cover, assertion, or liveness assertion	-	Defines the run target by filtering the list of properties specified in `hunt -run` by property type.
`-auto_helper_num`			
	N	300	Specifies the number of helper covers the tool is expected to generate automatically and append to the targets list in the `hunt -auto` flow.
`-auto_helper_type`			
	Type of helper covers	fsm, pattern	Specifies the type of helper covers the tool is expected to generate automatically and append to the targets list in the `hunt -auto` flow.
`-auto_keep_helpers`			
	-	-	Makes the generated helpers visible in the property table.
`-auto_cleanup_time_limit`			
	Time	10s	Specifies the amount of time the tool should spend cleaning up easy helper targets in the `hunt -auto` flow.
`-seed`	N	0	Specifies a hexadecimal value to be used by the tool as a seed to determine a strategy's parameters randomly selected that will be used in a `hunt` run. If you do not specify this switch, the default value is 0, which is a special value that tells the tool to select a nondeterministic seed.
`-stagnation_time_limit`			
	Time	0	Defines the maximum time interval to wait with no determined result in the `hunt -auto` flow.
`-stagnation_action`			
	stop, restart	stop	Specifies the action to take when the tool reaches the stagnation time limit.
`-helper_set_store_trace`			

	1, unlimited	1	Saves multiple traces for internally generated helper covers for the `hunt -auto` flow. If you want to save multiple traces, set to `unlimited`.

Trace Swarm Strategy

The Trace Swarm strategy hunts for bugs from cover traces in a trace queue. This strategy extends bug hunting from the last cycle of traces found by any `prove` and any `hunt` strategy using any specified engine. This strategy offers a broad and effective way to address liveness properties, which are particularly difficult to converge on and which are typically targeted by bug hunting approaches.

Example

The following example commands use State Swarm to find property traces and then configure Trace Swarm to hunt from the traces found by State Swarm.

```
# configure state swarm to find cover traces

hunt -config -strategy covers_ss -mode state_swarm -target_type cover


# configure trace_swarm strategy to hunt from the above traces

hunt -config -strategy liveness_asserts_ts -mode trace_swarm \-target_type assert_liveness -max_jobs
6 -per_trace_time_limit 20s


# run state_swarm strategy using trace_swarm strategy

hunt -run -task . -strategy covers_ss -use_strategy liveness_asserts_ts \-time_limit 50s
```

The following commands result in the same flow described above except that this example uses Simulation as the first strategy instead of State Swarm. The same can be done with the Formal strategy; that is, you can use Formal as the first strategy.

```
hunt -config -strategy sim -mode simulation -target_type cover
```

```
hunt -run -task . -strategy sim -use_strategy ts -time_limit 1h
```

- The helper cover generation and optimization associated with State Swarm is automatically skipped when you combine it with Trace Swarm.
- The `hunt -run -strategy` and `-use_strategy` commands with Trace Swarm do not support differential analysis properties (for example, X-Prop). The tool blocks the command and issues an error.

The following table lists relevant Trace Swarm switches and their valid and default values.

Switch	Valid Values	Default Values	Description
`-engine_mode`			
	engine	{Ht B}	Specifies the engine.
`-max_jobs`			
	N > 0	20	Specifies the number of engine jobs to be used for bug hunting from traces. You might see fewer engine jobs than the specified `max_jobs` if an insufficient number of traces are available.
`-per_engine_max_jobs`			
	N	1	Configures the number of `engine_jobs` per engine to dispatch.
`-tail_length`			
	N	0	Specifies the starting point for a new search that is N cycles before the end of found traces.
`-per_trace_time_limit`			
	N	*	Specifies the time limit for hunting from each trace. * If you do not specify this switch, the default value is the `time_limit` divided by 10, assuming 1m as the lower limit and 20m as the upper limit.

-target_type			
	cover, assertion, or liveness assertion	-	Defines run target by filtering the list of properties specified in `hunt -run` by property type.

You can use the following new commands for the integration of the `trace_swarm` strategy with `prove` and `hunt`:

- **`set_prove_orchestration_with_trace_swarm`**: Control whether or not `trace_swarm` runs along with proof orchestration.
- **`set_prove_orchestration_hunt_trace_swarm_jobs_ratio`**: Control the percentage of proof jobs to be shifted to `trace_swarm`.
- **`set_prove_orchestration_hunt_trace_swarm_start_time_ratio`**: Control the ratio of proof time after which `trace_swarm` runs.
- **`set_prove_orchestration_hunt_trace_swarm_strategy`**: Set the `trace_swarm` strategy that runs along with proof orchestration.

Trace Search Strategy

Although traces serve as evidence of design activity and reachability, they represent a narrow path through a design's reachable state space. The level of assertion checking outside the actual trace itself is unknown. The Trace Search strategy addresses this concern by searching to a specified depth along the entire trace for additional CEX and cover traces. This strategy launches a series of proofs from points along the trace to give a consistent depth analysis. With Trace Search strategy, the tool only analyzes the trace portion beyond the established proof bound.

The Trace Search strategy uses a specified target depth to determine how deep to search around the trace. The default setting is 10. And by default, the spacing between subsequent proofs matches the specified target depth, but you can manually configure the interval cycles. Smaller values might reduce proof complexity but create more proof stages. Larger values reduce the number of proof stages but might increase complexity.

> We recommend focusing on a smaller number of traces that are known to activate key design functions.

The Trace Search strategy also supports searching from a list of cover properties that have traces or a single cover trace. If you specify multiple cover properties, the tool only processes the default trace associated with each cover. If you specify a single property, you can also specify the trace ID for a specific trace you want the tool to process.

> This strategy is able to handle liveness properties with the default engine setup. Differential analysis properties are not allowed with the Trace Search strategy. The tool blocks the command and issues an error.

The following table lists relevant Trace Search switches and their valid and default values.

Switch	Valid Values	Default Values	Description
`-engine_mode`			
	engine	{B Bm}	Specifies the engine.
`-max_jobs`			
	N > 0	20	Specifies the number of engine jobs to be used for bug hunting from traces. You might see fewer engine jobs than the specified `max_jobs` if an insufficient number of traces and cycle intervals are available.
`-target_depth`			
	N > 0	10	Specifies the target depth to search from any point along the trace.
`-interval_cycles`			
	N > 0	Depth of target	Specifies the number of cycles between subsequent search points.

Guidepointing Strategy

The Guidepointing strategy proves a sequence of covers (path) and then runs formal verification at

the end. For this strategy, you will specify the path of covers.

Example

The following example commands configure a Guidepointing path and then run the strategy on specified properties to hunt for bugs.

```
# Configure guidepointing path

hunt -config -strategy gp -mode guidepoint \

    -path {cov1 cov2 cov3 cov4} \

    -engine_mode {B Ht Hp AM}


# Run guidepointing strategy

hunt -run -strategy gp -property {ast1 ast2 ast3}
```

The following table lists relevant Guidepointing switches and their valid and default values.

Switch	Valid Values	Default Values	Description
-tail_length			
	N	0	Specifies the starting point for a new search that is N cycles before the triggered at cycle of each cover trace in covers path.
-engine_mode			
	engine	{Hp Ht B L}	Specifies the engines to be used in both the covers path and in the final step target.
-path	Starting state	-	Specifies the list of covers. The tool targets the covers in the specified order.
-per_engine_max_jobs			

	N	*	Configures the number of engine_jobs per engine during the last phase of `guidepoint`. If you do not specify this switch, the default value is the value set for `proofgrid_per_engine_max_local_jobs`, if in local mode, or `proofgrid_per_engine_max_jobs`, otherwise.

Simulation Strategy

The Deep Bug Hunting simulation strategy runs random simulation engines in a constrained-random walk through states to find bugs. The settings are as follows:

- Five jobs with engine U, but you might also select engines Q3 or J.
- The `max_trace_length` of each job will be randomized between 50 and 1,000 cycles, but you can change the default with the `-max_trace_length` switch.
- The engines can make unlimited restarts, but you can use `-restarts` to specify the number of restarts the engine makes before abandoning the proof.
- The tool selects a nondeterministic seed for hunt reproducibility, but you can use `-seed` and specify a hexadecimal value.

  > Using the same seed does not guarantee the same results due to many randomizations used by the engines, but it improves the chance of getting those results.

Example commands:

```
hunt -config -strategy sim -mode simulation -max_jobs 10

hunt -run -strategy sim -task .
```

> This strategy is able to handle liveness properties only when using engine U.

The following table lists relevant Simulation Strategies switches and their values.

Switch	Valid Values	Default Values	Description
`-engine_mode`			
	engine	{U U2 J Q3}*	Specifies the engines used by this strategy. * If you do not specify this switch, the default value is engine U2.
`-max_jobs`			
	N	6	Specifies the number of threads to be dispatched, each thread with specific computed value for `max_trace_length`. If you specify the value 0 for this switch, the tool uses the number of all possible `max_trace_length` values as the value for `max_jobs`.
`-max_trace_length`			
	N	*	Specifies the limit for the length of traces the tool searches for. * If you do not specify this switch, the default distribution is {50%:[50..200] 50%:[200..1000]}.
`-restarts`			
	N	0 (unlimited)	Specifies the number of restarts the engine makes before abandoning the proof.
`-seed`	N	0	Specifies a hexadecimal value to be used by the tool as a seed to determine a strategy's parameters randomly selected and engine seeds that will be used in a `hunt` run.

For additional information about specifying seeds, refer to set_engineJ_seed and set_engineU_seed.

"hunt" Command Flow

The `hunt` command flow includes the following steps.

1. Define and view a strategy’s settings.

```
hunt -config -strategy strat -mode formal

strat

hunt -config -show strat

Strategy        = strat

Mode            = formal

Default configuration:

engine_mode = Ht Bm Hts B
```

2. (Optional) Change the strategy's configuration.

```
hunt -config -strategy strat -engine_mode {Bm B U}

hunt -config -show strat

Strategy        = strat

Mode            = formal

User configuration:

    engine_mode = Bm B U
```

3. (Optional) Customize the strategy with additional constraints.

```
hunt -config -strategy strat -add_constraint !valid

constraint:0

hunt -config -show strat

Strategy        = strat

Mode            = formal

User configuration:

    engine_mode = Bm B U

Constraints:

    constraint:0: !valid
```

4. Run the strategy on a task.

```
hunt -run -strategy strat -task <embedded>
INFO (IHT002): Settings used for this hunt:
    strategy                      = strat
    mode                          = formal
    time_limit                    = 86400s
    max_jobs                      = 0
    engine_mode                   = Bm B U
...
```

5. Run the strategy on a list of properties.
6. (Optional) Modify the strategy's configuration with `hunt -run`.

```
hunt -run -strategy strat -property {P1 P2 P3} \
     -max_jobs 10 \
     -time_limit 1h \
     -bg
INFO (IHT002): Settings used for this hunt:
    strategy                      = strat
    mode                          = formal
    time_limit                    = 3600s
    max_jobs                      = 10
    engine_mode                   = Bm B U
...
```

Hunt Limitations

Some `hunt` strategies do not support liveness properties due to engine limitations. Refer to the

following table for liveness support status.

Strategy	Liveness Support	Additional Information
Bound Swarm	Yes *See "Additional Information" for restrictions	`bound_swarm` with engine Bm does not support safety properties mixed with liveness properties in the target: • If the target contains both safety and liveness properties, the tool ignores the liveness properties. • If the target contains liveness properties only, the tool processes them.
Cycle Swarm	Yes *See "Additional Information" for restrictions	`cycle_swarm` with engine Bm does not support safety properties mixed with liveness properties in the target: • If the target contains both safety and liveness properties, the tool ignores the liveness properties. • If the target contains liveness properties only, the tool processes them.
Formal	Yes* *See "Additional Information" for restrictions	With default engines
Guidepoint	Yes* *See "Additional Information" for restrictions	With default engines
State Swarm	No	
Simulation	Yes*	Only with engine U
Trace Swarm	Yes* *See "Additional Information" for restrictions	With default engines

Trace Search	Yes* *See "Additional Information" for restrictions	With default engines

As an alternative, you can convert liveness properties to safety properties or run `prove -from` as follows:

1. Store cover traces for critical covers with `cover -set_store_trace`.
2. Run `hunt` on the task.

   > The `hunt` command ignores the liveness properties.

3. Run `prove` on liveness properties using `-from` and specify the trace length minus 10 cycles.

Example:

```
# Store cover traces for critical covers

set_prove_no_cover_traces false

cover -clear_store_trace "*"

cover -set_store_trace "<critical_prop_1>"

cover -set_store_trace "<critical_prop_2>"

cover -set_store_trace "..."

# Prove task, including critical covers -- liveness properties are ignored

hunt -config -strategy ss -mode state_swarm -unset no_cover_traces ...

hunt -run -strategy ss -task <embedded>


# Now backtrack 10 cycles from the end of all critical covers

foreach critical_prop {<critical_prop_1> <critical_prop_2> ...} {

  set from_cycle [expr [get_property_info $critical_prop -list trace_length] - 10]

  prove -property *live* -engine_mode {Ht Hp C I K N AB AD AM} \

        -from $critical_prop -cycle $from_cycle \
```

```
        -time_limit 1h -bg

}
```

Deadlock Hunting

The deadlock hunting flow relies on tool-generated deadlock cover and trap properties. You will use the commands `check_dlh -generate -cover` and `check_dlh -generate -trap` to configure your environment with covers that check for an exit from lack-of-forward-progress states and trap properties created around critical signals in the design to help find deadlocks. By default, the tool generates trap properties for counters, FSMs, and flop-related signals that match a specified pattern list. In addition, the tool analyzes undetermined and unprocessed properties to add helper covers to the target list to hunt for traces.

After the tool produces traces, you can use `check_dlh -verify` to analyze suspicious traces found in the hunt for deadlocks. This command automatically collects and searches traces for deadlocks, evaluating which traces are actual deadlocks and which ones are not. To see the results of the deadlock hunt, use the command `check_dlh -report`, which is similar to `hunt -report` but optimized to provide additional deadlock hunting information.

The general deadlock hunting flow is as follows:

1. Create a `hunt` strategy
2. Generate traps and related covers
3. Create random covers
4. Run a deep bug hunting search
5. For each trace, create an exit cover property and prove the cover

The following examples show the flow with default settings and the flow with a sequential deadlock hunting strategy.

> `check_dlh` is an initial release to gather feedback from early adopters and finalize implementation for an upcoming release. Contact support@cadence.com to provide feedback or comments.

Examples

```
# Verifying Deadlocks with the Default Settings
# -------------------------------------------
# Checking for deadlocks supports a default setting that does not require
# any configuration. The use model is:
analyze requirement.v
elaborate
reset rst
clock clk
task -create deadlock -set
check_dlh -generate -cover
check_dlh -generate -trap
assert -set_store_trace unlimited *
hunt -run -task deadlock
check_dlh -verify

# The set of commands above configures the deadlock environment and uses bug hunting
# to search for deadlocks. It first uses the default "hunt" strategy to
# determine covers and traps previously generated and then verifies the traces
# looking for deadlocks.

# Customizing a Sequential Deadlock Hunting Strategy
# ------------------------------------------------
# You can customize the strategy used to hunt for deadlocks by creating a new
# strategy with the desired mode and then customizing it. The use model is:

analyze requirement.v
elaborate
```

```
reset rst

clock clk

task -create deadlock -set

check_dlh -generate -cover

check_dlh -generate -trap

hunt -config -strategy ST1 -max_jobs 30 -time_limit 2h

hunt -run -strategy ST1 -task deadlock

check_dlh -verify


# The set of commands above hunts using a newly created state swarm strategy ST1,

# which has been customized to change the max number of jobs and the time limit.

# It first determines the covers and traps generated and then verifies the traces

# looking for deadlocks.
```

Additional Resources

For additional information about the deep bug hunting commands, refer to:

- check_dlh
- hunt

For additional information about engine selection, refer to the *Jasper Engine Selection Guide* (*Help – Application Guides*).

F

Appendix F: Custom Radix Files

Jasper Apps supports user-created radix files to define tags to display (instead of the original values) on the waveform and any related schematic and source browser. This feature is particularly useful for defining opcodes for debugging in a processor setting where you want to see the instruction type displayed instead of the bits that form the opcode (and thus avoid manually translating bit patterns into instructions during your debugging exercise). For example, you declare `ADD` for `32'b0101` and `SUB` for `32'b1010xxxx`, and your Visualize trace then displays `ADD` and `SUB` instead of the bit patterns.

- Radix File Format
- Example Radix Files
- Custom Radix JSON File
- Custom Radix Related Documentation

A new custom radix file can overwrite the current signal radix defined by the user or by another custom radix file.

Radix File Format

A radix file consists of definition clauses and use clauses. Definition clauses map values to tags with an optional color field and use clauses map signal names to a defined radix. Use clauses are optional.

Definition Clause Syntax

```
radix <radix_name>
[size <N>]
[base (b | o | d | h)]
begin
    <TAG> <value> [<color>]
    ...
end
```

Entry	Definition
`radix`	Name for the radix that will be used in the GUI displays.
`size` *N*	Number of bits. This entry is optional. If you define the size, it specifies the size for values used in the `TAG <`*value*`>` pairs within the `begin` and `end` block only. If you do not define the size, the tool uses `8` as the default size.
`base` *base*	Defines the radix (base) for the values in the `TAG <`*value*`>` pairs. If you define the base, it specifies the base for values used in the `TAG <`*value*`>` pairs within the `begin` and `end` block only. If you do not define the base, the tool uses binary as the default base.
`begin`	Marks the beginning of the definition clause block.
TAG	A label that appears in the GUI in place of the specified *value*, for example, an opcode. If you do not specify a tag for some values, the GUIs display the default radix: • For a signal vector that is greater than 4-bit, hexadecimal is the default base. • For a signal vector that is 4-bit or less, binary is the default base.

value	Value specified in the defined base. • Values are zero-extended if the size is not completely specified. For example, `10` is represented by `00000010` if the size is `8` and the base is binary. • A value must be matched by only one tag. • "x" is allowed as a wildcard, matching a single-digit value. No wildcards are supported in decimal base.

color	Highlight for specified values in Visualize waveforms.
	Definitions can be mixed to allow applying highlights for a subset of the radix values as shown below.
	`radix test_bin`
	`size 16`
	`base b`
	`begin`
	`t1 1000001`
	`t2 100000001000001 orange // Visualize highlights this value`
	`end`
	Jasper uses the Qt predefined color service. See the table below.

`end`	Marks the end of the definition clause block.

Qt Color	Hex Color Name and Code
white	White #ffffff
black	Black #000000
red	Red #ff0000
darkRed	Dark red #800000

green	Green #00ff00
darkGreen	Dark green #008000
blue	Blue #0000ff
darkBlue	Dark blue #000080
cyan	Cyan #00ffff
darkCyan	Dark cyan #008080
magenta	Magenta #ff00ff
darkMagenta	Dark magenta #800080
yellow	Yellow #ffff00
darkYellow	Dark yellow #808000
gray	Gray #a0a0a4
darkGray	Dark gray #808080
lightGray	Light gray #c0c0c0

Use Clause Syntax

The following shows and defines the syntax for use clauses in radix files.

> Use clauses are optional.

```
use [radix <radix_name> | radix_signal <signal_name>]
begin
    signal <signal_1>[, <signal_2>]
...
end
```

Entry	Definition
`use [radix` *`radix_name`* `| radix_signal` *`signal_name`*`]`	

	The radix you want to apply to listed signals. You can specify the radix with a radix name or signal name: `use radix <radix_name> clause` **or** `use radix_signal <signal_name> clause` When using a radix name, you must have defined it previously in the same file. (Refer to "Definition Clause Syntax".) Do not use wildcards in signal names.
`begin`	Marks the beginning of the use clause block.
`signal signal_1[, signal_2]`	
	The signal or signals that will use the specified radix.
`end`	Marks the end of the use clause block.

Predefined Radix Names

Add the following to your radix file before the use clause to use predefined radix names (`hex`, `signed_hex`, `bin`, `dec`, `signed_dec`, `oct`, `bits_set`, `floating_point`).

Jasper Apps uses the two's complement method to represent the signed decimal notation.

```
radix bin
begin
end

radix hex
begin
end
```

```
  radix dec
  begin
  end

  radix signed_dec
  begin
  end

  radix oct
  begin
  end

  radix bits_set
  begin
  end
```

You can set the radix for signals to one of these predefined radix names in use clauses, for example:

```
use radix dec
begin
  signal <signal_1>, <signal_2>
  signal <signal_3>
end

use radix hex
begin
  signal <signal_h1>, <signal_h2>
  signal <signal_h3>
end
```

Example Radix Files

This section includes examples of radix files:

- Radix File with Tag-Value Definition for Values 0, 1, 2, and 3
- Radix File with Color Definitions

Radix File with Tag-Value Definition for Values 0, 1, 2, and 3

The following example defines the tag-value pairs for values `0`, `1`, `2`, and `3`. Any value that falls outside of that range (`0-3`) will be displayed in the default radix in the GUIs.

```
radix radix_2b

size 3

base d

begin

SIZE_0 0

SIZE_1 1

SIZE_2 2

SIZE_3 3

end
```

Radix File with Color Definitions

The following example shows the contents of a radix file with color definitions. Refer to figure that follows to see the effects of the color definitions.

```
radix test_bin
```

```
size 16

base b

begin

    t1 1000001

    t2 100000001000001 orange

end

use radix test_bin

begin

    signal w16_bin

end

radix test_hex

size 16

base h

begin

    t1 41, 32

    t2 4041 green

end


use radix test_hex

begin

    signal w16_hex

end


radix testb

size 1

base b

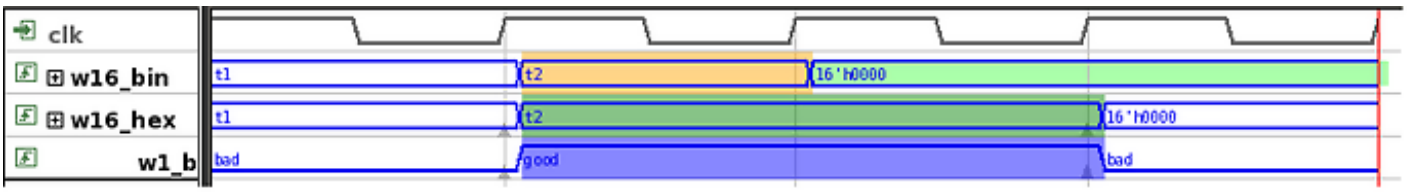
begin

    good 1 blue
```

```
    bad 0
end

use radix testb
begin
    signal w1_b
end
```

Example Custom Radix with Color



Custom Radix JSON File

Use a Custom Radix JSON File to define custom radices and radices usage using the JSON format.

> If the command fails to recognize the file as a JSON format, the file will be parsed as a regular Custom Radix File.

Header Object

The header object contains two mandatory values, the type value and the version value, as shown below.

```
"header" : {
    "type" : "visualize_radix_file",
    "version" : 1
}
```

- The header object is **mandatory**.
- The type value differentiates this type of JSON file from other JSON files. The single possible value for type is `visualize_radix_file`.
- The version value differentiates this JSON file from other future versions.

Radices Object

Radices objects contain all defined radices where the key is the name of the radix and the value contains the radix definition.

```
"radices" : {
    "fp_radix_a" : {
        "type" : "floating_point",
        "mapping" : {
            "16" : {"exponent" : 5, "mantissa" : 10},
            "32" : {"exponent" : 8, "mantissa" : 23},
            "64" : {"exponent" : 11, "mantissa" : 52}
        }
    },
    "fp_radix_b" : {
        "type" : "floating_point",
        "mapping" : {
            "18" : {"exponent" : 6, "mantissa" : 11},
            "34" : {"exponent" : 9, "mantissa" : 24}
        }
    },
    ...
}
```

The radices object is **optional**.

Floating-Point Radix

Floating-point radix lets you create radices with specific configurations following the IEEE 754 standard. You can specify different signal widths with different exponent and mantissa widths. It has two mandatory objects:

- `type`: It holds the type of the radix, and it is "floating_point" for the floating-point radix.
- `mapping`: Contains the floating-point definition for each signal size.

```
          16 bits signal
   +--+---------------+-------------+
   |  |               |             |
   +--+---------------+-------------+
   |15|14            7|6           0
   |  |               +---mantissa (7bits)
   |  +--------exponent (8bits)
   +----sign (1bit)

   "fp_radix_b" : {
       "type" : "floating_point",
       "mapping" : {
           "16" : {"exponent" : 8, "mantissa" : 7},
           "34" : {"exponent" : 9, "mantissa" : 24}
       }
   }
```

Each element in the mapping is an object with a signal width as a key and an object containing the exponent width and mantissa width.

A single floating-point radix can hold several mapping values, one for each size.

- Mapping values must respect the following constraint: `size = exponent+mantissa+1`
- The maximum supported signal size is 64.

You can also define a custom bias to be subtracted from the exponent.

```
"fp_radix_b" : {
  "type" : "floating_point",
  "mapping" : {
      "16" : {"exponent" : 8, "mantissa" : 7, "bias": 32},
      "34" : {"exponent" : 9, "mantissa" : 24, "bias": 1024}
  }
}
```

Fixed-Point Radix

Fixed-point radix lets you create radices with specific configurations.

You can specify different signal widths with different fractional widths and sign. It has two mandatory objects:

- `type`: It holds the type of the radix, and it is "fixed_point" for the fixed-point radix.
- `mapping`: Contains the fixed-point definition for each signal size.

```
           16 bits signal
    +--+---------------+-------------+
    |  |               |             |
    +--+---------------+-------------+
    |15|14            7|6           0
    |                  +---fractional (7bits)
    |
    +----sign (1bit)

    "fp_radix_b" : {
        "type" : "fixed_point",
        "mapping" : {
            "16" : {"fractional" : 8, "sign" : false},
            "34" : {"fractional" : 9, "sign" : true}
        }
    }
```

Each element in the mapping is an object with a signal width as a key and an object containing the fractional width and whether the radix is signed.

A single fixed-point radix can hold several mapping values, one for each size.

- Mapping values must respect the following constraints:
  - `signed: size >= fractional + 1`
  - `unsigned: size >= fractional`
- The maximum supported signal size is 64.

Usages Object

The usages object maps a signal to a specific radix. This radix can be any radix already defined or a built-in radix such as `dec`, `hex`, `bin`, and so on.

```
"usages" : {
    "signal_a" : "hex",
    "signal_b" : "fp_radix_a",
    "signal_c" : "dec",
    ...
    "signal_z" : "fp_radix_b"
}
```

- The usages object is **optional**.
- Use the full hierarchical path when specifying a signal.

Custom Radix Related Documentation

Refer to the following sections or additional resources for information about related commands and GUI options.

- `load_radix_file`
- `visualize` command, see "Utility Switches – General"
- *Jasper Visualize GUI Features* available from the tool (*Help – Application Guides*)

- *Jasper Platform and Formal Property Verification App User Guide* available from the tool (*Help – User Guide*):
  - "Menu bar" in the "Visualize" chapter
  - "Visualize Plot Preferences" in the "GUI Settings" appendix

G

Appendix G: Specifying Enum Values with Tcl

Any signal, parameter, localparameter, or enum declared in the RTL must include its instance hierarchy and scope as a prefix when using Tcl properties or any other Tcl command. However, prior to the 2022.03 release, the `assert`, `assume`, and `cover` commands did not accept enum literals with hierarchy and scope, and specifying enum literals without hierarchy and scope was ambiguous since the literal value was not fully scoped. See the example below:

```
assume {top.dut.mem_control.state == IDLE}
```

The tool might or might not have accepted this command if the literal value for IDLE was unclear.

Beginning with the 2022.03 release, you must fully specify the ENUM literal, that is, you must specify the design hierarchy and the package scope or compilation-unit scope where the literal is defined. If you do not do so, the tool generates an error message when the enum literal cannot be found.

You can define an enum in any of the following ways:

- Inside an Instance
- Within a SystemVerilog Package
- Within a Compilation Scope

Inside an Instance

To specify an ENUM literal defined inside an instance of module, provide the hierarchical path to the literal as shown in the following example:

```
assume {t1.me == t1.E_0}
```

See the related code below:

```
module tb;
```

```
  top t1();
endmodule
module top;
  typedef enum { E_0 =0, E_1 =1} my_enum;
  my_enum me;
endmodule // top
```

Within a SystemVerilog Package

To specify an ENUM literal defined within a SystemVerilog package, reference the package as shown in the following example:

```
assume {t1.me == test_pkg::E_0}
```

See the related code below:

```
package test_pkg;
  typedef enum { E_0 =0, E_1 =1} my_enum;
endpackage // test_pkg
module tb;
  top t1();
endmodule
module top;
  import test_pkg::*;
  my_enum me;
endmodule // top
```

Within a Compilation Scope

To specify an ENUM literal defined within a compilation scope with the default escaped naming,

reference the scope as shown in the following example:

```
assume {t1.me == \$unit_test_compunitscope_sv[]::E_1}
```

> The extra space highlighted in red above is necessary to terminate the escaped name.

See the related code below:

```
package test_pkg;

  typedef enum { E_0 =0, E_1 =1} my_enum;

endpackage // test_pkg

module tb;

    top t1();

endmodule

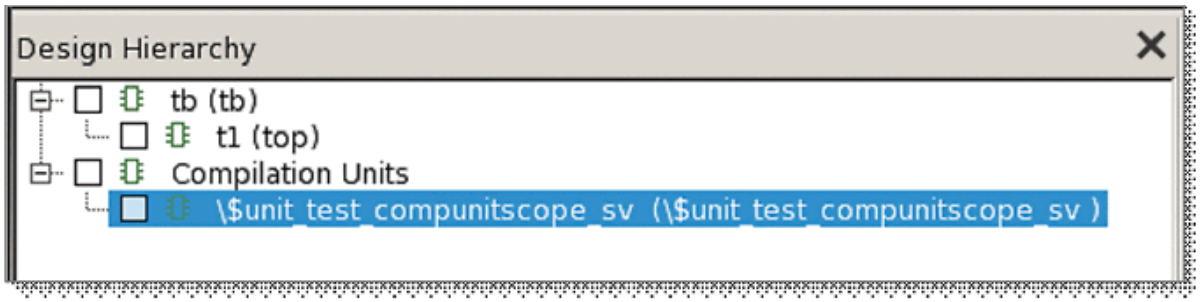
module top;

  import test_pkg::*;

  my_enum me;

endmodule // top
```

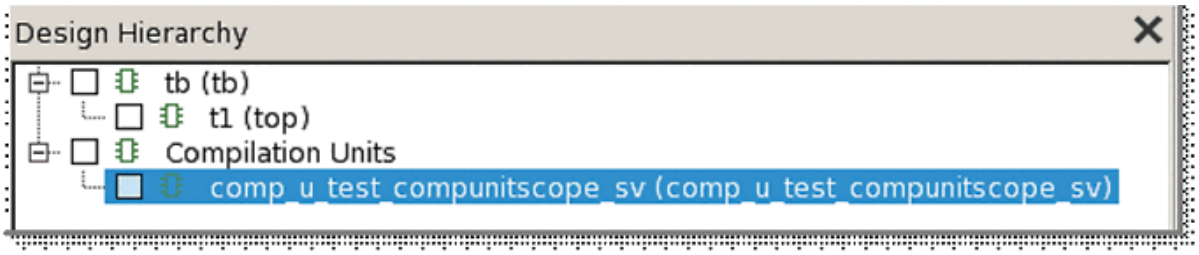
This is displayed in the design hierarchy as follows:



To specify an ENUM literal defined within a compilation scope with a naming prefix defined, reference the scope as shown in the following example:

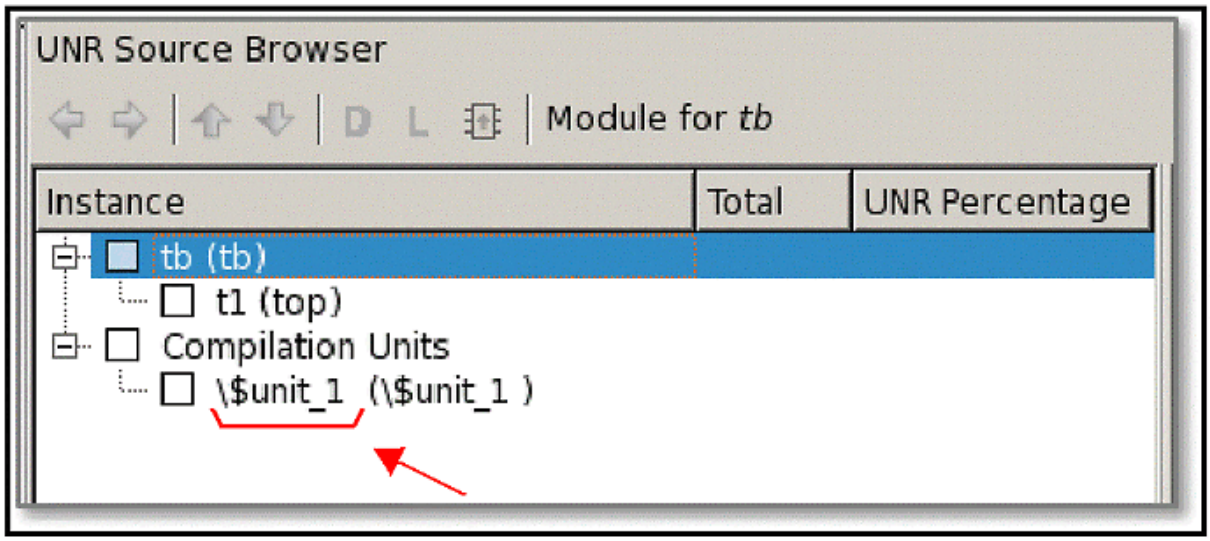
```
assume {t1.me == comp_u_test_compunitscope_sv::E_1}
```

This is displayed in the design hierarchy as follows:



Jasper UNR App Case

In the UNR App, when referencing an ENUM literal defined within the compilation scope, you must use the compilation-unit name assigned by Jasper, as seen in the UNR Source Browser window (see figure below). Do **not** use the Xcelium assigned compilation-unit name in this case.



H

Appendix H: Automatically Extracted Type Properties

Jasper Apps extracts and uses type information in `prove` and `visualize` runs. That is, during elaboration, the tool uses the VHDL type information and SystemVerilog enum variables to create type properties. This appendix includes the following sections:

- Introduction to VHDL Type Properties
- Introduction to SystemVerilog Type Properties
- Procedural Notes
- Type Property Limitations

Introduction to VHDL Type Properties

Jasper Apps synthesizes VHDL signals with integer types into bit vectors during the elaboration phase. The exact number of bits used for the synthesis is the smallest number needed to encode all the values defined by the type declaration. For instance, an integer signal with the range zero to seven will be synthesized into three bits to represent all values. In some cases though, an integer signal will have a range that does not contain an exact power of two elements. A signal with the type zero to six will still require three bits, but there will be one value (namely seven) that can be encoded in the bit vector that the signal should not be able to get according to its type definition. Jasper Apps automatically extracts verification directives that check that such type properties are not violated. This feature is useful for the following main purposes:

- Environment properties – When a type property directive is generated for an input port at the top elaboration level, the tool generates an assumption for that signal that ensures that the port is never driven with a value outside of the type range. This is the only type of integer signal for which the tool generates assumptions. For all other signals, the tool generates assertions instead.

- Bug detection – If a violation of the type property can actually occur, this is a design bug that should be fixed. Type overflows can occur in synthesized versions of the integer signal, and verifying such directives provides a quick sanity check of the design under verification.
- State Space Tunneling (SST) – Values outside the type definition should not be reachable from any legal state of a design. Type properties are therefore a natural starting point for an SST attempt, especially as type violations would likely render a design inoperable.

Just like integer signals, signals of enumeration types will be synthesized as bit vectors. Type properties will therefore be generated for such signals if the number of enumeration literals in the type declaration is not an exact power of two. For instance, directives will be generated for signals of the type "Color" below since it contains three literals, but not for the type "Direction," which contains four.

- Type Color is (Red, Blue, Green)
- Type Direction is (North, East, South, West)

> Jasper Apps does not generate type properties for variables.

Introduction to SystemVerilog Type Properties

SystemVerilog enumerated data types provide the capability to abstractly declare strongly typed variables without either a data type or data value and later adds the required data type and value for designs that require more definition. Enumerated data types also can be easily referenced or displayed using the enumerated names as opposed to the enumerated values.

To avoid the assignment of non-valid values for the enumeration type inputs, the tool creates assumptions that guarantee that only the listed values will be considered. On the other hand, for enumeration type outputs, the generated properties are assertions that check whether, in any situation, an invalid value for the signal can be reached.

The tool automatically extracts type properties for enumerations in which the number of enumeration literals is not an exact power of two. An enumerated type declares a set of integral named constants.

Procedural Notes

By default, the tool extracts VHDL type properties. If you prefer not to generate type properties, elaborate with the `-no_type_properties` switch.

For SystemVerilog, you must explicitly specify extraction by elaborating with the `-enable_sv_type_properties` switch.

With automatic extraction enabled (by default for VHDL or by using `-enable_sv_type_properties` for SystemVerilog), the tool creates the *<constraints>* task for type properties.

- `-no_type_properties` and `-enable_sv_type_properties` are mutually exclusive.
- See `elaborate` for command details.

Type Property Limitations

This section includes a list of known limitations for extracting type properties.

- If you use the VHDL attribute `enum_encoding`, extracted type properties for enums might have the wrong expression.
  Example:

```
attribute enum_encoding : string ;

type my_state is (start, stop, ready, off, warmup);

attribute enum_encoding of my_state : type is "001 010 011 100 111";
```

- The tool might not extract type properties when integer ranges depend on VHDL generics specified with `elaborate -parameter`.
  Example:

```
entity fifo is

  generic (fifo_length : fifo_length := 8);

  port (level    : out integer range 0 to fifo_length);

end fifo;
```

```
analyze -vhdl test.vhd

elaborate -vhdl -top fifo -parameter fifo_length 512
```

- Jasper Apps does not extract type properties from Proof Accelerator modules.
- Because undriven SystemVerilog enums are possible sources of spurious counterexamples in other assertions, the tool creates type assertions for them instead of assumptions.
- Type properties in black-boxed modules or instances might not work as expected.
- Type properties in signals with extended names or escaped identifiers might not work as expected.

Appendix I: Vacuity Checking with Automatically Extracted Covers

By default, Jasper Apps extracts cover properties for PSL and SVA assertions and assumptions that include triggering expressions. These cover properties are called preconditions. If the tool proves that an extracted cover property is unreachable, the related assertion is vacuous, and the property table displays a red exclamation point in the same cell with the property status indicator. In addition to the default precondition covers, you can specify that you want the tool to extract witness and late precondition covers. See "check_assumptions" for task-wide vacuity checking.

This appendix includes the following sections:

- Precondition Reachability Checking
- Refining Vacuity Checking with Additional Covers
- Managing Vacuity Checking
- Enabling and Disabling Related Covers
- Infinite and Finite Precondition Proof by Disabling
- Limitations

Precondition Reachability Checking

Jasper Apps elaborates PSL and SVA properties embedded in design files or in separate verification modules. Assertions and assumptions often use a precondition (triggering expression) to specify when the test expression should be checked. For assertions, if the precondition is unreachable, that is, it never triggers (always false: `1'b0`), the assertion is vacuous. In other words, the property does not check what was intended, and the tool will never find a violation trace regardless of the logic driving the test expression.

By default, Jasper Apps automatically extracts precondition reachability checks for liveness

assertions created with the `assert` command and for embedded PSL and SVA assertions and assumptions. The tool creates additional cover properties for each property (assertion or assumption) where it detects preconditions and adds them to the task in which the original property resides. These cover properties check whether preconditions are indeed reachable based on the related logic and environment constraints. To specify that you do not want the tool to extract precondition reachability checks for embedded properties, use the `elaborate -no_preconditions` command.

The tool names an automatically extracted cover property after the original property with a suffix identifying the precondition type:

- `:preconditionN`
- `:precondition_lateN`
- `:precondition_infiniteN`
- `:witnessN`
- `:contrapositiveN`
- `:infinite_contrapositiveN`

Property Examples

For the following PSL and SVA examples, the tool extracts a cover property equivalent to `req` evaluates to `true`:

- PSL property (Verilog) `assert (always ({req} |=> {ack})) @(posedge clk);`
- PSL property (VHDL) `assert (always ({req = '1'} |=> {ack = '1'})) @(rose (clk));`
- SVA property `assert property (@(posedge clk) req |=> ack);`

SVA Immediate Properties

The following example shows SVA immediate assertions embedded in `always` blocks. The tool extracts cover properties for the assertions `assert1`, `assert2`, and `assert3`.

```
always @(posedge clk) begin
```

```
  if (cond1) begin
    ...
    assert1: assert (test_expr1);
    // The tool extracts a cover property such as "cond1 is asserted".
  end

  else if (cond2)
    ...
    assert2: assert (test_expr2);
    // The tool extracts a cover property such as
    // "cond1 is deasserted and cond2 is asserted".
  end

  else begin
    ...
    assert3: assert (test_expr3);
    // The tool extracts a cover property such as
    // "cond1 and cond2 are deasserted".
  end
end
```

Nested Implications

As shown in the following SVA examples, the tool extracts a separate precondition for every implication in the property:

- For properties of the form `s1|=>(s2|=>P)`, the tool produces two precondition covers, `s1` and `s1##1s2`.
- For sub-properties consisting of nested instances of `if a ... else ...`, the tool takes this context into account. For example, given the following property,

  ```
  a |->
  ```

```
if (b)
    if_b
else if (c)
    if_c
else if (d)
    if_d
else if (e)
    if_e
$display ("something");
```

the tool generates the following set of preconditions and witnesses:

- `witness1 : a ##0 b ##0 if_b`
- `witness2 : a ##0 !b ##0 c ##0 if_c`
- `witness3 : a ##0 !b ##0 !c ##0 d ##0 if_d`
- `witness4 : a ##0 !b ##0 !c ##0 !d ##0 e ##0 if_e`
- `precondition1: a`
- `precondition2: a ##0 b`
- `precondition3: a ##0 !b`
- `precondition4: a ##0 !b ##0 c`
- `precondition5: a ##0 !b ##0 !c`
- `precondition6: a ##0 !b ##0 !c ##0 d`
- `precondition7: a ##0 !b ##0 !c ##0 !d`
- `precondition8: a ##0 !b ##0 !c ##0 !d ##0 e`

Refining Vacuity Checking with Additional Covers

To refine vacuity checking for embedded properties, use the command

`elaborate -create_related_covers...`. This switch accepts a Tcl list that specifies one or more types of cover statements (`precondition`, `late_precondition`, `infinite_precondition`, `witness`, `contrapositive` and `infinite_contrapositive`).

> Using `-create_related_covers` in `-f` files is not supported. It triggers a warning.

The tool creates related covers as follows. In these examples, `s1` and `s2` are sequences.

- The precondition cover of an SVA implication `s1 |=> P` is a finite trace where sequence `s1` matches somewhere on the trace. This is the default.
- The late precondition cover of an SVA implication `s1 |=> s2` is a cover of the sequence (`s1 ##1 1'b1`).
- The infinite precondition cover of an SVA implication `s1 |=> P`, where `P` is a liveness property, is an infinite trace where s1 matches somewhere on the trace.
- The witness cover of an SVA implication `s1 |=> s2`, where `s2` is a sequence, is the cover of the sequence (`s1 ##1 s2`).
- The witness cover of an SVA implication `s1 |-> s2`, where `s2` is a sequence, is the cover of the sequence (`s1 ##0 s2`).
- The contrapositive cover of an SVA implication `s1 |=> P`, is a finite trace where P is not true somewhere on the trace. This is a form of vacuity that might be an indication of underlying design or proof environment setup issues. Jasper creates the contrapositive cover property only when the right hand side of the property is Boolean. The feature does not support sequence/property operators. The following example triggers a contrapositive property only when `y` is Boolean.

  ```
  x |=> y  creates cover (~y)
  ```

- The infinite contrapositive cover of an SVA implication `s1 |-> s_eventually B`, where `B` is a Boolean expression, is an infinite trace where `B` never matches. Jasper automatically creates a related cover that checks if there is a scenario on a liveness property that indicates the RHS never happens.

- The tool's default behavior is equivalent to `elaborate -create_related_covers precondition`.
- It is an error to use `elaborate -no_preconditions` in combination with the switch `-create_related_covers precondition`.
- The `elaborate` switches `-no_preconditions` and `-create_related_covers` do not apply to liveness assertions created with the `assert` command. Instead, use the switch `-no_precondition` with the `assert` command or the command `set_tcl_no_precondition on`.
- The tool does not extract witness precondition covers or late precondition covers for liveness assertions created with the `assert` command.
- The tool does not support witness precondition covers for properties with local variable assignments.
- `:noConflict` can be considered an implicit related property for all other properties. Also `:live` can be considered an implicit related property for all liveness properties. See "check_assumptions".

The recommended engines for optimal precondition reachability checks are the following:

- **Multiple properties** – engine Ht and engine L
- **Single property** – engine B

Managing Vacuity Checking

Overconstraining is a common cause of vacuity (see "check_assumptions"). Apart from that, default preconditions cover the majority of vacuous proof situations; however, in some cases, the basic precondition is reachable but the witness precondition of the same property is not. This kind of vacuous proof can go unnoticed if you rely only on the default preconditions. Furthermore, the tool is able to generate basic precondition covers only for implication properties. Non-implication properties do not have default preconditions, but using the `-create_related_covers witness` switch with `elaborate` can generate witness covers for them and thereby provide coverage information for those properties.

Witness preconditions have the disadvantage of increasing computational overhead, so we do not

recommend using them in all cases. Carefully consider your design and proof environment before taking this step. If the majority of your assertions have implications and your environment has few assumptions, witnesses will probably add little value. However, if your proof environment has many related assumptions or few implication assertions, witness preconditions might greatly improve vacuity detection.

In general, we recommend a mixed-use approach. For daily work and nightly regressions, generate only the default preconditions to get reasonable confidence with fast performance, then, periodically generate the related covers for a deep scan of the proof environment.

Enabling and Disabling Related Covers

By default, if you disable a property, its enabled related covers are also disabled. Hence, related covers can be disabled for two reasons:

- You have explicitly disabled them.
- You have explicitly disabled all their main properties; that is, you have disabled the properties that "own" the related covers.

Exception: Although `assume -from_assert` and `assert -from_assume` disable the original property, the related covers are not disabled in these cases. Instead, they are linked to the new property created by the command.

For all other cases, this is the handling you can expect:

- If you disable the main property, the related covers are also disabled unless you run the command `set_auto_disable_related_covers off`.
- If you subsequently re-enable the main property, its related covers are also re-enabled unless you directly disabled the related covers. In that case, the covers remain disabled regardless of the state of their main properties.
- If you explicitly enable a disabled related cover, it is re-enabled even if the main property is disabled.
- Explicitly enabling a related cover does not automatically enable the main property.

Infinite and Finite Precondition Proof by Disabling

With the current disable semantics for infinite precondition, proof by disabling is treated as a kind of

vacuity.

For the following property:

```
assert property (disable iff (c) a |=> s_eventually b)
```

the tool produces the following infinite precondition check:

```
assert property (disable iff (c) a |=> s_eventually 0)
```

However, since this check is generated as a cover, an `unreachable` result is equivalent to a `proven` status, whereas a `covered` result is equivalent to a `CEX` status.

If the property is proved because the disable condition `c` is true, then the infinite precondition vacuity test will also be proved, and it will return `unreachable`, but a proof by the disable condition is regarded as a form of vacuity.

For the following finite case:

```
assert property (disable iff (c) a |-> b)
```

We generate the following precondition cover:

```
cover property (disable iff (c) a)
```

which (similar to the infinite case) is equivalent to the following:

```
assert property (disable iff (c) a |-> 0)
```

In general, a `disable iff` can make an assertion or an assumption "more true" and a cover "less true". Additionally, Jasper treats precondition covers (both finite and infinite) like ordinary covers in this respect.

Limitations

This section lists limitations in precondition reachability.

Excluded Cases

Jasper Apps does not extract precondition reachability checks for the following cases:

- Properties written in Verilog or VHDL. These properties do not have explicit preconditions.
- Non-liveness assertions you create with the `assert` command.
- Assumptions you create with the `assume` command.

Handling for Nested Implications for Local versus Non-Local Variables

Nested implications that have non-local variables compared to those with local variables result in a different number of precondition covers.

Nested Implication with Non-Local Variables

The tool generates non-local variable preconditions using a complex syntactical analysis that recognizes cases of nested implication with the form shown below:

```
A |-> (B |-> C)
```

For this case, the tool produces two preconditions, one for `A` and one for `A ##0 B`.

Nested Implication with Local Variables

The tool cannot generate local variable preconditions with the same kind of analysis it uses for non-local variables because of possible clashes between different instances of the same variable.

The local variable precondition identifies preconditions late in the analysis flow, so only the main implication (`A ##0 B |-> C`) remains. Therefore, the tool only produces one precondition for this case.

J

Appendix J: Understanding Visualize Semantics with SVA Equivalents

Jasper Apps adds the keywords `BT` (beyond trace) and `RC` (reset cycle) and the Boolean `NOCEX(p)` to the SVA language for the purpose of explaining the meaning of the `visualize` command. This appendix begins with an explanation of those keywords and the Boolean followed by SVA equivalents of `visualize` commands. The topics in this appendix are listed below:

- SystemVerilog BT/RC Keywords and NOCEX Boolean
- SystemVerilog Equivalent of Visualize At Least Once Options
- SystemVerilog Equivalent of Visualize Force Options
- SystemVerilog Equivalent of Visualize Not Options
- SystemVerilog Equivalent of Visualize for a Signal Sequence

SystemVerilog BT/RC Keywords and NOCEX Boolean

This section describes the added SystemVerilog `BT` and `RC` keywords and `NOCEX(p)` Boolean.

SystemVerilog BT Keyword

The SystemVerilog `BT` keyword added by Jasper Apps ensures that the `cover` covers all of the finite trace produced. This is only needed for the `[n:$]` case of `-force`. Refer to SystemVerilog Equivalent of Visualize Force Options.

The following `BT` expression is something that is only true beyond the finite trace considered here. You can think of it as having the meaning `cycle>trace_length` where `cycle` is the number of the

current cycle and `trace_length` is the length of the current trace. This cover will match on an extension (by one cycle) of the finite trace that is produced.

```
cover property ##(n-1) exprA[*0:$] ##1 BT;
```

SystemVerilog RC Keyword

The SystemVerilog `RC` keyword added by Jasper Apps matches the reset cycle, which is the cycle immediately before the trace. In the examples found in this appendix, the `RC` signal (reset cycle signal) is thought of as true at and only at the cycle before the first cycle.

SystemVerilog NOCEX(p) Boolean

The SystemVerilog `NOCEX(p)` Boolean added by Jasper Apps is true at a cycle if and only if there is no counterexample to the property ending in that cycle. For example, where `p = 'A|=>B',NOCEX(p) = !((A ##1 !B)$triggered).`

SystemVerilog Equivalent of Visualize At Least Once Options

The following examples show `visualize -at_least_once`, `visualize -at_least_once -after`, `visualize -at_least_once -before`, and `visualize -at_least_once -after -before` commands with their SVA equivalents when taking expressions as arguments.

Visualize Construct	Equivalent SVA Cover Property
`visualize -at_least_once exprA n:m`	`RC ##[n:m] exprA`
`visualize -at_least_once exprA n:$`	`RC ##[n:$] exprA`
`visualize -at_least_once exprA $:$`	`exprA ##1 BT`
`visualize -at_least_once exprA n:m -after exprB`	`exprB ##[n:m] exprA`
`visualize -at_least_once exprA n:$ -after exprB`	`exprB ##[n:$] exprA`

`visualize -at_least_once exprA n:m -before exprB`	`exprA ##[n:m] exprB`
`visualize -at_least_once exprA n:$ -before exprB`	`exprA ##[n:$] exprB`
`visualize -at_least_once exprA n:m -after exprB -before exprC`	`exprB ##1 exprA[=n:m] ##1 exprC`
`visualize -at_least_once exprA n:$ -after exprB -before exprC`	`exprB ##1 exprA[=n:$] ##1 exprC`

The following examples show `visualize -at_least_once`, `visualize -at_least_once -after`, and `visualize -at_least_once -before` commands with their SVA equivalents when taking sequences as arguments.

Visualize Construct	Equivalent SVA Cover Property
`visualize -at_least_once seqA n:m`	`RC ##[n:m] seqA`
`visualize -at_least_once seqA n:$`	`RC ##[n:$] seqA`
`visualize -at_least_once seqA $:$`	`RC ##[$:$] seqA`
`visualize -at_least_once seqA n:m -after exprB`	`exprB ##[n:m] seqA`
`visualize -at_least_once seqA n:$ -after exprB`	`exprB ##[n:$] seqA`
`visualize -at_least_once seqA n:m -before exprB`	`seqA ##[n:m] exprB`
`visualize -at_least_once seqA n:$ -before exprB`	`seqA ##[n:$] exprB`
`visualize -at_least_once exprA n:m -after exprB -before exprC`	`exprB ##1 exprA[=n:m] ##1 exprC`
`visualize -at_least_once exprA n:$ -after exprB -before exprC`	`exprB ##1 exprA[=n:$] ##1 exprC`

SystemVerilog Equivalent of Visualize Force Options

To help explain `visualize -force`, the following examples show `visualize` commands with their SVA equivalents when taking expressions as arguments.

Visualize Construct	Equivalent SVA Cover Property
`visualize -force exprA n:m`	`RC ##n exprA[*m-(n-1)]`
`visualize -force exprA n:$`	`RC ##n exprA[*1:$] ##1 BT`
`visualize -force exprA $:$`	`exprA ##1 BT`
`visualize -force exprA n:m -after exprB`	`exprB ##n exprA[*m-(n-1)]`
`visualize -force exprA n:$ -after exprB`	`exprB ##n exprA[*1:$] ##1 BT`
`visualize -force exprA n:m -before exprB`	`exprA[*m-(n-1)] ##n exprB`
`visualize -force exprA n:$ -before exprB`	`RC ##1 exprA[*1:$] ##n exprB`

To help explain `visualize -force`, the following examples show `visualize` commands with their SVA equivalents when taking properties as arguments.

Visualize Construct	Equivalent SVA Cover Property
`visualize -force propertyA n:m`	`RC ##n nocex(propertyA)[*m-(n-1)]`
`visualize -force propertyA n:$`	`RC ##n nocex(propertyA)[*1:$] ##1 BT`
`visualize -force propertyA $:$`	`nocex(propertyA) ##1 BT`
`visualize -force propertyA n:m -after exprB`	`exprB ##n nocex(propertyA)[*m-(n-1)]`
`visualize -force property n:$ -after exprB`	`exprB ##n nocex(propertyA)[*1:$] ##1 BT`
`visualize -force propertyA n:m -before exprB`	`nocex(propertyA)[*m-(n-1)] ##n exprB`

`visualize -force propertyA n:$ -before exprB`	`RC ##1 nocex(propertyA)[*1:$] ##n exprB`

SystemVerilog Equivalent of Visualize Not Options

To help explain `visualize -not`, the following examples show `visualize` commands with their SVA equivalents.

Visualize Construct	Equivalent SVA Cover Property
`visualize -not exprA n:m`	`RC ##n ~exprA[*m-(n-1)]`
`visualize -not exprA n:$`	`RC ##n ~exprA[*1:$] ##1 BT`
`visualize -not exprA $:$`	`~exprA ##1 BT`
`visualize -not exprA n:m -after exprB`	`exprB ##n ~exprA[*m-(n-1)]`
`visualize -not exprA n:$ -after exprB`	`exprB ##n ~exprA[*1:$] ##1 BT`
`visualize -not exprA n:m -before exprB`	`~exprA[*m-(n-1)] ##n exprB`
`visualize -not exprA n:$ -before exprB`	`RC ##1 ~exprA[*1:$] ##n exprB`

SystemVerilog Equivalent of Visualize for a Signal Sequence

To help explain `visualize -signal -sequence`, the following examples show `visualize` commands with their SVA equivalents.

Visualize Construct	Equivalent SVA Cover Property
`visualize -signal A -sequence {0 1 1 0 0 1 1 1 0}`	`cover property (RC ##1 A==0 ##1 A==1 ##1 A==1 ##1 A==0 ##1 A==0 ##1 A==1 ##1 A==1 ##1 A==1 ##1 A==0);`

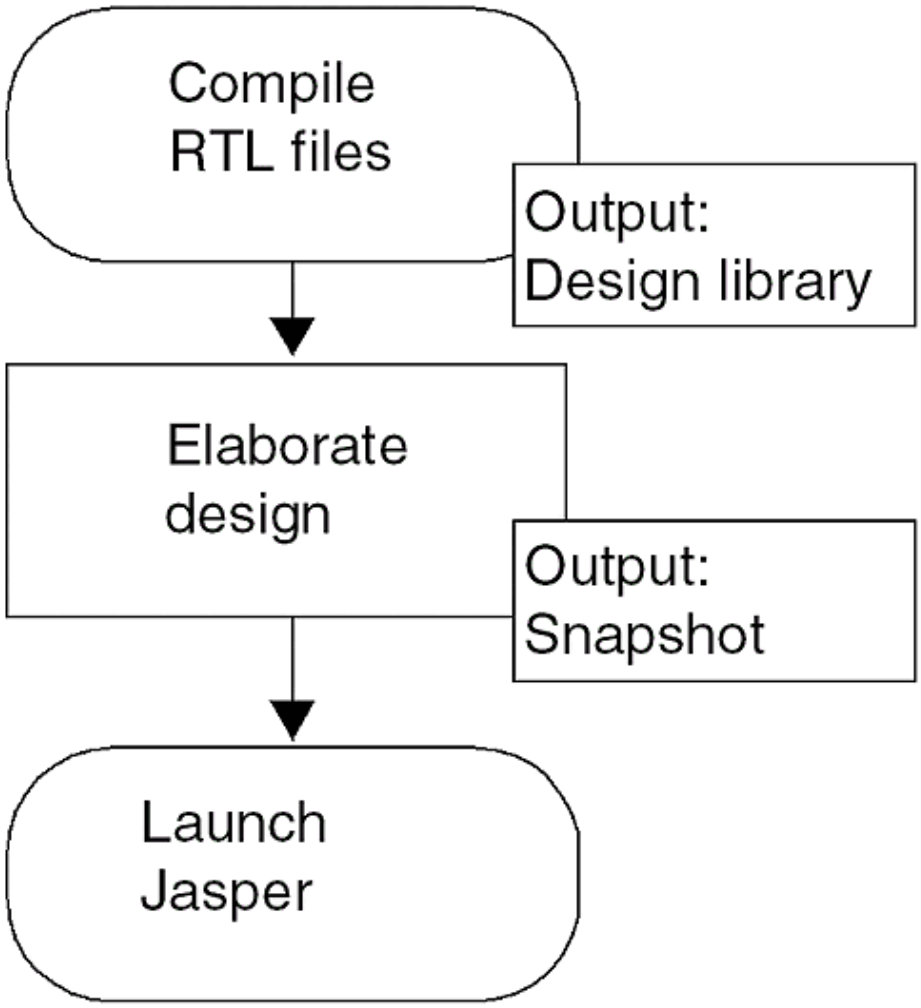
`visualize -signal B -sequence {0 1{3} 0 1 0}`	`cover property (RC ##1 B==0 ##1 B==1[*3] ##1 B==0 ##1 B==1 ##1 B==0);`
`visualize -signal C -sequence {0 1 1 1 0 0 x x 0}`	`cover property (RC ##1 C==0 ##1 C==1 ##1 C==1 ##1 C==1 ##1 C==0 ##1 C==0 ##1 C==x ##1 C==x ##1 C==0);`
`visualize -signal D -sequence {0 1{3} 0{2} x{3} 0}`	`cover property (RC ##1 D==0 ##1 D==1[*3] ##1 D==0[*2] ##1 D==x[*3] ##1 D==0);`
`visualize -signal busA -sequence {8'b0001 0000 8'hffff 8'h0 8'd0}`	`cover property (RC ##1 busA==8'b0001 ##1 busA==0000 ##1 busA==8'hffff ##1 busA==8'h0 ##1 busA==8'd0);`
`visualize -signal busB -sequence {8'b0001 0000 8'hffff 8'h0{2}}`	`cover property (RC ##1 busB==8'b0001 ##1 busB==0000 ##1 busB==8'hffff ##1 busB==8'h0[*2]);`

K

Appendix K: Jasper Apps-Xcelium Integration

xrun is a single, integrated, launching tool for various verification processes. It supports Jasper Apps with the Xcelium front end. When running the P2P flow, during the xmelab stage, the tool generates a Tcl file that launches Jasper. This Tcl file contains design information created by xmelab. To run this flow, both Xcelium and Jasper Apps installations must be available in the path.

xrun Process Flow



This appendix presents integration information, including the following:

- Supported Apps
- Jasper Apps-Xcelium Version Compatibility
- Launching Jasper with xrun: P2P and CPI2Jetlist
- Tips for Compiling VHDL Designs
- Differences in Synthesis for Mixed-Language Designs

Supported Apps

The apps officially supported by the integrated Xcelium-Jasper flow are listed below:

Jasper Apps	P2P Flow	CPI2Jetlist Flow
Coverage Unreachability (UNR) App	Yes	Yes
Functional Safety Verification (FSV) App	Only	No

Jasper Apps-Xcelium Version Compatibility

Jasper supports simulator version "mix-and-match." Key features of the simulator version "mix-and-match" include the following:

- Adds abstraction layer, which enables using Jasper for several Cadence simulator releases
- No need to update your Xcelium version when you need a new version of Jasper (and vice versa)

You only need to download one Jasper kit for each release. You can use it with multiple simulator streams.

- You must launch on RHEL 8.x or newer (see Supported Platforms for more information). Beginning with version 2025.09, the tool does not support RHEL 7.
- You must match 64-bit Jasper Apps with 64-bit Xcelium. Example for 64-bit Jasper installation with `xrun`:

  ```
  xrun -64bit -jg...
  ```

- `xrun` automatically blackboxes signals larger than 2048 bits.
- UNR has a native capability to automatically partition the design into multiple runs based on the number of uncovered coverage items extracted and merge results from all the runs. Use the following option to enable the auto partition flow: `xrun -jg_unr_partition` The minimum Xcelium versions required for the auto partition feature are the following: 23.09-a071 or newer (AGILE) and 22.09-s001 or newer (MAIN). Check the *Jasper Coverage Unreachability App User Guide* for additional information about the available options of the `xrun -jg_unr_partition` command.
- The UNR App supports enabling SVA assumes and SVA assertions turned into assumes. This support started in Jasper 2021.06 FCS. By default, the embedded assumes are disabled. To apply SVA assumes to the UNR analysis, add them using the `xrun -jg_pre_tcl <file>` command with a file containing the `check_unr -init -enable_assumption` command. If you run Jasper with the minimum Xcelium versions listed below, you can use the `xrun` switch `-jg_unr_assumption` with the command without the need of adding the `-enable_assumption` switch in the `check_unr -init` command line. This switch is automatically added to the autogenerated `check_unr -init` command when using the following minimum Xcelium versions:
  - 21.06-a001 – Agile
  - 21.03-s003 – Main
  - 21.09-s001 – Main

Launching Jasper with xrun: P2P and CPI2Jetlist

Jasper-Xcelium integration supports two flows, P2P and CPI2 Jetlist. The P2P flow is the default for UNR and FSV starting with
Xcelium 19.03-s001 when running with Jasper 2019.06 FCS or newer. Launch the apps with the following commands:

- UNR:

  ```
  xrun -64bit -jg -unr...

  xrun -64bit -jg -unr -synth_formalbuild...
  ```

- FSV

  ```
  xrun -64bit -jg -fsv...
  ```

  > Beginning with 2019.09, launching the FSV App with Jasper Apps no longer issues `check_fsv -init` automatically. With the 2024.03 release, the Jasper P2P flow supports Xcelium built snapshots with the `-newperf` switch. Compatible Xcelium versions include the following: 24.03-a071 and 24.03-s002

  - Using Jasper to launch the FSV App requires you to add `check_fsv -init` manually to the Tcl script before you run `elaborate` or `restore`.
  - Using Xcelium 19.09-s001 to launch xrun-based FSV automatically adds the command in `jgproject/jg_p2p_pre.tcl`.
  - Using a custom `jg_pre.tcl` with xrun requires you to add it manually if you use `restore`.
  - The FSV App automatically saves and restores the elaborated design when you use `-jg_save_restore <dir>` with xrun. This command behaves as follows:
    - If the `elaboratedDesign` exists already and can be loaded, the tool loads it.
    - If the `elaboratedDesign` does not exist or cannot be loaded, the tool analyzes, elaborates, and saves the `elaboratedDesign` to the default or specified directory.
    - By default, the FSV App saves the `elaboratedDesign` inside the Jasper project directory. Use the `<dir>` option to specify a different location

- The CPI2Jetlist flow is being deprecated.
- Refer to the *XRUN User Guide* for details on how to compile and elaborate a design.

Jasper-Specific xrun Command-Line Options

The following table lists and describes Jasper-specific tool options for xrun.

Option	Description
`-jg`	Run Jasper with xrun.
`-p2p -jg_analyze_opts -bb_sv_bind`	Run Jasper with xrun with the P2P flow. Use `-jg_analyze_opts -bb_sv_bind` to black box SystemVerilog bind modules in the formal flow. (This option propagates to the Jasper analyze flow.) The P2P flow is the default starting with Xcelium 19.03-s001 when running it with Jasper version 2019.06. Xcelium ignores any existing legacy flow switches and enforces the P2P UNR flow. As a consequence, you can omit the `-p2p` switch from the `xrun` command line and use `-jg` in newer tool versions. • If you use the `-jgsynthesis` switch on the xrun command line, the tool generates a warning with instructions for forcing the legacy flow with the `-synth_formalbuild` option. • If you use the `-formalbuildargs` switch on the xrun command line, the tool ignores it and issues a warning.

`-jg_bbinst <instance>`
 	
`-jg_bbinstlist`
`<file_list_of_instances>`
 	
`-jg_bbmod <module>`
 	
`-jg_bbmodlist`
`<file_list_of_modules>`
	Black box the specified instances or modules.

`-pragma`	Use the `-pragma` switch for the xrun elaboration step to produce sufficient information for the P2P flow to synthesize the design in accordance with your synthesis or coverage tool's requirements. In essence, the `-pragma` switch enables pragma processing later in P2P without impacting simulation.
`-gui`	Launch the tool in GUI mode.
`-64bit`	Launch a 64-bit Jasper Apps with a 64-bit Xcelium.
`-v93`	Compile VHDL source files that follow IEEE Std 1076-1993.
`-jg -fsv`	Run Jasper Functional Safety Verification App with xrun.
`-jg -unr`	Run Jasper UNR with xrun.
`-jg_coverage`	Process the specified coverage (`b:e:t:f:all`) This switch applies to UNR coverage.
`-jg_periodicexport` `<interval>`	Periodically dump the UNR covdb file.
`-jgargs <string>`	Pass the specified arguments to Jasper Apps.
`-synth_formalbuild`	Run formalbuild for Jasper in the CPI2Jetlist flow.
`-input <tcl_file>`	Pass the specified additional Tcl commands to Jasper Apps.
`-jg_pre_tcl`	Add a user-defined Tcl script prior to the tool-generated Tcl file while launching Jasper Apps from xrun.

`-libdirpath`	Use the specified path when creating xrun internal files. This switch replaces `-formalbuildargs -xmlibdirpath`.
`-jg_project`	Use the specified path for the Jasper project directory.
`-jg_unr_partition`	Use this switch to enable the Auto Partition flow in UNR.
`-jg_unr_partition_args (options)`	Use this switch to control and customize the Auto Partition flow in UNR. Refer to the *Jasper Coverage Unreachability App User Guide* for the description and explanation of the options you can pass to this switch. The minimum Xcelium versions required for the auto partition feature are the following: 23.09-a071 or newer (AGILE) and 22.09-s001 or newer (MAIN).
`-jg_unr_assumption`	To apply SVA assumes to the UNR analysis, add them using the `xrun -jg_pre_tcl <file>` command with a file containing the `check_unr -init -enable_assumption` command. If you run Jasper with the minimum Xcelium versions listed below, you can use the `xrun` switch `-jg_unr_assumption` with the command without the need of adding the `-enable_assumption` switch in the `check_unr -init` command line. This switch is automatically added to the autogenerated `check_unr -init` command when using the following minimum Xcelium versions: 21.06-a001 – Agile, 21.03-s003 – Main, 21.09-s001 – Main.
`-jg_pre_tcl <file>`	This switch gives you the capacity of providing Jasper Tcl commands listed in a file before the autogenerated `analyze` and `elaborate` commands in the UNR flow. To use `-jg_pre_tcl`, create a file with the Tcl commands and supply the file name to this switch.

For details on the `xrun` command, refer to the *XRUN User Guide*.

Tips for Compiling VHDL Designs

Jasper Apps offers three methods for compiling VHDL designs:

- **Method 1: Jasper Apps with the default front end** Use the `analyze` switches `-register` and `-sort` to handle file lists with an arbitrary order. With these switches, you can register specified library files for later sorting based on inter-file dependencies (see "analyze").
- **Method 2: Jasper Apps with the xrun front end** Use the `xrun` options `-smartorder` and `-smartlib` to handle files with library dependency. You can also use `-relax` to remove some strict compile errors. After xrun produces a simulation snapshot, use `xrun -p2p` and `-jg` options to force formalbuild to build a formal model and provide it to Jasper (see "Jasper-Specific xrun Command-Line Options").
- **Method 3: Leverage Method 2 with "-smartscript"** Leverage the xrun compile script from Method 2 but add `-smartscript` to specify that you want xrun to create an order-independent file list, which is a sorted file list that you can provide to the default Jasper front end without using `-register` and `-sort`. This method can provide better performance in terms of effective analysis and faster elaboration.

Differences in Synthesis for Mixed-Language Designs

This section describes the differences in the way Xcelium and Jasper handle mixed-language designs.

- These differences apply to the CPI2Jetlist flow. They do not apply to the P2P flow.
- To benefit from the latest enhancements provided by the P2P flow, we recommend that you transition UNR flows to the P2P flow. The CPI2Jetlist flow has entered the deprecation phase.

Mappings of Integer Parameters to Basic VHDL Enums

In mixed-language designs, both Jasper and Xcelium xrun accept mappings of integer (Verilog) parameters to VHDL generics (`std_logic`, bit, Boolean, character). For example:

```
module top (input clk);

    parameter int STD_2_INT = 1;

    parameter shortint STD_2_SHORT = 3;

    parameter longint STD_2_LONG = 9;

    m     #(STD_2_INT, STD_2_SHORT, STD_2_LONG) m1();

endmodule


library IEEE;

use IEEE.std_logic_1164.all;


entity m is

    generic (

             STD_2_INT: std_logic := '1';

             STD_2_SHORT: std_logic := '1';

             STD_2_LONG: std_logic := '1'

             );

end m;


architecture ARCH of m is

begin

end ARCH;
```

However, values are translated in different ways. While xrun maps numbers to either 0 or 1, the Jasper front end uses the table below by default. And for numbers outside this range, the tool generates an error message:

```
[ERROR (VHDL-1291)] top.v(12): value 9 is out of target constraint range 'U' to '-'
```

Verilog Integer Value	VHDL Enum Value
0	'U'
1	'X'
2	'0'
3	'1'
4	'Z'
5	'W'
6	'L'
7	'H'
8	'-'

To specify the xrun mapping shown below, elaborate with the `-vhdl_enum_to_verilog_int` switch.

Verilog Integer Value	VHDL Enum Value
0	'0'
1	'1'
2	'0'
3	'1'
4	'0'
5	'1'
6	'0'
7	'1'
8	'0'

Mappings of VHDL "time" to the Verilog "integer" Parameter

The Xcelium front end maps VHDL generics of type `time` to Verilog type `integer`. It converts the time unit specified in VHDL to the equivalent in femtoseconds and then passes the value to the Verilog parameter. However, the Jasper front end issues an error because in VHDL, `time` and `integer` are two different types, and it does not map them to each other in VHDL or mixed-language designs.

Consider the following VHDL and Verilog code:

```
entity top is
end top;


architecture ARCH of top is
constant time_const: time := 5 ps;
begin
    m1: entity work.m
    generic map (TIME_2_IMPLICIT => time_const);
end ARCH;
module m #(TIME_2_IMPLICIT = 1) ();
    wire [TIME_2_IMPLICIT-1:0] bla;
endmodule
```

While xrun correctly maps VHDL `time` to the Verilog `integer` parameter, the Jasper front end generates the following error:

```
[ERROR (VHDL-1272)] top.vhd(8): type error near time_const ; expected type of VHDL
generic does not match its Verilog connection
```

L

Appendix L: Covergroup Support

This appendix lists covergroup support for Jasper Apps according to the standard IEEE® Std 1800™ -2023. Refer to the following table for supported constructs.

- The page numbers listed in the "LRM Reference" column belong to the above standard.
- Starting with the 2025.06 release, all WCG warnings can be upgraded to errors with the `set_message -error <message_id>` command.

Item	Supported	Description	LRM Reference
Covergroup General			
Clocking event			
	Partial	Only clocking events (posedge or negedge of a clock, or clocking blocks). `with function sample`, `@@` and `sample()` method not supported.	19.3 Defining the coverage model: covergroup – Page 576

Example 1: Zero-dimensional Covergroup

```
covergroup cg_a @(cb);

a: coverpoint a;

endgroup

cg_a cg_a_i = new();

covergroup cg_b @(negedge clk);

b: coverpoint b;

endgroup

cg_b cg_b_i = new();
```

Example 2: Multi-dimensional Covergroups Instantiated with Parameters

```
parameter NUM_MODULES = 5;

logic [NUM_MODULES - 1:0] sub_module_output_signal

generate

for (genvar i = 0; i < NUM_MODULES; i = i + 1)

begin: gen_submodules

my_submodule_type my_submodule_inst

               (

                 …

                 .my_output_port (sub_module_output_signal[i]),

                 …

               )

endgenerate


covergroup cg_my_example_group (int my_module_number) @(posedge my_clock_name);

         type_option.comment = "Example Covergroup";

         option.per_instance = 1;

         option.auto_bin_max = 16;

         SUB_MODULE_OUTPUT_COVER: coverpoint
sub_module_output_signal[my_module_number]

{           bins ….        }

      endgroup : cg_my_example_group
```

You can make these covergroups work with Jasper by doing the following:

1. Declaring a set of them as an array

2. Constructing the covergroups within a generate block

```
cg_my_example_group my_submodule_coverage[NUM_MODULES];

generate

              for (genvar i = 0; i < NUM_MODULES; i = i + 1)

              begin: construct_covergroups

                     my_submodule_coverage[i] = new;

              end

endgenerate
```

Coverpoints			
	Partial	See "Coverpoints" section on this appendix for details	19.5 Defining coverage points – Page 582

Example:

```
logic [7:0] my_var;

covergroup my_var_cg @(cb);

my_var: coverpoint my_var {

bins min = {0};

bins mid[5] = {[1:254]};

bins max = {255};

}

endgroup

my_var_cg my_var_cg_i = new();
```

Cross coverage			
	Partial	See "Cross" section on this appendix for details	19.6 Defining cross coverage – Page 597

Example:

```
covergroup cg @(posedge clk);

packet_type: coverpoint my_packet.packet_type;

has_crc: coverpoint my_packet.has_crc;

cross_type_crc: cross packet_type, has_crc;

endgroup

cg cg_i = new();
```

Optional formal arguments			
	Yes	Constant and variable arguments are supported if the connected element is of the same size and type of the argument.	19.3 Defining the coverage model: covergroup – Page 576

Example:

```
logic [2:0] var_A;

logic [2:0] var_B;

covergroup cg_var (ref logic [2:0] my_var) @(posedge clk);

my_var1: coverpoint my_var;

endgroup

cg_var cg_var_A = new(var_A);

cg_var cg_var_B = new(var_B);
```

Coverage options			
	Partial	Covergroup type options such as `weight`, `goal`, `bin_auto_max`. Only `auto_bin_max` is supported for simple, single-dimension coverpoints.	19.7 Specifying coverage options – Page 605

Example:

```
logic [7:0] a;

covergroup cg_a @(posedge clk);

option.auto_bin_max = 10;

a: coverpoint a; // Will be created with 10 bins

endgroup

cg_a cg_a_i = new();
```

Scope			
	Partial	Only supported within module or interface. Class not supported.	19.5 Defining coverage points – Page 582

Example:

```
interface my_if (input logic clk);

logic req;

logic [3:0] port;

covergroup port_cg @(posedge clk);

port: coverpoint port iff (req);

endgroup

port_cg port_cg_i = new();

endinterface
```

Iff expression			
	Yes	Specifies enabling condition	19.3 Defining the coverage model: covergroup – Page 576

Example:

```
covergroup cg @ (posedge clk iff enable);
```

Coverpoints

Automatic bins			
	Yes	By default, 64 automatic bins are created and it is also controlled with `auto_bin_max`.	19.5.3 Automatic bin creation for coverage points – Page 594
Expressions/variable/var-select/globals			
	Yes	Simple variable coverpoint or expression	19.5 Defining coverage points – Page 582

Example:
`D0: coverpoint y;`
`D1: coverpoint y+z+k;`
`D2: coverpoint y&&z;`
`D3: coverpoint {y,z};`
`A: coverpoint i2.a;`			

iff expression			
	Yes	Specifies enabling condition	19.5 Defining coverage points – Page 582 A.2.11 Covergroup declarations – Page 1191
options			
	Partial	Covergroup type options such as `weight`, `goal`, `bin_auto_max`, and so forth. Only `auto_bin_max` is supported.	19.7 Specifying coverage options – Page 605
enum variable			
	Yes	Variables of enumerated type	6.19 Enumerations – Page 119 19.5.3 Automatic bin creation for coverage points – Page 594

named/unnamed			
	Yes	Coverpoints can specify name or receive automatic naming	19.5 Defining coverage points – Page 582
Bins (State)			
value, range, $			
	Yes	Specifies specific values or range of values	19.5.1 Specifying bins for values – Page 584

Example:
`bins fixed0[4] = { [1:9], 2, 5, 8 };`
`bins fixed1[] = { [1:9], 2, 5, 8 };`
`bins scalar = { [$:10], 2, 5, 8 };`			

function call in expression			
	Yes	Function calls can be used in an expression	19.5 Defining coverage points – Page 582
scalar, vector, vector auto type			
	Yes	Bins may be specified as scalar, fixed vector, or automatic type	19.5.1 Specifying bins for values – Page 584

Example:
`bins fixed [4] = { [1:9], 2, 5, 8 };`
`bins vec_auto[] = { [1:9], 2, 5, 8 };`
`bins scalar = { [1:9], 2, 5, 8 };`			

explicit and automatic bins			
	Yes	Bins may be specified explicitly or will be automatically created	19.5 Defining coverage points – Page 582

Example:
`D0: coverpoint y { bins f = {1,2}; }`
`D1: coverpoint y;`			

illegal/ignore			
	Yes	Excludes values. Assertions created for illegal values.	19.5.6 Specifying Illegal coverage point values or transitions – Page 595

Example:
`ignore_bins fixed [4] = { [1:9], 2, 5, 8 };`
`illegal_bins fixed = { [$:9], 2, 5, 8 };`			

with expression			
	Yes	Using expression as `with` argument	19.5.1.1 Coverpoint bin `with` covergroup expressions – Page 589

Example:
`a:` **coverpoint** `x`
`{`
bins `mod3[] = {[0:255]}` **with** `(item % 3 == 0);`
`}`			

with function			
	No	Using function as `with` argument	19.5.1.1 Coverpoint bin `with` covergroup expressions – Page 589

Example:
coverpoint `b`
`{`
bins `func[] = b` **with** `(myfunc(item));`
`}`			

wildcards			
	Yes	Wildcards used in bin values	19.5.4 Wildcard specification of coverage point bins – Page 594
Example: `wildcard_bins w = {8'bxxxxxxx}`			
default			
	Yes	Catches undefined values. Coverage is not collected. Assertions created for such values.	19.5 Defining coverage points – Page 582
signed/unsigned			
	Yes	Both signed and unsigned values can be used	19.5.7 Value resolution – Page 596
iff expression			
	Partial	Specifies enabling condition. Not supported for illegal and ignore bins.	19.5.2 Specifying bins for transitions – Page 590
Example: `bins tn = {1, 2} iff (inp1);`			
enum			
	Yes	Enumerated type used in bin specification	19.5.3 Automatic bin creation for coverage points – Page 594
Example: `bins a1 = {[VERILOG : VHDL] };`			
x value			
	Yes	x specified as bin value	19.5.1 Specifying bins for values – Page 584

arg, parameter usage			
	Yes	Bins using covergroup arguments or parameter values	19.5.1 Specifying bins for values – Page 584

Example:
`parameter L=4;`
`covergroup cg (int f1) @(posedge clk);`
`bins fixed [4] = { 1, f1, L };`			

bins value from expressions			
	Yes	Binary expressions used for creating bin value	Not applicable

Example:
`parameter L=4;`
`covergroup cg (int f1) @(posedge clk);`
`bins fixed [4] = { 1, f1+2, L<<3, 4-2 };`			

Bins (Transition)			
value, range			
	Yes	Transitions referencing specific values or a range of values	19.5.1 Specifying bins for values – Page 584
Example: `bins fixed [] = (1=>2=>3), ([3:5]=>[4:5]);`			
scalar, vector, vector auto type			
	Yes	Transition bins may be scalar, fixed vector, or automatic type	19.5.1 Specifying bins for values – Page 584

Example:
`bins fixed[] = (1=>[2:5]=>3);`
`bins scalar = (1=>2=>3);`			

illegal/ignore			

	Yes	Excludes transitions. Assertions created for illegal values	19.5.6 Specifying Illegal coverage point values or transitions – Page 595

Example:
`ignore_bins fixed[] = (1=>2=>3);`
`illegal_bins fixed[] = (1=>2=>3);`			

wildcards			
	Yes	Wildcards used in transition values	19.5.4 Wildcard specification of coverage point bins – Page 594
Example: `wildcard bins T0_3_array[] = (2'b0x => 2'b0x)`			
default_sequence			
	No	Catches undefined sequences. Coverage is not collected.	19.5 Defining coverage points – Page 582 19.5.1.2 Coverpoint bin set covergroup expressions – Page 590
Example: `bins others[] = default_sequence;`			
repetition -> * =>			
	No	Repetition of a sequence	19.5.2 Specifying bins for transitions – Page 590
signed/unsigned			
	Yes	Both signed and unsigned values supported	19.5.7 Value resolution – Page 596

Example: `bins fixed [] = (1=>2=>3), ([-3:5]=>[-4:5]);`			
iff expression			
	Partial	Specifies enabling condition. Not supported for illegal and ignore bins.	19.5.2 Specifying bins for transitions – Page 590
enum			
	Yes	Enumerated type used in bin specification	19.5.3 Automatic bin creation for coverage points – Page 594
Example: `bins b5[] = (zero => [zero : bigger] => zero);`			
number of transitions > 10000			
	No	Bins specifying >10000 transitions are not supported	Not applicable
arg, parameter usage			
	Yes	Bins using covergroup arguments or parameter values	19.5.1 Specifying bins for values – Page 584

Example:
`parameter L=4;`
`covergroup cg (int f1) @(posedge clk);`
`bins fixed [4] = { 1=>f1=> L };`			

bin value from expressions			
	Yes	Binary expressions used for creating bin value	Not applicable

Example:

```
parameter L=4;

covergroup cg (int f1) @(posedge clk);

bins fixed [4] = ( 1=> f1+2 => L<<3 );
```

Cross

cross of coverpoints			
	Yes	Cross coverage of two or more coverpoints	19.6 Defining cross coverage – Page 597

Example:

```
CP1: coverpoint v1;

CP2: coverpoint v2;

CR: cross CP1, CP2;
```

cross of variables and coverpoints			
	Yes	Cross coverage including variable(s) with coverpoint(s)	19.6 Defining cross coverage – Page 597

Example:

```
CP1: coverpoint v1;

CR: cross v1, CP2;
```

cross or external coverpoints/covergroups			
	Partial	Cross coverage of two or more coverpoints of external covergroups. This reference is limited to covergroups defined in the same module.	19.6 Defining cross coverage – Page 597
user bins			
	Yes	User specified cross bins	19.6.1 Defining cross coverage bins – Page 599

Example: `bins b1 = binsof(CP1) intersect {2, 3}`			
illegal/ignore bins			
	Yes	Excludes illegal bins. No check/error on illegal bins	19.6.2 Excluding cross products – Page 605 19.6.3 Specifying illegal cross products – Page 605
Example: `illegal_bins b1 = binsof(CP1) intersect {2, 3}`			
auto bins			
	Yes	Created excluding illegal/ignore bins values (if any)	19.6.2 Excluding cross products – Page 605
cross of cross			
	No	Cross coverage including cross bin(s)	19.6.1 Defining cross coverage bins – Page 599
binsof(coverpoint and bin) and intersect			
	Yes	Specifying a coverpoint and `bin` with `binsof`	19.6.1 Defining cross coverage bins – Page 599

Example:
`illegal_bins b1 = binsof(CP1) intersect {2, 3}`
`illegal_bins b1 = binsof(CP1.BIN1) intersect {2, 3}`			

cross bin with covergroup expr			
	Yes	Specifying expression with select_expression(binsof) and cross_identifier(X)	19.6.1.2 Cross bin with covergroup expressions – Page 602

Example:

bins apple = X **with** (a+b < 257);

bins cherry = (**binsof**(b) **intersect** {[0:50]} && **binsof**(a.low) **intersect** {[0:50]}) **with** (a==b));

bins plum = **binsof**(b.two) **with** (b > 12) || (a.low) **with** (a & b);

expression with coverpoint			
	Yes	Use coverpoint in expression	19.6.1.2 Cross bin with covergroup expressions – Page 602
expression with bin			
	Yes	Use bins in expression	19.6.1.4 Cross bin set expression – Page 603
expression logical operators			
	Yes	Logical operators `&&` and `||` are supported	19.6.1 Defining cross coverage bins – Page 599
expression - negated form			
	Yes	Using `!` operator	19.6.1 Defining cross coverage bins – Page 599

Example:

```
illegal_bins b2 = binsof(CP1) intersect {2, 3} || !binsof(CP2) intersect {2, 3}
```

Cross of transitions			
	Yes	Cross of transitions are fully supported	19.6.1 Defining cross coverage bins – Page 599
Cross bin set expression			

	No	Cross bin set expression not supported	19.6.1.4 Cross bin set expression – Page 603
CrossQueueType			
	No	CrossQueueType not supported	19.6.1.3 Cross bin automatically defined types – Page 603
binary expressions in bins values			
	Yes	Binary expressions for value of bins	19.6 Defining cross coverage – Page 597

Example:

```
illegal_bins ill = binsof(c1) intersect {v-2, [v : 15]};
```

M

Appendix M: Disabled Jasper and Front-End Messages

This appendix lists Jasper and front-end messages that are disabled by default. You can use the command `set_message` to enable the messages.

- Disabled Warnings
- Disabled Warning or Info Messages
- Disabled Error Messages

Disabled Warnings

Jasper Apps ignores a limited set of Jasper and front-end warning messages, which are listed in the following table. Use the command `set_message -warning` to enable the listed messages (see **`set_message`**).

> If you are using IP, such as the ACE IPK, the software includes a command to enable the listed warnings.

Message ID	Description
`VDB-1056`	Warning issued when a register is removed due to some optimization
`WOBS001`	Warning for an assertion that cannot be converted into an assumption
`VERI-1318`	Warning for variable with multiple concurrent drivers
`VERI-2371`	Warning for non-synthesizable delay constructions

VERI-2438	Warning for package parameter override
VERI-2580	Warning for latch inferred for net var_a
VERI-2790	Warning for multiple drivers in variables
VERI-2887	Warning when an arithmetic operation is between operands of different types
VERI-2889	Warning for comparison between operands of different types
VERI-2890	Warning for implicit conversion from floating point type real to boolean
VERI-2891	Warning for implicit conversion from real to int
VERI-2892	Warning for implicit conversion from real to shortreal
VERI-2893	Warning for implicit conversion widens from shortreal to real
VERI-2894	Warning for implicit conversion from packed struct/union to packed union/struct
VERI-2895	Warning for implicit conversion from integer to boolean
VERI-2896	Warning for implicit conversion from integer to real
VERI-2897	Warning for implicit conversion changes signedness
VERI-2898	Warning for comparison of differently signed types
VERI-2899	Warning for real value provided for integer format specifier
VERI-2900	Warning when "signed" or "unsigned" on port has no effect on type
VERI-2901	Warning when case does not have default statement
VERI-2902	Warning when case condition is already specified
VERI-2903	Warning when enumeration value is not handled in case statement
VERI-2904	Warning when numeration value is not explicitly handled in case statement
VERI-2906	Warning when case generate condition is already specified
VERI-2907	Warning when unique/priority case does not need to code a default statement
VERI-2908	Warning for useless cast

VERI-2909	Warning when ignoring return value of function
VERI-8003	Warning for ignored action block
VERI-8011	Warning for delay control not yet supported for synthesis
VERI-8017	Warning for X in replication expressions
VERI-8027	Warning for undefined instance that will be black boxed
VERI-8035	Warning for implicitly declared nets
VERI-8037	Warning for negative range limits
VERI-8044	Warning for expression size 32 truncated by a cast operation to fit in target size 4
VERI-8045	Warning for 32-bit expression truncated by a cast operation to fit in target size 4
VERI-8046	Warning for expression size 3 extended by a cast operation to fit in target size 4
VERI-8047	Warning for right shift count equal to width of value
VERI-8048	Warning for right shift count greater than width of value
VERI-9002	Warning for LHS width of an assignment larger than RHS width
VERI-9004	Comparison between unequal length operands of size 1 and 2
VERI-9009	Warning for else operand of size 1 extended to 32 bits for conditional operator
VERI-9010	Warning for generate block with no name
VERI-9018	Warning for expression truncated to fit in target size
VERI-9025	Warning for converting 4-state value to 2-state
VERI-9132	Warning when a slice of a variable is potentially used before its assignment
VHDL-1840	Warning for latch inferred for net `sig_a`
VHDL-8012	Warning for ignored report statements

Disabled Warning or Info Messages

Jasper Apps ignores a limited set of front-end messages that you can set as a warning *or* info message. These messages are listed in the following table. Use the command `set_message` with `-warning` or `-info` to enable the listed messages (see set_message).

Message ID	Description
`VERI-2320`	Back to file that included a file
`VERI-2610`	Jasper issues this message when processing translate off and translate on pragmas. Jasper does not issue messages for pragmas not processed by the tool, for instance, ones inside a `pragma translate_off` block.

`VERI-8023`	Reporting the name of the Compilation Units created during `analyze` commands. The format of the message is the following, if `analyze` is targeted to the work library:
	`[INFO (VERI-8023)] test.v(1): Creating compilation unit named '\$unit_test_v'`
	If the target is a different library, the format is the following:
	`[INFO (VERI-8023)] mod.v(1): Creating compilation unit named 'myname' into library 'myLib2'`

`VERI-9012` and `VHDL-9002`	Using default value of parameter since it has no actual
`VERI-9013` and `VHDL-9003`	Elaborating instance
`VERI-9014` and `VHDL-9004`	Done elaborating instance

Disabled Error Messages

Jasper Apps ignores a limited set of Jasper and front-end error messages, which are listed in the following table. Use the command `set_message -error` to enable the listed messages (see **`set_message`**).

Message ID	Description
`VERI-8053`	Error issued for mixed concurrent and procedural assignments on ports

N

Appendix N: get_design_info Transitivity

This appendix lists the transitivity of `get_design_info` scopes:

- Top instance (no scope switch)
- `-instance`
- `-module`
- `-interface_instance`
- `-interface_port`
- `-interface`

> For the complete command documentation, refer to get_design_info.

This appendix includes the following sections:

- Transitivity for Top Instance, "-instance", and "-module"
- Transitivity for SystemVerilog Interfaces

Transitivity for Top Instance, "-instance", and "-module"

The following table lists the transitivity of `get_design_info` scopes: top instance (no scope switch), `-instance`, and `-module`. In addition, it identifies limitations and exceptions.

Possible behaviors are as follows:

- Transitive:
  - Top instance scope: Lists elements from all instances of the design (top instance and

instances below it)

  - `-instance` scope: Lists elements from the specified instance and its children
  - `-module` scope: Lists elements from all instances of the specified module and its children

- Not transitive:
  - Top instance scope: Lists elements from the top instance
  - `-instance` scope: Lists elements from the specified instance
  - `-module` scope: Lists elements from all instances of the specified module
- Not transitive by default and transitive by default (top instance and `-instance` scopes): Supports a switch to specify transitivity
- Global (top instance scope): Transitivity does not apply to global enums

The table also includes the following categories:

- N/A: Transitivity does not apply to this enum
- – : The enum is not available for this scope

Enum	top instance	-instance	-module
`parameter`	Not transitive by default Use `-transitive` to return all parameters listed in the design.	Not transitive by default Use `-transitive` to return the parameters listed in the specified instance and instances under it.	Not transitive
`local_parameter`	Not transitive	Not transitive	–
`constant`	Not transitive	Not transitive	Not transitive
`enum`	Transitive	Transitive	Transitive for 1 instance: Lists elements from one instance of the specified module and its children

`output`	Not transitive	Not transitive	Not transitive for 1 instance: Lists elements from one instance of the specified module
`input`	Not transitive	Not transitive	Not transitive for 1 instance: Lists elements from one instance of the specified module
`inout`	Not transitive	Not transitive	Not transitive for 1 instance: Lists elements from one instance of the specified module
`stopat`	Transitive	Transitive	• Transitive for 1 instance: Lists elements from one instance of the specified module and its children • Only lists stopats on the COI of the current task.

undriven	Transitive	Transitive	Transitive for 1 instance: Lists elements from one instance of the specified module and its children
undriven_internal	Transitive	Transitive	Transitive for 1 instance: Lists elements from one instance of the specified module and its children
loop_index	Transitive	Transitive	Transitive for 1 instance: Lists elements from one instance of the specified module and its children
multiple_driven	Transitive	Transitive	Transitive for 1 instance: Lists elements from one instance of the specified module and its children
bbox_mod	Transitive	Transitive	Transitive
bbox_inst	Transitive	Transitive	Transitive
bbox_in	Transitive	Transitive	Transitive
bbox_out	Transitive	Transitive	Transitive
bbox_inout	Transitive	Transitive	Transitive

bbox_in_instance_port	Transitive	Transitive	Transitive
bbox_out_instance_port	Transitive	Transitive	Transitive
bbox_inout_instance_port	Transitive	Transitive	Transitive
basic	Transitive	Transitive	Transitive for 1 instance: Lists elements from one instance of the specified module and its children
counter	Transitive	Transitive	Transitive for 1 instance: Lists elements from one instance of the specified module and its children
fsm	Transitive	Transitive	Transitive for 1 instance: Lists elements from one instance of the specified module and its children
fifo	Transitive	Transitive	Transitive for 1 instance: Lists elements from one instance of the specified module and its children

`array`	Transitive	Transitive	Transitive for 1 instance: Lists elements from one instance of the specified module and its children
`register`	Transitive	Transitive	Transitive for 1 instance: Lists elements from one instance of the specified module and its children
`flop`	Transitive	Transitive	Transitive for 1 instance: Lists elements from one instance of the specified module and its children
`latch`	Transitive	Transitive	Transitive for 1 instance: Lists elements from one instance of the specified module and its children
`gate`	N/A Returns the number of gates	N/A Returns the number of gates	N/A Returns the number of gates

`multiplier`	Transitive	Transitive	Partially transitive: Transitive for black-boxed operators and not transitive for cell-defined operators
`divider`	Transitive	Transitive	Partially transitive: Transitive for black-boxed operators and not transitive for cell-defined operators
`modulus`	Transitive	Transitive	Partially transitive: Transitive for black-boxed operators and not transitive for cell-defined operators
`module`	Transitive	Transitive	Transitive
`module_no_param`	Transitive	Transitive	Transitive
`module_name`	N/A Returns the non-black-boxed module name for the top instance	N/A Returns the module name for the top instance	N/A Returns the specified module name
`module_name_no_param`	N/A Returns the specified module name without the non-default parameter values		

`instance`	Transitive by default Use `-depth` to filter the list of instances using the depth level, listing the instances until the chosen depth. Depth 0 is the top instance.	Transitive by default Use `-depth` to filter the list of instances using the depth level, listing the instances until the chosen depth. Depth 0 is the top instance.	Not transitive
`bind_instance`	Transitive by default Use `-depth` to filter the list of instances using the depth level, listing the instances until the chosen depth. Depth 0 is the top instance.	Transitive by default Use `-depth` to filter the list of instances using the depth level, listing the instances until the chosen depth. Depth 0 is the top instance.	Not transitive
`interface`	Transitive	–	–
`interface_instance`	Transitive	–	–
`interface_port`	Transitive	–	–
`prim_inst`	Transitive	Transitive	Transitive
`prim_mod`	Transitive	Transitive	Transitive
`celldefine`	Transitive	Transitive	Transitive
`package`	Global	–	–
`architecture`	Transitive	Transitive	Transitive
`signal`	Transitive	Transitive	Transitive for 1 instance: Lists elements from one instance of the specified module and its children

`wire`	Transitive	Transitive	Transitive for 1 instance: Lists elements from one instance of the specified module and its children
`property`	Transitive	Transitive	Transitive for 1 instance: Lists elements from one instance of the specified module and its children
`assert`	Transitive	Transitive	Transitive for 1 instance: Lists elements from one instance of the specified module and its children
`assume`	Transitive	Transitive	Transitive for 1 instance: Lists elements from one instance of the specified module and its children
`cover`	Transitive	Transitive	Transitive for 1 instance: Lists elements from one instance of the specified module and its children

`macro`	Global	–	–
`parent`	–	N/A Returns the parent of the specified instance	–
`empty_inst`	–	Transitive	–

Transitivity for SystemVerilog Interfaces

The following table lists the transitivity of `get_design_info` for SystemVerilog instance scopes: `-interface_instance`, `-interface_port`, and `-interface`. In addition, it provides return info when relevant.

Enum	-interface_instance	-interface_port	-interface
`interface_instance`	–	–	Not transitive
`interface_port`	–	–	Not transitive
`interface`	N/A Returns the interface name for the specified *`interface_instance`*.	N/A Returns the interface name for the specified *`interface_port`*.	–

O

Appendix O: HDL Sort Analyze Flow

Starting with the 2026.03 release, you can now run an integrated HDL sort flow in `analyze` using `analyze -register` and `analyze -hdl_sort`. This flow can handle SystemVerilog/Verilog, VHDL, and mixed designs, including different flavors of SystemVerilog or VHDL.

Unlike the `analyze -sort` option, this new option does not require an analysis mode or file to be informed as these have been defined in the corresponding register for each file using the `analyze -register` command.

An example of the new flow follows:

```
analyze -vhdl08 -register ha.vhd
analyze -vhdl -register counter.vhd
analyze -hdl_sort
elaborate -vhdl
```

Mixed VHDL/SystemVerilog designs are also supported:

```
analyze -register -vhdl counter.vhd
analyze -register -sv09 test.v
analyze -register -vhdl08 ha.vhd
analyze -register -sv pkg.v
analyze -hdl_sort
elaborate
```

The HDL sort flow has some limitations as listed below:

- **SystemVerilog/Verilog Limitations:**
  - By default, the Verilog file sorter looks into the files in SFCU mode and resets the macro table when it moves from one file to the next. In general, macros make it difficult to find exact file ordering because they can be defined and then undefined and yet again redefined. Because of this, macro preprocessing is a language within itself and does not allow the order of files to be determined.

- Libmaps/library declarations are ignored. Consequently, file name pattern to analysis library mappings are not honored.
- SystemVerilog configurations are automatically discovered; however, not all items within a configuration can be processed. Only the following items are honored:
  1. All the top level modules specified in the `design` statement.
  2. Modules specified with `cell` clause.
  3. Library lists specified with `liblist` only for `cell` clause.
  4. Modules specified with `use` clause.
- All the module names can be specified with a library name prefix, and the flow considers the library name in such cases. `default liblist` is ignored as it applies to all instances/rules in the sub-hierarchy. `analyze -hdl_sort` cannot handle that as it has no hierarchy information.

- **VHDL Limitations:**

  `analyze -hdl_sort` cannot sort the files in the correct order if any of the files has a configuration declaration that has some declaration before the binding indication. For example:

```
library Lib1 ;
 configuration Config1 of Entity1 is
   use Lib1.all ; -- We cannot handle this design because of this line
   for …
       …
   end for ;
 end configuration Config1 ;
```

6

Jasper TclHelp Reference

Jasper Command Help

Use one of the tabs to the left to navigate.

Search uses Full Text Search (FTS5) Query Syntax:

https://sqlite.org/fts5.html#full_text_query_syntax